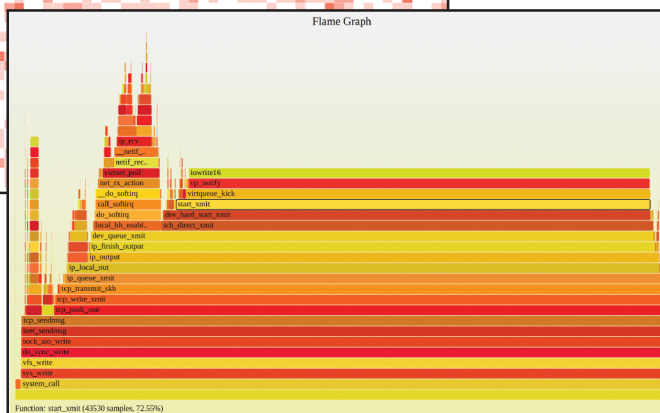
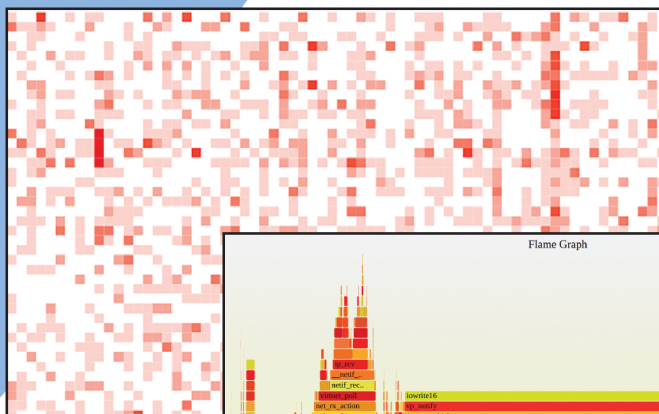


Производительность систем

Второе издание

на примере Linux

Брендан Грегг



Systems Performance

Enterprise and the Cloud

Second Edition

Brendan Gregg

◆◆Addison-Wesley

Boston • Columbus • New York • San Francisco • Amsterdam • Cape Town
Dubai • London • Madrid • Milan • Munich • Paris • Montreal • Toronto • Delhi • Mexico City
São Paulo • Sydney • Hong Kong • Seoul • Singapore • Taipei • Tokyo

Производительность систем

Второе издание

Брендан Грегг



Санкт-Петербург • Москва • Минск

2023

ББК 32.973.2-018.2
УДК 004.451
Г79

Грегг Брендан

Г79 Производительность систем. — СПб.: Питер, 2023. — 992 с.: ил. — (Серия «Для профессионалов»).

ISBN 978-5-4461-1818-2

Книга посвящена концепциям, стратегиям, инструментам и настройке операционных систем и приложений на примере систем на базе Linux. Понимание этих инструментов и методов критически важно при разработке современного ПО. Применение стратегий, изложенных в обновленном и переработанном издании, позволит перформанс-инженерам улучшить взаимодействие с конечными пользователями и снизить затраты, особенно для облачных сред.

Брендан Грегг — эксперт в области производительности систем и автор нескольких бестселлеров — лаконично, но емко излагает наиболее важные сведения о работе операционных систем, оборудования и приложений, которые позволят специалистам быстро добиться результатов, даже если раньше они никогда не занимались анализом производительности. Далее автор дает детальные объяснения по применению современных инструментов и методов, включая расширенный BPF, и показывает, как добиться максимальной эффективности ваших систем в облачных, веб- и крупных корпоративных средах.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.973.2-018.2
УДК 004.451

Права на издание получены по соглашению с Pearson Education, Inc. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги.

В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др.

Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-0136820154 англ.

Authorized translation from the English language edition, entitled Systems Performance, 2nd Edition by Brendan Gregg, published by Pearson Education, Inc, publishing as Addison Wesley Professional.

ISBN 978-5-4461-1818-2

© 2021 Pearson Education, Inc.
© Перевод на русский язык ООО «Прогресс книга», 2023
© Издание на русском языке, оформление ООО «Прогресс книга», 2023
© Серия «Для профессионалов», 2023

КРАТКОЕ СОДЕРЖАНИЕ

Предисловие.....	30
Благодарности.....	38
Об авторе	41
Глава 1. Введение.....	42
Глава 2. Методологии.....	66
Глава 3. Операционные системы.....	146
Глава 4. Инструменты наблюдения.....	195
Глава 5. Приложения.....	241
Глава 6. Процессоры.....	299
Глава 7. Память	396
Глава 8. Файловые системы	460
Глава 9. Диски.....	533
Глава 10. Сеть.....	622
Глава 11. Облачные вычисления.....	714
Глава 12. Бенчмаркинг.....	786
Глава 13. perf.....	821
Глава 14. Ftrace	857

Глава 15. BPF	904
Глава 16. Пример из практики	939
Приложение А. Метод USE: Linux	950
Приложение В. Краткий справочник по sag	957
Приложение С. Однострочные сценарии для bpftrace	959
Приложение D. Решения некоторых упражнений	966
Приложение Е. Производительность систем, кто есть кто	969
Глоссарий	974

ОГЛАВЛЕНИЕ

Предисловие	30
Об этом издании	30
Об этой книге	31
Благодарности.....	38
Об авторе	41
От издательства	41
Глава 1. Введение	42
1.1. Производительность системы.....	42
1.2. Роли.....	43
1.3. Действия	44
1.4. Перспективы	46
1.5. Сложности оценки производительности	46
1.5.1. Субъективность	47
1.5.2. Сложность	47
1.5.3. Множественные причины	48
1.5.4. Множественные проблемы с производительностью	48
1.6. Задержка.....	49
1.7. Наблюдаемость	50
1.7.1. Счетчики, статистики и метрики	50
1.7.2. Профилирование	52
1.7.3. Трассировка	54
1.8. Эксперименты	56
1.9. Облачные вычисления	58
1.10. Методологии.....	58
1.10.1. Анализ производительности Linux за 60 секунд.....	59

1.11. Практические примеры.....	60
1.11.1. Медленные диски.....	60
1.11.2. Изменение в программном обеспечении	63
1.11.3. Дополнительное чтение	65
1.12. Ссылки	65
Глава 2. Методологии.....	66
2.1. Терминология	67
2.2. Модели	68
2.2.1. Тестируемая система	68
2.2.2. Система массового обслуживания	69
2.3. Основные понятия	69
2.3.1. Задержка.....	70
2.3.2. Шкалы времени	71
2.3.3. Компромиссы	72
2.3.4. Настройка производительности	73
2.3.5. Целесообразность.....	74
2.3.6. Когда лучше остановить анализ.....	75
2.3.7. Рекомендации действительно на данный момент времени.....	76
2.3.8. Нагрузка и архитектура	77
2.3.9. Масштабируемость.....	78
2.3.10. Метрики	80
2.3.11. Потребление.....	81
2.3.12. Насыщенность.....	82
2.3.13. Профилирование.....	83
2.3.14. Кэширование	83
2.3.15. Известные неизвестные	86
2.4. Точки зрения	86
2.4.1. Анализ ресурсов.....	86
2.4.2. Анализ рабочей нагрузки.....	88
2.5. Методология	89
2.5.1. Антиметодология «Уличный фонарь»	91
2.5.2. Антиметодология «Случайное изменение»	92
2.5.3. Антиметодология «Виноват кто-то другой»	92

2.5.4. Специальный чек-лист	93
2.5.5. Формулировка проблемы	94
2.5.6. Научный метод	94
2.5.7. Цикл диагностики	96
2.5.8. Метод инструментов	96
2.5.9. Метод USE	97
2.5.10. Метод RED	105
2.5.11. Определение характеристик рабочей нагрузки	105
2.5.12. Анализ с увеличением детализации	107
2.5.13. Анализ задержек	109
2.5.14. Метод R	109
2.5.15. Трассировка событий	110
2.5.16. Базовые статистики	112
2.5.17. Статическая настройка производительности	113
2.5.18. Настройка кэширования	113
2.5.19. Микробенчмаркинг производительности	114
2.5.20. Мантры производительности	115
2.6. Моделирование	116
2.6.1. Корпоративные и облачные среды	117
2.6.2. Визуальная идентификация	117
2.6.3. Закон Амдала	119
2.6.4. Универсальный закон масштабируемости	120
2.6.5. Теория массового обслуживания	121
2.7. Планирование емкости	125
2.7.1. Пределы ресурсов	125
2.7.2. Факторный анализ	127
2.7.3. Решения масштабирования	128
2.8. Статистики	129
2.8.1. Количественная оценка прироста производительности	129
2.8.2. Усреднение	131
2.8.3. Стандартное отклонение, процентиля, медиана	132
2.8.4. Коэффициент вариации	133
2.8.5. Мультимодальные распределения	133
2.8.6. Выбросы	134

2.9. Мониторинг.....	135
2.9.1. Временные закономерности	135
2.9.2. Инструменты мониторинга	137
2.9.3. Сводная статистика, накопленная с момента загрузки	137
2.10. Визуализация	137
2.10.1. Линейная диаграмма.....	137
2.10.2. Диаграммы рассеяния	139
2.10.3. Тепловые карты.....	140
2.10.4. Временная шкала	141
2.10.5. График поверхности.....	142
2.10.6. Инструменты визуализации	143
2.11. Упражнения.....	144
2.12. Ссылки.....	144
Глава 3. Операционные системы.....	146
3.1. Терминология	147
3.2. Основы	148
3.2.1. Ядро.....	148
3.2.2. Ядро и пользовательские режимы.....	150
3.2.3. Системные вызовы.....	152
3.2.4. Прерывания.....	154
3.2.5. Часы и бездействие.....	158
3.2.6. Процессы	159
3.2.7. Стеки.....	162
3.2.8. Виртуальная память.....	164
3.2.9. Планировщики.....	165
3.2.10. Файловая система.....	167
3.2.11. Кэширование	169
3.2.12. Сеть	170
3.2.13. Драйверы устройств.....	171
3.2.14. Многопроцессорность	171
3.2.15. Вытеснение.....	172
3.2.16. Управление ресурсами	172
3.2.17. Наблюдаемость	173

3.3. Ядра.....	173
3.3.1. Unix	174
3.3.2. BSD.....	175
3.3.3. Solaris.....	176
3.4. Linux.....	177
3.4.1. Новые разработки в ядре Linux.....	178
3.4.2. systemd.....	184
3.4.3. KPTI (Meltdown)	185
3.4.4. Расширенный BPF	186
3.5. Другие темы	187
3.5.1. Ядра PGO.....	187
3.5.2. Одноцелевые ядра.....	188
3.5.3. Микроядра и гибридные ядра.....	188
3.5.4. Распределенные операционные системы.....	189
3.6. Сравнение ядер	189
3.7. Упражнения.....	190
3.8. Ссылки	191
3.8.1. Дополнительное чтение	193
Глава 4. Инструменты наблюдения.....	195
4.1. Покрытие инструментами.....	196
4.1.1. Инструменты статического анализа производительности.....	197
4.1.2. Инструменты анализа кризисных ситуаций	198
4.2. Типы инструментов	199
4.2.1. Фиксированные счетчики	200
4.2.2. Профилирование	202
4.2.3. Трассировка	203
4.2.4. Мониторинг.....	204
4.3. Источники информации.....	206
4.3.1. /proc	208
4.3.2. /sys.....	212
4.3.3. Учет задержек	213
4.3.4. netlink	214
4.3.5. Точки трассировки	214

4.3.6. kprobes	219
4.3.7. uprobes	222
4.3.8. USDT	224
4.3.9. Аппаратные счетчики (PMC)	225
4.3.10. Другие источники информации для наблюдения	229
4.4. sar	231
4.4.1. Область покрытия sar(1)	232
4.4.2. Мониторинг с sar(1)	232
4.4.3. Вывод оперативной информации с помощью sar(1)	236
4.4.4. Документация с описанием sar(1)	236
4.5. Инструменты трассировки	237
4.6. Наблюдение за наблюдаемостью	237
4.7. Упражнения	239
4.8. Ссылки	240
Глава 5. Приложения	241
5.1. Основы приложений	242
5.1.1. Цель	243
5.1.2. Оптимизация общего случая	245
5.1.3. Наблюдаемость	245
5.1.4. Нотация «О-большое»	245
5.2. Методы повышения производительности приложений	247
5.2.1. Выбор размеров блоков ввода/вывода	247
5.2.2. Кэширование	248
5.2.3. Буферизация	248
5.2.4. Опрос	248
5.2.5. Конкурентность и параллелизм	249
5.2.6. Неблокирующий ввод/вывод	254
5.2.7. Привязка к процессору	255
5.2.8. Мантры производительности	255
5.3. Языки программирования	256
5.3.1. Компилируемые языки	256
5.3.2. Интерпретируемые языки	258
5.3.3. Виртуальные машины	259
5.3.4. Сборка мусора	259

5.4. Методология	260
5.4.1. Профилирование процессора.....	261
5.4.2. Анализ времени ожидания вне процессора.....	264
5.4.3. Анализ системных вызовов.....	268
5.4.4. Метод USE.....	269
5.4.5. Анализ состояния потока	270
5.4.6. Анализ блокировок.....	275
5.4.7. Настройка статической производительности	276
5.4.8. Распределенная трассировка.....	277
5.5. Инструменты наблюдения	277
5.5.1. perf.....	278
5.5.2. profile.....	281
5.5.3. offcputime.....	282
5.5.4. strace.....	284
5.5.5. execsnoop.....	286
5.5.6. syscount	287
5.5.7. bpftrace	288
5.6. Проблемы	292
5.6.1. Отсутствие символов	293
5.6.2. Отсутствие стеков	294
5.7. Упражнения.....	295
5.8. Ссылки	297
Глава 6. Процессоры	299
6.1. Терминология	300
6.2. Модели	301
6.2.1. Архитектура процессора.....	301
6.2.2. Кэш-память процессора	302
6.2.3. Очереди на выполнение	303
6.3. Понятия	303
6.3.1. Тактовая частота	304
6.3.2. Инструкции	304
6.3.3. Вычислительный конвейер.....	305
6.3.4. Ширина инструкций.....	305
6.3.5. Размеры инструкций.....	306

6.3.6. SMT	306
6.3.7. IPC, CPI	306
6.3.8. Потребление	307
6.3.9. Время в режиме пользователя/ядра	308
6.3.10. Насыщенность	308
6.3.11. Вытеснение	309
6.3.12. Инверсия приоритета	309
6.3.13. Несколько процессов, несколько потоков	310
6.3.14. Размер слова	312
6.3.15. Оптимизации компилятора	312
6.4. Архитектура	312
6.4.1. Аппаратное обеспечение	312
6.4.2. Программное обеспечение	326
6.5. Методология	330
6.5.1. Метод инструментов	331
6.5.2. Метод USE	332
6.5.3. Определение характеристик рабочей нагрузки	333
6.5.4. Профилирование	335
6.5.5. Анализ тактов	338
6.5.6. Мониторинг производительности	339
6.5.7. Статическая настройка производительности	340
6.5.8. Настройка приоритетов	340
6.5.9. Управление ресурсами	341
6.5.10. Привязка к процессору	342
6.5.11. Микробенчмаркинг	342
6.6. Инструменты наблюдения	343
6.6.1. uptime	344
6.6.2. vmstat	347
6.6.3. mpstat	348
6.6.4. sar	349
6.6.5. ps	350
6.6.6. top	351
6.6.7. pidstat	352
6.6.8. time, ptime	353

6.6.9. turbostat	354
6.6.10. showboost.....	355
6.6.11. pmcarch	355
6.6.12. tlbbstat	356
6.6.13. perf	357
6.6.14. profile	367
6.6.15. cpudist.....	369
6.6.16. runqlat.....	370
6.6.17. runqlen	371
6.6.18. softirqs.....	372
6.6.19. hardirqs.....	373
6.6.20. bpftrace	373
6.6.21. Другие инструменты.....	376
6.7. Методы визуализации	379
6.7.1. Тепловая карта потребления	379
6.7.2. Тепловая карта с субсекундным смещением	380
6.7.3. Флейм-графики	381
6.7.4. FlameScope	384
6.8. Эксперименты	385
6.8.1. Ad hoc.....	386
6.8.2. SysBench	386
6.9. Настройка	387
6.9.1. Параметры компилятора	387
6.9.2. Приоритет и класс планирования	387
6.9.3. Параметры планировщика.....	388
6.9.4. Режимы масштабирования частоты	389
6.9.5. Состояния энергопотребления.....	390
6.9.6. Привязка к процессору.....	390
6.9.7. Исключительные процессорные наборы	390
6.9.8. Управление ресурсами.....	391
6.9.9. Параметры безопасной загрузки.....	391
6.9.10. Параметры процессора (настройка BIOS).....	392
6.10. Упражнения.....	392
6.11. Ссылки.....	393

Глава 7. Память	396
7.1. Терминология	397
7.2. Основные понятия	398
7.2.1. Виртуальная память	398
7.2.2. Подкачка страниц.....	399
7.2.3. Подкачка страниц по требованию	401
7.2.4. Чрезмерное выделение памяти.....	402
7.2.5. Подкачка процессов	403
7.2.6. Использование кэша файловой системы	403
7.2.7. Потребление и насыщение	403
7.2.8. Распределители.....	404
7.2.9. Разделяемая память.....	404
7.2.10. Размер рабочего набора	405
7.2.11. Размер слова	405
7.3. Архитектура	406
7.3.1. Аппаратное обеспечение.....	406
7.3.2. Программное обеспечение	411
7.3.3. Адресное пространство процесса.....	416
7.4. Методология	421
7.4.1. Метод инструментов	422
7.4.2. Метод USE.....	422
7.4.3. Определение характеристик потребления памяти.....	424
7.4.4. Анализ тактов	425
7.4.5. Мониторинг производительности.....	425
7.4.6. Выявление утечек.....	426
7.4.7. Статическая настройка производительности.....	427
7.4.8. Управление ресурсами.....	427
7.4.9. Микробенчмаркинг	428
7.4.10. Уменьшение потребления памяти.....	428
7.5. Инструменты наблюдения.....	428
7.5.1. vmstat	429
7.5.2. PSI.....	431
7.5.3. swapon.....	431
7.5.4. sar	431

7.5.5. slabtop	434
7.5.6. numastat	435
7.5.7. ps	436
7.5.8. top	437
7.5.9. pmap	438
7.5.10. perf	439
7.5.11. drsnoop	443
7.5.12. wss	443
7.5.13. bpftrace	444
7.5.14. Другие инструменты	449
7.6. Настройка	451
7.6.1. Настраиваемые параметры	452
7.6.2. Несколько размеров страниц	454
7.6.3. Распределители	455
7.6.4. Привязка NUMA	455
7.6.5. Управление ресурсами	456
7.7. Упражнения	456
7.8. Ссылки	458
Глава 8. Файловые системы	460
8.1. Терминология	461
8.2. Модели	462
8.2.1. Интерфейсы файловых систем	462
8.2.2. Кэш файловой системы	463
8.2.3. Кэш второго уровня	464
8.3. Основные понятия	464
8.3.1. Задержки в файловой системе	464
8.3.2. Кэширование	465
8.3.3. Произвольный и последовательный ввод/вывод	465
8.3.4. Предварительная выборка	466
8.3.5. Упреждающее чтение	467
8.3.6. Кэширование с отложенной записью	468
8.3.7. Синхронная запись	468
8.3.8. Прямой и низкоуровневый ввод/вывод	469
8.3.9. Неблокирующий ввод/вывод	470

8.3.10. Файлы, отображаемые в память	470
8.3.11. Метаданные.....	471
8.3.12. Логический и физический ввод/вывод.....	472
8.3.13. Неравноценность операций.....	474
8.3.14. Специальные файловые системы.....	475
8.3.15. Доступ к отметкам времени.....	475
8.3.16. Емкость	476
8.4. Архитектура	476
8.4.1. Стек ввода-вывода файловой системы	476
8.4.2. VFS	477
8.4.3. Кэши файловой системы	478
8.4.4. Особенности файловых систем	481
8.4.5. Типы файловых систем	482
8.4.6. Тома и пулы	490
8.5. Методология	491
8.5.1. Анализ дисков.....	492
8.5.2. Анализ задержек.....	492
8.5.3. Определение характеристик рабочей нагрузки.....	495
8.5.4. Мониторинг производительности.....	497
8.5.5. Статическая настройка производительности.....	498
8.5.6. Настройка кэша.....	498
8.5.7. Разделение рабочей нагрузки	498
8.5.8. Микробенчмаркинг	499
8.6. Инструменты наблюдения.....	501
8.6.1. mount.....	502
8.6.2. free.....	502
8.6.3. top.....	503
8.6.4. vmstat	503
8.6.5. sar	503
8.6.6. slabtop	504
8.6.7. strace.....	505
8.6.8. fatrace	505
8.6.9. LatencyTOP	506
8.6.10. opensnoop	507
8.6.11. filetop	508

8.6.12. cachestat	509
8.6.13. ext4dist (xfs, zfs, btrfs, nfs)	510
8.6.14. ext4slower (xfs, zfs, btrfs, nfs)	511
8.6.15. bpftrace	512
8.6.17. Другие инструменты	519
8.6.18. Визуализация	520
8.7. Эксперименты	521
8.7.1. Ad hoc	522
8.7.2. Инструменты микробенчмаркинга	522
8.7.3. Очистка кэша	524
8.8. Настройка	525
8.8.1. Функции	525
8.8.2. ext4	526
8.8.3. ZFS	529
8.9. Упражнения	530
8.10. Ссылки	532
Глава 9. Диски	533
9.1. Терминология	534
9.2. Модели	535
9.2.1. Простой диск	535
9.2.2. Кэширующие диски	536
9.2.3. Контроллер	536
9.3. Основные понятия	537
9.3.1. Измерение времени	537
9.3.2. Масштаб времени	540
9.3.3. Кэширование	541
9.3.4. Произвольный и последовательный ввод/вывод	542
9.3.5. Соотношение операций чтения и записи	543
9.3.6. Размер ввода/вывода	543
9.3.7. Несопоставимость количества операций в секунду	544
9.3.8. Дисковые команды, не связанные с передачей данных	544
9.3.9. Потребление	545
9.3.10. Насыщенность	546
9.3.11. Ожидание ввода/вывода	546

9.3.12. Синхронный и асинхронный ввод/вывод.....	547
9.3.13. Дисковый ввод/вывод и ввод/вывод в приложении.....	548
9.4. Архитектура	548
9.4.1. Типы дисков	548
9.4.2. Интерфейсы	557
9.4.3. Типы хранилищ.....	559
9.4.4. Стек дискового ввода/вывода в операционной системе	563
9.5. Методология	567
9.5.1. Метод инструментов	568
9.5.2. Метод USE.....	568
9.5.3. Мониторинг производительности.....	570
9.5.4. Определение характеристик рабочей нагрузки.....	571
9.5.5. Анализ задержек	573
9.5.6. Статическая настройка производительности.....	574
9.5.7. Настройка кэширования.....	575
9.5.8. Управление ресурсами.....	576
9.5.9. Микробенчмаркинг	576
9.5.10. Масштабирование.....	577
9.6. Инструменты наблюдения.....	578
9.6.1. iostat	579
9.6.2. sar	584
9.6.3. PSI.....	585
9.6.4. pidstat.....	586
9.6.5. perf.....	587
9.6.6. biolatenecy	590
9.6.7. biosnoop.....	592
9.6.8. iotop, biotop.....	594
9.6.9. biostacks	596
9.6.10. blktrace	597
9.6.11. bpftrace	601
9.6.12. MegaCli	605
9.6.13. smartctl.....	606
9.6.14. Журналирование SCSI	607
9.6.15. Другие инструменты.....	608

9.7. Визуализация.....	609
9.7.1. Линейные диаграммы	609
9.7.2. Диаграммы рассеяния задержек	609
9.7.3. Тепловые карты задержек.....	610
9.7.4. Тепловые карты смещений.....	611
9.7.5. Тепловые карты потребления	612
9.8. Эксперименты	612
9.8.1. Ad hoc.....	613
9.8.2. Пользовательские генераторы нагрузки.....	613
9.8.3. Инструменты микробенчмаркинга	614
9.8.4. Пример произвольного чтения.....	614
9.8.5. ioping	615
9.8.6. fio	615
9.8.7. blkreplay	615
9.9. Настройка	616
9.9.1. Параметры операционной системы	616
9.9.2. Настраиваемые параметры дисковых устройств	617
9.9.3. Настраиваемые параметры контроллера диска.....	618
9.10. Упражнения.....	618
9.11. Ссылки.....	619
Глава 10. Сеть.....	622
10.1. Терминология	623
10.2. Модели.....	624
10.2.1. Сетевой интерфейс.....	624
10.2.2. Контроллер.....	624
10.2.3. Стек протоколов.....	625
10.3. Основные понятия.....	626
10.3.1. Сети и маршрутизация.....	626
10.3.2. Протоколы	627
10.3.3. Инкапсуляция.....	628
10.3.4. Размер пакета	628
10.3.5. Задержка	629
10.3.6. Буферизация.....	631
10.3.7. Очередь запросов на подключение	632

10.3.8. Согласование интерфейса.....	632
10.3.9. Предотвращение перегрузки.....	633
10.3.10. Потребление	633
10.3.11. Локальные подключения	634
10.4. Архитектура	635
10.4.1. Протоколы	635
10.4.2. Оборудование.....	643
10.4.3. Программное обеспечение	646
10.5. Методология	655
10.5.1. Метод инструментов.....	655
10.5.2. Метод USE	656
10.5.3. Определение характеристик рабочей нагрузки.....	657
10.5.4. Анализ задержек.....	659
10.5.5. Мониторинг производительности	660
10.5.6. Перехват пакетов.....	661
10.5.7. Анализ TCP	662
10.5.8. Статическая настройка производительности	663
10.5.9. Управление ресурсами	664
10.5.10. Микробенчмаркинг.....	665
10.6. Инструменты наблюдения.....	665
10.6.1. ss	666
10.6.2. ip	668
10.6.3. ifconfig	670
10.6.4. nstat	670
10.6.5. netstat	671
10.6.6. sar	675
10.6.7. nicstat.....	678
10.6.8. ethtool	679
10.6.9. tcplife.....	680
10.6.10. tcptop	681
10.6.11. tcpretrans	682
10.6.12. bpftrace.....	683
10.6.13. tcpdump	691
10.6.14. Wireshark.....	693
10.6.15. Другие инструменты.....	694

10.7. Эксперименты	695
10.7.1. ping	695
10.7.2. traceroute	696
10.7.3. pathchar	697
10.7.4. iperf	698
10.7.5. netperf	699
10.7.6. tc	699
10.7.7. Другие инструменты	700
10.8. Настройка	701
10.8.1. Общесистемные	701
10.8.2. Параметры сокетов	707
10.8.3. Конфигурация	708
10.9. Упражнения	708
10.10. Ссылки	709
Глава 11. Облачные вычисления	714
11.1. Основы	715
11.1.1. Типы экземпляров	716
11.1.2. Масштабируемая архитектура	717
11.1.3. Планирование мощности	718
11.1.4. Хранилище	721
11.1.5. Мультиарендность	722
11.1.6. Оркестровка (Kubernetes)	723
11.2. Виртуализация оборудования	724
11.2.1. Реализация	726
11.2.2. Оверхед	727
11.2.3. Управление ресурсами	734
11.2.4. Наблюдаемость	738
11.3. Виртуализация операционной системы	746
11.3.1. Реализация	748
11.3.2. Оверхед	751
11.3.3. Управление ресурсами	755
11.3.4. Наблюдаемость	760
11.4. Легковесная виртуализация	775
11.4.1. Реализация	776

11.4.2. Оверхед.....	777
11.4.3. Управление ресурсами	777
11.4.4. Наблюдаемость	777
11.5. Другие типы виртуализации.....	779
11.6. Сравнение.....	780
11.7. Упражнения.....	782
11.8. Ссылки.....	783
Глава 12. Бенчмаркинг	786
12.1. Основы.....	787
12.1.1. Причины.....	787
12.1.2. Эффективный бенчмаркинг	788
12.1.3. Проблемы бенчмаркинга.....	790
12.2. Типы бенчмаркинга.....	798
12.2.1. Микробенчмаркинг	799
12.2.2. Моделирование.....	801
12.2.3. Воспроизведение	802
12.2.4. Отраслевые стандарты	803
12.3. Методология.....	805
12.3.1. Пассивный бенчмаркинг	805
12.3.2. Активный бенчмаркинг	806
12.3.3. Профилирование процессора	809
12.3.4. Метод USE	811
12.3.5. Определение характеристик рабочей нагрузки.....	811
12.3.6. Собственный бенчмаркинг	811
12.3.7. Пиковая нагрузка.....	812
12.3.8. Проверка правильности.....	814
12.3.9. Статистический анализ.....	815
12.3.10. Чек-лист бенчмаркинга	816
12.4. Вопросы о бенчмаркинге.....	817
12.5. Упражнения.....	819
12.6. Ссылки.....	819
Глава 13. perf.....	821
13.1. Обзор подкоманд.....	822
13.2. Однострочные сценарии.....	824

13.3. События perf	830
13.4. Аппаратные события.....	832
13.4.1. Дискретная выборка	833
13.5. Программные события	834
13.6. События точек трассировки.....	835
13.7. События зондов	836
13.7.1. kprobes.....	836
13.7.2. uprobes.....	838
13.7.3. USDT	840
13.8. perf stat	842
13.8.1. Параметры	843
13.8.2. Интервальные статистики	844
13.8.3. Баланс между процессорами	844
13.8.4. Фильтрация событий.....	844
13.8.5. Теневые статистики.....	845
13.9. perf record	845
13.9.1. Параметры	846
13.9.2. Профилирование процессора	846
13.9.3. Обход стека	847
13.10. perf report	848
13.10.1. TUI.....	848
13.10.2. STDIO.....	849
13.11. perf script	850
13.11.1. Флейм-графики.....	852
13.11.2. Сценарии обработки трассировок	852
13.12. perf trace	852
13.12.1. Версии ядра.....	853
13.13. Другие команды	854
13.14. Документация perf.....	855
13.15. Ссылки.....	855
Глава 14. Ftrace	857
14.1. Обзор возможностей.....	858
14.2. tracefs (/sys)	861
14.2.1. Содержимое tracefs.....	861

14.3. Профилировщик функций	863
14.4. Трассировщик function.....	865
14.4.1. Использование файла trace	866
14.4.2. Использование файла trace_pipe.....	867
14.4.3. Параметры	868
14.5. Точки трассировки.....	869
14.5.1. Фильтрация.....	870
14.5.2. Триггеры	871
14.6. Зонды kprobes.....	871
14.6.1. Трассировка событий	871
14.6.2. Аргументы	872
14.6.3. Возвращаемые значения.....	873
14.6.4. Фильтры и триггеры	874
14.6.5. Профилировщик kprobe	874
14.7. Зонды uprobes.....	875
14.7.1. Трассировка событий.....	875
14.7.2. Аргументы и возвращаемые значения.....	876
14.7.3. Фильтры и триггеры	876
14.7.4. Профилировщик uprobe	876
14.8. Трассировщик function_graph	876
14.8.1. Трассировка графа	877
14.8.2. Параметры	878
14.9. Трассировщик hwlat	878
14.10. Триггеры hist.....	879
14.10.1. Гистограмма с единственным ключом.....	880
14.10.2. Поля.....	881
14.10.3. Модификаторы	881
14.10.4. Фильтры PID	882
14.10.5. Гистограмма с несколькими ключами	882
14.10.6. Трассировки стека в роли ключей.....	883
14.10.7. Синтетические события	884
14.11. trace-cmd	887
14.11.1. Обзор подкоманд	887
14.11.2. Однострочные сценарии для trace-cmd	889

14.11.3. Сравнение trace-cmd и perf(1).....	891
14.11.4. trace-cmd function_graph.....	892
14.11.5. KernelShark.....	892
14.11.6. Документация для trace-cmd.....	894
14.12. perf ftrace.....	894
14.13. perf-tools.....	895
14.13.1. Покрывтие инструментами.....	895
14.13.2. Специализированные инструменты.....	896
14.13.3. Многоцелевые инструменты.....	898
14.13.4. Однострочные сценарии для perf-tools.....	898
14.13.5. Пример.....	901
14.13.6. Сравнение perf-tools и BCC/BPF.....	901
14.13.7. Документация.....	902
14.14. Документация для Ftrace.....	902
14.15. Ссылки.....	903
Глава 15. BPF.....	904
15.1. BCC.....	907
15.1.1. Установка.....	908
15.1.2. Покрывтие инструментами.....	908
15.1.3. Специализированные инструменты.....	909
15.1.4. Многоцелевые инструменты.....	911
15.1.5. Однострочные сценарии.....	911
15.1.6. Пример многоцелевого инструмента.....	913
15.1.7. Сравнение BCC и bpftrace.....	914
15.1.8. Документация.....	914
15.2. bpftrace.....	915
15.2.1. Установка.....	917
15.2.2. Инструменты.....	917
15.2.3. Однострочные сценарии.....	918
15.2.4. Программирование.....	920
15.2.5. Справочник.....	929
15.2.6. Документация.....	937
15.3. Ссылки.....	937

Глава 16. Пример из практики	939
16.1. Необъяснимый выигрыш	939
16.1.1. Постановка задачи	939
16.1.2. Стратегия анализа.....	940
16.1.3. Статистики	941
16.1.4. Конфигурация	943
16.1.5. Счетчики РМС.....	944
16.1.6. Программные события	945
16.1.7. Трассировка.....	946
16.1.8. Заключение	948
16.2. Дополнительная информация	949
16.3. Ссылки	949
Приложение А. Метод USE: Linux	950
Приложение В. Краткий справочник по sar	957
Приложение С. Однострочные сценарии для bpftrace	959
Приложение D. Решения некоторых упражнений	966
Приложение Е. Производительность систем, кто есть кто	969
Глоссарий	974

*Посвящается Дейдре Страуган,
удивительному специалисту
и удивительному человеку, —
мы сделали это!*

ПРЕДИСЛОВИЕ

Есть известные известные — вещи, о которых мы знаем, что знаем их. Есть также известные неизвестные — вещи, о которых мы знаем, что не знаем. Но еще есть неизвестные неизвестные — это вещи, о которых мы не знаем, что не знаем их.

Министр обороны США Дональд Рамсфельд, 12 февраля 2002 г.

Это заявление на пресс-брифинге встретили с усмешками. Однако оно определяет принцип, одинаково важный и для сложных технических систем, и для геополитики: проблемы с эффективностью могут возникать где угодно, включая те области системы, о которых вы ничего не знаете и поэтому не проверяете (неизвестные неизвестные). Эта книга поможет раскрыть многие из таких областей, а также предоставит методики и инструменты для их анализа.

ОБ ЭТОМ ИЗДАНИИ

Первое издание я написал восемь лет назад и рассчитывал, что оно будет актуально достаточно долго. Главы структурированы так, чтобы сначала охватить то, что постоянно (модели, архитектуры и методологии), а затем то, что быстро меняется (инструменты и настройки). Инструменты и приемы настройки устаревают, но знание базовых вещей поможет всегда оставаться в курсе последних изменений.

За последние восемь лет в Linux появилось большое дополнение: Extended BPF — технология ядра, которая поддерживает новое поколение инструментов анализа производительности и используется в Netflix и Facebook. В это новое издание я включил главу о BPF и инструментах BPF, а также опубликовал подробный справочник по BPF [Gregg 19]. Инструменты `perf` и `Ftrace` в ОС Linux также претерпели множество изменений, и я добавил для них отдельные главы. Ядро Linux получило множество новых технологий и параметров оценки производительности, которые тоже рассматриваются в этом издании книги. Гипервизоры, управляющие облачными виртуальными машинами, и контейнерные технологии тоже существенно изменились, и главы, посвященные им, были обновлены и дополнены.

Первое издание в равной степени охватывало Linux и Solaris. Однако доля рынка Solaris за эти годы сильно сократилась [ITJobsWatch 20], поэтому я почти полностью убрал из этого издания все, что касалось Solaris, освободив место для

дополнительной информации о Linux. Но как мне кажется, возможность сравнения с альтернативами укрепит общее понимание операционной системы или ядра. По этой причине я включил в это издание некоторые упоминания о Solaris и других ОС.

Последние шесть лет я работал старшим перформанс-инженером в Netflix, используя свои знания для оценки производительности микросервисов в Netflix и устранения проблем. Я занимался вопросами производительности гипервизоров, контейнеров, библиотек, ядер, баз данных и приложений. По мере необходимости я разрабатывал новые методологии и инструменты и обменивался опытом с экспертами в области производительности облачных систем и разработки ядра Linux. Все это в немалой степени улучшило второе издание книги.

ОБ ЭТОЙ КНИГЕ

Итак, добро пожаловать во второе издание «Производительности систем»! Книга посвящена производительности (performance) ОС и приложений в контексте операционной системы и охватывает корпоративные серверы и облачные среды. Большая часть информации из книги может пригодиться также для анализа производительности клиентских устройств и ОС настольных компьютеров. Моя цель — помочь вам получить максимальную отдачу от ваших систем, какими бы они ни были.

При работе с прикладным программным обеспечением, находящимся в постоянном развитии, может возникнуть соблазн думать об эффективности ОС, ядро которой разрабатывалось и настраивалось десятилетиями, как о решенной проблеме. Но это не так! Операционная система — это сложный комплекс ПО, управляющего множеством постоянно меняющихся физических устройств и различными прикладными рабочими нагрузками. Ядра тоже находятся в постоянном развитии, в них добавляются новые возможности увеличения производительности определенных рабочих нагрузок, а вновь возникающие узкие места устраняются по мере масштабирования системы. Изменения в ядре, например, устраняющие уязвимости Meltdown и представленные в 2018 году, тоже могут отрицательно сказываться на производительности. Анализ и работа над улучшением производительности ОС — это непрерывный процесс. Производительность приложений тоже можно проанализировать в контексте операционной системы, чтобы отыскать подсказки, которые можно упустить при использовании только инструментов для приложений; об этом я тоже расскажу.

Рассматриваемые операционные системы

Эта книга изучает производительность систем. В роли основного представителя выступают ОС на базе Linux для процессоров Intel. Но книга организована так, чтобы вы могли изучить и другие ядра для других аппаратных архитектур.

Для представленных примеров конкретный дистрибутив Linux не важен, если явно не указано иное. Основная масса примеров была получена в дистрибутиве

Ubuntu, и там, где это важно, в текст включены примечания, объясняющие отличия от других дистрибутивов. В книге приводятся примеры, полученные в системах различных типов: без операционной системы (на «голом железе») и в виртуальных средах, на производственных и тестовых машинах, на серверах и клиентских устройствах.

В своей работе я сталкивался со множеством разных операционных систем и ядер, что углубило мое понимание их дизайна. Чтобы вы могли во всем разобраться, в книгу включены некоторые упоминания о Unix, BSD, Solaris и Windows.

Другие материалы

Примеры скриншотов инструментов анализа производительности включены не только чтобы показать данные, но и для иллюстрации видов доступных данных. Инструменты часто представляют данные простым и понятным способом, многие из них реализованы в стиле, знакомом по более ранним инструментам для Unix. Учитывая это, скриншоты могут быть мощным средством показать назначение этих инструментов почти без дополнительного описания. (Если инструмент требует подробного объяснения, это может быть признаком неудачного дизайна.)

Там, где это уместно, я затрагиваю историю появления определенных технологий. Также полезно узнать немного о ключевых людях в этой отрасли: вы наверняка сталкивались с ними или их работой в сфере эффективности и в других контекстах. Их имена вы найдете в приложении E.

Некоторые темы, рассматриваемые в этом издании, также были освещены в моей предыдущей книге, «BPF Performance Tools»¹ [Gregg 19]: в частности, BPF, BCC, bpftrace, tracepoints, kprobes, uprobes и множество других инструментов на основе BPF. В этой книге вы найдете дополнительную информацию. Краткое изложение этих тем здесь часто основано на той, более ранней книге. Иногда я использую тот же текст и примеры.

О чем здесь не рассказывается

Эта книга посвящена производительности. Для выполнения всех приведенных примеров потребуются некоторые действия по администрированию системы, включая установку или компиляцию программного обеспечения (которые здесь не рассматриваются).

Также здесь кратко описывается внутреннее устройство операционной системы, которое более подробно рассматривается в специализированных изданиях. Вопросы, касающиеся углубленного анализа производительности, здесь рассмотрены в общих чертах — лишь для того, чтобы вы знали об их существовании и могли заняться их

¹ Грегг Б. «BPF: Профессиональная оценка производительности». Выходит в издательстве «Питер» в 2023 году.

изучением по другим источникам. См. раздел «Дополнительные источники, ссылки и библиография» в конце предисловия.

Структура

Глава 1 «Введение» — это введение в анализ производительности системы. Глава обобщает ключевые идеи и приводит примеры действий, направленных на улучшение производительности.

Глава 2 «Методологии» формирует основу для анализа и настройки производительности и описывает терминологию, основные понятия, модели, методологии для наблюдения и экспериментов, планирование емкости, анализ и статистику.

Глава 3 «Операционные системы» описывает внутреннее устройство ядра с точки зрения анализа производительности. Глава закладывает основы, необходимые для интерпретации и понимания действий операционной системы.

Глава 4 «Инструменты наблюдения» знакомит с доступными средствами наблюдения за системой, а также интерфейсами и фреймворками, на которых они построены.

Глава 5 «Приложения» обсуждает вопросы производительности приложений и наблюдение за ними из операционной системы.

Глава 6 «Процессоры» рассматривает процессоры, ядра, аппаратные потоки выполнения, кэш-память процессора, взаимосвязи между процессорами, взаимосвязи между устройствами и механизмы планирования в ядре.

Глава 7 «Память» посвящена виртуальной памяти, страничной организации, механизму подкачки, архитектурам памяти, шинам, адресным пространствам и механизмам распределения памяти.

Глава 8 «Файловые системы» посвящена производительности операций ввода/вывода с файловой системой, включая различные механизмы кэширования.

Глава 9 «Диски» описывает устройства хранения, рабочие нагрузки дискового ввода/вывода, контроллеры хранилищ, дисковые массивы RAID и подсистему ввода/вывода ядра.

Глава 10 «Сеть» посвящена сетевым протоколам, сокетам, интерфейсам и физическим соединениям.

Глава 11 «Облачные вычисления» знакомит с методами виртуализации операционных систем и оборудования, которые обычно используются для организации облачных вычислений, а также с их характеристиками производительности, изоляции и наблюдаемости. В этой главе также рассматриваются гипервизоры и контейнеры.

Глава 12 «Бенчмаркинг» показывает, как правильно проводить сравнительный анализ и как интерпретировать результаты бенчмаркинга. Это на удивление сложная тема, и в этой главе я покажу, как избежать типичных ошибок, и попробую объяснить их смысл.

Глава 13 «perf» кратко описывает стандартный профилировщик Linux, perf(1), и его многочисленные возможности. Ссылки на этот справочник по perf(1) вы будете встречать по всей книге.

Глава 14 «Ftrace» кратко описывает стандартное средство трассировки Linux, Ftrace, которое особенно подходит для исследования особенностей работы ядра.

Глава 15 «BPF» кратко описывает стандартные внешние интерфейсы BPF: BCC и bpftrace.

Глава 16 «Пример из практики» содержит пример исследования производительности системы, проводившегося в Netflix. Он показывает, как проводился анализ производительности.

Главы 1–4 содержат важную базовую информацию. Прочитав их, вы будете готовы перейти к любой из остальных глав в книге, в частности к главам 5–12, в которых рассматриваются конкретные цели для анализа.

Главы 13–15 посвящены продвинутому профилированию и трассировке и являются факультативным чтением для тех, кто хочет подробнее изучить один или несколько трассировщиков.

В главе 16 я привожу истории, которые позволят сформировать более широкое представление о работе перформанс-инженера. Если вы только начинаете заниматься анализом производительности, то можете прочитать сначала эту главу. Она покажет пример анализа производительности с использованием множества различных инструментов. Вы можете вернуться к ней после прочтения других глав.

Применимость в будущем

Я писал эту книгу так, чтобы ее полезность сохранялась на долгие годы. Основное внимание в ней уделяется опыту и методологиям анализа производительности систем.

Для этого многие главы были разделены на две части. Первая часть включает определения, описание понятий и методологий (часто соответствующие разделы именно так и называются), которые должны оставаться актуальными на протяжении многих лет. Во второй части приводятся примеры реализации: архитектура, инструменты анализа и настраиваемые параметры. Они могут устареть, но все же останутся полезными в качестве примеров.

Примеры трассировки

Часто требуется глубоко изучить операционную систему, чтобы знать, что можно сделать с помощью инструментов трассировки.

Уже после выхода первого издания была разработана и внедрена в ядро Linux расширенная технология BPF, в результате чего появилось новое поколение инструментов трассировки, использующих внешние интерфейсы BCC и bpftrace. Эта книга посвящена BCC и bpftrace, а также встроенному в ядро Linux трассировщику Ftrace. BPF, BCC и bpftrace более подробно описаны в моей предыдущей книге [Gregg 19].

В книге рассматривается еще один инструмент трассировки Linux — perf. Но perf здесь в основном используется для получения и анализа счетчиков контроля производительности (performance monitoring counter, PMC), а не для трассировки.

Возможно, вы захотите использовать другие инструменты трассировки, и это нормально. Представленные в этой книге инструменты показывают, какие вопросы можно задать системе. Часто именно эти вопросы и методологии, которые их ставят, труднее всего понять.

Для кого эта книга

В первую очередь книга адресована системным администраторам и операторам корпоративных и облачных вычислительных сред. Она послужит справочником для разработчиков, администраторов баз данных и администраторов веб-серверов, которые должны понимать, из чего складывается производительность операционных систем и приложений.

Как перформанс-инженеру в компании с гигантской вычислительной инфраструктурой (Netflix), мне часто приходится работать с SRE-инженерами и разработчиками, которым просто физически не хватает времени для решения сразу нескольких проблем с производительностью. Мне тоже доводилось работать дежурным инженером в Netflix CORE SRE, и я знаком с этой нехваткой времени на собственном опыте. Для многих людей обеспечение производительности не является их основной работой, и им достаточно знать ровно столько, чтобы решать текущие проблемы. Я понимаю, что у вас может быть мало времени, и я постарался сделать эту книгу как можно короче и проще по структуре.

Другая целевая аудитория — студенты. Книга поможет при изучении курса производительности систем. Я вел подобные курсы раньше и знаю, что лучше всего помогает студентам решать проблемы с успеваемостью. Этими знаниями я руководствовался при работе над книгой.

Кем бы вы ни были, упражнения в главах позволят проверить себя и надежнее усвоить материал. Среди них вы найдете особенно сложные упражнения, которые необязательно решать. (Они могут представлять проблемы, не имеющие решения, главная их цель — заставить задуматься.)

Наконец, с точки зрения размера компании эта книга содержит достаточно подробностей, чтобы удовлетворить потребности компаний от мала до велика. Для многих небольших компаний эта книга послужит справочником, в котором лишь некоторые части используются ежедневно.

Условные обозначения

В этой книге используются следующие условные обозначения:

Пример	Описание
<code>netif_receive_skb()</code>	Имя функции
<code>iostat(1)</code>	Команда со ссылкой на раздел в справочном руководстве <code>man</code> с ее описанием
<code>read(2)</code>	Системный вызов со ссылкой на раздел в справочном руководстве <code>man</code> с его описанием
<code>malloc(3)</code>	Имя функции из библиотеки языка C со ссылкой на раздел в справочном руководстве <code>man</code> с ее описанием
<code>vmstat(8)</code>	Команда администрирования со ссылкой на раздел в справочном руководстве <code>man</code> с ее описанием
<code>Documentation/...</code>	Каталог с документацией в дереве исходных текстов ядра Linux
<code>kernel/...</code>	Каталог в дереве исходных текстов ядра Linux
<code>fs/...</code>	Каталог с реализацией файловой системы в дереве исходных текстов ядра Linux
<code>CONFIG_...</code>	Параметры настройки ядра Linux (<code>Kconfig</code>)
<code>r_await</code>	Команда в командной строке и ее вывод
<code>mpstat 1</code>	Команда или ключевая деталь, на которую следует обратить особое внимание
<code>#</code>	Приглашение к вводу в командной оболочке суперпользователя (<code>root</code>)
<code>\$</code>	Приглашение к вводу в командной оболочке обычного пользователя (не <code>root</code>)
<code>^C</code>	Прерывание выполнения команды (комбинацией клавиш <code>Ctrl-C</code>)
<code>[...]</code>	Усечение

Дополнительные источники, ссылки и библиография

Список литературы приводится в конце каждой главы, а не в конце всей книги. Вы сразу можете смотреть источники, относящиеся к теме каждой главы. В списке ниже перечислены книги, из которых можно почерпнуть информацию об ОС и анализе производительности:

[Jain 91] Jain, R., «The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling», Wiley, 1991.

[Vahalia 96] Vahalia, U., «UNIX Internals: The New Frontiers», Prentice Hall, 1996.¹

[Cockcroft 98] Cockcroft, A., and Pettit, R., «Sun Performance and Tuning: Java and the Internet, Prentice Hall», 1998.

¹ Вахалия Ю. «UNIX изнутри». СПб.: Издательство «Питер».

- [Musumeci 02] Musumeci, G. D., and Loukides, M., «System Performance Tuning, 2nd Edition», O'Reilly, 2002.¹
- [Bovet 05] Bovet, D., and Cesati, M., «Understanding the Linux Kernel, 3rd Edition», O'Reilly, 2005.²
- [McDougall 06a] McDougall, R., Mauro, J., and Gregg, B., «Solaris Performance and Tools: DTrace and MDB Techniques for Solaris 10 and OpenSolaris», Prentice Hall, 2006.
- [Gove 07] Gove, D., «Solaris Application Programming», Prentice Hall, 2007.
- [Love 10] Love, R., «Linux Kernel Development, 3rd Edition», Addison-Wesley, 2010.³
- [Gregg 11a] Gregg, B., and Mauro, J., «DTrace: Dynamic Tracing in Oracle Solaris, Mac OS X and FreeBSD», Prentice Hall, 2011.
- [Gregg 13a] Gregg, B., «Systems Performance: Enterprise and the Cloud», Prentice Hall, 2013 (first edition).
- [Gregg 19] Gregg, B., «BPF Performance Tools: Linux System and Application Observability»⁴, Addison-Wesley, 2019.
- [ITJobsWatch 20] ITJobsWatch, «Solaris Jobs», https://www.itjobswatch.co.uk/jobs/uk/solaris.do#demand_trend, ссылка была действительна в феврале 2021.

¹ Мусумеси Дж.-П. Д. Лукидес М. «Настройка производительности UNIX-систем».

² Бовет Д., Чезати М. «Ядро Linux».

³ Лав Р. «Ядро Linux. Описание процесса разработки».

⁴ Грегг Б. «BPF: Профессиональная оценка производительности». Выходит в издательстве «Питер» в 2023 году.

БЛАГОДАРНОСТИ

Спасибо всем купившим первое издание, и особенно тем, кто рекомендовал прочитать его своим коллегам. Поддержка первой книги способствовала созданию второй. Спасибо вам.

Это моя последняя книга, посвященная производительности систем, но не первая. Я хотел бы поблагодарить авторов других книг по этой же тематике, на которые я опирался и на которые неоднократно ссылаюсь здесь. В частности, хотел бы поблагодарить Адриана Кокрофта (Adrian Cockcroft), Джима Мауро (Jim Mauro), Ричарда Макдугалла (Richard McDougall), Майка Лукидеса (Mike Loukides) и Раджа Джейна (Raj Jain). Все вы здорово помогли мне, и я надеюсь, что смогу помочь вам.

Я благодарен всем, кто давал мне обратную связь:

Дейдра Страуган (Deirdré Straughan) всячески поддерживала меня и использовала свой богатый опыт редактирования технических книг, чтобы каждая страница этой книги стала лучше. Слова, которые вы читаете, принадлежат нам обоим. Нам нравится не только вместе проводить время (сейчас мы женаты), но и работать. Спасибо.

Филиппу Мареку (Philipp Marek) — эксперту в информационных технологиях, ИТ-архитектору и перформанс-инженеру в Австрийском федеральном вычислительном центре. Он одним из первых делился мнением по каждой теме этой книги (настоящий подвиг) и даже обнаружил проблемы в тексте первого издания. Филипп начал программировать в 1983 году, еще для микропроцессора 6502, и практически сразу стал искать способы экономии тактов процессора. Спасибо, Филипп, за твой опыт и неустанную работу!

Дейл Хэмел (Dale Hamel, Shopify) тоже внимательно прочитал каждую главу, предоставил важные сведения о различных облачных технологиях и помог взглянуть на книгу другими глазами. Спасибо тебе, Дейл, что взял на себя этот труд! Это случилось сразу после того, как ты помог мне с книгой о BPF.

Даниэль Боркманн (Daniel Borkmann, Isovalent) тщательно прорецензировал некоторые главы, в частности главы о сетях. Это помогло мне лучше понять имеющиеся сложности и сопутствующие технологии. Даниэль уже много лет занимается сопровождением ядра Linux и обладает огромным опытом работы над сетевым стеком ядра и расширенным BPF. Спасибо тебе, Даниэль, за профессионализм и строгость оценок.

Я особенно благодарен мейнтейнеру инструмента perf Арнальдо Карвалью де Мело (Arnaldo Carvalho de Melo; Red Hat) за помощь с главой 13 «perf» и Стивену Ростедту (Steven Rostedt; VMware), создателю Ftrace, за помощь с главой 14

«Ftrace» — двумя темами, которые я недостаточно полно рассмотрел в первом издании. Я высоко ценю их не только за помощь в написании этой книги, но также за их работу над этими инструментами повышения производительности, которые я использовал для решения бесчисленных проблем в Netflix.

Было очень приятно, что Доминик Кэй (Dominic Kay) пролистал несколько глав и оставил множество советов по улучшению читаемости и повышению технической точности. Доминик также помогал мне с первым изданием (а еще раньше мы вместе работали в Sun Microsystems, где занимались вопросами производительности). Спасибо, Доминик.

Мой нынешний коллега по Netflix, Амер Атер (Amer Ather), оставил очень ценные отзывы к нескольким главам. Амер — инженер, разбирающийся в сложных технологиях. Захарий Джонс (Zachary Jones, Verizon) тоже дал обратную связь по особенно сложным вопросам и поделился опытом в области производительности, чем очень помог улучшить книгу. Спасибо вам, Амер и Захарий.

Несколько рецензентов, взяв несколько глав, участвовали в обсуждении конкретных тем: Алехандро Проаньо (Alejandro Proaño, Amazon), Бикаш Шарма (Bikash Sharma, Facebook), Кори Луенингонер (Cory Lueninghoener, Национальная лаборатория в Лос-Аламосе), Грег Данн (Greg Dunn, Amazon), Джон Аппасджид (John Arrasjid, Ottometric), Джастин Гаррисон (Justin Garrison, Amazon), Майкл Хаузенблас (Michael Hausenblas, Amazon) и Патрик Кейбл (Patrick Cable, Threat Stack). Спасибо всем за вашу помощь и энтузиазм.

Также спасибо Адитье Сарваде (Aditya Sarwade, Facebook), Эндрю Галлатину (Andrew Gallatin, Netflix), Басу Смиты (Bas Smit), Джорджу Невиллу-Нилу (George Neville-Neil, JUUL Labs), Йенсу Аксбоу (Jens Axboe, Facebook), Джоэлю Фернандесу (Joel Fernandes, Google), Рэндаллу Стюарту (Randall Stewart, Netflix), Стефану Эраниану (Stephane Eranian, Google) и Токе Хойланд-Йоргенсену (Toke Høiland-Jørgensen, Red Hat) за ответы на вопросы и своевременную техническую помощь.

Те, кто участвовал в создании моей предыдущей книги — «BPF Performance Tools», тоже косвенно помогли мне, потому что некоторые материалы этого издания основаны на предыдущем. Улучшению той книги в немалой степени способствовали Аластер Робертсон (Alastair Robertson, Yellowbrick Data), Алексей Старовойтов (Alexei Starovoitov, Facebook), Дэниел Боркманн (Daniel Borkmann), Джейсон Кох (Jason Koch, Netflix), Мэри Марчини (Mary Marchini, Netflix), Масами Хирамацу (Masami Hiramatsu, Linaro), Мэтью Дезнойерс (Mathieu Desnoyers, EfficiOS), Йонгхонг Сонг (Yonghong Song, Facebook) и многие другие. Полный список вы найдете в разделе «Благодарности» предыдущего издания.

Многие, помогавшие мне в работе над первым изданием, помогли в работе и над этим. В частности, я получил техническую поддержку по нескольким главам от Адама Левенталя (Adam Leventhal), Карлоса Карденаса (Carlos Cardenas), Дэррила Гоува (Darryl Gove), Доминика Кэя (Dominic Kay), Джерри Елинека (Jerry Jelinek), Джима Мауро (Jim Mauro), Макса Брунинга (Max Bruning), Ричарда Лоу (Richard Lowe) и Роберта Мустаччи (Robert Mustacchi). Я также получил отзывы

и поддержку от Адриана Кокрофта (Adrian Cockcroft), Брайана Кантрилла (Bryan Cantrill), Дэна Макдональда (Dan McDonald), Дэвида Пачеко (David Pacheco), Кита Весоловски (Keith Wesolowski), Марселля Кукульевича-Пирса (Marsell Kukuljevic-Pearce) и Пола Эгглтона (Paul Eggleton). Рох Бурбоннис (Roch Bourbonnais) и Ричард Макдугалл (Richard McDougall) многому научили меня на моей предыдущей работе, где мы занимались проблемами производительности, и тем самым оказали косвенную помощь в работе над этой книгой, как и Джейсон Хоффман (Jason Hoffman), косвенно помогавший в работе над первым изданием.

Ядро Linux — сложный и постоянно меняющийся программный продукт, и я ценю труд Джонатана Корбета (Jonathan Corbet) и Джейка Эджа (Jake Edge) из lwn.net по обобщению большого числа сложнейших тем. Многие из их статей упоминаются в этой книге.

Отдельное спасибо Грегу Доенчу (Greg Doench) — выпускающему редактору издательства Pearson за гибкость и поддержку, благодаря которым процесс двигался особенно эффективно. Спасибо продюсеру информационного наполнения Джулии Нахил (Julie Nahil; Pearson) и менеджеру проекта Рэйчел Пол (Rachel Paul) за внимание к деталям и помощь в создании качественного продукта. Спасибо редактору Киму Уимпсетту (Kim Wimpsett) за работу над еще одной из моих длинных и глубоко технических книг, за множество предложений по улучшению текста.

И спасибо тебе, Митчелл, за терпение и понимание.

Начиная с первого издания, я продолжал работать перформанс-инженером, устраняя проблемы по всему программно-аппаратному стеку. Теперь у меня еще больше опыта в работе с гипервизорами, в настройке производительности, в анализе среды выполнения (включая JVM), в применении трассировщиков, включая Ftrace и BPF, а также в реагировании на быстро меняющиеся микросервисы Netflix и ядро Linux. Много из этого недостаточно хорошо задокументировано, и порой было очень сложно определить, что отразить в книге. Но я люблю сложности.

ОБ АВТОРЕ

Брендан Грегг — эксперт в области производительности и облачных вычислений. Работает старшим перформанс-инженером в Netflix, где занимается проектированием, оценкой, анализом и настройкой производительности. Автор нескольких книг, в том числе «BPF Performance Tools¹». Обладатель награды USENIX LISA за выдающиеся достижения в системном администрировании. Работал инженером по поддержке ядра, руководил командой обеспечения производительности и профессионально занимался преподаванием технических дисциплин, был сопредседателем конференции USENIX LISA 2018. Создал множество инструментов оценки производительности для разных операционных систем, а также разработал средства и методы визуализации для анализа производительности, включая флейм-графики.

ОТ ИЗДАТЕЛЬСТВА

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

¹ Грегг Б. «BPF: Профессиональная оценка производительности». Выходит в издательстве «Питер» в 2023 году.

Глава 1

ВВЕДЕНИЕ

Производительность компьютеров — увлекательная, многообразная и сложная дисциплина. В этой главе вы познакомитесь с понятием эффективности и производительности систем.

Цели главы:

- познакомить с понятием производительности системы, кто и как ее обеспечивает и какие сложности встречаются на этом пути;
- показать разницу между инструментами наблюдения и инструментами проведения экспериментов;
- дать общее представление о способах и средствах оценки производительности, таких как параметры, профилирование, флейм-графики, трассировка, статические и динамические инструменты;
- представить роль методологий и короткий чек-лист для исследования производительности Linux.

Здесь будут даны ссылки на последующие главы, поэтому данную главу можно считать введением и в дисциплину производительности систем, и в книгу в целом. Глава заканчивается реальными примерами, показывающими, насколько важно уделять внимание производительности систем.

1.1. ПРОИЗВОДИТЕЛЬНОСТЬ СИСТЕМЫ

Под оценкой производительности системы понимается изучение производительности всей компьютерной системы, включая основные программные и аппаратные компоненты, — все, что находится на пути к данным, от устройств хранения до прикладного программного обеспечения, — потому что все они могут влиять на производительность. В случае с распределенными системами в этот список также входят все серверы и приложения. Если у вас нет схемы вашего окружения, отражающей путь к данным, найдите ее или нарисуйте сами; она поможет увидеть взаимосвязи между компонентами и не упустить из виду целые области.

Типичная цель оценки производительности системы — улучшить взаимодействие с конечным пользователем за счет уменьшения задержек и снижения затрат на

вычисления. Снижения затрат можно достигнуть за счет устранения неэффективности, повышения пропускной способности и общей настройки системы.

На рис. 1.1 показан обобщенный стек системного ПО на одном сервере, включая ядро операционной системы (ОС), базу данных и приложение. Иногда термин *фуллстек* используется для описания только прикладного окружения, включающего базы данных, приложения и веб-серверы. Но говоря о производительности системы, под словом *фуллстек* мы подразумеваем весь программный стек — от приложений до железа (оборудования), включая системные библиотеки, ядро и само оборудование. При оценке производительности системы рассматривается фуллстек.



Рис. 1.1. Обобщенный стек системного программного обеспечения

Компиляторы включены в обобщенный стек на рис. 1.1, потому что играют важную роль в производительности системы. Этот стек мы обсудим в главе 3 «Операционные системы» и подробно рассмотрим в последующих главах. В следующих разделах будут более подробно описаны особенности оценки производительности системы.

1.2. РОЛИ

Производительность системы обеспечивается различными специалистами, в том числе системными администраторами, SRE-инженерами, разработчиками приложений, сетевыми инженерами, администраторами баз данных, веб-администраторами и др. Для многих из этих специалистов обеспечение производительности является лишь

частью их работы, поэтому при оценке производительности они фокусируются только на своей сфере ответственности: группа сетевых администраторов проверяет производительность сетевого стека, группа администраторов баз данных проверяет базу данных, и т. д. Однако иногда для выяснения причин низкой производительности или факторов, способствующих этому, требуются совместные усилия нескольких команд.

В некоторых компаниях работают *перформанс-инженеры*, для которых обеспечение высокой производительности — основная деятельность. Они могут взаимодействовать с несколькими командами для проведения комплексного исследования среды, что нередко очень важно для решения сложных проблем с производительностью. Они также могут выступать в качестве центрального звена, осуществляющего поиск и разработку инструментов для анализа производительности и планирования ресурсов во всей среде.

Например, в Netflix есть команда по производительности облачных вычислений, в которую вхожу и я. Мы помогаем командам микросервисов и обеспечения надежности анализировать производительность и создаем инструменты оценки производительности для всех остальных.

Компании, нанимающие несколько перформанс-инженеров, могут позволить им специализироваться в одной или нескольких областях и обеспечивать более глубокую поддержку. Например, большая группа перформанс-инженеров может включать специалистов по производительности ядра, клиентских приложений, языка (например, Java), среды выполнения (например, JVM), по разработке инструментов для оценки производительности и т. д.

1.3. ДЕЙСТВИЯ

Анализ и увеличение производительности системы предполагают выполнение множества действий. Ниже приводится список таких действий, которые одновременно представляют этапы идеального жизненного цикла программного проекта — от идеи до разработки и развертывания в продакшене. В этой книге описаны методологии и инструменты, помогающие выполнять эти действия.

1. Определение целей и моделирование производительности будущего продукта.
2. Определение характеристик производительности прототипа программного и аппаратного обеспечения.
3. Анализ производительности разрабатываемых продуктов в тестовой среде.
4. Регрессионное тестирование новых версий продукта.
5. Бенчмаркинг производительности разных версий продуктов.
6. Проверка концепции в целевой промышленной среде.
7. Оптимизация производительности в промышленной среде.
8. Мониторинг ПО, действующего в промышленной среде.
9. Анализ производительности промышленных задач.

10. Ревью инцидентов, возникающих в промышленной среде.
11. Разработка инструментов для повышения эффективности анализа производительности в промышленной среде.

Шаги с 1-го по 5-й охватывают традиционный процесс разработки продукта, будь то продукт, продаваемый клиентам или используемый внутри компании. После разработки продукт вводится в эксплуатацию, иногда сначала проверяется пригодность использования продукта в целевой среде (клиента или внутри компании), а иногда сразу же производится развертывание и настройка. Если в целевой среде обнаружится проблема (шаги с 6-го по 9-й), это говорит только о том, что она не была обнаружена или исправлена на этапах разработки.

В идеале проектирование производительности должно начинаться до выбора какого-либо оборудования или создания программного обеспечения: первым шагом должны быть постановка целей и создание модели производительности. Однако часто продукты разрабатываются, минуя этот шаг, из-за чего работы по проектированию производительности откладываются на более позднее время, когда проблемы уже возникнут. С каждым следующим этапом процесса разработки становится все труднее устранять проблемы с производительностью, обусловленные ранее принятыми архитектурными решениями.

Облачные вычисления предлагают новые методы проверки концепции (шаг 6), которые побуждают пропускать более ранние шаги (с 1-го по 5-й). Один из таких методов — тестирование нового ПО на единственном экземпляре с небольшой рабочей нагрузкой: это называется *канареечным тестированием*. Другой метод превращает его в обычный шаг при развертывании: трафик постепенно перемещается в новый пул экземпляров, при этом старый пул остается в горячем резерве; этот метод известен как *синее-зеленое развертывание*¹. Применение таких приемов защиты от сбоев в новом ПО позволяет проводить тестирование в продакшене без всякого предварительного анализа производительности и при необходимости быстро возвращаться к исходному состоянию. Но я рекомендую всегда, когда это возможно, выполнить также первые этапы, чтобы достичь максимальной производительности (даже при том, что иногда могут быть веские причины миновать их, например, быстрый выход на рынок).

Некоторые из перечисленных этапов охватываются термином *планирование мощности* (capacity planning). На этапе проектирования он подразумевает изучение объема ресурсов, необходимых разрабатываемому ПО, чтобы увидеть, насколько полно его архитектура может удовлетворить целевые потребности. После развертывания он подразумевает мониторинг использования ресурсов для прогнозирования проблем до их возникновения.

В анализ производительности в промышленной среде (шаг 9) могут также вовлекаться SRE-инженеры. Далее следует шаг по ревью инцидентов в промышленной среде (шаг 10), когда производится анализ произошедшего, обмен опытом отладки

¹ В Netflix используется термин *красно-черное развертывание*.

и поиск способов избежать аналогичных инцидентов в будущем. Эти ревью похожи на *ретроспективы* разработчиков (см. [Соггу 20], где рассказывается, что такое ретроспективы и их антипаттерны).

Среды и мероприятия различаются для разных компаний и продуктов, и во многих случаях выполняются не все десять шагов. Ваша работа также может быть сосредоточена только на некоторых или только на одном из этих мероприятий.

1.4. ПЕРСПЕКТИВЫ

Помимо выполнения различных мероприятий, привлечение различных специалистов можно рассматривать как использование разных точек зрения — перспектив. На рис. 1.2 обозначены две точки зрения на анализ производительности: *анализ рабочей нагрузки* и *анализ ресурсов* — двух подходов к оценке программного стека.

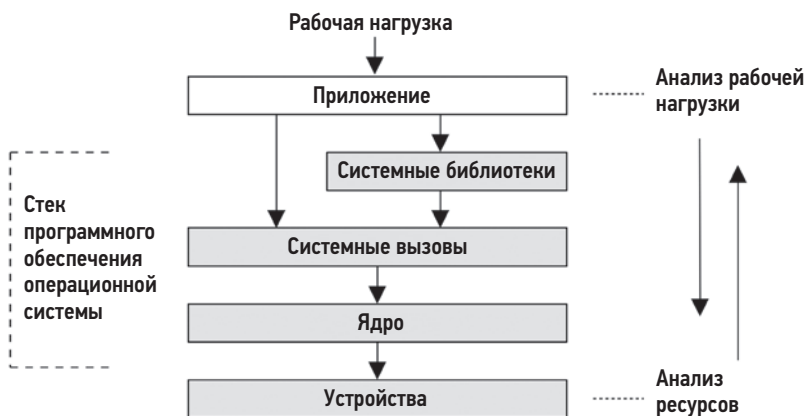


Рис. 1.2. Анализ с двух перспектив

Перспектива анализа ресурсов обычно используется системными администраторами, отвечающими за системные ресурсы. Разработчики приложений, отвечающие за производительность под рабочей нагрузкой, обычно сосредотачиваются на перспективе анализа рабочей нагрузки. Каждая перспектива имеет свои сильные стороны, которые подробно обсуждаются в главе 2 «Методологии». При решении сложных вопросов полезно попытаться проанализировать ситуацию с обеих сторон.

1.5. СЛОЖНОСТИ ОЦЕНКИ ПРОИЗВОДИТЕЛЬНОСТИ

Проектирование производительности системы — сложная область по многим причинам, в том числе из-за субъективности, природной сложности, отсутствия какой-то единственной основной причины и наличия множества связанных проблем.

1.5.1. Субъективность

Технические дисциплины тяготеют к объективности, причем настолько, что занимающиеся ими люди видят мир в черно-белом цвете. Это может быть верно в отношении неполадок в софте, когда ошибка либо есть, либо ее нет и либо она исправлена, либо не исправлена. Такие ошибки часто проявляются в виде сообщений, которые легко интерпретировать и идентифицировать как сообщения об ошибках.

Производительность, напротив, часто бывает *субъективной*. В отношении проблем с производительностью часто неочевидно, имела ли место проблема изначально, и если да, то когда она была устранена. Один пользователь может считать производительность «плохой» и рассматривать ее как проблему, а другой может считать ее «хорошей».

Например, представьте, что вам поступила такая информация:

Среднее время отклика подсистемы дискового ввода/вывода составляет 1 мс.

Это «хорошо» или «плохо»? Время отклика, или задержка, является одним из лучших доступных показателей, но интерпретировать информацию о задержке сложно. Часто выбор между оценками «хорошая» или «плохая» зависит от ожиданий разработчиков приложений и конечных пользователей.

Субъективную оценку производительности можно сделать объективной, определив четкие цели, например целевое среднее время отклика или попадание определенного процента запросов в некоторый диапазон задержек. Другие способы борьбы с субъективностью будут представлены в главе 2 «Методологии», в том числе и анализ задержки.

1.5.2. Сложность

Помимо субъективности, сложность оценки производительности может быть связана со свойственной системам сложностью и отсутствием очевидной отправной точки для анализа. В облачных средах порой даже нет возможности узнать, на какой экземпляр обратить внимание в первую очередь. Иногда мы начинаем с выдвижения гипотезы, например, обвиняя сеть или базу данных, а аналитик производительности должен выяснить, является ли эта гипотеза верной.

Проблемы с производительностью также могут возникать из-за сложных взаимодействий между подсистемами, которые показывают хорошие характеристики при их анализе в изоляции друг от друга. Падение производительности может произойти из-за каскадного сбоя, когда один отказавший компонент вызывает проблемы с производительностью в других компонентах. Чтобы понять причину возникшей проблемы, необходимо распутать клубок взаимосвязей между компонентами и понять, какой вклад они вносят.

Сложность также может быть обусловлена наличием неожиданных взаимосвязей, когда устранение проблемы в одном месте может привести к появлению проблемы

в другом месте системы, при этом общая производительность улучшится не так сильно, как ожидалось. Помимо сложности системы, проблемы с производительностью также могут быть вызваны сложным характером производственной нагрузки. Некоторые случаи могут просто не воспроизводиться в лабораторных условиях или возникать лишь периодически.

Решение сложных проблем производительности часто требует комплексного подхода. Возможно, потребуется исследовать всю систему — и ее внутренние элементы, и внешние взаимодействия. Для этого необходимо обладать широким спектром навыков, что может сделать проектирование производительности многообразным и интеллектуально сложным делом.

Для преодоления этих сложностей можно использовать разные методологии, как описано в главе 2. В главах 6–10 вы найдете описания конкретных методологий анализа конкретных системных ресурсов: процессоры, память, файловые системы, диски и сеть. (Комплексный анализ сложных систем, включая разливы нефти и крах финансовых систем, описан в [Dekker 18].)

В некоторых случаях проблема производительности может быть вызвана взаимодействием этих ресурсов.

1.5.3. Множественные причины

Некоторые проблемы с производительностью не имеют единственной первопричины и обусловлены множеством факторов. Представьте сценарий, когда одновременно происходят три вполне обычных события, которые в совокупности вызывают проблему с производительностью: каждое из этих событий — обычное и само по себе не является первопричиной.

Множественными могут быть не только причины, но и сами проблемы с производительностью.

1.5.4. Множественные проблемы с производительностью

В сложном программном обеспечении обычно бывает немало проблем с производительностью. Для примера попробуйте найти базу данных ошибок для вашей ОС или приложений и поищите по слову *performance* (производительность). Результаты могут удивить вас! Как правило, в таких базах данных даже для зрелого ПО, считающегося высокопроизводительным, есть множество известных, но пока не исправленных проблем с производительностью. Это создает еще одну трудность при анализе: настоящая задача не в том, чтобы найти проблему, а в том, чтобы определить, какая проблема или проблемы *наиболее важны*.

Для этого специалист по анализу производительности должен *количественно* оценить масштаб проблемы. Некоторые проблемы с производительностью могут быть не свойственны вашей рабочей нагрузке или проявляться в весьма незначительной степени. В идеале вы должны не только количественно оценить проблемы, но также

оценить потенциальное ускорение, которое можно получить за счет устранения каждой из них. Эта информация может пригодиться, когда менеджеры будут искать оправдание расходам на инженерные или операционные ресурсы.

Один из показателей, хорошо подходящих для количественной оценки производительности, если он доступен, — это *задержка*.

1.6. ЗАДЕРЖКА

Задержка характеризует время, затраченное на ожидание, и является важной метрикой производительности. В широком смысле под задержкой понимается время до завершения любой операции, такой как обработка запроса приложением или базой данных, операция файловой системы и т. д. Например, задержка может выражать время полной загрузки веб-страницы от щелчка на ссылке до появления полного изображения страницы на экране. Это важный показатель как для клиента, так и для владельца сайта: большая задержка может вызвать разочарование и желание у клиентов разместить заказ в другом месте.

В роли метрики задержка позволяет оценить максимальное ускорение. Например, на рис. 1.3 изображена временная диаграмма обработки запроса к базе данных, на что уходит 100 мс (это время и является задержкой), из которых 80 мс тратится на ожидание завершения операции чтения с диска. В данном случае за счет исключения операций чтения с диска (например, путем кэширования) можно добиться уменьшения времени обработки со 100 мс до 20 мс ($100 - 80$), то есть добиться пятикратного (в 5 раз) улучшения производительности. Это оценочное *ускорение*, и вычисления также помогли количественно оценить масштаб проблемы производительности: чтение с диска в 5 раз замедляет обработку запросов.

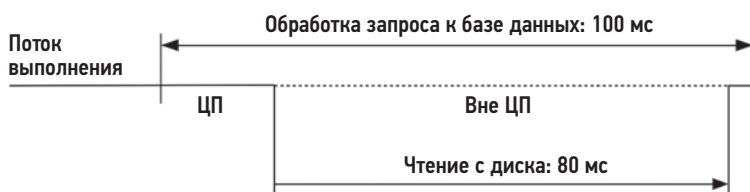


Рис. 1.3. Пример задержки, вызванной дисковым вводом/выводом

Подобные вычисления невозможны при использовании других метрик. Например, количество операций ввода/вывода в секунду (IOPS) зависит от типа ввода/вывода и часто напрямую несопоставимо для разных ситуаций. Если какое-то изменение приведет к уменьшению IOPS на 80 %, то трудно предсказать, как это отразится на производительности. Операций в секунду может быть в 5 раз меньше, но что, если каждая из этих операций обрабатывает в 10 раз больше данных?

Значение задержки также может быть неоднозначным без уточняющих терминов. Например, задержка в сети может означать время, необходимое для установления

соединения, но не время передачи данных; или общую продолжительность соединения, включая передачу данных (например, так обычно измеряется задержка DNS). По мере возможности в этой книге я буду использовать уточняющие термины: эти примеры лучше описать как задержку *соединения* и задержку *обработки запроса*. Терминология, связанная с задержкой, также приводится в начале каждой главы.

Задержка — полезная метрика, но она не всегда доступна. Некоторые системные области позволяют измерить только среднюю задержку; некоторые вообще не дают возможности измерения. С появлением новых инструментов наблюдения на основе BPF¹ задержку теперь можно измерять практически в любых точках и получить данные, описывающие полное распределение задержки.

1.7. НАБЛЮДАЕМОСТЬ

Под наблюдаемостью подразумевается исследование системы через наблюдение и инструменты, предназначенные для этого. Сюда входят инструменты, использующие счетчики, профилирование и трассировку, но не входят инструменты тестирования производительности, которые изменяют состояние системы, выполняя *эксперименты* с рабочей нагрузкой. В промышленных средах желательно сначала попробовать применить инструменты наблюдения, где это возможно, потому что инструменты для экспериментов могут препятствовать обработке промышленных рабочих нагрузок из-за конкуренции за ресурсы. В тестовых средах, которые большую часть времени простаивают, можно сразу начать с инструментов тестирования производительности для определения быстродействия оборудования.

В этом разделе я расскажу о счетчиках, метриках, профилировании и трассировке. Более подробно о наблюдаемости речь пойдет в главе 4, где рассмотрены общесистемная и индивидуальная наблюдаемость, инструменты наблюдения Linux и их внутреннее устройство. Кроме того, в главах 5–11 есть разделы, посвященные наблюдаемости, например, раздел 6.6 описывает инструменты наблюдения за процессором.

1.7.1. Счетчики, статистики и метрики

Приложения и ядро обычно предоставляют данные с информацией о своем состоянии и активности: счетчики операций, счетчики байтов, измеренные задержки, использованный объем ресурсов и частоту ошибок. Обычно эти данные доступны в виде целочисленных переменных, называемых *счетчиками*; они жестко «вшиты» в программное обеспечение, часть из них являются кумулятивными и постоянно увеличиваются. Эти кумулятивные счетчики можно читать в разное время инструментами оценки производительности для вычисления таких статистик, как скорость изменения во времени, среднее значение, процентиля и т. д.

¹ В настоящее время BPF является названием, а не аббревиатурой от Berkeley Packet Filter, как было первоначально.

Например, утилита `vmstat(8)` выводит общесистемную статистику по виртуальной памяти и другие данные на основе счетчиков ядра, доступных в файловой системе `/proc`. Вот пример вывода `vmstat(8)` на производственном сервере с 48 процессорами:

```
$ vmstat 1 5
procs -----memory----- --swap-- ----io---- -system-- -----cpu-----
 r  b swpd  free  buff  cache  si  so  bi  bo  in  cs  us  sy  id  wa  st
19  0    0 6531592 42656 1672040  0  0   0   0   7  21  33  51  4  46  0  0
26  0    0 6533412 42656 1672064  0  0   0   0 81262 188942 54  4  43  0  0
62  0    0 6533856 42656 1672088  0  0   0   0 80865 180514 53  4  43  0  0
34  0    0 6532972 42656 1672088  0  0   0   0 81250 180651 53  4  43  0  0
31  0    0 6534876 42656 1672088  0  0   0   0 74389 168210 46  3  51  0  0
```

Судя по этому примеру, доля занятости процессора в системе составляет около 57 % (столбцы `sru us + sy`). Более подробно значение столбцов объясняется в главах 6 и 7.

Метрика — это статистика, выбранная для оценки или мониторинга цели. Большинство компаний используют агенты мониторинга для записи выбранных статистик (метрик) через регулярные промежутки времени и построения графиков их изменения в графическом интерфейсе, чтобы видеть, как они меняются с течением времени. Программное обеспечение для мониторинга также может поддерживать создание специальных *предупреждений* на основе этих метрик, например отправку электронных писем для уведомления персонала об обнаружении проблем.

Эта иерархия от счетчиков до предупреждений изображена на рис. 1.4, который поможет вам понять эти термины, но их использование в отрасли не является жестко фиксированным. Термины *счетчики*, *статистики* и *метрики* часто используются как синонимы. Кроме того, оповещения могут генерироваться на любом уровне, а не только специальной системой оповещения.

В качестве примера графического представления метрик на рис. 1.5 показан скриншот инструмента на основе Grafana, который наблюдает за тем же сервером, на котором был получен предыдущий вывод `vmstat(8)`.

Эти линейные графики удобно использовать для планирования мощности — они помогают предсказать, когда наступит момент исчерпания ресурсов.

Ваша интерпретация статистик производительности улучшится, если вы поймете, как они вычисляются. Статистики, включая средние значения, распределения, моды и выбросы, более подробно описываются в главе 2 «Методологии», в разделе 2.8 «Статистики».

Иногда для решения проблемы с производительностью достаточно получить временные ряды с метриками. Время, когда проявилась проблема, может коррелировать с известным изменением в ПО или конфигурации, которое можно отменить. Иногда метрики лишь подсказывают направление, сообщая о проблеме с процессором или диском, но без объяснения причин. В таких случаях, чтобы копнуть глубже и отыскать причину, необходимо использовать инструменты профилирования.

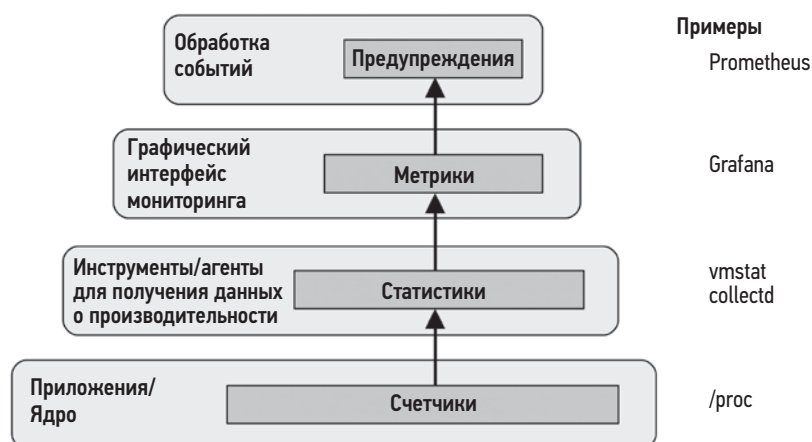


Рис. 1.4. Терминология, связанная с оценкой производительности

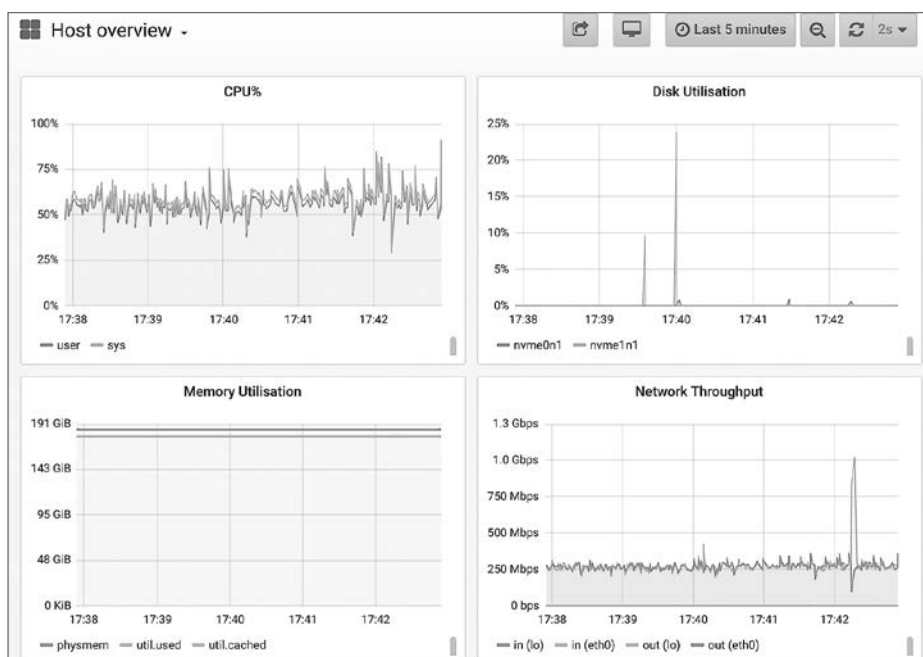


Рис. 1.5. Графический интерфейс с метриками (Grafana)

1.7.2. Профилирование

В контексте производительности систем термин *профилирование* обычно означает использование инструментов для выборки подмножества измерений и создания на их основе приблизительной картины, характеризующей цель. Часто целью профилирования является доля занятости процессора. Обычно для этого используется

метод профилирования, основанный на выборке путей в коде, выполняющемся на процессоре, с заданным интервалом времени.

Один из эффективных способов визуализации результатов профилирования процессора — *флейм-графики* (или графики пламени — flame graphs). Флейм-графики помогают добиться большего выигрыша в производительности, чем любой другой инструмент, после метрик. Они позволяют выявить не только проблемы с занятостью процессора, но и другие типы проблем, обнаруживаемые по характерным особенностям использования процессора. Проблемы, связанные с конфликтом блокировок, например, можно обнаружить по большим затратам процессорного времени в циклах ожидания блокировок; проблемы с памятью легко обнаруживаются по чрезмерным затратам процессорного времени в функциях распределения памяти (`malloc()`) и в коде, вызывающем эти функции; проблемы, связанные с ошибками в настройках сети, можно обнаружить по затратам процессорного времени в медленных или устаревших путях в коде; и т. д.

На рис. 1.6 приведен пример флейм-графика, показывающего процессорное время, потраченное инструментом тестирования сети `iperf(1)`.

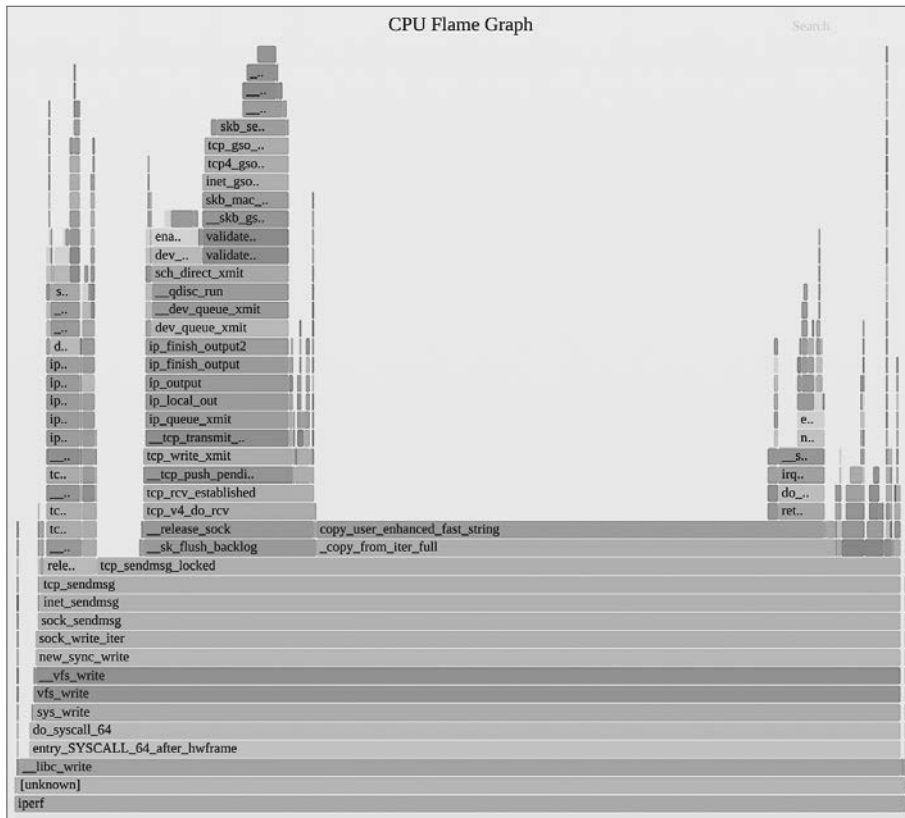


Рис. 1.6. Флейм-график расходования процессорного времени

На этом флейм-графике видно, насколько больше процессорного времени тратится на копирование байтов (путь в коде, заканчивающийся функцией `copy_user_enhanced_fast_string()`) по сравнению с передачей TCP-пакетов (второй большой пик слева, включающий функцию `tcp_write_xmit()`). Ширина пиков (или пирамид, как их еще называют) пропорциональна потраченному процессорному времени, а вдоль вертикальной оси откладывается путь в коде.

Подробнее приемы профилирования обсуждаются в главах 4, 5 и 6, а об использовании флейм-графиков рассказывается в главе 6 «Процессор», в разделе 6.7.3 «Флейм-графики».

1.7.3. Трассировка

Трассировка — это запись событий, когда данные о событиях фиксируются и сохраняются для последующего анализа или используются на лету для обобщения и выполнения других действий. Есть специальные инструменты трассировки для системных вызовов (например, `strace(1)` в Linux) и сетевых пакетов (например, `tcpdump(8)` в Linux), а также инструменты трассировки общего назначения, способные анализировать все программные и аппаратные события (например, `Ftrace`, `BCC` и `bpftrace` в Linux). Эти всевидящие трассировщики используют различные источники событий, в частности точки статической и динамической инструментации, а также механизм BPF, поддерживающий возможность программного управления.

Точки статической инструментации

Под точками статической инструментации подразумеваются жестко закодированные точки в ПО. В ядре Linux сотни таких точек, позволяющих выполнять трассировку дискового ввода/вывода, событий планировщика, системных вызовов и многого другого. Технология статической инструментации ядра Linux называется *tracepoints* (точки трассировки). Есть технология статической инструментации ПО в пространстве пользователя, которая называется *статически определяемой трассировкой на уровне пользователя* (User-level Statically Defined Tracing, USDT). Точки USDT используются во многих библиотеках (например, `libc`) для инструментации библиотечных вызовов и в приложениях для инструментации функций обработки запросов.

Примером утилиты, использующей статическую инструментацию, может служить `hexesnoop(8)`, которая выводит информацию о процессах, запущенных во время трассировки, получаемую путем инструментации точки трассировки в системном вызове `execve(2)`. Ниже показано, как `hexesnoop(8)` трассирует вход по SSH:

```
# hexesnoop
PCOMM      PID    PPID  RET  ARGS
ssh         30656  20063  0    /usr/bin/ssh 0
sshd        30657  1401  0    /usr/sbin/sshd -D -R
sh          30660  30657  0
env         30661  30660  0    /usr/bin/env -i PATH=/usr/local/sbin:/usr/local...
```

```

run-parts      30661 30660 0 /bin/run-parts --lsbsysinit /etc/update-motd.d
00-header     30662 30661 0 /etc/update-motd.d/00-header
uname         30663 30662 0 /bin/uname -o
uname         30664 30662 0 /bin/uname -r
uname         30665 30662 0 /bin/uname -m
10-help-text  30666 30661 0 /etc/update-motd.d/10-help-text
50-motd-news  30667 30661 0 /etc/update-motd.d/50-motd-news
cat           30668 30667 0 /bin/cat /var/cache/motd-news
cut           30671 30667 0 /usr/bin/cut -c -80
tr            30670 30667 0 /usr/bin/tr -d \000-\011\013\014\016-\037
head          30669 30667 0 /usr/bin/head -n 10
80-esm        30672 30661 0 /etc/update-motd.d/80-esm
lsb_release   30673 30672 0 /usr/bin/lsb_release -cs
[...]
```

Этот прием особенно удобен для трассировки короткоживущих процессов, которые могут остаться незамеченными другими инструментами наблюдения, такими как `top(1)`. Эти короткоживущие процессы могут быть источником проблем с производительностью.

Дополнительную информацию о точках трассировки и зондах USDT вы найдете в главе 4.

Динамическая инструментация

Технология динамической инструментации создает точки трассировки уже после запуска ПО путем подмены инструкций в памяти процесса вызовами процедур трассировки. Примерно так отладчики вставляют точки останова в любые функции в запущенном ПО. Разница лишь в том, что когда при отладке поток выполнения достигает точки останова, управление передается интерактивному отладчику, а при динамической инструментации вызывается процедура трассировки, по окончании которой целевое ПО продолжает работу как ни в чем не бывало. Динамическая инструментация позволяет собирать необходимую статистику производительности из любого выполняющегося ПО. Проблемы, которые раньше было невозможно или очень сложно решить из-за недостаточной наблюдаемости, теперь можно исправить.

Динамическая инструментация настолько отличается от традиционного наблюдения и мониторинга, что поначалу трудно понять ее роль. Возьмем для примера ядро операционной системы: анализ внутреннего устройства ядра часто похож на блуждание в темной комнате со свечами (системными счетчиками), установленными там, где разработчики ядра посчитали необходимым. Динамическую инструментацию я бы сравнил с фонариком, луч которого можно направить куда угодно.

Первые методы динамической инструментации были разработаны в 1990-х годах [Hollingsworth 94] вместе с инструментами, которые их используют. Эти инструменты называют *динамическими трассировщиками* (например, `kerninst` [Tamches 99]). Поддержка динамической инструментации для ядра Linux была разработана в 2000 году [Kleen 08] и начала внедряться в 2004 году (`kprobes`). Но эти технологии были малоизвестны и сложны в использовании. Ситуация изменилась, когда в 2005 году Sun Microsystems выпустила свою версию DTrace — простую

в использовании и безопасную для применения в производственной среде. Я разработал множество инструментов на основе DTrace, которые доказали свою важность для оценки производительности системы, получили широкое распространение и принесли широкую известность DTrace и динамической инструментации.

BPF

Механизм BPF, название которого первоначально произошло от Berkeley Packet Filter, поддерживает новейшие инструменты динамической трассировки для Linux. BPF создавался как миниатюрная виртуальная машина в ядре для ускорения выполнения выражений `tcpdump(8)`. Но в 2013 году был расширен (поэтому иногда его называют `eBPF`¹) и превратился в универсальную среду выполнения в ядре, обеспечивающую безопасный и быстрый доступ к ресурсам. Среди многочисленных новых применений обновленного механизма BPF — инструменты трассировки, для которых он обеспечивает поддержку программирования операций с применением коллекции компиляторов BPF (BPF Compiler Collection, BCC), и внешний интерфейс `bpftool`. Например, инструмент `hexdump(8)`, показанный выше, является инструментом BCC².

Подробнее о BPF рассказывается в главе 3, а глава 15 знакомит с интерфейсами трассировки BPF: BCC и `bpftool`. В других главах, в разделах о наблюдаемости, будут представлены многие инструменты трассировки на основе BPF; например, инструменты трассировки процессора описываются в главе 6 «Процессор» в разделе 6.6 «Инструменты наблюдения». Также ранее я опубликовал книги по инструментам трассировки (для DTrace [Gregg 11a] и BPF [Gregg 19]).

Оба инструмента, `perf(1)` и `Ftrace`, тоже являются трассировщиками и обладают возможностями, аналогичными интерфейсам BPF. Подробнее о `perf(1)` и `Ftrace` рассказывается в главах 13 и 14.

1.8. ЭКСПЕРИМЕНТЫ

Помимо инструментов наблюдения, также есть инструменты для *экспериментов*, большинство из которых применяется для бенчмаркинга. Они позволяют ставить эксперименты, применяя искусственную рабочую нагрузку к системе и измеряя ее производительность. Такие эксперименты следует проводить с осторожностью, потому что они могут ухудшать производительность тестируемых систем.

Есть инструменты *макробенчмаркинга*, которые имитируют реальную рабочую нагрузку, например действия клиентов, посылающих запросы приложениям, а есть

¹ Первое время для описания расширенного BPF (extended BPF) использовалось название `eBPF`; однако в настоящее время эта технология называется просто BPF.

² Сначала я разработал версию для DTrace, а потом и для других трассировщиков, включая BCC и `bpftool`.

инструменты *микробенчмаркинга*, которые тестируют конкретный компонент системы, например процессоры, диски или сети. В качестве аналогии: определение времени, необходимого для преодоления круга на трассе Laguna Seca Raceway, можно считать макробенчмаркингом, а определение времени разгона от 0 до 100 км/ч можно считать микробенчмаркингом. Оба типа тестов важны, но микробенчмарки обычно проще в разработке, отладке, использовании и понимании, и они более стабильны.

Следующий пример показывает использование `iperf(1)` на простаивающем сервере для микробенчмаркинга пропускной способности канала TCP с удаленным неактивным сервером. Этот бенчмарк выполнялся в течение 10 с (`-t 10`) и выводит средние значения в секунду (`-i 1`):

```
# iperf -c 100.65.33.90 -i 1 -t 10
```

```
-----
Client connecting to 100.65.33.90, TCP port 5001
TCP window size: 12.0 MByte (default)
-----
```

```
[ 3] local 100.65.170.28 port 39570 connected with 100.65.33.90 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 3]  0.0- 1.0 sec   582 MBytes  4.88 Gbits/sec
[ 3]  1.0- 2.0 sec   568 MBytes  4.77 Gbits/sec
[ 3]  2.0- 3.0 sec   574 MBytes  4.82 Gbits/sec
[ 3]  3.0- 4.0 sec   571 MBytes  4.79 Gbits/sec
[ 3]  4.0- 5.0 sec   571 MBytes  4.79 Gbits/sec
[ 3]  5.0- 6.0 sec   432 MBytes  3.63 Gbits/sec
[ 3]  6.0- 7.0 sec   383 MBytes  3.21 Gbits/sec
[ 3]  7.0- 8.0 sec   388 MBytes  3.26 Gbits/sec
[ 3]  8.0- 9.0 sec   390 MBytes  3.28 Gbits/sec
[ 3]  9.0-10.0 sec   383 MBytes  3.22 Gbits/sec
[ 3]  0.0-10.0 sec   4.73 GBytes  4.06 Gbits/sec
```

Как показывают результаты эксперимента, пропускная способность¹ в первые 5 с находилась на уровне около 4,8 Гбит/с, а затем упала до примерно 3,2 Гбит/с. Этот интересный результат демонстрирует бимодальную пропускную способность. Чтобы увеличить производительность, можно сосредоточиться на моде 3,2 Гбит/с и поискать другие метрики, объясняющие ее.

Рассмотрим недостатки отладки подобной проблемы производительности на рабочем сервере с использованием только инструментов наблюдения. Пропускная способность сети может меняться с течением времени из-за естественной разницы в рабочей нагрузке клиента, и подобное бимодальное поведение сети может оставаться незамеченным. Используя инструмент `iperf(1)`, генерирующий фиксированную рабочую нагрузку, можно избавиться от влияния изменчивости поведения клиентов и обнаружить отклонения, вызванные другими факторами (например, наличием ограничений во внешней сети, использованием буферов и т. д.).

¹ В этом выводе можно видеть термин «Bandwidth» (полоса пропускания), который часто используется не по назначению. Под полосой пропускания подразумевается максимальная возможная пропускная способность, которую `iperf(1)` не измеряет. `iperf(1)` измеряет текущую скорость передачи данных по сети: ее *пропускную способность*.

Как я рекомендовал выше, в производственных системах следует сначала попробовать применить инструменты наблюдения. Но их так много, что с ними можно работать часами, тогда как инструмент для экспериментов позволит быстрее прийти к результатам. Много лет назад старший перформанс-инженер Рох Бурбоннис (Roch Bourbonnais) рассказал мне о такой аналогии: у вас есть две руки, наблюдение и эксперименты. Использование инструментов только одного типа похоже на попытку решить проблему одной рукой.

Главы 6–10 включают разделы об инструментах для экспериментирования. Например, инструменты для экспериментирования с процессором описаны в главе 6 «Процессор» в разделе 6.8 «Эксперименты».

1.9. ОБЛАЧНЫЕ ВЫЧИСЛЕНИЯ

Облачные вычисления как способ развертывания вычислительных ресурсов по запросу позволяют быстро масштабировать приложения, развертывая их во все большем количестве небольших виртуальных систем, называемых *экземплярами* (*instances*). Этот подход избавляет от необходимости тщательно планировать емкость, потому что в кратчайшие сроки можно добавить дополнительные ресурсы из облака. В некоторых случаях это также увеличило потребность в анализе производительности, потому что использование меньшего количества ресурсов может означать меньшее количество систем. Поскольку за использование облачных ресурсов обычно взимается поминутная или часовая оплата, выигрыш в производительности, способствующий использованию меньшего количества систем, дает прямую экономию средств. Сравните этот сценарий с услугами корпоративного вычислительного центра, с которым вы можете быть связаны фиксированным договором на поддержку в течение многих лет без возможности экономии, пока срок действия договора не истечет.

Облачные вычисления и виртуализация принесли новые трудности, включая управление влиянием других подписчиков на производительность (иногда это называют *изоляцией производительности*) и возможность мониторинга физической системы со стороны каждого подписчика. Например, в отсутствие должного управления системой производительность дискового ввода/вывода может упасть из-за конфликта с соседом. В некоторых средах информация о фактической нагрузке на физические диски может быть недоступна подписчикам, что затрудняет выявление таких проблем.

Более подробно эти вопросы рассматриваются в главе 11 «Облачные вычисления».

1.10. МЕТОДОЛОГИИ

Методология — это способ задокументировать рекомендуемые шаги для решения различных задач по оценке производительности системы. Без методологии

исследование производительности может превратиться в рыбалку, когда рыбак пробует разные наживки в надежде на удачу. Это неэффективный и потенциально долгий путь с риском упустить из виду важные области. В главе 2 «Методологии» я привожу перечень методологий для оценки производительности систем. В следующем подразделе я покажу первое, что использую для решения любой проблемы с производительностью: чек-лист инструментов.

1.10.1. Анализ производительности Linux за 60 секунд

Это чек-лист инструментов анализа производительности для Linux, который можно выполнить за 60 секунд в самом начале исследования проблемы производительности. Здесь перечислены традиционные инструменты, доступные в большинстве дистрибутивов Linux [Gregg 15a]. В табл. 1.1 показаны конкретные команды, которые следует выполнить, а также разделы в этой книге, где соответствующие команды рассматриваются более подробно.

Таблица 1.1. Чек-лист для анализа производительности Linux за 60 секунд

№	Инструмент	Проверяет	Раздел
1	uptime	Средние значения нагрузки, чтобы определить, увеличивается или уменьшается нагрузка (можно сравнить средние значения за 1, 5 и 15 минут)	6.6.1
2	dmesg -T tail	Ошибки в ядре, включая события OOM (нехватки памяти)	7.5.11
3	vmstat -SM 1	Общесистемные статистики: длина очереди на выполнение, подкачка (swapping), общая доля занятости процессора	7.5.1
4	mpstat -P ALL 1	Баланс нагрузки по процессорам: увеличенная нагрузка на один из процессоров может указывать на плохое масштабирование потоков выполнения	6.6.3
5	pidstat 1	Потребление процессора каждым процессом: позволяет выявить непредвиденных потребителей процессорного времени и определить, сколько процессорного времени потрачено каждым процессом в пространстве ядра и в пространстве пользователя	6.6.7
6	iostat -sxz 1	Статистики дискового ввода/вывода: количество операций ввода/вывода в секунду (IOPS) и пропускная способность, среднее время ожидания, процент занятости	9.6.1
7	free -m	Потребление памяти, включая кэши файловой системы	8.6.2
8	sar -n DEV 1	Статистики сетевого устройства ввода/вывода: количество пакетов и пропускная способность	10.6.6
9	sar -n TCP,ETCP	Статистики TCP: частота приема соединений, частота повторных передач	10.6.6
10	top	Общий обзор	6.6.6

Этот чек-лист также можно использовать в графическом интерфейсе мониторинга, если в нем доступны те же метрики¹.

Глава 2 «Методологии» и следующие за ней содержат описание множества методологий анализа производительности, включая метод USE, определение характеристик рабочей нагрузки, анализ задержки и многие другие.

1.11. ПРАКТИЧЕСКИЕ ПРИМЕРЫ

Если вы новичок в вопросах оценки производительности систем, то практические примеры, показывающие, когда и почему выполняются различные действия, помогут вам провести аналогию с вашим текущим окружением. В этом разделе представлены два гипотетических примера. Первый показывает проблему производительности, связанную с дисковым вводом/выводом, а второй — тестирование производительности после изменения ПО.

Примеры описывают действия, о которых подробно рассказывается в других главах этой книги. Основная цель этих примеров — показать не правильный или единственный способ, а скорее *один из* способов, которым можно выполнить эти действия.

1.11.1. Медленные диски

Сумит — системный администратор в компании среднего размера. Группа обслуживания базы данных добавила тикет с жалобой на «медленные диски» на одном из серверов баз данных.

Первая задача Сумита — узнать как можно больше о проблеме, собрать необходимую информацию и сформулировать проблему. В тикете утверждается, что диски работают медленно, но не объясняется, действительно ли это является причиной проблемы в базе данных. В ответ Сумит задает следующие вопросы:

- Наблюдаются ли проблемы с производительностью базы данных сейчас? Как она измеряется?
- Как давно существует эта проблема?
- Изменилось ли что-нибудь в базе данных за последнее время?
- Почему подозрение пало на диски?

В ответ команда базы данных пишет: «В нашем отделе ведется журнал, в котором фиксируются запросы, продолжительность обработки которых превышает 1000 мс. Обычно такие запросы встречаются редко, но за последнюю неделю их число

¹ Можно даже создать свой дашборд для такого чек-листа, но имейте в виду, что этот чек-лист составлен с целью максимально использовать доступные инструменты командной строки, тогда как средства мониторинга могут иметь множество других (и более информативных) метрик. Я более склонен создавать дашборды для методологии USE и др.

выросло до нескольких десятков в час. Анализ с применением АсмеМон показал большую загруженность дисков».

Этот ответ подтверждает наличие проблемы с базой данных, но также показывает, что гипотеза о том, что причиной является низкая производительность диска, скорее всего, является предположением. Сумит решает проверить диски, а также другие ресурсы на тот случай, если гипотеза окажется неверной.

АсмеМон — это базовая система мониторинга серверов компании, предоставляющая исторические графики изменения стандартных метрик операционной системы, которые можно получить с помощью `mpstat(1)`, `iostat(1)` и других системных утилит. Сумит входит в АсмеМон, чтобы выполнить задуманные проверки.

На первом шаге Сумит применяет методологию USE (описывается в главе 2 «Методологии» в разделе 2.5.9), чтобы быстро проверить наличие узких мест в ресурсах. Как сообщила группа обслуживания базы данных, доля загруженности дисков достигла высокого уровня, около 80 %, тогда как потребление других ресурсов (процессор, сеть) намного ниже. По историческим данным выяснилось, что загруженность дисков неуклонно росла в течение последней недели, в то время как потребление процессора оставалось постоянным. Система АсмеМон не предоставляет статистики, характеризующие насыщенность или частоту ошибок для дисков, поэтому для применения методологии USE Сумит должен выполнить некоторые команды на самом сервере.

Он проверил счетчики ошибок дискового ввода/вывода в `/sys` — они оказались равны нулю. Запустил `iostat(1)`, задав интервал равным одной секунде, и понаблюдал за изменением метрик потребления и насыщенности с течением времени. Система АсмеМон сообщала об уровне загруженности 80 %, но проводила измерения с интервалом в одну минуту. Используя интервал измерений в одну секунду, Сумит увидел, что загруженность диска колеблется, часто достигая 100 % и вызывая увеличение уровня насыщенности и задержки дискового ввода/вывода.

Чтобы еще раз убедиться, что загруженность диска является причиной блокировки базы данных и возрастает синхронно с запросами к ней, он решил использовать инструмент трассировки BCC/BPF под названием `offcputime(8)` и с его помощью захватывать трассировки стека всякий раз, когда база приостанавливается ядром, а также определять продолжительность приостановки. Трассировки стека показали, что база данных часто блокируется на время чтения файловой системы в процессе обработки запроса. Для Сумита этого было достаточно.

Следующий вопрос — почему. Статистики производительности диска соответствуют высокой нагрузке. Сумит решил выяснить характеристики рабочей нагрузки, измерив с помощью `iostat(1)` частоту операций ввода/вывода, пропускную способность, среднюю задержку дискового ввода/вывода и соотношение операций чтения/записи. Для получения дополнительной информации Сумит мог бы использовать трассировку дискового ввода/вывода, однако ему достаточно информации, указывающей, что имеет место высокая нагрузка на диск, а не проблема с производительностью дисков.

Сумит добавляет дополнительные детали в тикет, указав, что было проверено, и включив скриншоты команд, использовавшихся для анализа работы дисков. На данный момент он пришел к выводу, что диски работают под высокой нагрузкой, из-за чего увеличивается задержка ввода/вывода и медленно обрабатываются запросы. Но судя по имеющимся данным, диски вполне справляются с нагрузкой, и он задает вопрос: есть ли простое объяснение случившемуся; увеличилась ли нагрузка на базу данных?

Команда обслуживания БД ответила, что простого объяснения нет и количество запросов (о которых не сообщает АсmeMon) остается постоянным. Похоже, это согласуется с более ранним выводом о неизменности нагрузки на процессор.

Сумит размышляет о том, какие еще причины могут вызвать увеличение нагрузки на дисковый ввод/вывод без заметного увеличения потребления процессора, и консультируется со своими коллегами. Один из них выдвигает предположение о сильной фрагментированности файловой системы, что вполне ожидаемо, когда заполненность файловой системы приближается к 100 %. Но, как выяснил Сумит, файловая система заполнена всего на 30 %.

Сумит знает, как провести более детальный анализ¹, чтобы выяснить точные причины, но на это требуется много времени. Основываясь на своем знании стека ввода/вывода в ядре, он пытается придумать другие простые объяснения, которые можно быстро проверить. Он помнит, что часто дисковые операции обусловлены промахами кэша файловой системы (кэша страниц).

Сумит проверяет коэффициент попаданий в кэш файловой системы с помощью `cachestat(8)`² и обнаруживает, что в данный момент он составляет 91 %. Выглядит неплохо (и даже очень хорошо), но у него нет исторических данных для сравнения. Тогда он заходит на другие серверы баз данных, обслуживающие аналогичные рабочие нагрузки, и обнаруживает, что в них коэффициент попадания в кэш превышает 98 %. Он также обнаруживает, что размер кэша файловой системы на других серверах намного больше.

Обратив внимание на размер кэша файловой системы и потребление памяти сервера, он замечает, что было упущено из виду: в разрабатываемом проекте имеется прототип приложения, потребляющий все больше памяти, хотя пока работает вхолостую. В результате для кэша файловой системы остается меньше свободной памяти, из-за чего снижается частота попаданий в кэш и увеличивается количество дисковых операций чтения с диска.

Сумит связался с командой разработчиков приложения и попросил их остановить приложение и переместить его на другой сервер, сославшись на проблему с базой данных. После того как они выполнили его просьбу, Сумит увидел в АсmeMon, как

¹ Об этом рассказывается в главе 2 «Методологии» в разделе 2.5.12 «Анализ с увеличением детализации».

² Инструмент трассировки ВСС, описывается в главе 8 «Файловые системы» в разделе 8.6.12 «`cachestat`».

по мере восстановления кэша файловой системы до исходного объема нагрузка на диск стала постепенно снижаться. Количество медленно обрабатываемых запросов упало до нуля, и он закрыл тикет как разрешенный.

1.11.2. Изменение в программном обеспечении

Памела — перформанс-инженер в небольшой компании, она занимается всем, что так или иначе связано с производительностью. Разработчики приложений реализовали новую возможность, но не уверены, что ее внедрение не ухудшит производительность. Памела решает провести регрессионное тестирование новой версии приложения перед внедрением в продакшен.

Для тестирования Памела выбирает простаивающий сервер, и теперь ей нужен имитатор рабочей нагрузки клиента. В недавнем прошлом группа разработчиков приложений написала такой имитатор, но у него есть различные ограничения и известные ошибки. Памела решает попробовать его, но прежде хочет убедиться, что он способен создавать рабочую нагрузку, адекватную текущей.

Она настраивает сервер в соответствии с текущей конфигурацией развертывания и запускает имитатор в другой системе для создания рабочей нагрузки на сервер. Нагрузку, имитирующую действия клиентов, можно оценить, изучив журнал доступа, и в компании уже есть подходящий для этого инструмент. Она запускает этот инструмент, передает ему журнал с рабочего сервера с данными за разные периоды в течение суток и сравнивает рабочие нагрузки. Как оказалось, имитатор создает среднюю рабочую нагрузку без всяких отклонений. Она отмечает это и продолжает анализ.

Памела знает несколько подходов, которые можно применить на этом этапе. Она выбирает самый простой: увеличивать нагрузку до достижения предела (иногда этот подход называют *стресс-тестированием*). Имитатор клиента можно настроить на отправку определенного количества клиентских запросов в секунду со значением по умолчанию 1000. Она решает начать со 100 запросов в секунду и постепенно увеличивать нагрузку с шагом 100, пока не будет достигнут предел, при этом на каждом уровне нагрузки тестирование продолжается в течение одной минуты. Она пишет сценарий на языке командной оболочки, который собирает результаты в файл для анализа другими инструментами.

При действующей нагрузке она проводит активный анализ производительности, чтобы выявить ограничивающие факторы. Ресурсы сервера кажутся свободными, а потоки выполнения по большей части бездействующими. Как показал имитатор, пропускная способность составила примерно 700 запросов в секунду.

После этого она запускает новую версию приложения и повторяет тесты. Достигается тот же уровень 700 запросов в секунду, и никаких ограничивающих факторов не выявляется.

Она наносит результаты на графики, отражающие зависимость количества обработанных запросов в секунду от нагрузки, чтобы визуально определить профиль

масштабируемости, и отмечает присутствие на обоих графиках резкой верхней границы.

По всей видимости, обе версии ПО имеют схожие характеристики производительности, но Памела разочарована, потому что не смогла выявить ограничивающий фактор, определяющий потолок масштабируемости. Она знает, что проверяла только ресурсы сервера, а ограничение может скрываться в логике приложения, в работе сети, в имитаторе клиента или где-то еще.

Памела решает использовать другой подход, например, запустить имитацию клиента с фиксированной частотой следования запросов и оценить потребление ресурсов (процессора, дискового ввода/вывода, сетевого ввода/вывода), чтобы потом выразить затраты ресурсов на обработку одного клиентского запроса. Она запускает имитатор с частотой 700 запросов в секунду и измеряет потребление ресурсов при использовании текущего и нового программного обеспечения. С текущим ПО в системе с 32 процессорами загрузка процессоров составила 20 %, а с новым программным обеспечением при той же нагрузке она увеличилась до 30 %. Похоже, что в новом ПО действительно имеется регресс производительности из-за увеличенного потребления вычислительных ресурсов.

Но было бы интересно понять причину предела в 700 запросов в секунду. Памела настраивает имитатор на более высокую нагрузку и исследует все компоненты на пути к данным, включая сеть, клиентскую систему и генератор клиентской рабочей нагрузки. Она также выполняет детальный анализ серверного и клиентского ПО, документируя проверенные метрики и скриншоты для справки.

Для исследования клиентского ПО она проанализировала состояния потоков выполнения и обнаружила, что оно однопоточное! Этот единственный поток потребляет все 100 % процессорного времени, что, по всей видимости, и является причиной столь резкой границы в 700 запросов в секунду, обнаруженной во время тестирования.

Чтобы подтвердить догадку, она запускает имитатор сразу на нескольких клиентских системах и доводит потребление процессора на сервере до 100 % как с текущим, так и с новым программным обеспечением. По результатам этого тестирования производительность текущей версии достигла 3500 запросов в секунду, а новой версии — 2300 запросов в секунду, что согласуется с более ранними данными о потреблении ресурсов.

Памела сообщает разработчикам, что в новой версии ПО наблюдается регресс, и приступает к профилированию потребления процессора с помощью флейм-графика, чтобы понять, какие пути в коде вносят наибольший вклад. Она отмечает, что тестирование выполнялось с нагрузкой, сопоставимой со средней на производственном сервере, и другие нагрузки не проверялись. Она также сообщает об ошибке в имитаторе клиентской рабочей нагрузки — что он однопоточный и может стать узким местом.

1.11.3. Дополнительное чтение

Подробный практический пример представлен в главе 16 «Пример из практики», где описывается решение конкретной проблемы с производительностью в облачной среде. В следующей главе будут представлены методологии, используемые для анализа производительности, а в последующих главах дается вся необходимая информация.

1.12. ССЫЛКИ

- [Hollingsworth 94] Hollingsworth, J., Miller, B., and Cargille, J., «Dynamic Program Instrumentation for Scalable Performance Tools», Scalable High-Performance Computing Conference (SHPCC), May 1994.
- [Tamches 99] Tamches, A., and Miller, B., «Fine-Grained Dynamic Instrumentation of Commodity Operating System Kernels», Proceedings of the 3rd Symposium on Operating Systems Design and Implementation, February 1999.
- [Kleen 08] Kleen, A., «On Submitting Kernel Patches», Intel Open Source Technology Center, <http://halobates.de/on-submitting-patches.pdf>, 2008.
- [Gregg 11a] Gregg, B., and Mauro, J., «DTrace: Dynamic Tracing in Oracle Solaris, Mac OS X and FreeBSD», Prentice Hall, 2011.
- [Gregg 15a] Gregg, B., «Linux Performance Analysis in 60,000 Milliseconds», Netflix Technology Blog, <http://techblog.netflix.com/2015/11/linux-performance-analysis-in-60s.html>, 2015.
- [Dekker 18] Dekker, S., «Drift into Failure: From Hunting Broken Components to Understanding Complex Systems», CRC Press, 2018.
- [Gregg 19] Gregg, B., «BPF Performance Tools: Linux System and Application Observability»¹, Addison-Wesley, 2019.
- [Corry 20] Corry, A., «Retrospectives Antipatterns», Addison-Wesley, 2020.

¹ Грегг Б. «BPF: Профессиональная оценка производительности». Выходит в издательстве «Питер» в 2023 году.

Глава 2

МЕТОДОЛОГИИ

Дай человеку рыбу, и он будет сыт один день.
Научи его ловить рыбу, и он будет сыт до конца жизни.

Китайская пословица

Свою карьеру я начинал младшим системным администратором и думал тогда, что смогу освоить оценку производительности, изучая только метрики и инструменты командной строки. Как же я ошибался! Я прочитал страницы справочного руководства от корки до корки и изучил, как определять сбои страниц (page faults), переключения контекста и множество других системных показателей, но я не знал, что с ними делать: как перейти от сигналов к решениям.

Я заметил, что всякий раз, когда возникала проблема с производительностью, старшие системные администраторы применяли свои методики использования инструментов и метрик, чтобы как можно быстрее добраться до первопричины. Они понимали, какие метрики наиболее информативны, когда они указывают на проблему, и как их использовать для сужения исследуемой области. Именно этого *ноу-хау* не хватало на страницах справочного руководства, и мы, молодежь, перенимали этот опыт, заглядывая через плечо старшего администратора или инженера.

С тех пор я собрал, разработал, задокументировал и опубликовал собственные *методологии* оценки эффективности и включил их в эту главу вместе с другой важной информацией о производительности системы: понятиями, терминологией, статистиками и приемами визуализации. Здесь та теория, которая пригодится в последующих главах, где мы перейдем к практическому применению этих методологий.

Цели главы:

- Познакомить с ключевыми метриками производительности: задержкой, коэффициентом использования (потреблением) и насыщенностью.
- Развить интуицию в выборе масштаба измеряемого времени вплоть до наносекунд.
- Научить правильно выстраивать компромиссы, определять цели и момент прекращения анализа.
- Показать, как выявлять проблемы рабочей нагрузки в зависимости от архитектуры.

- Научить анализировать ресурсы и рабочие нагрузки.
- Представить различные методологии оценки производительности, в том числе метод USE, определение характеристик рабочей нагрузки, анализ задержки, статическую настройку производительности и мантры производительности.
- Познакомить с основами статистики и теории массового обслуживания.

Из всех глав моей книги эта меньше всего изменилась по сравнению с первым изданием. Программное и аппаратное обеспечение, инструменты оценки и настройки производительности — все это не раз менялось в течение моей карьеры, но теория и методология, о которых идет речь в этой главе, оставались неизменными.

Глава делится на три части:

- **Основы:** знакомит с терминологией, базовыми моделями, ключевыми понятиями и перспективами. Практически вся оставшаяся часть книги предполагает знание этих основ.
- **Методологии:** рассматривает методологии анализа производительности, основанные как на наблюдениях, так и на экспериментах, моделирование, а также планирование мощности.
- **Метрики:** представляет статистики производительности, приемы мониторинга и визуализации.

Многие из представленных здесь методологий мы рассмотрим подробнее в последующих главах, в том числе в разделах, посвященных методологиям, в главах 5–10.

2.1. ТЕРМИНОЛОГИЯ

Ниже перечислены ключевые термины, используемые в сфере анализа производительности систем. В последующих главах будут представлены дополнительные термины с описаниями в различных контекстах.

- **IOPS** (input/output operations per second): количество операций ввода/вывода в секунду; является мерой частоты следования операций передачи данных. Для дискового ввода/вывода под количеством операций ввода/вывода (IOPS) подразумевается сумма операций чтения и записи в секунду.
- **Пропускная способность** (throughput): скорость выполненной работы. В частности, в области связи этот термин используется для обозначения *скорости передачи данных* (байтов или битов в секунду). В некоторых контекстах (например, в базах данных) под пропускной способностью может подразумеваться *скорость работы* (количество операций или транзакций в секунду).
- **Время отклика** (response time): время от начала до завершения операции. Включает в себя время на ожидание, на обслуживание (*время обслуживания* — service time) и на передачу результата.

- **Задержка (latency)**: время, в течение которого операция ожидает обслуживания. В некоторых контекстах под задержкой подразумевается все время выполнения операции, то есть время отклика. См. примеры в разделе 2.3 «Основные понятия».
- **Потребление (utilization)**: для ресурсов, необходимых для обслуживания, потребление определяет меру занятости ресурса, исходя из продолжительности в данном интервале времени, когда этот ресурс активно выполнял работу. Для ресурсов, которые обеспечивают хранение, под коэффициентом использования (потреблением) может подразумеваться потребленный объем (например, потребленный объем памяти).
- **Насыщенность (saturation)**: степень заполнения очереди заданий, которые ресурс не может обслужить немедленно.
- **Узкое место (bottleneck)**. В сфере анализа производительности систем под узким местом подразумевается ресурс, ограничивающий производительность системы. Основная работа по повышению производительности системы как раз заключается в выявлении и устранении узких мест.
- **Рабочая нагрузка (workload)**: рабочей нагрузкой называются входные данные для системы или приложенная к ней нагрузка. Для базы данных рабочей нагрузкой является поток запросов и команд, отправляемых клиентами.
- **Кэш (cache)**: область быстродействующего хранилища, в которой могут храниться копии или буферы ограниченного объема данных. Кэш используется, чтобы предотвратить операции с более медленными хранилищами и тем самым повысить производительность. По экономическим причинам кэш часто меньше основного и более медленного хранилища.

В глоссарии можно найти описания дополнительных терминов.

2.2. МОДЕЛИ

Следующие простые модели показывают некоторые основные принципы оценки производительности системы.

2.2.1. Тестируемая система

На рис. 2.1 показаны характеристики тестируемой системы (System Under Test, SUT).

Важно помнить, что на результат могут влиять посторонние возмущения (помехи), в том числе вызванные запланированной активностью системы, пользователями системы и другими рабочими нагрузками. Происхождение возмущений не всегда очевидно, и для их определения может потребоваться тщательное изучение характеристик системы. Это особенно сложно в некоторых облачных средах, где другие действия (со стороны подписчиков) в физической системе хоста могут быть недоступны для наблюдения из гостевой SUT.



Рис. 2.1. Схема тестируемой системы

Другая сложность современных окружений заключается в том, что они могут включать несколько сетевых компонентов, обслуживающих входную рабочую нагрузку, в том числе балансировщики нагрузки, прокси-серверы, веб-серверы, серверы кэширования, серверы приложений, серверы баз данных и системы хранения. Простое картографирование окружения может помочь выявить ранее упущенные из виду источники возмущений. Для аналитических исследований окружение также можно представить как сеть систем массового обслуживания (queueing systems).

2.2.2. Система массового обслуживания

Некоторые компоненты и ресурсы можно смоделировать как систему массового обслуживания и получить возможность предсказывать их производительность в различных ситуациях. Диски, например, часто моделируют как систему массового обслуживания, которая может предсказать, насколько ухудшится время отклика под нагрузкой. На рис. 2.2 показана простая система массового обслуживания.

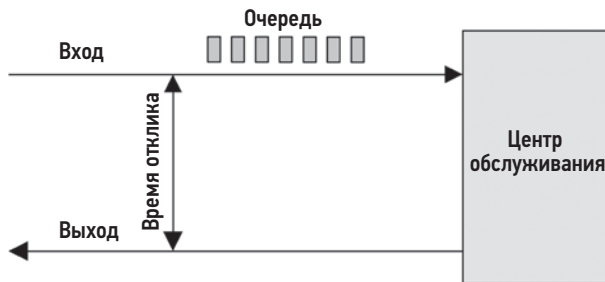


Рис. 2.2. Простая модель системы массового обслуживания

Теория массового обслуживания, представленная в разделе 2.6 «Моделирование», изучает системы массового обслуживания и сети систем массового обслуживания.

2.3. ОСНОВНЫЕ ПОНЯТИЯ

Далее приводится описание важных концепций в сфере оценки производительности систем, понимание которых понадобится в оставшейся части этой главы

и всей книги. Здесь приводится лишь общее описание понятий, а более подробные разъяснения, связанные с реализацией, будут даны в последующих главах — в разделах, посвященных архитектуре.

2.3.1. Задержка

В некоторых средах задержка — единственный фактор, позволяющий хоть как-то оценить производительность. В других средах это одна из двух основных метрик наряду с пропускной способностью.

В качестве примера на рис. 2.3 показана передача данных по сети, в данном случае запрос HTTP GET, с разделением времени на задержку и передачу данных.

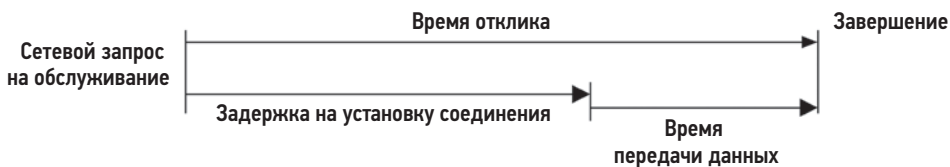


Рис. 2.3. Задержка при передаче данных по сети

Задержка — это время ожидания до завершения операции. В этом примере операцией является запрос к сетевой службе на передачу данных. Прежде чем эта операция завершится, система должна дожидаться, пока будет установлено сетевое соединение, что является задержкой для этой операции. *Время отклика* включает в себя эту задержку и время выполнения самой операции.

Поскольку задержку можно измерить в разных местах, ее часто выражают с использованием цели измерений. Например, время загрузки страницы веб-сайта может включать в себя три разных значения времени, измеренных в разных местах: *задержка DNS*, *задержка соединения TCP* и *время передачи данных TCP*. Задержка DNS охватывает всю операцию разрешения имен в DNS. Задержка соединения TCP охватывает только инициализацию соединения («рукопожатие» TCP).

На более высоком уровне все эти времена, включая время передачи данных TCP, можно рассматривать как задержку чего-то еще. Например, время с момента щелчка на ссылке до полной загрузки веб-страницы можно назвать *задержкой*, включающей время получения браузером страницы из сети и ее отображение на экране. Поскольку само слово «задержка» может быть неоднозначным, его лучше сопровождать уточняющими терминами, объясняющими, что именно измеряет эта задержка: задержка запроса, задержка соединения TCP и т. д.

Поскольку задержка является метрикой времени, ее можно использовать в различных вычислениях. Проблемы производительности можно количественно оценить с помощью задержки и затем ранжировать, потому что они выражаются в одних и тех же единицах (времени). Также, опираясь на задержки, можно рассчитать прогнозируемое ускорение, определив степень уменьшения задержки. Подобные

вычисления и прогнозы невозможны с использованием, например, только метрики IOPS.

В табл. 2.1 приводятся краткие обозначения единиц измерения времени и их масштаб.

Таблица 2.1. Единицы измерения времени

Единица	Краткое обозначение	Величина в секундах
Минута	мин	60
Секунда	с	1
Миллисекунда	мс	0,001, или 1/1000, или 1×10^{-3}
Микросекунда	мкс	0,000001, или 1/1000000, или 1×10^{-6}
Наносекунда	нс	0,000000001, или 1/1000000000, или 1×10^{-9}
Пикосекунда	пс	0,000000000001, или 1/1000000000000, или 1×10^{-12}

Преобразование других типов метрик в задержку или время, если это возможно, позволяет сравнивать их. Если вам нужно было бы выбирать между 100 сетевыми и 50 дисковыми операциями ввода/вывода, как бы вы узнали, что лучше? Это сложный вопрос, требующий учета множества факторов: количества сетевых переходов, частоты сброса сетевых пакетов и повторных передач, объема ввода/вывода, характера ввода/вывода — произвольный или последовательный, типов дисков и т. д. Но если сравнивать 100 мс сетевого ввода/вывода и 50 мс дискового ввода/вывода, разница очевидна.

2.3.2. Шкалы времени

Время можно сравнивать численно, но у нас также есть интуитивное понимание времени, и мы можем выдвигать разумные предположения относительно задержек из разных источников. Компоненты системы работают в совершенно разных временных масштабах (отличающихся порой на порядки), разных до такой степени, что порой трудно понять, насколько велики эти различия. В табл. 2.2 представлены примеры задержек, начиная с доступа к регистру процессора с тактовой частотой 3,5 ГГц. Чтобы наглядно показать разницу во временных масштабах, с которыми приходится работать, в таблице показано также среднее время выполнения каждой операции в масштабе воображаемой системы, в которой один такт процессора длительностью в 0,3 нс (примерно одна треть от одной миллиардной доли секунды) занимает целую секунду.

Как видите, один такт процессора длится очень короткое время. Чтобы преодолеть расстояние 0,5 м — примерно столько же, сколько от ваших глаз до этой страницы, — свету требуется около 1,7 нс. За это время современный процессор может выполнить пять тактов и обработать несколько инструкций.

Таблица 2.2. Масштабы задержек в системе

Событие	Задержка	В масштабе
1 такт процессора	0,3 нс	1 с
Доступ к кэшу 1-го уровня	0,9 нс	3 с
Доступ к кэшу 2-го уровня	3 нс	10 с
Доступ к кэшу 3-го уровня	10 нс	33 с
Доступ к оперативной памяти (DRAM из процессора)	100 нс	6 мин
Продолжительность ввода/вывода для твердотельного накопителя (флеш-память)	10–100 мкс	9–90 часов
Продолжительность ввода/вывода для вращающегося жесткого диска	1–10 мс	1–12 месяцев
Интернет: время передачи сигнала от Сан-Франциско до Нью-Йорка	40 мс	4 года
Интернет: время передачи сигнала от Сан-Франциско до Великобритании	81 мс	8 лет
Облегченная загрузка с аппаратной виртуализацией	100 мс	11 лет
Интернет: время передачи сигнала от Сан-Франциско до Австралии	183 мс	19 лет
Загрузка системы с виртуализацией на уровне ОС	< 1 с	105 лет
Задержка перед повторной передачей TCP	1–3 с	105–317 лет
Тайм-аут команды SCSI	30 с	3 тысячелетия
Загрузка системы с аппаратной виртуализацией	40 с	4 тысячелетия
Перезагрузка физической системы	5 мин	32 тысячелетия

Дополнительные сведения о тактах процессора и задержках вы найдете в главе 6 «Процессоры», а о задержках дискового ввода/вывода — в главе 9 «Диски». Задержки в интернете взяты из главы 10 «Сеть», где вы найдете еще множество примеров.

2.3.3. Компромиссы

Вы должны знать о некоторых типичных компромиссах, связанных с производительностью. Компромисс «выбора двух из трех» — качественно/быстро/дешево — показан на рис. 2.4 вместе с терминологией, скорректированной для ИТ-проектов.

Многие ИТ-проекты выбирают своевременность и экономичность, оставляя высокую эффективность на потом. Этот выбор может превратиться в проблему, когда принятые решения впоследствии будут препятствовать повышению производительности, как, например, выбор и реализация неоптимальной архитектуры хранилища,

использование неэффективного языка программирования или операционной системы или выбор компонента, в котором нет инструментов комплексного анализа производительности.



Рис. 2.4. Компромиссы: выбор двух из трех

Типичный компромисс при настройке производительности — это компромисс между процессором и памятью, потому что память можно использовать для кэширования результатов и тем самым снизить нагрузку на процессор. В современных системах с большим количеством процессоров выбор может измениться, например, в сторону использования процессора для сжатия данных, чтобы уменьшить потребление памяти.

Настраиваемые параметры часто требуют компромиссов. Вот пара примеров:

- **Размер записи в файловой системе** (или размер блока): записи небольшого размера, близкие к размеру блоков ввода/вывода в приложении, лучше подходят для рабочих нагрузок с произвольным вводом/выводом и позволяют эффективнее использовать кэш файловой системы во время работы приложения. Записи большого размера лучше подходят для рабочих нагрузок с потоковой передачей данных, включая резервное копирование файловой системы.
- **Размер сетевого буфера**: буферы небольшого размера уменьшают оверхед памяти на одно соединение, облегчая масштабирование системы. Буферы большого размера улучшают пропускную способность сети.

Учитывайте подобные компромиссы при внесении изменений в систему.

2.3.4. Настройка производительности

Настройка производительности наиболее эффективна, когда выполняется в непосредственной близости от места выполнения работы. Для рабочих нагрузок, управляемых приложениями, это означает настройку в самом приложении. В табл. 2.3 приведен пример программного стека с возможностями настройки.

Выполняя настройки на уровне приложения, можно исключить или уменьшить количество запросов к базе данных и значительно повысить производительность (например, в 20 раз). Спустившись на уровень устройства хранения, можно улучшить производительность операций ввода/вывода с хранилищем, но в этом случае оверхед на выполнение кода стека ОС никуда не исчезает, поэтому такая настройка может улучшить итоговую производительность приложения только на проценты (например, на 20 %).

Таблица 2.3. Примеры целей для настройки

Уровень	Примеры цели
Приложение	Логика приложения, размеры очередей запросов, выполнение запросов к базе данных
База данных	Структура таблиц, индексы, буферизация
Системные вызовы	Отображение в память или чтение/запись, флаги синхронного или асинхронного ввода/вывода
Файловая система	Размер записи, размер кэша, параметры файловой системы, журналирование
Устройства хранения	Уровень RAID, количество и типы дисков, параметры устройств хранения

Есть еще одна причина для поиска выигрышей в производительности на уровне приложений. Многие современные среды нацелены на быстрое развертывание, что способствует внедрению изменений в ПО еженедельно или даже ежедневно¹. Поэтому процессы разработки и тестирования приложений, как правило, сосредоточены на корректности, и в них почти не остается времени для оценки производительности или оптимизации перед развертыванием в производственной среде. Оценка и оптимизация делаются позже, когда производительность становится проблемой.

Уровень приложения может быть самым эффективным для настройки производительности, но это не обязательно самый эффективный уровень для наблюдения за производительностью. Неэффективность обработки запросов легче выявить по времени выполнения на процессоре или по операциям ввода/вывода с файловой системой и дисками, то есть по информации, которую легко получить с помощью инструментов ОС.

Во многих средах (особенно в облачных) уровень приложений постоянно развивается, и изменения внедряются в продакшен еженедельно и ежедневно. Кроме того, самый значительный выигрыш в производительности, включая исправление регрессий, достигается при изменении кода приложения. В этих средах легко упустить из виду настройки на уровне операционной системы и возможность наблюдения со стороны операционной системы. Помните, что анализ производительности ОС может выявить проблемы не только на уровне ОС, но и на уровне приложений, и в некоторых случаях такой анализ выполнить легче, чем анализ самого приложения.

2.3.5. Целесообразность

В разных организациях и средах предъявляются разные требования к производительности. Может так сложиться, что в компании, куда вы устроитесь, принято проводить гораздо более глубокий анализ, чем вы привыкли делать, и вы даже не

¹ Примером быстро меняющихся сред могут служить облака Netflix и Shopify, в которых изменения вносятся по несколько раз в день.

думали, что это возможно. Или, наоборот, на новом месте работы анализ, который вы считали базовым, может считаться углубленным и никогда раньше не проводился (легкая работа!).

Это не обязательно означает, что одни компании поступают правильно, а другие — нет. Многое зависит от окупаемости затрат и наличия опыта. Организации с большими вычислительными центрами или большими облачными средами могут позволить себе нанять команду перформанс-инженеров, которые анализируют все, включая внутренние механизмы ядра и счетчики производительности процессора, и часто используют различные инструменты трассировки. Они также могут формально моделировать производительность и разрабатывать точные прогнозы будущего роста. Для сред, тратящих миллионы в год на вычисления, легко оправдать затраты на такие группы специалистов, потому что выигрыши, которые они получают, — это рентабельность инвестиций. Небольшие стартапы со скромными затратами на вычисления могут выполнять только поверхностные проверки, доверяя мониторинг производительности сторонним решениям.

Но как было сказано в главе 1, производительность системы — это не только стоимость, но и удобство конечного пользователя. Стартап может посчитать необходимым инвестировать в оптимизацию производительности, чтобы уменьшить время отклика сайта или приложения. Рентабельность инвестиций в данном случае не обязательно означает снижение затрат, но уж точно будет означать наличие довольных, а не потерянных клиентов.

К наиболее экстремальным средам с точки зрения производительности относятся фондовые биржи и торговые площадки, где производительность и задержка имеют решающее значение, и увеличение эффективности может оправдать значительные усилия и затраты. В качестве примера можно привести прокладку трансатлантического кабеля, соединяющего биржи Нью-Йорка и Лондона, стоимостью 300 миллионов долларов, который позволил уменьшить задержку передачи на 6 мс [Williams 11].

Целесообразность также играет важную роль при принятии решения о том, когда прекратить анализ производительности.

2.3.6. Когда лучше остановить анализ

При анализе производительности всегда сложно правильно оценить, когда нужно остановиться, ведь инструментов и факторов, которые нужно исследовать, так много.

Когда я провожу уроки по анализу производительности (в последнее время я снова начал это делать), то часто даю студентам проблему для исследования, имеющую три причины. Выясняется, что некоторые останавливаются, найдя одну причину, другие — найдя две, третьи — все три, а некоторые продолжают исследования, пытаясь найти еще больше причин. Кто из них поступает правильно? Может показаться, что правильнее остановиться после того, как будут найдены все три причины, но в реальной жизни количество причин неизвестно.

Вот три сценария, когда анализ можно остановить, с некоторыми личными примерами:

- **Когда вы объяснили основной вклад в проблему производительности.** Приложение на Java начало потреблять в 3 раза больше процессорного времени, чем раньше. Первой я обнаружил проблему со стеком исключений, потребляющих процессорное время. Затем я подсчитал затраты времени в этих стеках и обнаружил, что на них приходится только 12 % от общего объема процессорного времени. Если бы эта цифра была близка к 66 %, я бы прекратил анализ, потому что это объясняло бы трехкратное замедление. Но получив цифру в 12 %, я решил продолжить поиски.
- **Когда потенциальная выгода меньше стоимости анализа.** Некоторые проблемы с производительностью, над которыми я работаю, могут приносить выигрыш, измеряемый десятками миллионов долларов в год. Такой выигрыш позволяет оправдать месяцы моего времени (инженерные затраты), потраченные на анализ. Другие выигрыши, например, за счет увеличения производительности крошечных микросервисов, могут измеряться сотнями долларов. Для их анализа может не хватить часа инженерного времени, разве что мне будет нечем заняться (чего в реальности никогда не бывает) или если я подозреваю, что эта проблема может быть предвестницей более серьезной проблемы и поэтому ее стоит решить до того, как она разрастется.
- **Когда окупаемость затрат на решение других проблем выше.** Если текущая ситуация не подпадает под предыдущие два сценария, исследование других проблем может принести больший выигрыш, и тогда имеет смысл перенести свое внимание на них.

Если вы работаете перформанс-инженером на полную ставку, определение приоритетности различных проблем на основе их потенциальной окупаемости, вероятно, станет для вас повседневной задачей.

2.3.7. Рекомендации действительны на данный момент времени

Характеристики производительности окружения со временем меняются из-за расширения круга пользователей, обновления оборудования и ПО или прошивки. Например, после увеличения пропускной способности сети с 10 до 100 Гбит/с диски или процессор могут превратиться в узкое место.

Рекомендации по производительности, в частности значения настраиваемых параметров, являются действительными только в определенный *момент времени*. Хороший на данный момент совет специалиста по производительности может стать недействительным после обновления программного или аппаратного обеспечения либо после появления новых пользователей.

Значения настраиваемых параметров, найденные в интернете, иногда могут дать быстрый выигрыш. Но могут также снизить производительность, если не подходят

для вашей системы или рабочей нагрузки. Возможно, когда-то они были подходящими, но не сейчас, или подходят только как временное решение для обхода ошибки в ПО, которая будет в одном из ближайших его релизов. Это как если бы вы начали искать в чужой аптечке лекарство, но нашли бы там только просроченное или то, что вам не годится.

Искать такие рекомендации полезно, чтобы узнать, какие настраиваемые параметры существуют и как они настраивались в прошлом. Следующая задача после знакомства с параметрами — определить, нужно ли их настраивать для вашей системы и рабочей нагрузки и как. При этом все равно есть риск пропустить важный параметр, если другим не приходилось его настраивать или они настраивали его, но не поделились своим опытом.

При изменении настраиваемых параметров полезно сохранить их в системе контроля версий с подробной историей. (Возможно, вы уже так и поступаете, используя инструменты управления конфигурациями — Puppet, Salt, Chef и т. д.) Благодаря этому можно изучать изменения настраиваемых параметров и причины позже.

2.3.8. Нагрузка и архитектура

Приложение может работать неэффективно из-за проблем с конфигурацией ПО и оборудования, на котором оно выполняется, то есть из-за проблем с архитектурой и реализацией среды. Но приложение также может работать неэффективно просто из-за слишком большой нагрузки, вызывающей появление больших очередей и длительных задержек. Нагрузка и архитектура показаны на рис. 2.5.

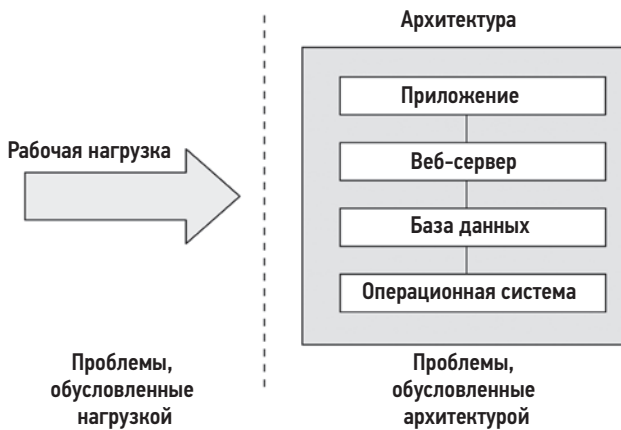


Рис. 2.5. Нагрузка и архитектура

Если анализ архитектуры показывает рост очередей с заданиями, но обрабатываются они достаточно эффективно, то проблема может быть в слишком большой

приложенной нагрузке. В облачных средах в такие моменты можно организовать увеличение количества экземпляров для обслуживания дополнительной нагрузки.

Проблема с архитектурой может заключаться в однопоточном характере приложения, которое полностью нагружает один процессор, тогда как другие процессоры простаивают. В этом случае производительность ограничена однопоточной архитектурой приложения. Еще одна проблема с архитектурой — конкуренция за единственную блокировку в многопоточной программе, из-за чего только один поток может двигаться вперед, а остальные вынуждены ждать.

Многопоточные приложения тоже могут страдать от проблемы слишком высокой нагрузки, когда все доступные процессоры уже задействованы, а запросы продолжают прибывать в очереди. В этом случае производительность ограничена вычислительной мощностью процессора или, иначе говоря, система испытывает нагрузку большую, чем могут обрабатывать ее процессоры.

2.3.9. Масштабируемость

Изменение производительности системы при возрастающей нагрузке определяется ее *масштабируемостью*. На рис. 2.6 показан типичный график изменения пропускной способности с увеличением нагрузки на систему.



Рис. 2.6. Изменение пропускной способности с увеличением нагрузки

Какое-то время наблюдается линейный рост пропускной способности. Затем достигается уровень нагрузки, отмеченный вертикальной пунктирной линией, на котором конкуренция за ресурсы начинает снижать пропускную способность. Эту точку можно описать как *точку перегиба*, которая является границей между двумя функциями. Правее этой точки график отклоняется от линейного из-за возрастания конкуренции за ресурсы. В итоге оверхеда на конкуренцию и когерентность приводят к уменьшению объема выполняемой работы и снижению пропускной способности.

Эта точка может возникнуть, когда доля использования (потребление) компонента достигает 100 %: *точки насыщения*. Также это может произойти, когда доля использования компонента приближается к 100 % и операции с очередями выполняются все чаще и отнимают все больше времени.

Примером системы с подобным профилем может служить приложение, выполняющее тяжелые вычисления и обслуживающее дополнительную нагрузку, запуская дополнительные потоки выполнения. По мере того как нагрузка на процессор приближается к 100 %, время отклика начинает деградировать с увеличением задержки планировщика. После достижения пиковой производительности при 100 %-ном использовании процессоров пропускная способность начинает снижаться с добавлением новых потоков выполнения, потому что увеличивается количество переключений контекста, которые потребляют ресурсы процессора, и для выполнения фактической работы остается меньше процессорного времени.

Эту же кривую можно получить, если заменить «нагрузку» на оси X таким ресурсом, как ядра процессора. Дополнительную информацию по этой теме можно найти в разделе 2.6 «Моделирование».

На рис. 2.7 показано, как деградирует производительность в случае нелинейной масштабируемости с точки зрения среднего времени отклика или задержки [Cockcroft 95].



Рис. 2.7. Деградация производительности

Большое время отклика — это, конечно, плохо. Профиль «быстрой» деградации особенно характерен для случаев исчерпания оперативной памяти, когда система начинает перемещать страницы памяти на диск, чтобы освободить оперативную память. Профиль «медленной» деградации более характерен для случаев высокой нагрузки на процессор.

Еще одним примером ситуации с профилем «быстрой» деградации производительности может служить увеличение операций дискового ввода/вывода. По мере увеличения нагрузки (и, соответственно, операций с диском) операции ввода/вывода с большей вероятностью будут оказываться в очереди за другими операциями ввода/вывода. Неактивный вращающийся (не твердотельный) диск может обслуживать ввод/вывод с временем отклика около 1 мс, но при увеличении нагрузки это время может увеличиться до 10 мс. Эта ситуация смоделирована в разделе 2.6.5 «Теория массового обслуживания» в виде системы M/D/1 с 60 % нагрузкой, а производительность дисков рассматривается в главе 9 «Диски».

Линейный профиль времени отклика может возникнуть, если при недоступности ресурсов приложение начнет возвращать ошибки вместо добавления заданий в очередь. Например, веб-сервер может возвращать ошибку 503 «Служба недоступна» («service unavailable») вместо добавления запросов в очередь, чтобы запросы, которые приняты к обслуживанию, были обработаны за установленное время.

2.3.10. Метрики

Метрики производительности — это статистические данные, генерируемые системой, приложениями или дополнительными инструментами, которые измеряют интересующую активность. Метрики изучаются при анализе и мониторинге производительности либо в числовом виде в командной строке, либо графически с использованием средств визуализации.

К общим метрикам производительности системы можно отнести:

- **пропускную способность:** количество выполняемых операций или объем обрабатываемых данных в секунду;
- **IOPS:** количество операций ввода/вывода в секунду;
- **доля использования (потребление):** уровень потребления ресурса в процентах;
- **задержка:** время выполнения операции в виде среднего значения или процентилей.

Значение пропускной способности зависит от контекста. Под пропускной способностью базы данных обычно понимается количество запросов (операций) в секунду. Пропускная способность сети — это количество битов или байтов (объем), передаваемых в секунду.

IOPS — это мера пропускной способности для операций ввода/вывода (чтения и записи). И снова смысл этой метрики может меняться в зависимости от контекста.

Оверхед

Метрики производительности имеют определенную стоимость, потому что за их сбор и хранение приходится платить тактами процессорного времени. Такой оверхед может отрицательно влиять на производительность объекта измерения. Это называется *эффектом наблюдателя*. (Его часто путают с принципом неопределенности Гейзенберга, который описывает предел точности, с которой можно измерить пары физических свойств, например положение и импульс.)

Проблемы

Вы можете полагать, что поставщик ПО предоставил хорошо подобранные и безошибочные метрики, обеспечивающие полную наглядность. Но на самом деле метрики могут быть запутанными, сложными, ненадежными, неточными и совсем

неверными (из-за ошибок). Иногда метрика может быть точной и верной в одной версии ПО и оказаться ошибочной в другой версии, потому что не отражает вновь появившиеся пути в коде.

Дополнительную информацию о проблемах с метриками ищите в главе 4 «Инструменты наблюдения» в разделе 4.6 «Наблюдение за наблюдаемостью».

2.3.11. Потребление

Термин *потребление* (utilization) часто применяется для описания уровня использования устройств, например процессоров и дисковых устройств. Потребление может зависеть от времени или мощности.

По времени

Потребление по времени в теории массового обслуживания формально определяется как (например, [Gunther 97])

среднее количество времени, в течение которого сервер или ресурс были заняты

и задается соотношением

$$U = B/T,$$

где U = потребление (utilization), B = суммарное время, когда ресурс был занят (busy) в течение времени наблюдения T .

Влияние на потребление является также одним из наиболее доступных средств повышения производительности операционной системы. Инструмент мониторинга дисков `iostat(1)` называет эту метрику `%b` (от *percent busy* — процент занятости); это название точно соответствует базовой метрике B/T .

Метрика потребления показывает уровень использования компонента: когда компонент используется на все 100 %, производительность может серьезно ухудшиться из-за возникающей конкуренции за ресурс. Чтобы убедиться, что компонент стал узким местом в системе, необходимо проверить также другие метрики.

Некоторые компоненты могут выполнять сразу несколько операций, и их производительность может не сильно пострадать при уровне использования 100 %, потому что они способны выполнять больше работы. Для примера рассмотрим строительный лифт. Он может считаться используемым, когда перемещается между этажами, и неиспользуемым, когда простаивает. Но лифт может перевозить большее количество пассажиров и грузов, даже когда занят все 100 % времени.

Диск, который занят на 100 %, тоже может выполнять больше работы, например, за счет буферизации записываемых данных в кэш-памяти диска и записи их позже. Массивы хранения часто работают со 100 %-ной загрузкой, потому что *некоторые* диски заняты 100 % времени, но при этом в массиве много бездействующих дисков, и поэтому он может выполнять больший объем работы.

По мощности

Другое определение потребления используется в информатике в контексте планирования мощности [Wong 97]:

Система или компонент (например, дисковый накопитель) могут обеспечить определенную пропускную способность. На любом уровне производительности система или компонент работают с определенной долей своей мощности. Эта доля называется потреблением.

В этом случае потребление определяется с точки зрения мощности, а не времени. Это означает, что диск, используемый на 100 %, *не может* выполнять больше работы. По определению, основанному на времени, под 100 %-ным потреблением подразумевается всего лишь, что он занят 100 % времени.

100 %-ная занятость не означает потребление 100 % мощности.

В примере с лифтом потребление 100 % мощности может означать, что лифт работает с максимальной полезной нагрузкой и не может перевозить больше пассажиров.

В идеальном мире мы могли бы измерить обе метрики потребления и узнать, например, когда диск занят на 100 % и производительность снижается из-за конкуренции, а когда он используется на 100 % мощности и не способен выполнять больше работы. К сожалению, обычно это невозможно. В случае с диском для этого нужно знать, что делает встроенный контроллер диска, и прогнозировать его мощность. В настоящее время диски не предоставляют эту информацию.

В этой книге термин *потребление* (или *использование*) обычно относится к версии определения на основе времени, которую также можно назвать *временем работы без простоев*. Версия определения на основе мощности применяется для описания занятости ресурсов на основе объема, например памяти.

2.3.12. Насыщенность

Степень превышения потребности в ресурсе над его возможностями называется *насыщенностью*. Насыщение наступает при пересечении отметки, равной 100 % потребления (по мощности), когда дополнительные задания не могут быть обработаны немедленно и ставятся в очередь. Эта ситуация показана на рис. 2.8.

На рисунке видно, что с увеличением нагрузки насыщенность продолжает расти линейно и после пересечения отметки 100 % доли потребления по мощности. Любая степень насыщенности — это проблема производительности, потому что при насыщении часть времени тратится на ожидание (задержку). При использовании метрики занятости на основе времени (процента занятости) насыщение и, следовательно, создание очередей, может не наступать на отметке 100 % потребления в зависимости от способности ресурса обслуживать задания параллельно.

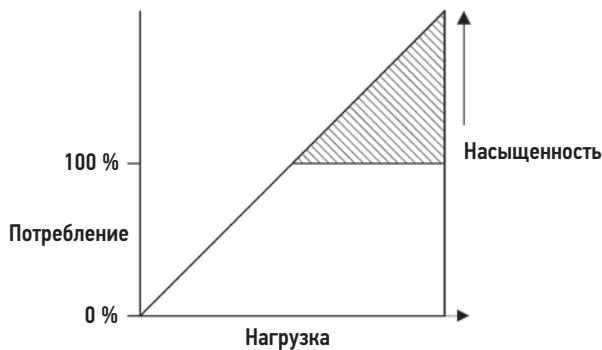


Рис. 2.8. Потребление и насыщенность

2.3.13. Профилирование

Профилирование создает картину цели, которую можно изучить. Профилирование с целью оценки производительности вычислений обычно выполняется путем *выборки*, или *семплинга* (sampling), состояния системы через определенные интервалы времени и последующего изучения набора выборок.

В отличие от рассмотренных ранее показателей, включая количество операций ввода/вывода в секунду и пропускную способность, выборки помогают получить *приблизительное* представление об активности цели. Степень приблизительности зависит от частоты дискретизации.

Например, с помощью профилирования путем частой выборки указателя инструкций процессора или трассировок стека можно достаточно полно понять, какие пути в коде потребляют больше всего процессорного времени. Более подробно об этом рассказывается в главе 6 «Процессоры».

2.3.14. Кэширование

Кэширование часто используется для повышения производительности. В кэше хранятся результаты, полученные из более медленного хранилища. Примером может служить кэширование дисковых блоков в оперативной памяти (ОЗУ).

Кэши могут быть многоуровневыми. Например, в процессорах используется несколько аппаратных кэш-уровней (1-го, 2-го и 3-го уровней). На уровне 1 находится самый быстрый и небольшой кэш, и с каждым следующим уровнем размер кэша и время доступа к нему увеличиваются. Это экономический компромисс между объемом и задержкой. Размеры для разных уровней выбраны с учетом компромисса между производительностью и доступным пространством на кристалле. Подробнее об этих кэшах рассказывается в главе 6 «Процессоры».

В системе есть множество других кэш-уровней, большинство из которых реализованы в ПО с использованием оперативной памяти. Список уровней кэширования вы найдете в главе 3 «Операционные системы» в разделе 3.2.11 «Кэширование».

Одной из метрик производительности кэша является *коэффициент попаданий в кэш* (hit ratio) — сколько раз необходимые данные были обнаружены в кэше (попадания) по отношению к общему числу обращений к кэшу (попадания + промахи):

$$\text{Коэффициент попаданий} = \text{попадания} / (\text{попадания} + \text{промахи}).$$

Чем больше значение этой метрики, тем лучше, ведь чем больше этот коэффициент, тем чаще данные получают из быстродействующего хранилища. На рис. 2.9 показано ожидаемое улучшение производительности с увеличением коэффициента попадания в кэш.

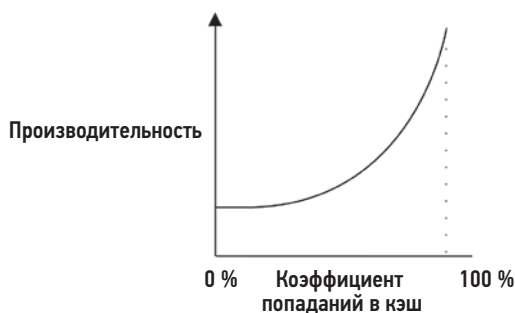


Рис. 2.9. Коэффициент попадания в кэш и производительность

Разница между производительностями, соответствующими коэффициентам попадания в кэш 98 % и 99 %, намного больше, чем между коэффициентами 10 % и 11 %. Эта нелинейность обусловлена разницей в скорости между попаданиями и промахами кэша — двумя уровнями хранения. Чем больше разница, тем круче кривая.

Еще одна метрика производительности кэша — *частота промахов в кэше* — выражает количество промахов в секунду. Она пропорциональна (линейно) потере производительности с каждым промахом, и ее проще интерпретировать.

Например, рабочие нагрузки А и В решают одну и ту же задачу с применением разных алгоритмов и используют кэш в оперативной памяти, чтобы избежать чтения с диска. Рабочая нагрузка А имеет коэффициент попадания в кэш 90 %, а рабочая нагрузка В имеет коэффициент попадания в кэш 80 %. Казалось бы, эта информация говорит о том, что рабочая нагрузка А действует эффективнее. Но что, если рабочая нагрузка А имеет частоту промахов 200 в секунду, а рабочая нагрузка В — 20 в секунду? Согласно этой информации, рабочая нагрузка В выполняет в 10 раз меньше операций чтения с диска, благодаря чему может справиться с задачей намного раньше, чем рабочая нагрузка А. Для большей уверенности можно рассчитать общее время выполнения каждой рабочей нагрузки как

$$\text{время выполнения} = (\text{коэффициент попаданий} \times \text{задержка попадания}) + (\text{частота промахов} \times \text{задержка промаха}).$$

В этой формуле используются средние значения задержек попаданий и промахов и предполагается, что все операции выполняются последовательно.

Алгоритмы

Алгоритмы и стратегии управления кэшем определяют, какие элементы будут продолжать храниться в ограниченном пространстве кэша.

Последний использовавшийся (most recently used, MRU) — это стратегия хранения в кэше, согласно которой там сохраняются объекты, которые использовались недавно. *Дольше всех не использовавшийся* (least recently used, LRU) — это похожая стратегия, но она описывает стратегию вытеснения из кэша: какие объекты удалять, когда требуется освободить место в кэше для новых объектов. Также есть стратегии *наиболее часто используемые* (most frequently used, MFU) и *наименее часто используемые* (least frequently used, LFU).

Иногда можно столкнуться со стратегией *нечасто используемые* (not frequently used, NFU), которая является менее дорогостоящей и менее полной версией LRU.

Горячие, холодные и теплые кэши

Эти характеристики широко используются для описания состояния кэша:

- **Холодный:** *холодный кэш* — это пустой кэш или заполненный посторонними данными. Коэффициент попаданий для холодного кэша равен нулю (и начинает увеличиваться по мере разогрева кэша).
- **Теплый:** *теплый кэш* — это кэш, наполненный полезными данными, но коэффициент попаданий в который пока недостаточно высок, чтобы можно было считать его горячим.
- **Горячий:** *горячий кэш* — это кэш, наполненный полезными данными и имеющий высокий коэффициент попаданий, например более 99 %.
- **Теплота:** *теплота кэша* описывает, насколько он горяч или холоден. Действие, увеличивающее «температуру» кэша, увеличивает коэффициент попаданий в него.

После инициализации кэш сначала холодный, а затем постепенно разогревается. Когда кэш очень большой или хранилище следующего уровня работает очень медленно (или и то и другое), на разогрев кэша может потребоваться много времени.

Например, мне довелось работать с устройством хранения, имевшим 128 Гбайт DRAM для кэширования файловой системы, 600 Гбайт флеш-памяти в качестве кэша второго уровня и вращающиеся диски для фактического хранения данных. При рабочей нагрузке с произвольным доступом диски производили около 2000 операций чтения в секунду. При размере блока ввода/вывода 8 Кбайт это означало, что кэш-память разогревалась со скоростью всего 16 Мбайт/с (2000×8 Кбайт). Сразу после включения, когда оба кэша еще холодные, требовалось более 2 часов для разогрева кэша в DRAM и более 10 часов для разогрева во флеш-памяти.

2.3.15. Известные неизвестные

Представленные в предисловии понятия *известные известные* (известные и изученные вещи), *известные неизвестные* (известные и неизученные вещи) и *неизвестные неизвестные* (неизвестные и неизученные вещи) очень важны в анализе производительности. Ниже для каждого из них приводятся примеры из анализа производительности систем:

- **Известные известные:** это то, что вы знаете. Вы знаете, какую метрику производительности следует проверить, и знаете ее текущее значение. Например, вы знаете, что должны проверить потребление процессора, и известно также, что эта метрика имеет среднее значение 10 %.
- **Известные неизвестные:** это то, о существовании чего вы знаете, но конкретные характеристики этого чего-то не знаете. Вы знаете, что можете проверить некоторую метрику или наличие подсистемы, но пока не сделали этого. Например, вы знаете, что можете выполнить профилирование, чтобы проверить, чем заняты процессоры, но еще не сделали этого.
- **Неизвестные неизвестные:** это то, о существовании чего вы и не подозреваете. Например, вы можете не подозревать о том, что прерывания от устройства могут оказывать существенную нагрузку на процессор, и, соответственно, не проверяете их.

Производительность — это область, в которой «чем больше знаешь, тем больше не знаешь». Чем больше вы узнаете о системах, тем больше неизвестных неизвестных выявляете; далее они становятся известными неизвестными и у вас появляется возможность проверить их.

2.4. ТОЧКИ ЗРЕНИЯ

Есть две общие точки зрения в анализе производительности со своими последовательностями, метриками и подходами: *анализ рабочей нагрузки* и *анализ ресурсов*. Их можно рассматривать как нисходящий и восходящий виды анализа программного стека операционной системы, как показано на рис. 2.10.

Конкретные стратегии для каждой из них будут представлены в разделе 2.5 «Методология», а здесь поговорим о самих точках зрения.

2.4.1. Анализ ресурсов

Анализ ресурсов начинается с анализа системных ресурсов: процессоров, памяти, дисков, сетевых интерфейсов, шин и соединений. Обычно это делают системные администраторы — те, кто отвечает за физические ресурсы. В число выполняемых мероприятий входят:

- **исследование проблем с производительностью:** чтобы определить меру ответственности конкретного ресурса за проблемы;

- **планирование мощности:** для получения информации, которая поможет определить размер и мощность новых систем и когда имеющиеся системные ресурсы будут исчерпаны.

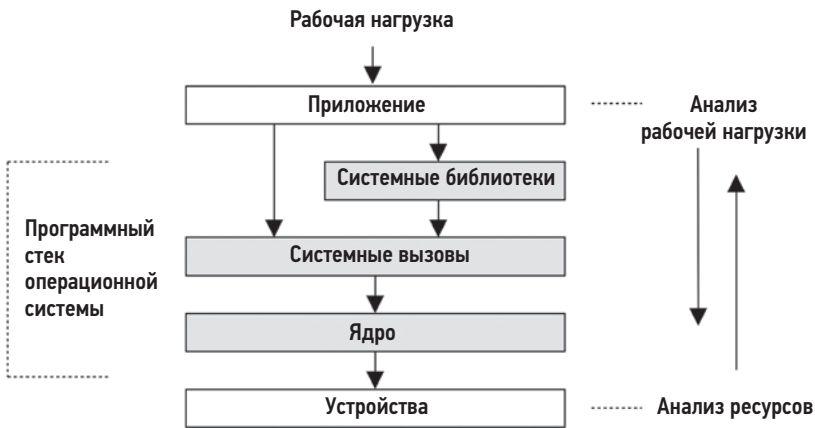


Рис. 2.10. Разные точки зрения на анализ

Эта точка зрения фокусируется на потреблении ресурсов, чтобы определить, когда произошло или произойдет исчерпание имеющихся ресурсов. Метрики потребления одних ресурсов, таких как процессоры, легкодоступны. Метрики потребления других ресурсов можно оценить на основе доступных метрик, например, оценить уровень потребления сетевого интерфейса, сравнив объем передаваемых и принимаемых данных в единицу времени (пропускной способности) с известной или ожидаемой максимальной пропускной способностью.

К числу метрик, которые лучше всего подходят для анализа ресурсов, относятся:

- IOPS;
- пропускная способность;
- потребление;
- насыщенность.

Они оценивают уровень потребления и насыщения ресурса для данной нагрузки. Другие метрики, такие как задержка, тоже могут пригодиться, помогая увидеть, насколько хорошо ресурс соответствует данной рабочей нагрузке.

Анализ ресурсов — это распространенный подход к анализу производительности, отчасти благодаря широкодоступной документации по этой теме. В этой документации основное внимание уделяется инструментам «stat» операционной системы: `vmstat(8)`, `iostat(1)`, `mpstat(1)`. При чтении такой документации важно понимать, что это одна из точек зрения, но не единственная.

2.4.2. Анализ рабочей нагрузки

Анализ рабочей нагрузки (рис. 2.11) исследует производительность приложений: прикладываемую рабочую нагрузку и реакцию приложения. Этот вид анализа чаще используют разработчики приложений и команда поддержки — те, кто отвечает за сопровождение ПО и настройку приложения.



Рис. 2.11. Анализ рабочей нагрузки

Цели анализа рабочей нагрузки:

- **вход:** прикладываемая рабочая нагрузка;
- **задержка:** время отклика приложения;
- **выход:** выявление ошибок.

Изучение прикладываемой рабочей нагрузки обычно включает проверку и обобщение атрибутов запросов: это процесс *определения характеристик рабочей нагрузки* (более подробно описывается в разделе 2.5 «Методология»). Для баз данных в число этих атрибутов могут входить клиентский хост, имя базы данных, используемые таблицы и строка запроса. Эти данные могут помочь определить ненужную или несбалансированную работу. Даже когда система хорошо справляется с текущей рабочей нагрузкой (показывает низкую задержку), изучение этих атрибутов поможет найти способы уменьшения объема выполняемой работы. Помните, что самый быстрый запрос — это запрос, который не требуется выполнять.

Задержка (время отклика) — самый важный показатель, выражающий производительность приложения. Для базы данных MySQL — это задержка обработки запроса, для веб-сервера Apache — это задержка обработки HTTP-запроса, и т. д. В этих случаях термин *задержка* используется для обозначения времени ответа (см. раздел 2.3.1 «Задержка», чтобы узнать больше о контексте).

Задачи анализа рабочей нагрузки включают выявление и подтверждение проблем, например, поиск задержек, превышающих допустимый порог, а затем поиск источника задержки и подтверждение, что задержка уменьшилась после внесения исправлений. Обратите внимание, что отправной точкой в этом анализе является приложение. Исследование задержки обычно включает более глубокое изучение приложения, библиотек и операционной системы (ядра).

При изучении особенностей обработки запроса могут быть выявлены системные проблемы, включая состояние ошибки. Обработка запроса может завершиться быстро, но с признаком ошибки, что вызовет повторную попытку отправки запроса, увеличивая задержку.

Для анализа рабочей нагрузки лучше всего подходят следующие метрики:

- пропускная способность (транзакций в секунду);
- задержка.

Они оценивают скорость обработки запросов и итоговую производительность.

2.5. МЕТОДОЛОГИЯ

Если вы столкнулись с неэффективным и сложным системным окружением, первая проблема, которую нужно решить, — определить, с чего начать анализ и как действовать дальше. Как отмечалось в главе 1, проблемы с производительностью могут возникать где угодно — в софте, в оборудовании и вообще в любом компоненте на пути к данным. Методологии могут помочь приступить к анализу таких сложных систем, показывая, с чего начать анализ, и предлагая эффективную последовательность шагов.

В этом разделе описывается множество методологий и процедур анализа и настройки производительности, часть из которых разработаны мной. Эти методики помогут новичкам приступить к работе и послужат напоминанием для экспертов. Также в этом разделе упоминаются некоторые *антиметодологии*.

Я разделил методологии на типы — анализ наблюдений и анализ результатов экспериментов (табл. 2.4).

Таблица 2.4. Методологии анализа производительности

Раздел	Методология	Тип
2.5.1	Антиметодология «Уличный фонарь»	Анализ наблюдений
2.5.2	Антиметодология «Случайное изменение»	Анализ результатов экспериментов
2.5.3	Антиметодология «Виноват кто-то другой»	Гипотетический анализ
2.5.4	Специальный чек-лист	Анализ наблюдений и результатов экспериментов
2.5.5	Формулировка проблемы	Сбор информации
2.5.6	Научный метод	Анализ наблюдений
2.5.7	Цикл диагностики	Анализ жизненного цикла
2.5.8	Метод инструментов	Анализ наблюдений
2.5.9	Метод USE	Анализ наблюдений

Таблица 2.4 (продолжение)

Раздел	Методология	Тип
2.5.10	Метод RED	Анализ наблюдений
2.5.11	Определение характеристик рабочей нагрузки	Анализ наблюдений, планирование мощности
2.5.12	Анализ с увеличением детализации	Анализ наблюдений
2.5.13	Анализ задержек	Анализ наблюдений
2.5.14	Метод R	Анализ наблюдений
2.5.15	Трассировка событий	Анализ наблюдений
2.5.16	Базовые статистики	Анализ наблюдений
2.5.17	Статическая настройка производительности	Анализ наблюдений, планирование мощности
2.5.18	Настройка кэширования	Анализ наблюдений, настройка
2.5.19	Микробенчмаркинг производительности	Анализ результатов экспериментов
2.5.20	Мантры производительности	Настройка
2.6.5	Теория массового обслуживания	Статистический анализ, планирование мощности
2.7	Планирование мощности	Планирование мощности, настройка
2.8.1	Количественная оценка прироста производительности	Статистический анализ
2.9	Мониторинг производительности	Анализ наблюдений, планирование мощности

Мониторинг производительности, теория массового обслуживания и планирование мощности рассматриваются далее в этой главе. Некоторые из упомянутых методологий подробнее обсуждаются в других главах в различных контекстах, где также представлены дополнительные методологии для конкретных целей анализа производительности. Эти дополнительные методологии перечислены в табл. 2.5.

Таблица 2.5. Дополнительные методологии анализа производительности

Раздел	Методология	Тип
1.10.1	Анализ производительности Linux за 60 секунд	Анализ наблюдений
5.4.1	Профилирование процессора	Анализ наблюдений
5.4.2	Анализ простоев	Анализ наблюдений
6.5.5	Анализ тактов	Анализ наблюдений
6.5.8	Настройка приоритетов	Настройка
6.5.9	Управление ресурсами	Настройка

Раздел	Методология	Тип
6.5.10	Привязка к процессору	Настройка
7.4.6	Определение утечек	Анализ наблюдений
7.4.10	Уменьшение потребления памяти	Анализ результатов экспериментов
8.5.1	Анализ дисков	Анализ наблюдений
8.5.7	Разделение рабочей нагрузки	Настройка
9.5.10	Масштабирование	Планирование мощности, настройка
10.5.6	Перехват пакетов	Анализ наблюдений
10.5.7	Анализ TCP	Анализ наблюдений
12.3.1	Пассивное тестирование производительности	Анализ результатов экспериментов
12.3.2	Активное тестирование производительности	Анализ наблюдений
12.3.6	Нестандартное тестирование производительности	Разработка программного обеспечения
12.3.7	Пиковая нагрузка	Анализ результатов экспериментов
12.3.8	Проверка правильности	Анализ наблюдений

Мы начнем с исследования часто используемых, но наиболее слабых методологий, включая антиметодологии. Приступая к анализу проблем с производительностью, в первую очередь нужно применить методологию постановки задачи и только потом переходить к другим.

2.5.1. Антиметодология «Уличный фонарь»

В действительности это методология *отсутствия* продуманной методологии. Исследователь анализирует производительность, выбирая знакомые инструменты наблюдения, найденные в интернете или просто наугад, и пытается обнаружить что-нибудь очевидное. Этот подход может увенчаться или не увенчаться успехом и сопряжен с риском упустить из виду многие типы проблем.

Аналогично можно попытаться настроить производительность, используя метод проб и ошибок, изменяя известные и знакомые настраиваемые параметры и наблюдая, дает ли такое изменение положительный результат. Даже если с помощью этого метода удастся обнаружить проблему, путь к ней может быть долгим, потому что перед этим случается опробовать инструменты или настройки, не связанные с проблемой, только потому, что они знакомы. По этой причине эта методология названа в честь эффекта, получившего название «эффект уличного фонаря», который отлично иллюстрирует притча:

Однажды ночью полицейский увидел, как пьяный что-то ищет под фонарем. Полицейский подошел поближе и спросил, что тот ищет. Пьяный отвечает, что потерял ключи. Полицейский начинает помогать ему и после долгих безуспешных поисков спрашивает: «Вы уверены, что потеряли их здесь, под фонарем?» Пьяный отвечает: «Нет, но здесь светлее».

Характеристики производительности можно пытаться искать в $\text{top}(1)$, но не потому, что это имеет смысл, а потому, что исследователь не умеет пользоваться другими инструментами.

Проблема, которую обнаруживает эта методология, может оказаться не *основной*, а *побочной* проблемой. Другие методологии позволяют количественно оценить результаты, позволяя исключить ложные срабатывания и уделить основное внимание более серьезным проблемам.

2.5.2. Антиметодология «Случайное изменение»

Эта антиметодология опирается на анализ результатов экспериментов. Исследователь пытается угадать причину проблемы, а затем меняет настройки, пока проблема не исчезнет. Чтобы определить, улучшилась ли производительность, после каждого изменения изучается метрика, например время выполнения приложения, время выполнения операции, задержка, скорость работы (операций в секунду) или пропускная способность (байтов в секунду). При этом используется следующий подход:

1. Выбирается случайная настройка для изменения (например, некоторый параметр).
2. Изменяется в одном направлении.
3. Измеряется производительность.
4. Затем эта настройка изменяется в другом направлении.
5. Снова измеряется производительность.
6. Результаты, полученные на шаге 3 или 5, сравниваются с исходными показателями. Если производительность улучшилась, изменения сохраняются и процесс повторяется с шага 1.

Этот процесс может в итоге помочь обнаружить настройку, которая подходит для тестируемой рабочей нагрузки, но он занимает очень много времени и может привести к настройкам, не имеющим смысла в долгосрочной перспективе. Например, изменение приложения для обхода ошибки в базе данных или в ОС может улучшить производительность, но позже эта ошибка может быть исправлена, а приложение все еще будет использовать настройку, которая потеряла смысл и суть которой никто не понял.

Также есть риск, что изменение, которое неправильно понято, вызовет более серьезную проблему при пиковой нагрузке, и тогда потребуются откатить его.

2.5.3. Антиметодология «Виноват кто-то другой»

Эта антиметодология включает следующие шаги:

1. Найти компонент системы или среды, за который вы не несете ответственности.
2. Выдвинуть предположение, что проблема в этом компоненте.

3. Направить задачу команде, ответственной за этот компонент.
4. Если окажется, что вы ошиблись, повторить процесс с шага 1.

«Может быть, все дело в сети? Уточните у сетевых администраторов, возможно, какие-то маршрутизаторы сбрасывают пакеты или типа того».

Вместо исследования проблемы с производительностью использующий эту методологию делает ее чужой проблемой, что может привести к напрасному расходованию ресурсов других команд, если окажется, что это не их проблема. Эту антиметодологию можно определить по отсутствию данных, обосновывающих гипотезу.

Чтобы не стать жертвой голословного обвинения, попросите обвинителя предоставить скриншоты, показывающие, какие инструменты были запущены и как интерпретировался вывод. Эти скриншоты и результаты интерпретации можно передать кому-нибудь, чтобы получить подкрепляющее мнение.

2.5.4. Специальный чек-лист

Методология пошагового выполнения чек-листа широко распространена среди специалистов служб поддержки и используется ими для проверки и настройки системы. Типичным сценарием может служить развертывание нового сервера или приложения в промышленной среде с последующей проверкой распространенных проблем под реальной нагрузкой. Такие чек-листы обычно являются специализированными — основанными на недавнем опыте и проблемах, характерных для систем этого типа.

Вот пример одного элемента такого чек-листа:

Запустить `iostat -x 1` и проверить столбец `r_await`. Если под нагрузкой его значение неизменно превышает 10 (мс), то либо операции чтения с диска выполняются медленно, либо диск перегружен.

Чек-лист может состоять из десятка подобных пунктов.

Чек-листы нередко оказываются очень ценными в условиях ограниченного времени, отпущенного на диагностику, но часто они действительны лишь на определенный период времени (см. раздел 2.3 «Основные понятия») и их следует обновлять как можно чаще, чтобы сохранить их актуальность. Кроме того, они имеют тенденцию сосредотачивать внимание на проблемах, для которых есть известные и легко выполнимые исправления, например установка настраиваемых параметров, но не на проблемах, решаемых нестандартными способами, требующими изменения исходного кода или окружения.

Если вы руководите службой поддержки, то специальный чек-лист может быть для вас эффективным способом убедиться, что сотрудники знают, как проверить наличие типичных проблем. Чек-лист может быть ясным и четким, показывающим, как идентифицировать каждую проблему и как ее решить. Но учтите, что этот список нужно постоянно обновлять.

2.5.5. Формулировка проблемы

Реагируя на появившуюся проблему, команда поддержки первым делом должна сформулировать проблему. Для этого клиенту задают следующие вопросы:

1. Почему вы думаете, что проблема именно с производительностью?
2. Хорошо ли работала система до этого?
3. Что изменилось за последнее время? Программное обеспечение? Аппаратное обеспечение? Нагрузка?
4. Можно ли обозначить проблему в терминах задержки или времени выполнения?
5. Влияет ли проблема на других людей либо приложения или только на вас?
6. Какое программное и аппаратное обеспечение используется? Версии? Конфигурация?

Простые вопросы и ответы на них часто помогают выявить непосредственную причину и решение. По этой причине формулировка проблемы включена как отдельная методология и должна использоваться при решении новых проблем.

Мне доводилось решать проблемы с производительностью по телефону, используя только метод формулировки проблемы, без входа на какой-либо сервер или просмотра каких-либо метрик.

2.5.6. Научный метод

Научный метод предназначен для изучения неизвестного и основывается на выдвижении гипотез и их проверке. Его можно кратко описать следующими шагами:

1. Вопрос.
2. Гипотеза.
3. Прогноз.
4. Проверка.
5. Анализ.

Вопрос — это формулировка проблемы. Сформулировав проблему, можно выдвинуть предположение о причине плохой работы. Затем создается тест, который может быть тестом наблюдения или экспериментальным, для проверки прогноза, основанного на гипотезе. И в заключение проводится анализ результатов, полученных в ходе тестирования.

Например, вы обнаруживаете, что производительность приложения снизилась после переноса в систему с меньшим объемом оперативной памяти, и предполагаете, что причиной снижения производительности является меньший размер кэша файловой системы. Вы можете использовать *тест наблюдения*, чтобы измерить частоту промахов кэша в обеих системах, предсказав, что в системе с меньшим кэшем частота промахов будет выше. *Экспериментальный тест* будет заключаться в увеличении

размера кэша (добавлении ОЗУ) и предсказания, что производительность улучшится. Другой, более простой экспериментальный тест — искусственное уменьшение размера кэша (с использованием настраиваемых параметров) и предсказание, что производительность после этого ухудшится.

Ниже приводятся еще несколько примеров.

Пример (наблюдение)

1. Вопрос: в чем причина медленной обработки запросов в базе данных?
2. Гипотеза: влияние действий других подписчиков (других клиентов облачного окружения), вызывающих дисковый ввод/вывод, из-за чего возникает конкуренция за дисковый ввод/вывод (через файловую систему).
3. Прогноз: если измерить задержку ввода/вывода в файловой системе во время выполнения запроса, то выяснится, что причина кроется в файловой системе.
4. Проверка: трассировка задержек в файловой системе относительно задержек в обработке запросов показала, что на ожидание файловой системы тратится менее 5 % времени.
5. Анализ: причина медленной обработки запросов не в файловой системе.

Проблема еще не решена, но некоторые крупные компоненты среды уже можно исключить из рассмотрения. Человек, проводящий исследование, может вернуться к шагу 2 и выдвинуть новую гипотезу.

Пример (эксперимент)

1. Вопрос: почему HTTP-запросы, посылаемые хостом А, обрабатываются хостом С дольше, чем запросы, посылаемые хостом В?
2. Гипотеза: хосты А и В находятся в разных вычислительных центрах.
3. Прогноз: если хост А перенести в вычислительный центр, где находится хост В, то это решит проблему.
4. Проверка: перенос хоста А и измерение производительности.
5. Анализ: производительность изменилась в соответствии с гипотезой, проблема решена.

Если проблема не была решена, то перед выдвижением новой гипотезы отмените экспериментальное изменение (в данном случае верните хост А назад) — одновременное изменение нескольких факторов затрудняет определение, какой из них оказал наибольшее влияние!

Пример (эксперимент)

1. Вопрос: почему производительность файловой системы снижается при увеличении размера кэша?

2. Гипотеза: в большом кэше хранится больше записей, и для управления большим кэшем требуется больше вычислительных ресурсов.
3. Прогноз: если постепенно уменьшать размер записи, то в кэш того же объема поместится больше записей, что, соответственно, повлечет ухудшение производительности.
4. Проверка: протестировать ту же рабочую нагрузку, постепенно уменьшая размер записей.
5. Анализ: результаты представлены в виде графиков и согласуются с прогнозом. Далее необходимо выполнить анализ процедур управления кэшем.

Это пример *отрицательного теста* — намеренного снижения производительности, чтобы узнать больше о целевой системе.

2.5.7. Цикл диагностики

Цикл диагностики похож на научный метод:

гипотеза → инструментальная проверка → результаты → гипотеза.

Как и научный метод, этот метод проверяет гипотезу путем сбора данных. Слово «цикл» в названии подчеркивает, что результаты измерений могут привести к новой гипотезе, ее проверке и уточнению и т. д. Примерно так врач проводит серию тестов для диагностики заболевания, уточняя диагноз по результатам каждого теста.

Оба подхода имеют хороший баланс теории и фактических данных. Старайтесь быстрее переходить от гипотез к данным, чтобы как можно раньше отбраковать ошибочные гипотезы и уделить больше внимания верным.

2.5.8. Метод инструментов

Инструментально-ориентированный подход заключается в следующем:

1. Составить список доступных инструментов анализа производительности (при желании можно установить или приобрести дополнительные инструменты).
2. Перечислить полезные метрики, измеряемые каждым инструментом.
3. Перечислить возможные способы интерпретации каждой метрики.

В результате должен получиться чек-лист, описывающий, какие метрики и с помощью каких инструментов можно получить и как эти метрики интерпретировать. Этот метод может быть очень эффективным, но полагается исключительно на доступные (или известные) инструменты, из-за чего может сложиться неполное представление о системе как в антиметодологии «Уличный фонарь». Хуже того, исследователь даже не подозревает об этой неполноте. Проблемы, требующие применения специальных инструментов (например, динамической трассировки), могут остаться невыявленными и нерешенными.

На практике метод инструментов действительно выявляет определенные узкие места, ошибки и проблемы других видов, но страдает некоторой неэффективностью.

При наличии большого количества инструментов и метрик их перебор может занять много времени. Ситуация усугубляется, когда имеется несколько инструментов с одинаковыми или почти одинаковыми функциональными возможностями, и исследователь тратит дополнительное время, пытаясь понять плюсы и минусы каждого из них. В некоторых случаях, например в микробенчмаркинге файловой системы, может быть более десятка инструментов на выбор, тогда как достаточно одного из них¹.

2.5.9. Метод USE

Метод USE (Utilization, Saturation, and Errors — потребление, насыщение и ошибки) следует использовать на ранних этапах исследования производительности для выявления узких мест [Gregg 13b]. Эта методология предназначена для анализа с точки зрения ресурсов, и в общем виде ее можно представить так:

Для каждого ресурса проверьте его потребление, насыщение и наличие ошибок.

Термины в этом определении интерпретируются так:

- **Ресурсы:** все функциональные компоненты физического сервера (процессоры, шины, ...). Также исследованию могут быть подвергнуты некоторые программные ресурсы, при условии, что метрики имеют смысл.
- **Потребление:** процент времени для заданного временного интервала, в течение которого ресурс обслуживал рабочую нагрузку. Часто, когда ресурс занят, он все еще может выполнять дополнительную работу; невозможность этого определяется уровнем насыщения.
- **Насыщение:** такое состояние ресурса, когда для него есть дополнительная работа, но он не может приступить к ее выполнению немедленно, из-за чего задания часто вынуждены ждать в очереди. Иногда это называют *давлением*.
- **Ошибки:** количество ошибок.

Для некоторых типов ресурсов, включая оперативную память, под потреблением подразумевается использованный объем. Это отличие от определения, основанного

¹ Как-то мне довелось услышать такой аргумент в поддержку использования нескольких инструментов, дублирующих друг друга: «Конкуренция — это хорошо». Я был бы осторожен с этим: конечно, использование похожих инструментов может быть полезно для перекрестной проверки результатов (и я часто проверяю инструменты BPF с помощью Ftrace), но необдуманное применение нескольких похожих инструментов может привести к пустой трате времени разработчика, которое можно было бы использовать более эффективно для решения других задач, а также к пустой трате времени конечных пользователей, которым приходится оценивать каждый выбор.

на времени, объяснялось выше в разделе 2.3.11 «Потребление». Когда потребление ресурса достигает 100 %, он оказывается не в состоянии принять дополнительные задания и либо ставит их в очередь (увеличивает насыщение), либо возвращает ошибки, которые также выявляются с помощью метода USE.

Ошибки важно изучать, потому что они могут снижать производительность, оставаясь незамеченными, особенно когда ресурс работает в режиме автоматического восстановления после отказов. Сюда можно отнести операции, выполняющие повторные попытки после завершения с ошибкой, и устройства, перемещающиеся в резервный пул после неудачи.

В отличие от метода инструментов, метод USE предполагает перебор системных ресурсов, а не инструментов. Этот метод поможет вам составить полный список вопросов, после чего вы сможете приступить к выбору инструментов для поиска ответов на них. Даже когда не удастся найти инструменты для ответа на некоторые вопросы, знание самих этих вопросов может быть чрезвычайно полезно для аналитика, потому что это «известные неизвестные».

Метод USE также фокусируется на анализе ограниченного числа ключевых метрик, поэтому все системные ресурсы проверяются очень быстро. После этого, если проблемы не обнаружались, можно использовать другие методики.

Процедура

На рис. 2.12 изображена схема метода USE. Проверка ошибок стоит первой, потому что обычно они легко интерпретируются (чаще всего ошибки — это объективная, а не субъективная метрика), и исключение ошибок перед исследованием других метрик позволяет эффективно экономить время. Затем проверяется насыщенность, потому что это состояние интерпретируется быстрее, чем потребление: проблемой может быть любой уровень насыщения.

Этот метод выявляет проблемы, которые могут быть узкими местами в системе. К сожалению, у системы может быть несколько проблем с производительностью, поэтому есть риск первой обнаружить незначительную проблему. Каждый выявленный недостаток можно исследовать с помощью дополнительных методологий перед возвратом к методу USE, если потребуется изучить другие ресурсы.

Выражение метрик

Метрики в методе USE обычно выражаются так:

- **Потребление:** в процентах за интервал времени (например, «один процессор загружен работой на 90 %»).
- **Насыщенность:** как длина очереди ожидания (например, «средняя длина очереди задач, готовых к выполнению на процессоре, равна четырем»).
- **Ошибки:** как количество зарегистрированных ошибок (например, «на этом диске было зафиксировано 50 ошибок»).

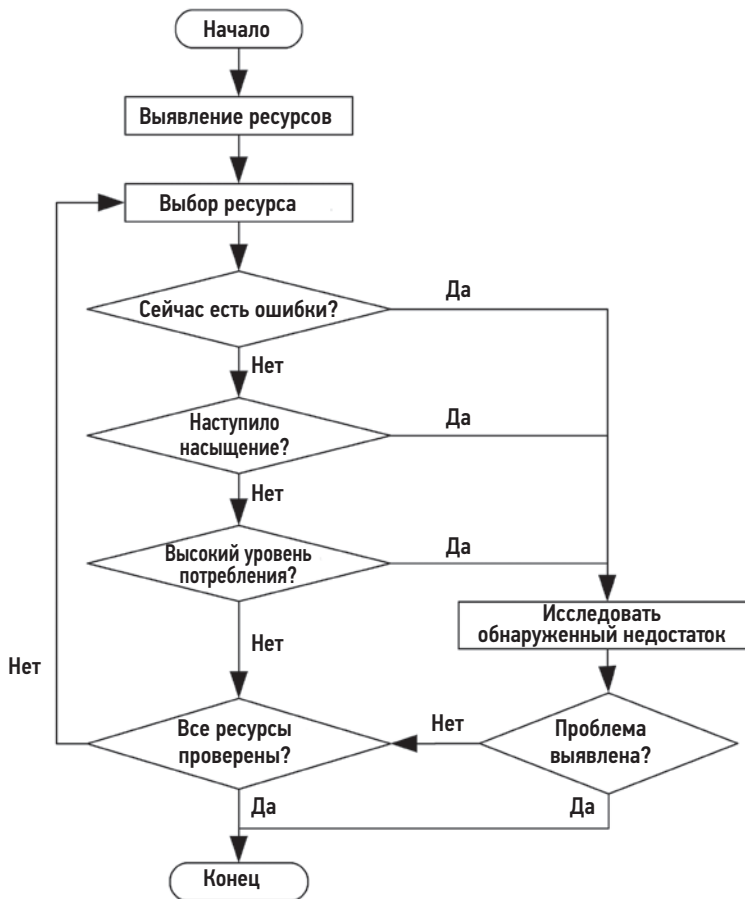


Рис. 2.12. Схема метода USE

Это может показаться нелогичным, но кратковременное увеличение потребления может вызвать проблемы с насыщенностью и производительностью, даже если общее потребление оставалось низким в течение длительного периода времени. Некоторые инструменты мониторинга сообщают среднюю величину потребления за последние 5 минут. Например, нагрузка на процессор может сильно меняться с каждой секундой, поэтому пятиминутное среднее значение может скрывать короткие периоды 100 %-ной нагрузки и, следовательно, насыщения.

Рассмотрим оплату проезда на платном шоссе. Коэффициент использования можно определить как количество пунктов оплаты, занятых обслуживанием автомобилей. Стопроцентная занятость означает, что вы не сможете найти пустую будку и должны встать в очередь (возникло состояние насыщения). Если бы я сказал, что средняя загрузка пункта оплаты в течение дня составляет 40 %, означает ли

это, что в течение дня ни один автомобиль не стоял в очереди? Вполне возможны часы пиковой нагрузки, когда занятость пункта оплаты достигает 100 %, но средняя занятость за день намного ниже.

Список ресурсов

Первый шаг в методе USE — создание списка ресурсов. Список должен быть как можно более полным. Вот типичный список аппаратных ресурсов сервера с конкретными примерами:

- **Процессоры:** тип сокета, количество ядер, аппаратные потоки (виртуальные процессоры).
- **Оперативная память:** DRAM.
- **Сетевые интерфейсы:** используемые порты Ethernet, высокоскоростная коммутируемая сеть.
- **Устройства хранения:** диски, адаптеры хранения.
- **Ускорители:** графические процессоры, TPU, FPGA и т. д., если используются.
- **Контроллеры:** хранилище, сеть.
- **Шины:** процессора, памяти, ввода/вывода.

Обычно каждый компонент интерпретируется как отдельный тип ресурса. Например, оперативная память — это ресурс *емкости*, а сетевые интерфейсы — это ресурс *ввода/вывода* (который может оцениваться такими метриками, как IOPS или пропускная способность). Некоторые компоненты могут иметь черты ресурсов нескольких типов, например, устройство хранения является одновременно ресурсом ввода/вывода и ресурсом емкости. Учитывать необходимо все типы, которые могут привести к снижению производительности. Также обратите внимание, что ресурсы ввода/вывода можно дополнительно исследовать как *системы массового обслуживания*, которые ставят запросы в очередь, а затем обслуживают их.

Некоторые физические компоненты, такие как аппаратные кэши (например, кэши процессора), можно не включать в чек-лист. Метод USE наиболее эффективен для ресурсов, производительность которых снижается при высоком потреблении или насыщении, тогда как высокий коэффициент использования кэша *улучшает* производительность. Эти компоненты можно проверить с помощью других методик. Если вы не уверены, следует ли включать ресурс в список, то включите его, а затем оцените, насколько это было оправданно.

Функциональная блок-схема

Другой способ составить список ресурсов — найти или нарисовать функциональную блок-схему системы, как, например, на рис. 2.13. Такая блок-схема также показывает отношения, которые могут пригодиться при поиске узких мест в потоке данных.

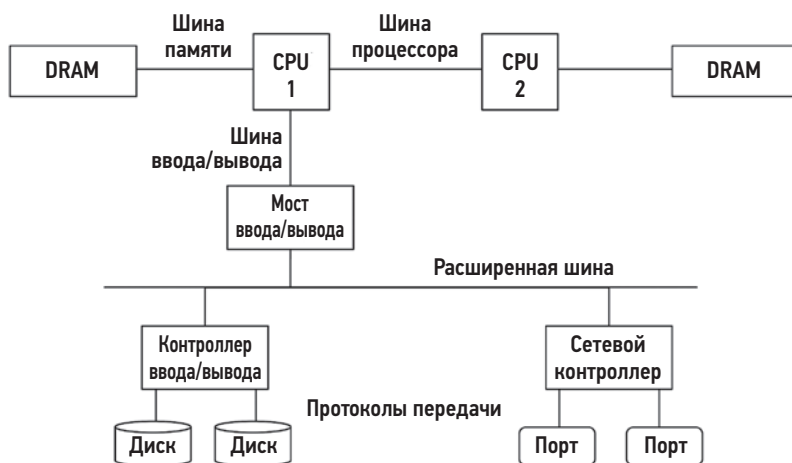


Рис. 2.13. Пример функциональной блок-схемы двухпроцессорной системы

Шины процессора, памяти и ввода/вывода часто упускаются из виду. К счастью, они редко оказываются узкими местами, потому что обычно проектируются с избыточной пропускной способностью. Однако если это не так, то решить проблемы с ними может быть очень сложно. Иногда для этого нужно обновить материнскую плату или уменьшить нагрузку; например, применение программных методов «зего сору» (ноль копирований) способствует уменьшению нагрузки на шину памяти.

Дополнительную информацию о шинах ищите в главе 6 «Процессоры» в разделе 6.4.1 «Аппаратное обеспечение».

Метрики

После составления списка ресурсов следует подумать о том, какие метрики использовать для оценки потребления, насыщения и наличия ошибок. В табл. 2.6 показаны некоторые примеры ресурсов и типы метрик, а также возможные метрики (для обобщенной ОС).

Эти метрики могут быть средними значениями за некоторый интервал времени или счетчиками.

Перечислите все возможные комбинации и добавьте в список инструкции по получению каждой метрики. Обратите внимание на метрики, которые в настоящее время недоступны, — это известные неизвестные. В итоге вы получите список, включающий примерно 30 метрик, часть из которых трудно, а часть вообще невозможно измерить. К счастью, наиболее распространенные проблемы обычно обнаруживаются с помощью простых метрик (например, насыщение процессора, насыщение памяти, потребление сетевого интерфейса, потребление диска), поэтому их можно проверить в первую очередь.

В табл. 2.7 представлены примеры некоторых более сложных комбинаций.

Таблица 2.6. Пример метрик для метода USE

Ресурс	Тип	Метрика
Процессор	Потребление	Потребление процессора (каждого процессора в отдельности или всех процессоров в общем)
Процессор	Насыщение	Длина очереди задач, готовых к выполнению, задержка планировщика, давление на процессор (подсистема PSI в ядре Linux)
Память	Потребление	Доступная свободная память (в системе в целом)
Память	Насыщение	Подкачка, сканирование страниц, события нехватки памяти, давление на память (подсистема PSI в ядре Linux)
Сетевой интерфейс	Потребление	Пропускная способность приема относительно максимальной ширины полосы пропускания, пропускная способность передачи относительно максимальной ширины полосы пропускания
Устройство хранения	Потребление	Процент времени, когда устройство было занято
Устройство хранения	Насыщение	Длина очереди ожидающих заданий, давление ввода/вывода (подсистема PSI в ядре Linux)
Устройство хранения	Ошибки	Ошибки в работе устройства (программные, аппаратные)

Таблица 2.7. Пример дополнительных метрик для метода USE

Ресурс	Тип	Метрика
Процессор	Ошибки	Например, исключения машинного контроля (machine check exceptions). Ошибки кэша процессора ¹
Память	Ошибки	Например, ошибки вызова malloc(), хотя с настройками ядра по умолчанию такие ошибки — большая редкость
Сеть	Насыщение	Ошибки, связанные с насыщением сетевого интерфейса, например «переполнения» в Linux
Контроллер устройств хранения	Потребление	В зависимости от типа контроллера в качестве метрики, характеризующей текущую активность, можно использовать максимальное количество операций ввода/вывода в секунду или пропускную способность
Шина процессора	Потребление	Пропускная способность порта относительно максимальной ширины полосы пропускания (счетчики производительности)
Шина памяти	Насыщение	Холостые циклы памяти, большое количество циклов на инструкцию (счетчики производительности)
Шина ввода/вывода	Потребление	Пропускная способность шины относительно максимальной ширины полосы пропускания (аппаратное обеспечение может поддерживать соответствующие счетчики производительности, например событий «uncore» в устройствах Intel)

¹ Например, ошибки, которые исправляются кодом коррекции ошибок (Error-Correcting Code, ECC) в строках кэша процессора (если поддерживается). Некоторые ядра отключают процессоры, обнаружив увеличение таких ошибок.

Часть из них может быть недоступна для стандартных инструментов ОС, и для их оценки необходимо использовать динамическую трассировку или счетчики производительности.

В приложении А я привожу примерный чек-лист для метода USE в Linux, в котором перечислены все комбинации аппаратных ресурсов с соответствующими инструментами наблюдения, а также некоторые программные ресурсы, в том числе и те, что описаны в следующем разделе.

Программные ресурсы

Аналогично можно исследовать некоторые программные ресурсы. Обычно это относится к более мелким компонентам ПО (не к целым приложениям), например:

- **Блокировки и мьютексы:** потребление можно определить как время удержания блокировки, а насыщение — по длине очереди ожидания, в которой находятся потоки, ожидающие освобождения блокировки.
- **Пулы потоков выполнения:** потребление можно определить как время, в течение которого потоки были заняты обработкой запросов, а насыщение — по количеству запросов, ожидающих обслуживания.
- **Емкость процессов/потоков:** система может ограничивать количество процессов и/или потоков. Потребление этой емкости можно определить как текущее количество выполняющихся процессов или потоков. Метрикой насыщения может служить время ожидания возможности запуска нового процесса или потока, а ошибкой — ситуация, когда запуск не удался (например, ошибка «cannot fork»).
- **Емкость пула файловых дескрипторов:** подобно емкости процессов/потоков, но для файловых дескрипторов.

Если у вас есть действенные метрики, используйте их. Либо подумайте об альтернативных методологиях, таких как анализ задержки.

Возможные интерпретации

Вот несколько общих предложений по интерпретации типов метрик:

- **Потребление:** 100 %-ное потребление обычно говорит о том, что этот ресурс является узким местом (проверьте его насыщение, чтобы подтвердить или опровергнуть это). Потребление на уровне более 60 % может говорить о вероятных проблемах, так как, в зависимости от интервала оценки, могут оставаться незамеченными короткие всплески до 100 % потребления. Кроме того, работу некоторых ресурсов, таких как жесткие диски (но не процессоры), нельзя прерывать во время операции даже для выполнения задания с более высоким приоритетом. По мере увеличения потребления задержки в очереди становятся более частыми и заметными. Более подробно уровень потребления 60 % описывается в разделе 2.6.5 «Теория массового обслуживания».

- **Насыщение:** любой уровень насыщения (отличный от нуля) может быть проблемой. Насыщение можно измерить как длину очереди ожидания или время, потраченное на ожидание в очереди.
- **Ошибки:** счетчики ошибок, отличные от нуля, заслуживают внимания, особенно если они увеличиваются при низкой производительности.

Отрицательные результаты оценки — низкое потребление, отсутствие насыщения, отсутствие ошибок — легко интерпретировать. Это более полезно, чем кажется, — сужение исследуемой сферы после выяснения, что проблема *не* связана с данным ресурсом, может помочь быстрее сосредоточиться на проблемной области.

Управление ресурсами

В облачных и контейнерных средах может применяться программное управление ресурсами для ограничения количества клиентов, совместно использующих систему. Также может быть ограничен объем доступной памяти, количество процессоров, объем дискового и сетевого ввода/вывода. Например, в Linux используется механизм контрольных групп `cgroup` для ограничения потребления ресурсов контейнерами. Каждое из установленных ограничений можно проверить с помощью метода `USE`, подобно проверке физических ресурсов.

Например, «потребление памяти» может означать использование памяти подписчиком относительно установленного верхнего предела. Наступление «насыщения емкости памяти» можно увидеть по ошибкам операций выделения новой памяти или срабатыванию механизма подкачки для клиента, даже если хост-система не испытывает нехватки памяти. Эти ограничения обсуждаются в главе 11 «Облачные вычисления».

Микросервисы

Архитектура микросервисов подвержена аналогичной проблеме слишком большого количества метрик. Для каждого сервиса их может быть так много, что проверить все очень сложно, и есть риск упустить из виду области, для которых метрик пока нет. Метод `USE` способен решить такие проблемы с микросервисами. Например, для типичного микросервиса в Netflix используются следующие метрики `USE`:

- **Потребление:** среднее потребление процессора по всему кластеру экземпляров.
- **Насыщение:** приблизительное значение — разница между 99-м перцентилем задержки и средней задержкой (предполагается, что 99-й перцентиль связан с насыщением).
- **Ошибки:** ошибки обработки запроса.

В Netflix эти три метрики проверяются для каждого микросервиса с помощью общесистемного инструмента мониторинга Atlas [Harrington 14].

Есть аналогичная методология, разработанная специально для сервисов: метод RED.

2.5.10. Метод RED

Основное внимание в этой методологии уделяется сервисам, обычно облачным сервисам в архитектуре микросервисов. Она определяет три метрики для мониторинга состояния сервисов с точки зрения пользователя согласно [Wilkie 18]:

Для каждого сервиса проверьте частоту запросов, ошибок и продолжительность.

Метрики:

- **Частота запросов:** количество запросов на обслуживание в секунду.
- **Ошибки:** количество неудачных запросов.
- **Продолжительность:** время обработки запросов (с учетом статистик распределения, таких как процентиля, в дополнение к среднему: см. раздел 2.8 «Статистики»).

Ваша задача — нарисовать схему системы микросервисов и убедиться, что эти три метрики извлекаются из всех сервисов. (Такую диаграмму можно получить с помощью инструментов распределенной трассировки.) Метод RED обладает теми же преимуществами, что и метод USE: скорость, простота использования и полнота охвата.

Метод RED был создан Томом Уилки (Tom Wilkie), который также разработал реализации метрик методов USE и RED для Prometheus и панели мониторинга на основе Grafana [Wilkie 18]. Эти методологии дополняют друг друга: метод USE используется для проверки работоспособности физической машины, а метод RED — работоспособности сервисов.

Включение метрики частоты запросов дает важную информацию, которая может пригодиться на ранних этапах исследования и помочь выяснить, обусловлена низкая производительность высокой нагрузкой или архитектурой (см. раздел 2.3.8 «Нагрузка и архитектура»). Если частота запросов оставалась постоянной, но продолжительность обработки увеличилась, то это указывает на проблему с архитектурой, то есть с самим сервисом. Если увеличилась и частота запросов, и продолжительность обработки, то проблема может быть в увеличении нагрузки. Эту ситуацию можно дополнительно исследовать с помощью метода определения характеристик рабочей нагрузки.

2.5.11. Определение характеристик рабочей нагрузки

Определение характеристик рабочей нагрузки — это простой и эффективный метод выявления проблем, связанных с приложенной нагрузкой. В центре внимания этого метода находится *вход* в систему, а не ее производительность. Система может не иметь проблем с архитектурой, реализацией или конфигурацией, но испытывать существенную нагрузку, превышающую ее возможности.

Рабочие нагрузки можно охарактеризовать, ответив на следующие вопросы:

- **Кто** вызывает нагрузку: идентификатор процесса, идентификатор пользователя или удаленный IP-адрес.
- **Почему** вызывается нагрузка: пути в коде, трассировки стека.
- **Что** характеризует нагрузку: IOPS, пропускная способность, направление (чтение/запись), тип. При необходимости можно добавить дисперсию (стандартное отклонение).
- **Как** меняется нагрузка с течением времени: имеются ли закономерности изменения нагрузки в течение суток.

Проверить все это нужно. Даже если вы абсолютно уверены в возможных ответах, результаты могут вас очень удивить.

Рассмотрим следующий сценарий. Есть проблема с производительностью базы данных, которой пользуется пул веб-серверов. Следует ли проверить IP-адреса тех, кто использует базу данных? Вы уверены, что согласно конфигурации они будут принадлежать веб-серверам, но все равно проверяете их и обнаруживаете, что базой данных, похоже, пользуется весь интернет, снижая ее производительность. Вы подверглись DoS-атаке!

Наибольший выигрыш для производительности дает *исключение ненужной работы*. Иногда ненужная работа обусловлена неправильной работой приложений, например, поток выполнения застревает в цикле, создавая ненужную нагрузку на процессор. Иногда она может быть вызвана неправильной конфигурацией, например выполнением общесистемного резервного копирования в часы пик, или даже DoS-атакой, как описано выше. Определение характеристик рабочей нагрузки может выявить эти проблемы, которые устраняются своевременным техническим обслуживанием или изменением конфигурации.

Если выявленную рабочую нагрузку нельзя уменьшить, то можно подойти иначе — ограничить ее с использованием системных механизмов управления ресурсами. Например, задача резервного копирования системы может мешать работе производственной базы данных, потребляя ресурсы процессора для сжатия резервной копии и сетевые ресурсы для ее передачи. Использование процессора и сетевых ресурсов можно регулировать с помощью механизмов управления ресурсами (если они имеются в системе), чтобы притормозить резервное копирование без ущерба для работы базы данных.

Помимо выявления проблем, определение характеристик рабочей нагрузки также можно использовать для разработки имитационных тестов. В идеале, оценивая характеристики рабочей нагрузки, необходимо не только фиксировать средний ее уровень, но и собрать подробную информацию о распределении и дисперсии. Это может пригодиться для моделирования разнообразных ожидаемых рабочих нагрузок, а не только средней нагрузки. Дополнительную информацию о средних значениях и дисперсии (стандартном отклонении) вы найдете в разделе 2.8 «Статистики» и в главе 12 «Бенчмаркинг».

Анализ рабочей нагрузки также помогает отделить проблемы, связанные с нагрузкой, от проблем с архитектурой. Подробнее об этом рассказывается в разделе 2.3.8 «Нагрузка и архитектура».

Конкретные инструменты и метрики для определения характеристик рабочей нагрузки зависят от цели. Некоторые приложения ведут подробные журналы действий клиентов, которые могут служить источником информации для статистического анализа. Приложения также могут поддерживать предоставление ежедневных или ежемесячных отчетов об использовании клиентами, что тоже пригодится для получения дополнительной информации.

2.5.12. Анализ с увеличением детализации

Анализ с увеличением детализации начинается с изучения проблемы в целом, а затем фокус внимания сужается, исходя из предыдущих результатов. Области, не представляющие интереса, отбрасываются, а перспективные области подвергаются более глубокому изучению. Процесс может включать погружение в более глубокие слои программного стека, а при необходимости и аппаратного.

Вот три основных этапа анализа производительности системы с увеличением детализации [McDougall 06a]:

1. **Мониторинг:** используется для непрерывной регистрации изменения высокоуровневой статистики с течением времени, а также для выявления возможных проблем.
2. **Идентификация:** при наличии подозрений на проблему сужает область исследования до конкретных ресурсов и помогает выявить возможные узкие места.
3. **Анализ:** дальнейшее изучение конкретных областей системы с целью выявления первопричины и количественной оценки проблемы.

Мониторинг можно делать в масштабах всей компании, а результаты агрегировать по серверам или облачным экземплярам. Исторически для этой цели применяется простой протокол мониторинга сети (simple network monitoring protocol, SNMP), который можно использовать для мониторинга любого устройства, подключенного к сети, если оно поддерживает этот протокол. Современные системы мониторинга используют *экспортеры*: программные агенты, которые выполняются в каждой системе и поставляют необходимые метрики. Полученные данные записываются системой мониторинга и отображаются в виде графиков и схем. Это помогает заметить протяженные закономерности, которые легко упустить из виду при использовании инструментов командной строки в течение коротких периодов времени. Многие решения для мониторинга способны генерировать предупреждения, если возникает подозрение на проблему. Это побуждает аналитика переходить к следующему этапу.

Выявление проблем происходит непосредственно через анализ сервера и проверку компонентов системы: процессоров, дисков, памяти и т. д. Исторически для этого использовались инструменты командной строки — `vmstat(8)`, `iostat(1)` и `mpstat(1)`.

Сейчас же есть множество дашбордов мониторинга с графическим UI, которые отображают те же метрики и ускоряют анализ.

На этапе анализа для более глубокого изучения подозрительных участков используются инструменты трассировки или профилирования. Также для этой цели могут создаваться свои инструменты и даже проводиться исследование исходного кода (если таковой имеется). На этом этапе происходит постепенное увеличение детализации с погружением во все более глубокие слои программного стека, чтобы выяснить первопричину. В Linux для этого обычно используются `strace(1)`, `perf(1)`, а также инструменты BCC, `bpfttrace` и `Ftrace`.

Вот, например, перечень технологий, использованных для реализации этой трех-этапной методологии в облачной среде Netflix:

1. **Мониторинг:** Netflix Atlas — облачная платформа мониторинга с открытым исходным кодом [Harrington 14].
2. **Идентификация:** Netflix perfdash (формально Netflix Vector) — графический интерфейс для анализа отдельного экземпляра с дашбордами мониторинга, отображающими метрики USE.
3. **Анализ:** Netflix FlameCommander — инструмент для создания различных типов флейм-графиков, а также разнообразные инструменты командной строки в сеансе SSH, включая инструменты `Ftrace`, BCC и `bpfttrace`.

В Netflix мы используем эти технологии примерно так: с помощью Atlas выявляем проблемный микросервис, затем, применяя perfdash, сужаем область внимания до ресурса и, наконец, с помощью FlameCommander определяем пути в коде, потребляющие этот ресурс. Далее этот код можно исследовать с помощью инструментов BCC и `bpfttrace`.

Пять «почему»

На этапе анализа с увеличением детализации можно использовать вспомогательную методологию под названием «Пять “почему”» [Wikipedia 20]: спросите себя «почему?», ответьте на вопрос и повторите его еще четыре раза (или больше). Вот простой пример:

1. База данных начала долго обрабатывать многие запросы. Почему?
2. Проблема в задержке дискового ввода/вывода из-за использования файла подкачки. Почему?
3. Потребление памяти базой данных слишком большое. Почему?
4. Механизм распределения памяти потребляет больше памяти, чем должен. Почему?
5. Механизм распределения памяти страдает от проблемы фрагментации памяти.

Это реальный пример того, как многократные попытки найти ответы на вопросы при изучении проблемы неожиданно привели к исправлению в библиотеке распределения системной памяти.

2.5.13. Анализ задержек

В анализе задержки исследуется время, потраченное на выполнение операции, с последующей разбивкой на более мелкие составляющие и выделением компонентов с наибольшей задержкой. Такой подход позволяет идентифицировать и количественно оценить основную причину. Как и анализ с увеличением детализации, анализ задержек может погружаться в более глубокие слои программного стека, чтобы обнаруживать источник проблем.

Анализ может начинаться с исследования особенностей обработки рабочей нагрузки в приложении, а затем углубляться в библиотеки операционной системы, системные вызовы, ядро и драйверы устройств.

Например, анализ задержек, возникающих в MySQL при обработке запросов, может включать поиск ответов на следующие вопросы (примеры ответов приведены в скобках):

1. Есть ли проблема с задержками обработки запросов? (да)
2. Основное время обработки запроса расходуется на вычисления на процессоре или на ожидание вне процессора? (на ожидание вне процессора)
3. На что расходуется время ожидания вне процессора? (на ввод/вывод файловой системы)
4. Задержка ввода/вывода файловой системы связана с дисковым вводом/выводом или конфликтом блокировок? (с дисковым вводом/выводом)
5. Основное время дискового ввода/вывода тратится на организацию очереди или на выполнение операций ввода/вывода? (на выполнение операций)
6. Основное время в операциях дискового ввода/вывода тратится на инициализацию ввода/вывода или на передачу данных? (на передачу данных)

В этом примере на каждом шаге ставится вопрос, делящий задержку на две части, после чего анализируется бóльшая из них: поиск методом дихотомии, если хотите. Этот процесс приведен на рис. 2.14.

Когда из А и В выявляется составляющая, которая вносит наибольший вклад, она, в свою очередь, снова делится на составляющие А и В, и анализ продолжается.

Анализ задержек в обработке запросов к базе данных является целью метода R.

2.5.14. Метод R

Метод R — это метод анализа производительности, разработанный для баз данных Oracle и заключающийся в поиске источника задержки по событиям трассировки Oracle [Millsap 03]. Он позиционируется как «метод оценки производительности по времени отклика, способный принести максимальную экономическую выгоду вашему бизнесу», и ориентирован на определение и количественную оценку затрат времени в ходе обработки запросов. Несмотря на то что этот метод предназначался

для исследования баз данных, лежащий в его основе подход можно применить к любой системе, и потому его стоит упомянуть здесь как один из возможных путей исследования.

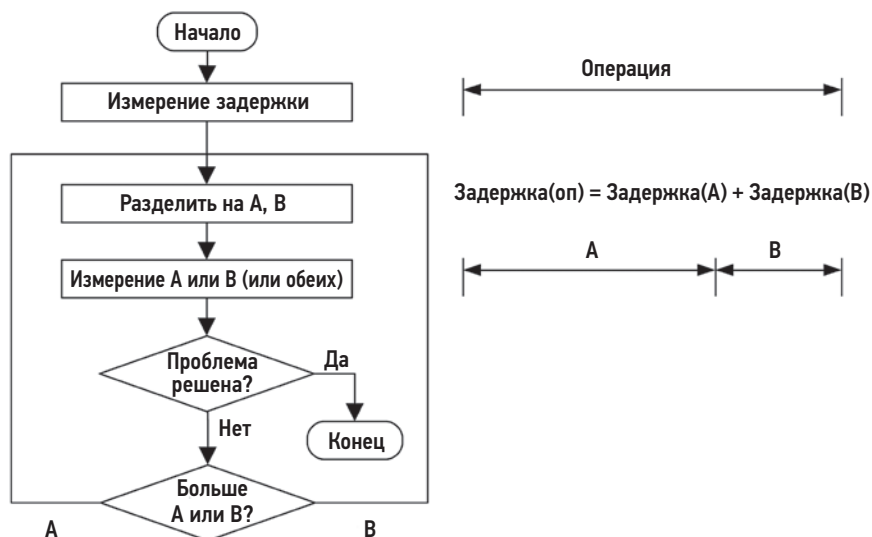


Рис. 2.14. Процедура анализа задержек

2.5.15. Трассировка событий

Основная работа систем заключается в обработке дискретных событий. К ним относятся инструкции процессора, дисковый ввод/вывод и другие дисковые команды, сетевые пакеты, системные вызовы, библиотечные вызовы, транзакции в приложениях, запросы к базе данных и т. д. При анализе производительности обычно изучаются сводные данные об этих событиях, например количество операций в секунду, байтов в секунду или средняя задержка. Иногда важные детали теряются в сводных данных, и поэтому события лучше изучать индивидуально.

Для устранения неполадок в сети часто приходится исследовать пакеты с помощью таких инструментов, как tcpdump(8). Вот пример обобщения пакетов в виде отдельных текстовых строк:

```
# tcpdump -ni eth4 -ttt
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth4, link-type EN10MB (Ethernet), capture size 65535 bytes
00:00:00.000000 IP 10.2.203.2.22 > 10.2.0.2.33986: Flags [P.], seq
1182098726:1182098918, ack 4234203806, win 132, options [nop,nop,TS val 1751498743
ecr 1751639660], length 192
00:00:00.000392 IP 10.2.0.2.33986 > 10.2.203.2.22: Flags [.), ack 192, win 501,
options [nop,nop,TS val 1751639684 ecr 1751498743], length 0
00:00:00.009561 IP 10.2.203.2.22 > 10.2.0.2.33986: Flags [P.], seq 192:560, ack 1,
```

```
win 132, options [nop,nop,TS val 1751498744 ecr 1751639684], length 368
00:00:00.000351 IP 10.2.0.2.33986 > 10.2.203.2.22: Flags [.], ack 560, win 501,
options [nop,nop,TS val 1751639685 ecr 1751498744], length 0
00:00:00.010489 IP 10.2.203.2.22 > 10.2.0.2.33986: Flags [P.], seq 560:896, ack 1,
win 132, options [nop,nop,TS val 1751498745 ecr 1751639685], length 336
00:00:00.000369 IP 10.2.0.2.33986 > 10.2.203.2.22: Flags [.], ack 896, win 501,
options [nop,nop,TS val 1751639686 ecr 1751498745], length 0
[...]
```

При необходимости `tcpdump(8)` может выводить больше или меньше информации (см. главу 10 «Сеть»).

Трассировку событий ввода/вывода для блочных устройств хранения можно выполнить с помощью инструмента `biosnoop(8)` (основанного на `BCC/BPF`):

```
# biosnoop
TIME(s)      COMM      PID    DISK    T  SECTOR  BYTES  LAT(ms)
0.000004     supervise  1950    xvda1   W  13092560 4096   0.74
0.000178     supervise  1950    xvda1   W  13092432 4096   0.61
0.001469     supervise  1956    xvda1   W  13092440 4096   1.24
0.001588     supervise  1956    xvda1   W  13115128 4096   1.09
1.022346     supervise  1950    xvda1   W  13115272 4096   0.98
[...]
```

В этом выводе команды `biosnoop(8)` можно видеть время завершения ввода/вывода (`TIME(s)`), информацию о процессе-инициаторе (`COMM`, `PID`), задействованное дисковое устройство (`DISK`), тип ввода/вывода (`T`), объем ввода/вывода (`BYTES`) и продолжительность (`LAT(ms)`). Дополнительную информацию об этом инструменте вы найдете в главе 9 «Диски».

Системные вызовы — еще одна распространенная цель для трассировки. В Linux трассировку системных вызовов можно выполнить с помощью команды `tracex` (см. главу 5 «Приложения») инструментов `strace(1)` и `perf(1)`. Эти инструменты также поддерживают возможность вывода отметок времени.

Трассируя события, ищите следующую информацию:

- **вход:** все атрибуты события запроса: тип, направление, размер и т. д.;
- **время:** время начала, время конца, задержка (разница);
- **результат:** наличие ошибки, результат события (например, объем успешно переданных данных).

Иногда причины проблем с производительностью можно понять, изучив атрибуты события запроса или результата. Отметки времени событий позволяют анализировать задержки, и их вывод поддерживается многими инструментами трассировки событий. В предыдущем примере `tcpdump(8)` я включил вывод разности отметок времени между пакетами, добавив параметр `-ttt`.

Изучение предшествующих событий дает дополнительную информацию. Событие с необычно высокой задержкой, известное как *выброс задержки*, может быть обусловлено предыдущими событиями, а не им самим. Например, событие в конце

очереди может иметь высокую задержку, обусловленную не его собственными свойствами, а событиями, поставленными в очередь ранее. Эту ситуацию можно определить по трассируемым событиям.

2.5.16. Базовые статистики

Во многих средах используются решения мониторинга для записи метрик производительности и их визуализации в виде линейных графиков с временем на оси X (см. раздел 2.9 «Мониторинг»). Такие графики могут показать, менялась ли метрика в последнее время и как она менялась. Иногда добавляются дополнительные графики с историческими данными, такими как средние значения или просто соответствующие другим временным диапазонам, для сравнения с текущим диапазоном. Например, многие дашборды в Netflix отображают дополнительный график, соответствующий тому же временному диапазону, но за предыдущую неделю, благодаря чему характер работы системы в 15:00 во вторник можно сравнить с характером работы в 15:00 в предыдущий вторник.

Такой подход хорошо зарекомендовал себя в анализе уже отслеживаемых метрик, графики изменений которых отображаются в GUI. Но в командной строке доступна масса системных метрик и подробностей, которые не отслеживаются решением мониторинга. Вы можете столкнуться с незнакомыми системными статистиками и задаться вопросом, являются ли они «нормальными» для сервера или свидетельствуют о проблеме.

Это не новая задача, и для ее решения есть своя методология, которая появилась раньше широкого распространения решений мониторинга с использованием линейных графиков. Она заключается в сборе *базовых статистик*. Под этим подразумевается сбор всех системных метрик, когда система находится под «нормальной» нагрузкой, и их запись в текстовый файл или базу данных для дальнейшего использования. Программное обеспечение для сбора базовых статистик может быть реализовано как сценарий командной оболочки, который запускает инструменты наблюдения и собирает информацию из других источников (например, извлекая содержимое файлов в /proc с помощью `cat(1)`). Применение профилировщиков и инструментов трассировки позволяет зафиксировать гораздо больше деталей, чем это возможно с помощью обычных средств мониторинга продуктов (но будьте осторожны: эти инструменты имеют высокий оверхед и могут ухудшить производительность системы). Базовые статистики могут собираться на регулярной основе (например, ежедневно), а также до и после изменений в системе или приложении, чтобы можно было проанализировать различия в производительности.

Если базовые статистики не собирались и мониторинг не производится, то для оценки текущей активности можно использовать некоторые инструменты наблюдения (основанные на счетчиках ядра), отображающие средние значения, накопленные с момента загрузки. Этот прием не дает достаточной точности, но все же лучше, чем ничего.

2.5.17. Статическая настройка производительности

Статическая настройка производительности фокусируется на проблемах сконфигурированной архитектуры, в отличие от других методологий, которые сосредоточены на оценке *динамической производительности* — производительности при обработке приложенной нагрузки [Elling 00]. Статический анализ производительности выполняется, когда система находится в состоянии покоя и без нагрузки.

Для статического анализа производительности и ее настройки нужно перечислить все компоненты системы и для каждого ответить на следующие вопросы:

- Есть ли смысл использовать данный компонент? (Устаревший, низкопроизводительный и т. д.)
- Имеет ли смысл данная конфигурация для предполагаемой рабочей нагрузки?
- Обеспечивает ли автоматически выбранная конфигурация компонента наилучшее соответствие предполагаемой рабочей нагрузке?
- Возникла ли в компоненте какая-то ошибка, из-за которой пострадала производительность?

Вот несколько примеров проблем, которые можно обнаружить при статической настройке производительности:

- согласование сетевого интерфейса: выбор 1 Гбит/с вместо 10 Гбит/с;
- неисправный диск в пуле RAID;
- старая версия операционной системы, приложений или прошивки;
- файловая система почти заполнена (что может вызвать проблемы с производительностью);
- несоответствие размера записи в файловой системе и размера блока ввода/вывода рабочей нагрузки;
- приложение выполняется в случайно оставленном включенным режиме отладки;
- сервер по ошибке настроен как сетевой маршрутизатор (включена переадресация трафика);
- сервер настроен на использование ресурсов, например аутентификации, удаленного вычислительного центра вместо локального.

К счастью, такие проблемы легко выявляются. Самое сложное — не забыть это сделать!

2.5.18. Настройка кэширования

Приложения и операционные системы могут использовать несколько кэшей для увеличения производительности ввода/вывода, от приложения до дисков. Полный список вы найдете в главе 3 «Операционные системы», в разделе 3.2.11 «Кэширование». Вот общая стратегия настройки кэширования на каждом уровне:

1. Старайтесь хранить данные в кэше как можно более высокого уровня, то есть как можно ближе к месту, где выполняется работа. Это уменьшит оверхед при попадании в кэш. В этом местоположении также должно быть доступно как можно больше метаданных, которые можно использовать для оптимизации политики хранения в кэше.
2. Убедитесь, что кэширование включено и работает.
3. Проверьте соотношение попаданий/промахов и частоту промахов.
4. Если размер кэша настраивается динамически, проверьте текущий размер.
5. Настройте кэш в соответствии с характером рабочей нагрузки. Эта задача зависит от доступных параметров настройки кэша.
6. Настройте рабочую нагрузку в соответствии с особенностями кэша, например, сократите количество потребителей кэша, чтобы освободить больше места для целевой рабочей нагрузки.

Проверьте, нет ли двойного кэширования, когда существуют два разных кэша, которые потребляют оперативную память и хранят одни и те же данные.

Также учитывайте общий прирост производительности, который дает настройка кэширования на каждом уровне. Настройка кэша процессора уровня 1 может сэкономить наносекунды, потому что промахи в этом кэше могут компенсироваться кэшем уровня 2. Но оптимизация кэширования в кэше процессора уровня 3 может дать гораздо больший прирост производительности за счет исключения обращений к более медленной оперативной памяти. (Кэши процессора описываются в главе 6 «Процессоры».)

2.5.19. Микробенчмаркинг производительности

Микробенчмаркинг производительности позволяет оценить производительность простых и искусственных рабочих нагрузок. Этим он отличается от *макробенчмаркинга производительности* (или *отраслевого эталонного тестирования*), который обычно направлен на оценку фактической и естественной рабочей нагрузки. Макробенчмаркинг выполняется путем моделирования рабочих нагрузок и нередко оказывается сложным в реализации и трудным для понимания делом.

Благодаря меньшему числу факторов, микробенчмаркинг проще в реализации и для понимания. Часто для микробенчмаркинга производительности в Linux используется инструмент *iperf(1)*, который оценивает пропускную способность стека ТСП: он может выявить узкие места во внешней сети (которые иначе трудно обнаружить) путем проверки счетчиков ТСП во время приложения промышленной нагрузки.

Микробенчмаркинг может выполняться с помощью *инструмента микробенчмаркинга*, который применяет рабочую нагрузку и оценивает производительность системы при ее обработке. Также можно использовать *генератор нагрузки*, который просто применяет рабочую нагрузку, оставляя измерения производительности другим

инструментам наблюдения (примеры генераторов нагрузки приводятся в главе 12 «Бенчмаркинг» в разделе 12.2.2 «Моделирование»). Оба подхода имеют право на жизнь, но инструменты микробенчмаркинга обычно безопаснее и позволяют проверить производительность с помощью других инструментов.

Вот некоторые примеры целей для микробенчмаркинга, включая второе измерение для тестов:

- **время выполнения системного вызова:** для `fork(2)`, `execve(2)`, `open(2)`, `read(2)`, `close(2)`;
- **операции чтения из файловой системы:** из кэшированного файла с разными размерами блоков от одного байта до одного мегабайта;
- **пропускная способность сети:** передача данных между конечными точками TCP с разными размерами буфера сокета.

В процессе микробенчмаркинга обычно целевая операция выполняется с максимальной частотой и измеряется время, необходимое для выполнения большого числа этих операций. После этого вычисляется среднее время (среднее время = общее время выполнения / количество операций).

В последующих главах будут описаны конкретные методологии микробенчмаркинга, перечислены цели и атрибуты. Более подробно тема тестирования рассматривается в главе 12 «Бенчмаркинг».

2.5.20. Мантры производительности

Цель этой методологии — показать, как лучше всего повысить производительность. Она перечисляет правила в порядке от наиболее эффективных к наименее эффективным:

1. Не делай этого.
2. Сделай это только один раз.
3. Делай меньше.
4. Делай позже.
5. Делай это, когда нет других дел.
6. Делай это параллельно.
7. Делай это оптимально.

Вот несколько примеров реализации этих правил:

1. Не делай этого: избавьтесь от ненужной работы.
2. Сделай это только один раз: кэширование.
3. Делай меньше: настройте извлечение и обновление данных так, чтобы они выполнялись как можно реже.

4. Делай позже: кэширование с отложенной записью.
5. Делай это, когда нет других дел: запланируйте выполнение работы в непииковые часы.
6. Делай это параллельно: используйте многопоточный режим вместо однопоточного.
7. Делай это оптимально: приобретите более производительное оборудование.

Это одна из моих любимых методологий, с которой меня познакомил Скотт Эммонс (Scott Emmons) в Netflix. Он приписывает ее авторство Крейгу Хэнсону (Craig Hanson) и Пэту Крейну (Pat Crain), но мне не удалось найти подтверждение его слов.

2.6. МОДЕЛИРОВАНИЕ

Аналитическое моделирование системы можно использовать для разных целей, в частности для *анализа масштабируемости*: изучения возможности масштабирования производительности с увеличением нагрузки или ресурсов. Ресурсы могут быть аппаратными (например, ядра процессора) или программными (процессы или потоки выполнения).

Аналитическое моделирование можно рассматривать как третий способ оценки производительности наряду с наблюдением за промышленной системой («измерение») и экспериментальным тестированием («симуляция») [Jain 91]. Оценка производительности точнее, когда применяются по крайней мере два из этих способов: аналитическое моделирование и симуляция или симуляция и измерение.

Анализ существующей системы можно начать с измерения: определения характера нагрузки и фактической производительности. Экспериментальный анализ путем симуляции рабочей нагрузки можно использовать, если в системе нет промышленной нагрузки или требуется оценить производительность при рабочих нагрузках, которые пока не наблюдаются в продакшене. Аналитическое моделирование можно использовать для прогнозирования производительности на основе результатов измерений или симуляции.

Анализ масштабируемости может показать, что производительность перестает линейно масштабироваться в определенной точке из-за ограниченности ресурсов. Эта точка называется *точкой перегиба*: в ней одна функция сменяется другой, в данном случае линейное масштабирование сменяется конкуренцией за ресурсы. Выявление таких точек поможет сфокусироваться на исследовании проблем производительности, препятствующих масштабируемости, — чтобы исправить их до того, как они проявятся в промышленной среде.

Более подробно эти шаги описываются в разделах 2.5.11 «Определение характеристик рабочей нагрузки» и 2.5.19 «Микробенчмаркинг производительности».

2.6.1. Корпоративные и облачные среды

Моделирование позволяет имитировать крупномасштабные корпоративные системы без затрат на владение ими, но производительность крупномасштабных сред складывается из очень большого числа факторов и ее трудно смоделировать точно.

С развитием облачных вычислений появилась возможность арендовать на короткий срок (на время тестирования) среду любого масштаба. Вместо создания математической модели для прогнозирования производительности можно определить характер рабочей нагрузки, смоделировать ее и затем протестировать в облачных средах разного масштаба. Некоторые результаты, такие как точки перегиба, могут остаться прежними, но теперь будут подтверждены фактическими измерениями, а не только теоретическими моделями. Кроме того, при тестировании в реальной среде можно обнаружить ограничители, не учтенные в вашей модели.

2.6.2. Визуальная идентификация

Когда есть возможность получить достаточный объем экспериментальных данных, построение графика зависимости производительности от параметра масштабирования может выявить закономерности.

На рис. 2.15 показана зависимость пропускной способности приложения от числа потоков выполнения. Как видно на графике, точка перегиба находится в районе восьми потоков, где наклон линии графика меняется. Теперь можно исследовать причины появления этой точки перегиба, например, изучив конфигурацию приложения и системы при количестве потоков, близком к восьми.

В данном случае система была оснащена восьмиядерным процессором, каждое ядро имело два аппаратных потока. Чтобы убедиться, что точка перегиба связана с количеством ядер процессора, можно исследовать производительность процессоров с меньшим или большим числом ядер (например, инструкций на такт — IPC; см. главу 6 «Процессоры»). Или можно повторить тестирование масштабируемости в системе с другим количеством ядер и подтвердить, что точка перегиба перемещается, как ожидалось.

Есть ряд профилей масштабируемости, которые можно идентифицировать визуально, без использования формальной модели. Они показаны на рис. 2.16.

Для каждого из них ось X определяет масштаб, а ось Y — итоговую производительность (пропускную способность, количество транзакций в секунду и т. д.). Вот эти профили:

- **Линейная масштабируемость:** производительность увеличивается пропорционально масштабированию ресурса. Линейный рост не может продолжаться вечно и часто является ранней стадией другого профиля масштабируемости.
- **Масштабируемость с конкуренцией:** некоторые компоненты архитектуры являются общими и могут использоваться только последовательно; конкуренция за эти общие ресурсы снижает эффективность масштабирования.

- **Масштабируемость с когерентностью:** затраты на поддержание когерентности данных, включая распространение изменений, начинают перевешивать преимущества масштабирования.
- **Точка перегиба:** имеет место фактор, который меняет профиль масштабируемости.
- **Потолок масштабируемости:** достигнут жесткий предел. Это может быть устройство, являющееся узким местом, например шина с ограниченной пропускной способностью или предел, установленный программно (в системе управления ресурсами).

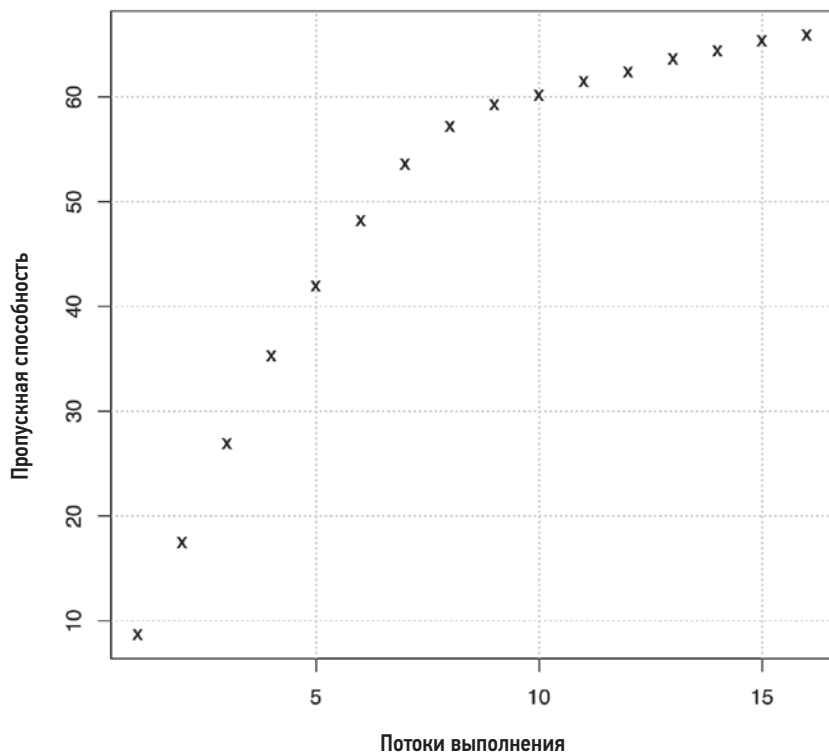


Рис. 2.15. Результаты тестирования масштабируемости

Визуальная идентификация может быть простой и эффективной, тем не менее использование математической модели позволит узнать больше о масштабируемости системы. Модель может иметь неожиданные отклонения от фактических данных, которые полезно исследовать и выяснить, заключается ли причина в модели и, следовательно, в вашем понимании системы или проблема в реальной масштабируемости. В следующих разделах представлены закон Амдала, универсальный закон масштабируемости и теория массового обслуживания.

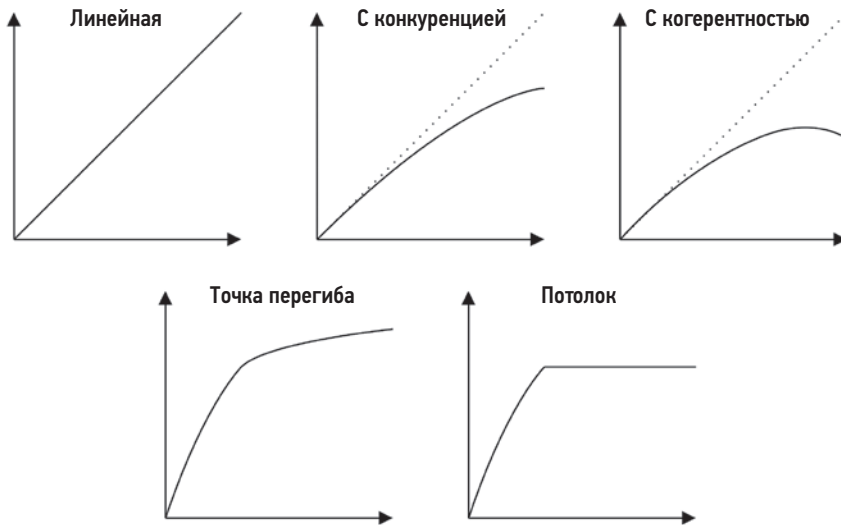


Рис. 2.16. Профили масштабируемости

2.6.3. Закон Амдала

Названный в честь компьютерного архитектора Джина Амдала (Gene Amdahl) [Amdahl 67], этот закон моделирует масштабируемость системы с учетом компонентов рабочих нагрузок, имеющих последовательный характер, которые нельзя обрабатывать параллельно. Его можно использовать для изучения масштабирования процессоров, потоков выполнения, рабочих нагрузок и т. д.

Закон Амдала можно наблюдать в профиле масштабируемости «с конкуренцией», показанном выше, который иллюстрирует конкуренцию за последовательный ресурс или компонент рабочей нагрузки. Его можно определить как [Gunther 97]:

$$C(N) = N / (1 + \alpha(N - 1)),$$

где $C(N)$ — относительная емкость, а N — размерность масштабирования, например количество процессоров или пользовательская нагрузка. Параметр α ($0 \leq \alpha \leq 1$) представляет степень последовательности, то есть степень отклонения от линейной масштабируемости.

Закон Амдала можно применить, выполнив следующие действия:

1. Собрать данные для диапазона N , наблюдая за действующей системой, либо экспериментально с помощью микробенчмаркинга или генераторов нагрузки.
2. Выполнить регрессионный анализ для определения параметра Амдала (α); это можно сделать с помощью ПО для статистических вычислений, например gnuplot или R.

3. Представить результаты для анализа. Собранные данные можно нанести на график вместе с функцией прогнозной модели масштабирования и выявить различия между данными и моделью. Это также можно сделать с помощью `gnuplot` или `R`.

Ниже приводится пример кода для `gnuplot`, реализующий регрессионный анализ закона Амдала, который дает представление о том, как выполнить этот шаг:

```
inputN = 10                # количество входных точек данных для модели
alpha = 0.1                # начальная точка
amdahl(N) = N1 * N / (1 + alpha * (N - 1))
# регрессионный анализ (нелинейная аппроксимация методом наименьших квадратов)
fit amdahl(x) filename every ::1::inputN using 1:2 via alpha
```

Примерно такой же объем кода требуется для обработки в `R`, включая функцию `nls()` нелинейной аппроксимации методом наименьших квадратов для вычисления коэффициентов, которые затем используются при построении графика. Полный код для `gnuplot` и `R` вы найдете в репозитории с набором моделей масштабируемости производительности, ссылка на который есть в конце главы [Gregg 14a].

Пример функции закона Амдала показан в следующем разделе.

2.6.4. Универсальный закон масштабируемости

Универсальный закон масштабируемости (universal scalability law, USL), ранее называвшийся *суперпоследовательной моделью* (super-serial model) [Gunther 97], был разработан доктором Нилом Гюнтером (Dr. Neil Gunther) и включает параметр задержки согласования. График этого закона был показан выше на рис. 2.16 как профиль масштабируемости с согласованием, который включает также эффекты конкуренции.

Универсальный закон масштабируемости можно определить как

$$C(N) = N / (1 + \alpha(N - 1) + \beta N(N - 1)),$$

где $C(N)$, N и α имеют тот же смысл, что и в законе масштабируемости Амдала. β — это параметр согласования. Когда $\beta = 0$, универсальный закон масштабируемости сводится к закону масштабируемости Амдала.

Примеры анализа с использованием универсального закона масштабируемости и закона масштабируемости Амдала показаны на рис. 2.17.

Во входных данных наблюдается высокая дисперсия, что затрудняет визуальное определение профиля масштабируемости. Первые десять точек данных, изображенные кружками, были переданы в модели. На график также нанесены дополнительные десять точек данных (крестики), которые помогают сравнить прогноз модели с реальностью.

Подробнее об анализе с применением универсального закона масштабируемости рассказывается в [Gunther 97] и [Gunther 07].

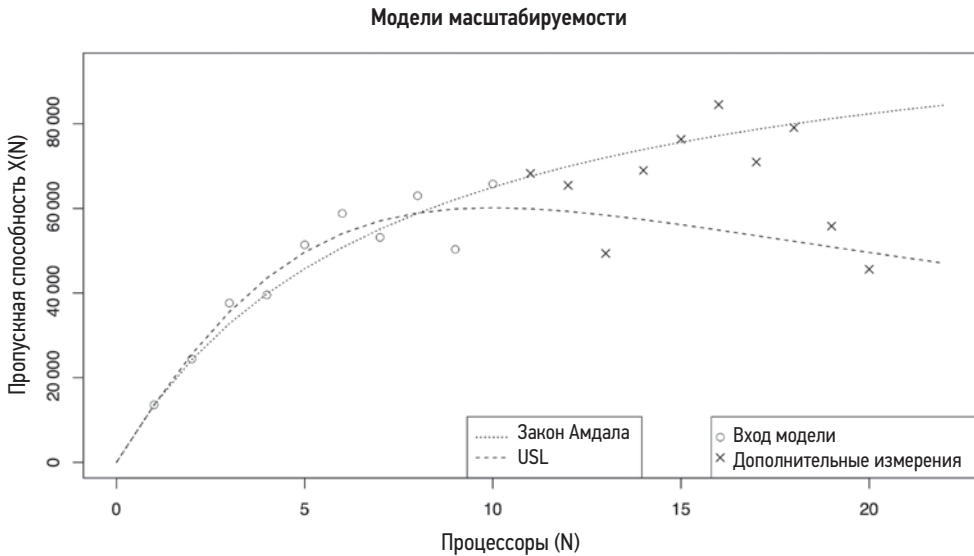


Рис. 2.17. Модели масштабируемости

2.6.5. Теория массового обслуживания

Теория массового обслуживания — это раздел теории вероятностей, исследующий системы с очередями и предлагающий методы анализа длины очереди, времени ожидания (задержки) и загрузки (на основе времени). Многие вычислительные компоненты, как программные, так и аппаратные, можно смоделировать как *системы массового обслуживания* (системы очередей). Системы с множеством очередей называются *сетями массового обслуживания* (сетями очередей).

В этом разделе кратко описывается роль теории массового обслуживания и приводится пример, который поможет понять эту роль. Это весьма обширная область исследований, и более подробное ее описание вы найдете в других книгах, например [Jain 91] и [Gunther 97].

Теория массового обслуживания основывается на различных областях математики и статистики, включая распределения вероятностей, случайные процессы, формулу Эрланга-С (Агнер Крауп Эрланг — автор теории массового обслуживания) и закон Литтла (Little). Закон Литтла можно выразить так:

$$L = \lambda W.$$

Он определяет среднее количество запросов в системе (L) как среднюю скорость поступления (λ), умноженную на среднее время обслуживания запроса (W). Применительно к очередям: L — это количество запросов в очереди, а W — среднее время ожидания в очереди.

Теорию массового обслуживания можно использовать для получения ответов на различные вопросы, в том числе:

- Как увеличится среднее время отклика при двойном увеличении нагрузки?
- Как повлияет добавление дополнительного процессора на среднее время отклика?
- Сможет ли система обеспечить время отклика меньше 100 мс в 90 % случаев при удвоении нагрузки?

Помимо времени отклика, можно изучить другие факторы, в том числе нагрузку, длину очереди и количество заданий в очереди.

На рис. 2.18 показана простая модель системы массового обслуживания.

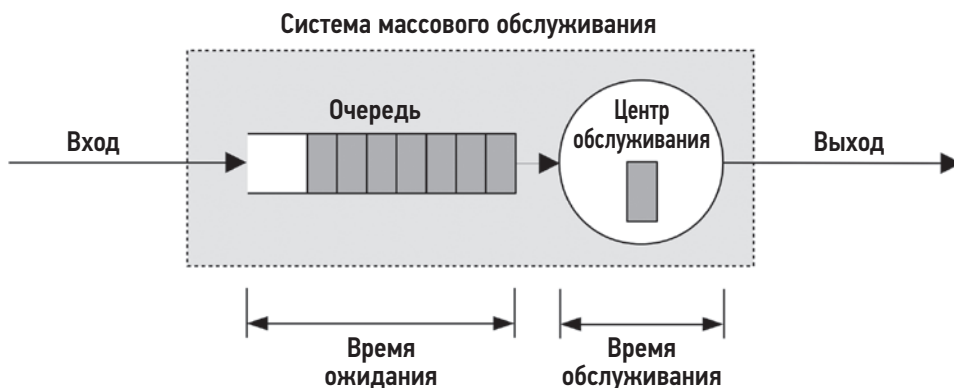


Рис. 2.18. Простая модель системы массового обслуживания

Здесь имеется единый центр обслуживания, обрабатывающий задания из очереди. Системы массового обслуживания могут иметь несколько таких центров, действующих параллельно. В теории массового обслуживания центры обслуживания часто называют *серверами*.

Системы массового обслуживания можно классифицировать по трем факторам:

- **Процесс поступления:** определяет время между поступлениями запросов, которое может быть случайным, фиксированным или описываться такими процессами, как, например, пуассоновский (в котором времена между получениями запросов подчиняются экспоненциальному распределению).
- **Распределение времени обслуживания:** описывает время обработки в центре обслуживания. Время может быть фиксированным (детерминированным) или подчиняться некоторому другому закону распределения, например экспоненциальному.
- **Количество центров обслуживания:** один или несколько.

Эти факторы можно записать в нотации Кендалла (Kendall).

Нотация Кендалла

В этой нотации для обозначения каждого атрибута используется свой код. Она выглядит так:

$$A/S/m.$$

Здесь A — это процесс поступления; S — распределение времени обслуживания; m — количество центров обслуживания. Есть также расширенная нотация Кендалла, включающая дополнительные факторы: количество буферов в системе, размер совокупности и дисциплину обслуживания.

Вот примеры некоторых типичных систем массового обслуживания:

- **M/M/1**: марковский процесс поступления (время между поступлениями запросов имеет экспоненциальное распределение), марковское время обслуживания (время обслуживания запросов имеет экспоненциальное распределение), один центр обслуживания.
- **M/M/c**: то же, что и M/M/1, но с несколькими серверами.
- **M/G/1**: марковский процесс поступления, обобщенное (любое) распределение времени обслуживания, один центр обслуживания.
- **M/D/1**: марковский процесс поступления, детерминированное (фиксированное) время обслуживания, один центр обслуживания.

Модель M/G/1 обычно применяется для изучения производительности вращающихся жестких дисков.

M/D/1 и 60 %-ное потребление

В качестве простого примера применения теории массового обслуживания рассмотрим диск, который (примем для простоты) детерминированно реагирует на рабочую нагрузку. Это определение соответствует модели M/D/1.

Попробуем ответить на вопрос: как изменится время отклика диска при увеличении потребления?

Теория массового обслуживания позволяет рассчитать время отклика для M/D/1 как

$$r = s(2 - \rho)/2(1 - \rho),$$

где время отклика r определяется в терминах времени обслуживания s и уровня потребления ρ .

На рис. 2.19 показано, как меняется время отклика при времени обслуживания 1 мс с увеличением потребления от 0 до 100 %.

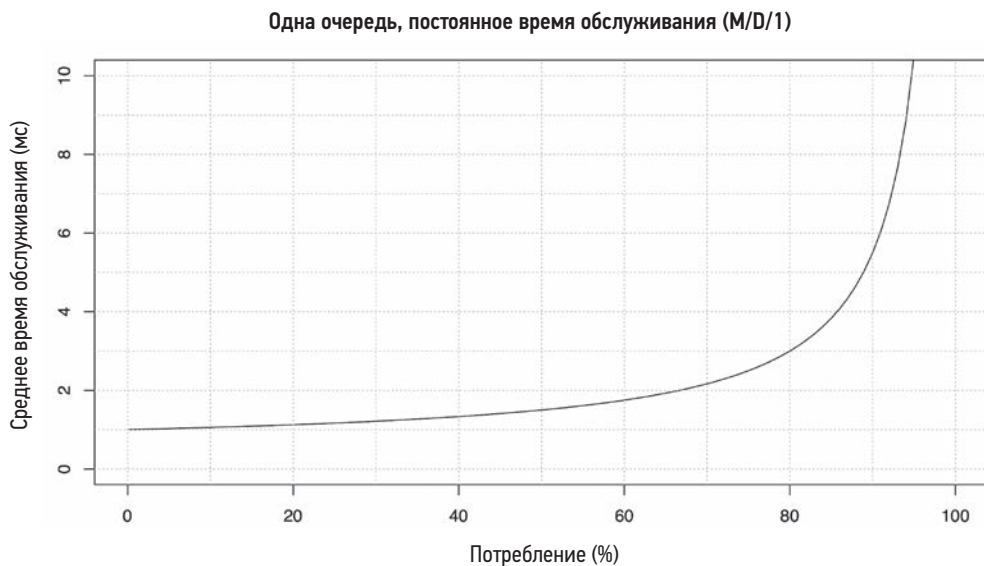


Рис. 2.19. Зависимость среднего времени отклика от потребления в модели M/D/1

При увеличении потребления чуть выше 60 % среднее время отклика удваивается. При увеличении потребления до 80 % оно выросло втрое. Поскольку задержка дискового ввода/вывода часто является ограничивающим ресурсом, увеличение средней задержки в два или более раза может оказывать существенное негативное влияние на производительность приложения. Вот почему увеличение потребления диска может стать проблемой задолго до того, как она достигнет 100 %: в системе массового обслуживания, где (обычно) нет возможности прервать обработку запросов, каждый запрос должен ждать своей очереди. В случае с процессорами, например, где задания с более высоким приоритетом могут вытеснять низкоприоритетные задания, ситуация обстоит иначе.

Этот график позволяет сразу ответить на поставленный ранее вопрос: «Как изменится время отклика диска при удвоении потребления?»

Эта модель проста и в некотором смысле показывает самый оптимистичный результат. Изменения во времени обслуживания могут привести к увеличению среднего времени отклика (например, в модели M/G/1 или M/M/1). Есть также распределение времени отклика, не показанное на рис. 2.19, в котором 90-й и 99-й процентиля при потреблении выше 60 % деградируют намного быстрее.

Как и в предыдущем примере использования gnuplot для визуализации закона Амдала, может быть полезно показать некоторый код, который поможет понять суть происходящего. Но на этот раз я приведу код для R [R Project 20]:

```
svc_ms <- 1           # среднее время обслуживания дискового ввода/вывода, мс
util_min <- 0          # диапазоны для построения графика
util_max <- 100        # "
```



```

ms_min <- 0                # "
ms_max <- 10               # "
# Построить график зависимости среднего времени отклика от потребления (M/D/1)
plot(x <- c(util_min:util_max), svc_ms * (2 - x/100) / (2 * (1 - x/100)),
     type="l", lty=1, lwd=1,
     xlim=c(util_min, util_max), ylim=c(ms_min, ms_max),
     xlab="Utilization %", ylab="Mean Response Time (ms)")

```

Здесь в функцию `plot()` передается предыдущее уравнение $M/D/1$. Большая часть этого кода определяет границы для построения графика, свойства линий и метки осей.

2.7. ПЛАНИРОВАНИЕ ЕМКОСТИ

При планировании емкости изучают, насколько хорошо система будет справляться с нагрузкой и как она будет масштабироваться с увеличением нагрузки. Такой анализ можно выполнить несколькими способами, включая изучение пределов ресурсов и факторный анализ, которые описаны ниже, а также моделирование, как было показано выше. В этом разделе будут представлены решения для масштабирования, включая балансировку нагрузки и сегментирование. Дополнительную информацию ищите в книге «The Art of Capacity Planning» [Allspaw 08].

Для планирования емкости конкретного приложения полезно иметь количественную оценку целевой производительности. Как ее определить, рассказывается в начале главы 5 «Приложения».

2.7.1. Пределы ресурсов

Этот метод заключается в выявлении ресурса, который станет узким местом после увеличения нагрузки. В случае с контейнерами ограничение на ресурс, который станет узким местом, может быть наложено программно. Вот шаги, реализующие этот метод:

1. Измерить частоту запросов к серверу и проследить, как она меняется с течением времени.
2. Измерить потребление аппаратных и программных ресурсов и проследить, как оно меняется с течением времени.
3. Выразить запросы к серверу в терминах используемых ресурсов.
4. Экстраполировать потребление ресурсов запросами к серверу с известными (или экспериментально установленными) ограничениями для каждого ресурса.

Для начала следует определить роль сервера и тип запросов, которые он обслуживает. Например, веб-сервер обслуживает запросы HTTP, сервер сетевой файловой системы (network file system, NFS) обслуживает запросы (операции) протокола NFS, а сервер базы данных обслуживает запросы к данным (или команды, подмножеством которых являются запросы к данным).

Следующий шаг — определение потребления ресурсов системы каждым запросом. В действующей системе можно измерить текущую частоту запросов вместе с потреблением ресурсов. После этого полученные данные можно экстраполировать, чтобы увидеть, потребление какого ресурса первым достигнет 100 % и какой при этом будет частота запросов.

Для анализа будущей системы можно использовать инструменты микробенчмаркинга или генераторы нагрузки, чтобы смоделировать предполагаемые запросы в тестовой среде и измерить потребление ресурсов. При достаточной клиентской нагрузке пределы можно определить экспериментальным путем.

Ресурсы для мониторинга:

- **Аппаратное обеспечение:** потребление процессора, потребление памяти, частота дисковых операций ввода/вывода в секунду, пропускная способность диска, емкость диска (используемого тома), пропускная способность сети.
- **Программное обеспечение:** потребление виртуальной памяти, процессы/задачи/потoki, дескрипторы файлов.

Допустим, вы изучаете действующую систему, которая обрабатывает 1000 запросов в секунду. Самыми загруженными ресурсами в этой системе являются 16 процессоров, среднее потребление которых составляет 40 %. Вы прогнозируете, что они станут узким местом, когда возникнет рабочая нагрузка, потребляющая 100 % процессорного времени. Нужно ответить на вопрос: при какой частоте запросов наступит этот момент?

$$\text{CPU\% на запрос} = \text{общее потребление CPU\%} / \text{количество запросов} = 16 \times 40\% / 1000 = 0,64\%.$$

$$\text{Максимальное число запросов в секунду} = 100\% \times 16 \text{ CPU} / \text{CPU\% на запрос} = 1600 / 0,64 = 2500.$$

Прогноз — 2500 запросов в секунду, после чего потребление процессоров достигнет 100 %. Это приблизительная оценка мощности в лучшем случае, потому что может возникнуть другой ограничивающий фактор, прежде чем частота запросов достигнет этой величины.

В этом упражнении использовалась только одна точка данных: пропускная способность приложения (1000 запросов в секунду) и потребление ресурса 40 %. Если есть возможность провести мониторинг в течение длительного времени, то в вычисления можно включить несколько точек данных с разной пропускной способностью и коэффициентами использования, чтобы повысить точность оценки. На рис. 2.20 показан визуальный метод их обработки и экстраполяции максимальной пропускной способности приложения.

Как вы думаете, 2500 запросов в секунду — это много или мало? Чтобы ответить на этот вопрос, нужно знать величину пиковой рабочей нагрузки, которая наблюдается в течение суток. Для действующей системы, за работой которой вы наблюдаете

в течение долгого времени, почти всегда можно довольно точно сказать, как будет выглядеть пик.

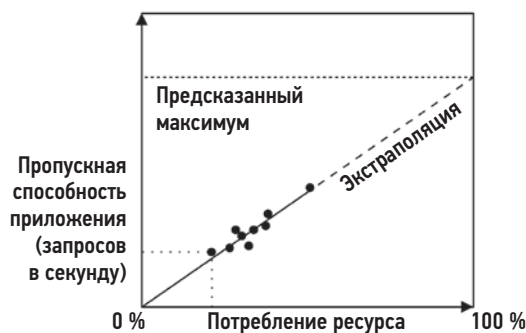


Рис. 2.20. Анализ пределов ресурсов

Рассмотрим сервер, обрабатывающий 100 000 посещений сайта в день. Может показаться, что число посещений огромно, но в среднем это чуть больше одного запроса в секунду — не много. Но может быть так, что большинство из 100 000 посещений произойдет в считанные секунды после публикации нового контента, поэтому пик окажется весьма значительным.

2.7.2. Факторный анализ

При покупке и развертывании новых систем часто есть возможность многое поменять, чтобы добиться желаемой производительности. Сюда можно отнести изменение количества дисков и процессоров, объема ОЗУ, использование SSD-накопителей, организацию дисков в массивы RAID, настройку файловой системы и т. д. Задача обычно состоит в достижении требуемой производительности при минимальных затратах.

Тестирование всех комбинаций позволит определить, какая из них имеет лучшее соотношение цена/качество; однако с ростом количества факторов объем тестирования увеличивается в геометрической прогрессии: чтобы проверить все комбинации из восьми бинарных факторов, потребуется провести 256 тестов.

Одно из решений этой проблемы состоит в том, чтобы протестировать ограниченный набор комбинаций. Вот подход, основанный на знании максимальной конфигурации системы:

1. Проверить производительность со всеми факторами, настроенными на максимум.
2. Поочередно изменять факторы, тестируя производительность (она должна снижаться в каждом случае).
3. Соотнести процентное снижение производительности, обусловленное каждым фактором, с экономией средств.

4. Начать с максимальной производительности (и стоимости), выбрать факторы, которые экономят затраты и при этом обеспечивают обработку требуемого количества запросов в секунду, если судить по совокупному падению производительности.
5. Повторно протестировать расчетную конфигурацию, чтобы подтвердить производительность.

Для системы с восемью факторами этот подход позволит ограничиться всего десятью тестами.

Рассмотрим планирование емкости новой системы хранения, которая должна иметь пропускную способность для чтения 1 Гбайт/с и рабочий объем памяти 200 Гбайт. В максимальной конфигурации такая система обеспечивает пропускную способность 2 Гбайт/с, оснащена четырьмя процессорами, 256 Гбайт DRAM и двумя двухпортовыми сетевыми картами 10 GbE, обеспечивающими передачу больших пакетов, при этом без сжатия и шифрования (которые ухудшают производительность). Производительность конфигурации с двумя процессорами снижается на 30 %, с одной сетевой картой — на 25 %, с обычной сетевой картой — на 35 %, с включенным шифрованием — на 10 %, с включенным сжатием — на 40 % и с меньшим объемом DRAM — на 90 %, потому что рабочая нагрузка будет кэшироваться не полностью. Учитывая эти значения падения производительности и соответствующую им экономию, можно рассчитать лучшую систему по соотношению цена/производительность, отвечающую требованиям; такой пропускной способностью должна обладать двухпроцессорная система с одной сетевой картой: $2 \times (1 - 0,30) \times (1 - 0,25) = 1,04$ Гбайт/с. Результаты расчетов было бы разумно подтвердить тестами на тот случай, если все вместе эти компоненты работают не так, как ожидалось.

2.7.3. Решения масштабирования

Удовлетворение более высоких требований к производительности часто означает использование более крупных систем. Эта стратегия называется *вертикальным масштабированием*. Распределение нагрузки между многочисленными системами, обычно с помощью так называемых *балансирующих нагрузки*, которые создают видимость единого целого, называется *горизонтальным масштабированием*.

Облачные вычисления расширяют возможности горизонтального масштабирования за счет создания небольших виртуализированных систем. Это обеспечивает при покупке более точный подбор вычислительных ресурсов для обработки предполагаемой нагрузки и позволяет масштабировать производительность небольшими приращениями. Поскольку при использовании облачных вычислений не требуется больших первоначальных вложений, как при покупке корпоративных мейнфреймов (включая плату за поддержку), то нет необходимости тщательно планировать мощность на ранних этапах проекта.

В облачных средах есть технологии автоматического масштабирования на основе оценки производительности. В AWS, например, такая технология называется *группой*

автоматического масштабирования (auto scaling group, ASG). Эта технология позволяет определить правило масштабирования для увеличения и уменьшения количества экземпляров на основе метрики потребления, как показано на рис. 2.21.

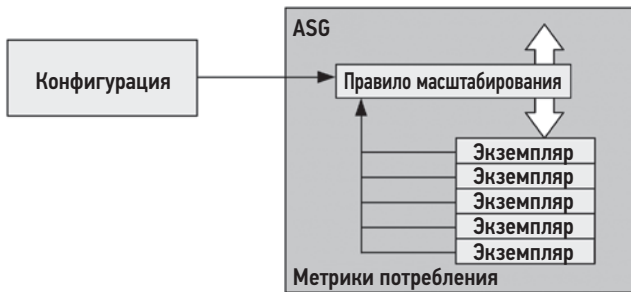


Рис. 2.21. Группа автоматического масштабирования

Группы автоматического масштабирования, используемые в Netflix, обычно нацелены на 60 %-ный уровень потребления процессора и запускают или останавливают экземпляры, стараясь удерживать потребление ниже этой цели.

Системы оркестрации контейнеров тоже могут поддерживать автоматическое масштабирование. Например, в Kubernetes есть автоматический механизм горизонтального масштабирования подов (horizontal pod autoscaler, HPA), который может масштабировать количество подов (контейнеров) в зависимости от уровня потребления процессора или другой настраиваемой метрики [Kubernetes 20a].

В базах данных распространена стратегия масштабирования, основанная на *шардировании* (sharding), когда данные разделяются на логические компоненты, каждый из которых управляется отдельной базой данных (или избыточной группой баз данных). Например, база данных клиентов может быть разделена по именам этих клиентов на алфавитные сегменты. Для равномерного распределения нагрузки между базами данных в этом случае важен выбор эффективного ключа сегментирования.

2.8. СТАТИСТИКИ

Важно хорошо понимать, как использовать статистики и их ограничения. В этом разделе мы рассмотрим количественную оценку проблем производительности с использованием статистик (метрик) и статистических типов, включая средние значения, стандартные отклонения и процентиля.

2.8.1. Количественная оценка прироста производительности

Количественная оценка проблем и возможного прироста производительности после их устранения позволяет сравнивать эти проблемы и определять очередность их устранения. Эту задачу можно решить с помощью наблюдения или экспериментов.

С помощью наблюдений

Чтобы количественно оценить проблему производительности с помощью наблюдений:

1. Выберите надежную метрику.
2. Оцените выигрыш в производительности от решения проблемы.

Например:

- Наблюдение: приложение обрабатывает запрос в течение 10 мс.
- Наблюдение: из них 9 мс расходуется на дисковый ввод/вывод.
- Предложение: настроить в приложении кэширование операций ввода/вывода в DRAM с ожидаемой задержкой около 10 мкс.
- Расчетное улучшение: $10 \text{ мс} \rightarrow 1,01 \text{ мс} (10 \text{ мс} - 9 \text{ мс} + 10 \text{ мкс}) = 9 \text{ раз}$.

Как отмечалось в разделе 2.3 «Основные понятия», на роль метрики хорошо подходит задержка (время), потому что позволяет напрямую сравнивать компоненты и делает такие вычисления возможными.

Используя задержки, убедитесь, что они измеряются как синхронный компонент запроса приложения. Некоторые события, например фоновый дисковый ввод/вывод (фактическая запись данных на диск), происходят асинхронно и не влияют напрямую на производительность приложения.

С помощью экспериментов

Чтобы количественно оценить проблему производительности с помощью экспериментов:

1. Примените исправление.
2. Определите количественную оценку выбранной метрики до и после исправления.

Например:

- Наблюдение: средняя задержка выполнения транзакции в приложении составляет 10 мс.
- Эксперимент: увеличить количество потоков выполнения в приложении, чтобы одновременно обрабатывать больше запросов.
- Наблюдение: средняя задержка выполнения транзакции в приложении составляет 2 мс.
- Улучшение: $10 \text{ мс} \rightarrow 2 \text{ мс} = 5 \text{ раз}$.

Этот подход может оказаться неприемлемым, если применение исправления в промышленной среде требует больших затрат. В таких ситуациях используйте метод на основе наблюдений.

2.8.2. Усреднение

Усреднение позволяет представить набор данных в виде единственного значения, отражающего преимущественную тенденцию. Наиболее распространенный тип среднего — *среднее арифметическое* (или просто *среднее*), которое определяется как сумма значений, деленная на их количество. В числе других типов можно назвать среднее геометрическое и среднее гармоническое.

Среднее геометрическое

Среднее геометрическое — это корень n -й степени (где n — количество значений) из произведения всех значений. Подробнее о среднем геометрическом рассказывается в [Jain 91], где также приводится пример использования этого метода для анализа производительности сети: если увеличение производительности каждого слоя сетевого стека в ядре измерять индивидуально, то как получить среднее увеличение производительности? Поскольку все слои вместе обрабатывают одни и те же пакеты, улучшение производительности имеет «мультипликативный» эффект, который лучше всего описывается с помощью среднего геометрического.

Среднее гармоническое

Среднее гармоническое — это отношение количества значений к сумме их обратных величин и больше подходит для усреднения скоростей, например для расчета средней скорости передачи 800 Мбайт данных, когда первые 100 Мбайт передаются со скоростью 50 Мбайт/с, а оставшиеся 700 Мбайт — с пониженной скоростью 10 Мбайт/с. В данном случае среднее гармоническое будет равно: $800 / (100/50 + 700/10) = 11,1$ Мбайт/с.

Усреднение по времени

В отношении производительности многие изучаемые нами метрики являются значениями, усредненными за определенный период времени. Процессор не всегда нагружен «на 50 %». Использование составило 50 % в течение некоторого интервала, который может быть секундой, минутой или часом. Исследуя средние значения, важно учитывать интервалы.

Однажды я исследовал проблему падения производительности у клиента, вызванную насыщением процессора (была задержка в работе планировщика), при этом инструменты мониторинга показывали, что потребление процессора никогда не превышало 80 %. Инструмент мониторинга сообщал пятиминутные средние значения, скрывавшие периоды, длившиеся по несколько секунд, когда потребление достигало 100 %.

Затухающее среднее

При оценке производительности систем иногда используется *затухающее среднее* (decayed average). Примером могут служить «средние значения нагрузки», сообщаемые различными инструментами, включая `uptime(1)`.

Затухающее среднее все так же измеряется в течение определенного промежутка времени, но недавний интервал имеет больший вес, чем более отдаленный в прошлое. Это уменьшает (гасит) кратковременные колебания среднего значения.

Дополнительную информацию ищите в подразделе «Средние значения нагрузки» в главе 6 «Процессоры» в разделе 6.6 «Инструменты наблюдения».

Ограничения

Среднее — это сводная статистика, скрывающая детали. Я неоднократно наблюдал случаи, когда задержка дискового ввода/вывода периодически превышала 100 мс, но при этом средняя задержка была близка к 1 мс. Для лучшего понимания ситуации можно использовать дополнительную статистику, описанную ниже в разделе 2.8.3 «Стандартное отклонение, процентиля, медиана», и приемы визуализации, описанные в разделе 2.10 «Визуализация».

2.8.3. Стандартное отклонение, процентиля, медиана

Стандартные отклонения и процентиля (например, 99-й процентиль) — это статистические метрики, позволяющие получить представление о *распределении* данных. Стандартное отклонение — это мера *дисперсии*: чем больше значение стандартного отклонения, тем больше отклоняются измерения от среднего (арифметического). 99-й процентиль показывает точку в распределении, которая включает 99 % значений. На рис. 2.22 показаны эти статистики для нормального распределения вместе с минимумом и максимумом.



Рис. 2.22. Статистические метрики

Такие процентиля, как 90-й, 95-й, 99-й и 99,9-й, используются при мониторинге производительности для количественной оценки случаев, когда запросы обрабатывались медленнее всего. Они также могут указываться в соглашениях об уровне

обслуживания (service level agreement, SLA) для представления производительности у большинства пользователей.

50-й процентиль, называемый *медианой*, можно использовать, чтобы показать, где сосредоточена основная часть данных.

2.8.4. Коэффициент вариации

Поскольку стандартное отклонение непосредственно связано со средним значением, рассматривать изменчивость данных можно только как сочетание стандартного отклонения и среднего. Значение 50 стандартного отклонения само по себе почти не несет информации. Но если добавить к нему среднее значение, равное 200, то эта комбинация может сказать о многом.

Существует другой способ выразить изменчивость в виде единственной метрики: как отношение стандартного отклонения к среднему. Эта метрика называется *коэффициентом вариации* (coefficient of variation, CoV или CV). В этом примере CV составляет 25 %. Более низкие значения CV соответствуют меньшей изменчивости.

Еще один способ выразить изменчивость в виде единственной метрики — значение z , которое показывает, на сколько стандартных отклонений отстоит данное значение от среднего.

2.8.5. Мультимодальные распределения

Метрики — среднее значение, стандартное отклонение и проценти́ли — имеют проблему, которая показана на графике (рис. 2.23): они характеризуют одномодальные распределения, близкие к *нормальному*. Производительность, напротив, часто имеет *двухмодальное* распределение. Такая двухмодальность может быть обусловлена, например, тем, что быстро выполняющиеся пути в коде дают низкие задержки, а медленно выполняющиеся — высокие; либо попадания в кэш дают низкие задержки, а промахи — высокие. Распределение также может иметь большее число мод.

На рис. 2.23 показано распределение задержек дискового ввода/вывода для смешанной рабочей нагрузки чтения и записи, которая выполняет произвольный и последовательный ввод/вывод.

Распределение представлено в виде гистограммы, у которой есть две моды. Мода слева соответствует задержкам менее 1 мс, которые обусловлены попаданием в дисковый кэш. Мода справа, с пиком в районе 7 мс, соответствует задержкам, которые обусловлены промахами дискового кэша, характерными для операций произвольного чтения. Среднее (арифметическое) задержек ввода/вывода составляет 3,3 мс и обозначено вертикальной линией. Как видите, это среднее значение не отражает преимущественную тенденцию (как говорилось выше). На самом деле все почти с точностью до наоборот. Среднее значение для этого распределения может ввести в глубокое заблуждение.

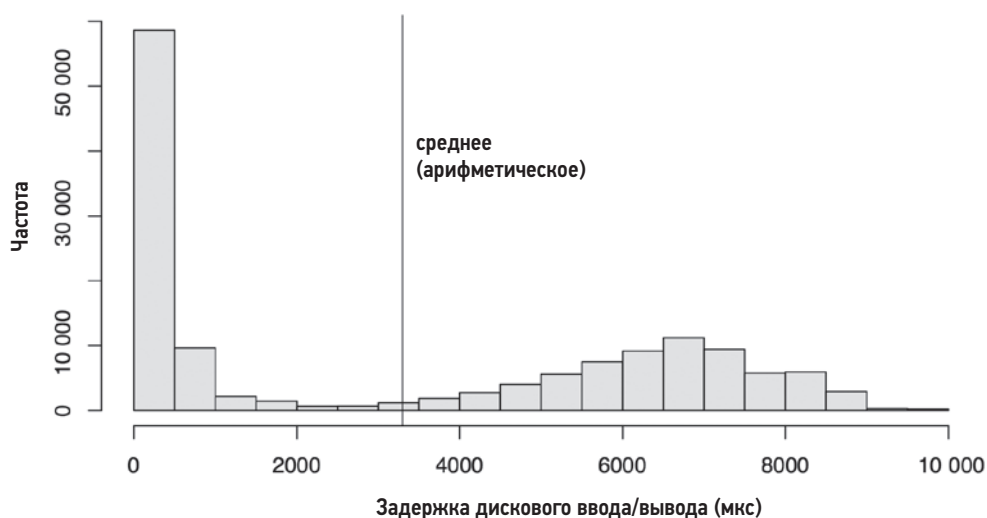


Рис. 2.23. Распределение задержек

А потом нашелся человек, утонувший при переходе через ручей со средней глубиной шесть дюймов.

У. И. Э. Геймс (W. I. E. Gates)

Каждый раз, когда вы видите среднее значение, используемое в качестве показателя производительности, особенно среднюю задержку, спрашивайте о характере распределения. В разделе 2.10 «Визуализация» приводится еще один пример и рассказывается, насколько эффективными могут быть различные средства визуализации и метрики для отображения таких распределений.

2.8.6. Выбросы

Другой статистической проблемой являются *выбросы*: небольшое количество чрезвычайно высоких или низких значений, которые не соответствуют ожидаемому распределению (одно- или мультимодальному).

Примером могут служить выбросы задержки дискового ввода/вывода — случайный дисковый ввод/вывод, выполняющийся более 1000 мс, когда продолжительность выполнения большей части аналогичных операций колеблется в диапазоне от 0 до 10 мс. Такие выбросы могут вызвать серьезные проблемы с производительностью, но их присутствие часто трудно идентифицировать по большинству известных метрик, за исключением максимального значения. Другой пример — выбросы задержки сетевого ввода/вывода, вызванные повторной передачей по таймеру TCP.

В нормальном распределении выбросы могут немного сдвинуть среднее, но не медианное значение (о чем полезно помнить). Стандартное отклонение и 99-й процентиль с большей вероятностью позволят выявить выбросы, но в этом случае многое зависит от их частоты.

В случае мультимодального распределения выбросы и другие сложные, но довольно типичные закономерности лучше определять визуализацией полного распределения, например, в виде гистограммы. Как это сделать, рассказывается в разделе 2.10 «Визуализация».

2.9. МОНИТОРИНГ

Мониторинг производительности заключается в записи информации о производительности с течением времени, чтобы прошлое можно было сравнить с настоящим и выявить закономерности во времени. Это пригодится для планирования емкости, количественной оценки роста и отображения пиковой нагрузки. Исторические значения также могут послужить основой для понимания текущих значений метрик производительности, показывая, какими были «нормальный» диапазон и среднее значение в прошлом.

2.9.1. Временные закономерности

На рис. 2.24, 2.25 и 2.26 показаны примеры временных закономерностей распределения операций чтения в файловой системе на облачном сервере в разные временные интервалы.



Рис. 2.24. Мониторинг активности: одни сутки



Рис. 2.25. Мониторинг активности: пять суток



Рис. 2.26. Мониторинг активности: 30 суток

На суточном графике видно, как количество операций чтения начинает нарастать в районе 8 часов утра, немного спадает после полудня, а затем постепенно уменьшается в течение ночи. На графиках, охватывающих более протяженные интервалы, видно, что в выходные дни активность ниже. На 30-дневном графике также видно два коротких всплеска.

В исторических данных часто можно увидеть различные циклы поведения, подобные показанным на рисунках, в том числе:

- **Часовые:** активность, обусловленная прикладным окружением, такая как задачи мониторинга и отчетности, может нарастать и спадать каждый час. Также нередко есть задачи, которые выполняются через пяти- или десятиминутные интервалы.
- **Суточные:** активность может иметь суточную закономерность, совпадая с рабочим временем (например, с 9 утра до 5 вечера), а если сервер обслуживает несколько часовых поясов, то эта закономерность может растягиваться во времени. Для серверов в интернете закономерность может следовать за активностью пользователей по всему миру. Еще один пример суточной закономерности — ночная ротация журналов и резервное копирование.
- **Недельные:** наряду с суточными закономерностями могут иметь место недельные закономерности, отражающие рабочие и выходные дни.
- **Квартальные:** финансовые отчеты составляются ежеквартально.
- **Годовые:** годовая закономерность нагрузки может быть связана с расписанием периодов обучения и каникул в школах.

Нерегулярное увеличение нагрузки может быть связано с другими видами активности — публикацией нового контента на сайте или распродажами (черная пятница и киберпонедельник в США). Нерегулярное снижение нагрузки может происходить из-за внешних причин, например частых перебоев в подаче электроэнергии или в интернет-соединении, а также из-за спортивных финалов (когда все смотрят игру, а не используют ваш продукт)¹.

¹ Работая в Netflix дежурным SRE-инженером, я познакомился с некоторыми нетрадиционными способами выявления подобных случаев: проверять соцсети на предмет объявлений об отключении электроэнергии и узнавать в чатах насчет значимых спортивных состязаний.

2.9.2. Инструменты мониторинга

Есть множество сторонних продуктов для мониторинга производительности. Обычно они поддерживают архивирование данных и представление их в виде интерактивных графиков в браузере, а также рассылку настраиваемых предупреждений.

Некоторые из них для сбора статистик используют *агентов* (также известных как *экспортеры*). Эти агенты либо запускают средства наблюдения ОС (например, `iostat(1)` или `sar(1)`) и анализируют их вывод (что считается неэффективным), либо читают данные напрямую, используя библиотеки операционной системы и интерфейсы ядра. Инструменты мониторинга поддерживают набор настраиваемых агентов для экспорта статистик из веб-серверов, баз данных и сред выполнения разных языков.

По мере распространения распределенных систем и облачных вычислений все чаще придется сталкиваться с необходимостью мониторинга многочисленных систем, насчитывающих сотни, тысячи и даже более экземпляров. Именно в таких ситуациях пригодится централизованное решение мониторинга, позволяющее контролировать всю среду из одного места.

Вот конкретный пример: облако Netflix состоит из более чем 200 000 экземпляров и контролируется с помощью инструмента мониторинга Atlas, охватывающего все облако и специально созданного в Netflix для работы в этом масштабе и имеющего открытый исходный код [Harrington 14]. Другие решения для мониторинга см. в главе 4 «Инструменты наблюдения» в разделе 4.2.4 «Мониторинг».

2.9.3. Сводная статистика, накопленная с момента загрузки

Если мониторинг не проводился, то проверьте доступность хотя бы *сводной статистики, накопленной с момента загрузки* (`summary-since-boot`), которую можно использовать для сравнения с текущими значениями.

2.10. ВИЗУАЛИЗАЦИЯ

Визуализация упрощает исследование больших объемов данных. Она также помогает распознавать и сопоставлять закономерности. Визуализация может оказаться более эффективным способом выявления корреляций между различными источниками метрик, которые трудно выявить программно, но легко — визуально.

2.10.1. Линейная диаграмма

Линейная диаграмма (или *линейный график*) — это хорошо известный прием визуализации. Обычно он используется для изучения характера изменения метрик производительности с течением времени, где время откладывается вдоль оси X.

На рис. 2.27 показан пример изменения средней (арифметической) задержки дискового ввода/вывода за 20-секундный период. Измерения производились на

промышленном облачном сервере с базой данных MySQL, где, как предполагалось, задержка дискового ввода/вывода являлась причиной медленной обработки запросов.

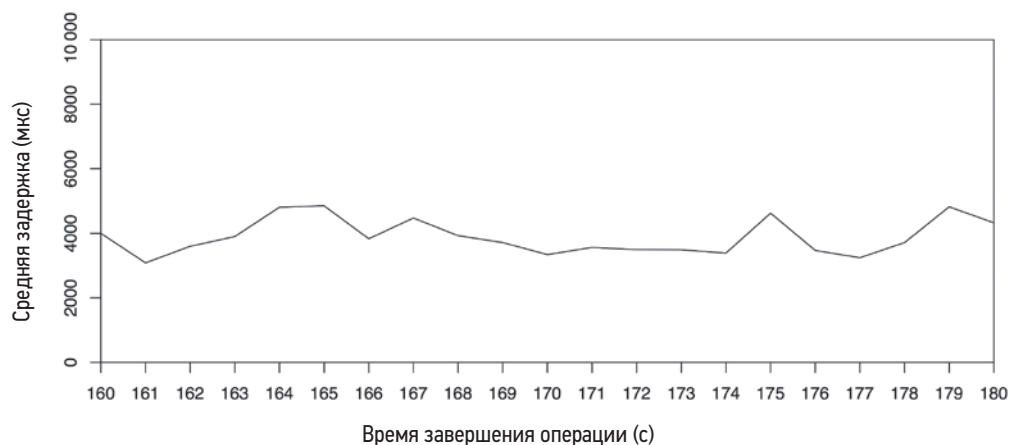


Рис. 2.27. Линейный график средней задержки

Этот график показывает довольно постоянную среднюю задержку чтения около 4 мс, что несколько больше, чем ожидалось для этих дисков.

Можно попробовать нарисовать несколько линий на одном графике и отобразить связанные данные, например, нарисовать отдельную линию для каждого диска, чтобы сравнить их производительности.

Также можно добавить линии, показывающие изменение некоторых статистик, чтобы получить больше информации о распределении данных. На рис. 2.28 показаны

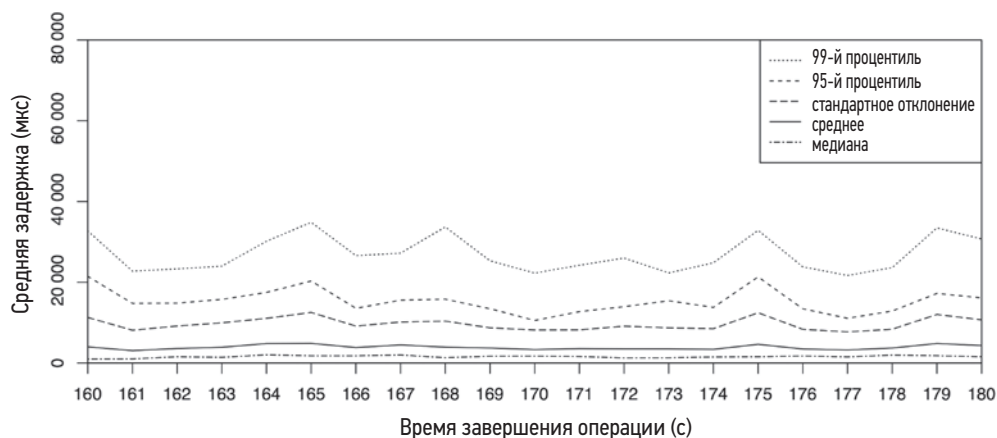


Рис. 2.28. Медиана, среднее, стандартное отклонение, процентиля

события дискового ввода/вывода для того же интервала времени с дополнительными линиями, отображающими посекундное изменение медианы, стандартного отклонения и процентов. Обратите внимание, что ось Y теперь охватывает гораздо больший диапазон, чем на предыдущем рисунке (в 8 раз).

Теперь ясно видно, почему среднее значение выше ожидаемого: в период проведения наблюдений имели место операции ввода/вывода с очень высокой задержкой. В частности, продолжительность 1 % операций превышала 20 мс, о чем свидетельствует 99-й процентиль. Медиана же соответствует ожидаемой задержке ввода/вывода в районе 1 мс.

2.10.2. Диаграммы рассеяния

На рис. 2.29 показаны события дискового ввода/вывода для того же интервала времени в виде диаграммы рассеяния, которая позволяет увидеть все данные. Каждая операция дискового ввода/вывода отображается в виде точки, время завершения которой откладывается по оси X, а задержка — по оси Y.

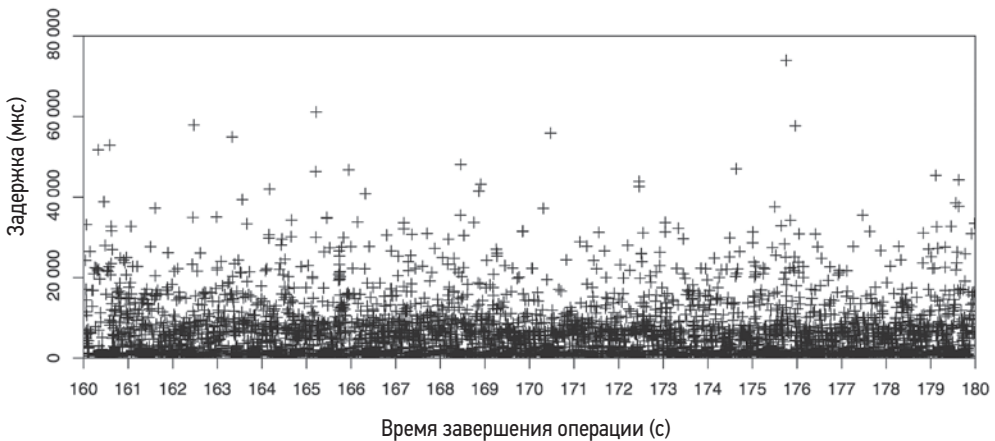


Рис. 2.29. Диаграмма рассеяния

Теперь полностью понятно, почему средняя задержка оказалась выше, чем ожидалось: довольно большое количество операций дискового ввода/вывода выполнялось с задержками 10 мс, 20 мс и даже более 50 мс. На диаграмме рассеяния показаны все данные, в том числе и выбросы.

Многие операции ввода/вывода длились около миллисекунды; они сосредоточены рядом с осью X. Именно в этой области сказывается проблема с разрешением диаграмм рассеяния: точки перекрывают друг друга, и их становится трудно различить. Чем больше данных, тем хуже ситуация: представьте диаграмму рассеяния с миллионами событий, собранных со всего облака: точки могут сливаться и образовывать полностью закрашенную область. Другая проблема — объем данных,

которые необходимо собрать и обработать: координаты X и Y для каждой операции ввода/вывода.

2.10.3. Тепловые карты

Тепловые карты (или, если точнее, *квантование по столбцам*) способны решить проблему недостаточной разрешающей способности диаграмм рассеяния путем квантования диапазонов X и Y в группы, называемые *сегментами*, или *корзинами*. Они отображаются в виде больших пикселей, окрашенных в зависимости от количества событий в этом диапазоне X и Y. Квантование также решает проблему отображения данных с большой плотностью размещения, что позволяет тепловым картам одинаково хорошо отображать данные из одной или из тысяч систем. Ранее тепловые карты использовались для отображения местоположений, например смещений на диске (например, TazTool [McDougall 06a]). Я предложил использовать их для отображения задержек и других метрик. Тепловые карты задержек впервые были включены в инструмент Analytics для устройства Sun Microsystems ZFS Storage, выпущенного в 2008 году [Gregg 10a], [Gregg 10b] и в настоящее время широко используются в решениях для мониторинга производительности, таких как Grafana [Grafana 20].

На рис. 2.30 показан тот же набор данных, что и выше, но уже в виде тепловой карты.

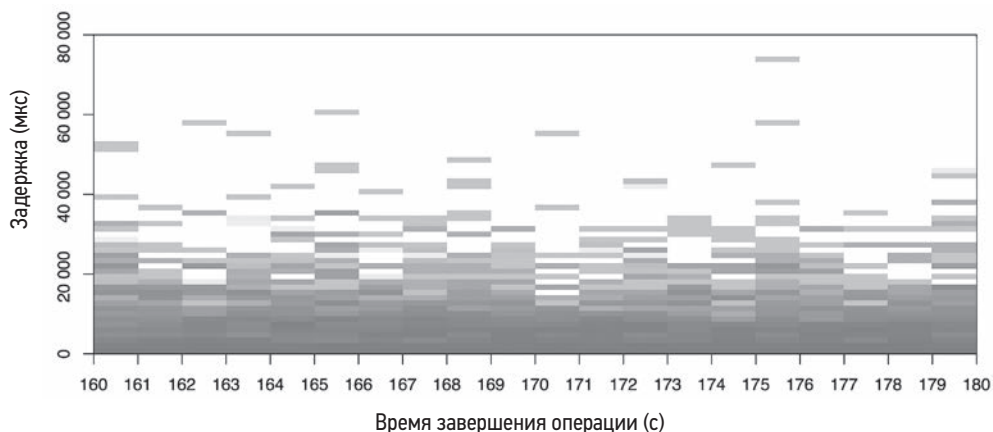


Рис. 2.30. Тепловая карта

Выбросы с высокой задержкой можно определить по блокам, расположенным в верхней части тепловой карты, обычно светлого цвета, потому что они охватывают небольшое количество операций ввода/вывода (часто одну). В основном объеме данных начинают проявляться закономерности, которые невозможно заметить на диаграмме рассеяния.

На рис. 2.31 показана тепловая карта, отражающая весь временной интервал наблюдений за дисковым вводом/выводом (не был показан ранее).

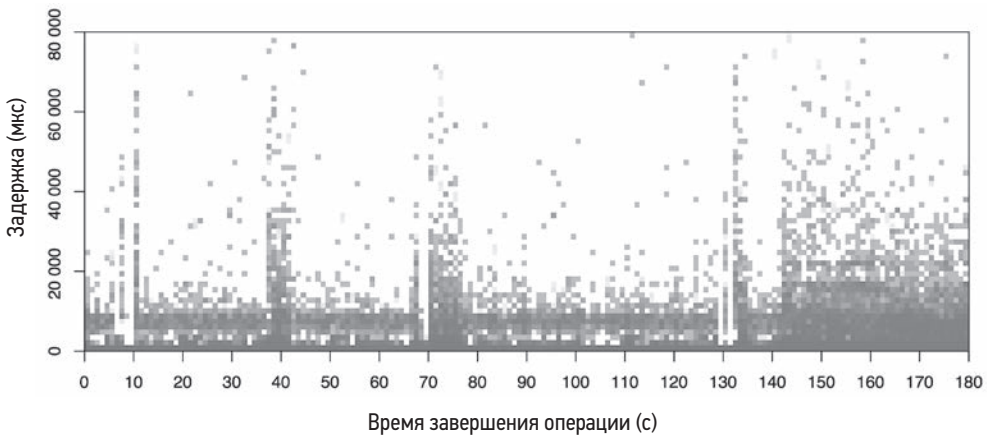


Рис. 2.31. Тепловая карта: весь временной интервал

Несмотря на охват диапазона, который в 9 раз больше, диаграмма по-прежнему хорошо читается. На большей части диапазона явно наблюдается бимодальный характер распределения задержек: одни операции ввода/вывода выполняются с задержкой, близкой к нулю (скорее всего, из-за попадания в дисковый кэш), а другие — немногим менее 1 мс (вероятно, из-за промаха дискового кэша).

В этой книге вы найдете и другие примеры тепловых карт, в том числе в главе 6 «Процессоры» в разделе 6.7 «Методы визуализации», в главе 8 «Файловые системы» в разделе 8.6.18 «Визуализация» и в главе 9 «Диски» в разделе 9.7.3 «Тепловые карты задержек». На моем сайте есть примеры тепловых карт задержек, потребления ресурсов и субсекундного смещения [Gregg 15b].

2.10.4. Временная шкала

На временной шкале набор действий отображается в виде полос. Обычно временные шкалы используются для анализа производительности на стороне внешнего интерфейса (в браузере), где они также называются *водопадными*, или *каскадными*, *диаграммами* и показывают продолжительность обработки сетевых запросов. На рис. 2.32 изображен пример из браузера Firefox.

На рис. 2.32 выделен первый сетевой запрос: кроме отображения его продолжительности в виде горизонтальной полосы, также показаны компоненты этой продолжительности разным цветом. В панели справа также описывается распределение времени между компонентами: большую часть времени в первом запросе занимает компонент «Waiting» (ожидание), то есть ожидание HTTP-ответа от сервера. Запросы со второго по шестой посылаются после того, как начинают поступать данные в ответ на первый запрос, и, вероятно, зависят от этих данных. Если в диаграмму включены явные стрелки зависимостей, она становится разновидностью диаграммы Ганта.

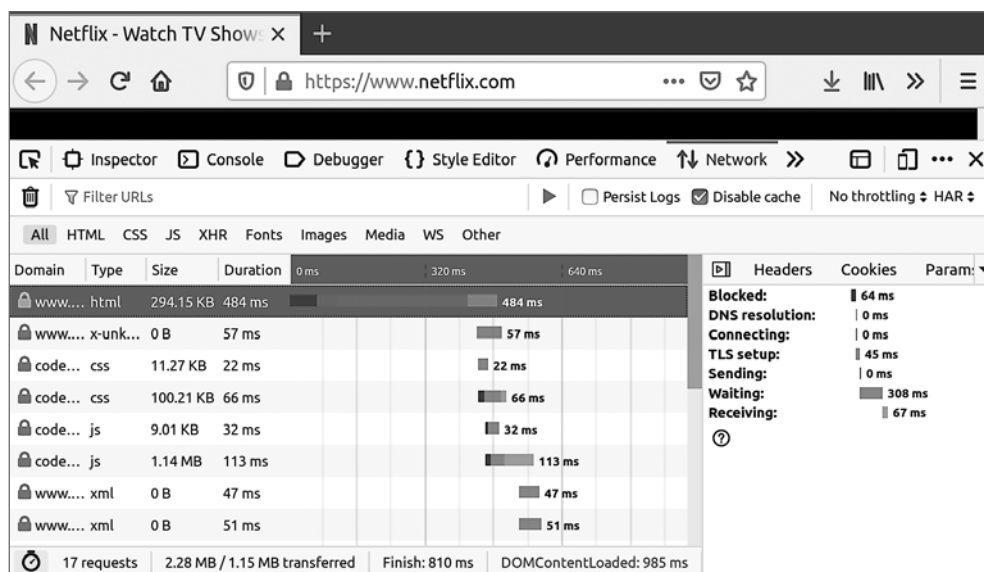


Рис. 2.32. Временная шкала в браузере Firefox

Для анализа производительности на стороне сервера используются аналогичные диаграммы, отображающие временные рамки потоков выполнения или процессов. В качестве примеров можно привести инструменты KernelShark [KernelShark 20] и Trace Compass [Eclipse 20]. Пример скриншота с изображением KernelShark вы найдете в главе 14 «Ftrace» в разделе 14.11.5 «KernelShark». Trace Compass тоже рисует стрелки, показывающие зависимости, когда один поток запускает другой.

2.10.5. График поверхности

Этот график представляет данные в виде трехмерной поверхности. Особенно хорошо он подходит для случаев, когда значение третьего измерения не часто, но резко меняется между точками, образуя поверхность, напоминающую холмы. Трехмерные графики поверхности часто изображаются в виде *каркасной модели*.

На рис. 2.33 показана такая каркасная поверхность потребления процессора. Он отображает значения, отбиравшиеся каждую секунду в течение 60 с со многих серверов (эта информация вырезана из диаграммы, но мониторинг выполнялся в вычислительном центре с более чем 300 физическими серверами и 5312 процессорами) [Gregg 11b].

Каждый сервер с его 16 процессорами представлен на диаграмме горизонтальными рядами точек шириной в 60 с (столбцы), при этом высота каждой точки соответствует уровню потребления процессора. Дополнительно уровень потребления отображается цветом. При желании можно использовать также оттенок и насыщенность цвета, чтобы добавить в график четвертое и пятое измерения. (При достаточном

разрешении можно даже использовать определенные узоры для обозначения шестого измерения.)

Прямоугольники 16×60 , соответствующие серверам, отображаются на поверхности в шахматном порядке. Даже без разметки на графике хорошо видны некоторые прямоугольники, соответствующие отдельным серверам. Например, прямоугольник справа, который выглядит как высокое плато, наглядно показывает, что процессоры соответствующего ему сервера почти все время работают с нагрузкой 100 %.

Использование линий сетки позволяет заметить даже незначительные изменения высоты. На том же плато видна одна линия, находящаяся намного ниже других, что говорит о постоянной низкой нагрузке на один из процессоров (несколько процентов).

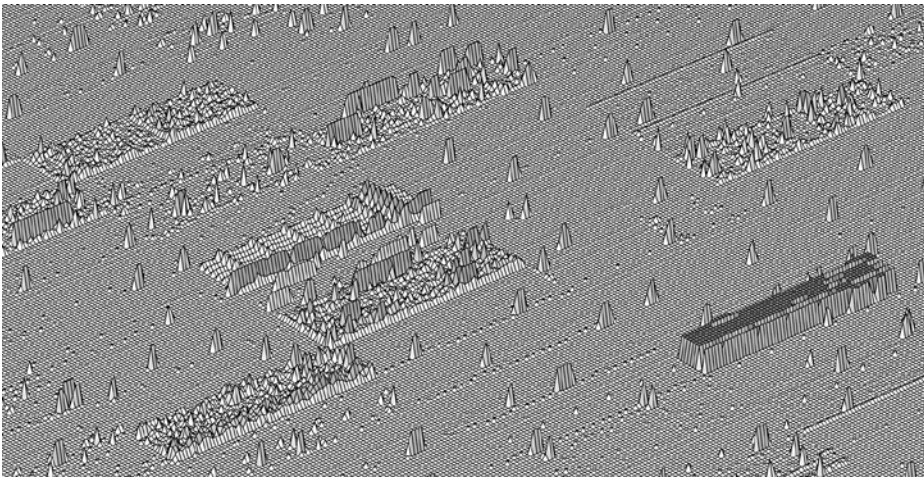


Рис. 2.33. Каркасная модель: потребление процессоров в вычислительном центре

2.10.6. Инструменты визуализации

Анализ производительности в Unix традиционно проводился с использованием текстовых инструментов, отчасти из-за ограниченной поддержки графического интерфейса. Такие инструменты можно быстро запускать и получать данные в режиме реального времени. Для получения визуального представления нужно больше времени, и часто требуется разделить процесс на циклы мониторинга и формирования отчетов. При работе с особенно острыми проблемами производительности скорость получения метрик может быть критичной.

Современные инструменты визуализации обеспечивают возможность наблюдения за производительностью системы в реальном времени с помощью браузера и мобильных устройств. Для этого есть множество продуктов, многие из которых могут мониторить все облако. В главе 1 «Введение», в разделе 1.7.1 «Счетчики, статистики и метрики» показан скриншот одного из таких продуктов — Grafana.

В главе 4 «Инструменты наблюдения» в разделе 4.2.4 «Мониторинг» мы познакомимся с другими продуктами мониторинга.

2.11. УПРАЖНЕНИЯ

1. Ответьте на следующие вопросы, касающиеся ключевых понятий:
 - Что такое IOPS?
 - Что такое потребление?
 - Что такое насыщенность?
 - Что такое задержка?
 - Что такое микробенчмаркинг производительности?
2. Выберите пять методологий для использования в вашей (или гипотетической) среде. Выберите порядок их применения и объясните причину выбора каждой из них.
3. Перечислите проблемы, связанные с использованием средней задержки в качестве единственной метрики производительности. Можно ли решить эти проблемы, добавив 99-й перцентиль?

2.12. ССЫЛКИ

- [Amdahl 67] Amdahl, G., «Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities», AFIPS, 1967.
- [Jain 91] Jain, R., «The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation and Modeling», Wiley, 1991.
- [Cockcroft 95] Cockcroft, A., «Sun Performance and Tuning», Prentice Hall, 1995.
- [Gunther 97] Gunther, N., «The Practical Performance Analyst», McGraw-Hill, 1997.
- [Wong 97] Wong, B., «Configuration and Capacity Planning for Solaris Servers», Prentice Hall, 1997.
- [Elling 00] Elling, R., «Static Performance Tuning», Sun Blueprints, 2000.
- [Millsap 03] Millsap, C., and J. Holt., «Optimizing Oracle Performance», O'Reilly, 2003¹.
- [McDougall 06a] McDougall, R., Mauro, J., and Gregg, B., «Solaris Performance and Tools: DTrace and MDB Techniques for Solaris 10 and OpenSolaris», Prentice Hall, 2006.
- [Gunther 07] Gunther, N., «Guerrilla Capacity Planning», Springer, 2007.
- [Allspaw 08] Allspaw, J., «The Art of Capacity Planning», O'Reilly, 2008².

¹ Миллсап К., Холт Дж. «Oracle. Оптимизация производительности».

² Оллспоу Дж. «Искусство планирования мощностей». СПб., издательство «Питер».

- [Gregg 10a] Gregg, B., «Visualizing System Latency», Communications of the ACM, July 2010.
- [Gregg 10b] Gregg, B., «Visualizations for Performance Analysis (and More)», USENIX LISA, <https://www.usenix.org/legacy/events/lisa10/tech/#gregg>, 2010.
- [Gregg 11b] Gregg, B., «Utilization Heat Maps», <http://www.brendangregg.com/HeatMaps/utilization.html>, 2011.
- [Williams 11] Williams, C., «The \$300m Cable That Will Save Traders Milliseconds», The Telegraph, <https://www.telegraph.co.uk/technology/news/8753784/The-300m-cable-that-will-save-traders-milliseconds.html>, 2011.
- [Gregg 13b] Gregg, B., «Thinking Methodically about Performance», Communications of the ACM, February 2013.
- [Gregg 14a] Gregg, B., «Performance Scalability Models», <https://github.com/brendangregg/PerfModels>, 2014.
- [Harrington 14] Harrington, B., and Rapoport, R., «Introducing Atlas: Netflix’s Primary Telemetry Platform», Netflix Technology Blog, <https://medium.com/netflix-techblog/introducing-atlas-netflixs-primary-telemetry-platform-bd31f4d8ed9a>, 2014.
- [Gregg 15b] Gregg, B., «Heatmaps», <http://www.brendangregg.com/heatmaps.html>, 2015.
- [Wilkie 18] Wilkie, T., «The RED Method: Patterns for Instrumentation & Monitoring», Grafana Labs, <https://www.slideshare.net/grafana/the-red-method-how-to-monitoring-your-microservices>, 2018.
- [Eclipse 20] Eclipse Foundation, «Trace Compass», <https://www.eclipse.org/tracecompass>, по состоянию на 2020.
- [Wikipedia 20] Wikipedia, «Five Whys», https://en.wikipedia.org/wiki/Five_whys, по состоянию на 2020¹.
- [Grafana 20] Grafana Labs, «Heatmap», <https://grafana.com/docs/grafana/latest/features/panels/heatmap>, по состоянию на 2020.
- [KernelShark 20] «KernelShark», <https://www.kernelshark.org>, по состоянию на 2020.
- [Kubernetes 20a] Kubernetes, «Horizontal Pod Autoscaler», <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale>, по состоянию на 2020.
- [R Project 20] R Project, «The R Project for Statistical Computing», <https://www.r-project.org>, по состоянию на 2020.

¹ Перевод статьи на русский язык: https://ru.wikipedia.org/wiki/Пять_почему. — *Примеч. пер.*

Глава 3

ОПЕРАЦИОННЫЕ СИСТЕМЫ

Для анализа производительности важно знать, как устроена операционная система и ее ядро. Вам часто придется разрабатывать, а затем проверять гипотезы о поведении системы, например о том, как выполняются системные вызовы, как ядро планирует потоки для выполнения на процессоре, как ограниченная емкость памяти может влиять на производительность или как файловая система обрабатывает ввод/вывод. Для этого вы должны знать устройство операционной системы и ядра.

Цели этой главы:

- познакомить с терминологией ядра: переключением контекста, подкачкой страниц, вытеснением и т. д.;
- пояснить роль ядра и системных вызовов;
- познакомить на практике с внутренним устройством ядра, в том числе с прерываниями, планировщиками, виртуальной памятью и стеком ввода/вывода;
- показать, какие средства оптимизации производительности ядра были добавлены из Unix в Linux;
- дать общее представление о механизме BPF.

В главе даются базовые сведения об операционных системах и ядрах, знание которых предполагается в остальной части книги. Если вы пропустили курс по операционным системам, то можете рассматривать эту главу как его ускоренную версию. Старайтесь ничего не упустить, потому что в конце вас ждет экзамен (шучу, всего лишь викторина). Подробную информацию о внутреннем устройстве ядра ищите в разделе 3.8 «Ссылки» в конце этой главы.

Глава делится на три раздела:

- в разделе **«Терминология»** перечислены основные термины;
- в разделе **«Основы»** кратко излагаются основные понятия операционной системы и ядра;
- в разделе **«Ядра»** обобщаются особенности реализации Linux и других ядер.

Более узкие области, связанные с производительностью, включая планирование процессорного времени, память, диски, файловые системы, сеть, а также специализированные инструменты анализа производительности, более подробно рассматриваются в последующих главах.

3.1. ТЕРМИНОЛОГИЯ

Ниже перечислены основные термины, связанные с операционной системой, которые используются в этой книге. Многие из этих понятий подробно рассматриваются в этой и в последующих главах.

- **Операционная система (ОС):** программное обеспечение и файлы, установленные в системе, обеспечивающие возможность загрузки и запуска программ. Операционная система включает ядро, инструменты администрирования и системные библиотеки.
- **Ядро:** программа, управляющая системой, включая (в зависимости от модели ядра) аппаратные устройства и память, и планирующая распределение процессорного времени. Ядро работает в привилегированном режиме процессора, обеспечивающем прямой доступ к оборудованию, который называется *режимом ядра*.
- **Процесс:** абстракция операционной системы и окружение для выполнения программы. Программа работает в *пользовательском режиме* и имеет возможность переключаться в режим ядра (например, для выполнения ввода/вывода) через системные вызовы или ловушки ядра.
- **Поток выполнения** (или просто **поток**): контекст выполнения, который можно запланировать для запуска на процессоре. Ядро имеет несколько потоков выполнения, а процессы — один или несколько.
- **Задача (task):** выполняемый объект Linux — процесс (с одним потоком), поток выполнения в многопоточном процессе или потоки ядра.
- **Программа BPF:** программа, действующая в пространстве ядра в среде выполнения BPF¹.
- **Основная память:** физическая память системы (например, ОЗУ).
- **Виртуальная память:** абстракция основной памяти, поддерживающая многозадачность и превышение ограничений. Это практически бесконечный ресурс.
- **Пространство ядра:** адресное пространство виртуальной памяти для ядра.
- **Пространство пользователя:** адресное пространство виртуальной памяти для процессов.

¹ Первоначально «BPF» расшифровывалось как «Berkeley Packet Filter», но в настоящее время эта технология имеет настолько мало общего с Беркли, пакетами и фильтрацией, что «BPF» превратилось из аббревиатуры в самостоятельное название.

- **Область пользователя:** программы и библиотеки пользовательского уровня (/usr/bin, /usr/lib ...).
- **Переключение контекста:** переключение с одного потока выполнения или процесса на другой. Это обычная функция планировщика в ядре, осуществляющая переключение набора регистров процессора (контекста потока) на новый набор.
- **Переключение режима:** переключение между режимами ядра и пользователя.
- **Системный вызов (syscall):** четко определенный протокол, посредством которого пользовательские программы запрашивают у ядра выполнение привилегированных операций, включая ввод/вывод.
- **Микросхема процессора:** не путайте с *процессом*, микросхема процессора — это физический чип, содержащий один или несколько процессоров.
- **Ловушка (trap):** сигнал, отправляемый ядру как запрос на выполнение системной процедуры (привилегированного действия). К ловушкам относятся: системные вызовы, исключения процессора и прерывания.
- **Аппаратное прерывание:** сигнал, отправляемый физическим устройством ядру, обычно для запроса обслуживания ввода/вывода. Прерывание — это разновидность ловушки.

В глоссарий в конце книги включены дополнительные термины, которые также встречаются в этой главе, в том числе *адресное пространство*, *буфер*, *центральный процессор (ЦП, CPU)*, *дескриптор файла*, *POSIX* и *регистры*.

3.2. ОСНОВЫ

В следующих разделах описываются общие понятия операционной системы и ядра, которые помогут получить общее представление об устройстве любой ОС. Для простоты в этот раздел включены некоторые детали реализации Linux. Следующие разделы, 3.3 «Ядра» и 3.4 «Linux», посвящены особенностям реализации ядра Unix, BSD и Linux.

3.2.1. Ядро

Ядро — это программное сердце ОС. Выполняемые им функции зависят от модели ядра: Unix-подобные операционные системы, включая Linux и BSD, имеют *монолитное* ядро, которое управляет процессорами, памятью, файловыми системами, сетевыми протоколами и системными устройствами (дисками, сетевыми интерфейсами и т. д.). Эта модель ядра показана на рис. 3.1.

Здесь также показаны системные библиотеки, которые часто используются, чтобы обеспечить более простой и богатый программный интерфейс по сравнению с системными вызовами. К приложениям относятся все запущенные программы пользовательского уровня, включая базы данных, веб-серверы, инструменты администрирования и командные оболочки операционной системы.

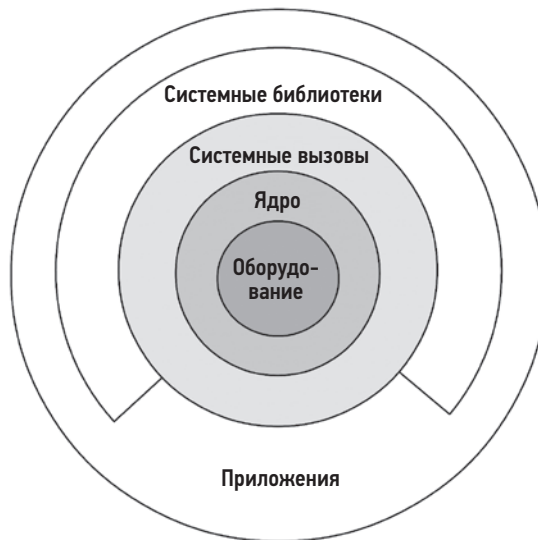


Рис. 3.1. Роль монолитного ядра операционной системы

Системные библиотеки изображены на рис. 3.1 в виде разорванного кольца, чтобы показать, что приложения могут обращаться к системным вызовам напрямую¹. Например, среда выполнения Golang имеет свой слой доступа к системным вызовам в обход системной библиотеки `libc`. Обычно на этой диаграмме кольца изображаются целыми, без разрывов, чтобы показать постепенное уменьшение уровня привилегий от ядра в центре к приложениям на периферии (эта модель впервые была реализована в Multics [Graham 68], предшественнице Unix).

Есть и другие модели ядра: в модели с *микроядром* используется небольшое ядро, а значительная доля функций перенесена в программы, действующие в пользовательском режиме; в модели *unikernel* (*одноцелевое ядро*) ядро и код приложения компилируются вместе в одну программу. Есть также *гибридные ядра*, такие как ядро Windows NT, в которых сочетаются подходы монолитных ядер и микроядер. Они кратко описаны в разделе 3.5 «Другие темы».

Модель ядра в Linux недавно претерпела изменения и теперь поддерживает новый тип программного обеспечения: расширенный механизм BPF, который обеспечивает

¹ Эта модель имеет некоторые исключения. Технологии обхода ядра, иногда используемые для работы с сетями, позволяют приложениям, действующим в режиме пользователя, напрямую обращаться к оборудованию (см. главу 10 «Сеть», раздел 10.4.3 «Программное обеспечение» подраздел «В обход ядра»). Ввод/вывод через устройство можно выполнять без затрат на интерфейс системных вызовов (впрочем, обратиться к системным вызовам все равно придется, хотя бы для инициализации), например, используя отображение ввода/вывода в память и механизм обработки отказов страниц (см. главу 7 «Память», раздел 7.2.3 «Подкачка страниц по требованию») или `sendfile(2)` и механизма `io_uring` в Linux (см. главу 5 «Приложения», раздел 5.2.6 «Неблокирующий ввод/вывод»).

безопасное выполнение приложений в режиме ядра и предлагает свой API ядра — вспомогательные вызовы BPF. Это позволяет переписать некоторые приложения и системные функции на BPF и обеспечить более высокий уровень безопасности и производительности. Эта возможность показана на рис. 3.2.

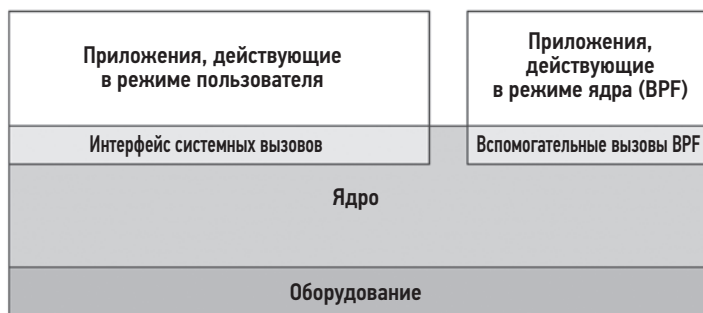


Рис. 3.2. Приложения BPF

Расширенный механизм BPF рассматривается в разделе 3.4.4 «Расширенный BPF».

Выполнение ядра

Ядро — это большая программа, объем кода которой исчисляется миллионами строк. В основном код выполняется по запросу, когда программа пользовательского уровня выполняет системный вызов или устройство посылает прерывание. Некоторые потоки ядра работают асинхронно и выполняют такие рутинные задачи, как обновление системных часов в ядре или управление памятью, но в целом они достаточно легковесны и потребляют очень мало вычислительных ресурсов.

Рабочие нагрузки, часто выполняющие операции ввода/вывода, например веб-серверы, действуют преимущественно в контексте ядра. Рабочие нагрузки, нужные для интенсивных вычислений, обычно выполняются в пользовательском режиме и не прерываются ядром. Может создаться впечатление, что ядро не влияет на производительность таких рабочих нагрузок, производящих массивные вычисления, но часто это не так. Наиболее очевидным влиянием является состязание за обладание процессором, когда другие потоки конкурируют за вычислительные ресурсы и планировщику ядра необходимо решить, какие из них будут выполняться, а какие — ждать. Ядро также решает, на каком процессоре будет выполняться поток, и может выбрать процессор с более горячими аппаратными кэшами или лучшей локализацией памяти для процесса, чтобы значительно повысить производительность.

3.2.2. Ядро и пользовательские режимы

Ядро работает в специальном режиме процессора, который называется *режимом ядра*, обеспечивающем неограниченный доступ к устройствам и выполнение привилегированных инструкций. Ядро регулирует доступ к устройствам для поддержки

многозадачности, предотвращая возможность доступа к данным других процессов и пользователей, если это явно не разрешено.

Пользовательские программы (процессы) выполняются в *пользовательском режиме*, где запрашивают выполнение привилегированных операций у ядра через системные вызовы, например, операций ввода/вывода.

Пользовательский режим и режим ядра реализуются аппаратно — процессорами — с использованием *колец привилегий* (или *колец защиты*) в соответствии с моделью, изображенной на рис. 3.1. Например, процессоры x86 поддерживают четыре кольца привилегий, пронумерованных от 0 до 3. Обычно используются только два или три кольца: для пользовательского режима, режима ядра и режима гипервизора, если он предусмотрен. Привилегированные инструкции для доступа к устройствам разрешены только в режиме ядра; попытка выполнить их в пользовательском режиме вызывает *исключение*, которое обрабатывается ядром (например, чтобы сгенерировать ошибку отказа в доступе).

В традиционном ядре системный вызов производит переключение в режим ядра, после чего исполняется остальной его код. Это показано на рис. 3.3.

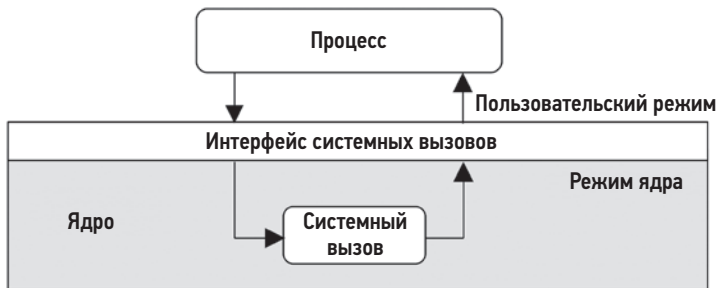


Рис. 3.3. Режимы выполнения системного вызова

Переключение между пользовательским режимом и режимом ядра называется *переключением режима*.

Переключение режима производят все системные вызовы. Некоторые системные вызовы также *переключают контекст*: системные вызовы, блокирующие дальнейшее выполнение вызвавшего потока, например, на время дискового и сетевого ввода/вывода, переключают контекст, чтобы позволить поработать другому потоку, пока первый ожидает завершения операции.

Поскольку переключение режима и контекста требует некоторого оверхеда (тактов процессора)¹, есть различные оптимизации, помогающие их избежать, в том числе:

- **Системные вызовы пользовательского режима:** некоторые системные вызовы можно реализовать на уровне библиотеки, действующей в пользовательском

¹ Из-за внедрения механизмов борьбы с уязвимостью Meltdown переключение контекста стало более дорогостоящим. См. раздел 3.4.3 «KPTI (Meltdown)».

режиме. Для этого ядро Linux экспортирует виртуальный динамически разделяемый объект (virtual Dynamic Shared Object, vDSO), который отображается в адресное пространство процесса и содержит такие системные вызовы, как `gettimeofday(2)` и `getcpu(2)` [Drysdale 14].

- **Отображение памяти:** используется для подкачки страниц памяти по требованию (см. главу 7 «Память», раздел 7.2.3 «Подкачка страниц по требованию»). Отображение памяти можно использовать для организации хранилищ данных и других операций ввода/вывода и при этом избежать обхода на системные вызовы.
- **В обход ядра:** этот прием позволяет программам пользовательского режима напрямую обращаться к устройствам, минуя системные вызовы ядра. Примером может служить технология Data Plane Development Kit (DPDK), используемая для работы с сетевыми устройствами.
- **Приложения режима ядра:** к ним относятся веб-сервер TUX [Lever 00], реализованный в ядре, а также расширенная технология BPF, изображенная на рис. 3.2.

Пользовательский режим и режим ядра имеют свои контексты выполнения, включая стек и регистры. Некоторые архитектуры процессоров (например, SPARC) используют отдельное адресное пространство для ядра, что означает необходимость изменения контекста виртуальной памяти при переключении режима.

3.2.3. Системные вызовы

Системные вызовы запрашивают у ядра выполнение привилегированных системных процедур. Есть сотни различных системных вызовов, но специалисты, занимающиеся разработкой ядра, прилагают все усилия, чтобы их было как можно меньше, а ядро оставалось максимально простым (как того требует философия Unix, [Thompson 78]). На их основе могут быть построены более сложные интерфейсы, доступные в пользовательской среде в виде системных библиотек, где их проще разрабатывать и поддерживать. Операционные системы обычно включают стандартную библиотеку для языка C, которая предоставляет более удобные интерфейсы ко многим системным вызовам (например, библиотеки `libc` или `glibc`).

Наиболее важные системные вызовы, о которых нужно помнить, перечислены в табл. 3.1.

Таблица 3.1. Наиболее важные системные вызовы

Системный вызов	Описание
<code>read(2)</code>	Читает байты
<code>write(2)</code>	Записывает байты
<code>open(2)</code>	Открывает файл
<code>close(2)</code>	Закрывает файл

Системный вызов	Описание
fork(2)	Создает новый процесс
clone(2)	Создает новый процесс или поток выполнения
exec(2)	Запускает новую программу
connect(2)	Устанавливает соединение с сетевым узлом
accept(2)	Принимает сетевое соединение
stat(2)	Извлекает статистики, имеющие отношение к файлу
ioctl(2)	Настраивает параметры ввода/вывода или выполняет другие функции
mmap(2)	Отображает файл в адресное пространство памяти
brk(2)	Расширяет область динамической памяти (кучи)
futex(2)	Быстрый мьютекс в пространстве пользователя

Системные вызовы хорошо документированы, для каждого есть своя страница в справочном руководстве man, которое обычно поставляется вместе с ОС. Они также имеют относительно простой и единообразный интерфейс и возвращают числовые коды для описания ошибок, когда это нужно. Например, код ENOENT описывает ошибку «нет такого файла или каталога» («no such file or directory»)¹.

Многие из этих системных вызовов имеют очевидную цель. Но есть и такие, применение которых может быть не столь очевидным. Вот некоторые из них:

- **ioctl(2)**: обычно используется для запроса выполнения различных действий в ядре; особенно часто — в инструментах системного администрирования или в программах, где другой (более очевидный) системный вызов не подходит. См. примеры ниже.
- **mmap(2)**: обычно используется для отображения выполняемых файлов и библиотек в адресное пространство процесса, а также для работы с файлами, отображаемыми в память. Иногда применяется для выделения памяти вместо malloc(2) и brk(2), чтобы уменьшить частоту обращений к системным вызовам и повысить производительность (этот прием не всегда дает положительный результат из-за необходимости предусматривать управление отображаемой памятью).
- **brk(2)**: используется для расширения области динамической памяти (кучи), размер которой определяет размер рабочей памяти процесса. Обычно вызывается автоматически библиотекой распределения системной памяти, когда вызов malloc(3) (выделение памяти) сталкивается с проблемой нехватки памяти в существующем объеме кучи. См. главу 7 «Память».
- **futex(2)**: этот системный вызов используется для управления блокировками в пространстве пользователя.

¹ glibc возвращает эти коды ошибок в целочисленной переменной errno (error number — номер ошибки).

Чтобы узнать больше о том или ином системном вызове, см. его страницу в справочном руководстве `man` (страницы с описаниями системных вызовов находятся в разделе 2 «Системные вызовы» (`syscalls`) руководства).

Системный вызов `ioctl(2)` является, пожалуй, самым сложным для изучения из-за его неоднозначной природы. Например, инструмент `perf(1)` в Linux (представленный в главе 6 «Процессоры») выполняет привилегированные действия для координации инструментов анализа производительности. Вместо множества системных вызовов для каждого из возможных действий в ядро был добавлен один системный вызов: `perf_event_open(2)`, который возвращает дескриптор файла для передачи в `ioctl(2)`. Имея этот дескриптор, можно вызвать `ioctl(2)` с разными аргументами, чтобы выполнить различные действия. Например, вызов `ioctl(fd, PERF_EVENT_IOC_ENABLE)` включит инструментацию. Аргументы, как `PERF_EVENT_IOC_ENABLE` в этом примере, могут добавляться и изменяться разработчиком.

3.2.4. Прерывания

Прерывание — это сигнал процессору о том, что произошло какое-то событие, требующее немедленной реакции; после получения сигнала выполнение текущего процесса прерывается для обработки этого события. Обычно это приводит к тому, что процессор переходит в режим ядра, если этого еще не было сделано, сохраняет текущее состояние потока, и затем запускается *процедура обработки прерывания* (*interrupt service routine, ISR*).

Есть асинхронные прерывания, генерируемые внешним оборудованием, и синхронные прерывания, генерируемые программными инструкциями. Они показаны на рис. 3.4.

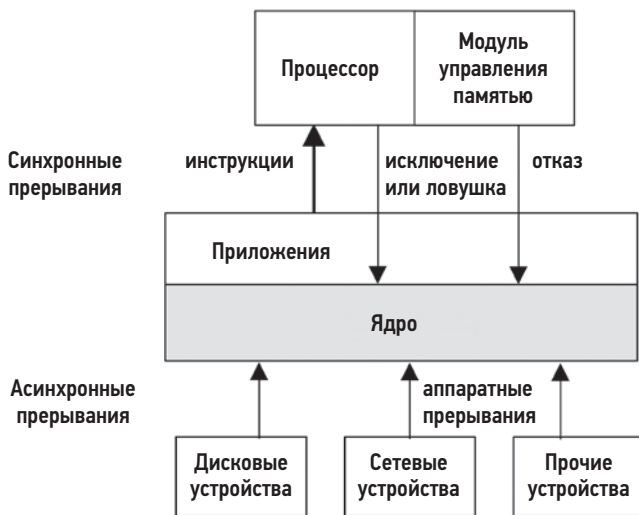


Рис. 3.4. Типы прерываний

Для простоты на рис. 3.4 показаны все прерывания, посылаемые ядру для обработки. Они сначала передаются процессору, который, в свою очередь, выбирает и запускает ISR для обработки события.

Асинхронные прерывания

Аппаратные устройства могут посылать процессору *запросы на обслуживание прерывания* (interrupt service request, IRQ) асинхронно по отношению к ПО, которое выполняется в данный момент. В числе примеров аппаратных прерываний можно назвать:

- прерывания от дисковых устройств, сигнализирующие о завершении дискового ввода/вывода;
- прерывания от оборудования, указывающие на состояние отказа;
- прерывания от сетевых интерфейсов, сигнализирующие о получении пакета;
- прерывания от устройств ввода: ввод с клавиатуры и мыши.

Чтобы объяснить идею асинхронных прерываний, на рис. 3.5 изображен пример сценария, показывающий, как проходит чтение файловой системы в базе данных (MySQL), выполняющейся на CPU 0. Содержимое файловой системы должно быть получено с диска, поэтому планировщик переключает контекст на другой поток (приложение на Java), пока база данных ждет завершения операции. Через какое-то время операция дискового ввода/вывода завершается, но в этот момент база данных уже не выполняется на CPU 0. Прерывание, сообщающее о завершении операции, произошло асинхронно по отношению к базе данных, как показывает пунктирная линия на рис. 3.5.

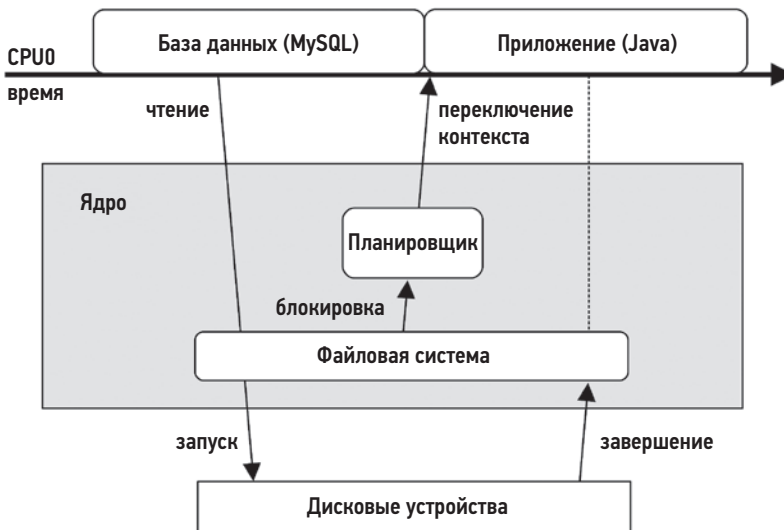


Рис. 3.5. Пример асинхронного прерывания

Синхронные прерывания

Синхронные прерывания генерируются программными инструкциями. Ниже описаны различные типы программных прерываний с использованием таких терминов, как *ловушки*, *исключения* и *отказы*. Часто эти термины используются как синонимы.

- **Ловушки:** преднамеренный вызов ядра, например, с помощью инструкции `int` (`interrupt` — прерывание). Есть реализации системных вызовов, обращения к которым начинаются с выполнения инструкции `int` с вектором обработчика системных вызовов (например, `int 0x80` в Linux x86). Инструкция `int` вызывает программное прерывание.
- **Исключения:** исключительные случаи, например, когда выполняется деление на ноль.
- **Отказы:** этот термин часто используется для обозначения событий, связанных с памятью, таких как *отказы страниц*, возникающих при попытке обратиться к области памяти, которая не была отображена модулем управления памятью (MMU). Подробности см. в главе 7 «Память».

Программное обеспечение, породившее эти прерывания, в этот момент все еще выполняется на процессоре.

Потоки выполнения для обработки прерываний

Процедуры обработки прерываний (ISR) реализуются так, чтобы обработать прерывание как можно быстрее и уменьшить влияние на активные потоки выполнения. Если для обработки прерывания требуется много работы, и особенно если в процессе обработки процедура может приостановиться на блокировке, то для обработки таких прерываний могут использоваться отдельные потоки выполнения, управляемые планировщиком ядра. Это показано на рис. 3.6.

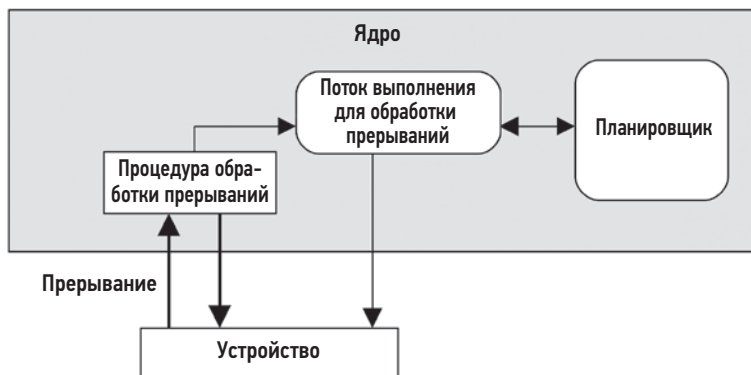


Рис. 3.6. Обработка прерывания

Фактическая реализация таких потоков зависит от версии ядра. В Linux драйверы устройств могут быть реализованы на основе модели двух половин. Верхняя половина в таком драйвере быстро обрабатывает прерывание и планирует задания для нижней половины, которая будет выполняться позже [Corbet 05]. Высокая скорость обработки прерываний очень важна, потому что верхняя половина работает в режиме *с отключенными прерываниями*, чтобы отложить доставку новых прерываний. Это может вызвать проблемы с задержкой в других потоках выполнения, если верхняя половина будет работать слишком долго. Нижняя половина может быть реализована как *tasklet* (tasklet) или как *очередь заданий*. Последняя обслуживается потоками выполнения, которые действуют под управлением планировщика и могут приостанавливаться, когда в очереди нет заданий.

Например, в сетевых драйверах в Linux верхняя половина обрабатывает запросы IRQ, порождаемые поступающими пакетами, и запускает нижнюю половину для продвижения пакета вверх по сетевому стеку. Нижняя половина реализована как *обработчик программного прерывания* (softirq).

Время от получения прерывания до момента его обслуживания — это *задержка обработки прерывания*, которая зависит от оборудования и реализации. Это особый предмет исследования для систем реального времени или систем с малой задержкой.

Маскировка прерываний

Некоторые пути в коде в ядре невозможно прервать, не опасаясь что-нибудь разрушить. Примером может служить код ядра, который получает спин-блокировку во время системного вызова, потому что обработчику прерывания тоже может потребоваться получить спин-блокировку. Запуск обработчика прерывания в такой момент может вызвать тупиковую ситуацию (deadlock). Чтобы этого не случилось, ядро может на время маскировать прерывания, устанавливая регистр *маски прерываний* в процессоре. Продолжительность маскировки прерываний должна быть как можно короче, потому что это может помешать своевременной реакции приложений, которые могут активироваться другими прерываниями. Это важный фактор для систем *реального времени*, имеющих строгие требования к времени отклика. Продолжительность выполнения с отключенными прерываниями также является одной из целей анализа производительности (такой вид анализа поддерживается трассировщиком irqsoff в Ftrace, который упоминается в главе 14 «Ftrace»).

Некоторые высокоприоритетные события не следует игнорировать, поэтому они реализованы как *немаскируемые прерывания* (non-maskable interrupts, NMI). Например, Linux может использовать сторожевой таймер интеллектуального интерфейса управления платформой (Intelligent Platform Management Interface, IPMI), который проверяет, не заблокировано ли ядро из-за отсутствия прерываний в течение определенного периода времени. Если ядро заблокировано, то сторожевой таймер может сгенерировать немаскируемое прерывание для перезагрузки системы¹.

¹ В Linux также есть программный сторожевой таймер, генерирующий немаскируемые прерывания при обнаружении зависаний [Linux 20d].

3.2.5. Часы и бездействие

Одним из основных компонентов оригинального ядра Unix является процедура `clock()`, обрабатывающая прерывания таймера. Исторически она вызывалась с частотой 60, 100 или 1000 раз в секунду¹ (или герц), и каждый ее вызов называется *тактом системных часов*². В ее задачи входило обновление системного времени, обновление таймеров и квантов времени для планирования потоков, подсчет статистики потребления процессора и выполнение запланированных подпрограмм ядра.

В реализации этой процедуры были проблемы, связанные с производительностью, которые устранили в более поздних версиях ядра, в том числе:

- **Задержка такта системных часов:** при частоте следования тактовых импульсов 100 Гц таймер мог срабатывать с опозданием до 10 мс из-за того, что запланированное время истекало сразу после предыдущего такта. Эта проблема была исправлена с помощью прерываний реального времени с высоким разрешением, так что теперь обработчик истечения таймера вызывается немедленно.
- **Оверхед на обработку тактов:** на обработку каждого такта системных часов расходуется процессорное время, что несколько мешает работе приложений и является одной из причин так называемого *колебания задержек операционной системы*. Кроме того, современные процессоры поддерживают динамическое управление питанием и могут отключать некоторые компоненты в периоды бездействия. Процедура `clock()` прерывает эти периоды, что может привести к повышенному потреблению электроэнергии.

В современных ядрах большая часть функций была перенесена из процедуры `clock()` в прерывания по требованию, чтобы не обрабатывать такты системных часов. Это позволило снизить оверхед и повысить энергоэффективность за счет того, что процессоры дольше оставались в состоянии бездействия.

Функции процедуры `clock()` в Linux выполняет функция `scheduler_tick()`. Кроме того, в Linux предусмотрена возможность не вызывать эту функцию, когда процессор бездействует. Сами часы обычно работают на частоте 250 Гц (настраивается параметром `CONFIG_HZ`), а количество вызовов сокращается за счет функции `NO_HZ` (настраивается параметром `CONFIG_NO_HZ`), которая в настоящее время обычно активна [Linux 20a].

Поток бездействия

Когда процессор не выполняет никакой работы, ядро планирует для выполнения специальный поток, который ожидает появления работы и называется *поток бездействия, или ожидания* (*idle thread*). В упрощенном представлении поток

¹ Для справки: в Linux 2.6.13 системные часы работают с частотой 250 Гц, в Ultrix — с частотой 256 Гц и в OSF/1 — с частотой 1024 Гц [Mills 94].

² Ядро Linux также отслеживает *миги* (*jiffies*), интервалы времени, аналогичные тактам.

просто проверяет наличие новой работы в цикле. В современном ядре Linux *задача бездействия* может вызывать инструкцию `hlt` (`halt` — останов) для выключения питания процессора до следующего прерывания, что позволяет экономить электроэнергию.

3.2.6. Процессы

Процесс — это среда для выполнения программы пользовательского уровня, включающая адресное пространство памяти, файловые дескрипторы, стеки потоков и регистры. В некотором смысле процесс похож на виртуальный компьютер, на котором выполняется только одна программа со своими собственными регистрами и стеками.

Процессы являются многозадачными благодаря ядру, которое поддерживает возможность выполнения тысяч процессов в одной системе. Процессы идентифицируются уникальными числовыми *идентификаторами процесса* (`process ID`, `PID`).

Процесс содержит один или несколько *потоков выполнения*, которые действуют в общем адресном пространстве процесса и используют одни и те же файловые дескрипторы. Поток выполнения — это контекст выполнения, включающий стек, регистры и указатель инструкций (который также называется *программным счетчиком*). Возможность запускать несколько потоков позволяет одному процессу выполняться параллельно на нескольких процессорах. В Linux потоки выполнения и процессы называют *задачами*.

Первый процесс, запускаемый ядром, называется «`init`» и получает идентификатор `PID`, равный 1. Выполняемый файл для запуска этого процесса по умолчанию находится в `/sbin/init`. Этот процесс, в свою очередь, запускает службы пространства пользователя. В Unix это подразумевало запуск сценариев начальной загрузки из каталога `/etc`. Этот метод теперь называется SysV (от Unix System V). В настоящее время для запуска служб и их зависимостей дистрибутивы Linux обычно используют программное обеспечение `systemd`.

Создание процесса

В системах Unix процессы обычно создаются с помощью системного вызова `fork(2)`. Библиотеки для языка C в Linux обычно реализуют свою функцию `fork`, обертывающую универсальный системный вызов `clone(2)`. Эти системные вызовы создают копию текущего процесса с собственным идентификатором процесса. После этого можно обратиться к системному вызову `exec(2)` (или его вариантам, например `execve(2)`), чтобы запустить в процессе другую программу.

На рис. 3.7 показан пример создания процесса командной оболочки `bash` (`bash`), выполняющей команду `ls`.

Системные вызовы `fork(2)` и `clone(2)` могут использовать стратегию копирования при записи (`copy-on-write`, `COW`) для повышения производительности. То есть

вместо копирования всего содержимого памяти процесса-родителя процесс-потомок получает ссылку. Но как только любой из процессов, родитель или потомок, попытается изменить содержимое памяти, для области памяти с изменениями создается отдельная копия. Эта стратегия откладывает копирование памяти, а иногда даже полностью устраняет необходимость в нем, уменьшая потребление памяти и процессора.

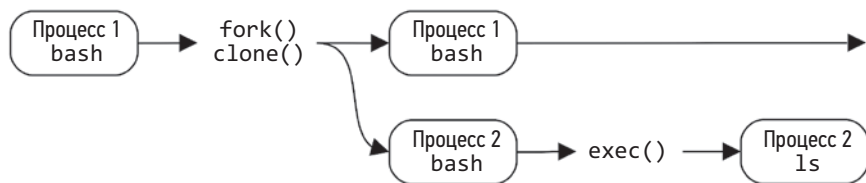


Рис. 3.7. Создание процесса

Жизненный цикл процесса

Жизненный цикл процесса показан на рис. 3.8. Это упрощенная диаграмма, потому что в современных многопоточных ОС запускаются и планируются потоки выполнения, а также имеются некоторые дополнительные тонкости реализации, касающиеся их отображения в состояние процесса (более подробные диаграммы приводятся в главе 5 на рис. 5.6 и 5.7).

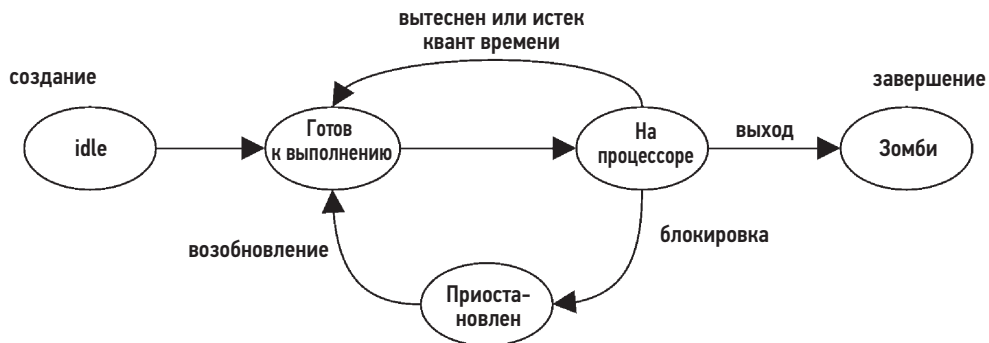


Рис. 3.8. Жизненный цикл процесса

Состояние «на процессоре» означает выполнение на процессоре. Состояние «готов к выполнению» означает, что процесс запущен, но в данный момент ожидает своей очереди выполнения на процессоре. Большинство операций ввода/вывода блокируют процесс, переводя его в состояние приостановки до тех пор, пока ввод/вывод не завершится, после чего процесс возобновляется. Состояние «зомби» возникает во время завершения процесса, когда процесс ждет, пока его статус не будет передан родителю или пока он не будет удален ядром.

Среда процесса

На рис. 3.9 показана среда процесса. Она включает данные в адресном пространстве процесса и метаданные (контекст) в ядре.

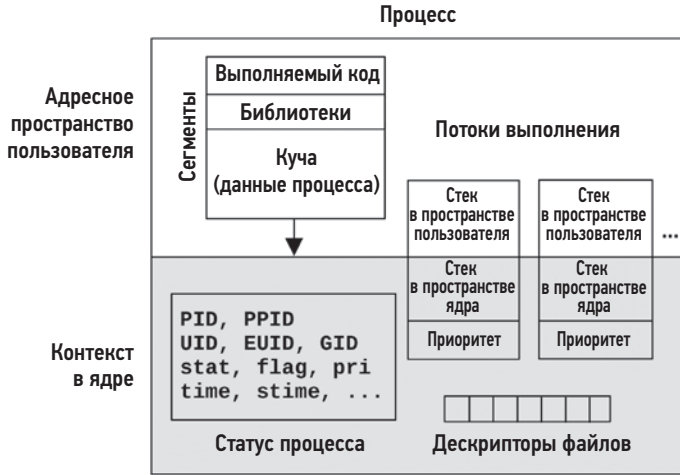


Рис. 3.9. Среда процесса

Контекст в ядре включает множество различных свойств и статистик, характеризующих процесс: его идентификатор процесса (PID), идентификатор пользователя-владельца (UID) и различные значения времени. Обычно их исследуют с помощью команд `ps(1)` и `top(1)`. Также в контекст включен набор дескрипторов файлов, которые ссылаются на открытые файлы и (обычно) совместно используются потоками выполнения.

На рис. 3.9 изображены два потока, каждый из которых содержит некоторые метаданные, включая приоритет в контексте в ядре¹ и стек в пользовательском адресном пространстве. Диаграмма не в масштабе — контекст в ядре занимает очень мало места по сравнению с адресным пространством процесса.

Пользовательское адресное пространство делится на несколько сегментов, содержащих выполняемый код, библиотеки и кучу (динамическую память). Дополнительные сведения ищите в главе 7 «Память».

В Linux каждый поток выполнения имеет свой стек в пространстве пользователя и стек исключений в пространстве ядра² [Owens 20].

¹ Контекст в ядре может быть полноценным адресным пространством (как в случае с процессорами SPARC) или ограниченной областью, не перекрывающейся с пользовательским адресным пространством (как в случае с процессорами x86).

² Также в пространстве ядра есть специализированные стеки для каждого процессора, в том числе используемые обработчиками прерываний.

3.2.7. Стеки

Стек — это область памяти для временных данных, организованная в виде списка «последним пришел — первым ушел» (last-in, first-out, LIFO). Стек используется для хранения менее важных данных, чем те, что помещаются в регистры процессора. Когда программа вызывает функцию, адрес возврата из нее сохраняется в стеке. Также в стеке могут быть сохранены некоторые регистры, если их значения понадобятся после вызова¹. Когда вызываемая функция завершает работу, она восстанавливает все необходимые регистры и, извлекая из стека адрес возврата, возвращается в вызвавшую ее функцию. Стек также можно использовать для передачи параметров функциям. Набор данных в стеке, связанный с выполнением функции, называется *фреймом стека*.

Цепочку вызовов, которая привела к текущей выполняющейся функции, можно получить, изучив адреса возврата, хранящиеся во всех фреймах в стеке потока выполнения (этот процесс называется *обходом стека*)². Эту цепочку вызовов обычно называют *обратной трассировкой стека*, или *трассировкой стека*. В анализе производительности ее часто для краткости называют просто стеком. Стеки позволяют ответить на вопрос, *почему* выполняется тот или иной код, и являются бесценным инструментом для отладки и анализа производительности.

Как читать стеки

Ниже приводится пример стека ядра (из Linux), показывающий цепочку передачи пакета TCP и полученный с помощью инструмента трассировки:

```
tcp_sendmsg+1
sock_sendmsg+62
SYSC_sendto+319
sys_sendto+14
do_syscall_64+115
entry_SYSCALL_64_after_hwframe+61
```

Стек обычно выводится в направлении от листа к корню, где первая строка соответствует текущей выполняемой функции, а за ней следуют ее родитель, затем

¹ Соглашение о вызовах, в зависимости от ABI (application binary interface — прикладной двоичный интерфейс) процессора, определяет, какие регистры должны восстанавливаться после вызова функции (*неизменяемые* регистры) и сохраняться в стеке вызываемой функцией (сохранение выполняет вызываемый). Другие регистры считаются *изменяемыми* и могут использоваться вызываемой функцией. Если вызывающий код хочет сохранить свои значения в этих регистрах, он должен сохранить их в стеке перед вызовом (сохранение выполняет вызывающий).

² Подробную информацию об обходе стека и различных способах реализации (в числе которых методы на основе указателя на список фреймов, с использованием отладочной информации, записи последней ветви и ORC) см. в главе 2 «Tech» в разделе 2.4 «Stack Trace Walking» в книге «BPF Performance Tools» [Gregg 19].

прародитель и т. д. В этом примере в текущий момент выполнялась функция `tcp_sendmsg()`. Она была вызвана из `sock_sendmsg()`. Также в этом примере стека, справа от имен функций, выводится смещение инструкции, показывающее местоположение внутри функции. Смещение 1 около имени `tcp_sendmsg()` в первой строке показывает, что ее вызов был второй инструкцией в `sock_sendmsg()`, которая сама имеет смещение 62. Эти смещения могут пригодиться, только если вы захотите изучить путь выполнения кода на еще более низком уровне — уровне инструкций.

Читая стек сверху вниз, можно увидеть всю «родословную» текущей функции: ее родителя, прародителя и т. д., читая его снизу вверх — проследить, каким путем выполнение кода привело к текущей функции.

Стеки показывают, как следует выполнение через функции, для которых часто нет никакой другой документации, кроме их исходного кода. В данном примере это исходный код ядра Linux. Исключение составляют случаи, когда функции являются частью API и задокументированы.

Стеки в пространствах пользователя и ядра

При выполнении системного вызова поток процесса имеет два стека: в пространстве пользователя и в пространстве ядра. Они показаны на рис. 3.10.

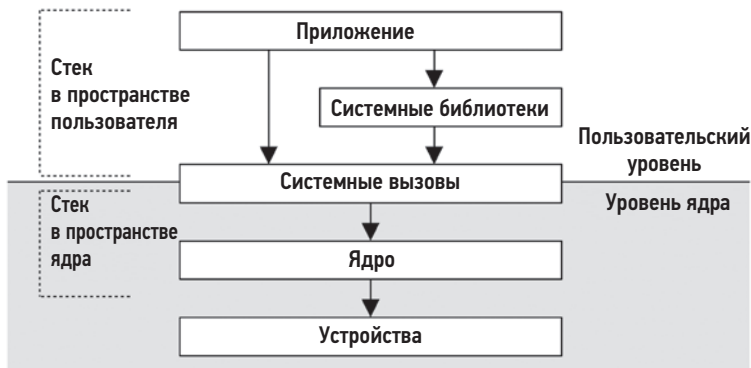


Рис. 3.10. Стеки в пространстве пользователя и в пространстве ядра

Стек пользовательского уровня заблокированного потока не изменяется во время выполнения системного вызова, потому что при выполнении в контексте ядра используется отдельный стек — стек уровня ядра. (Исключение — некоторые обработчики сигналов, которые в зависимости от конфигурации могут использовать стек пользовательского уровня.)

В Linux есть несколько стеков ядра, предназначенных для разных целей. Системные вызовы используют стек исключений ядра, связанный с каждым потоком, а кроме него есть и стеки для обработчиков программных и аппаратных прерываний [Bovet 05].

3.2.8. Виртуальная память

Виртуальная память — это абстракция основной памяти, предоставляющая процессам и ядру собственное, почти бесконечное¹ представление основной памяти. Виртуальная память поддерживает многозадачность, позволяя процессам и ядру работать в своих изолированных адресных пространствах, не беспокоясь о конфликтах. Она также поддерживает превышение лимита основной памяти, позволяя операционной системе при необходимости прозрачно отображать виртуальную память между основной памятью и вторичным хранилищем (дисками).

Идея виртуальной памяти показана на рис. 3.11. Первичная память — это основная память (ОЗУ), а вторичная память — это устройства хранения (диски).

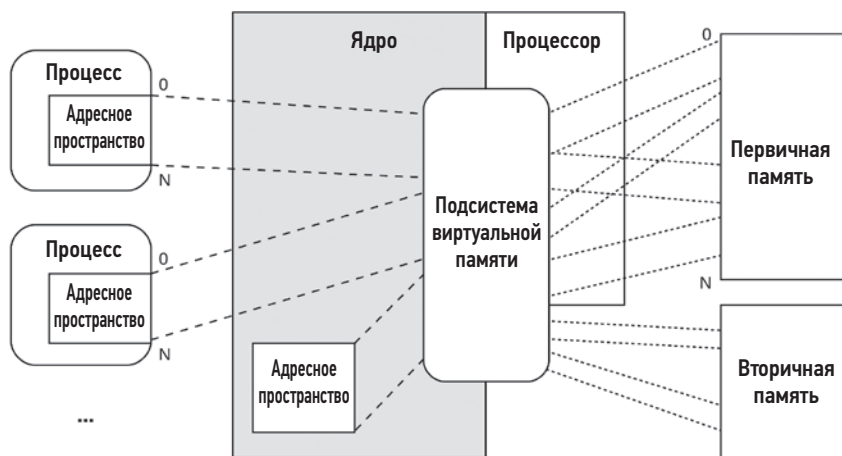


Рис. 3.11. Адресное пространство виртуальной памяти²

Поддержка виртуальной памяти обеспечивается процессором и операционной системой. Это не настоящая память, и большинство ОС отображают области

¹ Во всяком случае в 64-битных архитектурах. В 32-битных архитектурах объем виртуальной памяти ограничен 4 Гбайт из-за ограничений 32-битных адресов (ядро может еще больше ограничить этот объем).

² Для простоты начальный адрес виртуальной памяти процесса на этом рисунке равен 0. В современных ядрах виртуальное адресное пространство процесса обычно начинается с некоторого смещения, например 0x10000 или некоторого случайного адреса. Одно из преимуществ использования смещения заключается в том, что при таком подходе обычная программная ошибка разыменования пустого указателя NULL (0) приведет к сбою программы (SIGSEGV), потому что адрес 0 считается недействительным. Это предпочтительнее, чем ошибочное извлечение данных, находящихся по адресу 0, потому что в последнем случае программа продолжит работу с ошибочными данными.

виртуальной памяти в реальное адресное пространство только по запросу, когда происходит первое заполнение (запись) памяти данными.

Дополнительную информацию о виртуальной памяти ищите в главе 7 «Память».

Управление памятью

Виртуальная память позволяет расширять основную память за счет вторичного хранилища, однако ядро старается хранить наиболее часто используемые данные в основной памяти. Для этого в ядре реализованы две схемы:

- **подкачка процессов** — перемещение целых процессов между основной и вторичной памятью;
- **подкачка страниц** — перемещение небольших блоков памяти, называемых *страницами* (например, размером 4 Кбайт).

Подкачка процессов, впервые предложенная и реализованная в Unix, может вызвать серьезную потерю производительности. Подкачка страниц действует более эффективно, ее добавили в BSD с введением страничной организации виртуальной памяти. В обоих случаях наиболее давно использовавшаяся память перемещается во вторичное хранилище и возвращается в основную память, только когда она понадобится снова.

В Linux термин *подкачка* (swapping) используется для обозначения *подкачки страниц* (paging). Ядро Linux не поддерживает (более старую) подкачку процессов в стиле Unix, когда между первичной и вторичной памятью перемещаются целые потоки и процессы.

Дополнительную информацию о страничной организации и подкачке вы найдете в главе 7 «Память».

3.2.9. Планировщики

ОС Unix и ее производные — это системы с разделением времени, позволяющие запускать несколько процессов одновременно за счет деления времени выполнения между ними. Планирование процессов для выполнения на процессорах осуществляется *планировщиком*, ключевым компонентом ядра операционной системы. Принцип действия планировщика показан на рис. 3.12, где видно, что планировщик работает с потоками (*задачами* в Linux) и выбирает процессоры для их выполнения.

Основная цель планировщика — разделить процессорное время между активными процессами и потоками с учетом их *приоритетов*, чтобы более важная работа могла выполняться быстрее. Все потоки, готовые к выполнению, планировщик традиционно запоминает в разных очередях по приоритетам, которые называются *очередями на выполнение* [Bach 86]. Современные ядра могут реализовать такие очереди для каждого процессора, а также запомнить потоки с использованием других структур данных помимо очередей. Когда количество потоков, готовых к выполнению,

превышает количество процессоров, то потоки с более низким приоритетом ждут, пока выполнятся потоки с высоким приоритетом. В большинстве случаев потоки ядра имеют более высокий приоритет, чем процессы пользовательского уровня.

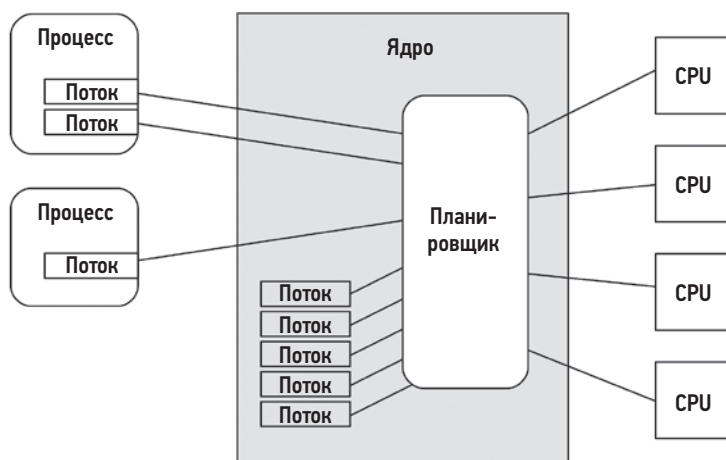


Рис. 3.12. Планировщик ядра

Планировщик может динамически изменять приоритет процесса для повышения производительности определенных рабочих нагрузок. В общем случае рабочие нагрузки можно разделить на следующие категории:

- **Вычислительные нагрузки:** приложения, выполняющие тяжелые вычисления, например научный и математический анализ, которые, как ожидается, будут иметь длительное время выполнения (секунды, минуты, часы, дни и даже дольше). Их производительность ограничивается вычислительной мощностью процессора.
- **Нагрузки с интенсивным вводом/выводом:** приложения, выполняющие большое количество операций ввода/вывода с небольшим объемом вычислений, например веб-серверы, файловые серверы и интерактивные оболочки, где нужна минимальная задержка. С увеличением нагрузки на такие приложения их производительность ограничивается пропускной способностью хранилища или сетевых ресурсов.

Типичная стратегия планирования, восходящая к временам UNIX, заключается в определении вычислительных рабочих нагрузок и снижении их приоритета, чтобы позволить чаще запускаться нагрузкам с интенсивным вводом/выводом, для которых желательна минимальная задержка. Это можно реализовать, вычисляя отношение продолжительности выполнения на процессоре к реальному времени выполнения и уменьшая приоритет процессов с высоким получившимся значением [Thompson 78]. Этот механизм отдает предпочтение более короткоживущим

процессам, которые обычно выполняют операции ввода/вывода, в том числе интерактивным процессам, взаимодействующим с человеком.

Современные ядра поддерживают несколько *классов*, или *стратегий планирования* (Linux), которые используют разные алгоритмы для управления приоритетом и выполняемыми потоками. В их число может входить *планирование реального времени*, использующее более высокий приоритет, чем вся остальная менее важная работа, включая потоки ядра. Наряду с поддержкой вытеснения (описанной ниже) планирование реального времени обеспечивает предсказуемое планирование с малой задержкой для систем, которым это необходимо.

Дополнительную информацию о планировщике ядра и других алгоритмах планирования ищите в главе 6 «Процессоры».

3.2.10. Файловая система

Файловые системы определяют способ организации данных на дисках в виде файлов и каталогов и предлагают интерфейс для доступа к ним, обычно основанный на стандарте POSIX. Ядра поддерживают несколько типов и экземпляров файловых систем. Поддержка файловой системы — одна из наиболее важных ролей операционной системы, а когда-то ее называли *самой* важной ролью [Ritchie 74].

Операционная система предоставляет глобальное пространство имен файлов, организованное в виде дерева, растущего сверху вниз, от корня («/»). Файловые системы присоединяются к дереву путем *монтирования* и подключают свое дерево к каталогу, который в этом случае называют *точкой монтирования*. Это позволяет конечному пользователю прозрачно перемещаться по пространству имен файлов, независимо от типа базовой файловой системы.

На рис. 3.13 показана типичная организация файлов в ОС.

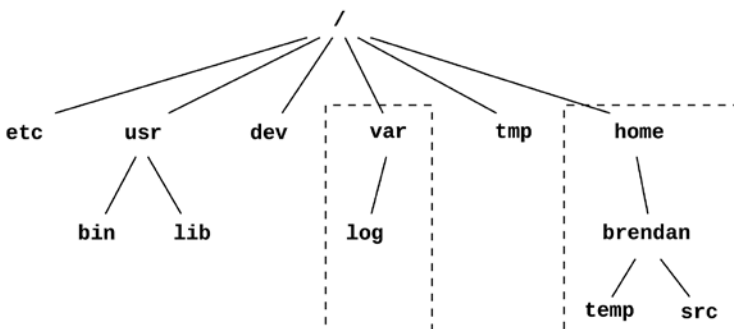


Рис. 3.13. Типичная организация файлов в операционной системе

На верхнем уровне находятся каталоги: *etc* — для файлов с настройками системы; *usr* — для программ и библиотек пользовательского уровня, предоставляемых

системой; `dev` — для узлов устройств; `var` — для часто меняющихся файлов, включая системные журналы; `tmp` — для временных файлов; и `home` — для домашних каталогов пользователей. Каталоги `var` и `home` в примере на рис. 13.3 могут находиться в отдельных экземплярах файловой системы и на отдельных устройствах, при этом они будут доступны, как любые другие компоненты дерева.

Большинство типов файловых систем используют устройства хранения (диски) для хранения своего содержимого. Некоторые типы файловых систем, например `/proc` и `/dev`, динамически создаются ядром.

Ядра обычно предоставляют различные способы изоляции частей файловой системы от процессов, включая `chroot(8)`, и один из таких способов — пространство имен файловых систем (`mount namespace`) — широко используется в Linux для изоляции контейнеров (см. главу 11 «Облачные вычисления»).

VFS

Виртуальная файловая система (virtual file system, VFS) — это интерфейс ядра, не зависящий от конкретных типов файловых систем, первоначально разработанный в Sun Microsystems, чтобы упростить интеграцию файловой системы Unix (Unix file system, UFS) и сетевой файловой системы (Network file system, NFS). Роль VFS показана на рис. 3.14.

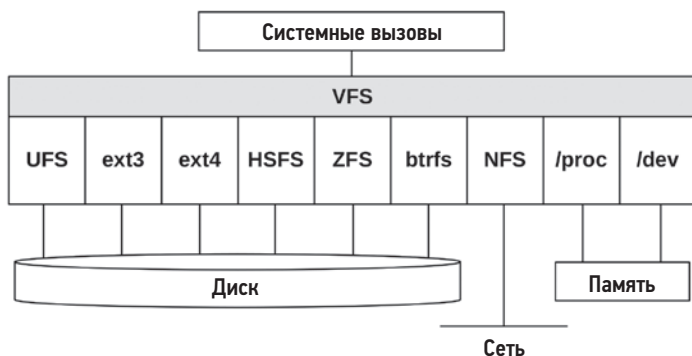


Рис. 3.14. Виртуальная файловая система

Интерфейс VFS упрощает добавление в ядро поддержки новых типов файловых систем. Он также позволяет организовать глобальное пространство имен файлов, как было показанное выше, чтобы пользовательские программы и приложения могли прозрачно обращаться к различным типам файловых систем.

Стек ввода/вывода

Путь от программного обеспечения пользовательского уровня к файловой системе на запоминающем устройстве называется *стеком ввода/вывода*. Это подмножество

всего программного стека, показанного ранее. Стек ввода/вывода в общем виде показан на рис. 3.15.

Слева на рис. 3.15 показан прямой путь к блочным устройствам в обход файловой системы. Этот путь иногда используется инструментами администрирования и базами данных.

Файловые системы и их производительность подробно описаны в главе 8 «Файловые системы», а устройства хранения, на которых они размещаются, описаны в главе 9 «Диски».

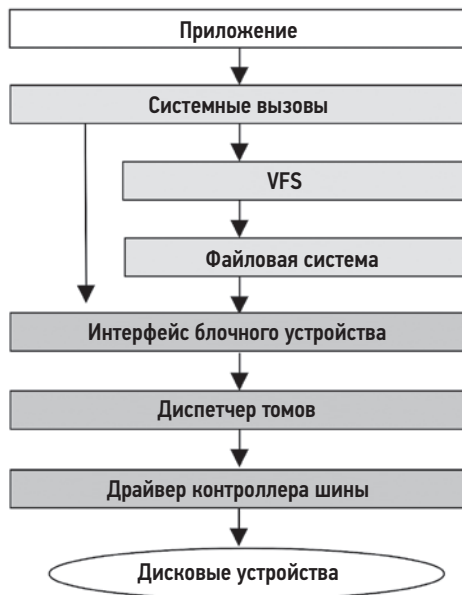


Рис. 3.15. Обобщенный стек ввода/вывода

3.2.11. Кэширование

Поскольку дисковый ввод/вывод традиционно имел высокую задержку, на многих уровнях стека программного обеспечения делаются попытки избежать их путем кэширования операций чтения и буферизации операций записи. В табл. 3.2 перечислены возможные виды кэшей (в том порядке, в каком они проверяются).

Например, кэш буферов — это область в основной памяти, где хранятся недавно использованные дисковые блоки. Данные для операций чтения с диска могут немедленно возвращаться из кэша, если в нем есть запрошенный блок, что позволяет избежать большой задержки, связанной с дисковым вводом/выводом.

Типы имеющихся кэшей зависят от системы и окружения.

Таблица 3.2. Уровни кэшей для дискового ввода/вывода

	Кэш	Примеры
1	Кэш клиента	Кэш браузера
2	Кэш приложения	—
3	Кэш веб-сервера	Кэш Apache
4	Служба кэширования	memcached
5	Кэш базы данных	Кэш буферов в MySQL
6	Кэш каталога	dcache
7	Кэш метаданных файлов	Кэш индексных узлов (inode)
8	Кэш буферов операционной системы	Кэш буферов
9	Основной кэш файловой системы	Кэш страниц, ZFS ARC
10	Вторичный кэш файловой системы	ZFS L2ARC
11	Кэш устройства	ZFS vdev
12	Кэш блоков	Кэш буферов
13	Кэш контроллера диска	Кэш карты RAID
14	Кэш массива устройств хранения	—
15	Кэш внутри диска	—

3.2.12. Сеть

Современные ядра предоставляют стек встроенных сетевых протоколов, позволяя системе обмениваться данными через сеть и участвовать в работе распределенных окружений. Этот стек протоколов называется *сетевым стеком*, или *стеком TCP/IP*, в честь широко используемых протоколов TCP и IP. Приложения пользовательского уровня могут обращаться к сети через программируемые конечные точки — *сокеты*.

Физическое устройство, которое подключается к сети, называется *сетевым интерфейсом* и обычно находится на *сетевой карте* (network interface card, NIC). Раньше одной из традиционных обязанностей системного администратора было назначение IP-адреса сетевому интерфейсу, чтобы тот мог подключиться к сети. Теперь эта работа обычно выполняется автоматически, благодаря протоколу динамической конфигурации хоста (dynamic host configuration protocol, DHCP).

Сетевые протоколы меняются не часто, но в последнее время наблюдается рост популярности нового транспортного протокола QUIC (кратко описывается в главе 10 «Сеть»). Расширения протокола и его параметры меняются чаще, как, например, новые параметры и алгоритмы управления заторами в протоколе TCP. Новые протоколы и улучшения обычно требуют поддержки ядра (за исключением реализаций протоколов, действующих в пространстве пользователя). Еще одно

частое изменение — поддержка различных сетевых карт, для которых требуется включение в ядро новых драйверов.

Дополнительные сведения о сети и ее производительности см. в главе 10 «Сеть».

3.2.13. Драйверы устройств

Ядро должно взаимодействовать с широким спектром физических устройств. Это достигается с помощью *драйверов устройств*: программных компонентов ядра, управляющих работой устройств и вводом/выводом. Драйверы устройств часто разрабатываются производителями этих устройств. Некоторые ядра поддерживают *подключаемые* драйверы устройств, которые можно загружать и выгружать без перезагрузки системы.

Драйверы устройств могут поддерживать *символьный* и/или *блочный* интерфейс. Символьные устройства, также называемые *небуферизованными*, обеспечивают небуферизованный последовательный доступ к вводу/выводу вплоть до каждого отдельного символа, в зависимости от устройства. К таким устройствам относятся клавиатуры и последовательные порты (а в раннюю эпоху развития Unix — накопители на бумажных лентах и последовательные принтеры).

Блочные устройства выполняют ввод/вывод блоками, обычно по 512 байт в каждом. К ним можно обращаться произвольно, указывая смещения блоков, счет которых начинается с 0. Первоначально в Unix для повышения производительности интерфейс блочного устройства обеспечивал также кэширование буферов блочного устройства в области основной памяти, называемой *кэшем буферов*. В Linux этот кэш теперь является частью кэша страниц.

3.2.14. Многопроцессорность

Поддержка нескольких процессоров позволяет операционной системе использовать несколько процессоров для параллельного выполнения работы. Обычно это реализуется как *симметричная многопроцессорность* (symmetric multiprocessing, SMP), в которой все процессоры обрабатываются одинаково. Реализовать многопроцессорность было технически сложно из-за проблем доступа и совместного использования памяти и процессоров потоками, выполняющимися параллельно. В многопроцессорных архитектурах с *неравномерным доступом к памяти* (non-uniform memory access, NUMA) могут быть банки основной памяти, подключенные к разным физическим процессорам, что также создает проблемы с производительностью. Более подробную информацию, включая планирование и синхронизацию потоков, вы найдете в главе 6 «Процессоры», а сведения о доступе к памяти и архитектуре — в главе 7 «Память».

ИП

В многопроцессорных системах процессорам иногда нужно координировать свои действия, например, для согласования записей в кэше (для информирования других

процессоров о том, что запись, если она есть в кэше, теперь устарела). Процессор может запросить другие или все процессоры немедленно выполнить необходимые действия, сгенерировав *межпроцессорное прерывание* (inter-processor interrupt, IPI, также известное как *вызов SMP*, или *перекрестный вызов процессоров*). IPI — это процессорное прерывание, предназначенное для быстрого выполнения, чтобы минимизировать время приостановки других потоков.

IPI также можно использовать для вытеснения.

3.2.15. Вытеснение

Поддержка вытеснения дает возможность высокоприоритетным потокам пользовательского уровня прерывать ядро и выполняться. Это позволяет системам реального времени, например, используемым для управления самолетами или медицинским оборудованием, выполнять работу в определенные периоды времени. Ядро, поддерживающее вытеснение, называется *полностью вытесняемым*, хотя на практике в нем все равно будут иметься некоторые небольшие критические участки кода, работу которых нельзя прервать.

Другой подход, который поддерживается в Linux, — *добровольное вытеснение ядра* (voluntary kernel preemption), когда в ядро добавляются логические точки его приостановки, где проверяются и обрабатываются запросы на вытеснение. Этот подход позволяет избежать некоторых сложностей, связанных с поддержкой полностью вытесняемого ядра, и обеспечивает вытеснение с малой задержкой для обычных рабочих нагрузок. В Linux добровольное вытеснение обычно включается с помощью параметра CONFIG_PREEMPT_VOLUNTARY. Кроме него есть также параметр CONFIG_PREEMPT, позволяющий вытеснять весь код ядра (кроме критических участков), и CONFIG_PREEMPT_NONE, отключающий возможность вытеснения, повышая пропускную способность за счет более высоких задержек.

3.2.16. Управление ресурсами

Операционная система может предоставлять различные средства управления для тонкой настройки доступа к системным ресурсам, таким как процессоры, память, диск и сеть. Их называют *средствами управления ресурсами*. Они могут использоваться для управления производительностью в системах, где выполняются разные приложения или обслуживаются несколько подписчиков (в облачных средах). Такие средства управления могут ограничивать доступность ресурсов на уровне процессов (или групп процессов) или использовать более гибкий подход, позволяющий распределять между потребителями недоиспользованный объем.

Ранние версии Unix и BSD имели базовые средства управления ресурсами на уровне процессов, включая приоритеты, задаваемые с помощью nice(1), и некоторые ограничения ресурсов, задаваемые с помощью ulimit(1).

В 2008 году для Linux были разработаны группы управления (cgroups), интегрированные в ядро в версии 2.6.24. После этого было добавлено множество других средств

управления. Они задокументированы в исходном коде ядра в `Documentation/cgroups`. В 2016 году была разработана улучшенная унифицированная иерархическая схема, получившая название *cgroup v2* и интегрированная в ядро в версии 4.5. Она задокументирована в файле `Documentation/adminguide/cgroup-v2.rst`.

Конкретные средства управления ресурсами будут упоминаться в последующих главах по мере необходимости. Пример их использования для управления производительностью виртуализации ОС описан в главе 11 «Облачные вычисления».

3.2.17. Наблюдаемость

Операционная система состоит из ядра, библиотек и программ. В число программ входят также инструменты для наблюдения за активностью системы и анализа производительности, которые обычно устанавливаются в `/usr/bin` и `/usr/sbin`. В системе также могут быть установлены сторонние инструменты для обеспечения дополнительной наблюдаемости.

Инструменты наблюдения и компоненты ОС, на которых они построены, представлены в главе 4.

3.3. ЯДРА

В следующих разделах обсуждаются детали реализации Unix-подобного ядра с упором на производительность. В качестве фундамента обсуждаются характеристики производительности более ранних ядер: Unix, BSD и Solaris. Ядро Linux обсуждается более подробно в разделе 3.4 «Linux».

Кроме всего прочего, ядра отличаются друг от друга поддерживаемыми файловыми системами (см. главу 8 «Файловые системы»), интерфейсом системных вызовов, архитектурой сетевого стека, поддержкой выполнения в реальном времени и алгоритмами планирования процессорного времени, а также стеком дискового и сетевого ввода/вывода.

В табл. 3.3 для сравнения перечислены Linux и другие версии ядра с количеством поддерживаемых системных вызовов, которые упоминаются в разделе 2 справочного руководства `man`. Это грубое сравнение, но оно позволяет увидеть некоторые различия.

Таблица 3.3. Версии ядер с количеством поддерживаемых системных вызовов

Версия ядра	Количество системных вызовов
UNIX Version 7	48
SunOS (Solaris) 5.11	142
FreeBSD 12.0	222
Linux 2.6.32-21-server	408

Таблица 3.3 (окончание)

Версия ядра	Количество системных вызовов
Linux 2.6.32-220.el6.x86_64	427
Linux 3.2.6-3.fc16.x86_64	431
Linux 4.15.0-66-generic	480
Linux 5.3.0-1010-aws	493

Здесь указано число документированных системных вызовов, но кроме них имеются также скрытые системные вызовы, предназначенные исключительно для использования программным обеспечением операционной системы.

Первоначально в UNIX имелось двадцать системных вызовов, а в современном Linux — прямом потомке — их больше тысячи... Меня просто беспокоит все возрастающая сложность и размер ядра.

Кен Томпсон, ACM Turing Centenary Celebration, 2012 г.

Ядро Linux становится все сложнее и раскрывает эту сложность перед пользователем через системные вызовы и другие интерфейсы. Увеличенная сложность делает обучение, программирование и отладку более трудоемкими.

3.3.1. Unix

Операционная система Unix была разработана Кеном Томпсоном, Деннисом Ричи и другими сотрудниками AT&T Bell Labs в 1969 году и продолжала развиваться ими в последующие годы. Полная история создания описана в книге «The UNIX Time-Sharing System» [Ritchie 74]:

Первая версия была написана, когда один из нас (Томпсон), недовольный имеющимися возможностями компьютеров, обнаружил мало используемый PDP-7 и решил создать более дружелюбное окружение.

Разработчики UNIX уже имели опыт разработки операционной системы Multiplexed Information and Computer Services (Multics). UNIX разрабатывалась как *легкая* многозадачная операционная система и ядро, первоначально названное UNiplexed Information and Computing Service (UNICS) — игра слов от Multics. Вот цитата из «UNIX Implementation» [Thompson 78]:

Ядро — единственный код в UNIX, который пользователь не может заменить по своему желанию. По этой причине ядро должно принимать как можно меньше реальных решений. Это не означает, что пользователю дается миллион способов сделать что-то. Скорее это означает разрешить только один способ, но такой, который является наиболее универсальным из всех возможных вариантов.

Хотя ядро было небольшим, оно поддерживало некоторые средства для обеспечения высокой производительности. Процессам назначались приоритеты, благодаря чему высокоприоритетные задания ненадолго задерживались в очереди на выполнение. Дисковый ввод/вывод выполнялся большими (по 512 байт) блоками и кэшировался в кэше буферов. Неактивные процессы могли выгружаться на диск, чтобы освободить память для более нагруженных процессов. И система, конечно же, была многозадачной, позволяя нескольким процессам работать одновременно и тем самым повышать общую производительность.

Для поддержки сети, нескольких файловых систем, подкачки страниц и других функций, которые мы теперь считаем стандартными, ядро должно было расти и развиваться. А благодаря множеству производных, включая BSD, SunOS (Solaris) и позднее Linux, производительность ядра стала одним из факторов в конкурентной борьбе, что привело к добавлению дополнительных возможностей и кода.

3.3.2. BSD

Операционная система Berkeley Software Distribution (BSD) начиналась как усовершенствование 6-й редакции Unix (Unix 6th Edition) в Калифорнийском университете в Беркли и была выпущена в 1978 году. Поскольку исходный код Unix требовал покупки лицензии на программное обеспечение у AT&T, к началу 1990-х годов весь код, заимствованный из Unix, был переписан в BSD под новой лицензией BSD, допускающей бесплатное распространение ОС семейства BSD, включая FreeBSD.

Вот некоторые основные разработки в ядре BSD, в том числе связанные с производительностью:

- **Выгружаемая виртуальная память:** BSD привнесла в Unix выгружаемую виртуальную память: вместо выгрузки процессов целиком, чтобы освободить основную память, стало возможным выгружать небольшие, редко используемые блоки памяти. См. главу 7 «Память», раздел 7.2.2 «Подкачка страниц».
- **Подкачка страниц по требованию:** откладывает отображение физической памяти в виртуальную до момента первой операции записи, что помогает избежать преждевременных, а иногда и ненужных затрат на размещение в памяти страниц, которые могут никогда не использоваться. Подкачка по требованию была перенесена в Unix из BSD. См. главу 7 «Память», раздел 7.2.3 «Подкачка страниц по требованию».
- **FFS:** быстрая файловая система (Fast File System, FFS) реализовала возможность распределения данных между дисками по группам цилиндров, что значительно уменьшило фрагментацию и повысило производительность вращающихся дисков, а также обеспечило поддержку дисков большего размера и другие улучшения. FFS стала основой для многих других файловых систем, включая UFS. См. главу 8 «Файловые системы», раздел 8.4.5 «Типы файловых систем».
- **Сетевой стек TCP/IP:** для BSD был разработан первый высокопроизводительный сетевой стек TCP/IP, включенный в 4.2BSD (1983). BSD по-прежнему славится производительностью своего сетевого стека.

- **Сокеты:** сокеты Беркли — это API для конечных точек сетевых соединений. Добавленные в 4.2BSD, они стали стандартом работы с сетью. См. главу 10 «Сеть».
- **Клетки (jails):** упрощенная виртуализация на уровне операционной системы, позволяющая нескольким гостевым системам совместно использовать одно ядро. Впервые клетки появились в FreeBSD 4.0.
- **TLS-ядро:** в настоящее время поддержка безопасности транспортного уровня (transport layer security, TLS) широко используется в интернете, поэтому было создано TLS-ядро, выполняющее большую часть обработки TLS, что способствовало повышению производительности [Stewart 15]¹.

ОС BSD не так популярна, как Linux, но часто используется для организации критически важных и высокопроизводительных окружений. Примерами могут служить сеть доставки контента (content delivery network, CDN) в Netflix, а также файловые серверы в NetApp, Isilon и других компаниях. В Netflix высоко оценили производительность FreeBSD как основу своей CDN в 2019 году [Looney 19]:

Используя FreeBSD и стандартные компоненты, мы достигли пропускной способности 90 Гбит/с в соединениях с шифрованием TLS при ~55 %-ной нагрузке на 16-ядерный процессор с тактовой частотой 2,6 ГГц.

Есть отличный справочник по внутреннему устройству FreeBSD: «The Design and Implementation of the FreeBSD Operating System, 2nd Edition» [McKusick 15].

3.3.3. Solaris

Solaris — это операционная система (ОС Unix и ядро на основе BSD), созданная в Sun Microsystems в 1982 году. Первоначально эта операционная система называлась SunOS и была оптимизирована для рабочих станций Sun. К концу 1980-х годов AT&T разработала новый стандарт Unix — Unix System V Release 4 (SVR4), — основанный на технологиях SVR3, SunOS, BSD и Xenix. После этого в Sun было создано новое ядро на основе стандарта SVR4, а операционная система переименована в Solaris.

Вот некоторые основные разработки в ядре Solaris, в том числе связанные с производительностью:

- **VFS:** виртуальная файловая система (virtual file system, VFS) — абстракция и интерфейс, позволяющий легко сосуществовать нескольким файловым системам. Первоначально VFS создавалась в Sun для обеспечения сосуществования NFS и UFS. Подробнее о VFS рассказывается в главе 8 «Файловые системы».
- **Полностью вытесняемое ядро:** обеспечило низкую задержку для высокоприоритетных заданий, включая задания реального времени.

¹ Разработано для увеличения производительности устройств с открытым подключением (Open Connect Appliances, OCA) в Netflix, действующих под управлением FreeBSD и обслуживающих Netflix CDN.

- **Поддержка многопроцессорных систем:** в начале 1990-х Sun вложила значительные средства в поддержку многопроцессорности, разработав поддержку симметричной и асимметричной многопроцессорной обработки (SMP и ASMP) [Mauro 01].
- **Распределитель блоков (slab allocator):** заменив метод выделения памяти «приятель» (buddy allocator) SVR4, распределитель блоков памяти ядра обеспечил лучшую производительность за счет кэширования предварительно выделенных буферов, которые можно было быстро использовать повторно. Этот тип распределителя и его производные стали стандартом для многих ядер, включая Linux.
- **DTrace:** фреймворк и инструмент статической и динамической трассировки, обеспечивающий практически неограниченные возможности наблюдения за всем программным стеком в реальном времени. В Linux для подобных наблюдений имеются BPF и bpftrace.
- **Зоны (Zones):** технология виртуализации на основе операционной системы для создания экземпляров ОС с общим ядром, подобная ранее появившейся в FreeBSD технологии клеток. Виртуализация на уровне ОС в настоящее время широко используется для организации контейнеров в Linux. См. главу 11 «Облачные вычисления».
- **ZFS:** файловая система с функциями и производительностью корпоративного уровня. Теперь она доступна и в других ОС, включая Linux. См. главу 8 «Файловые системы».

В 2010 году Oracle приобрела Sun Microsystems, и теперь Solaris называется Oracle Solaris. Solaris более подробно рассматривается в первом издании этой книги.

3.4. LINUX

ОС Linux была создана в 1991 году Линусом Торвальдсом как бесплатная операционная система для персональных компьютеров Intel. Он объявил о своем проекте в Usenet:

Я делаю (бесплатную) операционную систему (это всего лишь хобби, и она не будет большой и профессиональной, как gnu) для клонов 386 (486) AT. Работа над ней началась в апреле, и скоро она будет готова. Я хотел бы получить любые отзывы о том, что вам нравится или не нравится в minix, потому что моя ОС чем-то похожа на нее (кроме всего прочего, она использует ту же организацию файловой системы (по практическим причинам)).

Линус ссылается на ОС MINIX, которая разрабатывалась как бесплатная небольшая (мини) версия Unix для небольших компьютеров. Создатели BSD тоже стремились создать бесплатную версию Unix, но в то время у них были проблемы с лицензированием.

Ядро Linux разрабатывалось с использованием идей, заимствованных у его предшественников, в том числе:

- **Unix (и Multics):** уровни операционной системы, системные вызовы, многозадачность, процессы, приоритеты процессов, виртуальная память, глобальная файловая система, разрешения файловой системы, узлы устройств, кэш буферов.
- **BSD:** выгружаемая виртуальная память, подкачка по требованию, быстрая файловая система (FFS), сетевой стек TCP/IP, сокет.
- **Solaris:** VFS, NFS, кэш страниц, унифицированный кэш страниц, распределитель блоков.
- **Plan 9:** ветвление ресурсов (rfork) для создания разных уровней совместного использования ресурсов процессами и потоками (задачами).

В настоящее время Linux широко используется на серверах, в облачных экземплярах и на встраиваемых устройствах, включая мобильные телефоны.

3.4.1. Новые разработки в ядре Linux

Далее перечислены новые разработки, появившиеся в ядре Linux, в том числе связанные с производительностью (многие из этих описаний включают версию ядра Linux, в которой эти разработки появились впервые):

- **Классы планирования процессорного времени:** разработаны различные дополнительные алгоритмы планирования процессорного времени, включая домены планирования (2.6.7), для принятия более обоснованных решений в архитектурах с неоднородным доступом к памяти (non-uniform memory access, NUMA). См. главу 6 «Процессоры».
- **Классы планирования ввода/вывода:** разработаны различные алгоритмы планирования блочного ввода/вывода, включая алгоритм по ближайшему сроку завершения (deadline, 2.5.39), упреждающий (anticipatory, 2.5.75), а также абсолютно справедливая организация очередей (Completely Fair Queueing, CFQ; 2.6.6). Они были доступны в ядрах Linux до версии 5.0, но затем были удалены, уступив место новым планировщикам ввода/вывода с несколькими очередями. См. главу 9 «Диски».
- **Алгоритмы управления заторами в протоколе TCP:** Linux позволяет настраивать различные алгоритмы управления заторами в протоколе TCP и поддерживает Reno, Cubic и другие алгоритмы в ядрах, упомянутых в этом списке. См. также главу 10 «Сеть».
- **Overcommit¹:** наряду с компонентом ядра Out-Of-Memory Killer (OOM Killer)² эта стратегия позволяет делать больше с меньшим объемом основной памяти. См. главу 7 «Память».

¹ Стратегия выделения памяти больше имеющегося объема. — *Примеч. пер.*

² Останавливает процессы при критической нехватке памяти. — *Примеч. пер.*

- **Futex (2.5.7)**: сокращение от *fast user-space mutex* (быстрый мьютекс в пространстве пользователя), используется для реализации высокопроизводительных примитивов синхронизации на уровне пользователя.
- **Огромные страницы (2.5.36)**: обеспечивает поддержку заранее выделенных больших страниц памяти ядром и блоком управления памятью (memory management unit, MMU). См. главу 7 «Память».
- **OProfile (2.5.43)**: профилировщик системы, помогающий изучать потребление процессора и другие события как в ядре, так и в приложениях.
- **RCU (2.5.43)**: механизм синхронизации «чтение-модификация-запись» (read-copy-update, RCU), который позволяет выполнять несколько операций чтения параллельно с операциями изменения, повышая производительность и масштабируемость данных, которые чаще читаются, чем изменяются.
- **epoll (2.5.46)**: системный вызов для эффективного ожидания завершения операций ввода/вывода с несколькими открытыми файловыми дескрипторами, повышающий производительность серверных приложений.
- **Модульное планирование ввода/вывода (2.6.10)**: Linux поддерживает подключаемые алгоритмы планирования ввода/вывода для блочных устройств. См. главу 9 «Диски».
- **DebugFS (2.6.11)**: простой неструктурированный интерфейс ядра для передачи данных на уровень пользователя, который используется некоторыми инструментами анализа производительности.
- **Наборы процессоров (cpuset; 2.6.12)**: механизм назначения процессоров процессам.
- **Добровольное вытеснение ядра (2.6.13)**: обеспечивает планирование с малой задержкой без сложностей, связанных с реализацией полного вытеснения.
- **inotify (2.6.13)**: фреймворк для мониторинга событий в файловой системе.
- **blktrace (2.6.17)**: фреймворк и инструмент для трассировки событий блочного ввода/вывода (позже был перенесен в точки трассировки tracepoints).
- **splice (2.6.17)**: системный вызов для быстрого перемещения данных между файловыми дескрипторами и конвейерами без выхода в пространство пользователя. (Системный вызов `sendfile(2)`, который эффективно перемещает данные между файловыми дескрипторами, теперь реализован как обертка вокруг `splice(2)`.)
- **Учет задержек (2.6.18)**: трассировка состояний задержки для каждой задачи. См. главу 4 «Инструменты наблюдения».
- **Учет ввода/вывода (2.6.20)**: измерение различных статистик ввода/вывода для каждого процесса.
- **DynTicks (2.6.21)**: механизм, позволяющий не генерировать прерывания таймера ядра (часов) во время простоя и тем самым экономить электроэнергию и вычислительные ресурсы.
- **SLUB (2.6.22)**: новая и упрощенная версия распределителя блоков памяти.

- **CFS (2.6.23)**: абсолютно справедливый планировщик (Completely Fair Scheduler). См. главу 6 «Процессоры».
- **cgroups (2.6.24)**: группы управления (control groups) позволяют измерять и ограничивать потребление ресурсов для групп процессов.
- **TCP LRO (2.6.24)**: стратегия снижения нагрузки при приеме больших объемов данных по TCP (TCP Large Receive Offload, LRO) позволяет сетевым драйверам и оборудованию объединять принимаемые пакеты в пакеты большего размера перед их передачей в сетевой стек. В Linux также поддерживается стратегия снижения нагрузки при передаче больших объемов данных (Large Send Offload, LSO).
- **latencytop (2.6.25)**: инструмент для наблюдения за источниками задержки в операционной системе.
- **Точки трассировки (tracepoints; 2.6.28)**: статические точки трассировки в ядре (также известные как *статические зонды* — *static probes*), которые инструментуют логические точки выполнения в ядре (раньше назывались *маркерами ядра*) и могут использоваться инструментами трассировки. Инструменты трассировки представлены в главе 4 «Инструменты наблюдения».
- **perf (2.6.31)**: набор инструментов для анализа производительности, включая профилирование счетчиков производительности процессоров, а также статическую и динамическую трассировку. См. главу 6 «Процессоры».
- **No BKL (2.6.37)**: окончательно устраняет узкое место в производительности из-за большой блокировки ядра (Big Kernel Lock, BKL).
- **Прозрачные огромные страницы (2.6.38)**: фреймворк, позволяющий использовать огромные страницы памяти. См. главу 7 «Память».
- **KVM**: виртуальная машина на основе ядра (Kernel-based Virtual Machine, KVM) — технология виртуализации, разработанная для Linux компанией Qumranet, которая была приобретена компанией Red Hat в 2008 году. KVM позволяет создавать экземпляры виртуальных операционных систем с собственными ядрами. См. главу 11 «Облачные вычисления».
- **BPF JIT (3.0)**: динамический (Just-In-Time, JIT) компилятор для пакетного фильтра Беркли (Berkeley Packet Filter, BPF), увеличивающий производительность фильтрации пакетов за счет компиляции байт-кода BPF в машинные инструкции.
- **Управление пропускной способностью CFS (3.2)**: алгоритм планирования процессорного времени с поддержкой квот и регулирования.
- **Функция устранения избыточной сетевой буферизации в TCP (anti-bufferbloat; 3.3+)**: в Linux 3.3 и более поздних версиях были внесены различные улучшения для решения проблемы с избыточной буферизацией, в том числе алгоритм ограничения размеров очередей Byte Queue Limits (BQL), используемый при передаче пакетных данных (3.3), алгоритм управления очередями CoDel (3.5), поддержка маленьких очередей TCP (3.6) и планировщик пакетов с улучшенным пропорционально-интегральным контроллером (Proportional Integral controller Enhanced, PIE) (3.14).

- **uprobes (3.5):** фреймворк для динамической трассировки программного обеспечения пользовательского уровня, используемый другими инструментами (perf, SystemTap и т. д.).
- **Ранняя повторная передача TCP (3.5):** RFC 5827 для уменьшения количества повторяющихся подтверждений, необходимых для быстрого запуска повторной передачи.
- **TFO (3.6, 3.7, 3.13):** алгоритм быстрого открытия соединения TCP (TCP Fast Open, TFO) способен сжать трехэтапное подтверждение соединения TCP до передачи одного пакета SYN с cookie-файлом TFO и тем самым повысить производительность. Этот алгоритм начал использоваться по умолчанию в 3.13.
- **Балансировка NUMA (3.8+):** добавляет возможность автоматической балансировки нагрузки в системах с несколькими узлами NUMA, уменьшения трафика между процессорами ЦП и увеличения производительности.
- **SO_REUSEPORT (3.9):** параметр настройки сокетов, позволяющий нескольким сокетам-приемникам привязываться к одному и тому же порту, что улучшает многопоточную масштабируемость.
- **Устройства SSD в роли кэш-памяти (3.9):** поддержка SSD в механизме отображения устройств с целью использования их в качестве кэша для медленных вращающихся дисков.
- **bcache (3.10):** технология кэширования SSD для блочного интерфейса.
- **TCP TLP (3.10):** TCP Tail Loss Probe (TLP) — это схема, помогающая избежать дорогостоящих повторных передач по таймеру путем отправки новых данных или последнего неподтвержденного сегмента после более короткого тайм-аута проверки, чтобы быстрее запустить процедуру восстановления.
- **NO_HZ_FULL (3.10, 3.12):** параметр ядра, также известный как *многозадачность без таймера*, или *бестактовое ядро*, позволяющий выполнять потоки без приостановки на обработку прерываний часов [Corbet 13a].
- **Блочный ввод/вывод с несколькими очередями (3.13):** создает очереди отправки ввода/вывода для каждого процессора вместо одной общей очереди запросов, улучшая масштабируемость, особенно для устройств SSD с высоким IOPS [Corbet 13b].
- **SCHED_DEADLINE (3.14):** дополнительная стратегия планирования, реализующая алгоритм планирования по ближайшему сроку завершения [Linux 20b].
- **TCP autocorking (3.14):** функция, позволяющая ядру объединять небольшие пакеты и уменьшать общее количество отправляемых пакетов. Автоматизированная версия функции TCP_CORK setsockopt(2).
- **Блокировки MCS и быстрые спин-блокировки (3.15):** эффективные блокировки ядра с использованием таких методов, как создание структур для каждого процессора. Название MCS дано в честь изобретателей такой блокировки (Меллор—Крамми и Скотт) [Mellor-Crummey 91], [Corbet 14].

- **Расширенный BPF (3.18+)**: окружение выполнения в ядре для запуска программ в режиме ядра. Основная часть расширенного BPF была добавлена в серию 4.x. Поддержка подключения к kprobes была добавлена в 3.19, к точкам трассировки `tracerepoints` — в 4.7, к программным и аппаратным событиям — в 4.9 и к группам управления (`cgroups`) — в 4.10. В 5.3 были добавлены ограниченные циклы, что расширило возможности создания сложных приложений. См. раздел 3.4.4 «Расширенный BPF».
- **Overlayfs (3.18)**: каскадная файловая система, включенная в Linux. Создает виртуальные файловые системы поверх других, которые также можно изменять без изменения первых. Часто используется для контейнеров.
- **DCTCP (3.18)**: алгоритм управления заторами в центрах обработки данных (Data Center TCP, DCTCP), нацеленный на обеспечение высокой устойчивости к всплескам, малой задержки и высокой пропускной способности [Borkmann 14a].
- **DAX (4.0)**: прямой доступ (Direct Access, DAX) позволяет из пользовательского пространства напрямую читать данные с устройств постоянного хранения без дополнительных затрат на буферизацию. Файловая система `ext4` поддерживает DAX.
- **Очереди спин-блокировок (4.2)**: предлагают лучшую производительность в условиях значительной конкуренции; стали использоваться по умолчанию, начиная с версии 4.2.
- **Приемник TCP без блокировки (4.4)**: из пути к приемнику TCP были убраны блокировки, что повысило производительность.
- **cgroup v2 (4.5, 4.15)**: унифицированная иерархическая схема для организации групп управления присутствовала и в более ранних ядрах, но только в 4.5 была признана стабильной и получила название `cgroup v2` [Нео 15]. Контроллер процессора в `cgroup v2` был добавлен в 4.15.
- **Масштабирование `epoll` (4.5)**: многопоточное масштабирование `epoll(7)` помогает избежать одновременного возобновления работы всех потоков, ожидающих на одном и том же файловом дескрипторе при каждом событии, что вызывало проблему с производительностью, известную как «громовое стадо» [Corbet 15].
- **КСМ (4.6)**: мультиплексор соединений ядра (Kernel Connection Multiplexor, КСМ) обеспечивает эффективный интерфейс обмена сообщениями через TCP.
- **TCP NV (4.8)**: New Vegas (NV) — новый алгоритм управления заторами в TCP, подходящий для сетей с высокой пропускной способностью (которые работают на скоростях выше 10 Гбит/с).
- **XDP (4.8, 4.18)**: eXpress Data Path (XDP) — программируемый высокопроизводительный путь к данным на основе BPF для высокопроизводительных сетей [Herbert 16]. Семейство адресов сокетов `AF_XDP`, которое может работать в обход большей части сетевого стека. Добавлено в 4.18.
- **TCP BBR (4.9)**: Bottleneck Bandwidth and RTT (BBR) — алгоритм управления заторами в TCP, который обеспечивает меньшую задержку и лучшую пропускную способность в сетях, страдающих от потерь пакетов и переполнения буфера [Cardwell 16].

- **Аппаратный трассировщик задержки (4.9):** трассировщик Ftrace, способный обнаруживать задержки, вызванные аппаратным и микропрограммным обеспечением, включая прерывания управления системой (System Management Interrupts, SMI).
- **perf c2c (4.10):** подкоманда «cache-to-cache» (c2c) трассировщика perf может помочь выявить проблемы с производительностью кэш-памяти процессора, включая ложное совместное использование.
- **Intel CAT (4.10):** поддержка технологии выделения кэшей Intel (Cache Allocation Technology, CAT), позволяющая задачам выделять кэш-память процессора. Может использоваться контейнерами, чтобы помочь с проблемой «шумного соседа».
- **Планировщики ввода/вывода с несколькими очередями: BPQ, Kyber (4.12):** планировщик ввода/вывода с равномерным распределением бюджета (Budget Fair Queueing, BFQ) обеспечивает ввод/вывод с низкой задержкой для интерактивных приложений, особенно при использовании медленных устройств хранения. BFQ был значительно усовершенствован в версии 5.2. Планировщик ввода/вывода Kyber подходит для быстрых устройств с несколькими очередями [Corbet 17].
- **TLS-ядро (4.13, 4.17):** TLS-версия ядра Linux [Edge 15].
- **MSG_ZEROCOPY (4.14):** флаг для системного вызова send(2), позволяющий предотвращать избыточное копирование байтов пакета между приложением и сетевым интерфейсом [Linux 20c].
- **PCID (4.14):** поддержка идентификатора контекста процесса (Process-Context ID, PCID) — функция управления памятью (MMU) процессора, помогающая избежать сброса буфера ассоциативной трансляции (Translation Lookaside Buffer, TLB) при переключении контекста. Внедрение этой функции снизило влияние на производительность исправлений в механизме изоляции таблицы страниц ядра (Kernel Page Table Isolation, KPTI), необходимых для устранения уязвимости Meltdown. См. раздел 3.4.3 «KPTI (Meltdown)».
- **PSI (4.20, 5.2):** Pressure Stall Information (PSI) — это набор новых метрик, показывающих время, затраченное в ожидании доступности процессора, памяти или ввода/вывода. В версии 5.2 были добавлены уведомления о достижении порогов PSI для поддержки мониторинга PSI.
- **TCP EDT (4.20):** алгоритм ранней отправки (Early Departure Time, EDT): использует планировщик с колесом времени для отправки пакетов, уменьшает потребление процессора и памяти для очередей [Jacobson 18].
- **Ввод/вывод с несколькими очередями (5.0):** в версии 5.0 по умолчанию стали использоваться планировщики блочного ввода/вывода с несколькими очередями, а классические планировщики были удалены.
- **UDP GRO (5.0):** универсальный механизм уменьшения затрат на прием UDP (Generic Receive Offload, GRO) повышает производительность; позволяет объединять пакеты в драйвере и сетевой карте и передавать их в стек.

- **io_uring (5.1)**: универсальный асинхронный интерфейс для быстрых взаимодействий между приложениями и ядром с использованием общих кольцевых буферов. Основные области применения: быстрый дисковый и сетевой ввод/вывод.
- **MADV_COLD, MADV_PAGEOUT (5.4)**: эти флаги в системном вызове `madvise(2)` подсказывают ядру, что в ближайшее время потребуется память. `MADV_PAGEOUT` также подсказывает, что память будет немедленно освобождена. Такие подсказки особенно полезны для встраиваемых устройств с Linux и ограниченным объемом памяти.
- **MultiPath TCP (5.6)**: позволяет использовать несколько сетевых каналов (например, 3G и Wi-Fi) для повышения производительности и надежности одного TCP-соединения.
- **Трассировка времени загрузки (5.6)**: позволяет `Ftrace` трассировать процесс начальной загрузки. (Характеристики поздней загрузки может предоставить `systemd`: см. раздел 3.4.2 «system».)
- **Температурное давление (5.7)**: позволяет планировщику учитывать регулирование частоты процессора в зависимости от температуры, чтобы принимать более сбалансированные решения.
- **Флейм-графики perf (5.8)**: `perf(1)` поддерживает создание флейм-графиков.

В этот список не попало множество других небольших улучшений производительности блокировок, драйверов, VFS, файловых систем, асинхронного ввода/вывода, распределителей памяти, NUMA, поддержки новых инструкций процессора, графических процессоров и инструментов анализа производительности `perf(1)` и `Ftrace`. Также за счет использования `systemd` было улучшено время загрузки системы.

Далее я подробно опишу три улучшения в Linux, особенно важные для производительности: `systemd`, KPTI и расширенный BPF.

3.4.2. systemd

`systemd` — системный диспетчер сервисов, широко используемый в Linux. Был разработан как замена `init` — первоначальной системы инициализации в UNIX. `systemd` выполняет множество разных функций, включая запуск сервисов с учетом зависимостей и сбор статистики о времени работы сервисов.

Иногда перформанс-инженерам приходится заниматься настройкой времени загрузки системы, и метрики, которые собирает диспетчер `system`, могут подсказать, где и что можно настроить. Общее время загрузки можно узнать с помощью `systemd-analyze(1)`:

```
# systemd-analyze
```

```
Startup finished in 1.657s (kernel) + 10.272s (userspace) = 11.930s
graphical.target reached after 9.663s in userspace
```

Как показывает этот вывод, система загрузилась (в данном случае достигла точки `graphical.target`) за 9,663 с. Дополнительную информацию можно получить с помощью подкоманды `critical-chain`:

`systemd-analyze critical-chain`

The time when unit became active or started is printed after the "@" character.
The time the unit took to start is printed after the "+" character.

```
graphical.target @9.663s
├─multi-user.target @9.661s
│   └─snapd.seeded.service @9.062s +62ms
│       └─basic.target @6.336s
│           ├──sockets.target @6.334s
│           │   └─snapd.socket @6.316s +16ms
│           └─sysinit.target @6.281s
│               └─cloud-init.service @5.361s +905ms
│                   └─systemd-networkd-wait-online.service @3.498s +1.860s
│                       └─systemd-networkd.service @3.254s +235ms
│                           └─network-pre.target @3.251s
│                               └─cloud-init-local.service @2.107s +1.141s
│                                   └─systemd-remount-fs.service @391ms +81ms
│                                       └─systemd-journald.socket @387ms
│                                           └─system.slice @366ms
│                                               └─.slice @366ms
```

Здесь показан *критический путь*: последовательность шагов (в данном случае запуск сервисов), вызывающих задержку. Дольше всего запускался сервис `systemd-networkd-wait-online.service` — 1,86 с.

В `system` есть и другие полезные подкоманды: `blame` показывает самое медленное время инициализации, а `plot` создает диаграмму в формате SVG. Дополнительную информацию см. на странице `systemd-analysis(1)` в справочном руководстве `man`.

3.4.3. KPTI (Meltdown)

Исправления в механизме изоляции таблицы страниц ядра (kernel page table isolation, KPTI), добавленные в Linux 4.14 в 2018 году, помогают смягчить уязвимость процессоров Intel под названием «meltdown» (расплавление). В более старых версиях ядра Linux аналогичную цель преследовали исправления KAISER. В других ядрах тоже были свои средства защиты. Они решают проблему безопасности, но при этом снижают производительность процессора из-за расходования добавочных тактов процессора на дополнительную очистку буфера ассоциативной трансляции TLB при переключениях контекста и обращении к системным вызовам. В Linux в той же версии ядра была добавлена поддержка идентификатора контекста процесса (PCID), помогающая избежать избыточной очистки TLB, при условии, что процессор поддерживает `pcid`.

Я оцениваю влияние KPTI на производительность для рабочих нагрузок в облачной среде Netflix в диапазоне от 0,1 до 6 %, в зависимости от частоты обращений к системным вызовам из рабочей нагрузки (чем выше частота, тем больше потеря производительности) [Gregg 18a]. Снизить затраты помогают дополнительные

настройки: использование больших страниц помогает быстрее «разогреть» буфер TLB после очистки, а с помощью инструментов трассировки можно выявить наиболее часто используемые системные вызовы и потом определить пути снижения частоты их вызова. Несколько инструментов трассировки реализованы с использованием расширенного BPF.

3.4.4. Расширенный BPF

Название BPF происходит от Berkeley Packet Filter (фильтр пакетов Беркли), малоизвестной технологии, разработанной в 1992 году, которая улучшила производительность инструментов перехвата пакетов [McCanne 92]. В 2013 году Алексей Старовойтов предложил существенно измененную версию BPF [Starovoitov 13], которая была доработана им и Даниэлем Боркманном и включена в ядро Linux в 2014 году [Borkmann 14b]. Это превратило BPF в универсальный механизм выполнения, который можно использовать для решения множества задач, включая организацию сетевых взаимодействий, наблюдение и поддержание безопасности.

BPF — это гибкая и эффективная технология, определяющая набор инструкций, объектов хранения (ассоциативных массивов, или карт) и вспомогательных функций. Из-за поддержки виртуальных инструкций BPF можно рассматривать как виртуальную машину. Программы для BPF работают в режиме ядра (как было показано на рис. 3.2) и запускаются по событиям: сокетов, точек, зондов USDT, kprobes, uprobes и perf_events. Все они показаны на рис. 3.16.

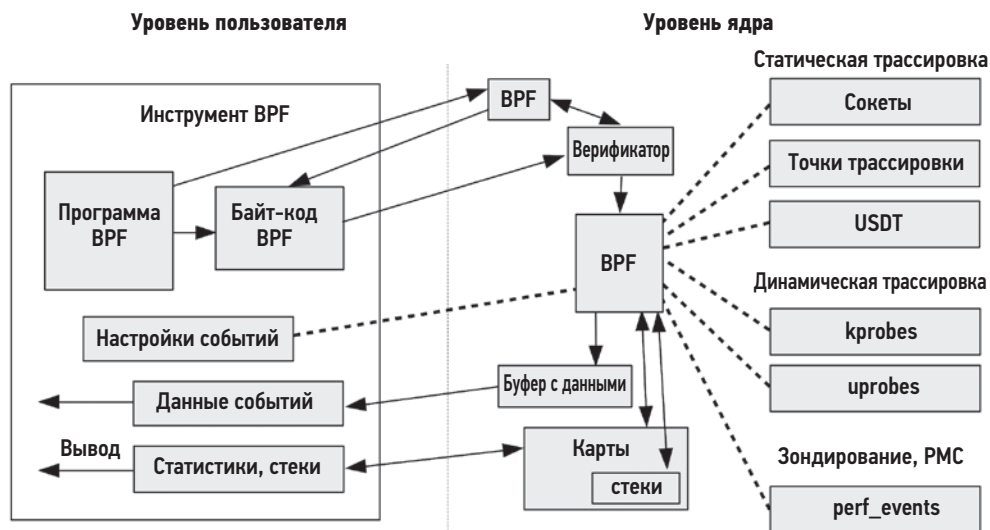


Рис. 3.16. Компоненты BPF

Байт-код BPF сначала попадает в верификатор, который проверяет его безопасность и гарантирует, что программа BPF не приведет к сбою или повреждению ядра.

Он также может использовать систему представления типов (BPF Type Format, BTF) для определения типов и структур данных. Программы BPF могут выводить данные через кольцевой буфер `perf`, эффективный способ передачи данных о каждом событии, или через карты (ассоциативные массивы), которые лучше всего подходят для передачи статистик.

Поскольку BPF — это основа для нового поколения эффективных и безопасных инструментов трассировки, этот механизм занимает важное место в сфере анализа производительности. Он обеспечивает возможность программирования существующих источников событий ядра: точек трассировки, зондов `kprobes` и `uprobes`, а также событий производительности `perf_events`. Программа BPF может, например, записать отметку времени в начале и в конце ввода/вывода, чтобы дать возможность определить продолжительность операции, или сохранить эти данные в гистограмме. В этой книге приводится множество программ для BPF, использующих интерфейсы BCC и `bpftrace`. Мы рассмотрим их в главе 15.

3.5. ДРУГИЕ ТЕМЫ

В числе других тем, касающихся ядра и операционных систем, которые заслуживают внимания, можно назвать: ядра PGO, одноцелевые ядра (`unikernel`), микроядра, гибридные ядра и распределенные операционные системы.

3.5.1. Ядра PGO

Профильная оптимизация (`profile-guided optimization`, PGO), также известная как оптимизация с обратной связью (`feedback-directed optimization`, FDO), основана на использовании информации профилирования процессора для улучшения решений компилятора [Yuan 14a]. Вот примерная процедура применения такой оптимизации для сборки ядра:

1. Во время промышленной эксплуатации системы собрать информацию о потреблении процессора (профиль).
2. Скомпилировать ядро, опираясь на эту информацию.
3. Развернуть новое ядро в продакшене.

Эта процедура создаст ядро с улучшенной производительностью для конкретной рабочей нагрузки. Некоторые окружения времени выполнения, такие как JVM, делают это автоматически, перекомпилируя методы Java после оценки их производительности во время выполнения в сочетании с JIT-компиляцией. В отличие от них, процесс профильной оптимизации ядра включает ручные шаги.

Оптимизации компиляции сопутствует оптимизация времени компоновки (`link-time optimization`, LTO), когда двоичный файл компилируется целиком, чтобы обеспечить оптимизацию всей программы. Ядро Microsoft Windows активно использует и LTO, и PGO, получая прирост производительности от 5 до 20 % по сравнению

с PGO [Bearman 20]. Google тоже использует ядра LTO и PGO для повышения производительности [Tolvanen 20].

Компиляторы gcc и clang, а также само ядро Linux поддерживают профильную оптимизацию. Обычно процесс профильной оптимизации включает запуск специально инструментированного ядра для сбора данных профилирования. Компания Google выпустила инструмент AutoFDO, который позволяет обойтись без такого специального ядра: AutoFDO может собирать информацию из обычного ядра с помощью perf(1) и затем преобразовывать ее в формат, пригодный для использования компиляторами [Google 20a].

Единственная достаточно свежая информация о создании ядра Linux с PGO или AutoFDO — это доклады с конференции Linux Plumber Conference 2020 представителей компаний Microsoft [Bearman 20] и Google [Tolvanen 20]¹.

3.5.2. Одноцелевые ядра

Одноцелевое ядро (unikernel) — это образ машины с одним приложением, объединяющий ядро, библиотеку и прикладное ПО, которые обычно могут выполняться в едином адресном пространстве на аппаратной виртуальной машине или на «голом железе». Такие ядра имеют потенциальные преимущества с точки зрения производительности и безопасности: меньшее количество машинных инструкций, что предполагает более высокий коэффициент попадания в кэш процессора и меньшее количество уязвимостей безопасности. Но такие ядра также создают специфические проблемы: вам могут быть недоступны ни SSH, ни командные оболочки, ни инструменты анализа производительности, и вы можете не иметь возможности войти в систему и выполнить действия по анализу и отладке работы системы.

Чтобы одноцелевые ядра можно было настроить, нужно создать новые инструменты и метрики производительности для них. В качестве доказательства жизнеспособности этой идеи я создал простой профилировщик процессорного времени, который запускался из Xen dom0 и выполнял профилирование гостевого одноцелевого ядра domU, а затем строил флейм-график процессорного времени [Gregg 16a].

Примером одноцелевых ядер может служить MirageOS [MirageOS 20].

3.5.3. Микроядра и гибридные ядра

Большая часть этой главы посвящена Unix-подобным ядрам, которые еще называют *монолитными ядрами*. Весь код, управляющий устройствами в таких ядрах, выполняется вместе как одна большая программа. В модели *микроядер* программное обеспечение, составляющее ядро, уменьшено до минимума. Микроядро поддерживает только самые основные функции, такие как управление памятью и потоками выполнения и межпроцессные взаимодействия (Inter-Process Communication, IPC).

¹ Самая последняя документация была выпущена для Linux 3.13 [Yuan 14b] в 2014 году, что препятствовало внедрению новых ядер.

Файловые системы, сетевой стек и драйверы реализованы как ПО пользовательского режима, что упрощает изменение и замену этих компонентов. Представьте, что у вас есть возможность выбрать не только то, какую базу данных или веб-сервер установить, но и какой сетевой стек использовать. Микроядро также более устойчиво к отказам: сбой в драйвере не приводит к сбою всего ядра. Примерами микроядер могут служить QNX и Minix 3.

Недостатком микроядер является необходимость дополнительных шагов ИРС для выполнения ввода/вывода и других функций, что, конечно же, отрицательно сказывается на производительности. Одно из решений этой проблемы — *гибридные ядра*, сочетающие преимущества микроядер и монолитных ядер. В модели гибридных ядер все критически важные для производительности службы выполняются в пространстве ядра (и вызывают функции напрямую, минуя механизмы ИРС), как в монолитных ядрах, но имеют модульную организацию и отказоустойчивость микроядер. Примерами гибридных ядер могут служить ядро Windows NT и ядро Plan 9.

3.5.4. Распределенные операционные системы

Распределенная операционная система выполняется как единый экземпляр на множестве компьютеров, объединенных в сеть. Обычно на каждом из компьютеров используется микроядро. Примерами распределенных ОС могут служить Plan 9 компании Bell Labs и Inferno.

Несмотря на новаторский дизайн, эта модель не получила широкого распространения. Роб Пайк (Rob Pike), один из создателей Plan 9 и Inferno, описал некоторые причины этой неудачи [Pike 00]:

В конце 1970-х — начале 1980-х годов было заявлено, что Unix уничтожила исследования операционных систем, потому что у многих пропало желание попробовать что-нибудь другое. Тогда я не поверил. Сегодня я вынужден согласиться с тем, что это утверждение может быть правдой (несмотря на Microsoft).

В современных облачных средах распространенной моделью масштабирования является балансировка нагрузки между наборами идентичных экземпляров ОС, число которых может меняться в зависимости от нагрузки (см. главу 11 «Облачные вычисления», раздел 11.1.3 «Планирование мощности»).

3.6. СРАВНЕНИЕ ЯДЕР

Какое ядро самое быстрое? Ответ на этот вопрос отчасти зависит от конфигурации ОС и рабочей нагрузки, а также от того, насколько ядро вовлечено в работу. Вообще я ожидаю, что Linux превзойдет другие ядра благодаря активной работе, направленной на повышение производительности, поддержку приложений и драйверов, а также широкому использованию и обширному сообществу, которое выискивает

проблемы с производительностью и сообщает о них. Список TOP500 самых мощных суперкомпьютеров, который ведется с 1993 года, в 2017 году стал на 100 % Linux [TOP500 17]. Но останутся и некоторые исключения; например, Netflix использует Linux в облаке и FreeBSD для своей сети доставки контента (CDN)¹.

Производительность ядер обычно сравнивается с помощью микробенчмарков, но этот подход подвержен ошибкам. Бенчмарки могут обнаружить, что одно ядро намного быстрее в определенном системном вызове, но этот системный вызов не используется рабочей нагрузкой. (Или используется, но с наборами флагов, не используемых микробенчмарком, которые сильно влияют на производительность.) Точное сравнение производительности ядра — это задача старшего перформанс-инженера, и она может занять недели. См. главу 12 «Бенчмаркинг», раздел 12.3.2 «Активный бенчмаркинг», где описывается одна из возможных методологий.

В первом издании этой книги этот раздел я завершил замечанием о том, что в Linux нет развитых средств динамической трассировки, без которых можно упустить большой выигрыш в производительности. С тех пор я вплотную занялся Linux и помог разработать динамические трассировщики, которых так не хватало: BCC и bpftrace, основанные на расширенном BPF. Они описаны в главе 15 и в моей предыдущей книге [Gregg 19].

В разделе 3.4.1 «Новые разработки в ядре Linux» перечислены многие другие изменения, направленные на улучшение производительности, которые произошли между первым и текущим изданием этой книги, включая изменения в версиях ядра 3.1 и 5.8. Важным событием, не отмеченным выше, является включение в основную ветку ядра Linux поддержки OpenZFS — высокопроизводительной и зрелой файловой системы.

Развитие ядра Linux сопряжено со множеством сложностей. В нем так много механизмов и параметров настройки, что стало очень сложно настраивать их для каждой рабочей нагрузки. Я был свидетелем множества случаев, когда ядро разворачивалось без настройки. Помните об этом, сравнивая производительность ядер: все ли ядра были правильно настроены? Последующие главы в этой книге и разделы, посвященные настройке, помогут вам.

3.7. УПРАЖНЕНИЯ

1. Ответьте на вопросы, касающиеся терминологии операционных систем:
 - В чем разница между процессом, потоком и задачей?
 - Что такое переключение режима и переключение контекста?

¹ FreeBSD обеспечивает более высокую производительность для рабочей нагрузки Netflix CDN и, в частности, благодаря улучшениям, внесенным командой Netflix OCA. Ядра регулярно тестируются, например, в последний раз тестирование в продакшене состоялось в 2019 году. Тогда сравнивались Linux 5.0 и FreeBSD, и я помогал анализировать полученные результаты.

- В чем разница между подкачкой страниц и подкачкой процессов?
 - В чем разница между рабочими нагрузками с интенсивным вводом/выводом и вычислительными нагрузками?
2. Ответьте на концептуальные вопросы:
 - Опишите роль ядра.
 - Опишите роль системных вызовов.
 - Опишите роль VFS и ее положение в стеке ввода/вывода.
 3. Ответьте на вопросы «со звездочкой»:
 - Перечислите причины, почему поток выполнения может оставить процессор.
 - Опишите преимущества виртуальной памяти и подкачки по требованию.

3.8. ССЫЛКИ

- [Graham 68] Graham, B., «Protection in an Information Processing Utility», Communications of the ACM, May 1968.
- [Ritchie 74] Ritchie, D. M., and Thompson, K., «The UNIX Time-Sharing System», Communications of the ACM 17, no. 7, pp. 365–75, July 1974.
- [Thompson 78] Thompson, K., «UNIX Implementation», Bell Laboratories, 1978.
- [Bach 86] Bach, M. J., «The Design of the UNIX Operating System», Prentice Hall, 1986.
- [Mellor-Crummey 91] Mellor-Crummey, J. M., and Scott, M., «Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors», ACM Transactions on Computing Systems, Vol. 9, No. 1, https://www.cs.rochester.edu/u/scott/papers/1991_TOCS_synch.pdf, 1991.
- [McCanne 92] McCanne, S., and Jacobson, V., «The BSD Packet Filter: A New Architecture for User-Level Packet Capture», USENIX Winter Conference, 1993.
- [Mills 94] Mills, D., «RFC 1589: A Kernel Model for Precision Timekeeping», Network Working Group, 1994.
- [Lever 00] Lever, C., Eriksen, M. A., and Molloy, S. P., «An Analysis of the TUX Web Server», CITI Technical Report 00-8, <http://www.citi.umich.edu/techreports/reports/citi-tr-00-8.pdf>, 2000.
- [Pike 00] Pike, R., «Systems Software Research Is Irrelevant», http://doc.cat-v.org/bell_labs/utah2000/utah2000.pdf, 2000.
- [Mauro 01] Mauro, J., and McDougall, R., «Solaris Internals: Core Kernel Architecture», Prentice Hall, 2001.
- [Bovet 05] Bovet, D., and Cesati, M., «Understanding the Linux Kernel, 3rd Edition», O'Reilly, 2005¹.
- [Corbet 05] Corbet, J., Rubini, A., and Kroah-Hartman, G., «Linux Device Drivers», 3rd Edition, O'Reilly, 2005.

¹ Бовет Д., Чезати М. «Ядро Linux».

- [Corbet 13a] Corbet, J., «Is the whole system idle?» LWN.net, <https://lwn.net/Articles/558284>, 2013.
- [Corbet 13b] Corbet, J., «The multiqueue block layer», LWN.net, <https://lwn.net/Articles/552904>, 2013.
- [Starovoitov 13] Starovoitov, A., «[PATCH net-next] extended BPF», Linux kernel mailing list, <https://lkml.org/lkml/2013/9/30/627>, 2013.
- [Borkmann 14a] Borkmann, D., «net: tcp: add DCTCP congestion control algorithm», <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=e3118e8359bb7c59555aca60c725106e6d78c5ce>, 2014.
- [Borkmann 14b] Borkmann, D., «[PATCH net-next 1/9] net: filter: add jited flag to indicate jit compiled filters», netdev mailing list, <https://lore.kernel.org/netdev/1395404418-25376-1-git-send-email-dborkman@redhat.com/T>, 2014.
- [Corbet 14] Corbet, J., «MCS locks and qspinlocks», LWN.net, <https://lwn.net/Articles/590243>, 2014.
- [Drysdale 14] Drysdale, D., «Anatomy of a system call, part 2», LWN.net, <https://lwn.net/Articles/604515>, 2014.
- [Yuan 14a] Yuan, P., Guo, Y., and Chen, X., «Experiences in Profile-Guided Operating System Kernel Optimization», APSys, 2014.
- [Yuan 14b] Yuan P., Guo, Y., and Chen, X., «Profile-Guided Operating System Kernel Optimization», <http://coolypf.com>, 2014.
- [Corbet 15] Corbet, J., «Epoll evolving», LWN.net, <https://lwn.net/Articles/633422>, 2015.
- [Edge 15] Edge, J., «TLS in the kernel», LWN.net, <https://lwn.net/Articles/666509>, 2015.
- [Heo 15] Heo, T., «Control Group v2», Linux documentation, <https://www.kernel.org/doc/Documentation/cgroup-v2.txt>, 2015.
- [McKusick 15] McKusick, M. K., Neville-Neil, G. V., and Watson, R. N. M., «The Design and Implementation of the FreeBSD Operating System, 2nd Edition», Addison-Wesley, 2015¹.
- [Stewart 15] Stewart, R., Gurney, J. M., and Long, S., «Optimizing TLS for High-Bandwidth Applications in FreeBSD», AsiaBSDCon, https://people.freebsd.org/~rrs/asiabsd_2015_tls.pdf, 2015.
- [Cardwell 16] Cardwell, N., Cheng, Y., Stephen Gunn, C., Hassas Yeganeh, S., and Jacobson, V., «BBR: Congestion-Based Congestion Control», ACM queue, <https://queue.acm.org/detail.cfm?id=3022184>, 2016.
- [Gregg 16a] Gregg, B., «Unikernel Profiling: Flame Graphs from dom0», <http://www.brendangregg.com/blog/2016-01-27/unikernel-profiling-from-dom0.html>, 2016.
- [Herbert 16] Herbert, T., and Starovoitov, A., «eXpress Data Path (XDP): Programmable and High Performance Networking Data Path», https://github.com/iovisor/bpf-docs/raw/master/Express_Data_Path.pdf, 2016.
- [Corbet 17] Corbet, J., «Two new block I/O schedulers for 4.12», LWN.net, <https://lwn.net/Articles/720675>, 2017.

¹ Маккузик М. К., Невилл-Нил Дж. В. «FreeBSD. Архитектура и реализация».

- [TOP500 17] TOP500, «List Statistics», <https://www.top500.org/statistics/list>, 2017.
- [Gregg 18a] Gregg, B., «KPTI/KAISER Meltdown Initial Performance Regressions», <http://www.brendangregg.com/blog/2018-02-09/kpti-kaiser-meltdown-performance.html>, 2018.
- [Jacobson 18] Jacobson, V., «Evolving from AFAP: Teaching NICs about Time», netdev 0x12, <https://netdevconf.info/0x12/session.html?evolving-from-afap-teaching-nics-about-time>, 2018.
- [Gregg 19] Gregg, B., «BPF Performance Tools: Linux System and Application Observability»¹, Addison-Wesley, 2019.
- [Looney 19] Looney, J., «Netflix and FreeBSD: Using Open Source to Deliver Streaming Video», FOSDEM, https://papers.freebsd.org/2019/fosdem/looney-netflix_and_freebsd, 2019.
- [Bearman 20] Bearman, I., «Exploring Profile Guided Optimization of the Linux Kernel», Linux Plumber's Conference, <https://linuxplumbersconf.org/event/7/contributions/771>, 2020.
- [Google 20a] Google, «AutoFDO», <https://github.com/google/autofdo>, по состоянию на 2020.
- [Linux 20a] «NO_HZ: Reducing Scheduling-Clock Ticks», Linux documentation, https://www.kernel.org/doc/html/latest/timers/no_hz.html, по состоянию на 2020.
- [Linux 20b] «Deadline Task Scheduling», Linux documentation, <https://www.kernel.org/doc/Documentation/scheduler/sched-deadline.rst>, по состоянию на 2020.
- [Linux 20c] «MSG_ZEROCOPY», Linux documentation, https://www.kernel.org/doc/html/latest/networking/msg_zerocopy.html, по состоянию на 2020.
- [Linux 20d] «Softlockup Detector and Hardlockup Detector (aka nmi_watchdog)», Linux documentation, <https://www.kernel.org/doc/html/latest/admin-guide/lockup-watchdogs.html>, по состоянию на 2020.
- [MirageOS 20] MirageOS, «Mirage OS», <https://mirage.io>, по состоянию на 2020.
- [Owens 20] Owens, K., et al., «4. Kernel Stacks», Linux documentation, <https://www.kernel.org/doc/html/latest/x86/kernel-stacks.html>, по состоянию на 2020.
- [Tolvanen 20] Tolvanen, S., Wendling, B., and Desaulniers, N., «LTO, PGO, and AutoFDO in the Kernel», Linux Plumber's Conference, <https://linuxplumbersconf.org/event/7/contributions/798>, 2020.

3.8.1. Дополнительное чтение

Операционные системы и их ядра — увлекательная и обширная тема. В этой главе я привел только самые основные сведения. В дополнение к упоминавшимся в этой главе источникам посмотрите следующие материалы по ОС на базе Linux и другим:

- [Goodheart 94] Goodheart, B., and Cox J., «The Magic Garden Explained: The Internals of UNIX System V Release 4, an Open Systems Design», Prentice Hall, 1994.
- [Vahalia 96] Vahalia, U., «UNIX Internals: The New Frontiers», Prentice Hall, 1996².

¹ Грегг Б. «BPF: Профессиональная оценка производительности». Выходит в издательстве «Питер» в 2023 году.

² Вахалия Ю. «UNIX изнутри». СПб., издательство «Питер».

- [Singh 06] Singh, A., «Mac OS X Internals: A Systems Approach», Addison-Wesley, 2006.
- [McDougall 06b] McDougall, R., and Mauro, J., «Solaris Internals: Solaris 10 and OpenSolaris Kernel Architecture», Prentice Hall, 2006.
- [Love 10] Love, R., «Linux Kernel Development, 3rd Edition», Addison-Wesley, 2010¹.
- [Tanenbaum 14] Tanenbaum, A., and Bos, H., «Modern Operating Systems, 4th Edition», Pearson, 2014².
- [Yosifovich 17] Yosifovich, P., Ionescu, A., Russinovich, M. E., and Solomon, D. A., «Windows Internals, Part 1 (Developer Reference), 7th Edition», Microsoft Press, 2017³.

¹ Лав Р. «Ядро Linux: описание процесса разработки. 3-е издание».

² Таненбаум Э., Бос Х. «Современные операционные системы». СПб., издательство «Питер».

³ Русинович М., Соломон Д. «Внутреннее устройство Windows». СПб., издательство «Питер».

Глава 4

ИНСТРУМЕНТЫ НАБЛЮДЕНИЯ

Операционные системы традиционно предлагали множество инструментов для наблюдения за работой системного ПО и аппаратных компонентов. Широкий спектр доступных инструментов и метрик заставляет новичков думать, что все или, по крайней мере, все важное можно наблюдать. На самом деле пробелов очень много, и анализ производительности систем превратился в искусство интерпретации и выводов: решения о необходимых действиях принимаются на основе косвенных инструментов и статистик. Например, сетевые пакеты можно исследовать по отдельности, а дисковый ввод/вывод — нет (по крайней мере, выделить отдельные блоки очень нелегко).

Благодаря появлению инструментов динамической трассировки, включая ВСС и `bpfttrace` на основе BPF, наблюдаемость в Linux значительно улучшилась. Темные углы теперь неплохо подсвечиваются, в том числе появилась возможность наблюдать отдельные операции дискового ввода/вывода с помощью `biosnoop`(8). Но многие компании и коммерческие продукты мониторинга еще не внедрили трассировку системы и упускают из виду открывающиеся возможности. Я был первым, кто разработал, опубликовал и объяснил новые инструменты трассировки, которые уже используются в том числе Netflix и Facebook.

Цели этой главы:

- определить инструменты статического анализа производительности и кризисных ситуаций;
- познакомиться с типами инструментов и их оверхедом: подсчет, профилирование и трассировка;
- перечислить источники информации для наблюдения, включая: `/proc`, `/sys`, точки трассировки, `kprobes`, `uprobes`, `USDT` и `PMC`;
- показать, как настроить `sar`(1) для архивирования статистики.

В главе 1 я уже перечислил виды инструментов: подсчет, профилирование и трассировка, а также упомянул статические и динамические инструменты. В этой главе я подробно расскажу об инструментах наблюдения и их источниках данных, в том числе об инструменте `sar`(1), формирующем отчеты о деятельности системы, и кратко — об инструментах трассировки. Это даст базис для понимания наблюдаемости

в Linux. В последующих главах (6–11) я расскажу, как использовать инструменты и источники данных для решения конкретных проблем. В главах 13–15 мы подробно рассмотрим приемы трассировки.

В этой главе в качестве примера используется дистрибутив Ubuntu Linux. Большинство из описываемых инструментов доступны также в других дистрибутивах Linux. Аналогичные инструменты есть и для других ядер и операционных систем.

4.1. ПОКРЫТИЕ ИНСТРУМЕНТАМИ

На рис. 4.1 показана схема ОС Linux, на которую я нанес инструменты наблюдения за рабочей нагрузкой для каждого компонента¹.

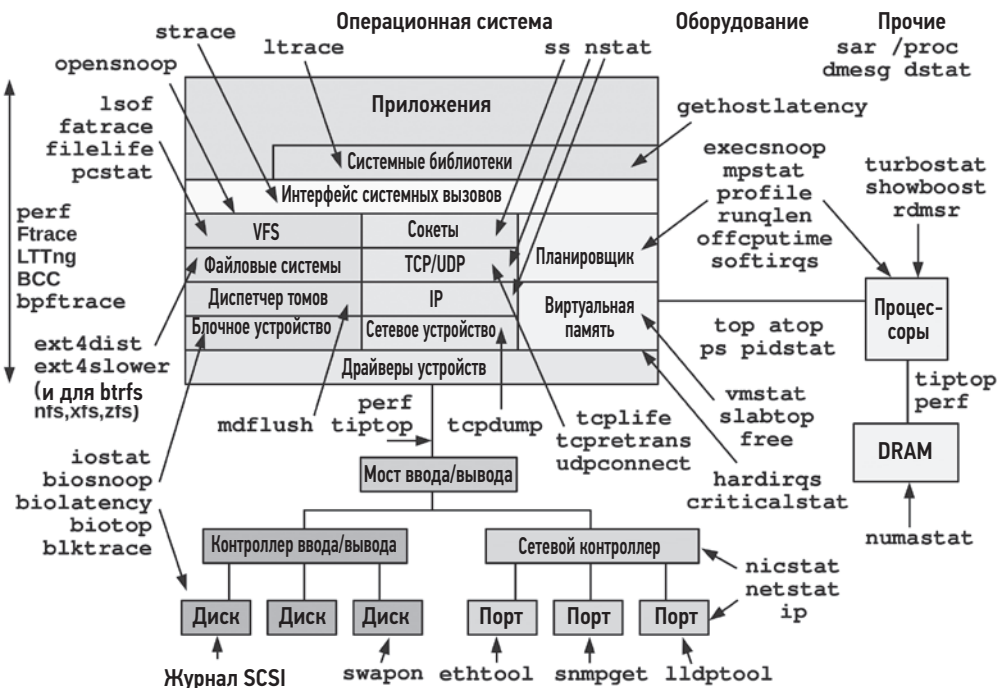


Рис. 4.1. Инструменты наблюдения за рабочей нагрузкой в Linux

¹ На курсах по анализу производительности, которые я вел в середине 2000-х, я рисовал свою схему ядра на доске, наносил на нее различные инструменты анализа производительности и подписывал — за чем они наблюдают. Как показала практика, такие схемы оказались эффективным средством для объяснения покрытия инструментами и формирования ментальной карты. С тех пор я не раз публиковал свои схемы в электронном виде, и теперь они украшают стены офисов по всему миру. Вы можете скачать их на моем сайте [Gregg 20a].

Большинство этих инструментов специализированы для наблюдения за конкретным ресурсом — процессором, памятью или дисками — и рассматриваются в последующих главах, посвященных отдельным ресурсам. Но есть несколько универсальных инструментов, которые могут анализировать разные области. Они будут представлены далее в этой главе: perf, Ftrace, BCC и bpftrace.

4.1.1. Инструменты статического анализа производительности

Есть еще один тип наблюдений, в центре внимания которого находятся атрибуты системы в состоянии покоя. В главе 2 «Методологии» в разделе 2.5.17 «Статическая настройка производительности» этот подход был описан как методология *статической настройки производительности*, а соответствующие инструменты показаны на рис. 4.2.

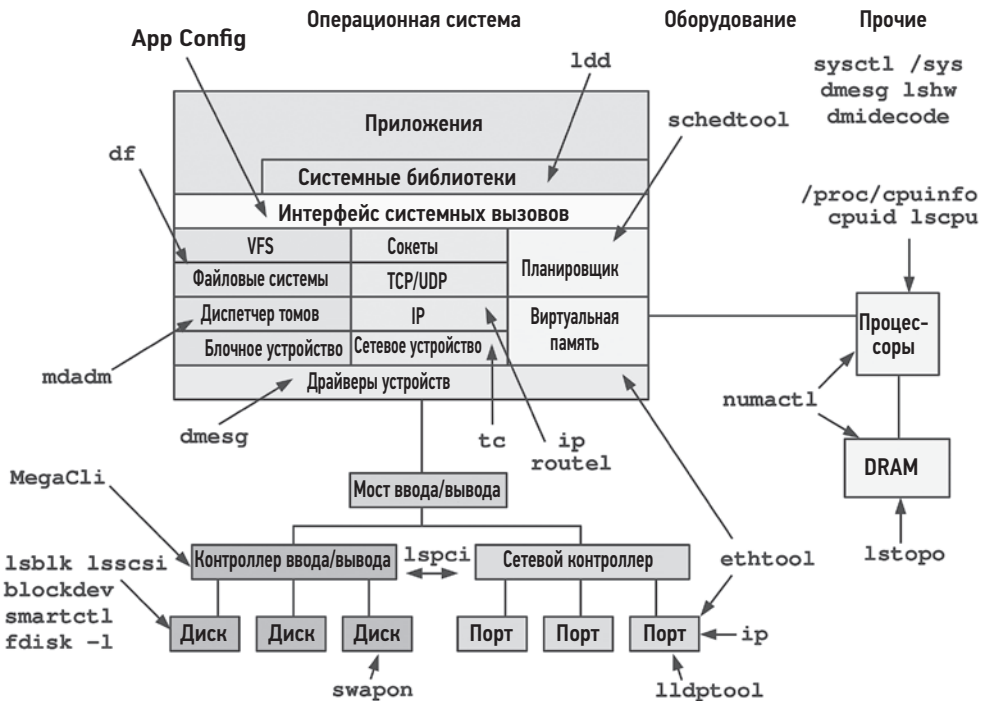


Рис. 4.2. Инструменты статической настройки производительности в Linux

Не забывайте использовать инструменты, перечисленные на рис. 4.2, для проверки на наличие проблем с конфигурацией и компонентами. Иногда проблемы с производительностью возникают просто из-за неправильной конфигурации.

4.1.2. Инструменты анализа кризисных ситуаций

Когда у вас случается критическое падение производительности, для устранения которого требуется применение различных инструментов, то может оказаться, что ни один из них не установлен в кризисной системе. Хуже того, из-за проблем с производительностью установка инструментов на сервер может занять гораздо больше времени, чем обычно длится кризис.

В табл. 4.1 перечислены рекомендуемые установочные пакеты и репозитории с исходным кодом для Linux, содержащие такие антикризисные инструменты. Имена пакетов указаны для Ubuntu/Debian, в других дистрибутивах они могут отличаться.

Таблица 4.1. Пакеты инструментов анализа кризисных ситуаций в Linux

Пакет	Содержит инструменты
procs	ps(1), vmstat(8), uptime(1), top(1)
util-linux	dmesg(1), lsblk(1), lscpu(1)
sysstat	iostat(1), mpstat(1), pidstat(1), sar(1)
iproute2	ip(8), ss(8), nstat(8), tc(8)
numactl	numastat(8)
linux-tools-common linux-tools-\$(uname -r)	perf(1), turbostat(8)
bcc-tools (он же bpfcc-tools)	opensnoop(8), execsnoop(8), runqlat(8), runqlen(8), softirqs(8), hardirqs(8), ext4slower(8), ext4dist(8), biotop(8), biosnoop(8), biolatency(8), tcptop(8), tcplife(8), trace(8), argdist(8), funccount(8), stackcount(8), profile(8) и многие другие
bpfttrace	bpfttrace, basic versions of opensnoop(8), execsnoop(8), runqlat(8), runqlen(8), biosnoop(8), biolatency(8) и многие другие
perf-tools-unstable	версии opensnoop(8), execsnoop(8), iolateness(8), iosnoop(8), bitesize(8), funccount(8), kprobe(8) для Ftrace
trace-cmd	trace-cmd(1)
nicstat	nicstat(1)
ethtool	ethtool(8)
tiptop	tiptop(1)
msr-tools	rdmsr(8), wrmsr(8)
github.com/brendangregg/msr-cloud-tools	showboost(8), cpuhot(8), cputemp(8)
github.com/brendangregg/pmc-cloud-tools	pmcarch(8), cpucache(8), icache(8), tlbstat(8), resstalls(8)

В крупных компаниях, например в Netflix, есть перформанс-команды, занимающиеся поддержкой анализа производительности ОС. Они следят за тем, чтобы все

эти пакеты были установлены в промышленных системах. По умолчанию в дистрибутиве Linux обычно устанавливаются только `procs` и `util-linux`, поэтому все остальные нужно добавить самостоятельно.

В контейнерных средах бывает желательно создать привилегированный отладочный контейнер, имеющий полный доступ к системе¹ и всем установленным инструментам. Образ этого контейнера можно установить на узлы для размещения контейнеров и разворачивать при необходимости.

Простого добавления пакетов инструментов часто бывает недостаточно: может потребоваться также настроить ядро и ПО пространства пользователя для поддержки этих инструментов. Инструменты трассировки обычно требуют включения определенных конфигурационных параметров ядра, таких как `CONFIG_FTRACE` и `CONFIG_BPF`. Для инструментов профилирования бывает необходимо, чтобы ПО было настроено для поддержки возможности обхода стека либо за счет его компиляции с поддержкой указателей фреймов (включая системные библиотеки: `libc`, `libpthread` и т. д.), либо за счет использования `debuginfo`-пакетов с отладочной информацией. Если ваша компания еще не сделала этого, то следует проверить работоспособность каждого инструмента анализа производительности и устранить имеющиеся недостатки до того, как эти инструменты срочно понадобятся в кризисной ситуации.

В следующих разделах мы подробно рассмотрим инструменты наблюдения за производительностью.

4.2. ТИПЫ ИНСТРУМЕНТОВ

Инструменты наблюдения удобно классифицировать по области покрытия — *система целиком* (*system-wide*) или *отдельный процесс* (*per-process*), а также по тому, на чем они основаны — на счетчиках или *событиях*. Пример такой классификации инструментов для Linux показан на рис. 4.3.

Некоторые инструменты присутствуют в нескольких квадрантах. Например, `top(1)`, помимо прочего, выводит информацию о системе в целом, а инструменты анализа всей системы, основанные на событиях, часто поддерживают фильтрацию для вывода сведений о конкретном процессе (`-p PID`).

К инструментам, основанным на событиях, относятся профилировщики и трассировщики. Профилировщики наблюдают за активностью, выполняя серию моментальных снимков по событиям и создавая грубую картину цели. Трассировщики отслеживают каждое интересующее событие и могут обрабатывать их, что можно использовать, например, для создания нестандартных счетчиков. Счетчики, трассировка и профилирование были описаны в главе 1.

¹ Также можно настроить общее пространство имен для совместного использования с целевым контейнером.



Рис. 4.3. Типы инструментов наблюдения

В следующих разделах описываются инструменты Linux, использующие фиксированные счетчики, трассировку и профилирование, а также инструменты, выполняющие мониторинг (метрики).

4.2.1. Фиксированные счетчики

Ядра поддерживают различные счетчики с системными статистиками. Обычно они реализованы как целые числа без знака, которые увеличиваются при возникновении событий. Например, есть счетчики полученных сетевых пакетов, выполненных операций дискового ввода/вывода и произошедших прерываний. Они отображаются программным обеспечением для мониторинга в виде *метрик* (см. раздел 4.2.4 «Мониторинг»).

Обычно ядро поддерживает кумулятивные счетчики парами: один для подсчета количества событий, а другой для подсчета общего времени, потраченного на обработку событий. Это позволяет определить не только общее количество событий, но и среднее время (или задержку) обработки события путем деления общего времени на количество. Поскольку счетчики являются кумулятивными, то, извлекая из них информацию через определенные интервалы времени (например, каждую секунду), можно вычислить дельту, а также частоту событий в секунду и среднюю задержку. Именно так вычисляются системные статистики.

С точки зрения производительности считается, что счетчики не имеют оверхеда, потому что они включены по умолчанию и постоянно поддерживаются ядром. Единственные дополнительные расходы при их использовании — это чтение их значений из пространства пользователя (но эти расходы весьма незначительны). Далее приводятся примеры инструментов, которые читают счетчики для анализа работы системы в целом или отдельного процесса.

Инструменты для анализа системы в целом

Эти инструменты исследуют активность системы в целом в контексте системного программного обеспечения или аппаратных ресурсов, используя счетчики ядра. В их число входят:

- **vmstat(8)**: статистика использования виртуальной и физической памяти в системе в целом;
- **mpstat(1)**: потребление процессора;
- **iostat(1)**: использование дискового ввода/вывода, сообщаемое интерфейсом блочного устройства;
- **nstat(8)**: статистика стека TCP/IP;
- **sar(1)**: различные статистики; их можно также архивировать для последующего сравнительного анализа.

Эти инструменты обычно доступны всем пользователям системы (не требуют полномочий root). Соответствующие статистики чаще всего отображаются в виде графиков с помощью софта для мониторинга.

Многие следуют устоявшимся соглашениям об использовании и принимают необязательные *интервал* и *количество*. Вот пример запуска **vmstat(8)** с интервалом в одну секунду и количеством подсчитываемых событий, равным трем:

```
$ vmstat 1 3
procs -----memory----- ---swap-- -----io----- -system-- -----cpu-----
 r  b   swpd  free   buff  cache  si   so    bi    bo    in   cs  us  sy  id  wa  st
 4  0 1446428 662012 142100 5644676   1   4    28   152   33   1 29  8 63  0  0
 4  0 1446428 665988 142116 5642272   0   0     0   284 4957 4969 51  0 48  0  0
 4  0 1446428 685116 142116 5623676   0   0     0     0 4488 5507 52  0 48  0  0
```

Первая строка в выводе содержит сводную информацию — средние значения — за весь период с момента загрузки системы. Последующие строки выводятся с интервалом в одну секунду, они содержат обновленную сводную информацию и отражают текущую активность. По крайней мере такова была цель: эта версия инструмента для Linux смешивает в первой строке суммарные усредненные значения с момента загрузки и текущие (в столбцах, отражающих потребление памяти, выводятся текущие значения; подробнее о **vmstat(8)** рассказывается в главе 7).

Инструменты для анализа отдельных процессов

Эти инструменты ориентированы на исследование отдельных процессов и используют счетчики, которые ядро поддерживает для каждого процесса. В их число входят:

- **ps(1)**: показывает состояние и различные статистики процесса, включая потребление памяти и процессора;
- **top(1)**: показывает самые активные процессы; может сортировать список процессов по потреблению процессора или другой статистике;
- **mpstat(1)**: перечисляет сегменты памяти процесса со статистиками потребления.

Эти инструменты обычно читают статистики из файловой системы /proc.

4.2.2. Профилирование

Профилирование нужно для определения характеристик цели путем сбора образцов или моментальных снимков ее поведения. Типичной целью профилирования является потребление процессора. В этом случае профилировщик отбирает по таймеру значение указателя инструкций или трассировки стека, которые позволяют судить о путях в коде, где процесс проводит больше всего времени. Отбор образцов обычно производится с фиксированной частотой, например 100 Гц (раз в секунду), для всех процессоров в течение короткого промежутка времени, например одной минуты. Инструменты профилирования, или *профилировщики*, часто используют частоту 99 Гц вместо 100 Гц, чтобы избежать попадания в одну и ту же точку целевой активности, которое может привести к завышению или занижению оценок.

Профилирование также может основываться на не связанных по времени аппаратных событиях, таких как промахи аппаратного кэша процессора или активность шины. В этом случае профилирование помогает выявить пути, ответственные за эти события, или дает разработчикам ценную информацию для оптимизации потребления памяти их кодом.

В отличие от фиксированных счетчиков, профилирование (и трассировка) обычно производится только при необходимости, так как может вызвать довольно высокий оверхед на сбор и хранение информации. Величина таких оверхедов зависит от инструмента и частоты событий, которые он обрабатывает. Профилировщики, выполняющие сбор образцов по таймеру, обычно более безопасны: частота событий известна, поэтому оверхед можно предсказать, а кроме того, частоту событий можно выбрать такой, чтобы сделать оверхеды пренебрежимо малыми.

Инструменты для анализа системы в целом

В число инструментов для профилирования системы в целом входят:

- **perf(1)**: стандартный профилировщик Linux, поддерживающий подкоманды профилирования;
- **profile(8)**: профилировщик процессора на основе BPF из репозитория BCC (описывается в главе 15 «BPF»), который подсчитывает трассировки стека в контексте ядра;
- **Intel VTune Amplifier XE**: профилирование Linux и Windows с графическим интерфейсом, поддерживает просмотр исходного кода.

Их также можно использовать для анализа отдельных процессов.

Инструменты для анализа отдельных процессов

В число инструментов для профилирования отдельных процессов входят:

- **gprof(1)**: профилировщик GNU, анализирующий информацию профилирования, добавленную компиляторами (например, `gcc -pg`);

- **cachegrind**: инструмент из набора valgrind, способный профилировать аппаратный кэш (и многое другое) и визуализировать результаты с помощью kcachegrind;
- **Java Flight Recorder (JFR)**: многие языки программирования имеют свои специализированные профилировщики, которые могут исследовать контекст языка, как, например, JFR для Java.

Дополнительную информацию об инструментах профилирования ищите в главе 6 «Процессоры» и главе 13 «perf».

4.2.3. Трассировка

Трассировка позволяет отследить каждое событие и сохранить подробные сведения о событиях для последующего анализа или вычисления сводных данных. Трассировка похожа на профилирование, но ее цель состоит в том, чтобы собрать или исследовать все события, а не только образцы. Трассировка может иметь более высокий оверхед, чем профилирование, и существенно замедлить работу цели. Учитывайте это, принимая решение о трассировке промышленной рабочей нагрузки, а также не забывайте, что нагрузка, создаваемая трассировщиком, может исказить результаты измерения времени. Как и в случае с профилированием, трассировка обычно используется только по мере необходимости.

Журналирование, в ходе которого нечастые события, такие как ошибки и предупреждения, записываются в файл журнала, можно рассматривать как низкочастотную трассировку, которая включена по умолчанию. В число таких журналов входит и системный журнал.

Далее приводятся примеры инструментов трассировки системы в целом и отдельных процессов.

Инструменты для анализа системы в целом

Эти инструменты трассировки исследуют активность системы в целом в контексте системного программного обеспечения или аппаратных ресурсов, используя средства трассировки ядра. В их число входят:

- **tcpdump(8)**: трассировка сетевых пакетов (использует libpcap);
- **biosnoop(8)**: трассировка блочного ввода/вывода (использует BCC или bpftrace);
- **execsnoop(8)**: трассировка событий запуска новых процессов (использует BCC или bpftrace);
- **perf(1)**: стандартный профилировщик Linux, способный также трассировать отдельные события;
- **perf trace**: специальная подкоманда профилировщика perf, позволяющая трассировать системные вызовы в масштабе всей системы;
- **Ftrace**: трассировщик, встроенный в Linux;
- **BCC**: библиотека трассировки и набор инструментов на основе BPF;
- **bpftrace**: трассировщик на основе BPF (bpftrace(8)) и набор инструментов.

perf(1), Ftrace, BCC и bpftrace будут представлены в разделе 4.5 «Инструменты трассировки» и подробно описаны в главах 13–15. Есть более сотни инструментов трассировки, созданных с использованием BCC и bpftrace, включая biosnoop(8) и exectnoop(8) из этого списка. Далее в книге вы найдете множество примеров их применения.

Инструменты для анализа отдельных процессов

Эти инструменты трассировки ориентированы на исследование отдельных процессов, а также операционных систем, на которых они основаны. В их число входят:

- **strace(1)**: трассировка системных вызовов;
- **gdb(1)**: отладчик на уровне исходного кода.

Отладчики позволяют исследовать данные для каждого события, но для этого они должны приостанавливать и возобновлять работу цели. Это может привести к огромному оверхеду, что делает их непригодными для использования в промышленной среде.

Инструменты трассировки для анализа системы в целом, такие как perf(1) и bpftrace, поддерживают фильтры, позволяющие исследовать конкретный процесс, и могут работать с гораздо меньшим оверхедом, что делает их предпочтительным выбором, если они доступны.

4.2.4. Мониторинг

С понятием мониторинга мы познакомились в главе 2 «Методологии». В отличие от других типов инструментов, инструменты мониторинга ведут постоянное наблюдение и сохраняют статистики на случай, если они понадобятся позже.

sar(1)

Традиционным инструментом мониторинга отдельного хоста является System Activity Reporter, sar(1), созданный на базе соответствующего инструмента из AT&T Unix. Он основан на счетчиках и имеет агент, который запускается по графику (через cron) и сохраняет состояние общесистемных счетчиков. Инструмент sar(1) позволяет просматривать данные в командной строке, например:

```
# sar
Linux 4.15.0-66-generic (bgregg) 12/21/2019      _x86_64_      (8 CPU)
12:00:01 AM  CPU      %user   %nice   %system %iowait  %steal   %idle
12:05:01 AM  all       3.34    0.00    0.95    0.04    0.00   95.66
12:10:01 AM  all       2.93    0.00    0.87    0.04    0.00   96.16
12:15:01 AM  all       3.05    0.00    1.38    0.18    0.00   95.40
12:20:01 AM  all       3.02    0.00    0.88    0.03    0.00   96.06
[...]
Average:     all       0.00    0.00    0.00    0.00    0.00   0.00
```


По умолчанию `sar(1)` читает статистики из архива (если он ведется) и выводит последние сохраненные статистики. При желании можно указать временной интервал и исследовать текущую активность с указанной частотой.

`sar(1)` может сохранять десятки различных статистик, помогающих получить достаточно полное представление о потреблении процессора, памяти, дисков и сети, о прерываниях, энергопотреблении и многом другом. Более подробно об этих его возможностях рассказывается в разделе 4.4 «`sar`». Сторонние продукты мониторинга часто основаны на `sar(1)` или тех же статистиках, которые он использует, и позволяют получать эти метрики по сети.

SNMP

Традиционно для мониторинга по сети используется простой протокол управления сетью (Simple Network Management Protocol, SNMP). Устройства и операционные системы могут поддерживать SNMP по умолчанию, не требуя установки сторонних агентов или инструментов экспорта. Протокол SNMP поддерживает множество основных показателей работы ОС, но не охватывает современные приложения. По этой причине в большинстве сред используются настраиваемые средства мониторинга на основе агентов.

Агенты

Современное ПО мониторинга запускает агенты (также известные как *экспортеры*, или *плагины*) во всех системах, где требуется фиксировать показатели работы ядра и приложений. К ним относятся агенты для конкретных приложений и целей, например, для сервера базы данных MySQL, веб-сервера Apache и системы кэширования Memcached. Такие агенты могут собирать подробные метрики производительности приложений, которые нельзя получить с помощью одних только системных счетчиков.

В числе ПО мониторинга и агентов для Linux можно назвать:

- **Performance Co-Pilot (PCP)**: PCP поддерживает десятки различных агентов (называемых агентами метрик производительности: Performance Metric Domain Agents, PMDA), включая метрики на основе BPF [PCP 20].
- **Prometheus**: программное обеспечение мониторинга Prometheus поддерживает десятки различных экспортеров для баз данных, оборудования, систем обмена сообщениями, хранилищ, HTTP, API и журналов [Prometheus 20].
- **collectd**: поддерживает десятки различных плагинов.

На рис. 4.4 показан пример архитектуры мониторинга, включающей сервер базы данных хранения метрик производительности и веб-сервер для обслуживания пользовательского интерфейса клиента. Метрики передаются на сервер базы данных агентами, а затем становятся доступными в пользовательском интерфейсе клиента, где отображаются на дашбордах в виде линейных графиков или числовых значений. Примером сервера базы данных для мониторинга может служить Graphite Carbon, а примером веб-сервера, поддерживающего дашборды мониторинга, — Grafana.

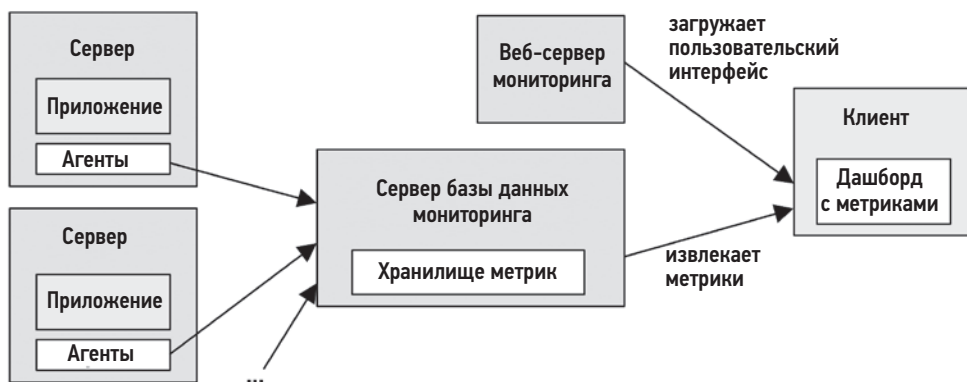


Рис. 4.4. Пример архитектуры мониторинга

Есть десятки продуктов для мониторинга и сотни различных агентов для разных целей. Их описание выходит за рамки этой книги. Однако у всех у них есть один общий знаменатель: системные статистики (основанные на счетчиках ядра). Системные статистики, отображаемые продуктами мониторинга, часто те же, что отображаются системными инструментами: `vmstat(8)`, `iostat(1)` и т. д. Их изучение поможет понять работу продуктов мониторинга, даже если вам не приходится использовать инструменты командной строки. Эти инструменты рассматриваются в следующих главах.

Некоторые продукты для мониторинга получают метрики, запуская системные инструменты и анализируя их вывод, что неэффективно. Более совершенные продукты получают метрики напрямую, используя библиотеки и интерфейсы ядра — все то же самое, что используют инструменты командной строки. Эти источники информации рассматриваются в следующем разделе, где основное внимание уделяется наиболее типичному общему знаменателю: интерфейсам ядра.

4.3. ИСТОЧНИКИ ИНФОРМАЦИИ

В следующих разделах описываются различные интерфейсы в Linux, с помощью которых инструменты наблюдения могут извлекать данные о производительности. Они перечислены в табл. 4.2.

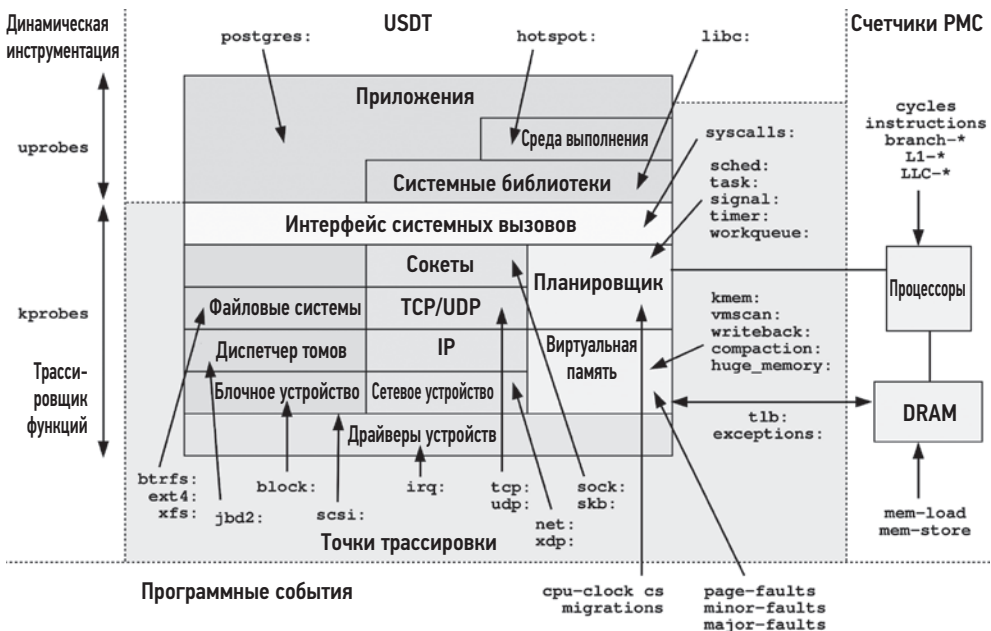
Сначала мы рассмотрим основные системные источники статистик производительности: `/proc` и `/sys`. Затем перейдем к другим источникам в Linux: учет задержек, `netlink`, точки трассировки, `kprobes`, `USDT`, `uprobes`, `PMU` и др.

Многие из этих источников, особенно общесистемные трассировки, используют трассировщики, описанные в главе 13 «`perf`», в главе 14 «`Ftrace`» и в главе 15 «`BPF`». Область, покрываемая этими источниками, показана на рис. 4.5 вместе с именами событий и групп: например, имя `block`: охватывает все точки трассировки блочного ввода/вывода, включая `block:block_rq_issue`.

Таблица 4.2. Источники информации о производительности в Linux

Тип	Источник
Счетчики производительности для отдельных процессов	/proc
Общесистемные счетчики	/proc, /sys
Настройки и счетчики устройств	/sys
Статистики групп управления (cgroups)	/sys/fs/cgroup
Трассировки для отдельных процессов	ptrace
Аппаратные счетчики производительности (PMC)	perf_event
Сетевые статистики	netlink
Перехват сетевых пакетов	libpcap
Метрики задержки для отдельных потоков выполнения	Учет задержек
Общесистемные трассировки	Функции профилирования (Ftrace), точ- ки трассировки, программные события, kprobes, uprobes, perf_event

На рис. 4.5 показаны лишь несколько примеров источников USDT для базы данных PostgreSQL (postgres:), компилятора JVM hotspot (hotspot:) и libc (libc:). У вас их может быть намного больше, в зависимости от установленного ПО уровня пользователя.

**Рис. 4.5.** Источники трассировок в Linux

Дополнительную информацию о работе точек трассировки, зондов kprobes и uprobes и их внутреннем устройстве ищите в главе 2 книги «BPF Performance Tools» [Gregg 19].

4.3.1. /proc

Этот интерфейс имеет вид файловой системы и содержит статистики ядра. В /proc есть несколько каталогов, имена которых совпадают с идентификаторами представляемых ими процессов. В каждом таком каталоге есть несколько файлов с информацией и статистиками о каждом процессе, отображаемыми из структур данных в ядре. В /proc есть также файлы с общесистемными статистиками.

Файловая система /proc динамически создается ядром и никак не связана с устройствами хранения (она целиком находится в памяти). В основном она доступна только для чтения и предоставляет статистики для инструментов наблюдения. Некоторые файлы доступны для записи и позволяют управлять процессами и поведением ядра.

Интерфейс файловой системы очень удобен: это интуитивно понятная структура для предоставления статистик ядра через дерево каталогов, которая имеет хорошо известный программный интерфейс доступа через вызовы файловой системы POSIX: open(), read(), close(). Ее можно также изучать из командной строки, используя утилиты cd, cat(1), grep(1) и awk(1). Кроме того, файловая система обеспечивает безопасность на уровне пользователя за счет поддержки разрешений на доступ к файлам. В редких случаях, когда нет возможности использовать типичные инструменты наблюдения за процессами (ps(1), top(1) и т. д.), некоторые действия по отладке процессов все же можно выполнить, применяя встроенные команды оболочки к файлам в каталоге /proc.

Оверхед на чтение большинства файлов в /proc незначителен. Исключение составляют некоторые файлы, связанные с массивами в памяти, при отображении которых выполняется обход таблицы страниц.

Статистики по процессам

В /proc доступны различные файлы со статистиками по процессам. Вот, например, что может быть доступно (в Linux 5.4), здесь исследуется процесс с PID 18733¹:

```
$ ls -F /proc/18733
arch_status      environ          mountinfo        personality       statm
attr/            exe@             mounts           projid_map        status
autogroup        fd/              mountstats       root@             syscall
auxv             fdinfo/          net/             sched             task/
cgroup           gid_map          ns/              schedstat         timers
clear_refs       io               numa_maps        sessionid          timerslack_ns
cmdline          limits           oom_adj          setgroups         uid_map
```

¹ Также можете попробовать изучить каталог /proc/self, соответствующий текущему процессу (командной оболочки).

comm	loginuid	oom_score	smaps	wchan
coredump_filter	map_files/	oom_score_adj	smaps_rollup	
cpuset	maps	pagemap	stack	
cwd@	mem	patch_state	stat	

Точный список доступных файлов зависит от версии ядра и опций CONFIG.

Те, которые связаны с наблюдаемостью производительности отдельных процессов, включают:

- **limits**: действующие ограничения ресурсов;
- **maps**: отображаемые области памяти;
- **sched**: различные статистики планировщика;
- **schedstat**: время выполнения на процессоре, задержка и кванты времени;
- **smaps**: отображаемые области памяти со статистиками использования;
- **stat**: состояние процесса и статистики, включая общее потребление процессора и памяти;
- **statm**: сводная информация о потреблении памяти в единицах страниц;
- **status**: информация из **stat** и **statm** с подписями;
- **fd**: каталог файловых дескрипторов для символических ссылок (см. также **fdinfo**);
- **cgroup**: информация о членстве в управляющей группе;
- **task**: каталог статистик по задачам (потокам выполнения).

Ниже показано, как **top(1)** читает статистики по процессам (трассировка выполнена с помощью **strace(1)**):

```
stat("/proc/14704", {st_mode=S_IFDIR|0555, st_size=0, ...}) = 0
open("/proc/14704/stat", O_RDONLY) = 4
read(4, "14704 (sshd) S 1 14704 14704 0 -"... , 1023) = 232
close(4)
```

Как вы видите, этот инструмент открыл файл с именем «stat» в каталоге с именем, совпадающим с идентификатором процесса (14704), и прочитал его содержимое.

top(1) повторяет эту последовательность действий для всех активных процессов в системе. В некоторых системах, где одновременно выполняется большое количество процессов, переход **top(1)** может стать заметным, особенно в версиях инструмента, которые повторяют эти действия для каждого процесса при каждом обновлении экрана. Это может привести к тому, что **top(1)** сообщит о самом себе как о самом большом потребителе процессорного времени.

Общесистемные статистики

В Linux файловая система **/proc** дополнена файлами и каталогами с общесистемными статистиками:

```
$ cd /proc; ls -Fd [a-z]*
acpi/      dma          kallsyms   mdstat      schedstat  thread-self@
buddyinfo  driver/      kcore      meminfo     scsi/      timer_list
bus/        execdomains  keys       misc        self@      tty/
cgroups    fb           key-users  modules     slabinfo   uptime
cmdline    filesystems  kmsg       mounts@     softirqs   version
consoles   fs/          kpagecgrou mtrr        stat       vmallocinfo
cpuinfo     interrupts   kpagecount net@         swaps      vmstat
crypto      iomem        kpageflags pagetypeinfo sys/        zoneinfo
devices     iports       loadavg    partitions  sysrq-trigger
diskstats   irq/         locks      sched_debug sysvipc/
```

Общесистемные файлы, связанные с наблюдаемостью производительности, включают:

- **cpuinfo**: информация о физическом процессоре, включая каждый виртуальный процессор, название модели, тактовую частоту и размеры кэшей;
- **diskstats**: статистики дискового ввода/вывода для всех дисковых устройств;
- **interrupts**: счетчики прерываний по процессорам;
- **loadavg**: средняя нагрузка;
- **meminfo**: характеристики использования системной памяти;
- **net/dev**: статистики сетевого интерфейса;
- **net/netstat**: сетевые статистики для системы в целом;
- **net/tcp**: информация об активности TCP-сокетов;
- **pressure/**: файлы с информацией о времени, затраченном на ожидание доступности процессора, памяти или ввода/вывода (Pressure Stall Information, PSI);
- **schedstat**: статистики планировщика процессорного времени для системы в целом;
- **self**: символическая ссылка на каталог, соответствующий текущему процессу; предназначена для удобства;
- **slabinfo**: статистика кэша распределителя блоков памяти ядра;
- **stat**: сборник статистик, связанных с потреблением ресурсов ядра и системы: процессоры, диски, подкачка, процессы;
- **zoneinfo**: информация о зонах памяти.

Эти файлы читаются общесистемными инструментами. Вот пример того, как `vmstat(8)` читает информацию из файловой системы `/proc` (трассировка выполнена с помощью `strace(1)`):

```
open("/proc/meminfo", O_RDONLY)      = 3
lseek(3, 0, SEEK_SET)                = 0
read(3, "MemTotal:      889484 kB\nMemF"... , 2047) = 1170
open("/proc/stat", O_RDONLY)         = 4
read(4, "cpu 14901 0 18094 102149804 131"... , 65535) = 804
open("/proc/vmstat", O_RDONLY)       = 5
lseek(5, 0, SEEK_SET)                = 0
read(5, "nr_free_pages 160568\nnr_inactive"... , 2047) = 1998
```

Здесь видно, что `vmstat(8)` читал файлы `meminfo`, `stat` и `vmstat`.

Точность статистик процессора

Файл `/proc/stat` содержит общесистемные статистики о потреблении процессора и используется многими инструментами (`vmstat(8)`, `mpstat(1)`, `sar(1)`, агентами мониторинга). Точность этих статистик зависит от конфигурации ядра. Конфигурация по умолчанию (`CONFIG_TICK_CPU_ACCOUNTING`) оценивает потребление процессора с точностью до такта системных часов [Weisbecker 13], длительность которого может составлять 4 мс (в зависимости от `CONFIG_HZ`). Обычно этого достаточно. Есть варианты увеличения точности за счет использования счетчиков с более высоким разрешением, пусть и с небольшим оверхедом (`VIRT_CPU_ACCOUNTING_NATIVE` и `VIRT_CPU_ACCOUNTING_GEN`), а также возможность более точно измерять время `IRQ` (`IRQ_TIME_ACCOUNTING`). Другой подход к получению более точных оценок потребления процессора — использование моделезависимых регистров (model-specific registers, MSR) или счетчиков производительности PMC.

Содержимое файла

Файлы в файловой системе `/proc` обычно имеют текстовый формат, что позволяет читать их прямо из командной строки и обрабатывать с помощью инструментов сценариев командной оболочки. Например:

```
$ cat /proc/meminfo
MemTotal:      15923672 kB
MemFree:       10919912 kB
MemAvailable:  15407564 kB
Buffers:       94536 kB
Cached:        2512040 kB
SwapCached:    0 kB
Active:        1671088 kB
[...]
$ grep Mem /proc/meminfo
MemTotal:      15923672 kB
MemFree:       10918292 kB
MemAvailable:  15405968 kB
```

Это довольно удобно, но добавляет небольшой оверхед на перевод статистик в текстовый вид в ядре и на их интерпретацию в самом инструменте, который анализирует текст. Более эффективен двоичный интерфейс `netlink`, описанный в разделе 4.3.4 «`netlink`».

Содержимое `/proc` задокументировано в странице справочного руководства `proc(5)` и в документации ядра Linux: `Documentation/filesystems/proc.txt` [Bowden 20]. Для некоторых компонентов есть расширенная документация: например, статистики по дискам подробно описываются в `Documentation/iostats.txt`, а статистики планировщика — в `Documentation/scheduler/sched-stats.txt`. Помимо документации, вы можете также изучить исходный код ядра, чтобы понять, как формируются

элементы /proc. Кроме того, полезно заглянуть в исходный код инструментов, которые их используют.

Наличие некоторых элементов /proc зависит от конфигурации ядра: файлы `sched-stat` доступны, только если включен параметр `CONFIG_SCHEDSTATS`, `sched` — если включен параметр `CONFIG_SCHED_DEBUG`, а `pressure` — если включен параметр `CONFIG_PSI`.

4.3.2. /sys

В Linux есть и файловая система `sysfs`, смонтированная в каталог `/sys`. Она появилась в ядре 2.6 и предназначалась для организации иерархического представления статистик ядра. Она отличается от `/proc`, которая развивалась долгие годы, и предлагает различные системные статистики, в основном добавляемые в каталог верхнего уровня. Файловая система `sysfs` изначально предназначалась для предоставления статистик драйверов устройств. Потом ее расширили, и теперь она может включать статистики любого типа.

Например, вот список файлов в `/sys` для процессора CPU0 (усеченный):

```
$ find /sys/devices/system/cpu/cpu0 -type f
/sys/devices/system/cpu/cpu0/uevent
/sys/devices/system/cpu/cpu0/hotplug/target
/sys/devices/system/cpu/cpu0/hotplug/state
/sys/devices/system/cpu/cpu0/hotplug/fail
/sys/devices/system/cpu/cpu0/crash_notes_size
/sys/devices/system/cpu/cpu0/power/runtime_active_time
/sys/devices/system/cpu/cpu0/power/runtime_active_kids
/sys/devices/system/cpu/cpu0/power/pm_qos_resume_latency_us
/sys/devices/system/cpu/cpu0/power/runtime_usage
[...]
/sys/devices/system/cpu/cpu0/topology/die_id
/sys/devices/system/cpu/cpu0/topology/physical_package_id
/sys/devices/system/cpu/cpu0/topology/core_cpus_list
/sys/devices/system/cpu/cpu0/topology/die_cpus_list
/sys/devices/system/cpu/cpu0/topology/core_siblings
[...]
```

Многие из этих файлов содержат информацию об аппаратных кэшах процессора. Ниже приводится пример их содержимого (получено с помощью `grep(1)`, чтобы в вывод было включено имя файла):

```
$ grep . /sys/devices/system/cpu/cpu0/cache/index*/level
/sys/devices/system/cpu/cpu0/cache/index0/level:1
/sys/devices/system/cpu/cpu0/cache/index1/level:1
/sys/devices/system/cpu/cpu0/cache/index2/level:2
/sys/devices/system/cpu/cpu0/cache/index3/level:3
$ grep . /sys/devices/system/cpu/cpu0/cache/index*/size
/sys/devices/system/cpu/cpu0/cache/index0/size:32K
/sys/devices/system/cpu/cpu0/cache/index1/size:32K
/sys/devices/system/cpu/cpu0/cache/index2/size:1024K
/sys/devices/system/cpu/cpu0/cache/index3/size:33792K
```


Здесь видно, что процессор CPU0 имеет доступ к двум кэшам Level 1, каждый по 32 Кбайт, кэшу Level 2 размером 1 Мбайт и кэшу Level 3 размером 33 Мбайт.

Файловая система /sys обычно содержит десятки тысяч статистик в файлах, доступных только для чтения, а также множество файлов, доступных для записи и предназначенных для изменения состояния ядра. Например, процессор можно включить или выключить, записав «1» или «0» в файл с именем «online». По аналогии со статистиками, доступными для чтения, некоторые настройки можно произвести, выполняя запись текстовых строк в командной строке (`echo 1 > filename`), без использования двоичного интерфейса.

4.3.3. Учет задержек

Ядро Linux, скомпилированное с параметром `CONFIG_TASK_DELAY_ACCT`, отслеживает время пребывания задачи в следующих состояниях:

- **задержка планировщика:** ожидание в очереди на выполнение;
- **блочный ввод/вывод:** ожидание завершения блочного ввода/вывода;
- **подкачка:** ожидание выделения памяти (при ее нехватке);
- **освобождение памяти:** ожидание процедуры освобождения памяти.

Технически задержка планировщика извлекается из файла `schedstat` в файловой системе /proc (упоминавшегося выше), но суммируется с другими состояниями учета задержки (находится в `struct sched_info`, а не в `struct task_delay_info`).

Эти статистики можно прочесть с помощью инструментов пользовательского уровня из `taskstats` — интерфейса на основе `netlink`, которой используется для получения статистик задач и процессов. В исходном коде ядра можно найти:

- `Documentation/Accounting/delay-Accounting.txt`: документацию;
- `tools/Accounting/getdelays.c`: пример получения статистик.

Вот пример вывода, полученный в результате выполнения `getdelays.c`:

```
$ ./getdelays -dp 17451
print delayacct stats ON
PID      17451
```

CPU	count	real total	virtual total	delay total	delay average
	386	3452475144	31387115236	1253300657	3.247ms
IO	count	delay total	delay average		
	302	1535758266	5ms		
SWAP	count	delay total	delay average		
	0	0	0ms		
RECLAIM	count	delay total	delay average		
	0	0	0ms		

Время выводится в наносекундах, если не указано иное. Этот пример получен в системе с сильно нагруженным процессором, и исследуемый процесс страдает от задержек планировщика.

4.3.4. netlink

netlink — это специальное семейство адресов сокетов (AF_NETLINK), предназначенных для получения информации о ядре. Процедура использования netlink включает открытие сетевого сокета с семейством адресов AF_NETLINK и последующее выполнение серии вызовов send(2) и recv(2) для передачи запросов и получения информации в двоичных структурах. Это более сложный интерфейс, чем /proc, но и более эффективный, к тому же поддерживающий уведомления. Библиотека libnetlink упрощает использование этого интерфейса.

По аналогии с инструментами, представленными выше, можно воспользоваться утилитой strace(1), чтобы посмотреть, откуда берется информация о ядре. Вот пример трассировки инструмента статистики сокетов ss(8):

```
# strace ss
[...]  
socket(AF_NETLINK, SOCK_RAW|SOCK_CLOEXEC, NETLINK_SOCK_DIAG) = 3  
[...]
```

Он открывает сокет AF_NETLINK для группы NETLINK_SOCK_DIAG, через который возвращается информация о сокетах. Подробное описание можно найти на странице справочного руководства man для sock_diag(7). В число групп netlink входят:

- **NETLINK_ROUTE**: информация о маршрутах (доступна также в /proc/net/route);
- **NETLINK_SOCK_DIAG**: информация о сокете;
- **NETLINK_SELINUX**: уведомления о событиях SELinux;
- **NETLINK_AUDIT**: аудит (безопасность);
- **NETLINK_SCSITRANSPORT**: транспорт SCSI;
- **NETLINK_CRYPT**: информация о поддержке шифрования в ядре.

Интерфейс netlink использует большое число команд, в том числе ip(8), ss(8), routel(8) и более старые ifconfig(8) и netstat(8).

4.3.5. Точки трассировки

Точки трассировки — это источник событий ядра Linux, основанный на *статической инструментации* (этот термин я объяснял в главе 1 «Введение», в разделе 1.7.3 «Трассировка»). Точки трассировки — это жестко закодированные точки инструментации, размещенные в определенных местах в коде ядра. Например, точки трассировки есть в начале и в конце системных вызовов, в местах, где генерируются события планировщика, в операциях с файловой системой и дискового ввода/вывода¹. Инфраструктура точек трассировки была разработана Матье Деснуайерсом

¹ Некоторые из них управляются параметрами Kconfig и могут быть недоступны, если ядро скомпилировано без них. Примером может служить параметр CONFIG_RCU_TRACE, отвечающий за точки трассировки rcu.

(Mathieu Desnoyers) и стала доступной в версии Linux 2.6.32 в 2009 году. Точки трассировки — это стабильный API¹, и их количество ограничено.

Точки трассировки — это важный источник информации о производительности, так как они позволяют извлекать не только сводную статистику, но и получать более глубокое понимание поведения ядра. Трассировка функций может обеспечить аналогичные возможности (например, см. раздел 4.3.6 «kprobes»), но только точки трассировки гарантируют стабильный интерфейс, что позволяет разрабатывать надежные инструменты.

В этом разделе мы поговорим о точках трассировки. Они могут использоваться трассировщиками, представленными в разделе 4.5 «Инструменты трассировки», и подробно рассматриваются в главах 13–15.

Пример использования точек трассировки

Список доступных точек трассировки можно получить с помощью команды `perf list` (синтаксис `perf(1)` рассматривается в главе 14):

```
# perf list tracepoint
```

```
List of pre-defined events (to be used in -e):
```

```
[...]
  block:block_rq_complete           [Tracepoint event]
  block:block_rq_insert             [Tracepoint event]
  block:block_rq_issue              [Tracepoint event]
[...]
  sched:sched_wakeup                [Tracepoint event]
  sched:sched_wakeup_new            [Tracepoint event]
  sched:sched_waking                 [Tracepoint event]
  scsi:scsi_dispatch_cmd_done         [Tracepoint event]
  scsi:scsi_dispatch_cmd_error        [Tracepoint event]
  scsi:scsi_dispatch_cmd_start        [Tracepoint event]
  scsi:scsi_dispatch_cmd_timeout      [Tracepoint event]
[...]
  skb:consume_skb                    [Tracepoint event]
  skb:kfree_skb                       [Tracepoint event]
[...]
```

Я привел лишь часть списка с дюжиной примеров точек трассировки для блочных устройств, планировщика и SCSI. В моей системе насчитывается 1808 различных точек трассировки, 634 из которых предназначены для инструментации системных вызовов.

С помощью точек трассировки можно не только узнать, когда произошло событие, но также получить контекстные данные о событии. Например, следующая команда

¹ Я бы назвал его «условно стабильным». В своей практике мне доводилось видеть, как меняются точки трассировки.

perf(1) трассирует точку block:block_rq_issue и выводит события в реальном времени:

```
# perf trace -e block:block_rq_issue
[...]
0.000 kworker/u4:1-e/20962 block:block_rq_issue:259,0 W 8192 () 875216 + 16
[kworker/u4:1]
255.945 :22696/22696 block:block_rq_issue:259,0 RA 4096 () 4459152 + 8 [bash]
256.957 :22705/22705 block:block_rq_issue:259,0 RA 16384 () 367936 + 32 [bash]
[...]
```

Первые три поля — это отметка времени (секунды), сведения о процессе (имя/идентификатор потока) и описание события (поля разделены двоеточиями вместо пробелов). Остальные поля являются *аргументами* точки трассировки и генерируются *строкой формата*, описанной далее. Описание конкретной строки формата, предназначенной для block:block_rq_issue, ищите в главе 9 «Диски», в разделе 9.6.5 «perf».

Уточнение по терминологии: *точки трассировки* (*tracepoints*) технически являются функциями трассировки (их также называют хуками — *hooks*) в исходном коде ядра. Например, trace_sched_wakeup() — это точка трассировки, а ее вызов можно увидеть в kernel/sched/core.c. Эту точку трассировки можно инструментировать с помощью трассировщиков по имени «sched:sched_wakeup». Но фактически это *событие трассировки*, определяемое макросом TRACE_EVENT. Макрос TRACE_EVENT также определяет и форматирует свои аргументы, автоматически генерирует код trace_sched_wakeup() и помещает событие трассировки в интерфейсы tracefs и perf_event_open(2) [Ts'o 20]. Инструменты трассировки в первую очередь инструментировали события трассировки, хотя их можно называть «точками трассировки». В терминологии perf(1) события трассировки называются «событиями точек трассировки» (tracerpoint event), что может вызвать путаницу, потому что в kprobe и uprobe события трассировки тоже называются «событиями точек трассировки».

Аргументы и строка формата точки трассировки

Для каждой точки трассировки определена своя строка формата, содержащая аргументы события: дополнительный контекст о событии. Структуру строки формата можно увидеть в файле «format» в /sys/kernel/debug/tracing/events. Например:

```
# cat /sys/kernel/debug/tracing/events/block/block_rq_issue/format
name: block_rq_issue
ID: 1080
format:
    field:unsigned short common_type; offset:0; size:2; signed:0;
    field:unsigned char common_flags; offset:2; size:1; signed:0;
    field:unsigned char common_preempt_count; offset:3; size:1; signed:0;
    field:int common_pid;    offset:4; size:4; signed:1;

    field:dev_t dev;          offset:8; size:4; signed:0;
    field:sector_t sector;    offset:16; size:8; signed:0;
```

```

field:unsigned int nr_sector;   offset:24; size:4; signed:0;
field:unsigned int bytes;      offset:28; size:4; signed:0;
field:char rwbs[8];           offset:32; size:8; signed:1;
field:char comm[16];           offset:40; size:16; signed:1;
field:__data_loc char[] cmd;   offset:56; size:4; signed:1;

```

```

print fmt: "%d,%d %s %u (%s) %llu + %u [%s]", ((unsigned int) ((REC->dev) >> 20)),
((unsigned int) ((REC->dev) & ((1U << 20) - 1))), REC->rwbs, REC->bytes,
__get_str(cmd), (unsigned long long)REC->sector, REC->nr_sector, REC->comm

```

Последняя строка демонстрирует строку формата и аргументы. Ниже показана строка формата, извлеченная из этого вывода, и пример отформатированной строки из предыдущего вывода команды `perf script`:

```

%d,%d %s %u (%s) %llu + %u [%s]
259,0 W 8192 () 875216 + 16 [kworker/u4:1]

```

Они совпадают.

Трассировщики обычно могут обращаться к аргументам строк формата по их именам. Например, следующая команда `perf(1)` трассирует и выводит события блочного ввода/вывода, если объем этого ввода/вывода (аргумент `bytes`) превышает 65 536¹:

```

# perf trace -e block:block_rq_issue --filter 'bytes > 65536'
0.000 jbd2/nvme0n1p1/174 block:block_rq_issue:259,0 WS 77824 () 2192856 + 152
[jbd2/nvme0n1p1-]
5.784 jbd2/nvme0n1p1/174 block:block_rq_issue:259,0 WS 94208 () 2193152 + 184
[jbd2/nvme0n1p1-]
[...]

```

В качестве примера ниже показано использование другого трассировщика — `bpfttrace` — для вывода аргумента `bytes` только для указанной точки трассировки (синтаксис `bpfttrace` рассматривается в главе 15 «BPF»; я буду использовать `bpfttrace` в последующих примерах, потому что эта команда имеет более компактный синтаксис и требует меньшего количества команд):

```

# bpfttrace -e 't:block:block_rq_issue { printf("size: %d bytes\n", args->bytes); }'
Attaching 1 probe...
size: 4096 bytes
size: 49152 bytes
size: 40960 bytes
[...]

```

Эта команда выводит по одной строке для каждого события ввода/вывода с указанием размера.

Точки трассировки — это стабильный API, включающий имена точек трассировки, строки формата и аргументы.

¹ Аргумент `--filter` был добавлен в команду `perf trace` в Linux 5.5. В старых ядрах то же самое можно сделать с помощью команды `perf trace -e block:block_rq_issue --filter 'bytes>65536' -a; perf script`

Интерфейс точек трассировки

Инструменты трассировки могут использовать точки трассировки через свои файлы событий трассировки в `tracefs` (обычно монтируется в `/sys/kernel/debug/tracing`) или через обращение к системному вызову `perf_event_open(2)`. Например, мой инструмент `iosnoop(8)` на основе `Ftrace` использует файлы в `tracefs`:

```
# strace -e openat ~/Git/perf-tools/bin/iosnoop
chdir("/sys/kernel/debug/tracing") = 0
openat(AT_FDCWD, "/var/tmp/.ftrace-lock", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
[...]
openat(AT_FDCWD, "events/block/block_rq_issue/enable", O_WRONLY|O_CREAT|O_TRUNC,
0666) = 3
openat(AT_FDCWD, "events/block/block_rq_complete/enable", O_WRONLY|O_CREAT|O_TRUNC,
0666) = 3
[...]
```

Как видите, `iosnoop(8)` вызывает `chdir(2)` для перехода в каталог `tracefs` и открывает файлы «enable» с точками трассировки блочного ввода/вывода. Он также открывает файл `/var/tmp/.ftrace-lock`: это предусмотренная мной мера предосторожности, которая блокирует конкурентную работу двух или более экземпляров инструмента, потому что не поддерживается интерфейсом `tracefs`. Интерфейс `perf_event_open(2)` поддерживает конкурентность и потому выглядит предпочтительнее там, где это возможно. Он используется моей новой версией того же инструмента для `BCC`:

```
# strace -e perf_event_open /usr/share/bcc/tools/biosnoop
perf_event_open({type=PERF_TYPE_TRACEPOINT, size=0 /* PERF_ATTR_SIZE_??? */,
config=2323, ...}, -1, 0, -1, PERF_FLAG_FD_CLOEXEC) = 8
perf_event_open({type=PERF_TYPE_TRACEPOINT, size=0 /* PERF_ATTR_SIZE_??? */,
config=2324, ...}, -1, 0, -1, PERF_FLAG_FD_CLOEXEC) = 10
[...]
```

`perf_event_open(2)` — это интерфейс к подсистеме ядра `perf_events`, которая обеспечивает различные возможности профилирования и трассировки. Подробную информацию об этом интерфейсе ищите на странице справочного руководства `man` и в описании `perf(1)` в главе 13.

Оверхед точек трассировки

После активации точки трассировки немного увеличивают время обработки каждого события. Инструмент трассировки может добавить свой оверхед для завершающей обработки событий, а также для записи результатов в файл. Насколько высок этот оверхед и может ли он нарушить нормальную работу промышленных приложений, зависит от частоты следования событий и количества процессоров. Все это вы должны учитывать при использовании точек трассировки.

По моим прикидкам, в типичных современных системах (насчитывающих от 4 до 128 процессоров) частота событий менее 10 000 в секунду влечет незначительный оверхед, и только при частотах больше 100 000 оверхед становится измеримым. Например, события дискового ввода/вывода обычно следуют с частотой менее

10 000 в секунду, но события планировщика легко могут превысить 100 000 в секунду и, соответственно, их трассировка может оказаться слишком дорогостоящей.

В свое время я проанализировал оверхед для конкретной системы и обнаружил, что минимальная величина задержки на обработку точки трассировки составляет 96 нс [Gregg 19]. В 2018 году в Linux 4.7 появился новый тип точек трассировки — *необработанные точки трассировки* (raw tracepoints), позволяющий избежать затрат на создание стабильных аргументов точек трассировки и уменьшающий оверхед.

Помимо явного оверхеда, который несут активированные точки трассировки, есть и неявный, связанный с постоянной готовностью неактивных точек трассировки к активизации. Неактивная точка трассировки становится небольшим количеством инструкций: для x86_64 — это 5-байтовая инструкция `nop` (no-operation). Кроме того, в конец функции добавлен обработчик точки трассировки, который немного увеличивает размер кода. Этот оверхед очень мал, но его тоже нужно учитывать при добавлении точек трассировки в ядро.

Документация с описанием точек трассировки

Документация с подробным описанием технологии точек трассировки есть в исходном коде ядра, в файле `Documentation/trace/tracepoints.rst`. Сами точки трассировки (не все) задокументированы в заголовочных файлах, где определяются. Они находятся в исходном коде ядра Linux в `include/trace/events`. Некоторые продвинутые темы, например, как точки трассировки добавляются в код ядра и как они работают на уровне инструкций, я описал в своей книге «BPF Performance Tools» в главе 2 [Gregg 19].

Иногда бывает нужно выполнить трассировку ПО, не имеющего точек трассировки; в таких случаях можно попробовать нестабильный интерфейс `kprobes`.

4.3.6. kprobes

`kprobes` (сокр. от «kernel probes» — зонды ядра) — это источник событий ядра Linux для трассировщиков, основанный на *динамической инструментации* (этот термин я ввел в главе 1 «Введение», в разделе 1.7.3 «Трассировка»). `kprobes` позволяет трассировать любые функции или инструкции ядра и доступен, начиная с версии Linux 2.6.9, выпущенной в 2004 году. `kprobes` считается нестабильным API, потому что открывает доступ к коду ядра, работа и аргументы которого могут изменяться в зависимости от версии.

`kprobes` может действовать по-разному. Стандартный метод — замена кода некоторой инструкции в нужном месте в коде ядра вызовом функции трассировки. При инструментации точки входа в функцию для оптимизации `kprobes` использует функцию трассировки `Ftrace`, потому что она дает меньший оверхед¹.

¹ Эту оптимизацию можно включать и выключить с помощью переменной `debug.kprobes-optimisation` в `sysctl(8)`.

kprobes очень важен, поскольку является последним спасительным источником¹ практически неограниченной информации о поведении ядра в промышленной среде, которая очень важна для диагностики проблем производительности, неразрешимых с применением других инструментов. Может использоваться трассировщиками, представленными в разделе 4.5 «Инструменты трассировки», и подробно рассматривается в главах 13–15.

В табл. 4.3 приводится сравнение kprobes и точек трассировки.

Таблица 4.3. Сравнение kprobes и точек трассировки

Характеристика	kprobes	Точки трассировки
Тип	Динамический	Статический
Примерное число событий	50 000+	1000+
Поддержка в ядре	Не требуется	Обязательна
Оверхед в неактивном состоянии	Нет	Небольшие (инструкция NOP и небольшой объем метаданных)
Стабильный API	Нет	Да

Технология kprobes позволяет трассировать не только точки входа в функции, но и любые другие инструкции внутри функций. Механизм kprobes создает *события kprobe* (события трассировки на основе kprobe). Эти события kprobe есть только тогда, когда их создает трассировщик: по умолчанию код ядра выполняется без изменений.

Пример использования kprobes

Команда bpftrace показывает пример использования kprobes. Она инструментует функцию ядра `do_nanosleep()` и выводит имя процесса, выполняющегося на процессоре:

```
# bpftrace -e 'kprobe:do_nanosleep { printf("sleep by: %s\n", comm); }'
Attaching 1 probe...
sleep by: mysqld
sleep by: mysqld
sleep by: sleep
^C
#
```

Здесь видно, что в период, когда выполнялась трассировка, процесс «mysqld» приостанавливался два раза, а процесс «sleep» (вероятно, `/bin/sleep`) — один раз.

¹ Без kprobes последним спасительным вариантом было бы изменение кода ядра, явное добавление инструментующего кода, компиляция, сборка и повторное развертывание измененного ядра.

Как показывают полученные результаты, `do_nanosleep()` обычно выполняется очень быстро — она 1280 раз выполнялась 0 мс (здесь выполняется округление в меньшую сторону). В двух случаях продолжительность составила от 512 до 1024 мс.

Синтаксис `bpfttrace` объясняется в главе 15 «BPF», где приводится аналогичный пример оценки продолжительности выполнения `vfs_read()`.

Интерфейс и оверхед kprobes

Интерфейс `kprobes` похож на точки трассировки. Инструментацию можно выполнить через файлы `/sys`, через системный вызов `perf_event_open(2)` (что предпочтительнее), а также через API ядра `register_kprobe()`. Оверхед `kprobes` сопоставим с оверхедом точек трассировки, когда инструментруются точки входа в функции (используется `ftrace`, если возможно), и выше, когда инструментруются инструкции, отстоящие на некотором расстоянии от точек входа в функции (используется метод точек останова), или когда используются `kretprobes` (с помощью функции `kretprobe_trampoline()`). В одной из своих систем я измерил минимальные затраты времени на обработку `kprobe`, и они составили 76 нс. Минимальные затраты на `kretprobe` составили 212 нс [Gregg 19].

Документация с описанием kprobes

Документацию с подробным описанием `kprobes` можно найти в исходном коде ядра Linux, в файле `Documentation/kprobes.txt`. Инструментируемые с их помощью функции ядра, как правило, не документируются вне исходного кода ядра (потому что большинство из них не являются составной частью API, а кроме того, в этом нет никакой необходимости). Некоторые продвинутые темы — например, как `kprobes` работает на уровне инструкций — я описал в своей книге «BPF Performance Tools» в главе 2 [Gregg 19].

4.3.7. uprobes

Зонды `uprobes` (`user-space probes` — зонды для пространства пользователя) похожи на `kprobes`, но предназначены для пространства пользователя. Они могут динамически инструментировать функции в приложениях и библиотеках и предлагают нестабильный API для погружения во внутреннее устройство программного обеспечения на глубины, недоступные другим инструментам. `uprobes` появились в 2012 году в Linux 3.5.

`uprobes` могут использоваться трассировщиками, представленными в разделе 4.5 «Инструменты трассировки», и подробно рассматриваются в главах 13–15.

Пример использования uprobes

Команда `bpfttrace` перечисляет доступные для инструментации точки входа в функции в командной оболочке `bash(1)`:

```
# bpftrace -l 'uprobe:/bin/bash:*'
uprobe:/bin/bash:rl_old_menu_complete
uprobe:/bin/bash:maybe_make_export_env
uprobe:/bin/bash:initialize_shell_builtins
uprobe:/bin/bash:extglob_pattern_p
uprobe:/bin/bash:dispose_cond_node
uprobe:/bin/bash:decode_prompt_string
[...]
```

Полный список содержит 1507 доступных для uprobes точек. Механизм uprobes инструментирует код и при необходимости создает *события uprobes* (события трассировки на основе uprobes); по умолчанию код в пространстве пользователя выполняется без изменений. Этот подход напоминает действия отладчика при добавлении точки останова в функцию: если точек останова нет, функция выполняется без изменений.

Аргументы uprobes

Механизм uprobes позволяет получить аргументы функций в пользовательском пространстве. Например, вот как можно вывести первый строковый аргумент функции `decode_prompt_string()` в командной оболочке `bash` с помощью `bpftrace`:

```
# bpftrace -e 'uprobe:/bin/bash:decode_prompt_string { printf("%s\n", str(arg0));
}'
Attaching 1 probe...
\[ \e[31;1m\] \u@\h: \w> \[ \e[0m\]
\[ \e[31;1m\] \u@\h: \w> \[ \e[0m\]
^C
```

Как видите, это строка приглашения к вводу в `bash(1)` в системе. Событие `uprobe` для `decode_prompt_string()` создается в момент запуска программы `bpftrace` и удаляется по ее завершении (`Ctrl+C`).

uretprobes

Трассировку возвратов из пользовательских функций и их возвращаемые значения можно выполнить с помощью *uretprobes* (сокр. от «user-level return probes» — зонды для точек возврата в пространстве пользователя), который похож на uprobes. В сочетании с uprobes и трассировщиком, который сохраняет отметки времени, можно измерить продолжительность выполнения функции в пространстве пользователя. Имейте в виду, что оверхед на обслуживание uretprobes может значительно исказить результаты для быстрых функций.

Интерфейс и оверхед uretprobes

Интерфейс uprobes похож на kprobes. Инструментацию можно выполнить через файлы `/sys` и через системный вызов `perf_event_open(2)`, что предпочтительнее.

Сейчас работа механизма uprobes основана на использовании обработчиков в ядре. По этой причине оверхед на uprobes выше, чем на kprobes или точки трассировки.

В одной из своих систем я измерил минимальные затраты времени на обработку `uprobe`, и они составили 1287 нс. Минимальные затраты на обслуживание `uretprobe` — 1931 нс [Gregg 19]. Оверхед `uretprobe` выше, потому что складывается из затрат на обслуживание `uprobe` и выполнение функции-обработчика.

Документация с описанием `uprobe`

Документацию с подробным описанием технологии `uprobes` можно найти в исходном коде ядра Linux, в файле `Documentation/trace/uprobetracer.rst`. Некоторые продвинутые темы, как зонды `uprobes` работают на уровне инструкций, я описал в своей книге «BPF Performance Tools» в главе 2 [Gregg 19]. Инструментируемые с их помощью функции в пространстве пользователя обычно не документируются вне исходного кода приложений (потому что большинство из них не являются составной частью API, а кроме того, в этом нет никакой необходимости). Если вам нужны документированные средства трассировки функций пользовательского пространства, используйте USDT.

4.3.8. USDT

USDT (user-level statically-defined tracing — статически определенные пользователем точки трассировки) — это версия точек трассировки для пространства пользователя. USDT отличаются от `uprobes` так же, как точки трассировки отличаются от `kprobes`. Некоторые приложения и библиотеки добавляют USDT, предлагая стабильный (и документированный) API для трассировки событий на уровне приложений. Например, USDT есть в Java JDK, в PostgreSQL и в `libc`. Ниже приводится список USDT в OpenJDK, полученный с помощью `bpfftrace`:

```
# bpfftrace -lv 'usdt:/usr/lib/jvm/openjdk/libjvm.so:*'
usdt:/usr/lib/jvm/openjdk/libjvm.so:hotspot:class__loaded
usdt:/usr/lib/jvm/openjdk/libjvm.so:hotspot:class__unloaded
usdt:/usr/lib/jvm/openjdk/libjvm.so:hotspot:method__compile__begin
usdt:/usr/lib/jvm/openjdk/libjvm.so:hotspot:method__compile__end
usdt:/usr/lib/jvm/openjdk/libjvm.so:hotspot:gc__begin
usdt:/usr/lib/jvm/openjdk/libjvm.so:hotspot:gc__end
[...]
```

Здесь перечислены зонды USDT для трассировки загрузки и выгрузки классов Java, компиляции методов и сборки мусора. Это лишь часть списка: полный список включает 524 зонда USDT.

Многие приложения уже поддерживают средства журналирования событий, которые можно включить, настроить и применять для анализа производительности. Отличительная черта зондов USDT — возможность использовать их в различных трассировщиках, которые могут комбинировать контекст приложения с такими событиями ядра, как дисковый и сетевой ввод/вывод. Средство журналирования уровня приложения может сообщить, что запрос к базе данных выполнялся медленно из-за задержек ввода/вывода в файловой системе. Трассировщик же может

дать больше информации: например, сообщить, что долгое обслуживание запроса было обусловлено конфликтом блокировок в файловой системе, а не дисковым вводом/выводом, как вы предполагали.

Некоторые приложения содержат зонды USDT, но они отключены в коробочной версии приложения (как в случае с OpenJDK). Для их использования нужно пере-собрать приложение из исходного кода с соответствующим конфигурационным параметром. Этот параметр может называться `--enable-dtrace-probes` в честь трассировщика DTrace, способствовавшего внедрению USDT в приложениях.

Зонды USDT должны быть скомпилированы в исполняемый файл. Но для языков с динамической компиляцией, например Java, которая обычно компилирует исходный код на лету, это невозможно. Решением здесь станет использование *динамического механизма USDT*, который предварительно компилирует зонды как разделяемую библиотеку и предоставляет интерфейс для их вызова из динамически скомпилированного кода. Библиотеки динамического механизма USDT бывают для Java, Node.js и т. д. Интерпретируемые языки имеют аналогичную проблему и нуждаются в динамическом USDT.

Зонды USDT реализованы в Linux с помощью `uprobes`: см. предыдущий раздел, где описывается механизм `uprobes` и его оверхед. Помимо явного оверхеда, который несет активированные зонды USDT, есть и неявный на выполнение инструкций `por`, как и в случае с точками трассировки.

Зонды USDT могут использоваться трассировщиками, представленными в разделе 4.5 «Инструменты трассировки», и подробно рассматриваются в главах 13–15 (но важно помнить, что использование USDT с `Ftrace` требует дополнительной работы).

Документация с описанием USDT

Если приложение реализует зонды USDT, они должны быть описаны в документации приложения. Некоторые продвинутые темы — например, как зонды USDT добавляются в код приложения, как работают и прочие вопросы динамических зондов USDT — я описал в своей книге «BPF Performance Tools» в главе 2 [Gregg 19].

4.3.9. Аппаратные счетчики (PMC)

Процессор и другие устройства обычно поддерживают аппаратные счетчики для наблюдения за их активностью. Основным источником служат процессоры; их счетчики обычно называют *счетчиками мониторинга производительности* (performance monitoring counters, PMC). Они также известны как *счетчики производительности процессоров* (CPU performance counters, CPC), *инструментальные счетчики производительности* (performance instrumentation counters, PIC) и *события модуля мониторинга событий производительности* (performance monitoring unit events, PMU события). Все эти названия обозначают одно и то же: программируемые аппаратные регистры процессора, которые предоставляют низкоуровневую информацию о производительности на уровне тактов процессора.

РМС — чрезвычайно важный ресурс для анализа производительности. Только с помощью РМС можно оценить эффективность инструкций процессора, долю попаданий в кэш процессора, потребление памяти и нагрузку на шины устройств, количество холостых тактов и т. д. Их использование в анализе помогает в поиске различных оптимизаций производительности.

Примеры РМС

Есть множество разных РМС, но Intel выбрала семь из них, основав «архитектурный набор», который позволяет получить общее представление об основных параметрах функционирования [Intel 16]. Наличие такого архитектурного набора счетчиков производительности РМС можно проверить с помощью инструкции `cpuid`. В табл. 4.4 показан пример набора полезных РМС.

Таблица 4.4. Архитектурный набор РМС компании Intel

Название события	UMask	Селектор события	Пример мнемоники маски события
UnHalted Core Cycles (счетчик непрерывных тактов ядра)	00H	3CH	CPU_CLK_UNHALTED.THREAD_P
Instruction Retired (счетчик выполненных инструкций)	00H	C0H	INST_RETIRED.ANY_P
UnHalted Reference Cycles (счетчик непрерывных опорных тактов)	01H	3CH	CPU_CLK_THREAD_UNHALTED.REF_XCLK
LLC References (счетчик обращений к кэшу последнего уровня)	4FH	2EH	LONGEST_LAT_CACHE.REFERENCE
LLC Misses (счетчик промахов кэша последнего уровня)	41H	2EH	LONGEST_LAT_CACHE.MISS
Branch Instruction Retired (счетчик выполненных инструкций ветвления)	00H	C4H	BR_INST_RETIRED.ALL_BRANCHES
Branch Misses Retired (счетчик ошибочно выбранных инструкций ветвления)	00H	C5H	BR_MISP_RETIRED.ALL_BRANCHES

Вот небольшой пример использования РМС: если запустить команду `perf stat` без указания событий (без параметра `-e`), то по умолчанию она использует архитектурные РМС. Следующая команда применяет `perf stat` к команде `gzip(1)`:

```
# perf stat gzip words
```

```
Performance counter stats for 'gzip words':
```

156.927428	task-clock (msec)	# 0.987 CPUs utilized
1	context-switches	# 0.006 K/sec
0	cpu-migrations	# 0.000 K/sec
131	page-faults	# 0.835 K/sec
209,911,358	cycles	# 1.338 GHz
288,321,441	instructions	# 1.37 insn per cycle
66,240,624	branches	# 422.110 M/sec
1,382,627	branch-misses	# 2.09% of all branches

```
0.159065542 seconds time elapsed
```

В первой колонке выводятся фактические значения счетчиков. За символом хеша (#) следует некоторая статистика, в том числе такая важная метрика производительности, как количество инструкций на такт (*insn per cycle*). Она показывает, насколько эффективно процессоры выполняют инструкции: чем выше значение этой метрики, тем лучше. Подробнее о ней я расскажу в главе 6 «Процессоры» в разделе 6.3.7 «IPC, CPI».

Интерфейс РМС

В Linux счетчики РМС доступны через системный вызов `perf_event_open(2)` и используются многими инструментами, включая `perf(1)`.

В современных системах доступны сотни аппаратных счетчиков РМС, но количество регистров в процессорах для их одновременного измерения ограничено, обычно всего шесть. Поэтому вы должны выбрать, какие счетчики РМС измерять в этих шести регистрах, либо циклически перебирать разные наборы РМС (`perf(1)` в Linux поддерживает это автоматически). Программные счетчики не страдают от этих ограничений.

Счетчики РМС могут использоваться в разных режимах: в режиме *счета*, когда ведется простой подсчет событий практически с нулевым оверхедом, а также в режиме *выборки при переполнении*, когда для определенного настраиваемого числа событий генерируется прерывание, чтобы обработать это состояние. Режим счета можно использовать для количественной оценки проблем. Режим выборки при переполнении можно использовать для определения пути в коде, ответственного за эти события.

`perf(1)` может выполнять счет с помощью подкоманды `stat`, а выборку — с помощью подкоманды `record`; см. главу 13 «`perf`».

Проблемы РМС

Две общие проблемы при использовании РМС — точность их работы в режиме выборки при переполнении и доступность в облачных средах.

Метод выборки при переполнении может давать неверный адрес инструкции, вызвавшей событие, из-за задержки прерывания (иногда эффект задержки называют сдвигом (*skid*)) или из-за выполнения инструкции вне очереди. Для профилирования тактов процессора это не проблема, а некоторые профилировщики даже намеренно вносят сдвиг, чтобы избежать попадания в одну и ту же точку (или используют частоту выборки со смещением, например, 99 Гц). Но для измерения других событий, например промахов кэша последнего уровня, адрес инструкции, ответственной за событие, должен быть точным.

Решением является поддержка в процессоре так называемых *точных событий*. В Intel для этой цели применяется технология под названием *точная выборка на основе событий* (*precise event-based sampling*, PEBS)¹, которая использует аппаратные буферы для более точной фиксации состояния указателя инструкций в момент появления события РМС. В AMD используется прием выборки на основе инструкций (*instruction-based sampling*, IBS) [Drongowski 07]. Команда `perf(1)` в Linux поддерживает точные события (см. главу 13 «perf», раздел 13.9.2 «Профилирование процессора»).

Еще одна проблема — облачные вычисления, потому что многие облачные среды отключают доступ к РМС для гостевых систем. Технически организовать такой доступ возможно: например, в гипервизоре Xen есть параметр командной строки `vrmi`, который позволяет предоставлять гостевым системам различные наборы РМС² [Xenbits 20]. Например, Amazon предоставляет множество счетчиков РМС в гипервизоре Nitro для гостевых систем³. Кроме того, некоторые облачные провайдеры предлагают «экземпляры на голом железе», предоставляя гостевым системам полный доступ к процессору и, следовательно, к счетчикам РМС.

Документация с описанием РМС

Счетчики РМС зависят от процессора и описываются в соответствующем руководстве разработчика ПО для этого процессора. Вот несколько примеров по производителям процессоров:

- **Intel:** глава 19 «Performance Monitoring Events» в «Intel® 64 and IA-32 Architectures Software Developer’s Manual Volume 3» [Intel 16];

¹ В некоторых документах Intel аббревиатура PEBS расшифровывается иначе, например: выборка на основе событий *процессора* — *processor event-based sampling*.

² Я написал код для Xen, который допускает различные режимы РМС: «*ipc*» — только с поддержкой счетчика инструкций на такт и «*arch*» — для архитектурного набора Intel. Мой код работал как простой брандмауэр для существующей поддержки `vrmi` в Xen.

³ Сейчас это возможно только для крупных экземпляров Nitro, когда виртуальная машина монопольно владеет процессором.

- **AMD:** раздел 2.1.1 «Performance Monitor Counters» в «Open-Source Register Reference For AMD Family 17h Processors Models 00h-2Fh» [AMD 18];
- **ARM:** раздел D7.10 «PMU Events and Event Numbers» в «Arm® Architecture Reference Manual Armv8, for Armv8-A architecture profile» [ARM 19].

В свое время была проведена работа по созданию стандартной схемы именования счетчиков РМС, которая могла бы поддерживаться всеми процессорами. Ее результатом стало появление *интерфейса прикладного программирования производительности* (performance application programming interface, PAPI) [UTK 20]. Поддержка PAPI в операционных системах не лишена недостатков: она требует частых обновлений для приведения в соответствие имен PAPI с кодами РМС производителей.

Более подробно о реализации счетчиков РМС я расскажу в главе 6 «Процессоры», в разделе 6.4.1 «Аппаратное обеспечение», подраздел «Аппаратные счетчики (РМС)». Там же вы найдете дополнительные примеры РМС.

4.3.10. Другие источники информации для наблюдения

Другие источники наблюдаемости включают:

- **MSR:** счетчики РМС реализуются с использованием регистров, зависящих от модели (model-specific registers, MSR). Есть и другие такие регистры для представления информации о конфигурации и работоспособности системы, включая тактовую частоту процессора, нагрузку, температуру и энергопотребление. Перечень поддерживаемых регистров MSR зависит от типа (модели) процессора, версии и настроек BIOS, а также настроек гипервизора. Одно из применений — точное измерение нагрузки на процессор с точностью до такта.
- **ptrace(2):** этот системный вызов управляет трассировкой процесса; он используется отладчиком gdb(1) для отладки процессов и трассировщиком strace(1) для трассировки системных вызовов. Системный вызов основан на точках останова и может замедлить цель более чем в 100 раз. В Linux также имеются точки трассировки, представленные в разделе 4.3.5 «Точки трассировки», предлагающие более эффективный способ трассировки системных вызовов.
- **Профилирующие функции:** в начало всех невстроенных (non-inlined) функций ядра на платформе x86 добавляются вызовы профилирующих функций (mcount() или __fentry__()), обеспечивающие эффективную трассировку с помощью Ftrace. Когда они не нужны, вместо них вставляются инструкции пор. См. главу 14 «Ftrace».
- **Анализ сетевых пакетов (libpcap):** эти интерфейсы позволяют перехватывать сетевые пакеты, поступающие из сетевых устройств, для детального исследования производительности механизмов обработки пакетов и протоколов. В Linux перехват пакетов поддерживается через библиотеку libpcap и /proc/net/dev и используется утилитой tcpdump(8). Не следует забывать, что перехват и изучение пакетов сопряжены с расходами не только процессорного времени, но и памяти

для хранения пакетов. За дополнительной информацией о перехвате сетевых пакетов обращайтесь к главе 10.

- **netfilter conntrack:** технология netfilter в Linux способна вызывать пользовательские обработчики событий не только брандмауэра, но и трассировщика соединений (connection tracking — conntrack), что позволяет журналировать сетевые потоки [Ayuso 12].
- **Механизм учета процессов:** этот механизм восходит к временам больших ЭВМ, когда требовалось выставлять счета отделам и пользователям за использование машин в зависимости от времени выполнения их процессов. В той или иной форме он есть в Linux и в других системах и иногда может пригодиться для анализа производительности на уровне процессов. Например, инструмент atop(1) в Linux использует механизм учета процессов для сбора и отображения информации о короткоживущих процессах, которые в противном случае можно не заметить при создании моментальных снимков /proc [Atoptool 20].
- **Программные события:** они подобны аппаратным событиям, но генерируются программно. Примером может служить отказ страницы. Программные события доступны через интерфейс perf_event_open(2) и используются в perf(1) и bpftrace. Они изображены на рис. 4.5.
- **Системные вызовы:** некоторые метрики производительности можно получить с помощью системных или библиотечных вызовов. К ним относится системный вызов getrusage(2), который процессы могут использовать для получения собственной статистики использования ресурсов, включая время выполнения в пространстве пользователя и системы, количество отказов, сообщений и переключений контекста.

Если вам интересно, как работает каждый из этих источников, поищите соответствующую документацию для разработчиков, создающих инструменты на основе этих интерфейсов.

И другие...

В зависимости от версии ядра и включенных параметров конфигурации могут быть доступны другие источники информации. О некоторых я расскажу в последующих главах этой книги. В случае с Linux к ним относятся механизм учета ввода/вывода, blktrace, timer_stats, lockstat и debugfs.

Один из способов найти такие источники — исследовать исходный код ядра, наблюдение за которым вас интересует, и посмотреть, какие статистики или точки трассировки в нем есть.

В некоторых случаях нужных статистик может не быть. Тогда помимо средств динамической инструментации (kprobes и uprobes в Linux) можно использовать отладчики — gdb(1) и lldb(1) — для получения значений переменных ядра и приложений. Это позволит пролить свет на интересующую проблему.

Kstat в Solaris

Еще одним способом получения статистик системы может служить система поддержки статистик ядра (kernel statistics, Kstat) в Solaris, реализующая согласованную иерархическую структуру статистик ядра, каждая из которых получает имя с использованием кортежа из четырех элементов:

модуль:экземпляр:имя:статистика

Вот их краткое описание:

- **модуль:** обычно это модуль ядра, создающий статистику, например `sd` для драйвера диска SCSI или `zfs` для файловой системы ZFS;
- **экземпляр:** некоторые модули действуют в виде нескольких экземпляров, например, для каждого диска SCSI используется свой экземпляр модуля `sd`. Экземпляр — это перечисление;
- **имя:** имя группы статистик;
- **статистика:** имя отдельной статистики.

Доступ к системе Kstat осуществляется через двоичный интерфейс ядра, и для этого есть различные библиотеки.

Вот пример использования системы Kstat, где команда `kstat(1M)` читает статистику «прос», определяемую полным кортежем:

```
$ kstat -p unix:0:system_misc:nproc
unix:0:system_misc:nproc 94
```

Эта статистика показывает текущее количество запущенных процессов.

Для сравнения: источники в стиле `/proc/stat` в Linux имеют несовместимое форматирование и обычно требуют синтаксического анализа текста для извлечения нужной информации. А это дополнительная нагрузка на процессор.

4.4. SAR

Инструмент мониторинга `sar(1)` был представлен в разделе 4.2.4 «Мониторинг». Хотя последнее время возник ажиотаж в отношении суперспособностей BPF в трассировке (часть ответственности за это лежит и на мне), но не упускайте из виду `sar(1)` — это важный инструмент оценки производительности системы, способный помочь решить многие проблемы производительности без привлечения других инструментов. Версия `sar(1)` для Linux хорошо спроектирована, имеет понятные заголовки столбцов, группы сетевых метрик и подробную документацию (`man`).

`sar(1)` распространяется в составе пакета `sysstat`.

4.4.1. Область покрытия sar(1)

На рис. 4.6 показана область покрытия инструментом sar(1) с использованием разных параметров командной строки.

Вы видите, что sar(1) имеет широкое покрытие ядра и устройств и ему доступна даже информация о состоянии вентиляторов. Параметр `-m` (управление питанием) поддерживает еще целый ряд аргументов, не показанных на этом рисунке, в том числе `IN` для получения величины входного напряжения, `TEMP` для получения температуры устройства и `USB` для получения статистик питания USB-устройств.

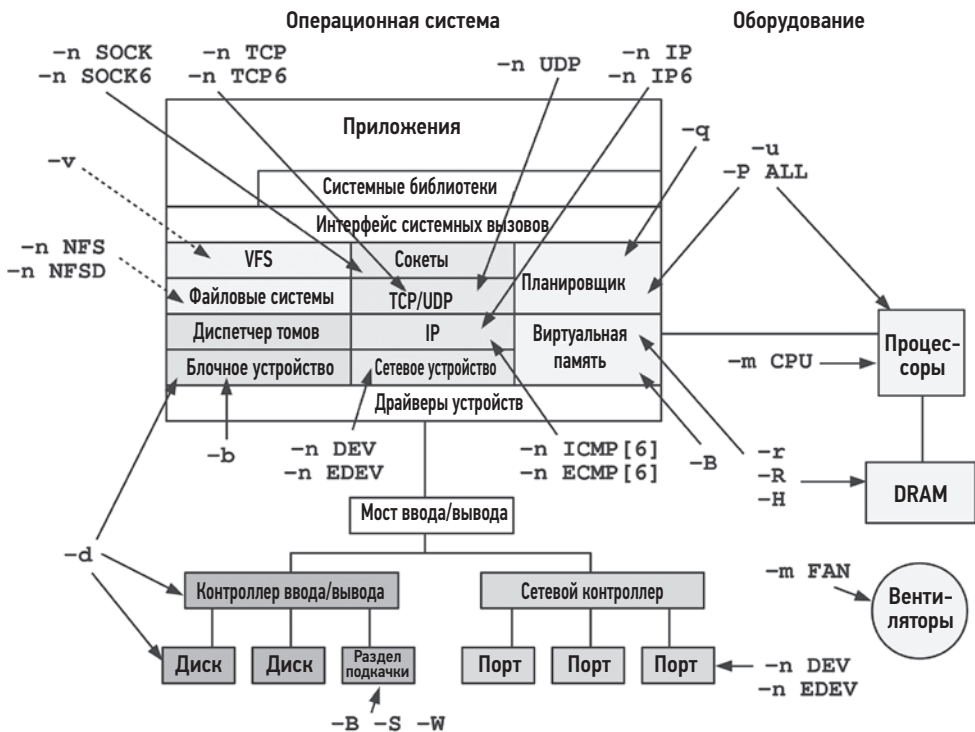


Рис. 4.6. Область охвата sar(1) в Linux

4.4.2. Мониторинг с sar(1)

Достаточно часто сбор (мониторинг) данных в sar(1) уже включен в системах Linux. Если это не так, нужно включить его. Для проверки просто запустите `sar` без параметров. Например:

```
$ sar
Cannot open /var/log/sysstat/sa19: No such file or directory
Please check if data collecting is enabled
```

В этом примере мы видим сообщение о том, что в системе сбор данных в `sar(1)` еще не включен (файл `sa19` в этом примере — это ежедневный архив за 19-е число месяца). В разных дистрибутивах включение сбора данных происходит по-разному.

Конфигурация (Ubuntu)

В этой системе Ubuntu можно включить сбор данных в `sar(1)`, отредактировав файл `/etc/default/sysstat` и присвоив параметру `ENABLED` значение `true`:

```
ubuntu# vi /etc/default/sysstat
#
# Настройки по умолчанию для файлов /etc/init.d/sysstat, /etc/cron.d/sysstat
# и /etc/cron.daily/sysstat
#
# Должна ли утилита sads собирать информацию об активности системы? Допустимые
# значения: "true" и "false". Не используйте никакие другие значения!
# Они будут затерты утилитой debconf!
ENABLED="true"
```

и затем перезапустив сервис `sysstat`:

```
ubuntu# service sysstat restart
```

Расписание записи статистик можно изменить в файле `crontab` для `sysstat`:

```
ubuntu# cat /etc/cron.d/sysstat
# Первый элемент в пути — это каталог, где находится сценарий debian-sa1
PATH=/usr/lib/sysstat:/usr/sbin:/usr/sbin:/usr/bin:/sbin:/bin

# Собирать данные об активности каждые 10 минут ежедневно
5-55/10 * * * * root command -v debian-sa1 > /dev/null && debian-sa1 1 1

# Дополнительно запускать в 23:59 для ротации файла со статистиками
59 23 * * * root command -v debian-sa1 > /dev/null && debian-sa1 60 2
```

Синтаксис `5-55/10` означает, что запись будет производиться каждые 10 минут в диапазоне минут от 5 до 55 текущего часа. Вы можете изменить расписание в соответствии с желаемым разрешением: синтаксис задокументирован на странице справочного руководства `man` для `crontab(5)`. Более частый сбор данных увеличит размер файлов архива `sar(1)`, которые сохраняются в `/var/log/sysstat`.

Я часто меняю расписание, как показано ниже:

```
*/5 * * * * root command -v debian-sa1 > /dev/null && debian-sa1 1 1 -S ALL
```

Здесь запись будет выполняться каждые 5 минут (`*/5`), а параметр `-S ALL` требует записывать все статистики. По умолчанию `sar(1)` записывает большинство (но не все) группы статистик. Параметр `-S ALL` требует сохранять все группы статистик — он передается в `sadc(1)` и задокументирован на странице справочного руководства для `sadc(1)`. Есть и расширенная версия параметра — `-S XALL`, которая записывает статистики с дополнительной разбивкой.

Получение отчетов

sar(1) можно запустить с любыми параметрами, показанными на рис. 4.6, чтобы получить выбранную группу статистик. Также можно указать несколько параметров. Например, следующий отчет содержит статистики о потреблении процессора (-u), TCP (-n TCP) и ошибках TCP (-n ETCP):

```
$ sar -u -n TCP,ETCP
Linux 4.15.0-66-generic (bgregg) 01/19/2020      _x86_64_      (8 CPU)

10:40:01 AM      CPU      %user      %nice      %system      %iowait      %steal      %idle
10:45:01 AM      all        6.87        0.00        2.84        0.18        0.00        90.12
10:50:01 AM      all        6.87        0.00        2.49        0.06        0.00        90.58
[...]
10:40:01 AM      active/s  passive/s      iseg/s      oseg/s
10:45:01 AM          0.16          0.00        10.98        9.27
10:50:01 AM          0.20          0.00        10.40        8.93
[...]
10:40:01 AM      atmpth/s  estres/s  retrans/s  isegerr/s  orsts/s
10:45:01 AM          0.04          0.02          0.46          0.00          0.03
10:50:01 AM          0.03          0.02          0.53          0.00          0.03
[...]
```

Первая строка содержит сводную информацию о системе: тип и версию ядра, имя хоста, дату, архитектуру и количество процессоров.

Запуск `sar -A` выводит все статистики.

Форматы вывода

В состав пакета `sysstat` входит утилита `sadf(1)` для просмотра статистик `sar(1)` в различных форматах, включая JSON, SVG и CSV. Далее приводятся примеры отображения статистик TCP (-n TCP) в этих форматах.

JSON (-j)

JSON легко можно проанализировать и импортировать во многих языках, что делает его удобным форматом вывода при создании ПО на основе `sar(1)`.

```
$ sadf -j -- -n TCP
{"sysstat": {
  "hosts": [
    {
      "nodename": "bgregg",
      "sysname": "Linux",
      "release": "4.15.0-66-generic",
      "machine": "x86_64",
      "number-of-cpus": 8,
      "file-date": "2020-01-19",
      "file-utc-time": "18:40:01",
      "statistics": [
        {
          "timestamp": {"date": "2020-01-19", "time": "18:45:01", "utc": 1,
```

```
"interval": 300},
  "network": {
    "net-tcp": {"active": 0.16, "passive": 0.00, "iseg": 10.98, "oseg": 9.27}
  },
  [...]
}
```

Данные в JSON можно обрабатывать в командной строке с помощью jq(1).

SVG (-g)

sadf(1) может генерировать файлы в формате SVG для просмотра в браузерах. Пример такого файла показан на рис. 4.7. Этот формат можно использовать для создания простых дашбордов.

Linux 4.15.0-66-generic (bgregg) 01/19/2020 _x86_64_ (8 CPU)

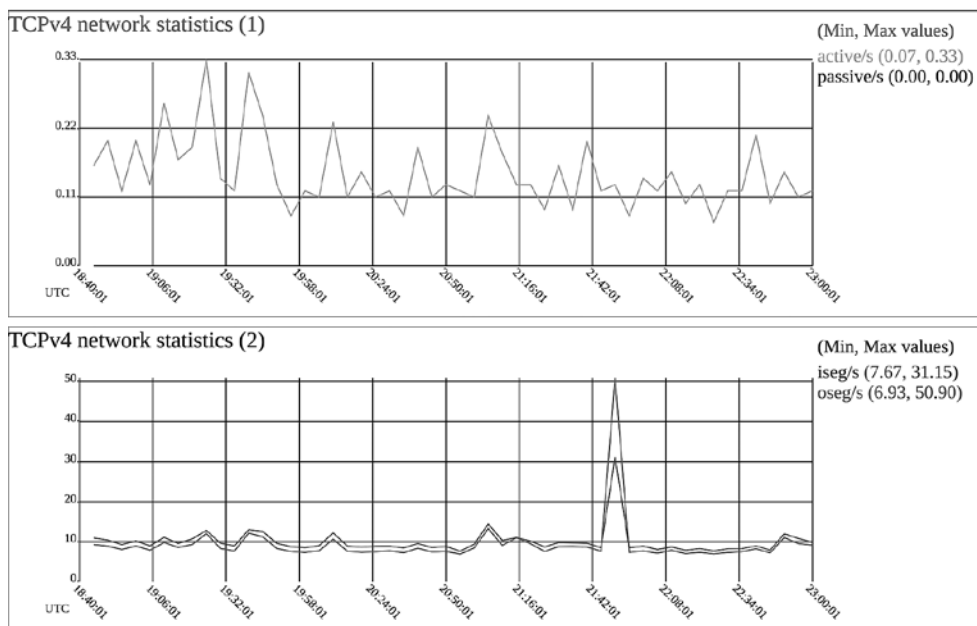


Рис. 4.7. Вывод статистик sar(1) утилитой sadf(1) в формате SVG¹

CSV (-d)

Формат CSV предназначен для экспорта в базы данных (и в качестве разделителя использует точку с запятой):

¹ Обратите внимание, что я отредактировал файл SVG, чтобы сделать этот рисунок более удобочитаемым, изменив цвета и увеличив размер шрифта.

```
$ sadf -d -- -n TCP
# hostname;interval;timestamp;active/s;passive/s;iseg/s;oseg/s
bgregg;300;2020-01-19 18:45:01 UTC;0.16;0.00;10.98;9.27
bgregg;299;2020-01-19 18:50:01 UTC;0.20;0.00;10.40;8.93
bgregg;300;2020-01-19 18:55:01 UTC;0.12;0.00;9.27;8.07
[...]
```

4.4.3. Вывод оперативной информации с помощью sar(1)

При запуске команды с интервалом и необязательным счетчиком sar(1) создает оперативные отчеты. Этот режим можно использовать, даже если сбор данных выключен.

Вот пример отображения статистик TCP с интервалом в одну секунду и значением счетчика, равным 5:

```
$ sar -n TCP 1 5
Linux 4.15.0-66-generic (bgregg) 01/19/2020      _x86_64_      (8 CPU)
03:09:04 PM active/s passive/s   iseg/s   oseg/s
03:09:05 PM      1.00      0.00      33.00      42.00
03:09:06 PM      0.00      0.00     109.00      86.00
03:09:07 PM      0.00      0.00     107.00      67.00
03:09:08 PM      0.00      0.00     104.00     119.00
03:09:09 PM      0.00      0.00      70.00      70.00
Average:          0.20      0.00      84.60      76.80
```

Сбор данных предназначен для анализа длительных интервалов, например 5 или 10 минут, тогда как оперативные отчеты позволяют просматривать статистики с разрешением до секунды.

В главах ниже приводится множество примеров получения оперативных статистик с помощью sar(1).

4.4.4. Документация с описанием sar(1)

Страница справочного руководства sar(1) описывает отдельные статистики и включает имена SNMP в квадратных скобках. Например:

```
$ man sar
[...]
    active/s
        Количество раз в секунду, когда соединения TCP
        совершали прямой переход в состояние SYN-SENT из
        состояния CLOSED [tcpActiveOpens].
    passive/s
        Количество раз в секунду, когда соединения TCP
        совершали прямой переход в состояние SYN-RCVD из
        состояния LISTEN [tcpPassiveOpens].
    iseg/s
        Общее количество сегментов, полученных за секунду,
        включая сегменты, принятые с ошибкой [tcpInSegs].
        В это число входят сегменты, полученные в текущих
        установленных соединениях.
[...]
```


Далее в книге вы найдете варианты использования `sar(1)`, см. главы 6–10. Общая сводка параметров и метрик `sar(1)` приведена в приложении С.

4.5. ИНСТРУМЕНТЫ ТРАССИРОВКИ

Инструменты трассировки в Linux используют ранее описанные интерфейсы событий (точки трассировки, `kprobes`, `uprobes`, `USDT`) для расширенного анализа производительности. К основным инструментам трассировки относятся:

- **perf(1)**: официальный профилировщик в Linux. Отлично подходит для профилирования процессорного времени (за счет выборки трассировок стека) и анализа счетчиков производительности PMC. Также может регистрировать другие события и записывать их в выходной файл для последующей обработки.
- **Ftrace**: официальный трассировщик в Linux. Многофункциональный инструмент, состоящий из различных утилит трассировки. Подходит для анализа путей в коде ядра и для систем с ограниченными ресурсами, так как его можно использовать без зависимостей.
- **BPF (BCC, bpftrace)**: расширенный BPF. Был представлен в главе 3 «Операционные системы», в разделе 3.4.4 «Расширенный BPF». Поддерживает улучшенные инструменты трассировки, основными из которых являются BCC и `bpftrace`. BCC предоставляет мощные инструменты, а `bpftrace` — язык высокого уровня для создания пользовательских однострочных и коротких программ.
- **SystemTap**: язык высокого уровня и трассировщик со множеством библиотек для трассировки различных целей [Eigler 05][Sourceware 20]. Недавно для него была разработана поддержка BPF — рекомендую обратить на это внимание (см. страницу справочного руководства для `starbpf(8)`).
- **LTtng**: трассировщик, оптимизированный для записи в «черный ящик»: оптимизирует запись большого количества событий для последующего анализа [LTtng 20].

Первые три трассировщика рассматриваются в главах 13 «`perf`», 14 «`Ftrace`» и 15 «`BPF`». В следующих главах (5–12) будут описаны разные способы использования этих трассировщиков, показаны конкретные команды и способы интерпретации результатов. Такой порядок выбран намеренно: сначала рассматриваются примеры использования и анализа производительности, а затем подробно рассматриваются сами трассировщики.

В Netflix я использую `perf(1)` для анализа потребления процессора, `Ftrace` — для исследования особенностей работы кода ядра и `BCC/bpftrace` — для всего остального (память, файловые системы, диски, сеть и трассировка приложений).

4.6. НАБЛЮДЕНИЕ ЗА НАБЛЮДАЕМОСТЬЮ

Инструменты наблюдения и статистики, на которых они основаны, реализованы программно, а любое ПО может содержать ошибки. То же верно и в отношении

документации. Относитесь со здоровым скептицизмом к любой новой для вас статистике, проверяйте, верна ли она и что она на самом деле означает.

Метрики могут быть подвержены любой из следующих проблем:

- инструменты и их результаты измерений иногда содержат ошибки;
- страницы справочного руководства не всегда содержат верную информацию;
- доступные метрики могут быть неполными;
- доступные метрики могут быть плохо спроектированы и вводить в заблуждение;
- средства сбора метрик (например, инструменты анализа результатов) могут содержать ошибки¹;
- обработка метрик (алгоритмы/электронные таблицы) также может выполняться с ошибками.

Если есть несколько инструментов наблюдения с перекрывающимися областями покрытия, используйте их для перекрестной проверки. В идеале они будут использовать разные средства инструментации, чтобы, в свою очередь, проверить их на наличие ошибок. Для этой цели особенно удобны средства динамической инструментации, поскольку они позволяют создавать свои инструменты для двойной проверки метрик.

Еще один метод проверки — применение *известных рабочих нагрузок* с последующей проверкой того, насколько результаты инструментов наблюдения согласуются с вашими ожиданиями. Для этого можно использовать инструменты микробенчмаркинга, сообщающие свою статистику для сравнения.

Иногда ошибки бывают не в инструменте или статистике, а в документации, включая страницы справочного руководства *tap*. Документация может просто не успевать обновляться.

У вас может не быть времени для перепроверки каждой метрики производительности, и вы решите сделать это, только обнаружив необычные результаты или результаты, критичные для компании. Даже если вы не перепроверяете результаты, иногда полезно зафиксировать тот факт, что вы не выполняли проверки и полагались на исправную работу инструмента.

¹ В этом случае инструмент и измерения верны, а ошибки внес инструмент сбора метрик. На *Surge 2013* я рассказал об одном удивительном случае [Gregg 13c]: компания, занимающаяся бенчмаркингом, сообщила о плохих показателях для продукта, который я поддерживал. Разумеется, я приступил к исследованиям, и оказалось, что сценарий на языке командной оболочки, который они использовали для автоматизации бенчмарка, содержал две ошибки. Во-первых, при обработке вывода `fiio(1)` он получал результат «100KB/s» и использовал регулярное выражение для исключения нецифровых символов, в том числе и «KB/s», превращая значение в «100». Поскольку `fiio(1)` может сообщать результаты в разных единицах измерения (байты, килобайты, мегабайты), то это приводило к значительным (1024х) ошибкам. Во-вторых, они также отбрасывали десятичную точку, поэтому результат «1.6» превращался в «16».

Метрики также могут быть неполными. Если вы встречаете большое количество инструментов и метрик, возникает соблазн предположить, что они обеспечивают полное и эффективное покрытие. Часто это не так: метрики могут добавляться разработчиками для отладки собственного кода, а затем встраиваться в инструменты наблюдения без глубокого изучения реальных потребностей клиентов. Некоторые разработчики могли вообще не добавить их в новые подсистемы.

Отсутствие метрик определить труднее, чем наличие некачественных метрик. Найти эти недостающие метрики поможет глава 2 «Методологии» — там рассматриваются вопросы, на которые вы должны ответить, приступая к анализу производительности.

4.7. УПРАЖНЕНИЯ

Ответьте на следующие вопросы, касающиеся инструментов наблюдения (возможно, для этого потребуется вернуться к главе 1 «Введение», где описываются некоторые из упоминаемых терминов):

1. Назовите несколько статических инструментов наблюдения за производительностью.
2. Что такое профилирование?
3. Почему профилировщики используют частоту выборки 99 Гц вместо 100 Гц?
4. Что такое трассировка?
5. Что такое статическая инструментация?
6. Объясните важность динамической инструментации.
7. Чем отличаются точки трассировки и зонды `kprobes`?
8. Опишите ожидаемый оверхед процессорного времени (как низкий, средний или высокий) в следующих случаях:
 - счетчики дисковых операций ввода/вывода в секунду (как, например, в `iostat(1)`);
 - трассировка каждого события дискового ввода/вывода с помощью точек трассировки или `kprobes`;
 - трассировка каждого события переключения контекста (точки трассировки или `kprobes`);
 - трассировка каждого события запуска (`execve(2)`) нового процесса (точки трассировки или `kprobes`);
 - трассировка каждого вызова `malloc()` из библиотеки `libc` через `uprobes`.
9. Объясните, в чем ценность счетчиков производительности РМС для анализа производительности.
10. На примере любого инструмента наблюдения опишите, как можно определить, какие источники он использует.

4.8. ССЫЛКИ

- [Eigler 05] Eigler, F. Ch., et al. «Architecture of SystemTap: A Linux Trace/Probe Tool», <http://sourceware.org/systemtap/archpaper.pdf>, 2005.
- [Drongowski 07] Drongowski, P., «Instruction-Based Sampling: A New Performance Analysis Technique for AMD Family 10h Processors», AMD (Whitepaper), 2007.
- [Ayuso 12] Ayuso, P., «The Conntrack-Tools User Manual», <http://conntrack-tools.netfilter.org/manual.html>, 2012.
- [Gregg 13c] Gregg, B., «Benchmarking Gone Wrong», Surge 2013: Lightning Talks, <https://www.youtube.com/watch?v=vm1GJMp0QN4#t=17m48s>, 2013.
- [Weisbecker 13] Weisbecker, F., «Status of Linux dynticks», OSPERT, <http://www.ertl.jp/~shinpei/conf/ospert13/slides/FredericWeisbecker.pdf>, 2013.
- [Intel 16] «Intel 64 and IA-32 Architectures Software Developer's Manual Volume 3B: System Programming Guide, Part 2, September 2016», <https://www.intel.com/content/www/us/en/architecture-and-technology/64-ia-32-architectures-software-developer-vol-3b-part-2-manual.html>, 2016.
- [AMD 18] «Open-Source Register Reference for AMD Family 17h Processors Models 00h-2Fh», <https://developer.amd.com/resources/developer-guides-manuals>, 2018.
- [ARM 19] «Arm® Architecture Reference Manual Armv8, for Armv8-A architecture profile», https://developer.arm.com/architectures/cpu-architecture/a-profile/docs?_ga=2.78191124.1893781712.1575908489-930650904.1559325573, 2019.
- [Gregg 19] Gregg, B., «BPF Performance Tools: Linux System and Application Observability»¹, Addison-Wesley, 2019.
- [Atoptool 20] «Atop», www.atoptool.nl/index.php, по состоянию на 2020.
- [Bowden 20] Bowden, T., Bauer, B., et al., «The /proc Filesystem», Linux documentation, <https://www.kernel.org/doc/html/latest/filesystems/proc.html>, по состоянию на 2020.
- [Gregg 20a] Gregg, B., «Linux Performance», <http://www.brendangregg.com/linuxperf.html>, по состоянию на 2020.
- [LTTng 20] «LTTng», <https://lttng.org>, по состоянию на 2020.
- [PCP 20] «Performance Co-Pilot», <https://pcp.io>, по состоянию на 2020.
- [Prometheus 20] «Exporters and Integrations», <https://prometheus.io/docs/instrumenting/exporters>, по состоянию на 2020.
- [Sourceware 20] «SystemTap», <https://sourceware.org/systemtap>, по состоянию на 2020.
- [Ts'o 20] Ts'o, T., Zefan, L., and Zanussi, T., «Event Tracing», Linux documentation, <https://www.kernel.org/doc/html/latest/trace/events.html>, по состоянию на 2020.
- [Xenbits 20] «Xen Hypervisor Command Line Options», <https://xenbits.xen.org/docs/4.11-testing/misc/xen-command-line.html>, по состоянию на 2020.
- [UTK 20] «Performance Application Programming Interface», <http://icl.cs.utk.edu/papi>, по состоянию на 2020.

¹ Грегг Б. «BPF: Профессиональная оценка производительности». Выходит в издательстве «Питер» в 2023 году.

Глава 5

ПРИЛОЖЕНИЯ

Производительность лучше всего настраивать как можно ближе к месту выполнения работы: в приложениях. К ним относятся базы данных, веб-серверы, серверы приложений, балансировщики нагрузки, файловые серверы и т. д. В последующих главах мы рассмотрим приложения с точки зрения потребляемых ими ресурсов: процессорного времени, памяти, файловых систем, дисков и пропускной способности сети. В этой главе обсудим уровень приложений в целом.

Сами приложения могут быть чрезвычайно сложными, особенно в распределенных средах, включающих множество компонентов. Изучение внутреннего устройства приложения — прерогатива разработчиков приложений и может включать использование сторонних инструментов для интроспекции (самоанализа). Для тех, кто занимается оценкой производительности систем, анализ производительности приложений предполагает поиск конфигурации приложения, обеспечивающей оптимальное потребление системных ресурсов, описание особенностей использования системы приложением и анализ типичных недостатков.

Цели этой главы:

- описать цели настройки производительности;
- познакомить с приемами повышения производительности, включая многопоточное программирование, хеш-таблицы и неблокирующий ввод/вывод;
- представить общие примитивы блокировки и синхронизации;
- описать проблемы, создаваемые разными языками программирования;
- познакомить с методологией анализа состояния потока;
- показать приемы профилирования потребления процессора;
- продемонстрировать анализ системных вызовов, включая трассировку выполнения процесса;
- обозначить подводные камни трассировки стека: отсутствующие символы и стеки.

Мы рассмотрим основы приложений и факторы, влияющие на их производительность, языки программирования и компиляторы, общие стратегии анализа производительности приложений, а также системные инструменты наблюдения за приложениями.

5.1. ОСНОВЫ ПРИЛОЖЕНИЙ

Прежде чем углубляться в анализ производительности приложения, познакомимся с его ролью, основными характеристиками и экосистемой. Это сформирует контекст и упростит понимание активности приложения. Также это позволит узнать об общих проблемах и особенностях настройки производительности и определить направления для дальнейшего изучения. Для этого попробуйте ответить на вопросы:

- **Функциональность:** роль приложения. Это сервер базы данных, веб-сервер, балансировщик нагрузки, файловый сервер, хранилище объектов?
- **Эксплуатация:** какие запросы обслуживает приложение или какие операции выполняет? Базы данных обрабатывают *запросы к данным* (и команды), веб-серверы обслуживают *HTTP-запросы* и т. д. Эксплуатацию можно измерить как частоту следования обрабатываемых запросов, чтобы оценить нагрузку и использовать ее для планирования емкости.
- **Требования к производительности:** есть ли у компании, эксплуатирующей приложение, цели уровня обслуживания (Service Level Objective, SLO)? Например, 99,9 % запросов должны обслуживаться с задержкой < 100 мс.
- **Режим выполнения:** на каком уровне действует приложение, пользователя или ядра? Большинство приложений выполняются на уровне пользователя в виде одного или нескольких процессов, но некоторые приложения могут быть реализованы как сервисы ядра (например, NFS). Программы BPF тоже выполняются на уровне ядра.
- **Конфигурация:** как настроено приложение и почему? Эту информацию можно найти в файле конфигурации или получить с помощью инструментов администрирования. Проверьте, не были ли изменены какие-либо настраиваемые параметры, влияющие на производительность, включая размеры буферов, кэшей, степень параллелизма (количество используемых процессов или потоков) и др.
- **Хост:** на чем размещается приложение? Сервер или облачный экземпляр? Каковы процессоры, топология памяти, устройства хранения и т. д.? Каковы пределы?
- **Метрики:** предоставляет ли приложение какие-либо метрики производительности, например интенсивность эксплуатации? Они могут предоставляться встроенными инструментами или инструментами сторонних производителей, через запросы API или путем обработки журналов.
- **Журналы:** какие журналы создает приложение? Какие журналы можно включить? Какие метрики производительности, включая задержку, доступны в журналах? Например, MySQL поддерживает *журнал медленных запросов*, предоставляя ценную информацию о производительности для каждого запроса, который обрабатывался медленнее определенного порога.
- **Версия:** это последняя версия приложения? Были ли отмечены исправления или улучшения производительности в примечаниях к релизу для последних версий?
- **Ошибки:** поддерживается ли для приложения база данных ошибок? Какие ошибки, связанные с производительностью, указаны в этой базе данных для

вашей версии приложения? Если у вас есть проблемы с производительностью, поищите в базе данных ошибок, не случалось ли что-то подобное раньше, какие исследования проводились в связи с этим и какие причины были выявлены.

- **Исходный код:** это приложение с открытым исходным кодом? Если да, то можно исследовать пути кода с помощью профилировщиков и трассировщиков и определить меры, которые могут привести к увеличению производительности. Вы можете сами изменить код приложения для повышения производительности и предложить улучшения разработчикам для включения в официальную версию приложения.
- **Сообщество:** есть ли у приложения сообщество, члены которого делятся найденными методами повышения производительности? Сообщества могут иметь форумы, блоги, каналы Internet Relay Chat (IRC), другие каналы общения (например, Slack), проводить митапы и конференции. После митапов и конференций в интернете часто публикуют презентации и видео, которые долгие годы могут оставаться полезными источниками информации. В сообществе также может быть менеджер, который публикует новости и обновления.
- **Книги:** есть ли книги о приложении и/или его производительности? Насколько они хороши (например, написаны экспертами, содержат практичные/действенные рекомендации, не пускаются в пространные рассуждения, актуальны и т. д.)?
- **Эксперты:** есть ли признанные эксперты в области производительности приложения? Вы можете попробовать найти их публикации.

В любом случае постарайтесь понять особенности приложения на высоком уровне — что оно делает, как работает и насколько эффективно. *Функциональная диаграмма*, которая описывает внутреннее устройство приложения, если вам удастся ее отыскать, может оказаться чрезвычайно полезным ресурсом.

В следующих разделах мы рассмотрим другие основы анализа приложений — постановку цели, оптимизацию общего случая, наблюдаемость и нотацию «О-большое».

5.1.1. Цель

Цель анализа производительности определяет направление вашей работы и помогает выбрать нужные действия. Без четкой цели анализ производительности рискует превратиться в поиск «на удачу».

Анализ производительности приложения можно начать с выяснения того, какие операции выполняет приложение (как было описано выше), и определения цели анализа производительности. Целью может быть:

- **задержка:** низкое или постоянное время отклика приложения;
- **пропускная способность:** высокая скорость обработки запросов или передачи данных;
- **потребление ресурсов:** эффективность для данной рабочей нагрузки;

- **цена:** улучшение баланса цена/производительность, снижение затрат на вычисления.

Хорошо, если цели можно количественно оценить с помощью метрик, вытекающих из требований бизнеса или уровня качества обслуживания. Например:

- средняя продолжительность обработки запроса приложением 5 мс;
- 95 % запросов обрабатываются с задержкой 100 мс или меньше;
- устранение выбросов по задержке: ни один запрос не должен обрабатываться дольше 1000 мс;
- максимальная пропускная способность должна составлять не менее 10 000 запросов в секунду на сервер заданной мощности¹;
- средняя нагрузка на диск должна быть не выше 50 % при частоте обработки 10 000 запросов в секунду.

После выбора цели выявите ограничения, стоящие на пути к ней. Для задержки ограничением может быть дисковый или сетевой ввод/вывод. Для пропускной способности — нагрузка на процессор. Стратегии, описанные в этой и других главах, помогут их выявить. Если целью является пропускная способность, то имейте в виду, что не все операции равноценны с точки зрения производительности или стоимости. Если целью является определенная скорость выполнения операций, то важно будет указать тип этих операций. Величина пропускной способности может образовывать распределение в зависимости от ожидаемых или измеренных рабочих нагрузок.

В разделе 5.2 «Методы увеличения производительности приложений» описаны общие методы. Некоторые из них могут иметь смысл для одной цели, но не подходить для другой. Например, увеличение размеров блоков ввода/вывода может повысить пропускную способность за счет увеличения задержки. Помните о цели, которую вы преследуете, когда будете определять подходящие методы.

Apdex

Некоторые компании используют в качестве цели и метрики для мониторинга индекс производительности приложения (application performance index, ApDex, или Apdex). Он может лучше отражать клиентский опыт и основывается на оценках производительности клиентами, таких как «удовлетворительно», «терпимо» или «неудовлетворительно». Индекс Apdex вычисляется с использованием этих оценок, как показано ниже [Apdex 20]:

$$\text{Apdex} = (\text{удовлетворительно} + 0,5 \times \text{терпимо} + 0 \times \text{неудовлетворительно}) / \text{общее количество оценок}.$$

¹ Если размер сервера может меняться (что особенно характерно для облачных экземпляров). В таких случаях максимальную пропускную способность лучше выражать в терминах ограничивающего ресурса: например, не менее 1000 запросов в секунду на процессор для преимущественно вычислительной рабочей нагрузки.

Получившийся индекс Apdex находится в диапазоне от 0 (нет довольных клиентов) до 1 (все клиенты довольны).

5.1.2. Оптимизация общего случая

Программное обеспечение может иметь сложное устройство, со множеством различных путей в коде и в поведении. Это особенно заметно при просмотре исходного кода: приложения обычно состоят из десятков тысяч строк кода, а ядра ОС — до сотен тысяч. Выбор области оптимизации наугад может потребовать много работы и не принести большой выгоды.

Один из эффективных способов увеличить производительность приложения — найти самый частый путь в коде, связанный с рабочей нагрузкой, и начать с его оптимизации. Если приложение обрабатывает преимущественно вычислительные нагрузки, искомые пути часто оказываются, как говорят, «на процессоре». Если приложение в основном выполняет операции ввода/вывода, то ищите пути в коде, ведущие к вводу/выводу. Все эти пути можно определить путем анализа и профилирования приложения, а также изучения трассировок стеков и флейм-графиков, как описано в следующих главах. Дополнительную информацию более высокого уровня также можно получить с помощью инструментов наблюдения за приложением.

5.1.3. Наблюдаемость

Во многих главах этой книги я часто повторяю, что наибольший выигрыш в производительности можно получить за счет исключения ненужной работы.

Этот факт иногда упускается из виду, когда выбор приложения делается на основе производительности. Если бенчмаркинг показал, что приложение А на 10 % быстрее приложения В, то может возникнуть соблазн выбрать приложение А. Но если приложение А непрозрачно, а приложение В предоставляет богатый набор инструментов наблюдения, весьма вероятно, что приложение В окажется лучшим выбором в долгосрочной перспективе. Инструменты наблюдения помогут избавиться от ненужной работы, а также лучше понять, как действует приложение, и настроить его. Выигрыш в производительности, достигнутый за счет лучшей наблюдаемости, может затмить начальную разницу в производительности 10 %. То же самое верно и при выборе языков и сред выполнения: например, сравните выбор Java или С, которые являются зрелыми и имеют множество инструментов наблюдения, с выбором нового языка.

5.1.4. Нотация «О-большое»

Нотация «О-большое», о которой часто говорят на курсе информатики, используется для анализа сложности алгоритмов и моделирования их производительности при изменении масштаба входного набора данных. Под буквой О подразумевается *порядок* (order) функции, описывающий скорость ее роста. Нотация помогает

программистам выбирать более эффективные и производительные алгоритмы при разработке приложений [Knuth 76], [Knuth 97].

В табл. 5.1 перечислены некоторые распространенные нотации «О-большого» и примеры соответствующих алгоритмов.

Таблица 5.1. Примеры нотаций «О-большого»

Нотация	Примеры
$O(1)$	Проверка логического условия
$O(\log n)$	Поиск в отсортированном массиве методом дихотомии
$O(n)$	Линейный поиск в связанном списке
$O(n \log n)$	Алгоритм быстрой сортировки (средний случай)
$O(n^2)$	Алгоритм пузырьковой сортировки (средний случай)
$O(2^n)$	Разложение чисел, экспоненциальный рост
$O(n!)$	Решение задачи коммивояжера методом прямого перебора вариантов

Нотация позволяет программистам сравнивать производительность различных алгоритмов и определять, оптимизация каких участков кода даст наибольший выигрыш. Например, алгоритм поиска в отсортированном массиве из 100 элементов методом дихотомии, по сравнению с алгоритмом линейного поиска, в 21 раз производительнее ($100/\log(100)$).

Производительности этих алгоритмов и динамика их изменения по мере масштабирования показаны на рис. 5.1.

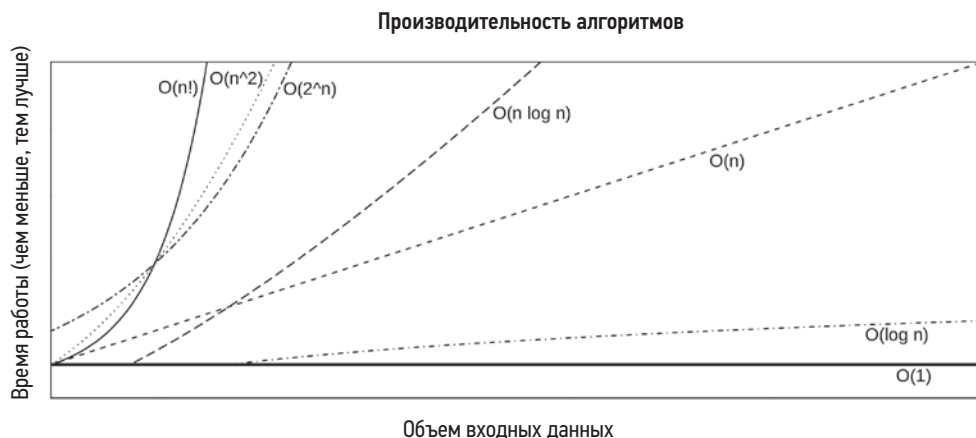


Рис. 5.1. Зависимость времени выполнения от объема входных данных для разных алгоритмов

Эта классификация помогает специалистам по анализу производительности систем заметить, что некоторые алгоритмы плохо масштабируются. Проблемы

с производительностью могут появиться, когда приложения вынуждены обслуживать больше пользователей или объектов данных, чем прежде, и в этот момент алгоритмы, например $O(n^2)$, могут вызвать патологическое падение производительности. Для исправления такой проблемы разработчик может использовать более эффективный алгоритм или как-то иначе разделить совокупность данных.

Нотация «О-большое» не учитывает некоторые постоянные вычислительные затраты, свойственные каждому алгоритму. В случаях, когда значение n (объем входных данных) невелико, такие затраты могут преобладать.

5.2. МЕТОДЫ ПОВЫШЕНИЯ ПРОИЗВОДИТЕЛЬНОСТИ ПРИЛОЖЕНИЙ

В этом разделе описаны некоторые распространенные методы повышения производительности приложений: выбор размеров блоков ввода/вывода, кэширование, буферизация, опрос, конкурентность и параллелизм, неблокирующий ввод/вывод и привязка к процессору. В документации приложения вы можете узнать, какие из этих методов уже используются, а также о любых дополнительных особенностях приложения.

5.2.1. Выбор размеров блоков ввода/вывода

К затратам, связанным с вводом/выводом, можно отнести инициализацию буферов, обращение к системному вызову, переключение режима или контекста, размещение метаданных в ядре, соблюдение прав и ограничений процесса, отображение адресного пространства в устройство, выполнение кода ядра и драйвера для передачи данных и, наконец, освобождение метаданных и буферов. «Налог на инициализацию» уплачивается для любых операций ввода/вывода, независимо от объема данных, вовлеченных в операцию. Поэтому с точки зрения эффективности чем больше данных передается с каждой операцией ввода/вывода, тем лучше.

Увеличение размеров блоков ввода/вывода — распространенная стратегия, используемая приложениями для увеличения пропускной способности. Обычно, с учетом любых фиксированных затрат на ввод/вывод, гораздо эффективнее выполнить ввод/вывод одного блока размером 128 Кбайт, чем 128 блоков по 1 Кбайт. В частности, операции ввода/вывода с вращающимися дисками традиционно имеют высокую стоимость из-за затрат времени на позиционирование головок.

Но у этой стратегии есть и обратная сторона, когда приложению не требуется выполнять ввод/вывод больших объемов данных. Например, база данных, выполняющая произвольное чтение 8 Кбайт данных, может работать медленнее при размере блока дискового ввода/вывода 128 Кбайт, потому что впустую тратится время на передачу ненужных 120 Кбайт данных. Это приводит к задержке ввода/вывода, которую можно уменьшить, выбрав меньший размер блока ввода/вывода, точнее

соответствующий нуждам приложения. Неоправданно большие размеры блоков ввода/вывода также могут напрасно расходовать пространство в кэш-памяти.

5.2.2. Кэширование

Для повышения производительности операций чтения из файловой системы и экономии времени на выделение памяти операционная система использует кэши. Приложения тоже часто используют кэши по той же причине. Чтобы каждый раз не выполнять дорогостоящую операцию ввода/вывода, приложение может сохранять результаты частых операций в локальном кэше и использовать их в будущем. Примером может служить буферный кэш базы данных, в котором хранятся данные часто выполняемых запросов к базе.

Типичная задача при развертывании приложений — определение кэшей, которые можно включить, и настройка их размеров в соответствии с особенностями системы.

Кэши увеличивают производительность операций чтения, но их пространство также часто используется для хранения буферов, чтобы увеличить производительность операций записи.

5.2.3. Буферизация

Для увеличения производительности операций записи данные можно объединить в буфер перед отправкой на следующий уровень. Это увеличивает размер блока ввода/вывода и эффективность операции. В зависимости от типа операции записи этот прием может также увеличить задержку, потому что первая записанная в буфер порция данных будет ждать заполнения буфера последующими операциями записи.

Кольцевой (или *циклический*) буфер — это разновидность буфера фиксированного размера, который можно использовать для непрерывной передачи данных между компонентами, действующими асинхронно. Реализуется такой буфер с использованием указателей начала и конца, которые могут перемещаться каждым компонентом при добавлении или удалении данных.

5.2.4. Опрос

Опрос (polling) — это метод организации ожидания некоторого события путем проверки состояния в цикле с паузами между проверками. Когда события возникают редко, использование метода опроса может приводить к проблемам с производительностью из-за:

- значительного оверхеда процессорного времени на повторные проверки;
- высокой задержки между появлением события и следующей проверкой в цикле опроса.

Если это действительно проблема, разработчики приложения могут изменить его поведение и реализовать немедленное уведомление приложения о событии и вызов требуемой процедуры.

Системный вызов *poll()*

Для проверки файловых дескрипторов можно использовать системный вызов *poll(2)*, который выполняет ту же функцию, что и цикл опроса, но его работа основана на событиях и поэтому не влечет дополнительных затрат на опрос.

Интерфейс *poll(2)* позволяет контролировать сразу несколько файловых дескрипторов, передаваемых в виде массива, из-за чего приложение должно просканировать массив, чтобы определить файловые дескрипторы, вызвавшие событие. Это сканирование имеет сложность $O(n)$ (см. раздел 5.1.4 «Нотация “О-большое”»), соответственно, переход на сканирование может вызвать проблемы производительности при масштабировании. Альтернатива в Linux — это системный вызов *epoll(2)*, позволяющий избежать сканирования и имеющий сложность $O(1)$. В BSD эквивалентом является *kqueue(2)*.

5.2.5. Конкурентность и параллелизм

Системы с разделением времени (включая все производные Unix) обеспечивают возможность *конкурентности* (concurrency): возможности загрузить и начать выполнять сразу несколько программ. Хотя их время выполнения может совпадать, они не обязательно запускаются на процессоре в один и тот же момент. Каждая из программ может быть прикладным процессом.

Чтобы воспользоваться преимуществами многопроцессорной системы, приложение должно выполняться на нескольких процессорах одновременно. Такой *параллелизм* приложение может реализовать, запуская несколько процессов (*мультипроцесс*) или несколько потоков (*многопоточность*), каждый из которых решает свою задачу. По причинам, описанным в главе 6 «Процессоры», в разделе 6.3.13 «Несколько процессов, несколько потоков», подход с использованием нескольких потоков (или задач) более эффективен и поэтому предпочтительнее.

Помимо увеличения пропускной способности процессора, выполнение в нескольких потоках (или процессах) — это один из способов конкурентного выполнения операций ввода/вывода, потому что другие потоки имеют возможность продолжать выполнение, пока заблокированный поток ждет завершения операции ввода/вывода. (Другой способ — асинхронный ввод/вывод.)

Поддержка архитектур с несколькими процессами или потоками означает, что ядро с помощью своего планировщика решает, когда и какому процессу или потоку дать возможность поработать на том или ином процессоре и учитывать переход на переключение контекста. Другой подход: реализация в приложении пользовательского режима (user-mode application) своего механизма планирования и модели программирования, чтобы он мог обслуживать различные запросы в одном и том же потоке операционной системы. К таким механизмам можно отнести:

- **Волокна (fibers):** также называются *легковесными потоками* — это разновидность потоков выполнения в пользовательском режиме, где каждое волокно представляет планируемую программу. Приложение может использовать собственную

логику планирования, выбирая волокно для запуска. Волокна, например, можно создавать для обработки каждого запроса, и эта операция имеет меньший оверхед, чем создание потоков выполнения ОС. Волокна поддерживаются, например, в Microsoft Windows¹.

- **Корутины (co-routines):** еще более легковесные, чем волокна. Корутины — это подпрограммы, выполнение которых может планироваться приложением пользовательского режима с помощью своего механизма управления конкурентностью.
- **Конкурентность на основе событий:** программы могут быть разбиты на множество обработчиков событий, а создание событий можно планировать и извлекать из очереди. Это делается, например, путем размещения метаданных для каждого запроса, к которому будут обращаться обработчики событий. В частности, среда выполнения Node.js поддерживает конкурентность на основе событий с использованием единственного *потока обработки событий* (который может стать узким местом, потому что способен выполняться только на одном процессоре).

При всем богатстве доступных механизмов операции ввода/вывода все еще должны обрабатываться ядром, поэтому переключение потоков ОС практически неизбежно². Кроме того, для параллельного выполнения все равно придется использовать несколько потоков ОС, чтобы их выполнение можно было планировать на нескольких процессорах.

Некоторые среды выполнения используют и корутины, как легковесный механизм поддержки конкурентного выполнения, и потоки ОС для обеспечения возможности параллельного выполнения на нескольких процессорах. Примером может служить среда выполнения Golang, которая использует *горутины* (goroutines) в комплексе с пулом потоков операционной системы. Для увеличения производительности, когда горутинa выполняет блокирующий вызов, планировщик Golang автоматически перемещает другие горутины из заблокированного потока в другие потоки, чтобы они выполнялись дальше [Golang 20].

Вот три распространенные модели многопоточного программирования:

- **Пул служебных потоков:** пул потоков обслуживает сетевые запросы, причем каждый поток обслуживает одно соединение с клиентом.

¹ Официальная документация Microsoft предупреждает о проблемах, которые могут создавать волокна: например, локальная память потока совместно используется всеми волокнами, поэтому нужно использовать локальную память волокон, и любая процедура, завершившая работу потока, автоматически завершит все волокна в этом потоке. В документации говорится: «В общем случае волокна не дают преимуществ перед хорошо спроектированным многопоточным приложением» [Microsoft 18].

² За некоторыми исключениями, например использованием `sendfile(2)` вместо обращений к системным вызовам ввода/вывода и интерфейса `io_uring` в Linux, который позволяет пользовательскому пространству планировать ввод/вывод путем записи и чтения из очередей `io_uring` (этот механизм кратко описан в разделе 5.2.6 «Неблокирующий ввод/вывод»).

- **Пул процессорных потоков:** для каждого процессора создается один поток. Этот прием обычно используется для организации длительных пакетных вычислений, например для кодирования видео.
- **Поэтапная архитектура, управляемая событиями (staged event-driven architecture, SEDA):** запросы в этой архитектуре разбиваются на этапы (уровни), которые могут обрабатываться пулами из одного или нескольких потоков.

Поскольку все потоки выполнения используют одно и то же адресное пространство процесса, они могут передавать данные через память напрямую, не прибегая к тяжеловесным интерфейсам (например, к механизмам межпроцессных взаимодействий (inter-process communication, IPC), которые вынуждены использовать процессы). Целостность данных в этом случае обеспечивается примитивами синхронизации, которые предотвращают повреждение данных при одновременных попытках чтения и записи из нескольких потоков.

Примитивы синхронизации

Примитивы синхронизации обеспечивают целостность данных, управляя доступом к памяти, и могут работать как светофор, регулирующий движение через перекресток. Как и светофор, они могут приостанавливать поток трафика, вызывая задержку. Вот наиболее часто используемые типы примитивов синхронизации для приложений:

- **Мьютексы (взаимоисключающие блокировки):** продолжать работу может только держатель блокировки. Другие потоки блокируются и ожидают освобождения блокировки вне процессора.
- **Спин-блокировки:** позволяют работать держателю блокировки, а в это время другие потоки выполняют цикл ожидания на процессоре (*вращаются* в цикле, откуда и пошло название «спин» — вращение), проверяя возможность захватить блокировку. Такие блокировки способны обеспечить доступ с малой задержкой — заблокированный поток не покидает процессор и сможет продолжить основную работу уже через несколько тактов процессора после освобождения блокировки, но они также увеличивают потребление процессора, пока потоки вращаются в цикле ожидания.
- **Блокировки чтения/записи:** блокировки чтения/записи гарантируют целостность, разрешая одновременную работу нескольким читающим потокам или единственному пишущему потоку.
- **Семафоры:** это разновидность переменной, которая ведет счет и может разрешить одновременное выполнение определенного количества операций. Частный случай — двоичный семафор, разрешающий выполнение только одной операции (фактически это мьютекс).

Мьютексы могут быть реализованы в библиотеке или в ядре как гибриды мьютексов и спин-блокировок, когда потоки продолжают вращаться в цикле ожидания, если держатель выполняется на другом процессоре, и блокируются в противном случае.

(или если достигнут порог итераций в цикле ожидания). В Linux они впервые были реализованы в 2009 году [Zijlstra 09] и теперь имеют три пути в зависимости от состояния блокировки (как описано в `Documentation/locking/mutex-design.rst` [Molnar 20]):

1. **Быстрый путь:** попытка получить блокировку с помощью инструкции `cmpxchg` для установки владельца. Успех достигается только в том случае, если блокировка свободна.
2. **Промежуточный путь:** также известный как *оптимистическое вращение* (*optimistic spinning*). Заблокированный поток вращается в цикле ожидания, пока держатель блокировки продолжает работать, в надежде, что она скоро освободится и ее можно будет получить без длительной приостановки.
3. **Медленный путь:** когда поток блокируется, перемещается планировщиком в очередь ожидания и остается там до тех пор, пока блокировка не освободится.

Механизм чтения-копирования-обновления (*read-copy-update*, RCU) в Linux — еще один механизм синхронизации, активно используемый в коде ядра. Он позволяет выполнять операции чтения без приобретения блокировки, улучшая производительность по сравнению с блокировками других типов. С помощью RCU операции записи создают копию защищенных данных и изменяют эту копию, в то время как операции чтения по-прежнему имеют доступ к оригиналу. Механизм RCU автоматически определяет, когда прекращается чтение защищенных данных (на основе различных условий для каждого процессора), и заменяет оригинал обновленной копией [Linux 20e].

Исследование проблем производительности, связанных с блокировками, может занять много времени и часто требует знания исходного кода приложения. Обычно такое исследование выполняют разработчики.

Хеш-таблицы

Для оптимизации количества блокировок при работе с большим количеством структур данных можно использовать хеш-таблицу блокировок. Я лишь кратко опишу эту идею, потому что это довольно сложная тема, предполагающая опыт программирования.

Представьте следующие два подхода:

- Единый глобальный мьютекс для всех структур данных. Хотя это решение простое, при конкурентном доступе возникнет борьба за блокировку и задержка при ее ожидании. Несколько потоков, которым нужна блокировка, будут выполнять *сериализацию* последовательно, а не конкурентно.
- Мьютекс для каждой структуры данных. Это помогает ослабить конкуренцию и уменьшить время ожидания до минимально необходимого уровня, когда несколько потоков конкурентно обращаются к одной и той же структуре данных. Но тогда возникает оверхед на память для хранения блокировки

и на процессор для создания и уничтожения блокировки каждой структуры данных.

Хеш-таблица блокировок — это компромиссное решение и подходит для случаев, когда ожидается, что конкуренция за блокировки будет незначительной. Суть заключается в создании фиксированного количества блокировок и использовании алгоритма хеширования для выбора блокировки той или иной структуры данных. Это позволяет избежать затрат на создание и уничтожение блокировок, а также проблем, свойственных единой глобальной блокировке.

На рис. 5.2 показан пример хеш-таблицы, включающей четыре записи, которые называются *корзинами* (buckets), каждая из которых содержит свою блокировку.

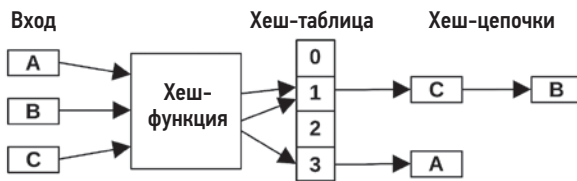


Рис. 5.2. Пример хеш-таблицы

В этом примере также показан один из подходов к разрешению *конфликтов хеширования*, когда две или более структуры данных хешируются в один и тот же сегмент. В этом случае создается цепочка структур данных для хранения в одном сегменте, где они будут обнаруживаться функцией хеширования. Эти хеш-цепочки могут создавать проблемы с производительностью, если становятся слишком длинными и обрабатываются последовательно, потому что защищены только одной блокировкой, которая может удерживаться длительное время. Хеш-функцию и размер таблицы можно выбирать, чтобы обеспечить равномерное распределение структур данных по множеству сегментов и свести длину хеш-цепочек к минимуму. Длину хеш-цепочек следует проверять для производственных нагрузок на случай, если алгоритм работает не так, как задумано, и создает длинные цепочки, ухудшая производительность.

В идеале количество сегментов в хеш-таблице должно быть не меньше количества процессоров, чтобы максимально раскрыть потенциал параллелизма. Алгоритм хеширования может быть таким же простым, как взятие младших битов¹ адреса структуры данных и использование их в качестве индекса в массиве блокировок с размером, равным степени двойки. Такие простые алгоритмы также очень производительны и позволяют быстро находить структуры данных.

При наличии массива смежных блокировок в памяти может возникнуть проблема производительности, когда блокировки попадают в одну и ту же строку

¹ Или средние биты. Младшие биты адресов массивов структур могут порождать слишком много конфликтов.

кэша. Два процессора, обновляющие разные блокировки в одной и той же строке кэша, столкнутся с оверхедом на согласование кэша, при этом каждый процессор аннулирует строку кэша в кэше другого процессора. Эта ситуация называется *ложным совместным использованием* и обычно решается дополнением блокировок неиспользуемыми байтами, чтобы в каждой строке кэша находилась только одна блокировка.

5.2.6. Неблокирующий ввод/вывод

Как показывает описание жизненного цикла процессов Unix в главе 3 «Операционные системы», на время ввода/вывода процессы блокируются и переходят в ожидание. У этой модели есть несколько проблем с производительностью:

- Каждая блокирующая операция ввода/вывода вынуждает поток (или даже процесс) простаивать. Чтобы одновременно выполнять множество операций ввода/вывода, приложение должно создать множество потоков (обычно по одному для каждого клиента) и нести оверхед, связанный с созданием и уничтожением потоков, а также выделять память для их стеков.
- Когда короткие операции ввода/вывода следуют часто, может возникать большой оверхед на переключение контекста, потребляющий ресурсы процессора и увеличивающий задержки в приложении.

Модель неблокирующего ввода/вывода предлагает операции ввода/вывода, выполняемые асинхронно, без блокировки текущего потока, который вместо ожидания может заняться другой полезной работой. Эта модель была ключевой особенностью Node.js [Node.js 20], серверной среды выполнения для приложений на JavaScript, которая требует использовать в коде неблокирующие приемы.

Есть множество механизмов неблокирующего или асинхронного ввода/вывода, в том числе:

- **open(2)**: при передаче флага `O_ASYNC`. Когда файловый дескриптор становится доступным для ввода/вывода, процесс автоматически уведомляется об этом.
- **io_submit(2)**: асинхронный ввод/вывод в Linux (Asynchronous I/O, AIO).
- **sendfile(2)**: копирует данные из одного файлового дескриптора в другой, перекладывая выполнение операции ввода/вывода на ядро¹.
- **io_uring_enter(2)**: интерфейс `io_uring` в Linux позволяет выполнять асинхронный ввод/вывод с использованием кольцевого буфера, который совместно используется пространствами пользователя и ядра [Axboe 19].

О других механизмах вы сможете узнать в документации к своей ОС.

¹ Этот системный вызов широко используется в сети Netflix CDN для отправки видеоресурсов клиентам без дополнительных затрат на ввод/вывод на уровне пользователя.

5.2.7. Привязка к процессору

Для сред NUMA часто выгодно, чтобы процесс или поток продолжил выполняться на том же процессоре, на котором запустил операцию ввода/вывода. Это может улучшить локальность памяти приложения, сократить циклы ввода/вывода в памяти и повысить общую производительность приложения. Разработчики ОС хорошо знают об этом и предусматривают поддержку привязки потоков приложений к одним и тем же процессорам (*привязка к процессору*, или CPU affinity). Подробнее об этом в главе 7 «Память».

Некоторые приложения принудительно вызывают такое поведение, *привязывая* себя к процессорам. В некоторых системах это может значительно улучшить производительность, но может и ухудшить ее, если одни привязки конфликтуют с другими, например, отображение прерываний устройств на процессоры.

Будьте осторожны с привязками к процессорам, если в той же системе работают другие такие же арендаторы (tenants) или приложения. Я столкнулся с этой проблемой в средах виртуализации операционной системы (в контейнерах), где приложение видит все процессоры, а затем привязывается к некоторым из них, если оно оказывается единственным приложением на сервере. Когда сервер совместно используется другими такими же приложениями арендаторов, которые тоже привязывают себя к процессорам, несколько арендаторов могут непреднамеренно привязаться к одним и тем же процессорам и вызвать конкуренцию за обладание процессором и задержки при планировании, при том что другие процессоры могут простаивать.

За время жизни приложения хост-система тоже может измениться, и необновляемые привязки могут не помогать, а ухудшать производительность, например, когда без необходимости привязываются к процессорам в нескольких сокетах.

5.2.8. Мантры производительности

Другие приемы увеличения производительности приложений см. в главе 2 в разделе «Мантры производительности». Перечислю их еще раз:

1. Не делай этого.
2. Сделай это только один раз.
3. Делай меньше.
4. Делай позже.
5. Делай это, когда нет других дел.
6. Делай это параллельно.
7. Делай это оптимально.

Первый пункт «Не делай этого» подразумевает отказ от выполнения ненужной работы. Более подробно об этом я рассказываю в главе 2 в разделе 2.5.20 «Мантры производительности».

5.3. ЯЗЫКИ ПРОГРАММИРОВАНИЯ

Языки программирования могут быть компилируемыми или интерпретируемыми, а также предполагать выполнение на виртуальной машине. Многие языки приписывают себе «оптимизацию производительности» как особенность, но строго говоря, это типичная функция программного обеспечения, *выполняющего* код на языке, а не самого языка. Например, ПО виртуальной машины Java HotSpot включает JIT-компилятор для динамического увеличения производительности.

Интерпретаторы и виртуальные машины также обеспечивают различные уровни поддержки наблюдения за производительностью, предлагая свои специализированные инструменты. Анализ производительности системы с использованием этих инструментов иногда может дать быстрые решения. Например, причина высокой нагрузки на процессор может заключаться в особенностях сборки мусора (garbage collection, GC) и может быть исправлена изменением некоторых параметров. Или причина может заключаться в известной ошибке в коде, зарегистрированной в базе данных ошибок, и может быть исправлена путем обновления версии программного обеспечения (такое случается довольно часто).

В следующих разделах описаны основные характеристики производительности для языков программирования каждого типа. Чтобы узнать больше о производительности конкретного языка, поищите статьи, описывающие этот язык.

5.3.1. Компилируемые языки

Компиляция преобразует исходный код программы в машинные инструкции до ее выполнения и сохраняет их в двоичных выполняемых файлах, которые иногда называют *бинарными*, обычно в формате исполняемых и компокуемых модулей (Executable and Linking Format, ELF) в Linux и других производных Unix либо в формате переносимых исполняемых файлов (Portable Executable, PE) в Windows. Их можно запустить в любой момент без повторной компиляции. К компилируемым языкам относятся C, C++ и ассемблер. У некоторых языков могут быть и интерпретаторы, и компиляторы.

Скомпилированный код обычно имеет высокую производительность, потому что не требует дополнительной трансляции перед выполнением на процессоре. Типичным примером скомпилированного кода является ядро Linux, которое написано в основном на C, а некоторые критические пути написаны на ассемблере.

Анализ производительности программ на компилируемых языках обычно не вызывает сложностей, потому что исполняемый машинный код близко соответствует исходной программе (хотя это зависит от оптимизаций, применяемых при компиляции). Во время компиляции может создаваться таблица символов, отображающая адреса в имена функций и объектов. При последующем профилировании и трассировке полученные результаты можно напрямую связать с этими именами и изучить особенности выполнения программы. Трассировки стека и содержащиеся в них

числовые адреса также можно отобразить в имена функций и проследить цепочку вызовов функций в коде.

Компиляторы могут повышать производительность за счет *оптимизаций компилятора* — процедур, оптимизирующих выбор и размещение машинных инструкций.

Оптимизации компилятора

Компилятор gcc(1) предлагает семь уровней оптимизации: 0, 1, 2, 3, s, fast и g. Числовые уровни определяют диапазоны оптимизаций, где уровню 0 соответствует наименьшее количество оптимизаций, а уровню 3 — наибольшее. Уровень «s» включает оптимизации по размеру, уровень «g» предназначен для отладки, а уровень «fast», кроме всех прочих, включает ряд дополнительных оптимизаций, которые не соответствуют стандартам. Вот как можно обратиться к gcc(1) и узнать, какие оптимизации используются на разных уровнях:

```
$ gcc -Q -O3 --help=optimizers
```

```
The following options control optimizations:
```

```
-O<number>
-Ofast
-Og
-Os
-faggressive-loop-optimizations      [enabled]
-falign-functions                    [disabled]
-falign-jumps                        [disabled]
-falign-label                        [enabled]
-falign-loops                        [disabled]
-fassociative-math                   [disabled]
-fasynchronous-unwind-tables        [enabled]
-fauto-inc-dec                       [enabled]
-fbranch-count-reg                   [enabled]
-fbranch-probabilities               [disabled]
-fbranch-target-load-optimize        [disabled]
[...]
-fomit-frame-pointer                 [enabled]
[...]
```

Полный список оптимизаций для gcc версии 7.4.0 включает около 230 параметров, часть из которых имеется даже на уровне `-O0`. Давайте посмотрим, как описывается одна из этих оптимизаций, `-fomit-frame-pointer`, на странице справочного руководства для gcc(1):

Запрещает сохранение указателя на фрейм стека в регистре для функций, которым он не нужен. Это позволяет исключить из скомпилированного кода инструкции, сохраняющие, настраивающие и восстанавливающие указатели фреймов, а также освободить дополнительный регистр, который может пригодиться во многих функциях. **Этот параметр также делает невозможной отладку на некоторых машинах.**

Это пример компромисса: отсутствие указателя фрейма обычно нарушает работу инструментов, выполняющих профилирование с использованием трассировок стека.

С учетом полезности профилирования, используя этот параметр, вы жертвуете возможными выгодами в производительности в будущем, которые будет трудно обнаружить без указателей фреймов и которые могут намного перевешивать выигрыш, предлагаемый данным параметром. Решением проблемы может стать компиляция с параметром `-fno-omit-frame-pointer`, чтобы избежать этой оптимизации¹. Еще один рекомендуемый параметр — это `-g`, предусматривающий включение отладочной информации в скомпилированный модуль, которая может упростить последующую отладку. Если потребуется, отладочную информацию можно удалить позже².

При проблемах с производительностью может возникнуть соблазн просто повторно скомпилировать приложение с пониженным уровнем оптимизации (например, с `-O2` вместо `-O3`) в надежде удовлетворить потребности в отладке. Но все не так просто: изменения в выводе компилятора могут оказаться значительными и важными и повлиять на поведение программы, которую вы решили проанализировать.

5.3.2. Интерпретируемые языки

Программный код на интерпретируемых языках транслируется в машинные инструкции прямо во время выполнения, что увеличивает оверхед на выполнение. Обычно никто не ждет высокой производительности от программ на интерпретируемых языках. Их используют в ситуациях, когда другие факторы, например простота программирования и отладки оказываются более важными. Примером интерпретируемого языка может служить *язык сценариев командной оболочки* (shell scripting).

Отсутствие специализированных инструментов наблюдения может затруднять анализ производительности программ на интерпретируемых языках. Профилирование процессора здесь может отражать работу интерпретатора, включая синтаксический анализ, трансляцию и выполнение инструкций, но не раскрывать имена функций в исходной программе, оставляя основной программный контекст загадкой. Анализ самого интерпретатора нельзя назвать абсолютно бесплодным занятием, потому что проблемы с производительностью могут возникнуть в самом интерпретаторе, даже если интерпретируемый код, который он выполняет, спроектирован безупречно.

В некоторых интерпретаторах контекст программы может быть доступен в виде аргументов функций интерпретатора, которые можно получить с помощью динамических инструментов. Другой подход — исследовать память процесса с учетом структуры программы (например, с помощью системного вызова `process_vm_readv(2)`).

¹ Некоторые профилировщики поддерживают дополнительные возможности для обхода стека, такие как использование отладочной информации, LBR, BTS и т. д. В главе 13 «perf» в разделе 13.9 «perf record» описаны способы использования разных инструментов обхода стека в профилировщике perf(1).

² Если вы распространяете двоичные файлы без отладочной информации, подумайте о создании пакетов с отладочной информацией, чтобы ее можно было добавить при необходимости.

Часто эти программы изучают, просто добавляя операторы печати и отметки времени. Более тщательный анализ производительности встречается редко, поскольку интерпретируемые языки обычно не выбирают для высокопроизводительных приложений.

5.3.3. Виртуальные машины

Виртуальная машина языка (или *виртуальная машина процесса*) — это ПО, имитирующее компьютер. Программы на некоторых языках, включая Java и Erlang, выполняются с использованием виртуальных машин (virtual machine, VM), которые предоставляют среду выполнения, не зависящую от платформы. Прикладная программа компилируется в набор команд виртуальной машины (*байт-код*), а затем выполняется виртуальной машиной. Это обеспечивает переносимость скомпилированных модулей при условии, что на целевой платформе доступна виртуальная машина для их выполнения.

Байт-код исполняется виртуальной машиной языка по-разному. Виртуальная машина Java HotSpot поддерживает выполнение через интерпретацию, а также JIT-компиляцию, когда байт-код компилируется в машинные инструкции для выполнения непосредственно на процессоре. Это дает преимущества высокой производительности скомпилированного кода и переносимости виртуальной машины.

Программы, выполняемые виртуальными машинами, как правило, являются наиболее сложными для наблюдения. К тому времени, когда программа окажется на процессоре, может быть пройдено несколько этапов компиляции или интерпретации, и информация об исходной программе становится недоступна. Основное внимание при анализе производительности таких программ обычно уделяется наборам инструментов, поставляемым с языковой виртуальной машиной, многие из которых поддерживают зонды USDT, а также сторонним инструментам.

5.3.4. Сборка мусора

Некоторые языки поддерживают автоматическое управление памятью, позволяя не освобождать явно выделенную память и оставлять эту задачу процессу асинхронной сборки мусора. Это упрощает написание программ, но имеет свои недостатки:

- **Утечки памяти:** ослабление контроля над использованием памяти приводит к ее утечкам, если объекты, размещенные в памяти, не идентифицируются автоматически как подлежащие освобождению. Когда объем памяти, занятой приложением, достигает определенной величины, может произойти ошибка исчерпания доступной памяти или включится системный механизм подкачки, что серьезно ухудшит производительность.
- **Потребление процессора:** сборщик мусора обычно запускается через определенные интервалы времени и производит в памяти поиск объектов, которые можно освободить. При этом на короткие периоды времени потребляются ресурсы процессора, сокращая процессорное время, доступное приложению. С ростом потребления памяти приложением потребление процессора сборщиком мусора

тоже растет. Иногда сборщик мусора потребляет все процессорное время, выделенное приложению.

- **Выбросы по задержке:** выполнение приложения может быть приостановлено на время работы сборщика мусора, что может вызывать случайные высокие задержки в обработке запросов¹. Это зависит от того, какой тип у сборщика мусора: блокирующий, инкрементальный или конкурентный.

Сборщик мусора — обычная цель для настройки производительности с основной задачей снизить потребление процессора и устранить выбросы по задержке. Например, JVM предлагает множество параметров для выбора типа сборщика мусора, количества используемых потоков, максимального размера кучи, целевой доли свободной памяти в куче и т. д.

Если настройка не дает желаемого эффекта, то проблема может заключаться в том, что приложение создает слишком много мусора или происходит утечка ссылок. Эти проблемы решает разработчик приложения. Один из подходов — стараться размещать в куче как можно меньше объектов, чтобы уменьшить нагрузку на сборщик мусора. Инструменты наблюдения, которые показывают распределение объектов и их пути в коде, можно использовать для поиска потенциальных целей для устранения.

5.4. МЕТОДОЛОГИЯ

В этом разделе описаны методологии анализа и настройки приложений. Инструменты для анализа будут представлены здесь или в других главах. Краткий список методологий приводится в табл. 5.2.

Таблица 5.2. Методологии анализа и настройки производительности приложений

Раздел	Методология	Тип
5.4.1	Профилирование процессора	Анализ наблюдений
5.4.2	Профилирование периодов вне процессора	Анализ наблюдений
5.4.3	Анализ системных вызовов	Анализ наблюдений
5.4.4	Метод USE	Анализ наблюдений
5.4.5	Анализ состояния потоков выполнения	Анализ наблюдений
5.4.6	Анализ блокировок	Анализ наблюдений
5.4.7	Статическая настройка производительности	Анализ наблюдений, настройка
5.4.8	Распределенная трассировка	Анализ наблюдений

¹ Методам сокращения времени сборки мусора и приостановки приложений уделяется самое пристальное внимание. В одном из примеров, представленном в [Schwartz 18], показано, как использовать метрики системы и приложения, чтобы определить наилучший момент для вызова сборщика мусора.

См. главу 2 «Методологии», где кратко описываются некоторые из этих методологий, а также дополнительные общие методологии, в частности профилирование процессора, определение характеристик рабочей нагрузки и анализ с увеличением детализации. См. также следующие главы, посвященные анализу потребления системных ресурсов и виртуализации.

Эти методологии можно использовать по отдельности или в сочетании друг с другом. Попробуйте применить их в том порядке, что указан в табл. 5.2.

Поищите описание специальных методов анализа для конкретного приложения и языка программирования, на котором оно написано. В них могут упоминаться характерные особенности поведения приложения, включая известные проблемы. Это поможет получить дополнительный выигрыш в производительности.

5.4.1. Профилирование процессора

Профилирование процессора — важный шаг в анализе производительности приложений. Оно подробно описывается в главе 6 «Процессоры», начиная с раздела 6.5.4 «Профилирование». В этом разделе кратко рассказывается о профилировании процессора и флейм-графиках, а также о том, как профилирование процессора используют для анализа периодов вне процессора.

Для Linux есть множество инструментов, поддерживающих профилирование процессора, в том числе `perf(1)` и `profile(8)`, краткое описание которых приводится в разделе 5.5 «Инструменты наблюдения». Оба этих инструмента используют прием на основе выборки трассировок стека по времени. Они действуют в режиме ядра и могут захватывать трассировки стека, как ядра, так и пользовательских приложений, создавая профиль *смешанного режима*. Это дает (почти) полную картину потребления процессора.

Приложения и среды выполнения иногда предлагают свои профилировщики, работающие в пользовательском режиме. Они не могут отображать потребление процессора ядром. Такие профилировщики могут давать искаженное представление о расходовании процессорного времени, потому что могут не знать, когда ядро исключило приложение из процесса планирования, и не учитывать этого. Я всегда начинаю с профилировщиков, действующих в режиме ядра (`perf(1)` и `profile(8)`), и использую профилировщики пользовательского режима только в крайнем случае.

Профилировщики, основанные на выборке трассировок стека, отбирают множество образцов: типичный профиль потребления процессора в Netflix строится на основе образцов, отбираемых с частотой 49 Гц на (примерно) 32 процессорах в течение 30 с: в общей сложности получается 47 040 образцов.

Для анализа этого множества образцов профилировщики обычно предоставляют различные средства для их обобщения или визуализации. Одно из таких средств визуализации отобранных трассировок стека — это *флейм-графики*, которые я изобрел.

Флейм-графики потребления процессора

Пример флейм-графика потребления процессора был показан в главе 1, и еще один пример см. на рис. 2.15. Пример графика на рис. 5.3 включает обозначение «ext4» для дальнейшего использования. Эти флейм-графики были получены в *смешанном режиме* и показывают стеки в обоих пространствах, ядра и пользователя.

Каждый прямоугольник на флейм-графике — это фрейм из трассировки стека, а ось Y показывает поток выполнения кода: на самом верху отображается текущая функция, а в направлении вниз — ее предки. Ширина фрейма пропорциональна его присутствию в профиле, а его положение на оси X не имеет значения (в этом примере фреймы выводятся в алфавитном порядке). Читая флейм-график, сначала ищите самые широкие «плато» или «башни» — именно на них тратится основная часть процессорного времени. Более подробную информацию о флейм-графиках ищите в главе 6 «Процессоры» раздела 6.7.3 «Флейм-графики».

Больше всего процессорного времени в примере на рис. 5.3 потребляет функция `crc32_z()`, охватывающая около 40 % ширины графика (центральное плато). Башня

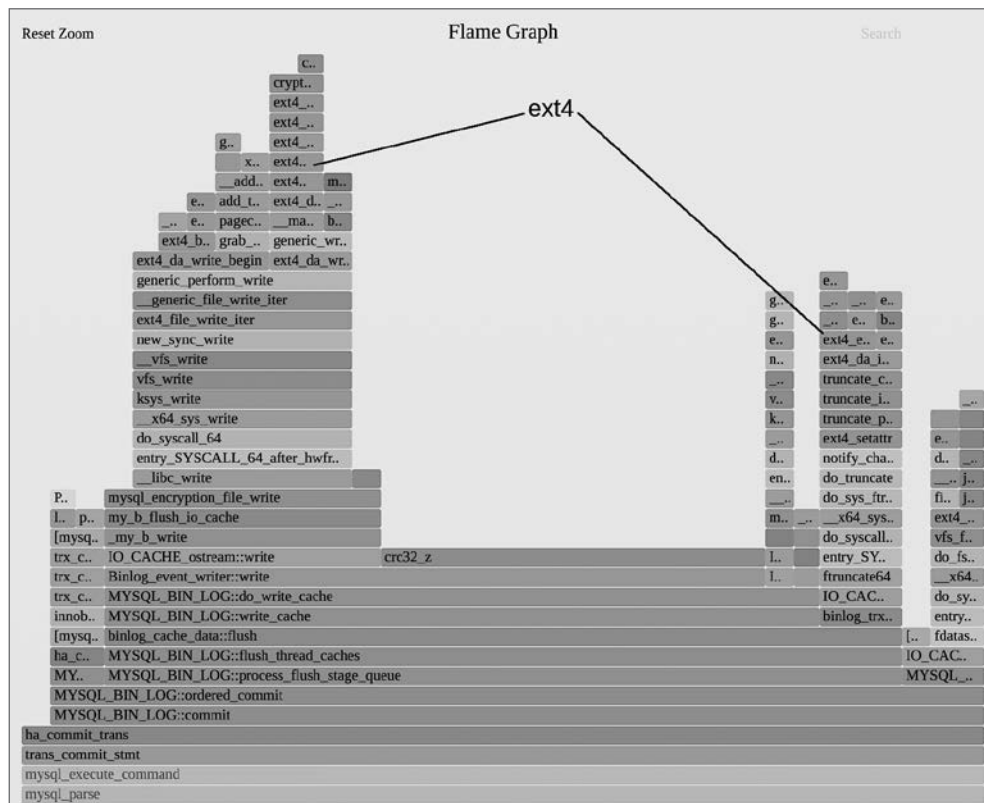


Рис. 5.3. Фрагмент флейм-графика потребления процессора

слева показывает путь к системному вызову `write(2)` в ядре, на выполнение которого в целом уходит около 30 % процессорного времени. Беглого взгляда оказалось достаточно, чтобы определить две низкоуровневые цели для оптимизации. Исследование исходного кода пути (вниз) к этим функциям помогает выявить высокоуровневые цели: в данном случае все процессорное время потребляется функцией `MYSQL_BIN_LOG::commit()`.

Я не могу сказать, что делают функции `crc32_z()` и `MYSQL_BIN_LOG::commit()`, хотя и догадываюсь. Профили потребления процессора раскрывают внутреннюю работу приложений, и если вы не разработчик приложения, то вы едва ли будете знать, что это за функции. Нужно будет изучить их, чтобы разработать действенные меры по увеличению производительности.

Для примера я погуглил по названию функции `MYSQL_BIN_LOG::commit()` и быстро нашел статьи, описывающие, как работает механизм двоичного журналирования в MySQL, который используется для восстановления и репликации базы данных. Также я нашел способы его настройки или полного отключения. Быстрый поиск по имени `crc32_z()` показал, что это функция вычисления контрольной суммы из библиотеки `zlib`. Может быть, есть более новая и быстрая версия `zlib`? Поддерживает ли процессор оптимизированную инструкцию CRC и использует ли ее библиотека `zlib`? Нужно ли в MySQL вообще вычислять контрольную сумму CRC или ее вычисление можно отключить? Узнать больше о таком стиле мышления вы сможете в главе 2 «Методологии» в разделе 2.5.20 «Мантры производительности».

В разделе 5.5.1 «perf» вы найдете обобщенные инструкции по созданию флейм-графиков потребления процессора с использованием `perf(1)`.

Определение следов ожидания вне процессора

Профили потребления процессора могут показать не только время выполнения на процессоре. С их помощью можно искать доказательства наличия проблем других типов, связанных с ожиданием вне процессора. Дисковый ввод/вывод, например, можно определить по потреблению процессора функциями доступа к файловой системе и инициализации блочного ввода/вывода. Это похоже на поиск следов медведя: вы не видите медведя, но по следам вы можете узнать, что он есть.

Изучая флейм-график потребления процессора, можно найти доказательства использования операций ввода/вывода файловой системы, дискового ввода/вывода, сетевого ввода/вывода, конфликтов блокировок и т. д. На рис. 5.3 для примера показан ввод/вывод с файловой системой `ext4`. Просмотрев достаточно большое количество флейм-графиков, вы быстро познакомитесь с именами функций, которые нужно искать: с префикса «`tcp_*`» начинаются имена функций TCP-стека ядра, с префикса «`blk_*`» — имена функций блочного ввода/вывода в ядре и т. д. Вот некоторые условия поиска, которые можно использовать в Linux:

- «`ext4`» (или «`btrfs`», «`xfs`», «`zfs`»): для поиска операций с файловой системой;
- «`blk`»: для операций блочного ввода/вывода;

- **«tcp»**: для поиска операций сетевого ввода/вывода;
- **«utex»**: для поиска конфликтов блокировок («mutex» или «futex»);
- **«alloc»** или **«object»**: для поиска путей в коде, выделяющих память.

Этот метод позволяет только определить наличие тех или иных действий, но не их масштабы. Флейм-график потребления процессора показывает величину потребления, а не время, проведенное в ожидании вне процессора. Чтобы напрямую измерить время ожидания, можно использовать анализ времени ожидания вне процессора, о котором пойдет речь ниже, но имейте в виду, что измерение этого времени обычно сопряжено с большим оверхедом.

5.4.2. Анализ времени ожидания вне процессора

Анализ времени ожидания вне процессора заключается в исследовании потоков, которые не выполняются на процессоре прямо сейчас. Это состояние называется *вне процессора* (off-CPU). Обычно анализ включает выявление всех причин блокировки потоков: дисковый ввод/вывод, сетевой ввод/вывод, конфликт блокировок, явная приостановка выполнения, вытеснение планировщиком и т. д. Их разбор и изучение проблем с производительностью, которые они вызывают, как правило, предполагает использование разных инструментов. Анализ времени ожидания вне процессора — это один метод для исследования всех этих причин и может поддерживаться единственным инструментом профилирования времени ожидания вне процессора.

Профилирование времени ожидания вне процессора выполняется разными способами, в том числе:

- **Выборкой**: когда по таймеру выбираются потоки, находящиеся вне процессора, или просто все потоки (так называемая *выборка по часам*).
- **Трассировкой планировщика**: путем инструментации планировщика в ядре для определения времени, в течение которого потоки находятся вне процессора, и фиксации этого времени вместе с трассировкой стека. Трассировка стека не изменяется, когда поток находится вне процессора (так как он не выполняется и не может ее изменить), поэтому трассировку стека достаточно прочитать только один раз для каждого события блокировки.
- **Инструментацией приложений**: некоторые приложения имеют встроенные точки инструментации в тех путях в коде, которые часто блокируются, например ведущих к дисковому вводу/выводу. Такие точки инструментации могут предоставлять контекст приложения. Несмотря на удобство и полезность этого подхода, он обычно не учитывает события, связанные с оставлением процессора (вытеснение планировщиком, отказы страниц и т. д.).

Первые два подхода предпочтительнее, потому что пригодны для всех приложений и позволяют увидеть все события оставления процессора, но они сопряжены с большим оверхедом. Выборка трассировок стека на частоте 49 Гц не должна приводить к значительным оверхедам, скажем, в системе с восемью процессорами, но выборка

потоков, находящихся вне процессора, должна просматривать пул потоков, а не пул процессоров. Одна и та же система может иметь 10 000 потоков, большинство из которых простаивает, поэтому оверхед на их выборку может оказаться в 1000 раз больше¹ (представьте себе профилирование потребления процессора в системе с 10 000 процессоров). Трассировка планировщика тоже может стоить значительного оверхеда, потому что в системе может происходить 100 000 и даже больше событий планировщика в секунду.

Трассировка планировщика — это широко используемый сейчас метод, основанный на моих собственных инструментах, таких как `offcputime(8)` (раздел 5.5.3 «`offcputime`»). В ходе трассировки я использую оптимизацию, которая заключается в записи только событий оставления процессора на периоды, превышающие определенный порог, что уменьшает количество выборок². Я также использую BPF для обобщения стеков в контексте ядра вместо отправки всех выборок в пространство пользователя, что еще больше снижает оверхеды. Несмотря на полезность этих методов, будьте осторожны с профилированием времени ожидания вне процессора в промышленных средах и оценивайте оверхед в тестовой среде перед использованием.

Флейм-графики времени вне процессора

Профили времени ожидания вне процессора можно представить в виде флейм-графика. На рис. 5.4 показан 30-секундный профиль в масштабе всей системы. Я увеличил масштаб, чтобы показать потоки сервера MySQL, обрабатывающие команды (запросы).

Основная часть времени ожидания вне процессора приходится на путь в коде, ведущий к `fsync()` и файловой системе `ext4`. Курсор на рис. 5.4 находится над функцией `Prepared_statement::execute()`, чтобы показать, что информационная строка внизу сообщает продолжительность ожидания вне процессора в этой функции: всего 3,97 с. Интерпретация флейм-графика этого вида аналогична интерпретации флейм-графика потребления процессора: в первую очередь ищите и исследуйте самые широкие башни.

Используя флейм-графики потребления процессора и ожидания, можно получить полное представление о распределении времени в пути кода: это очень мощный

¹ Пожалуй, даже больше, потому что здесь требуется получить трассировки стека для потоков, находящихся вне процессора, и их стеки едва ли останутся в кэше процессора (в отличие от стеков потоков, выполняющихся на процессоре). Ограничив профилирование одним приложением, можно уменьшить количество потоков, но в этом случае профиль получится неполным.

² Прежде чем вы скажете: «А что, если таких коротких простоев, которые исключает эта оптимизация, окажется очень много?», — вы должны увидеть свидетельство этого в профиле потребления процессора в виде частого вызова планировщика и только потом подумать об отключении этой оптимизации.

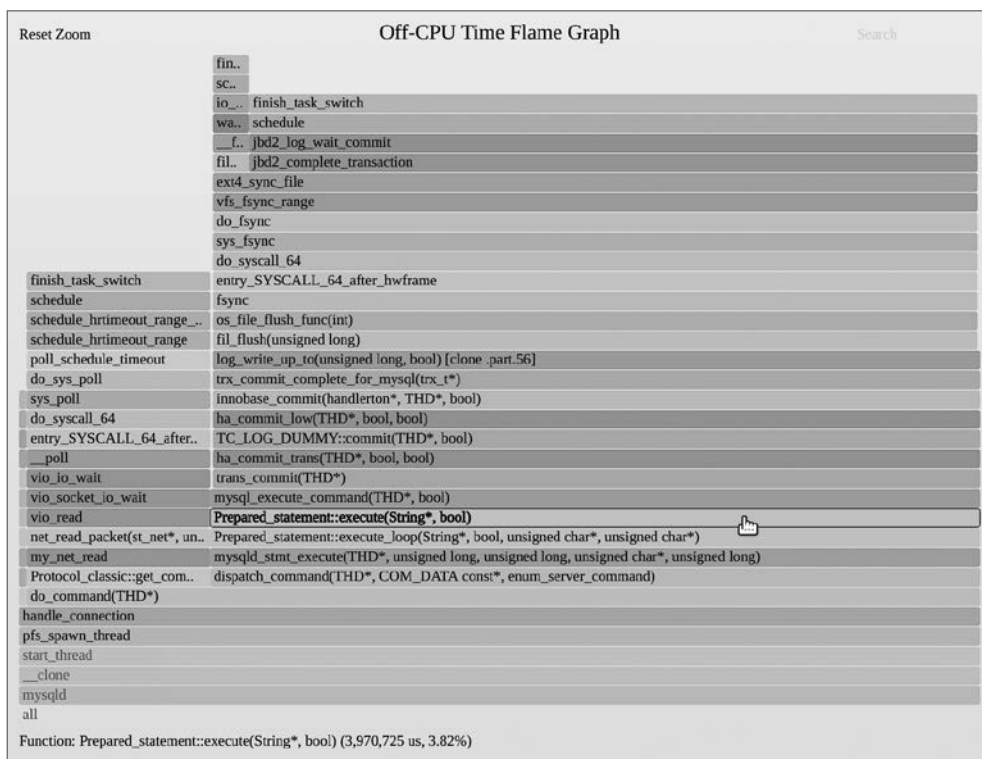


Рис. 5.4. Флейм-график времени ожидания вне процессора, в увеличенном масштабе

инструмент. Я обычно создаю их как отдельные флейм-графики. Их можно объединить в один флейм-график, который я называю *графиком горячего/холодного пламени*. Но этот прием дает не так много преимуществ, потому что время на процессоре сжимается и превращается в узкую башню из-за того, что большая часть графика горячего/холодного пламени отображает время ожидания. Это обусловлено еще и тем, что количество потоков, ожидающих вне процессора, может превышать количество потоков, выполняющихся на процессоре, на два порядка, в результате чего график горячего/холодного пламени на 99 % состоит из времени ожидания вне процессора, которое (если его не отфильтровать) в основном складывается из времени простоя.

Время простоя

Помимо оверхеда на сбор профилей времени вне процессора, есть еще одна проблема, связанная с их интерпретацией: в этих профилях может преобладать время простоя — время, когда потоки простаивают в ожидании работы. На рис. 5.5 показан тот же график времени вне процессора, но без увеличения масштаба для детального исследования конкретного потока.

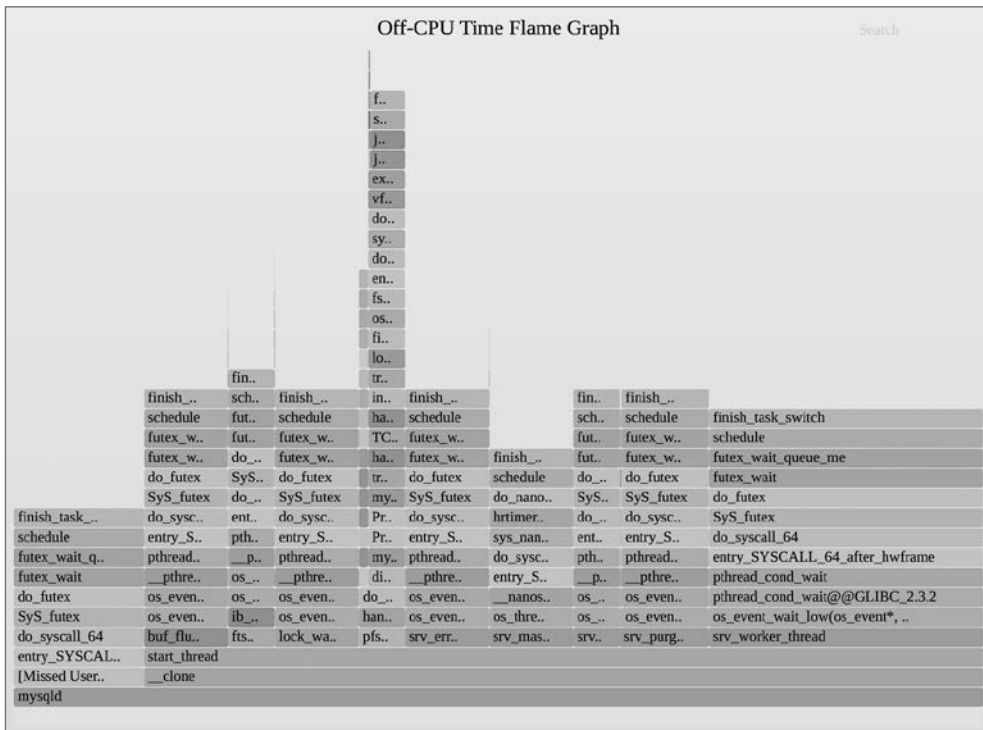


Рис. 5.5. Флейм-график времени ожидания вне процессора, полный

Большая часть времени в этом флейм-графике приходится на пути кода, ведущие к функциям `pthread_cond_wait()` и `futex()`: это потоки, ожидающие работы. Функции потоков можно увидеть на флейм-графике, справа налево: `srv_worker_thread()`, `srv_purge_coordinator_thread()`, `srv_monitor_thread()` и т. д.

Порекомендую пару приемов определения времени ожидания вне процессора:

- Увеличьте масштаб (или отфильтруйте лишнее) функций обработки запросов, потому что наибольший интерес представляет время, проведенное потоком вне процессора во время обработки запросов. Для сервера MySQL это функция `do_command()`. Поиск `do_command()` с последующим увеличением масштаба дает флейм-график как на рис. 5.4. Это очень эффективный прием, но для его использования вы должны знать, какую функцию искать в конкретном приложении.
- Используйте фильтр во время сбора данных, чтобы исключить состояния потока, не представляющие интереса. Эффективность этого приема зависит от ядра; в Linux сопоставление с признаком `TASK_UNINTERRUPTIBLE` помогает сосредоточить внимание на многих интересных событиях оставления процессора, но исключает некоторые другие, не менее интересные события.

Иногда можно встретить пути в коде, блокирующие приложение и ожидающие чего-то еще, например освобождения блокировки. Чтобы глубже изучить детали, нужно знать, почему держателю блокировки потребовалось так много времени, чтобы освободить ее. Помимо анализа блокировок, описанного в разделе 5.4.7 «Настройка статической производительности», широко используется метод, основанный на инструментации события возобновления (пробуждения). Это довольно сложный вид анализа: см. главу 14 в книге «BPF Performance Tools» [Gregg 19], а также инструменты `wakeuptime(8)` и `offwaketime(8)` из ВСС.

В разделе 5.5.3 «`offcputime`» приводятся инструкции по созданию флейм-графиков времени вне процессора с использованием `offcputime(8)` из ВСС. Еще одна полезная цель для изучения приложений, помимо событий планировщика, — события системных вызовов.

5.4.3. Анализ системных вызовов

Для изучения проблем производительности, обусловленных потреблением ресурсов, можно использовать прием инструментации системных вызовов. Его цель — выяснить, на что тратится время системного вызова, включая тип системного вызова и причину, по которой он вызывается.

Вот некоторые цели для анализа системных вызовов:

- **Трассировка запуска нового процесса:** трассируя системный вызов `execve(2)`, можно фиксировать попытки запуска новых процессов и анализировать проблемы короткоживущих процессов. См. описание инструмента `execsnoop(8)` в разделе 5.5.5 «`execsnoop`».
- **Профилирование ввода/вывода:** трассируя `read(2)`/`write(2)`/`send(2)`/`recv(2)` и их варианты, а также изучая объем ввода/вывода, флаги и пути в коде, можно выявить проблемы неоптимального ввода/вывода, например большое количество операций ввода/вывода небольшими порциями. См. описание инструмента `bpfttrace` в разделе 5.5.7 «`bpfttrace`».
- **Анализ времени выполнения в пространстве ядра:** когда система сообщает о большом количестве времени выполнения в режиме ядра, которое часто обозначается как «%sys», инструментация системных вызовов может помочь определить причину. См. описание инструмента `syscount(8)` в разделе 5.5.6 «`syscount`». Системные вызовы потребляют большую часть процессорного времени, потраченного в режиме ядра, но не все; к исключениям можно отнести отказы страниц, асинхронные потоки ядра и прерывания.

Системные вызовы — это хорошо задокументированный API (на страницах справочного руководства `man`), что делает их легкодоступным источником событий для изучения. Кроме того, они вызываются приложением синхронно, а это означает, что выборка трассировки стека из системных вызовов покажет путь в коде приложения, ответственный за обращение к этому системному вызову. Такие трассировки стека можно визуализировать в виде флейм-графика.

5.4.4. Метод USE

Представленный в главе 2 «Методологии» и продемонстрированный в последующих главах, метод USE проверяет коэффициент использования (Utilization), насыщение (Saturation) и наличие ошибок (Errors) для всех аппаратных ресурсов. Он позволяет решить многие проблемы с производительностью приложений, помогая выявить ресурс, ставший узким местом.

Метод USE также может применяться к программным ресурсам. Если у вас есть функциональная диаграмма, показывающая внутренние компоненты приложения, определите коэффициент использования, насыщение и наличие ошибок для каждого программного ресурса и оцените, в чем может быть проблема.

Например, приложение может использовать пул рабочих потоков для обработки запросов, которые помещаются в очередь перед обработкой. Если рассматривать эту очередь как ресурс, то перечисленные три метрики можно определить так:

- **Коэффициент потребления:** среднее количество потоков, занятых обработкой запросов в течение периода трассировки, в процентах от общего числа потоков. Например, 50 % будет означать, что в среднем половина потоков была занята обработкой запросов.
- **Насыщение:** средняя длина очереди запросов в течение периода трассировки. Это поможет узнать, сколько запросов находилось в очереди и ожидало, пока освободится рабочий поток.
- **Ошибки:** количество запросов, отклоненных или не выполненных по какой-либо причине.

Затем нужно найти способ, как измерить эти метрики. Они могут храниться где-то в приложении либо их нужно добавить или измерить другим способом, например динамической трассировкой.

Системы массового обслуживания, как в этом примере, также исследуют с привлечением теории массового обслуживания (см. главу 2 «Методологии»).

В качестве другого примера рассмотрим файловые дескрипторы. Система может накладывать ограничение на количество файловых дескрипторов, так что дескрипторы — это конечный ресурс. Тремя метриками, характеризующими его, могут быть:

- **Коэффициент потребления:** количество используемых файловых дескрипторов в процентах от предельного значения.
- **Насыщение:** зависит от поведения операционной системы: если потоки блокируются в ожидании файловых дескрипторов, то насыщение может характеризоваться количеством потоков, заблокированных в ожидании этого ресурса.
- **Ошибки:** количество ошибок при попытке получить дескриптор, таких как EFILE, «Too many open files» («слишком много открытых файлов»).

Повторите это упражнение для компонентов вашего приложения и пропустите все метрики, которые не имеют смысла. Так вы сможете выработать свой чек-лист

для проверки работоспособности приложения, прежде чем переходить к другим методологиям.

5.4.5. Анализ состояния потока

Это самая первая методология, которую я использую, приступая к решению любых проблем с производительностью, но и не самая простая в Linux. Цель состоит в том, чтобы в общих чертах определить, где прикладные потоки проводят основное время. Эта информация позволяет быстро решить одни проблемы и выбрать направление для дальнейшего анализа других. В ходе такого анализа нужно разделить время каждого прикладного потока на несколько значимых состояний.

Поток имеет как минимум два состояния: на процессоре и вне процессора. Определить, находятся ли потоки в состоянии выполнения на процессоре, можно с помощью стандартных метрик и инструментов (например, `top(1)`), и при необходимости выполнить профилирование времени на процессоре или вне процессора (см. разделы 5.4.1 «Профилирование процессора» и 5.4.2 «Анализ времени ожидания вне процессора»). Но эта методология более эффективна, когда число состояний больше двух.

Девять состояний

Я отобрал девять состояний потока, которые обеспечивают более эффективный анализ, чем два предыдущих состояния (на процессоре и вне процессора):

- **пользователь:** выполняется на процессоре в пользовательском режиме;
- **ядро:** выполняется на процессоре в режиме ядра;
- **готов к выполнению:** время вне процессора в ожидании своей очереди на выполнение;
- **подкачка** (анонимная подкачка страниц): готов к выполнению, но заблокирован в ожидании подкачки страниц;
- **дисковый ввод/вывод:** ожидает завершения ввода/вывода блочным устройством: чтение/запись, подкачка данных/кода;
- **сетевой ввод/вывод:** ожидает завершения ввода/вывода сетевым устройством: чтение/запись через сокет;
- **приостановлен:** преднамеренная приостановка;
- **заблокирован:** ожидает освобождения блокировки синхронизации (ожидает, пока какой-то другой поток освободит блокировку);
- **простой:** ожидание работы.

Эта модель с девятью состояниями изображена на рис. 5.6.

Производительность обработки запросов в приложении можно увеличить, сократив время пребывания во всех состояниях, кроме простоя. При прочих равных это означает меньшую задержку при обработке запросов и возможность обслуживать большую нагрузку.

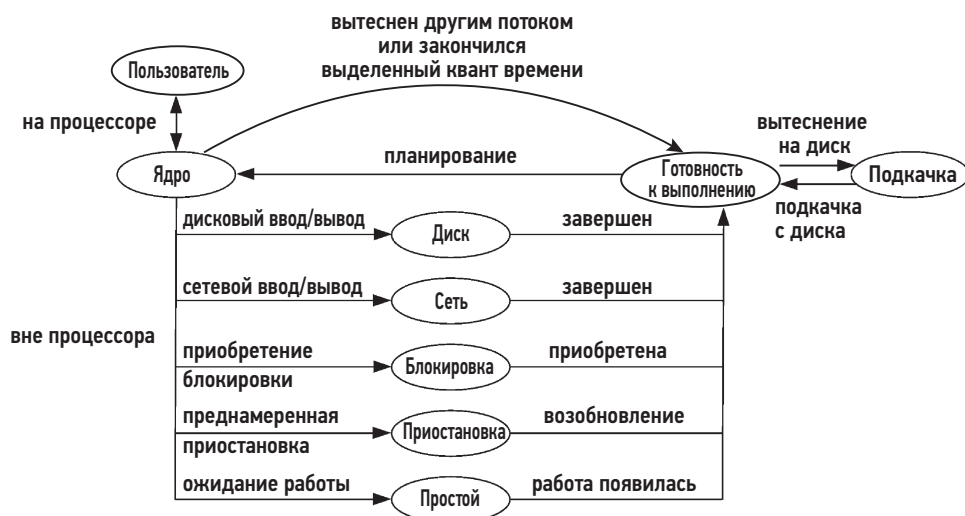


Рис. 5.6. Модель девяти состояний потока

Установив, в каких состояниях потоки расходуют больше всего времени, можно исследовать их дальше:

- **Пользователь** или **ядро**: профилирование может определить, какие пути в коде выполняются процессором, включая время, потраченное на ожидание спин-блокировок. См. раздел 5.4.1 «Профилирование процессора».
- **Готов к выполнению**: пребывание в этом состоянии означает, что приложению требуется больше процессорного времени. Изучите потребление процессора для всей системы и любые ограничения на доступ к процессору для приложения (например, через средства управления ресурсами).
- **Подкачка** (анонимная подкачка страниц): недостаточный объем оперативной памяти, доступной приложению, может вызвать задержки из-за подкачки страниц. Изучите потребление памяти для всей системы и любые имеющиеся ограничения на доступ к памяти. Подробности см. в главе 7 «Память».
- **Дисковый ввод/вывод**: это состояние включает дисковый ввод/вывод непосредственно и отказы страниц. Описание анализа этих проблем см. в разделе 5.4.3 «Анализ системных вызовов», в главе 8 «Файловые системы» и в главе 9 «Диски». Определение характера рабочей нагрузки поможет решить многие проблемы дискового ввода/вывода: изучите имена файлов, объемы и типы ввода/вывода.
- **Сетевой ввод/вывод**: в этом состоянии поток пребывает, когда блокируется в ожидании завершения сетевого ввода-вывода (отправка/получение), но не в ожидании подтверждения создания нового соединения (это время относится к времени простоя). Анализ этих проблем описан в разделе 5.4.3 «Анализ системных вызовов», в разделе 5.5.7 «bpfftrace» и его подразделе «Профилирование ввода/вывода», а также в главе 10 «Сеть». Определение характера рабочей

нагрузки также полезно при проблемах с сетевым вводом/выводом: изучите имена хостов, протоколы и пропускную способность.

- **Приостановлен:** проанализируйте причину (путь в коде) и продолжительность приостановок.
- **Заблокирован:** определите блокировку, поток, удерживающий ее, и причину, по которой эта блокировка удерживается так долго. Причина может быть в том, что держатель блокируется на другой блокировке, что требует дальнейшего исследования. Обычно этот анализ выполняется разработчиком программного обеспечения, хорошо знакомым с приложением и его иерархией блокировок. Я создал инструмент ВСС, помогающий в проведении такого анализа: `offwaketime(8)` (включен в ВСС), который показывает трассировки стека, ведущие к блокировке и ее освобождению.

Из-за особенностей ожидания работы приложениями часто можно обнаружить, что время в состояниях ожидания сетевого ввода/вывода и освобождения блокировки — это время простоя. Рабочий поток приложения может ждать появления работы, ожидая завершения сетевого ввода/вывода со следующим запросом (например, HTTP keep-alive) или когда условная переменная (состояние «заблокирован») возобновит выполнение процесса.

Ниже я кратко описал процесс измерения времени пребывания потоков в этих состояниях в Linux.

Linux

На рис. 5.7 показана модель состояния потока для потока ядра в Linux.

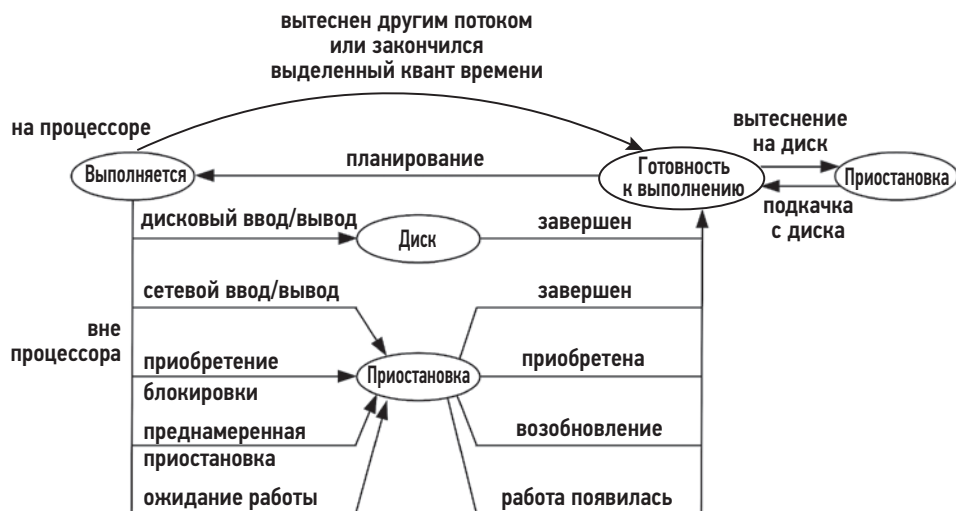


Рис. 5.7. Модель состояний потока в Linux

Состояние потока ядра зависит от состояния (state) в структуре `task_struct` ядра: состоянию «готов к выполнению» соответствует значение `TASK_RUNNING`, состоянию «дисковый ввод/вывод» — значение `TASK_UNINTERRUPTIBLE`, а состоянию «приостановлен» — значение `TASK_INTERRUPTIBLE`. Эти состояния отображаются многими инструментами, включая `ps(1)` и `top(1)`, с помощью однобуквенных кодов: R (Runnable — готов к выполнению), D (Disk — дисковый ввод/вывод) и S (Sleep — приостановлен) соответственно. (Есть и другие состояния, которые я не включил в этот список, например «остановлен отладчиком».)

Эти обозначения дают некоторые подсказки для дальнейшего анализа, но они не определяют разделение времени на девять состояний, описанных выше. Для точной классификации состояния требуется дополнительная информация: например, состояние Runnable (готов к выполнению) можно разделить на время пребывания в пространстве пользователя и в пространстве ядра, используя статистики из `/proc` или полученные с помощью `getrusage(2)`.

Другие ядра обычно предлагают большее количество состояний, что упрощает применение этой методологии. Первоначально я разработал и использовал эту методологию для ядра Solaris, вдохновившись его поддержкой широкого круга микросостояний. Она сохраняла время пребывания потока в восьми различных состояниях: пользователь, система, ловушка, ошибка в коде, ошибка в данных, блокировка, приостановка и пребывание в очереди на выполнение (задержка планировщика). Этот список не соответствует моим идеальным состояниям, но он лучшая отправная точка.

Далее я расскажу о трех подходах, которые использую в Linux: анализ косвенных признаков, анализ времени вне процессора и прямое измерение.

Анализ косвенных признаков

При этом анализе можно начать с типовых инструментов ОС — `pidstat(1)` и `vmstat(8)`, что позволит определить, в каком состоянии поток проводит основное время. Вот названия инструментов и соответствующих колонок в их выводе:

- **Пользователь:** `pidstat(1)` «%usr» (время в этом состоянии измеряется напрямую).
- **Ядро:** `pidstat(1)` «%system» (время в этом состоянии измеряется напрямую).
- **Готов к выполнению:** `vmstat(8)` «r» (для системы в целом).
- **Подкачка:** `vmstat(8)` «si» и «so» (для системы в целом).
- **Дисковый ввод/вывод:** `pidstat(1)` -d «iodelay» (включает время пребывания в состоянии подкачки).
- **Сетевой ввод/вывод:** `sar(1)` -n DEV «rxkB/s» и «txkB/s» (для системы в целом).
- **Приостановлен:** трудно поддается определению.
- **Заблокирован:** `perf(1)` `top` (может напрямую определять время в ожидании освобождения спин-блокировки).
- **Простой:** трудно поддается определению.

Некоторые из этих статистик оцениваются для системы в целом. Если после запуска `vmstat(8)` обнаружится, что в системе в целом часто происходит подкачка, то это состояние можно исследовать с помощью более глубоких инструментов и выяснить, насколько подкачка влияет на производительность интересующего приложения. Эти инструменты описаны в следующих разделах и главах.

Анализ времени вне процессора

Многие состояния характеризуются пребыванием вне процессора (все, кроме состояний выполнения в режимах пользователя и ядра). Чтобы определить конкретное состояние потока, можно применить анализ времени вне процессора. См. раздел 5.4.2 «Анализ времени ожидания вне процессора».

Прямое измерение

Вот как можно точно измерить время пребывания потока в разных состояниях:

Пользователь: время выполнения на процессоре в пользовательском режиме можно определить с помощью множества инструментов, а также `/proc/PID/stat` и `getrusage(2)`. `pidstat(1)` сообщает это время в колонке «%usr».

Ядро: время выполнения на процессоре в режиме ядра можно определить с помощью `/proc/PID/stat` и `getrusage(2)`. `pidstat(1)` сообщает это время в колонке «%system».

Готов к выполнению: это состояние определяется механизмом ядра `schedstats` в наносекундах и доступно через `/proc/PID/schedstat`. Его также можно оценить, измерив некоторый оверхед с помощью инструментов трассировки, включая подкоманду `sched` в `perf(1)` и команду `runqlat(8)` в ВСС; обе рассматриваются в главе 6 «Процессоры».

Подкачка: время, потраченное на подкачку (анонимную подкачку страниц) в наносекундах, можно оценить *измерением задержки*, как описывается в главе 4 «Инструменты наблюдения» в разделе 4.3.3 «Учет задержек», где представлен пример использования инструмента `getdelays.c`. Для определения задержек, вызванных подкачкой, также можно использовать инструменты трассировки.

Дисковый ввод/вывод: `pidstat(1) -d` показывает в колонке «`iodelay`» количество тактов системных часов, в течение которых процесс ждал завершения блочного ввода/вывода и подкачки. Без времени подкачки для системы в целом (которое сообщает `vmstat(8)`) легко по ошибке предположить, что любая задержка ввода/вывода соответствует пребыванию в состоянии дискового ввода/вывода. Учет задержек и другие механизмы учета, такие как `iotop(8)`, если они включены, тоже позволяют оценить время, потраченное на блочный ввод/вывод. Также можно использовать инструменты трассировки, такие как `biotop(8)` в ВСС.

Сетевой ввод/вывод: время пребывания в состоянии сетевого ввода/вывода можно исследовать с помощью инструментов трассировки — ВСС и `bpfttrace`, включая `tcptop(8)` для сетевого ввода/вывода через TCP. У приложения могут быть свои инструменты для определения времени ввода/вывода (сетевого и дискового).

Приостановлен: время добровольного пребывания в состоянии приостановки можно исследовать с помощью инструментов трассировки и событий, включая точку трассировки `syscalls:sys_enter_nanosleep`. Мой инструмент `napttime.bt` трассирует эти события приостановки и выводит PID процесса и продолжительность приостановки [Gregg 19], [Gregg 20b].

Заблокирован: время пребывания в заблокированном состоянии можно оценить с помощью инструментов трассировки, включая `klockstat(8)` из ВСС и из репозитория `bpf-perf-tools-book`, `pmlock.bt` и `pmheld.bt` для ожидания на мьютексах `pthread`, а также `mlock.bt` и `mhheld.bt` для ожидания на мьютексах ядра.

Простой: пути в коде приложения, ведущие к ожиданию появления работы, можно исследовать с помощью инструментов трассировки.

Иногда может показаться, что приложения постоянно приостановлены: они остаются заблокированными вне процессора, показывая нулевой объем ввода/вывода и полное отсутствие других событий. Чтобы определить, в каком состоянии находятся потоки такого приложения, может потребоваться отладчик, например `pstack(1)` или `gdb(1)`. Им можно проверить трассировки стека потоков или прочитать соответствующие файлы `/proc/PID/stack`. Обратите внимание, что подобные отладчики могут сами приостанавливать работу целевого приложения и вызывать проблемы с производительностью. Прежде чем пробовать их в продакшене, выясните, как их использовать, и оцените связанные с ними риски.

5.4.6. Анализ блокировок

Блокировки в многопоточных приложениях легко могут стать узким местом, препятствуя параллелизму и масштабируемости. Однопоточные приложения могут приостанавливаться на блокировках в ядре (например, на блокировках в файловой системе).

Для блокировок анализируют:

- наличие конкуренции;
- чрезмерное время удержания.

В первом случае определяют наличие проблем *в данный момент*. Чрезмерное время удержания не всегда бывает непосредственной проблемой, но может стать ею в будущем при еще большем распараллеливании нагрузки. В любом случае попробуйте определить имя блокировки (если оно есть) и путь в коде, ведущий к ее использованию.

Для анализа блокировок есть специализированные инструменты, но иногда проблемы можно решить только с помощью профилирования процессорного времени. Конкуренция за спин-блокировки сопровождается повышенным потреблением процессора, и ее легко выявить профилированием трассировок стека. Конкуренция за *адаптивные мьютексы* тоже часто сопровождается несколько повышенным потреблением процессора, которое также можно определить профилированием трассировок стека.

Но имейте в виду, что профиль потребления процессорного времени дает только часть картины, потому что потоки могут блокироваться и ждать освобождения блокировок вне процессора. См. раздел 5.4.1 «Профилирование процессора».

Конкретные инструменты анализа блокировок в Linux описываются в разделе 5.5.7 «bpftrace».

5.4.7. Настройка статической производительности

Основное внимание при настройке статической производительности уделяется проблемам в настроенной среде. Анализируя производительность приложения, изучите следующие аспекты статической конфигурации:

- Какая версия приложения используется и какие зависимости оно имеет? Есть ли более новые версии? Упоминаются ли в их примечаниях к выпуску улучшения производительности?
- Есть ли известные проблемы с производительностью? Есть ли их список в базе данных с описанием ошибок?
- Как настроено приложение?
- Есть ли настройки, отличающиеся от настроек по умолчанию, и в чем причина их изменения? (Были ли эти изменения основаны на измерениях и анализе или только на предположениях?)
- Использует ли приложение кэш объектов? Какого размера?
- Работает ли приложение конкурентно? Как оно настроено (например, размер пула потоков)?
- Работает ли приложение в специальном режиме? (Например, мог быть включен режим отладки, снижающий производительность, или приложение собрано для отладки вместо релиза сборки.)
- Какие системные библиотеки использует приложение? Каких версий?
- Какой распределитель памяти использует приложение?
- Настроено ли приложение для использования больших страниц в куче?
- Написано ли приложение на компилируемом языке? Какая версия компилятора использовалась? Какие параметры компилятора и оптимизации применялись? Версия приложения 64-битная?
- Есть ли в выполняемом коде расширенные инструкции? (Должны ли быть?) Например, инструкции SIMD или векторные инструкции, включая Intel SSE.
- Есть ли ошибки в приложении и не находится ли оно в режиме пониженной производительности? Возможно, оно неправильно настроено и всегда выполняется в режиме пониженной производительности?
- Есть ли системные ограничения и используются ли средства управления ресурсами, ограничивающие потребление процессора, памяти, файловой системы, диска или сети? (Такие средства широко используются в облачных средах.)

Ответы на эти вопросы помогут выявить варианты конфигурации, которые были упущены из виду.

5.4.8. Распределенная трассировка

В распределенной среде приложение может состоять из сервисов, работающих в разных системах. Каждый сервис можно исследовать как отдельное мини-приложение, но нужно также изучить работу распределенного приложения в целом. Это требует применения новых методологий и инструментов и обычно выполняется с использованием распределенной трассировки.

Распределенная трассировка включает регистрацию информации по каждому запросу с последующим объединением этой информации и ее изучением. Каждый запрос, охватывающий несколько сервисов, затем можно разбить на запросы к зависимостям и определить сервис, ответственный за высокую задержку приложения или ошибки.

Собранная информация может включать:

- уникальный идентификатор запроса (внешний идентификатор запроса);
- информацию о местоположении запроса в иерархии зависимостей;
- время начала и окончания обработки;
- код ошибки.

Основная сложность в распределенной трассировке — это количество генерируемых данных для журналирования: каждому запросу может соответствовать несколько записей. Один из способов преодоления этой сложности — *выборка на основе заголовка*, когда при встрече начала («заголовка») запроса принимается решение, следует ли включать его в выборку («трассировку»): например, трассировать только один из каждых 10 000 запросов. Этого достаточно для анализа производительности обработки большинства запросов, но такой подход затрудняет анализ периодических ошибок или выбросов из-за ограниченности данных. Некоторые распределенные трассировщики принимают решение на основе *хвоста* — сначала фиксируют все события, а затем решают, следует ли оставить информацию о том или ином запросе, исходя из величины задержки или наличия ошибок.

После выявления проблемного сервиса его можно проанализировать более подробно, используя другие методологии и инструменты.

5.5. ИНСТРУМЕНТЫ НАБЛЮДЕНИЯ

В этом разделе представлены инструменты наблюдения за производительностью приложений для ОС на базе Linux. Стратегии их использования описаны в предыдущем разделе.

Инструменты перечислены в табл. 5.3.

Таблица 5.3. Инструменты наблюдения в Linux

Раздел	Инструмент	Описание
5.5.1	perf	Профилирование процессора, создание флейм-графиков процессорного времени, трассировка системных вызовов
5.5.2	profile	Профилирование процессора методом выборки трассировок стека по времени
5.5.3	offcputime	Профилирование времени вне процессора методом трассировки планировщика
5.5.4	strace	Трассировка системных вызовов
5.5.5	execsnoop	Трассировка запуска новых процессов
5.5.6	syscount	Подсчет обращений к системным вызовам
5.5.7	bpfftrace	Трассировка сигналов, профилирование ввода/вывода, анализ блокировок

Первыми следуют инструменты профилирования процессора, а за ними — инструменты трассировки. Многие инструменты трассировки основаны на BPF и используют интерфейсы BCC и bpfftrace (глава 15); к ним относятся: profile(8), offcputime(8), execsnoop(8) и syscount(8). Полный перечень возможностей каждого инструмента вы найдете в соответствующей документации, включая справочные страницы man.

Также поищите инструменты для увеличения производительности конкретных приложений, не указанные в этой таблице. Кроме того, для анализа приложений можно использовать инструменты, ориентированные на ресурсы, — процессоры, память, диски и т. д., — которые описываются в последующих главах.

Многие из следующих инструментов собирают трассировки стека приложений. Если вы обнаружите, что трассировки содержат фреймы «[unknown]» (неизвестно) или выглядят невероятно короткими, прочитайте раздел 5.6 «Проблемы», в котором кратко описаны типичные проблемы и методы их устранения.

5.5.1. perf

perf(1) — это стандартный профилировщик Linux, многоцелевой инструмент со множеством применений. Подробнее о нем речь пойдет в главе 13 «perf». Профилирование процессора очень важно для анализа производительности приложений, поэтому здесь приводится краткое описание приемов профилирования с использованием perf(1). Более подробно о профилировании процессора и флейм-графиках я расскажу в главе 6 «Процессоры».

Профилирование процессора

Ниже приводится пример использования perf(1) для выборки трассировок стека (-g) на всех процессорах (-a) с частотой 49 Гц (-F 49: выборка в секунду) в течение 30 с с последующим выводом полученного списка трассировок:

```
# perf record -F 49 -a -g -- sleep 30
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.560 MB perf.data (2940 samples) ]
# perf script
mysqld 10441 [000] 64918.205722: 10101010 cpu-clock:pppH:
    5587b59bf2f0 row_mysql_store_col_in_innbase_format+0x270 (/usr/sbin/mysqld)
    5587b59c3951 [unknown] (/usr/sbin/mysqld)
    5587b58803b3 ha_innbase::write_row+0x1d3 (/usr/sbin/mysqld)
    5587b47e10c8 handler::ha_write_row+0x1a8 (/usr/sbin/mysqld)
    5587b49ec13d write_record+0x64d (/usr/sbin/mysqld)
    5587b49ed219 Sql_cmd_insert_values::execute_inner+0x7f9 (/usr/sbin/mysqld)
    5587b45dfd06 Sql_cmd_dml::execute+0x426 (/usr/sbin/mysqld)
    5587b458c3ed mysql_execute_command+0xb0d (/usr/sbin/mysqld)
    5587b4591067 mysql_parse+0x377 (/usr/sbin/mysqld)
    5587b459388d dispatch_command+0x22cd (/usr/sbin/mysqld)
    5587b45943b4 do_command+0x1a4 (/usr/sbin/mysqld)
    5587b46b22c0 [unknown] (/usr/sbin/mysqld)
    5587b5cfff0a [unknown] (/usr/sbin/mysqld)
    7fdbf66a9669 start_thread+0xd9 (/usr/lib/x86_64-linux-gnu/libpthread-2.30.so)
[...]
```

Всего в этом профиле 2940 образцов стека, но я показал только один. Подкоманда `script` выводит все образцы стека, сохраненные в профиле (файл `perf.data`). `perf(1)` имеет также подкоманду `report`, которая обобщает профиль в виде иерархии путей в коде. Профиль также можно визуализировать в виде флейм-графика процессорного времени.

Флейм-графики процессорного времени

В Netflix используется автоматизированная процедура получения флейм-графиков процессорного времени, поэтому операторы и разработчики могут запрашивать их из пользовательского интерфейса в браузере. Графики могут создаваться полностью с использованием открытого ПО, в том числе из GitHub. Флейм-график на рис. 5.3, например, был получен так:

```
# perf record -F 49 -a -g -- sleep 10; perf script --header > out.stacks
# git clone https://github.com/brendangregg/FlameGraph; cd FlameGraph
# ./stackcollapse-perf.pl < ./out.stacks | ./flamegraph.pl --hash > out.svg
```

После их выполнения файл `out.svg` можно открыть в браузере.

Скрипт `flamegraph.pl` позволяет использовать настраиваемые цветовые палитры для разных языков: например, для приложений на Java можно передать сценарию параметр `--color = java`. Чтобы получить список всех параметров, выполните команду `flamegraph.pl -h`.

Трассировка системных вызовов

Подкоманда `perf(1) trace` по умолчанию трассирует системные вызовы и является версией `strace(1)` для `perf(1)` (см. раздел 5.5.4 «`strace`»). Вот пример трассировки серверного процесса MySQL:

```
# perf trace -p $(pgrep mysqld)
? ( ): mysqld/10120 ... [continued]: futex())
= -1 ETIMEDOUT (Connection timed out)
  0.014 ( 0.002 ms): mysqld/10120 futex(uaddr: 0x7fbddc37ed48, op: WAKE|
PRIVATE_FLAG, val: 1) = 0
  0.023 (10.103 ms): mysqld/10120 futex(uaddr: 0x7fbddc37ed98, op: WAIT_BITSET|
PRIVATE_FLAG, utime: 0x7fbdc9cfcbc0, val3: MATCH_ANY) = -1 ETIMEDOUT (Connection
timed out)
[...]
```

Здесь показана лишь пара строк из вывода, соответствующих вызовам `futex(2)`, с помощью которых различные потоки в MySQL ждут появления работы (они преобладали на флэйм-графике времени вне процессора на рис. 5.5).

Преимущество `perf(1)` — в использовании буферов для каждого процессора, уменьшающих оверхед, что делает этот инструмент более безопасным в использовании, чем текущая реализация `strace(1)`. Он также может делать трассировку в масштабе всей системы, тогда как инструмент `strace(1)` требует указать конкретный процесс или процессы и может трассировать события, не связанные с обращениями к системным вызовам. Но `perf(1)` не обладает такими богатыми возможностями по трансляции аргументов системных вызовов, как `strace(1)`. Вот единственная строка из вывода `strace(1)` для сравнения:

```
[pid 10120] futex(0x7fbddc37ed98, FUTEX_WAIT_BITSET_PRIVATE, 0, {tv_sec=445110,
tv_nsec=427289364}, FUTEX_BITSET_MATCH_ANY) = -1 ETIMEDOUT (Connection timed out)
```

Здесь `strace(1)` развернул структуру `utime`. Сейчас ведется работа по улучшению подкоманды `trace` в `perf(1)` и использованию BPF для расширения возможностей интерпретации аргументов в символическую форму. Конечной целью `perf(1) trace` может стать полная замена `strace(1)`. (Подробнее о `strace(1)` см. в разделе 5.5.4 «`strace`».)

Анализ времени выполнения в ядре

`perf(1) trace` показывает время, проведенное в системных вызовах. Это помогает понять, из чего складывается время выполнения в режиме ядра, которое обычно отображается инструментами мониторинга, хотя анализ проще начинать с обобщенной сводки, чем с распределения времени по событиям. Такую обобщенную сводку по системным вызовам можно получить с помощью параметра `-s`:

```
# perf trace -s -p $(pgrep mysqld)
mysqld (14169), 225186 events, 99.1%
```

syscall	calls	total (msec)	min (msec)	avg (msec)	max (msec)	stddev (%)
sendto	27239	267.904	0.002	0.010	0.109	0.28%
recvfrom	69861	212.213	0.001	0.003	0.069	0.23%
poll	15478	201.183	0.002	0.013	0.412	0.75%
[...]						

Как видите, в выводе сообщается количество обращений к системным вызовам и время, проведенное в них.

Вывод, полученный в результате трассировки вызовов `futex(2)`, сам по себе малоинтересен, а при трассировке любого нагруженного приложения `perf(1) trace` генерирует лавину информации. Поэтому лучше начать с получения этой сводки, а затем использовать `perf(1) trace` с фильтром для исследования только интересных системных вызовов.

Профилирование ввода/вывода

Системные вызовы ввода/вывода представляют особый интерес, и некоторые из них можно заметить в предыдущем примере вывода. Вот пример трассировки вызовов `sendto(2)` с помощью фильтра (-e):

```
# perf trace -e sendto -p $(pgrep mysqld)
 0.000 ( 0.015 ms): mysqld/14097 sendto(fd: 37<socket:[833323]>, buff:
0x7fbdac072040, len: 12664, flags: DONTWAIT) = 12664
 0.451 ( 0.019 ms): mysqld/14097 sendto(fd: 37<socket:[833323]>, buff:
0x7fbdac072040, len: 12664, flags: DONTWAIT) = 12664
 0.624 ( 0.011 ms): mysqld/14097 sendto(fd: 37<socket:[833323]>, buff:
0x7fbdac072040, len: 11, flags: DONTWAIT) = 11
 0.788 ( 0.010 ms): mysqld/14097 sendto(fd: 37<socket:[833323]>, buff:
0x7fbdac072040, len: 11, flags: DONTWAIT) = 11
[...]
```

В этом выводе можно видеть две операции отправки данных по 12 664 байта, за которыми следуют две операции отправки по 11 байт. Все операции выполняются с флагом `DONTWAIT`. Если бы я увидел поток небольших посылов, то мог бы спросить: можно ли улучшить производительность, объединив их или избегая флага `DONTWAIT`?

Несмотря на то что `perf(1) trace` можно использовать для профилирования ввода/вывода, мне часто хочется углубиться в аргументы и обобщить их по-своему. Например, результаты трассировки `sendto(2)` выше сообщают дескриптор файла (37) и номер сокета (833323), но я предпочел бы видеть тип сокета, IP-адреса и порты. Для такой расширенной трассировки можно использовать `bpfttrace` (см. раздел 5.5.7 «`bpfttrace`»).

5.5.2. profile

`profile(8)`¹ — это профилировщик процессора для ВСС (глава 15), который выбирает трассировки стека с заданным интервалом. Он использует BPF для уменьшения оверхеда, обобщая трассировки в контексте ядра и передавая в пространство пользователя только уникальные стеки и их счетчики.

¹ Немного истории: я разработал версию `profile(8)` для ВСС 15 июля 2016 года, взяв за основу код Саши Гольдштейна (Sasha Goldshtein), Эндрю Бирчалла (Andrew Birchall), Евгения Верещагина (Evgeny Vereshchagin) и Тена Циня (Teng Qin).

Вот пример использования `profile(8)` для профилирования всех процессоров с частотой 49 Гц в течение 10 с:

```
# profile -F 49 10
```

```
Sampling at 49 Hertz of all threads by user + kernel stack for 10 secs.
```

```
[...]
```

```
SELECT_LEX::prepare(THD*)
Sql_cmd_select::prepare_inner(THD*)
Sql_cmd_dml::prepare(THD*)
Sql_cmd_dml::execute(THD*)
mysql_execute_command(THD*, bool)
Prepared_statement::execute(String*, bool)
Prepared_statement::execute_loop(String*, bool)
mysqld_stmt_execute(THD*, Prepared_statement*, bool, unsigned long, PS_PARAM*)
dispatch_command(THD*, COM_DATA const*, enum_server_command)
do_command(THD*)
[unknown]
[unknown]
start_thread
-                               mysqld (10106)
```

```
13
```

```
[...]
```

Здесь показана только одна трассировка стека, из которой видно, что в период профилирования этот путь к функции `SELECT_LEX::prepare()` был выполнен 13 раз.

О `profile(8)` рассказано в главе 6 «Процессоры», в разделе 6.6.14 «`profile`» — там перечислены различные параметры и представлены инструкции по созданию флейм-графиков процессорного времени на основе результатов профилирования.

5.5.3. `offcputime`

`offcputime(8)`¹ — это инструмент для BCC и `bpfttrace` (глава 15), подсчитывающий время, потраченное потоками в ожидании на блокировках и в очереди на выполнение, и отображающий соответствующие трассировки стека. Он позволяет проанализировать время вне процессора (раздел 5.4.2 «Анализ времени ожидания вне процессора»). `offcputime(8)` — это аналог `profile(8)`: вместе они показывают все время, потраченное потоками в системе.

Вот пример трассировки в течение 5 с с использованием `offcputime(8)` для BCC:

```
# offcputime 5
```

```
Tracing off-CPU time (us) of all threads by user + kernel stack for 5 secs.
```

```
[...]
```

```
finish_task_switch
schedule
```

¹ Немного истории: я разработал методологию анализа времени вне процессора и инструменты, реализующие ее, в 2005 году. Версию `offcputime(8)` для BCC я написал 13 января 2016 года.

```

jbd2_log_wait_commit
jbd2_complete_transaction
ext4_sync_file
vfs_fsync_range
do_fsync
__x64_sys_fdatasync
do_syscall_64
entry_SYSCALL_64_after_hwframe
fdatasync
IO_CACHE_ostream::sync()
MYSQL_BIN_LOG::sync_binlog_file(bool)
MYSQL_BIN_LOG::ordered_commit(THD*, bool, bool)
MYSQL_BIN_LOG::commit(THD*, bool)
ha_commit_trans(THD*, bool, bool)
trans_commit(THD*, bool)
mysql_execute_command(THD*, bool)
Prepared_statement::execute(String*, bool)
Prepared_statement::execute_loop(String*, bool)
mysqld_stmt_execute(THD*, Prepared_statement*, bool, unsigned long, PS_PARAM*)
dispatch_command(THD*, COM_DATA const*, enum_server_command)
do_command(THD*)
[unknown]
[unknown]
start_thread
-      mysqld (10441)
352107
[...]
```

Инструмент выводит уникальные трассировки стека и время вне процессора в микросекундах, приходящееся на них. Этот пример демонстрирует операцию синхронизации в файловой системе ext4, путь к которой пролегает через `MYSQL_BIN_LOG::sync_binlog_file()`. Всего на эту операцию за время, пока делалась трассировка, было потрачено 352 мс.

Для большей эффективности `offcputime(8)` обобщает трассировки стека в контексте ядра и передает в пространство пользователя только уникальные стеки. Кроме того, `offcputime(8)` фиксирует трассировки стека, только если продолжительность времени вне процессора превышает пороговое значение, по умолчанию равное 1 мкс, которое можно настроить с помощью параметра `-m`.

Также инструмент поддерживает параметр `-M` для установки максимального времени трассировки стеков. Почему может понадобиться исключить из анализа стеки, соответствующие длительному ожиданию? Например, чтобы отфильтровать неинтересные стеки: потоки, ожидающие работы и пребывающие в заблокированном состоянии одну или несколько секунд в цикле. Попробуйте использовать `-M 900000`, чтобы исключить из вывода стеки с продолжительностью ожидания больше 900 мс.

Флейм-графики времени вне процессора

Несмотря на то что инструмент выводит только уникальные стеки, полный вывод в предыдущем примере содержит более 200 000 строк. Для анализа такого объема информации можно использовать флейм-график времени вне процессора. Пример

такого графика показан на рис. 5.4. Создается он так же, как и флейм-график по результатам profile (8):

```
# git clone https://github.com/brendangregg/FlameGraph; cd FlameGraph
# offcputime -f 5 | ./flamegraph.pl --bgcolors=blue \
  --title="Off-CPU Time Flame Graph"> out.svg
```

На этот раз я выбрал синий фон для визуального напоминания, что это флейм-график времени вне процессора, а не обычный график процессорного времени.

5.5.4. strace

Команда `strace(1)` — это трассировщик системных вызовов в Linux¹. Она может трассировать системные вызовы и выводить для каждого однострочную сводку, а также подсчитывать обращения к системным вызовам и печатать отчет.

Вот пример трассировки обращений к системным вызовам из процесса с PID 1884:

```
$ strace -ttt -T -p 1884
1356982510.395542 close(3)          = 0 <0.000267>
1356982510.396064 close(4)          = 0 <0.000293>
1356982510.396617 ioctl(255, TIOCGPRG, [1975]) = 0 <0.000019>
1356982510.396980 rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0 <0.000024>
1356982510.397288 rt_sigprocmask(SIG_BLOCK, [CHLD], [], 8) = 0 <0.000014>
1356982510.397365 wait4(-1, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], WSTOPPED|
WCONTINUED, NULL) = 1975 <0.018187>
[...]
```

В этом вызове использованы следующие параметры (полный список параметров вы найдете на странице `strace(1)` в справочном руководстве `man`):

- **-ttt**: выводит первый столбец с временем, прошедшим от начала эпохи, в секундах с точностью до микросекунд;
- **-T**: выводит в последнем поле (*<time>*) продолжительность выполнения системного вызова в секундах с точностью до микросекунд;
- **-p PID**: определяет процесс для трассировки. В `strace(1)` также можно указать команду для запуска трассируемого процесса.

В числе других параметров, не использованных здесь, можно назвать: **-f** — для трассировки дочерних потоков и **-o filename** — для записи вывода в файл с указанным именем.

В выводе можно заметить одну из замечательных особенностей `strace(1)` — преобразование аргументов системного вызова в удобочитаемую форму. Это особенно полезно при исследовании вызовов `ioctl(2)`.

¹ В числе трассировщиков системных вызовов для других ОС можно назвать: `ktrace(1)` для BSD, `truss(1)` для Solaris, `dtruss(1)` для OS X (первая версия этого инструмента была разработана мной) и `logger.exe` и `ProcMon` для Windows.

Для обобщения активности системных вызовов можно использовать параметр `-с`. В следующем примере показан запуск и трассировка команды `dd(1)` вместо подключения к уже запущенному процессу с указанным PID:

```
$ strace -c dd if=/dev/zero of=/dev/null bs=1k count=5000k
512000+0 records in
512000+0 records out
524288000 bytes (5.2 GB) copied, 140.722 s, 37.3 MB/s
% time      seconds  usecs/call   calls   errors syscall
-----
 51.46      0.008030          0   5120005         read
 48.54      0.007574          0   5120003         write
  0.00      0.000000          0         20        13 open
[...]
-----
100.00      0.015604          0  10240092         19 total
```

Вывод начинается с трех строк, которые выводит команда `dd(1)`, а за ними следует сводка `strace(1)`. Столбцы:

- **time**: процентное соотношение, показывающее, на что было потрачено процессорное время в режиме ядра;
- **seconds**: общее процессорное время в режиме ядра в секундах;
- **usecs/call**: среднее процессорное время на вызов в режиме ядра в микросекундах;
- **calls**: количество обращений к системным вызовам;
- **syscall**: имя системного вызова.

Оверхед strace

ВНИМАНИЕ: текущая версия `strace(1)` выполняет трассировку с использованием точек останова через интерфейс `ptrace(2)`. Она устанавливает точки останова на входе и выходе из всех системных вызовов (даже если использован параметр `-e`, определяющий конкретные системные вызовы). Такое поведение влечет значительный оверхед, поэтому производительность приложений, часто обращающихся к системным вызовам, может уменьшиться на порядок. Вот пример с той же командой `dd(1)`, запущенной без `strace(1)`:

```
$ dd if=/dev/zero of=/dev/null bs=1k count=5000k
512000+0 records in
512000+0 records out
524288000 bytes (5.2 GB) copied, 1.91247 s, 2.7 GB/s
```

`dd(1)` выводит в последней строке подсчитанную величину пропускной способности: сравнивая эти значения, можно заключить, что из-за `strace(1)` производительность `dd(1)` ухудшилась в 73 раза. Это особенно серьезный случай, потому что `dd(1)` очень часто обращается к системным вызовам.

В зависимости от требований приложения такой способ трассировки приемлем только в течение короткого времени и только для определения типов используемых системных вызовов. Трассировщик `strace(1)` был бы более полезен, если бы не страдал проблемой высокого оверхеда. Другие трассировщики, в том числе `perf(1)`, `Ftrace`, `BCC` и `bpftrace`, имеют значительно меньший оверхед на трассировку благодаря буферизации результатов, когда события записываются в общий кольцевой буфер в пространстве ядра, а трассировщик уровня пользователя периодически читает этот буфер. Это уменьшает частоту переключений между контекстом пользователя и ядра, снижая оверхед.

В будущей версии `strace(1)` проблема оверхеда может быть решена, если `strace` превратится в псевдоним для подкоманды `trace` в `perf(1)` (описанной выше в разделе 5.5.1 «`perf`»). В число других высокопроизводительных трассировщиков системных вызовов для Linux, основанных на BPF, входят `vttrace` от Intel [Intel 18] и версия `ProcMon` для Linux от Microsoft [Microsoft 20].

5.5.5. `execsnoop`

`execsnoop(8)`¹ — это инструмент для `BCC` и `bpftrace`, трассирующий запуск новых процессов в масштабе системы. Он помогает обнаруживать проблемы, связанные с короткоживущими процессами, потребляющими вычислительные ресурсы, а также может использоваться для отладки выполнения ПО, включая сценарии запуска приложений.

Вот пример вывода, полученный с помощью версии для `BCC`:

```
# execsnoop
PCOMM          PID    PPID   RET   ARGS
oltp_read_write 13044 18184   0    /usr/share/sysbench/oltp_read_write.lua --db-driver=mysql --mysql-password=... --table-size=100000 run
oltp_read_write 13047 18184   0    /usr/share/sysbench/oltp_read_write.lua --db-driver=mysql --mysql-password=... --table-size=100000 run
sh              13050 13049   0    /bin/sh -c command -v debian-sa1 > /dev/null &&
debian-sa1 1 1 -S XALL
debian-sa1      13051 13050   0    /usr/lib/sysstat/debian-sa1 1 1 -S XALL
sa1             13051 13050   0    /usr/lib/sysstat/sa1 1 1 -S XALL
sadc            13051 13050   0    /usr/lib/sysstat/sadc -F -L -S DISK 1 1 -S XALL
/var/log/sysstat
[...]
```

Я запустил эту команду в своей системе баз данных в надежде обнаружить что-нибудь интересное, и это удалось: первые две строки показывают, что микробенчмаркинг чтения/записи все еще работает. Запуская команды `oltp_read_write` в цикле, я случайно оставил его работающим на несколько дней! Поскольку база

¹ Немного истории: первую версию `execsnoop` я написал 24 марта 2004 года. Версию для `BCC` я написал 7 февраля 2016 года, а версию для `bpftrace` — 15 ноября 2017 года. Подробности см. в [Gregg 19].

данных обслуживает другую рабочую нагрузку, мой огрех трудно было заметить по другим системным метрикам, показывающим потребление процессора и диска. Строки после `oltp_read_write` показывают, что в системе работает `sar(1)`, собирающий системные метрики.

`hexesnoop(8)` трассирует обращения к системному вызову `execve(2)` и выводит однострочную сводку для каждого уникального вызова. Инструмент поддерживает несколько параметров, в том числе `-t`, включающий вывод отметок времени.

В главе 1 показан еще один пример использования `hexesnoop(8)`. Я также опубликовал инструмент `threadnoop(8)` для `bpfttrace`, позволяющий трассировать создание потоков через вызов функции `pthread_create()` из библиотеки `libpthread`.

5.5.6. syscount

`syscount(8)`¹ — это инструмент для BCC и `bpfttrace`, подсчитывающий количество системных вызовов во всей системе.

Вот пример вывода, полученный версией инструмента для BCC:

```
# syscount
Tracing syscalls, printing top 10... Ctrl+C to quit.
^C[05:01:28]
SYSCALL          COUNT
recvfrom         114746
sendto           57395
ppoll            28654
futex            953
io_getevents     55
bpf              33
rt_sigprocmask   12
epoll_wait       11
select           7
nanosleep        6
Detaching...
```

Как показывает этот вывод, чаще других осуществлялся системный вызов `recvfrom(2)`: за время трассировки к нему было выполнено 114 746 обращений. При желании можно было бы продолжить исследование и с помощью других инструментов трассировки определить аргументы системного вызова, величину задержки и трассировки стека. Например, можно использовать `perf(1) trace` с фильтром `-e recvfrom` или с помощью `bpfttrace` инструментировать точку трассировки `sys_enter_recvfrom`. Описание этих трассировщиков ищите в главах 13–15.

`syscount(8)` с параметром `-P` может также подсчитывать обращения к системным вызовам по процессам:

¹ Немного истории: первую версию, основанную на `Ftrace` и `perf(1)`, я написал для коллекции `perf-tools` 7 июля 2014 года, а 15 февраля 2017 года Саша Гольдштейн выпустил версию для BCC.

```
# syscount -P
Tracing syscalls, printing top 10... Ctrl+C to quit.
^C[05:03:49]
PID    COMM                COUNT
10106  mysqld                155463
13202  oltp_read_only.      61779
9618   sshd                  36
344    multipathd           13
13204  syscount-bpfcc       12
519    accounts-daemon      5
```

В этом выводе перечислены процессы и количество обращений к системным вызовам для каждого из них.

5.5.7. bpftrace

bpftrace — это трассировщик, основанный на BPF, который поддерживает высокоуровневый язык программирования, позволяющий создавать мощные и короткие сценарии. Хорошо подходит для анализа приложений на основе подсказок, полученных с помощью других инструментов.

Подробнее о bpftrace рассказывается в главе 15. В этом разделе я приведу лишь несколько примеров анализа приложений.

Трассировка сигналов

Следующая однострочная программа на языке bpftrace трассирует сигналы, посылаемые процессом (через системный вызов `kill(2)`), и выводит PID и имя процесса-отправителя, а также PID процесса-получателя и номер сигнала:

```
# bpftrace -e 't:syscalls:sys_enter_kill { time("%H:%M:%S ");
    printf("%s (PID %d) send a SIG %d to PID %d\n",
        comm, pid, args->sig, args->pid); }'
Attaching 1 probe...
09:07:59 bash (PID 9723) send a SIG 2 to PID 9723
09:08:00 systemd-journal (PID 214) send a SIG 0 to PID 501
09:08:00 systemd-journal (PID 214) send a SIG 0 to PID 550
09:08:00 systemd-journal (PID 214) send a SIG 0 to PID 392
...
```

Как показывает вывод, командная оболочка `bash` отправляет себе сигнал 2 (`Ctrl+C`), а затем `systemd-journal` отправляет сигнал 0 некоторым другим процессам. Сигнал 0 ничего не делает: обычно он используется для проверки существования другого процесса по значению, возвращаемому системным вызовом.

Этот однострочный сценарий пригодится для отладки непонятных проблем в приложениях, например преждевременного завершения работы. Отметки времени включены в вывод для перекрестной проверки проблем с производительностью в ПО мониторинга. Трассировку сигналов также можно выполнить с помощью автономного инструмента `killsnoop(8)` в BCC и `bpftrace`.

[8K, 16K)	0			
[16K, 32K)	0			
[32K, 64K)	1			

Здесь показан путь в коде держателя блокировки.

Если у блокировки есть символическое имя, оно будет выведено вместо адреса. Если символического имени нет, блокировку можно идентифицировать по трассировке стека: в данном случае блокировка приобретается в `THD::set_query()` со смещением 147. Заглянув в исходный код этой функции, можно увидеть, что она приобретает только одну блокировку: `LOCK_thd_query`.

Трассировка блокировок увеличивает оверхед, а события приобретения блокировок могут следовать с большой частотой. См. описание оверхеда, связанного с `uprobes` в главе 4 «Инструменты наблюдения» в разделе 4.3.7 «`uprobes`». Возможно, когда-нибудь разработают аналогичные инструменты на основе `kprobes` для функций ядра, обслуживающих фьютексы (`futex` — быстрые мьютексы), которые помогут немного уменьшить оверхед. Альтернативное решение с незначительным оверхедом — использовать профилирование процессорного времени. Оверхед на профилирование процессорного времени обычно невелик из-за ограниченной частоты отбора трассировок стека, а ожидание на тяжелой блокировке может продолжаться достаточно долго, чтобы отобразиться в профиле.

Внутренние особенности приложения

При необходимости можно создать свои инструменты, учитывающие внутренние особенности приложения. Для начала проверьте доступность точек трассировки `USDT` или возможность включить их (обычно путем компиляции с некоторыми параметрами). Если таких точек трассировки нет или их недостаточно, подумайте об использовании зондов `uprobes`. Примеры использования `uprobes` и `USDT` из `bpfttrace` см. в главе 4 «Инструменты наблюдения», в разделах 4.3.7 «`uprobes`» и 4.3.8 «`USDT`». В разделе 4.3.8 также описывается динамический механизм `USDT`, который пригодится для исследования динамически компилируемого (JIT-компиляция) ПО, которое невозможно исследовать с помощью `uprobes`.

Примером сложного для исследования ПО может служить код на Java: с помощью зондов `uprobes` можно инструментировать среду выполнения JVM (код на C++) и библиотеки ОС. Статические точки трассировки `USDT` позволяют инструментировать высокоуровневые события JVM, а динамические точки трассировки `USDT` могут внедряться в код на Java для исследования особенностей выполнения некоторого метода.

5.6. ПРОБЛЕМЫ

В следующих разделах описаны общие проблемы, встречающиеся при анализе производительности приложений, в частности отсутствующие символы и трассировки

стека. Впервые с этим можно столкнуться при изучении профиля потребления процессора, обнаружив, что в флейм-графике нет имен функций и трассировки стека.

Преодоление этих проблем — сложная тема, которую я подробно освещаю в главах 2, 12 и 18 в книге «BPF Performance Tools», а коротко — в этом разделе.

5.6.1. Отсутствие символов

Когда профилировщик или трассировщик не может отыскать соответствия между адресом инструкции в приложении с именем функции (символом), то вместо имени он может вывести шестнадцатеричное число или строку «[unknown]». Решение этой проблемы зависит от компилятора, среды выполнения и настроек приложения, а также от самого профилировщика.

Двоичные файлы ELF (C, C++, ...)

Отсутствие символов — характерная проблема для скомпилированных двоичных файлов, особенно для упакованных и подготовленных к распространению с помощью `strip(1)` с целью уменьшения их размеров. Одно из решений — настроить процесс сборки, чтобы исключить удаление символов. Другое решение — использовать иной источник информации о символах, например пакеты с отладочной информацией или файлы в формате BPF Type Format (BTF). Профилирование в Linux с помощью `perf(1)`, BCC и `bpfttrace` поддерживает использование отладочной информации.

Среды выполнения с JIT-компиляцией (Java, Node.js, ...)

Отсутствие символов обычно характерно для сред с динамической (JIT) компиляцией — Java и Node.js. В этих случаях JIT-компилятор создает свою таблицу символов, которая изменяется во время выполнения и не является частью предварительно скомпилированных таблиц символов в двоичном файле. Обычно эта проблема решается использованием дополнительных таблиц символов, созданных средой выполнения, которые помещаются в файлы `/tmp/perf-<PID>.map` и используются как `perf(1)`, так и BCC.

Например, в Netflix используется `perf-map-agent` [Rudolph 18], который может подключаться к действующему процессу Java и выгружать файлы с дополнительными символами. Я автоматизировал его применение с помощью другого инструмента под названием `jmaps` [Gregg 20c], который следует запускать сразу после профилирования и перед трансляцией символов. Вот примеры с использованием `perf(1)` (глава 13):

```
# perf record -F 49 -a -g -- sleep 10; jmaps
# perf script --header > out.stacks
# [...]
```

и bpftrace (глава 15):

```
# bpftrace --unsafe -e 'profile:hz:49 { @[ustack] = count(); }
    interval:s:10 { exit(); } END { system("jmaps"); }'
```

Соответствие символов может измениться в период между созданием профиля и получением таблицы символов, что может привести к появлению ошибочных имен функций в профиле. Это называется *устареванием символов* (symbol churn), и запуск jmaps сразу после perf record уменьшает его вероятность. Пока эта проблема не вставала остро, но если такое случится, можно попробовать получить дамп символов до и после профилирования и выявить изменения.

Есть и другие подходы к разрешению JIT-символов. Один из них — фиксировать отметки времени символов. Эта возможность поддерживается в perf(1) и решает проблему устаревания символов, хотя и за счет более высокого оверхеда. Другой подход, предназначенный для perf(1), заключается в использовании своего средства обхода стека среды выполнения (который обычно используется для обхода стека при появлении исключений). Эти средства иногда называют *вспомогательными обходчиками стека*, и в Java этот подход реализован с помощью async-profiler [Pangin 20].

Обратите внимание, что любая среда выполнения с JIT-компиляцией также может иметь предварительно скомпилированные компоненты: JVM, например, использует библиотеки libjvm и libc. Подробнее о проблемах с двоичными файлами ELF см. в предыдущем разделе.

5.6.2. Отсутствие стеков

Еще одна распространенная проблема — отсутствие или неполная трассировка стека, включающая всего пару фреймов. Вот пример профилирования времени вне процессора для сервера MySQL:

```
finish_task_switch
schedule
futex_wait_queue_me
futex_wait
do_futex
__x64_sys_futex
do_syscall_64
entry_SYSCALL_64_after_hwframe
pthread_cond_timedwait@@GLIBC_2.3.2
[unknown]
```

Этот стек неполный: за pthread_cond_timedwait() следует единственный фрейм «[unknown]». В этом стеке нет функций MySQL, которые действительно нужны для понимания контекста приложения.

Иногда трассировка стека может быть представлена единственным фреймом:

```
send
```

На флейм-графиках такие стеки могут выглядеть как «трава»: множество тонких одиночных фреймов вдоль нижнего края профиля.

Неполные трассировки стека, к сожалению, распространены и обычно обусловлены двумя факторами: 1) применение инструмента наблюдения, который для чтения трассировок стека использует подход на основе указателя фрейма, и 2) целевой двоичный файл не резервирует регистр (RBP на x86_64) для хранения указателя фрейма — вместо этого компилятор использует его как регистр общего назначения для оптимизации производительности. Инструмент наблюдения читает этот регистр, ожидая найти в нем указатель фрейма, но в действительности в нем может находиться все что угодно: числа, адреса объектов, указатели на строки и т. д. Инструмент наблюдения пытается разрешить это число по таблице символов, и если повезет, не найдет соответствия и выведет «[unknown]». Но если не повезет, то это случайное число будет разрешено в совершенно посторонний символ, и в трассировку стека добавится ошибочное имя функции, сбивающее с толку вас, конечного пользователя.

Библиотека `libc` обычно компилируется без указателей фреймов, поэтому отсутствие стеков — обычное дело для любых путей в коде, пролегающих через `libc`, включая два предыдущих примера: `pthread_cond_timedwait()` и `send()`¹.

Самое простое решение этой проблемы — включить поддержку указателей фреймов:

- **ПО на C/C++** и других языках, компилируемых с помощью `gcc(1)` или `LLVM`, можно скомпилировать с флагом `-fno-omit-frame-pointer`;
- **ПО на Java** можно запустить командой `java(1)` с флагом `-XX:+PreserveFramePointer`.

Это может привести к снижению производительности, но часто такое снижение оказывается меньше 1 %. Преимущества трассировки стека для поиска выигрышей в производительности обычно намного перевешивают этот недостаток.

Другой подход — использовать метод обхода стека, не основанный на указателях фрейма. `perf(1)` поддерживает обход стека на основе DWARF, ORC и записи последней ветви (last branch record, LBR). Другие методы обхода стека упоминаются в главе 13 «perf» в разделе 13.9 «perf record».

На момент написания этих строк обход стека на основе DWARF и LBR не поддерживался в BPF, а ORC пока был недоступен для ПО пользовательского уровня.

5.7. УПРАЖНЕНИЯ

1. Ответьте на вопросы, касающиеся терминологии:

- Что такое кэш?
- Что такое кольцевой буфер?

¹ Лично я отправлял инструкции по сборке мейнтейнерам пакетов с просьбой собрать пакет `libc` с указателями фреймов.

- Что такое спин-блокировка?
 - Что такое адаптивный мьютекс?
 - Чем отличаются конкурентность и параллелизм?
 - Что такое привязка к процессору?
2. Ответьте на следующие концептуальные вопросы:
- Каковы основные достоинства и недостатки ввода/вывода блоками большого размера?
 - Для чего используется хеш-таблица блокировок?
 - Опишите общие характеристики производительности среды выполнения компилируемых и интерпретируемых языков и виртуальных машин.
 - Объясните роль сборки мусора и то, как она может повлиять на производительность.
3. Выберите приложение и ответьте на следующие основные вопросы о нем:
- Назначение приложения.
 - Какую дискретную операцию выполняет приложение?
 - В каком режиме работает приложение, пользователя или ядра?
 - Какие настройки есть у приложения? Какие ключевые параметры из доступных связаны с производительностью?
 - Какие метрики производительности предоставляет приложение?
 - Какие журналы создает приложение? Содержат ли они информацию о производительности?
 - Устранены ли проблемы с производительностью в самой последней версии приложения?
 - Есть ли в приложении известные ошибки, связанные с производительностью?
 - Есть ли у приложения сообщество (например, каналы IRC, чаты, форумы)? Занимается ли сообщество проблемами производительности?
 - Есть ли книги, посвященные приложению? Его производительности?
 - Есть ли известные эксперты по проблемам производительности приложения? Кто они?
4. Выберите приложение, работающее под нагрузкой, и выполните следующие задачи (многие из них могут потребовать использовать динамическую трассировку):
- Прежде чем проводить какие-либо измерения, выясните, на какие операции приложение тратит основное время — вычислительные или ввода/вывода? Объясните свои рассуждения.
 - Определите с помощью инструментов наблюдения, действительно ли приложение выполняет в основном вычислительные операции или операции ввода/вывода.

- Сгенерируйте флейм-график процессорного времени для приложения. При этом вам может потребоваться решить проблемы с отсутствующими символами и трассировками стека. Какой путь в коде потребляет больше всего процессорного времени?
 - Создайте флейм-график времени вне процессора для приложения. На какой блокировке приложение ожидает дольше всего в процессе обработки запросов (игнорируйте стеки, соответствующие простоям в ожидании работы)?
 - Определите размеры блоков ввода/вывода (например, в операциях чтения/записи с файловой системой или приема/передачи с сетью).
 - Есть ли в приложении кэши? Определите их размеры и процент попаданий.
 - Измерьте задержку (время отклика) для операции, которую обслуживает приложение. Определите среднее, минимальное, максимальное и полное распределение задержек.
 - Выполните анализ операции с детализацией, исследуя причину основной задержки.
 - Охарактеризуйте рабочую нагрузку, приложенную к приложению (в частности, источник и тип нагрузки).
 - Сделайте чек-лист настройки статической производительности.
 - Приложение выполняется конкурентно? Изучите использование примитивов синхронизации.
5. (опционально) Разработайте инструмент для Linux под названием tsastat(8), который выполняет анализ состояния потоков и выводит столбцы с их состояниями и временем пребывания в каждом состоянии. Этот инструмент может работать как pidstat(1) и производить прокатный вывод¹.

5.8. ССЫЛКИ

[Knuth 76] Knuth, D., «Big Omicron and Big Omega and Big Theta», ACM SIGACT News, 1976.

[Knuth 97] Knuth, D., «The Art of Computer Programming, Volume 1: Fundamental Algorithms, 3rd Edition»², Addison-Wesley, 1997.

[Zijlstra 09] Zijlstra, P., «mutex: implement adaptive spinning», <http://lwn.net/Articles/314512>, 2009.

¹ Интересный факт: я не знаю никого, кто решил бы эту задачу, впервые предложенную в первом издании книги. Я планировал выступить с докладом об анализе состояния потока (Thread State Analysis, TSA) на OSCON и запланировал разработку инструмента Linux TSA для выступления. Но заявку отклонили (это моя вина: я оформил заявку кое-как), и поэтому мне еще предстоит разработать этот инструмент. Организаторы EuroBSDcon пригласили меня выступить с основным докладом, в котором я должен был представить TSA, и при подготовке я разработал инструмент tstates.d для FreeBSD [Gregg 17a].

² Кнут Д. «Искусство программирования. Том 1. Основные алгоритмы».

- [Gregg 17a] Gregg, B., «EuroBSDcon: System Performance Analysis Methodologies», EuroBSDcon, <http://www.brendangregg.com/blog/2017-10-28/bsd-performance-analysis-methodologies.html>, 2017.
- [Intel 18] «Tool tracing syscalls in a fast way using eBPF linux kernel feature», <https://github.com/pmem/vltrace>, последнее обновление 2018.
- [Microsoft 18] «Fibers», Windows Dev Center, <https://docs.microsoft.com/en-us/windows/win32/procthread/fibers>, 2018¹.
- [Rudolph 18] Rudolph, J., «perf-map-agent», <https://github.com/jvm-profiling-tools/perf-map-agent>, последнее обновление 2018.
- [Schwartz 18] Schwartz, E., «Dynamic Optimizations for SBCL Garbage Collection», 11th European Lisp Symposium, <https://european-lisp-symposium.org/static/proceedings/2018.pdf>, 2018.
- [Axboe 19] Axboe, J., «Efficient IO with io_uring», https://kernel.dk/io_uring.pdf, 2019.
- [Gregg 19] Gregg, B., «BPF Performance Tools: Linux System and Application Observability»², Addison-Wesley, 2019.
- [Apdex 20] Apdex Alliance, «Apdex», <https://www.apdex.org>, по состоянию на 2020.
- [Golang 20] «Why goroutines instead of threads?» документация для языка Golang, <https://golang.org/doc/faq#goroutines>, по состоянию на 2020.
- [Gregg 20b] Gregg, B., «BPF Performance Tools», <https://github.com/brendangregg/bpf-perf-tools-book>, последнее обновление 2020.
- [Gregg 20c] Gregg, B., «jmaps», <https://github.com/brendangregg/FlameGraph/blob/master/jmaps>, последнее обновление 2020.
- [Linux 20e] «RCU Concepts», документация для Linux, <https://www.kernel.org/doc/html/latest/RCU/rcu.html>, по состоянию на 2020.
- [Microsoft 20] «Procmon Is a Linux Reimagining of the Classic Procmon Tool from the Sysinternals Suite of Tools for Windows», <https://github.com/microsoft/ProcMon-for-Linux>, последнее обновление 2020.
- [Molnar 20] Molnar, I., and Bueso, D., «Generic Mutex Subsystem», документация для Linux, <https://www.kernel.org/doc/Documentation/locking/mutex-design.rst>, по состоянию на 2020.
- [Node.js 20] «Node.js», <http://nodejs.org>, по состоянию на 2020.
- [Pangin 20] Pangin, A., «async-profiler», <https://github.com/jvm-profiling-tools/async-profiler>, последнее обновление 2020.

¹ Перевод на русский язык: <https://docs.microsoft.com/ru-ru/windows/win32/procthread/fibers>.

² Грегг Б. «BPF: Профессиональная оценка производительности». Выходит в издательстве «Питер» в 2023 году.

Глава 6

ПРОЦЕССОРЫ

Центральные процессоры (CPU) управляют всем программным обеспечением, и часто именно они — первая цель для анализа производительности системы. В этой главе я расскажу об особенностях аппаратного и программного обеспечения процессора, а также покажу, как можно детально исследовать потребление процессора для определения направлений улучшения производительности.

На высоком уровне можно исследовать потребление CPU в масштабе всей системы и проанализировать его использование процессами или потоками, на более низком уровне — профилировать и изучать пути в коде приложений и ядра, а также использовать CPU прерываниями. На самом низком уровне можно проанализировать выполнение отдельных инструкций и циклов. Также можно исследовать другие особенности поведения, в том числе задержку планировщика, когда задачи ожидают своей очереди выполнения на процессоре, что снижает производительность.

Цели этой главы:

- познакомить с моделями CPU и основными понятиями;
- познакомить с внутренним устройством CPU;
- познакомить с внутренним устройством планировщика CPU;
- представить различные методики анализа потребления CPU;
- познакомить с приемами интерпретации средней нагрузки и набором метрик PSI, показывающих время ожидания доступности CPU, памяти или ввода/вывода;
- познакомить с приемами определения характера потребления CPU в системе;
- показать, как выявлять проблемы, связанные с задержкой планировщика, и количественно оценивать их;
- продемонстрировать приемы анализа циклов CPU для выявления неэффективности;
- показать приемы анализа потребления CPU с использованием профилировщиков и флейм-графиков;
- показать, как определять потребление CPU программными и аппаратными прерываниями;

- продемонстрировать приемы интерпретации флейм-графиков процессорного времени и других визуализаций;
- познакомить с параметрами настройки процессора.

Глава состоит из шести частей. В первых трех речь идет об основах анализа потребления процессоров, а в следующих трех рассказывается о практическом применении этого анализа в системах на базе Linux. Вот эти части:

- **Основы** представляет терминологию, основные модели и ключевые идеи производительности процессоров.
- **Архитектура** описывает архитектуру процессора и планировщика в ядре.
- **Методология** описывает методологии анализа эффективности, как наблюдательные, так и экспериментальные.
- **Инструменты наблюдения** описывает инструменты анализа производительности процессора в системах на базе Linux, включая профилирование, трассировку и визуализацию.
- **Экспериментирование** обобщает возможности инструментов тестирования производительности процессоров.
- **Настройка** включает примеры настраиваемых параметров.

Мы рассмотрим и влияние чтения/записи в память на производительность процессора, в том числе циклы процессора, расходуемые на ожидание доступности памяти, и производительность кэшей процессора. В главе 7 «Память» продолжается обсуждение чтения/записи в память, включая MMU, NUMA/UMA, взаимосвязи и шины памяти.

6.1. ТЕРМИНОЛОГИЯ

Ниже перечислены основные термины, связанные с CPU и используемые в этой главе.

- **Процессор (CPU):** физическая микросхема, которая вставляется в гнездо на системной или процессорной плате и содержит один или более процессоров, реализованных как ядра или аппаратные потоки.
- **Ядро процессора:** независимый экземпляр процессора в *многоядерном процессоре*. Технология создания многоядерных процессоров обеспечивает возможность масштабирования, которую называют *многопроцессорностью на уровне кристалла* (chip-level multiprocessing, CMP).
- **Аппаратный поток:** процессорная архитектура, поддерживающая параллельное выполнение нескольких потоков на одном ядре (включая технологию Intel Hyper-Threading), где каждый поток является независимым экземпляром процессора. Такой подход к масштабированию называется *одновременной многопоточностью* (simultaneous multithreading, SMT).

- **Инструкция процессора:** отдельная операция из набора команд процессора. Есть инструкции, выполняющие арифметические операции, операции ввода/вывода с памятью и операции управления выполнением.
- **Логический процессор:** также называется *виртуальным процессором*¹, экземпляр процессора операционной системы (объект планирования). Может быть реализован как аппаратный поток (в этом случае его можно назвать *виртуальным ядром*), как ядро или как одноядерный процессор.
- **Планировщик:** подсистема в ядре ОС, которая планирует выполнение потоков на процессорах.
- **Очередь на выполнение:** очередь потоков, готовых к выполнению и ожидающих обслуживания процессорами. Современные ОС могут использовать для хранения потоков, готовых к выполнению, некоторую другую структуру данных (например, красно-черное дерево), но мы по-прежнему используем термин «очередь на выполнение».

Далее будут вводиться и другие термины. В глоссарий для справки включена базовая терминология, в том числе *процессор*, *цикл процессора* и *стек*. См. также разделы «Терминология» в главах 2 и 3.

6.2. МОДЕЛИ

Модели, представленные в этом разделе, показывают основные особенности процессоров и характеристики производительности процессоров. Раздел 6.4 «Архитектура» содержит более подробную информацию и описывает тонкости, зависящие от реализации.

6.2.1. Архитектура процессора

На рис. 6.1 показан пример архитектуры процессора с четырьмя ядрами и восемью аппаратными потоками. Здесь изображена физическая архитектура и то, как ее видит ОС².

Каждый аппаратный поток адресуется как *логический процессор*, поэтому эта микросхема процессора видится ОС как восемь процессоров. Операционная система может использовать некоторые дополнительные сведения о топологии для оптимизации своих решений по планированию, например, какие логические процессоры находятся на одном ядре и как совместно используются кэши процессоров.

¹ Иногда его действительно называют *виртуальным процессором*. Но чаще этот термин используется для обозначения экземпляров виртуальных процессоров, предоставляемых технологией виртуализации. См. главу 11 «Облачные вычисления».

² Для Linux есть инструмент `lstopo(1)`, который генерирует подобные диаграммы для текущей системы. Пример его использования см. в разделе 6.6.21 «Другие инструменты».

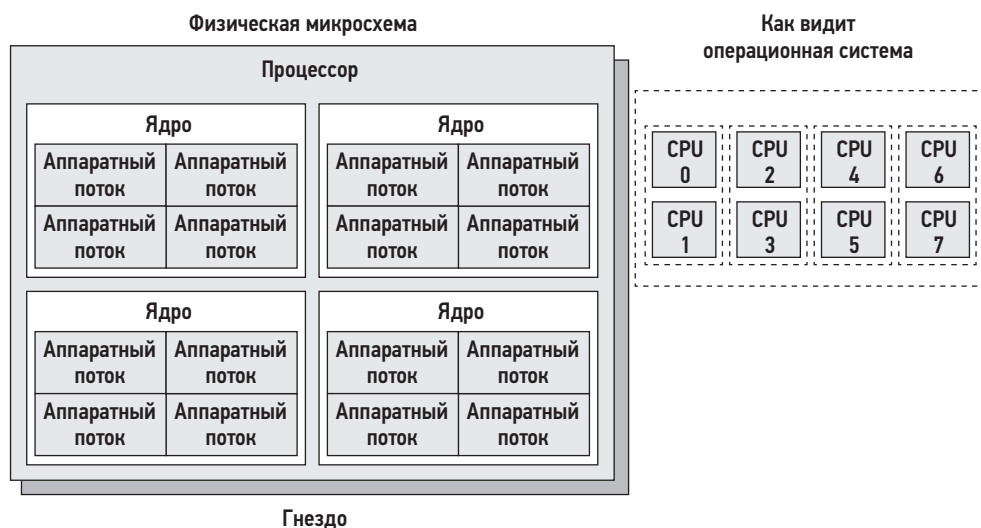


Рис. 6.1. Архитектура процессора

6.2.2. Кэш-память процессора

Процессоры имеют различные аппаратные кэши для повышения производительности операций чтения/записи в память. На рис. 6.2 показано соотношение кэшей разных уровней, размеры которых уменьшаются, а производительность увеличивается при приближении к ядру процессора.

Наличие кэшей, а также их размещение в процессоре (интегрированные кэши) или во внешней памяти зависит от типа процессора. В ранних процессорах было меньше уровней интегрированного кэша.



Рис. 6.2. Размеры кэшей процессора

6.2.3. Очереди на выполнение

На рис. 6.3 показана очередь на выполнение, которой управляет планировщик в ядре ОС.



Рис. 6.3. Очередь на выполнение на процессоре

Состояния потоков на рис. 6.3 «готов к выполнению» и «на процессоре» показаны на рис. 3.8 в главе 3 «Операционные системы».

Количество программных потоков, готовых к выполнению и стоящих в очереди, — это важная метрика производительности, указывающая на насыщение процессора. На рис. 6.3 в очереди находятся четыре потока и еще один поток выполняется на процессоре. Время, проведенное в очереди на выполнение, иногда называют *задержкой в очереди на выполнение* или *задержкой в очереди диспетчера*. В этой книге часто используется термин *задержка планировщика*, так как он подходит для всех планировщиков, в том числе и не использующих очереди (см. часть про CFS в разделе 6.4.2 «Программное обеспечение»).

В многопроцессорных системах ядро ОС обычно поддерживает несколько очередей на выполнение — по числу процессоров — и стремится помещать потоки в одну и ту же очередь. Это означает, что потоки с большей вероятностью будут продолжать выполняться на тех же процессорах, которые хранят данные этих потоков в своих кэшах. Эти кэши называют *разогретыми кэшами*, и такая стратегия, позволяющая поддерживать выполнение потоков на одних и тех же процессорах, называется *привязкой к процессору* (CPU affinity). В системах NUMA наличие отдельной очереди на выполнение для каждого процессора также улучшает *локальность памяти*. Это повышает производительность за счет использования потоками одних и тех же узлов памяти (как описывается в главе 7 «Память») и исключения затрат на синхронизацию потоков (блокировок на мьютексах) при выполнении операций с очередями, которые могли бы ухудшить масштабируемость, если бы очередь на выполнение была одной для всех процессоров.

6.3. ПОНЯТИЯ

Далее описываются некоторые важные понятия, так или иначе связанные с анализом производительности процессора. Сначала будет представлен обзор внутренних особенностей процессора — тактовой частоты и порядка выполнения инструкций. Это необходимый базис для анализа производительности, в частности, для понимания метрики IPC — число инструкций на цикл (instructions per cycle).

6.3.1. Тактовая частота

Такт — это цифровой сигнал, управляющий всей логикой процессора. Для выполнения каждой инструкции процессору требуется один или несколько тактов (их еще называют *циклами процессора*). Процессоры работают с определенной тактовой частотой. Например, процессор с тактовой частотой 4 ГГц выполняет 4 миллиарда циклов в секунду.

Некоторые процессоры могут менять тактовую частоту, увеличивая ее для повышения производительности или уменьшая для снижения энергопотребления. Частота может меняться по запросу ОС или динамически самим процессором. Например, поток ядра, обслуживающий состояние бездействия системы, может запросить у процессора уменьшить мощность для экономии энергии.

Тактовая частота часто позиционируется как основная характеристика процессора, но может вводить в заблуждение. Даже если процессор в вашей системе кажется полностью загруженным (является узким местом), установка процессора с более высокой тактовой частотой может не повысить производительность — многое зависит от того, какую работу в действительности выполняют эти более быстрые циклы. Если они расходуются на ожидание доступа к памяти, более быстрое их выполнение не увеличит скорость выполнения инструкций или пропускную способность рабочей нагрузки.

6.3.2. Инструкции

Процессоры выполняют инструкции из их наборов команд. Обработка каждой инструкции включает следующие шаги, выполняемые компонентом процессора, который называется *функциональным блоком*:

1. Извлечение инструкции.
2. Декодирование инструкции.
3. Выполнение.
4. Доступ к памяти.
5. Запись в регистры.

Последние два шага необязательны и зависят от инструкции. Многие инструкции работают только с регистрами и не требуют шага доступа к памяти.

Для выполнения каждого из этих шагов требуется по крайней мере один тактовый цикл. Доступ к памяти часто является самым медленным шагом, потому что для чтения или записи в оперативную память могут потребоваться десятки тактов, во время которых инструкции не выполняются (такие такты называются *циклами приостановки — stall cycles*). Вот почему кэширование важно для производительности процессоров, как описано в разделе 6.4.1 «Аппаратное обеспечение»: оно может значительно сократить количество циклов, необходимых для доступа к памяти.

6.3.3. Вычислительный конвейер

Вычислительный конвейер — это процессорная архитектура, способная выполнять сразу несколько инструкций, одновременно совершая разные шаги обработки разных инструкций. Это похоже на заводской сборочный конвейер, на котором этапы производства могут выполняться параллельно, увеличивая производительность.

Обратите внимание на шаги, перечисленные выше. Если бы для каждого из них требовался один такт, то для выполнения всей инструкции потребовалось бы пять тактов. На каждом шаге обработки инструкции активен только один функциональный блок, а другие четыре простаивают. При использовании конвейерной обработки несколько функциональных модулей могут действовать одновременно, обрабатывая разные инструкции как на конвейере. В идеале процессор во время каждого такта выполнять одну инструкцию.

Конвейерная обработка инструкций может включать деление инструкции на несколько простых шагов для параллельного выполнения. (В зависимости от процессора эти шаги могут превращаться в простые операции, которые называют *микрооперациями* (uOps) и выполняются в области процессора, называемой *внутренней частью*. *Внешняя часть* такого процессора отвечает за выборку инструкций и прогнозирование ветвлений.)

Предсказатель ветвлений

Современные процессоры могут выполнять конвейерную обработку без сохранения очередности — более поздние инструкции могут быть уже выполнены, пока более ранние инструкции простаивают в ожидании завершения цикла обращения к памяти. Все это улучшает пропускную способность инструкций. Однако инструкции условного перехода создают проблему. Обычные инструкции перехода просто выполняют переход к другой инструкции, а инструкции условных переходов делают это, опираясь на результаты проверки некоторого условия. В случае условных переходов процессор не знает, какие инструкции будут следующими. Для оптимизации многие процессоры реализуют *предсказатель ветвлений*, пытаясь угадать результат проверки и приступая к обработке соответствующих инструкций. Если позже прогноз окажется ошибочным, то выполнение инструкций в конвейере придется прекратить, что ухудшит производительность. Чтобы повысить вероятность правильного прогнозирования, программисты могут размещать подсказки в коде (например, макросы `likely()` и `unlikely()` в исходном коде ядра Linux).

6.3.4. Ширина инструкций

Но инструкции могут обрабатываться еще быстрее. В процессоре бывает несколько функциональных блоков одного типа, благодаря чему на каждом такте может обрабатываться больше инструкций. Эта процессорная архитектура называется *суперскалярной* и обычно используется в сочетании с конвейерной обработкой для достижения высокой пропускной способности команд.

Ширина инструкций описывает целевое количество инструкций, обрабатываемых параллельно. Современные процессоры имеют *ширину* 3 или 4 (3-wide или 4-wide), то есть могут выполнять до трех или четырех инструкций за такт. Конкретная реализация зависит от процессора, так как для каждого шага в конвейере бывает разное количество функциональных блоков.

6.3.5. Размеры инструкций

Другая характеристика инструкций — *размер*. В некоторых процессорных архитектурах разные инструкции имеют разный размер: например, в архитектуре x86, которая классифицируется как *компьютер со сложным набором инструкций* (complex instruction set computer, CISC), имеются инструкции размером до 15 байт. Архитектура ARM, которая классифицируется как *компьютер с сокращенным набором команд* (reduced instruction set computer, RISC), имеет 4-байтные инструкции с 4-байтным выравниванием для AArch32/A32 и 2- или 4-байтные инструкции для ARM Thumb.

6.3.6. SMT

Одновременная многопоточность использует суперскалярную архитектуру и аппаратную поддержку многопоточности (процессором) для улучшения параллелизма. Это позволяет ядру процессора выполнять более одного потока, эффективно распределяя выполнение инструкций между ними, например, когда одна инструкция приостанавливается в ожидании завершения доступа к памяти. Ядро ОС представляет эти аппаратные потоки как виртуальные процессоры и планирует выполнение программных потоков и процессов на них как обычно. Об этом шла речь в разделе 6.2.1 «Архитектура процессора».

Пример реализации — это технология Intel Hyper-Threading, в которой каждое процессорное ядро имеет два аппаратных потока. Другой пример — технология POWER8, в которой на одно процессорное ядро приходится восемь аппаратных потоков.

Производительность каждого аппаратного потока отличается от производительности отдельного процессорного ядра и зависит от рабочей нагрузки. Чтобы избежать проблем с производительностью, ядро ОС может распределять нагрузку между процессорными ядрами так, чтобы на каждом ядре был занят только один аппаратный поток. Рабочие нагрузки с большим количеством циклов приостановки (низкое значение IPC) могут иметь лучшую производительность, чем нагрузки с большим количеством инструкций, выполняемых за такт (высокое значение IPC), потому что циклы приостановки уменьшают конкуренцию за процессорное ядро.

6.3.7. IPC, CPI

Число инструкций на цикл (instructions per cycle, IPC) — важная высокоуровневая метрика, описывающая, как процессор расходует свои тактовые циклы, и помогающая понять природу использования процессора. Эту метрику также можно

выразить как *число циклов на инструкцию* (cycles per instruction, CPI), обратную метрике IPC. IPC чаще используется сообществом Linux и профилировщиком perf(1), а CPI чаще используется Intel и другими организациями¹.

Низкое значение IPC указывает на то, что процессоры часто приостанавливаются, обычно для доступа к памяти. Высокое значение IPC указывает на то, что процессоры редко приостанавливаются и имеют высокую пропускную способность. Эти метрики подсказывают, на чем сосредоточиться при настройке производительности.

Например, производительность рабочих нагрузок, интенсивно использующих память, можно улучшить, установив более быструю память (DRAM), повысив локальность памяти (настройкой программного обеспечения) или уменьшив количество операций чтения/записи в память. Установка процессоров с более высокой тактовой частотой может не дать улучшения производительности до ожидаемой степени, потому что процессорам может потребоваться такое же количество времени для завершения операции чтения/записи в память. Иначе говоря, установив более быстрый процессор, можно получить большее количество циклов простоя и то же количество инструкций, выполняемых в секунду.

Фактические величины высокого или низкого значения IPC зависят от процессора и его характеристик, и их можно определить экспериментально, запустив рабочие нагрузки с известными характеристиками. Например, вы можете обнаружить, что рабочие нагрузки с низким значением IPC характеризуются величиной IPC на уровне 0,2 или ниже, а рабочие нагрузки с высоким значением IPC характеризуются величиной IPC выше 1,0 (что вполне возможно при конвейерной обработке и ширине команд больше 1, как было описано выше). В Netflix облачные рабочие нагрузки характеризуются значениями IPC от 0,2 (низкая производительность) до 1,5 (хорошая производительность). При выражении в виде CPI этот диапазон охватывает значения от 5,0 до 0,66.

Следует отметить, что IPC показывает эффективность *обработки* инструкций, но не самих инструкций. Представьте изменение в ПО, в результате которого добавился неэффективный программный цикл, который работает в основном с регистрами процессора (без тактов приостановки): такое изменение может привести к увеличению значения IPC, но также и к более высокому потреблению CPU.

6.3.8. Потребление

Потребление процессора измеряется временем, когда экземпляр CPU занят выполнением работы в течение определенного интервала времени, и выражается в процентах. Его можно измерить как время, в течение которого процессор выполнял не поток обслуживания бездействия системы, а потоки приложений пользовательского уровня и уровня ядра или обрабатывал прерывания.

¹ В первом издании я использовал метрику CPI, но впоследствии стал заниматься в основном ОС Linux и переключился на использование IPC.

Высокое потребление процессора не всегда проблема, скорее это признак того, что система выполняет какую-то работу. Некоторые также считают это показателем возврата инвестиций (return of investment, ROI): высокое потребление процессора рассматривается как признак высокой окупаемости системы, а простаивающая система — как пустая трата денег. В отличие от других типов ресурсов (дисков), производительность при высоком потреблении падает постепенно, потому что ядро операционной системы поддерживает приоритеты, вытеснение и разделение времени. Вместе все эти приемы позволяют ядру ОС определить более приоритетные задачи и гарантировать их преимущественное выполнение.

Метрика потребления процессора охватывает все тактовые циклы, затраченные на выполнение работы, включая циклы приостановки в ожидании доступа к памяти. Это может вводить в заблуждение: потребление процессора может выглядеть высоким, но не из-за большого количества выполняемых инструкций, а из-за частых приостановок в ожидании доступа к памяти, как было описано в предыдущем разделе. Именно так обстоит дело с облаком Netflix, где основное потребление процессора связано с циклами приостановки в ожидании доступа к памяти [Gregg 17b]. Потребление процессора часто делят на отдельные метрики: время выполнения в режиме ядра и в режиме пользователя.

6.3.9. Время в режиме пользователя/ядра

Процессорное время, потраченное на выполнение ПО уровня пользователя, называется *временем выполнения в режиме пользователя*, или просто *временем пользователя*, а время, потраченное на выполнение ПО уровня ядра, — *временем выполнения в режиме ядра*, или просто *временем ядра*. Время ядра включает время выполнения системных вызовов, потоков ядра и прерываний. При измерениях для всей системы в целом соотношение пользовательского времени и времени ядра помогает определить тип рабочей нагрузки.

Приложения, выполняющие интенсивные вычисления, почти все свое время могут тратить на выполнение кода уровня пользователя и иметь соотношение времени пользователя и ядра, близкое к 99/1. Примерами могут служить обработка изображений, машинное обучение, геномика и анализ данных.

Приложения, выполняющие интенсивный ввод/вывод, часто обращаются к системным вызовам, которые выполняют код ядра для ввода/вывода. Например, веб-сервер, производящий сетевой ввод/вывод, может иметь соотношение времени пользователя и ядра около 70/30.

Эти числа зависят от многих факторов и представлены здесь для примера возможных ожидаемых соотношений.

6.3.10. Насыщенность

При потреблении процессора на уровне 100 % наступает *насыщение*, и потоки начинают сталкиваться с *задержкой планировщика*, потому что часть времени

тратится на ожидание выполнения, что снижает общую производительность. Эта задержка — время, проведенное в очереди на выполнение или в другой структуре, используемой для управления потоками.

Другая форма насыщения процессора связана с управлением ресурсами CPU, которое может иметь место в многопользовательской облачной среде. Процессор может быть использован не на 100 %, но из-за достижения наложенного ограничения выполняемые потоки вынуждены будут ждать своей очереди. Насколько это видно пользователям системы, зависит от типа виртуализации (см. главу 11 «Облачные вычисления»).

Процессор, работающий в режиме насыщения, представляет меньшую проблему, чем другие типы ресурсов, потому что задание с более высоким приоритетом может вытеснить текущий поток.

6.3.11. Вытеснение

Вытеснение, как рассказывалось в главе 3 «Операционные системы», позволяет потоку с более высоким приоритетом вытеснить текущий поток и продолжить выполнение на процессоре вместо него. Эта возможность избавляет высокоприоритетные потоки от задержки в очереди на выполнение и увеличивает их производительность.

6.3.12. Инверсия приоритета

Инверсия приоритета происходит, когда поток с более низким приоритетом удерживает ресурс и блокирует выполнение потока с более высоким приоритетом. Это снижает производительность потоков с высоким приоритетом, потому что они блокируются в ожидании освобождения ресурса.

Решить эту проблему можно с помощью схемы *наследования приоритетов*. Вот пример, как это можно реализовать (на основе реальной ситуации):

1. Поток А выполняет мониторинг и имеет низкий приоритет. Он приобретает блокировку адресного пространства в промышленной базе данных, чтобы проверить потребление памяти.
2. Сразу после этого запускается поток В, выполняющий рутинную задачу по сжатию системных журналов.
3. Для одновременной работы обоих не хватает процессоров. Поток В вытесняет поток А и запускается.
4. Поток С из промышленной базы данных имеет высокий приоритет и находится в режиме ожидания завершения ввода/вывода. Ввод/вывод завершается, и поток С возвращается в состояние выполнения.
5. Поток С вытесняет поток В, запускается и тут же блокируется на блокировке, удерживаемой потоком А. Поток С оставляет процессор.

6. Планировщик выбирает для выполнения следующий поток с наивысшим приоритетом: поток В.
7. При запущенном потоке В высокоприоритетный поток С фактически блокируется потоком с более низким приоритетом — потоком В. Это и есть инверсия приоритета.
8. Наследование приоритета придает потоку А высокий приоритет, вытесняя поток В, до освобождения блокировки. Теперь поток С сможет быстрее приступить к работе.

В Linux, начиная с версии 2.6.18, имеется мьютекс пользовательского уровня, который поддерживает наследование приоритетов, предназначенное для рабочих нагрузок в реальном времени [Corbet 06a].

6.3.13. Несколько процессов, несколько потоков

У большинства микросхем процессоров несколько процессорных ядер. Чтобы приложение могло воспользоваться преимуществами многопроцессорной архитектуры, необходимы отдельные потоки выполнения, работающие параллельно. Например, в системе с 64 процессорами приложение сможет выполняться до 64 раз быстрее или обрабатывать в 64 раза большую нагрузку, если сумеет максимально эффективно использовать все процессоры. Степень эффективного масштабирования приложения с увеличением количества процессоров является мерой *масштабируемости*.

Существует два основных способа масштабирования приложений в зависимости от количества процессоров: выполнение в *нескольких процессах* и выполнение в *нескольких потоках*, как показано на рис. 6.4. (Обратите внимание, что это программная многопоточность, а не аппаратная одновременная многопоточность SMT, упоминавшаяся выше.)

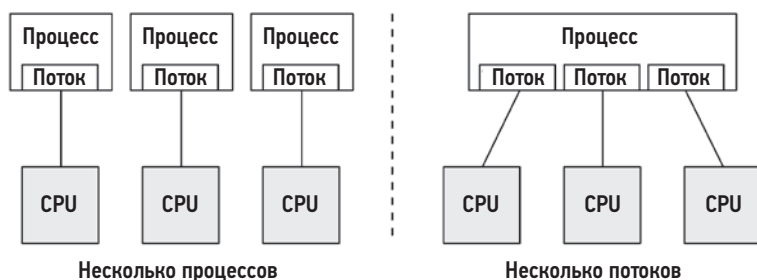


Рис. 6.4. Приемы программного масштабирования в зависимости от количества процессоров

В Linux могут использоваться обе модели, как на основе нескольких процессов, так и на основе нескольких потоков, и обе реализуются задачами.

Различия между этими моделями перечислены в табл. 6.1.

Таблица 6.1. Атрибуты, различающие модели с несколькими процессами и потоками

Атрибут	Несколько процессов	Несколько потоков
Разработка	Может быть проще. Используются системные вызовы <code>fork(2)</code> и <code>clone(2)</code>	Используется API потоков выполнения (<code>pthread</code> s)
Оверхед на использование памяти	Для каждого процесса выделяется свое адресное пространство (оверхед на организацию этих пространств бывает несколько снижен за счет применения технологии копирования при записи на уровне страниц)	Невысокий. Требуется только дополнительная память для стеков и данных, локальных для потоков
Вычислительный оверхед	Довольно высокий. На обращение к системным вызовам <code>fork(2)/clone(2)/exit(2)</code> , а также на работу модуля управления памятью (MMU) для управления адресными пространствами	Невысокий. На вызов функций API
Взаимодействия	Посредством механизмов IPC. Это влечет повышенный оверхед на переключение контекста при перемещении данных между адресными пространствами, если, конечно, не используются общие области памяти	Быстрые. Есть прямой доступ к общей памяти. Целостность данных обеспечивается механизмами синхронизации (например, мьютексами)
Устойчивость к сбоям	Высокая, процессы действуют независимо друг от друга	Низкая, любая ошибка может вызвать аварийное завершение всего приложения
Потребление памяти	Часть памяти может дублироваться, отдельные процессы могут вызывать <code>exit(2)</code> и возвращать всю свою память системе	Через системный механизм распределения памяти. Это может вызвать некоторую конкуренцию за процессор из-за наличия нескольких потоков и фрагментацию памяти из-за ее многократного использования

С учетом всех преимуществ, перечисленных в табл. 6.1, многопоточность обычно считается более предпочтительным способом масштабирования, даже при том, что она сложнее в реализации. Приемы многопоточного программирования рассматриваются в [Stevens 13].

Какой бы способ ни использовался, важно, чтобы было создано достаточное число процессов или потоков, охватывающих желаемое количество CPU, которое для достижения максимальной производительности может совпадать с количеством доступных CPU. Некоторые приложения могут иметь более высокую производительность при использовании меньшего количества CPU, например, когда затраты на синхронизацию потоков и уменьшенную локальность памяти (NUMA) перевешивают выгоды от выполнения на большем количестве CPU.

Параллельные архитектуры обсуждаются в главе 5 «Приложения» в разделе 5.2.5 «Конкурентность и параллелизм», где также кратко описываются корутины.

6.3.14. Размер слова

Процессоры рассчитаны на максимальный размер слова — 32 или 64 бита, — представляющий размер целого числа и размер регистра. Размер слова также обычно используется, в зависимости от процессора, для представления размера адресного пространства и ширины данных (которую иногда называют *разрядностью*).

Обычно чем больше размер, тем выше производительность, хотя все не так просто, как кажется. Большой размер может сопровождаться повышенным оверхедом памяти для хранения данных некоторых типов из-за неиспользуемых битов. Также с увеличением размера указателей (слова) увеличивается объем данных, что может потребовать большего количества операций чтения/записи с памятью. В 64-разрядной архитектуре x86 этот оверхед компенсируется увеличением емкости регистров и более эффективным соглашением об операциях с регистрами, поэтому 64-разрядные приложения, вероятно, будут выполняться быстрее, чем их 32-разрядные версии.

Процессоры и ОС могут поддерживать слова разных размеров и выполнять приложения, скомпилированные с разными размерами слов. Если программное обеспечение скомпилировано с меньшим размером слова, оно может работать вполне успешно, но менее эффективно.

6.3.15. Оптимизации компилятора

Время работы приложений на CPU можно значительно увеличить благодаря опциям компилятора (включая установку размера слова) и оптимизации. Компиляторы часто обновляются, чтобы использовать преимущества новейших наборов инструкций процессора и реализовать другие оптимизации. Иногда производительность приложения может быть значительно повышена просто за счет использования нового компилятора.

Эта тема более подробно рассматривается в главе 5 «Приложения» в разделе 5.3.1 «Компилируемые языки».

6.4. АРХИТЕКТУРА

В этом разделе представлена архитектура и реализация процессора, как самого оборудования, так и его ПО. Простые модели процессоров были представлены в разделе 6.2 «Модели», а общие понятия — в предыдущем разделе.

Здесь я обобщу эти темы, чтобы заложить фундамент для анализа производительности. За дополнительными сведениями обращайтесь к руководствам по процессорам от производителей и к документации с описанием внутреннего устройства операционной системы. Некоторые из них перечислены в конце этой главы.

6.4.1. Аппаратное обеспечение

Аппаратное обеспечение процессора включает микросхему процессора и его подсистемы, а также соединения между процессорами в многопроцессорных системах.

Микросхема процессора

На рис. 6.5 показаны компоненты стандартного двухъядерного процессора.

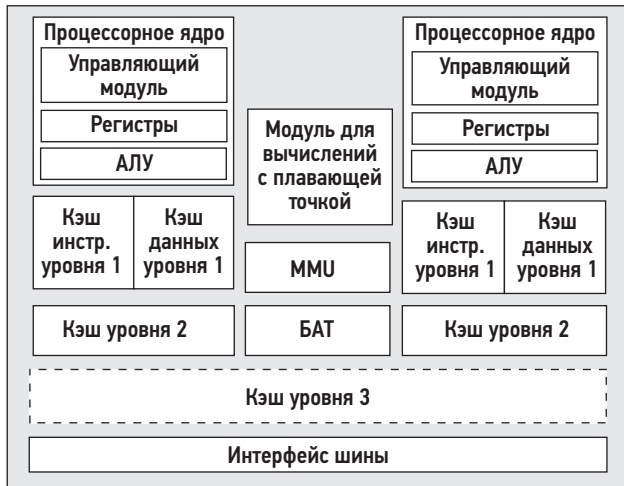


Рис. 6.5. Компоненты стандартного двухъядерного процессора

Управляющий блок — это сердце процессора. Он выбирает и декодирует инструкции, управляет их выполнением и сохраняет результаты.

В примере процессора на рис. 6.5 изображен общий модуль для вычислений с плавающей точкой и (необязательно) общий кэш уровня 3. Фактическое устройство вашего процессора может отличаться в зависимости от типа и модели. Вот другие компоненты, имеющие отношение к производительности:

- **Р-кэш (P-cache):** кэш предварительной выборки (отдельно для каждого ядра);
- **W-кэш (W-cache):** кэш записи (отдельно для каждого ядра);
- **тактовый генератор (clock):** генератор сигналов тактовой частоты (может быть реализован в виде внешнего устройства);
- **счетчик времени (timestamp counter):** счетчик времени с высоким разрешением, увеличиваемый тактовым генератором;
- **ПЗУ с микрокодом:** быстро преобразует инструкции в сигналы схемы;
- **датчики температуры:** используются для контроля перегрева;
- **сетевые интерфейсы** (если имеются на кристалле): служат для увеличения производительности.

Некоторые типы процессоров используют датчики температуры при принятии решения о разгоне отдельных ядер (включая технологию Intel Turbo Boost), увеличивая тактовую частоту, чтобы ядро оставалось в пределах установленного температурного диапазона. Возможные тактовые частоты могут определяться Р-состояниями.

Р-состояния и С-состояния

Стандарт *усовершенствованного интерфейса управления конфигурацией и энергопотреблением* (advanced configuration and power interface, ACPI), используемый процессорами Intel, определяет *состояния производительности процессора* (Р-состояния — performance states) и *состояния питания процессора* (С-состояния — power states) [ACPI 17].

Р-состояния задают различные уровни производительности при нормальном выполнении за счет изменения частоты процессора: состояние P0 характеризуется самой высокой тактовой частотой (для некоторых процессоров Intel это самый высокий уровень «турбоускорения»), а состояния P1 ... N — это более низкочастотные состояния. Эти состояния могут контролироваться как аппаратными средствами (например, в зависимости от температуры процессора), так и программными (например, режимами энергосбережения). Текущую частоту и доступные состояния можно наблюдать с помощью регистров, зависящих от модели (model-specific registers, MSR; например, с использованием инструмента showboost(8), о котором идет речь в разделе 6.6.10 «showboost»).

С-состояния задают различные состояния ожидания при остановке выполнения, что позволяет экономить электроэнергию. С-состояния перечислены в табл. 6.2: C0 — состояние нормальной работы, а C1 и выше — это состояния ожидания: чем больше число, тем глубже состояние ожидания.

Таблица 6.2. Состояния питания процессора (С-состояния)

С-состояние	Описание
C0	Нормальная работа. Процессор снабжается электроэнергией в полной мере и выполняет инструкции
C1	Выполнение остановлено. Вход в это состояние происходит по инструкции hlt. Кэши поддерживаются. Задержка выхода из такого состояния самая низкая
C1E	Расширенное состояние остановки с пониженным энергопотреблением (поддерживается некоторыми процессорами)
C2	Выполнение остановлено. Вход в это состояние происходит по аппаратному сигналу. Это более глубокое состояние спящего режима с большей задержкой выхода из него
C3	Более глубокий спящий режим с повышенной экономией энергии по сравнению с состояниями C1 и C2. Кэши могут сохранять свое содержимое, но их согласование прекращается и перекладывается на ОС

Производители процессоров могут определять дополнительные состояния, выше C3. Некоторые процессоры Intel определяют дополнительные уровни вплоть до C10, когда отключаются дополнительные функции процессора, включая сохранение содержимого кэша.

Кэши процессора

Различные аппаратные кэши включаются в состав микросхемы процессора (они называются *встроенными*, *на кристалле* или *интегрированными*) либо реализуются в виде дополнительных микросхем (*внешние*). Эти кэши увеличивают производительность операций с памятью за счет использования более быстрых типов памяти для кэширования операций чтения и буферизации записи. На рис. 6.6 показаны кэши с разными уровнями доступа в типичном процессоре.

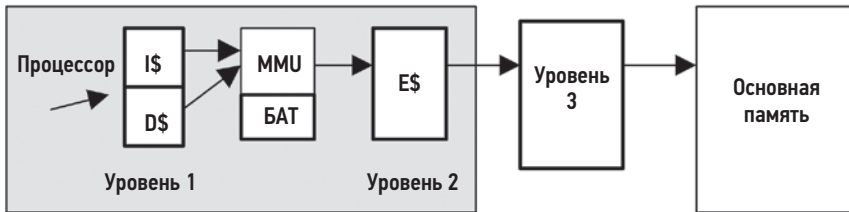


Рис. 6.6. Иерархия кэшей в процессоре

В том числе:

- кэш инструкций уровня 1 (I\$);
- кэш данных уровня 1 (D\$);
- буфер ассоциативной трансляции (БАТ);
- кэш уровня 2 (Е\$);
- кэш уровня 3 (может отсутствовать).

Буква *Е* в Е\$ изначально означала *external cache* (*внешний кэш*), но после интеграции кэшей уровня 2 в микросхему процессора его стали называть *встроенным кэшем* (*embedded cache*). Сейчас вместо обозначения в стиле «Е\$» используется термин «Уровень», что позволяет избежать путаницы.

Последний кэш перед основной памятью, который может отсутствовать, предпочтительнее называть кэшем уровня 3. В Intel для его обозначения используется термин *кэш последнего уровня* (last level cache, LLC). У этого кэша наибольшая задержка.

Кэши, доступные в каждом процессоре, зависят от его типа и модели. Со временем количество и размеры кэшей увеличиваются. Эта тенденция видна в табл. 6.3, где перечислены модели процессоров Intel, выпускавшиеся с 1978 года, включая последние достижения в области кэширования [Intel 19a], [Intel 20a].

В многоядерных и многопоточных процессорах некоторые кэши могут совместно использоваться ядрами и потоками. Например, все процессоры в табл. 6.3, начиная с Intel Xeon 7460 (2008), имеют несколько кэшей уровня 1 и уровня 2, обычно по одному на каждое ядро (размеры кэшей в табл. 6.3 указаны для каждого ядра, а не для всего процессора в целом).

Таблица 6.3. Размеры кэшей в процессорах Intel, выпускавшихся с 1978 по 2019 год

Процессор	Год	Тактовая частота	Ядер/потоков	Транзисторов	Шина данных (бит)	Уровень 1	Уровень 2	Уровень 3
8086	1978	8 МГц	1/1	29 тыс.	16	–	–	–
Intel 286	1982	12,5 МГц	1/1	134 тыс.	16	–	–	–
Intel 386 DX	1985	20 МГц	1/1	275 тыс.	32	–	–	–
Intel 486 DX	1989	25 МГц	1/1	1,2 млн	32	8 Кбайт	–	–
Pentium	1993	60 МГц	1/1	3,1 млн	64	16 Кбайт	–	–
Pentium Pro	1995	200 МГц	1/1	5,5 млн	64	16 Кбайт	256/512 Кбайт	–
Pentium II	1997	266 МГц	1/1	7 млн	64	32 Кбайт	256/512 Кбайт	–
Pentium III	1999	500 МГц	1/1	42 млн	64	32 Кбайт	512 Кбайт	–
Intel Xeon	2001	1,7 ГГц	1/1	42 млн	64	8 Кбайт	512 Кбайт	–
Pentium M	2003	1,6 ГГц	1/1	77 млн	64	64 Кбайт	1 Мбайт	–
Intel Xeon MP	2005	3,33 ГГц	1/2	675 млн	64	16 Кбайт	1 Мбайт	8 Мбайт
Intel Xeon 7140M	2006	3,4 ГГц	2/4	1,3 млрд	64	16 Кбайт	1 Мбайт	16 Мбайт
Intel Xeon 7460	2008	2,67 ГГц	6/6	1,9 млрд	64	64 Кбайт	3 Мбайт	16 Мбайт
Intel Xeon 7560	2010	2,26 ГГц	8/16	2,3 млрд	64	64 Кбайт	256 Кбайт	24 Мбайт
Intel Xeon E7-8870	2011	2,4 ГГц	10/20	2,2 млрд	64	64 Кбайт	256 Кбайт	30 Мбайт
Intel Xeon E7-8870v2	2014	3,1 ГГц	15/30	4,3 млрд	64	64 Кбайт	256 Кбайт	37,5 Мбайт
Intel Xeon E7-8870v3	2015	2,9 ГГц	18/36	5,6 млрд	64	64 Кбайт	256 Кбайт	45 Мбайт
Intel Xeon E7-8870v4	2016	3,0 ГГц	20/40	7,2 млрд	64	64 Кбайт	256 Кбайт	50 Мбайт
Intel Platinum 8180	2017	3,8 ГГц	28/56	8,0 млрд	64	64 Кбайт	1 Мбайт	38,5 Мбайт
Intel Xeon Platinum 9282	2019	3,8 ГГц	56/112	8,0 млрд	64	64 Кбайт	1 Мбайт	77 Мбайт

Помимо увеличения количества и размеров кэшей, существует также тенденция к их размещению на кристалле, за счет чего минимизируется задержка доступа.

Задержка

Наличие нескольких уровней кэшей обеспечивает оптимальное соотношение размера и задержки. Время доступа к кэшу уровня 1 обычно составляет несколько тактов, а к более объемному кэшу уровня 2 — около десятка тактов. Доступ к основной памяти может занимать около 60 нс (примерно 240 тактов для процессора с тактовой частотой 4 ГГц), а преобразование адресов в блоке управления памятью еще больше увеличивает задержку.

Время задержки кэша для конкретного процессора можно определить экспериментально с помощью микробенчмаркинга [Ruggiero 08]. На рис. 6.7 показан результат такого микробенчмаркинга — график задержки доступа к памяти для Intel Xeon E5620 с тактовой частотой 2,4 ГГц. Тестирование проводилось с увеличивающимися объемами памяти с помощью Lmbench [McVoy 12].

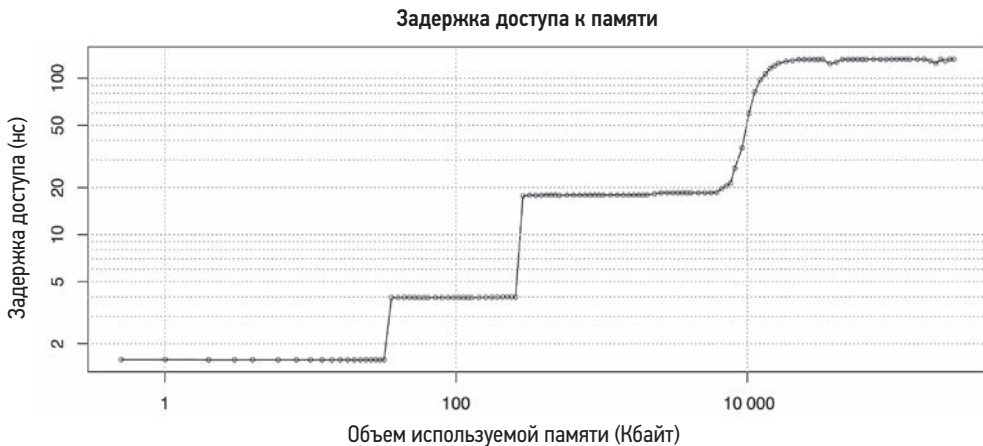


Рис. 6.7. Результаты тестирования задержки доступа к памяти

Обе оси имеют логарифмический масштаб. Перегибы на графике показывают точки превышения емкости кэша, когда задержка доступа увеличивается из-за переключения на использование кэша следующего (более медленного) уровня.

Ассоциативность

Ассоциативность — это характеристика кэша, описывающая ограничения, связанные с поиском новых записей в кэше. Кэш может быть:

- **Полностью ассоциативным:** новые записи могут находиться в кэше где угодно. Например, для вытеснения всего кэша можно использовать алгоритм наиболее давно использовавшихся (Least Recently Used, LRU).

- **С прямым отображением (direct mapped):** каждая запись имеет только одно допустимое место в кэше, например соответствующий хешу адрес памяти, когда адрес в кэше формируется с использованием подмножества битов адреса в памяти.
- **Множественно-ассоциативным (set associative):** когда подмножество кэша идентифицируется путем отображения (например, хеширования), для поиска внутри которого может использоваться другой алгоритм (например, LRU). Он описывается с точки зрения размера подмножества; например, *четырёхканальный множественно-ассоциативный* (four-way set associative) кэш отображает адрес в четыре возможных местоположения, а затем выбирает лучшее из них (например, наиболее давно использованное местоположение).

В процессорах часто используются множественно-ассоциативные кэши как компромисс между полностью ассоциативными (требующими больших затрат на выполнение) и кэшами с прямым отображением (имеющими низкую частоту попаданий).

Строки кэша

Еще одна характеристика кэшей процессора — *размер строки кэша*. Это количество байтов, хранящихся и передающихся как единое целое для увеличения пропускной способности памяти. Типичный размер строки кэша в процессорах x86 составляет 64 байта. Компиляторы учитывают это при оптимизации производительности. Иногда так поступают и программисты; см. подраздел «Хеш-таблицы» в главе 5 «Приложения», в разделе 5.2.5 «Конкуренция и параллелизм».

Согласование кэшей

Память может кэшироваться одновременно в нескольких кэшах разных процессоров. Когда один процессор изменяет память, все кэши должны узнать, что их кэшированная копия *устарела* и ее следует отбросить, чтобы при любом последующем чтении была получена только что измененная копия. Этот процесс — *согласование кэшей* (cache coherency) — гарантирует, что у процессора всегда будет верное состояние памяти.

Одно из последствий согласования кэшей — это издержки на доступ к LLC. Примеры ниже позволяют получить примерное представление о величине издержек (взяты из [Levinthal 09]):

- попадание в LLC, строка уникальная: ~ 40 тактов процессора;
- попадание в LLC, строка используется в другом ядре: ~ 65 циклов тактов процессора;
- попадание в LLC, строка изменена в другом ядре: ~ 75 циклов тактов процессора.

Согласование кэшей — одна из самых серьезных проблем при разработке масштабируемых многопроцессорных систем, потому что содержимое памяти может быстро меняться.

Блок управления памятью

Блок управления памятью (memory management unit, MMU) отвечает за преобразование виртуальных адресов в физические.

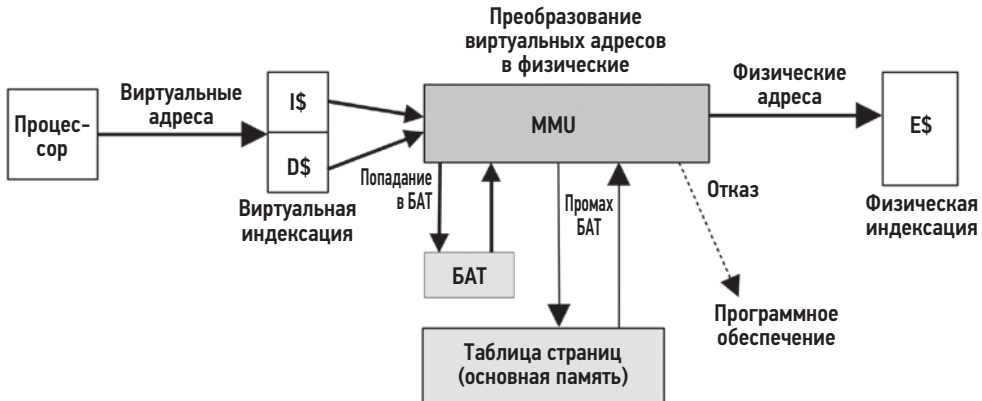


Рис. 6.8. Блок управления памятью и кэши процессора

На рис. 6.8 изображен типичный MMU вместе с кэшами процессора. Этот блок использует встроенный буфер ассоциативной трансляции (translation lookaside buffer, TLB) для кэширования преобразованных адресов. Промахи в кэше компенсируются таблицами трансляции в основной памяти (DRAM), которые называются *таблицами страниц*, читаются непосредственно модулем управления памятью (аппаратно) и поддерживаются ядром операционной системы.

Эти факторы зависят от процессора. Некоторые (более старые) процессоры обрабатывают промахи в буфере ассоциативной трансляции (TLB), используя ПО ядра операционной системы для обхода таблиц страниц, а затем заполняют буфер TLB полученными соответствиями. Такое ПО может поддерживать свой собственный, более крупный кэш преобразований адресов в памяти, называемый *буфером хранения преобразований* (translation storage buffer, TSB). Новые процессоры могут обслуживать промахи TLB аппаратно, значительно снижая их стоимость.

Соединения между процессорами

В многопроцессорных архитектурах процессоры соединяются друг с другом с помощью общей системной или выделенной шины. Это обусловлено архитектурой памяти системы, унифицированным доступом к памяти (uniform memory access, UMA), или NUMA, как описано в главе 7 «Память».

Общая системная шина, называемая «передней» шиной (front side bus, FSB) и использовавшаяся в ранних процессорах Intel, показана на рис. 6.9, на примере четырехпроцессорной архитектуры.

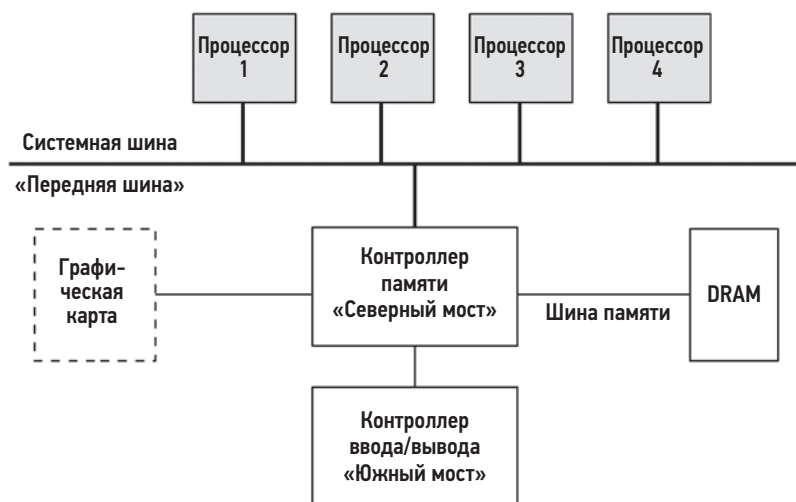


Рис. 6.9. Пример системной шины в системе с четырьмя процессорами Intel

При использовании системной шины возникают проблемы с масштабируемостью при увеличении числа процессоров из-за конкуренции за ресурс общей шины. Современные серверы, как правило, многопроцессорные, NUMA, и вместо этого используют выделенные соединения между процессорами.

Выделенные соединения могут также связывать другие компоненты, отличные от процессоров, такие как контроллеры ввода/вывода. В качестве примеров выделенных соединений можно привести Intel Quick Path Interconnect (QPI), Intel Ultra Path Interconnect (UPI), AMD HyperTransport (HT), ARM CoreLink Interconnect (существует три разных типа) и IBM Coherent Accelerator Processor Interface (CAPI). На рис. 6.10 показан пример архитектуры Intel QPI с четырьмя процессорами.

Выделенные соединения между процессорами обеспечивают бесконкурентный доступ, а также более высокую пропускную способность, чем общая системная шина. В табл. 6.4 приведены некоторые примеры скорости передачи данных для Intel FSB и QPI [Intel 09], [Mулникс 17].

Как скорость передачи соотносится с пропускной способностью, я объясню на примере QPI с тактовой частотой 3,2 ГГц. QPI поддерживает *двойную передачу*, выполняя передачу данных как по переднему, так и по заднему фронту тактового импульса¹. Это удваивает скорость передачи ($3,2 \text{ ГГц} \times 2 = 6,4 \text{ ГТ/с}$). В результате пропускная способность составляет 25,6 Гбайт/с в обоих направлениях — отправки и приема ($6,4 \text{ ГТ/с} \times \text{ширина в 2 байта} \times 2 \text{ направления} = 25,6 \text{ ГБ/с}$).

¹ Есть и *счетверенная передача*, когда данные передаются по нарастающему фронту, верхнему пику, падающему фронту и нижнему пику тактового импульса. Счетверенная передача используется системной шиной Intel FSB.



Рис. 6.10. Пример архитектуры Intel QPI с четырьмя процессорами

Таблица 6.4. Примеры пропускной способности выделенных соединений между процессорами Intel

Intel	Скорость передачи	Ширина	Пропускная способность
FSB (2007)	1,6 ГТ/с ¹	8 байт	12,8 Гбайт/с
QPI (2008)	6,4 ГТ/с	2 байта	25,6 Гбайт/с
UPI (2017)	10,4 ГТ/с	2 байта	41,6 Гбайт/с

Интересная деталь QPI — это возможность настройки режима согласования кэшей в BIOS выбором таких параметров, как Home Snooper (для оптимизации пропускной способности памяти), Early Snooper (для оптимизации задержки памяти) и Directory Snooper (для улучшения масштабируемости, включая трассировку совместно используемой памяти). Технология UPI, пришедшая на смену QPI, поддерживает только Directory Snooper.

Помимо внешних выделенных соединений, процессоры имеют внутренние соединения для обмена данными между ядрами.

Выделенные соединения обычно рассчитаны на большую пропускную способность, поэтому редко оказываются узким местом. Но если это произойдет, то производительность будет снижаться, так как процессоры будут вынуждены совершать

¹ Транзакций (пересылок) в секунду, в данном случае гигабайт/секунду. — Примеч. пер.

холостые циклы в периоды выполнения операций с соединениями, таких как чтение/запись в удаленную память. Ключевым показателем этого является падение числа инструкций на цикл (instructions per cycle, IPC). Инструкции процессора, такты, IPC, холостые циклы простоя и чтение/запись в память можно анализировать с помощью счетчиков производительности процессора.

Аппаратные счетчики производительности (PMC)

Счетчики мониторинга производительности (performance monitoring counter, PMC) были представлены как источники наблюдаемых статистик в главе 4 «Инструменты наблюдения», в разделе 4.3.9 «Аппаратные счетчики (PMC)». В этом разделе я более подробно расскажу об их реализации в процессорах и приведу дополнительные примеры.

Счетчики PMC — это аппаратные регистры процессора, которые можно запрограммировать для подсчета низкоуровневой активности процессора. Обычно с их помощью подсчитываются:

- **такты (циклы) процессора:** включая холостые циклы простоя и типы тактов простоя;
- **инструкции процессора:** выполненные;
- **обращения к кэшам уровней 1, 2, 3:** попадания, промахи;
- **модуль для вычислений с плавающей точкой:** операции;
- **ввод/вывод с памятью:** операции чтения, операции записи, холостые циклы;
- **ресурсы ввода/вывода:** операции чтения, операции записи, холостые циклы.

Каждый процессор имеет небольшое количество регистров, обычно от двух до восьми, которые можно запрограммировать для регистрации подобных событий. Перечень событий, доступных для регистрации, зависит от типа и модели CPU и задокументирован в руководстве по CPU.

Рассмотрим простой пример: семейство процессоров Intel P6 поддерживает счетчики производительности через четыре регистра. Два регистра являются счетчиками и доступны только для чтения. Два других регистра, называемые регистрами *выбора события*, используются для программирования счетчиков и доступны для чтения и записи. Счетчики производительности — это 40-битные регистры, а регистры выбора события — 32-битные. Формат регистра выбора события показан на рис. 6.11.

Счетчик идентифицируется селектором события и маской категории UMASK. Селектор события определяет тип события для подсчета, а UMASK — подтип или группу подтипов. Биты OS и USR можно установить так, чтобы счетчик увеличивался только в режиме ядра (OS) или в режиме пользователя (USR) в зависимости от текущего кольца защиты процессора. В маске счетчика CMASK можно установить порог количества событий, по достижении которого следует производить увеличение счетчика.

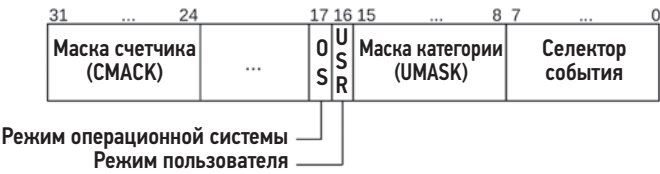


Рис. 6.11. Пример регистра выбора события в процессорах Intel

В руководстве по процессору Intel (том 3В [Intel 19b]) перечислены десятки событий, которые можно подсчитать с их значениями селектора события и маски UMASK. В табл. 6.5 приводятся некоторые примеры, позволяющие получить представление о разнообразии доступных целей (функциональных блоков процессора), за работой которых можно наблюдать, и выдержки из описания в руководстве. Чтобы узнать, что доступно в вашем конкретном случае, почитайте руководство по вашему CPU.

Таблица 6.5. Некоторые примеры счетчиков производительности в процессорах Intel

Селектор события	UMASK	Модуль	Имя	Описание
0x43	0x00	Кэш данных	DATA_MEM_REFS	Все операции чтения из памяти любого типа. Все операции записи в память любого типа. Обращение к памяти каждого типа учитывается отдельно. ... Не включает операции ввода/вывода или другие обращения, не имеющие отношения к памяти
0x48	0x00	Кэш данных	DCU_MISS_OUTSTANDING	Взвешенное количество циклов ожидания восполнения промахов кэша в модуле кэширования данных (data cache unit, DCU), увеличивается на количество промахов кэша в любой конкретный момент времени. Учитываются только кэшируемые запросы на чтение
0x80	0x00	Модуль выборки инструкций	IFU_IFETCH	Количество выборок инструкций, как кэшируемых, так и не-кэшируемых, включая некэшированные выборки
0x28	0x0F	Кэш L2	L2_IFETCH	Количество выборок инструкций из L2...

Таблица 6.5 (окончание)

Селектор события	UMASK	Модуль	Имя	Описание
0xC1	0x00	Модуль для вычислений с плавающей точкой	FLOPS	Количество выполненных операций с плавающей точкой
0x7E	0x00	Логика внешней шины	BUS_SNOOP_STALL	Количество циклов, в течение которых шина бездействовала
0xC0	0x00	Декодирование и выполнение инструкций	INST_RETIRED	Количество выполненных инструкций
0xC8	0x00	Прерывания	HW_INT_RX	Количество полученных аппаратных прерываний
0xC5	0x00	Ветвление	BR_MISS_PRED_RETIRED	Количество ошибочно выполненных ветвлений, которые были неправильно предсказаны
0xA2	0x00	Простои	RESOURCE_STALLS	Увеличивается на единицу с каждым циклом, выполненным вхолостую из-за задержки, связанной с ресурсами
0x79	0x00	Часы	CPU_CLK_UNHALTED	Количество циклов, в течение которых процессор не останавливался

Есть множество других счетчиков, особенно в новых процессорах.

Еще одна важная деталь, которую следует знать, — количество регистров аппаратных счетчиков, предлагаемых процессором. Например, микроархитектура Intel Skylake предлагает три фиксированных счетчика на каждый аппаратный поток и дополнительно восемь программируемых («универсальных») счетчиков на ядро. При чтении эти счетчики возвращаются как 48-битные значения.

Дополнительные примеры счетчиков PMC из архитектурного набора Intel см. в табл. 4.4, в разделе 4.3.9. В разделе 4.3.9 также приводятся ссылки на документацию с описанием счетчиков PMC в процессорах AMD и ARM.

Графические процессоры

Графические процессоры (graphics processing unit, GPU) были созданы для поддержки графических дисплеев и теперь применяются в машинном обучении, искусственном интеллекте, аналитике, обработке изображений и майнинге криптовалюты. Для серверов и облачных экземпляров GPU — процессороподобный ресурс, который может выполнять часть рабочей нагрузки, называемой *вычислительным ядром*, которая подходит для высокопараллельной обработки данных, например

матричных преобразований. Универсальные GPU от Nvidia, использующие ее архитектуру унифицированного вычислительного устройства (Compute Unified Device Architecture, CUDA), получили широкое распространение. CUDA предоставляет API и программные библиотеки для использования GPU Nvidia.

Если CPU может иметь десяток ядер, то GPU — сотни и даже тысячи маленьких ядер, которые называются *потоковыми процессорами* (streaming processor, SP)¹. Каждое из них может обслуживать отдельный поток выполнения. Поскольку рабочие нагрузки для GPU имеют очень высокий уровень параллелизма, потоки, которые могут выполняться параллельно, группируются в *блоки потоков*, где могут взаимодействовать друг с другом. Блоки потоков могут выполняться группами потоковых процессоров (SP), которые называются *потоковыми мультипроцессорами* (streaming multiprocessor, SM), и могут также предоставлять другие ресурсы, включая кэш-память. В табл. 6.6 приводится дополнительное сравнение CPU и GPU [Ather 19].

Таблица 6.6. Сравнение CPU и GPU

Атрибут	CPU	GPU
Корпус	Микросхема процессора включается в разъем на системной плате и подключается непосредственно к системной шине или соединением с другими CPU	Графический процессор обычно поставляется в виде карты расширения и подключается через шину расширения (например, PCIe). Графические процессоры также встраивают в системную плату или в корпус процессора (на кристалле)
Масштабируемость на уровне корпусов	Конфигурации с несколькими гнездами, связанными внутренними соединениями (например, Intel UPI)	Конфигурации с несколькими GPU тоже возможны. В этом случае используются внутренние соединения между GPU (например, NVIDIA NVLink)
Ядра	Типичный современный процессор имеет от 2 до 64 ядер	GPU может иметь такое же количество потоковых мультипроцессоров (SM)
Потоки	Типичное процессорное ядро может иметь два аппаратных потока выполнения (или больше, в зависимости от модели процессора)	Потоковый мультипроцессор (SM) может иметь десятки или сотни потоковых процессоров (SP). Каждый потоковый процессор может выполнять только один поток
Кэши	Каждое ядро имеет свои кэши L1 и L2, и все они могут иметь один общий кэш L3	Каждый потоковый мультипроцессор имеет свой кэш L1, и все они могут использовать один общий кэш L2
Тактовый генератор	Высокая частота (например, 3,4 ГГц)	Относительно низкая частота (например, 1,0 ГГц)

Для наблюдения за производительностью GPU нужно использовать специальные инструменты. Возможные метрики производительности GPU: количество инструкций на такт, коэффициент попадания в кэш и потребление шины памяти.

¹ В Nvidia их еще называют *ядрами CUDA* [Verma 20].

Другие ускорители

Помимо GPU есть и другие ускорители, предназначенные для разгрузки процессора и решения узкоспециализированных прикладных задач. К ним относятся программируемые пользователем вентильные матрицы (field-programmable gate arrays, FPGA) и тензорные процессоры (tensor processing unit, TPU). При наличии их использование и производительность следует анализировать вместе с процессорами, хотя обычно для их анализа требуются специальные инструменты.

GPU и FPGA широко применяются для повышения производительности майнинга криптовалюты.

6.4.2. Программное обеспечение

В ПО ядра операционной системы, обеспечивающее поддержку процессоров, входят: планировщик, классы планирования и поток бездействия (idle thread).

Планировщик

Основные функции планировщика процессорного времени в ядре ОС показаны на рис. 6.12.

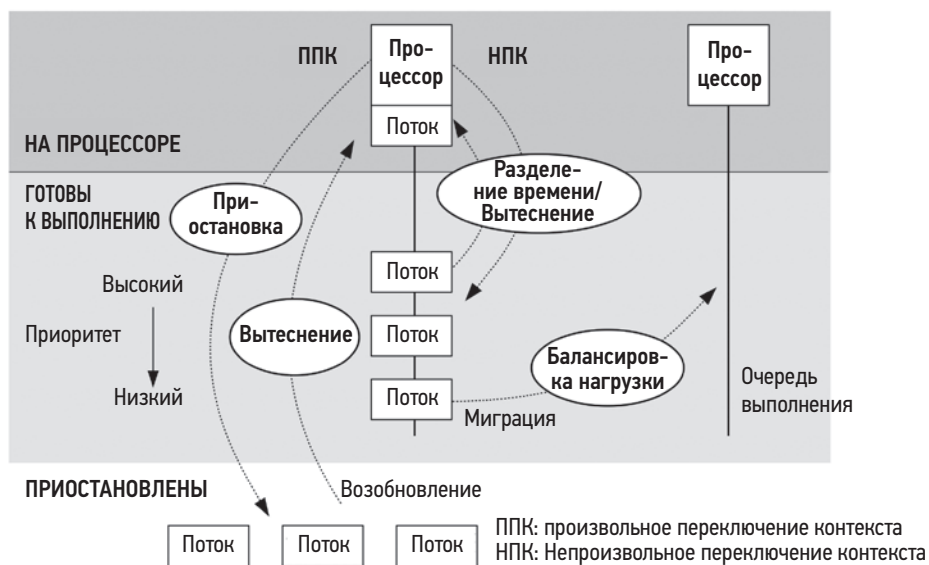


Рис. 6.12. Функции планировщика в ядре ОС

Вот эти функции:

- **Разделение времени:** многозадачность между выполняемыми потоками, когда преимущество в выполнении отдается потокам с наивысшим приоритетом.

- **Вытеснение:** если поток с высоким приоритетом перешел в разряд готовых к выполнению, планировщик может вытеснить текущий поток, чтобы поток с более высоким приоритетом начал выполняться немедленно.
- **Балансировка нагрузки:** перемещение потоков, готовых к выполнению и простаивающих в очереди на выполнение менее загруженных процессоров.

На рис. 6.12 показаны очереди на выполнение, как они изначально были реализованы. Эта терминология и модель все еще используются для описания ожидающих задач. Но современный планировщик CFS (*completely fair scheduler* — абсолютно справедливый планировщик) в Linux вместо очереди использует красно-черное дерево.

В Linux разделение времени реализовано с помощью прерываний системного таймера и функции `scheduler_tick()`, которая вызывает функции планировщика классов для управления приоритетами и слежения за истечением единиц процессорного времени, называемых *квантами времени*. Вытеснение происходит, когда потоки переходят в категорию готовых к выполнению, и реализуется функцией `check_preempt_curr()` планировщика. Переключаются потоки функцией `_schedule()`, которая выбирает поток с наивысшим приоритетом вызовом `pick_next_task()`. Балансировка нагрузки реализуется функцией `load_balance()`.

Также планировщик в Linux использует особую логику, стараясь избежать миграций, когда ожидается, что затраты на миграцию превысят выгоды от нее и предпочтительнее оставить потоки выполняться на том же процессоре, где кэши все еще должны оставаться разогретыми (привязка к процессору). В исходном коде Linux за это отвечают функции `idle_balance()` и `task_hot()`.

Обратите внимание, что имена всех этих функций могут измениться. Подробную информацию ищите в исходном коде ядра Linux и в документации в каталоге Documentation.

Классы планирования

Классы планирования управляют поведением потоков, готовых к выполнению. В частности, их приоритетами, выделением *квантов процессорного времени* и продолжительностью этих квантов. Есть и дополнительные возможности управления с применением политик планирования, которые могут выбираться для класса планирования и управлять планированием потоков с одинаковым приоритетом.

Они показаны на рис. 6.13 вместе с диапазоном приоритетов потоков.

На приоритет потоков пользовательского уровня влияет определяемое пользователем значение *nice* (степень уступчивости), установкой которого можно понизить приоритет выполнения неважной работы (чтобы проявить *уступчивость* по отношению к другим пользователям системы). В Linux значение *nice* устанавливает *статический приоритет* потока, который отличается от *динамического приоритета*, вычисляемого планировщиком.

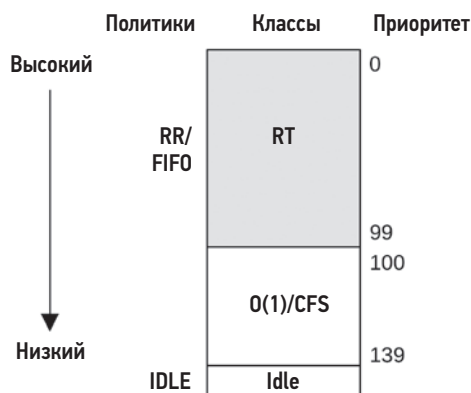


Рис. 6.13. Приоритеты планировщика потоков в Linux

Ядра Linux поддерживают следующие классы планирования:

- **RT (Real Time — реального времени):** присваивает фиксированные высокие приоритеты рабочим нагрузкам, выполняющимся в режиме реального времени. Ядро поддерживает приоритетное вытеснение как на уровне пользователя, так и на уровне ядра, что позволяет планировать задачи реального времени с минимальной задержкой. Диапазон приоритета: 0–99 (MAX_RT_PRIO-1).
- **O(1):** планировщик O(1) появился в ядре Linux 2.6 как планировщик разделения времени по умолчанию для пользовательских процессов. Свое название этот планировщик получил от сложности алгоритма O(1) (см. главу 5 «Приложения», где приводится обзор нотации «О-большое»). Предшествовавший ему планировщик использовал процедуры, которые выполняли итерации по всем задачам, и имел сложность O(n), что вызывало проблемы масштабируемости. Планировщик O(1) динамически повышает приоритет рабочих нагрузок, связанных с вводом/выводом, отдавая им предпочтение перед преимущественно вычислительными рабочими нагрузками, чтобы уменьшить задержку интерактивных рабочих нагрузок и рабочих нагрузок, связанных с вводом/выводом.
- **CFS:** абсолютно справедливый планировщик был добавлен в ядро Linux 2.6.23 как планировщик разделения времени по умолчанию для пользовательских процессов. Вместо традиционных очередей на выполнение этот планировщик использует красно-черное дерево, ключами в котором служат процессорные времена задач. Это позволяет легко находить и запускать потоки с низким уровнем потребления процессора, отдавая им предпочтение перед преимущественно вычислительными рабочими нагрузками и повышая производительность интерактивных рабочих нагрузок и рабочих нагрузок, связанных с вводом/выводом.
- **Idle:** запускает потоки с наименьшим возможным приоритетом.
- **Deadline:** добавлен в Linux 3.14, применяет планирование по принципу «сначала самый ранний дедлайн» (earliest deadline first, EDF), используя три параметра: *runtime*, *period* и *deadline*. Задача должна получать *runtime* микросекунд

процессорного времени в течение каждой *period* микросекунд и до установленного срока *deadline*.

Для выбора класса планирования процессы уровня пользователя выбирают *политику планирования*, которая отображается в класс системным вызовом `sched_setscheduler(2)` или инструмента `chrt(1)`.

Политики планирования:

- **RR (Round Robin):** `SCHED_RR` — это алгоритм планирования циклическим перебором. Как только поток исчерпает свой квант времени, он перемещается в конец очереди на выполнение для своего уровня приоритета. Это позволяет запускать другие потоки с таким же приоритетом. Использует класс планирования `RT`.
- **FIFO (First In, First Out):** `SCHED_FIFO` — это алгоритм планирования «первым пришел, первым вышел», который продолжает выполнять поток, находящийся в голове очереди на выполнение, пока тот не оставит процессор добровольно или не появится поток с более высоким приоритетом. Поток продолжает выполняться, даже если в очереди на выполнение есть другие потоки с таким же приоритетом. Использует класс планирования `RT`.
- **NORMAL:** `SCHED_NORMAL` (ранее известный как `SCHED_OTHER`) — это алгоритм планирования с разделением времени. Используется по умолчанию для пользовательских процессов. Планировщик динамически корректирует приоритет в зависимости от класса планирования. Для $O(1)$ длительность кванта времени устанавливается на основе статического приоритета: чем выше приоритет, тем больше квант времени. Для класса `CFS` размер кванта выбирается динамически. Использует класс планирования `CFS`.
- **BATCH:** `SCHED_BATCH` — этот алгоритм действует как `SCHED_NORMAL`, но ожидается, что поток будет привязан к процессору и не должен прерывать другие интерактивные задачи, связанные с вводом/выводом. Использует класс планирования `CFS`.
- **IDLE:** `SCHED_IDLE` использует класс планирования `Idle`.
- **DEADLINE:** `SCHED_DEADLINE` использует класс планирования `Deadline`.

Со временем в ядро могут быть добавлены другие классы и политики. Проводились исследования алгоритмов планирования, учитывающих *гиперпоточность* [Bulpin 05] и *температуру* [Otto 06], которые оптимизируют производительность за счет учета дополнительных факторов.

Когда планировщик не обнаруживает потоков, готовых к выполнению, он запускает специальную *задачу бездействия* (так называемый *поток бездействия* — *idle thread*) и продолжает выполнять ее, пока не появится другой поток, готовый к выполнению.

Поток бездействия

Представленный в главе 3 поток бездействия в ядре (или задача бездействия) выполняется на процессоре, когда нет другого потока, готового к выполнению, и имеет

минимально возможный приоритет. Обычно он сообщает процессору, что работа может быть остановлена (инструкция остановки) или тактовая частота снижена для экономии энергии. Процессор возобновит работу при следующем аппаратном прерывании.

Группировка NUMA

Производительность в системах NUMA можно значительно улучшить, добавив в ядро поддержку NUMA, чтобы оно могло планировать и принимать оптимальные решения о размещении памяти. Такая поддержка может автоматически обнаруживать и создавать группы локализованных ресурсов процессоров и памяти и организовывать их в топологию, отражающую архитектуру NUMA. Эта топология позволяет оценить стоимость любого доступа к памяти.

В системах Linux они называются *доменами планирования*, которые образуют топологию, начинающуюся с *корневого домена*.

Группировка вручную может быть выполнена системным администратором путем привязки процессов к одному или нескольким процессорам или путем создания отдельного набора процессоров для выполнения процессов. См. раздел 6.5.10 «Привязка к процессору».

Учет ресурсов процессоров

Топология ресурсов процессоров также может учитываться ядром для принятия оптимальных решений по планированию, для управления питанием, использования аппаратного кэша и балансировки нагрузки.

6.5. МЕТОДОЛОГИЯ

В этом разделе описаны различные методологии и упражнения анализа и настройки производительности процессора. Краткий перечень методологий приводится в табл. 6.7.

Таблица 6.7. Методологии анализа и настройки производительности процессора

Раздел	Методология	Тип
6.5.1	Метод инструментов	Анализ наблюдения
6.5.2	Метод USE	Анализ наблюдения
6.5.3	Определение характеристик рабочей нагрузки	Анализ наблюдения, планирование мощности
6.5.4	Профилирование	Анализ наблюдения
6.5.5	Анализ тактов	Анализ наблюдения

Раздел	Методология	Тип
6.5.6	Мониторинг производительности	Анализ наблюдения, планирование мощности
6.5.7	Статическая настройка производительности	Анализ наблюдения, планирование мощности
6.5.8	Настройка приоритетов	Настройка
6.5.9	Управление ресурсами	Настройка
6.5.10	Привязка к процессору	Настройка
6.5.11	Микробенчмаркинг	Анализ экспериментов

Список дополнительных методологий и краткое введение в них вы найдете в главе 2 «Методологии». От вас не требуется использовать все перечисленные методологии — воспринимайте этот список как сборник кулинарных рецептов, которые можно использовать как по отдельности, так и вместе.

Я предлагаю использовать их в таком порядке: мониторинг производительности, метод USE, профилирование, микробенчмаркинг и статическая настройка производительности.

В разделе 6.6 «Инструменты наблюдения» и последующих показано применение этих методологий с использованием инструментов ОС.

6.5.1. Метод инструментов

Метод инструментов — это процесс перебора доступных инструментов с целью изучения ключевых метрик, которые они возвращают. Это очень простая методология, но при ее использовании могут оставаться незамеченными некоторые проблемы, которые плохо обнаруживаются или вообще не обнаруживаются инструментами. Также на ее применение может уйти много времени.

При анализе производительности процессора метод инструментов можно использовать для проверки следующих метрик (Linux):

- **uptime/top**: проверьте средние значения нагрузки, чтобы узнать, как меняется нагрузка с течением времени. Нагрузка может меняться во время анализа — не забывайте об этом при использовании инструментов, перечисленных далее.
- **vmstat**: запустите `vmstat(1)` с интервалом в одну секунду и проверьте потребление процессора в масштабе всей системы («us» + «sy»). Потребление, близкое к 100 %, с большой вероятностью указывает на задержки планировщика.
- **mpstat**: изучите статистики производительности процессоров и проверьте отдельные горячие (наиболее загруженные) процессоры, чтобы выявить возможную проблему масштабируемости потоков.
- **top**: посмотрите, какие процессы и пользователи являются основными потребителями процессора.

- **pidstat**: выделите основных потребителей процессорного времени в режиме пользователя и ядра.
- **perf/profile**: выполните профилирование трассировок стека основных потребителей процессорного времени в режиме пользователя и ядра, чтобы определить причину высокого потребления.
- **perf**: измерьте число инструкций на цикл (instructions per cycle, IPC), которое может служить индикатором неэффективности на уровне тактов.
- **showboost/turboboost**: проверьте текущую тактовую частоту процессора, возможно, она слишком низкая.
- **dmesg**: проверьте сообщения о перегреве процессора («cpu clock throttled»).

При обнаружении проблемы изучите все поля в выводе доступных инструментов, чтобы получить больше информации. Дополнительную информацию о каждом инструменте вы найдете в разделе 6.6 «Инструменты наблюдения».

6.5.2. Метод USE

Метод USE используют на ранних этапах исследования производительности для выявления узких мест и ошибок во всех компонентах, прежде чем переходить к более глубоким и трудоемким стратегиям.

Для каждого процессора проверьте:

- **Потребление**: время, в течение которого процессор был занят (когда не выполнялся поток бездействия).
- **Насыщенность**: время, в течение которого потоки, готовые к выполнению, ждали своей очереди выполнения на процессоре.
- **Ошибки**: ошибки процессора, включая исправимые ошибки.

Сначала можно проверить ошибки — их легко интерпретировать, и это занимает мало времени. Некоторые процессоры и ОС обнаруживают увеличение количества исправляемых ошибок (error-correction code, ECC) и отключают процессор в качестве меры предосторожности, прежде чем неисправимая ошибка приведет к отказу процессора. Простейший способ проверить отсутствие таких ошибок — убедиться, что все процессоры все еще включены.

Уровень потребления обычно сообщается инструментами ОС как *процент занятости*. Эту метрику нужно проверить для каждого CPU, чтобы не пропустить проблемы с масштабируемостью. Высокое потребление процессора выявляют с помощью профилирования и анализа тактов.

Для сред, реализующих ограничения или квоты на потребление процессора (средства управления ресурсами: например, наборы задач и контрольные группы cgroups в Linux), что особенно характерно для облачных вычислений, потребление процессора следует оценивать с точки зрения наложенного ограничения в дополнение к физическому пределу. Ваша система может исчерпать свою квоту задолго

до того, как физическое потребление достигнет 100 %, что приведет к насыщению раньше, чем ожидалось.

Метрики насыщенности обычно предоставляются в масштабе всей системы, в том числе как часть средних значений нагрузки. Эта метрика количественно определяет степень перегруженности CPU или использования квоты, если она установлена.

Аналогично можно проверять производительность GPU и других ускорителей, если они используются, в зависимости от доступности соответствующих метрик.

6.5.3. Определение характеристик рабочей нагрузки

Определение характеристик приложенной нагрузки — важный шаг при планировании мощностей, сравнительном анализе и моделировании рабочих нагрузок. Это также помогает значительно увеличить производительность за счет выявления и устранения ненужной работы.

Вот основные атрибуты, характеризующие нагрузку на процессор:

- средняя нагрузка на процессор (потребление + насыщение);
- отношение времени выполнения в режиме пользователя и ядра;
- частота обращений к системным вызовам;
- частота намеренного переключения контекста;
- частота прерываний.

Цель состоит в том, чтобы охарактеризовать приложенную нагрузку, а не обеспеченную производительность. Средние значения нагрузки в некоторых ОС (например, Solaris) показывают только нагрузку на CPU, что делает их основной метрикой, характеризующей рабочую нагрузку на процессор. Но в Linux средние значения включают другие типы нагрузки. См. пример и дальнейшее объяснение в разделе 6.6.1 «uptime».

Метрики частоты интерпретировать немного сложнее — они отражают как приложенную нагрузку, так и в некоторой степени обеспеченную производительность, которая может занижать их частоту¹.

Отношение времени выполнения в режиме пользователя к времени выполнения в режиме ядра показывает тип приложенной нагрузки, как было отмечено в разделе 6.3.9 «Время в режиме пользователя/ядра». Большая доля времени выполнения в режиме пользователя говорит о том, что приложения тратят больше времени на свои вычисления. Большая доля времени выполнения в режиме ядра, напротив,

¹ Представьте, что при работе на более быстрых процессорах та же самая рабочая нагрузка, производящая пакетные вычисления, имеет более высокую частоту обращений к системным вызовам. Она завершит вычисления раньше!

указывает, что много времени тратится на выполнение функций ядра и следует продолжить изучение характера рабочей нагрузки по системным вызовам и частоте прерываний. Рабочие нагрузки, связанные с вводом/выводом, имеют большее время выполнения в режиме ядра и более высокую частоту обращений к системным вызовам и намеренного переключения контекста, чем преимущественно вычислительные рабочие нагрузки, потому что потоки блокируются в ожидании завершения ввода/вывода.

Вот пример описания рабочей нагрузки, показывающий использование этих атрибутов для выражения:

На типичном сервере приложений с 48 процессорами средняя нагрузка колеблется в течение дня от 30 до 40. Отношение времени выполнения в режиме пользователя/ядра составляет 95/5, потому что это вычислительная рабочая нагрузка, интенсивно использующая процессор. Частота обращений к системным вызовам составляет 325 000 в секунду, а частота намеренных переключений контекста — около 80 000 в секунду.

Эти характеристики могут изменяться с течением времени из-за приложения различных нагрузок.

Чек-лист для определения дополнительных характеристик рабочей нагрузки

При необходимости можно выяснить дополнительные сведения, характеризующие рабочую нагрузку. Они перечислены здесь как вопросы и могут служить чек-листом при исследовании проблем с процессором:

- Какова величина потребления процессора ЦП в масштабе всей системы? Каждого процессора? Каждого ядра?
- Насколько равномерно нагружены процессоры? Сколько потоков имеется в процессоре?
- Какие приложения или пользователи потребляют процессор? В каком объеме?
- Какие потоки ядра ОС потребляют процессор? В каком объеме?
- Сколько процессорного времени расходуется на обработку прерываний?
- Насколько загружены внутренние соединения между процессорами?
- Какие пути в коде (в режиме пользователя и ядра) потребляют процессор?
- Какие типы холостых циклов встречаются?

Более подробно эта методология и характеристики, которые необходимо измерить (кто, почему, зачем, как), описываются в главе 2 «Методологии». В следующих разделах мы подробно рассмотрим два последних вопроса в этом списке. Я расскажу, как проанализировать пути в коде с помощью профилирования и холостые циклы с помощью анализа тактов.

6.5.4. Профилирование

Профилирование создает картину изучаемой цели. Профилирование процессора можно выполнять разными способами, в том числе:

- **Выборкой по времени**, когда по таймеру выбираются трассировки стека текущей выполняющейся функции. На практике обычно используется выборка с частотой 99 Гц (выборка в секунду) на процессор. Это позволяет получить представление о потреблении процессора, достаточно подробное для решения больших и мелких проблем. Величина 99 используется, чтобы избежать попадания в одну и ту же точку, которое может произойти на частоте 100 Гц и привести к искажению профиля. При необходимости частоту таймера можно уменьшить, а интервал профилирования увеличить до значений, когда оверхед еще остается незначительным для продакшена.
- **Трассировка функций** инструментацией всех или некоторых функций для измерения продолжительности их выполнения. Этот подход обеспечит точное представление о работе функций, но оверхед может оказаться недопустимым для промышленной среды, часто превышая 10 %, потому что трассировка функций добавляет к каждому вызову время на работу инструмента.

Большинство профилировщиков, используемых в промышленной среде, и те, что описаны в этой книге, используют выборку по времени. Как они это делают, показано на рис. 6.14, где во время выборки трассировок стека приложение вызывает функцию A(), которая вызывает функцию B(), и т. д. См. главу 3 «Операционные системы», раздел 3.2.7 «Стеки», где рассказывается, что такое трассировки стека и как их читать.

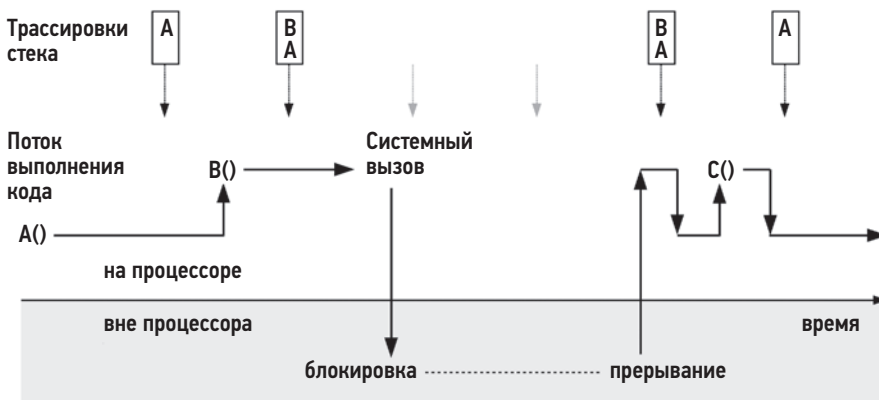


Рис. 6.14. Профилирование процессора выборкой по времени

Как показано на рис. 6.14, трассировки отбираются, только когда процесс находится на процессоре: две трассировки относятся к функции A(), а две — к функции B(),

которая вызывается из A(). Время вне процессора, пока выполняется системный вызов, не измеряется. Кроме того, функция C(), выполняющаяся очень короткое время, не попала ни в одну выборку.

Ядра ОС обычно поддерживают для процессов две трассировки стека: стек уровня пользователя и стек ядра, когда выполняется код ядра (например, системные вызовы). Для получения полного профиля процессора профилировщик должен фиксировать оба стека, если они доступны.

Помимо выборки трассировок стека, профилировщики также могут записывать только указатель инструкции, определяющий функцию, выполняемую на процессоре, и смещение текущей инструкции от начала функции. Иногда этого достаточно для решения проблем без дополнительного оверхеда на сбор трассировок стека.

Обработка трассировок

Как говорилось в главе 5, типичный профиль процессора в Netflix включает трассировки стека уровня пользователя и ядра и строится на основе образцов, отбираемых с частотой 49 Гц на (примерно) 32 процессорах в течение 30 с. Это дает в общей сложности 47 040 трассировок и создает две проблемы:

1. **Затраты на операции ввода/вывода с хранилищем.** Профилировщики обычно сохраняют образцы в файл профиля, который затем можно исследовать разными способами. Но запись такого количества образцов в файловую систему может привести к выполнению значительного количества операций ввода/вывода и снизить производительность промышленного приложения. Инструмент `profile(8)` на основе BPF решает проблему ввода/вывода путем обобщения выборок в памяти ядра и передачи в пользовательское пространство только сводной информации. Для промежуточного хранения профиля файлы не используются.
2. **Сложность анализа:** прочитать 47 040 многострочных трассировок стека в профиле практически невозможно, поэтому для анализа профиля необходимо использовать методы обобщения и инструменты визуализации. Обычно для визуального представления трассировок стека используют *флейм-графики* (примеры см. в главах 1 и 5). В этой главе будут дополнительные примеры.

На рис. 6.15 изображен обобщенный алгоритм создания флейм-графиков процессорного времени с помощью инструментов `perf(1)` и `profile`, что позволяет решить проблему сложности анализа. Он также показывает, как решается проблема ввода/вывода: `profile(8)` не использует промежуточные файлы, что снижает оверхед. Конкретные команды перечислены в разделе 6.6.13 «`perf`».

Решение на основе BPF имеет меньший оверхед, зато решение на основе `perf(1)` сохраняет исходные образцы (с отметками времени), которые можно повторно обрабатывать с помощью других инструментов, включая FlameScore (раздел 6.7.4).

1. Просмотрите график сверху вниз и выделите большие «плато». Они показывают, что одна и та же функция часто оказывалась на процессоре в момент получения выборки, и могут подсказать направления для быстрого получения выигрышей в производительности. На рис. 6.16 справа — два плато, соответствующие функциям `unmap_page_range()` и `page_remove_map()`. Оба они связаны со страницами памяти. Возможно, быстрым выигрышем здесь может стать переключение приложения на использование страниц большего размера.
2. Просмотрите график снизу вверх, чтобы понять иерархию кода. В этом примере оболочка `bash(1)` вызывала системный вызов `execve(2)`, который в итоге вызывал функции обслуживания страниц. Возможно, еще больший выигрыш можно получить, если как-то избежать обращения к `execve(2)`, например, используя встроенные функции `bash` вместо внешних процессов или переключившись на другой язык программирования.
3. Просмотрите более внимательно график сверху вниз и поищите разрозненную, но типичную нагрузку на процессор. Возможно, на графике будет множество небольших фреймов, связанных с одной и той же проблемой, например с состязанием за блокировку. Инвертирование порядка слияния флейм-графика так, чтобы они объединялись от листа к корню, становясь «графиками сосулек», может помочь выявить такие случаи.

Другой пример интерпретации флейм-графика процессорного времени приводится в главе 5 «Приложения», в разделе 5.4.1 «Профилирование процессора».

Дополнительная информация

Команды для профилирования процессора и создания флейм-графиков представлены в разделе 6.6 «Инструменты наблюдения». Также см. раздел 5.4.1, посвященный анализу потребления процессора приложениями, и раздел 5.6 «Проблемы», в котором описаны общие проблемы профилирования, связанные с отсутствием трассировок стека и символов.

Для профилирования потребления определенных ресурсов процессора, например кэшей и внутренних соединений, вместо таймера можно использовать триггеры событий на основе счетчиков РМС. Этот прием описан в следующем разделе.

6.5.5. Анализ тактов

Для анализа потребления процессора на уровне тактов используйте счетчики мониторинга производительности (РМС). С их помощью можно определить, сколько тактов расходуется вхолостую из-за промахов кэшей 1-го, 2-го или 3-го уровня, операций ввода/вывода с памятью или ресурсами, операций с плавающей точкой либо других действий. Эта информация поможет выявить возможности улучшения производительности за счет настройки параметров компилятора или изменения кода.

Начните анализ с измерения IPC (или обратной величины CPI). Если у IPC низкое значение, продолжайте исследовать типы холостых тактов. Если IPC

имеет высокое значение, поищите в коде возможности уменьшить количество выполняемых инструкций. Понятия «высокое» и «низкое» значение IPC зависят от процессора, но вообще низкими можно считать значения меньше 0,2, а высокими — больше 1. Более полное представление об этих значениях вы получите, понаблюдав за известными рабочими нагрузками, которые активно используют память или выполняют интенсивные вычисления. Измерьте величину IPC для каждой.

Помимо измерения значений счетчики РМС можно настроить так, чтобы они прерывали работу ядра ОС при превышении заданного порога. Например, на каждые 10 000 промахов кэша уровня 3 ядро может прерываться для выборки трассировки стека. Со временем ядро создаст профиль путей в коде, вызывающих промахи в кэше 3-го уровня, без чрезмерного оверхеда на обработку каждого промаха. Эта возможность часто используется интегрированными средами разработки для аннотирования кода, который интенсивно использует память и вызывает выполнение холостых тактов.

Как рассказывалось в главе 4 «Инструменты наблюдения» в разделе 4.3.9, подразделе «Проблемы РМС», прием выборки при превышении порога может зафиксировать неверный адрес инструкции, вызвавшей событие, из-за эффекта сдвига и выполнения вне очереди. В архитектуре Intel эта проблема решается точной выборкой на основе событий (*precise event-based sampling*, PEBS), которая в Linux поддерживается утилитой `perf(1)`.

Анализ тактов — сложная задача, и ее решение с помощью инструментов командной строки может занять несколько дней. Подробнее об этом в разделе 6.6 «Инструменты наблюдения». Следует учесть и то, что придется потратить время на изучение руководств вашего производителя процессоров. Сэкономить время помогут инструменты анализа производительности — Intel vTune [Intel 20b] и AMD uProf [AMD 20], которые запрограммированы на поиск интересующих вас счетчиков РМС.

6.5.6. Мониторинг производительности

Мониторинг производительности может помочь обнаружить проблемы и модели поведения, проявляющиеся с течением времени. Ключевыми метриками для процессоров являются:

- **Потребление:** величина нагрузки в процентах.
- **Насыщенность:** длина очереди на выполнение или задержка планировщика.

Потребление следует контролировать для каждого CPU, чтобы не пропустить проблемы с масштабируемостью потоков. В средах с поддержкой ограничений или квот на потребление процессора, например облачных, потребление процессора следует оценивать с учетом этих ограничений.

Выбор правильного интервала для измерения и архивирования — это сложная задача при мониторинге потребления процессора. Некоторые инструменты мониторинга

используют пятиминутные интервалы, из-за которых более короткие всплески нагрузки могут быть незаметны. Посекундные измерения выглядят предпочтительнее, но знайте, что всплески могут длиться меньше одной секунды. Их можно определить по насыщенности и исследовать с помощью инструмента FlameScope (раздел 6.7.4), который был создан для анализа на уровне секунд.

6.5.7. Статическая настройка производительности

Статическая настройка производительности фокусируется на проблемах сконфигурированной архитектуры. Анализируя производительность процессора, изучите следующие аспекты статической конфигурации:

- Сколько процессоров доступно для использования? Являются ли они отдельными ядрами или аппаратными потоками?
- Доступны и используются ли GPU или другие ускорители?
- Архитектура одно- или многопроцессорная?
- Какой размер имеют кэши процессора? Используются ли они совместно?
- На какой тактовой частоте работает процессор? Изменяется ли она динамически (например, с помощью технологий Intel Turbo Boost и SpeedStep)? Включена ли поддержка этой возможности в BIOS?
- Какие другие особенности, связанные с процессором, включены или отключены в BIOS? Например, турбоускорение, настройки шины, настройки энергосбережения?
- Есть ли у этой модели процессора проблемы с производительностью? Перечислены ли они в списке известных проблем?
- Какая версия микрокода? Реализует ли она меры по снижению уязвимости (например, Spectre/Meltdown), влияющие на производительность?
- Есть ли проблемы с производительностью у этой версии прошивки BIOS?
- Есть ли программные ограничения на потребление процессора (средства управления ресурсами)? Какие?

Ответы на эти вопросы помогут выявить ранее упущенные из виду варианты конфигурации.

Последний вопрос особенно актуален для облачных сред, где потребление процессора обычно ограничивается провайдером.

6.5.8. Настройка приоритетов

Изначально в Unix есть системный вызов `nice()` для настройки приоритета процесса, который устанавливает уровень «уступчивости» (niceness). Положительные значения `nice` соответствуют низкому приоритету процесса (процесс становится более «уступчивым» по отношению к другим процессам), а отрицательные

значения, которые может установить только суперпользователь (`root`)¹, соответствуют более высокому приоритету. Есть и команда `nice(1)`, позволяющая запускать программы с заданным значением `nice`. Позже (в BSD) была добавлена команда `renice(1M)`, которая меняет значения `nice` для уже запущенных процессов. На странице справочного руководства `man` из 4-го издания Unix представлен следующий пример [TUHS 73]:

Для программ, которые должны выполняться продолжительное время и не привлекать внимания администратора, рекомендуется выбирать значение 16.

Значение `nice` — по-прежнему полезный инструмент для настройки приоритета процесса. Этот прием наиболее эффективен, когда есть конкуренция за обладание процессором, вызывающая большую задержку планировщика в высокоприоритетных задачах. Ваша цель — определить низкоприоритетную работу, к которой можно отнести работу агентов мониторинга и плановое резервное копирование, и запускать ее с подходящим значением `nice`. Также можно провести дополнительный анализ, чтобы убедиться в эффективности выбранной настройки и в том, что задержка планировщика остается низкой для высокоприоритетных задач.

Помимо значения `nice`, у ОС бывают дополнительные средства управления приоритетами процессов — изменение класса и политики планирования, а также настраиваемые параметры. В Linux поддерживается *класс планирования реального времени*, который позволяет процессам вытеснять любые другие задачи. Это помогает устранить задержку планировщика (только не за счет других процессов реального времени и прерываний), но убедитесь, что вы понимаете последствия. Если в приложении реального времени обнаружится ошибка, из-за которой несколько потоков войдут в бесконечный цикл, это приведет к тому, что никакие другие процессы не смогут продолжить свою работу, включая административную оболочку, необходимую для устранения проблемы вручную².

6.5.9. Управление ресурсами

Операционная система может предоставлять низкоуровневые средства управления для распределения тактов процессора между процессами или группами процессов, позволяющие устанавливать фиксированные ограничения на потребление процессора и реализовывать более гибкий подход к использованию холостых тактов процессора на основе потребления. Как это работает, зависит от реализации и рассматривается в разделе 6.9 «Настройка».

¹ Начиная с Linux 2.6.12, «потолок `nice`» можно изменить для каждого процесса, что позволяет процессам с привилегиями простого пользователя устанавливать более низкие значения `nice`. Например, с помощью: `prlimit --nice = -19 -p PID`.

² В Linux, начиная с версии 2.6.25, появилось решение этой проблемы: параметр `RLIMIT_RTTIME`, который устанавливает ограничение процессорного времени в микросекундах, которое поток реального времени может потребить до выполнения блокирующего системного вызова.

6.5.10. Привязка к процессору

Еще один способ настройки производительности процессора основан на привязке процессов и потоков к отдельным процессорам или их коллекциям. Это может увеличить разогрев кэша процессора для процесса и за счет этого повысить производительность операций чтения/записи с памятью. В системах NUMA привязка к процессору также улучшает локальность памяти и способствует улучшению производительности.

Есть два способа:

- **Связывание процессора (CPU binding):** настройка процесса для запуска только на одном процессоре или на любом процессоре из определенного набора.
- **Определение исключительных наборов процессоров:** выделение набора процессоров, которые могут использоваться только определенными процессами. Это может дополнительно увеличить разогрев кэша процессора, потому что пока назначенный процесс простаивает, другие процессы не могут использовать эти процессоры.

В системах на базе Linux подход с определением исключительных наборов процессоров можно реализовать с использованием *cpuset*s. Примеры конфигурации показаны в разделе 6.9 «Настройка».

6.5.11. Микробенчмаркинг

Инструменты для микробенчмаркинга производительности CPU обычно измеряют время, затрачиваемое на многократное выполнение простой операции. Такой операцией могут быть:

- **Машинные инструкции:** целочисленная арифметика, операции с плавающей точкой, извлечение данных из памяти и их сохранение в память, ветвления и другие инструкции.
- **Доступ к памяти:** для исследования задержки различных кэшей процессора и пропускной способности основной памяти.
- **Инструкции на языках высокого уровня:** сродни тестированию с использованием машинных инструкций, с той лишь разницей, что операции написаны на интерпретируемом или компилируемом языке высокого уровня.
- **Операции операционной системы:** для тестирования функций системных библиотек и системных вызовов, создающих преимущественно вычислительную нагрузку, например *getpid(2)* и создание процесса.

Ранними примерами бенчмарка CPU можно назвать Whetstone, написанный в Национальной физической лаборатории (National Physical Laboratory) в 1972 году на языке Algol 60 и предназначенный для моделирования рабочей нагрузки, характерной для научных вычислений. В 1984 году был создан бенчмарк Dhrystone для моделирования вычислительных рабочих нагрузок с целочисленной арифметикой, ставший в то время популярным средством сравнения производительности. Эти

и множество других бенчмарков для Unix, включая создание процессов и тестирование пропускной способности конвейера, были включены в сборник *UnixBench*, первоначально созданный в Университете Монаша (Monash University) и опубликованный журналом *BYTE* [Hinnant 84]. Более современные бенчмарки тестируют скорости сжатия, вычисления простых чисел, шифрования и кодирования.

Независимо от выбранного бенчмарка, сравнивая результаты тестирования разных систем, важно понимать, что тестируется на самом деле. Подобные бенчмарки часто сводятся к тестированию различий в оптимизации между версиями компилятора, а не кода теста или скорости процессора. Многие бенчмарки выполняются в однопоточном режиме, и их результаты теряют смысл в многопроцессорных системах. (Система с четырьмя процессорами может сделать бенчмарки немного быстрее, чем система с восемью процессорами, но последняя, вероятно, обеспечит гораздо более высокую пропускную способность при наличии достаточного количества параллельно выполняемых потоков.)

Подробную информацию по теме см. в главе 12 «Бенчмаркинг».

6.6. ИНСТРУМЕНТЫ НАБЛЮДЕНИЯ

В этом разделе представлены инструменты наблюдения за производительностью процессора, доступные в ОС на базе Linux. Методологии их использования описываются в предыдущем разделе.

Инструменты перечислены в табл. 6.8.

Таблица 6.8. Инструменты наблюдения за производительностью процессора в Linux

Раздел	Инструмент	Описание
6.6.1	uptime	Средние значения нагрузки
6.6.2	vmstat	Общесистемные средние значения нагрузки на процессор
6.6.3	mpstat	Статистики по процессорам
6.6.4	sar	Исторические статистики
6.6.5	ps	Состояние процессов
6.6.6	top	Мониторинг потребления процессора по процессам/потокам
6.6.7	pidstat	Разбивка потребления процессора по процессам/потокам
6.6.8	time, ptime	Время выполнения команд с разбивкой по процессорам
6.6.9	turbobust	Показывает тактовую частоту процессора и другие состояния
6.6.10	showboost	Показывает тактовую частоту процессора и турбоускорение
6.6.11	pmcarch	Показывает общее потребление тактов процессора
6.6.12	tibstat	Обобщает информацию о тактах, затраченных на обслуживание буфера ассоциативной трансляции (TLB)

Таблица 6.8 (окончание)

Раздел	Инструмент	Описание
6.6.13	perf	Профилирование процессора и анализ счетчиков РМС
6.6.14	profile	Выборка трассировок стека
6.6.15	cpudist	Обобщает информацию о времени выполнения на процессоре
6.6.16	runqlat	Обобщает информацию о задержках в очередях на выполнение
6.6.17	runqlen	Обобщает информацию о длине очередей на выполнение
6.6.18	softirqs	Обобщает время на обработку программных прерываний
6.6.19	hardirqs	Обобщает время на обработку аппаратных прерываний
6.6.20	bpfftrace	Трассировка программ при анализе потребления процессора

Это набор инструментов и возможностей для поддержки раздела 6.5 «Методология». Начнем с традиционных инструментов получения статистик потребления процессора, а затем перейдем к инструментам для более глубокого анализа на основе профилирования путей в коде, анализа тактов процессора и инструментов трассировки. Некоторые из традиционных инструментов доступны (а иногда первоначально созданы) в других Unix-подобных ОС, в том числе: `uptime(1)`, `vmstat(8)`, `mpstat(1)`, `sar(1)`, `ps(1)`, `top(1)` и `time(1)`. В число инструментов трассировки на основе BPF и интерфейсов BCC и `bpfftrace` (глава 15) входят: `profile(8)`, `cpudist(8)`, `runqlat(8)`, `runqlen(8)`, `softirqs(8)` и `hardirqs(8)`.

Полный перечень возможностей каждого инструмента ищите в документации, в том числе в справочном руководстве `man`.

6.6.1. uptime

`uptime(1)` — одна из нескольких команд, позволяющих получить *средние значения нагрузки*:

```
$ uptime
 9:04pm up 268 day(s), 10:16,  2 users,  load average: 7.76, 8.32, 8.60
```

Последние три числа — это средние значения нагрузки за последние 1, 5 и 15 минут. Сравнивая эти числа, можно определить, как менялась нагрузка в последние 15 минут: увеличивалась, уменьшалась или оставалась неизменной. Эта информация может пригодиться, когда вы реагируете на проблему производительности в промышленной системе и обнаруживаете, что нагрузка уменьшается, что может свидетельствовать о незамеченной проблеме. Если нагрузка увеличивается, то проблема может усугубиться!

В следующих разделах я подробно расскажу о средних значениях нагрузки. Но они — лишь отправная точка, поэтому не тратьте на них больше 5 минут, прежде чем перейти к другим метрикам.

Средние значения нагрузки

Средние значения нагрузки указывают на потребность в системных ресурсах: чем выше средняя нагрузка, тем выше потребность. В некоторых ОС (например, Solaris) средние значения нагрузки показывают нагрузку на процессор, как и ранние версии Linux. Но в 1993 году подход к измерению средней нагрузки в Linux изменился, и теперь средние значения нагрузки в этой системе отражают общесистемную потребность: процессоров, дисков и других ресурсов¹. Эта особенность была реализована путем включения потоков в состояние TASK_UNINTERRUPTIBLE, которое показывается некоторыми инструментами как состояние «D» (оно упоминалось в главе 5 «Приложения» в разделе 5.4.5 «Анализ состояния потока»).

Нагрузка измеряется как текущее использование (потребление) ресурсов плюс запросы в очереди (насыщенность). Представьте пункт оплаты проезда на платной дороге: вы можете измерить нагрузку на разные пункты в течение дня, подсчитав количество обслуженных машин (нагрузка) плюс количество машин, стоявших в очереди (насыщенность).

Среднее — это экспоненциально затухающее скользящее среднее, отражающее нагрузку за последние 1, 5 и 15 минут (времена — это фактически константы, используемые при расчете экспоненциальной скользящей суммы [Myer 73]). На рис. 6.17 показаны результаты простого эксперимента, в ходе которого был запущен единственный поток, создающий вычислительную нагрузку, и средние значения нагрузки.

К 1-, 5- и 15-минутным отметкам средние значения нагрузки достигли примерно 61 % от известной нагрузки, равной 1,0.

Средние значения нагрузки были введены в Unix на ранних этапах развития BSD и вычислялись на основе средней длины очереди планировщика и средних нагрузок, использовавшихся в более ранних ОС (CTSS, Multics [Saltzer 70], TENEX [Bobrow 72]). Они были описаны в RFC 546 [Thomas 73]:

[1] Средняя нагрузка в TENEX — это мера потребности процессора. Средняя нагрузка — это среднее количество готовых к выполнению процессов за определенный период времени. Например, значение 10 средней часовой нагрузки будет означать, что (для системы с одним ЦП) в любое время в течение часа можно ожидать, что один процесс будет выполняться, а девять других будут готовы к выполнению (то есть не заблокированы в операции ввода/вывода), ожидая своей очереди выполнения на процессоре.

¹ Это изменение произошло так давно, что причина уже забылась и документальных свидетельств не осталось. Это был период до переноса Linux в git и на другие ресурсы. В конце концов я нашел оригинальный патч в онлайн-архиве старой почтовой системы. Это изменение внес Матиас Урлихс (Matthias Urlichs), указав, что при перемещении потребности с процессоров на диски средние значения нагрузки должны оставаться прежними, потому что потребность не изменилась [Gregg 17c]. Я задал ему вопрос (это было мое первое письмо ему), касающийся этого изменения, сделанного 24 года тому назад, и уже через час получил ответ!

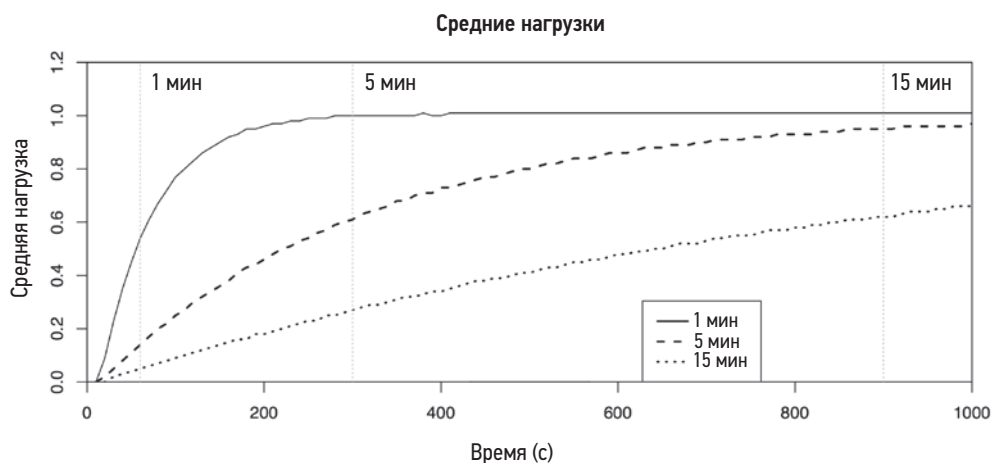


Рис. 6.17. Экспоненциально затухающее скользящее среднее

Обратимся к современному примеру и рассмотрим систему с 64 процессорами и средней нагрузкой 128. Если бы нагрузка была только нагрузкой на процессор, это значило бы, что в среднем на каждом процессоре всегда выполняется один поток и есть по одному потоку в очереди на выполнение для каждого процессора. Для той же системы средняя нагрузка 20 будет указывать на значительный запас, потому что эта система способна запустить еще 44 потока до того, как все процессоры окажутся заняты. (Некоторые компании отслеживают нормализованную метрику средней нагрузки, которая автоматически делится на количество процессоров. Это позволяет интерпретировать ее, не зная количества имеющихся процессоров.)

Pressure Stall Information (PSI)

В первом издании я описал, как можно предоставить средние значения нагрузки для ресурсов каждого типа, чтобы упростить интерпретацию. Теперь в этом нет нужды, потому что в Linux 4.20 появился интерфейс, обеспечивающий такую разбивку, — Pressure Stall Information (PSI). Он позволяет получить средние значения для процессора, памяти и ввода/вывода. Среднее значение показывает процент времени, в течение которого потоки простаивали в ожидании доступности ресурса (только при насыщении). В табл. 6.9 эта информация о простоях сравнивается со средними значениями нагрузки.

Таблица 6.9. Сравнение средних значений нагрузки и информации о простоях в Linux

Атрибут	Средние значения нагрузки	Информация о простоях
Ресурсы	Для системы в целом	Процессор, память, ввод/вывод (все по отдельности)

Атрибут	Средние значения нагрузки	Информация о простоях
Метрика	Количество задач, выполняющихся и ожидающих выполнения	Процент времени, потраченного на ожидание
Времена	1 мин, 5 мин, 15 мин	10 с, 60 с, 300 с
Среднее	Экспоненциально затухающее скользящее среднее	Экспоненциально затухающее скользящее среднее

В табл. 6.10 приводятся примеры значений этой метрики в разных сценариях.

Таблица 6.10. Примеры средних значений нагрузки и информации о простоях в Linux

Сценарий	Средние значения нагрузки	Информация о простоях
2 процессора, 1 поток вычислительной нагрузки	1,0	0,0
2 процессора, 2 потока вычислительной нагрузки	2,0	0,0
2 процессора, 3 потока вычислительной нагрузки	3,0	50,0
2 процессора, 4 потока вычислительной нагрузки	4,0	100,0
2 процессора, 5 потоков вычислительной нагрузки	5,0	100,0

Вот пример получения метрик в сценарии с двумя процессорами и тремя потоками вычислительной нагрузки:

```
$ uptime
 07:51:13 up 4 days, 9:56, 2 users, load average: 3.00, 3.00, 2.55
$ cat /proc/pressure/cpu
some avg10=50.00 avg60=50.00 avg300=49.70 total=1031438206
```

Значение 50,0 означает, что потоки («some» — «некоторые») простаивали 50 % времени. При выводе информации о ресурсах ввода/вывода (**io**) и памяти (**memory**) добавляется вторая строка («full» — «все»), когда простаивали все потоки, не являющиеся потоками бездействия. Метрики PSI фактически отвечают на вопрос: насколько вероятно, что задача будет простаивать в ожидании ресурсов?

Какую бы метрику вы ни использовали, среднюю нагрузку или PSI, старайтесь быстрее перейти к метрикам, характеризующим нагрузку более подробно, например к метрикам, возвращаемым инструментами `vmstat(1)` и `mpstat(1)`.

6.6.2. vmstat

Команда получения статистик использования виртуальной памяти `vmstat(8)` выводит в последних столбцах средние значения нагрузки на процессор для всей системы и количество выполняемых потоков в первом столбце. Вот пример вывода версии для Linux:

```
$ vmstat 1
procs -----memory----- ---swap-- ----io---- -system-- -----cpu-----
 r  b   swpd   free   buff   cache   si   so   bi   bo   in   cs   us   sy   id   wa   st
15  0       0 451732  70588 866628    0    0    1   10   43   38   2   1  97   0   0
15  0       0 450968  70588 866628    0    0    0  612 1064 2969 72 28   0   0   0
15  0       0 450660  70588 866632    0    0    0    0  961 2932 72 28   0   0   0
15  0       0 450952  70588 866632    0    0    0    0 1015 3238 74 26   0   0   0
[...]
```

В первой строке должна выводиться обобщенная информация с момента загрузки. Но в Linux вывод в столбцах **procs** и **memory** начинается со значений, характеризующих текущее состояние. (Возможно, когда-то это исправят.) С процессором связаны столбцы:

- **r**: длина очереди на выполнение — общее количество потоков, готовых к выполнению.
- **us**: процент времени выполнения в режиме пользователя.
- **sy**: процент времени выполнения в режиме системы (ядра).
- **id**: процент времени бездействия.
- **wa**: процент времени, проведенного потоками в ожидании завершения ввода/вывода, когда потоки оставались заблокированными на время дискового ввода/вывода.
- **st**: процент заимствованного времени — время, потраченное на обслуживание других клиентов в виртуализированных средах.

Все эти значения — общесистемные средние значения для всех процессоров, кроме **r**. Это суммарное значение.

В Linux столбец **r** — это общее количество задач, ожидающих выполнения *плюс* выполняющихся. В других ОС (например, Solaris) в столбце **r** отображаются только задачи, ожидающие выполнения, но не выполняющиеся. Инструмент **vmstat(1)** создали Билл Джой (Bill Joy) и Озалп Бабаоглу (Ozalp Babaoglu) для 3BSD в 1979 году, начав со столбца **RQ**, в котором выводилось количество готовых к выполнению *и* выполняющихся процессов, как это делает в настоящее время **vmstat(8)** в Linux.

6.6.3. mpstat

Инструмент **mpstat(1)** может сообщать статистику по процессорам. Вот пример вывода версии для Linux:

```
$ mpstat -P ALL 1
Linux 5.3.0-1009-aws (ip-10-0-239-218) 02/01/20      _x86_64_      (2 CPU)

18:00:32 CPU   %usr   %nice   %sys %iowait  %irq   %soft  %steal  %guest  %gnice   %idle
18:00:33 all  32.16    0.00  61.81    0.00    0.00    0.00    0.00    0.00    0.00    6.03
18:00:33  0  32.00    0.00  64.00    0.00    0.00    0.00    0.00    0.00    0.00    4.00
18:00:33  1  32.32    0.00  59.60    0.00    0.00    0.00    0.00    0.00    0.00    8.08
```



```

18:00:33 CPU    %usr    %nice    %sys %iowait  %irq    %soft  %steal  %guest  %gnice   %idle
18:00:34 all   33.83    0.00   61.19    0.00    0.00    0.00    0.00    0.00    0.00    4.98
18:00:34  0   34.00    0.00   62.00    0.00    0.00    0.00    0.00    0.00    0.00    4.00
18:00:34  1   33.66    0.00   60.40    0.00    0.00    0.00    0.00    0.00    0.00    5.94
[...]
```

Параметр `-P ALL` был использован для вывода отчета по каждому процессору. По умолчанию `mpstat(1)` выводит только сводную информацию для всей системы (строка `all`). Вот расшифровка столбцов:

- **CPU**: логический идентификатор процессора или `all` — в строке со сводной информацией.
- **%usr**: время работы в режиме пользователя, исключая `%nice`.
- **%nice**: время работы в режиме пользователя для процессов с пониженным приоритетом.
- **%sys**: время работы в режиме системы (ядра).
- **%iowait**: ожидание завершения ввода/вывода.
- **%irq**: потребление процессора аппаратными прерываниями.
- **%soft**: потребление процессора программными прерываниями.
- **%steal**: время, потраченное на обслуживание других пользователей.
- **%guest**: время, потраченное в гостевых виртуальных машинах.
- **%gnice**: время для выполнения гостя с пониженным приоритетом.
- **%idle**: время бездействия.

Ключевые столбцы: `%usr`, `%sys` и `%idle`. Они определяют потребление процессоров и показывают соотношение времени в режиме пользователя/ядра (см. раздел 6.3.9 «Время в режиме пользователя/ядра»). С их помощью можно определить и «горячие» процессоры — работающие со 100 %-ной нагрузкой (`%usr + %sys`), когда нагрузка на другие меньше, — что может быть обусловлено работой однопоточных приложений или отображением прерываний устройств.

Обратите внимание, что время, сообщаемое этим и другими инструментами, которые используют ту же статистику ядра (`/proc/stat` и т. д.), и точность этих статистик зависит от конфигурации ядра. См. подраздел «Точность статистик процессора» в главе 4 «Инструменты наблюдения» в разделе 4.3.1 «`/proc`».

6.6.4. sar

Генератор отчетов о работе системы (system activity reporter) `sar(1)` используют для наблюдения за текущей активностью. Его можно настроить для архивирования статистик и генерирования хронологических отчетов. Он был представлен в главе 4 «Инструменты наблюдения» в разделе 4.4 «`sar`» и упоминается в других главах, где это необходимо.

Версия для Linux предлагает следующие возможности для анализа производительности процессора:

- **-P ALL:** то же, что и `mpstat -P ALL`;
- **-u:** выводит ту же информацию, что и `mpstat(1)` без параметров, — только среднее общесистемное значение;
- **-q:** включает размер очереди на выполнение как `runq-sz` (ту же информацию включает в вывод параметр `r` утилиты `vmstat(1)`) и средние значения нагрузки.

Можно настроить `sar(1)` для сбора данных, чтобы исследовать эти данные позже. За более подробной информацией обращайтесь к разделу 4.4 «sar».

6.6.5. ps

Команда вывода состояния процессов `ps(1)` может отображать подробную информацию обо всех процессах, включая статистику потребления процессора. Например:

```
$ ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root             1  0.0  0.0  23772  1948 ?        Ss   2012   0:04 /sbin/init
root             2  0.0  0.0      0     0 ?        S    2012   0:00 [kthreadd]
root             3  0.0  0.0      0     0 ?        S    2012   0:26 [ksoftirqd/0]
root             4  0.0  0.0      0     0 ?        S    2012   0:00 [migration/0]
root             5  0.0  0.0      0     0 ?        S    2012   0:00 [watchdog/0]
[...]
web          11715 11.3  0.0 632700 11540 pts/0    Sl   01:36   0:27 node indexer.js
web          11721 96.5  0.1 638116 52108 pts/1    Rl+  01:37   3:33 node proxy.js
[...]
```

Этот стиль использования возник из BSD, о чем говорит отсутствие дефиса перед параметрами `aux`. Параметры требуют перечислить всех пользователей (`a`), вывести расширенные сведения о пользователях (`u`) и включить в список процессы без терминала (`x`). Терминал отображается в столбце `TTY`.

В другом стиле, отличном от SVR4, параметрам предшествует дефис:

```
$ ps -ef
UID          PID    PPID  C  STIME TTY          TIME CMD
root             1      0  0 Nov13 ?        00:00:04 /sbin/init
root             2      0  0 Nov13 ?        00:00:00 [kthreadd]
root             3      2  0 Nov13 ?        00:00:00 [ksoftirqd/0]
root             4      2  0 Nov13 ?        00:00:00 [migration/0]
root             5      2  0 Nov13 ?        00:00:00 [watchdog/0]
[...]
```

Эти параметры требуют перечислить все процессы (`-e`) с полной информацией (`-f`). `ps(1)` поддерживает множество других параметров, включая `-o` для настройки вывода и определения перечня отображаемых столбцов.

Ключевые столбцы для анализа производительности процессора — `TIME` и `%CPU` (предыдущий пример).

В столбце **TIME** отображается общее процессорное время, потраченное процессом (в режиме пользователя и ядра) с момента его создания, в часах:минутах:секундах.

В Linux столбец **%CPU** из первого примера показывает среднюю нагрузку на CPU за все время работы процесса, суммированную по всем CPU. Для однопоточного вычислительного процесса в этом столбце будет выводиться 100 %, для двухпоточного вычислительного процесса — 200 %. Другие ОС могут нормализовать процент процессорного времени количеством CPU, чтобы максимальное значение не превышало 100 %, и показывать только недавнее или текущее потребление процессора, а не среднее значение за все время выполнения. В Linux увидеть текущее потребление CPU процессами можно с помощью **top(1)**.

6.6.6. top

Утилита **top(1)** была создана Уильямом Лефевром (William LeFebvre) в 1984 году для BSD. Его вдохновила команда **MONITOR PROCESS/TOPCPU** в VMS, которая показывала задания с наибольшим потреблением процессора в виде ASCII-гистограммы (не в виде таблицы).

Команда **top(1)** наблюдает за запущенными процессами, регулярно обновляя информацию на экране. Вот пример, полученный в Linux:

```
$ top
top - 01:38:11 up 63 days,  1:17,  2 users,  load average: 1.57, 1.81, 1.77
Tasks: 256 total,  2 running, 254 sleeping,  0 stopped,  0 zombie
Cpu(s):  2.0%us,  3.6%sy,  0.0%ni, 94.2%id,  0.0%wa,  0.0%hi,  0.2%si,  0.0%st
Mem: 49548744k total, 16746572k used, 32802172k free, 182900k buffers
Swap: 100663292k total,      0k used, 100663292k free, 14925240k cached

  PID USER      PR  NI  VIRT  RES  SHR  S %CPU  %MEM    TIME+  COMMAND
11721 web       20   0  623m  50m 4984 R   93   0.1   0:59.50 node
11715 web       20   0  619m  20m 4916 S   25   0.0   0:07.52 node
   10 root       20   0      0    0   0 S    1   0.0 248:52.56 ksoftirqd/2
   51 root       20   0      0    0   0 S    0   0.0   0:35.66 events/0
11724 admin     20   0 19412 1444  960 R    0   0.0   0:00.07 top
    1 root       20   0 23772 1948 1296 S    0   0.0   0:04.35 init
```

В верхней части выводится сводная информация о системе в целом, а ниже — список процессов/задач, по умолчанию отсортированный по убыванию потребления процессорного времени. Общесистемная сводка включает средние значения нагрузки и состояния CPU: **%us, %sy, %ni, %id, %wa, %hi, %si, %st**. Эти состояния эквивалентны тем, которые выводит утилита **mpstat(1)**, как было описано выше, и они усредняются по всем процессорам.

Потребление CPU видно в столбцах **TIME** и **%CPU**. **TIME** — это общее процессорное время, потребляемое процессом с точностью до сотых долей секунды. Например, «1:36.53» означает, что в общей сложности процесс получил 1 минуту 36.53 секунды процессорного времени. Некоторые версии **top(1)** дополнительно выводят «накопленное время», включающее процессорное время, полученное уже завершившимися дочерними процессами.

Столбец %CPU показывает общую нагрузку на CPU за текущий интервал между обновлениями экрана. В Linux эта величина не нормализуется количеством CPU, поэтому для процесса с двумя вычислительными потоками `top(1)` будет сообщать значение 200 %. В терминологии `top(1)` этот режим называется «режимом Irix», в память о таком поведении утилиты в системе IRIX. Его можно переключить в «режим Solaris» (нажав клавишу `T`), в котором нагрузка на CPU делится на количество CPU. В этом случае для процесса с двумя вычислительными потоками на сервере с 16 процессорами `top(1)` будет сообщать значение 12,5 %.

`top(1)` часто используется начинающими специалистами по анализу производительности. Важно знать, что утилита `top(1)` сама может оказаться в числе наиболее существенных потребителей процессорного времени в одной из верхних строчек в списке процессов, который выводит `top(1)`. Это обусловлено частым обращением к системным вызовам, которые используются для чтения информации о множестве процессов из `/proc` — `open(2)`, `read(2)`, `close(2)`. Некоторые версии `top(1)` в других ОС стремятся сократить оверхед, оставляя файловые дескрипторы открытыми и вызывая `pread(2)`.

Есть вариант `top(1)` под названием `htop(1)`, который поддерживает больше интерактивных возможностей, настроек и вывод ASCII-гистограмм, иллюстрирующих нагрузку на процессор. Но эта утилита вызывает в 4 раза больше системных вызовов, внося еще большее возмущение в систему. Я редко пользуюсь ею.

Поскольку утилита `top(1)` создает моментальные снимки `/proc`, то может пропустить короткоживущие процессы, которые успевают запуститься и завершиться между моментами создания снимков. Такое часто случается во время сборки ПО, когда CPU могут быть сильно нагружены множеством короткоживущих процессов, запускаемых инструментами сборки. Вариант `top(1)` для Linux с названием `atop(1)` использует механизм учета процессов, чтобы не пропустить короткоживущие процессы и включить их в отображаемую таблицу.

6.6.7. pidstat

Инструмент `pidstat(1)` в Linux выводит данные о потреблении CPU по процессам или потокам, в том числе с разбивкой по времени в режиме пользователя/ядра. По умолчанию в интервальный вывод включаются только активные процессы. Например:

```
$ pidstat 1
Linux 2.6.35-32-server (dev7)  11/12/12      _x86_64_      (16 CPU)

22:24:42      PID    %usr %system %guest   %CPU  CPU Command
22:24:43      7814    0.00   1.98   0.00   1.98    3 tar
22:24:43      7815   97.03   2.97   0.00  100.00   11 gzip

22:24:43      PID    %usr %system %guest   %CPU  CPU Command
22:24:44      448    0.00   1.00   0.00   1.00    0 kjournald
22:24:44      7814    0.00   2.00   0.00   2.00    3 tar
22:24:44      7815   97.00   3.00   0.00  100.00   11 gzip
22:24:44      7816    0.00   2.00   0.00   2.00    2 pidstat
[...]
```

Этот пример был получен в момент, когда выполнялось резервное копирование системы с использованием команды `tar(1)` для чтения файлов из файловой системы и команды `gzip(1)` для их сжатия. Время в режиме пользователя для `gzip(1)`, как и ожидалось, оказалось довольно большим из-за преимущественно вычислительной природы реализации алгоритма сжатия. Команда `tar(1)` большую часть времени выполнялась в режиме ядра, читая данные из файловой системы.

Для вывода всех процессов, включая простаивающие, можно использовать параметр `-p ALL`. Параметр `-t` выводит статистики по потокам. Описание других параметров `pidstat(1)` приводится в других главах этой книги по мере необходимости.

6.6.8. time, ptime

Команду `time(1)` можно использовать для запуска программ и получения информации о потреблении ими процессора. Эта команда входит в состав ОС и доступна в виде выполняемого файла в каталоге `/usr/bin` и в виде встроенной команды командной оболочки.

В примере ниже команда `time` дважды используется для запуска утилиты `cksum(1)`, которая вычисляет контрольную сумму большого файла:

```
$ time cksum ubuntu-19.10-live-server-amd64.iso
1044945083 883949568 ubuntu-19.10-live-server-amd64.iso

real 0m5.590s
user 0m2.776s
sys 0m0.359s
$ time cksum ubuntu-19.10-live-server-amd64.iso
1044945083 883949568 ubuntu-19.10-live-server-amd64.iso

real 0m2.857s
user 0m2.733s
sys 0m0.114s
```

В первом случае утилите `cksum(1)` потребовалось 5.6 с, чтобы вычислить контрольную сумму, из которых 2.8 с пришлось на выполнение в режиме пользователя. Время выполнения в режиме ядра составило 0.4 с — столько времени потребовалось системным вызовам для чтения файла. Недостающие 2.4 с ($5.6 - 2.8 - 0.4$) — это, скорее всего, время ожидания на блокировках, пока завершится ввод/вывод с диска, потому что этот файл был кэширован лишь частично. Вторая попытка завершилась быстрее, за 2.9 с, почти без ожидания на блокировках. Это вполне ожидаемо, потому что файл мог быть полностью кэширован в основной памяти на первой попытке.

Версия `/usr/bin/time` в Linux поддерживает вывод более подробных сведений. Например:

```
$ /usr/bin/time -v cp fileA fileB
Command being timed: "cp fileA fileB"
User time (seconds): 0.00
System time (seconds): 0.26
Percent of CPU this job got: 24%
```

```

Elapsed (wall clock) time (h:mm:ss or m:ss): 0:01.08
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 3792
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 0
Minor (reclaiming a frame) page faults: 294
Voluntary context switches: 1082
Involuntary context switches: 1
Swaps: 0
File system inputs: 275432
File system outputs: 275432
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0

```

Версия, встроенная в командную оболочку, обычно не поддерживает параметр `-v`.

6.6.9. turbostat

`turbostat(1)` — это инструмент, основанный на использовании регистров, зависящих от модели (*model-specific registers*, MSR). Он показывает состояние процессоров и обычно доступен в пакете `linux-tools-common`. Регистры MSR упоминались в главе 4 «Инструменты наблюдения» в разделе 4.3.10 «Другие источники информации для наблюдения». Вот пример вывода:

```

# turbostat
turbostat version 17.06.23 - Len Brown <lenb@kernel.org>
CUID(0): GenuineIntel 22 CUID levels; family:model:stepping 0x6:8e:a (6:142:10)
CUID(1): SSE3 MONITOR SMX EIST TM2 TSC MSR ACPI-TM TM
CUID(6): APERF, TURBO, DTS, PTM, HWP, HWPnotify, HWPwindow, HWPcapp, No-HWPpkg, EPB
cpu0: MSR_IA32_MISC_ENABLE: 0x00850089 (TCC EIST No-MWAIT PREFETCH TURBO)
CUID(7): SGX
cpu0: MSR_IA32_FEATURE_CONTROL: 0x00040005 (Locked SGX)
CUID(0x15): eax_crystal: 2 ebx_tsc: 176 ecx_crystal_hz: 0
TSC: 2112 MHz (24000000 Hz * 176 / 2 / 1000000)
CUID(0x16): base_mhz: 2100 max_mhz: 4200 bus_mhz: 100
[...]

Core   CPU    Avg_MHz  Busy%   Bzy_MHz  TSC_MHz  IRQ    SMI    C1     C1E
      C3     C6       C7s     C8       C9       C10    C1%    C1E%   C3%
      C6%    C7s%     C8%     C9%     C10%    CPU%c1  CPU%c3 CPU%c6 CPU%c7
CoreTmp PkgTmp  GFX%rc6 GFXMHz  Totl%C0 Any%C0  GFX%C0 CPUGFX% Pkg%pc2
Pkg%pc3 Pkg%pc6 Pkg%pc7 Pkg%pc8 Pkg%pc9 Pk%pc10 PkgWatt CorWatt GFXWatt
RAMWatt PKG_%  RAM_%

[...]
0      0      97       2.70    3609     2112    1370   0      41     293
      41     453     0       693     0       311    0.24   1.23   0.15
      5.35   0.00    39.33   0.00    50.97    7.50   0.18   6.26   83.37

```

52	75	91.41	300	118.58	100.38	8.47	8.30	0.00
0.00	0.00	0.00	0.00	0.00	0.00	17.69	14.84	0.65
1.23	0.00	0.00						

[...]

Сначала `turbostat(8)` выводит информацию о процессоре и регистрах MSR, которая может включать более 50 строк (здесь вывод показан не полностью). Затем с пяти-секундным интервалом по умолчанию выводятся интервальные сводки метрик для всех процессоров вместе и каждого в отдельности. В этом примере строка с интервальной сводной информацией имеет длину 389 символов и переносилась пять раз, что затрудняет чтение. В число столбцов входят: номер процессора (CPU), средняя тактовая частота в мегагерцах (`Avg_MHz`), информация о C-состоянии, температура (`*Tmp`) и мощность (`*Watt`).

6.6.10. showboost

До появления `turbostat(8)` в облачной среде Netflix я разработал инструмент `showboost(1)`, отображающий тактовую частоту процессора с разбивкой по интервалам. Название `showboost(1)` — это сокращение от «show turbo boost», и этот инструмент тоже использует регистры MSR. Вот пример вывода:

```
# showboost
Base CPU MHz : 3000
Set CPU MHz  : 3000
Turbo MHz(s) : 3400 3500
Turbo Ratios : 113% 116%
CPU 0 summary every 1 seconds...
```

TIME	C0_MCYC	C0_ACYC	UTIL	RATIO	MHz
21:41:43	3021819807	3521745975	100%	116%	3496
21:41:44	3021682653	3521564103	100%	116%	3496
21:41:45	3021389796	3521576679	100%	116%	3496

[...]

Как показывает этот вывод, процессор CPU0 работает на тактовой частоте 3496 МГц. Базовая частота процессора составляет 3000 МГц: она повышается до 3496 с помощью Intel Turbo Boost. Возможные уровни турбоускорения, или «шаги», тоже перечислены в выводе: 3400 и 3500 МГц.

`showboost(8)` находится в моем репозитории `msr-cloud-tools` [Gregg 20d], названном так потому, что хранит инструменты для использования в облачной среде. Поскольку я продолжаю работать только в среде Netflix, `showboost(8)` может не работать в другом месте из-за различий в процессорах. Если вы столкнетесь с этим, то попробуйте `turboboost(1)`.

6.6.11. pmcarch

`pmcarch(8)` помогает получить общее представление о производительности на уровне тактов процессора. Этот инструмент основан на счетчиках РМС, входящих

в «архитектурный набор» Intel, что объясняет название инструмента (счетчики РМС были описаны в главе 4 «Инструменты наблюдения», в разделе 4.3.9 «Аппаратные счетчики (РМС)»). В некоторых облачных средах счетчики из архитектурного набора — единственно доступные (например, в некоторых экземплярах AWS EC2). Вот пример вывода:

```
# pmcarch
K_CYCLES   K_INSTR    IPC BR_RETIRED  BR_MISPRED  BMR% LLCREF   LLCMISS  LLC%
96163187   87166313   0.91 19730994925  679187299   3.44 656597454  174313799  73.45
93988372   87205023   0.93 19669256586  724072315   3.68 666041693  169603955  74.54
93863787   86981089   0.93 19548779510  669172769   3.42 649844207  176100680  72.90
93739565   86349653   0.92 19339320671  634063527   3.28 642506778  181385553  71.77
[...]
```

Инструмент выводит необработанное содержимое счетчиков, а также некоторые отношения в процентах. Вот значение некоторых столбцов:

- **K_CYCLES**: тактов процессора $\times 1000$;
- **K_INSTR**: инструкций процессора $\times 1000$;
- **IPC**: количество инструкций на такт;
- **BMR%**: доля ошибочно предсказанного выбора ветвей в процентах;
- **LLC%**: процент попаданий в кэш последнего уровня в процентах.

О метрике IPC говорилось в разделе 6.3.7 «IPC, CPI», и там же приводились примеры значений. Другие предоставленные отношения, **BMR%** и **LLC%**, дают некоторое представление о том, почему значение IPC может быть низким и чем обусловлено выполнение холостых тактов.

Я разработал `pmcarch(8)` для своего репозитория `pmc-cloud-tools`, в котором также есть инструмент `srcusache(8)`, позволяющий получить дополнительные статистики, характеризующие использование кэш-памяти процессора [Gregg 20e]. В этих инструментах используются обходные приемы и счетчики РМС, зависящие от процессора, поэтому они работают в облаке AWS EC2, но могут не работать в других средах. Если у вас эти инструменты не заработают, загляните в исходный код, где приводятся примеры полезных РМС, которые можно задействовать напрямую с помощью `perf(1)` (см. раздел 6.6.13 «perf»).

6.6.12. tlbbstat

`tlbbstat(8)` — еще один инструмент из репозитория `pmc-cloud-tools`, который показывает статистики кэширования в TLB. Вот пример вывода:

```
# tlbbstat -C0 1
K_CYCLES   K_INSTR    IPC DTLB_WALKS  ITLB_WALKS  K_DTLBCYC  K_ITLBCYC  DTLB%  ITLB%
2875793    276051     0.10 89709496    65862302    787913     650834     27.40  22.63
2860557    273767     0.10 88829158    65213248    780301     644292     27.28  22.52
2885138    276533     0.10 89683045    65813992    787391     650494     27.29  22.55
2532843    243104     0.10 79055465    58023221    693910     573168     27.40  22.63
[...]
```


В этом примере — наихудший сценарий для исправлений KPTI, которые устраняют уязвимость Meltdown (о влиянии KPTI на производительность рассказывалось в главе 3 «Операционные системы», в разделе 3.4.3 «KPTI (Meltdown)»). KPTI очищает кэш TLB при обращении к системным вызовам и другим событиям, заставляя процессор выполнять холостые такты во время обхода TLB: это демонстрируют последние два столбца. В этом примере процессор тратит примерно половину своего времени на обход TLB, из-за чего выполняет рабочую нагрузку примерно вдвое медленнее.

Вот значение некоторых столбцов:

- **K_CYCLES**: тактов процессора $\times 1000$;
- **K_INSTR**: инструкций процессора $\times 1000$;
- **IPC**: количество инструкций на такт;
- **DTLB_WALKS**: обходов буфера TLB данных (количество);
- **ITLB_WALKS**: обходов буфера TLB инструкций (количество);
- **K_DTLBCYC**: тактов, когда хотя бы один PMH активен во время обхода TLB данных $\times 1000$;
- **K_ITLBCYC**: тактов, когда хотя бы один PMH активен во время обхода TLB инструкций $\times 1000$;
- **DTLB%**: отношение числа активных тактов TLB данных к общему числу тактов;
- **ITLB%**: отношение числа активных тактов TLB инструкций к общему числу тактов.

Как и `rmcarch(8)`, этот инструмент может не работать в вашей среде из-за различий в процессорах. Тем не менее он может стать полезным источником идей.

6.6.13. perf

`perf(1)` — это официальный профилировщик в Linux, многофункциональный инструмент с множеством возможностей. В главе 13 дается подробное описание `perf(1)`. В этом разделе показано его применение для анализа производительности процессора.

Однострочные сценарии

Следующие однострочные сценарии показывают различные возможности `perf(1)` для анализа производительности процессора. Некоторые подробно описаны в следующих разделах.

Выборка трассировок стека указанной команды с частотой 99 Гц:

```
perf record -F 99 command
```

Выборка трассировок стека (через указатели фреймов) в масштабе всей системы в течение 10 с:

```
perf record -F 99 -a -g -- sleep 10
```

Выборка трассировок стека для процесса с заданным идентификатором PID с использованием отладочной информации в формате для раскручивания стека:

```
perf record -F 99 -p PID --call-graph dwarf -- sleep 10
```

Регистрация событий запуска новых процессов через системный вызов `exec`:

```
perf record -e sched:sched_process_exec -a
```

Регистрация переключений контекста с трассировками стека в течение 10 с:

```
perf record -e sched:sched_switch -a -g -- sleep 10
```

Регистрация миграций потоков между процессорами в течение 10 с:

```
perf record -e migrations -a -- sleep 10
```

Регистрация миграций между всеми процессорами в течение 10 с:

```
perf record -e migrations -a -c 1 -- sleep 10
```

Вывод `perf.data` в виде текстового отчета с объединенными данными, суммами и процентами:

```
perf report -n --stdio
```

Вывод всех событий `perf.data` с заголовками (рекомендуется):

```
perf script --header
```

Вывод статистик РМС для всей системы, накопленных за 5 с:

```
perf stat -a -- sleep 5
```

Вывод статистик, характеризующих работу кэша процессора последнего уровня (LLC), для указанной команды `command`:

```
perf stat -e LLC-loads,LLC-load-misses,LLC-stores,LLC-prefetches command
```

Вывод пропускной способности шины памяти в системе с интервалом в 1 с:

```
perf stat -e uncore_imc/data_reads/,uncore_imc/data_writes/ -a -I 1000
```

Вывод частоты переключений контекста в секунду:

```
perf stat -e sched:sched_switch -a -I 1000
```

Вывод частоты принудительного переключения контекста в секунду (когда предыдущим было состояние `TASK_RUNNING`):

```
perf stat -e sched:sched_switch --filter 'prev_state == 0' -a -I 1000
```

Вывод частоты переключения режима и контекста в секунду:

```
perf stat -e cpu_clk_unhalted.ring0_trans,cs -a -I 1000
```

Регистрация профиля планировщика в течение 10 с:

```
perf sched record -- sleep 10
```

Вывод задержки планировщика для каждого процесса из профиля планировщика:

```
perf sched latency
```

Вывод задержек планировщика для каждого события из профиля планировщика:

```
perf sched timehist
```

Еще множество однострочных сценариев для `perf(1)` будет представлено в главе 13 «`perf`», в разделе 13.2 «Однострочные сценарии».

Профилирование процессора в масштабе всей системы

`perf(1)` можно использовать для профилирования процессора, чтобы выявить пути в коде как в пространстве ядра, так и в пространстве пользователя, на выполнение которых тратится процессорное время. Это делают с помощью команды `record`, которая сохраняет образцы стека в файл `perf.data`. По окончании профилирования содержимое этого файла можно исследовать с помощью команды `report`. При профилировании используется самый точный из доступных таймеров: на основе тактов процессора, если он доступен, или программный (событие `cru-clock`).

Команда профилирования в следующем примере выбирает все (`-g`) трассировки стека вызовов со всех (`-a`¹) процессоров с частотой 99 Гц (`-F 99`) в течение 10 с (`sleep 10`). Параметр `--stdio` в команде `report` отключает интерактивный режим и выводит все результаты сразу.

```
# perf record -a -g -F 99 -- sleep 10
[ perf record: Woken up 20 times to write data ]
[ perf record: Captured and wrote 5.155 MB perf.data (1980 samples) ]
# perf report --stdio
[...]
```

#	Children	Self	Command	Shared Object	Symbol
#	29.49%	0.00%	mysqld	libpthread-2.30.so	[.] start_thread
			---start_thread		
			0x55dadd7b473a		
			0x55dadac140fe0		
			--29.44%--do_command		
				--26.82%--dispatch_command	
					--25.51%--mysqld_stmt_execute

¹ В Linux 4.11 параметр `-a` стал параметром по умолчанию.

```

Prepared_statement::execute_loop          | --25.05%--
      |                                   |
Prepared_statement::execute                | --24.90%--
      |                                   |
mysql_execute_command                      | --24.34%--
      |                                   |
[...]                                     |

```

Полный вывод занимает много страниц и содержит информацию о вызовах в порядке убывания значений счетчиков. Счетчики выводятся в процентах затраченного процессорного времени, что позволяет выявить основных потребителей процессора. В этом примере 29,44 % процессорного времени было потрачено на выполнение `do_command()` и вызываемых ею функций, включая `mysql_execute_command()`. Символы с именами функций в ядре и в процессе доступны, только если есть соответствующие файлы с отладочной информацией, иначе отображаются простые шестнадцатеричные адреса.

В Linux 4.4 порядок вывода стека изменился с прямого (когда первой следует функция, выполняющаяся на процессоре, а затем все ее потомки) на обратный (когда первой следует родительская функция, а затем все ее потомки). Прямой порядок возвращает параметр `-g`:

```

# perf report -g callee --stdio
[...]
19.75%    0.00%  mysqld          mysqld          [.]
Sql_cmd_dml::execute_inner
      |
      ---Sql_cmd_dml::execute_inner
         Sql_cmd_dml::execute
         mysql_execute_command
         Prepared_statement::execute
         Prepared_statement::execute_loop
         mysqld_stmt_execute
         dispatch_command
         do_command
         0x55dad140fe0
         0x55dadd7b473a
         start_thread
[...]

```

При анализе профиля попробуйте оба порядка. Если в командной строке будет сложно разобраться, попробуйте средства визуализации, например флейм-график.

Флейм-графики процессора

Флейм-графики процессорного времени можно сгенерировать из профиля `perf`. data командой `flamegraph`, добавленной в Linux 5.8¹. Например:

¹ Спасибо за это Андреасу Герстмайру (Andreas Gerstmayr).

```
# perf record -F 99 -a -g -- sleep 10
# perf script report flamegraph
```

Эти команды создадут флейм-график с использованием файла шаблона `d3-flame-graph` в `/usr/share/d3-flame-graph/d3-flamegraph-base.html` (если у вас нет этого файла, создайте его с помощью `d3-flame-graph` [Spier 20b]). Их также можно объединить в одну команду:

```
# perf script flamegraph -a -F 99 sleep 10
```

В более старых версиях Linux для визуализации образцов, собранных командой `perf script`, используйте мое оригинальное ПО `flamegraph`, как показано ниже (эти шаги также были показаны в главе 5):

```
# perf record -F 99 -a -g -- sleep 10
# perf script --header > out.stacks
$ git clone https://github.com/brendangregg/FlameGraph; cd FlameGraph
$ ./stackcollapse-perf.pl < ../out.stacks | ./flamegraph.pl --hash > out.svg
```

Файл `out.svg` — это флейм-график процессорного времени, который можно открыть в браузере. Он включает сценарий на JavaScript для поддержки интерактивности: щелчок мышью изменяет масштаб, а комбинация `Ctrl-F` открывает поиск. См. раздел 6.5.4 «Профилирование», где эти шаги показаны на рис. 6.15.

Можно изменить эти шаги и направить вывод `perf script` прямо на вход `stackcollapse-perf.pl`, минуя сохранение исходных данных в файле `out.stacks`. Но я считаю полезным архивировать эти файлы для дальнейшего использования и анализа с помощью других инструментов (например, `FlameScope`).

Параметры

`flamegraph.pl` поддерживает множество параметров, в том числе:

- **--title TEXT**: устанавливает текст `TEXT` как заголовок;
- **--subtitle TEXT**: устанавливает текст `TEXT` как подзаголовок;
- **--width NUM**: задает ширину создаваемого изображения (по умолчанию 1200 пикселей);
- **--countname TEXT**: изменяет текст метки счетчика (по умолчанию «samples»);
- **--colors PALETTE**: устанавливает палитру цветов для окрашивания фреймов стека. Некоторые из них используют искомые фразы или аннотации для окрашивания разных путей в коде разными цветами. Возможные варианты: `hot` (по умолчанию), `mem`, `io`, `java`, `js`, `perl`, `red`, `green`, `blue`, `yellow`;
- **--bgcolors COLOR**: устанавливает цвет фона. Варианты с градиентом: `yellow` (по умолчанию), `blue`, `green`, `grey`; чтобы получить фон без градиента, используйте обозначения цветов в формате «#rrggbb»;
- **--hash**: для единообразия цвета отмечаются хешем имени функции;
- **--reverse**: создает флейм-график с обратным упорядочением стека, от листа к корню;

- **--inverted**: поворачивает ось Y так, чтобы вместо «графика пламени» получился «график сосулек»;
- **--flamechart**: создает флейм-диаграмму (время по оси X).

Вот набор параметров, которые я использую для создания флейм-графиков процессорного времени для Java:

```
$ ./flamegraph.pl --colors=java --hash
  --title="CPU Flame Graph, $(hostname), $(date)" < ...
```

В график добавляются имя хоста и дата создания.

Порядок интерпретации флейм-графиков см. в разделе 6.7.3 «Флейм-графики».

Профилирование отдельных процессов

Кроме профилирования всех CPU, `perf(1)` позволяет профилировать отдельные процессы, для чего можно передать параметр `-p PID` или команду запуска процесса:

```
# perf record -F 99 -g command
```

Часто перед командой добавляются два дефиса «--», что предотвращает обработку параметров, предназначенных для команды `command`, утилитой `perf(1)`.

Задержка планировщика

Команда `sched` фиксирует и выводит статистики планировщика. Например:

```
# perf sched record -- sleep 10
[ perf record: Woken up 63 times to write data ]
[ perf record: Captured and wrote 125.873 MB perf.data (1117146 samples) ]
# perf sched latency
```

Task	Runtime ms	Switches	Average delay ms	Maximum delay ms
jbd2/nvme0n1p1-:175	0.209 ms	3	avg: 0.549 ms	max: 1.630 ms
kauditd:22	0.180 ms	6	avg: 0.463 ms	max: 2.300 ms
oltp_read_only.:(4)	3969.929 ms	184629	avg: 0.007 ms	max: 5.484 ms
mysqld:(27)	8759.265 ms	96025	avg: 0.007 ms	max: 4.133 ms
bash:21391	0.275 ms	1	avg: 0.007 ms	max: 0.007 ms
[...]				
TOTAL:	12916.132 ms	281395		

Этот отчет о задержке обобщает среднюю и максимальную задержку планировщика (также известную как задержка в очереди на выполнение) для каждого процесса. Несмотря на большое количество переключений контекста в процессах `oltp_read_only` и `mysqld`, средняя и максимальная задержки планировщика в этом примере оставались низкими. (Чтобы уместить результаты по ширине книжной страницы, я опустил последний столбец «Maximum delay at» (максимальная задержка в).)

События планировщика следуют очень часто, поэтому на трассировку этого типа расходуются значительные вычислительные ресурсы и объем дискового пространства. Размер файла perf.data в этом случае составил 125 Мбайт при продолжительности трассировки всего 10 с. Частота следования событий планировщика может привести к переполнению кольцевых буферов в perf(1) и повлечь потерю событий: если такое случится, об этом будет сказано в конце в отчета. Будьте осторожны, принимая решение о выполнении такой трассировки, потому что значительный оверхед может отрицательно сказаться на работе промышленных приложений.

perf(1) sched умеет создавать отчеты map и timehist с разными способами отображения профиля планировщика. Отчет timehist включает подробную информацию о каждом событии:

```
# perf sched timehist
Samples do not have callchains.
      time      cpu task name      wait time sch delay run time
              [tid/pid]              (msec)      (msec)      (msec)
-----
437752.840756 [0000] mysqld[11995/5187]      0.000      0.000      0.000
437752.840810 [0000] oltp_read_only.[21483/21482] 0.000      0.000      0.054
437752.840845 [0000] mysqld[11995/5187]      0.054      0.000      0.034
437752.840847 [0000] oltp_read_only.[21483/21482] 0.034      0.002      0.002
[...]
437762.842139 [0001] sleep[21487]      10000.080      0.004      0.127
```

В этом отчете показано каждое событие переключения контекста с указанием времени ожидания (wait time), задержки планировщика (sch delay) и времени выполнения на процессоре (runtime). Все времена выводятся в миллисекундах. Последняя строка показывает характеристики команды sleep(1), использовавшейся для установки длительности выполнения perf record, которая простаивала 10 с.

Счетчики РМС (аппаратные события)

Подкоманда stat подсчитывает события и выводит сводную информацию без записи событий в файл perf.data. По умолчанию perf stat подсчитывает несколько счетчиков РМС, чтобы получить высокоуровневую сводку с информацией о тактах процессора. Вот пример получения сводной информации о команде gzip(1):

```
$ perf stat gzip ubuntu-19.10-live-server-amd64.iso
Performance counter stats for 'gzip ubuntu-19.10-live-server-amd64.iso':

    25235.652299      task-clock (msec)      # 0.997 CPUs utilized
           142      context-switches      # 0.006 K/sec
           25      cpu-migrations      # 0.001 K/sec
          128      page-faults      # 0.005 K/sec
    94,817,146,941      cycles      # 3.757 GHz
    152,114,038,783      instructions      # 1.60 insn per cycle
    28,974,755,679      branches      # 1148.167 M/sec
    1,020,287,443      branch-misses      # 3.52% of all branches

    25.312054797      seconds time elapsed
```

В числе статистик выводится количество тактов и инструкций, а также IPC. Как отмечалось выше, это чрезвычайно полезная высокоуровневая метрика, помогающая определить типы и количество холостых тактов. В данном случае величина IPC равна 1,6 — это «хорошо».

Вот пример оценки IPC в масштабе всей системы, на этот раз с бенчмарком *Shopify*, используемым для исследования настройки NUMA. По результатам этой оценки в итоге удалось повысить пропускную способность приложения на 20–30 %. Команды ниже производят измерения на всех процессорах в течение 30 с.

До:

```
# perf stat -a -- sleep 30
[...]
      404,155,631,577      instructions      #    0.72  insns per cycle
[100.00%]
[...]
```

После настройки NUMA:

```
# perf stat -a -- sleep 30
[...]
      490,026,784,002      instructions      #    0.89  insns per cycle
[100.00%]
[...]
```

Величина IPC повысилась с 0,72 до 0,89 — на 24 %, что говорит о большой победе. (См. еще один пример оценки IPC в промышленной системе в главе 16 «Пример из практики».)

Выбор аппаратного события

Есть множество аппаратных событий, которые можно подсчитывать. Их можно перечислить командой `perf list`:

```
# perf list
[...]
branch-instructions OR branches      [Hardware event]
branch-misses                        [Hardware event]
bus-cycles                          [Hardware event]
cache-misses                        [Hardware event]
cache-references                    [Hardware event]
cpu-cycles OR cycles                [Hardware event]
instructions                        [Hardware event]
ref-cycles                          [Hardware event]
[...]
LLC-load-misses                    [Hardware cache event]
LLC-loads                          [Hardware cache event]
LLC-store-misses                   [Hardware cache event]
LLC-stores                         [Hardware cache event]
[...]
```


Обратите внимание на категории «Hardware event» (аппаратное событие) и «Hardware cache event» (событие аппаратного кэша). Для некоторых процессоров можно найти дополнительные категории счетчиков РМС. Более подробный пример см. в главе 13 «perf», в разделе 13.3 «События perf». Доступность зависит от архитектуры процессора, а описание можно найти в руководствах по процессорам (например, в *Software Developer's Manual* для процессоров Intel).

События можно выбрать с помощью параметра `-e`. Например (для Intel Xeon):

```
$ perf stat -e instructions,cycles,L1-dcache-load-misses,LLC-load-misses,dTLB-load-misses gzip ubuntu-19.10-live-server-amd64.iso
```

```
Performance counter stats for 'gzip ubuntu-19.10-live-server-amd64.iso':
```

152,226,453,131	instructions	# 1.61 insn per cycle
94,697,951,648	cycles	
2,790,554,850	L1-dcache-load-misses	
9,612,234	LLC-load-misses	
357,906	dTLB-load-misses	

```
25.275276704 seconds time elapsed
```

Помимо инструкций и тактов, в этом примере измерялись:

- **L1-dcache-load-misses:** промахи чтения из кэша данных уровня 1. Это значение дает представление о нагрузке на память, создаваемой приложением, с учетом того, что некоторые операции чтения были удовлетворены из кэша уровня 1. Это значение сопоставляют с другими счетчиками событий кэша L1, чтобы определить коэффициент попаданий в кэш.
- **LLC-load-misses:** промахи чтения из кэша последнего уровня. После неудачной попытки получить данные из кэша последнего уровня следует обращение к основной памяти, то есть это значение — мера нагрузки на основную память. Разница между этим счетчиком и `L1-dcache-load-misses` дает представление об эффективности кэшей процессора за пределами уровня 1, но для полноты картины желательно определить значения других счетчиков.
- **dTLB-load-misses:** промахи в буфере данных ассоциативной трансляции. Этот счетчик показывает эффективность блока управления памятью (MMU) в части кэширования отображений страниц для рабочей нагрузки и может измерять величину рабочей нагрузки на память (рабочий набор).

Можно проверить и многие другие счетчики. `perf(1)` поддерживает и описательные имена (как в этом примере), и шестнадцатеричные значения. Последние могут потребоваться для исследования загадочных счетчиков, которые можно найти в руководствах по процессорам и для которых нет описательных имен.

Трассировка программного обеспечения

`perf` может также записывать и подсчитывать программные события. Вот список некоторых событий, связанных с процессором:

```
# perf list
[...]
```

context-switches OR cs	[Software event]
cpu-migrations OR migrations	[Software event]

```
[...]
sched:sched_kthread_stop          [Tracepoint event]
sched:sched_kthread_stop_ret      [Tracepoint event]
sched:sched_wakeup                [Tracepoint event]
sched:sched_wakeup_new            [Tracepoint event]
sched:sched_switch                [Tracepoint event]
[...]
```

В примере ниже показано использование программного события переключения контекста для определения моментов, когда ПО оставляет процессор и собирает образцы стека вызовов в течение одной секунды:

```
# perf record -e sched:sched_switch -a -g -- sleep 1
[ perf record: Woken up 46 times to write data ]
[ perf record: Captured and wrote 11.717 MB perf.data (50649 samples) ]
# perf report --stdio
[...]
```

16.18%	16.18%	prev_comm=mysqlld	prev_pid=11995	prev_prio=120	prev_state=S ==>
next_comm=swapper/1 next_pid=0 next_prio=120					

```

|
---__sched_text_start
schedule
schedule_hrttimeout_range_clock
schedule_hrttimeout_range
poll_schedule_timeout.constprop.0
do_sys_poll
__x64_sys_ppoll
do_syscall_64
entry_SYSCALL_64_after_hwframe
ppoll
vio_socket_io_wait
vio_read
my_net_read
Protocol_classic::read_packet
Protocol_classic::get_command
do_command
start_thread
[...]
```

Этот усеченный вывод показывает переключение контекста `mysql` при блокировке на сокете через `poll(2)`. Подробности о методологии анализа состояний вне процессора ищите в главе 5 «Приложения», в разделе 5.4.2 «Анализ времени ожидания вне процессора», а о вспомогательных инструментах — в разделе 5.5.3 «offcpu».

В главе 9 «Диски» представлен еще один пример статической трассировки с помощью `perf(1)` и точек трассировки блочного ввода/вывода. В главе 10 «Сеть» приводится пример использования `perf(1)` для динамической инструментации функции ядра `tcp_sendmsg()`.

Аппаратная трассировка

perf(1) может также использовать аппаратную трассировку для анализа каждой инструкции, если такая возможность поддерживается процессором. Это довольно низкоуровневая методика, которая здесь не рассматривается, но упоминается в главе 13 «perf» в разделе 13.13 «Другие команды».

Документация

Подробнее о perf(1) идет речь в главе 13 «perf». Также дополнительные сведения можно найти в странице справочного руководства man, в документации в исходном коде ядра Linux в каталоге tools/perf/Documentation, на моей странице «perf Examples» [Gregg 20f], в учебном руководстве «Perf Tutorial» [Perf 15] и на странице «The Unofficial Linux Perf Events Web-Page» [Weaver 11].

6.6.14. profile

profile(8) — это инструмент для ВСС, который выбирает трассировки стека с заданным интервалом и сообщает частоту каждой из них. Это самый полезный инструмент в ВСС для понимания особенностей потребления CPU, потому что обобщает практически все пути кода, характеризующиеся высоким потреблением вычислительных ресурсов. (См. описание инструмента hardirqs(8) в разделе 6.6.19, где описываются дополнительные приемы выявления потребителей процессора.) По сравнению с perf(1) у него малый оверхед, потому что в пространство пользователя передается только обобщенная информация с результатами трассировки стека. Разница в оверхеде показана на рис. 6.15. Использование profile(8) в качестве профилировщика приложений кратко описывается в главе 5 «Приложения», в разделе 5.5.2 «profile».

По умолчанию profile(8) выбирает трассировки стека для всех процессоров в пространстве пользователя и ядра с частотой 49 Гц. Все это можно настроить с помощью параметров, при этом выбранные параметры выводятся перед результатами. Например:

profile

Sampling at 49 Hertz of all threads by user + kernel stack... Hit Ctrl-C to end.

^C

[...]

```
finish_task_switch
__sched_text_start
schedule
schedule_hrtimeout_range_clock
schedule_hrtimeout_range
poll_schedule_timeout.constprop.0
do_sys_poll
__x64_sys_ppoll
do_syscall_64
entry_SYSCALL_64_after_hwframe
ppoll
```

```

vio_socket_io_wait(Vio*, enum_vio_io_event)
vio_read(Vio*, unsigned char*, unsigned long)
my_net_read(NET*)
Protocol_classic::read_packet()
Protocol_classic::get_command(COM_DATA*, enum_server_command*)
do_command(THD*)
start_thread
-                               mysqld (5187)
151

```

Как видите, трассировки стека выводятся в виде списков функций, за которыми следуют дефис («-»), имя процесса, его PID в скобках и в конце число — сколько раз встретилась эта трассировка. Трассировки стека выводятся в порядке увеличения частоты встречаемости.

Полный вывод в этом примере состоит из 8261 строки, но здесь показан только последний, наиболее часто встречающийся стек. Он показывает, что функции планировщика выполнялись на процессоре после вызова из пути кода, ведущего к poll(2). В ходе трассировки этот конкретный стек наблюдался 151 раз.

gprofile(8) поддерживает множество параметров, в том числе:

- **-U**: выбирать трассировки стека только в пространстве пользователя;
- **-K**: выбирать трассировки стека только в пространстве ядра;
- **-a**: добавить аннотации во фреймы стека (например, «_[k]» во фреймы ядра);
- **-d**: добавить разделители между стеками в пространстве ядра/пользователя;
- **-f**: вывести результаты в свернутом формате;
- **-p PID**: трассировать только этот процесс;
- **--stack-storage-size SIZE**: количество уникальных трассировок стека (по умолчанию 16 384).

Если gprofile(8) выведет предупреждение такого вида:

```
WARNING: 5 stack traces could not be displayed.
```

это означает, что хранилище трассировок было переполнено. В таких случаях попробуйте увеличить объем хранилища с помощью параметра **--stack-storage-size**.

Создание флейм-графиков из результатов профилирования

Параметр **-f** позволяет вывести результаты профилирования в формате, пригодном для импорта моей программой, создающей флейм-графики. Например:

```

# profile -af 10 > out.stacks
# git clone https://github.com/brendangregg/FlameGraph; cd FlameGraph
# ./flamegraph.pl --hash < out.stacks > out.svg

```

Получившийся файл out.svg можно открыть в браузере.

`profile(8)` и инструменты, описываемые далее (`runqlat(8)`, `runqlen(8)`, `softirqs(8)`, `hardirqs(8)`), — это инструменты на основе BPF из репозитория BCC, о котором рассказывается в главе 15.

6.6.15. cpudist

`cpudist(8)`¹ — это инструмент для BCC, предназначенный для анализа распределения времени выполнения потоков на процессоре после их возобновления. Эту информацию можно использовать для определения характеристик рабочих нагрузок и обоснования последующих настроек и проектных решений. Вот пример вывода, полученный на двухпроцессорном экземпляре сервера базы данных:

```
# cpudist 10 1
Tracing on-CPU time... Hit Ctrl-C to end.
```

usecs	: count	distribution
0 -> 1	: 0	
2 -> 3	: 135	
4 -> 7	: 26961	*****
8 -> 15	: 123341	*****
16 -> 31	: 55939	*****
32 -> 63	: 70860	*****
64 -> 127	: 12622	****
128 -> 255	: 13044	****
256 -> 511	: 3090	*
512 -> 1023	: 2	
1024 -> 2047	: 6	
2048 -> 4095	: 1	
4096 -> 8191	: 2	

Как показывает этот вывод, время выполнения базы данных на процессоре от 4 до 63 мкс подряд. Это не много.

Поддерживаемые параметры:

- **-m**: выводить время в миллисекундах (по умолчанию время измеряется в микросекундах);
- **-O**: вместо времени выполнения на процессоре выводить время вне процессора;
- **-P**: выводить гистограммы для каждого процесса отдельно;
- **-p PID**: трассировать только этот процесс.

Его можно использовать вместе с `profile(8)`, чтобы не только определить, как долго приложение выполнялось на процессоре, но и что оно делало.

¹ Немного истории: Саша Гольдштейн разработал версию `cpudist(8)` для BCC 29 июня 2016 года. Я написал инструмент `cpudists` для Solaris в 2005 году.

6.6.16. runqlat

`runqlat(8)`¹ — это инструмент для BCC и `bpfttrace`, предназначенный для анализа задержек планировщика, которые часто называют задержкой в очереди на выполнение (даже если она больше не реализована с использованием очередей выполнения). (Даже при том, что современные реализации планировщиков не используют очереди.) Его можно использовать для выявления и количественной оценки проблем с насыщением процессора, когда спрос на вычислительные ресурсы превышает возможности системы. `runqlat(8)` измеряет время, которое каждый поток (задача) тратит на ожидание своей очереди выполнения на процессоре.

Ниже показан результат запуска `runqlat(8)` для BCC на промышленном экземпляре облачной базы данных MySQL с двумя процессорами, который работает с нагрузкой на процессоры, примерно равной 15 % в масштабе всей системы. Аргументы «10 1» для `runqlat(8)` определяют 10-секундный интервал и однократный вывод:

```
# runqlat 10 1
```

```
Tracing run queue latency... Hit Ctrl-C to end.
```

usecs	: count	distribution
0 -> 1	: 9017	*****
2 -> 3	: 7188	****
4 -> 7	: 5250	***
8 -> 15	: 67668	*****
16 -> 31	: 3529	**
32 -> 63	: 315	
64 -> 127	: 98	
128 -> 255	: 99	
256 -> 511	: 9	
512 -> 1023	: 15	
1024 -> 2047	: 6	
2048 -> 4095	: 2	
4096 -> 8191	: 3	
8192 -> 16383	: 1	
16384 -> 32767	: 1	
32768 -> 65535	: 2	
65536 -> 131071	: 88	

Результат может оказаться неожиданным для такой малонагруженной системы: судя по выводу, в 88 случаях была высокая задержка планировщика в диапазоне от 65 до 131 мс. Как выяснилось позже, задержка была обусловлена ограничением потребления процессора со стороны гипервизора.

¹ Немного истории: версию `runqlat` для BCC я выпустил 7 февраля 2016 года, а версию для `bpfttrace` — 17 сентября 2018 года. Они были созданы по образу и подобию инструмента для Solaris `dispqlat.d` (dispatcher queue latency — задержка в очереди диспетчера, так называется задержка в очереди на выполнение в терминологии Solaris).

Поддерживаемые параметры:

- **-m**: выводить время в миллисекундах;
- **-P**: выводить гистограммы для каждого процесса отдельно;
- **--pidns**: выводить гистограммы для каждого пространства имен PID;
- **-p PID**: трассировать только этот процесс;
- **-T**: включать в вывод отметки времени.

Чтобы определить время от возобновления (пробуждения) до запуска, `runqlat(8)` инструментирует события возобновления после ожидания и переключения контекста. В высоконагруженных системах эти события могут следовать очень часто и достигать миллиона событий в секунду и даже больше. Несмотря на все оптимизации, используемые в BPF, при такой частоте следования событий добавление даже одной микро-секунды на событие может вызвать заметный оверхед. Используйте этот инструмент с осторожностью и при возможности подумайте о замене его инструментом `runqlen(8)`.

6.6.17. runqlen

`runqlen(8)`¹ — это инструмент для BCC и `bpfftrace`, предназначенный для выборки длин очередей на выполнение, подсчета количества задач, ожидающих своей очереди, и представления полученной информации в виде линейной гистограммы. Этот инструмент можно использовать для дальнейшего исследования проблем с задержками в очереди на выполнение или для приближенной оценки. Выборка производится с частотой 99 Гц, поэтому оверхед обычно незначителен. В отличие от этого инструмента, `runqlat(8)` делает выборку для каждого переключения контекста, которые могут следовать с частотой до миллионов в секунду.

Ниже — пример использования `runqlen(8)` для BCC на экземпляре базы данных MySQL с двумя процессорами, который работает с нагрузкой на процессоры, примерно равной 15 % в масштабе всей системы (это тот же экземпляр, что был выше в примере с `runqlat(8)`). Аргументы «10 1» для `runqlen(8)` определяют 10-секундный интервал и однократный вывод:

```
# runqlen 10 1
Sampling run queue length... Hit Ctrl-C to end.

runqlen      : count      distribution
  0           : 1824      |*****|
  1           : 158       |***   |
```

Согласно полученным результатам, большую часть времени у очереди на выполнение была нулевая длина, и примерно в 8 % случаев длина очереди выполнения была равна единице. Это означает, что потокам приходилось ждать своей очереди на выполнение.

¹ Немного истории: версию для BCC я выпустил 12 декабря 2016 года, а версию для `bpfftrace` — 7 октября 2018 года. За основу был взят инструмент `dispqlen.d`, написанный мною ранее.

Поддерживаемые параметры:

- **-C**: выводить отдельную гистограмму для каждого процессора;
- **-O**: выводить заполнение очереди на выполнение;
- **-T**: включать в вывод отметки времени.

Заполнение очереди на выполнение — это отдельная метрика, показывающая долю времени, проведенную потоком в ожидании. Ее можно использовать, когда для мониторинга, оповещения и построения графиков требуется единственная метрика.

6.6.18. softirqs

`softirqs(8)`¹ — это инструмент для ВСС, который показывает время, затраченное на обработку программных прерываний. Это время легко узнать с помощью других инструментов. Например, `mpstat(1)` выводит его в столбце `%soft`. Есть и псевдофайл `/proc/softirqs`, откуда можно прочитать количество программных прерываний. Инструмент `softirqs(8)` отличается тем, что может сообщить время на обработку каждого типа программных прерываний.

Вот пример десятисекундной трассировки экземпляра базы данных с двумя процессорами:

```
# softirqs 10 1
Tracing soft irq event time... Hit Ctrl-C to end.

SOFTIRQ          TOTAL_usecs
net_tx            9
rcu                751
sched            3431
timer            5542
tasklet         11368
net_rx          12225
```

Как показывает этот вывод, больше всего времени было потрачено на прерывания `net_rx`, всего 12 мс. Этот инструмент помогает выявить потребителей процессора, которые не попадают в обычные профили, потому что профилировщики не могут прерывать обработчики прерываний.

Поддерживаемые параметры:

- **-d**: вывести распределение времени обработки прерываний в виде гистограмм;
- **-T**: добавить отметки времени в вывод.

Параметр `-d` можно использовать для изучения распределения времени, расходуемого на обработку прерываний.

¹ Немного истории: версию для ВСС я выпустил 20 октября 2015 года.

6.6.19. hardirqs

`hardirqs(8)`¹ — это инструмент для БСС, который показывает время, затраченное на обработку аппаратных прерываний. Это время легко узнать с помощью разных инструментов. Например, `mpstat(1)` выводит его в столбце `%irq`. Есть и псевдофайл `/proc/interrupts`, откуда можно прочесть количество аппаратных прерываний. Инструмент `hardirqs(8)` отличается тем, что может сообщить время на обработку каждого типа аппаратных прерываний.

Вот результаты 10-секундной трассировки экземпляра базы данных с двумя процессорами:

```
# hardirqs 10 1
Tracing hard irq event time... Hit Ctrl-C to end.

HARDIRQ                TOTAL_usecs
nvme0q2                  35
ena-mgmt@pci:0000:00:05.0    72
ens5-Tx-Rx-1             326
nvme0q1                  878
ens5-Tx-Rx-0             5922
```

Как показывает этот вывод, за 10 с трассировки на обработку прерывания `ens5-Tx-Rx-0` ушло около 5,9 мс. Подобно `softirqs(8)`, этот инструмент помогает выявить потребителей процессора, которые остаются незаметными для профилировщиков.

`hardirqs(8)` поддерживает те же параметры, что и `softirqs(8)`.

6.6.20. bpftrace

`bpftrace` — это трассировщик на основе BPF, представляющий высокоуровневый язык программирования и позволяющий создавать мощные однострочные и короткие сценарии. Он хорошо подходит для организации анализа приложений на основе подсказок, полученных с помощью других инструментов. В репозитории `bpftrace` есть `bpftrace`-версии инструментов `runqlat(8)` и `runqlen(8)`, представленных выше [Iovisor 20a].

Подробнее о `bpftrace` см. в главе 15. В этом разделе показаны некоторые примеры однострочных сценариев для анализа производительности процессора.

Однострочные сценарии

Следующие однострочные сценарии показывают различные возможности `bpftrace`.

Трассировка создания новых процессов с регистрацией аргументов:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_execve { join(args->argv); }'
```

¹ Немного истории: версию для БСС я выпустил 19 октября 2015 года, взяв за основу написанный мной инструмент `inttimes.d`, который, в свою очередь, был основан на идее еще одного инструмента — `intr.d`.

Подсчет количества обращений к системным вызовам по процессам:

```
bpfttrace -e 'tracepoint:raw_syscalls:sys_enter { @[pid, comm] = count(); }'
```

Подсчет количества обращений к системным вызовам по их именам:

```
bpfttrace -e 'tracepoint:syscalls:sys_enter_* { @[probe] = count(); }'
```

Выборка имен работающих процессов с частотой 99 Гц:

```
bpfttrace -e 'profile:hz:99 { @[comm] = count(); }'
```

Выборка всех трассировок стека в системе в пространствах пользователя и ядра с частотой 49 Гц с именами процессов:

```
bpfttrace -e 'profile:hz:49 { @[kstack, ustack, comm] = count(); }'
```

Выборка трассировок стека в пространстве пользователя с частотой 49 Гц для PID 189:

```
bpfttrace -e 'profile:hz:49 /pid == 189/ { @[ustack] = count(); }'
```

Выборка трассировок стека глубиной до 5 фреймов в пространстве пользователя с частотой 49 Гц для PID 189:

```
bpfttrace -e 'profile:hz:49 /pid == 189/ { @[ustack(5)] = count(); }'
```

Выборка трассировок стека в пространстве пользователя с частотой 49 Гц для процесса с именем «mysqld»:

```
bpfttrace -e 'profile:hz:49 /comm == "mysqld"/ { @[ustack] = count(); }'
```

Подсчет пересечений точек трассировки планировщика в ядре:

```
bpfttrace -e 'tracepoint:sched:* { @[probe] = count(); }'
```

Подсчет трассировок стека в ядре вне процессора по событиям переключения контекста:

```
bpfttrace -e 'tracepoint:sched:sched_switch { @[kstack] = count(); }'
```

Подсчет количества вызовов функций ядра, имена которых начинаются с «vfs_»:

```
bpfttrace -e 'kprobe:vfs_* { @[func] = count(); }'
```

Трассировка создания новых потоков выполнения вызовом функции pthread_create():

```
bpfttrace -e 'u:/lib/x86_64-linux-gnu/libpthread-2.27.so:pthread_create {  
  printf("%s by %s (%d)\n", probe, comm, pid); }'
```

Примеры

Ниже пример использования bpfttrace для профилирования сервера базы данных MySQL с частотой 49 Гц и с выборкой только первых трех фреймов пользовательского стека:

```
# bpftrace -e 'profile:hz:49 /comm == "mysqld"/ { @[ustack(3)] = count(); }'
Attaching 1 probe...
^C
[...]
@[
  my_lengthsp_8bit(Charset_info const*, char const*, unsigned long)+32
  Field::send_to_protocol(Protocol*) const+194
  THD::send_result_set_row(List<Item>*)+203
]: 8
@[
  ppoll+166
  vio_socket_io_wait(Vio*, enum_vio_io_event)+22
  vio_read(Vio*, unsigned char*, unsigned long)+236
]: 10
[...]
```

Приведен неполный вывод — я оставил только два стека, которые были выбраны 8 и 10 раз. Оба соответствуют путям в коде, выполняющим сетевые операции.

Внутренние особенности планирования

Вы можете создать свои инструменты для анализа поведения планировщика. Для начала исследуйте список доступных точек трассировки:

```
# bpftrace -l 'tracepoint:sched:*'
tracepoint:sched:sched_kthread_stop
tracepoint:sched:sched_kthread_stop_ret
tracepoint:sched:sched_waking
tracepoint:sched:sched_wakeup
tracepoint:sched:sched_wakeup_new
tracepoint:sched:sched_switch
tracepoint:sched:sched_migrate_task
tracepoint:sched:sched_process_free
[...]
```

У каждой есть свои аргументы, которые можно увидеть с помощью параметра `-lv`. Если точек трассировки окажется недостаточно, подумайте о возможности динамической инструментации с `kprobes`. Вот как можно получить список доступных зондов `kprobe` в функциях ядра с именами, начинающимися с «`sched`»:

```
# bpftrace -lv 'kprobe:sched*'
kprobe:sched_itmt_update_handler
kprobe:sched_set_itmt_support
kprobe:sched_clear_itmt_support
kprobe:sched_set_itmt_core_prio
kprobe:schedule_on_each_cpu
kprobe:sched_copy_attr
kprobe:sched_free_group
[...]
```

В этой версии ядра (5.3) есть 24 точки трассировки в категории «`sched`» и 104 функции с именами, начинающимися с «`sched`», которые можно инструментировать с помощью `kprobes`.

События планировщика могут следовать очень часто, поэтому их инструментация повлечет значительный оверхед. Будьте осторожны и поищите способы его уменьшить: используйте карты для обобщения статистик вместо вывода сведений о каждом событии и старайтесь трассировать как можно меньшее количество событий.

6.6.21. Другие инструменты

В табл. 6.11 перечислены инструменты наблюдения за работой процессора, представленные в других главах этой книги и в «BPF Performance Tools» [Gregg 19].

Таблица 6.11. Другие инструменты наблюдения за работой процессора

Раздел	Инструмент	Описание
5.5.3	offcputime	Профилирование времени вне процессора методом трассировки планировщика
5.5.5	execsnoop	Трассировка запуска новых процессов
5.5.6	syscount	Подсчет обращений к системным вызовам
[Gregg 19]	runqslower	Выводит задержки в очереди на выполнение, превышающие заданное пороговое значение
[Gregg 19]	cpufreq	Выбирает значения частоты процессора по процессам
[Gregg 19]	smpcalls	Трассирует время выполнения вызовов функций на параллельных процессорах (SMP-вызовов)
[Gregg 19]	llcstat	Обобщает частоту попаданий в кэш LLC по процессам

К числу других инструментов наблюдения и источников информации о работе процессора в Linux относятся:

- **oprofile**: оригинальный инструмент профилирования процессора Джона Левона (John Levon);
- **atop**: предлагает большое количество статистик, характеризующих работу системы в целом, и использует механизм учета для выявления короткоживущих процессов;
- **/proc/cpuinfo**: файл, доступный для чтения, с подробной информацией о процессоре, включая тактовую частоту и флаги поддерживаемых особенностей;
- **lscpu**: сообщает информацию об архитектуре процессора;
- **lstopo**: показывает топологию оборудования (распространяется в составе пакета hwloc);
- **cpupower**: показывает состояние питания процессора;
- **getdelays.c**: содержит пример наблюдения за учетом задержек, включая задержку планировщика, для каждого процесса. Его использование я показал в главе 4 «Инструменты наблюдения»;

- **valgrind**: набор инструментов для отладки и профилирования памяти [Valgrind 20]. Включает инструмент callgrind для трассировки вызовов функций и создания графа вызовов, который можно визуализировать с помощью kcachegrind, а также инструмент cachegrind для анализа использования аппаратного кэша данной программой.

На рис. 6.18 показан пример вывода `lstopo(1)` в формате SVG.

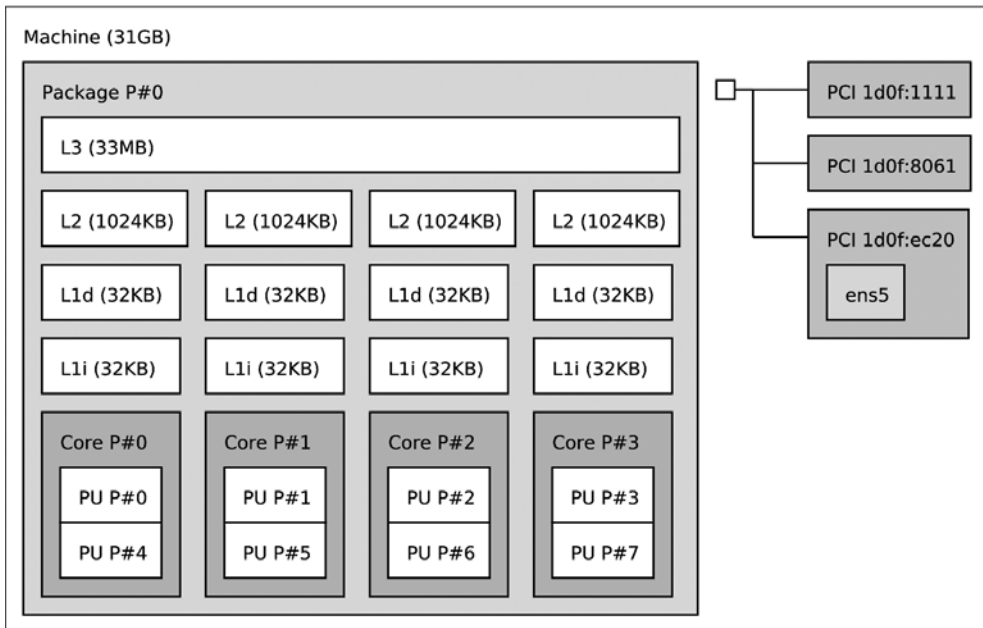


Рис. 6.18. Вывод утилиты `lstopo(1)` в формате SVG

На этом изображении, полученном через `lstopo(1)`, видно, каким ядрам процессора соответствуют логические процессоры (например, CPU 0 и 4 соответствуют ядру 0).

Еще один инструмент, о котором стоит сказать, — `cpupower(1)`. Вот пример его вывода:

```
# cpupower idle-info
CPUidle driver: intel_idle
CPUidle governor: menu
analyzing CPU 0:

Number of idle states: 9
Available idle states: POLL C1 C1E C3 C6 C7s C8 C9 C10
POLL:
Flags/Description: CPUIDLE CORE POLL IDLE
```

```
Latency: 0
Usage: 80442
Duration: 36139954
C1:
Flags/Description: MWAIT 0x00
Latency: 2
Usage: 3832139
Duration: 542192027
C1E:
Flags/Description: MWAIT 0x01
Latency: 10
Usage: 10701293
Duration: 1912665723
[...]
C10:
Flags/Description: MWAIT 0x60
Latency: 890
Usage: 7179306
Duration: 48777395993
```

Здесь не только перечисляются состояния энергопотребления процессора, но и приводятся некоторые статистики: **Usage** показывает, сколько раз активизировалось это состояние, **Duration** — продолжительность в микросекундах пребывания в состоянии, а **Latency** — задержку выхода в микросекундах. Здесь показан пример только для CPU 0; информацию о других процессорах можно получить из их файлов в каталоге `/sys`. Например, продолжительность можно узнать из `/sys/devices/system/cpu/cpu*/cpuidle/state0/time` [Wysocki 19].

Есть и комплексные продукты для анализа производительности процессора, например Intel vTune [22] и AMD uprof [23].

Графические процессоры

Пока что не было создано исчерпывающего набора стандартных инструментов для анализа работы GPU. Производители GPU обычно выпускают специальные инструменты, которые работают только с их собственными продуктами, например:

- **nvidia-smi**, **nvperf** и **Nvidia Visual Profiler**: для GPU Nvidia;
- **intel_gpu_top** и **Intel vTune**: для GPU Intel;
- **radeontop**: для GPU Radeon.

Эти инструменты предлагают базовые статистики — частоту выполнения инструкций и потребление ресурсов GPU. Другие возможные источники информации для наблюдения — счетчики РМС и точки трассировки (попробуйте выполнить команду `perf list | grep gpu`).

Профилирование GPU отличается от профилирования CPU, потому что графические процессоры не имеют трассировок стека, показывающих пути в коде. Вместо

этого профилировщики могут инструментировать вызовы API и передачи памяти, а также время их выполнения.

6.7. МЕТОДЫ ВИЗУАЛИЗАЦИИ

Для визуализации потребления процессора традиционно использовались линейные графики потребления или средней нагрузки, включая оригинальный инструмент для X11 (`xload(1)`). Такие линейные графики эффективно показывают изменение нагрузки, позволяя визуально сравнивать величины. Также они отображают закономерности, зависящие от времени, как было показано в главе 2 «Методологии», в разделе 2.9 «Мониторинг».

Но линейные графики не масштабируются с увеличением количества процессоров, которое мы сейчас наблюдаем, особенно в облачных средах, включающих десятки тысяч процессоров. Тогда график потребления может превратиться в нечитаемый узор из 10 000 линий.

Конечно, для решения проблемы масштабируемости можно строить линейные графики средних значений, стандартных отклонений, максимумов и процентилей, имеющих определенную ценность. Но нагрузка на процессор часто имеет *бимодальный* характер, когда одни процессоры находятся в режиме ожидания или почти бездействуют, а другие работают со 100 %-ной нагрузкой. И это неэффективно отражается такими статистиками. Часто изучения требует полное распределение. Для этого используйте тепловые карты.

В следующих разделах представлены приемы создания тепловых карт потребления процессоров, тепловых карт с субсекундным смещением, флэйм-графиков и FlameScore. Я разрабатывал эти методы визуализации для решения проблем при анализе производительности в промышленных и облачных средах.

6.7.1. Тепловая карта потребления

Зависимость потребления от времени можно представить в виде тепловой карты, где насыщенность цвета каждого пикселя отражает количество процессоров с этим уровнем потребления и временной диапазон [Gregg 10a]. Тепловые карты были представлены в главе 2 «Методологии».

На рис. 6.19 показано потребление процессоров в центре обработки данных в общедоступной облачной среде. Эта тепловая карта построена на основе более 300 физических серверов и 5312 процессоров.

Более насыщенный цвет пикселей в нижней части этой тепловой карты показывает, что большинство процессоров работает с нагрузкой от 0 до 30 %. Сплошная линия вверху показывает, что некоторые процессоры работают с нагрузкой 100 %. Насыщенность цвета этой линии подчеркивает, что с такой нагрузкой работают несколько процессоров, а не один.

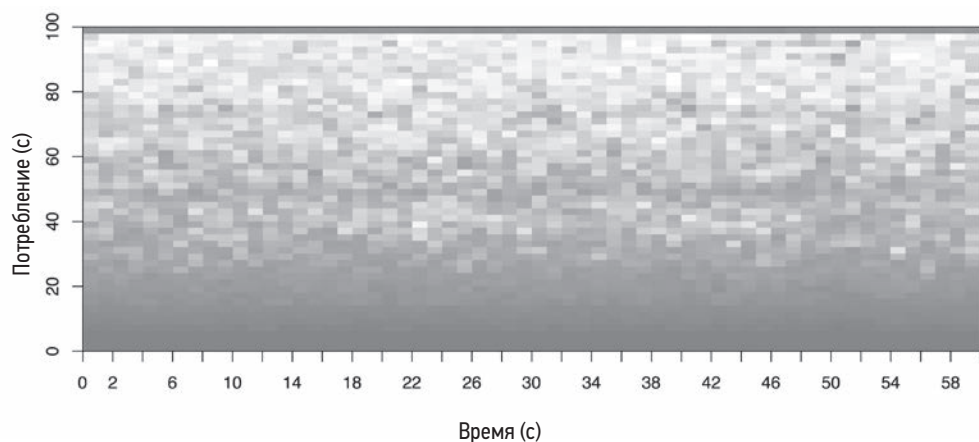


Рис. 6.19. Тепловая карта потребления 5312 процессоров

6.7.2. Тепловая карта с субсекундным смещением

Тепловые карты этого вида позволяют исследовать активность в течение секунды. Активность процессора обычно измеряется микросекундами или миллисекундами, и ее представление в виде средних значений за целую секунду может скрыть полезную информацию. Тепловая карта с субсекундным смещением показывает субсекундное смещение вдоль оси Y и отображает количество активных процессоров в каждом смещении через насыщенность цвета пикселей. То есть каждой секунде соответствует отдельный столбец, который «раскрашивается» снизу вверх.

На рис. 6.20 показана тепловая карта процессоров с субсекундным смещением для облачной базы данных (Riak).

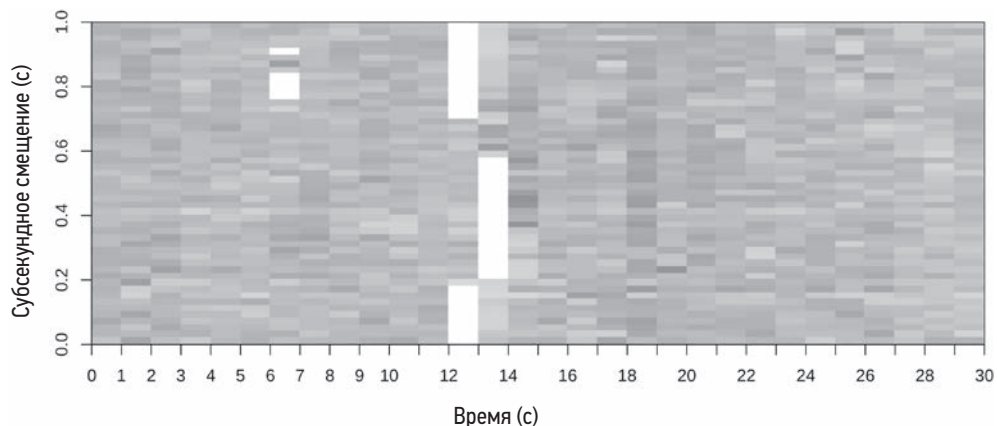


Рис. 6.20. Тепловая карта активности процессоров с субсекундным смещением

Здесь интересно не время, когда процессоры были заняты обслуживанием базы данных, а время, когда они простаивали, — оно обозначено белыми столбиками. Также интересна продолжительность этих периодов: сотни миллисекунд, в течение которых ни один из потоков базы данных не выполнялся на процессоре. Анализ этой тепловой карты привел к выявлению проблем с блокировками, когда вся база данных целиком блокировалась на сотни миллисекунд.

Если бы анализ проводился с помощью линейного графика, то падение нагрузки на процессоры не выглядело бы как переменчивость нагрузки и дальше не исследовалось.

6.7.3. Флейм-графики

Профилирование трассировок стека — эффективный способ, объясняющий потребление процессора и показывающий, какие пути в коде на уровне ядра или пользователя потребляют основное процессорное время. Но при этом могут выводиться тысячи страниц с результатами. Флейм-графики процессорного времени визуализируют фреймы стека профилей, помогая быстрее и точнее выявить основных потребителей процессора [Gregg 16b]. В примере на рис. 6.21 показан флейм-график с результатами профилирования ядра Linux, полученными с использованием `perf(1)`.

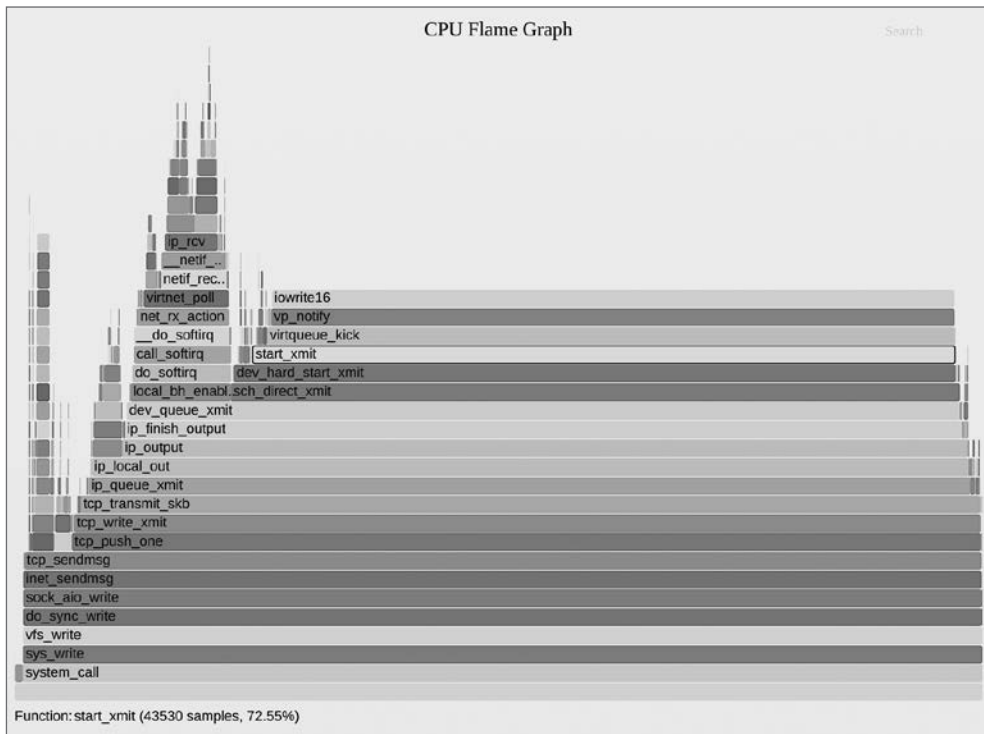


Рис. 6.21. Флейм-график с результатами профилирования ядра Linux

Флейм-график можно построить на основе любого профиля процессора, включающего трассировки стека, в том числе полученного с помощью `perf(1)`, `profile(8)`, `brftrace` и других профилировщиков. Флейм-графики могут отображать не только профили процессорного времени. В этом разделе описаны флейм-графики процессорного времени, сгенерированные через `flamegraph.pl` [Gregg 20g]. (Есть и другие реализации, включая `d3`, созданную моим коллегой Мартином Спиром [Spier 20a].)

Характеристики

У флейм-графика процессорного времени следующие характеристики:

- Каждый прямоугольник представляет функцию в стеке («фрейм стека»).
- **Ось Y** отражает глубину стека (количество фреймов в стеке). На самом верху отображается функция, выполняющаяся на процессоре. Все, что ниже, — это путь, приведший к этой функции. Функция, находящаяся непосредственно под выполняющейся функцией, является ее родителем, как и в трассировках стека, показанных выше.
- **Ось X** охватывает совокупность выборки. Важно отметить, что ось X не показывает ход времени слева направо, как в большинстве графиков. Порядок размещения фреймов слева направо не важен (в данном примере они отсортированы по именам функций в алфавитном порядке).
- **Ширина** прямоугольников отражает общее время, в течение которого функция выполнялась на процессоре, включая время выполнения ее потомков. Функции, представленные широкими прямоугольниками, могли выполняться медленнее, чем функции, представленные узкими прямоугольниками, или просто вызывались чаще. Количество вызовов не отображается (его нельзя получить методом выборки трассировок стека).

Суммарное время выборок может превышать время профилирования, если одновременно выполнялось несколько потоков и выборки извлекались параллельно.

Цветовые палитры

Фреймы можно окрашивать разными способами. По умолчанию каждый фрейм окрашивается случайно выбранным теплым цветом, что помогает различать соседние пирамиды. Со временем я добавил несколько цветовых схем. Я обнаружил, что для конечных пользователей флейм-графики намного полезнее, когда:

- **Оттенок** определяет тип кода¹. Например, красный обозначает собственный код в пространстве пользователя, оранжевый — собственный код в пространстве

¹ Это решение было предложено моим коллегой Амером Азером (Amer Ather). Моя первая реализация этого решения была основана на использовании регулярного выражения, написанного на скорую руку.

ядра, желтый — код на C++, зеленый — функции на интерпретируемом языке, голубой — встроенные функции, и т. д. в зависимости от используемого языка. Пурпурный можно использовать для выделения совпадений, найденных в процессе поиска. Некоторые разработчики настраивают флейм-графики так, чтобы их код всегда выделялся.

- **Насыщенность** зависит от имени функции. Это обеспечивает некоторую цветовую градацию, помогающую различать соседние пирамиды, сохраняя при этом одинаковые цвета для одинаковых имен функций, что упрощает сравнение нескольких флейм-графиков.
- **Цвет фона** служит визуальным напоминанием о типе флейм-графика. Например, желтый фон можно использовать для флейм-графиков распределения процессорного времени, синий — для флейм-графиков времени вне процессора или ввода/вывода и зеленый — для флейм-графиков потребления памяти.

Еще одна полезная цветовая схема — схема, используемая для представления на графиках количества инструкций на такт (IPC), где дополнительное измерение, величина IPC, визуализируется как градиентная окраска каждого фрейма от синего цвета к белому или красному.

Интерактивность

Флейм-графики интерактивны. Мой оригинальный сценарий `flamegraph.pl` генерирует флейм-графики в формате SVG со встроенной JavaScript-процедурой, которая при открытии в браузере отображает детали в нижней части при наведении курсора на элементы, а также реализует другие интерактивные функции. В примере на рис. 6.21 выделен фрейм `start_xmit()`, а в нижней части выведена подсказка, сообщающая, что он есть в 72,55 % стеков, имеющихся в профиле.

Щелкнув по фрейму, можно увеличить его горизонтальный масштаб¹. Нажав `Ctrl-F`², можно ввести слово и выполнить его поиск. При поиске также отображается совокупный процент, определяющий, как часто встречается трассировка стека, содержащая искомое слово. Это позволяет быстро выяснить, какая доля профиля приходится на определенные области кода. Например, можно выполнить поиск по строке «`tcp_`» и узнать, какая доля времени приходится на выполнение кода TCP в ядре.

Интерпретация

Чтобы объяснить, как интерпретировать флейм-графики, рассмотрим синтетический флейм-график процессорного времени (рис. 6.22).

¹ Функцию горизонтального масштабирования для флейм-графиков разработал Адриен Махье (Adrien Mahieux).

² Поддержку поиска в свою реализацию флейм-графиков первым добавил Торстен Лоренц (Thorsten Lorenz).

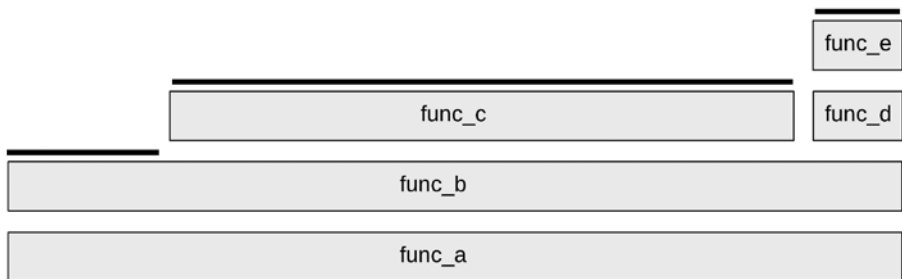


Рис. 6.22. Синтетический флейм-график процессорного времени

Верхний край выделен линией: так отображаются функции, выполняющиеся непосредственно на процессоре. Функция `func_c()` выполнялась непосредственно на процессоре в течение 70 % времени, функция `func_b()` — 20 % времени, функция `func_e()` — 10 % времени. Другие функции, `func_a()` и `func_d()`, ни в одной выборке не были отмечены как выполнявшиеся непосредственно на процессоре.

Читая флейм-график, сначала ищите самые широкие пирамиды и пробуйте в них разобраться. На рис. 6.22 это путь в коде `func_a() -> func_b() -> func_c()`. На рис. 6.21 самая широкая пирамида заканчивается плато с функцией `iowrite16()`.

В больших профилях, включающих тысячи выборок, некоторые пути в коде могут встречаться лишь несколько раз и образовывать настолько узкие пирамиды, что в них не остается места для включения имен функций. Как оказывается, это преимущество: все свое внимание вы, естественно, уделите более широким пирамидам, имеющим разборчивые имена функций, и их исследование позволит понять основную часть профиля.

Обратите внимание, что для рекурсивных функций каждый уровень рекурсии будет представлен отдельным фреймом.

Дополнительные советы по интерпретации флейм-графиков ищите в разделе 6.5.4 «Профилирование», а описание порядка их создания с помощью `perf(1)` — в разделе 6.6.13 «`perf`».

6.7.4. FlameScope

FlameScope — это инструмент с открытым исходным кодом, разработанный в Netflix, который предлагает возможность создания описанных выше тепловых карт с субсекундным смещением и флейм-графиков [Gregg 18b]. Тепловая карта с субсекундным смещением отображает профиль процессора, и диапазоны с субсекундными поддиапазонами для отображения флейм-графика можно выбрать только для этого диапазона. На рис. 6.23 показана тепловая карта с аннотациями и инструкциями, созданная с помощью FlameScope.

FlameScope прекрасно подходит для исследования проблем, аномалий и отклонений. Они могут быть слишком маленькими, чтобы их можно было увидеть на

флейм-графике, отображающем сразу весь профиль: аномалия продолжительностью 100 мс в 30-секундном профиле займет лишь 0,3 % ширины флейм-графика. В FlameScore та же аномалия отобразится в виде вертикальной полосы, занимающей 1/10 высоты тепловой карты. Несколько таких аномалий можно видеть в примере на рис. 6.23. При выборе аномального участка отображается флейм-график процессорного времени только для этого диапазона, показывающий соответствующие пути в коде.

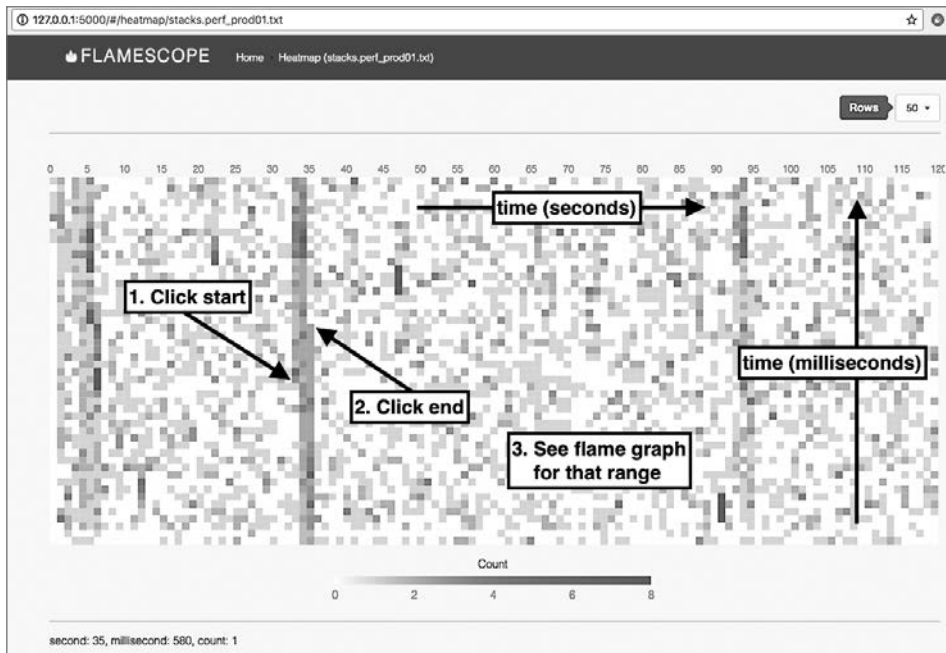


Рис. 6.23. FlameScope (инструкции на графике: 1. Щелкнуть на начале диапазона. 2. Щелкнуть на конце диапазона. 3. Посмотреть флейм-график для этого диапазона)

FlameScope — это открытое ПО [Netflix 19], которое используется в Netflix для поиска возможных оптимизаций.

6.8. ЭКСПЕРИМЕНТЫ

В этом разделе описаны инструменты для активного тестирования производительности CPU. Дополнительную информацию ищите в разделе 6.5.11 «Микробенчмаркинг».

При использовании инструментов, представленных ниже, рекомендую оставить постоянно работающим `mpstat(1)`, чтобы подтвердить потребление процессора и параллелизм.

6.8.1. Ad hoc

Этот прием тривиально прост и фактически ничего не измеряет, но он станет полезной известной рабочей нагрузкой, с помощью которой можно подтвердить, что инструменты наблюдения действительно показывают то, что должны. Код ниже создает однопоточную вычислительную рабочую нагрузку («горячую для одного процессора»):

```
# while ;; do ;; done &
```

Это программа на языке командной оболочки Bourne shell, которая работает в фоновом режиме и выполняет бесконечный цикл. Ее нужно будет принудительно остановить, как только надобность в ней отпадет.

6.8.2. SysBench

SysBench — комплект бенчмарков производительности для системы. Он включает простой инструмент для оценки производительности процессора, который вычисляет простые числа. Например:

```
# sysbench --num-threads=8 --test=cpu --cpu-max-prime=100000 run
sysbench 0.4.12: multi-threaded system evaluation benchmark
```

Running the test with following options:

Number of threads: 8

Doing CPU performance benchmark

Threads started!

Done.

Maximum prime number checked in CPU test: 100000

Test execution summary:

total time:	30.4125s
total number of events:	10000
total time taken by event execution:	243.2310
per-request statistics:	
min:	24.31ms
avg:	24.32ms
max:	32.44ms
approx. 95 percentile:	24.32ms

Threads fairness:

events (avg/stddev):	1250.0000/1.22
execution time (avg/stddev):	30.4039/0.01

В этом примере запускается восемь потоков, которые вычисляют простые числа меньше 100 000. Время выполнения теста составило 30,4 с. Его можно использовать для сравнения с другими системами или конфигурациями (при условии, что при компиляции ПО использовались идентичные параметры компилятора, см. главу 12 «Бенчмаркинг»).

6.9. НАСТРОЙКА

Наибольший выигрыш в производительности процессоров обычно достигается за счет устранения ненужной работы, что является эффективной формой настройки. В разделах 6.5 «Методология» и 6.6 «Инструменты наблюдения» представлено множество способов анализа и идентификации выполняемой работы, помогающих выявлять любую ненужную работу. Были представлены и другие методы настройки: настройка приоритетов и привязка к процессору. Этот раздел включает эти и другие примеры настройки.

Специфика настройки — доступные параметры и особенности их настройки — зависит от типа процессора, версии ОС и предполагаемой рабочей нагрузки. Ниже приводятся примеры доступных параметров, сгруппированные по типам, и порядок их настройки. Когда и почему эти параметры должны настраиваться, рассказано в предыдущих разделах, посвященных методологиям.

6.9.1. Параметры компилятора

Компиляторы и их параметры оптимизации кода могут существенно влиять на производительность процессора. К общим параметрам относятся: компиляция 64-разрядных версий вместо 32-разрядных и выбор уровня оптимизации. Параметры оптимизации компилятора обсуждаются в главе 5 «Приложения».

6.9.2. Приоритет и класс планирования

Для настройки приоритета процесса можно использовать команду `nice(1)`. Положительные значения уступчивости (`nice`) уменьшают приоритет, а отрицательные значения (которые может установить только суперпользователь) — увеличивают приоритет. Значения уступчивости можно изменять в диапазоне от -20 до $+19$. Например:

```
$ nice -n 19 command
```

запустит *command* со значением `nice`, равным 19, — наименьшим приоритетом, доступным команде `nice(1)`. Изменить приоритет уже запущенного процесса можно с помощью команды `renice(1)`.

Команда `chrt(1)` в Linux может напрямую показывать и изменять приоритет и класс планирования. Например:

```
$ chrt -b command
```

запустит *command* с классом планирования `SCHED_BATCH` (см. подраздел «Классы планирования» в разделе 6.4.2 «Программное обеспечение»).

Обеим утилитам, `nice(1)` и `chrt(1)`, можно передать PID запущенного процесса вместо команды (см. соответствующие страницы справочного руководства `man`).

Приоритет планирования также можно установить с помощью системного вызова `setpriority(2)`, а политику планирования — с помощью системного вызова `sched_setscheduler(2)`.

6.9.3. Параметры планировщика

Используемое вами ядро может предлагать настраиваемые параметры для управления поведением планировщика, хотя маловероятно, что их потребуется настраивать.

В системах Linux есть множество параметров CONFIG, управляющих поведением планировщика на высоком уровне, которые можно настроить во время компиляции ядра. В табл. 6.12 показаны примеры из Ubuntu 19.10 и ядра Linux 5.3.

Таблица 6.12. Примеры параметров CONFIG в ядре Linux, управляющих поведением планировщика

Параметр	Значение по умолчанию	Описание
CONFIG_CGROUP_SCHED	y	Позволяет группировать задачи и выделять процессорное время на основе принадлежности к группам
CONFIG_FAIR_GROUP_SCHED	y	Позволяет планировщику CFS группировать задачи
CONFIG_RT_GROUP_SCHED	n	Позволяет группировать задачи реального времени
CONFIG_SCHED_AUTOGROUP	y	Автоматически идентифицирует и создает группы задач (например, задания сборки)
CONFIG_SCHED_SMT	y	Поддержка гиперпоточности
CONFIG_SCHED_MC	y	Поддержка многоядерных процессоров
CONFIG_HZ	250	Устанавливает частоту хода часов ядра (прерываний таймера)
CONFIG_NO_HZ	y	Энергосберегающий режим без прерываний таймера
CONFIG_SCHED_HRTICK	y	Использование таймеров с высоким разрешением
CONFIG_PREEMPT	n	Полная вытесняемость ядра (кроме циклов активного ожидания спин-блокировок и прерываний)
CONFIG_PREEMPT_NONE	n	Отключение возможности вытеснения
CONFIG_PREEMPT_VOLUNTARY	y	Вытеснение ядра в специальных точках

Есть и динамические настройки планировщика, доступные через `sysctl(8)`, которые можно настраивать прямо в работающей системе. Часть из них, доступных в той же системе Ubuntu, перечислена в табл. 6.13 вместе со значениями по умолчанию.

Таблица 6.13. Примеры динамических настроек планировщика, доступных в Linux через sysctl(8)

sysctl	Значение по умолчанию	Описание
kernel.sched_cfs_bandwidth_slice_us	5000	Количество квантов процессорного времени, используемое для вычисления пропускной способности в CFS
kernel.sched_latency_ns	12000000	Целевая задержка вытеснения. Увеличение этого параметра может увеличить время работы задачи на процессоре за счет задержки вытеснения
kernel.sched_migration_cost_ns	500000	Стоимость задержки миграции задачи, используемая для расчетов привязки к процессору. Считается, что для задач, которые выполнялись дольше этого значения, кэш достаточно разогрет
kernel.sched_nr_migrate	32	Определяет, сколько задач можно перенести за раз для балансировки нагрузки
kernel.sched_schedstats	0	Включает сбор дополнительных статистик планировщика и точки трассировки sched:sched_stat*

Эти параметры sysctl(8) также доступны для настройки в /proc/sys/sched.

6.9.4. Режимы масштабирования частоты

Linux поддерживает различные режимы масштабирования, которые управляют тактовой частотой процессоров программно. Их можно настроить через файлы в /sys. Например, для CPU 0:

```
# cat /sys/devices/system/cpu/cpufreq/policy0/scaling_available_governors
performance powersave
# cat /sys/devices/system/cpu/cpufreq/policy0/scaling_governor
powersave
```

Этот пример получен в ненастроенной системе: текущий режим — «powersave» (энергосберегающий) — предписывает использовать пониженную тактовую частоту процессоров для экономии энергии. При желании можно установить режим «performance» (высокая производительность), чтобы процессоры всегда работали на максимальной тактовой частоте. Например:

```
# echo performance > /sys/devices/system/cpu/cpufreq/policy0/scaling_governor
```

Это нужно сделать для всех процессоров (policy0..N). Каталог policyN содержит также файлы для непосредственной установки частоты (scaling_setspeed) и определения диапазона возможных частот (scaling_min_freq, scaling_max_freq).

Настройка процессоров на постоянную работу с максимальной частотой может вредно сказываться на окружающей среде. Если эта настройка не обеспечивает значительного улучшения производительности, то предпочтительнее продолжать использовать настройку «powersave» ради блага планеты. У хостов, где есть доступ к регистрам MSR, которые управляют питанием (гостевым системам в облаке они могут быть недоступны), можно измерить потребляемую мощность в обоих режимах, «powersave» и «performance», чтобы количественно оценить (частично¹) экологические издержки.

6.9.5. Состояния энергопотребления

Состояния энергопотребления процессора можно включать и отключать с помощью инструмента `cripower(1)`. Как было показано в разделе 6.6.21 «Другие инструменты», более глубокие состояния сна характеризуются большей задержкой выхода на рабочий режим (выход из состояния C10, как было показано, занял 890 мкс). Отдельные состояния можно отключить с помощью параметра `-d`, а параметр `-D latency` отключит все состояния, задержка выхода из которых превышает *latency* (в микросекундах). Это позволяет точно настроить, какие состояния с низким энергопотреблением можно использовать, и отключать состояния с чрезмерной задержкой.

6.9.6. Привязка к процессору

Процесс может быть привязан к одному или нескольким CPU, что часто способствует увеличению его производительности за счет большего разогрева кэша и локализации памяти.

В Linux это делается командой `taskset(1)`, которая принимает маску процессора или диапазон. Например:

```
$ taskset -pc 7-10 10790
pid 10790's current affinity list: 0-15
pid 10790's new affinity list: 7-10
```

Эта команда привяжет процесс с PID 10790 к процессорам с 7-го по 10-й.

Команда `numactl(8)` тоже может выполнить привязку к процессорам, а также к узлам памяти (см. главу 7 «Память», раздел 7.6.4 «Привязка NUMA»).

6.9.7. Исключительные процессорные наборы

Linux поддерживает *процессорные наборы* (*cpusets*), позволяющие определять группы процессоров и привязывать к ним процессы. Этот прием повышает

¹ При измерении энергопотребления на уровне хоста не учитываются экологические издержки на кондиционирование воздуха в помещении для серверов, изготовление и транспортировку серверов и другие затраты.

производительность подобно привязке к процессорам, но при этом позволяет повысить производительность еще больше, если сделать процессорные наборы исключительно, запретив использовать их другими процессами. Отрицательная сторона — уменьшение количества процессоров, доступных для остальной части системы.

Следующий пример с комментариями показывает создание исключительного набора:

```
# mount -t cgroup -ocpuset cpuset /sys/fs/cgroup/cpuset # необязательно
# cd /sys/fs/cgroup/cpuset
# mkdir prodset # создать набор процессоров с именем "prodset"
# cd prodset
# echo 7-10 > cpuset.cpus # добавить в набор процессоры с 7-го по 10-й
# echo 1 > cpuset.cpu_exclusive # сделать набор prodset исключительным
# echo 1159 > tasks # привязать процесс с PID 1159 к набору prodset
```

Дополнительную информацию ищите на странице справочного руководства `man` для `cpuset(7)`.

При создании процессорных наборов можно определить, какие из процессоров в наборе будут продолжать обслуживать прерывания. Демон `irqbalance(1)` попытается распределить прерывания между процессорами так, чтобы обеспечить максимальную производительность. Кроме того, можно вручную установить связь между процессорами и IRQ через файлы `/proc/irq/IRQ/smp_affinity`.

6.9.8. Управление ресурсами

Помимо привязки процессов к процессорам, современные ОС предлагают средства управления ресурсами, позволяющие с высокой точностью настроить распределение потребления CPU.

В Linux есть группы управления (сgroups), которые могут управлять потреблением ресурсов процессами или группами процессов. Потребление процессора можно контролировать с помощью общих ресурсов, а планировщик CFS позволяет устанавливать фиксированные ограничения (*пропускную способность процессора*) в микросекундах процессорного времени.

Глава 11 «Облачные вычисления» описывает вариант управления потреблением процессора гостевыми ОС и то, как можно использовать общие доли и ограничения.

6.9.9. Параметры безопасной загрузки

Различные средства защиты ядра от уязвимостей Meltdown и Spectre имеют побочный эффект в виде снижения производительности. В некоторых сценариях, когда безопасностью можно пренебречь и нужна высокая производительность, такие средства лучше отключить. Поскольку этот подход не рекомендуется (из-за угрозы безопасности), я не буду перечислять здесь все параметры, но вы должны знать, что они есть. Это параметры загрузчика `grub`, в том числе `nospectre_v1` и `nospectre_v2`.

Они задокументированы в файле `Documentation/admin-guide/kernel-parameters.txt` в дереве исходных текстов Linux [Linux 20f]. Вот выдержка из этого файла:

```
nospectre_v1    [PPC] Отключить защиту от уязвимости Spectre Variant 1 (обход
                  проверки границ). При отключении этой защиты возможны утечки данных
                  в системе.

nospectre_v2    [X86, PPC_FSL_BOOK3E, ARM64] Отключить защиту от уязвимости
                  Spectre Variant 2 (косвенное предсказание ветвлений). При
                  отключении этой защиты система может допустить утечку данных.
```

6.9.10. Параметры процессора (настройка BIOS)

Процессоры предоставляют настройки для включения, отключения и коррекции некоторых их особенностей. В системах x86 они обычно доступны через меню настроек BIOS.

Как правило, настройки по умолчанию обеспечивают максимальную производительность: и их не нужно изменять. Я иногда отключаю только одну настройку — Intel Turbo Boost, чтобы бенчмарки выполнялись с постоянной тактовой частотой (но имейте в виду: на промышленных серверах настройка Turbo Boost должна быть включена, чтобы обеспечить чуть более высокую производительность).

6.10. УПРАЖНЕНИЯ

1. Ответьте на следующие вопросы по терминологии процессоров:
 - В чем разница между процессом и процессором?
 - Что такое аппаратный поток?
 - Что такое очередь выполнения?
 - В чем разница между временем выполнения в пространстве пользователя и в пространстве ядра?
2. Ответьте на следующие концептуальные вопросы:
 - Опишите такие характеристики процессора, как потребление и насыщенность.
 - Расскажите, каким образом вычислительный конвейер увеличивает пропускную способность процессора.
 - Опишите, как ширина инструкций процессора увеличивает пропускную способность процессора.
 - Опишите преимущества многопроцессорных и многопоточных моделей.
3. Ответьте на следующие, более сложные вопросы:
 - Опишите, что происходит, когда системные процессоры перегружены заданиями, готовыми к выполнению, и как это влияет на производительность приложений.
 - Что делают процессоры в отсутствие заданий, готовых к выполнению?

- Какие признаки говорят о наличии возможных проблем с производительностью процессоров? Назовите две методики, которые вы использовали бы в первую очередь, и объясните почему.
4. Разработайте следующие процедуры для своей среды:
 - Чек-лист для метода USE с целью анализа потребления процессора. Опишите, как получить каждую метрику (например, какую команду выполнить) и как интерпретировать результаты. Перед установкой или использованием дополнительных программных продуктов постарайтесь ограничиться инструментами наблюдения, имеющимися в ОС.
 - Чек-лист для определения характеристик рабочей нагрузки с точки зрения потребления процессора. Опишите, как получить каждую метрику, и для начала попробуйте ограничиться инструментами наблюдения, имеющимися в операционной системе.
 5. Решите задачи:
 - Вычислите среднюю нагрузку для следующей системы с постоянной общей нагрузкой и несущественной нагрузкой ввода/вывода:
 - в системе есть 64 процессора;
 - нагрузка на процессор в масштабе всей системы составляет 50 %;
 - насыщенность процессора в масштабе всей системы, измеренная как общее количество выполняемых потоков и потоков, находящихся в очереди, в среднем составляет 2,0.
 - Выберите приложение и выполните профилирование потребления процессора на уровне пользователя. Покажите, какие пути в коде потребляют большую часть процессорного времени.
 6. (опционально) Разработайте `bustop(1)` — инструмент, показывающий нагрузку на физическую шину или соединения между процессорами, который действует как `iostat(1)` и выводит список шин, столбцы с измеренной пропускной способностью в каждом направлении и нагрузкой. По возможности добавьте вывод метрик с величиной насыщенности и количеством ошибок. Для этого используйте счетчик РМС.

6.11. ССЫЛКИ

- [Saltzer 70] Saltzer, J., and Gintell, J., «The Instrumentation of Multics», Communications of the ACM, August 1970.
- [Bobrow 72] Bobrow, D. G., Burchfiel, J. D., Murphy, D. L., and Tomlinson, R. S., «TENEX: A Paged Time Sharing System for the PDP-10*», Communications of the ACM, March 1972.
- [Myer 73] Myer, T. H., Barnaby, J. R., and Plummer, W. W., TENEX Executive Manual, Bolt, Baranek and Newman, Inc., April 1973.
- [Thomas 73] Thomas, B., «RFC 546: TENEX Load Averages for July 1973», Network Working Group, <http://tools.ietf.org/html/rfc546>, 1973.

- [TUHS 73] «V4», The Unix Heritage Society, <http://minnie.tuhs.org/cgi-bin/utree.pl?file=V4>, materials from 1973.
- [Hinnant 84] Hinnant, D., «Benchmarking UNIX Systems», BYTE magazine 9, no. 8, August 1984.
- [Bulpin 05] Bulpin, J., and Pratt, I., «Hyper-Threading Aware Process Scheduling Heuristics», USENIX, 2005.
- [Corbet 06a] Corbet, J., «Priority inheritance in the kernel», LWN.net, <http://lwn.net/Articles/178253>, 2006.
- [Otto 06] Otto, E., «Temperature-Aware Operating System Scheduling», University of Virginia (Thesis), 2006.
- [Ruggiero 08] Ruggiero, J., «Measuring Cache and Memory Latency and CPU to Memory Bandwidth», Intel (Whitepaper), 2008.
- [Intel 09] «An Introduction to the Intel QuickPath Interconnect», Intel (Whitepaper), 2009.
- [Levinthal 09] Levinthal, D., «Performance Analysis Guide for Intel® Core™ i7 Processor and Intel® Xeon™ 5500 Processors», Intel (Whitepaper), 2009.
- [Gregg 10a] Gregg, B., «Visualizing System Latency», Communications of the ACM, July 2010.
- [Weaver 11] Weaver, V., «The Unofficial Linux Perf Events Web-Page», http://web.eece.maine.edu/~vweaver/projects/perf_events, 2011.
- [McVoy 12] McVoy, L., «LMBench — Tools for Performance Analysis», <http://www.bitmover.com/lmbench>, 2012.
- [Stevens 13] Stevens, W. R., and Rago, S., «Advanced Programming in the UNIX Environment, 3rd Edition», Addison-Wesley 2013¹.
- [Perf 15] «Tutorial: Linux kernel profiling with perf», perf wiki, <https://perf.wiki.kernel.org/index.php/Tutorial>, последнее обновление 2015.
- [Gregg 16b] Gregg, B., «The Flame Graph», Communications of the ACM, Volume 59, Issue 6, pp. 48–57, June 2016.
- [ACPI 17] Advanced Configuration and Power Interface (ACPI) Specification, https://uefi.org/sites/default/files/resources/ACPI%206_2_A_Sept29.pdf, 2017.
- [Gregg 17b] Gregg, B., «CPU Utilization Is Wrong», <http://www.brendangregg.com/blog/2017-05-09/cpu-utilization-is-wrong.html>, 2017.
- [Gregg 17c] Gregg, B., «Linux Load Averages: Solving the Mystery», <http://www.brendangregg.com/blog/2017-08-08/linux-load-averages.html>², 2017.
- [Mulnix 17] Mulnix, D., «Intel® Xeon® Processor Scalable Family Technical Overview», <https://software.intel.com/en-us/articles/intel-xeon-processor-scalable-family-technical-overview>, 2017.
- [Gregg 18b] Gregg, B., «Netflix FlameScope», Netflix Technology Blog, <https://netflixtechblog.com/netflix-flamescope-a57ca19d47bb>, 2018.

¹ Уильям Ричард Стивенс, Стивен А. Раго. «UNIX. Профессиональное программирование. 3-е изд.», СПб., издательство «Питер».

² Перевод на русский язык: <https://bjdarticles.blogspot.com/2019/07/httpwww.html>. — *Примеч. пер.*

- [Ather 19] Ather, A., «General Purpose GPU Computing», <http://techblog.cloudperf.net/2019/12/general-purpose-gpu-computing.html>, 2019.
- [Gregg 19] Gregg, B., «BPF Performance Tools: Linux System and Application Observability»¹, Addison-Wesley, 2019.
- [Intel 19a] Intel 64 and IA-32 Architectures Software Developer's Manual, Combined Volumes 1, 2A, 2B, 2C, 3A, 3B, and 3C. Intel, 2019.
- [Intel 19b] Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3B, System Programming Guide, Part 2. Intel, 2019.
- [Netflix 19] «FlameScope Is a Visualization Tool for Exploring Different Time Ranges as Flame Graphs», <https://github.com/Netflix/flamescope>, 2019.
- [Wysocki 19] Wysocki, R., «CPU Idle Time Management», Linux documentation, <https://www.kernel.org/doc/html/latest/driver-api/pm/cpuidle.html>, 2019.
- [AMD 20] «AMD μ Prof», <https://developer.amd.com/amd-uprof>, по состоянию на 2020.
- [Gregg 20d] Gregg, B., «MSR Cloud Tools», <https://github.com/brendangregg/msr-cloudtools>, последнее обновление 2020.
- [Gregg 20e] Gregg, B., «PMC (Performance Monitoring Counter) Tools for the Cloud», <https://github.com/brendangregg/pmc-cloud-tools>, последнее обновление 2020.
- [Gregg 20f] Gregg, B., «perf Examples», <http://www.brendangregg.com/perf.html>, по состоянию на 2020.
- [Gregg 20g] Gregg, B., «FlameGraph: Stack Trace Visualizer», <https://github.com/brendangregg/FlameGraph>, последнее обновление 2020.
- [Intel 20a] «Product Specifications», <https://ark.intel.com>, по состоянию на 2020.
- [Intel 20b] «Intel® VTune™ Profiler», <https://software.intel.com/content/www/us/en/develop/tools/vtune-profiler.html>, по состоянию на 2020.
- [Iovisor 20a] «bpfttrace: High-level Tracing Language for Linux eBPF», <https://github.com/iovisor/bpfttrace>, последнее обновление 2020.
- [Linux 20f] «The Kernel's Command-Line Parameters», Linux documentation, <https://www.kernel.org/doc/html/latest/admin-guide/kernel-parameters.html>, по состоянию на 2020.
- [Spier 20a] Spier, M., «A D3.js Plugin That Produces Flame Graphs from Hierarchical Data», <https://github.com/spiermar/d3-flame-graph>, последнее обновление 2020.
- [Spier 20b] Spier, M., «Template», <https://github.com/spiermar/d3-flame-graph#template>, последнее обновление 2020.
- [Valgrind 20] «Valgrind Documentation», <http://valgrind.org/docs/manual>, May 2020.
- [Verma 20] Verma, A., «CUDA Cores vs Stream Processors Explained», <https://graphicscardhub.com/cuda-cores-vs-stream-processors>, 2020.

¹ Грегг Б. «BPF: Профессиональная оценка производительности». Выходит в издательстве «Питер» в 2023 году.

Глава 7

ПАМЯТЬ

В основной памяти системы хранятся инструкции приложения и ядра, их рабочие данные и кэши файловой системы. Вторичные хранилища для этих данных — устройства хранения (диски), которые работают намного медленнее. После заполнения основной памяти система может начать перекачивать данные из основной памяти на устройства хранения и обратно. Это очень медленный процесс, который часто становится узким местом системы, резко снижая производительность. Система также может завершить процесс, потребляющий наибольший объем памяти, что повлечет сбой в работе приложений.

При анализе производительности следует учитывать затраты процессорного времени на выделение и освобождение памяти, копирование памяти и управление отображением адресного пространства. В архитектурах с несколькими физическими процессорами еще одним фактором, ограничивающим производительность, может стать локальность памяти, так как память, подключенная к физическим процессорам, имеет меньшую задержку доступа, чем удаленная память.

Цели этой главы:

- дать основные понятия, касающиеся памяти;
- познакомить с внутренним устройством памяти;
- познакомить с внутренним устройством ядра ОС и механизма распределения памяти;
- дать практические сведения о MMU и TLB;
- перечислить различные методики анализа потребления памяти;
- охарактеризовать использование памяти в масштабах всей системы и каждого процесса;
- описать возможные проблемы, вызываемые нехваткой доступной памяти;
- показать, как определять характер потребления памяти в адресном пространстве процесса и ядра;
- показать, как анализировать потребление памяти с использованием профилировщиков, трассировщиков и флейм-графиков;
- познакомить с параметрами настройки памяти.

Глава состоит из пяти частей. В первых трех я расскажу об основах анализа потребления памяти, а в следующих двух покажу их практическое применение в системах на базе Linux.

- **Основы** — представляет терминологию, имеющую отношение к памяти, и ключевые идеи производительности памяти.
- **Архитектура** — описывает общую аппаратную и программную архитектуру памяти.
- **Методология** — описывает методологию анализа производительности.
- **Инструменты наблюдения** — описывает инструменты анализа производительности памяти.
- **Настройка** — включает примеры настраиваемых параметров.

Внутренние кэши процессоров (уровни 1/2/3, TLB) рассматриваются в главе 6 «Процессоры».

7.1. ТЕРМИНОЛОГИЯ

Ниже перечислены основные термины, связанные с памятью и используемые в этой главе:

- **Основная память**, также называемая *физической памятью*, — это высокопроизводительное хранилище данных в компьютере, которое обычно называют ОЗУ (или DRAM).
- **Виртуальная память** — абстракция основной памяти, которая (почти) бесконечна и неисчерпаема. Виртуальная память — это не настоящая память.
- **Резидентная память** — память, которая в настоящее время располагается в основной памяти.
- **Анонимная память** — память, не имеющая имени или пути в файловой системе. Включает рабочие данные из адресного пространства процесса, называемые *кучей*.
- **Адресное пространство** — контекст памяти. Для каждого процесса и ядра есть свои виртуальные адресные пространства.
- **Сегмент** — область виртуальной памяти, предназначенная для определенной цели, например для хранения выполняемого кода или данных.
- **Машинные инструкции** — инструкции, находящиеся в памяти, обычно в сегменте, и выполняемые процессором.
- **ООМ (Out Of Memory)** — ошибка нехватки памяти, возникает, когда ядро обнаруживает нехватку доступной памяти.
- **Страница** — единица памяти, используемая операционной системой и процессорами. Традиционно страницы имеют размер 4 или 8 Кбайт. Современные процессоры *поддерживают несколько размеров страниц*.

- **Сбой страницы (page fault)** — ошибка обращения к недоступной памяти. Это нормальное явление при использовании виртуальной памяти.
- **Подкачка страниц (paging)** — передача страниц между основной памятью и запоминающими устройствами.
- **Подкачка** — термин «подкачка» (swapping) в Linux обозначает *анонимную подкачку страниц* с устройства подкачки (передача страниц). В Unix и других ОС под подкачкой подразумевается передача целых процессов между основной памятью и устройствами подкачки. В этой книге используется версия термина для Linux.
- **Своп (swap)** — область на диске для хранения выгружаемых анонимных данных. Это может быть область на запоминающем устройстве, которую называют *физическим устройством подкачки*, или файл в файловой системе, который называют *файлом подкачки*. В некоторых инструментах термин *своп* (swap) используется для обозначения виртуальной памяти (что сбивает с толку и в корне неверно).

В главе будут вводиться и другие термины. Глоссарий включает базовую терминологию, в том числе *адрес*, *буфер* и *ОЗУ*. Также см. разделы «Терминология» в главах 2 и 3.

7.2. ОСНОВНЫЕ ПОНЯТИЯ

Ниже описываются некоторые важные понятия, касающиеся памяти и ее производительности.

7.2.1. Виртуальная память

Виртуальная память — это абстракция, которая предоставляет каждому процессу и ядру отдельное большое и линейное адресное пространство. Виртуальная память упрощает разработку ПО, оставляя управление распределением физической памяти за операционной системой. Она также поддерживает многозадачность (виртуальные адресные пространства разделены по определению) и превышение ограничений (объем используемой памяти может превышать объем основной памяти). Первое знакомство с виртуальной памятью состоялось в главе 3 «Операционные системы», в разделе 3.2.8 «Виртуальная память». Историческую справку см. в [Denning 70].

На рис. 7.1 показана организация виртуальной памяти процесса в системе с устройством подкачки (вторичным хранилищем). Здесь изображена страница памяти, потому что большинство реализаций виртуальной памяти основаны на страницах.

Адресное пространство процесса отображается подсистемой виртуальной памяти в основную память и физическое устройство подкачки. По мере необходимости ядро может перемещать страницы памяти между ними — этот процесс в Linux называют *подкачкой* (swapping, а в других ОС — *анонимной подкачкой страниц*). Это позволяет ядру использовать памяти больше, чем имеется в наличии основной памяти.

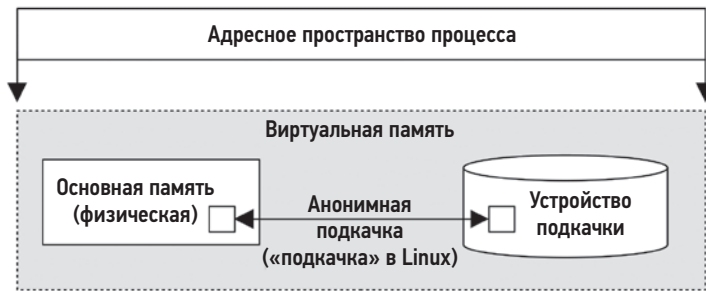


Рис. 7.1. Виртуальная память процесса

Ядро может ограничивать общий объем виртуальной памяти. Обычно этот предел определяется размером основной памяти и емкостью физических устройств подкачки. Ядро может потерпеть неудачу при попытке выделить память выше этого предела. Такие ошибки «нехватки виртуальной памяти» иногда могут сбивать с толку, потому что виртуальная память сама по себе абстрактный ресурс.

В Linux также допускаются другие варианты поведения, в том числе *отсутствие* ограничений на выделение памяти. Эта стратегия называется *overcommit* (чрезмерное выделение памяти) и описывается несколькими разделами ниже, после обсуждения механизмов подкачки страниц и подкачки по требованию, которые используются стратегией *overcommit*.

7.2.2. Подкачка страниц

Подкачка страниц заключается в перемещении страниц в основную память и из нее; это называют *загрузкой* (page-in) и *выгрузкой* (page-out) соответственно. Первые подкачка страниц была реализована компанией Atlas Computer в 1962 году [Corbató 68], что позволило:

- выполнять частично загруженные программы;
- выполнять программы, размер которых превышает объем основной памяти;
- эффективно перемещать программы между основной памятью и устройствами хранения.

Эти возможности актуальны и по сей день. В отличие от более ранней методики подкачки программ целиком, подкачка страниц — более детализированный подход к управлению основной памятью, так как размер страницы относительно мал (например, 4 Кбайт).

Поддержка подкачки страниц в виртуальной памяти (*страничная виртуальная память*) появилась в Unix BSD [Babaoglu 79] и стала стандартом.

С последующим добавлением кэша страниц для совместного использования страниц файловой системы (см. главу 8 «Файловые системы») стали доступны два типа

подкачки страниц: *подкачка страниц файловой системы* и *анонимная подкачка страниц*.

Подкачка страниц файловой системы

Подкачка страниц файловой системы происходит при чтении и записи страниц в файлах, отображаемых в память. Это нормальное поведение для приложений, использующих отображение файлов в память (`mmap(2)`), и для файловых систем, использующих кэш страниц (большинства из них; см. главу 8 «Файловые системы»). Ее называют «хорошей» подкачкой страниц [McDougall 06a].

При необходимости ядро может освободить память, выгружая некоторые страницы. Здесь терминология усложняется: если страница файловой системы была изменена в основной памяти (такие страницы называют *грязными* — *dirty*), то для выгрузки страницы потребуются записать ее на диск. Если страница файловой системы не была изменена (такие страницы называют *чистыми* — *clean*), то память, занимаемая страницей, просто освобождается и немедленно становится доступной для повторного использования, потому что копия страницы уже хранится на диске. По этой причине термин *выгрузка страницы* (page-out) означает удаление страницы из памяти с возможной ее записью в устройство хранения (в других книгах может встретиться другое определение «выгрузки страницы»).

Анонимная подкачка страниц (swapping)

Анонимная подкачка страниц предполагает загрузку/выгрузку собственных данных процессов: динамическую память (кучу) и стек. Она называется *анонимной* из-за отсутствия именованного местоположения в ОС (то есть пути в файловой системе). Анонимная выгрузка страниц требует перемещения данных в физические устройства подкачки или файлы подкачки. В Linux этот тип подкачки называют просто *подкачкой* (swapping).

Анонимная подкачка страниц снижает производительность, и поэтому ее называют «плохой» [McDougall 06a]. Приложения, обращающиеся к страницам памяти, которые были выгружены, блокируются в операциях дискового ввода/вывода, выполняющих чтение страниц обратно в основную память¹. Такая *анонимная загрузка страниц* вызывает синхронную задержку в работе приложения. Анонимная выгрузка страниц может не оказывать прямого влияния на производительность приложения, так как операции записи выполняются ядром асинхронно.

Для производительности лучше, когда анонимной подкачки страниц (swapping) нет. Этого можно добиться, настраивая приложения так, чтобы они оставались в доступной основной памяти, и наблюдая за сканированием страниц, потреблением

¹ Если в качестве устройств подкачки используются быстрые запоминающие устройства, такие как 3D XPoint с задержкой менее 10 мкс, то подкачка может оказаться не такой «плохой», как раньше, а скорее простым способом расширения основной памяти со зрелой поддержкой в ядре.

памяти и анонимной подкачкой страниц, чтобы убедиться в отсутствии признаков нехватки памяти.

7.2.3. Подкачка страниц по требованию

Операционные системы, поддерживающие подкачку страниц по запросу (большинство из них), отображают страницы виртуальной памяти в физическую память по требованию, как показано на рис. 7.2. При таком подходе затраты процессорного времени на создание отображений откладываются до момента, когда эти страницы действительно потребуются (при первом обращении к ним), а не при первой попытке выделить диапазон памяти.

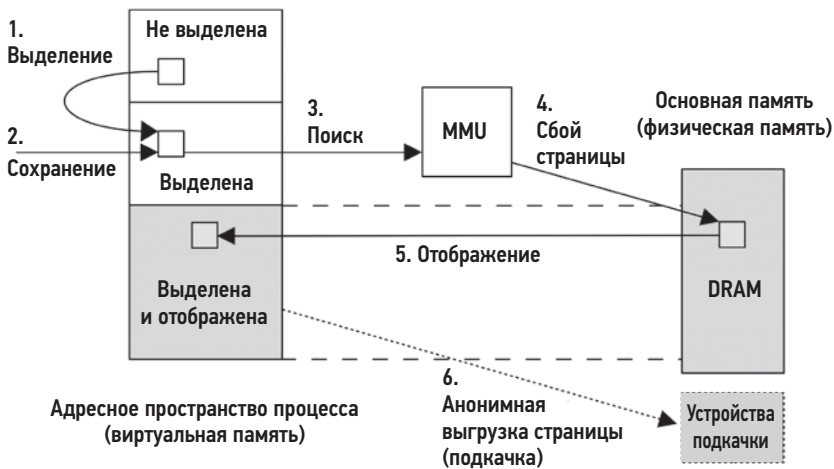


Рис. 7.2. Пример сбоя страницы

Последовательность, показанная на рис. 7.2, начинается с выделения памяти — вызова функции `malloc()` (шаг 1), которая предоставляет выделенную память, после чего выполняется инструкция сохранения данных (шаг 2) в эту вновь выделенную память. Чтобы блок управления памятью определил место в основной памяти, выполняется поиск страницы в физической памяти (шаг 3), который терпит неудачу, потому что эта виртуальная страница еще не отображена в физическую память. Эта неудача, которая называется *сбоем страницы* (шаг 4), вынуждает ядро создать отображение по требованию (шаг 5). Какое-то время спустя страница памяти может быть выгружена в устройства подкачки для освобождения основной памяти (шаг 6).

Шаг 2 может инициироваться инструкцией загрузки, как в случае отображения файла в память, который содержит требуемые процессу данные, но еще не был отображен в его адресное пространство.

Если отображение можно выполнить, начиная с другой страницы в памяти, то это называется *минорным сбоем* (minor fault). Это может происходить при отображении

новой страницы из доступной памяти во время увеличения объема памяти процесса (как показано на рисунке). Это также может произойти при отображении в другую существующую страницу, например, при чтении страницы из разделяемой библиотеки.

Сбои страниц, требующие доступа к запоминающему устройству (на рисунке не показаны), как, например, доступ к некешированному файлу, отображаемому в память, называются *мажорным сбоем* (major fault).

В результате использования модели виртуальной памяти и распределения по требованию любая страница виртуальной памяти может быть в одном из следующих состояний:

- A. Не распределена.
- B. Выделена, но не отображена (не заполнена и еще не потерпела сбоя).
- C. Выделена и отображена в основную память (ОЗУ).
- D. Выделена и отображена в физическое устройство подкачки (диск).

Состояние (D) наступает, когда страница выгружается из-за нехватки системной памяти. Переход из состояния (B) в состояние (C) является сбоем страницы. Если при этом возникает необходимость выполнить дисковый ввод/вывод, то это мажорный сбой страницы, в противном случае — минорный сбой страницы.

На основе этих состояний можно определить два термина, имеющих отношение к использованию памяти:

- **Размер резидентного набора** (resident set size, RSS): размер страниц, размещенных в основной памяти (C).
- **Размер виртуальной памяти**: размер всех выделенных областей (B + C + D).

Подкачка страниц по требованию была добавлена в Unix BSD вместе с поддержкой страничной виртуальной памяти. Она стала стандартом и используется в Linux.

7.2.4. Чрезмерное выделение памяти

Linux поддерживает возможность *чрезмерного выделения памяти* (overcommit), позволяющую выделить больше памяти, чем система может хранить, — больше объема физической памяти и емкости устройства подкачки, вместе взятых. Она основана на подкачке страниц по требованию и тенденции приложений не использовать большую часть выделенной ими памяти.

При поддержке чрезмерного выделения памяти запросы приложений на получение дополнительной памяти (например, вызовом `malloc(3)`) всегда будут успешными; даже если в отсутствие такой поддержки они потерпели бы неудачу. Вместо консервативного выделения памяти, чтобы не выйти за пределы виртуальной памяти, программист может щедро выделять память в своем приложении, а затем использовать ее редко и по требованию.

В Linux стратегию `overcommit` можно настроить с помощью параметра. Более подробно об этом см. в разделе 7.6 «Настройка». Последствия чрезмерного выделения памяти зависят от того, как ядро справляется с нехваткой памяти. См. часть про компонент ядра Out-Of-Memory Killer (OOM Killer) в разделе 7.3 «Архитектура».

7.2.5. Подкачка процессов

Подкачка процессов (`process swapping`) — это перемещение целых процессов между основной памятью и физическим устройством или файлом подкачки. Эта методика зародилась в Unix и была предназначена для управления основной памятью. Именно от нее произошел термин «своп» (`swap`) [Thompson 78].

Чтобы выгрузить процесс, все его данные должны быть записаны в устройство подкачки, включая динамическую память (анонимные данные), таблицу открытых файлов и другие метаданные, которые нужны, только когда процесс активен. Данные, полученные из файловых систем и не изменившиеся, при необходимости могут быть просто выброшены, так как есть возможность прочитывать их из исходного местоположения.

Подкачка процессов серьезно снижает производительность, потому что для возобновления выгруженного процесса требуется выполнить множество операций ввода/вывода с диском. Она имела определенный смысл для машин прошлого времени, например PDP-11 на ранних версиях Unix. Максимальный размер процесса составлял тогда всего 64 Кбайт [Bach 86]. (В современных системах размеры процессов измеряются гигабайтами.)

Описание этой методики дано исключительно для исторической справки. Системы Linux вообще не поддерживают подкачку процессов и полагаются только на подкачку страниц.

7.2.6. Использование кэша файловой системы

Увеличение потребления памяти после загрузки системы — нормальное явление, потому что ОС использует доступную память для кэширования файловой системы, чтобы повысить производительность. При этом используется простой принцип: если есть свободная основная память, ее можно использовать для чего-нибудь полезного. Это может огорчить неискушенных пользователей, наблюдающих, как после загрузки объем доступной свободной памяти постепенно сокращается почти до нуля. Но это не проблема для приложений, потому что ядро способно быстро освобождать память, занятую кэшем файловой системы, и передавать ее приложениям.

Дополнительные сведения о различных кэшах файловых систем, которые могут использовать основную память, см. в главе 8 «Файловые системы».

7.2.7. Потребление и насыщение

Потребление основной памяти можно рассчитать как отношение объема использованной памяти к общему объему памяти. Память, занятую кэшем файловой

системы, можно рассматривать как неиспользуемую, потому что она доступна для повторного использования приложениями.

Если потребность в памяти превышает объем основной памяти, наступает *насыщение* основной памяти. В этом случае ОС может освободить память, используя подкачку страниц, подкачку процессов (если поддерживается), а в Linux — задействовать механизм OOM Killer (описывается ниже). Любое из этих действий — это показатель насыщения основной памяти.

Виртуальную память тоже можно изучить с точки зрения потребления, если система накладывает ограничение на объем виртуальной памяти (в Linux стратегия *overcommit* не поддерживается). Если объем виртуальной памяти ограничен, то после его исчерпания ядро не сможет выделить память. Например, `malloc(3)` потерпит неудачу и вернет ошибку `ENOMEM` в `errno`.

Обратите внимание, что доступную в настоящий момент виртуальную память иногда (не совсем точно) называют *доступным пространством в свопе* (*available swap*).

7.2.8. Распределители

Виртуальная память обеспечивает поддержку многозадачности в физической памяти, но фактическое распределение и размещение в виртуальном адресном пространстве часто реализуется распределителями (*allocators*). Это могут быть библиотеки пользовательского уровня или процедуры в ядре, предлагающие программистам простой интерфейс для использования памяти (например, `malloc(3)`, `free(3)`).

Распределители могут существенно влиять на производительность, и система может предоставлять несколько библиотек распределителей уровня пользователя. Разные распределители могут давать более высокую производительность за счет использования кэширования объектов по потокам, но также могут снижать производительность, если в результате работы распределителя увеличивается фрагментированность памяти и избыточное ее расходование. Конкретные примеры ищите в разделе 7.3 «Архитектура».

7.2.9. Разделяемая память

Память может совместно использоваться процессами. Это особенно характерно при использовании системных библиотек, когда для экономии в памяти размещается единственная копия библиотеки, доступная только для чтения всем процессам, которые ее используют.

Это создает определенные сложности для инструментов наблюдения, показывающих потребление основной памяти каждым процессом, например: следует ли включать такую разделяемую память в отчет об общем потреблении памяти процессом? Один из методов, используемых в Linux, заключается в предоставлении дополнительной меры — *пропорционального размера набора* (*proportional set size, PSS*), включающего собственную память процесса (неразделяемую) плюс объем разделяемой памяти,

деленный на количество потребителей. Описание инструмента, отображающего метрику PSS, ищите в разделе 7.5.9 «rmap».

7.2.10. Размер рабочего набора

Размер рабочего набора (working set size, WSS) — это объем основной памяти, часто используемый процессом в работе. Это полезное понятие для настройки производительности памяти: производительность должна значительно улучшиться, если WSS умещается в кэши процессора. И наоборот, производительность значительно ухудшится, если WSS превысит емкость основной памяти и в процессе работы приложения системе придется прибегнуть к подкачке страниц.

Несмотря на полезность, рабочий размер трудно измерить на практике: в инструментах наблюдения нет статистики WSS (обычно они оценивают RSS, а не WSS). В разделе 7.4.10 «Уменьшение потребления памяти» описывается экспериментальная методология оценки WSS, а в разделе 7.5.12 «wss» представлен инструмент wss(8) для оценки размера рабочего набора экспериментальным путем.

7.2.11. Размер слова

Как отмечалось в главе 6 «Процессоры», процессоры могут поддерживать разные размеры слов, например 32- и 64-разрядные, что позволяет запускать ПО с любыми из них. Поскольку размер адресного пространства зависит от размера слова, приложения, требующие более 4 Гбайт памяти, слишком велики для 32-разрядного адресного пространства и должны быть скомпилированы с поддержкой слов размером 64 или более разрядов¹.

В зависимости от ядра и процессора часть адресного пространства может быть зарезервирована для ядра и недоступна для использования приложениями. Крайний случай — Windows с 32-разрядным размером слова, в которой по умолчанию для ядра зарезервировано 2 Гбайт и приложениям остается всего 2 Гбайт. В Linux (или Windows с параметром /3GB) за ядром резервируется 1 Гбайт памяти. При 64-разрядном размере слова (если CPU поддерживает это) адресное пространство настолько большое, что резервирование памяти для ядра не проблема.

В зависимости от архитектуры процессора увеличение разрядности тоже способствует увеличению производительности памяти, потому что инструкции могут работать со словами большего размера. Правда, при этом некоторый небольшой объем памяти может тратиться впустую, когда типы данных используют не все биты в словах с большей разрядностью.

¹ Есть механизм расширения физических адресов (physical address extension, PAE) для архитектуры x86, позволяющий 32-разрядным процессорам использовать более обширные диапазоны памяти (но не в одном процессе).

7.3. АРХИТЕКТУРА

В этом разделе описывается архитектура памяти, как аппаратная, так и программная, включая особенности процессора и операционной системы.

Здесь я обобщу эти темы, чтобы заложить фундамент для анализа производительности. Дополнительные сведения смотрите в руководствах по процессорам от производителей и в документации с описанием внутреннего устройства ОС. Некоторые из них перечислены в конце этой главы.

7.3.1. Аппаратное обеспечение

Аппаратное обеспечение памяти включает основную память, шины, кэши процессора и блок управления памятью.

Основная память

Наиболее распространенный тип основной памяти — *динамическая память с произвольным доступом* (dynamic random-access memory, DRAM). Это энергозависимая память, теряющая свое содержимое при отключении питания. DRAM обеспечивает высокую плотность хранения, потому что каждый бит реализован с помощью всего двух логических компонентов: конденсатора и транзистора. Конденсатор требует периодической подзарядки.

Корпоративные серверы снабжаются разными объемами DRAM в зависимости от назначения, обычно от одного гигабайта до одного терабайта и больше. Облачные экземпляры обычно получают меньший объем памяти, от 512 Мбайт до 256 Гбайт каждый¹. Но облачные среды обеспечивают возможность распределения нагрузки по пулу экземпляров, поэтому вместе могут предоставлять распределенному приложению намного больше оперативной памяти, хотя и с гораздо более высокими затратами на согласование.

Задержка

Время доступа к основной памяти можно измерить как задержку *строба адреса столбца* (column address strobe, CAS): время между передачей желаемого адреса (столбца) блоку памяти и моментом, когда данные станут доступны для чтения. Это зависит от типа памяти (для DDR4 задержка составляет от 10 до 20 нс [Crucial 18]). Для операций чтения/записи с памятью эта задержка может увеличиваться в несколько раз из-за необходимости использования шины памяти (например, шириной 64 бита) для передачи строки кэша (например, длиной 64 *байта*). Есть также другие задержки, которые вносятся процессором и блоком MMU при последующем чтении вновь доступных данных. Инструкции чтения избегают этих задержек, если данные

¹ Исключение составляют экземпляры AWS EC2, в которых объем памяти достигает 24 Тбайт [Amazon 20].

возвращаются из кэша процессора. Инструкции записи тоже могут их избежать, если процессор поддерживает кэширование с отложенной записью (как, например, процессоры Intel).

Архитектура основной памяти

На рис. 7.3 показан пример архитектуры основной памяти для универсальной двухпроцессорной системы с *унифицированным доступом к памяти* (uniform memory access, UMA).

Все процессоры имеют одинаковую задержку доступа ко всей памяти через общую системную шину. При управлении одним экземпляром ядра ОС, которое равномерно использует все процессоры, эта архитектура также является симметричной многопроцессорной (symmetric multiprocessing, SMP).

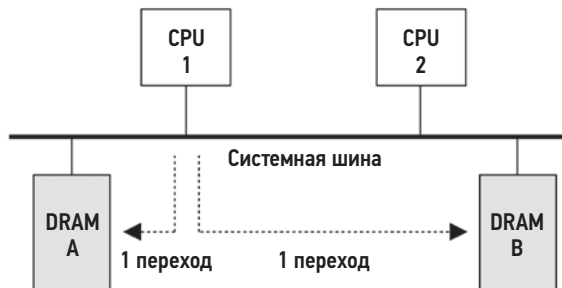


Рис. 7.3. Пример архитектуры основной памяти с UMA в двухпроцессорной системе

На рис. 7.4 показан пример двухпроцессорной системы с *неоднородным доступом к памяти* (non-uniform memory access, NUMA), где используется внутренняя шина между процессорами, являющаяся частью архитектуры памяти. Время доступа к основной памяти в этой архитектуре зависит от ее расположения относительно процессоров.



Рис. 7.4. Пример основной памяти в архитектуре NUMA с двумя процессорами

Процессор 1 может выполнять чтение/запись в DRAM A напрямую через свою шину памяти. Эта память называется *локальной памятью*. Процессор 1 выполняет

чтение/запись в DRAM В через процессор 2 и внутреннюю шину между процессорами (два перехода). Эта память называется *удаленной памятью* и имеет более высокую задержку доступа.

Банки памяти, подключенные к каждому процессору, называются *узлами памяти*, или просто *узлами*. ОС может определить топологию узла памяти, опираясь на информацию, предоставленную процессором, и затем выбирать память и планировать потоки в зависимости от местоположения памяти, отдавая предпочтение локальной памяти для повышения производительности.

Шины

Как было показано выше, порядок физического подключения основной памяти к системе зависит от архитектуры основной памяти. Фактическая реализация может включать дополнительные контроллеры и шины между процессорами и памятью. Доступ к основной памяти можно получить одним из способов:

- **Общая системная шина:** в одно- или многопроцессорной системе через общую системную шину, контроллер моста памяти и шину памяти. Эта архитектура изображена на рис. 7.3 как пример UMA и «передней» шины Intel (см. рис. 6.9 в главе 6 «Процессоры»). Контроллером памяти в этом примере служит серверный мост.
- **Прямой:** в однопроцессорной системе, где память подключена напрямую через шину памяти.
- **Внутренняя шина:** в многопроцессорной системе, где каждый процессор напрямую связан со своим банком памяти через шину памяти и процессоры связаны между собой посредством внутренней шины. Эта архитектура изображена на рис. 7.4 как пример NUMA. Внутренняя шина, соединяющая процессоры, обсуждается в главе 6 «Процессоры».

Если вы подозреваете, что архитектура вашей системы не соответствует ни одному из вышеперечисленных примеров, найдите функциональную схему системы и проследите путь, который преодолевают данные между процессорами и памятью, отмечая на этом пути все компоненты.

DDR SDRAM

Скорость шины памяти для любой архитектуры часто определяется стандартом интерфейса памяти, который поддерживается процессором и системной платой. Сейчас широко используется стандарт 1996 года — *синхронная динамическая память с произвольным доступом и удвоением частоты шины данных* (double data rate synchronous dynamic random-access memory, DDR SDRAM). Термин *удвоенная частота шины данных* означает, что данные передаются как по переднему, так и по заднему фронту тактового сигнала (также называется *двойной подкачкой*). Термин *синхронный* подразумевает синхронизацию памяти с процессорами.

В табл. 7.1 приводятся примеры стандартов DDR SDRAM.

Таблица 7.1. Примеры пропускной способности для стандарта DDR

Стандарт	Год публикации	Тактовая частота памяти (МГц)	Скорость передачи (МТ/с)	Пиковая пропускная способность (Мбайт/с)
DDR-200	2000	100	200	1600
DDR-300	2000	167	333	2667
DDR2-667	2003	167	667	5333
DDR2-800	2003	200	800	6400
DDR3-1333	2007	167	1333	10 667
DDR3-1600	2007	200	1600	12 800
DDR4-3200	2012	200	3200	25 600
DDR5-4800	2020	200	4800	38 400
DDR5-6400	2020	200	6400	51 200

Стандарт DDR5 был опубликован 14 июля 2020 года ассоциацией JEDEC Solid State Technology Association. Эти стандарты также обозначаются аббревиатурой «PC-», за которой следует скорость передачи данных в мегабайтах в секунду, например PC-1600.

Многоканальная память

Некоторые системные архитектуры поддерживают возможность одновременного использования нескольких шин памяти для увеличения пропускной способности. Типичные реализации имеют два, три или четыре канала. Например, процессоры Intel Core i7 поддерживают возможность подключения до четырех каналов памяти DDR3-1600 с максимальной пропускной способностью памяти 51,2 Гбайт/с.

Кэши процессоров

Процессоры обычно имеют аппаратные кэши на кристалле для повышения производительности доступа к памяти. Кэши могут включать следующие уровни в порядке уменьшения скорости доступа и увеличения размера:

- **Уровень 1:** обычно разделяется на отдельные кэши инструкций и данных.
- **Уровень 2:** кэш для инструкций и данных.
- **Уровень 3:** еще один уровень кэша большого объема.

В зависимости от процессора на уровень 1 обычно ссылаются адреса в виртуальной памяти, а на уровень 2 и далее — адреса в физической памяти.

Эти кэши подробно обсуждались в главе 6 «Процессоры». В этой главе рассмотрим еще один тип аппаратного кэша: TLB.

MMU

MMU (Memory Management Unit — блок управления памятью) отвечает за преобразование виртуальных адресов в физические. Такие преобразования выполняются для каждой страницы, а смещения внутри страниц отображаются напрямую. MMU был представлен в главе 6 «Процессоры» при обсуждении кэшей, ближайших к процессорам.

На рис. 7.5 изображен типичный блок MMU с кэшами процессора и основной памятью.

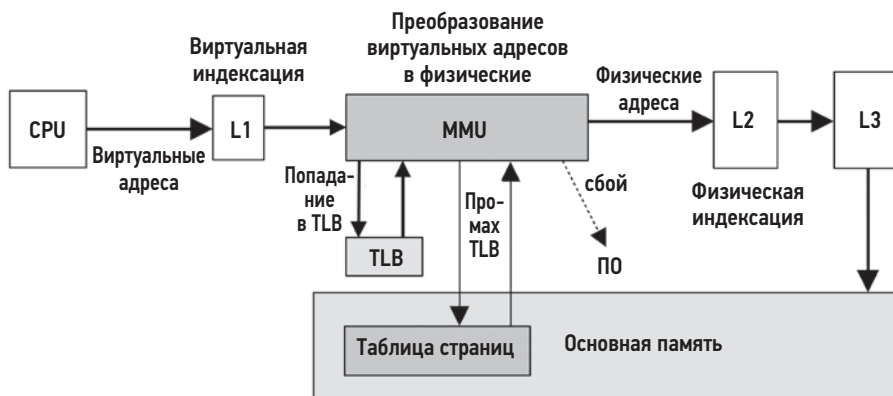


Рис. 7.5. Блок управления памятью

Поддержка нескольких размеров страниц

Современные процессоры поддерживают несколько размеров страниц, что позволяет ОС и MMU использовать страницы разных размеров (например, 4 Кбайт, 2 Мбайт, 1 Гбайт). Механизм *больших страниц* (huge pages) в Linux поддерживает страницы большого размера, например 2 Мбайт или 1 Гбайт.

TLB

MMU, изображенный на рис. 7.5, использует буфер ассоциативной трансляции (translation lookaside buffer, TLB) в качестве кэша первого уровня при трансляции адресов в таблице страниц в основной памяти. У буфера TLB есть отдельные кэши для страниц инструкций и данных.

Поскольку TLB хранит ограниченное количество записей, использование страниц больших размеров увеличивает диапазон адресов в памяти, которые можно получить из его кэша (этот диапазон называется *охватом*), снижает количество промахов TLB и увеличивает производительность системы. Буфер TLB может делиться на отдельные кэши для каждого из этих размеров страницы, повышая вероятность сохранности в кэше более крупных отображений.

В качестве примера в табл. 7.2 представлены четыре буфера TLB в типичном процессоре Intel Core i7 [Intel 19a].

Таблица 7.2. Буферы TLB в типичном процессоре Intel Core i7

Тип	Размер страницы	Записей
Инструкции	4 К	64 на поток, 128 на ядро
Инструкции	большой	7 на поток
Данные	4 К	64
Данные	большой	32

Этот процессор имеет один уровень данных в TLB. Микроархитектура Intel Core поддерживает два уровня, подобно тому как процессоры предоставляют несколько уровней кэша основной памяти.

Точная организация TLB зависит от типа процессора. Подробные сведения о TLB в вашем процессоре и дополнительную информацию о его работе смотрите в руководствах по процессорам от производителей.

7.3.2. Программное обеспечение

В состав ПО для управления памятью входят: система виртуальной памяти, а также механизмы преобразования адресов, подкачки и выделения памяти. В этом разделе мы рассмотрим темы, связанные с производительностью: освобождение памяти, список свободных страниц, сканирование страниц, подкачка, организация адресного пространства процесса и распределители памяти.

Освобождение памяти

Когда объем доступной памяти в системе опускается ниже некоторого порога, ядро использует различные методы освобождения памяти и добавления освободившихся блоков в *список свободных страниц*. На рис. 7.6 показаны методы, используемые в Linux в том порядке, в каком они применяются по мере уменьшения объема доступной памяти.

Вот эти методы:

- **Список свободных страниц:** список страниц, которые не используются (их еще называют *неиспользуемой памятью*) и доступны для немедленного выделения. Обычно такой список реализуется в виде нескольких списков, по одному для каждой группы локальности (NUMA).
- **Кэш страниц:** кэш файловой системы. Настраиваемый параметр *swappiness* (вытесняемость) определяет степень предпочтительности освобождения памяти из кэша страниц вместо подкачки.

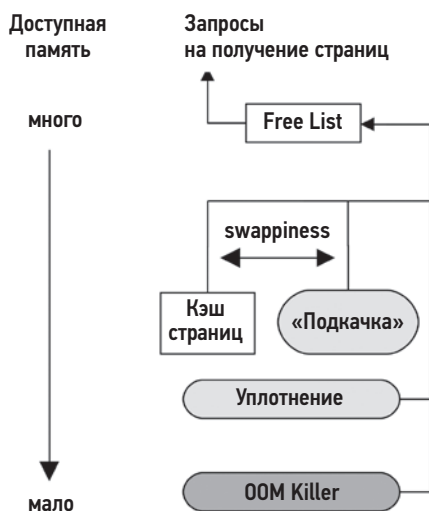


Рис. 7.6. Управление доступностью памяти в Linux

- **Подкачка** (swapping): выгрузка страниц демоном kswapd, который ищет давно не использовавшиеся страницы для добавления в список свободных страниц, включая память приложения. Найденные страницы выгружаются, что может означать запись в файл подкачки или в устройство подкачки. Естественно, подкачка доступна, только если был настроен файл или устройство подкачки.
- **Уплотнение** (reaping): при уменьшении объема доступной памяти ниже порогового значения модулям ядра и механизму выделения блоков памяти в ядре может быть дана команда немедленно освободить любую память, которую легко освободить.
- **OOM Killer**: этот механизм освобождает память, отыскивая и останавливая непривилегированный процесс. Поиск и остановка выполняются вызовом функций `select_bad_process()` и `oom_kill_process()` соответственно. Этот факт может быть зафиксирован в системном журнале (`/var/log/messages`) с сообщением «Out of memory: Kill process» (Недостаточно памяти: процесс остановлен).

Параметр `swappiness` в Linux определяет, следует ли освобождать память путем вытеснения страниц на диск или за счет уменьшения кэша страниц. Этот параметр является числом от 0 до 100 (значение по умолчанию 60), где более высокие значения означают предпочтительность освобождения памяти за счет вытеснения страниц на диск (подкачки). Правильный баланс между этими методами освобождения памяти позволяет повысить пропускную способность системы и сохранить кэш файловой системы теплым за счет вытеснения холодных страниц из памяти приложения [Corbet 04].

Интересно отметить, что произойдет, если в системе не настроено устройство или файл подкачки. Это ограничит размер виртуальной памяти, и ошибка распределения

памяти возникнет намного раньше. Кроме того, в Linux намного раньше будет приведен в действие механизм OOM Killer.

Рассмотрим приложение, постоянно увеличивающее потребление памяти. При наличии свопа сначала, скорее всего, возникнет проблема производительности из-за все возрастающего количества операций подкачки, что позволит оперативно отладить проблему. В отсутствие свопа нет периода подкачки, поэтому либо приложение выдаст ошибку «Out of memory» (недостаточно памяти), либо механизм OOM Killer завершит его. Это может усложнить отладку проблемы, особенно если она проявляется только через несколько часов работы.

В облаке Netflix экземпляры обычно не используют подкачку, поэтому приложения удаляются механизмом OOM Killer, если исчерпают доступную память. Приложения распределяются по большому пулу экземпляров, и остановка одного из них приведет к немедленному перенаправлению трафика на другие работоспособные экземпляры. Это считается более предпочтительным, чем медленная работа одного экземпляра из-за подкачки.

Когда используются контрольные группы (cgroups), для управления памятью в этих группах могут применяться те же методы освобождения памяти, что были показаны на рис. 7.6. Даже при наличии большого объема свободной памяти в системе может возникнуть проблема падения производительности из-за подкачки или остановки процессов механизмом OOM Killer, потому что контейнер исчерпал свой лимит, контролируемый группой cgroup [Evans 17]. Дополнительные сведения о контрольных группах и контейнерах см. в главе 11 «Облачные вычисления».

В следующих разделах мы поговорим про списки свободных страниц, уплотнение и демон подкачки страниц.

Списки свободных страниц

Первоначально механизм распределения памяти в Unix использовал карту памяти и искал свободные блоки по принципу «первый подходящий». С появлением в BSD страничной виртуальной памяти были добавлены *список свободных страниц* и *демон подкачки страниц* [Babaoglu 79]. Список свободных страниц, изображенный на рис. 7.7, позволяет быстро найти доступную память.

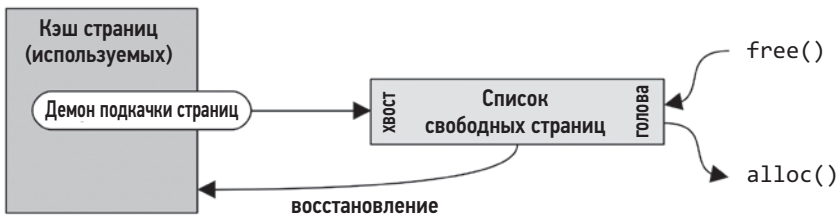


Рис. 7.7. Принцип действия списка свободных страниц

Освободившаяся память добавляется в голову списка для будущих распределений. Память, которая освобождается демоном подкачки страниц и может содержать полезные кэшированные страницы файловой системы, добавляется в хвост списка. Если в будущем запрос к одной из этих страниц произойдет до того, как она будет повторно использована для других нужд, демон сможет *восстановить* ее и удалить из списка свободных страниц.

Форма списка свободных страниц, как показано на рис. 7.6, все еще используется в системах на базе Linux. Списки свободных страниц обычно используются через распределители памяти, такие как распределитель блоков (slab-allocator) для выделения памяти в ядре и функцию malloc() в библиотеке libc для выделения памяти в пространстве пользователя (который имеет свои списки свободных страниц). В свою очередь, они обеспечивают доступ к страницам через свой API распределения памяти.

В Linux для управления страницами используется распределитель, основанный на «системе двойников» (buddy allocator). Он поддерживает несколько списков свободных страниц для распределения блоков памяти разных размеров, кратных степени двойки. Термин *двойник* (buddy) относится к поиску соседних страниц свободной памяти, которые можно выделить вместе. Историческую справку см. в [Peterson 77].

Списки свободных «двойников» находятся внизу следующей иерархии, начиная с узла памяти pg_data_t:

- **Узлы:** банки памяти, с поддержкой NUMA.
- **Зоны:** диапазоны памяти для определенных целей (прямой доступ к памяти [DMA]¹, обычная память, физическая память за пределами виртуальной памяти ядра).
- **Типы миграции:** непереключаемый, восстанавливаемый, перемещаемый и т. д.
- **Размеры:** количество страниц, равное степени двойки.

Выделение памяти из списков свободных страниц в пределах узла улучшает локальность памяти и производительность. Для наиболее распространенных случаев, когда распределяются отдельные страницы, buddy allocator поддерживает списки отдельных страниц для каждого процессора, чтобы уменьшить конкуренцию между процессорами.

Уплотнение

Уплотнение (reaping) заключается в освобождении памяти в основном из кэшей распределителя блоков ядра. Эти кэши содержат неиспользуемую память в виде фрагментов размером с блок, готовых к повторному использованию. Уплотнение возвращает эту память системе для распределения в виде страниц.

¹ Зона ZONE_DMA может быть удалена [Корбет 18a].

В Linux модули ядра могут вызывать `register_shrinker()` для регистрации функций, выполняющих уплотнение их собственной памяти.

Сканирование страниц

Освобождением памяти путем ее вытеснения управляет демон подкачки. Когда объем доступной основной памяти в списке свободных страниц опускается ниже порогового значения, демон подкачки начинает сканировать страницы. Сканирование происходит, только когда это необходимо. Хорошо сбалансированная система может сканировать страницы редко и только в короткие периоды.

В Linux демон подкачки называется `kswapd`. Он сканирует LRU-списки страниц неактивной и активной памяти и пытается найти страницы, которые можно освободить. Активность демона управляется объемом свободной памяти и двумя пороговыми значениями, обеспечивающими гистерезисное поведение, как показано на рис. 7.8.

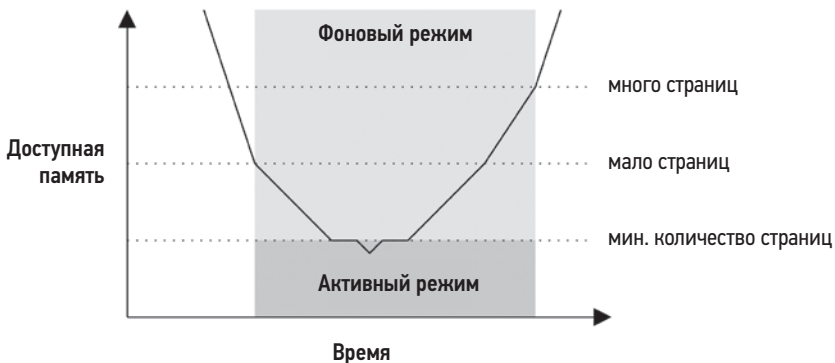


Рис. 7.8. Режимы работы `kswapd`

Когда объем свободной памяти достигает минимального порога, `kswapd` запускается в активном режиме, синхронно освобождая страницы памяти по мере появления запросов на распределение. Иногда этот режим называют режимом *немедленного освобождения* [Gorman 04]. Этот самый низкий порог настраивается через параметр `vm.min_free_kbytes`, а остальные автоматически вычисляются на его основе («мало свободных страниц» — в 2 раза и «много свободных страниц» — в 3 раза больше минимально допустимого количества страниц). Для обслуживания рабочих нагрузок, активно распределяющих память и опережающих ее освобождение демоном `kswapd`, в Linux предоставляются дополнительные настройки, обеспечивающие более активное сканирование: `vm.watermark_scale_factor` и `vm.watermark_boost_factor` (см. раздел 7.6.1 «Настраиваемые параметры»).

В кэше поддерживаются отдельные списки для *активных* и *неактивных страниц*. Они обслуживаются алгоритмом LRU (least recently used — наиболее давно

использовавшиеся), что позволяет демону kswapd быстро находить свободные страницы. Эти списки показаны на рис. 7.9.

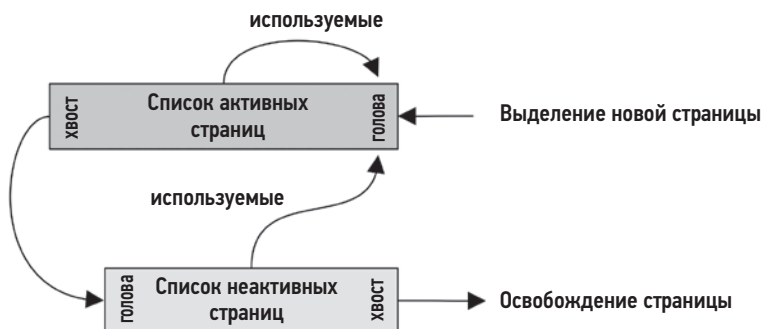


Рис. 7.9. Списки, поддерживаемые демоном kswapd

kswapd сначала сканирует список неактивных страниц и при необходимости — список активных страниц. Под термином *сканирование* подразумевается последовательная проверка страниц в списке: страница может быть недоступна для освобождения, если она заблокирована или «грязная». В терминологии kswapd *сканирование* имеет другое значение, чем в оригинальном демоне подкачки страниц UNIX, который сканирует всю память.

7.3.3. Адресное пространство процесса

Виртуальное адресное пространство процесса, управляемое как аппаратным, так и программным обеспечением, — это диапазон виртуальных страниц, которые при необходимости отображаются в физические страницы. Адреса разделены на области, называемые *сегментами*, для хранения стеков потоков, выполняемого кода процессов, библиотек и динамической памяти (кучи). На рис. 7.10 показаны примеры 32-битных процессов в Linux для процессоров x86 и SPARC.

В архитектуре SPARC ядро находится в отдельном полноценном адресном пространстве (которое не показано на рис. 7.10). Обратите внимание, что в SPARC по одному только значению указателя невозможно отличить адресные пространства пользователя и ядра. В x86 используется другая схема, в которой адресные пространства пользователя и ядра не перекрываются¹.

¹ Обратите внимание, что для 64-битных адресов полный 64-битный диапазон может не поддерживаться процессором: спецификация AMD позволяет реализациям поддерживать только 48-битные адреса, где неиспользуемые старшие биты устанавливаются равными последнему биту: это создает два диапазона адресов, называемых *каноническими адресами*, от 0 до 0x00007fffffffff — для пространства пользователя и от 0xffff800000000000 до 0xffffffffffffffff — для пространства ядра. Вот почему адреса в ядре для архитектуры x86 начинаются с 0xffff.

Сегмент выполняемой программы содержит отдельные сегменты с инструкциями и данными. Библиотеки также состоят из отдельных сегментов с инструкциями и данными. К этим различным типам сегментов относятся:

- **Выполняемые инструкции:** содержит инструкции процесса, выполняемые процессором. Отображается из сегмента инструкций двоичной программы в файловой системе. Доступен только для чтения с разрешением на выполнение.
- **Выполняемые данные:** содержит инициализированные переменные. Отображается из сегмента данных двоичной программы. Имеет разрешение на чтение/запись, поэтому переменные могут изменяться во время работы программы. Также этот сегмент отмечен флагом, препятствующим сохранению изменений на диск.
- **Динамическая память (куча):** рабочая и анонимная память программы (не имеющая отображения в файловой системе). Этот сегмент увеличивается по мере необходимости, и память для него выделяется через `malloc(3)`.
- **Стек:** содержит стеки запущенных потоков, доступные для чтения/записи.

Сегменты с выполняемыми инструкциями библиотек могут совместно использоваться несколькими процессами, применяющими ту же библиотеку, при этом каждый будет иметь свою копию сегмента данных библиотеки.

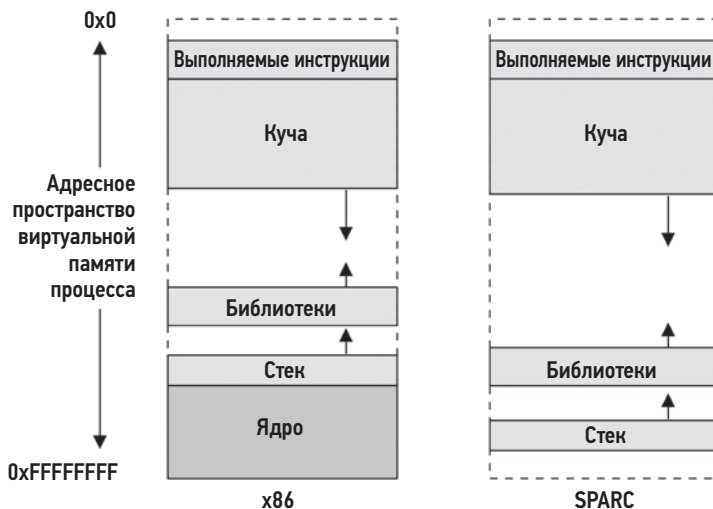


Рис. 7.10. Пример адресного пространства виртуальной памяти процесса

Рост кучи

Распространенный источник путаницы — постоянный рост кучи. Что это? Утечка памяти? В простых распределителях функция `free(3)` не возвращает память ОС,

а сохраняет ее для распределения в будущем. Это означает, что резидентная память процесса может только увеличиваться, и это нормально. Для сокращения потребления системной памяти процессы используют следующие методы:

- **Повторный запуск:** вызовом `execve(2)`, чтобы начать выполнение с пустым адресным пространством.
- **Отображение в память:** с использованием `mmap(2)` и `munmap(2)`, которые возвращают память системе.

Файлы, отображаемые в память, описаны в главе 8 «Файловые системы», в разделе 8.3.10 «Файлы, отображаемые в память».

Библиотека `glibc`, широко используемая в Linux, предлагает улучшенный распределитель памяти, поддерживающий режим работы `mmap`, а также функцию `malloc_trim(3)`, возвращающую освобождаемую память системе. Функция `malloc_trim(3)` автоматически вызывается из `free(3)`, когда объем свободной памяти в верхней части кучи достигает определенного предела¹ и освобождает ее с помощью системного вызова `sbrk(2)`.

Распределители

На уровнях пользователя и ядра есть множество распределителей памяти. Роль распределителей, включая некоторые общие их типы, показана на рис. 7.11.

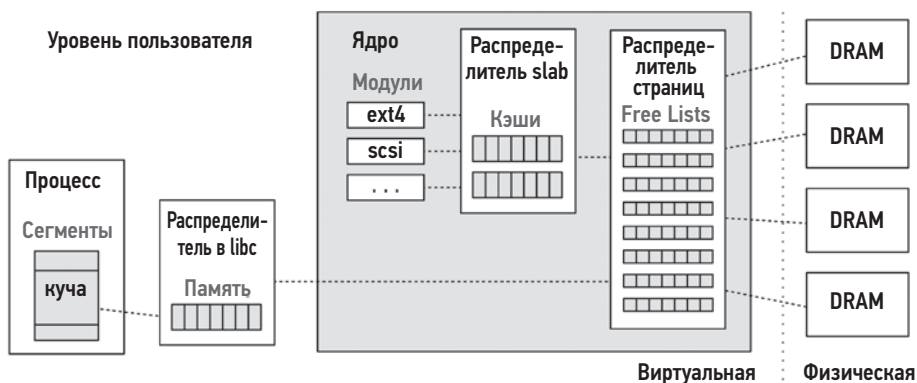


Рис. 7.11. Распределители памяти на уровнях пользователя и ядра

Управление страницами было описано выше в разделе 7.3.2 «Программное обеспечение», в подразделе «Списки свободных страниц».

¹ Больше, чем указано в параметре `M_TRIM_THRESHOLD` функции `mallot(3)`, который по умолчанию равен 128 Кбайт.

Распределитель памяти может поддерживать следующие свойства:

- **Предоставление простого API:** например, `malloc(3)`, `free(3)`.
- **Эффективное использование памяти:** при многократном распределении/освобождении блоков памяти различных размеров динамическая память может *фрагментироваться* и иметь множество неиспользуемых фрагментов, без пользы занимающих память. Распределители могут объединять неиспользуемые фрагменты, чтобы потом их можно было выделить более крупным блоком и тем самым повысить эффективность использования памяти.
- **Производительность:** запросы на распределение памяти могут следовать очень часто, что может приводить к падению скорости распределения в многопоточных средах из-за конкуренции за примитивы синхронизации. С этой точки зрения разработчики распределителей могут предусмотреть экономное использование блокировок, а также использовать кэши для каждого потока или процессора для улучшения локальности памяти.
- **Наблюдаемость:** распределитель может предоставлять статистики о своей работе и поддерживать режимы отладки, чтобы показать, как он используется и какие пути в коде приводят к распределению памяти.

В следующих разделах я опишу распределители уровня ядра: slab и SLUB — и распределители уровня пользователя: glibc, TCMalloc и jemalloc.

Slab

Распределитель slab управляет кэшами объектов определенного размера, позволяя быстро выделять память без дополнительных затрат на выделение страниц. Это качество особенно востребовано в ядре ОС, которое часто выделяет память для хранения структур фиксированного размера.

Ниже приводятся две строки кода, взятые из реализации ZFS в `arc.c`¹:

```
df = kmem_alloc(sizeof (l2arc_data_free_t), KM_SLEEP);
head = kmem_cache_alloc(hdr_cache, KM_PUSHPAGE);
```

Первая строка, вызывающая `kmem_alloc()`, показывает традиционный прием распределения памяти в ядре, когда размер требуемого блока передается в аргументе. Исходя из этого размера, ядро отображает блок в соответствующий кэш slab (выделение блоков очень большого размера производится по-другому, в *области для блоков большого размера* — oversized arena). Вызов `kmem_cache_alloc()` во второй строке взаимодействует непосредственно с нестандартным кэшем блоков, в данном случае `(kmem_cache_t *)hdr_cache`.

Разработанный для Solaris 2.4 [Bonwick 94], распределитель slab позже был дополнен поддержкой кэшей для каждого процессора, которые называются *магазинами* [Bonwick 01]:

¹ Единственная причина, почему я привожу именно эти строки, — я сам написал их.

Базовая идея состоит в том, чтобы для каждого процессора организовать свой кэш объектов из M элементов, который называется магазином, по аналогии с магазинами автоматического оружия. Магазин при каждом процессоре сможет удовлетворить M запросов на распределение памяти, прежде чем процессору потребуется перезарядка, то есть замена пустого магазина полным.

Помимо высокой производительности, оригинальный распределитель slab имеет средства отладки и анализа, включая аудит, для трассировки особенностей распределения памяти и стека.

Механизм распределения блоков был заимствован различными ОС. В BSD есть распределитель slab, который называется универсальным распределителем памяти (universal memory allocator, UMA), обладает высокой эффективностью и поддерживает NUMA. Распределитель slab был также включен в ядро Linux, начиная с версии 2.2, и в течение многих лет использовался по умолчанию. Но позднее Linux перешел на использование по умолчанию другого распределителя — SLUB.

SLUB

Распределитель SLUB в ядре Linux основан на распределителе slab и предназначен для решения проблем, главным образом связанных со сложностью распределителя slab. В числе усовершенствований можно назвать отказ от очередей объектов и кэшей для процессоров, когда оптимизация NUMA перекладывается на распределитель страниц (см. подраздел «Списки свободных страниц» выше).

Распределитель SLUB стал использоваться по умолчанию, начиная с версии Linux 2.6.23 [Lameter 07].

glibc

Распределитель GNU libc, действующий на уровне пользователя, основан на распределителе dlmalloc, созданном Дугом Ли (Doug Lea). Поведение распределителя зависит от размера запрошенного блока. Запросы на выделение блоков небольшого размера удовлетворяются из сегментов памяти с блоками одинакового размера, которые могут объединяться с использованием алгоритма «двойников» (buddy). Для более эффективного поиска при выделении крупных блоков можно использовать поиск по дереву. При выделении очень больших блоков происходит переключение на использование mmap(2). В результате получился высокопроизводительный распределитель, выигрывающий от использования разных стратегий распределения.

TCMalloc

TCMalloc — это механизм выделения памяти для потоков пользовательского уровня, использующий отдельные кэши для каждого потока, из которых удовлетворяются запросы на выделение небольших блоков памяти. Это помогает уменьшить конкуренцию за обладание блокировками и увеличить производительность [Ghemawat 07]. При периодической сборке мусора память перемещается обратно в центральную кучу для распределения в будущем.

jemalloc

Созданный как распределитель `libc` пользовательского уровня для FreeBSD, `libjemalloc` также доступен в Linux. Он использует такие методики, как деление памяти на несколько областей, кэширование по потокам и выделение памяти небольшими блоками для объектов, чтобы улучшить масштабируемость и уменьшить фрагментацию памяти. Для получения памяти от системы может использовать как `mmap(2)`, так и `sbrk(2)`, но предпочтение отдается `mmap(2)`. Компания Facebook использует `jemalloc` в своем ПО и добавила в него поддержку профилирования и другие оптимизации [Facebook 11].

7.4. МЕТОДОЛОГИЯ

В этом разделе описаны различные методологии и упражнения для анализа и настройки памяти. Краткий перечень методологий приводится в табл. 7.3.

Таблица 7.3. Методологии анализа и настройки производительности памяти

Раздел	Методология	Тип
7.4.1	Метод инструментов	Анализ наблюдения
7.4.2	Метод USE	Анализ наблюдения
7.4.3	Определение характеристик потребления памяти	Анализ наблюдения, планирование мощности
7.4.4	Анализ тактов	Анализ наблюдения
7.4.5	Мониторинг производительности	Анализ наблюдения, планирование мощности
7.4.6	Выявление утечек	Анализ наблюдения
7.4.7	Статическая настройка производительности	Анализ наблюдения, планирование мощности
7.4.8	Управление ресурсами	Настройка
7.4.9	Микробенчмаркинг	Анализ результатов экспериментов
7.4.10	Уменьшение потребления памяти	Анализ результатов экспериментов

Список дополнительных методологий и краткое введение ищите в главе 2 «Методологии». Эти методологии можно применять по отдельности или в сочетании друг с другом. При устранении проблем с памятью предлагаю начать с использования следующих стратегий в таком порядке: мониторинг производительности, метод USE и определение характеристик потребления памяти.

В разделе 7.5 «Инструменты наблюдения» показано применение этих методологий с использованием инструментов ОС.

7.4.1. Метод инструментов

Метод инструментов — это процесс перебора доступных инструментов с целью изучения ключевых метрик, которые они возвращают. Это очень простая методология, но при ее использовании могут оставаться незамеченными некоторые проблемы, которые плохо обнаруживаются или вообще не обнаруживаются инструментами. Также на ее применение может уйти много времени.

При анализе производительности памяти метод инструментов можно использовать для проверки следующих метрик (в Linux):

- **Сканирование страниц:** обращайте внимание на продолжительные периоды сканирования страниц (более 10 с), которые могут служить признаком нехватки памяти. Это можно сделать с помощью команды `sar -B` и проверки столбцов `pgscan` в ее выводе.
- **Информация о задержках доступа** (pressure stall information, PSI): с помощью команды `cat /proc/pressure/memory` (Linux 4.20+) можно проверить статистики, характеризующие насыщение памяти, и их изменение с течением времени.
- **Подкачка** (swapping): если подкачка включена, то подкачка страниц памяти (так в Linux называется подкачка) может служить еще одним признаком нехватки памяти. Для анализа можно использовать столбцы `si` и `so` в выводе `vmstat(8)`.
- **vmstat:** запустите `vmstat 1` и проверьте столбец `free`, показывающий объем доступной памяти.
- **События OOM Killer:** эти события можно увидеть в системном журнале `/var/log/messages` или в `dmesg(1)`. Ищите сообщения «Out of memory» (недостаточно памяти).
- **top:** посмотрите, какие процессы и пользователи являются основными потребителями физической (резидентной) и виртуальной памяти (описание столбцов в выводе команды `top` смотрите на странице справочного руководства `man`, они могут различаться в зависимости от версии). `top(1)` также сообщает об объеме свободной памяти.
- **perf(1)/BCC/bpfttrace:** проанализируйте операции распределения памяти по трассировкам стека, чтобы выявить конкретные причины повышенного потребления памяти. Имейте в виду, что трассировка может быть сопряжена со значительным оверхедом. Более дешевое, хотя и грубое решение — профилирование процессора (выборка трассировок стека по времени) и поиск в коде путей, ведущих к распределению памяти.

Дополнительные сведения о каждом инструменте вы найдете в разделе 7.5 «Инструменты наблюдения».

7.4.2. Метод USE

Метод USE можно использовать на ранних этапах исследования производительности для выявления узких мест и ошибок во всех компонентах, прежде чем переходить к более глубоким и трудоемким стратегиям.

Для системы в целом проверьте:

- **Потребление:** сколько памяти используется и сколько доступно. Проверять нужно и физическую, и виртуальную память.
- **Насыщенность:** частоту сканирования страниц, подкачки страниц и событий OOM Killer как основных признаков мер, предпринимаемых системой для уменьшения давления на память.
- **Ошибки:** программные или аппаратные ошибки.

Сначала можно проверить насыщенность, так как постоянное насыщение является ярким признаком проблем с памятью. Соответствующие метрики обычно легко получить с помощью инструментов ОС. Например, статистики, характеризующие частоту подкачки страниц, можно получить с помощью `vmstat(8)` и `sar(1)`, а сообщения OOM Killer — с помощью `dmesg(1)`. Для систем с настроенным отдельным устройством подкачки любые операции с этим устройством — это один из признаков нехватки памяти. Статистика насыщения памяти в Linux также предоставляется как часть информации о задержках доступа (pressure stall information, PSI).

Уровень потребления физической памяти сообщают разные инструменты, но они могут учитывать или не учитывать страницы кэша файловой системы, на которые нет ссылок, или неактивные страницы. Система может сообщить, что в ее распоряжении всего 10 Мбайт доступной памяти, тогда как на самом деле 10 Гбайт занято кэшем файловой системы, и эта память при необходимости может быть немедленно освобождена приложениями. Загляните в документацию с описанием инструмента, чтобы узнать, какие сведения он предоставляет.

Желательно проверить и потребление виртуальной памяти, если система поддерживает стратегию `overcommit`. В системах, не поддерживающих эту стратегию, попытки распределения памяти потерпят неудачу сразу же, как только виртуальная память будет исчерпана, с сообщениями об ошибке.

Ошибки распределения памяти могут быть вызваны ПО, например попыткой выделить слишком большой объем памяти, или действиями механизма OOM Killer в Linux, или аппаратным обеспечением, например ошибками ECC. Традиционно обязанность сообщать об ошибках распределения памяти возлагается на приложения, но не все приложения делают это (а с учетом поддержки стратегии выделения памяти без ограничений в Linux многие разработчики не считают это нужным). Аппаратные ошибки тоже сложно диагностировать. Некоторые инструменты могут сообщать об ошибках, исправляемых с помощью ECC (например, `dmidecode(8)`, `edac-utils`, `ipmi-tool sel` в Linux), когда используется память ECC. Эти исправимые ошибки могут использоваться как метрики ошибок в методе USE и служить признаком того, что вскоре могут появиться неисправимые ошибки. При фактических (неисправимых) ошибках памяти можно столкнуться с необъяснимыми и невозпроизводимыми сбоями (включая ошибки сегментации и сигналы ошибок шины) в любых приложениях.

В средах, поддерживающих ограничения или квоты памяти (средствами управления ресурсами), как в некоторых облачных средах, для оценки потребления и насыщения

памяти могут потребоваться другие подходы к измерениям. Для экземпляра ОС может быть установлен программный предел объема доступной памяти и пространства подкачки, даже если на хосте достаточно физической памяти. См. главу 11 «Облачные вычисления».

7.4.3. Определение характеристик потребления памяти

Определение характеристик потребления памяти — важный шаг при планировании мощностей, сравнительном анализе и моделировании рабочих нагрузок. Это также поможет значительно увеличить производительность за счет выявления и исправления неправильных настроек. Например, в базе данных может быть настроен либо слишком маленький кэш, который из-за этого имеет низкую частоту попаданий, либо слишком большой, что вызывает подкачку.

Определение характеристик потребления памяти включает выяснение того, где и сколько памяти используется:

- Потребление физической и виртуальной памяти в масштабе всей системы.
- Степень насыщения: наличие событий подкачки и сообщений OOM Killer.
- Потребление памяти для кэшей ядра и файловой системы.
- Потребление физической и виртуальной памяти каждым процессом.
- Использование средств управления ресурсами памяти, если есть.

Пример ниже показывает, как эти атрибуты можно выразить в одном описании:

Система имеет 256 Гбайт основной памяти, из которых 1 % используется процессами и 30 % занято кэшем файловой системы. Самый главный потребитель памяти — процесс базы данных, занимающий 2 Гбайт основной памяти (Resident Set Size, RSS), что является настроенным для него пределом, унаследованным из предыдущей системы при переносе.

Эти характеристики могут меняться со временем, потому что для кэширования рабочих данных может потребоваться больше памяти. Кроме обычного увеличения размера кэша, память, занимаемая ядром или приложением, тоже может увеличиваться с течением времени из-за утечек памяти, обусловленных ошибками в ПО.

Расширенный анализ потребления/чек-лист

В списке ниже перечислены дополнительные характеристики для рассмотрения. Он также может служить чек-листом для более тщательного изучения проблем с памятью:

- Каков размер рабочего набора (working set size, WSS) для приложений?
- На какие нужды ядро использует память? Сколько памяти выделяется ядром на каждый блок (slab)?

- Какая доля кэша файловой системы активна/неактивна?
- Для каких целей используется память процесса (инструкции, кэши, буферы, объекты и т. д.)?
- Почему процессы выделяют дополнительную память (пути в коде)?
- Почему ядро выделяет дополнительную память (пути в коде)?
- Наблюдаются ли странности с отображением библиотек в память процессов (например, изменения с течением времени)?
- Какие процессы активно используют подкачку?
- Какие процессы ранее использовали подкачку?
- Могут ли процессы или ядро иметь утечки памяти?
- Насколько хорошо память распределена между узлами в системе NUMA?
- Как часто следуют холостые такты в ожидании доступа к памяти и сколько инструкций выполняется за такт (IPC)?
- Насколько сбалансированы шины памяти?
- Каково соотношение операций чтения/записи с локальной и удаленной памятью?

Ответы на некоторые из этих вопросов можно найти в следующих разделах. Более подробное описание этой методологии и метрик, которые нужно измерить (кто, что, почему и как), вы найдете в главе 2 «Методологии».

7.4.4. Анализ тактов

Нагрузку на шину памяти легко определить с помощью счетчиков мониторинга производительности процессора (performance monitoring counters, PMC), которые можно запрограммировать для подсчета холостых тактов в ожидании доступа к памяти, использования шины памяти и т. д. Для начала можно определить количество инструкций на такт (IPC), отражающее, насколько нагрузка на процессор зависит от памяти. См. главу 6 «Процессоры».

7.4.5. Мониторинг производительности

Мониторинг производительности может помочь обнаружить проблемы и модели поведения, проявляющиеся со временем. Ключевые метрики для памяти:

- **Потребление:** процент использованной памяти, который можно определить по объему доступной памяти.
- **Насыщенность:** наличие событий подкачки и сообщений OOM Killer в журнале.

В средах, реализующих ограничения или квоты памяти (средствами управления ресурсами), может также потребоваться собрать статистики, характеризующие наложенные ограничения.

Также можно выполнить мониторинг ошибок (если есть такая возможность), как описано в разделе 7.4.2 «Метод USE».

Мониторинг потребления памяти с течением времени, особенно по процессам, может помочь выявить наличие и величину утечек памяти.

7.4.6. Выявление утечек

Эта проблема возникает, когда объем памяти, потребляемой приложением или модулем ядра из списка свободных страниц, из кэша файловой системы и в итоге из других процессов, постоянно увеличивается. Это можно заметить по появлению событий подкачки страниц или по сообщениям OOM Killer о завершении приложения в ответ на чрезмерное потребление памяти.

Проблемы этого типа обычно обусловлены одной из причин:

- **Утечка памяти:** из-за программной ошибки, вследствие которой выделенная и больше не используемая память не освобождается. Такие проблемы исправляются изменением программного кода или применением исправлений или обновлений (которые изменяют код).
- **Рост потребляемой памяти:** программа потребляет ровно столько памяти, сколько ей необходимо, но с гораздо большей скоростью, чем желательно для системы. Такие проблемы исправляются настройкой конфигурации программного обеспечения или изменением способа потребления памяти приложением в программном коде.

Проблему роста потребляемой памяти часто ошибочно принимают за утечку памяти. Чтобы избежать этого, в первую очередь нужно ответить на вопрос: должно ли так быть? Проверьте потребление памяти, конфигурацию приложения и поведение распределителей памяти в нем. Приложение может быть настроено на быстрое заполнение кэша, и наблюдаемый рост потребления может быть обусловлен разогревом кэша.

Анализ утечек памяти зависит от ПО и типа языка, на котором оно написано. Некоторые распределители поддерживают режимы отладки и предоставляют подробную информацию об операциях распределения памяти, которую затем можно проанализировать, чтобы выявить пути в коде, порождающие утечки. В некоторых средах времени выполнения имеются методы для анализа дампа кучи и другие инструменты, помогающие исследовать утечки памяти.

В наборе инструментов трассировки для ВСС в Linux есть инструмент memleak(8) для анализа роста потребления и утечек памяти: он наблюдает за выделяемыми блоками памяти и отмечает те из них, которые не были освобождены в течение определенного интервала, а также пути в коде, которые выделили эти блоки. Он не различает утечки и нормальный рост потребления, поэтому проанализируйте пути в коде, чтобы определить конкретный тип проблемы. (Обратите внимание, что у этого инструмента высокий оверхед при высокой частоте следования операций

распределения памяти.) ВСС рассматривается в главе 15 «BPF», в разделе 15.1 «ВСС».

7.4.7. Статическая настройка производительности

Статическая настройка производительности фокусируется на проблемах сконфигурированной среды. Анализируя производительность памяти, рассмотрите следующие аспекты статической конфигурации:

- Сколько всего основной памяти?
- Какой объем памяти выделен для использования приложениями (в их собственных конфигурациях)?
- Какие распределители памяти используются приложениями?
- Какова скорость работы основной памяти? Это самый быстрый доступный тип (DDR5)?
- Тестировалась ли когда-нибудь основная память (например, с помощью memtester в Linux)?
- Какова архитектура системы? NUMA, UMA?
- Поддерживает ли ОС архитектуру NUMA? Предоставляет ли она настройки NUMA?
- Подключена ли память к одному и тому же физическому процессору или разделена между несколькими физическими процессорами?
- Сколько шин памяти?
- Сколько кэшей и какого размера имеют процессоры? TLB?
- Какие настройки сделаны в BIOS?
- Настроены ли и используются ли большие страницы?
- Доступна ли и настроена ли стратегия overcommit?
- Какие еще настройки системной памяти используются?
- Есть ли программные ограничения на потребление памяти (средствами управления ресурсами)?

Ответы на эти вопросы могут помочь выявить варианты конфигурации, которые были упущены из виду.

7.4.8. Управление ресурсами

Операционная система может предоставлять низкоуровневые средства управления, регулирующие выделение памяти процессам или группам процессов и позволяющие устанавливать фиксированные ограничения на потребление основной и виртуальной памяти. Как это работает, зависит от реализации и обсуждается в разделе 7.6 «Настройка» и в главе 11 «Облачные вычисления».

7.4.9. Микробенчмаркинг

Микробенчмаркинг можно использовать для определения быстродействия основной памяти и таких характеристик, как объем кэша процессора и размеров строк кэша. Они пригодятся при анализе различий между системами, потому что скорость доступа к памяти может влиять на производительность больше, чем тактовая частота процессора, в зависимости от особенностей приложения и рабочей нагрузки.

В главе 6 «Процессоры» в подразделе «Задержка» (раздел 6.4.1 «Аппаратное обеспечение») показан результат микробенчмаркинга задержки доступа к памяти для определения характеристик кэшей процессора.

7.4.10. Уменьшение потребления памяти

Это метод оценки размера рабочего набора (*working set size*, WSS), основанный на проведении *отрицательного* эксперимента и требующий настройки устройств подкачки для этого эксперимента. В ходе эксперимента объем основной памяти, доступной приложению, постепенно уменьшается и одновременно измеряются его производительность и объем подкачки: точка, в которой производительность начинает резко падать, а подкачка увеличиваться, показывает, когда WSS перестает уместиться в доступной памяти.

Отрицательный эксперимент — это пример, достойный упоминания, но не рекомендую проводить его в промышленном окружении, так как при этом намеренно снижается производительность. Информацию о других методах оценки WSS ищите в описании экспериментального инструмента `wss(8)` в разделе 7.5.12 «`wss`», а также в статье, посвященной оценке WSS, на моем сайте [Gregg 18c].

7.5. ИНСТРУМЕНТЫ НАБЛЮДЕНИЯ

В этом разделе представлены инструменты наблюдения за распределением памяти, доступные в ОС на базе Linux. Стратегии их использования описываются в предыдущем разделе.

Инструменты перечислены в табл. 7.4.

Таблица 7.4. Инструменты наблюдения за распределением памяти в Linux

Раздел	Инструмент	Описание
7.5.1	<code>vmstat</code>	Статистики виртуальной и физической памяти
7.5.2	<code>PSI</code>	Информация о задержках доступа к памяти
7.5.3	<code>swapon</code>	Использование устройства подкачки
7.5.4	<code>sar</code>	Исторические статистики

Раздел	Инструмент	Описание
7.5.5	slabtop	Статистики распределителя блоков в ядре
7.5.6	numastat	Статистики NUMA
7.5.7	ps	Состояние процессов
7.5.8	top	Мониторинг потребления памяти по процессам/потокам
7.5.9	pmap	Статистики адресного пространства процесса
7.5.10	perf	Анализ счетчиков РМС и точек трассировки, связанных с распределением памяти
7.5.11	drsnnoop	Трассировка непосредственной утилизации памяти
7.5.12	wss	Оценка размера рабочего набора
7.5.13	bpfttrace	Трассировка программ в ходе анализа потребления памяти

Описание этого набора инструментов и возможностей дополняет раздел 7.4 «Методология». Оно начинается с традиционных инструментов получения статистик потребления памяти в масштабе всей системы, а затем переходит к инструментам для более глубокого анализа отдельных процессов и трассировки операций распределения памяти. Некоторые из традиционных инструментов доступны (а иногда первоначально созданы) в других Unix-подобных ОС, в том числе `vmstat(8)`, `sar(1)`, `ps(1)`, `top(1)` и `pmap(1)`. `drsnnoop(8)` — это инструмент BPF из ВСС (глава 15).

Полный перечень возможностей каждого инструмента ищите в соответствующей документации, включая страницы справочного руководства `man`.

7.5.1. vmstat

Команда вывода статистик использования виртуальной памяти `vmstat(8)` позволяет получить общее представление о состоянии системной памяти, включая текущий объем свободной памяти и статистики подкачки. Также она выводит статистики потребления процессора, как описано в главе 6 «Процессоры».

Когда этот инструмент был разработан Биллом Джоем и Озалпом Бабаоглу в 1979 году для BSD, на его странице в справочном руководстве было написано:

ОШИБКИ. Выводит так много информации, что иногда сложно понять, на что смотреть.

Вот пример вывода версии для Linux:

```
$ vmstat 1
procs -----memory----- --swap--  -----io----- -system--  ----cpu-----
 r  b   swpd   free   buff  cache   si   so    bi   bo    in   cs us sy id wa
 4  0       0 34454064 111516 13438596  0  0    0    5    2    0  0  0 100  0
 4  0       0 34455208 111516 13438596  0  0    0    0 2262 15303 16 12 73  0
```

```

5 0      0 34455588 111516 13438596  0  0  0  0 1961 15221 15 11 74  0
4 0      0 34456300 111516 13438596  0  0  0  0 2343 15294 15 11 73  0
[...]
```

Эта версия `vmstat(8)` не выводит обобщенные значения с момента загрузки в столбцах `procs` или `memory` в первой строке, а сразу показывает текущее состояние. По умолчанию величины в столбцах указываются в килобайтах:

- **swpd**: объем выгружаемой памяти;
- **free**: свободная доступная память;
- **buff**: память в кэше буферов;
- **cache**: память в кэше страниц;
- **si**: объем памяти, загруженной с устройства подкачки;
- **so**: объем памяти, выгруженной на устройство подкачки.

Кэши буферов и страниц описаны в главе 8 «Файловые системы». После загрузки системы объем свободной памяти постепенно уменьшается и занимается этими кэшами для повышения производительности — это нормально. При необходимости эта память может беспрепятственно использоваться приложениями.

Если в столбцах `si` и `so` постоянно выводятся значения, отличные от нуля, это означает, что система испытывает нехватку памяти и выполняет подкачку с устройства или файла подкачки (см. описание `swapon(8)`). Также для выявления основных потребителей памяти можно использовать другие инструменты, в том числе показывающие потребление памяти процессами (например, `top(1)`, `ps(1)`).

В системах с большим объемом памяти значения в столбцах могут не совпадать с заголовками, и их трудно читать. В таком случае можно попробовать изменить единицы вывода на мегабайты, добавив параметр `-S` (где `m` соответствует единицам, кратным 1000000, а `M` — кратным 1048576):

```

$ vmstat -Sm 1
procs -----memory----- ---swap-- ----io---- -system-- ----cpu-----
r b  swpd  free  buff  cache  si  so   bi   bo   in   cs us sy id wa
4 0      0 35280  114 13761  0  0   0    5    2    1  0  0 100  0
4 0      0 35281  114 13761  0  0   0    0 2027 15146 16 13 70  0
[...]
```

Есть параметр `-a` для вывода разбивки по признаку активной/неактивной памяти в кэше страниц:

```

$ vmstat -a 1
procs -----memory----- ---swap-- ----io---- -system-- ----cpu-----
r b  swpd  free  inact active si  so   bi   bo   in   cs us sy id wa
5 0      0 34453536 10358040 3201540 0  0   0    5    2    0  0  0 100  0
4 0      0 34453228 10358040 3200648 0  0   0    0 2464 15261 16 12 71  0
[...]
```

Эти статистики можно вывести в виде списка, добавив параметр `-s`.

7.5.2. PSI

Информация о задержках доступа в Linux (pressure stall information, PSI), добавленная в Linux 4.20, включает статистики, характеризующие насыщение памяти. Они показывают не только наличие задержек, но и то, как они менялись за последние 5 минут. Вот пример вывода:

```
# cat /proc/pressure/memory
some avg10=2.84 avg60=1.23 avg300=0.32 total=1468344
full avg10=1.85 avg60=0.66 avg300=0.16 total=702578
```

Мы видим, что нагрузка на память увеличивается — среднее значение за последние 10 с (2.84) выше среднего значения за последние 300 с (0.32). Средние значения представляют процент времени, в течение которого задача простаивала в ожидании доступа к памяти. Строка *some* (некоторые) обобщает случаи, когда задержкой были затронуты некоторые задачи (поток), а строка *full* (все) обобщает случаи, когда задержкой были затронуты все выполняемые задачи.

Статистики PSI также трассируются механизмом контрольных групп *cgroup2* (контрольные группы *cgroup* рассматриваются в главе 11 «Облачные вычисления») [Facebook 19].

7.5.3. swapon

swapon(1) позволяет узнать о наличии устройств подкачки в системе и какой их объем используется. Например:

```
$ swapon
NAME      TYPE      SIZE    USED    PRIO
/dev/dm-2 partition 980M  611.6M   -2
/swap1    file      30G    10.9M   -3
```

Согласно этому выводу, система имеет два устройства подкачки: раздел физического диска размером 980 Мбайт и файл */swap1* размером 30 Гбайт. Вывод также показывает, какой объем на обоих устройствах используется. Сейчас во многих системах подкачка вообще не настроена, в этом случае *swapon(1)* не выводит никаких результатов.

Если устройство подкачки активно используется, это можно заметить по значениям в столбцах *si* и *so* в выводе *vmstat(1)* и информации об объеме ввода/вывода с устройствами в данных *iostat(1)* (глава 9).

7.5.4. sar

Генератор отчетов о работе системы (system activity reporter) *sar(1)* можно использовать для наблюдения за текущей активностью и настроить для архивирования статистик и генерирования хронологических отчетов. Этот инструмент упоминается в нескольких главах этой книги (из-за разнообразия собираемых им статистик)

и достаточно полно был представлен в главе 4 «Инструменты наблюдения», в разделе 4.4 «sag».

Версия для Linux предлагает несколько статистик, характеризующих потребление памяти, доступных при использовании следующих параметров:

- **-B**: статистики подкачки страниц;
- **-H**: статистики использования огромных страниц;
- **-r**: потребление памяти;
- **-S**: статистики потребления пространства подкачки;
- **-W**: статистики подкачки.

Они включают метрики, характеризующие потребление памяти, активность демона подкачки страниц и использование больших страниц. См. раздел 7.3 «Архитектура», где описываются эти темы.

Некоторые из доступных статистик перечислены в табл. 7.5.

Таблица 7.5. Статистики из утилиты sag в Linux

Параметр	Статистика	Описание	Единицы измерения
-B	pgpgin/s	Объем загрузки страниц из устройства подкачки	Кбайт/с
-B	pgpgout/s	Объем выгрузки страниц в устройства подкачки	Кбайт/с
-B	fault/s	Количество мажорных и минорных сбоев страниц вместе в секунду	Число/с
-B	majflt/s	Количество мажорных сбоев страниц в секунду	Число/с
-B	pgfree/s	Количество страниц, добавляемых в список свободных страниц, в секунду	Число/с
-B	pgscank/s	Количество страниц, отсканированных демоном подкачки страниц (ksward) в фоновом режиме, в секунду	Число/с
-B	pgscand/s	Количество прямых сканирований страниц в секунду	Число/с
-B	pgsteal/s	Частота освобождения страниц в памяти и в устройстве подкачки	Число/с
-B	%meff	Отношение числа освобождаемых страниц к числу сканированных страниц, показывающее эффективность механизма освобождения страниц	%
-H	hbhugfree	Свободный объем памяти, выделенной для больших страниц	Кбайт
-H	hbhugused	Использованный объем памяти, выделенной для больших страниц	Кбайт
-H	%hugused	Потребление больших страниц	%
-r	kbmemfree	Объем свободной памяти (вообще не использованной)	Кбайт

Параметр	Статистика	Описание	Единицы измерения
-r	kbavail	Объем доступной памяти, включая память в кэше страниц, которая с легкостью может быть освобождена в любой момент	Кбайт
-r	kbmemused	Объем использованной памяти (исключая ядро)	Кбайт
-r	%memused	Потребление памяти	%
-r	kbbuffers	Размер кэша буферов	Кбайт
-r	kbcached	Размер кэша страниц	Кбайт
-r	kbcommit	Выделенный объем основной памяти: оценка объема памяти, необходимого для обслуживания текущей рабочей нагрузки	Кбайт
-r	%commit	Выделенный объем основной памяти для текущей рабочей нагрузки, оценка	%
-r	kbactive	Размер списка активных страниц памяти	Кбайт
-r	kbinact	Размер списка неактивных страниц памяти	Кбайт
-r	kbdirtw	Объем измененной памяти для записи на диск	Кбайт
-r ALL	kbanonpg	Объем анонимной памяти процесса	Кбайт
-r ALL	kbslab	Размер кэша блоков (slab) ядра	Кбайт
-r ALL	kbkstack	Размер стека ядра	Кбайт
-r ALL	kbpgtbl	Минимальный размер таблицы страниц	Кбайт
-r ALL	kbvmused	Использованный объем виртуальной памяти	Кбайт
-S	kbswpfree	Свободный объем в пространстве подкачки	Кбайт
-S	kbswpused	Использованный объем в пространстве подкачки	Кбайт
-S	%swpused	Доля использованного объема в пространстве подкачки	%
-W	pswpin/s	Объем загрузки страниц из устройства подкачки	Кбайт/с
-W	pswpout/s	Объем выгрузки страниц в устройства подкачки	Кбайт/с

Названия многих статистик включают единицы измерения: pg — страницы, kb — килобайты, % — проценты и /s — в секунду. Полный список статистик включает дополнительные метрики, измеряемые в процентах; вы найдете его на странице справочного руководства `man`.

Важно помнить, что при необходимости можно получить много обобщенной информации о потреблении памяти и о работе подсистем, управляющих ее распределением. Для более детального исследования используйте трассировщики `perf(1)` и `bpfttrace`. С их помощью можно инструментировать точки трассировки в функциях ядра, управляющих распределением памяти, как описывается в последующих разделах. Также можно исследовать исходный код в каталоге `mm`, в частности `mm/vmscan.c`.

В списке рассылки `linux-mm` вы найдете большое количество сообщений с дополнительной информацией, в которых разработчики обсуждают перечень желаемых статистик.

Метрика `%vmeff` — отличный показатель эффективности освобождения страниц. Высокое значение указывает, что страницы успешно удаляются из списка неактивных страниц, а низкое — что система испытывает нехватку памяти. На странице справочного руководства `man` говорится, что высокими считаются значения, близкие к 100 %, а низкими — ниже 30 %.

Еще одна полезная метрика — `pgscand`, показывающая как часто приложение блокируется в операциях выделения памяти и выполняется прямое освобождение страниц (чем выше это значение, тем хуже). Чтобы узнать время, потраченное приложениями в ожидании завершения прямого освобождения страниц, можно использовать инструменты трассировки: см. раздел 7.5.11 «`drsnop`».

7.5.5. slabtop

Команда `slabtop(1)` в Linux выводит информацию об использовании кэша блоков памяти ядра из распределителя `slab`. Так же, как и `top(1)`, она постоянно обновляет экран.

Вот пример вывода:

```
# slabtop -sc
Active / Total Objects (% used) : 686110 / 867574 (79.1%)
Active / Total Slabs (% used)   : 30948 / 30948 (100.0%)
Active / Total Caches (% used)  : 99 / 164 (60.4%)
Active / Total Size (% used)    : 157680.28K / 200462.06K (78.7%)
Minimum / Average / Maximum Object : 0.01K / 0.23K / 12.00K

  OBJS ACTIVE  USE OBJ SIZE  SLABS OBJ/SLAB  CACHE SIZE NAME
45450  33712  74%   1.05K  3030      15   48480K ext4_inode_cache
161091  81681  50%   0.19K  7671      21   30684K dentry
222963 196779  88%   0.10K  5717      39   22868K buffer_head
35763  35471  99%   0.58K  2751      13   22008K inode_cache
26033  13859  53%   0.57K  1860      14   14880K radix_tree_node
93330  80502  86%   0.13K  3111      30   12444K kernfs_node_cache
  2104   2081  98%   4.00K   263       8    8416K kmalloc-4k
   528   431  81%   7.50K   132       4    4224K task_struct
[...]
```

В верхней части вывода содержится сводная информация, за которой следует список блоков, включая количество объектов (`OBJJS`), в том числе активных (`ACTIVE`), процент использованного пространства (`USE`), размер объектов (`OBJ SIZE`, в байтах) и общий размер кэша (`CACHE SIZE`, в байтах). В этом примере использовался параметр `-sc` для сортировки по размеру кэша в порядке убывания: наибольший размер здесь у кэша `ext4_inode_cache`.

Статистики распределения блоков в памяти ядра находятся в файле `/proc/slabinfo`. Их также можно получить командой `vmstat -m`.

7.5.6. numastat

Инструмент `numastat(8)`¹ выводит статистики для систем с неоднородным доступом к памяти (NUMA), которые обычно имеют несколько физических процессоров. Вот пример вывода, полученный в двухпроцессорной системе:

```
# numastat
                node0          node1
numa_hit        210057224016    151287435161
numa_miss        9377491084     291611562
numa_foreign     291611562      9377491084
interleave_hit   36476          36665
local_node       210056887752    151286964112
other_node       9377827348      292082611
```

В этой системе есть два узла NUMA, по одному для каждого банка памяти, подключенного к каждому процессору. Ядро Linux стремится выделять память на ближайшем узле NUMA, и `numastat(8)` показывает, насколько ему это удается. Вот основные статистики:

- **numa_hit**: количество распределений памяти на предпочтительном узле NUMA.
- **numa_miss + numa_foreign**: количество распределений памяти не на предпочтительном узле NUMA. (`numa_miss` сообщает количество выделений локальной памяти, которая должна была бы быть выделена на другом узле, а `numa_foreign` показывает количество выделений удаленной памяти, которая должна была бы быть выделена локально.)
- **other_node**: количество распределений памяти на этом узле, когда процесс выполнялся в другом месте.

Как показывают выходные данные, политика выделения NUMA в этой системе работает довольно хорошо: количество попаданий намного больше по сравнению с другими статистиками. Если коэффициент попаданий слишком низкий, можно исследовать возможность настройки параметров NUMA в `sysctl(8)` или использовать другие подходы для улучшения локальности памяти (например, разделение рабочих нагрузок или систем или выбор другой системы, с меньшим количеством узлов NUMA). Если нет возможности повысить эффективность распределения памяти NUMA, то `numastat(8)` хотя бы поможет объяснить плохую производительность операций чтения/записи с памятью.

`numastat(8)` поддерживает параметр `-n` для вывода статистик в мегабайтах и `-m` — для вывода в стиле `/proc/meminfo`. В зависимости от дистрибутива Linux `numastat(8)` может быть доступен в пакете `numactl`.

¹ Немного истории: примерно в 2003 году Энди Клин (Andi Kleen) написал оригинальную версию `numastat` на языке Perl. Текущую версию в 2012 году написал Билл Грей (Bill Gray).

7.5.7. ps

Команда `ps(1)` выводит подробную информацию обо всех процессах, включая статистику потребления памяти. Особенности ее использования были описаны в главе 6 «Процессоры».

Вот пример использования параметров в стиле BSD:

```
$ ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY  STAT  START   TIME COMMAND
[...]
bind      1152  0.0  0.4 348916 39568 ?    Ssl   Mar27   20:17 /usr/sbin/named -u bind
root      1371  0.0  0.0  39004  2652 ?    Ss    Mar27   11:04 /usr/lib/postfix/master
root      1386  0.0  0.6 207564 50684 ?    Sl    Mar27    1:57 /usr/sbin/console-kit-
daemon --no-daemon
rabbitmq  1469  0.0  0.0  10708   172 ?    S     Mar27    0:49 /usr/lib/erlang/erts-
5.7.4/bin/epmd -daemon
rabbitmq  1486  0.1  0.0 150208  2884 ?    Ssl   Mar27  453:29 /usr/lib/erlang/erts-
5.7.4/bin/beam.smp -W w -K true -A30 ...
```

Особый интерес представляют следующие столбцы:

- **%MEM**: доля основной памяти (физической памяти, RSS) в процентах от общего объема памяти в системе;
- **RSS**: размер резидентного набора в килобайтах;
- **VSZ**: размер виртуальной памяти в килобайтах.

Столбец RSS показывает объем основной памяти, используемой процессом, но в это число входят также сегменты разделяемой памяти, например, занятые системными библиотеками, которые могут отображаться в адресные пространства десятков процессов. Сложив все значения в столбце RSS, можно обнаружить, что получившаяся сумма превышает объем памяти, доступной в системе, что объясняется многократным суммированием такой разделяемой памяти. Дополнительную информацию о разделяемой памяти см. в разделе 7.2.9 «Разделяемая память», а приемы анализа потребления разделяемой памяти — в разделе с описанием инструмента `rtop(1)` ниже.

Столбцы можно выбирать с помощью параметра `-o` в стиле SVR4, например:

```
# ps -eo pid,pmem,vsz,rss,comm
      PID %MEM  VSZ   RSS COMMAND
[...]
13419  0.0  5176  1796 /opt/local/sbin/nginx
13879  0.1 31060 22880 /opt/local/bin/ruby19
13418  0.0  4984  1456 /opt/local/sbin/nginx
15101  0.0  4580    32 /opt/riak/lib/os_mon-2.2.6/priv/bin/memsup
10933  0.0  3124   2212 /usr/sbin/rsyslogd
[...]
```

Версия для Linux также может выводить столбцы с количеством значительных и незначительных сбоев (`majflt`, `minflt`).

Вывод команды `ps(1)` можно отсортировать по столбцам со статистиками, характеризующими потребление памяти, чтобы быстро выявить самых активных потребителей. Или попробуйте использовать команду `top(1)`, которая поддерживает интерактивную сортировку.

7.5.8. top

Команда `top(1)` выводит информацию о процессах, выполняющихся в системе, включая статистики потребления памяти. Эта команда была представлена в главе 6 «Процессоры». Вот пример вывода, полученный в Linux:

```
$ top -o %MEM
top - 00:53:33 up 242 days,  2:38,  7 users,  load average: 1.48, 1.64, 2.10
Tasks: 261 total,  1 running, 260 sleeping,  0 stopped,  0 zombie
Cpu(s):  0.0%us,  0.0%sy,  0.0%ni, 99.9%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Mem:   8181740k total,  6658640k used,  1523100k free,  404744k buffers
Swap:  2932728k total,  120508k used,  2812220k free,  2893684k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
29625	scott	20	0	2983m	2.2g	1232	S	45	28.7	81:11.31	node
5121	joshw	20	0	222m	193m	804	S	0	2.4	260:13.40	tmux
1386	root	20	0	202m	49m	1224	S	0	0.6	1:57.70	console-kit-dae
6371	stu	20	0	65196	38m	292	S	0	0.5	23:11.13	screen
1152	bind	20	0	340m	38m	1700	S	0	0.5	20:17.36	named
15841	joshw	20	0	67144	23m	908	S	0	0.3	201:37.91	mosh-server
18496	root	20	0	57384	16m	1972	S	3	0.2	2:59.99	python
1258	root	20	0	125m	8684	8264	S	0	0.1	2052:01	l2tpns
16295	wesolows	20	0	95752	7396	944	S	0	0.1	4:46.07	sshd
23783	brendan	20	0	22204	5036	1676	S	0	0.1	0:00.15	bash

[...]

Блок со сводной информацией вверху сообщает общий, использованный и свободный, объем как основной памяти (Mem), так и виртуальной (Swap). Также показаны размеры кэша буферов (buffers) и кэша страниц (cached).

В этом примере процессы были отсортированы по столбцу `%MEM` с помощью параметра `-o`, который определяет столбец для сортировки. Больше всего памяти в этом примере использует процесс `node` — 2,2 Гбайт основной памяти и почти 3 Гбайт виртуальной памяти.

Столбцы, отображающие процент использованной основной памяти (`%MEM`), размер виртуальной памяти (`VIRT`) и размер резидентного набора (`RES`), сообщают те же значения, что и эквивалентные столбцы в выводе `ps(1)`, описанные выше. Дополнительную информацию о статистиках потребления памяти, которые сообщает команда `top(1)`, ищите в разделе «Linux Memory Types» (типы памяти Linux) на странице справочного руководства `man` для `top(1)`. Там же объясняется, какой тип памяти отображается в каждом из столбцов. Во время сеанса работы с `top(1)` также можно ввести «?», чтобы получить список интерактивных команд.

7.5.9. pmap

Команда `pmap(1)` выводит список сегментов, отображенных в адресное пространство процесса, их размеры, разрешения и отображенные объекты. Это позволяет подробно изучить потребление памяти процессом и количественно оценить размер разделяемой памяти.

Вот пример, полученный в системе на базе Linux:

```
# pmap -x 5187
5187: /usr/sbin/mysqld
Address      Kbytes    RSS      Dirty Mode  Mapping
000055dadb0dd000  58284    10748      0 r-x--  mysqld
000055dade9c8000   1316     1316     1316 r----  mysqld
000055dadeb11000   3592      816      764 rw---  mysqld
000055dadee93000   1168     1080     1080 rw---  [ anon ]
000055dae08b5000   5168     4836     4836 rw---  [ anon ]
00007f018c000000    4704     4696     4696 rw---  [ anon ]
00007f018c498000   60832      0        0 ----- [ anon ]
00007f0190000000    132      24       24 rw---  [ anon ]
[...]
00007f01f99da000     4         4         0 r----  ld-2.30.so
00007f01f99db000    136        136      0 r-x--  ld-2.30.so
00007f01f99fd000     32         32      0 r----  ld-2.30.so
00007f01f9a05000     4         0         0 rw-s- [aio] (deleted)
00007f01f9a06000     4         4         4 r----  ld-2.30.so
00007f01f9a07000     4         4         4 rw---  ld-2.30.so
00007f01f9a08000     4         4         4 rw---  [ anon ]
00007ffd2c528000    132        52        52 rw---  [ stack ]
00007ffd2c5b3000     12         0         0 r----  [ anon ]
00007ffd2c5b6000     4         4         0 r-x--  [ anon ]
fffffffff6000000     4         0         0 --x--  [ anon ]
-----
total kB      1828228  450388  434200
```

Здесь показана структура памяти сервера баз данных MySQL, включая виртуальную память (Kbytes), основную память (RSS), анонимную память процесса (Anon) и разрешения (Mode). Многие сегменты анонимной памяти малы, и многие сегменты доступны только для чтения (r-...), что позволяет использовать соответствующие страницы совместно с другими процессами. Особенно это касается системных библиотек. Основная часть используемой памяти в этом примере находится в куче (в этом усеченном выводе ей соответствует первая волна сегментов [anon]).

Параметр `-x` обеспечивает вывод дополнительных полей. Есть возможность использовать параметр `-X` для вывода более подробной информации и `-XX` для вывода всей информации, которую может предоставить ядро. Вот примеры, показывающие лишь заголовки этих дополнительных полей:

```
# pmap -X $(pgrep mysqld) | head -2
5187: /usr/sbin/mysqld
Address Perm  Offset Device  Inode  Size  Rss  Pss Referenced
Anonymous LazyFree ShmemPmdMapped Shared_Hugetlb Private_Hugetlb Swap SwapPss Locked
```

```

THNPeligible ProtectionKey Mapping
[...]
# pmap -XX $(pgrep mysqld) | head -2
5187: /usr/sbin/mysqld
      Address Perm  Offset Device  Inode    Size KernelPageSize MMUPageSize
Rss    Pss Shared_Clean Shared_Dirty Private_Clean Private_Dirty Referenced Anonymous
LazyFree AnonHugePages ShmemPmdMapped Shared_Hugetlb Private_Hugetlb Swap SwapPss
Locked THNPeligible ProtectionKey                               VmFlags Mapping
[...]

```

Перечень дополнительных полей зависит от версии ядра. К их числу относятся поля с информацией о потреблении больших страниц, использовании подкачки и пропорциональном размере набора (Pss) сегментов (выделены жирным). Поле PSS сообщает объем анонимной памяти в сегменте плюс объем разделяемой памяти, деленный на количество пользователей. Это обеспечивает более реалистичное представление о потреблении основной памяти.

7.5.10. perf

perf(1) — это стандартный профилировщик Linux, многоцелевой инструмент со множеством применений. Подробнее о нем я расскажу в главе 13 «perf». В этом разделе рассмотрим приемы его использования для анализа потребления памяти. Также см. главу 6, где шла речь об использовании perf(1) для анализа потребления памяти с применением счетчиков РМС.

Однострочные сценарии

Однострочные сценарии ниже показывают различные возможности perf(1) для анализа потребления памяти.

Выборка сбоев страниц (рост RSS) с трассировками стека в масштабе всей системы. Выборка производится до нажатия Ctrl-C:

```
perf record -e page-faults -a -g
```

Запись в файл всех сбоев страниц с трассировками стека для PID 1843 в течение 60 с:

```
perf record -e page-faults -c 1 -p 1843 -g -- sleep 60
```

Запись событий увеличения кучи путем трассировки обращений к системному вызову brk(2). Запись производится до нажатия Ctrl-C:

```
perf record -e syscalls:sys_enter_brk -a -g
```

Запись событий миграции страниц в системах NUMA:

```
perf record -e migrate:mm_migrate_pages -a
```

Подсчет всех событий kmem с выводом отчета каждую секунду:

```
perf stat -e 'kmem:*' -a -I 1000
```

Подсчет всех событий vmscan с выводом отчета каждую секунду:

```
perf stat -e 'vmscan:*' -a -I 1000
```

Подсчет всех событий уплотнения памяти с выводом отчета каждую секунду:

```
perf stat -e 'compaction:*' -a -I 1000
```

Трассировка событий пробуждения демона kswapd с трассировками стека. Трассировка производится до нажатия Ctrl-C:

```
perf record -e vmscan:mm_vmscan_wakeup_kswapd -ag
```

Профилирование обращений к памяти для указанной команды:

```
perf mem record command
```

Профилирование обращений к памяти в масштабе всей системы:

```
perf mem report
```

Для команд, которые записывают или выбирают события, используйте `perf report` для обобщения результатов профилирования или `perf script --header`, чтобы вывести все результаты.

Еще больше однострочных сценариев с `perf(1)` ищите в главе 13 «perf», в разделе 13.2 «Однострочные сценарии». В разделе 7.5.13 «bpftrace» вашему вниманию будут представлены программы наблюдения за многими из этих же событий.

Выборка сбоев страниц

`perf(1)` может записывать трассировки стека по событиям сбоев страниц, показывающие пути в коде, вызвавшие эти события. Поскольку сбои страниц возникают по мере увеличения размера резидентного набора (RSS) процесса, их анализ может объяснить, почему растет потребление основной памяти в процессе. См. рис. 7.2, где показана роль сбоев страниц в потреблении памяти.

Пример ниже показывает наблюдение за программными событиями сбоя страниц на всех процессорах (`-a`¹) с записью трассировок стека (`-g`) в течение 60 с и последующим их выводом:

```
# perf record -e page-faults -a -g -- sleep 60
[ perf record: Woken up 4 times to write data ]
[ perf record: Captured and wrote 1.164 MB perf.data (2584 samples) ]
# perf script
[...]
```

sleep 4910 [001] 813638.716924: 1 page-faults:
 ffffffff9303f31e __clear_user+0x1e ([kernel.kallsyms])

¹ Начиная с версии Linux 4.11, параметр `-a` подразумевается по умолчанию.

```

ffffff9303f37b clear_user+0x2b ([kernel.kallsyms])
ffffff92941683 load_elf_binary+0xf33 ([kernel.kallsyms])
ffffff928d25cb search_binary_handler+0x8b ([kernel.kallsyms])
ffffff928d38ae __do_execve_file.isra.0+0x4fe ([kernel.kallsyms])
ffffff928d3e09 __x64_sys_execve+0x39 ([kernel.kallsyms])
ffffff926044ca do_syscall_64+0x5a ([kernel.kallsyms])
ffffff9320008c entry_SYSCALL_64_after_hwframe+0x44 ([kernel.kallsyms])
7fb53524401b execve+0xb (/usr/lib/x86_64-linux-gnu/libc-2.30.so)
[...]
mysqld 4918 [000] 813641.075298:          1 page-faults:
7fc6252d7001 [unknown] (/usr/lib/x86_64-linux-gnu/libc-2.30.so)
562cacae282 pfs_malloc_array+0x42 (/usr/sbin/mysqld)
562cacafd582 PFS_buffer_scalable_container<PFS_prepared_stmt, 1024, 1024,
PFS_buffer_default_array<PFS_prepared_stmt>,
PFS_buffer_default_allocator<PFS_prepared_stmt> >::allocate+0x262 (/usr/sbin/mysqld)
562cacafd820 create_prepared_stmt+0x50 (/usr/sbin/mysqld)
562cacadbef [unknown] (/usr/sbin/mysqld)
562cab3719ff mysqld_stmt_prepare+0x9f (/usr/sbin/mysqld)
562cab3479c8 dispatch_command+0x16f8 (/usr/sbin/mysqld)
562cab348d74 do_command+0x1a4 (/usr/sbin/mysqld)
562cab464fe0 [unknown] (/usr/sbin/mysqld)
562cacad873a [unknown] (/usr/sbin/mysqld)
7fc625ceb669 start_thread+0xd9 (/usr/lib/x86_64-linux-gnu/libpthread-
2.30.so)
[...]
```

Здесь я оставил только две трассировки стека. Первая соответствует команде `sleep(1)`, которая вызывается командой `perf(1)`, а вторая — серверу MySQL. При трассировке в масштабе всей системы можно заметить множество стеков короткоживущих процессов, которые ненадолго резервируют память, вызывая сбой страниц. Для наблюдения за конкретным процессом вместо `-a` используйте параметр `-p PID`.

Полный вывод в этом примере составил 222 582 строки. Команда `perf report` обобщила пути в коде в виде иерархии, но и после этого объем вывода составил 7592 строки. Для более эффективной визуализации профиля используйте флейм-графики.

Флейм-графики сбоев страниц

На рис. 7.12 показан флейм-график сбоев страниц, созданный на основе предыдущего профиля.

Флейм-график на рис. 7.12 показывает, что более половины прироста потребления памяти в сервере MySQL обусловлено выполнением `JOIN::optimize()` (левая большая пирамида). Если навести курсор на фрейм с `JOIN::optimize()`, то можно увидеть, что эта и другие функции, вызываемые ею, породили 3226 сбоев страниц; для размера одной страницы 4 Кбайт это означает общий прирост потребления основной памяти на 12 Мбайт.



Рис. 7.12. Флейм-график сбоев страниц

Для создания этого флейм-графика использовались следующие команды, включая команду `perf(1)` для записи сбоев страниц:

```
# perf record -e page-faults -a -g -- sleep 60
# perf script --header > out.stacks
$ git clone https://github.com/brendangregg/FlameGraph; cd FlameGraph
$ ./stackcollapse-perf.pl < ../out.stacks | ./flamegraph.pl --hash \
  --bgcolor=green --count=pages --title="Page Fault Flame Graph" > out.svg
```

Я выбрал зеленый фон для напоминания, что это не обычный флейм-график процессорного времени (желтый фон), а флейм-график потребления памяти (зеленый фон).

7.5.11. drsnoop

`drsnoop(8)`¹ — это инструмент ВСС для трассировки событий прямого освобождения памяти, показывающий затронутый процесс и задержку: время, затраченное на освобождение памяти. Его можно использовать для количественной оценки влияния приложения на производительность системы с ограниченным объемом памяти. Например:

```
# drsnoop -T
TIME(s)      COMM      PID      LAT(ms)  PAGES
0.000000000  java      11266    1.72     57
0.004007000  java      11266    3.21     57
0.011856000  java      11266    2.02     43
0.018315000  java      11266    3.09     55
0.024647000  acpid     1209     6.46     73
[...]
```

Как показывает этот вывод, для процесса Java произошли события непосредственного освобождения памяти, длившиеся от одной до семи миллисекунд. Частота этих операций и их продолжительность могут учитываться при количественной оценке влияния на затронутое приложение.

В своей работе этот инструмент использует точки трассировки в функции `vmscan: mm_vm_scan_direct_reclaim_begin` и `mm_vm_scan_direct_reclaim_end`. Предполагается, что эти события будут возникать нечасто (как правило, пакетами), поэтому оверхед должен быть незначительным.

`drsnoop(8)` поддерживает параметр `-T` для включения отметок времени и `-p PID` для организации наблюдения за конкретным процессом.

7.5.12. wss

`wss(8)` — это экспериментальный инструмент, который я разработал, чтобы показать, как можно измерить размер рабочего набора процесса (WSS), используя бит «accessed» в записи в таблице страниц (page table entry, PTE). Инструмент был создан в процессе длительного исследования способов определения размера рабочего набора [Gregg 18c]. Я включил описание `wss(8)` в книгу потому, что размер рабочего набора (объем часто используемой памяти) — это важный показатель, помогающий понять характер использования памяти, а наличие хотя бы экспериментального инструмента с оговорками лучше, чем отсутствие всякого инструмента.

Пример ниже — измерение WSS сервера баз данных MySQL (`mysqld`) с помощью `wss(8)` и с выводом совокупного размера WSS каждую секунду:

```
# ./wss.pl $(pgrep -n mysqld) 1
Watching PID 423 page references grow, output every 1 seconds...
Est(s)      RSS(MB)    PSS(MB)    Ref(MB)
1.014       403.66      400.59      86.00
```

¹ Немного истории: этот инструмент создал Венбо Чжан (Wenbo Zhang) 10 февраля 2019 года.

2.034	403.66	400.59	90.75
3.054	403.66	400.59	94.29
4.074	403.66	400.59	97.53
5.094	403.66	400.59	100.33
6.114	403.66	400.59	102.44
7.134	403.66	400.59	104.58
8.154	403.66	400.59	106.31
9.174	403.66	400.59	107.76
10.194	403.66	400.59	109.14

Согласно полученным результатам, к пятой секунде сервер `mysqld` использовал около 100 Мбайт памяти. RSS для `mysqld` составлял 400 Мбайт. Вывод также включает оценку времени, прошедшего с начала трассировки, в том числе времени, необходимого для установки и чтения бита «accessed» (`Est(s)`), и PSS, который учитывает страницы, совместно используемые с другими процессами.

В процессе работы этот инструмент сбрасывает бит «accessed» в PTE для каждой страницы, используемой процессом, приостанавливается на заданный интервал, а затем проверяет биты, чтобы определить, какие из них были установлены. Поскольку инструмент анализирует страницы, он показывает результаты с точностью до страниц, размер которых обычно составляет 4 Кбайт. Считайте, что числа в выводе округлены до величины, кратной размеру страницы.

ВНИМАНИЕ: этот инструмент использует `/proc/PID/clear_refs` и `/proc/PID/smaps`, что может вызвать увеличение задержек в работе приложения (например, 10 %) на время, необходимое ядру для обхода страниц. Для больших процессов (занимающих больше 100 Гбайт) время, когда инструмент будет интенсивно использовать процессор, может составить больше одной секунды. Помните об оверхеде. Этот инструмент также сбрасывает упомянутый флаг «accessed», что может помешать ядру правильно выбирать страницы для освобождения, особенно если активна подкачка. Кроме того, он активирует некоторый старый код в ядре, который раньше мог не использоваться в вашем окружении. Сначала опробуйте инструмент в тестовой среде, чтобы оценить оверхед.

7.5.13. bpftrace

`bpftrace` — это трассировщик, основанный на BPF, который поддерживает язык программирования высокого уровня, позволяющий создавать мощные и короткие сценарии. Хорошо подходит для анализа приложений на основе подсказок, полученных с помощью других инструментов. В репозитории `bpftrace` есть инструменты для анализа памяти, в том числе `oomkill.bt` [Robertson 20].

Подробнее о `bpftrace` идет речь в главе 15 «BPF». В этом разделе я покажу примеры однострочных сценариев для анализа потребления памяти.

Однострочные сценарии

Однострочные сценарии ниже показывают различные возможности `bpftrace`.

Подсчет количества байтов, выделяемых обращениями к `malloc()` в `libc`, по трассировкам стека в пространстве пользователя и процессам (имеет высокий оверхед):

```
bpfftrace -e 'uprobe:/lib/x86_64-linux-gnu/libc.so.6:malloc {
    @[ustack, comm] = sum(arg0); }'
```

Подсчет количества байтов, выделяемых обращениями к `malloc()` в `libc`, по трассировкам стека в пространстве пользователя в процессе PID 181 (высокий оверхед):

```
bpfftrace -e 'uprobe:/lib/x86_64-linux-gnu/libc.so.6:malloc /pid == 181/ {
    @[ustack] = sum(arg0); }'
```

Показывает в виде гистограмм с шагом, равным степени двойки, распределение размеров блоков памяти, выделяемых обращениями к `malloc()`, по трассировкам стека в пространстве пользователя в процессе PID 181 (высокий оверхед):

```
bpfftrace -e 'uprobe:/lib/x86_64-linux-gnu/libc.so.6:malloc /pid == 181/ {
    @[ustack] = hist(arg0); }'
```

Подсчет количества байтов, выделенных в кэше ядра `kmem` по трассировкам стека в пространстве ядра:

```
bpfftrace -e 't:kmem:kmem_cache_alloc { @bytes[kstack] = sum(args->bytes_alloc); }'
```

Подсчет событий увеличения кучи процесса (`brk(2)`) по трассировкам стека:

```
bpfftrace -e 'tracepoint:syscalls:sys_enter_brk { @[ustack, comm] = count(); }'
```

Подсчет сбоев страниц по процессам:

```
bpfftrace -e 'software:page-fault:1 { @[comm, pid] = count(); }'
```

Подсчет сбоев страниц в пространстве пользователя по трассировкам стека в пространстве пользователя:

```
bpfftrace -e 't:exceptions:page_fault_user { @[ustack, comm] = count(); }'
```

Подсчет операций `vmscan` по точкам трассировки:

```
bpfftrace -e 'tracepoint:vmscan:* { @[probe] = count(); }'
```

Подсчет событий подкачки страниц по процессам:

```
bpfftrace -e 'kprobe:swap_readpage { @[comm, pid] = count(); }'
```

Подсчет миграций страниц:

```
bpfftrace -e 'tracepoint:migrate:mm_migrate_pages { @ = count(); }'
```

Трассировка событий уплотнения памяти:

```
bpfftrace -e 't:compaction:mm_compaction_begin { time(); }'
```


Мы видим, что в период трассировки этот путь в коде вызвал `malloc()` 676 раз, чтобы выделить память размером 32 Кбайт до 64 Кбайт, и 338 раз, чтобы выделить память размером 64 Кбайт до 128 Кбайт.

Флейм-график размеров памяти, выделяемой обращением к `malloc()`

Вывод предыдущего однострочного сценария очень длинный, поэтому его лучше анализировать в форме флейм-графика. Создать график можно следующими командами:

```
# bpfttrace -e 'u:/lib/x86_64-linux-gnu/libc.so.6:malloc /pid == 4840/ {
    @[ustack] = hist(arg0); }' > out.stacks
$ git clone https://github.com/brendangregg/FlameGraph; cd FlameGraph
$ ./stackcollapse-bpfttrace.pl < ../out.stacks | ./flamegraph.pl --hash \
  --bgcolor=green --count=bytes --title="malloc() Bytes Flame Graph" > out.svg
```

ВНИМАНИЕ: запросы на выделение памяти в пространстве пользователя могут следовать очень часто, до миллионов раз в секунду. Стоимость инструментации здесь невелика, но если умножить ее на высокую частоту запросов, есть риск получить значительный оверхед и снижение производительности целевого процесса в два и более раза. Так что используйте этот прием с осторожностью. Из-за высокого оверхеда я сначала делаю профилирование процессора, чтобы получить информацию о путях в коде, заканчивающихся распределением памяти, или трассирую сбои страниц, как показано в следующем разделе.

Флейм-графики сбоев страниц

Трассировка сбоев страниц помогает узнать, когда процесс увеличивает объем используемой памяти. Предыдущий однострочный сценарий, трассирующий обращения к функции `malloc()`, позволяет определить пути в коде, заканчивающиеся выделением памяти. Пример трассировки сбоев страниц был показан выше в разделе 7.5.10 «perf», и там же было показано, как получить флейм-график. Преимущество использования `bpfttrace` вместо `perf` заключается в возможности увеличить эффективность за счет обобщения трассировок стека в пространстве ядра и передачи в пространство пользователя только уникальных стеков и счетчиков.

Вот пример использования `bpfttrace` для сбора трассировок стека, ведущих к сбоям страниц, и последующего создания флейм-графика:

```
# bpfttrace -e 't:exceptions:page_fault_user { @[ustack, comm] = count(); }
  ' > out.stacks
$ git clone https://github.com/brendangregg/FlameGraph; cd FlameGraph
$ ./stackcollapse-bpfttrace.pl < ../out.stacks | ./flamegraph.pl --hash \
  --bgcolor=green --count=pages --title="Page Fault Flame Graph" > out.svg
```

Примеры трассировок стека, ведущих к сбоям страниц, и флейм-графика ищите в разделе 7.5.10 «perf».

Исследование внутренних особенностей управления памятью

При необходимости можно написать свои инструменты для более глубокого исследования распределения памяти и внутренних особенностей управления ею. Для начала попробуйте использовать точки трассировки для событий распределения памяти в ядре и зонды USDT для распределителей в библиотеках, например `libc`. Вот так можно получить список точек трассировки:

```
# bpftrace -l 'tracepoint:kmem:*'
tracepoint:kmem:kmalloc
tracepoint:kmem:kmem_cache_alloc
tracepoint:kmem:kmalloc_node
tracepoint:kmem:kmem_cache_alloc_node
tracepoint:kmem:kfree
tracepoint:kmem:kmem_cache_free
[...]
# bpftrace -l 't:*:mm_*'
tracepoint:huge_memory:mm_khugepaged_scan_pmd
tracepoint:huge_memory:mm_collapse_huge_page
tracepoint:huge_memory:mm_collapse_huge_page_isolate
tracepoint:huge_memory:mm_collapse_huge_page_swapin
tracepoint:migrate:mm_migrate_pages
tracepoint:compaction:mm_compaction_isolate_migratepages
tracepoint:compaction:mm_compaction_isolate_freepages
[...]
```

У каждой из этих точек трассировки есть аргументы, которые можно узнать, добавив параметр `-lv`. В этом ядре (5.3) — 12 точек трассировки `kmem` и 47 точек трассировки, имена которых начинаются с «`mm_`».

Список зондов USDT для `libc` в Ubuntu:

```
# bpftrace -l 'usdt:/lib/x86_64-linux-gnu/libc.so.6:'
usdt:/lib/x86_64-linux-gnu/libc.so.6:libc:setjmp
usdt:/lib/x86_64-linux-gnu/libc.so.6:libc:longjmp
usdt:/lib/x86_64-linux-gnu/libc.so.6:libc:longjmp_target
usdt:/lib/x86_64-linux-gnu/libc.so.6:libc:l1l_lock_wait_private
usdt:/lib/x86_64-linux-gnu/libc.so.6:libc:memory_mallocopt_arena_max
usdt:/lib/x86_64-linux-gnu/libc.so.6:libc:memory_mallocopt_arena_test
usdt:/lib/x86_64-linux-gnu/libc.so.6:libc:memory_tunable_tcach_max_bytes
[...]
```

В этой версии `libc` (6) есть 33 зонда USDT.

Если точек трассировки и зондов USDT окажется недостаточно, подумайте о возможности использовать динамическую инструментацию с помощью `kprobes` и `uprobes`.

Для наблюдения за памятью есть зонды типа `watchpoint` (точки наблюдения). Они позволяют наблюдать за событиями обращения к памяти — чтением, записью или выполнением.

Так как события обращения к памяти могут следовать очень часто, их инструментация может повлечь значительный оверхед. Функция `malloc(3)` может вызываться из пространства пользователя миллионы раз в секунду, и с учетом оверхеда на зонды `uprobes` (см. главу 4 «Инструменты наблюдения» раздел 4.3.7 «`uprobes`») их трассировка способна снизить производительность целевого процесса в два или более раза. Будьте осторожны и ищите способы уменьшить оверхед, например используйте карты для обобщения статистик вместо вывода сведений о каждом событии и трассируйте как можно меньшее количество событий одновременно.

7.5.14. Другие инструменты

В табл. 7.6 перечислены инструменты наблюдения за потреблением памяти, представленные в других главах этой книги и в книге «BPF Performance Tools» [Gregg 19].

Таблица 7.6. Другие инструменты наблюдения за потреблением памяти

Раздел	Инструмент	Описание
6.6.11	<code>pmcarch</code>	Показывает общее потребление тактов процессора, включая промахи кэша последнего уровня
6.6.12	<code>tlbstat</code>	Обобщает информацию о тактах, затраченных на обслуживание буфера ассоциативной трансляции (TLB)
8.6.2	<code>free</code>	Статистики емкости кэшей
8.6.12	<code>cachestat</code>	Статистики кэша страниц
[Gregg 19]	<code>oomkill</code>	Выводит дополнительную информацию о событиях механизма OOM Killer
[Gregg 19]	<code>memleak</code>	Выводит пути в коде, возможно, вызывающие утечки памяти
[Gregg 19]	<code>mmapsnoop</code>	Трассирует системный вызов <code>mmap(2)</code> для системы в целом
[Gregg 19]	<code>brkstack</code>	Выводит трассировки стека в пространстве пользователя, ведущие к вызову <code>brk()</code>
[Gregg 19]	<code>shmsnoop</code>	Трассирует обращения к разделяемой памяти и выводит дополнительные детали
[Gregg 19]	<code>faults</code>	Выводит трассировки стека в пространстве пользователя, вызывающие сбои страниц
[Gregg 19]	<code>ffaults</code>	Выводит события сбоев страниц по именам файлов
[Gregg 19]	<code>vmscan</code>	Измеряет время, потребовавшееся сканеру виртуальной машины на уплотнение и освобождение памяти
[Gregg 19]	<code>swapi</code>	Выводит события извлечения страниц из устройства подкачки по процессам
[Gregg 19]	<code>hfaults</code>	Выводит события сбоев больших страниц по процессам

Ниже перечислены дополнительные инструменты наблюдения и источники информации о потреблении памяти в Linux:

- **dmesg**: позволяет проверить наличие сообщений «Out of memory», которые оставляет механизм OOM Killer;
- **dmidecode**: показывает информацию о банках памяти из BIOS;
- **tiptop**: версия `top(1)`, отображающая статистики РМС по процессам;
- **valgrind**: комплект инструментов для анализа производительности, включающий `memcheck` — обертку для распределителей на уровне пользователя, помогающую исследовать потребление памяти и обнаруживать утечки. Требуется значительного оверхеда. В руководстве говорится, что может снизить производительность целевого процесса в 20–30 раз [Valgrind 20];
- **iostat**: если устройство подкачки — это физический диск или раздел, за операциями ввода/вывода с этим устройством, которые свидетельствуют о подкачке, можно наблюдать с помощью `iostat(1)`;
- **/proc/zoneinfo**: содержит статистики по зонам памяти (DMA и т. д.);
- **/proc/buddyinfo**: содержит статистики, характеризующие работу распределителя памяти `buddy`, управляющего страницами ядра;
- **/proc/pagetypeinfo**: содержит статистики страниц свободной памяти ядра; может использоваться для устранения проблем, связанных с фрагментацией памяти ядра;
- **/sys/devices/system/node/node*/numastat**: содержит статистики, характеризующие работу узлов NUMA;
- **SysRq m**: `SysRq` в сочетании с `m` позволяет вывести информацию о памяти в консоль.

Вот пример вывода инструмента из `dmidecode(8)`, показывающий информацию о банках памяти:

```
# dmidecode
[...]
Memory Device
    Array Handle: 0x0003
    Error Information Handle: Not Provided
    Total Width: 64 bits
    Data Width: 64 bits
    Size: 8192 MB
    Form Factor: SODIMM
    Set: None
    Locator: ChannelA-DIMM0
    Bank Locator: BANK 0
    Type: DDR4
    Type Detail: Synchronous Unbuffered (Unregistered)
    Speed: 2400 MT/s
    Manufacturer: Micron
    Serial Number: 00000000
    Asset Tag: None
    Part Number: 4ATS1G64HZ-2G3A1
    Rank: 1
    Configured Clock Speed: 2400 MT/s
```

```
Minimum Voltage: Unknown
Maximum Voltage: Unknown
Configured Voltage: 1.2 V
```

[...]

Этот вывод содержит информацию, полезную для настройки статической производительности (например, здесь видно, что в системе есть ОЗУ типа DDR4, а не DDR5). К сожалению, эта информация обычно недоступна для гостевых систем в облачных средах.

Вот пример вывода, полученный комбинацией SysRq m:

```
# echo m > /proc/sysrq-trigger
# dmesg
[...]
[334849.389256] sysrq: Show Memory
[334849.391021] Mem-Info:
[334849.391025] active_anon:110405 inactive_anon:24 isolated_anon:0
                active_file:152629 inactive_file:137395 isolated_file:0
                unevictable:4572 dirty:311 writeback:0 unstable:0
                slab_reclaimable:31943 slab_unreclaimable:14385
                mapped:37490 shmem:186 pagetables:958 bounce:0
                free:37403 free_pcp:478 free_cma:2289
[334849.391028] Node 0 active_anon:441620kB inactive_anon:96kB active_file:610516kB
                inactive_file:549580kB unevictable:18288kB isolated(anon):0kB isolated(file):0kB
                mapped:149960kB dirty:1244kB writeback:0kB shmem:744kB shmem_thp: 0kB
                shmem_pmdmapped: 0kB anon_thp: 2048kB writeback_tmp:0kB unstable:0kB
                all_unreclaimable? no
[334849.391029] Node 0 DMA free:12192kB min:360kB low:448kB high:536kB ...
[...]
```

Этот прием полезен, если система окажется заблокирована, потому что даже в таком случае можно запросить информацию, нажав последовательность клавиш SysRq на клавиатуре консоли, если она доступна [Linux 20g].

Приложения и виртуальные машины (например, Java VM) также могут предоставлять свои инструменты для анализа потребления памяти. См. главу 5 «Приложения».

7.6. НАСТРОЙКА

Самая важная настройка потребления памяти — обеспечить размещение всех приложений в основной памяти и сделать так, чтобы подкачка страниц происходила как можно реже. В разделе 7.4 «Методология» и в разделе 7.5 «Инструменты наблюдения» мы уже обсудили, как выявить эту проблему. В этом разделе обсудим другие настройки: параметры ядра, настройку больших страниц и распределителей, а также средства управления ресурсами.

Конкретные настройки — доступные параметры и оптимальные значения — зависят от версии ОС и предполагаемой рабочей нагрузки. В следующих разделах, организованных по типам настроек, приводятся примеры доступных настраиваемых параметров и описываются причины, почему может потребоваться их настройка.

7.6.1. Настраиваемые параметры

В этом разделе представлены примеры параметров, доступных для настройки в последних версиях ядра Linux.

Параметры, управляющие распределением памяти, описаны в документации в исходном коде ядра в `Documentation/sysctl/vm.txt` и могут настраиваться с помощью `sysctl(8)`. В табл. 7.7 приводятся примеры значений по умолчанию параметров из Ubuntu 19.10 в ядре 5.3 (значения, указанные в первом издании этой книги, остались прежними).

Таблица 7.7. Примеры параметров, управляющих распределением памяти в Linux

Параметр	Значение по умолчанию	Описание
<code>vm.dirty_background_bytes</code>	0	Верхний предел объема «грязной» (измененной) памяти в <i>байтах</i> , после превышения которого демон <code>pdflush</code> начинает запись измененных страниц в фоновом режиме
<code>vm.dirty_background_ratio</code>	10	Верхний предел объема «грязной» (измененной) памяти в <i>процентах</i> , после превышения которого демон <code>pdflush</code> начинает запись измененных страниц в фоновом режиме
<code>vm.dirty_bytes</code>	0	Верхний предел объема «грязной» (измененной) памяти в <i>байтах</i> , после превышения которого процесс начинает запись измененных страниц
<code>vm.dirty_ratio</code>	20	Верхний предел объема «грязной» (измененной) памяти в <i>процентах</i> , после превышения которого процесс начинает запись измененных страниц
<code>vm.dirty_expire_centisecs</code>	3000	Минимальное время (в сотых долях секунды), в течение которого «грязная» (измененная) память может игнорироваться демоном <code>pdflush</code> (способствует <i>отмене записи</i>)
<code>vm.dirty_writeback_centisecs</code>	500	Интервал (в сотых долях секунды) активации демона <code>pdflush</code> (0 — запуск демона отключен)
<code>vm.min_free_kbytes</code>	выбирается динамически	Желаемый объем свободной памяти (может использоваться некоторыми атомарными операциями распределения в ядре)
<code>vm.watermark_scale_factor</code>	10	Расстояние между пороговыми уровнями в демоне <code>ksward</code> (минимальный, низкий, высокий), которые управляют его активностью (измеряется в 10 000-ных долях, то есть 10 означает 0,1 % объема системной памяти)

Параметр	Значение по умолчанию	Описание
vm.watermark_boost_factor	5000	Насколько выше верхнего предела заходит kswarp при сканировании, когда память фрагментирована (недавно имели место события фрагментации); измеряется в 10 000-ных долях, то есть 5000 означает, что kswarp может подниматься выше верхнего предела до 150 %; 0 — отключено
vm.percpu_pagelist_fraction	0	С помощью этого параметра можно переопределить максимальную долю страниц для списков страниц каждого процессора (значение 10 соответствует 1/10 доле страниц)
vm.overcommit_memory	0	0 = использовать эвристику для разумного превышения пределов доступной памяти при распределении; 1 = всегда разрешать превышать пределы; 2 = никогда не разрешать превышать пределы
vm.swappiness	60	Степень предпочтительности освобождения памяти из кэша страниц вместо подкачки
vm.vfs_cache_pressure	100	Уровень активности по отношению к освобождению кэшированных объектов каталогов и индексных узлов (inode); чем ниже значение, тем неохотнее освобождаются страницы, занятые этими объектами; 0 означает, что эта память никогда не будет освобождаться — это может быстро привести к нехватке памяти
kernel.numa_balancing	1	Разрешает автоматическую балансировку NUMA
kernel.numa_balancing_scan_size_mb	256	Сколько мегабайт проверяется при каждом сканировании страниц во время балансировки NUMA

Для обозначения настроек используется согласованная схема именования, включающая единицы измерения. Обратите внимание, что `dirty_background_bytes` и `dirty_background_ratio` взаимно исключают друг друга, так же как `dirty_bytes` и `dirty_ratio` (когда один параметр установлен, он отменяет действие другого).

Размер `vm.min_free_kbytes` выбирается динамически как доля основной памяти. Алгоритм выбора этого параметра нелинейный, потому что потребность в свободной памяти не связана линейно с объемом основной памяти. (Это задокументировано в исходном коде Linux в `mm/page_alloc.c`.) Значение `vm.min_free_kbytes` можно уменьшить, чтобы освободить часть памяти для приложений, но это также может привести к увеличению нагрузки на ядро в периоды нехватки памяти и раннему срабатыванию механизма OOM Killer. Увеличение параметра поможет избежать остановки процессов механизмом OOM Killer.

Другой параметр, позволяющий избежать срабатывания OOM Killer, — `vm.overcommit_memory`. Ему можно присвоить значение 2, чтобы отключить возможность выделения памяти без ограничений и избежать случаев, когда такие попытки приводят к срабатыванию OOM Killer. Если у вас возникнет необходимость управлять механизмом OOM Killer на уровне процессов, то проверьте в своем ядре наличие таких параметров, как `oom_adj` или `oom_score_adj`. Они описаны в `Documentation/filesystems/proc.txt`.

Параметр `vm.swappiness` может значительно ухудшить производительность, если из-за него подкачка страниц памяти приложения начнется раньше, чем требуется. Этот параметр имеет значение в диапазоне от 0 до 100, при этом чем выше значение, тем выше склонность к использованию подкачки и, следовательно, сохранению кэша страниц на диск. Иногда желательно присвоить этому параметру 0, чтобы память приложения оставалась в кэше страниц как можно дольше. Если возникнет состояние нехватки памяти, ядро все еще сможет использовать подкачку.

В системах Netflix с более ранними версиями ядер (близких к Linux 3.13) параметру `kernel.numa_balancing` присваивалось значение 0, потому что чрезмерно активное сканирование NUMA потребляло слишком много вычислительных ресурсов [Gregg 17d]. Эта проблема была исправлена в более поздних ядрах, а кроме того, появились другие настройки, включая `kernel.numa_balancing_scan_size_mb`, позволяющие регулировать активность сканирования NUMA.

7.6.2. Несколько размеров страниц

Использование страниц больших размеров может улучшить производительность операций чтения/записи с памятью за счет увеличения коэффициента попадания в кэш TLB (увеличения его покрытия). Большинство современных процессоров поддерживают несколько размеров страниц, например, 4 Кбайт по умолчанию и большие страницы размером 2 Мбайт.

В Linux большие страницы (в этой ОС их называют *огромными страницами* — *huge pages*) можно настроить несколькими способами. Для справки см. `Documentation/vm/hugetlbpage.txt`.

Обычно процесс настройки начинается с создания огромных страниц:

```
# echo 50 > /proc/sys/vm/nr_hugepages
# grep Huge /proc/meminfo
AnonHugePages:      0 kB
HugePages_Total:    50
HugePages_Free:      50
HugePages_Rsvd:      0
HugePages_Surp:      0
Hugepagesize:       2048 kB
```

Использовать огромные страницы в приложении можно, например, посредством сегментов разделяемой памяти и флага `SHM_HUGETLBS` для `shmget(2)`. Другой

способ предполагает создание огромной страничной файловой системы, которую приложения будут отображать в память:

```
# mkdir /mnt/hugetlbfs
# mount -t hugetlbfs none /mnt/hugetlbfs -o pagesize=2048K
```

В числе других способов — использование флагов `MAP_ANONYMOUS` | `MAP_HUGETLB` при вызове `mmap(2)` и использование библиотеки `libhugetlbfs` [Gorman 10].

Еще один механизм, позволяющий использовать огромные страницы, — *прозрачные огромные страницы* (transparent huge pages, THP). Он автоматически преобразует обычные страницы в огромные и обратно, не требуя настраивать использование огромных страниц в приложении [Corbet 11]. Описание этого механизма можно найти в исходном коде Linux в `Documentation/vm/transhuge.txt` и в `admin-guide/mm/transhuge.rst`¹.

7.6.3. Распределители

Для многопоточных приложений доступны различные распределители памяти в пространстве пользователя, способные повысить их производительность. Выбор распределителя можно сделать во время компиляции или во время выполнения, задав переменную среды `LD_PRELOAD`.

Вот пример того, как можно выбрать распределитель `libtcmalloc`:

```
export LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libtcmalloc_minimal.so.4
```

Эту команду можно добавить в сценарий запуска.

7.6.4. Привязка NUMA

В системах NUMA с помощью команды `numactl(8)` можно привязать процессы к узлам NUMA. Это позволит повысить производительность приложений, которым не требуется более одного узла NUMA в основной памяти. Вот пример использования:

```
# numactl --membind = 0 3161
```

Эта команда свяжет процесс PID 3161 с узлом NUMA 0. После этого последующие запросы на выделение памяти в этом процессе будут терпеть неудачу, если их не получится удовлетворить из этого узла. Обязательно познакомьтесь с параметром `--physcpubind`, чтобы вместе с привязкой к определенному узлу NUMA ограничить процесс использованием процессоров, подключенных к этому узлу. Обычно я использую привязки NUMA и привязки к процессору вместе, чтобы ограничить

¹ Обратите внимание, что прозрачным огромным страницам традиционно сопутствовали проблемы производительности, сдерживавшие их использование. Надеемся, что эти проблемы были исправлены.

процесс одним физическим процессором и избежать потери производительности из-за использования внутренней шины между процессорами.

Используйте `numastat(8)` (см. раздел 7.5.6) для вывода списка доступных узлов NUMA.

7.6.5. Управление ресурсами

Базовые средства управления ресурсами, включая ограничение объема основной и виртуальной памяти, доступны через `ulimit(1)`.

В Linux подсистема контрольных групп (`cgroups`) предоставляет дополнительные средства управления памятью, в том числе:

- **memory.limit_in_bytes**: максимальный объем памяти в пространстве пользователя в байтах, включая кэш файлов;
- **memory.memsw.limit_in_bytes**: максимальный объем памяти и пространства подкачки в байтах (когда используется подкачка);
- **memory.kmem.limit_in_bytes**: максимальный объем памяти ядра в байтах;
- **memory.tcp.limit_in_bytes**: максимальный объем памяти для буферов TCP в байтах;
- **memory.swappiness**: напоминает параметр `vm.swappiness`, описанный выше, но может устанавливаться для контрольной группы;
- **memory.oom_control**: может иметь значение 0, разрешающее применение OOM Killer к этой контрольной группе, или 1 — запрещающее это.

Linux также позволяет настроить общесистемную конфигурацию в `/etc/security/limits.conf`.

Дополнительные сведения об управлении ресурсами см. в главе 11 «Облачные вычисления».

7.7. УПРАЖНЕНИЯ

1. Ответьте на следующие вопросы, касающиеся терминологии:
 - Что такое страница памяти?
 - Что такое резидентная память?
 - Что такое виртуальная память?
 - В чем разница между подкачкой страниц (`paging`) и анонимной подкачкой (`swapping`) с точки зрения терминологии, принятой в Linux?
2. Ответьте на следующие концептуальные вопросы:
 - В чем состоит цель подкачки страниц по требованию?
 - Опишите характеристики потребления и насыщения памяти.

- Для чего предназначены MMU и TLB?
 - Для чего предназначен демон подкачки страниц?
 - Для чего предназначен механизм OOM Killer?
3. Ответьте на следующие, более сложные вопросы:
- Что такое анонимная подкачка страниц (swapping) и почему ее анализ важнее, чем анализ подкачки страниц файловой системы?
 - Опишите, какие шаги выполняет ядро Linux, чтобы освободить больше памяти, когда свободная память исчерпывается.
 - Опишите, какие преимущества в производительности дает распределение slab.
4. Разработайте следующие процедуры для своей ОС:
- Чек-лист анализа потребления памяти для метода USE. Опишите, как получить каждую метрику (например, какую команду выполнить) и как интерпретировать результаты. Перед установкой или использованием дополнительных программных продуктов постарайтесь ограничиться инструментами наблюдения, имеющимися в ОС.
 - Чек-лист для определения характеристик рабочей нагрузки с точки зрения потребления памяти. Опишите, как получить каждую метрику, и для начала попробуйте ограничиться инструментами наблюдения, имеющимися в ОС.
5. Решите следующие задачи:
- Выберите приложение и выясните, какие пути в коде ведут к выделению памяти (malloc(3)).
 - Выберите приложение, постоянно наращивающее объем потребляемой памяти (вызовом brk(2) или sbrk(2)), и выясните, какие пути в коде ведут к этому наращиванию.
 - Охарактеризуйте потребление памяти в Linux, опираясь только на следующий вывод:

```
# vmstat 1
procs -----memory----- --swap-- -----io----- --system-- -----cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo    in   cs  us  sy  id  wa  st
 2  0  413344  62284    72   6972    0    0    17    12    1    1  0  0 100  0  0
 2  0  418036  68172    68   3808    0  4692  4520  4692 1060 1939 61 38  0  1  0
 2  0  418232  71272    68   1696    0  196 23924  196 1288 2464 51 38  0 11  0
 2  0  418308  68792    76   2456    0   76  3408    96 1028 1873 58 39  0  3  0
 1  0  418308  67296    76   3936    0    0  1060    0 1020 1843 53 47  0  0  0
 1  0  418308  64948    76   3936    0    0    0    0 1005 1808 36 64  0  0  0
 1  0  418308  62724    76   6120    0    0  2208    0 1030 1870 62 38  0  0  0
 1  0  422320  62772    76   6112    0  4012    0  4016 1052 1900 49 51  0  0  0
 1  0  422320  62772    76   6144    0    0    0    0 1007 1826 62 38  0  0  0
 1  0  422320  60796    76   6144    0    0    0    0 1008 1817 53 47  0  0  0
 1  0  422320  60788    76   6144    0    0    0    0 1006 1812 49 51  0  0  0
 3  0  430792  65584    64   5216    0  8472  4912  8472 1030 1846 54 40  0  6  0
 1  0  430792  64220    72   6496    0    0  1124    16 1024 1857 62 38  0  0  0
```

2	0	434252	68188	64	3704	0	3460	5112	3460	1070	1964	60	40	0	0	0
2	0	434252	71540	64	1436	0	0	21856	0	1300	2478	55	41	0	4	0
1	0	434252	66072	64	3912	0	0	2020	0	1022	1817	60	40	0	0	0

[...]

6. (опционально) Найдите или разработайте метрики, оценивающие работу политик ядра, управляющих локальностью памяти NUMA. Разработайте рабочие нагрузки с хорошей или плохой локальностью памяти для тестирования этих метрик.

7.8. ССЫЛКИ

- [Corbató 68] Corbató, F. J., A Paging Experiment with the Multics System, MIT Project MAC Report MAC-M-384, 1968.
- [Denning 70] Denning, P., «Virtual Memory», ACM Computing Surveys (CSUR) 2, no. 3, 1970.
- [Peterson 77] Peterson, J., and Norman, T., «Buddy Systems», Communications of the ACM, 1977.
- [Thompson 78] Thompson, K., UNIX Implementation, Bell Laboratories, 1978.
- [Babaoglu 79] Babaoglu, O., Joy, W., and Porcar, J., Design and Implementation of the Berkeley Virtual Memory Extensions to the UNIX Operating System, Computer Science Division, Department of Electrical Engineering and Computer Science, University of California, Berkeley, 1979.
- [Bach 86] Bach, M. J., The Design of the UNIX Operating System, Prentice Hall, 1986.
- [Bonwick 94] Bonwick, J., «The Slab Allocator: An Object-Caching Kernel Memory Allocator», USENIX, 1994.
- [Bonwick 01] Bonwick, J., and Adams, J., «Magazines and Vmem: Extending the Slab Allocator to Many CPUs and Arbitrary Resources», USENIX, 2001.
- [Corbet 04] Corbet, J., «2.6 swapping behavior», LWN.net, <http://lwn.net/Articles/83588>, 2004.
- [Gorman 04] Gorman, M., Understanding the Linux Virtual Memory Manager, Prentice Hall, 2004.
- [McDougall 06a] McDougall, R., Mauro, J., and Gregg, B., Solaris Performance and Tools: DTrace and MDB Techniques for Solaris 10 and OpenSolaris, Prentice Hall, 2006.
- [Ghemawat 07] Ghemawat, S., «TCMalloc: Thread-Caching Malloc», <https://gperftools.github.io/gperftools/tcmalloc.html>, 2007.
- [Lameter 07] Lameter, C., «SLUB: The unqueued slab allocator V6», Linux kernel mailing list, <http://lwn.net/Articles/229096>, 2007.
- [Gorman 10] Gorman, M., «Huge pages part 2: Interfaces», LWN.net, <http://lwn.net/Articles/375096>, 2010.
- [Corbet 11] Corbet, J., «Transparent huge pages in 2.6.38», LWN.net, <http://lwn.net/Articles/423584>, 2011.
- [Facebook 11] «Scalable memory allocation using jemalloc», Facebook Engineering, <https://www.facebook.com/notes/facebook-engineering/scalable-memory-allocation-using-jemalloc/480222803919>, 2011.

- [Evans 17] Evans, J., «Swapping, memory limits, and cgroups», <https://jvns.ca/blog/2017/02/17/mystery-swap>, 2017.
- [Gregg 17d] Gregg, B., «AWS re:Invent 2017: How Netflix Tunes EC2», <http://www.brendangregg.com/blog/2017-12-31/reinvent-netflix-ec2-tuning.html>, 2017.
- [Corbet 18a] Corbet, J., «Is it time to remove ZONE_DMA?» LWN.net, <https://lwn.net/Articles/753273>, 2018.
- [Crucial 18] «The Difference between RAM Speed and CAS Latency», <https://www.crucial.com/articles/about-memory/difference-between-speed-and-latency>, 2018.
- [Gregg 18c] Gregg, B., «Working Set Size Estimation», <http://www.brendangregg.com/wss.html>, 2018.
- [Facebook 19] «Getting Started with PSI», Facebook Engineering, <https://facebookmicrosites.github.io/psi/docs/overview>, 2019.
- [Gregg 19] Gregg, B., «BPF Performance Tools: Linux System and Application Observability»¹, Addison-Wesley, 2019.
- [Intel 19a] Intel 64 and IA-32 Architectures Software Developer’s Manual, Combined Volumes: 1, 2A, 2B, 2C, 3A, 3B and 3C, Intel, 2019.
- [Amazon 20] «Amazon EC2 High Memory Instances», <https://aws.amazon.com/ec2/instance-types/high-memory>, по состоянию на 2020.
- [Linux 20g] «Linux Magic System Request Key Hacks», Linux documentation, <https://www.kernel.org/doc/html/latest/admin-guide/sysrq.html>, по состоянию на 2020.
- [Robertson 20] Robertson, A., «bpftrace», <https://github.com/iovisor/bpftrace>, последнее обновление 2020.
- [Valgrind 20] «Valgrind Documentation», <http://valgrind.org/docs/manual>, May 2020.

¹ Грегг Б. «BPF: Профессиональная оценка производительности». Выходит в издательстве «Питер» в 2023 году.

Глава 8

ФАЙЛОВЫЕ СИСТЕМЫ

Производительность файловых систем для приложений часто намного важнее, чем производительность дисков или устройств хранения. Приложения взаимодействуют именно с файловыми системами и ожидают, когда те выполнят свои операции. Чтобы исключить задержки на уровне диска (или удаленной системы хранения), файловые системы могут использовать кэширование, буферизацию и асинхронный ввод/вывод.

Инструменты наблюдения и анализа производительности традиционно фокусируются на работе дисков, обходя вниманием работу файловой системы. Эта глава проливает свет на файловые системы, показывает, как они работают, и рассказывает, как оценить возникающие в них задержки. Все это позволяет вычеркнуть файловые системы и используемые ими дисковые устройства из числа причин низкой производительности и перейти к исследованию других аспектов.

Цели этой главы:

- представить модели файловых систем и основные понятия;
- показать, как рабочие нагрузки на файловые системы влияют на производительность;
- познакомить с кэшированием в файловых системах;
- познакомить с внутренним устройством файловых систем и средствами обеспечения высокой производительности;
- перечислить различные методики анализа производительности файловых систем;
- познакомить с приемами измерения задержек в файловых системах для определения модальностей и выбросов;
- познакомить с приемами использования инструментов трассировки для изучения характера потребления файловых систем;
- показать, как тестировать производительность файловых систем с помощью микробенчмарков;
- познакомить с параметрами настройки файловых систем.

В главе шесть частей. В первых трех я расскажу об основах анализа производительности файловых систем, а в следующих трех покажу их практическое применение в системах на базе Linux.

- **Основы** представляет терминологию файловых систем и основные модели, иллюстрирующие принципы работы файловых систем и ключевые идеи производительности.
- **Архитектура** описывает обобщенную архитектуру файловых систем и некоторые специфические особенности.
- **Методология** описывает методологии анализа эффективности, как наблюдательные, так и экспериментальные.
- **Инструменты наблюдения** описывает инструменты наблюдения за работой файловых систем в Linux, включая статическую и динамическую инструментацию.
- **Экспериментирование** обобщает возможности инструментов бенчмаркинга файловых систем.
- **Настройка** описывает параметры настройки файловых систем.

8.1. ТЕРМИНОЛОГИЯ

Ниже перечислены основные термины этой главы, связанные с файловыми системами:

- **Файловая система** — организация данных в виде файлов и каталогов с файловым интерфейсом доступа к ним и разрешениями, управляющими доступом. Дополнительно файловая система может определять специальные типы файлов для устройств, сокетов и каналов, а также поддерживать некоторые метаданные, включая время последнего доступа к файлам.
- **Кэш файловой системы** — область в основной памяти (обычно DRAM), которая используется для кэширования содержимого файловой системы и может включать кэши для различных типов данных и метаданных.
- **Операции** — операции файловой системы — это запросы к файловой системе, включая `read(2)`, `write(2)`, `open(2)`, `close(2)`, `stat(2)`, `mkdir(2)` и др.
- **Ввод/вывод** — понятие ввода/вывода в файловой системе можно определить несколькими способами. В этой главе термин используется только для обозначения операций, которые непосредственно выполняют чтение и/или запись (осуществляют ввод/вывод), включая `read(2)`, `write(2)`, `stat(2)` (читает статистики) и `mkdir(2)` (запись нового элемента каталога). Операции `open(2)` и `close(2)` на считаются вводом/выводом (несмотря на то, что обновляют метаданные и косвенно могут запускать операции дискового ввода/вывода).
- **Логический ввод/вывод** — ввод/вывод между приложением и файловой системой.

- **Физический ввод/вывод** — ввод/вывод между файловой системой и дисками непосредственно (или через низкоуровневый ввод/вывод).
- **Размер блока** — размер групп данных файловой системы на диске, также известный как *размер записи* (record size). См. подраздел «Блоки и экстенды» в разделе 8.4.4 «Особенности файловых систем».
- **Пропускная способность (throughput)** — текущая скорость передачи данных между приложениями и файловой системой, измеряемая в байтах в секунду.
- **Индексный узел (inode)** — структура данных, содержащая метаданные, описывающие объект файловой системы, включая разрешения, отметки времени и указатели на данные.
- **VFS** — виртуальная файловая система, интерфейс ядра, абстрагирующий поддержку различных типов файловых систем.
- **Том** — экземпляр хранилища, обеспечивающий дополнительную гибкость по сравнению с использованием устройств хранения целиком. Том может быть частью устройства или объединять несколько устройств.
- **Диспетчер томов** — ПО для гибкого управления физическими устройствами хранения и создания *виртуальных томов* для использования операционной системой.

В главе будут вводиться и другие термины. Глоссарий включает базовую терминологию для справки, в том числе: *fsck*, *IOPS (операций ввода/вывода в секунду)*, *частота операций* и *POSIX*. См. также разделы «Терминология» в главах 2 и 3.

8.2. МОДЕЛИ

Ниже описываются простые модели, иллюстрирующие некоторые базовые принципы устройства файловых систем и их работы.

8.2.1. Интерфейсы файловых систем

На рис. 8.1 изображена упрощенная модель файловой системы с точки зрения ее интерфейсов.

На рисунке также обозначены места, где выполняются логические и физические операции. Подробнее об этом рассказывается в разделе 8.3.12 «Логический и физический ввод/вывод».

На рис. 8.1 перечислены типичные операции с объектами. Но ядра могут реализовать дополнительные варианты этих операций: например, в Linux поддерживаются *readv(2)*, *writev(2)*, *openat(2)* и др.

Один из подходов к изучению производительности файловой системы — рассматривать ее как черный ящик, уделяя особое внимание задержкам в операциях с объектами. Более подробно речь об этом идет в разделе 8.5.2 «Анализ задержек».

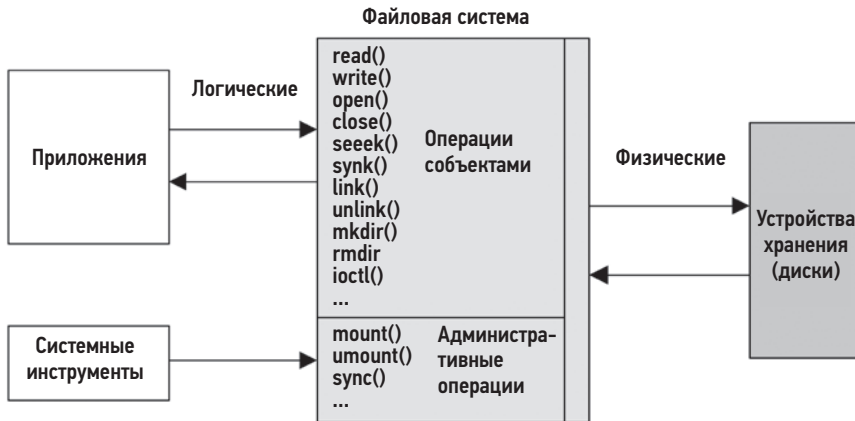


Рис. 8.1. Интерфейсы файловой системы

8.2.2. Кэш файловой системы

На рис. 8.2 изображен типичный кэш файловой системы, хранящийся в основной памяти и обслуживающий операции чтения.

Операция чтения возвращает данные либо из кэша (в случае *попадания в кэш*), либо с диска (в случае *промаха кэша*). В случае промахов данные, прочитанные с диска, сохраняются в кэше, заполняя (разогревая) его.

Кэш файловой системы также используется для буферизации записываемых данных, которые будут записаны (вытолкнуты) позже. В разных файловых системах используются разные механизмы буферизации; они описываются в разделе 8.4 «Архитектура».

Ядра часто позволяют выполнять операции в обход кэша. См. раздел 8.3.8 «Прямой и низкоуровневый ввод/вывод».

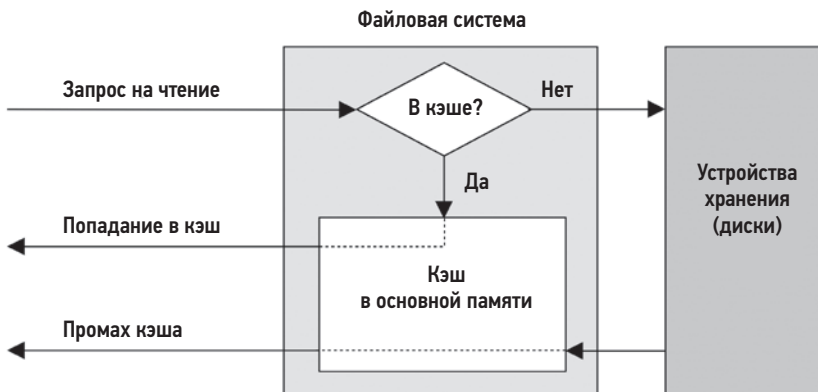


Рис. 8.2. Кэш файловой системы в основной памяти

8.2.3. Кэш второго уровня

Кэш второго уровня может быть в памяти любого типа. На рис. 8.3 показан вариант, когда он находится во флеш-памяти. Этот тип кэша был разработан мной для ZFS в 2007 году.



Рис. 8.3. Кэш второго уровня для файловой системы

8.3. ОСНОВНЫЕ ПОНЯТИЯ

Ниже я расскажу о некоторых важных понятиях, касающихся файловых систем и их производительности.

8.3.1. Задержки в файловой системе

Задержка в файловой системе — это основная метрика, характеризующая производительность файловой системы. Она измеряется как время от логического запроса к файловой системе до завершения его выполнения. Сюда входит время в файловой системе и в подсистеме дискового ввода/вывода, а также ожидание физического ввода/вывода в дисковых устройствах. Потоки выполнения в приложениях часто блокируются на время выполнения запроса, чтобы дождаться результатов от файловой системы. По этой причине задержки в файловой системе *прямо и пропорционально* влияют на производительность приложений.

К случаям, когда производительность приложений не зависит напрямую от задержек в файловой системе, относится использование неблокирующего ввода/вывода и предварительной выборки (раздел 8.3.4) и ситуация, когда ввод/вывод выполняется из асинхронного (например, из фонового) потока. Эти случаи можно выявить, если приложение предоставляет подробные метрики использования файловой системы. В противном случае применяется универсальный подход, основанный на использовании инструментов трассировки ядра, способных отображать трассировки стека в пространстве пользователя, ведущие к логическому вводу/выводу. Эти трассировки стека можно изучить, чтобы увидеть, какие процедуры в приложении ответственны за это.

Исторически сложилось так, что ОС мало внимания уделяли простоте наблюдения за задержками в файловой системе, предлагая вместо этого статистики уровня дисковых устройств. Есть много случаев, когда такая статистика не связана с производительностью приложения, и кроме того, вводит в заблуждение. Например, файловые системы выполняют фоновую очистку записанных данных (background

flushing)), что может выглядеть как всплески дискового ввода/вывода с высокой частотой. По статистикам уровня дисковых устройств это выглядит настораживающе. При этом в системе может не быть ни одного приложения, ожидающего завершения этих операций. Дополнительные сведения см. в разделе 8.3.12 «Логический и физический ввод/вывод».

8.3.2. Кэширование

Для хранения кэшей и увеличения производительности файловая система обычно использует основную память (ОЗУ). Этот процесс не затрагивает приложения: поддержка логического ввода/вывода для приложений становится намного меньше, потому что запросы могут удовлетворяться из основной памяти, а не из гораздо более медленных дисковых устройств.

Со временем объем кэша увеличивается, а объем свободной памяти ОС уменьшается. Подобное может встревожить начинающих пользователей, но это нормально. Здесь действует принцип: если есть свободная основная память, желательно использовать ее для чего-нибудь полезного. Когда приложениям требуется больше памяти, ядро должно быстро освободить ее, сократив кэш файловой системы.

Файловые системы используют кэширование для увеличения производительности операций чтения и буферизацию (в кэше) для увеличения производительности операций записи. Файловая система и подсистема блочных устройств обычно используют кэши нескольких типов, включая перечисленные в табл. 8.1.

Таблица 8.1. Примеры типов кэшей

Кэш	Пример
Кэш страниц	Кэш страниц операционной системы
Первичный кэш файловой системы	ZFS ARC
Вторичный кэш файловой системы	ZFS L2ARC
Кэш каталогов	Кэш записей каталогов (dentry cache)
Кэш индексных узлов (inode)	Кэш индексных узлов (inode)
Кэш устройств	ZFS vdev
Кэш блочных устройств	Кэш буферов

Конкретные типы кэшей описаны в разделе 8.4 «Архитектура», а в главе 3 «Операционные системы» приводится полный список кэшей (включая кэши приложений и устройств).

8.3.3. Произвольный и последовательный ввод/вывод

Последовательность операций логического ввода/вывода можно описать как *произвольную* или *последовательную*, в зависимости от смещения каждой операции от

начала файла. При последовательном вводе/выводе каждая следующая операция начинается с позиции, где завершилась предыдущая. При произвольном вводе/выводе очевидной связи между операциями нет и смещение изменяется произвольным (случайным) образом. Под произвольной рабочей нагрузкой на файловую систему может также подразумеваться произвольный доступ к множеству разных файлов.



Рис. 8.4. Последовательный и произвольный ввод/вывод

На рис. 8.4 показаны эти последовательности операций — операции ввода/вывода и соответствующие им смещения в файле.

Из-за особенностей работы конкретных устройств хранения (описываются в главе 9 «Диски») файловые системы традиционно пытались уменьшить количество произвольных операций ввода/вывода, размещая данные на диске последовательно и непрерывно. Термин *фрагментация* описывает ситуацию, когда файловые системы плохо справляются с этой задачей и файлы «разбрасываются» по всему диску, из-за чего последовательный логический ввод/вывод приводит к произвольному физическому вводу/выводу.

Файловые системы могут оценивать характер логического ввода/вывода, определяя последовательные рабочие нагрузки и увеличивая производительность ввода/вывода за счет предварительной выборки или упреждающего чтения. Это полезно для вращающихся дисков и чуть менее полезно для твердотельных накопителей.

8.3.4. Предварительная выборка

Типичная рабочая нагрузка на файловую систему предполагает последовательное чтение большого объема данных из файлов, например, при создании резервной копии файловой системы. Объем данных может быть слишком большим, чтобы уместиться в кэше, или они могут читаться только один раз и едва ли сохранятся в кэше (в зависимости от политики управления кэшем). Такая рабочая нагрузка будет иметь относительно низкую производительность из-за низкого коэффициента попадания в кэш.

Предварительная выборка — это функция файловой системы, помогающая решить эту проблему. Файловая система может идентифицировать рабочие нагрузки с последовательным чтением по текущему и предыдущему смещениям в файлах, а затем сделать прогноз и выполнять операции чтения с диска до того, как приложение запросит их. При таком подходе кэш файловой системы постоянно пополняется,

и если приложение выполняет ожидаемое чтение, то происходит попадание в кэш (необходимые данные уже есть в кэше). Вот пример такого сценария:

1. Приложение вызывает `read(2)` для чтения файла, передавая управление ядру.
2. Искомых данных в кэше нет, поэтому файловая система выполняет чтение с диска.
3. Предыдущее смещение в файле сравнивается с текущим, и если они идут последовательно, то файловая система выполняет дополнительные операции чтения (предварительная выборка).
4. Первая операция чтения завершается, и ядро возвращает данные приложению.
5. Завершаются операции предварительной выборки, кэш заполняется данными для последующих операций чтения.
6. Последующие последовательные операции чтения, выполняемые приложением, получают данные из кэша в ОЗУ и завершаются намного быстрее.

Этот сценарий также показан на рис. 8.5, где приложение читает данные со смещения 1, а затем со смещения 2, после чего автоматически выполняется предварительная выборка для следующих трех смещений.

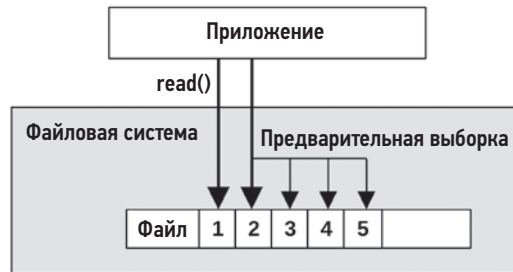


Рис. 8.5. Предварительная выборка, выполняемая файловой системой

Когда механизм прогнозирования предварительной выборки работает хорошо, приложения показывают гораздо более высокую производительность последовательного чтения — диски опережают запросы приложений (при условии, что у них для этого достаточная пропускная способность). Когда механизм прогнозирования предварительной выборки работает плохо, выполняются операции ввода/вывода, результаты которых не нужны приложению. Это загрязняет кэш и потребляет дисковые ресурсы и ресурсы для передачи ввода/вывода. Файловые системы обычно позволяют настраивать предварительную выборку по мере необходимости.

8.3.5. Упреждающее чтение

Предварительная выборка прежде была известна как *упреждающее чтение*. В Linux термин *упреждающее чтение* (`read-ahead`) используется для обозначения системного

вызова `readahead(2)`, который позволяет приложениям явно разогревать кэш файловой системы.

8.3.6. Кэширование с отложенной записью

Кэширование с отложенной записью (*write-back caching*) обычно используется файловыми системами для увеличения производительности операций записи. В этом случае операции записи считаются завершенными сразу после передачи данных в основную память, а фактическая запись их на диск происходит в некоторый момент позже, *асинхронно*. Процесс записи этих «грязных» данных на диск называется *очисткой* (*flushing*). Примерная последовательность действий выглядит следующим образом:

1. Приложение вызывает `write(2)` для записи данных в файл, передавая управление ядру.
2. Данные копируются из адресного пространства приложения в адресное пространство ядра.
3. Ядро завершает выполнение системного вызова `write(2)` и передает управление обратно приложению.
4. Через некоторое время асинхронная задача ядра обнаруживает данные для записи и выполняет запись на диск.

За производительность в этом случае приходится платить надежностью. Основная память ОЗУ энергозависима, и записываемые данные в случае сбоя питания могут быть потеряны. Данные могут быть записаны на диск *не полностью*, что приведет к *повреждению* данных на диске.

При повреждении метаданных файловой системы система может не загружаться. Восстановить систему после подобных повреждений можно только из резервных копий, а это сопряжено с длительным простоем. Хуже того, если окажется повреждено содержимое файла, который приложение читает и использует, то весь производственный процесс может оказаться под угрозой. Чтобы сбалансировать требования к скорости и надежности, файловые системы могут предлагать кэширование с отложенной записью по умолчанию и давать возможность выполнять запись синхронно, чтобы обеспечить более высокую надежность записи критических данных непосредственно на устройства хранения.

8.3.7. Синхронная запись

Синхронная запись завершается только тогда, когда данные полностью записаны в устройство хранения (например, на дисковое устройство), включая запись любых изменений в метаданных файловой системы. Синхронная запись выполняется намного медленнее асинхронной (кэширование с отложенной записью), потому что ждет завершения ввода/вывода с дисковым устройством (и, возможно, нескольких операций ввода/вывода для сохранения метаданных файловой системы). Синхронная запись часто используется, например, базами данных для записи в журналы, когда риск повреждения данных из-за асинхронной записи недопустим.

Есть две формы синхронной записи: индивидуальный ввод/вывод, который записывается синхронно, и групповой, когда происходит синхронная фиксация предыдущих операций записи.

Индивидуальная синхронная запись

Запись происходит синхронно, когда файл открывается с флагом `O_SYNC` или с одним из вариантов `O_DSYNC` и `O_RSYNC` (которые в Linux 2.6.31 отображаются библиотекой `glibc` в `O_SYNC`). Некоторые файловые системы поддерживают параметры монтирования, которые требуют, чтобы все операции записи во все файлы выполнялись синхронно.

Синхронная фиксация предыдущих операций записи

Вместо синхронного выполнения отдельных операций записи приложение может синхронно фиксировать предыдущие асинхронные операции записи в контрольных точках, используя системный вызов `fsync(2)`. Это повышает производительность за счет группировки операций записи, а также экономит время, необходимое для выполнения обновлений метаданных.

Есть и другие ситуации, когда происходит фиксация предыдущих операций записи, например, при закрытии дескрипторов файлов или когда для файла накопилось слишком много незаписанных буферов. Ситуации первого вида можно заметить по длительным паузам при распаковке архива с множеством файлов, особенно по NFS.

8.3.8. Прямой и низкоуровневый ввод/вывод

Это еще две разновидности ввода/вывода, которые может использовать приложение, если они поддерживаются ядром или файловой системой.

Низкоуровневый ввод/вывод выполняется непосредственно в смещения на диске, полностью в обход файловой системы. Такой ввод/вывод используется некоторыми приложениями, особенно базами данных, способными управлять своими данными и кэшировать их лучше, чем кэш файловой системы. Недостаток этого — более сложное ПО и трудности с администрированием: для резервного копирования/восстановления или наблюдения невозможно использовать обычные инструменты файловой системы.

Прямой ввод/вывод позволяет приложениям использовать файловую систему, но в обход ее кэша, например, используя флаг `O_DIRECT` при обращении к системному вызову `open(2)` в Linux. Этот вид ввода/вывода напоминает синхронную запись (но без гарантий, которые дает флаг `O_SYNC`), а кроме того, может использоваться для чтения. Это более сложный вид ввода/вывода, чем низкоуровневый ввод/вывод, потому что отображение смещений в файлах в дисковые смещения по-прежнему должно выполняться файловой системой. Также может потребоваться привести размер блоков ввода/вывода в соответствие с размерами блоков, которые используются файловой системой с учетом разметки диска (размер его записи), иначе это может привести к ошибке (`EINVAL`). В зависимости от типа файловой системы прямой

ввод/вывод может отключить не только кэширование чтения и буферизацию записи, но и предварительную выборку.

8.3.9. Неблокирующий ввод/вывод

Обычно операция ввода/вывода с файловой системой завершается либо немедленно (например, когда данные извлекаются из кэша), либо после некоторого ожидания (например, когда выполняется физический ввод/вывод). Если требуется ожидание, то поток приложения блокируется и оставляет процессор, позволяя поработать другим потокам в течение времени ожидания. Заблокированный поток не может выполнять другую работу, но обычно это не проблема — многопоточные приложения могут запускать дополнительные потоки для решения текущих задач, пока другие ожидают завершения ввода/вывода.

Иногда, чтобы избежать дополнительных затрат на создание новых потоков, желательно использовать неблокирующий ввод/вывод. Неблокирующую операцию ввода/вывода можно запустить, используя флаг `O_NONBLOCK` или `O_NDELAY` при обращении к системному вызову `open(2)`. Эти флаги заставляют операции чтения и записи возвращать ошибку `EAGAIN`, если их невозможно выполнить без блокировки потока, которая сообщает приложению, что оно должно повторить попытку позже. (Поддержка этого ввода/вывода зависит от файловой системы, которая может обеспечивать неблокирующий режим только для рекомендательных или обязательных блокировок файлов.)

ОС может также предоставлять отдельные функции асинхронного ввода/вывода, такие как `aio_read(3)` и `aio_write(3)`. В Linux 5.1 появился новый интерфейс асинхронного ввода/вывода, `io_uring`, более простой в использовании и с улучшенной эффективностью и производительностью [Ахбие 19].

Неблокирующий ввод/вывод также обсуждался в главе 5 «Приложения», в разделе 5.2.6 «Неблокирующий ввод/вывод».

8.3.10. Файлы, отображаемые в память

Производительность ввода/вывода файловой системы некоторых приложений и рабочих нагрузок можно улучшить за счет отображения файлов в адресное пространство процесса и прямого доступа к смещениям в памяти. Это позволяет избежать оверхеда на выполнение системных вызовов и переключение контекста, возникающего при обращении к системным вызовам `read(2)` и `write(2)` для доступа к данным файла. Этот прием позволяет избежать также двойного копирования данных, если ядро поддерживает прямое отображение буфера с данными из файла в адресное пространство процесса.

Отображения в память создаются системным вызовом `mmap(2)` и удаляются с помощью `munmap(2)`. Отображения можно настраивать с помощью `madvise(2)`, как описано в разделе 8.8 «Настройка». Некоторые приложения позволяют использовать системный вызов `mmap` (который можно назвать «режимом `mmap`»), предлагая

параметры конфигурации. Например, база данных Riak может использовать mmap для хранения данных в памяти.

Я заметил тенденцию использовать mmap(2) для решения проблем с производительностью файловой системы без предварительного их анализа. Если проблема в высокой задержке операций ввода/вывода с дисковыми устройствами, то исключение небольшого оверхеда на системные вызовы с помощью mmap(2) не даст желаемого результата.

Недостатком использования отображений в память в многопроцессорных системах может быть оверхед на поддержание синхронизации MMU каждого процессора, в частности перекрестные вызовы между CPU для удаления отображений (*TLB shootdowns*). В зависимости от ядра и отображения их можно свести к минимуму, откладывая обновление TLB (*lazy shootdowns*) [Vahalia 96].

8.3.11. Метаданные

Данные описывают содержимое файлов и каталогов, а *метаданные* описывают информацию о них. Под метаданными может пониматься информация, доступная через интерфейс файловой системы (POSIX), или информация, необходимая для реализации макета файловой системы на диске. Такие метаданные называются логическими и физическими метаданными соответственно.

Логические метаданные

Логические метаданные — это информация, которая читается из файловой системы и записывается в файловую систему потребителями (приложениями) двумя способами:

- **явно:** чтением статистик файлов (`stat(2)`), созданием и удалением файлов (`creat(2)`, `unlink(2)`) и каталогов (`mkdir(2)`, `rmdir(2)`), установкой свойств файлов (`chown(2)`, `chmod(2)`) — либо
- **неявно:** обновлением отметок времени доступа к файловой системе, обновлением отметок времени изменения каталога, обновлением битовых карт использованных блоков, изменением статистик свободного пространства.

Рабочие нагрузки, «перегруженные метаданными», обычно влияют на логические метаданные. К таким нагрузкам относятся, например, веб-серверы, которые вызывают `stat(2)`, чтобы убедиться, что они не изменились после кэширования быстрее, чем произошло чтение содержимого данных файла.

Физические метаданные

Физические метаданные — это метаданные разметки на диске, необходимые для записи информации обо всей файловой системе. Используемые типы метаданных зависят от файловой системы и могут включать суперблоки, индексные узлы, блоки указателей на данные (первичные, вторичные...) и списки свободных блоков.

Логические и физические метаданные — одна из причин различий между логическим и физическим вводом/выводом.

8.3.12. Логический и физический ввод/вывод

Может показаться нелогичным, но операции ввода/вывода, выполняемые приложением с файловой системой (логический ввод/вывод), по разным причинам могут не совпадать с дисковым вводом/выводом (физический ввод/вывод).

Файловые системы решают гораздо более широкий круг задач, чем просто представление файлового интерфейса для доступа к хранилищу (дискам). Они кэшируют операции чтения, буферизуют операции записи, отображают файлы в адресное пространство процесса и выполняют дополнительные операции ввода/вывода для поддержки физического размещения метаданных на диске, которые нужны для сохранения информации о местонахождении данных. Все это может вызывать несвязанный, косвенный, неявный, увеличенный или уменьшенный дисковый ввод/вывод по сравнению с вводом/выводом, выполняемым приложением. Ниже приводятся примеры таких видов ввода/вывода.

Несвязанный ввод/вывод

Дисковый ввод/вывод, который не связан с приложением и может быть обусловлен:

- **другими приложениями;**
- **другими пользователями:** дисковый ввод/вывод выполняется другим клиентом в облачной среде (некоторые технологии виртуализации позволяют отмечать такие события с помощью системных инструментов);
- **другими задачами ядра:** например, когда ядро перестраивает том программного массива RAID или выполняет асинхронную проверку контрольной суммы файловой системы (см. раздел 8.4 «Архитектура»);
- **задачами администрирования:** например, резервным копированием.

Косвенный ввод/вывод

Дисковый ввод/вывод, вызванный приложением, но не имеющий прямой связи с соответствующим вводом/выводом, выполняемым приложением. Это может быть обусловлено:

- **работой механизма предварительной выборки файловой системы:** дополнительные операции ввода/вывода, результаты которых могут использоваться или не использоваться приложением;
- **буферизацией в файловой системе:** использование кэширования с отложенной записью для отсрочки и объединения операций записи с последующей записью данных на диск. Некоторые системы могут буферизовать записываемые данные в течение десятков секунд и потом записывать их на диск все сразу, что выглядит как нечастые операции записи больших объемов данных.

Неявный ввод/вывод

Дисковый ввод/вывод, обусловленный событиями в приложении, за исключением явных операций чтения и записи, например:

- **загрузка/сохранение файлов, отображаемых в память:** при использовании файлов, отображаемых в память (`mmap(2)`), инструкции чтения и сохранения данных могут вызывать соответствующие операции дискового ввода/вывода. Записываемые данные могут помещаться в буфер и записываться на диск позже. Такое поведение может сбивать с толку при анализе операций с файловой системой (`read(2)`, `write(2)`) и затруднять поиск источника ввода/вывода (потому что он запускается инструкциями для работы с памятью, а не системными вызовами).

Уменьшенный (*deflated*) ввод/вывод

Дисковый ввод/вывод, объем которого меньше объема ввода/вывода в приложении или вообще отсутствует. Это может быть связано с:

- **кэшированием в файловой системе:** запросы на чтение удовлетворяются из основной памяти;
- **отменой записи в файловой системе:** данные с одним и тем же смещением в файле могли многократно меняться перед фактической записью на диск;
- **сжатием:** уменьшение объема данных между логическим и физическим вводом/выводом;
- **объединением:** объединение последовательных операций ввода/вывода перед записью на диск (это уменьшает количество операций ввода/вывода, но не общий объем данных);
- **размещением файловой системы в памяти:** содержимое такой файловой системы никогда не записывается на диск (например, `tmpfs`¹).

Увеличенный (*inflated*) ввод/вывод

Дисковый ввод/вывод, объем которого превышает объем ввода/вывода в приложении. Обычно увеличение ввода/вывода обусловлено:

- **поддержкой метаданных файловой системы:** для обслуживания метаданных выполняются дополнительные операции ввода/вывода;
- **размером записи в файловой системе:** из-за округления объема ввода/вывода до размера, кратного размеру записи (выводятся дополнительные байты) или фрагментации ввода/вывода (увеличение числа записей);
- **журналированием в файловой системе:** поддержка журналирования может удвоить количество операций записи на диск: одна операция осуществляет запись в журнал, а другая — в требуемое место назначения;

¹ Хотя `tmpfs` может вытесняться на устройства подкачки.

- **контролем четности в диспетчере томов:** циклы чтения-изменения-записи добавляют дополнительные операции ввода/вывода;
- **расширением RAID:** запись дополнительных данных с контролем четности или данных в зеркальные тома.

Пример

Чтобы показать, как эти факторы могут сочетаться друг с другом, ниже я описал, что может произойти при записи 1 байта данных приложением:

1. Приложение выполняет запись 1 байта данных в существующий файл.
2. Файловая система определяет его местоположение в записи размером 128 Кбайт, которой нет в кэше (но есть метаданные для ссылки на нее).
3. Файловая система запрашивает загрузку этой записи с диска.
4. Дисковое устройство разбивает чтение 128 Кбайт на более мелкие порции, подходящие для устройства.
5. Диски выполняют несколько операций чтения общим объемом 128 Кбайт.
6. Файловая система заменяет 1 байт в записи новым байтом.
7. Через некоторое время файловая система или ядро запрашивают сохранение «грязной» записи размером 128 Кбайт обратно на диск.
8. Запись размером 128 Кбайт записывается на диск (и при необходимости разбивается).
9. Файловая система записывает обновленные метаданные, например, чтобы обновить ссылки (для копирования при записи) или время последнего доступа.
10. Диски выполняют дополнительные операции записи.

В ответ на запись 1 байта приложением диски выполнили несколько операций чтения (прочитав 128 Кбайт) и еще больше операций записи (записав больше 128 Кбайт).

8.3.13. Неравноценность операций

В предыдущих разделах мы увидели, что операции файловой системы могут показывать разную производительность в зависимости от их типа. Нельзя точно охарактеризовать рабочую нагрузку только по показателю скорости «500 операций в секунду». Некоторые операции могут возвращать данные из кэша файловой системы и выполняться со скоростью доступа к основной памяти. Другие могут возвращать данные с диска и выполняться на несколько порядков медленнее. В числе других определяющих факторов можно назвать последовательный или произвольный характер операций, чтение или запись, синхронность или асинхронность записи, объем ввода/вывода, выполнение операций других типов, стоимость их выполнения процессором (и степень нагрузки на процессор в системе), а также характеристики устройства хранения.

Для определения этих характеристик производительности обычно используется микробенчмаркинг различных операций файловой системы. В качестве примера в табл. 8.2 приводятся результаты тестирования файловой системы ZFS на простаивающем сервере с многоядерным процессором Intel Xeon 2,4 ГГц.

Таблица 8.2. Примеры задержек операций файловой системы

Операция	Средняя задержка (мкс)
open(2) (из кэша ¹)	2,2
close(2) (чистые данные ²)	0,7
read(2) 4 Кбайт (из кэша)	3,3
read(2) 128 Кбайт (из кэша)	13,9
write(2) 4 Кбайт (асинхронно)	9,3
write(2) 128 Кбайт (асинхронно)	55,2

Тесты не затрагивали устройства хранения и отражают быстродействие ПО файловой системы и процессора. Некоторые специальные файловые системы никогда не обращаются к устройствам хранения.

Кроме того, тесты выполнялись в одном потоке. На производительность параллельного ввода/вывода могут также влиять тип и организация блокировок, используемых файловой системой.

8.3.14. Специальные файловые системы

Цель файловой системы — постоянное хранение данных, но есть специальные типы файловых систем, которые используются в Linux для других целей, включая временные файлы (/tmp), пути к устройствам ядра (/dev), предоставление доступа к системным статистикам (/proc) и конфигурации системы (/sys)³.

8.3.15. Доступ к отметкам времени

Многие файловые системы поддерживают отметки времени, которые фиксируют время последнего доступа (чтения) к каждому файлу и каталогу. Это требует обновления метаданных при каждом чтении файлов, что вызывает выполнение дополнительных операций записи, потребляющих ресурсы дискового ввода/вывода. В разделе 8.8 «Настройка» показано, как отключить эти обновления.

¹ Из кэша индексных узлов файлов.

² Без наличия измененных данных, которые нужно вытолкнуть на диск.

³ Список специальных файловых систем в Linux, не использующих устройства хранения, выводится командой: `grep '^nodev' /proc/filesystems`.

Некоторые файловые системы оптимизируют запись отметок времени последнего доступа, откладывая и группируя их, чтобы уменьшить влияние на активную рабочую нагрузку.

8.3.16. Емкость

По мере заполнения файловых систем их производительность может снижаться по нескольким причинам. Во-первых, при записи новых данных может потребоваться больше процессорного времени и дискового ввода/вывода для поиска свободных блоков на диске¹. Во-вторых, свободные области на диске, вероятно, будут меньше и разбросаны по всему диску, что отрицательно скажется на производительности из-за уменьшения объемов каждой операции ввода/вывода или необходимости выполнять произвольный ввод/вывод.

Острота этой проблемы зависит от типа файловой системы, ее структуры на диске, использования приема копирования при записи и особенностей устройств хранения. В следующем разделе описаны различные типы файловых систем.

8.4. АРХИТЕКТУРА

В этом разделе представлена общая архитектура файловой системы, включая стек ввода/вывода, VFS, механизм кэширования в файловой системе и его особенности, общие типы файловых систем, томов и пулов. Эта информация полезна при определении компонентов файловой системы для анализа и настройки. Более подробные сведения о внутреннем устройстве файловой системы и ответы на другие вопросы можно найти в исходном коде, а также в других источниках. Некоторые из них перечислены в конце этой главы.

8.4.1. Стек ввода-вывода файловой системы

На рис. 8.6 изображена общая модель стека ввода/вывода файловой системы с акцентом на интерфейс файловой системы. Конкретные компоненты, уровни и API зависят от типа и версии ОС и используемой файловой системы. Более обобщенное описание стека ввода/вывода приводится в главе 3 «Операционные системы», и еще одно, более подробно показывающее компоненты диска, — в главе 9 «Диски».

Здесь показан путь ввода/вывода от приложений и системных библиотек к системным вызовам через ядро. Путь от системных вызовов непосредственно к подсистеме дисковых устройств — это *низкоуровневый ввод/вывод*. Путь через VFS и файловую систему — это ввод/вывод через файловую систему, включая прямой ввод/вывод, который происходит в обход кэша файловой системы.

¹ ZFS, например, переключается на другой, более медленный алгоритм поиска свободных блоков, когда объем занятого дискового пространства превышает пороговое значение (первоначально 80 %, позднее 99 %). См. статью «Pool performance can degrade when a pool is very full» [Oracle 12].

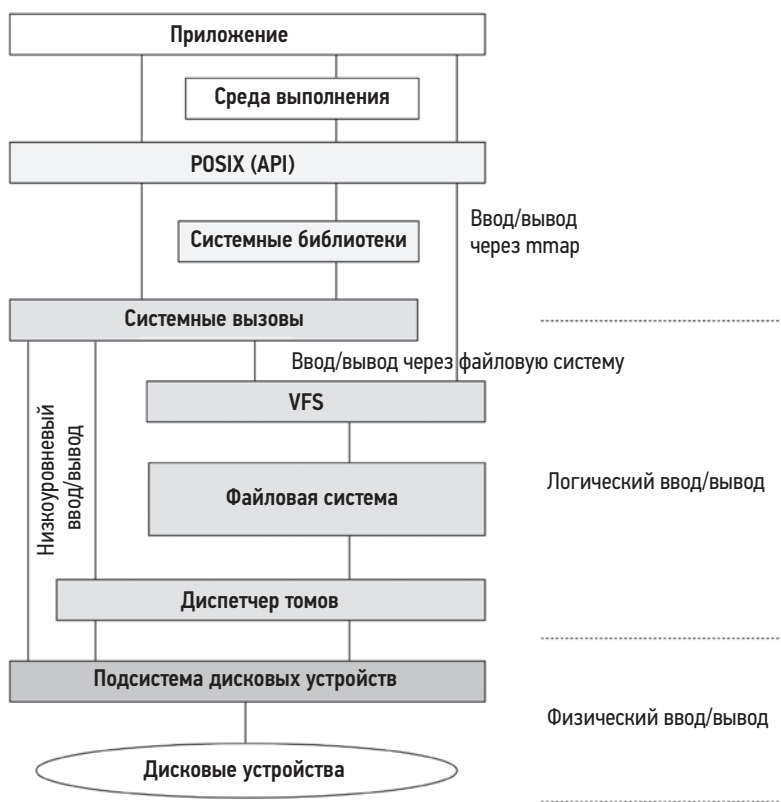


Рис. 8.6. Обобщенная модель стека ввода/вывода файловой системы

8.4.2. VFS

Виртуальная файловая система (virtual file system, VFS) предоставляет универсальный интерфейс для доступа к файловым системам разных типов. Ее местоположение показано на рис. 8.7.

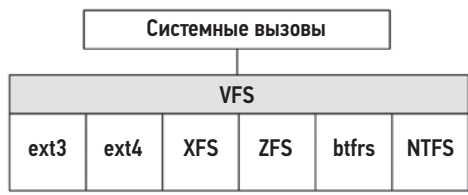


Рис. 8.7. Виртуальная файловая система

Первоначально VFS была создана в SunOS и стала затем стандартной абстракцией для файловых систем.

Терминология, используемая в VFS в Linux, может запутать, потому что в ней для обозначения объектов VFS повторно используются термины *индексные узлы* (inodes) и *суперблоки* (superblocks), заимствованные из дисковых структур данных файловой системы Unix. Термины, использующиеся для описания дисковых структур данных в Linux, обычно начинаются с префикса, описывающего тип файловой системы, например `ext4_inode` и `ext4_super_block`. Индексные узлы и суперблоки VFS находятся исключительно в памяти.

VFS может также служить общей точкой для измерения производительности любой файловой системы. Для этого можно использовать статистики, предоставляемые ОС, либо статическую или динамическую инструментацию.

8.4.3. Кэши файловой системы

Первоначально в Unix был только кэш буферов, увеличивающий скорость доступа к блочным устройствам. Сейчас Linux поддерживает несколько типов кэшей. На рис. 8.8 показана структура кэшей файловой системы в Linux — кэши, доступные для файловых систем стандартных типов.

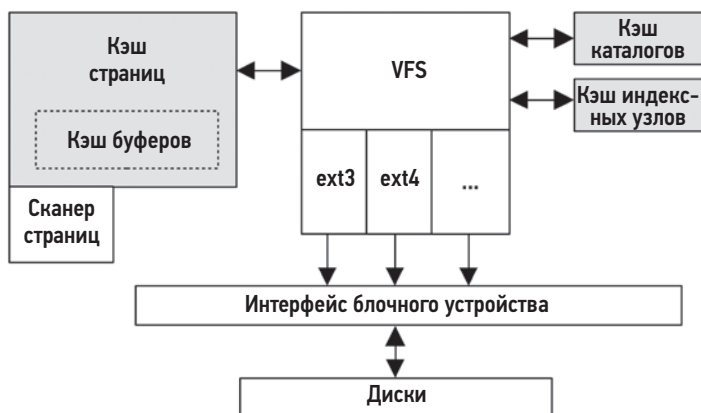


Рис. 8.8. Кэши файловой системы в Linux

Кэш буферов

В Unix кэш буферов использовался на уровне интерфейса блочного устройства для кэширования дисковых блоков. Это был отдельный кэш фиксированного размера. Позднее, с добавлением кэша страниц, стали возникать проблемы с настройкой балансировки нагрузки между ними, а также увеличился оверхед, обусловленный двойным кэшированием и необходимостью синхронизации. Эти проблемы устранили за счет использования кэша страниц для хранения кэша буферов. Это решение, реализованное в SunOS, получило название *унифицированного кэша буферов* (unified buffer cache).

В Linux первое время тоже использовался кэш буферов, как в Unix. Но начиная с Linux 2.4, кэш буферов переместился в кэш страниц (именно поэтому на рис. 8.8 кэш буферов заключен в пунктирную рамку). Это позволило избежать оверхеда на двойное кэширование и синхронизацию. Кэш буферов есть до сих пор, он улучшает производительность ввода/вывода с блочными устройствами. Этот термин все еще используется в инструментах наблюдения Linux (например, в `free(1)`).

Размер кэша буферов изменяется динамически и его можно узнать в `/proc`.

Кэш страниц

Впервые кэш страниц появился в SunOS, после того как в 1985 году подсистема виртуальной памяти была переписана и добавлена в SVR4 Unix [Vahalia 96]. В ту пору он кэшировал страницы виртуальной памяти, в том числе страницы, отображаемые из файловой системы, что улучшало производительность ввода/вывода с файлами и каталогами. Он показывал большую эффективность при работе с файлами, чем кэш буферов, который требовал преобразования смещений в файлах в смещения на диске для каждой операции поиска. Кэш страниц может использоваться самыми разными файловыми системами, включая UFS и NFS. Размер кэша изменяется динамически: он растет, пока не будет занята вся доступная память, и затем освобождает ее, когда память потребуется приложениям.

Кэш страниц в Linux характеризуется теми же особенностями. Размер кэша страниц в Linux также изменяется динамически, также есть возможность настройки для установки баланса между освобождением страниц из кэша и анонимной подкачкой (`vm.swappiness`; см. главу 7 «Память»).

Грязные (измененные) страницы памяти, которые нужны для использования файловой системой, сбрасываются на диск потоками ядра. До Linux 2.6.32 был пул потоков очистки грязных страниц (*pdflush*), число которых изменялось от двух до восьми по мере необходимости. Но позднее его заменили *потоки очистки* (*flusher threads*), называемые *flush*, которые создаются для каждого устройства, чтобы лучше сбалансировать рабочую нагрузку на устройство и повысить пропускную способность. Страницы сбрасываются на диск по следующим событиям:

- по истечении определенного интервала (30 с);
- при обращении к системным вызовам `sync(2)`, `fsync(2)`, `msync(2)`;
- когда число грязных страниц превышает определенный порог (параметры `dirty_ratio` и `dirty_bytes`);
- когда в кэше страниц не оказывается свободных страниц.

При нехватке системной памяти другой поток ядра, демон подкачки страниц (*kswapd*, также известный как *сканер страниц*), может отыскать и запланировать запись на диск грязных страниц, чтобы освободить память для повторного использования (см. главу 7 «Память»). В инструментах наблюдения потоки `kswapd` и `flush` отображаются как задачи ядра. Подробнее о сканере страниц шла речь в главе 7 «Память».

Кэш каталогов

Кэш каталогов (dentry cache, Dcache) запоминает соответствие между записью с данными о каталоге (struct dentry) и индексным узлом VFS, как и более ранний *кэш поиска имен каталогов* (Directory Name Lookup Cache, DNLC) в Unix. Кэш каталогов повышает производительность поиска имен каталогов (например, в `open(2)`): при обходе имен каталогов в пути к файлу поиск каждого имени может проверять Dcache на наличие прямого соответствия с индексным узлом вместо пошагового просмотра содержимого каталога. Записи Dcache хранятся в хеш-таблице, что обеспечивает высокую скорость и масштабируемость поиска (хеширование производится по имени родительского и текущего каталога).

За годы производительность была увеличена еще больше, в том числе за счет применения алгоритма *чтения-копирования-обновления-обхода* (read-copy-update-walk, RCU-walk) [Corbet 10]. Алгоритм выполняет обход имен в пути без обновления счетчиков ссылок в записях. Это вызывало проблемы с масштабируемостью из-за необходимости согласования кэшей процессоров в многопроцессорных системах, где очень часто выполнялась операция поиска имен каталогов. Если обнаруживается запись, которой нет в кэше, алгоритм RCU-walk переключается на более медленный алгоритм обхода с подсчетом ссылок (reference-count walk, ref-walk), так как подсчет ссылок потребует во время поиска и блокировки файловой системы. Для больших рабочих нагрузок ожидается, что данные каталогов с большой вероятностью будут помещены в кэш и алгоритм RCU-walk выполнится успешно.

Dcache выполняет *отрицательное кэширование*, запоминая результаты поиска несуществующих записей. Это повышает производительность неудачных поисков, которые обычно возникают при поиске разделяемых библиотек.

Dcache динамически увеличивается в размерах и сокращается за счет удаления наиболее давно использовавшихся записей (least recently used, LRU), когда системе требуется больше памяти. Его размер можно узнать через `/proc`.

Кэш индексных узлов

Этот кэш хранит индексные узлы (inodes) виртуальной файловой системы (struct inodes), описывающие свойства объектов файловой системы. Многие из этих свойств возвращаются системным вызовом `stat(2)`. К этим свойствам часто обращаются многие рабочие нагрузки, например, для проверки разрешений при открытии файлов или обновлении времени последнего изменения. Индексные узлы VFS хранятся в хеш-таблице, что обеспечивает высокую скорость и масштабируемость поиска (хеширование выполняется по номеру индексного узла и суперблоку файловой системы), но большая часть операций поиска использует кэш каталогов.

Кэш индексных узлов динамически увеличивается в размерах и хранит как минимум все индексные узлы, на которые сылается Dcache. При нехватке памяти кэш сокращается за счет удаления индексных узлов не связанных с кэшем каталогов. Его размер можно узнать из файлов `/proc/sys/fs/inode*`.

8.4.4. Особенности файловых систем

Ниже описаны дополнительные особенности файловой системы, влияющие на производительность.

Блоки и экстенты

Блочные файловые системы хранят данные в блоках фиксированного размера, на которые ссылаются указатели, хранящиеся в блоках метаданных. Для больших файлов может потребоваться много указателей на блоки с данными и много блоков метаданных, а сами блоки могут оказаться разбросанными по всему диску, что влечет за собой произвольный ввод/вывод. Некоторые блочные файловые системы стараются размещать блоки непрерывно и последовательно, чтобы избежать этого. Другое решение — использование *блоков переменного размера*, чтобы по мере роста файла можно было использовать блоки больших размеров. Это снижает overhead на обработку метаданных.

Файловые системы на основе экстентов заранее выделяют непрерывное пространство для файлов (экстенты), увеличивая его по мере необходимости. Экстенты имеют переменную длину и формируются из одного или нескольких смежных блоков.

Это улучшает производительность потоковой передачи и может даже улучшить производительность произвольного ввода/вывода за счет локализации файловых данных. Также улучшается производительность обработки метаданных — требуется следить за меньшим количеством объектов, не жертвуя пространством неиспользуемых блоков в экстенсте.

Журналирование

Журнал (или *лог*) файловой системы хранит изменения в файловой системе, так что в случае сбоя системы или отключения питания изменения можно атомарно воспроизвести (в случае успеха) или отменить (в случае неудачи). Это позволяет файловым системам быстро восстанавливать согласованное состояние. Нежурналируемые файловые системы могут быть повреждены во время сбоя системы, если сопутствующие изменению данные и метаданные были записаны не полностью. Восстановление после такого сбоя требует обхода всех структур файловой системы, что может занять часы для больших файловых систем.

Журнал записывается на диск синхронно, а некоторые файловые системы позволяют настроить хранение журнала на отдельном устройстве. Некоторые журналы хранят и данные, и метаданные, что увеличивает потребление ресурсов, потому что все операции ввода/вывода выполняются дважды. Другие журналы хранят только метаданные и обеспечивают целостность данных за счет копирования при записи.

Есть особый тип файловой системы, состоящей только из журнала: *файловая система с журнальной структурой* (log-structured file system), в которой все обновления данных и метаданных записываются в непрерывный и циклический журнал.

Такое решение оптимизирует производительность операций записи, потому что эти операции всегда выполняются последовательно и их можно объединить, чтобы записать соответствующие им данные за один прием.

Копирование при записи

Файловая система, использующая копирование при записи (copy-on-write, COW), не перезаписывает существующие блоки. Она выполняет следующие шаги:

1. Записывает блоки в новое место (создает новую копию).
2. Обновляет ссылки на новые блоки.
3. Добавляет старые блоки в список свободных блоков.

Это помогает поддерживать целостность файловой системы в случае сбоя, а также повышает производительность за счет преобразования произвольных операций записи в последовательные.

Когда заполнение файловой системы приближается к верхнему пределу, использование алгоритма COW вызывает фрагментацию размещения данных на диске и снижение производительности (это особенно характерно для жестких дисков). Восстановить производительность помогает дефрагментация файловой системы, если она доступна.

Скраббинг

Скраббинг (scrubbing) — это особенность файловой системы, заключающаяся в асинхронном чтении всех блоков данных и проверке контрольных сумм для раннего выявления неисправностей дисковых устройств, в идеале — до того момента, когда проблему уже нельзя будет исправить с помощью RAID. Но такой контроль при выполнении операций чтения снижает производительность, поэтому его следует запускать с низким приоритетом или в периоды пониженной рабочей нагрузки.

Другие особенности

К другим особенностям, влияющим на производительность файловой системы, относится создание моментальных снимков, сжатие, избыточность, дедупликация (поиск и удаление повторяющихся данных), поддержка обрезки и многое другое. Некоторые из этих особенностей описываются в следующем разделе применительно к конкретным файловым системам.

8.4.5. Типы файловых систем

Большая часть этой главы описывает общие *характеристики*, свойственные всем типам файловых систем. В следующих разделах я расскажу о конкретных особенностях некоторых распространенных файловых систем, оказывающих влияние на производительность. Их анализ и настройка рассматриваются в последующих разделах.

FFS

Многие файловые системы прямо или опосредованно основаны на *быстрой файловой системе* Беркли (fast file system, FFS), которая была разработана для решения проблем в оригинальной файловой системе Unix¹. Немного предыстории поможет объяснить современное состояние файловых систем.

Организация оригинальной файловой системы Unix на диске состояла из таблицы индексных узлов, 512-байтных блоков данных и суперблока с информацией, которая используется при распределении ресурсов [Ritchie 74], [Lions 77]. Таблица индексных узлов и блоки данных разделяли диска на два диапазона, что вызывало проблемы с производительностью из-за необходимости перемещения магнитных головок между ними. Другой проблемой был маленький фиксированный размер блока — 512 байт, что ограничивало пропускную способность и увеличивало количество метаданных (указателей), необходимых для хранения больших файлов. В [McKusick 84] описан эксперимент по удвоению размера блоков до 1024 байт и возникшее узкое место:

Пропускная способность увеличилась вдвое, но старая файловая система по-прежнему использовала не более четырех процентов пропускной способности диска. Основная проблема была в списке свободных блоков: изначально этот список был упорядочен и обеспечивал оптимальную скорость доступа, но быстро фрагментировался по мере создания и удаления файлов. В итоге порядок следования свободных блоков в списке был полностью нарушен, в результате чего блоки на диске распределялись случайным образом. По этой причине дисковому устройству приходилось позиционировать магнитные головки перед каждым доступом к блоку. Сразу после создания старые файловые системы обеспечивали скорость передачи до 175 Кбайт в секунду, но за несколько недель умеренного использования скорость упала до 30 Кбайт в секунду из-за такой рандомизации размещения блоков данных.

В этом отрывке описывается *фрагментация* списка свободных блоков, которая с течением временем снижает производительность файловой системы.

Производительность FFS была увеличена за счет деления раздела на многочисленные группы цилиндров, как показано на рис. 8.9, каждая со своим массивом индексных узлов и блоками данных. Индексные узлы и данные файлов по возможности хранятся в одной группе цилиндров, что уменьшает время на позиционирование головок диска. Другие связанные данные тоже размещаются поблизости, в том числе индексные узлы каталогов и их записи. Структура индексного узла подобна иерархии указателей и блоков данных, как показано на рис. 8.10 (трехуровневые блоки с косвенными указателями здесь не показаны) [Bach 86].

¹ Не путайте оригинальную файловую систему Unix с более поздними файловыми системами UFS, которые основаны на FFS.

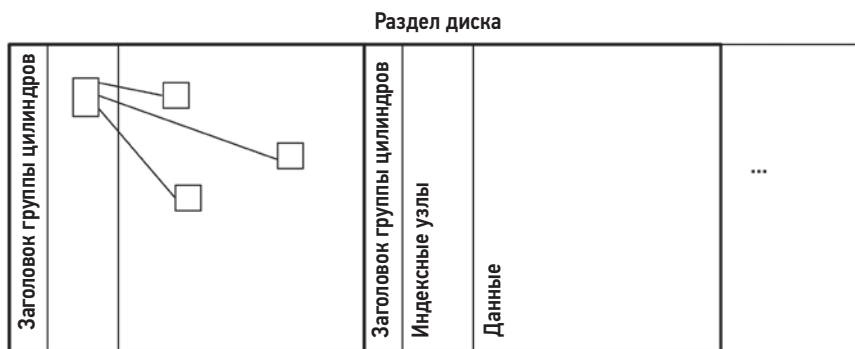


Рис. 8.9. Группы цилиндров

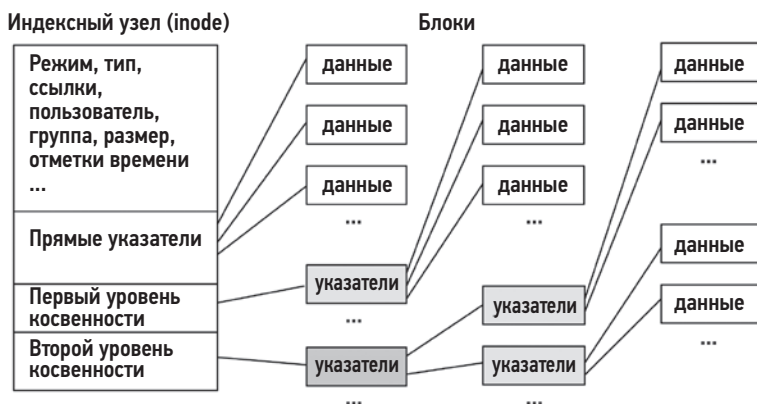


Рис. 8.10. Структура индексного узла

Минимальный размер блока был увеличен до 4 Кбайт, что повысило пропускную способность. Благодаря этому уменьшилось количество блоков данных, необходимых для хранения файла, и количество косвенных блоков, необходимых для ссылки на блоки данных. Также уменьшилось количество требуемых блоков с косвенными указателями, потому что их размеры также увеличились. Для экономии места при хранении небольших файлов каждый блок можно разбить на фрагменты размером 1 Кбайт.

Другая особенность FFS, влияющая на производительность, — *чередование блоков*: последовательные блоки файлов размещаются на диске с интервалом в один или несколько блоков между ними [Доеррнер 10]. Эти дополнительные блоки давали ядру и процессору время для запуска следующей операции последовательного чтения файла. Без чередования следующий блок (на вращающемся диске) может проскочить под головкой до того, как ядро успеет подготовиться к запуску операции чтения, что приведет к задержке в ожидании, пока диск не сделает полный оборот.

ext3

Расширенная файловая система Linux (extended file system, ext) была разработана в 1992 году как первая файловая система для Linux и VFS, основанная на оригинальной файловой системе Unix. Вторая версия, ext2 (1993), добавила поддержку нескольких отметок времени и групп цилиндров из FFS. Третья версия, ext3 (1999), добавила поддержку роста файловой системы и журналирование.

Вот основные характеристики, влияющие на производительность, в том числе добавленные с момента релиза:

- **Журналирование:** либо *упорядоченный режим*, только для метаданных, либо *режим журнала* для метаданных и данных. Журналирование увеличивает скорость загрузки после сбоя системы и избавляет от необходимости запускать fsck. Также журналирование может улучшить производительность некоторых рабочих нагрузок записи за счет объединения операций записи метаданных.
- **Поддержка отдельного устройства для журнала:** позволяет использовать внешнее устройство для хранения журнала, чтобы операции с журналом не конкурировали с рабочей нагрузкой чтения.
- **Распределитель блоков Орлова:** распределяет каталоги верхнего уровня по группам цилиндров, благодаря чему увеличивается вероятность компактного размещения подкаталогов с их содержимым и уменьшается вероятность произвольного ввода/вывода.
- **Индексы каталогов:** добавляют в файловую систему хешированные B-деревья для более быстрого поиска в каталогах.

Настраиваемые особенности описаны на странице справочного руководства man для mke2fs(8).

ext4

В 2008 году была выпущена файловая система ext4 для Linux, добавившая в ext3 новые возможности и улучшившая производительность: экстененты, увеличенная емкость, предварительное выделение пространства с помощью `fallocate(2)`, отложенное выделение, поддержка контрольных сумм в журнале, более быстрая утилита fsck, многоблочный распределитель, отметки времени с точностью до наносекунд и моментальные снимки.

Вот основные характеристики, влияющие на производительность, в том числе добавленные с момента релиза:

- **Экстененты:** экстененты улучшают непрерывное размещение, уменьшают количество операций произвольного ввода/вывода и увеличивают объем операций последовательного ввода/вывода. Подробнее об экстенентах см. в разделе 8.4.4 «Особенности файловых систем».
- **Предварительное выделение пространства:** с помощью системного вызова `fallocate(2)` позволяет приложениям предварительно выделять пространство,

которое с большой долей вероятности будет непрерывным, что повышает производительность последующих операций записи.

- **Отложенное распределение:** выделение пространства для блоков откладывается до тех пор, пока они не будут сброшены на диск. Это позволяет записывать блоки группами (через многоблочный распределитель) и уменьшать фрагментацию.
- **Быстрая утилита fsck:** нераспределенные блоки и индексные узлы маркируются соответствующим образом, что сокращает время работы fsck.

Состояние некоторых особенностей можно получить из файловой системы /sys. Например:

```
# cd /sys/fs/ext4/features
# grep . *
batched_discard:supported
casefold:supported
encryption:supported
lazy_itable_init:supported
meta_bg_resize:supported
metadata_csum_seed:supported
```

Настраиваемые особенности описаны на странице справочного руководства man для mke2fs(8). В файловой системе ext3 может быть реализована в том числе и поддержка экстенгов.

XFS

XFS была создана компанией Silicon Graphics в 1993 году для их ОС IRIX, чтобы устранить ограничения масштабируемости, которые были в EFS — предыдущей файловой системе IRIX (основанной на FFS) [Sweeney 96]. Поддержка XFS была добавлена в ядро Linux в начале 2000-х. Сейчас XFS поддерживается большинством дистрибутивов Linux и может использоваться в качестве корневой файловой системы. В Netflix, например, XFS используется для экземпляров базы данных Cassandra, так как отличается высокой производительностью для этой рабочей нагрузки (в качестве корневой файловой системы используется ext4).

Вот основные характеристики, влияющие на производительность, в том числе добавленные с момента релиза:

- **Группы распределения:** раздел делится на группы распределения (Allocation Groups, AG) равного размера, к которым можно обращаться параллельно. Для ограничения конкуренции метаданные — индексные узлы и списки свободных блоков в каждой группе — управляются независимо, в то время как файлы и каталоги охватывают несколько групп.
- **Экстенги:** (см. описание в разделе «ext4»).
- **Журналирование:** журналирование увеличивает скорость загрузки после сбоя системы и избавляет от необходимости запускать fsck(8). Также журналирование

улучшает производительность некоторых рабочих нагрузок записи за счет объединения операций записи метаданных.

- **Поддержка отдельного устройства для журнала:** позволяет использовать внешнее устройство для хранения журнала, чтобы операции с журналом не конкурировали с операциями доступа к данным.
- **Чередующееся распределение:** если файловая система создается на дисковом массиве RAID с чередованием или LVM, можно настроить шаг чередования для данных и журнала, чтобы оптимизировать распределение данных с учетом особенностей оборудования.
- **Отложенное выделение:** выделение экстендов откладывается до тех пор, пока данные не будут сброшены на диск, что позволяет объединять операции записи и уменьшить фрагментацию. Блоки для файлов в памяти резервируются заранее, чтобы к моменту сброса данных для них было свободное пространство.
- **Оперативная дефрагментация:** XFS предоставляет утилиту дефрагментации, которая может работать с файловой системой при ее активном использовании. Несмотря на то что XFS использует экстенды и поддерживает отложенное выделение для предотвращения фрагментации, некоторые рабочие нагрузки при определенных условиях фрагментируют файловую систему.

Настраиваемые особенности описаны на странице справочного руководства `man` для `mkfs.xfs(8)`. Внутренние данные о производительности XFS можно получить из `/proc/fs/xfs/stat`. Они предназначены для углубленного анализа: дополнительную информацию см. на сайте XFS [XFS 10].

ZFS

ZFS была разработана в Sun Microsystems и выпущена в 2005 году. Она объединяет файловую систему с диспетчером томов и поддерживает многочисленные функции, востребованные на крупных предприятиях, что делает ее привлекательным выбором для организации файловых серверов. ZFS распространяется с открытым исходным кодом и используется несколькими ОС, но обычно в виде надстройки, так как ZFS использует лицензию CDDL. Основная разработка ведется в рамках проекта OpenZFS, который в 2019 году объявил о поддержке Linux в качестве основной ОС [Ahrens 19]. Несмотря на рост поддержки этой файловой системы в Linux, из-за лицензии на исходный код есть некоторое сопротивление, в том числе со стороны Линуса Торвальдса [Torvalds 20a].

Вот основные характеристики, влияющие на производительность ZFS, в том числе добавленные с момента релиза:

- **Объединенное хранилище (пул):** все выбранные устройства хранения помещаются в пул, в котором создаются файловые системы. Это позволяет использовать все устройства параллельно для максимальной пропускной способности и производительности. Поддерживается возможность использовать различные

типы RAID: 0, 1, 10, Z (на основе RAID-5), Z2 (двойная четность) и Z3 (тройная четность).

- **COW**: измененные блоки копируются, затем группируются и последовательно записываются.
- **Журналирование**: ZFS сбрасывается на диск *группы транзакций* (transaction groups, TXG) с изменениями в виде пакетов, которые в случае успеха сохраняются все вместе, или, в случае неудачи, не сохраняется ни одна, поэтому данные на диске всегда остаются в согласованном состоянии.
- **Адаптивный кэш замены** (Adaptive Replacement Cache, ARC): обеспечивает высокую частоту попаданий в кэш за счет одновременного использования нескольких алгоритмов кэширования: последнего использовавшегося (most recently used, MRU) и наиболее часто используемого (most frequently used, MFU). Распределение основной памяти между ними балансируется с учетом их производительности, которая известна благодаря поддержке дополнительных метаданных (*списков призраков*), которые позволяют увидеть, как будет действовать каждый из них при управлении всей основной памятью.
- **Интеллектуальная предварительная выборка**: при необходимости ZFS применяет разные виды предварительной выборки: для метаданных, для z-узлов (znodes; с содержимым файлов) и виртуальных устройств vdevs.
- **Несколько потоков предварительной выборки**: поддержка чтения данных из одного файла в несколько потоков может создавать произвольную рабочую нагрузку ввода/вывода из-за необходимости переключений между ними. ZFS определяет отдельные потоки предварительной выборки и позволяет присоединять к ним новые потоки.
- **Моментальные снимки**: благодаря архитектуре COW, снимки можно создавать почти мгновенно, откладывая копирование новых блоков до момента, когда они понадобятся.
- **Конвейер ZIO**: операции ввода/вывода с устройством обрабатываются конвейером этапов, каждый этап обслуживается пулом потоков выполнения для повышения производительности.
- **Сжатие**: поддерживается несколько алгоритмов, которые обычно снижают производительность из-за нагрузки на процессор. Алгоритм lzjb (Lempel-Ziv Jeff Bonwick) — легковесный и может немного улучшить производительность хранилища за счет уменьшения объема ввода/вывода (из-за сжатия) ценой некоторой нагрузки на процессор.
- **SLOG**: отдельный журнал намерений в ZFS позволяет синхронно выполнять операции записи на отдельные устройства и избегать конкуренции с операциями с дисковым пулом. Записи в SLOG читаются только в случае сбоя системы, когда требуется их воспроизведение.
- **L2ARC**: адаптивный кэш замены второго уровня (Level 2 ARC) — это второй уровень кэша после основной памяти. Он предназначен для кэширования операций

произвольного чтения на твердотельных накопителях (SSD). Он не буферизует операции записи и хранит только чистые данные, которые уже находятся на дисках. Также может дублировать данные в ARC, чтобы система могла быстрее восстановиться в случае каких-либо нарушений в процессе выталкивания данных из кэша в основной памяти.

- **Дедупликация данных:** эта особенность файловой системы позволяет избежать записи нескольких копий одних и тех же данных. Может оказывать значительное влияние на производительность, как положительное (сокращение объема ввода/вывода), так и отрицательное (когда хеш-таблица не уместается в основной памяти и выполняется избыточное количество операций ввода/вывода с устройством). Первоначальная версия предназначалась только для рабочих нагрузок, когда ожидается, что хеш-таблица всегда будет уместаться в основной памяти.

Некоторые особенности поведения ZFS могут снизить ее производительность по сравнению с другими файловыми системами: по умолчанию ZFS выдает команды выталкивания кэша на устройства хранения, чтобы гарантировать завершение записи на случай отключения питания. Это одна из особенностей поддержки целостности ZF, и за это приходится платить задержкой операций, вынужденных ждать выталкивания кэша.

btrfs

Файловая система *btrfs* основана на структурах B-деревьев и работает по принципу «копирование при записи» (copy-on-write). Это современная архитектура, совмещающая и файловую систему, и диспетчер томов, подобно ZFS; как ожидается, в итоге она будет обладать аналогичным набором возможностей. Сейчас она поддерживает такие особенности, как объединенное хранилище (пул), увеличенная емкость, экстенды, COW, динамическое изменение размеров томов, вложенные тома, добавление и удаление блочных устройств, моментальные снимки, клонирование, сжатие и различные контрольные суммы (включая crc32c, xxhash64, sha256 и blake2b). Разработка начата компанией Oracle в 2007 году.

Вот основные характеристики, влияющие на производительность:

- **Объединенное хранилище (пул):** все выбранные устройства хранения объединяются в том, в котором создаются файловые системы. Это позволяет использовать все устройства параллельно для максимальной пропускной способности и производительности. Поддерживается возможность использовать типы RAID: 0, 1 и 10.
- **COW:** измененные блоки копируются, затем группируются и записываются последовательно.
- **Оперативная балансировка:** объекты перемещаются между устройствами хранения для балансировки нагрузки на них.

- **Экстененты:** обеспечивают последовательное расположение блоков и улучшают производительность.
- **Моментальные снимки:** благодаря архитектуре COW, снимки можно создавать почти мгновенно, откладывая копирование новых блоков до момента, когда они понадобятся.
- **Сжатие:** поддерживаются алгоритмы zlib и LZO.
- **Журналирование:** поддерживается возможность создания дерева журнала для каждого субтома, чтобы улучшить производительность журналирования для синхронных рабочих нагрузок COW.

В будущем планируется добавить поддержку RAID-5 и 6, RAID на уровне объектов, инкрементные дампы и дедупликацию данных.

8.4.6. Тома и пулы

Традиционно файловые системы располагались на одном диске или в его разделе. Тома и пулы позволяют создавать файловые системы на нескольких дисках и настраиваются с помощью различных стратегий RAID (см. главу 9 «Диски»).

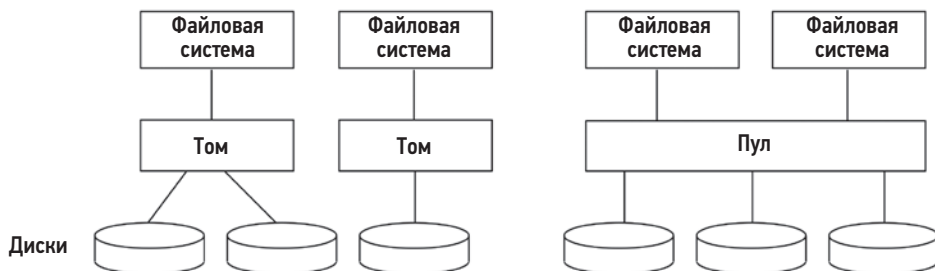


Рис. 8.11. Тома и пулы

Тома представляют несколько дисков как один виртуальный диск, на котором создается файловая система. Охватывая целые диски (а не срезы или разделы), тома изолируют рабочие нагрузки, уменьшая проблемы производительности, связанные с конкуренцией.

ПО управления томами для систем на базе Linux включает диспетчер логических томов (Logical Volume Manager, LVM). Тома, или виртуальные диски, также могут организовываться аппаратными контроллерами RAID.

Объединенное хранилище собирает несколько дисков в *пул*, в котором можно создать несколько файловых систем. Для сравнения пулы и тома показаны на рис. 8.11. Пулы предлагают больше гибкости, чем тома, позволяя увеличивать и уменьшать файловые системы независимо от емкости отдельных устройств хранения, входящих в пул. Этот подход используется в современных файловых системах, включая ZFS и btrfs, он также возможен при использовании LVM.

Объединенное хранилище может использовать все дисковые устройства для всех файловых систем, повышая производительность. Рабочие нагрузки не изолируются. В некоторых случаях для разделения рабочих нагрузок можно использовать несколько пулов, хотя и за счет потери гибкости, потому что дисковые устройства должны быть изначально включены в тот или иной пул. Обратите внимание, что в пул могут объединяться диски разных типов и размеров, тогда как для организации томов необходимо, чтобы все диски были одного размера.

Вот некоторые соображения, касающиеся производительности при использовании программных диспетчеров томов или объединенных хранилищ:

- **Ширина полосы:** должна соответствовать рабочей нагрузке.
- **Наблюдаемость:** метрики потребления виртуальных устройств могут вводить в заблуждение. Проверяйте метрики отдельных физических устройств.
- **Нагрузка на процессор:** особенно заметна в периоды вычисления четности в RAID. С появлением более мощных современных процессоров эта проблема уже не так актуальна. (Вычисление четности также можно переложить на аппаратные RAID-контроллеры.)
- **Восстановление** (или *перенастройка*): возникает, когда в группу RAID добавляется пустой диск (например, взамен вышедшего из строя) и заполняется данными, необходимыми для присоединения к группе. Восстановление значительно влияет на производительность, потому что потребляет ресурсы ввода/вывода и может длиться часами или даже днями.

Проблема снижения производительности в периоды восстановления носит усугубляющийся характер, потому что емкость устройств хранения увеличивается быстрее, чем их пропускная способность. Из-за этого увеличивается время восстановления и риск сбоев или менее серьезных ошибок в ходе восстановления. При возможности отключайте рабочую нагрузку на период восстановления отмонтированных дисков — это сократит время восстановления.

8.5. МЕТОДОЛОГИЯ

Ниже описаны методологии анализа и настройки файловой системы. Краткий перечень методологий приводится в табл. 8.3.

Список дополнительных методологий и их краткое описание ищите в главе 2 «Методологии».

Методологии можно применять по отдельности или в сочетании друг с другом. Предлагаю начинать анализ с использования следующих стратегий в указанном порядке: анализ задержек, мониторинг производительности, определение характеристик рабочей нагрузки и статическая настройка производительности. Выбирайте тот порядок и комбинации, которые лучше подходят для вашей среды.

В разделе 8.6 «Инструменты наблюдения» приведены приемы применения этих методологий с использованием инструментов ОС.

Таблица 8.3. Методологии анализа производительности файловых систем

Раздел	Методология	Тип
8.5.1	Анализ дисков	Анализ наблюдения
8.5.2	Анализ задержек	Анализ наблюдения
8.5.3	Определение характеристик рабочей нагрузки	Анализ наблюдения, планирование мощности
8.5.4	Мониторинг производительности	Анализ наблюдения, планирование мощности
8.5.5	Статическая настройка производительности	Анализ наблюдения, планирование мощности
8.5.6	Настройка кэширования	Анализ наблюдения, настройка
8.5.7	Разделение рабочей нагрузки	Настройка
8.5.8	Микробенчмаркинг	Анализ экспериментов

8.5.1. Анализ дисков

Распространенной стратегией устранения неполадок было игнорирование файловой системы и концентрация внимания на производительности диска. Это предполагает, что низкая производительность ввода/вывода обусловлена низкой производительностью дискового ввода/вывода, поэтому, анализируя только диски, вы фокусируетесь на ожидаемом источнике проблем.

С простыми файловыми системами и небольшими кэшами такая стратегия себя оправдывала. Но сейчас такой подход запутывает и упускает из виду целые классы проблем (см. раздел 8.3.12 «Логический и физический ввод/вывод»).

8.5.2. Анализ задержек

Анализ задержек лучше начинать с измерения задержек операций в файловой системе. К таким операциям относятся все операции с объектами, а не только ввод/вывод (включая, например, `sync(2)`).

Задержка операции = время (завершения операции) — время (отправки запроса на выполнение операции).

Эти времена можно измерить на четырех уровнях, перечисленных в табл. 8.4.

Таблица 8.4. Уровни для анализа задержек в файловой системе

Уровень	Достоинства	Недостатки
Приложение	На этом уровне проще всего оценить влияние задержки файловой системы на приложение; позволяет также учесть контекст приложения и определить — влияет ли задержка на производительность основной функции приложения или она асинхронна	Для разных приложений и версий прикладного ПО может потребоваться использовать разные приемы
Интерфейс системных вызовов	Хорошо документированный интерфейс. Обычно есть возможность анализа с применением инструментов ОС и статической трассировки	Системные вызовы покрывают все типы файловых систем, включая файловые системы, не использующие устройства хранения (статистики, сокет), что может сбивать с толку, если не использовать фильтрацию. Кроме того, для выполнения одной и той же операции файловой системы может использоваться несколько системных вызовов. Например, для чтения могут вызываться <code>read(2)</code> , <code>pread64(2)</code> , <code>preadv(2)</code> , <code>preadv2(2)</code> и т. д., каждый из которых необходимо проанализировать
VFS	Стандартный интерфейс для всех файловых систем; каждой операции в файловой системе соответствует один вызов (например, <code>vfs_write()</code>)	Через VFS следуют запросы ко всем типам файловых систем, включая файловые системы, не использующие устройства хранения, что может сбивать с толку, если не использовать фильтрацию
Интерфейс файловой системы	Анализируется только целевая файловая система. Есть возможность учитывать внутренние особенности файловой системы для получения дополнительных деталей	Зависит от типа файловой системы; приемы трассировки могут отличаться в зависимости от версии программного обеспечения файловой системы (хотя файловая система может иметь интерфейс, подобный VFS, который отображается в интерфейс VFS и поэтому меняется редко)

Выбор уровня зависит от наличия подходящего инструмента, поэтому, оказавшись перед выбором, проверьте:

- **документацию к приложению:** некоторые приложения предоставляют метрики, помогающие оценить задержки файловой системы, или позволяют включить их сбор;
- **инструменты операционной системы:** ОС могут предоставлять метрики, в идеале в виде отдельных статистик для каждой файловой системы или приложения;
- **возможность динамической инструментации:** если в вашей системе поддерживается динамическая инструментация (`kprobes` и `uprobes` в Linux, которые используются различными трассировщиками), то исследуйте все уровни с помощью специальных программ трассировки без перезапуска чего-либо.

Задержки могут предоставляться в виде средних значений за интервал, в виде распределений (например, гистограмм или тепловых карт: см. раздел 8.6.18) или в виде списка операций и их задержек. Для файловых систем с высоким коэффициентом попадания в кэш (более 99 %) средние значения за интервал могут в большей степени отражать задержку попадания в кэш. Это может вызывать досадные недоразумения при появлении отдельных случаев с высокой задержкой (выбросов), которые важно идентифицировать, но трудно заметить по усредненным значениям. Изучение полных распределений или задержек по операциям позволяет исследовать такие выбросы, а также влияние различных классов задержек, включая попадания и промахи кэша файловой системы.

Обнаружив высокие задержки, проведите углубленный анализ файловой системы, чтобы определить их источник.

Стоимость транзакции

Другой способ представления задержек в файловой системе — общее время, потраченное на ожидание в файловой системе во время транзакции (например, при выполнении запроса к базе данных):

Процент времени в файловой системе = $100 \times \text{общее время задержки в файловой системе} / \text{время выполнения транзакции в приложении}$.

Так можно количественно оценить стоимость операций с файловой системой с точки зрения производительности приложения и спрогнозировать возможное улучшение производительности. Метрика может быть представлена как среднее значение для всех транзакций за интервал или для некоторых из них.

На рис. 8.12 показано время, затрачиваемое прикладным потоком на обслуживание транзакции. Транзакция запускает операцию чтения из файловой системы. Приложение блокируется и ожидает завершения операции, оставляя процессор. Общее время задержки в файловой системе в данном случае — это время одной операции чтения. Если во время транзакции вызывается несколько блокирующих операций ввода/вывода, то общее время — это их сумма.



Рис. 8.12. Приложение и задержка в файловой системе

Рассмотрим конкретный пример: прикладная транзакция выполняется 200 мс, из которых 180 мс — это время ожидания завершения нескольких операций ввода/

вывода в файловой системе. Время, в течение которого приложение было заблокировано файловой системой, составляет 90 % ($100 \times 180 \text{ мс} / 200 \text{ мс}$). То есть устранение задержки в файловой системе может повысить производительность до 10 раз.

Другой пример: если прикладная транзакция выполняется 200 мс, из которых 2 мс было потрачено на ожидание файловой системы, то на работу файловой системы — и всего стека дискового ввода/вывода — тратится лишь 1 % от времени выполнения транзакции. Этот результат невероятно полезен, потому что может направить исследование на анализ истинного источника задержки.

Используя *неблокирующий* ввод/вывод, приложение может продолжать выполнение на процессоре, пока файловая система готовит ответ. В этом случае измерение задержки в файловой системе дает только время, в течение которого приложение было заблокировано вне процессора.

8.5.3. Определение характеристик рабочей нагрузки

Определение характеристик приложенной нагрузки — важный шаг при планировании мощностей, сравнительном анализе и моделировании рабочих нагрузок. Оно помогает значительно увеличить производительность за счет выявления и устранения ненужной работы.

Вот основные атрибуты, характеризующие нагрузку на файловую систему:

- частота и типы операций;
- пропускная способность файлового ввода/вывода;
- объем файлового ввода/вывода;
- соотношение операций чтения/записи;
- частота операций синхронной записи;
- произвольный и последовательный доступ к данным в файлах.

Частота операций и пропускная способность определены в разделе 8.1 «Терминология». Операции синхронной записи, а также операции с произвольным и последовательным доступом были описаны в разделе 8.3 «Основные понятия».

Эти характеристики могут быстро меняться со временем, особенно когда в системе есть задачи, запускающие приложения через определенные промежутки времени. Чтобы точнее охарактеризовать рабочую нагрузку, фиксируйте как максимальные, так и средние значения. Еще лучше изучить полное распределение значений во времени.

Пример ниже показывает, как все эти атрибуты можно выразить в одном описании:

На сервере баз данных для системы финансовой торговли файловая система выполняет в среднем 18 000 операций произвольного чтения в секунду при среднем объеме читаемых данных 4 Кбайт. Общая частота операций составляет 21 000 в секунду, включая чтение, получение статистик, открытие, закрытие

и примерно 200 операций синхронной записи в секунду. Частота операций записи стабильна, в то время как частота операций чтения меняется, достигая 39 000 операций чтения в секунду в пиковые моменты.

Эти характеристики могут описывать один экземпляр файловой системы или все экземпляры одного типа.

Чек-лист для определения дополнительных характеристик рабочей нагрузки

При необходимости можно выяснить дополнительные сведения, характеризующие рабочую нагрузку. Они перечислены ниже в форме вопросов, которые могут служить чек-листом при исследовании проблем с файловой системой:

- Каков коэффициент попаданий в кэш файловой системы? Промехов?
- Какова емкость кэша файловой системы и его текущее потребление?
- Какие еще есть кэши (каталогов, индексных узлов, буферов) и каковы статистики их использования?
- Предпринимались ли в прошлом попытки настроить файловую систему? Имеют ли какие-либо параметры файловой системы значения, отличные от значений по умолчанию?
- Какие приложения или пользователи используют файловую систему?
- К каким файлам и каталогам осуществляется доступ? Какие создаются и удаляются?
- Были ли обнаружены ошибки? Эти ошибки обусловлены неверными запросами или проблемами с файловой системой?
- Почему выполняется ввод/вывод через файловую систему (пути в коде на уровне пользователя)?
- В какой степени приложения напрямую (синхронно) запрашивают ввод/вывод через файловую систему?
- Как распределяется продолжительность выполнения операций ввода/вывода?

Многие из этих вопросов можно задать по отдельным приложениям или файлам. Любые из метрик также можно исследовать со временем, чтобы определить максимальные и минимальные значения, а также характер изменения во времени. См. также раздел 2.5.11 «Определение характеристик рабочей нагрузки» в главе 2 «Методологии», где приводится обобщенное описание этих и других характеристик (кто, почему, зачем, как).

Определение характеристик производительности

В предыдущих чек-листах перечислены метрики, определяющие характеристики применяемой рабочей нагрузки. В списке ниже перечислены метрики результирующей производительности:

- Какова средняя задержка файловой системы?
- Есть ли выбросы с большой задержкой?
- Как распределены задержки?
- Есть ли и активны ли системные средства управления ресурсами файловой системы либо дискового ввода/вывода?

Первые три вопроса можно задать для каждого типа операций.

Трассировка событий

Инструменты трассировки используют для записи деталей всех операций с файловой системой в журнал с целью последующего анализа. В число деталей могут входить: тип операции, аргументы, путь к файлу, отметки времени начала и окончания, код завершения, а также идентификатор и имя процесса. Это идеальный подход к определению характеристик рабочей нагрузки, но на практике у него большой оверхед из-за высокой частоты следования операций с файловой системой, что делает его неприменимым без фильтрации (например, обеспечивающей запись в журнал только медленных операций ввода/вывода: см. `ext4slower(8)` в разделе 8.6.14).

8.5.4. Мониторинг производительности

Мониторинг производительности помогает обнаружить проблемы и модели поведения, проявляющиеся со временем. Ключевые метрики для файловой системы:

- частота операций;
- задержки при выполнении операций.

Частота операций — основная характеристика применяемой рабочей нагрузки, а задержки — результирующая производительность. Величина нормальной или ненормальной задержки зависит от рабочей нагрузки, среды и требований к задержке. В случае сомнений можно провести микробенчмаркинг с использованием заведомо умеренных и тяжелых рабочих нагрузок, чтобы изучить задержки (например, для рабочих нагрузок, у которых высокий коэффициент попадания в кэш файловой системы в противовес нагрузкам с большой долей промахов). См. раздел 8.7 «Эксперименты».

Метрика задержки при выполнении операций может вычисляться как среднее значение в секунду и включать другие значения, такие как максимальное и стандартное отклонение. В идеале желательно исследовать полное распределение задержек, например, построив гистограмму или тепловую карту, чтобы не пропустить выбросы и другие закономерности.

Значения частоты и задержек также можно фиксировать для каждого типа операций (чтение, запись, получение статистик, открытие, закрытие и т. д.). Это поможет в расследовании изменений характера рабочей нагрузки и производительности за счет выявления различий в конкретных типах операций.

В средах, поддерживающих управление ресурсами файловой системы, можно включить сбор статистик, подсказывающих, когда и как использовались механизмы регулирования.

К сожалению, в Linux практически нет доступных статистик для операций с файловой системой (исключение — NFS, для которой есть инструмент `nfsstat(8)`).

8.5.5. Статическая настройка производительности

Статическая настройка производительности фокусируется на проблемах сконфигурированной архитектуры. Анализируя производительность файловой системы, изучите следующие аспекты статической конфигурации:

- Сколько файловых систем смонтировано и активно используется?
- Каков размер записи в файловой системе?
- Включена ли поддержка отметок времени последнего доступа?
- Какие еще параметры файловой системы включены (сжатие, шифрование, ...)?
- Как настроен кэш файловой системы? Каков его максимальный размер?
- Как настроены другие кэши (каталогов, индексных узлов, буферов)?
- Есть ли и используется ли кэш второго уровня?
- Сколько устройств хранения имеется и используется?
- Какая конфигурация устройств хранения используется? RAID?
- Какие типы файловых систем используются?
- Какие версии файловых систем (или ядра) используются?
- Есть ли известные ошибки и исправления для файловой системы, которые следует учитывать?
- Используются ли средства управления ресурсами файловой системы?

Ответы на эти вопросы помогут выявить варианты конфигурации, которые были упущены из виду. Иногда система была настроена для одной рабочей нагрузки, а затем перепрофилирована для другой. Этот метод напугает вас о необходимости пересмотреть эти варианты.

8.5.6. Настройка кэша

Ядро и файловая система могут использовать множество разных кэшей, включая кэш буферов, кэш каталогов, кэш индексных узлов и кэш файловой системы (кэш страниц). Эти виды кэшей были описаны в разделе 8.4 «Архитектура». Их можно исследовать и даже настраивать в зависимости от наличия параметров, доступных для настройки.

8.5.7. Разделение рабочей нагрузки

Некоторые типы рабочих нагрузок работают лучше, если настроены на использование собственных эксклюзивных файловых систем и дисковых устройств. Такой

подход известен как использование «отдельных шпинделей», потому что операции произвольного ввода/вывода из двух отдельных рабочих нагрузок плохо сказываются на производительности обеих из-за необходимости часто позиционировать магнитные головки вращающихся дисков (см. главу 9 «Диски»).

Например, для баз данных почти всегда предпочтительнее выделять для файлов журнала и файлов с данными отдельные файловые системы и диски. Руководства по установке БД часто содержат рекомендации по размещению хранилищ данных.

8.5.8. Микробенчмаркинг

Для тестирования производительности различных типов файловых систем или настроек в файловой системе для заданных рабочих нагрузок могут использоваться инструменты бенчмаркинга файловых систем и дисков (таких инструментов очень много). Вот типичные факторы, которые могут быть протестированы:

- **Типы операций:** частота операций чтения, записи и др.
- **Объем ввода/вывода:** от 1 байта до 1 мегабайта и больше.
- **Характер изменения смещения в файле:** произвольный или последовательный.
- **Характер произвольного доступа:** равномерный, случайный или согласуется с распределением Парето.
- **Тип операций записи:** асинхронный или синхронный (O_SYNC).
- **Размер рабочего набора:** насколько хорошо соответствует размеру кэша файловой системы.
- **Конкурентность:** число операций ввода/вывода, выполняющихся одновременно, или потоков выполнения, производящих ввод/вывод.
- **Отображение в память:** доступ к файлам через mmap(2) вместо read(2)/write(2).
- **Состояние кэша:** кэш файловой системы — «холодный» (незаполненный) или «теплый».
- **Параметры файловой системы:** к ним можно отнести поддержку сжатия, дубликации данных и т. д.

К типичным комбинациям относятся: произвольное чтение, последовательное чтение, произвольная запись и последовательная запись. Я не включил прямой ввод/вывод в этот список, потому что его цель при микробенчмаркинге — проверка производительности дисковых устройств за счет выполнения операций в обход файловой системы (см. главу 9 «Диски»).

Важным фактором при микробенчмаркинге файловых систем является *размер рабочего набора* (working set size, WSS): объем данных, к которым осуществляется доступ. В зависимости от вида тестирования это может быть общий размер используемых файлов. Рабочий набор небольшого размера может полностью возвращаться из кэша файловой системы в основной памяти, если не используется флаг прямого ввода/вывода. Рабочий набор большого размера может возвращаться в основном

с устройств хранения (дисков). Производительность в этих случаях может на порядок отличаться. Бенчмаркинг недавно смонтированной файловой системы, а затем его повтор после заполнения кэшей и сравнение результатов — это хорошая иллюстрация WSS. (Также см. раздел 8.7.3 «Очистка кэша».)

В табл. 8.5 перечислены общие ожидаемые результаты для различных бенчмарков, включая общий размер файлов (WSS).

Таблица 8.5. Ожидаемые результаты бенчмаркинга файловых систем

Системная память	Общий размер файлов (WSS)	Бенчмарк	Ожидаемый результат
128 Гбайт	10 Гбайт	Произвольное чтение	100 % попаданий в кэш
128 Гбайт	10 Гбайт	Произвольное чтение, прямой ввод/вывод	100 % операций чтения выполняется с диска (из-за прямого ввода/вывода)
128 Гбайт	1000 Гбайт	Произвольное чтение	Основная часть операций чтения выполняется с диска, доля попаданий в кэш около 12 %
128 Гбайт	10 Гбайт	Последовательное чтение	100 % попаданий в кэш
128 Гбайт	1000 Гбайт	Последовательное чтение	Смесь попаданий в кэш (в основном из-за опережающего чтения) и чтений с диска
128 Гбайт	10 Гбайт	Буферизованная запись	Подавляющее большинство операций попадают в кэш (буферизуются), и лишь отдельные операции записи блокируют рабочую нагрузку в зависимости от особенностей поведения файловой системы
128 Гбайт	10 Гбайт	Синхронная запись	100 % операций записи производят непосредственную запись данных на диск

Некоторые инструменты бенчмаркинга файловой системы не дают четкого представления о том, что они тестируют, и могут подразумевать бенчмаркинг *диска*. Они используют файлы небольшого размера, которые полностью умещаются в кэше и поэтому фактически не тестируют диски. См. раздел 8.3.12 «Логический и физический ввод/вывод», чтобы понять разницу между тестированием файловой системы (логический ввод/вывод) и дисков (физический ввод/вывод).

Некоторые инструменты бенчмаркинга *дисков* работают через файловую систему, используя прямой ввод/вывод, чтобы избежать кэширования и буферизации. В таких случаях файловая система все так же играет второстепенную роль, но добавляется оверхед на выполнение дополнительных путей в коде, и тестирование отражает различия между размещением данных в файле и на диске.

Подробную информацию о бенчмаркинге см. в главе 12.

8.6. ИНСТРУМЕНТЫ НАБЛЮДЕНИЯ

В этом разделе представлены инструменты наблюдения за работой файловой системы, доступные в ОС на базе Linux. Стратегии их использования описываются в предыдущем разделе.

Инструменты перечислены в табл. 8.6.

Таблица 8.6. Инструменты наблюдения за файловой системой

Раздел	Инструмент	Описание
8.6.1	mount	Список файловых систем и их флаги монтирования
8.6.2	free	Статистики, отражающие емкость кэшей
8.6.3	top	Включает сводную информацию о потреблении памяти
8.6.4	vmstat	Статистики потребления виртуальной и физической памяти
8.6.5	sar	Различные статистики, включая исторические
8.6.6	slabtop	Статистики распределителя блоков в ядре
8.6.7	strace	Трассировка системных вызовов
8.6.8	fatrace	Трассировка операций с файловой системой с использованием интерфейса fanotify
8.6.9	latencytop	Показывает источники задержек в масштабе всей системы
8.6.10	opensnoop	Трассировка операций открытия файлов
8.6.11	filetop	Определяет наиболее интенсивно используемые файлы по количеству операций и объему ввода/вывода
8.6.12	cachestat	Выводит статистики, характеризующие использование кэша страниц
8.6.13	ext4dist (xfs, zfs, btrfs, nfs)	Выводит распределение задержек операций в ext4
8.6.14	ext4slower (xfs, zfs, btrfs, nfs)	Выводит информацию о медленных операциях в ext4
8.6.15	bpfftrace	Настраиваемая трассировка файловой системы

Описание этого набора инструментов и возможностей дополняет раздел 8.5 «Методология». Оно начинается с описания традиционных инструментов, а затем переходит к инструментам на основе трассировки. Некоторые из традиционных инструментов доступны (а иногда первоначально созданы) в других Unix-подобных ОС, в том числе mount(8), free(1), top(1), vmstat(8) и sar(1). Многие из инструментов трассировки основаны на BPF и используют интерфейсы BCC и bpfftrace (глава 15), в том числе opensnoop(8), filetop(8), cachestat(8), ext4dist(8) и ext4slower(8).

Полный перечень возможностей каждого инструмента ищите в соответствующей документации, включая страницы справочного руководства man.

8.6.1. mount

Команда `mount(1)` в Linux выводит список смонтированных файловых систем и их флаги монтирования:

```
$ mount
/dev/nvme0n1p1 on / type ext4 (rw,relatime,discard)
devtmpfs on /dev type devtmpfs (rw,relatime,size=986036k,nr_inodes=246509,mode=755)
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev)
[...]
```

Первая строка показывает, что файловая система `ext4`, находящаяся в разделе `/dev/nvme0n1p1`, смонтирована в каталог `/` с флагами монтирования `rw`, `relatime` и `discard`. `relatime` — это параметр, предназначенный для увеличения производительности, который отключает обновление времени последнего доступа к индексным узлам и сопутствующие затраты на дисковый ввод/вывод. Время последнего доступа обновляется, только если изменяется время последнего изменения или если последнее обновление было более суток тому назад.

8.6.2. free

Команда `free(1)` в Linux выводит статистики распределения основной памяти и пространства в файле подкачки. Ниже примеры обычного и широкого (`-w`) вывода объемов свободной памяти в мегабайтах (`-m`):

```
$ free -m
              total        used        free      shared  buff/cache   available
Mem:           1950         568         163           0         1218         1187
Swap:           0           0           0
$ free -mw
              total        used        free      shared    buffers       cache   available
Mem:           1950         568         163           0          84        1133        1187
Swap:           0           0           0
```

Широкий вывод показывает столбец `buffers`, в котором отображается размер кэша буферов, и столбец `cache` с размером кэша страниц. В выводе по умолчанию эти значения объединяются в столбце `buff/cache`.

Особо важен столбец `available` (новое дополнение в `free(1)`), который показывает, сколько памяти доступно для приложений без использования подкачки. При этом учитывается память, которую нельзя немедленно освободить.

Значения для этих полей можно получить и из `/proc/meminfo`, где они указаны в килобайтах.

8.6.3. top

Некоторые версии команды `top(1)` включают сведения о кэше файловой системы. Строки ниже выводит Linux-версия `top(1)`. Они включают статистики `buff/cache` и `available (avail Mem)`, которые выводит `free(1)`:

```
MiB Mem :    1950.0 total,    161.2 free,    570.3 used,    1218.6 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    1185.9 avail Mem
```

Дополнительные сведения о `top(1)` см. в главе 6 «Процессоры».

8.6.4. vmstat

Команда `vmstat(1)`, как и `top(1)`, выводит информацию о кэше файловой системы. Дополнительные подробности о `vmstat(1)` см. в главе 7 «Память».

Ниже — пример запуска команды `vmstat(1)` с параметром `1`, обеспечивающим вывод статистик раз в секунду:

```
$ vmstat 1
procs -----memory----- --swap--  -----io----- -system--  -----cpu-----
 r  b   swpd   free   buff  cache   si   so    bi   bo    in   cs  us  sy  id  wa  st
 0  0       0 167644  87032 1161112    0    0     7   14   14    1  4  2 90  0  5
 0  0       0 167636  87032 1161152    0    0     0    0  162  376  0  0 100  0  0
[...]
```

Столбец `buff` показывает размер кэша буферов, а столбец `cache` — размер кэша страниц, в обоих случаях размер выводится в килобайтах.

8.6.5. sar

Генератор отчетов о работе системы (system activity reporter) `sar(1)` можно использовать для получения самых разных статистик о работе файловой системы, настроив его для периодического их архивирования. Этот инструмент упоминается в нескольких главах книги (из-за разнообразия собираемых им статистик) и достаточно полно был представлен в главе 4 «Инструменты наблюдения», в разделе 4.4 «sar».

Вот пример запуска `sar(1)` для получения информации о текущей активности с интервалом в одну секунду:

```
# sar -v 1
Linux 5.3.0-1009-aws (ip-10-1-239-218)    02/08/20    _x86_64_    (2 CPU)
21:20:24   dentunusd   file-nr   inode-nr   pty-nr
21:20:25       27027       1344    52945        2
21:20:26       27012       1312    52922        2
21:20:27       26997       1248    52899        2
[...]
```

Параметр `-v` обеспечивает вывод следующих столбцов:

- **dentunusd**: количество неиспользуемых (свободных) записей в кэше каталогов;
- **file-nr**: количество используемых дескрипторов файлов;
- **inode-nr**: количество используемых индексных узлов;
- **pty-nr**: количество используемых псевдотерминалов.

Поддерживается также параметр `-r`, который выводит столбцы `kbbuffers` и `kbcached` с размерами кэша буферов и кэша страниц в килобайтах.

8.6.6. slabtop

Команда `slabtop(1)` в Linux выводит информацию об использовании кэша блоков памяти ядра, часть которых используется кэшами файловой системы:

```
# slabtop -o
Active / Total Objects (% used)   : 604675 / 684235 (88.4%)
Active / Total Slabs (% used)     : 24040 / 24040 (100.0%)
Active / Total Caches (% used)    : 99 / 159 (62.3%)
Active / Total Size (% used)      : 140593.95K / 160692.10K (87.5%)
Minimum / Average / Maximum Object : 0.01K / 0.23K / 12.00K

  OBJS ACTIVE  USE OBJ SIZE  SLABS OBJ/SLAB  CACHE SIZE NAME
165945 149714  90%   0.10K   4255    39    17020K buffer_head
107898  66011  61%   0.19K   5138    21    20552K dentry
  67350  67350 100%   0.13K   2245    30     8980K kernfs_node_cache
  41472  40551  97%   0.03K    324   128     1296K kmalloc-32
  35940  31460  87%   1.05K   2396    15    38336K ext4_inode_cache
  33514  33126  98%   0.58K   2578    13    20624K inode_cache
  24576  24576 100%   0.01K     48   512     192K kmalloc-8
[...]
```

Некоторые блоки в памяти ядра, связанные с кэшами файловой системы, видны в этом выводе — это записи `dentry`, `ext4_inode_cache` и `inode_cache`. Без параметра `-o` (once — режим однократного вывода) `slabtop(1)` будет постоянно обновлять экран.

В число блоков могут входить:

- **buffer_head**: используется кэшем буферов;
- **dentry**: кэш каталогов;
- **inode_cache**: кэш индексных узлов;
- **ext3_inode_cache**: кэш индексных узлов для ext3;
- **ext4_inode_cache**: кэш индексных узлов для ext4;
- **xfs_inode**: кэш индексных узлов для XFS;
- **btrfs_inode**: кэш индексных узлов для btrfs.

`slabtop(1)` использует информацию из файла `/proc/slabinfo`, который создается, если ядро собрано с параметром `CONFIG_SLAB`.

8.6.7. strace

Задержку файловой системы можно измерить на уровне системных вызовов с помощью инструментов трассировки, включая `strace(1)` для Linux. Но текущая реализация `strace(1)` на основе `ptrace(2)` может серьезно ухудшить производительность системы, поэтому этот прием применяют, когда оверхед допустим и нет возможности использовать другие методы анализа. Дополнительную информацию о `strace(1)` см. в главе 5 в разделе 5.5.4 «`strace`».

Пример ниже показывает измерение времени чтения в файловой системе `ext4` с помощью `strace(1)`:

```
$ strace -ttT -p 845
[...]
```

18:41:01.513110	read(9, "\334\260/\224\356k..."...	65536)	=	65536	<0.018225>
18:41:01.531646	read(9, "\371X\265 \244\317..."...	65536)	=	65536	<0.000056>
18:41:01.531984	read(9, "\357\311\347\1\241..."...	65536)	=	65536	<0.005760>
18:41:01.538151	read(9, "*\263\264\204 \370..."...	65536)	=	65536	<0.000033>
18:41:01.538549	read(9, "\205q\327\304f\370..."...	65536)	=	65536	<0.002033>
18:41:01.540923	read(9, "\6\2738>zw\321\353..."...	65536)	=	65536	<0.000032>

Параметр `-tt` обеспечивает вывод отметок времени слева, а `-T` — продолжительность выполнения системных вызовов справа. Все вызовы `read(2)` прочитали по 64 Кбайт, продолжительность выполнения первого вызова составила 18 мс, второго 56 мкс (вероятно, данные были получены из кэша) и третьего 5 мс. Операции чтения выполнялись с файловым дескриптором 9. Чтобы проверить, относится ли этот дескриптор к файловой системе (а не к сокету), поищите в выводе выше вызов `open(2)` либо воспользуйтесь другим инструментом, таким как `lsof(8)`. Также информацию о дескрипторе 9 можно найти в файловой системе `/proc`: `/proc/845/fd{,info}/9`.

Учитывая оверхед, характерный для текущей реализации `strace(1)`, измеренная задержка может быть искажена из-за эффекта наблюдателя. Обратите внимание на новые инструменты трассировки, включая `ext4slower(8)`, которые для уменьшения оверхеда и более точного измерения задержек используют трассировку с буферизацией для каждого процессора и BPF.

8.6.8. fatrace

`fatrace(1)` — специализированный трассировщик, использующий прикладной интерфейс `fanotify` (file access notify — уведомление о попытках доступа к файлу) в Linux. Вот пример вывода этой команды:

```
# fatrace
sar(25294): O /etc/ld.so.cache
sar(25294): RO /lib/x86_64-linux-gnu/libc-2.27.so
sar(25294): C /etc/ld.so.cache
sar(25294): O /usr/lib/locale/locale-archive
sar(25294): O /usr/share/zoneinfo/America/Los_Angeles
sar(25294): RC /usr/share/zoneinfo/America/Los_Angeles
```

```
sar(25294): R0 /var/log/sysstat/sa09
sar(25294): R /var/log/sysstat/sa09
[...]
```

В каждой строке выводятся: имя процесса, идентификатор процесса PID, тип события, полный путь и необязательный статус. Поддерживаются типы событий: (O) — открытие (open), (R) — чтение (read), (W) — запись (write) и (C) — закрытие (close). `fatrace(1)` можно использовать для определения таких характеристик рабочей нагрузки, как имена файлов, к которым осуществляется доступ, и поиска случаев, когда выполняется ненужная работа, которой можно избежать.

Но для высоконагруженных файловых систем `fatrace(1)` может выводить десятки тысяч строк в секунду и расходовать значительные вычислительные ресурсы. Это немного смягчается фильтрацией по типу события. Также можно использовать инструменты трассировки на основе BPF, включая `opensnoop(8)` (раздел 8.6.10), отличающиеся незначительным оверхедом.

8.6.9. LatencyTOP

LatencyTOP — это инструмент для составления отчетов об источниках задержки по системе в целом и по отдельным процессам. LatencyTOP сообщает и о задержках в файловой системе. Например:

Cause	Maximum	Percentage
Reading from file	209.6 msec	61.9 %
synchronous write	82.6 msec	24.0 %
Marking inode dirty	7.9 msec	2.2 %
Waiting for a process to die	4.6 msec	1.5 %
Waiting for event (select)	3.6 msec	10.1 %
Page fault	0.2 msec	0.2 %
Process gzip (10969)	Total: 442.4 msec	
Reading from file	209.6 msec	70.2 %
synchronous write	82.6 msec	27.2 %
Marking inode dirty	7.9 msec	2.5 %

В верхней части выводится сводная информация для системы в целом, а в нижней — для единственного процесса `gzip(1)`, который в данный момент сжимает файл. Больше всего задержек вносят операции чтения исходного файла (`Reading from file`) — 70,2 % и операции синхронной записи (`synchronous write`) — 27,2 % в новый сжатый файл.

Инструмент LatencyTOP был разработан в Intel, но давно не обновлялся, и сайт проекта больше не доступен. Кроме того, для поддержки этого инструмента ядро должно быть скомпилировано с определенными параметрами, которые обычно выключены¹. Возможно, будет проще измерить задержку в файловой системе с помощью инструментов трассировки BPF, см. разделы 8.6.13–8.6.15.

¹ CONFIG_LATENCYTOP и CONFIG_HAVE_LATENCYTOP_SUPPORT.

8.6.10. opensnoop

opensnoop(8)¹ — это инструмент для BCC и bpftrace, который трассирует операции открытия файлов. Может использоваться для определения местоположения файлов данных, журналов и конфигураций. Он также обнаруживает проблемы с производительностью, вызванные частым открытием, и помогает устранить проблемы с отсутствующими файлами. Вот пример вывода, полученный в производственной системе, с параметром -T для включения в вывод отметок времени:

```
# opensnoop -T
TIME(s)    PID    COMM      FD ERR PATH
0.000000000 26447  sshd      5   0 /var/log/btmp
[...]
1.961686000 25983  mysqld    4   0 /etc/mysql/my.cnf
1.961715000 25983  mysqld    5   0 /etc/mysql/conf.d/
1.961770000 25983  mysqld    5   0 /etc/mysql/conf.d/mysql.cnf
1.961799000 25983  mysqld    5   0 /etc/mysql/conf.d/mysqldump.cnf
1.961818000 25983  mysqld    5   0 /etc/mysql/mysql.conf.d/
1.961843000 25983  mysqld    5   0 /etc/mysql/mysql.conf.d/mysql.cnf
1.961862000 25983  mysqld    5   0 /etc/mysql/mysql.conf.d/mysqld.cnf
[...]
2.438417000 25983  mysqld    4   0 /var/log/mysql/error.log
[...]
2.816953000 25983  mysqld   30   0 ./binlog.000024
2.818827000 25983  mysqld   31   0 ./binlog.index_crash_safe
2.820621000 25983  mysqld    4   0 ./binlog.index
[...]
```

В этом выводе зафиксирован запуск базы данных MySQL, и как видите, opensnoop(2) обнаружил файлы конфигурации, файл журнала, файлы данных (двоичные журналы) и многое другое.

opensnoop(8) трассирует варианты системного вызова open(2): open(2) и openat(2). Оверхед на трассировку обычно незначителен, потому что open(2) вызывается нечасто.

В числе параметров версии для BCC можно назвать:

- **-T**: добавить колонку с отметками времени;
- **-x**: показать только неудачные вызовы open;
- **-p PID**: трассировать только процесс с этим PID;
- **-n NAME**: выводить только имена процессов, содержащие NAME.

Параметр -x можно использовать при решении проблем: он выводит информацию о неудачных попытках открыть файл.

¹ Немного истории: первую версию opensnoop я создал в 2004 году, версию для BCC — 17 сентября 2015 года, а для bpftrace — 8 сентября 2018 года.

8.6.11. filetop

filetop(8)¹ — это инструмент для ВСС, действующий как утилита top(1), только для файлов. Он показывает имена наиболее часто используемых файлов. Вот пример вывода:

```
# filetop
Tracing... Output every 1 secs. Hit Ctrl-C to end

19:16:22 loadavg: 0.11 0.04 0.01 3/189 23035

TID   COMM                READS  WRITES R_Kb   W_Kb   T FILE
23033 mysqld               481    0      7681   0      R sb1.ibd
23033 mysqld                3     0       48    0      R mysql.ibd
23032 oltp_read_only.     3     0       20    0      R oltp_common.lua
23031 oltp_read_only.     3     0       20    0      R oltp_common.lua
23032 oltp_read_only.     1     0       19    0      R Index.xml
23032 oltp_read_only.     4     0       16    0      R openssl.cnf
23035 systemd-udevd     4     0       16    0      R sys_vendor
[...]
```

По умолчанию выводятся двадцать наиболее часто используемых файлов, отсортированных по столбцу прочитанных байтов. Как показывает верхняя строка, mysqld выполнил 481 операцию чтения из файла sb1.ibd общим объемом 7681 Кбайт.

Этот инструмент используют для определения характера рабочей нагрузки и наблюдения за файловой системой в целом. Как top(1) выявляет процессы с неожиданно большим потреблением процессора, этот инструмент помогает обнаружить файлы, использующиеся необычно интенсивно.

По умолчанию filetop выводит информацию только об обычных файлах. Но если добавить параметр -a, то инструмент выведет информацию обо всех файлах, включая сокеты TCP:

```
# filetop -a
[...]
```

TID	COMM	READS	WRITES	R_Kb	W_Kb	T FILE
21701	sshd	1	0	16	0	O ptmx
23033	mysqld	1	0	16	0	R sbtest1.ibd
23335	sshd	1	0	8	0	S TCP
1	systemd	4	0	4	0	R comm

```
[...]
```

Этот вывод содержит информацию о дополнительных типах файлов: другой (O, other) и сокет (S, socket). В этом случае к другому типу отнесен файл ptmx — специальный файл символьного устройства в каталоге /dev.

¹ Немного истории: версию для ВСС я создал 6 февраля 2016 года, вдохновившись утилитой top(1) Уильяма Лефевра.

В числе поддерживаемых параметров можно назвать:

- **-C**: не очищать экран — выводить данные с прокруткой;
- **-a**: показать файлы всех типов;
- **-r ROWS**: выводить указанное число строк (по умолчанию 20);
- **-p PID**: трассировать только процесс с этим PID.

Экран обновляется каждую секунду (как в `top(1)`), если не задан параметр `-C`. Я предпочитаю использовать команду с параметром `-C`, чтобы вывод сохранялся в буфере прокрутки терминала на тот случай, если мне понадобится исследовать данные.

8.6.12. cachestat

`cachestat(8)`¹ — это инструмент для ВСС, который выводит статистику попаданий и промахов кэша страниц. Его можно использовать для проверки доли попаданий и эффективности кэша страниц, а также для исследования влияния разных настроек системы и приложений и получения информации об эффективности кэша. Вот пример вывода:

```
$ cachestat -T 1
TIME      HITS    MISSES  DIRTIES  HITRATIO  BUFFERS_MB  CACHED_MB
21:00:48   586         0    1870   100.00%      208       775
21:00:49   125         0    1775   100.00%      208       776
21:00:50   113         0    1644   100.00%      208       776
21:00:51    23         0    1389   100.00%      208       776
21:00:52   134         0    1906   100.00%      208       777
[...]
```

Этот вывод показывает рабочую нагрузку чтения, которая получает все данные исключительно из кэша (количество попаданий выводится в столбце `HITS`, а доля попаданий — 100 % в данном примере — в столбце `HITRATIO`), и более высокую рабочую нагрузку записи (столбец `DIRTIES`). В идеале желательно, чтобы доля попаданий была как можно ближе к 100 %, чтобы операции чтения не блокировали приложения для выполнения дискового ввода/вывода.

Столкнувшись с низкой долей попаданий в кэш, из-за чего может наблюдаться снижение производительности, попробуйте немного уменьшить объем памяти, выделяемой приложению, чтобы для кэша страниц осталось больше места. Если настроены устройства подкачки, то настройте подкачку, чтобы предпочтение отдавалось удалению страниц из кэша вместо подкачки.

Из параметров можно назвать `-T`, включающий вывод отметок времени.

¹ Немного истории: первую экспериментальную версию инструмента на основе `Ftrace` я создал 28 декабря 2014 года. На ее основе Аллан Макаливи (Allan McAleavy) реализовал версию для ВСС 6 ноября 2015 года.

Этот инструмент дает ценную информацию о доле попаданий в кэш страниц, но пока остается экспериментальным и использует зонды `kprobes` для трассировки определенных функций ядра. Может потребоваться изменить его исходный код для работы с разными версиями ядра. Еще лучше было бы переписать этот инструмент и использовать в нем точки трассировки или статистики из `/proc`, чтобы улучшить стабильность. Лучшее применение этого инструмента на сегодняшний день — демонстрация того, что такое вообще возможно.

8.6.13. `ext4dist` (`xfs`, `zfs`, `btrfs`, `nfs`)

`ext4dist(8)`¹ — это инструмент для БСС и `bpftrace`, позволяющий инструментировать файловую систему `ext4` и исследовать гистограммы распределения задержек типичных операций: чтения, записи, открытия и синхронизации. Есть версии и для других файловых систем: `xfsdist(8)`, `zfsdist(8)`, `btrfsdist(8)` и `nfsdist(8)`. Вот пример вывода:

```
# ext4dist 10 1
```

```
Tracing ext4 operation latency... Hit Ctrl-C to end.
```

```
21:09:46:
```

```
operation = read
```

usecs	: count	distribution
0 -> 1	: 783	*****
2 -> 3	: 88	**
4 -> 7	: 449	*****
8 -> 15	: 1306	*****
16 -> 31	: 48	*
32 -> 63	: 12	
64 -> 127	: 39	*
128 -> 255	: 11	
256 -> 511	: 158	****
512 -> 1023	: 110	***
1024 -> 2047	: 33	*

```
operation = write
```

usecs	: count	distribution
0 -> 1	: 1073	*****
2 -> 3	: 324	*****
4 -> 7	: 1378	*****
8 -> 15	: 1505	*****
16 -> 31	: 183	****
32 -> 63	: 37	
64 -> 127	: 11	
128 -> 255	: 9	

```
operation = open
```

usecs	: count	distribution
0 -> 1	: 672	*****

¹ Немного истории: версию для БСС я создал 12 февраля 2016 года, а версию для `bpftrace` — 2 февраля 2019 года [Gregg 19]. За основу был взят аналогичный инструмент для ZFS, который я написал в 2012 году.

```

      2 -> 3      : 10      |
operation = fsync
  usecs          : count    distribution
    256 -> 511    : 485      |*****|
    512 -> 1023   : 308      |*****|
    1024 -> 2047  : 1779     |*****|
    2048 -> 4095  : 79       |*      |
    4096 -> 8191  : 26       |        |
    8192 -> 16383 : 4        |        |

```

Параметры команды в этом примере определяют 10-секундный интервал и однократный вывод результатов трассировки, накопленных в течение этого интервала. Как показывает вывод, у задержек чтения бимодальное распределение: одна мода находится в интервале от 0 до 15 мкс и, вероятно, связана с попаданиями в кэш, а другая — в интервале от 256 до 2048 мкс и, вероятно, соответствует операциям чтения с диска. Можно исследовать и распределение задержек в других операциях. Запись выполняется быстро, вероятно, из-за сохранения данных в буферы, которые позже выталкиваются на диск с помощью более медленной операции синхронизации (fsync).

Этот и сопутствующий ему инструмент `ext4slower(8)` (см. следующий раздел) показывают задержки, с которыми могут столкнуться приложения. Измерение задержек на уровне диска возможно и показано в главе 9, но приложения могут не блокироваться операциями дискового ввода/вывода, что затрудняет интерпретацию таких измерений. По возможности я использую инструменты `ext4dist(8)` и `ext4slower(8)` перед инструментами измерения задержки дискового ввода/вывода. См. раздел 8.3.12, где описываются различия между логическим вводом/выводом на уровне файловых систем, продолжительность которого измеряется этим инструментом, и физическим вводом/выводом на уровне дисков.

В числе поддерживаемых параметров можно назвать:

- **-m**: выводит время в миллисекундах;
- **-p PID**: трассировать только процесс с этим PID.

Вывод этого инструмента можно представить в виде тепловой карты задержек. Чтобы получить больше информации о медленном вводе/выводе файловой системы, используйте `ext4slower(8)` и его варианты.

8.6.14. `ext4slower` (xfs, zfs, btrfs, nfs)

`ext4slower(8)`¹ трассирует выполнение обычных операций в файловой системе ext4 (чтение, запись, открытие и синхронизация) и сообщает о тех, продолжительность которых превышает определенный порог. Вот пример вывода:

¹ Немного истории: этот инструмент я создал 11 февраля 2016 года, взяв за основу аналогичный инструмент для ZFS, который написал в 2011 году.

```
# ext4slower
Tracing ext4 operations slower than 10 ms
TIME      COMM      PID    T BYTES  OFF_KB  LAT(ms)  FILENAME
21:36:03 mysqld    22935  S  0      0      12.81 sbtest1.ibd
21:36:15 mysqld    22935  S  0      0      12.13 ib_logfile1
21:36:15 mysqld    22935  S  0      0      10.46 binlog.000026
21:36:15 mysqld    22935  S  0      0      13.66 ib_logfile1
21:36:15 mysqld    22935  S  0      0      11.79 ib_logfile1
[...]
```

В столбцах выводятся: текущее время (TIME), имя процесса (COMM), идентификатор процесса (PID), тип операции (T: R — чтение, W — запись, O — открытие и S — синхронизация), смещение в килобайтах (OFF_KB), задержка в миллисекундах (LAT(ms)) и имя файла (FILENAME).

В этом примере видно несколько операций синхронизации (S), длившихся дольше 10 мс (пороговое значение по умолчанию). Порог можно передать в виде аргумента. Если указать 0 мс, то в выводе будут показаны все операции:

```
# ext4slower 0
Tracing ext4 operations
21:36:50 mysqld    22935  W 917504 2048      0.42 ibdata1
21:36:50 mysqld    22935  W 1024   14165     0.00 ib_logfile1
21:36:50 mysqld    22935  W 512    14166     0.00 ib_logfile1
21:36:50 mysqld    22935  S 0       0        3.21 ib_logfile1
21:36:50 mysqld    22935  W 1746   21714     0.02 binlog.000026
21:36:50 mysqld    22935  S 0       0        5.56 ibdata1
21:36:50 mysqld    22935  W 16384  4640      0.01 undo_001
21:36:50 mysqld    22935  W 16384  11504     0.01 sbtest1.ibd
21:36:50 mysqld    22935  W 16384  13248     0.01 sbtest1.ibd
21:36:50 mysqld    22935  W 16384  11808     0.01 sbtest1.ibd
21:36:50 mysqld    22935  W 16384  1328      0.01 undo_001
21:36:50 mysqld    22935  W 16384  6768      0.01 undo_002
[...]
```

В этом примере видна закономерность: mysqld выполняет синхронизацию после записи в файлы.

При трассировке всех операций можно получить большой объем вывода с соответствующим оверхедом. Я выполняю такую трассировку только короткое время (например, не дольше 10 с), чтобы понять закономерности работы файловой системы, которые не видны в выводе других инструментов (ext4dist(8)).

В числе поддерживаемых параметров можно назвать `-p PID` для трассировки только одного процесса и `-j` для вывода результатов в формате CSV.

8.6.15. bpftrace

bpftrace — это трассировщик на основе BPF, который предоставляет язык программирования высокого уровня, позволяющий создавать мощные однострочные и короткие сценарии. Он хорошо подходит для анализа работы файловой системы на основе подсказок, полученных с помощью других инструментов.

Подробнее о bpftrace я рассказываю в главе 15. В этом разделе показаны некоторые примеры анализа файловой системы: однострочные сценарии и приемы трассировки системных вызовов, VFS и внутренних механизмов файловой системы.

Однострочные сценарии

Однострочные сценарии ниже показывают различные возможности bpftrace.

Трассировка операций открытия файлов через вызов `openat(2)` с именами процессов:

```
bpftrace -e 't:syscalls:sys_enter_openat { printf("%s %s\n", comm, str(args->filename)); }'
```

Подсчет обращений к системным вызовам из семейства `read`:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_*read* { @[probe] = count(); }'
```

Подсчет обращений к системным вызовам из семейства `write`:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_*write* { @[probe] = count(); }'
```

Выводит гистограмму с распределением вызовов `read()` по запрошенным размерам блоков:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_read { @ = hist(args->count); }'
```

Выводит гистограмму с распределением вызовов `read()` по размерам прочитанных блоков (и кодам ошибок):

```
bpftrace -e 'tracepoint:syscalls:sys_exit_read { @ = hist(args->ret); }'
```

Подсчитывает количество вызовов `read()`, завершившихся ошибкой, для каждого кода ошибки:

```
bpftrace -e 't:syscalls:sys_exit_read /args->ret < 0/ { @[- args->ret] = count(); }'
```

Подсчитывает количество вызовов VFS:

```
bpftrace -e 'kprobe:vfs_* { @[probe] = count(); }'
```

Подсчитывает количество вызовов VFS для процесса с PID 181:

```
bpftrace -e 'kprobe:vfs_* /pid == 181/ { @[probe] = count(); }'
```

Подсчитывает количество прохождений через точки трассировки в `ext4`:

```
bpftrace -e 'tracepoint:ext4:* { @[probe] = count(); }'
```

Подсчитывает количество прохождений через точки трассировки в `xfs`:

```
bpftrace -e 'tracepoint:xfs:* { @[probe] = count(); }'
```

Подсчитывает число операций чтения файлов в файловой системе в ext4 по именам процессов и трассировкам стека в пространстве пользователя:

```
bpfftrace -e 'kprobe:ext4_file_read_iter { @[ustack, comm] = count(); }'
```

Определяет время выполнения `sra_sync()` в ZFS:

```
bpfftrace -e 'kprobe:sra_sync { time("%H:%M:%S ZFS sra_sync()\n"); }'
```

Подсчитывает число обращений к кэшу каталогов по именам и идентификаторам (PID) процессов:

```
bpfftrace -e 'kprobe:lookup_fast { @[comm, pid] = count(); }'
```

Трассировка системных вызовов

Системные вызовы — отличная цель для многих инструментов трассировки. Но некоторым системным вызовам не хватает контекста файловой системы, что затрудняет их анализ. Я покажу примеры того, что возможно осуществить с предлагаемыми средствами (трассировка `openat(2)`), а что может вызвать затруднения (трассировка `read(2)`).

openat(2)

Выполняя трассировку семейства системных вызовов `open(2)`, можно узнать, какие файлы открываются. Сейчас наиболее часто используется вариант `openat(2)`. Вот пример его трассировки:

```
# bpfftrace -e 't:syscalls:sys_enter_openat { printf("%s %s\n", comm,
    str(args->filename)); }'
Attaching 1 probe...
sal /etc/sysstat/sysstat
sadc /etc/ld.so.cache
sadc /lib/x86_64-linux-gnu/libsensors.so.5
sadc /lib/x86_64-linux-gnu/libc.so.6
sadc /lib/x86_64-linux-gnu/libm.so.6
sadc /sys/class/i2c-adapter
sadc /sys/bus/i2c/devices
sadc /sys/class/hwmon
sadc /etc/sensors3.conf
[...]
```

Здесь мне удалось поймать запуск `sar(1)` для архивирования статистик, и теперь я могу видеть, какие файлы он открывал. Этот сценарий использует аргумент `filename` точки трассировки. Все доступные аргументы можно получить с помощью параметров `-lv`:

```
# bpfftrace -lv t:syscalls:sys_enter_openat
tracepoint:syscalls:sys_enter_openat
    int __syscall_nr;
    int dfd;
```

```
const char * filename;
int flags;
umode_t mode;
```

Как видите, эта точка трассировки получает в аргументах номер системного вызова, дескриптор файла, имя файла, флаги открытия и режим открытия. Этой информации достаточно для однострочных сценариев и инструментов, например `opensnoop(8)`.

read(2)

`read(2)` мог бы быть полезной целью трассировки для понимания задержек операций чтения в файловой системе. Но взгляните на аргументы точки трассировки (сможете ли вы определить проблему?):

```
# bpftrace -lv t:syscalls:sys_enter_read
tracepoint:syscalls:sys_enter_read
    int __syscall_nr;
    unsigned int fd;
    char * buf;
    size_t count;
```

`read(2)` может вызываться для выполнения операций с файловыми системами, сокетами, `/proc` и другими целями, но имеющиеся аргументы не позволяют различить их. Чтобы понять, какие это вызывает сложности, взгляните на результаты сценария, подсчитывающего обращения к системному вызову `read(2)` по именам процессов:

```
# bpftrace -e 't:syscalls:sys_enter_read { @[comm] = count(); }'
Attaching 1 probe...
^C
```

```
@[systemd-journal]: 13
@[sshd]: 141
@[java]: 3472
```

В период трассировки Java-процесс 3472 раза обратился к системному вызову `read(2)`, но с какой целью — для чтения данных из файловой системы, сокета или откуда-то еще? (Процесс `sshd`, вероятнее всего, читал данные из сокета.)

При трассировке системного вызова `read(2)` можно узнать дескриптор файла (FD), представленный целым числом, но это всего лишь число, которое не отражает тип FD (`bpftrace` работает в ограниченном режиме ядра и не может выполнить поиск информации о FD в `/proc`). У этой проблемы минимум четыре решения:

- Вывести значения PID и FD в сценарии `bpftrace`, а затем отыскать FD с помощью `lsof(8)` или `/proc`, чтобы узнать, что он представляет.
- Вспомогательная функция `get_fd_path()` в BPF, внедрение которой ожидается в ближайшее время, может возвращать путь к файлу, соответствующему дескриптору FD. Это поможет отличить операции чтения из файловой системы (имеющие путь) от других типов чтения.


```

[2, 4)          57 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[4, 8)          53 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[8, 16)         86 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[16, 32)        2 | |
[...]

@us[proc]:
[0]             242 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[1]             41 | @@@@@@@@@@ |
[2, 4)          40 | @@@@@@@@@@ |
[4, 8)          61 | @@@@@@@@@@ |
[8, 16)         44 | @@@@@@@@@@ |
[16, 32)        40 | @@@@@@@@@@ |
[32, 64)        6 | @ |
[64, 128)       3 | |

@us[ext4]:
[0]             653 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[1]             447 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[2, 4)          70 | @@@@ |
[4, 8)          774 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[8, 16)         417 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[16, 32)        25 | @ |
[32, 64)        7 | |
[64, 128)       170 | @@@@@@@@@@@@@@ |
[128, 256)      55 | @@@ |
[256, 512)     59 | @@@ |
[512, 1K)     118 | @@@@@@@@@@ |
[1K, 2K)       3 | @@ |

```

Вывод (усеченный) также включает гистограммы распределения задержек для: sysfs, devpts, pipefs, devtmpfs, tmpfs и anon_inofs.

Вот исходный код сценария:

```

#!/usr/local/bin/bpftrace
#include <linux/fs.h>

BEGIN
{
    printf("Tracing vfs_read() by type... Hit Ctrl-C to end.\n");
}

kprobe:vfs_read
{
    @file[tid] = ((struct file *)arg0)->f_inode->i_sb->s_type->name;
    @ts[tid] = nsecs;
}

kretprobe:vfs_read
/@ts[tid]/
{
    @us[str(@file[tid])] = hist((nsecs - @ts[tid]) / 1000);
    delete(@file[tid]); delete(@ts[tid]);
}

```

```
END
{
    clear(@file); clear(@ts);
}
```

При желании этот инструмент можно расширить, включив в него другие операции — `vfs_readv()`, `vfs_write()`, `vfs_writev()` и т. д. Чтобы понять этот код, прочитайте раздел 15.2.4 «Программирование», в котором объясняются основы измерения задержек в `vfs_read()`.

Обратите внимание, что эта задержка может и не влиять напрямую на производительность приложения, как говорилось в разделе 8.3.1 «Задержки в файловой системе». Многое зависит от того, чем обусловлена задержка — запросом от приложения или работой асинхронной фоновой задачи. Чтобы ответить на этот вопрос, можно добавить в гистограмму дополнительный ключ — трассировку стека в пространстве пользователя (`ustack`), который может показать, был ли вызов `vfs_read()` произведен самим приложением.

Внутренние механизмы файловой системы

При необходимости можно разработать собственные инструменты, показывающие поведение внутренних компонентов файловой системы. Прежде всего проверьте имеющиеся точки трассировки. Вот как получить список точек трассировки для `ext4`:

```
# bpftrace -l 'tracepoint:ext4:*'
tracepoint:ext4:ext4_other_inode_update_time
tracepoint:ext4:ext4_free_inode
tracepoint:ext4:ext4_request_inode
tracepoint:ext4:ext4_allocate_inode
tracepoint:ext4:ext4_evict_inode
tracepoint:ext4:ext4_drop_inode
[...]
```

Каждая из них имеет свои аргументы, список которых можно получить с помощью параметров `-lv`. Если точек трассировки недостаточно (или они недоступны для целевой файловой системы), воспользуйтесь динамической инструментацией с помощью `kprobes`. Вот список зондов `kprobe` для `ext4`:

```
# bpftrace -lv 'kprobe:ext4_*'
kprobe:ext4_has_free_clusters
kprobe:ext4_validate_block_bitmap
kprobe:ext4_get_group_number
kprobe:ext4_get_group_no_and_offset
kprobe:ext4_get_group_desc
kprobe:ext4_wait_block_bitmap
[...]
```

В этом ядре (5.3) файловая система `ext4` имеет 105 точек трассировки и 538 зондов `kprobe`.

8.6.17. Другие инструменты

В табл. 8.7 перечислены инструменты наблюдения за работой файловой системы, представленные в других главах этой книги и в «BPF Performance Tools» [Gregg 19].

Таблица 8.7. Другие инструменты наблюдения за работой файловой системы

Раздел	Инструмент	Описание
5.5.6	syscount	Подсчитывает обращения к системным вызовам, включая связанные с файловыми системами
[Gregg 19]	statsnoop	Трассирует вызовы вариантов функции stat(2)
[Gregg 19]	syncsnoop	Трассирует вызовы вариантов функции sync(2) с отметками времени
[Gregg 19]	mmapfiles	Подсчитывает вызовы mmap(2) для файлов
[Gregg 19]	scread	Подсчитывает вызовы read(2) для файлов
[Gregg 19]	fmapfault	Подсчитывает сбои отображения файлов
[Gregg 19]	filelife	Трассирует короткоживущие файлы и определяет продолжительность их существования в секундах
[Gregg 19]	vfsstat	Возвращает статистики, характеризующие работу VFS
[Gregg 19]	vfscount	Подсчитывает все операции с VFS
[Gregg 19]	vfssize	Выводит распределение объемов данных в операциях чтения/записи
[Gregg 19]	fsrwstat	Подсчитывает операции чтения/записи по типам файловых систем
[Gregg 19]	fileslower	Определяет медленные операции чтения/записи
[Gregg 19]	filetype	Подсчитывает операции чтения/записи по типам файлов и процессам
[Gregg 19]	ioprofile	Подсчитывает трассировки стека для операций ввода/вывода
[Gregg 19]	writesync	Подсчитывает операции записи с флагом sync в обычные файлы
[Gregg 19]	writeback	Выводит события отложенной записи и задержки
[Gregg 19]	dcstat	Выводит статистику попаданий в кэш каталогов
[Gregg 19]	dcsnoop	Трассирует операции поиска в кэше каталогов
[Gregg 19]	mountsnoop	Трассирует события монтирования и размонтирования файловых систем в системе в целом
[Gregg 19]	icstat	Выводит статистику попаданий в кэш индексных узлов
[Gregg 19]	bufgrow	Выводит события увеличения буферов в байтах по процессам
[Gregg 19]	readahead	Показывает попадания в кэш после операций опережающего чтения и их эффективность

Другие инструменты для анализа файловых систем в Linux:

- **df(1)**: сообщает статистики потребления и мощности файловой системы;
- **inotify**: фреймворк для мониторинга событий файловой системы в Linux.

Некоторые типы файловых систем имеют собственные инструменты анализа производительности, дополняющие набор инструментов, которые есть в ОС. Одна из таких файловых систем — ZFS.

ZFS

В состав ZFS входит инструмент `zpool(1M)`, имеющий подкоманду `iostat`. Она возвращает статистики пула ZFS, в том числе частоту операций (чтение и запись) и пропускную способность пула.

Популярное дополнение — инструмент `arcstat.pl`, который сообщает размер ARC и L2ARC, а также частоту попаданий и промахов. Например:

```
$ arcstat 1
      time read miss miss% dmis dm% pmis pm% mmis mm% arcsz  c
04:45:47    0    0    0    0    0    0    0    0    0    14G  14G
04:45:49  15K   10    0   10    0    0    0    1    0   14G  14G
04:45:50  23K   81    0   81    0    0    0    1    0   14G  14G
04:45:51  65K   25    0   25    0    0    0    4    0   14G  14G
[...]
```

Статистики относятся к текущему интервалу:

- **read, miss**: общее количество обращений к первичному кэшу (ARC) и промахов;
- **miss%, dm%, pm%, mm%**: общий процент промахов ARC, процент промахов данных, предварительной выборки и метаданных;
- **dmis, pmis, mmis**: промахи данных, предварительной выборки и метаданных;
- **arcsz, c**: размер первичного кэша (ARC), целевой размер первичного кэша.

`arcstat.pl` — это программа на языке Perl, которая читает статистики из `kstat`.

8.6.18. Визуализация

Изменение нагрузки, приложенной к файловым системам, можно изобразить в виде линейного графика. Такой график помогает выявить временные закономерности изменения нагрузки. Иногда полезно строить отдельные графики для разных операций с файловой системой — чтение, запись и т. д.

Распределение задержек в файловой системе обычно имеет бимодальный вид: одна мода с низкой задержкой соответствует попаданиям в кэш файловой системы, а другая — с высокой задержкой — промахам кэша (когда выполняется чтение с устройства хранения). По этой причине представление распределения единственным значением, таким как среднее, мода или медиана, может вводить в заблуждение.

Одно из решений этой проблемы — использовать визуализацию, показывающую полное распределение, например тепловую карту. Тепловые карты были представлены в главе 2 «Методологии» в разделе 2.10.3 «Тепловые карты». На рис. 8.13

показан пример тепловой карты задержек в файловой системе: здесь течение времени отображается по оси X, а задержка ввода/вывода — по оси Y [Gregg 09a].

Эта тепловая карта показывает разницу в величине задержек до и после включения устройства L2ARC для NFSv3. Устройство L2ARC — это вторичный кэш ZFS, обычно основанный на флеш-памяти (об этом шла речь в разделе 8.3.2 «Кэширование»). Система, представленная на рис. 8.13, имеет 128 Гбайт DRAM и устройство L2ARC (твердотельный диск SSD с оптимизацией чтения) емкостью 600 Гбайт. Левая половина тепловой карты показывает распределение задержек в отсутствие устройства L2ARC (L2ARC был отключен), а правая половина — распределение задержек с включенным устройством L2ARC.

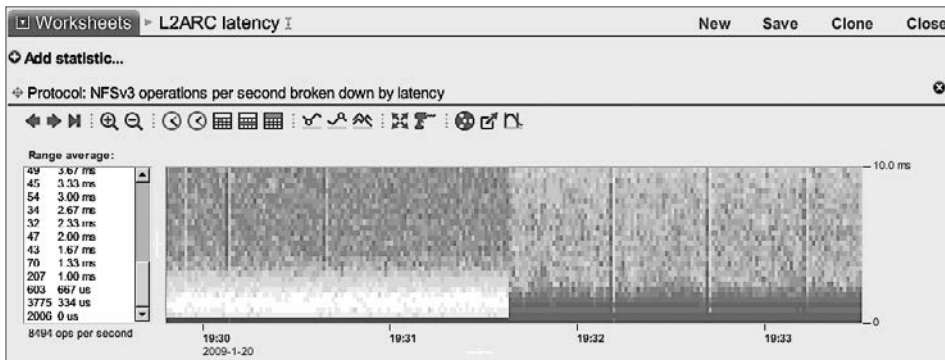


Рис. 8.13. Тепловая карта задержек в файловой системе

На тепловой карте слева четко просматриваются две области — низких и высоких задержек, — разделенные промежутком. Низкие задержки — это синяя полоса внизу около 0 мс. Вероятнее всего, они обусловлены попаданием в кэш в основной памяти. Высокие задержки начинаются примерно с 3 мс и простираются до самого верха в виде «облака». Они, скорее всего, обусловлены задержками в устройствах с вращающимися дисками. Такое двухмодальное распределение задержек типично для файловых систем на вращающихся дисках.

Картина справа соответствует распределению задержек с включенным устройством L2ARC. Теперь задержки часто оказываются ниже 3 мс, и количество задержек, обусловленных вращающимися дисками, значительно меньше. Как видите, задержки в L2ARC заполнили диапазон, где раньше был промежуток, и задержки в файловой системе в целом уменьшились.

8.7. ЭКСПЕРИМЕНТЫ

В этом разделе описаны инструменты для активного тестирования производительности файловой системы. Дополнительную информацию см. в разделе 8.5.8 «Микробенчмаркинг», где представлена предлагаемая стратегия тестирования.

При использовании представленных ниже инструментов рекомендуется оставить постоянно работающим `iostat` (1). Это позволит убедиться, что рабочая нагрузка достигает или не достигает дисков в соответствии с ожиданиями. Например, при тестировании с рабочим набором, размер которого позволяет ему уместиться в кэше файловой системы, ожидается, что 100 % операций чтения будут попадать в кэш. Поэтому `iostat(1)` не должен показывать значительный объем дискового ввода/вывода. Подробнее о `iostat(1)` идет речь в главе 9 «Диски».

8.7.1. Ad hoc

Команду `dd(1)` (копирует с устройства на устройство) используют для специальных тестов производительности последовательных операций с файловой системой. Команды ниже записывают, а затем читают файл с именем `file1` и размером в 1 Гбайт блоками по 1 Мбайт:

```
write: dd if=/dev/zero of=file1 bs=1024k count=1k
read: dd if=file1 of=/dev/null bs=1024k
```

Версия `dd(1)` в Linux по завершении выводит статистики. Например:

```
$ dd if=/dev/zero of=file1 bs=1024k count=1k
1024+0 records in
1024+0 records out
1073741824 bytes (1.1 GB, 1.0 GiB) copied, 0.76729 s, 1.4 GB/s
```

Здесь видно, что у файловой системы пропускная способность записи 1,4 Гбайт/с (используется кэширование с отложенной записью, поэтому фактическая запись на диск производится позже, в зависимости от параметров `vm.dirty_*`: см. главу 7 «Память», раздел 7.6.1 «Настраиваемые параметры»).

8.7.2. Инструменты микробенчмаркинга

Для тестирования файловой системы есть множество инструментов, включая `Bonnie`, `Bonnie++`, `iozone`, `tiobench`, `SysBench`, `fiio` и `FileBench`. Некоторые из них приводятся ниже в порядке возрастания сложности. См. также главу 12 «Бенчмаркинг». Я рекомендую использовать `fiio`.

Bonnie, Bonnie++

`Bonnie` — это простая программа, написанная на языке C и предназначенная для проверки нескольких рабочих нагрузок с одним файлом из одного потока. Она была написана Тимом Бреем (Tim Bray) в 1989 году [Bray 90]. Ее можно запускать без аргументов (в этом случае будут использоваться значения по умолчанию):

```
$ ./Bonnie
File './Bonnie.9598', size: 104857600
[...]
```

-----Sequential Output----- ---Sequential Input-- --Random--

```

      -Per Char- --Block--- -Rewrite-- -Per Char- --Block--- --Seeks---
Machine    MB K/sec %CPU K/sec %CPU K/sec %CPU K/sec %CPU K/sec %CPU /sec %CPU
      100 123396 100.0 1258402 100.0 996583 100.0 126781 100.0 2187052 100.0
164190.1 299.0

```

Среди прочих значений выводится величина потребления процессора в каждом тесте: 100 % показывает, что программа Bonnie не блокировалась дисковым вводом/выводом и всегда оставалась на процессоре, получая данные из кэша. Причина в том, что размер целевого файла в этом примере составляет 100 Мбайт и полностью уместается в кэше в этой системе. Размер файла можно изменить, передав параметр `-s <size>`.

Есть 64-разрядная версия Bonnie-64, которая позволяет тестировать файлы большего размера. Есть версия Bonnie++, переписанная на C++ Расселом Кокером (Russell Coker) [Coker 01].

К сожалению, инструменты бенчмаркинга файловых систем, такие как Bonnie, могут приводить в замешательство, если вы не понимаете, что они тестируют. Первый результат, тест `putc(3)`, зависит от реализации системной библиотеки, и именно она становится предметом тестирования, а не файловая система. См. пример в главе 12 «Бенчмаркинг», в разделе 12.3.2 «Активный бенчмаркинг».

fio

fio (Flexible IO Tester) Йенса Аксбо (Jens Axboe) — это гибкий инструмент для бенчмаркинга файловых систем, обладающий множеством дополнительных возможностей [Axboe 20]. Вот две из них — именно они побудили меня использовать его вместо других:

- **неоднородные случайные распределения**, которые более точно моделируют реальные нагрузки (например, `-random_distribution=pareto:0.9`);
- **вывод информации о распределении задержек в процентилях**, включая 99,00; 99,50; 99,90; 99,95; 99,99.

Вот результаты имитации рабочей нагрузки произвольного чтения с размером блока ввода/вывода 8 Кбайт, размером рабочего набора 5 Гбайт и неоднородной закономерностью доступа (`pareto:0.9`):

```

# fio --runtime=60 --time_based --clocksource=clock_gettime --name=randread --
numjobs=1 --rw=randread --random_distribution=pareto:0.9 --bs=8k --size=5g --
filename=fio.tmp
randread: (g=0): rw=randread, bs=8K-8K/8K-8K/8K-8K, ioengine=sync, iodepth=1
fio-2.0.13-97-gdd8d
Starting 1 process
Jobs: 1 (f=1): [r] [100.0% done] [3208K/0K/0K /s] [401 /0 /0 iops] [eta 00m:00s]
randread: (groupid=0, jobs=1): err= 0: pid=2864: Tue Feb 5 00:13:17 2013
  read: io=247408KB, bw=4122.2KB/s, iops=515 , runt= 60007msec
    clat (usec): min=3 , max=67928 , avg=1933.15, stdev=4383.30
    lat (usec): min=4 , max=67929 , avg=1934.40, stdev=4383.31
    clat percentiles (usec):

```

```

| 1.00th=[ 5], 5.00th=[ 5], 10.00th=[ 5], 20.00th=[ 6],
| 30.00th=[ 6], 40.00th=[ 6], 50.00th=[ 7], 60.00th=[ 620],
| 70.00th=[ 692], 80.00th=[ 1688], 90.00th=[ 7648], 95.00th=[10304],
| 99.00th=[19584], 99.50th=[24960], 99.90th=[39680], 99.95th=[51456],
| 99.99th=[63744]
bw (KB/s) : min= 1663, max=71232, per=99.87%, avg=4116.58, stdev=6504.45
lat (usec) : 4=0.01%, 10=55.62%, 20=1.27%, 50=0.28%, 100=0.13%
lat (usec) : 500=0.01%, 750=15.21%, 1000=4.15%
lat (msec) : 2=3.72%, 4=2.57%, 10=11.50%, 20=4.57%, 50=0.92%
lat (msec) : 100=0.05%
cpu       : usr=0.18%, sys=1.39%, ctx=13260, majf=0, minf=42
IO depths  : 1=100.0%, 2=0.0%, 4=0.0%, 8=0.0%, 16=0.0%, 32=0.0%, >=64=0.0%
submit     : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
complete   : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
issued     : total=r=30926/w=0/d=0, short=r=0/w=0/d=0

```

Процентили задержек (`clat`) показывают очень низкие задержки вплоть до 50-го перцентиля, которые, если судить по их величине (от 5 до 7 мс), обусловлены попаданиями в кэш. Остальные процентили показывают эффект промахов кэша, включая хвост очереди. В этом случае 99,99-й перцентиль показывает задержку 63 мс.

Только одних перцентилей недостаточно, чтобы понять, что распределение задержек многомодальное. Но при этом они показывают не менее интересный аспект: хвост более медленной моды (дисковый ввод/вывод).

В качестве альтернативы можно использовать похожий, но более простой инструмент SysBench (пример использования SysBench для анализа потребления процессора представлен в главе 6 в разделе 6.8.2 «SysBench»). Но если вам нужен еще больший контроль, попробуйте FileBench.

FileBench

FileBench — это инструмент для бенчмаркинга файловых систем, позволяющий моделировать рабочие нагрузки, описывая их на языке моделирования рабочих нагрузок Workload Model Language. С помощью FileBench можно моделировать разные шаблоны поведения потоков выполнения, в том числе и синхронный. Вместе с инструментом распространяется множество конфигураций, называемых *характерами* (personalities), в том числе для имитации модели ввода/вывода в базе данных Oracle. К сожалению, FileBench сложен для изучения и использования и может быть интересен только тем, кто работает с файловыми системами на постоянной основе.

8.7.3. Очистка кэша

Linux предоставляет возможность очищать кэши файловой системы (удалять записи), что может пригодиться для бенчмаркинга из согласованного и «холодного» состояния кэша, как бывает после загрузки системы. В исходном коде ядра этот механизм описан просто (Documentation/sysctl/vm.txt):

Чтобы очистить кэш страниц:

```
echo 1 > /proc/sys/vm/drop_caches
```

Чтобы очистить кэши объектов в ядре (в том числе кэши каталогов и индексных узлов):

```
echo 2 > /proc/sys/vm/drop_caches
```

Чтобы очистить кэши страниц и объектов в ядре:

```
echo 3 > /proc/sys/vm/drop_caches
```

Иногда полезно очистить все кэши (3) перед запуском других бенчмарков, чтобы бенчмаркинг начинался с известного состояния (холодный кэш). Это поможет обеспечить согласованность результатов бенчмаркинга.

8.8. НАСТРОЙКА

В разделе 8.5 «Методология» мы рассмотрели множество подходов к настройке, включая настройку кэша и определение характеристик рабочей нагрузки. В последнем случае, выявляя и устраняя ненужную работу, можно добиться наивысших результатов. В этот раздел включено описание конкретных параметров, доступных для настройки.

Конкретные настройки — доступные параметры и оптимальные значения — зависят от версии ОС и предполагаемой рабочей нагрузки. В следующих разделах, организованных по типам настроек, приводятся примеры доступных настраиваемых параметров и описываются причины, почему может потребоваться их настройка. В них я представляю функции, доступные из приложений, и два примера для файловых систем ext4 и ZFS. Описание настройки кэша страниц вы найдете в главе 7 «Память».

8.8.1. Функции

В разделе 8.3.7 «Синхронная запись» говорилось о том, как можно повысить производительность операций синхронной записи, используя `fsync(2)` для сбрасывания на диск данных группами, а не по отдельности, с флагами `O_DSYNC/O_RSYNC` при обращении к системному вызову `open(2)`.

Ниже перечислены другие функции, вызов которых повышает производительность, — `posix_fadvise()` и `madvise(2)`. Они предоставляют файловой системе подсказки по требованиям к кэшированию.

posix_fadvise()

Эта библиотечная функция (обертка вокруг системного вызова `fadvise64(2)`) работает с областью файла и имеет сигнатуру:

```
int posix_fadvise(int fd, off_t offset, off_t len, int advice);
```

Флаги, которые можно передать в аргументе `advice`, перечислены в табл. 8.8.

Таблица 8.8. Флаги для аргумента `advise` функции `posix_fadvise()` в ядре Linux

Флаг	Описание
<code>POSIX_FADV_SEQUENTIAL</code>	Доступ к указанному диапазону данных будет последовательным
<code>POSIX_FADV_RANDOM</code>	Доступ к указанному диапазону данных будет произвольным
<code>POSIX_FADV_NOREUSE</code>	Данные не будут использоваться повторно
<code>POSIX_FADV_WILLNEED</code>	Данные будут использоваться повторно в ближайшем будущем
<code>POSIX_FADV_DONTNEED</code>	Данные не будут использоваться повторно в ближайшем будущем

Ядро может использовать эту информацию для повышения производительности. Она помогает ему решить, когда стоит производить предварительную выборку данных и когда есть смысл кэшировать данные. Эти подсказки могут повысить коэффициент попадания в кэш для данных с высоким приоритетом. Полный список флагов для аргумента `advise` см. на странице справочного руководства `man` для вашей системы.

madvise()

Этот системный вызов работает с файлами, отображенными в память, и имеет такую сигнатуру:

```
int madvise(void *addr, size_t length, int advice);
```

Флаги, которые можно передать в аргументе `advise`, перечислены в табл. 8.9.

Таблица 8.9. Флаги для аргумента `advise` системного вызова `madvise(2)` в ядре Linux

Флаг	Описание
<code>MADV_RANDOM</code>	Доступ к данным в файле будет произвольным
<code>MADV_SEQUENTIAL</code>	Доступ к данным в файле будет последовательным
<code>MADV_WILLNEED</code>	Данные будут использоваться повторно (кэширование желательно)
<code>MADV_DONTNEED</code>	Данные не будут использоваться повторно (кэширование нежелательно)

Как и в случае с `posix_fadvise()`, ядро может использовать эту информацию для увеличения производительности, в том числе для принятия более эффективных решений по кэшированию.

8.8.2. ext4

Файловые системы `ext2`, `ext3` и `ext4` в Linux можно настроить с помощью:

- параметров монтирования;
- команды `tune2fs(8)`;

- файлов свойств /sys/fs/ext4;
- команды `e2fsck(8)`.

Параметры монтирования и команда `tune2fs`

Параметры монтирования можно установить во время монтирования с помощью команды `mount(8)` или во время загрузки в `/boot/grub/menu.lst` и `/etc/fstab`. Список доступных параметров ищите на странице справочного руководства `man` для `mount(8)`. Вот несколько вариантов:

```
# man mount
[...]
ПАРАМЕТРЫ МОНТИРОВАНИЯ, НЕ ЗАВИСЯЩИЕ ОТ ТИПА ФАЙЛОВОЙ СИСТЕМЫ
[...]
    atime      Не использовать поддержку noatime. В этом случае поддержка
                обновления времени последнего доступа к индексным узлам
                управляется настройками по умолчанию в ядре. См. также описание
                параметров монтирования relatime и strictatime.

    noatime     Не обновлять время последнего доступа к индексным узлам в этой
                файловой системе (например, для более быстрого доступа к файлам
                на серверах новостей).

[...]
    relatime    Обновлять время последнего доступа к индексным узлам
                одновременно с обновлением времени последнего изменения. Время
                последнего доступа обновляется, только если предыдущее время
                доступа было раньше, чем текущее время последнего изменения.
                (Напоминает noatime, но не нарушает работу
                mutt и других приложений, которым необходимо знать, выполнялось
                ли чтение файла с момента последнего его изменения.)

                Начиная с Linux 2.6.30, поведение ядра по умолчанию соответствует
                этому параметру (если не указан параметр noatime), а для
                возврата к традиционной семантике следует использовать параметр
                strictatime. Кроме того, начиная с Linux 2.6.30, время последнего
                доступа к файлу всегда обновляется, если оно оказалось в прошлом
                больше чем на 1 день.1

[...]

```

Параметр `noatime` исторически использовался для повышения производительности за счет предотвращения обновления отметок времени последнего доступа и связанных с этим операций дискового ввода/вывода. На странице справочного руководства описано, что в настоящее время по умолчанию используется параметр `relatime`, который тоже уменьшает число этих обновлений.

На странице справочного руководства для `mount(8)` описаны не только общие параметры монтирования, но и параметры монтирования, характерные для файловой системы. Но для параметров монтирования файловой системы `ext4` есть своя страница справочного руководства `ext4(5)`:

¹ Это перевод текста из справочного руководства. — *Примеч. пер.*

```
# man ext4
[...]
Параметры монтирования для ext4
[...]
    Параметры journal_dev, journal_path, norecovery, noload, data,
    commit, orlov, oldalloc, [no]user_xattr, [no]acl, bsddf, minixdf, debug,
    errors, data_err, grpid, bsdgroups, nogrpid, sysvgroups, resgid, resuid,
    sb, quota, noquota, nouid32, grpquota, usrquota, usrjquota,
    grpjquota и jqfmt обратно совместимы с ext3 или ext2.

    journal_checksum | nojournal_checksum
        Параметр journal_checksum включает подсчет контрольных сумм
        транзакций с журналом. Это обеспечивает поддержку кода
        восстановления в e2fsck и...1
[...]

```

Текущие настройки монтирования можно узнать с помощью `tune2fs -l device` и `mount` (без параметров). `tune2fs (8)` может устанавливать или сбрасывать различные параметры монтирования, как описано на соответствующей странице справочного руководства `man`.

Чаще всего для повышения производительности используется параметр монтирования `noatime`: он позволяет отключить обновление времени последнего доступа к файлам, если оно не требуется пользователям файловой системы, и за счет этого сократить количество внутренних операций ввода/вывода.

Файлы свойств /sys/fs

Некоторые дополнительные параметры можно установить в реальном времени через файловую систему `/sys`. Вот пример для `ext4`:

```
# cd /sys/fs/ext4/nvme0n1p1

# ls
delayed_allocation_blocks  last_error_time          msg_ratelimit_burst
err_ratelimit_burst       lifetime_write_kbytes    msg_ratelimit_interval_ms
err_ratelimit_interval_ms max_writeback_mb_bump    reserved_clusters
errors_count              mb_group_prealloc        session_write_kbytes
extent_max_zeroout_kb     mb_max_to_scan           trigger_fs_error
first_error_time          mb_min_to_scan           warning_ratelimit_burst
inode_goal                mb_order2_req            warning_ratelimit_interval_ms
inode_readahead_blks     mb_stats
journal_task              mb_stream_req
# cat inode_readahead_blks
32

```

Как показывает этот вывод, `ext4` будет читать не более 32 блоков из таблицы индексных узлов. Не все эти файлы позволяют изменять их: некоторые доступны только для чтения. Они описаны в исходном коде Linux в файле `Documentation/admin-guide/ext4.rst` [Linux 20h], где также приведены параметры монтирования.

¹ Это перевод текста из справочного руководства. — *Примеч. пер.*

e2fsck

Команда `e2fsck(8)` поддерживает возможность переиндексации каталогов в файловой системе `ext4`, что помогает повысить производительность. Например:

```
e2fsck -D -f /dev/hdX
```

Другие параметры `e2fsck(8)` связаны с проверкой и восстановлением файловой системы.

8.8.3. ZFS

ZFS поддерживает большое количество настраиваемых параметров (называемых *свойствами*) для каждой файловой системы и несколько параметров — для системы в целом. Список параметров можно получить командой `zfs(1)`. Например:

```
# zfs get all zones/var
NAME          PROPERTY      VALUE          SOURCE
[...]
zones/var     recordsize    128K           default
zones/var     mountpoint    legacy         local
zones/var     sharenfs      off            default
zones/var     checksum      on             default
zones/var     compression   off            inherited from zones
zones/var     atime         off            inherited from zones
[...]
```

Вывод включает столбцы с именами свойств, текущими значениями и источниками. Источник сообщает, как тот или иной параметр был установлен: унаследован из набора данных ZFS более высокого уровня, выбрано значение по умолчанию или настроен локально для данной файловой системы.

Параметры также можно установить с помощью команды `zfs(1M)`, они описаны на ее странице справочного руководства `man`. Ключевые параметры, связанные с производительностью, перечислены в табл. 8.10.

Таблица 8.10. Ключевые настраиваемые параметры ZFS

Параметр	Варианты значений	Описание
<code>recordsize</code>	от 512 до 128 K	Желаемый размер блока для файлов
<code>compression</code>	<code>on</code> <code>off</code> <code>lzjb</code> <code>gzip</code> <code>gzip-[1–9]</code> <code>zle</code> <code>lz4</code>	Использование легковесных алгоритмов (например, <code>lzjb</code>) может улучшить производительность в некоторых ситуациях за счет уменьшения объема внутреннего ввода/вывода
<code>atime</code>	<code>on</code> <code>off</code>	Обновление времени последнего доступа (при включении будет вызывать операции записи после операций чтения)
<code>primarycache</code>	<code>all</code> <code>none</code> <code>metadata</code>	Политика первичного кэша ARC. Загрязнение кэша данными из файловых систем с низким приоритетом (например, архивов) можно уменьшить, установив значение «по» или «metadata»

Таблица 8.10 (окончание)

Параметр	Варианты значений	Описание
secondarycache	all none metadata	Политика вторичного кэша L2ARC
logbias	latency throughput	Подсказка для операций синхронной записи: «latency» означает использовать устройства журналов, а «throughput» — устройства пула
sync	standard always disabled	Поведение операций синхронной записи

Наиболее важный параметр для настройки — `recordsize` (размер записи). Желательно, чтобы его значение соответствовало характеру ввода/вывода в приложении. Обычно он имеет значение по умолчанию 128 Кбайт, что может быть неэффективно для небольших случайных операций ввода/вывода. Обратите внимание, что это не относится к файлам, размер которых меньше размера записи и которые сохраняются с использованием динамического размера записи, равного длине файла. Отключение `atime` тоже может улучшить производительность, если время последнего доступа не нужно.

ZFS также предоставляет общесистемные настройки, включая настройки времени (TXG) синхронизации группы транзакций (`zfs_txg_synctime_ms`, `zfs_txg_timeout`), а также порогового значения метаблоков для переключения между алгоритмами распределения, оптимизированными по памяти и по времени (`metaslab_df_free_pct`). Уменьшение TXG может улучшить производительность за счет уменьшения числа конфликтов и очередей с другими операциями ввода/вывода.

Обязательно ознакомьтесь с описанием настраиваемых параметров в документации, где можно найти исчерпывающий список, подробное описание и предупреждения.

8.9. УПРАЖНЕНИЯ

- Ответьте на следующие вопросы, касающиеся файловых систем:
 - В чем разница между логическим и физическим вводом/выводом?
 - В чем разница между произвольным и последовательным вводом/выводом?
 - Что такое прямой ввод/вывод?
 - Что такое неблокирующий ввод/вывод?
 - Что такое размер рабочего набора?
- Ответьте на следующие концептуальные вопросы:
 - Какую роль играет VFS?
 - Что такое задержка в файловой системе и как ее можно измерить?

- Какова цель предварительной выборки (упреждающего чтения)?
 - Какова цель прямого ввода/вывода?
3. Ответьте на следующие более сложные вопросы:
- Опишите преимущества `fsync(2)` перед `O_SYNC`.
 - Опишите достоинства и недостатки использования `mmap(2)` вместо `read(2)/write(2)`.
 - Опишите причины, почему объем логического ввода/вывода может быть *больше* объема физического ввода/вывода.
 - Опишите причины, почему объем логического ввода/вывода может быть *меньше* объема физического ввода/вывода.
 - Объясните, как использование алгоритма копирования при записи может повысить производительность файловой системы.
4. Разработайте следующие процедуры для своей среды:
- Чек-лист настройки кэша файловой системы. Он должен включать способы получения списка существующих кэшей файловой системы, определения их текущих размеров и потребления, а также коэффициентов попадания.
 - Чек-лист определения характеристик рабочей нагрузки на файловую систему. Он должен включать способы получения всех необходимых деталей, и в первую очередь с использованием инструментов ОС.
5. Решите следующие задачи:
- Выберите приложение и измерьте задержки в операциях с файловой системой, включая:
 - полное распределение задержек, а не только среднее значение;
 - какую часть каждой секунды каждый поток выполнения в приложении тратит на операции с файловой системой.
 - С помощью инструмента микробенчмаркинга определите экспериментально размер кэша файловой системы. Объясните свой выбор инструмента. Также покажите снижение производительности (с использованием любой метрики) в случаях, когда рабочий набор не уместается в кэше.
6. (опционально) Разработайте инструмент, позволяющий оценить производительность синхронной и асинхронной записи. Он должен измерять скорость операций и задержку, а также поддерживать возможность определения идентификаторов процессов, выполнивших эти операции; это позволит использовать инструмент для определения характеристик рабочей нагрузки.
7. (опционально) Разработайте инструмент для получения статистик косвенного и увеличенного ввода/вывода: количество дополнительных байтов и операций ввода/вывода, не инициированных приложениями непосредственно. Инструмент должен разбивать дополнительные операции ввода/вывода по типам, чтобы можно было объяснить их причину.

8.10. ССЫЛКИ

- [Ritchie 74] Ritchie, D. M., and Thompson, K., «The UNIX Time-Sharing System», Communications of the ACM 17, no. 7, pp. 365–75, July 1974.
- [Lions 77] Lions, J., A «Commentary on the Sixth Edition UNIX Operating System», University of New South Wales, 1977.
- [McKusick 84] McKusick, M. K., Joy, W. N., Leffler, S. J., and Fabry, R. S., «A Fast File System for UNIX». ACM Transactions on Computer Systems (TOCS) 2, no. 3, August 1984.
- [Bach 86] Bach, M. J., «The Design of the UNIX Operating System», Prentice Hall, 1986.
- [Bray 90] Bray, T., «Bonnie», <http://www.textuality.com/bonnie>, 1990.
- [Sweeney 96] Sweeney, A., «Scalability in the XFS File System», USENIX Annual Technical Conference, <https://www.cs.princeton.edu/courses/archive/fall09/cos518/papers/xfs.pdf>, 1996.
- [Vahalia 96] Vahalia, U., «UNIX Internals: The New Frontiers», Prentice Hall, 1996¹.
- [Coker 01] Coker, R., «bonnie++», <https://www.coker.com.au/bonnie++>, 2001.
- [Gregg 09a] Gregg, B., «L2ARC Screenshots», <http://www.brendangregg.com/blog/2009-01-30/l2arc-screenshots.html>, 2009.
- [Corbet 10] Corbet, J., «Dcache scalability and RCU-walk», LWN.net, <http://lwn.net/Articles/419811>, 2010.
- [Doeppner 10] Doeppner, T., «Operating Systems in Depth: Design and Programming», Wiley, 2010.
- [XFS 10] «Runtime Stats», https://xfs.org/index.php/Runtime_Stats, 2010.
- [Oracle 12] «ZFS Storage Pool Maintenance and Monitoring Practices», Oracle Solaris Administration: ZFS File Systems, https://docs.oracle.com/cd/E36784_01/html/E36835/storage-9.html, 2012.
- [Ahrens 19] Ahrens, M., «State of OpenZFS», OpenZFS Developer Summit 2019, https://drive.google.com/file/d/197jS8_MWtfdW2LyvIFnH58uUasHuNszz/view, 2019.
- [Axboe 19] Axboe, J., «Efficient IO with io_uring», https://kernel.dk/io_uring.pdf, 2019.
- [Gregg 19] Gregg, B., «BPF Performance Tools: Linux System and Application Observability»², Addison-Wesley, 2019.
- [Axboe 20] Axboe, J., «Flexible I/O Tester», <https://github.com/axboe/fio>, последнее обновление 2020.
- [Linux 20h] «ext4 General Information», Linux documentation, <https://www.kernel.org/doc/html/latest/admin-guide/ext4.html>, по состоянию на 2020.
- [Torvalds 20a] Torvalds, L., «Re: Do not blame anyone. Please give polite, constructive criticism», <https://www.realworldtech.com/forum/?threadid=189711&curpostid=189841>, 2020.

¹ Вахалия Ю. «UNIX изнутри». СПб., издательство «Питер».

² Грегг Б. «BPF: Профессиональная оценка производительности». Выходит в издательстве «Питер» в 2023 году.

Глава 9

ДИСКИ

Дисковый ввод/вывод может вызывать значительные задержки в работе приложений, поэтому он важная цель для анализа производительности. При высокой нагрузке диски становятся узким местом, в результате чего процессоры простаивают, пока система ожидает завершения дискового ввода/вывода. Выявление и устранение этих узких мест увеличивает производительность и пропускную способность приложений на порядки.

Термин *диски* относится к основным устройствам хранения в системе, в том числе к твердотельным накопителям (SSD) на основе флеш-памяти и магнитным вращающимся дискам. Твердотельные накопители изначально предназначались для повышения производительности дискового ввода/вывода, с чем они прекрасно справляются. Но требования к емкости, скорости ввода/вывода и пропускной способности тоже растут, и устройства на основе флеш-памяти не лишены своих проблем с производительностью.

Цели этой главы:

- представить модели дисков и основные понятия;
- показать, как модели доступа к диску влияют на производительность;
- описать опасности интерпретации потребления диска;
- познакомить с характеристиками и «внутренностями» дисковых устройств;
- познакомить с путями в коде ядра между файловыми системами и устройствами;
- познакомить с уровнями RAID и их производительностями;
- представить различные методологии анализа производительности дисков;
- охарактеризовать общесистемный и межпроцессный дисковый ввод/вывод;
- познакомить с приемами измерения задержек дискового ввода/вывода и выявления выбросов;
- показать, как определять пути в коде приложения, заканчивающиеся дисковым вводом/выводом;
- детально изучить дисковый ввод/вывод с помощью трассировщиков;
- познакомить с параметрами настройки дисков.

Глава состоит из шести частей. В первых трех обсуждаются основы анализа производительности дискового ввода/вывода, а в следующих трех показано их практическое применение в системах на базе Linux:

- **Основы** представляет терминологию, связанную с хранением данных, основные модели дисковых устройств и ключевые идеи производительности дисков.
- **Архитектура** описывает обобщенную аппаратную и программную архитектуру хранилища.
- **Методология** описывает методологии анализа эффективности, как наблюдательные, так и экспериментальные.
- **Инструменты наблюдения** описывает инструменты наблюдения за работой дисков в Linux, включая трассировку и визуализацию.
- **Экспериментирование** обобщает возможности инструментов тестирования дисков.
- **Настройка** описывает параметры настройки дисков.

В предыдущей главе рассматривалась производительность файловых систем, построенных на дисках, так что она заслуживает особого внимания для понимания производительности приложений.

9.1. ТЕРМИНОЛОГИЯ

Ниже даны основные термины этой главы, связанные с дисками:

- **Виртуальный диск:** имитация запоминающего устройства. Система видит его как один физический диск, но он может состоять из нескольких физических дисков или части диска.
- **Транспорт:** физическая шина, через которую происходит обмен информацией, включая передачу данных (ввод/вывод) и другие дисковые команды.
- **Сектор:** блок хранения на диске, обычно размером 512 байт, но сейчас секторы нередко имеют размер 4 Кбайт.
- **Ввод/вывод:** строго говоря, ввод/вывод подразумевает только чтение и запись на диск и не включает другие дисковые команды. Ввод/вывод можно описать как минимум направлением (чтение или запись), адресом на диске (местоположением) и размером (в байтах).
- **Дисковые команды:** диску можно передавать команды на выполнение других операций, не связанных с передачей данных (например, очистка кэша).
- **Пропускная способность (throughput):** под пропускной способностью дисков обычно понимается текущая скорость передачи данных, измеряемая в байтах в секунду.
- **Полоса пропускания (bandwidth):** максимально возможная скорость передачи данных для транспорта или контроллера. Ширина полосы пропускания ограничена аппаратными возможностями.

- **Задержка ввода/вывода:** время выполнения операции ввода/вывода от начала до конца. Более точно понятие времени определяется в разделе 9.3.1 «Измерение времени». Имейте в виду, что в сетевых взаимодействиях термин *задержка* обозначает время, необходимое для инициирования ввода/вывода и последующей передачи данных.
- **Выбросы задержки:** дисковый ввод/вывод с необычно высокой задержкой.

Глоссарий включает базовую терминологию для справки, в том числе: *диск, контроллер диска, дисковый массив, локальные диски, удаленные диски и IOPS (операций ввода/вывода в секунду)*. См. также разделы «Терминология» в главах 2 и 3.

9.2. МОДЕЛИ

Далее описываются простые модели, иллюстрирующие некоторые базовые принципы устройства дисков.

9.2.1. Простой диск

Современные диски включают в себя очередь для запросов ввода/вывода (рис. 9.1).

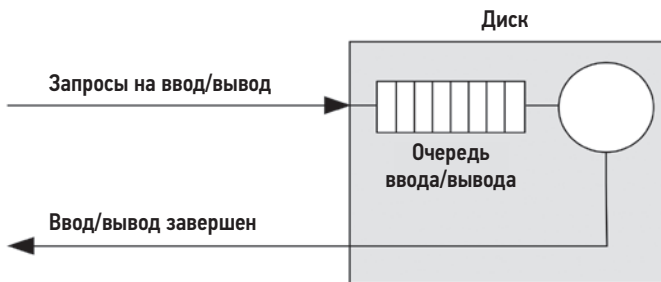


Рис. 9.1. Простой диск с очередью

Операции ввода/вывода, полученные диском, либо находятся в очереди, либо обслуживаются. Эта простая модель напоминает кассу в супермаркете, где клиенты выстраиваются в очередь для оплаты покупок. Модель хорошо подходит и для анализа с использованием теории массового обслуживания.

Часто такая очередь действует по принципу «первым пришел — первым обслужился», но дисковый контроллер может использовать также другие алгоритмы для оптимизации производительности. Эти алгоритмы могут учитывать время, необходимое для перемещения магнитных головок во вращающихся дисках (см. обсуждение в разделе 9.4.1 «Типы дисков») или организовывать отдельные очереди для операций чтения и записи (это особенно характерно для дисков на основе флеш-памяти).

9.2.2. Кэширующие диски

Наличие кэша в дисковых устройствах позволяет удовлетворять некоторые запросы на чтение из более быстрой памяти, как показано на рис. 9.2. Такой кэш может быть реализован с использованием микросхем памяти небольшого объема (ОЗУ) внутри физического дискового устройства.

При попадании в кэш данные возвращаются с очень низкой задержкой (что хорошо), но промахи все еще случаются достаточно часто и приводят к высокой задержке из-за необходимости чтения данных с физического диска.

Кэш в дисковом устройстве также может использоваться для увеличения производительности *записи* — он используется для хранения буферов *отложенной записи*. В таких случаях дисковое устройство сообщает о завершении записи сразу после сохранения данных в кэше, то есть до фактической их записи на диск. Иногда кэши используются в режиме *сквозной записи*, когда признак завершения записи возвращается только после полной передачи данных на следующий уровень.

На практике кэши отложенной записи часто комплектуются батареями, чтобы буферизованные данные можно было сохранить позже в случае сбоя питания. Такие батареи могут встраиваться в дисковое устройство или в контроллер.

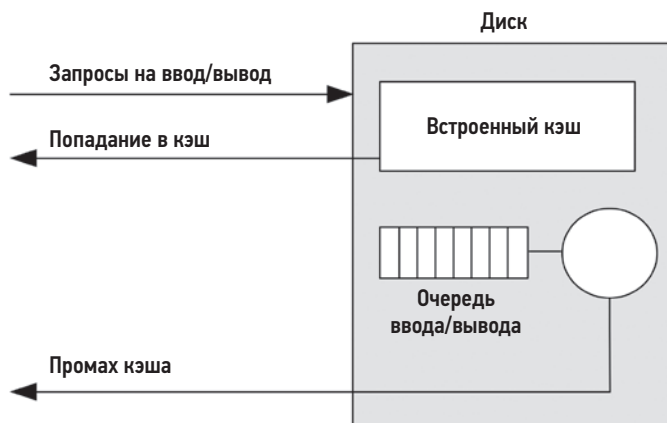


Рис. 9.2. Простой диск со встроенным кэшем

9.2.3. Контроллер

На рис. 9.3 показан простой контроллер диска, соединяющий транспорт ввода/вывода процессора с транспортом системы хранения и подключенными дисковыми устройствами. Контроллеры также называют *адаптерами главной шины* (host bus adapter, HBA).

Производительность может ограничиваться любой из этих шин, контроллером диска или самими дисками. Подробности о контроллерах дисков ищите в разделе 9.4 «Архитектура».

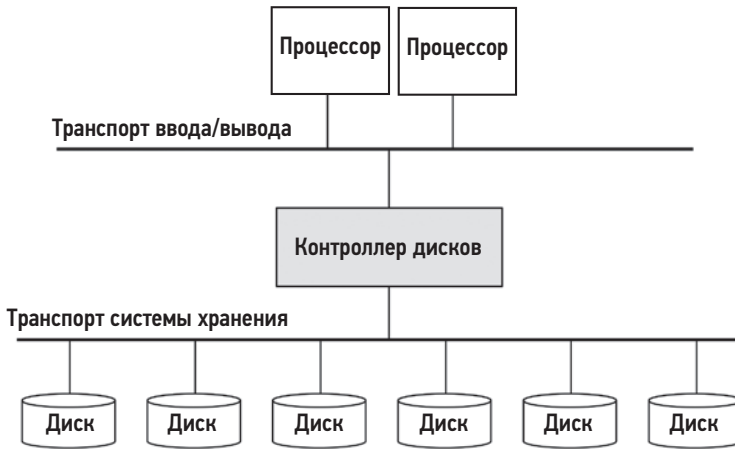


Рис. 9.3. Простой контроллер дисков с подключенными к нему транспортами

9.3. ОСНОВНЫЕ ПОНЯТИЯ

Ниже даны некоторые важные понятия, касающиеся дисков и их производительности.

9.3.1. Измерение времени

Время ввода/вывода можно измерить как:

- **время обработки запроса ввода/вывода** (также называется **временем отклика ввода/вывода**): все время от запуска ввода/вывода до его завершения;
- **время ожидания ввода/вывода**: время ожидания в очереди;
- **время обслуживания ввода/вывода**: время, в течение которого производился ввод/вывод (без ожидания).

Соответствующие интервалы изображены на рис. 9.4.

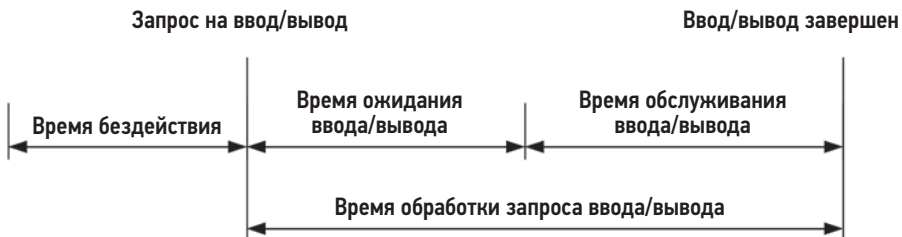


Рис. 9.4. Понятия времени ввода/вывода (обобщенные)

Термин *время обслуживания* возник во времена, когда диски были устроены проще и управлялись непосредственно операционной системой, которая знала, когда диск активно обслуживал ввод/вывод. Современные диски организуют собственные внутренние очереди, а время обслуживания включает время ожидания в очередях в ядре.

По возможности я буду использовать поясняющие термины, чтобы указать, что измеряется, от какого начального события и до какого конечного события. Начальные и конечные события могут происходить в ядре или в дисковом устройстве. Время в ядре при этом измеряется с помощью интерфейса блочного ввода/вывода для дисковых устройств (как показано на рис. 9.5).

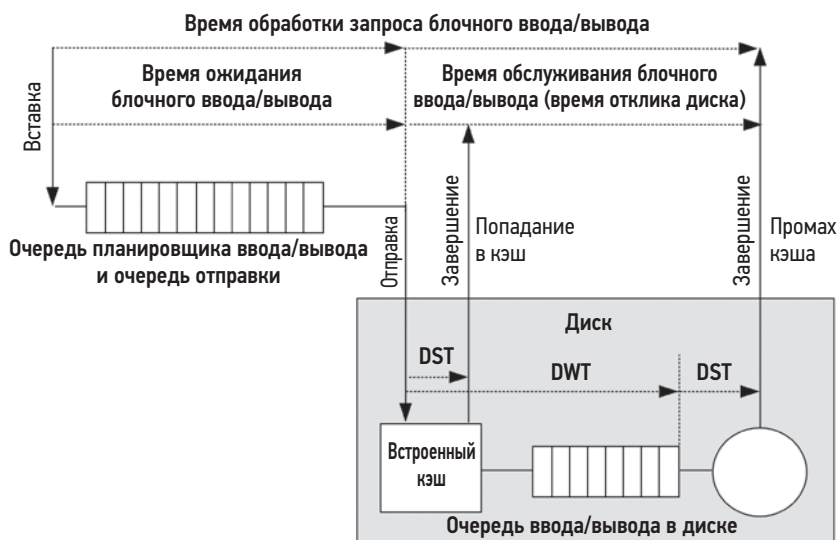


Рис. 9.5. Понятия времени в ядре и диске

В ядре:

- **Время ожидания блочного ввода/вывода (или время ожидания ОС)** — это время, прошедшее от момента создания и добавления запроса на ввод/вывод в очередь в ядре до момента, когда запрос покинул очередь в ядре и был передан дисковому устройству. Этот интервал может охватывать ожидание в нескольких очередях в ядре, включая очередь блочного ввода/вывода и очередь дискового устройства.
- **Время обслуживания блочного ввода/вывода** — это время от отправки запроса устройству до получения прерывания от устройства, сигнализирующего о завершении его обработки.
- **Время обработки запроса блочного ввода/вывода** — это время ожидания запроса на блочный ввод/вывод плюс время обслуживания блочного ввода/вывода: полное время от создания запроса до завершения его обработки.

В дисковом устройстве:

- **Время ожидания диска** — это время, проведенное запросом в очереди внутри диска.
- **Время обслуживания диском** — это время, необходимое на активную обработку запроса после того, как он покинул очередь в диске.
- **Время обработки запроса диском** (также называется **временем отклика диска и задержкой дискового ввода/вывода**) — это время ожидания диска плюс время обслуживания диском. Оно равно времени обслуживания блочного ввода/вывода.

Соответствующие интервалы изображены на рис. 9.5, где DWT — это время ожидания диска (disk wait time), а DST — время обслуживания диском (disk service time). На схеме также показан кэш в диске и то, как попадание в кэш диска способствует значительному уменьшению времени обслуживания диском (DST).

Задержка ввода/вывода — еще один широко используемый термин, представленный в главе 1. Значение этого термина, как и многих других, зависит от места проведения измерений. Часто под задержкой ввода/вывода подразумевается время обработки запроса блочного ввода/вывода, то есть все время, потраченное на выполнение операции ввода/вывода. В приложениях и инструментах анализа производительности для обозначения времени обработки запроса диском обычно используется термин *задержка дискового ввода/вывода*, обозначающий все время обработки запроса устройством. Инженеры, занимающиеся аппаратным обеспечением, могут использовать термин *задержка дискового ввода/вывода* для обозначения времени ожидания диска.

Время обслуживания блочного ввода/вывода обычно интерпретируется как мера текущей производительности диска (его показывают старые версии `iostat(1)`), но все же это слишком упрощенное представление. На рис. 9.7 изображен обобщенный стек ввода/вывода с тремя возможными уровнями драйверов под интерфейсом блочного устройства. Любой из них может иметь свою очередь или блокировать обработку запросов на мьютексах, увеличивая задержку ввода/вывода. Эта задержка входит во время обслуживания блочного ввода/вывода.

Расчет времени

Время обслуживания диском обычно фиксируется ядром напрямую, но его среднее значение можно оценить по значениям IOPS (операций ввода/вывода в секунду) и потребления:

$$\text{Время обслуживания диском} = \text{потребление} / \text{IOPS}.$$

Например, при уровне потребления 60 % и 300 операциях ввода/вывода в секунду получаем среднее время обслуживания 2 мс (600 мс / 300 операций в секунду). Эта формула предполагает, что величина потребления относится к единственному устройству (или *центру обслуживания*), которое обрабатывает запросы ввода/вывода последовательно, по одному. Современные диски могут обрабатывать сразу несколько операций ввода/вывода, что делает эту формулу неточной.

Чтобы получить точное время обслуживания диском, с помощью трассировки событий можно фиксировать моменты времени, соответствующие передаче запросов в устройство и получения ответов. Это можно сделать с помощью инструментов, описанных в этой главе (например, `biolatency(8)` в разделе 9.6.6 «`biolatency`»).

9.3.2. Масштаб времени

Время выполнения дискового ввода/вывода широко варьируется — от десятков микросекунд до тысяч миллисекунд. На самом медленном конце диапазона большое время отклика может быть обусловлено единичным медленным дисковым вводом/выводом, на самом быстром дисковый ввод/вывод может стать проблемой только при большом количестве операций (сумма времени выполнения большого количества быстрых операций ввода/вывода сопоставима с временем медленной операции).

Таблица 9.1 дает общее представление о возможном диапазоне задержек дискового ввода/вывода. Чтобы узнать точные текущие значения, обратитесь к документации поставщика дисков и проведите свой микробенчмаркинг. Чтобы получить представление о масштабах времени, отличных от дискового ввода/вывода, см. главу 2 «Методологии».

Для более точного представления о порядках величин в столбце «В масштабе» приводятся значения, масштабированные по величине воображаемой задержки попадания в кэш на диске, равной одной секунде.

Таблица 9.1. Масштабы задержек дискового ввода/вывода

Событие	Задержка	В масштабе
Попадание в кэш на диске	< 100 мкс ¹	1 с
Чтение из флеш-памяти	от ~100 до 1000 мкс (в зависимости от объема ввода/вывода)	от 1 до 10 с
Последовательное чтение с вращающегося диска	~1 мс	10 с
Произвольное чтение с вращающегося диска (7200 об/мин)	~8 мс	1,3 мин
Произвольное чтение с вращающегося диска (медлен- ное с очередью)	> 10 мс	1,7 мин
Произвольное чтение с вращающегося диска (с десятка- ми запросов в очереди)	> 100 мс	17 мин
Худший случай с виртуальным диском (аппаратный кон- троллер, RAID-5, очередь, произвольный ввод/вывод)	> 1000 мс	2,8 ч

¹ От 10 до 20 мкс для запоминающих устройств с энергонезависимой памятью (Non-Volatile Memory, NVMe): обычно это флеш-память, подключенная через адаптер к шине PCIe.

Эти задержки можно интерпретировать по-разному, в зависимости от среды. Работая в сфере корпоративных систем хранения данных, я считал необычно медленным любой дисковый ввод/вывод, занимающий более 10 мс (и, следовательно, рассматривал это как потенциальный источник проблем). В сфере облачных вычислений к большим задержкам относятся терпимее, особенно это относится к веб-приложениям, для которых большие задержки в сети нормальны. В таких средах дисковый ввод/вывод будет проблемой, только если его продолжительность превышает 50 мс (отдельный случай или в целом в течение всего времени обработки запроса).

Эта таблица также показывает, что задержки дискового ввода/вывода могут быть двух типов: при попадании в кэш на диске (задержка менее 100 мкс) и при промахе (1–8 мс и выше, в зависимости от характера доступа и типа устройства). Поскольку при работе диска будут наблюдаться задержки обоих типов, то выражение их в виде *средней* задержки (как это делает `iostat(1)`) может ввести в заблуждение, так как такое распределение по сути бимодальное. См. рис. 2.23 в главе 2 «Методологии», где показан пример распределения задержки дискового ввода/вывода в виде гистограммы.

9.3.3. Кэширование

Наилучшая производительность дискового ввода/вывода — при полном отсутствии такового. Многие уровни программного стека пытаются избежать дискового ввода/вывода путем кэширования операций чтения и буферизации операций записи вплоть до самого диска. Полный список кэшей, включающий кэши на уровне приложений и файловой системы, был приведен в табл. 3.2 в главе 3 «Операционные системы». На уровне драйверов дисковых устройств и ниже могут быть свои кэши, перечисленные в табл. 9.2.

Таблица 9.2. Кэши, используемые при выполнении дискового ввода/вывода

Кэш	Пример
Кэш устройства	ZFS vdev
Блочный кэш	Кэш буферов
Кэш контроллера дисков	Кэш на карте RAID
Кэш дискового массива	Кэш массива
Кэш внутри дискового устройства	Контроллер данных в диске (Disk Data Controller, DDC) с микросхемами DRAM

Блочный кэш буферов описан в главе 8 «Файловые системы». Эти кэши, используемые при выполнении дискового ввода/вывода, особенно важны для увеличения производительности рабочих нагрузок с произвольным вводом/выводом.

9.3.4. Произвольный и последовательный ввод/вывод

Последовательность операций дискового ввода/вывода можно описать как *произвольную* или *последовательную*, в зависимости от смещения каждой последующей операции от предыдущей (имеется в виду смещение на диске). Эти термины мы обсуждали в главе 8 «Файловые системы», когда знакомились с особенностями доступа к файлам.

Последовательные рабочие нагрузки также часто называют *потокowymi*. Термин *потокowy* обычно используется на уровне приложений для описания потокового чтения и записи «на диск» (в файловую систему).

Произвольный и последовательный характер дискового ввода/вывода было важно изучить в эпоху магнитных вращающихся дисков. В них произвольный ввод/вывод вызывает дополнительную задержку из-за необходимости позиционирования головок и ожидания, пока диск повернется на нужный угол и требуемый сектор на диске окажется под головкой. Это показано на рис. 9.6, где для перемещения от сектора 1 к сектору 2 требуется выполнить позиционирование головок и дожждаться, пока нужный сектор окажется под головкой (фактический путь будет более прямым). Настройка производительности включала выявление таких операций произвольного ввода/вывода и попытки устранить их, например, кэшированием, распределением произвольных операций ввода/вывода между разными дисками и размещением данных на диске так, чтобы уменьшить расстояние, которое должна преодолеть головка при позиционировании.



Рис. 9.6. Вращающийся диск

Для других типов дисков, в том числе твердотельных накопителей на основе флеш-памяти (SSD), производительность произвольного и последовательного ввода/вывода обычно одна и та же. Впрочем, в зависимости от диска, бывает небольшая разница, обусловленная другими факторами, например, кэш поиска адреса может охватывать последовательный доступ, но не охватывать произвольный. В операциях записи, объем которых меньше размера блока, иногда наблюдается потеря

производительности из-за цикла «чтение-изменение-запись», что особенно характерно для произвольных операций записей.

Обратите внимание, что смещения на диске, которые видно в ОС, могут не совпадать со смещениями на физическом диске. Например, аппаратный виртуальный диск может отображать непрерывный диапазон смещений на несколько дисков. Диски могут переназначать смещения по-своему (через контроллер данных). Иногда произвольный ввод/вывод не определяется по смещениям, но может определяться по увеличению времени обслуживания диском.

9.3.5. Соотношение операций чтения и записи

Помимо характера рабочих нагрузок (произвольных и последовательных), еще одна характерная мера, тесно связанная с количеством операций в секунду (IOPS) и пропускной способностью, — это соотношение операций чтения и записи. Она может быть выражена как отношение во времени в процентах, например: «Из всех операций ввода/вывода, выполненных системой с момента загрузки, 80 % приходится на операции чтения».

Знание этого соотношения помогает при проектировании и настройке систем. Система с большой долей операций чтения может получить наибольшую выгоду от добавления кэша, а система с большой долей операций записи — от добавления дополнительных дисков, способствующих увеличению пропускной способности и количества операций ввода/вывода в секунду.

Операции чтения и записи сами по себе могут иметь разный характер: чтение может быть произвольным, а запись — последовательной (что особенно характерно для файловых систем с поддержкой копирования при записи). Они также могут иметь разные размеры.

9.3.6. Размер ввода/вывода

Еще одна характеристика рабочей нагрузки — средний размер ввода/вывода (количество байтов) или распределение размеров ввода/вывода. Чем больше размер ввода/вывода, тем обычно больше пропускная способность, но при этом увеличивается задержка каждой отдельной операции ввода/вывода.

Размер ввода/вывода может изменяться подсистемой дискового устройства (например, делиться на блоки по 512 байт, соответствующие размерам секторов). Размер также может увеличиваться или уменьшаться после запуска операции приложением компонентами ядра, такими как файловые системы, диспетчеры томов и драйверы устройств. См. подразделы «Увеличенный ввод/вывод» и «Уменьшенный ввод/вывод» в главе 8 «Файловые системы» в разделе 8.3.12 «Логический и физический ввод/вывод».

Некоторые дисковые устройства, особенно на основе флеш-памяти, работают по-разному с разными объемами чтения и записи. Например, флеш-накопитель

может показывать наибольшую эффективность при чтении данных блоками по 4 Кбайт и записи блоками по 1 Мбайт. Идеальные размеры ввода/вывода могут упоминаться производителем диска в документации, либо их можно определить с помощью микробенчмаркинга. Текущий размер ввода/вывода можно узнать с помощью инструментов наблюдения (см. раздел 9.6 «Инструменты наблюдения»).

9.3.7. Несопоставимость количества операций в секунду

Из-за трех последних характеристик количество операций ввода/вывода в секунду (IOPS) для разных устройств и рабочих нагрузок нельзя сравнивать напрямую. Величина IOPS сама по себе мало что значит.

Например, вращающийся диск может потратить на выполнение 5000 последовательных операций ввода/вывода значительно меньше времени, чем на 1000 произвольных операций. IOPS для твердотельных накопителей тоже трудно сравнивать, потому что их производительность часто зависит от размера и направления ввода/вывода (чтение или запись).

Более того, IOPS может вообще не иметь значения для приложения. Рабочая нагрузка, состоящая из произвольных запросов, обычно более чувствительна к задержкам, и в этом случае желательно высокое значение IOPS. Поточковая (последовательная) рабочая нагрузка зависит от пропускной способности, из-за чего может оказаться желательной низкая величина IOPS при более объемных операциях ввода/вывода.

Чтобы понять значение IOPS, добавьте в уравнение другие детали: случайный или последовательный характер ввода/вывода, размер, отношение чтение/запись, буферизованный/прямой ввод/вывод и количество операций ввода/вывода, выполняемых параллельно. Также рассмотрите возможность использования метрик на основе времени — уровня потребления и времени обслуживания, которые отражают итоговую производительность и лучше подходят для сравнения.

9.3.8. Дисковые команды, не связанные с передачей данных

Дискам можно посылать множество команд, не связанных с чтением и записью данных. Например, диску с внутренним кэшем можно послать команду вытолкнуть содержимое кэша на диск. Такая команда не является передачей данных. Данные были отправлены раньше, во время операции записи.

Еще один пример — команды для уничтожения данных: ATA TRIM или SCSI UNMAP. Они сообщают диску, что диапазон секторов больше не понадобится. Это помогает накопителям SSD поддерживать высокую производительность записи.

Эти дисковые команды могут влиять на производительность и приводить к тому, что диск будет занят их выполнением, вынуждая другие операции ввода/вывода ожидать.

9.3.9. Потребление

Потребление может быть рассчитано как время, в течение которого диск был занят выполнением работы.

Диск с уровнем потребления 0 % «простаивает», а диск с уровнем потребления 100 % — постоянно занят выполнением операций ввода/вывода (и других команд). Диски со 100 %-ным уровнем потребления — это почти всегда источник проблем с производительностью, особенно если их потребление остается на уровне 100 % некоторое время. Но к снижению производительности может приводить любой уровень потребления, потому что дисковый ввод/вывод — это, как правило, медленное действие.

Между 0 % и 100 % также может быть уровень (например, 60 %), при пересечении которого производительность диска становится неудовлетворительной из-за повышенной вероятности постановки запросов в очередь, будь то очереди внутри диска или в ОС. Точное значение такого уровня потребления зависит от требований к диску, рабочей нагрузке и задержке. См. подраздел «М/D/1 и 60 % нагрузка» в главе 2 «Методологии» в разделе 2.6.5 «Теория массового обслуживания».

Чтобы убедиться, что проблемы в приложении вызывает высокое потребление, изучите время отклика диска и определите, блокируется ли приложение на время выполнения ввода/вывода. Приложение или ОС могут выполнять ввод/вывод асинхронно, и в таких случаях медленный ввод/вывод не вынуждает приложение простаивать.

Обратите внимание, что потребление — интервальная метрика. Дисковый ввод/вывод может выполняться пакетами, особенно когда происходит выталкивание буферов записи, что может остаться незамеченным при вычислении метрики за более длительные интервалы. Более подробно особенности измерения потребления рассматриваются в главе 2 «Методологии» в разделе 2.3.11 «Потребление».

Потребление виртуального диска

Для виртуальных дисков, организованных аппаратно (например, контроллером диска или сетевым хранилищем), ОС может знать, когда виртуальный диск был занят, но ничего не знает о производительности физических дисков, из которых он состоит. Из-за этого потребление виртуального диска, которое сообщается ОС, значительно отличается от фактического положения дел на физических дисках (и выглядит нелогично):

- Виртуальный диск с уровнем потребления 100 %, состоящий из нескольких физических дисков, может выполнить больше работы. В этом случае величина 100 % может означать, что лишь некоторые диски были заняты на 100 %, а некоторые простаивали.
- Виртуальные диски, содержащие кэш отложенной записи, могут казаться не сильно нагруженными во время записи, потому что контроллер диска немедленно возвращает признак завершения записи, даже если после этого физические диски какое-то время остаются занятыми.

- Диски могут оставаться занятыми из-за перестройки аппаратного RAID, при этом ОС может не видеть соответствующих операций ввода/вывода.

По тем же причинам трудно интерпретировать потребление виртуальных дисков, созданных программно (программный RAID). Но ОС должна также предоставлять доступ к информации о потреблении физических дисков.

Когда потребление физического диска достигает 100 % и запрашиваются дополнительные операции ввода/вывода, наступает состояние насыщения.

9.3.10. Насыщенность

Насыщенность — это мера, характеризующая объем работы, поставленной в очередь, который превышает возможности ресурса. Для дисковых устройств насыщенность рассчитывается как средняя длина очереди ожидания устройства в ОС (если она поддерживается).

Насыщенность обеспечивает оценку производительности, когда потребление превышает отметку 100 %. Диск со 100 %-ным потреблением может не быть насыщенным (с пустой очередью) или иметь значительный уровень насыщенности, отрицательно влияющий на производительность из-за того, что запросы долгое время ожидают обработки в очереди.

Вы можете предположить, что для дисков с потреблением менее 100 % насыщение не наступает, но на самом деле многое зависит от интервала измерения: 50 %-ное потребление диска долгое время может означать, что половину этого времени было 100 %-ное потребление, а остальное время диск простаивал. Любая статистика, измеряемая за интервал, может скрывать подобные проблемы. Когда важно иметь точное представление о происходящем, можно использовать инструменты трассировки и ими анализировать события ввода/вывода.

9.3.11. Ожидание ввода/вывода

Ожидание ввода/вывода — это метрика производительности, измеряемая для каждого процессора отдельно. Она показывает время бездействия процессора, когда в очереди планировщика находились потоки, заблокированные в ожидании завершения дискового ввода/вывода. Она вычисляется как отношение времени бездействия процессора к времени отсутствия любых заданий и ожидания завершения операций дискового ввода/вывода. Большая величина ожидания ввода/вывода для процессора может сигнализировать, что диски — это узкое место и процессор вынужден простаивать, ожидая их.

Ожидание ввода/вывода бывает очень запутанной метрикой. Когда появляется другой процесс с высоким потреблением ресурсов процессора, значение ожидания ввода/вывода может упасть, ведь теперь процессору есть чем заняться. Но дисковый ввод/вывод никуда не исчез и все так же блокирует потоки выполнения, несмотря на уменьшение метрики ожидания ввода/вывода. Бывает и наоборот, когда

системные администраторы после установки обновленного и более эффективного прикладного ПО обнаруживают, что ожидание ввода/вывода увеличилось. Здесь можно подумать, что в новой версии есть проблема с диском и худшая производительность, хотя на самом деле потребление диска осталось прежним, но потребление процессора уменьшилось.

Более надежной метрикой является время, за которое прикладные потоки ожидают завершения дискового ввода/вывода. Она отражает падение производительности потоков, вызванное дисками, независимо от того, какую другую работу делают процессоры в это время. Эту метрику можно измерить путем статической или динамической инструментации.

Ожидание ввода/вывода по-прежнему популярная метрика в системах Linux, и, несмотря на ее запутанный характер, она успешно используется для выявления узкого места, которое характеризуется высокой занятостью дисков и низкой занятостью процессоров. Один из вариантов ее интерпретации — рассматривать любое ожидание ввода/вывода как признак наличия узкого места в системе, а затем настроить систему так, чтобы минимизировать его, даже если при увеличении нагрузки на процессор будет выполняться тот же объем ввода/вывода. Параллельный ввод/вывод с большей вероятностью будет неблокирующим и с меньшей вероятностью будет вызывать проблемы непосредственно. Непараллельный ввод/вывод, который вызывает ожидание ввода/вывода, с большей вероятностью заблокирует приложение.

9.3.12. Синхронный и асинхронный ввод/вывод

Важно понимать, что задержка дискового ввода/вывода может не влиять напрямую на производительность приложения, если операции ввода/вывода в приложении и дисковый ввод/вывод выполняются асинхронно. Обычно это происходит при кэшировании с отложенной записью, когда операция ввода/вывода в приложении завершается раньше, а дисковый ввод/вывод выполняется позже.

Приложения могут использовать упреждающее чтение для асинхронного чтения, которое может не блокировать приложение, пока диск выполняет ввод/вывод. Файловая система может инициировать этот процесс сама, чтобы нагреть кэш (предварительная выборка).

Даже если приложение выполняет ввод/вывод синхронно, этот путь в коде приложения может быть не критическим и асинхронным по отношению к запросам клиентов. Это может быть рабочий поток ввода/вывода, созданный приложением для управления вводом/выводом, в то время как другие потоки продолжают выполнять основную работу.

Ядра ОС тоже часто поддерживают асинхронный или неблокирующий ввод/вывод, предоставляя приложениям API для запуска ввода/вывода и получения уведомления о его завершении через некоторое время. Дополнительную информацию по этим темам ищите в главе 8 «Файловые системы», в разделах 8.3.9 «Неблокирующий ввод/вывод», 8.3.5 «Упреждающее чтение», 8.3.4 «Предварительная выборка» и 8.3.7 «Синхронная запись».

9.3.13. Дисковый ввод/вывод и ввод/вывод в приложении

Дисковый ввод/вывод — это конечный результат работы различных компонентов ядра, в том числе файловых систем и драйверов устройств. Есть много причин, по которым частота и объем дискового ввода/вывода могут не соответствовать вводу/выводу, выполняемому приложением:

- Уменьшенный, увеличенный и несвязанный ввод/вывод файловой системы. См. главу 8 «Файловые системы», раздел 8.3.12 «Логический и физический ввод/вывод».
- Подкачка страниц из-за нехватки системной памяти. См. главу 7 «Память», раздел 7.2.2 «Подкачка страниц».
- Размер блока ввода/вывода в драйвере устройства: драйвер округляет объем ввода/вывода в большую сторону или фрагментирует ввод/вывод.
- Запись зеркальных блоков и контрольных сумм или проверка прочитанных данных в RAID.

Это несоответствие может сбивать с толку, если не знать о нем. Его можно исследовать, изучив архитектуру и проведя анализ.

9.4. АРХИТЕКТУРА

В этом разделе я описываю архитектуру диска, которую обычно изучают при планировании емкости с целью определить ограничения для различных компонентов и конфигураций. Ее также важно изучить при исследовании более поздних проблем с производительностью, потому что проблема может быть связана с архитектурным выбором, а не с текущей нагрузкой и настройками.

9.4.1. Типы дисков

Сейчас в основном используются два типа дисков: магнитные вращающиеся и твердотельные накопители (SSD) на основе флеш-памяти. Оба обеспечивают постоянное хранение данных. В отличие от энергозависимой памяти, их содержимое сохраняется и после выключения питания.

9.4.1.1. Магнитные вращающиеся диски

Диски этого типа, которые также называют *приводами жестких дисков* (hard disk drive, HDD), состоят из одного или нескольких дисков — *пластин*, — покрытых частицами оксида железа. Небольшой объем этих частиц может быть намагничен в одном из двух направлений — эта ориентация используется для хранения битов информации. Пластины вращаются, а над ними перемещается механический рычаг со схемой для чтения и записи данных. Эта схема включает *магнитные головки*, и на рычаге бывает несколько головок, что позволяет одновременно читать и записывать

несколько битов. Данные хранятся на пластине в виде концентрических дорожек, каждая из которых разделена на секторы.

Как всякое механическое устройство, жесткие диски работают довольно медленно, особенно когда производится произвольный ввод/вывод. С развитием технологии производства флеш-памяти твердотельные накопители начали вытеснять вращающиеся диски, и вполне вероятно, что однажды вращающиеся диски выйдут из употребления. Тем не менее вращающиеся диски остаются вполне конкурентоспособными в некоторых сценариях, например, когда требуются недорогие хранилища высокой плотности (с низкой стоимостью хранения мегабайта), особенно для хранилищ данных¹.

Ниже кратко описаны некоторые факторы, влияющие на производительность вращающихся дисков.

Позиционирование и вращение

Невысокая скорость ввода/вывода магнитных вращающихся дисков обычно обусловлена затратами времени на позиционирование головок и ожиданием, пока диск повернется на требуемый угол. То и другое может исчисляться миллисекундами. В лучшем случае следующий ввод/вывод должен производиться в конце текущего ввода/вывода, обслуживаемого в данный момент, поэтому диску не требуется позиционировать головки или ждать поворота пластин на дополнительный угол. Как отмечалось выше, такой ввод/вывод называется *последовательным*, а ввод/вывод, требующий позиционирования головки или ожидания поворота диска на требуемый угол, — *произвольным*.

Есть множество стратегий сокращения времени ожидания позиционирования и поворота, в том числе:

- кэширование: полное устранение дискового ввода/вывода;
- размещение и поведение файловой системы, включая копирование при записи (делающее запись последовательной, но последующие чтения могут стать произвольными);
- распределение разных рабочих нагрузок по разным дискам, чтобы избежать излишних затрат времени на позиционирование головок между рабочими нагрузками;
- распределение разных рабочих нагрузок по разным системам (некоторые облачные среды дают такую возможность, чтобы уменьшить влияние разных клиентов друг на друга);

¹ Серверы Open Connect Appliance (OCA) в Netflix, на которых размещается видео для потоковой передачи, могут выглядеть вполне подходящими для использования жестких дисков, но поддержка большого количества клиентов, одновременно просматривающих разные видеофильмы, может вызвать произвольный ввод/вывод. По этой причине некоторые OCA были переведены на использование флеш-накопителей [Netflix 20].

- использование внутри дисков алгоритмов, сокращающих затраты времени на позиционирование;
- использование дисков с более высокой плотностью хранения данных, чтобы ограничить рабочую нагрузку;
- настройка разделов (или «срезов»), например, с коротким ходом.

Еще одна стратегия сокращения времени ожидания поворота — использование дисков с высокими скоростями вращения. Сейчас доступны жесткие диски с разными скоростями вращения, включая 5400, 7200, 10 000 и 15 000 оборотов в минуту. Но учтите: более высокие скорости могут привести к сокращению срока службы дисков из-за повышенного нагрева и износа.

Теоретическая максимальная пропускная способность

Если известно максимальное количество секторов на дорожку диска, то его пропускную способность можно рассчитать по формуле:

Максимальная пропускная способность = максимальное количество секторов на дорожку × размер сектора × об/мин/60 с.

Эта формула более подходила для старых дисков, которые точно отображали исходную информацию. Современные диски предоставляют ОС виртуальный образ диска и искусственные значения этих атрибутов.

Короткий ход

Короткий ход имеет место, когда рабочая нагрузка использует только внешние дорожки диска, а остальные либо не используются, либо используются рабочими нагрузками с низкими требованиями к пропускной способности (например, архивами). Это сокращает время позиционирования, потому что головки перемещаются на короткие расстояния и диск может оставить их в состоянии покоя на внешнем крае, уменьшая время на первое позиционирование после простоя. Кроме того, внешние дорожки обычно имеют более высокую пропускную способность из-за зонирования секторов (см. следующий подраздел). Следите за короткими ходами при изучении опубликованных результатов тестирования производительности дисков, особенно результатов, в которых не указаны цены и при получении которых могли использоваться несколько дисков с коротким ходом.

Зонирование секторов

Все дорожки на диске имеют разную длину, самые короткие находятся ближе к центру диска, а самые длинные — у внешнего края. Зонирование секторов (также называемое *многозонной записью*) позволяет не фиксировать количество секторов (и битов), приходящееся на дорожку, а увеличивать его для более длинных дорожек, которые физически позволяют разбить их на большее число секторов. Поскольку скорость вращения постоянна, более длинные внешние дорожки обеспечивают более высокую пропускную способность (мегабайт в секунду) по сравнению с внутренними.

Размер сектора

В индустрии хранения данных был разработан новый стандарт для дисковых устройств, получивший название «Advanced Format». Он определяет поддержку секторов большего размера, в частности 4 Кбайт. Это снижает оверхед на операции ввода/вывода, увеличивает пропускную способность, а также сокращает оверхед на хранение метаданных в каждом секторе диска. Секторы размером 512 байт по-прежнему могут поддерживаться микропрограммой диска через стандарт эмуляции «Advanced Format 512e». В некоторых дисках это может увеличить оверхед на запись, вызывая цикл чтения-изменения-записи для отображения 512 байт в сектор размером 4 Кбайт. К другим проблемам с производительностью, о которых следует знать, относятся смещенные 4-килобайтные операции ввода/вывода, охватывающие два сектора, которые увеличивают объем ввода/вывода для их обслуживания.

Кэш внутри дискового устройства

Общим компонентом современных жестких дисков является небольшой объем памяти, используемый для кэширования результатов чтения и буферизации записи. Эта память позволяет помещать операции ввода/вывода (команды) в очередь на устройстве и переупорядочивать их для большей эффективности. В SCSI эта очередь называется Tagged Command Queueing (TCQ), в SATA — Native Command Queuing (NCQ).

Алгоритм лифта для позиционирования

Алгоритм лифта — это один из способов повышения эффективности работы за счет переупорядочения команд в очереди. Этот алгоритм переупорядочивает операции ввода/вывода с учетом их местоположения на диске так, чтобы минимизировать перемещение головок. Своим действием алгоритм напоминает строительный лифт, который обслуживает этажи не в порядке нажатия кнопок этажей, а движется вверх и вниз, останавливаясь на ближайшем этаже из запрошенных в данный момент.

Такое поведение легко выявляется при исследовании трассировки дискового ввода/вывода — сортировка операций ввода/вывода по времени их начала не соответствует сортировке по времени окончания, то есть операции завершаются не по порядку.

Кажется, что это дает очевидный выигрыш в производительности, но представьте следующий сценарий: на диск отправлен пакет операций ввода/вывода со смещением 1000 и одна операция со смещением 2000. Головки диска находятся в позиции со смещением 1000. Когда будет обслужен ввод/вывод со смещением 2000? Теперь представьте, что при обслуживании ввода/вывода со смещением, близким к 1000, поступает еще множество операций со смещением, близким к 1000, и такие операции продолжают поступать в достаточном количестве, чтобы удерживать диск занятым работой около смещения 1000 в течение 10 с. Когда

будет обслужен ввод/вывод со смещением 2000 и какова будет его окончательная задержка ввода/вывода?

Целостность данных

Диски хранят код исправления ошибок (error-correcting code, ECC) в конце каждого сектора. Это обеспечивает целостность данных и то, что диск может проверить их правильность при чтении или исправить любые возникающие ошибки. Если сектор был прочитан неправильно, диск может попытаться повторно прочитать его на следующем обороте (и повторить попытку несколько раз, немного меняя положение головки). Иногда этим объясняется необычно медленный ввод/вывод. Накопитель может возвращать ОС программные ошибки, объясняющие происходящее. Часто полезно наблюдать за частотой программных ошибок, так как ее увеличение может указывать на то, что вскоре диск может выйти из строя.

Одно из преимуществ перехода с 512-байтных секторов на 4-килобайтные в том, что для одного и того же объема данных требуется меньше битов ECC, потому что ECC более эффективен для секторов большого размера [Smith 09].

Обратите внимание, что для проверки данных также могут использоваться другие контрольные суммы. Например, для проверки передачи данных в шину может использоваться циклический избыточный код (cyclic redundancy check, CRC), а файловые системы могут использовать другие контрольные суммы.

Вибрация

В отличие от поставщиков дисковых устройств, хорошо знакомых с проблемой вибрации, в программной индустрии не всегда знали об этой проблеме и не воспринимали ее всерьез. В 2008 году, изучая загадочную проблему с производительностью, я провел эксперимент, подвергнув вибрационному воздействию дисковый массив во время выполнения теста записи. Это вызвало всплеск увеличения времени выполнения операций ввода/вывода. Мой эксперимент был снят на видео и размещен на YouTube. Он стал вирусным и был описан как первая демонстрация влияния вибрации на производительность диска [Turner 10]. Видео набрало более 1 700 000 просмотров, поспособствовав повышению осведомленности о проблемах влияния вибрации на диски [Gregg 08]. Судя по электронным письмам, я также случайно дал толчок к развитию индустрии звукоизоляции центров обработки данных: теперь вы можете нанять профессионалов, которые проанализируют уровни шумов и выполнят мероприятия, способствующие улучшению производительности дисков за счет гашения вибраций.

Ленивые диски

Актуальная проблема производительности некоторых вращающихся дисков связана с открытием так называемых *ленивых дисков* (sloth disks). Эти диски иногда выполняют ввод/вывод очень медленно, тратя более одной секунды и не сообщая при этом ни о каких ошибках. Это время намного превышает время, необходимое

для повторных попыток чтения, когда получен неверный код ECC. Было бы лучше, если бы такие диски возвращали сообщение об отказе, а не занимали так много времени, чтобы ОС или контроллеры дисков могли предпринять корректирующие действия, например отключение диска в избыточных средах и отправку сообщения о сбое. Ленивые диски доставляют неудобства, особенно когда это часть виртуального диска, представленного дисковым массивом, и ОС не видит отдельные диски напрямую, что затрудняет их идентификацию¹.

Черепичная магнитная запись

Приводы с поддержкой черепичной магнитной записи (Shingled Magnetic Recording, SMR) обеспечивают более высокую плотность хранения данных за счет использования более узких дорожек. Такие дорожки слишком узкие для записывающей головки, но достаточно широкие для читающей, поэтому запись выполняется с частичным перекрытием дорожек, что напоминает укладку кровельной черепицы (отсюда и название). Диски, использующие SMR, обеспечивают увеличение плотности примерно на 25 % за счет снижения производительности записи, потому что перекрывающиеся данные уничтожаются и должны быть записаны повторно. Эти диски подходят для хранения архивов, которые записываются один раз и затем только читаются, но не подходят для рабочих нагрузок с большим объемом записи в конфигурациях RAID [Mellor 20].

Дисковый контроллер данных

Механические диски представляют простой интерфейс для систем, предполагающих фиксированное количество секторов на дорожке и непрерывный диапазон адресуемых смещений. Что происходит на диске, зависит от контроллера данных — внутреннего микропроцессора, выполняющего микропрограмму. На дисках могут быть реализованы алгоритмы, включая зонирование секторов, влияющее на определение смещений. Об этом нужно знать, но такие тонкости сложно анализировать, потому что дисковый контроллер данных недоступен ОС.

9.4.1.2. Твердотельные накопители

Твердотельные накопители (solid-state drives, SSD) также называют твердотельными дисками (solid-state disks). Термин *твердотельный* означает использование твердотельной электроники — программируемой энергонезависимой памяти, обладающей гораздо более высокой производительностью, чем вращающиеся диски. Эти диски, не имеющие движущихся частей, более прочные физически и не подвержены проблемам, связанным с вибрацией.

Производительность дисков этого типа обычно одинакова для разных смещений (нет задержки, необходимой для позиционирования и ожидания поворота

¹ Если используется система Linux Distributed Replicated Block Device (DRBD), то она предоставляет параметр «disk-timeout».

на нужный угол) и предсказуема для известного объема ввода/вывода. Произвольный или последовательный характер рабочей нагрузки здесь менее важен, чем для вращающихся дисков. Все это упрощает их анализ и планирование мощностей. Но при появлении проблем с производительностью из-за особенностей внутреннего устройства разобраться в них так же сложно, как и с вращающимися дисками.

Некоторые твердотельные накопители используют энергонезависимую память (NV-DRAM), но в большинстве конструкций применяется флеш-память.

Флеш-память

Твердотельные накопители на основе флеш-памяти обеспечивают высокую производительность чтения, в том числе и произвольного чтения, которая может в разы превосходить производительность вращающихся дисков. Большинство из них основаны на использовании флеш-памяти NAND, в которой применяются электронные носители, захватывающие заряд и способные хранить электроны долгое время¹, даже в отсутствие питания [Cornwell 12]. Название «флеш» относится к способу записи данных, требующему одновременного удаления всего блока памяти (до нескольких страниц, обычно 8 или 64 Кбайт на страницу) и перезаписи содержимого. Из-за перепада на запись флеш-память отличается асимметричной производительностью операций чтения и записи: чтение выполняется быстро, а запись — медленно. Накопители обычно сглаживают это различие, используя кэши с отложенной записью для повышения производительности записи и небольшой конденсатор в качестве резервного источника питания на случай сбоя.

Есть несколько типов флеш-памяти:

- **с одноуровневыми ячейками** (single-level cell, **SLC**): хранит по одному биту в каждой ячейке;
- **с многоуровневыми ячейками** (multi-level cell, **MLC**): хранит несколько битов в каждой ячейке (обычно два, что требует четырех уровней напряжения);
- **с многоуровневыми ячейками промышленного уровня** (enterprise multi-level cell, **eMLC**): флеш-память MLC с расширенным микропрограммным обеспечением, предназначенная для использования в промышленных условиях;
- **с трехуровневыми ячейками** (tri-level cell, **TLC**): хранит три бита (восемь уровней напряжения) в каждой ячейке;
- **с четырехуровневыми ячейками** (quad-level cell, **QLC**): хранит четыре бита в каждой ячейке;
- **3D NAND/Vertical NAND (V-NAND)**: многослойная архитектура, включающая несколько слоев флеш-памяти (например, TLC) для увеличения плотности и емкости хранилища.

¹ Но не до бесконечности. Ошибки хранения данных в современных MLC могут возникнуть уже через несколько месяцев после выключения питания [Cassidy 12], [Cai 15].

Этот список составлен в приблизительном хронологическом порядке, последними перечислены новейшие технологии: 3D NAND стала доступна для коммерческого распространения с 2013 года.

Флеш-память SLC, как правило, имеет более высокую производительность и надежность по сравнению с другими типами. Она предпочтительнее для промышленного использования, хотя и стоит дороже. На предприятиях часто используется флеш-память MLC, обладающая более высокой плотностью, несмотря на ее не самую высокую надежность. Надежность флеш-памяти часто измеряется как количество операций записи блока (циклов программирования/стирания). Для SLC это количество составляет от 50 000 до 100 000 циклов, для MLC от 5000 до 10 000 циклов, для TLC около 3000 циклов, а для QLC около 1000 циклов [Liu 20].

Контроллер

Контроллер для SSD решает следующие задачи [Leventhal 13]:

- **ВВОД:** читает и записывает страницы (обычно по 8 Кбайт), запись может происходить только в стертые страницы, страницы стираются блоками от 32 до 64 штук (256–512 Кбайт);
- **ВЫВОД:** имитирует интерфейс жесткого диска: читает или записывает произвольные секторы (по 512 байт или 4 Кбайт).

Преобразование между вводом и выводом выполняется уровнем трансляции флеш-памяти (flash translation layer, FTL) контроллера, который также должен следить за свободными блоками. По сути, для этой цели используется собственная файловая система, например журналируемая файловая система.

Особенности записи могут вызывать проблемы с производительностью в пишущих рабочих нагрузках, особенно когда данные записываются блоками размером меньше размера блока флеш-памяти (который может достигать 512 Кбайт). Это может вызвать *увеличение объема записи*, когда оставшаяся часть блока копируется в другое место перед стиранием, а также задержку, по крайней мере, на продолжительность цикла стирания/записи. Некоторые флеш-накопители стараются ослабить проблему задержки, предоставляя внутренний буфер (на основе ОЗУ) с резервной батареей, чтобы записываемые данные можно было сохранить в буфер и записать позже даже в случае сбоя питания.

Самый распространенный флеш-накопитель промышленного уровня, который я использовал, оптимально работал при чтении данных блоками по 4 Кбайт и записи блоками по 1 Мбайт, что объяснялось структурой флеш-памяти. Эти значения различаются для разных приводов, и их можно определить путем микробенчмаркинга размеров ввода/вывода.

Учитывая несоответствие между внутренними операциями и блочным интерфейсом флеш-памяти, есть место для совершенствования операционной системы и ее файловых систем. Примером может служить команда TRIM: она сообщает твердотельному накопителю SSD, что некоторая область больше не используется; это

упрощает для SSD сборку своего пула свободных блоков и ослабляет проблему увеличения объема записи. (Для SCSI то же самое можно реализовать с помощью команд UNMAP или WRITE SAME, для ATA — с помощью команд DATA SET MANAGEMENT. Поддержка Linux включает параметр `discard` команды `mount` и команду `fstrim(8)`.)

Продолжительность жизни

Использование флеш-памяти NAND в качестве носителя информации сопряжено с разными проблемами, включая выгорание, затухание данных и нарушение чтения [Cornwell 12]. Их можно решить с помощью контроллера SSD, который способен перемещать данные, чтобы избежать этих проблем. Обычно контроллер использует алгоритм *выравнивания износа*, который распределяет записываемые данные по разным блокам, чтобы сократить количество циклов записи для отдельных блоков, и *избыточное выделение памяти*, резервирующее дополнительную память, которую можно было бы использовать для обслуживания.

Все эти методы увеличивают срок службы, но блоки в SSD по-прежнему имеют ограниченное количество циклов записи, в зависимости от типа флеш-памяти и алгоритмов выравнивания износа, используемых накопителем. В накопителях промышленного уровня используется избыточное выделение памяти и самый надежный тип флеш-памяти SLC, допускающий от 1 миллиона циклов записи и выше. Приводы потребительского уровня на основе MLC могут допускать всего 1000 циклов.

Патологии

Твердотельные накопители с флеш-памятью страдают некоторыми патологиями, о которых следует знать и помнить:

- выбросы по величине задержки из-за старения, обусловленные тем, что SSD пытается извлечь правильные данные (проверяя их с помощью ECC);
- высокая задержка из-за фрагментации (эту проблему можно исправить, выполнив форматирование и очистив карты блоков FTL);
- низкая пропускная способность, если в SSD реализовано внутреннее сжатие.

Проверьте наличие других разработок, влияющих на производительность SSD, и выявленных проблем.

9.4.1.3. Энергонезависимая память

Для поддержки кэшей отложенной записи в контроллерах устройств хранения используется энергонезависимая память в форме DRAM с питанием от батарей¹. Производительность памяти этого типа на порядки выше производительности

¹ Батарей или суперконденсатора.

флеш-памяти, но стоимость и ограниченный срок службы батарей позволяют использовать их только в особых случаях.

Новый тип энергонезависимой памяти под названием 3D XPoint, разработанный компаниями Intel и Micron, расширяет круг использования памяти этого типа благодаря привлекательному соотношению цена/производительность, по которому она занимает промежуточное положение между DRAM и флеш-памятью. 3D XPoint хранит биты в наращиваемом массиве в виде сетки и имеет байтовую адресацию. По результатам тестирования, проведенного в Intel, память 3D XPoint показала задержку доступа в 14 мкс против 200 мкс для твердотельных накопителей 3D NAND [Hady 18]. 3D XPoint также показала стабильность задержки, тогда как для 3D NAND был отмечен разброс задержек в более широком диапазоне, достигающем 3 мс.

Для коммерческого использования память 3D XPoint стала доступна в 2017 году. Intel выпускает модули энергонезависимой памяти и твердотельные накопители под маркой Intel Optane.

9.4.2. Интерфейсы

Интерфейс — это протокол, поддерживаемый приводом для взаимодействия с системой, обычно через контроллер диска. Ниже приводится краткое описание интерфейсов SCSI, SAS, SATA, FC и NVMe. Проверьте, какие интерфейсы используются прямо сейчас, и изучите их пропускную способность — они имеют свойство меняться по мере развития существующих и появления новых спецификаций.

SCSI

Интерфейс малых вычислительных систем (Small Computer System Interface, SCSI) первоначально имел вид параллельной транспортной шины, использующей несколько электрических разъемов для параллельной передачи битов. Первая версия SCSI-1, появившаяся в 1986 году, имела ширину шины данных 8 бит и позволяла передавать один байт, обеспечивая пропускную способность 5 Мбайт/с. Она подключалась с помощью 50-контактного разъема Centronics C50. В поздних версиях SCSI использовались более широкие шины данных с разъемами до 80 контактов, пропускная способность достигала сотен мегабайт.

Поскольку параллельный интерфейс SCSI — это общая шина, он может испытывать проблемы с производительностью из-за конфликтов на шине, например, когда запланированное резервное копирование системы насыщает шину низкоприоритетным вводом/выводом. Для решения этой проблемы устройства с низким приоритетом часто подключались к отдельной шине SCSI или контроллеру.

На высоких скоростях синхронизация параллельных шин становится проблемой. Наряду с другими проблемами (включая ограниченное количество устройств и необходимость использования терминаторов SCSI) это привело к переходу на последовательную версию: SAS.

SAS

Последовательный интерфейс SCSI (Serial Attached SCSI) проектировался как высокоскоростной двухточечный транспорт, позволяющий избежать конфликтов на шине параллельного интерфейса SCSI. Первоначальная спецификация SAS-1 поддерживала пропускную способность 3 Гбит/с (выпущена в 2003 году). За ней последовали спецификации SAS-2 с пропускной способностью 6 Гбит/с (2009), SAS-3 — 12 Гбит/с (2012) и SAS-4 — 22,5 Гбит/с (2017). Спецификация SAS поддерживает агрегирование каналов, что позволяет объединить несколько портов для достижения более высокой пропускной способности. Фактическая скорость передачи данных составляет 80 % полосы пропускания из-за кодирования 8-битных данных в 10-битные.

В числе других особенностей SAS можно назвать поддержку приводов с двумя портами для подключения резервных разъемов и резервных архитектур, многоканальный ввод/вывод, домены SAS, горячую замену и поддержку совместимости с устройствами SATA. Благодаря этому SAS стал популярен в промышленном секторе, особенно с резервными архитектурами.

SATA

По тем же причинам, что SCSI и SAS, стандарт параллельного интерфейса ATA (он же IDE) превратился в последовательный интерфейс обмена данными с накопителями информации (Serial ATA). Первоначальная спецификация SATA 1.0 (2003) поддерживала пропускную способность 1,5 Гбит/с. Более поздняя версия SATA 2.0 — 3,0 Гбит/с (2004) и SATA 3.0 — 6,0 Гбит/с (2008). В основные и промежуточные версии спецификации были добавлены такие особенности, как встроенная поддержка очереди команд. SATA использует кодирование 8-битных данных в 10-битные, поэтому скорость передачи составляет 80 % от пропускной способности. Интерфейс SATA широко используется в настольных компьютерах и ноутбуках.

FC

Оптоволоконный канал (Fibre Channel, FC) — это стандарт высокоскоростного интерфейса для передачи данных, изначально предназначавшийся только для использования с оптоволоконным кабелем (отсюда и название), но позднее в него была добавлена поддержка медных кабелей. FC обычно используется в промышленных средах для создания сетей хранения данных (storage area networks, SAN), в которых несколько устройств хранения могут быть подключены к нескольким серверам через оптоволоконную сеть (Fibre Channel Fabric). Он обеспечивает большую масштабируемость и доступность, чем другие интерфейсы, и действует подобно хостам в сети. Как и в обычной сети, FC позволяет использовать *коммутаторы* для соединения нескольких локальных конечных точек (сервера и хранилища). Разработка стандарта Fibre Channel началась в 1988 году, и первая версия, одобренная в ANSI, вышла в 1994 году [FICA 20]. С тех пор появилось множество вариантов

и улучшений: в недавнем стандарте Gen 7 256GFC пропускная способность достигла 51 200 Мбайт/с в полнодуплексном режиме [FICA 18].

NVMe

Высокопроизводительная энергонезависимая память (Non-Volatile Memory express, NVMe) — это спецификация шины PCIe для устройств хранения. В отличие от устройств хранения, подключающихся к карте контроллера хранилища, устройство NVMe само по себе — это карта, которая подключается непосредственно к шине PCIe. Работы над спецификацией NVMe начались в 2011 году. Первая версия 1.0e вышла в 2013 году, а последняя версия 1.4 — в 2019-м [NVMe 20]. В новые спецификации были добавлены различные особенности: управление температурным режимом и команды для самотестирования, проверка и очистка данных (исключающая возможность восстановления). Пропускная способность карт NVMe ограничена пропускной способностью шины PCIe. PCIe версии 4.0, широко используемая сегодня, имеет однонаправленную пропускную способность 31,5 Гбайт/с для карты x16 (ширина канала).

Главное преимущество NVMe перед традиционными SAS и SATA — поддержка нескольких аппаратных очередей. Эти очереди могут использоваться одним и тем же процессором для «разогрева» кэша (а с поддержкой нескольких очередей в Linux можно даже избежать использования общих блокировок в ядре). Эти очереди дают обширные возможности буферизации, позволяя сохранять в каждой очереди до 64 тысяч команд, тогда как типичные SAS и SATA ограничены 256 и 32 командами соответственно.

NVMe также поддерживает виртуализацию ввода/вывода с единым корнем (single-root I/O virtualization, SR-IOV) для повышения производительности хранилища виртуальных машин (см. главу 11 «Облачные вычисления», раздел 11.2 «Виртуализация оборудования»).

NVMe используется для флеш-устройств с малой задержкой ввода/вывода менее 20 мкс.

9.4.3. Типы хранилищ

Хранилище может быть организовано на сервере несколькими способами. В подразделах ниже описаны четыре наиболее распространенные архитектуры: дисковые устройства, RAID, массивы хранения и сетевые хранилища (NAS).

Дисковые устройства

Самая простая архитектура — это сервер с внутренними дисками, которыми ОС управляет по отдельности. Диски подключаются к контроллеру дисков, реализованному в виде схемы на материнской плате или отдельной карты расширения, который обеспечивает доступ к дисковым устройствам. В этой архитектуре контроллер дисков действует просто как канал связи, позволяющий системе обмениваться данными с дисками. В типичном персональном компьютере или ноутбуке диск, играющий роль основного хранилища, подключен именно таким образом.

Эту архитектуру проще всего анализировать с помощью инструментов оценки производительности, потому что каждый диск известен ОС и за ними можно наблюдать по отдельности.

В терминологии некоторых контроллеров дисков такая архитектура называется *простой связкой дисков* (just a bunch of disks, JBOD).

RAID

Усовершенствованные контроллеры дисков могут поддерживать архитектуру избыточного массива независимых дисков (redundant array of independent disks, RAID) для дисковых устройств (первоначально эта архитектура называлась избыточным массивом *недорогих* дисков — redundant array of *inexpensive* disks [Patterson 88]). RAID может представлять несколько дисков как один большой, быстрый и надежный виртуальный диск. Эти контроллеры часто имеют встроенный кэш для увеличения производительности чтения и записи.

Если дисковый массив формируется с использованием карты контроллера RAID, то такой массив называется *аппаратным* RAID. Дисковый массив может быть организован и программно операционной системой, но традиционно предпочтение отдавалось аппаратному RAID, поскольку дорогостоящие вычисления контрольных сумм и четности предпочтительнее выполнять на выделенном оборудовании. Также такое оборудование часто включает блок резервного питания от батареи для повышения отказоустойчивости. Но с появлением процессоров с большим количеством ядер оверхед на вычисление четности стал менее ощутимым. В ряде решений для хранения данных снова стали использовать программный RAID (например, с применением ZFS), что помогло снизить сложность и стоимость оборудования и улучшило наблюдаемость со стороны ОС. В случае серьезного сбоя программный RAID также часто проще восстановить, чем аппаратный RAID (представьте вышедшую из строя карту RAID).

В подразделах ниже описаны характеристики производительности RAID. Говоря о RAID, часто можно услышать термин *чередование* (stripe): он относится к случаям, когда данные, сгруппированные в блоки, записываются на несколько дисков (как бы прочерчивая полосу — stripe — через все диски).

Типы

Для разных потребностей в смысле емкости, производительности и надежности доступны разные типы RAID. В этом подразделе основное внимание уделим характеристикам, приведенным в табл. 9.3.

Уровень RAID-0 с чередованием обладает самой высокой производительностью, но не имеет избыточности, что делает его малоприменимым для промышленного использования. Возможные исключения — отказоустойчивые облачные среды, которые не хранят важные данные и где отказавшие экземпляры автоматически замещаются новыми, а также серверы хранения, используемые только для кэширования.

Таблица 9.3. Типы RAID

Уровень	Описание	Производительность
0 (последовательный)	Диски заполняются последовательно, друг за другом	Увеличивает производительность произвольного чтения, когда в работе может участвовать несколько дисков
0 (с чередованием)	Диски заполняются параллельно, данные распределяются (чередуются) сразу по нескольким дискам	Как предполагается, такая организация обеспечивает наилучшую производительность произвольного и последовательного ввода/вывода (зависит от размера полосы чередования и характера рабочей нагрузки)
1 (зеркалирование)	Диски объединяются в группы (обычно по два) и хранят одни и те же данные, обеспечивая резервирование	Хорошая производительность произвольного и последовательного чтения (чтение может выполняться со всех дисков одновременно, в зависимости от реализации). Скорость записи ограничена самым медленным диском в зеркале, а оверхед удваивается (если диски группируются по два)
10	Комбинация RAID-0 с чередованием и RAID-1 с группировкой дисков, обеспечивающая емкость и избыточность	Характеристики производительности аналогичны уровню RAID-1, но допускается участие большего количества групп дисков для увеличения пропускной способности как в RAID-0
5	Данные хранятся с чередованием на нескольких дисках вместе с дополнительной информацией о четности для избыточности	Невысокая производительность записи из-за цикла «чтение-изменение-запись» и вычислений четности
6	RAID-5 с двумя дисками для хранения с чередованием информации о четности	Близкая к RAID-5, но немного хуже

Наблюдаемость

Как описывалось выше в разделе о потреблении виртуальных дисков, использование аппаратных виртуальных дисков может затруднить наблюдение за ними из ОС, которая не знает, чем заняты физические диски. Если дисковый массив RAID организован программно, то обычно есть возможность наблюдать за работой отдельных дисковых устройств, потому что ОС сама управляет ими.

Чтение-изменение-запись

Когда данные хранятся с чередованием и с информацией о четности, как в RAID-5, каждая операция записи может повлечь за собой дополнительные операции чтения и затраты времени на вычисления. Это связано с тем, что для записи блока данных, размер которого меньше размера полосы, может потребоваться прочитать всю полосу, изменить в ней требуемые байты, повторно вычислить четности и записать полосу обратно. Чтобы избежать этого, можно использовать оптимизацию для RAID-5: вместо всей полосы прочитать только часть полосы (полос), включающую

измененные данные, вместе с информацией о четности, выполнить последовательность операций XOR, чтобы вычислить обновленное значение четности, и записать его вместе с измененной частью полосы.

Запись блока, охватывающего всю полосу, можно произвести без предварительного чтения. Производительность в таких средах можно улучшить, выбрав размер полосы равным среднему размеру операций записи, чтобы уменьшить дополнительный оверхед на чтение.

Кэши

Контроллеры дисков, реализующие RAID-5, могут ослабить проблему падения производительности из-за цикла «чтение-изменение-запись» за счет использования кэша отложенной записи. Эти кэши должны иметь резервное питание от батареи, чтобы в случае пропажи питания они могли завершить буферизованную запись.

Дополнительные возможности

Имейте в виду, что усовершенствованные контроллеры дисков могут поддерживать дополнительные возможности, влияющие на производительность. Загляните в документацию производителя, чтобы получить хотя бы общее представление об имеющихся возможностях. Например, вот пара возможностей, которыми обладает контроллер Dell PERC 5 [Dell 20]:

- **Patrol read:** каждые несколько дней контроллер читает все блоки диска и проверяет их контрольные суммы. Если диски заняты обслуживанием запросов, то на patrol read выделяется меньше ресурсов, чтобы исключить конкуренцию с рабочей нагрузкой.
- **Интервал выталкивания кэша:** время в секундах между принудительным выталкиванием измененных данных из кэша на диск. Установка большего интервала может уменьшить объем дискового ввода/вывода за счет сокращения операций записи и улучшения агрегирования записываемых данных. При этом может увеличиться задержка операций чтения в моменты, когда на диск выталкиваются значительные объемы данных из кэша.

Оба этих фактора могут существенно повлиять на производительность.

Массивы хранения

Массивы хранения позволяют подключать к системе множество дисков. Для их организации используются усовершенствованные контроллеры дисков, позволяющие настроить RAID и обычно имеющие кэш большой емкости (измеряемой гигабайтами) для увеличения производительности чтения и записи. Также эти кэши обычно питаются от батарей, что позволяет им работать в режиме отложенной записи. Распространенная тактика — переключение в режим сквозной записи в случае отказа батареи, что может быть заметно по внезапному падению производительности записи из-за цикла «чтение-изменение-запись».

На производительность влияет и способ подключения к системе массива хранения — обычно для этой цели используется внешний контроллер хранения. Контроллер и транспорт, соединяющий его с массивом хранения, оба будут иметь свои ограничения на количество операций ввода/вывода в секунду и пропускную способность. Для увеличения производительности и надежности массивы хранения часто поддерживают двойное подключение, то есть их можно подключать двумя физическими кабелями к одному или двум контроллерам хранения.

Сетевые хранилища

Сетевые хранилища (network attached storage, NAS) подключаются к системе по сети с использованием сетевого протокола, такого как NFS, SMB/CIFS или iSCSI. Обычно они организуются как выделенные системы, которые называют устройствами NAS. Это отдельные системы, и их следует анализировать как таковые. Однако и на стороне клиента можно выполнить некоторый анализ производительности, например оценить прикладываемую рабочую нагрузку и измерить задержки ввода/вывода. Важным фактором в таких средах становится производительность сети, и проблемы могут возникать из-за перегрузки сети и задержек с несколькими сетевыми переходами.

9.4.4. Стек дискового ввода/вывода в операционной системе

Точный состав компонентов и слоев в стеке дискового ввода/вывода зависит от ОС, версии, а также используемых программных и аппаратных технологий. На рис. 9.7 изображена обобщенная модель. Аналогичная модель, включающая приложения, описывается в главе 3 «Операционные системы».

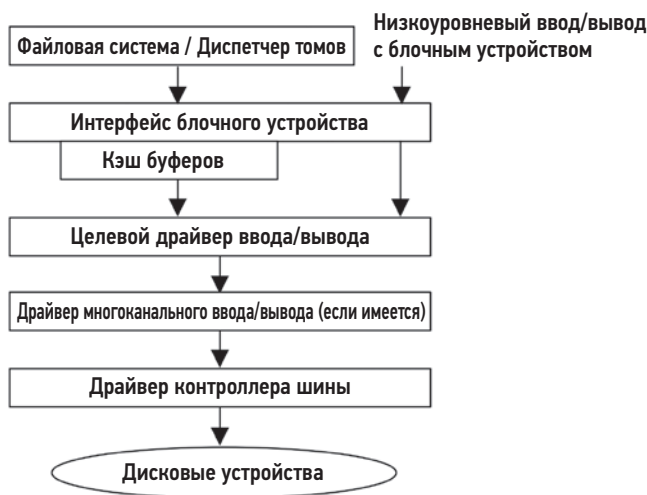


Рис. 9.7. Обобщенный стек дискового ввода/вывода

Интерфейс блочного устройства

Интерфейс блочного устройства был разработан на ранних этапах развития Unix для доступа к устройствам с единицами хранения в виде блоков по 512 байт и поддержки кэша буферов для повышения производительности. Этот интерфейс перекочевал в Linux, хотя важность кэша буферов уменьшилась с появлением других кэшей в файловых системах, о которых рассказывается в главе 8 «Файловые системы».

Unix поддерживает возможность выполнения ввода/вывода в обход кэша буферов, которая называется *низкоуровневым вводом/выводом с блочным устройством* (или просто *низкоуровневым вводом/выводом*), через специальные файлы символьных устройств (см. главу 3 «Операционные системы»). Эти файлы больше недоступны по умолчанию в Linux. Низкоуровневый ввод/вывод с блочным устройством отличается от функции «прямого ввода/вывода» в файловой системе, описанной в главе 8 «Файловые системы», но имеет некоторые схожие черты.

Интерфейс блочного ввода/вывода обычно можно исследовать с помощью инструментов ОС (`iostat(1)`). Он поддерживает широкий выбор точек для статической инструментации, а в последнее время появилась возможность исследования с помощью динамической инструментации. Linux расширил эту область ядра дополнительными возможностями.

Linux

На рис. 9.8 показаны основные компоненты стека блочного ввода/вывода в Linux.

В Linux усовершенствовали поддержку блочного ввода/вывода. В нее была добавлена возможность слияния ввода/вывода, более эффективные планировщики, поддержка группировки нескольких устройств в диспетчерах томов и средства отображения физических устройств для создания виртуальных устройств.

Слияние ввода/вывода

При создании новых запросов на выполнение ввода/вывода Linux может объединять и группировать их, как показано на рис. 9.9.

Такая группировка ввода/вывода уменьшает оверхед процессора на каждый ввод/вывод в подсистеме хранения ядра и оверхед дисков, а также повышает пропускную способность. Статистики слияний спереди и сзади доступны в `iostat(1)`.

После слияния выполняется планирование передачи запросов ввода/вывода на диски.

Планировщики ввода/вывода

Запросы на ввод/вывод ставятся в очередь и планируются на уровне блочного интерфейса либо классическими планировщиками (присутствующими только в версиях Linux старше 5.0), либо более новыми планировщиками с несколькими

очередями. Планировщики могут переупорядочивать (или перепланировать) запросы для оптимизации. Такое поведение может улучшить производительность и более справедливо распределить потребление ресурсов, особенно при использовании устройств с высокими задержками ввода/вывода (вращающиеся диски).



Рис. 9.8. Стек ввода/вывода в Linux

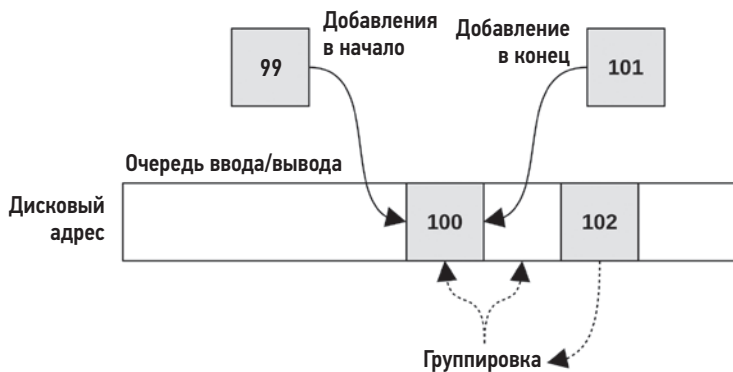


Рис. 9.9. Типы слияния ввода/вывода

К классическим планировщикам относятся:

- **Noop**: этот планировщик не выполняет никакого планирования (noop на языке процессоров означает no-operation — ничего не делать) и может использоваться, когда оверхед на планирование считается избыточным (например, для RAM-диска).
- **Deadline**: пытается установить крайний срок задержки. Используя этот планировщик, можно, например, выбрать предельное время в миллисекундах, в течение которого должна быть выполнена операция чтения или записи. Это может пригодиться в системах реального времени, когда определенность является одним из важных качеств. Также этот планировщик может решить проблему *голодания*: когда запрос на выполнение ввода/вывода испытывает нехватку дисковых ресурсов из-за того, что вновь запущенный ввод/вывод перескакивает через очередь, вследствие чего происходит выброс по задержке. Голодание может быть обусловлено проблемой *задержки чтения из-за выполнения записи* (writes starving reads), удерживающей головки в одной области диска из-за интенсивного ввода/вывода, из-за чего другие операции ввода/вывода испытывают голодание. Планировщик deadline частично решает эту проблему, используя три отдельные очереди ввода/вывода: очередь для чтения (FIFO), очередь для записи (FIFO) и отсортированную очередь [Love 10].
- **CFQ**: полностью справедливый планировщик очередей (completely fair queueing) распределяет временные интервалы ввода/вывода между процессами, по аналогии с планированием процессорного времени, для справедливого распределения дисковых ресурсов. Он также позволяет устанавливать приоритеты и классы для пользовательских процессов с помощью команды `ionice(1)`.

Проблема классических планировщиков заключается в использовании единственной очереди запросов, защищенной единственной блокировкой, которая часто оказывалась узким местом при высоких скоростях ввода/вывода. Драйвер с несколькими очередями (`blk-mq`, добавленный в Linux 3.13) решает эту проблему, используя отдельные очереди передачи для каждого процессора и несколько очередей для устройств. Это обеспечивает лучшую производительность и меньшую задержку ввода/вывода по сравнению с классическими планировщиками, потому что запросы могут обрабатываться параллельно и на том же процессоре, на котором был инициирован ввод/вывод. Это было нужно для поддержки устройств на основе флеш-памяти и других типов устройств, способных выполнять миллионы операций ввода/вывода в секунду [Corbet 13b].

К планировщикам с несколькими очередями относятся:

- **None**: планировщик без очереди.
- **BFQ**: планировщик очередей со справедливым распределением бюджета (budget fair queueing), действующий как и планировщик CFQ, но добавок к времени ввода/вывода распределяющий еще и полосу пропускания. Он создает очередь для каждого процесса, выполняющего дисковый ввод/вывод, и поддерживает бюджет для каждой очереди, измеряемый в секторах. Также есть общесистемный

бюджет тайм-аутов, чтобы ни один процесс не мог удерживать устройство слишком долго. Кроме того, планировщик BFQ поддерживает контрольные группы cgroups.

- **mq-deadline:** версия планировщика deadline (описан выше) с драйвером blk-mq.
- **Kyber:** планировщик, регулирующий длину очереди на чтение и запись в зависимости от производительности, чтобы обеспечить достижение целевых задержек при чтении или записи. Этот простой планировщик имеет всего две настройки: целевая задержка чтения (`read_lat_nsec`) и целевая задержка синхронной записи (`write_lat_nsec`). Планировщик Kyber хорошо зарекомендовал `ctmz` в облаке Netflix, где используется по умолчанию.

Начиная с Linux 5.0, планировщики с несколькими очередями используются по умолчанию (классические планировщики отключены).

Планировщики ввода/вывода подробно описываются в исходном коде Linux в каталоге `Documentation/block`.

После этапа планирования запрос помещается в очередь для отправки блочному устройству.

9.5. МЕТОДОЛОГИЯ

В этом разделе описаны методологии анализа и настройки дискового ввода/вывода. Краткий перечень методологий приводится в табл. 9.4.

Таблица 9.4. Методологии анализа производительности дисков

Раздел	Методология	Тип
9.5.1	Метод инструментов	Анализ наблюдения
9.5.2	Метод USE	Анализ наблюдения
9.5.3	Мониторинг производительности	Анализ наблюдения, планирование мощности
9.5.4	Определение характеристик рабочей нагрузки	Анализ наблюдения, планирование мощности
9.5.5	Анализ задержек	Анализ результатов наблюдения
9.5.6	Статическая настройка производительности	Анализ наблюдения, планирование мощности
9.5.7	Настройка кэширования	Анализ наблюдения, настройка
9.5.8	Управление ресурсами	Настройка
9.5.9	Микробенчмаркинг	Анализ результатов экспериментов
9.5.10	Масштабирование	Планирование мощности, настройка

Список дополнительных методологий и краткое введение в них ищите в главе 2 «Методологии».

Эти методологии можно применять по отдельности или в сочетании друг с другом. При устранении проблем с дисками предлагаю начать с использования следующих стратегий в таком порядке: метод USE, мониторинг производительности, определение характеристик рабочей нагрузки, анализ задержек, микробенчмаркинг, статический анализ и трассировка событий.

В разделе 9.6 «Инструменты наблюдения» показаны приемы применения этих методологий с использованием инструментов ОС.

9.5.1. Метод инструментов

Метод инструментов — это процесс перебора доступных инструментов с целью изучения ключевых метрик, которые они возвращают. Это очень простая методология, но при ее использовании могут оставаться незамеченными некоторые проблемы, которые плохо обнаруживаются или вообще не обнаруживаются инструментами, и на ее применение может уйти много времени.

При анализе дисков можно использовать следующие инструменты (для Linux):

- **iostat**: при использовании в расширенном режиме помогает выявить высокое потребление дисков (более 60 %), высокое среднее время обслуживания (скажем, более 10 мс) и большое количество операций ввода/вывода в секунду (в зависимости от обстоятельств);
- **iotop/biotop**: помогает определить, какой процесс генерирует дисковый ввод/вывод;
- **biolatency**: с помощью этого инструмента можно получить распределение задержек ввода/вывода в виде гистограммы и выявить мультимодальные распределения и выбросы задержек (более 100 мс);
- **biosnoop**: позволяет исследовать отдельные операции ввода/вывода;
- **perf(1)/BCC/bpftrace**: позволяет исследовать стеки в пространствах пользователя и ядра, ведущих к дисковому вводу/выводу;
- **инструменты для исследования конкретных контроллеров дисков** (от поставщика).

При обнаружении проблемы изучите все поля в выводе доступных инструментов, чтобы получить больше информации. Подробности о каждом инструменте ищите в разделе 9.6 «Инструменты наблюдения». Для выявления других типов проблем используйте и другие методологии.

9.5.2. Метод USE

Метод USE можно использовать на ранних этапах исследования производительности для выявления узких мест и ошибок во всех компонентах, прежде чем переходить к более глубоким и трудоемким стратегиям. В подразделах ниже описано,

как можно применить метод USE к дисковым устройствам и контроллерам, а в разделе 9.6 «Инструменты наблюдения» перечислены инструменты для оценки конкретных показателей.

Дисковые устройства

Для каждого диска проверьте:

- **Потребление:** время, в течение которого устройство было занято.
- **Насыщенность:** время, в течение которого запросы на ввод/вывод ждали своей очереди.
- **Ошибки:** ошибки устройства.

Сначала можно проверить ошибки. Иногда на них не обращают внимания, потому что система работает нормально, хотя и медленнее, несмотря на сбои дисков: диски обычно объединяются в пул с целью увеличения отказоустойчивости. Помимо стандартных счетчиков ошибок в ОС, дисковые устройства могут поддерживать более широкий спектр своих счетчиков, значения которых можно получить с помощью специальных инструментов (например, SMART data¹).

Если дисковые устройства — это физические диски, то оценка их потребления не должна быть сложной задачей. Если это виртуальные диски, то оценка потребления может не отражать характера работы физических дисков. См. раздел 9.3.9 «Потребление», где этот вопрос обсуждается более подробно.

Контроллеры дисков

Для каждого контроллера дисков проверьте:

- **Потребление:** текущую и максимальную пропускную способность, время и аналогичные метрики для частоты операций.
- **Насыщенность:** время, в течение которого запросы ждали своей очереди из-за насыщения контроллера.
- **Ошибки:** ошибки контроллера.

Здесь метрика потребления определяется не с точки зрения времени, а с точки зрения ограничений контроллера диска: пропускной способности (байтов в секунду) и скорости работы (операций в секунду). Под операциями подразумеваются чтение/запись и другие дисковые команды. Пропускная способность или скорость работы также могут ограничиваться транспортом, соединяющим контроллер с системой или контроллер с отдельными дисками. Каждый транспорт нужно проверять одинаково: ошибки, потребление, насыщение.

¹ В Linux см. такие инструменты, как MegaCLI и smartctl (рассматриваются ниже), cciiss-vol-status, cpqarrayd, varmon и dpt-i2o-raidutils.

Вы можете обнаружить, что инструменты наблюдения (например, `iostat(1)` в Linux) предоставляют метрики не для контроллеров, а для дисков. Эта проблема имеет обходное решение: если в системе только один контроллер, то можно определить количество операций ввода/вывода в секунду и пропускную способность контроллера, суммируя эти метрики для всех дисков. Если в системе несколько контроллеров, то нужно определить, какие диски к какому контроллеру подключены, и просуммировать соответствующие метрики.

Производительность контроллеров и транспортов часто упускается из виду. К счастью, они редко становятся узким местом в системе, потому что их пропускная способность обычно превышает пропускную способность подключенных дисков. Если общая пропускная способность диска или количество операций ввода/вывода в секунду всегда снижаются с определенной скоростью, даже при разных рабочих нагрузках, то это может указывать на то, что источником проблем действительно являются контроллеры или транспорты.

9.5.3. Мониторинг производительности

Мониторинг производительности помогает обнаружить проблемы и модели поведения, проявляющиеся со временем. Ключевые метрики для дискового ввода/вывода:

- **потребление диска;**
- **время отклика.**

Потребление диска на уровне 100 % в течение нескольких секунд, скорее всего, признак проблемы. В зависимости от среды уровень потребления более 60 % тоже может вызвать снижение производительности из-за увеличения очереди. «Нормальный» или «высокий» уровень потребления зависит от рабочей нагрузки, среды и требований к задержкам. Если вы не уверены, то можно провести микробенчмаркинг с использованием заведомо умеренных и тяжелых рабочих нагрузок, чтобы понять, как их можно определить, опираясь на метрики потребления диска. См. раздел 9.8 «Эксперименты».

Эти метрики следует оценивать для каждого диска, чтобы выявить несбалансированные рабочие нагрузки и отдельные плохо работающие диски. Метрика времени отклика может вычисляться как среднее значение в секунду и включать другие значения, например максимальное и стандартное отклонение. В идеале нужно исследовать полное распределение времени отклика, например построить гистограмму или тепловую карту, чтобы не пропустить выбросы и другие закономерности.

Если в системе используются средства управления дисковыми ресурсами, то можно также собрать статистику, показывающую, как и когда эти ресурсы использовались. Дисковый ввод/вывод может быть узким местом из-за наложенных ограничений, а не из-за высокой активности самого диска.

Потребление и время отклика показывают результирующую производительность диска. Для определения характеристик рабочей нагрузки можно добавить дополнительные метрики, включая количество операций ввода/вывода в секунду и пропускную способность, которые служат важными данными при планировании мощности (см. следующий раздел и раздел 9.5.10 «Масштабирование»).

9.5.4. Определение характеристик рабочей нагрузки

Определение характеристик приложенной нагрузки — важный шаг при планировании мощностей, сравнительном анализе и моделировании рабочих нагрузок. Это также помогает значительно увеличить производительность за счет выявления и устранения ненужной работы.

Основные атрибуты, характеризующие нагрузку на дисковый ввод/вывод:

- частота операций ввода/вывода;
- пропускная способность ввода/вывода;
- объем ввода/вывода;
- соотношение операций чтения/записи;
- произвольный и последовательный доступ к данным на дисках.

Влияние на производительность произвольного и последовательного ввода/вывода, соотношения операций чтения/записи и объема ввода/вывода было описано в разделе 9.3 «Основные понятия». Частота операций ввода/вывода (IOPS) и пропускная способность были определены в разделе 9.1 «Терминология».

Эти характеристики могут быстро меняться со временем, особенно когда в системе выполняются приложения, активно использующие файловые системы и периодически буферизующие и выталкивающие на диск блоки данных. Чтобы точнее охарактеризовать рабочую нагрузку, фиксируйте как максимальные, так и средние значения. Еще лучше изучить полное распределение значений во времени.

Пример ниже показывает, как все эти атрибуты можно выразить в одном описании:

Системные диски испытывают небольшую рабочую нагрузку произвольного чтения, в среднем 350 операций в секунду с пропускной способностью 3 Мбайт/с. Количество операций чтения составляет 96 %. Время от времени бывают короткие всплески последовательных операций записи, продолжительностью от 2 до 5 с, в результате частота операций дискового ввода/вывода достигает максимального значения 4800 в секунду, а пропускная способность возрастает до 560 Мбайт/с. В среднем чтение выполняется блоками по 8 Кбайт, а запись — по 128 Кбайт.

Эти характеристики можно измерить не только в масштабе всей системы, но также для отдельных дисков и контроллеров.

Чек-лист для определения дополнительных характеристик рабочей нагрузки

Для характеристики рабочей нагрузки можно включить дополнительные сведения. Они перечислены ниже в форме вопросов, которые могут послужить чек-листом при исследовании проблем с дисками:

- Какова общесистемная частота операций ввода/вывода в секунду? Для каждого диска? Для каждого контроллера?
- Какова пропускная способность всей системы? Каждого диска? Каждого контроллера?
- Какие приложения или пользователи используют диски?
- К каким файловым системам или файлам осуществляется доступ?
- Были ли обнаружены ошибки? Эти ошибки обусловлены неверными запросами или проблемами с диском?
- Насколько сбалансирован ввод/вывод между доступными дисками?
- Какова частота операций ввода/вывода для каждой задействованной транспортной шины?
- Какая доля пропускной способности приходится на каждую задействованную транспортную шину?
- Какие выполняются команды, не связанные с передачей данных?
- Почему выполняются операции дискового ввода/вывода (пути в коде, ведущие к системным вызовам в ядре)?
- Какая доля дискового ввода/вывода приходится на операции, выполняемые приложениями синхронно?
- Как распределяется время поступления запросов на ввод/вывод?

Вопросы, затрагивающие частоту операций и пропускную способность, можно задать отдельно для операций чтения и записи. Любые из этих метрик можно исследовать во времени, чтобы найти максимальные и минимальные значения, а также характер изменения во времени. См. также раздел 2.5.11 «Определение характеристик рабочей нагрузки» в главе 2 «Методологии», где приводится обобщенное описание этих и других характеристик (кто, почему, зачем, как).

Определение характеристик производительности

В предыдущих чек-листах перечисляются метрики, определяющие характеристики применяемой рабочей нагрузки. В списке ниже перечислены метрики результирующей производительности:

- Какова величина потребления каждого диска (нагрузка)?
- Насколько насыщен каждый диск операциями ввода/вывода (величина очереди ожидания)?

- Каково среднее время обслуживания ввода/вывода?
- Каково среднее время ожидания ввода/вывода?
- Имеются ли выбросы ввода/вывода с большой задержкой?
- Каково полное распределение задержки ввода/вывода?
- Имеются ли и активны ли системные средства управления ресурсами, в частности ресурсами ввода/вывода?
- Какова задержка выполнения дисковых команд, не связанных с передачей данных?

Трассировка событий

Инструменты трассировки можно использовать для записи деталей всех операций с файловой системой в журнал для последующего анализа (см., например, раздел 9.6.7 «biosnoop»). В число деталей могут входить: идентификатор дискового устройства, тип операции ввода/вывода или команды, смещение, размер, отметки времени начала и окончания, код завершения, а также идентификатор и имя процесса (если возможно). На основе отметок времени начала и окончания можно вычислять задержку ввода/вывода (или записывать ее в журнал напрямую). Изучая последовательность отметок времени начала и окончания обработки запроса, можно идентифицировать их переупорядочивание устройством. Трассировка может быть идеальным подходом к определению характеристик рабочей нагрузки, но на практике она часто сопряжена со значительным оверхедом из-за высокой частоты следования дисковых операций. Если запись данных трассировки на диск событий происходит в период трассировки, они могут не только загрязнять данные для анализа, но также создать петлю обратной связи и проблемы с производительностью.

9.5.5. Анализ задержек

Анализ задержек предполагает более глубокое изучение системы для поиска источника задержек. В случае с дисками исследование часто заканчивается на интерфейсе диска, где измеряется время между передачей запроса на ввод/вывод и прерыванием, уведомляющим о его выполнении. Если это время соответствует задержке ввода/вывода на уровне приложения, то можно с уверенностью предположить, что задержка ввода/вывода обусловлена особенностями работы дисков, и сосредоточить внимание на них. Если задержка отличается, ее измерение на разных уровнях стека ОС позволит определить источник.

На рис. 9.10 показан общий стек ввода/вывода с задержками на разных уровнях, соответствующих двум выбросам, А и В.

Задержка А одинакова на всех уровнях от приложения до драйверов диска. Это указывает на то, что ее причиной являются диски (или драйверы дисков). Об этом нетрудно догадаться по схожим задержкам на разных уровнях, измеряемым независимо.

Задержка В, по всей видимости, возникает на уровне файловой системы (блокировка или ожидание в очереди), потому что уровни ниже вносят гораздо меньший вклад. Имейте в виду, что на разных уровнях стека количество операций ввода/вывода может уменьшаться или увеличиваться, а это означает, что объем ввода/вывода, количество операций и величина задержки будут отличаться на разных уровнях. Пример с задержкой В может соответствовать ситуации, когда на нижних уровнях наблюдается одна задержка (10 мс), но не учитываются другие связанные операции ввода/вывода, которые обусловлены обслуживанием того же самого ввода/вывода в файловой системе (например, для обновления метаданных).

Задержки на каждом уровне можно представить в виде:

- **средних значений за интервал:** обычно сообщаются инструментами ОС;
- **полных распределений:** в виде гистограмм или тепловых карт, см. раздел 9.7.3 «Тепловые карты задержек»;
- **значений задержек:** см. предыдущий подраздел «Трассировка событий».

Последние два представления помогают определить источник выбросов и выявить случаи, когда запросы на ввод/вывод разделялись или объединялись.



Рис. 9.10. Анализ задержек в стеке ввода/вывода

9.5.6. Статическая настройка производительности

Статическая настройка производительности фокусируется на проблемах сконфигурированной архитектуры. Анализируя производительность дисков, изучите следующие аспекты статической конфигурации:

- Сколько имеется дисков? Каких типов (например, SMR, MLC)? Какой емкости?
- Версия прошивки диска?
- Сколько имеется контроллеров дисков? Какие типы интерфейсов они имеют?
- Подключены ли контроллеры дисков к высокоскоростным слотам?
- Сколько дисков подключено к каждому контроллеру?
- Имеются ли резервные батареи для питания дисков/контроллеров и какой мощности?
- Версия прошивки контроллера диска?
- Настроен ли RAID? Как именно, включая ширину полосы?
- Доступен и настроен ли режим многоканального доступа?
- Версия драйвера дискового устройства?
- Каков объем основной памяти сервера? Какой объем используется для кэшей страниц и буферов?
- Есть ли известные ошибки и исправления для каких-либо драйверов устройств хранения?
- Используются ли средства управления ресурсами дискового ввода/вывода?

Имейте в виду, что в драйверах устройств и прошивке могут быть ошибки, влияющие на производительность, которые в идеале исправляются обновлениями от поставщика.

Эти ответы помогут выявить ранее упущенные из виду варианты конфигурации. Иногда бывает так, что систему настроили для одной рабочей нагрузки, а затем перепрофилировали для другой. Этот метод напомним о необходимости вернуться к выбору параметров.

Когда я работал ведущим специалистом по производительности хранилищ на ZFS в Sun, то наиболее частые жалобы на производительность, которые я получал, были вызваны неправильной конфигурацией: использованием половины JBOD (12 дисков) в RAID-Z2 (с широкими полосами). Эта конфигурация обеспечивает высокую надежность, но уступает по производительности отдельным дискам. Я научился сначала запрашивать у клиентов детали конфигурации и только потом входить к ним в систему и тратить время на изучение задержек ввода/вывода.

9.5.7. Настройка кэширования

В системе может быть много разных кэшей, включая кэши в приложениях, в файловых системах, в контроллерах дисков и в самих дисках. Они были перечислены в разделе 9.3.3 «Кэширование». Их можно настроить, как описано в главе 2 «Методологии», в разделе 2.5.18 «Настройка кэширования». Поэтому проверьте, какие кэши есть, убедитесь, что они работают, и проверьте, насколько хорошо они работают, а затем настройте рабочую нагрузку для кэша и настройте кэш для рабочей нагрузки.

9.5.8. Управление ресурсами

Операционная система может предоставлять средства управления для распределения ресурсов дискового ввода/вывода между процессами или группами процессов, позволяющие устанавливать фиксированные ограничения по частоте операций ввода/вывода и пропускной способности или ее доли. Как это работает, зависит от реализации и обсуждается в разделе 9.9 «Настройка».

9.5.9. Микробенчмаркинг

Микробенчмаркинг дискового ввода/вывода был представлен в главе 8 «Файловые системы», где объяснялась разница между тестированием ввода/вывода файловой системы и тестированием дискового ввода/вывода. Под тестированием дискового ввода/вывода обычно подразумевается тестирование через пути к устройствам, которые есть в ОС. В частности, через низкоуровневый интерфейс к устройствам, если он доступен, позволяющий исключить из тестирования поведение файловой системы (включая кэширование, буферизацию, разделение и объединение ввода/вывода, оверхед на выполнение кода и отображение смещений).

Вот основные факторы, которые можно проверить:

- **направление:** чтение или запись;
- **характер доступа к диску:** произвольный или последовательный;
- **диапазон смещений:** весь диск или узкие диапазоны (например, тестируется только смещение 0);
- **размер ввода/вывода:** от 512 байт (типичный минимум) до 1 Мбайт;
- **конкуренция:** количество операций ввода/вывода, выполняемых одновременно, или количество потоков, выполняющих операции ввода/вывода;
- **количество устройств:** один диск или несколько дисков (для изучения ограничений контроллера и шины).

В подразделах ниже показано, как можно комбинировать эти факторы для исследования производительности диска и контроллера. Подробную информацию о конкретных инструментах для проведения этих тестов ищите в разделе 9.8 «Эксперименты».

Диски

Микробенчмаркинг может быть выполнен для каждого диска для определения следующих метрик и предлагаемых рабочих нагрузок:

- **максимальная пропускная способность диска** (мегабайт в секунду): последовательное чтение блоками по 128 Кбайт или 1 Мбайт;
- **максимальная частота операций с диском (IOPS):** чтение блоками по 512 байт¹, только из смещения 0;

¹ Этот размер должен соответствовать наименьшему размеру блока на диске. Во многих современных дисках используются блоки размером 4 Кбайт.

- **максимальное количество произвольных чтений (IOPS):** чтение блоками по 512 байт из случайно выбираемых смещений;
- **профиль задержки чтения** (средние значения в микросекундах): многократное последовательное чтение блоками по 512 байт, 1 Кбайт, 2 Кбайт, 4 Кбайт и т. д.;
- **профиль задержки произвольного ввода/вывода** (средние значения в микросекундах): многократное чтение блоками по 512 байт в полном диапазоне смещений, только в области начальных смещений, только в области конечных смещений.

Эти тесты можно повторить для операций записи. Случай «только из смещения 0» позволяет протестировать кэширование данных на диске и оценить время доступа к кэшу¹.

Контроллеры дисков

Для микробенчмаркинга контроллеров дисков можно применить рабочую нагрузку к нескольким дискам, чтобы достичь предельных возможностей контроллера. При тестировании контроллеров можно определить следующие метрики и характер рекомендуемых рабочих нагрузок:

- **максимальная пропускная способность контроллера** (мегабайт в секунду): чтение блоками по 128 Кбайт, только из смещения 0;
- **максимальная частота выполнения операций контроллером (IOPS):** чтение блоками по 512 байт, только из смещения 0.

Примените рабочую нагрузку, последовательно увеличивая количество охватываемых дисков, и следите за ограничениями. Чтобы найти предел в контроллере дисков, может потребоваться приложить нагрузку к десятку дисков.

Этот размер должен соответствовать наименьшему размеру блока на диске. Во многих современных дисках используются блоки размером 4 Кбайт.

9.5.10. Масштабирование

Диски и контроллеры дисков имеют ограничения по пропускной способности и количеству операций ввода/вывода в секунду (IOPS), что можно показать с помощью микробенчмаркинга, как было описано выше. Настройка может улучшить производительность только до этих пределов. Если требуется повышенная производительность диска, а кэширование, например, не дает желаемого результата, то дисковую подсистему нужно масштабировать.

¹ Поговаривают, что некоторые производители накопителей оптимизируют прошивки для ускорения операций ввода/вывода с сектором 0, что завышает результаты такого теста. Это можно проверить, сравнив производительность операций с сектором 0 и производительность операций с любым другим сектором по вашему выбору.

Вот простой метод, основанный на планировании емкости ресурсов:

1. Определите целевую рабочую нагрузку в терминах пропускной способности и количества операций ввода/вывода в секунду. Если это новая система, то см. главу 2 «Методологии», раздел 2.7 «Планирование мощности». Если система уже имеет рабочую нагрузку, выразите количество пользователей в терминах текущей дисковой пропускной способности и IOPS и масштабируйте эти числа для целевой группы пользователей. (Если параллельно нельзя масштабировать кэш, то рабочая нагрузка на диск может увеличиться, потому что уменьшается доля объема кэша, приходящаяся на одного пользователя.)
2. Рассчитайте количество дисков, необходимых для поддержки этой рабочей нагрузки. Учтите фактор конфигурации RAID. Не используйте в расчетах максимальную пропускную способность и IOPS на диск, так как это приведет к 100 %-ной нагрузке на диски, что тут же повлечет проблемы с производительностью из-за насыщения и увеличения очередей. Выберите целевое потребление (скажем, 50 %) и соответственно масштабируйте значения.
3. Рассчитайте количество контроллеров дисков, которое нужно для поддержки этой рабочей нагрузки.
4. Убедитесь, что не превышены пределы возможностей транспортов, и при необходимости масштабируйте транспорты.
5. Рассчитайте количество тактов процессора, расходуемых на дисковый ввод/вывод и нужное количество процессоров (для этого может потребоваться несколько процессоров и организация параллельного ввода/вывода).

Максимальная пропускная способность каждого диска и величина IOPS будут зависеть от типа ввода/вывода и типов дисков. См. раздел 9.3.7 «Несопоставимость количества операций в секунду». Микробенчмаркинг можно использовать для определения конкретных ограничений для заданного размера и типа ввода/вывода, а определение характеристик рабочей нагрузки — на существующих рабочих нагрузках, чтобы увидеть, какие размеры и типы имеют значение.

Для удовлетворения требований, предъявляемых рабочей нагрузкой, нередко могут понадобиться серверы с десятками дисков, объединенных в массивы хранения. Раньше мы говорили: «Добавляйте больше шпинделей». Теперь же говорим: «Добавляйте больше флеш-памяти».

9.6. ИНСТРУМЕНТЫ НАБЛЮДЕНИЯ

В этом разделе представлены инструменты наблюдения за работой дисков, доступные в ОС на базе Linux. Стратегии их использования описаны в предыдущем разделе.

Инструменты перечислены в табл. 9.5.

Таблица 9.5. Инструменты наблюдения за работой дисков

Раздел	Инструмент	Описание
9.6.1	iostat	Различные статистики, характеризующие дисковый ввод/вывод
9.6.2	sar	Исторические статистики
9.6.3	PSI	Информация о времени, затраченном на ожидание доступности диска
9.6.4	pidstat	Разбивка потребления дисков по процессам
9.6.5	perf	Точки трассировки для анализа производительности блочного ввода/вывода
9.6.6	biolatency	Распределение задержек дискового ввода/вывода в виде гистограммы
9.6.7	biosnoop	Трассировка дискового ввода/вывода, генерируемого процессами, с выводом информации о задержках
9.6.8	iotop, biotop	Аналоги команды top для дисков: обобщают характеристики блочного ввода/вывода по процессам
9.6.9	biostacks	Отображает дисковый ввод/вывод со стеками инициализации
9.6.10	blktrace	Трассировка событий дискового ввода/вывода
9.6.11	bpfttrace	Настраиваемая трассировка дискового ввода/вывода
9.6.12	MegaCli	Инструмент компании LSI, выводит статистики работы контроллера
9.6.13	smartctl	Статистики работы контроллера

Описание этого набора инструментов и возможностей дополняет раздел 9.5 «Методология». Он начинается с описания традиционных инструментов, затем переходит к инструментам трассировки и, наконец, к инструментам для получения статистик работы контроллера диска. Некоторые из традиционных инструментов, вероятно, доступны (а иногда первоначально созданы) в других Unix-подобных ОС, в том числе iostat(8) и sar(1). Многие из инструментов трассировки основаны на BPF и используют интерфейсы BCC и bpftrace (глава 15), в том числе biolatency(8), biosnoop(8), biotop(8) и biostacks(8).

Полный перечень возможностей каждого инструмента вы найдете в соответствующей документации, включая страницы справочного руководства man.

9.6.1. iostat

iostat(1) выводит сводную статистику ввода/вывода для каждого диска, сообщая метрики, характеризующие рабочую нагрузку, потребление и насыщение. Эта команда может запускаться рядовыми пользователями и обычно используется при исследовании проблем дискового ввода/вывода. Статистики, возвращаемые этим инструментом, также можно получить с помощью ПО для мониторинга, поэтому полезно поближе познакомиться с iostat(1), чтобы углубить понимание статистик

мониторинга. Кроме того, предоставляемые статистики по умолчанию собираются ядром¹, поэтому `iostat(1)` не оказывает большой нагрузки на систему.

Название «`iostat`» — это сокращение от «I/O statistics» (статистики ввода/вывода). Но правильнее, пожалуй, было бы назвать этот инструмент «`diskiostat`», чтобы подчеркнуть тип ввода/вывода, характеристики которого он сообщает. Иногда такое название могло запутать, когда пользователь знал, что приложение выполняет ввод/вывод (в файловую систему), но не видел его через `iostat(1)` (диски).

Утилита `iostat(1)` была написана в начале 1980-х для Unix, и сейчас доступны разные ее версии для разных ОС. Ее можно добавить в системы на базе Linux, установив пакет `sysstat`. Ниже описана версия для Linux.

Вывод `iostat` по умолчанию

После запуска без аргументов и параметров команда выводит сводку о потреблении процессоров и дисков, накопленную с момента загрузки. Ниже приведен пример вывода по умолчанию для первичного знакомства с инструментом. Но чаще приходится использовать расширенный режим, описанный ниже, который дает больше полезной информации.

```
$ iostat
Linux 5.3.0-1010-aws (ip-10-1-239-218)      02/12/20      _x86_64_      (2 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           0.29    0.01   0.18    0.03    0.21   99.28

Device            tps    kB_read/s    kB_wrtn/s    kB_read    kB_wrtn
loop0              0.00         0.05         0.00       1232         0
[...]
nvme0n1            3.40        17.11        36.03     409902     863344
```

В первой строке дана общая информация о системе, включая версию ядра, имя хоста, дату, архитектуру и количество процессоров. В следующих строках выводятся статистики, накопленные с момента загрузки системы, в том числе среднее потребление процессора (`avg-cpu`, эта статистика была описана в главе 6 «Процессоры») и дисковых устройств (в разделе `Device:`).

Каждому дисковому устройству соответствует отдельная строка с основными сведениями в столбцах. Заголовки столбцов в листинге выделены жирным. Среди них:

- **tps**: количество транзакций в секунду (IOPS);
- **kB_read/s**, **kB_wrtn/s**: объемы чтения и записи в килобайтах в секунду соответственно;
- **kB_read**, **kB_wrtn**: суммарный объем прочитанных и записанных данных в килобайтах.

¹ Сбор статистик можно отключить через файл `/sys/block/<dev>/queue/iostats`, но я еще не встречал никого, кто сделал бы это.

Некоторые устройства SCSI, включая CD-ROM, могут не отображаться командой `iostat(1)`. Информацию о ленточных накопителях SCSI можно получить с помощью утилиты `tapestat(1)` из того же пакета `sysstat`. Также обратите внимание, что `iostat(1)` сообщает информацию об операциях чтения и записи с блочными устройствами, но может исключать некоторые другие типы команд, посылаемых дисковым устройствам, в зависимости от ядра (см. реализацию функции ядра `blk_do_io_stat()`). При использовании в расширенном режиме `iostat(1)` выводит дополнительные поля с информацией об этих командах.

Параметры iostat

Команде `iostat(1)` можно передавать различные параметры, за которыми следуют необязательные значения интервала и счетчик. Например:

```
# iostat 1 10
```

10 раз выведет статистики с интервалом в 1 с. А:

```
# iostat 1
```

будет выводить статистики бесконечно (до нажатия Ctrl-C) с интервалом в 1 с.

Вот некоторые наиболее часто используемые параметры:

- **-c**: выводит отчет о потреблении процессора;
- **-d**: выводит отчет о потреблении дисков;
- **-k**: выводит объемы в килобайтах вместо блоков (по 512 байт);
- **-m**: выводит объемы в мегабайтах вместо блоков (по 512 байт);
- **-p**: добавляет статистики по разделам;
- **-t**: выводит отметки времени;
- **-x**: выводит расширенные статистики;
- **-s**: режим узкого вывода;
- **-z**: пропускает вывод информации об устройствах с нулевой активностью.

Есть переменная среды `POSIXLY_CORRECT=1` для вывода объемов в блоках (по 512 байт каждый) вместо килобайтов. Некоторые старые версии поддерживали параметр `-n` для получения статистик о NFS. Начиная с версии `sysstat 9.1.3`, эта возможность была реализована в виде команды `nfsiostat`.

Режим расширенного узкого вывода iostat

В режиме расширенного вывода (`-x`) `iostat(1)` выводит дополнительные столбцы, которые пригодятся при выполнении описанных выше действий. В этих дополнительных столбцах выводятся: метрики IOPS и пропускной способности для определения характеристик рабочей нагрузки, потребление дисков и размеры очередей

для метода USE, а также время отклика диска для определения характеристик производительности и анализа задержек.

Со временем в расширенный вывод добавлялось все больше полей, и в последней версии (12.3.1, декабрь 2019-го) вывод был шириной 197 символов. Он не умещается не только по ширине книжной страницы, но и во многих терминалах, что затрудняет чтение вывода из-за переноса строк. В 2017 году было добавлено решение, параметр `-s`, обеспечивающий «узкий» вывод и предназначенный для отображения в терминалах с шириной строк в 80 символов.

Вот пример узкого (`-s`) и расширенного (`-x`) вывода статистик с пропуском устройств с нулевой активностью (`-z`):

```
$ iostat -sxz 1
[...]
avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           15.82    0.00   10.71   31.63    1.53   40.31

Device            tps        kB/s        rqm/s    await  aqu-sz   areq-sz   %util
nvme0n1           1642.00    9064.00    664.00    0.44   0.00     5.52   100.00
[...]

```

Значение столбцов:

- **tps**: количество транзакций в секунду (IOPS);
- **kB/s**: килобайт в секунду;
- **rqm/s**: количество запросов в секунду, добавленных в очередь и объединенных;
- **await**: среднее время отклика ввода/вывода, включая время ожидания в очереди драйвера и время отклика устройства ввода/вывода (миллисекунд);
- **aqu-sz**: среднее количество запросов, ожидающих в очереди запросов драйвера и обрабатываемых устройством;
- **areq-sz**: средний размер запроса в килобайтах;
- **%util**: процент времени, в течение которого устройство было занято обработкой запросов ввода/вывода (потребление).

Наиболее важная метрика производительности — это **await**, показывающая общее среднее время ожидания ввода/вывода. Какое значение считать «хорошим» или «плохим», зависит от ваших потребностей. Как показано в примере вывода, среднее время ожидания (**await**) составило 0,44 мс, что вполне удовлетворительно для этого сервера базы данных. Это время может увеличиваться по разным причинам: из-за ожидания в очередях (из-за высокой нагрузки), больших размеров ввода/вывода, произвольного ввода/вывода на вращающихся дисках и ошибок устройств.

Метрика **%util** важна для планирования мощности и потребления ресурсов, но учтите, что это всего лишь оценка потребления (доля времени, в течение которого устройство было занято обработкой запросов) и она может почти ничего не значить для виртуальных устройств, охватывающих несколько дисков. Такие устройства

лучше исследовать по значению приложенной нагрузки: **tps** (IOPS) и **kB/s** (пропускная способность).

Ненулевые значения в столбце **rqm/s** показывают, что последовательные запросы объединялись перед отправкой на устройство для повышения производительности. Этот показатель также является признаком последовательной нагрузки.

Поскольку величина **areq-sz** измеряется после слияния запросов, небольшие значения (8 Кбайт или меньше) могут служить признаком рабочей нагрузки произвольного ввода/вывода, запросы которой не удалось объединить. Большие размеры могут соответствовать либо большим операциям ввода/вывода, либо объединенным запросам рабочей нагрузки последовательного ввода/вывода (как в примере выше).

Режим расширенного вывода *iostat*

Без параметра **-s** команда *iostat*(1) с параметром **-x** выводит намного больше столбцов. Вот сводная информация, накопленная с момента загрузки системы (однократный вывод), которую выводит *iostat*(1) из пакета *sysstat* версии 12.3.2 (апрель 2020-го):

```
$ iostat -x
[...]
```

Device	r/s	rkB/s	rrqm/s	%rrqm	r_await	rareq-sz	w/s	wkB/s	
wrqm/s	%wrqm	w_await	wareq-sz	d/s	dkB/s	drqm/s	%drqm	d_await	dareq-sz
f/s	f_await	aq-sz	%util						
nvme0n1		0.23	9.91	0.16	40.70	0.56	43.01	3.10	33.09
0.92	22.91	0.89	10.66	0.00	0.00	0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.12						

Многие метрики, доступные с параметрами **-sx**, разбиты на компоненты, соответствующие операциям чтения, записи, синхронизации (**flush**) и усечения (**discard**). Вот краткое описание дополнительных столбцов:

- **r/s, w/s, d/s, f/s**: запросов на чтение, запись, усечение и синхронизацию, выполняемых дисковым устройством в секунду (после слияния);
- **rkB/s, wkB/s, dkB/s**: объем чтения, записи и усечения в килобайтах в секунду;
- **%rrqm/s, %wrqm/s, %drqm/s**: количество запросов в секунду на чтение, запись и усечение, добавленных в очередь и объединенных, в процентах от общего количества запросов каждого типа;
- **r_await, w_await, d_await, f_await**: среднее время отклика для чтения, записи, усечения и синхронизации, включая время ожидания в очереди драйвера и время отклика устройства (миллисекунд);
- **reduq-sz, wareq-sz, dareq-sz**: средний размер чтения, записи и усечения (килобайт).

Возможность исследовать операции чтения и записи по отдельности очень важна. Приложения и файловые системы обычно используют приемы уменьшения

задержки записи (например, кэширование с отложенной записью) и тем самым уменьшают вероятность блокировки приложения на время выполнения записи на диск. Это означает, что любые метрики, суммирующие операции чтения и записи, искажаются компонентом, который может прямо не влиять на производительность (операции записи). Благодаря разделению вы можете начать изучение метрики `r_wait`, которая показывает среднюю задержку чтения и, вероятно, является наиболее важным показателем производительности приложения.

Метрики, оценивающие производительность чтения и записи в виде количества операций ввода/вывода в секунду (`r/s`, `w/s`) и пропускной способности (`rkB/s`, `wkB/s`), можно использовать для определения характеристик рабочей нагрузки.

Поддержка вывода статистик отмены и синхронизации появилась в `iostat(1)` недавно. Операции усечения (`discard`) освобождают блоки на диске (команда ATA TRIM), а соответствующая статистика была добавлена в ядро Linux 4.19. Статистика для операций синхронизации (`flush`) была добавлена в Linux 5.5. Их изучение помогает сузить поле поиска причин задержки диска.

Вот еще одна полезная комбинация параметров `iostat(1)`:

```
$ iostat -dmstxz -p ALL 1
Linux 5.3.0-1010-aws (ip-10-1-239-218)    02/12/20    _x86_64_    (2 CPU)

02/12/20 17:39:29
Device            tps          MB/s      rqm/s    await   areq-sz   aqu-sz   %util
nvme0n1            3.33         0.04       1.09     0.87    12.84     0.00     0.12
nvme0n1p1          3.31         0.04       1.09     0.87    12.91     0.00     0.12

02/12/20 17:39:30
Device            tps          MB/s      rqm/s    await   areq-sz   aqu-sz   %util
nvme0n1          1730.00      14.97     709.00     0.54     8.86     0.02    99.60
nvme0n1p1        1538.00      14.97     709.00     0.61     9.97     0.02    99.60
[...]
```

Первый вывод — это сводная информация о системе, за которой следуют блоки статистик, выводящиеся с интервалом в одну секунду. Параметр `-d` требует выводить только статистики потребления диска (без статистик процессора), `-m` — значения объемов в мегабайтах и `-t` — значения отметок времени, что может пригодиться при сравнении результатов из других источников с отметками времени, а параметр `-p ALL` включает вывод статистик по разделам.

К сожалению, текущая версия `iostat(1)` не выводит информацию об ошибках диска. В противном случае все метрики метода `USE` можно было бы получить с помощью одного инструмента.

9.6.2. sar

Генератор отчетов о работе системы (system activity reporter) `sar(1)` можно использовать для наблюдения за текущей активностью или настроить для архивирования статистик и генерирования хронологических отчетов. Он был представлен в главе 4

«Инструменты наблюдения», в разделе 4.4 «sar» и упоминается в других главах, где используется для получения соответствующих статистик.

При вызове с параметром `-d` инструмент `sar(1)` выводит сводную информацию о потреблении диска с интервалом в одну секунду, как показано в следующих примерах. Вывод довольно широкий, поэтому здесь я разбил его на две части (`sysstat 12.3.2`):

```
$ sar -d 1
Linux 5.3.0-1010-aws (ip-10-0-239-218) 02/13/20 _x86_64_ (2 CPU)
09:10:22 DEV tps rkB/s wkB/s dkB/s areq-sz \ ...
09:10:23 dev259-0 1509.00 11100.00 12776.00 0.00 15.82 / ...
[...]
```

и остальные столбцы:

```
$ sar -d 1
09:10:22 \ ... \ aqu-sz await %util
09:10:23 / ... / 0.02 0.60 94.00
[...]
```

Эти столбцы есть в выводе `iostat(1)` с параметром `-x`, они были описаны в предыдущем разделе. В этом выводе мы видим смешанную рабочую нагрузку чтения/записи со значением `await` 0,6 мс, что обусловлено потреблением диска на уровне 94 %.

Предыдущие версии `sar(1)` включали столбец `svctm` (service time — время обслуживания): среднее (расчетное) время отклика диска в миллисекундах. Информацию о времени обслуживания см. в разделе 9.3.1 «Измерение времени». Но так как упрощенный алгоритм вычисления времени обслуживания давал неверные результаты для современных дисков, поддерживающих возможность параллельного ввода/вывода, в более поздних версиях столбец `svctm` удалили.

9.6.3. PSI

Информация о задержках доступа в Linux (pressure stall information, PSI), добавленная в Linux 4.20, включает статистики, характеризующие насыщение ввода/вывода. Они не только показывают наличие задержек, но и то, как они менялись за последние 5 минут. Вот пример вывода:

```
# cat /proc/pressure/io
some avg10=63.11 avg60=32.18 avg300=8.62 total=667212021
full avg10=60.76 avg60=31.13 avg300=8.35 total=622722632
```

Как показывает этот вывод, насыщение ввода/вывода увеличивается, потому что среднее значение за 10 с (63,11) выше среднего за 300 с (8,62). Эти средние значения представляют процент времени, в течение которого задача была заблокирована в ожидании ввода/вывода. Строка `some` (некоторые) обобщает случаи, когда задержкой были затронуты некоторые задачи (потoki), а строка `full` (все) обобщает случаи, когда задержкой были затронуты все выполняемые задачи.

Так же как со средними нагрузками, эти значения могут служить высокоуровневыми метриками для рассылки оповещений. Узнав о появлении проблемы

с производительностью диска, вы можете использовать другие инструменты для поиска основных причин, включая `pidstat(8)` для получения статистик потребления диска по процессам.

9.6.4. pidstat

По умолчанию инструмент `pidstat(1)` в Linux выводит данные о потреблении процессора по процессам, но еще поддерживает параметр `-d` для вывода статистик потребления дискового ввода/вывода. Эта возможность доступна в ядрах 2.6.20 и выше. Например:

```
$ pidstat -d 1
Linux 5.3.0-1010-aws (ip-10-0-239-218) 02/13/20 _x86_64_ (2 CPU)
09:47:41      UID      PID    kB_rd/s    kB_wr/s kB_ccwr/s iodelay Command
09:47:42          0    2705   32468.00         0.00         0.00      5 tar
09:47:42          0    2706         0.00  8192.00         0.00      0 gzip
[...]
09:47:56      UID      PID    kB_rd/s    kB_wr/s kB_ccwr/s iodelay Command
09:47:57          0     229         0.00    72.00         0.00      0 systemd-journal
09:47:57          0     380         0.00     4.00         0.00      0 auditd
09:47:57          0    2699         4.00         0.00         0.00    10 kworker/
u4:1-flush-259:0
09:47:57          0    2705   15104.00         0.00         0.00      0 tar
09:47:57          0    2706         0.00  6912.00         0.00      0 gzip
```

Вот значение некоторых столбцов:

- **kB_rd/s**: прочитано килобайт в секунду;
- **kB_wr/s**: записано килобайт в секунду;
- **kB_ccwr/s**: отменено для записи килобайт в секунду (например, при повторной записи или при удалении перед выталкиванием на диск);
- **iodelay**: время, в течение которого процесс оставался заблокированным в ожидании завершения дискового ввода/вывода (в тактах часов), включая подкачку.

Роль рабочей нагрузки для этого примера играла команда `tar`, читающая файлы и передающая данные через конвейер команде `gzip`, которая, в свою очередь, читала данные из конвейера, сжимала их и записывала в файл архива. Операциям чтения в `tar` в столбце `iodelay` соответствует значение 5 (тактов), тогда операции записи в `gzip` не блокировались благодаря кэшированию с отложенной записью в кэше страниц. Некоторое время спустя кэш страницы был вытолкнут на диск, о чем можно судить по задержке `iodelay`, возникшей в процессе `kworker/u4:1-flush-259:0`.

`iodelay` — достаточно новое дополнение, помогающее оценить масштабы проблем с производительностью: время, в течение которого приложение ожидало ввода/вывода. Остальные столбцы отражают характеристики приложенной рабочей нагрузки.

Обратите внимание: чтобы получить доступ к статистикам потребления диска для процессов, запущенных от лица других пользователей, нужно обладать привилегиями суперпользователя (`root`). Они доступны также в `/proc/PID io`.

9.6.5. perf

Инструмент `perf(1)` (глава 13) для Linux поддерживает возможность инструментации точек трассировки блочного вывода. Вот как можно получить их список:

```
# perf list 'block:*
```

List of pre-defined events (to be used in `-e`):

block:block_bio_backmerge	[Tracepoint event]
block:block_bio_bounce	[Tracepoint event]
block:block_bio_complete	[Tracepoint event]
block:block_bio_frontmerge	[Tracepoint event]
block:block_bio_queue	[Tracepoint event]
block:block_bio_remap	[Tracepoint event]
block:block_dirty_buffer	[Tracepoint event]
block:block_getrq	[Tracepoint event]
block:block_plug	[Tracepoint event]
block:block_rq_complete	[Tracepoint event]
block:block_rq_insert	[Tracepoint event]
block:block_rq_issue	[Tracepoint event]
block:block_rq_remap	[Tracepoint event]
block:block_rq_requeue	[Tracepoint event]
block:block_sleeprq	[Tracepoint event]
block:block_split	[Tracepoint event]
block:block_touch_buffer	[Tracepoint event]
block:block_unplug	[Tracepoint event]

Вот пример регистрации событий передачи запросов блочному устройству с трассировками стека. Команда `sleep 10` задает продолжительность трассировки.

```
# perf record -e block:block_rq_issue -a -g sleep 10
[ perf record: Woken up 22 times to write data ]
[ perf record: Captured and wrote 5.701 MB perf.data (19267 samples) ]
# perf script --header
[...]
```

mysqld 1965 [001] 160501.158573: block:block_rq_issue: 259,0 WS 12288 () 10329704 + 24 [mysqld]

```

ffffffffffb12d5040 blk_mq_start_request+0xa0 ([kernel.kallsyms])
ffffffffffb12d5040 blk_mq_start_request+0xa0 ([kernel.kallsyms])
ffffffffffb1532b4c nvme_queue_rq+0x16c ([kernel.kallsyms])
ffffffffffb12d7b46 __blk_mq_try_issue_directly+0x116 ([kernel.kallsyms])
ffffffffffb12d87bb blk_mq_request_issue_directly+0x4b ([kernel.kallsyms])
ffffffffffb12d8896 blk_mq_try_issue_list_directly+0x46 ([kernel.kallsyms])
ffffffffffb12dce7e blk_mq_sched_insert_requests+0xae ([kernel.kallsyms])
ffffffffffb12d86c8 blk_mq_flush_plug_list+0x1e8 ([kernel.kallsyms])
ffffffffffb12cd623 blk_flush_plug_list+0xe3 ([kernel.kallsyms])
ffffffffffb12cd676 blk_finish_plug+0x26 ([kernel.kallsyms])
ffffffffffb119771c ext4_writepages+0x77c ([kernel.kallsyms])
ffffffffffb10209c3 do_writepages+0x43 ([kernel.kallsyms])
ffffffffffb1017ed5 __filemap_fdatawrite_range+0xd5 ([kernel.kallsyms])
ffffffffffb10186ca file_write_and_wait_range+0x5a ([kernel.kallsyms])
ffffffffffb118637f ext4_sync_file+0x8f ([kernel.kallsyms])
ffffffffffb1105869 vfs_fsync_range+0x49 ([kernel.kallsyms])
```

```

fffffffb11058fd do_fsync+0x3d ([kernel.kallsyms])
fffffffb1105944 __x64_sys_fsync+0x14 ([kernel.kallsyms])
fffffffb0e044ca do_syscall_64+0x5a ([kernel.kallsyms])
fffffffb1a0008c entry_SYSCALL_64_after_hwframe+0x44 ([kernel.kallsyms])
7f2285d1988b fsync+0x3b (/usr/lib/x86_64-linux-gnu/libpthread-2.30.so)
55ac10a05ebe Fil_shard::redo_space_flush+0x44e (/usr/sbin/mysqld)
55ac10a06179 Fil_shard::flush_file_redo+0x99 (/usr/sbin/mysqld)
55ac1076ff1c [unknown] (/usr/sbin/mysqld)
55ac10777030 log_flusher+0x520 (/usr/sbin/mysqld)
55ac10748d61
std::thread::_State_impl<std::thread::_Invoker<std::tuple<Runnable, void (*) (log_t*),
log_t*> > >::_M_run+0xc1 (/usr/sbin/mysqld)
7f2285d1988b [unknown] (/usr/lib/x86_64-linux-gnu/libstdc++.so.6.0.28)
7f226c3652c0 [unknown] ([unknown])
55ac107499f0
std::thread::_State_impl<std::thread::_Invoker<std::tuple<Runnable, void (*) (log_t*),
log_t*> > >::~_~State_impl+0x0 (/usr/sbin/
5441554156415741 [unknown] ([unknown])
[...]
```

Для каждого события выводится строка со сводной информацией о событии с трассировкой стека, которая привела к этому событию. Строка со сводной информацией начинается с полей, которые идут по умолчанию: имени процесса, идентификатора потока, идентификатора процессора, отметки времени и имени события (см. главу 13 «perf», раздел 13.11 «Сценарий perf»). Остальные поля относятся к точке трассировки. В частности, для точки трассировки `block:block_rq_issue` выводятся:

- **старший и младший номера диска:** 259,0;
- **тип ввода/вывода:** `WS` (synchronous writes — синхронная запись);
- **объем ввода/вывода:** 12288 (байт);
- **командная строка ввода/вывода:** ();
- **адрес сектора:** 10329704;
- **количество секторов:** 24;
- **процесс:** `mysqld`.

Эти поля извлекаются из строки формата точки трассировки (см. главу 4 «Инструменты наблюдения», раздел 4.3.5 «Точки трассировки», подраздел «Аргументы и строка формата точки трассировки»).

Трассировка стека помогает объяснить природу дискового ввода/вывода. Здесь инициатором является подпрограмма `log_flusher()` в `mysqld`, которая вызвала `fsync(2)`. Путь к коду ядра показывает, что ввод/вывод обрабатывался файловой системой `ext4` и превратился в проблему дискового ввода/вывода из-за вызова `blk_mq_try_issue_list_directly()`.

Операции ввода/вывода часто ставятся в очередь, а затем выполняются потоком ядра, поэтому при наблюдении за точкой трассировки `block:block_rq_issue`

информация об исходном процессе или трассировка стека в пространстве пользователя могут оказаться недоступными. В таких случаях понаблюдайте за точкой трассировки `block:block_rq_insert`, которая находится в операции постановки в очередь. Обратите внимание, что она не замечает ввода/вывода, который производится в обход очереди.

Однотрочные сценарии

Однотрочные сценарии ниже показывают использование фильтров с точками трассировки `block`.

Регистрация всех завершенных блочных операций размером не менее 100 Кбайт до нажатия комбинации `Ctrl-C`¹:

```
perf record -e block:block_rq_complete --filter 'nr_sector > 200'
```

Регистрация всех завершенных блочных операций синхронной записи до нажатия `Ctrl-C`:

```
perf record -e block:block_rq_complete --filter 'rwbs == "WS"
```

Регистрация всех завершенных блочных операций записи любых типов до нажатия `Ctrl-C`:

```
perf record -e block:block_rq_complete --filter 'rwbs ~ "*W*"'
```

Задержка дискового ввода/вывода

Задержку дискового ввода/вывода (описанную выше как *время обработки запроса диском*) также можно определить путем регистрации событий отправки и завершения обработки запроса для последующего анализа. Пример ниже регистрирует эти события в течение 60 с, а затем выводит в файл `out.disk01.txt`:

```
perf record -e block:block_rq_issue,block:block_rq_complete -a sleep 60
perf script --header > out.disk01.txt
```

Обработать выходной файл можно с помощью любого удобного вам инструмента: `awk`(1), Perl, Python, R, Google Spreadsheets и т. д. Нужно найти соответствующие друг другу события отправки и завершения запросов и использовать отметки времени для вычисления задержки.

Следующие инструменты, `biolateness`(8) и `biosnoop`(8), эффективно вычисляют задержку дискового ввода/вывода в пространстве ядра с помощью BPF и включают величину задержки непосредственно в вывод.

¹ При размере сектора 512 байт объему 100 Кбайт соответствует 200 секторов.

9.6.6. biolatenсy

biolatenсy(8)¹ — это инструмент для ВСС и bpftrace, отображающий задержки дискового ввода/вывода в виде гистограммы. Под *задержкой ввода/вывода* здесь понимается время между передачей запроса устройству и завершением его выполнения (также известное как время обработки запроса диском).

Ниже приведен пример использования biolatenсy(8) для трассировки блочного ввода/вывода в ВСС в течение 10 с:

```
# biolatenсy 10 1
```

```
Tracing block device I/O... Hit Ctrl-C to end.
```

usecs	: count	distribution
0 -> 1	: 0	
2 -> 3	: 0	
4 -> 7	: 0	
8 -> 15	: 0	
16 -> 31	: 2	
32 -> 63	: 0	
64 -> 127	: 0	
128 -> 255	: 1065	*****
256 -> 511	: 2462	*****
512 -> 1023	: 1949	*****
1024 -> 2047	: 373	*****
2048 -> 4095	: 1815	*****
4096 -> 8191	: 591	*****
8192 -> 16383	: 397	*****
16384 -> 32767	: 50	

В этом выводе мы видим бимодальное распределение, в котором одна мода охватывает интервал от 128 до 1023 мкс, а вторая — от 2048 до 4095 мкс (от 2,0 до 4,1 мс). Теперь, зная, что распределение задержек в устройстве имеет бимодальный характер, я могу заняться настройкой, перемещающей больше операций ввода/вывода в более быстрый интервал. Например, медленно может выполняться произвольный ввод/вывод или ввод/вывод большого объема (источник которого можно идентифицировать с помощью других инструментов BPF) или операции, выполняемые с другими флагами (можно узнать с помощью параметра -F). Самые медленные операции ввода/вывода в этом примере выполнялись от 16 до 32 мс: похоже, что на устройстве образовалась очень длинная очередь.

Версия biolatenсy(8) для ВСС поддерживает следующие параметры:

- **-m**: выводить время в миллисекундах;
- **-Q**: включать время, проведенное в очередях ОС (*время обработки запроса операционной системой*);

¹ Немного истории: версию biolatenсy для ВСС я создал 20 сентября 2015 года, а версию для bpftrace — 13 сентября 2018 года, взяв за основу свой ранний инструмент iolatenсy. Буква «b» была добавлена, чтобы подчеркнуть, что инструмент трассирует блочный (block) ввод/вывод.

- **-F:** показать гистограмму отдельно для каждого установленного флага ввода/вывода;
- **-D:** показать гистограмму отдельно для каждого дискового устройства.

Параметр **-Q** заставляет `biolatency(8)` сообщать полное время ввода/вывода, от создания и добавления запроса в очередь ядра до завершения его обработки устройством, которое выше было описано как *время обработки запроса блочного ввода/вывода*.

`biolatency(8)` для ВСС также принимает необязательные аргументы со значениями интервала в секундах и счетчика.

Флаги

Параметр **-F** особенно полезен — он позволяет разбить распределение задержек по флагам ввода/вывода. Вот пример гистограмм, полученных с дополнительным параметром **-m** для вывода времени в миллисекундах:

```
# biolatency -Fm 10 1
```

```
Tracing block device I/O... Hit Ctrl-C to end.
```

```
flags = Sync-Write
msecs          : count  distribution
  0 -> 1        : 2      |*****|

flags = Flush
msecs          : count  distribution
  0 -> 1        : 1      |*****|

flags = Write
msecs          : count  distribution
  0 -> 1        : 14     |*****|
  2 -> 3        : 1      |**      |
  4 -> 7        : 10     |*****|
  8 -> 15       : 11     |*****|
 16 -> 31      : 11     |*****|

flags = NoMerge-Write
msecs          : count  distribution
  0 -> 1        : 95     |*****|
  2 -> 3        : 152    |*****|
  4 -> 7        : 266    |*****|
  8 -> 15       : 350    |*****|
 16 -> 31      : 298    |*****|

flags = Read
msecs          : count  distribution
  0 -> 1        : 11     |*****|

flags = ReadAhead-Read
msecs          : count  distribution
  0 -> 1        : 5261   |*****|
  2 -> 3        : 1238   |*****|
  4 -> 7        : 481    |***     |
  8 -> 15       : 5      |         |
 16 -> 31      : 2      |         |
```

Флаги могут обрабатываться устройствами хранения по-разному. Такое разделение помогает исследовать их по отдельности. Гистограммы выше показывают, что запись выполнялась медленнее, чем чтение, и могут объяснить выявленное ранее бимодальное распределение.

`biolatency(8)` обобщает информацию о задержках дискового ввода/вывода. Для исследования задержек отдельных операций ввода/вывода используйте `biosnoop(8)`.

9.6.7. biosnoop

`biosnoop(8)`¹ — это инструмент для BCC и `bpfttrace`, который выводит короткие однострочные сводки для каждой операции дискового ввода/вывода. Например:

```
# biosnoop
TIME(s)      COMM          PID    DISK    T  SECTOR    BYTES    LAT(ms)
0.009165000  jbd2/nvme0n1p1 174    nvme0n1 W  2116272    8192     0.43
0.009612000  jbd2/nvme0n1p1 174    nvme0n1 W  2116288    4096     0.39
0.011836000  mysqld          1948    nvme0n1 W  10434672   4096     0.45
0.012363000  jbd2/nvme0n1p1 174    nvme0n1 W  2116296    8192     0.49
0.012844000  jbd2/nvme0n1p1 174    nvme0n1 W  2116312    4096     0.43
0.016809000  mysqld          1948    nvme0n1 W  10227712   262144   1.82
0.017184000  mysqld          1948    nvme0n1 W  10228224   262144   2.19
0.017679000  mysqld          1948    nvme0n1 W  10228736   262144   2.68
0.018056000  mysqld          1948    nvme0n1 W  10229248   262144   3.05
0.018264000  mysqld          1948    nvme0n1 W  10229760   262144   3.25
0.018657000  mysqld          1948    nvme0n1 W  10230272   262144   3.64
0.018954000  mysqld          1948    nvme0n1 W  10230784   262144   3.93
0.019053000  mysqld          1948    nvme0n1 W  10231296   131072   4.03
0.019731000  jbd2/nvme0n1p1 174    nvme0n1 W  2116320    8192     0.49
0.020243000  jbd2/nvme0n1p1 174    nvme0n1 W  2116336    4096     0.46
0.020593000  mysqld          1948    nvme0n1 R  4495352    4096     0.26
[...]
```

Как показывает этот вывод, основную рабочую нагрузку записи на диск `nvme0n1` создает процесс `mysqld` с PID 174, выполняющий ввод/вывод разного объема. Вот значения столбцов:

- **TIME(s)**: время выполнения операции в секундах;
- **COMM**: имя процесса (если известно этому инструменту);
- **PID**: идентификатор процесса (если известен этому инструменту);
- **DISK**: имя устройства хранения;
- **T**: тип: R == чтение (reads), W == запись (writes);
- **SECTOR**: адрес на диске в 512-байтных секторах;
- **BYTES**: объем ввода/вывода;

¹ Немного истории: версию для BCC я написал 16 сентября 2015 года, а версию для `bpfttrace` — 15 ноября 2017 года, взяв за основу свой же инструмент, созданный в 2003 году. Полная история происхождения `biosnoop(8)` приводится в моей книге [Gregg 19].

- **LAT(ms)**: продолжительность ввода/вывода от передачи запроса устройству до завершения его обработки устройством (время отклика диска).

Примерно в середине этого примера можно видеть серию из операций записи по 262 144 байта, при выполнении которых задержка постепенно увеличивается с 1,82 до 4,03 мс. Я часто наблюдаю такую закономерность, а ее вероятную причину можно определить по другому столбцу: **TIME(s)**. Если вычесть значение в столбце **LAT(ms)** из значения в столбце **TIME(s)**, то получится время начала ввода/вывода. В данном случае все эти операции были инициированы примерно в одно и то же время. Похоже, что эта группа операций записи была запущена почти одновременно, затем они были поставлены в очередь на устройстве, а после этого выполнены по очереди, что объясняет увеличение задержки каждой последующей операции.

Путем тщательного изучения времени начала и конца можно также определить переупорядочение операций на устройстве. Поскольку вывод инструмента может состоять из многих тысяч строк, я часто прибегал к помощи программного обеспечения на R для статистических расчетов, чтобы преобразовать полученный вывод в диаграмму рассеяния и упростить идентификацию таких закономерностей (см. раздел 9.7 «Визуализация»).

Анализ выбросов

Ниже описан метод поиска и анализа выбросов по задержке с помощью `biosnoop(8)`.

1. Сохраните вывод в файл:

```
# biosnoop > out.biosnoop01.txt
```

2. Отсортируйте вывод по столбцу со значением задержки и выведите последние пять записей (с наибольшей задержкой):

```
# sort -n -k 8,8 out.biosnoop01.txt | tail -5
31.344175  logger          10994  nvme0n1 W 15218056  262144  30.92
31.344401  logger          10994  nvme0n1 W 15217544  262144  31.15
31.344757  logger          10994  nvme0n1 W 15219080  262144  31.49
31.345260  logger          10994  nvme0n1 W 15218568  262144  32.00
46.059274  logger          10994  nvme0n1 W 15198896  4096    64.86
```

3. Откройте получившийся файл в текстовом редакторе (например, `vi(1)` или `vim(1)`):

```
# vi out.biosnoop01.txt
```

4. Просмотрите выбросы от самого медленного к самому быстрому, сравнивая время в первом столбце. Самый медленный здесь составил 64,86 мс, время обработки — 46,059274 (в секундах от начала трассировки). Ищем 46.059274:

```
[...]
45.992419  jbd2/nvme0n1p1 174    nvme0n1 W 2107232   8192    0.45
45.992988  jbd2/nvme0n1p1 174    nvme0n1 W 2107248   4096    0.50
46.059274  logger          10994  nvme0n1 W 15198896   4096    64.86
[...]
```

Посмотрите, какие события происходили до выброса, и сравните их задержки. Если величины задержек сопоставимы, значит, задержка — это результат ожидания в очереди (аналогично линейному изменению задержек от 1,82 до 4,03 мс, наблюдаемому в первом примере вывода `biosnoop(8)`). Вероятно, вам удастся обнаружить какие-то другие подсказки. Здесь ситуация иная: предыдущее событие произошло примерно на 6 мс раньше и имело задержку 0,5 мс. Возможно, устройство переупорядочило запросы и первыми обработало другие. Если бы предыдущее событие завершения обработки отстояло от текущего на 64 мс в прошлом, то разрыв можно было бы объяснить другими факторами: например, эта система — экземпляр виртуальной машины и была перепланирована гипервизором во время ввода/вывода, что добавило дополнительное время к времени ввода/вывода.

Время в очереди

Параметр `-Q` в версии `biosnoop(8)` для ВСС можно использовать для отображения времени между созданием запроса на выполнение ввода/вывода и его передачи в устройство (выше это время было определено как *время ожидания блочного ввода/вывода*, или *время ожидания ОС*). Большую часть этого времени запрос проводит в очередях ОС, но оно также может включать время, необходимое для выделения памяти и получения блокировок. Например:

```
# biosnoop -Q
TIME(s)   COMM          PID   DISK   T SECTOR   BYTES   QUE(ms)   LAT(ms)
0.000000  kworker/u4:0    9491  nvme0n1 W 5726504   4096     0.06     0.60
0.000039  kworker/u4:0    9491  nvme0n1 W 8128536   4096     0.05     0.64
0.000084  kworker/u4:0    9491  nvme0n1 W 8128584   4096     0.05     0.68
0.000138  kworker/u4:0    9491  nvme0n1 W 8128632   4096     0.05     0.74
0.000231  kworker/u4:0    9491  nvme0n1 W 8128664   4096     0.05     0.83
[...]
```

Время в очереди выводится в столбце `QUE(ms)`.

9.6.8. iotop, biotop

Первую версию `iotop` я написал в 2005 году для систем на основе Solaris [McDougall 06a]. Сейчас есть множество версий, включая инструмент `iotop(1)` для Linux, основанный на статистиках ядра¹ [Chazarain 13], и мой собственный инструмент `biotop(8)`, основанный на BPF.

iotop

Обычно `iotop` устанавливается в составе одноименного пакета `iotop`. При запуске без аргументов команда обновляет экран каждую секунду и выводит список

¹ `iotop(1)` требует, чтобы ядро было собрано с параметрами `CONFIG_TASK_DELAY_ACCT`, `CONFIG_TASK_IO_ACCOUNTING`, `CONFIG_TASKSTATS` и `CONFIG_VM_EVENT_COUNTERS`.

процессов, являющихся основными потребителями дискового ввода/вывода. Для непрерывного вывода (без очистки экрана) можно использовать пакетный режим (-b). Он показан в примере ниже, где используется для вывода только процессов, осуществляющих ввод/вывод (-o), с интервалом 5 с (-d5):

```
# iotop -bod5
Total DISK READ:      4.78 K/s | Total DISK WRITE:      15.04 M/s
   TID  PRIO  USER      DISK READ  DISK WRITE  SWAPIN      IO    COMMAND
22400 be/4  root       4.78 K/s    0.00 B/s    0.00 %   13.76 % [flush-252:0]
   279 be/3  root       0.00 B/s  1657.27 K/s    0.00 %    9.25 % [jbd2/vda2-8]
22446 be/4  root       0.00 B/s   10.16 M/s    0.00 %    0.00 % beam.smp -K true ...
Total DISK READ:      0.00 B/s | Total DISK WRITE:      10.75 M/s
   TID  PRIO  USER      DISK READ  DISK WRITE  SWAPIN      IO    COMMAND
   279 be/3  root       0.00 B/s    9.55 M/s    0.00 %    0.01 % [jbd2/vda2-8]
22446 be/4  root       0.00 B/s   10.37 M/s    0.00 %    0.00 % beam.smp -K true ...
   646 be/4  root       0.00 B/s  272.71 B/s    0.00 %    0.00 % rsyslogd -n -c 5
[...]
```

Как показывает этот вывод, процесс beam.smp (Riak) выполняет запись на диск со скоростью около 10 Мбайт/с. Вот значения столбцов:

- **DISK READ:** скорость чтения в килобайтах в секунду;
- **DISK WRITE:** скорость записи в килобайтах в секунду;
- **SWAPIN:** процент времени, в течение которого поток ожидал завершения ввода/вывода подкачки;
- **IO:** процент времени, в течение которого поток ожидал завершения ввода/вывода.

iotop(8) поддерживает еще несколько параметров, включая -a для вывода накопленной статистики (вместо интервальной), -p PID для отображения данных, касающихся только определенного процесса, и -d SEC для установки интервала обновления экрана.

Рекомендую проверить работу утилиты iotop(8) с известной рабочей нагрузкой и убедиться, что она отображает верные числа. Я только что попробовал сделать это у себя (iotop версии 0.6) и обнаружил, что она сильно недооценивает рабочие нагрузки записи. Также можно использовать инструмент biotop(8), который использует другой источник информации и у меня, например, выводит значения, соответствующие моей тестовой рабочей нагрузке.

biotop

biotop(8) — это инструмент для ВСС и еще одна версия утилиты top(1) для дисков. Вот пример вывода:

```
# biotop
Tracing... Output every 1 secs. Hit Ctrl-C to end

08:04:11 loadavg: 1.48 0.87 0.45 1/287 14547

PID      COMM                D MAJ MIN  DISK      I/O  Kbytes  AVGms
```

```

14501 cksum          R 202 1   xvda1      361    28832    3.39
6961  dd             R 202 1   xvda1     1628   13024    0.59
13855 dd             R 202 1   xvda1     1627   13016    0.59
326   jbd2/xvda1-8   W 202 1   xvda1       3     168     3.00
1880  supervise      W 202 1   xvda1       2       8     6.71
1873  supervise      W 202 1   xvda1       2       8     2.51
1871  supervise      W 202 1   xvda1       2       8     1.57
1876  supervise      W 202 1   xvda1       2       8     1.22
[...]
```

В этом выводе можно видеть команды `cksum(1)` и `dd(1)`, производящие чтение, и процессы `supervise`, производящие запись. Это быстрый способ определить, какие процессы и в каком объеме выполняют дисковый ввод/вывод. Вот значения столбцов:

- **PID**: кэшированный идентификатор процесса (может не соответствовать истинному процессу);
- **COMM**: кэшированное имя процесса (может не соответствовать истинному процессу);
- **D**: направление (R == чтение (read), W == запись (write));
- **MAJ MIN**: старший и младший номера дискового устройства (идентификатор в ядре);
- **DISK**: имя диска;
- **I/O**: количество операций дискового ввода/вывода за интервал;
- **Kbytes**: общая пропускная способность диска за интервал (килобайты);
- **AVGms**: среднее время ввода/вывода (задержка) от передачи запроса устройству до завершения его выполнения (миллисекунды).

К моменту передачи запроса дисковому устройству процесс, пославший этот запрос, может находиться вне процессора и его идентификация может быть затруднена. `biotor(8)` делает все возможное, чтобы отобразить верные значения в столбцах **PID** и **COMM**, но не гарантирует абсолютной точности.

`biotor(8)` поддерживает необязательные аргументы, определяющие интервал и счетчик (по умолчанию интервал равен одной секунде). Параметр `-c` предотвращает очистку экрана перед выводом очередной порции данных, а с помощью параметра `-r MAXROWS` можно задать максимальное количество отображаемых процессов.

9.6.9. biostacks

`biostacks(8)`¹ — это инструмент для `bpftrace`, который трассирует запросы блочного ввода/вывода (от постановки в очередь в ОС до завершения обработки устройством) и выводит соответствующие трассировки стека. Например:

¹ Немного истории: я создал этот инструмент 19 марта 2019 года [Gregg 19].


```
# biostacks.bt
Attaching 5 probes...
Tracing block I/O with init stacks. Hit Ctrl-C to end.
^C
[...]
@usecs[
    blk_account_io_start+1
    blk_mq_make_request+1069
    generic_make_request+292
    submit_bio+115
    submit_bh_wbc+384
    ll_rw_block+173
    ext4_bread+102
    __ext4_read_dirblock+52
    ext4_dx_find_entry+145
    ext4_find_entry+365
    ext4_lookup+129
    lookup_slow+171
    walk_component+451
    path_lookupat+132
    filename_lookup+182
    user_path_at_empty+54
    sys_access+175
    do_syscall_64+115
    entry_SYSCALL_64_after_hwframe+61
]:
[2K, 4K)           2 |@@|
[4K, 8K)           37 |@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|
[8K, 16K)          15 |@@@@@@@@@@@@@@@@@@@@@@@@|
[16K, 32K)          9 |@@@@@@@@@@@@|
[32K, 64K)          1 |@|
```

В выводе отображается гистограмма распределения задержек (в микросекундах) дискового ввода/вывода, а также стек, инициировавший ввод/вывод через системный вызов `access(2)`, `filename_lookup()` и `ext4_lookup()`. Этот ввод/вывод обусловлен поиском путей при проверке прав доступа к файлам. В выводе было много таких стеков, которые показывают, что дисковый ввод/вывод вызван действиями, отличными от чтения и записи.

Я видел случаи, когда происходил загадочный дисковый ввод/вывод, не инициированный никаким выполняющимся приложением. Причина была в выполнении фоновых задач файловой системы. (В одном случае это был фоновый скруббер ZFS, периодически проверяющий контрольные суммы.) `biostacks(8)` способен определить истинную причину дискового ввода/вывода, отображая трассировку стека ядра.

9.6.10. blktrace

`blktrace(8)` — это инструмент для трассировки событий блочного ввода/вывода в Linux, который использует трассировщик `blktrace` в ядре. Это специализированный трассировщик, управляемый через системные вызовы `BLKTRACE_IOCTL(2)`

к файлам дисковых устройств. К инструментам, составляющим внешний интерфейс, относятся `blktrace(8)`, `blkparse(1)` и `btrace(8)`.

`blktrace(8)` запускает трассировку драйвера блочного устройства в ядре и возвращает данные трассировки, которые можно обработать с помощью `blkparse(1)`. Для удобства был также создан инструмент `btrace(8)`, который запускает `blktrace(8)` и `blkparse(1)`, то есть следующие команды эквивалентны:

```
# blktrace -d /dev/sda -o - | blkparse -i -
# btrace /dev/sda
```

`blktrace(8)` — низкоуровневый инструмент, отображающий множество событий для каждой операции ввода/вывода.

Вывод по умолчанию

Ниже дан пример вывода `btrace(8)` по умолчанию для одной операции чтения с диска командой `cksum(1)`:

```
# btrace /dev/sdb
8,16      3      1      0.429604145 20442 A R 184773879 + 8 <- (8,17) 184773816
8,16      3      2      0.429604569 20442 Q R 184773879 + 8 [cksum]
8,16      3      3      0.429606014 20442 G R 184773879 + 8 [cksum]
8,16      3      4      0.429607624 20442 P N [cksum]
8,16      3      5      0.429608804 20442 I R 184773879 + 8 [cksum]
8,16      3      6      0.429610501 20442 U N [cksum] 1
8,16      3      7      0.429611912 20442 D R 184773879 + 8 [cksum]
8,16      1      1      0.440227144      0 C R 184773879 + 8 [0]
[...]
```

Этой единственной операции дискового ввода/вывода соответствует восемь строк в выводе, по одной на каждое действие (событие), связанное с очередью блочного устройства или с самим устройством.

По умолчанию выводится семь столбцов:

1. Старший и младший номер устройства.
2. Идентификатор процессора.
3. Порядковый номер.
4. Время события в секундах от начала трассировки.
5. Идентификатор процесса.
6. Идентификатор действия: тип события (см. подраздел «Идентификаторы действий» ниже).
7. Описание RWBS: флаги ввода/вывода (см. подраздел «Описание RWBS» ниже).

Столбцы в выводе можно настроить с помощью параметра `-f`, указав в нем строку формата для вывода информации о действиях.

Отображаемый результат зависит от действия. Например, `184773879 + 8 [cksum]` означает ввод/вывод в блок с адресом 184773879 и размером 8 (секторов), инициированный процессом с именем `cksum`.

Идентификаторы действий

Вот описания идентификаторов из страницы справочного руководства для `blkparse(1)`:

A	Ввод/вывод был отображен на другое устройство
B	Дополнительный ввод/вывод
C	Ввод/вывод завершен
D	Запрос передан драйверу
F	Запрос добавлен в начало другого запроса, находившегося в очереди
G	Выделение памяти для запроса
I	Запрос добавлен в очередь планировщика ввода/вывода
M	Запрос добавлен в конец другого запроса, находившегося в очереди
P	Блокировка очереди для накопления нескольких запросов
Q	Ввод/вывод передан коду, который управляет очередью
S	Ожидание освобождения структуры для размещения запроса
T	Разблокировка очереди по тайм-ауту
U	Разблокировка очереди
X	Запрос разделен ¹

Я привел здесь этот список потому, что он также показывает, какие события может наблюдать фреймворк `blktrace`.

Описание RWBS

Для нужд мониторинга ядро позволяет описать тип каждой операции ввода/вывода с помощью строки символов — *rwbs*. Она применяется в `blktrace(8)` и в других инструментах трассировки, определена в функции ядра `blk_fill_rwbs()` и использует следующие символы:

- **R**: чтение (`read`);
- **W**: запись (`write`);
- **M**: метаданные (`metadata`);
- **S**: синхронная (`synchronous`);
- **A**: опережающее чтение (`read-ahead`);
- **F**: выталкивание на диск или принудительное обращение к блоку (`flush or force`);
- **D**: отмена (`discard`);
- **E**: стирание (`erase`);
- **N**: нет операции (`none`).

¹ Это перевод текста из справочного руководства. — *Примеч. пер.*

Символы могут комбинироваться. Например, «WM» означает «запись метаданных».

Фильтрация действий

Команды `blktrace(8)` и `btrace(8)` способны фильтровать действия и отображать только события конкретного типа. Например, для трассировки только действий D (передача запроса драйверу) используйте параметр `-a issue`:

```
# blktrace -a issue /dev/sdb
8,16      1      1      0.000000000  448 D   W 38978223 + 8 [kjournald]
8,16      1      2      0.000306181  448 D   W 104685503 + 24 [kjournald]
8,16      1      3      0.000496706  448 D   W 104685527 + 8 [kjournald]
8,16      1      1      0.010441458 20824 D   R 184944151 + 8 [tar]
[...]
```

Описание других фильтров можно найти на странице справочного руководства `man` для `blktrace(8)`, в том числе фильтры для трассировки только чтения (`-a read`), только записи (`-a write`) или только синхронных операций (`-a sync`).

Анализ

В пакете `blktrace` есть утилита `btt(1)`, предназначенная для анализа трассировки ввода/вывода. Вот пример, в котором `blktrace(8)` применяется для наблюдения за `/dev/nvme0n1p1` во время записи файлов трассировки (здесь используется новый каталог, потому что эти команды создают несколько файлов):

```
# mkdir tracefiles; cd tracefiles
# blktrace -d /dev/nvme0n1p1 -o out -w 10
=== nvme0n1p1 ===
CPU 0:          20135 events,          944 KiB data
CPU 1:          38272 events,         1795 KiB data
Total:          58407 events (dropped 0),       2738 KiB data
# blkparse -i out.blktrace.* -d out.bin
259,0      1      1      0.000000000  7113 A  RM 161888 + 8 <- (259,1) 159840
259,0      1      1      0.000000000  7113 A  RM 161888 + 8 <- (259,1) 159840
[...]
```

btt -i out.bin

```
===== All Devices =====
      ALL      MIN      AVG      MAX      N
-----
Q2Q      0.000000001  0.000365336  2.186239507  24625
Q2A      0.000037519  0.000476609  0.001628905   1442
Q2G      0.000000247  0.000007117  0.006140020  15914
G2I      0.000001949  0.000027449  0.000081146    602
Q2M      0.000000139  0.000000198  0.000021066    8720
I2D      0.000002292  0.000008148  0.000030147    602
M2D      0.000001333  0.000188409  0.008407029    8720
D2C      0.000195685  0.000885833  0.006083538   12308
Q2C      0.000198056  0.000964784  0.009578213   12308
[...]
```

Эти статистики выражаются в секундах и показывают продолжительность каждого этапа обработки ввода/вывода. Наибольший интерес представляют:

- **Q2C**: общее время от отправки запроса ввода/вывода до его выполнения (время обработки стеком блочного ввода/вывода);
- **D2C**: время от передачи запроса устройству до его выполнения (задержка дискового ввода/вывода);
- **I2D**: время от добавления запроса в очередь планировщика ввода/вывода до передачи устройству (время ожидания в очереди);
- **M2D**: время от слияния с другим запросом до передачи устройству.

Как показывает вывод в примере, среднее время D2C составляет 0,86 мс, а максимальное время M2D — 8,4 мс. Такие максимальные значения могут быть причиной выбросов по задержке ввода/вывода.

Дополнительную информацию ищите в руководстве пользователя btt [Brunelle 08].

Визуализация

Инструмент blktrace(8) позволяет записывать события в файлы трассировки для последующей визуализации с помощью утилиты iowatcher(1), также входящей в пакет blktrace, и с помощью утилиты seekwatcher Криса Мейсона [Mason 08].

9.6.11. bpftrace

bpftrace — это трассировщик, основанный на BPF, который поддерживает высокоуровневый язык программирования, позволяющий создавать мощные и короткие сценарии. Хорошо подходит для анализа дискового ввода/вывода на основе подсказок, полученных с помощью других инструментов.

Подробнее о bpftrace я расскажу в главе 15, а в этом разделе приведу несколько примеров анализа размеров и задержек дискового ввода/вывода.

Однострочные сценарии

Однострочные сценарии ниже показывают различные возможности bpftrace.

Подсчет событий блочного ввода/вывода по точкам трассировки:

```
bpftrace -e 'tracepoint:block:* { @[probe] = count(); }'
```

Выводит размеры блочного ввода/вывода в виде гистограммы:

```
bpftrace -e 't:block:block_rq_issue { @bytes = hist(args->bytes); }'
```

Подсчет операций ввода/вывода с группировкой по трассировкам стека в пространстве пользователя:

```
bpftrace -e 't:block:block_rq_issue { @[ustack] = count(); }'
bpftrace -e 't:block:block_rq_insert { @[ustack] = count(); }'
```



```
# biolateness.bt
Attaching 4 probes...
Tracing block device I/O... Hit Ctrl-C to end.
^C

@usecs:
[32, 64)          2 |@
[64, 128)         1 |
[128, 256)        1 |
[256, 512)       27 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[512, 1K)        43 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[1K, 2K)         54 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[2K, 4K)         41 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[4K, 8K)         47 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[8K, 16K)        16 | @@@@@@@@@@@@@@@@@@
[16K, 32K)        4 | @@@
```

Как показывают эти результаты, в большинстве случаев на выполнение ввода/вывода уходило от 256 мкс до 16 мс (16К мкс).

Вот исходный код инструмента:

```
#!/usr/local/bin/bpftrace
BEGIN
{
    printf("Tracing block device I/O... Hit Ctrl-C to end.\n");
}
tracepoint:block:block_rq_issue

{
    @start[args->dev, args->sector] = nsecs;
}

tracepoint:block:block_rq_complete
/@start[args->dev, args->sector]/
{
    @usecs = hist((nsecs - @start[args->dev, args->sector]) / 1000);
    delete(@start[args->dev, args->sector]);
}

END
{
    clear(@start);
}
```

Измерение задержки требует сохранить время начала каждого ввода/вывода, а затем извлечь ее, когда ввод/вывод будет завершен, чтобы вычислить прошедшее время. При измерении задержки в VFS (в главе 8 «Файловые системы», в разделе 8.6.15 «bpftrace») время начала сохранялось в карте BPF с идентификатором потока в роли ключа. Это было оправданно, потому что поток с тем идентификатором будет выполняться на процессоре в моменты начала и окончания обработки запроса. С дисковым вводом/выводом ситуация иная, потому что к моменту появления события завершения на процессоре может выполняться другой поток. По этой причине в роли уникального идентификатора в biolateness.bt была выбрана комбинация из

номеров устройства и сектора: предполагается, что только одна операция ввода/вывода будет ссылаться на этот сектор в данный момент времени.

По аналогии с однострочным сценарием для анализа размера ввода/вывода можно добавить в карту ключ `args->rwbs`, чтобы дополнительно сгруппировать время задержки по типам ввода/вывода.

Ошибки дискового ввода/вывода

Код ошибки ввода/вывода доступен как аргумент точки трассировки `block:block_rq_complete`, и следующий инструмент `bioerr(8)`¹ использует его для вывода информации об операциях ввода/вывода, в которых возникла ошибка (однострочная версия этого инструмента была представлена выше):

```
#!/usr/local/bin/bpfttrace

BEGIN
{
    printf("Tracing block I/O errors. Hit Ctrl-C to end.\n");
}

tracepoint:block:block_rq_complete
/args->error != 0/
{
    time("%N:%M:%S ");
    printf("device: %d,%d, sector: %d, bytes: %d, flags: %s, error: %d\n",
        args->dev >> 20, args->dev & ((1 << 20) - 1), args->sector,
        args->nr_sector * 512, args->rwbs, args->error);
}
```

Найти дополнительную информацию об ошибке дискового ввода/вывода помогут более низкоуровневые инструменты, например `MegaCli`, `smartctl` и журналирование SCSI, которые описываются ниже.

9.6.12. MegaCli

Контроллеры дисков (адаптеры главной шины) состоят из аппаратного и программного обеспечения, внешних по отношению к системе. Инструменты анализа ОС и даже динамические трассировщики не могут напрямую наблюдать за их работой изнутри. Но иногда об особенностях этой работы можно судить, внимательно анализируя результаты реакции контроллера диска на серию операций ввода/вывода (в том числе полученные методом статической или динамической инструментации ядра).

Но сейчас появились инструменты для анализа конкретных контроллеров дисков, например `MegaCli` от LSI. Ниже показаны события контроллера:

```
# MegaCli -AdpEventLog -GetLatest 50 -f lsi.log -aALL
# more lsi.log
```

¹ Немного истории: я создал его 19 марта 2019 года для своей книги [Gregg 19].

```

seqNum: 0x0000282f
Time: Sat Jun 16 05:55:05 2012
Code: 0x00000023
Class: 0
Locale: 0x20
Event Description: Patrol Read complete
Event Data:
=====
None

seqNum: 0x000027ec
Time: Sat Jun 16 03:00:00 2012
Code: 0x00000027
Class: 0
Locale: 0x20
Event Description: Patrol Read started
[...]
```

Последние два события показывают, что между 3:00 и 5:55 утра произошло Patrol Read (влияющее на производительность). Patrol Read упоминалось в разделе 9.4.3 «Типы хранилищ». Контроллер производит Patrol Read блоков на диске с целью проверки их контрольных сумм.

MegaCli имеет множество других параметров для отображения информации об адаптере, о дисковом устройстве, о виртуальном устройстве, о среде, о состоянии батареи и о физических ошибках. Это помогает выявить проблемы в конфигурации и ошибки. Но даже при наличии этой информации бывает трудно выявить некоторые проблемы, например, почему именно тот или иной ввод/вывод продолжался сотни миллисекунд.

Чтобы узнать, какой интерфейс, если он есть, используется для анализа контроллера диска, обращайтесь к документации производителя.

9.6.13. smartctl

Дисковое устройство выполняет определенную программную логику, управляя своей работой, включая организацию очереди, кэширование и обработку ошибок. Как и в случае с контроллерами дисков, внутреннее поведение дискового устройства недоступно для наблюдения со стороны ОС, но о нем обычно можно судить по результатам наблюдения за запросами ввода/вывода и их задержкой.

Многие современные диски предоставляют данные SMART (Self-Monitoring, Analysis and Reporting Technology — технология самоконтроля, анализа и отчетности) с разнообразными статистиками, характеризующими состояние устройства. Следующий вывод команды smartctl(8) в Linux показывает, какие данные доступны (получен обращением к первому диску в виртуальном RAID с использованием параметров -d megaraid, 0):

```

# smartctl --all -d megaraid,0 /dev/sdb
smartctl 5.40 2010-03-16 r3077 [x86_64-unknown-linux-gnu] (local build)
Copyright (C) 2002-10 by Bruce Allen, http://smartmontools.sourceforge.net
```

```
Device: SEAGATE ST3600002SS          Version: ER62
Serial number: 3SS0LM01
Device type: disk
Transport protocol: SAS
Local Time is: Sun Jun 17 10:11:31 2012 UTC
Device supports SMART and is Enabled
Temperature Warning Disabled or Not Supported
SMART Health Status: OK

Current Drive Temperature:      23 C
Drive Trip Temperature:        68 C
Elements in grown defect list: 0
Vendor (Seagate) cache information
  Blocks sent to initiator = 317280756
  Blocks received from initiator = 2618189622
  Blocks read from cache and sent to initiator = 854615302
  Number of read and write commands whose size <= segment size = 30848143
  Number of read and write commands whose size > segment size = 0
Vendor (Seagate/Hitachi) factory information
  number of hours powered up = 12377.45
  number of minutes until next internal SMART test = 56
```

```
Error counter log:
      Errors Corrected by           Total Correction      Gigabytes    Total
      ECC      rereads/           errors    algorithm    processed    uncorrected
      fast | delayed rewrites    corrected invocations [10^9 bytes] errors
read:   7416197      0      0   7416197   7416197   1886.494      0
write:    0      0      0      0      0   1349.999      0
verify: 142475069      0      0  142475069  142475069  22222.134      0
```

```
Non-medium error count:      2661
```

```
SMART Self-test log
Num  Test              Status      segment  LifeTime  LBA_first_err [SK ASC ASQ]
     Description                               number    (hours)
# 1  Background long    Completed   16        3          - [- - -]
# 2  Background short   Completed   16        0          - [- - -]
```

```
Long (extended) Self Test duration: 6400 seconds [106.7 minutes]
```

Несмотря на всю полезность, эта информация не дает ответа на вопросы об отдельных медленных операциях дискового ввода/вывода, как это делают фреймворки трассировки ядра. Информация об исправленных ошибках пригодится для мониторинга: опираясь на нее, можно предсказать сбой диска до того, как он произойдет, а также подтвердить выход диска из строя или состояние, близкое к выходу из строя.

9.6.14. Журналирование SCSI

В Linux есть встроенное средство для журналирования событий SCSI. Его можно включить через `sysctl(8)` или `/proc`. Например, обе команды ниже устанавливают максимальный уровень журналирования для всех типов событий (будьте осторожны:

при высокой нагрузке на диск это может привести к переполнению системного журнала):

```
# sysctl -w dev.scsi.logging_level=03333333333
# echo 0333333333 > /proc/sys/dev/scsi/logging_level
```

Числа в этих примерах представляют битовую маску, которая устанавливает уровень журналирования от 1 до 7 для десяти различных типов событий (здесь числа записаны в восьмеричном формате, в шестнадцатеричном формате они будут иметь вид 0x1b6db6db). Эта битовая маска определена в файле `drivers/scsi/scsi_logging.h`. Для ее настройки в пакете `sg3-utils` есть инструмент `scsi_logging_level(8)`. Например:

```
# scsi_logging_level -s --all 3
```

Примеры событий:

```
# dmesg
[...]
[542136.259412] sd 0:0:0:0: tag#0 Send: scmd 0x0000000001fb89dc
[542136.259422] sd 0:0:0:0: tag#0 CDB: Test Unit Ready 00 00 00 00 00 00
[542136.261103] sd 0:0:0:0: tag#0 Done: SUCCESS Result: hostbyte=DID_OK
driverbyte=DRIVER_OK
[542136.261110] sd 0:0:0:0: tag#0 CDB: Test Unit Ready 00 00 00 00 00 00
[542136.261115] sd 0:0:0:0: tag#0 Sense Key : Not Ready [current]
[542136.261121] sd 0:0:0:0: tag#0 Add. Sense: Medium not present
[542136.261127] sd 0:0:0:0: tag#0 0 sectors total, 0 bytes done.
[...]
```

Поддержку журналирования можно использовать для выявления ошибок и таймаутов. Несмотря на наличие отметок времени (первый столбец), их едва ли можно использовать для вычисления задержек ввода/вывода, когда нет уникальных идентификационных данных.

9.6.15. Другие инструменты

В табл. 9.6 перечислены инструменты наблюдения за работой дисков, представленные в других главах этой книги и в «BPF Performance Tools» [Gregg 19].

Таблица 9.6. Другие инструменты наблюдения за работой дисков

Раздел	Инструмент	Описание
7.5.1	vmstat	Статистики виртуальной памяти, включая подкачку
7.5.3	swapon	Использование устройства подкачки
[Gregg 19]	seeksize	Отображает расстояния для перемещения головок
[Gregg 19]	biopattern	Идентифицирует закономерность произвольного/последовательного обращения к диску
[Gregg 19]	bioerr	Трассирует дисковые ошибки

Раздел	Инструмент	Описание
[Gregg 19]	mdflush	Трассирует запросы на выталкивание в драйверах для групп устройств
[Gregg 19]	iosched	Обобщает информацию о задержках планировщика ввода/вывода
[Gregg 19]	scsilatency	Отображает распределение задержек при выполнении команд SCSI
[Gregg 19]	scsiresult	Отображает коды завершения команд SCSI
[Gregg 19]	nvmelateny	Обобщает задержки выполнения команд в драйвере NVMe (твердотельного накопителя)

Другие инструменты наблюдения за дисками и источников информации в Linux:

- **/proc/diskstats**: высокоуровневые статистики по дискам;
- **seekwatcher**: визуализирует закономерности доступа к диску [Mason 08].

Поставщики дисков могут предлагать дополнительные инструменты для доступа к статистикам микропрограммы или для установки отладочной версии прошивки.

9.7. ВИЗУАЛИЗАЦИЯ

Для анализа производительности дискового ввода/вывода есть множество разных приемов визуализации. Некоторые из них показаны в этом разделе вместе со скриншотами. Обсуждение визуализации вы найдете в главе 2 «Методологии», в разделе 2.10 «Визуализация».

9.7.1. Линейные диаграммы

Решения для мониторинга производительности обычно отображают изменение частоты операций ввода/вывода (IOPS), пропускной способности и потребления диска со временем в виде линейных графиков. Это помогает увидеть временные закономерности — изменение нагрузки в течение дня или повторяющиеся события, например интервалы очистки кэшей файловой системы.

Учитывайте особенности метрики, изображенной на графике. Средняя задержка может скрывать мультимодальные распределения и выбросы. Средние значения по всем дисковым устройствам могут скрыть несбалансированное поведение, включая чрезмерную нагрузку на одно устройство. Средние значения за длительные периоды времени могут скрывать краткосрочные колебания.

9.7.2. Диаграммы рассеяния задержек

Диаграмма рассеяния позволяет визуализировать задержки ввода/вывода для всех событий, а их могут быть тысячи. Вдоль оси X такой диаграммы откладывается время завершения операции, а по оси Y — время отклика (задержка). На рис. 9.11 дан

пример диаграммы рассеяния, построенной с использованием R на основе 1400 событий ввода/вывода, зафиксированных с помощью `iosnoop(8)` на промышленном сервере базы данных MySQL.

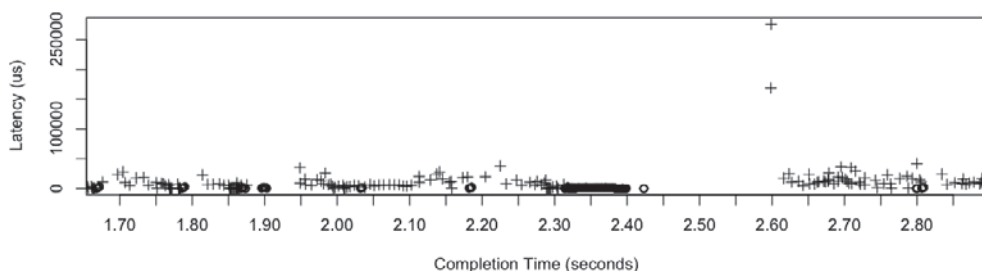


Рис. 9.11. Диаграмма рассеяния задержек дисковых операций чтения и записи

События чтения и записи на этой диаграмме рассеяния изображены разными значками: + — чтение, ° — запись. Также можно построить диаграмму, откладывая вдоль оси Y другие метрики, например адрес блока диска.

Здесь можно заметить пару выбросов задержек чтения с величинами больше 150 мс. Причина этих выбросов ранее не была известна. Эта диаграмма рассеяния и другие, включающие аналогичные выбросы, показали, что они возникают после большого количества операций записи. Операции записи имеют низкую задержку, потому что используют кэш отложенной записи в RAID-контроллере, который выполняет фактическую запись на устройство чуть позже. Я подозреваю, что эти операции чтения оказались позади операций записи в очереди на устройстве.

На этой диаграмме показана работа одного сервера в течение нескольких секунд. Если увеличить массив данных, добавив несколько серверов или используя более длинные интервалы, то события на графике могут начать сливаться и его будет трудно читать. В таких случаях предпочтительнее использовать тепловую карту задержек.

9.7.3. Тепловые карты задержек

Тепловую карту можно использовать для визуализации задержек, откладывая время вдоль оси X, задержку ввода/вывода вдоль оси Y и количество операций ввода/вывода в конкретное время и диапазон задержек вдоль оси Z, которая отображается цветом (чем темнее, тем больше). О тепловых картах шла речь в главе 2 «Методологии», в разделе 2.10.3 «Тепловые карты». На рис. 9.12 показан один интересный пример.

Рабочая нагрузка здесь была получена экспериментальным путем: я организовал последовательное чтение с нескольких дисков один за другим, чтобы исследовать ограничения шины и контроллера. Получившаяся тепловая карта оказалась неожиданной (была описана как «птеродактиль»), и она показывает информацию, которая была бы упущена при анализе только средних значений. Каждая из рассматриваемых

деталей имеет техническое объяснение: например, «клюв» охватывает восемь дисков, что равно количеству подключенных портов SAS (два порта x4), а «голова» начинается с девятого диска, когда возникает конкуренция за эти порты.

Я придумал создавать тепловые карты для визуализации изменения задержек во времени, вдохновившись инструментом *taztool*, описанным в следующем разделе. Рисунок 9.12 взят из раздела «Analytics» в описании устройства Sun Microsystems ZFS Storage [Gregg 10a]: я построил эту и другие интересные тепловые карты задержки, чтобы поделиться ими и популяризовать их применение.

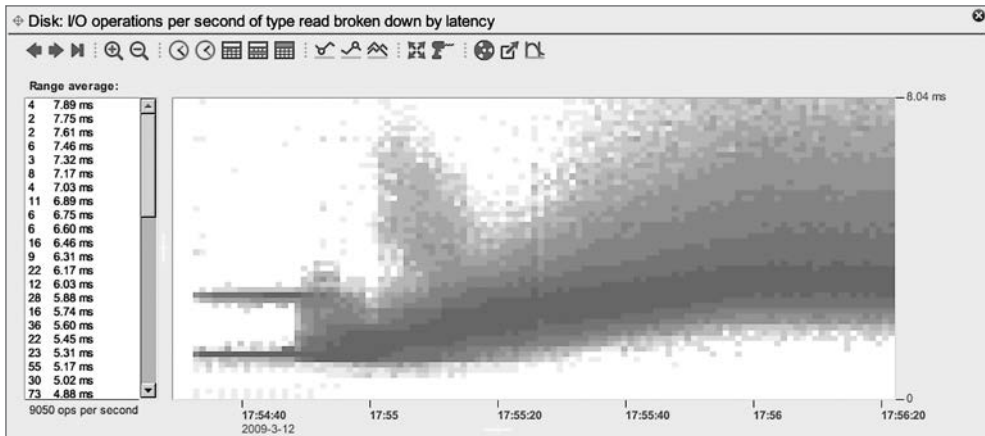


Рис. 9.12. Тепловая карта задержек дискового ввода/вывода в форме птеродактиля

Оси X и Y аналогичны одноименным осям на диаграмме рассеяния задержек. Основное преимущество тепловых карт в том, что они могут масштабироваться до миллионов событий, когда диаграмма рассеяния превращается в полностью закрашенные полосы, на которых невозможно ничего различить. Эта проблема обсуждалась в разделах 2.10.2 «Диаграммы рассеяния» и 2.10.3 «Тепловые карты».

9.7.4. Тепловые карты смещений

Местоположение ввода/вывода, или смещение, тоже можно визуализировать в виде тепловой карты (желательно перед созданием тепловых карт задержек). Пример такой тепловой карты показан на рис. 9.13.

Смещение на диске (адрес блока) отображается вдоль оси Y, а время — вдоль оси X. Каждый пиксель окрашивается в зависимости от количества операций ввода/вывода, попавших в этот диапазон времени и задержки. Чем больше значение, тем темнее цвет. В данном примере визуализирована рабочая нагрузка, созданная процедурой архивирования файловой системы, которая перемещается по всему диску, начиная с блока 0. Темные линии соответствуют последовательному вводу/выводу, а светлые — произвольному.

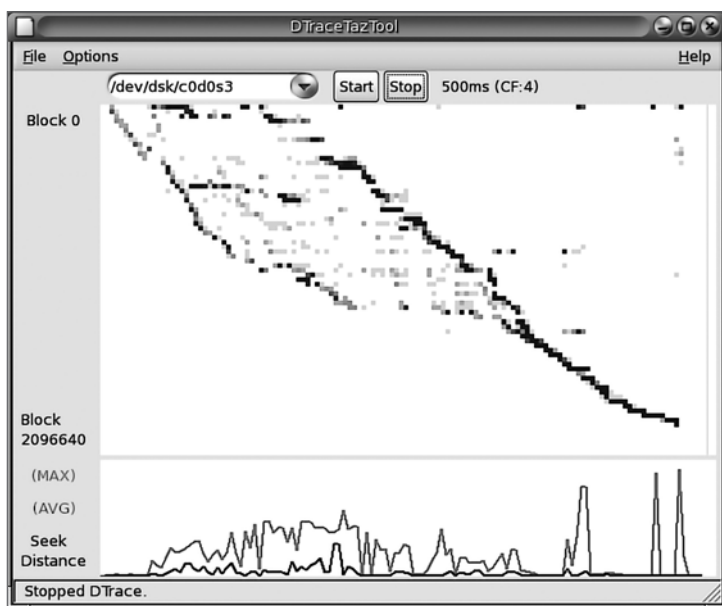


Рис. 9.13. DTraceTazTool

Эта диаграмма создана Ричардом Макдугаллом (Richard McDougall) в 1995 году с помощью инструмента taztool. Скриншот получен с помощью DTraceTazTool, версии инструмента taztool, которую я написал в 2006 году. Тепловые карты смещений дискового ввода/вывода можно получить с помощью других инструментов, включая seekwatcher (Linux).

9.7.5. Тепловые карты потребления

Потребление каждого устройства тоже можно показать в виде тепловой карты, чтобы выявить баланс потребления устройств и отдельные выбросы [Gregg 11b]. В этом случае вдоль оси Y откладывается процент потребления, а более темный цвет означает большее количество дисков с этим уровнем потребления. Тепловые карты этого вида пригодятся для выявления отдельных «горячих» дисков, в том числе «ленивых», в виде линий в верхней части тепловой карты (100 %). Пример тепловой карты потребления приводится в главе 6 «Процессоры», в разделе 6.7.1 «Тепловая карта потребления».

9.8. ЭКСПЕРИМЕНТЫ

В этом разделе описываются инструменты для активного тестирования производительности дискового ввода/вывода. Общее описание предлагаемой методологии приводится в разделе 9.5.9 «Микробенчмаркинг».

При использовании этих инструментов рекомендуется оставить постоянно работающим `iostat(1)`, чтобы любой результат можно было немедленно перепроверить. Некоторым инструментам микробенчмаркинга может потребоваться режим «прямого» доступа в обход кэша файловой системы, чтобы сосредоточить внимание на производительности дискового устройства.

9.8.1. Ad hoc

Для выполнения бенчмаркинга последовательного доступа к дискам можно использовать команду `dd(1)` (выполняет копирование с устройства на устройство). Вот пример тестирования последовательного чтения с размером ввода/вывода 1 Мбайт:

```
# dd if=/dev/sda1 of=/dev/null bs=1024k count=1k
1024+0 records in
1024+0 records out
1073741824 bytes (1.1 GB) copied, 7.44024 s, 144 MB/s
```

Поскольку ядро может кэшировать и буферизовать данные, измеренная с помощью `dd(1)` пропускная способность может характеризовать производительность кэша и диска вместе, а не только диска. Чтобы протестировать только производительность диска, используйте специальное символьное устройство для диска: в Linux его можно создавать в `/dev/raw` командой `raw(8)` (если доступна). Последовательную запись тестируют аналогично. Остерегайтесь уничтожения всех данных на диске, включая главную загрузочную запись и таблицы разделов!

Более безопасный подход — использовать флаг прямого ввода/вывода с командой `dd(1)` и файлы в файловой системе вместо дисковых устройств. Но имейте в виду, что в этом случае тест включает некоторый оверхед файловой системы. Вот пример тестирования записи в файл с именем `out1`:

```
# dd if=/dev/zero of=out1 bs=1024k count=1000 oflag=direct
1000+0 records in
1000+0 records out
1048576000 bytes (1.0 GB, 1000 MiB) copied, 1.79189 s, 585 MB/s
```

Утилита `iostat(1)`, запущенная в другом терминале, подтвердила, что пропускная способность дискового ввода/вывода при записи составила около 585 Мбайт/с.

Используйте `iflag=direct` для прямого чтения их входных файлов.

9.8.2. Пользовательские генераторы нагрузки

Для создания своих рабочих нагрузок можно написать собственный генератор нагрузки и измерить итоговую производительность с помощью `iostat(1)`. Роль такого генератора нагрузки может играть короткая программа на языке C, которая открывает устройство и применяет предполагаемую рабочую нагрузку. В Linux специальные файлы блочных устройств можно открывать с флагом `O_DIRECT`, чтобы исключить буферизацию. Если вы пользуетесь языками более высокого

уровня, попробуйте использовать системные интерфейсы, чтобы как минимум обойти механизмы буферизации в стандартных библиотеках (например, `sysread()` в Perl) и, желательно, также механизм буферизации в ядре (например, применив флаг `O_DIRECT`).

9.8.3. Инструменты микробенчмаркинга

Доступные инструменты бенчмаркинга дисков включают, например, `hdparm(8)` в Linux:

```
# hdparm -Tt /dev/sdb
/dev/sdb:
Timing cached reads:   16718 MB in 2.00 seconds = 8367.66 MB/sec
Timing buffered disk reads: 846 MB in 3.00 seconds = 281.65 MB/sec
```

Параметр `-T` тестирует чтение из кэша, а `-t` — чтение с дискового устройства. Полученные результаты показывают, насколько велика разница между попаданиями и промахами дискового кэша.

Изучите документацию по инструменту, чтобы понимать все достоинства и недостатки, и загляните в главу 12 «Бенчмаркинг» — там приводятся дополнительные сведения о микробенчмаркинге. См. также главу 8 «Файловые системы», где описываются инструменты для проверки производительности диска через файловую систему (которых доступно гораздо больше).

9.8.4. Пример произвольного чтения

В качестве примера эксперимента я написал инструмент, выполняющий произвольное чтение блоками по 8 Кбайт из файла дискового устройства. Одновременно выполнялись от одного до пяти экземпляров инструмента и утилита `iostat(1)`. Столбцы, отражающие нулевой объем записи, были удалены:

Device:	rrqm/s	r/s	rkB/s	avgrq-sz	aqu-sz	r_await	svctm	%util
sda	878.00	234.00	2224.00	19.01	1.00	4.27	4.27	100.00
[...]								
Device:	rrqm/s	r/s	rkB/s	avgrq-sz	aqu-sz	r_await	svctm	%util
sda	1233.00	311.00	3088.00	19.86	2.00	6.43	3.22	100.00
[...]								
Device:	rrqm/s	r/s	rkB/s	avgrq-sz	aqu-sz	r_await	svctm	%util
sda	1366.00	358.00	3448.00	19.26	3.00	8.44	2.79	100.00
[...]								
Device:	rrqm/s	r/s	rkB/s	avgrq-sz	aqu-sz	r_await	svctm	%util
sda	1775.00	413.00	4376.00	21.19	4.01	9.66	2.42	100.00
[...]								
Device:	rrqm/s	r/s	rkB/s	avgrq-sz	aqu-sz	r_await	svctm	%util
sda	1977.00	423.00	4800.00	22.70	5.04	12.08	2.36	100.00

Обратите внимание на ступенчатое увеличение `aqu-sz` и увеличивающуюся задержку `r_await`.

9.8.5. ioping

ioping(1) — это интересный инструмент для микробенчмаркинга дисков, напоминающий утилиту ring(8). Вот пример запуска ioping(1) на дисковом устройстве nvme0n1:

```
# ioping /dev/nvme0n1
4 KiB <<< /dev/nvme0n1 (block device 8 GiB): request=1 time=438.7 us (warmup)
4 KiB <<< /dev/nvme0n1 (block device 8 GiB): request=2 time=421.0 us
4 KiB <<< /dev/nvme0n1 (block device 8 GiB): request=3 time=449.4 us
4 KiB <<< /dev/nvme0n1 (block device 8 GiB): request=4 time=412.6 us
4 KiB <<< /dev/nvme0n1 (block device 8 GiB): request=5 time=468.8 us
^C
--- /dev/nvme0n1 (block device 8 GiB) ioping statistics ---
4 requests completed in 1.75 ms, 16 KiB read, 2.28 k iops, 8.92 MiB/s
generated 5 requests in 4.37 s, 20 KiB, 1 iops, 4.58 KiB/s
min/avg/max/mdev = 412.6 us / 437.9 us / 468.8 us / 22.4 us
```

По умолчанию ioping(1) выполняет чтение со скоростью 4 Кбайт/с и выводит задержку ввода/вывода в микросекундах. По окончании выводятся различные статистики.

В отличие от других инструментов бенчмаркинга ioping(1) создает небольшую рабочую нагрузку. Вот что сообщает iostat(1) во время работы ioping(1):

```
$ iostat -xsx 1
[...]
```

Device	tps	kB/s	rqm/s	await	aqu-sz	areq-sz	%util
nvme0n1	1.00	4.00	0.00	0.00	0.00	4.00	0.40

Потребление диска составило всего 0,4 %. ioping(1) можно использовать для отладки проблем в промышленной среде, где применение других инструментов микробенчмаркинга будет невозможным из-за того, что они создают слишком высокую нагрузку на диски.

9.8.6. fio

fio (Flexible IO Tester) — это гибкий инструмент для бенчмаркинга файловых систем, который может пролить свет на производительность дисковых устройств, особенно при использовании с параметром `--direct=true`, запрещающим использовать буферизацию (если небуферизованный ввод/вывод поддерживается файловой системой). Он был представлен в главе 8 «Файловые системы», в разделе 8.7.2 «Инструменты микробенчмаркинга».

9.8.7. blkreplay

blkreplay (block I/O replay) — это инструмент воспроизведения блочного ввода/вывода, который может воссоздавать нагрузки блочного ввода/вывода, зафиксированные с помощью blktrace (см. раздел 9.6.10 «blktrace») или Windows DiskMon

[Schöbel-Theuer 12]. Пригодится при отладке проблем с диском, которые трудно воспроизвести с помощью инструментов микробенчмаркинга.

См. главу 12 «Бенчмаркинг», раздел 12.2.3 «Воспроизведение», где показан пример, как воспроизведение дискового ввода/вывода может ввести в заблуждение, если целевая система изменилась.

9.9. НАСТРОЙКА

В разделе 9.5 «Методология» мы уже рассмотрели множество подходов к настройке, включая настройку кэша, масштабирование и определение характеристик рабочей нагрузки, которые помогают выявить и устранить ненужную работу. Другая важная область настройки — это конфигурация хранилища, которую можно исследовать как часть методологии статической настройки производительности.

В разделах ниже показаны области, доступные для настройки: операционная система, дисковые устройства и контроллер диска. Конкретные настройки — доступные параметры и оптимальные значения — зависят от версии ОС, а также моделей дисков, контроллеров и их микропрограмм. Дополнительную информацию ищите в соответствующей документации. Изменить настройки легко, но настройки по умолчанию обычно достаточно разумные и редко нуждаются в дополнительной коррекции.

9.9.1. Параметры операционной системы

К ним относятся `ionice(1)`, средства управления ресурсами и настраиваемые параметры ядра.

ionice

В Linux с помощью команды `ionice(1)` можно установить класс планирования ввода/вывода и приоритет процесса. Классы планирования идентифицируются числовыми значениями:

- **0, нет:** класс не указан, поэтому ядро выберет значение по умолчанию — наилучшее из возможных с учетом приоритета, исходя из значения `nice` (уступчивости) процесса;
- **1, в реальном времени:** доступ к диску с наивысшим приоритетом. При неправильном применении может привести к нехватке ресурсов для других процессов (по аналогии с классом планирования процессора RT);
- **2, наилучший из возможных:** класс планирования по умолчанию, поддерживающий приоритеты 0–7, где 0 — наивысший;
- **3, бездействие:** дисковый ввод/вывод разрешен только после некоторого периода бездействия диска.

Пример использования:

```
# ionice -c 3 -p 1623
```

Эта команда поместит процесс с идентификатором 1623 в класс планирования ввода/вывода в режиме бездействия. Это может быть желательно для продолжительных заданий резервного копирования, чтобы уменьшить вероятность создания помех промышленной рабочей нагрузке.

Управление ресурсами

Современные ОС поддерживают средства управления ресурсами для регулирования потребления ввода/вывода, дискового или через файловые системы.

Управление ресурсами устройств хранения для процессов или групп процессов в Linux обеспечивает подсистема контрольных групп (cgroups) блочного ввода/вывода (blkio). Ограничение может устанавливаться как пропорциональная доля потребления или фиксированное значение. Пределы могут устанавливаться независимо для чтения и записи в отношении IOPS или пропускной способности (байтов в секунду). Дополнительные сведения см. в главе 11 «Облачные вычисления».

Настраиваемые параметры

В число настраиваемых параметров Linux входят:

- **/sys/block/*/queue/scheduler**: позволяет устанавливать политики планирования ввода/вывода, такие как noop, deadline, cfq и др., см. описание в разделе 9.4 «Архитектура»;
- **/sys/block/*/queue/nr_requests**: количество запросов чтения или записи, которые могут быть приняты блочным уровнем;
- **/sys/block/*/queue/read_ahead_kb**: максимальный размер опережающего чтения в килобайтах.

Как и в случае с другими настройками ядра, прежде чем что-то изменять, ознакомьтесь с полным списком доступных параметров, их описаниями и предупреждениями в документации, которая находится в исходном коде Linux: Documentation/block/queue-sysfs.txt.

9.9.2. Настраиваемые параметры дисковых устройств

В Linux есть инструмент `hdparm(8)`, позволяющий настраивать различные параметры дискового устройства, включая управление питанием и тайм-ауты остановки [Archlinux 20]. Будьте очень осторожны при использовании этого инструмента и изучите страницу справочного руководства `man` для `hdparm(8)`: некоторые параметры отмечены как «DANGEROUS» («ОПАСНО») — их изменение может привести к потере данных.

9.9.3. Настраиваемые параметры контроллера диска

Перечень доступных для настройки параметров контроллера диска зависит от модели контроллера и производителя. Чтобы вы могли получить представление о том, какие параметры могут быть доступны, ниже показаны некоторые настройки контроллера Dell PERC 6, полученные с помощью команды MegaCli:

```
# MegaCli -AdpAllInfo -aALL
[...]
Predictive Fail Poll Interval      : 300sec
Interrupt Throttle Active Count   : 16
Interrupt Throttle Completion    : 50us
Rebuild Rate                      : 30%
PR Rate                          : 0%
BGI Rate                         : 1%
Check Consistency Rate           : 1%
Reconstruction Rate             : 30%
Cache Flush Interval             : 30s
Max Drives to Spinup at One Time : 2
Delay Among Spinup Groups        : 12s
Physical Drive Coercion Mode     : 128MB
Cluster Mode                    : Disabled
Alarm                           : Disabled
Auto Rebuild                    : Enabled
Battery Warning                  : Enabled
Ecc Bucket Size                 : 15
Ecc Bucket Leak Rate            : 1440 Minutes
Load Balance Mode               : Auto
[...]
```

Имя каждого параметра достаточно описательное, и его назначение подробно объясняется в документации.

9.10. УПРАЖНЕНИЯ

1. Ответьте на следующие вопросы, касающиеся дисковых устройств:
 - Что такое IOPS?
 - Чем отличается время обслуживания от времени ожидания?
 - Как определяется время ожидания дискового ввода/вывода?
 - Что такое выброс задержки?
 - Что такое дисковая команда, не связанная с передачей данных?
2. Ответьте на следующие концептуальные вопросы:
 - Опишите такие характеристики, как потребление и насыщенность диска.
 - Опишите различия в производительности между произвольным и последовательным дисковым вводом/выводом.
 - Опишите роль внутреннего дискового кэша для операций чтения и записи.

3. Ответьте на следующие, более сложные вопросы:

- Объясните, почему метрика потребления виртуальных дисков может вводить в заблуждение.
- Объясните, почему метрика «ожидание ввода/вывода» может вводить в заблуждение.
- Опишите характеристики производительности RAID-0 (с чередованием) и RAID-1 (зеркалирование).
- Опишите, что происходит, когда диски перегружены работой, включая влияние на производительность приложений.
- Опишите, что происходит, когда контроллер хранилища перегружен работой (с точки зрения пропускной способности или IOPS), включая влияние на производительность приложения.

4. Разработайте следующие процедуры для своей ОС:

- Чек-лист для метода USE с целью анализа потребления дисковых ресурсов (дисков и контроллеров). Опишите, как получить каждую метрику (например, какую команду выполнить) и как интерпретировать результаты. Перед установкой или использованием дополнительных программных продуктов постарайтесь ограничиться инструментами наблюдения, имеющимися в ОС.
- Чек-лист для определения характеристик рабочей нагрузки на дисковые ресурсы. Опишите, как получить каждую метрику, и для начала попробуйте ограничиться инструментами наблюдения в ОС.

5. Опишите поведение диска по следующему выводу `iostat(1)` в Linux:

```
$ iostat -x 1
[...]
```

avg-cpu:	%user	%nice	%system	%iowait	%steal	%idle			
	3.23	0.00	45.16	31.18	0.00	20.43			

Device:		rrqm/s	wrqm/s	r/s	w/s	rkB/s	wkB/s	avgrq-sz	
avgqu-sz	await	r_await	w_await	svctm	%util				
vda		39.78	13156.99	800.00	151.61	3466.67	41200.00	93.88	
11.99	7.49	0.57	44.01	0.49	46.56				
vdb		0.00	0.00	0.00	0.00	0.00	0.00	0.00	
0.00	0.00	0.00	0.00	0.00	0.00				

6. (опционально) Разработайте инструмент для трассировки всех дисковых команд, кроме операций чтения и записи. Для этого может понадобиться организовать трассировку на уровне SCSI.

9.11. ССЫЛКИ

[Patterson 88] Patterson, D., Gibson, G., and Kats, R., «A Case for Redundant Arrays of Inexpensive Disks», ACM SIGMOD, 1988.

[McDougall 06a] McDougall, R., Mauro, J., and Gregg, B., «Solaris Performance and Tools: DTrace and MDB Techniques for Solaris 10 and OpenSolaris», Prentice Hall, 2006.

- [Brunelle 08] Brunelle, A., «btt User Guide», blktrace package, /usr/share/doc/blktrace/btt.pdf, 2008.
- [Gregg 08] Gregg, B., «Shouting in the Datacenter», <https://www.youtube.com/watch?v=tDacjrSCeq4>, 2008.
- [Mason 08] Mason, C., «Seekwatcher», <https://oss.oracle.com/~mason/seekwatcher>, 2008.
- [Smith 09] Smith, R., «Western Digital's Advanced Format: The 4K Sector Transition Begins», <https://www.anandtech.com/show/2888>, 2009.
- [Gregg 10a] Gregg, B., «Visualizing System Latency», Communications of the ACM, July 2010.
- [Love 10] Love, R., «Linux Kernel Development, 3rd Edition», Addison-Wesley, 2010¹.
- [Turner 10] Turner, J., «Effects of Data Center Vibration on Compute System Performance», USENIX SustainIT, 2010.
- [Gregg 11b] Gregg, B., «Utilization Heat Maps», <http://www.brendangregg.com/HeatMaps/utilization.html>, 2011.
- [Cassidy 12] Cassidy, C., «SLC vs MLC: Which Works Best for High-Reliability Applications?», <https://www.eetimes.com/slc-vs-mlc-which-works-best-for-high-reliability-applications/#>, 2012.
- [Cornwell 12] Cornwell, M., «Anatomy of a Solid-State Drive», Communications of the ACM, December 2012.
- [Schöbel-Theuer 12] Schöbel-Theuer, T., «blkreplay — a Testing and Benchmarking Toolkit», <http://www.blkreplay.org>, 2012.
- [Chazarain 13] Chazarain, G., «Iotop», <http://guichaz.free.fr/iotop>, 2013.
- [Corbet 13b] Corbet, J., «The multiqueue block layer», LWN.net, <https://lwn.net/Articles/552904>, 2013.
- [Leventhal 13] Leventhal, A., «A File System All Its Own», ACM Queue, March 2013.
- [Cai 15] Cai, Y., Luo, Y., Haratsch, E. F., Mai, K., and Mutlu, O., «Data Retention in MLC NAND Flash Memory: Characterization, Optimization, and Recovery», IEEE 21st International Symposium on High Performance Computer Architecture (HPCA), 2015. https://users.ece.cmu.edu/~omutlu/pub/flash-memory-data-retention_hpca15.pdf
- [FICA 18] «Industry's Fastest Storage Networking Speed Announced by Fibre Channel Industry Association — 64GFC and Gen 7 Fibre Channel», Fibre Channel Industry Association, <https://fibrechannel.org/industrys-fastest-storage-networking-speed-announced-by-fibre-channel-industry-association-%E2%94%80-64gfc-and-gen-7-fibre-channel>, 2018.
- [Hady 18] Hady, F., «Achieve Consistent Low Latency for Your Storage-Intensive Workloads», <https://www.intel.com/content/www/us/en/architecture-and-technology/optane-technology/low-latency-for-storage-intensive-workloads-article-brief.html>, 2018.
- [Gregg 19] Gregg, B., «BPF Performance Tools: Linux System and Application Observability»², Addison-Wesley, 2019.

¹ Лав Р. «Ядро Linux: описание процесса разработки. 3-е издание».

² Грегг Б. «BPF: Профессиональная оценка производительности». Выходит в издательстве «Питер» в 2023 году.

- [Archlinux 20] «hdparm», <https://wiki.archlinux.org/index.php/Hdparm>, последнее обновление 2020.
- [Dell 20] «PowerEdge RAID Controller», <https://www.dell.com/support/article/en-us/sln312338/poweredge-raid-controller?lang=en>, по состоянию на 2020.
- [FCIA 20] «Features», Fibre Channel Industry Association, <https://fibrechannel.org/fibre-channel-features>, по состоянию на 2020.
- [Liu 20] Liu, L., «Samsung QVO vs EVO vs PRO: What's the Difference? [Clone Disk]», <https://www.partitionwizard.com/clone-disk/samsung-qvo-vs-evo.html>, 2020.
- [Mellor 20] Mellor, C., «Western Digital Shingled Out in Lawsuit for Sneaking RAIDunfriendly Tech into Drives for RAID arrays», TheRegister, https://www.theregister.com/2020/05/29/wd_class_action_lawsuit, 2020.
- [Netflix 20] «Open Connect Appliances», <https://openconnect.netflix.com/en/appliances>, по состоянию на 2020.
- [NVMe 20] «NVM Express», <https://nvmexpress.org>, по состоянию на 2020.

Глава 10

СЕТЬ

Поскольку системы становятся все более распределенными, особенно в среде облачных вычислений, сеть играет все большую роль в производительности. Типичные задачи, связанные с производительностью сети, включают уменьшение задержек и увеличение пропускной способности, а также устранение выбросов задержек, которые могут быть вызваны сбросом или задержкой сетевых пакетов.

Анализ сети охватывает аппаратное и программное обеспечение. Аппаратное обеспечение — это физическая инфраструктура сети, включающая сетевые карты, коммутаторы, маршрутизаторы и шлюзы (обычно они тоже работают под управлением программного обеспечения). Программное обеспечение — это сетевой стек ядра, включающий драйверы сетевых устройств, очереди пакетов и планировщики пакетов, а также реализацию сетевых протоколов. Протоколы низкого уровня обычно реализуются в ядре (IP, TCP, UDP и т. д.), а протоколы высокого уровня — в библиотеках или приложениях (например, HTTP).

В низкой производительности часто винят сеть, учитывая потенциальную перегруженность и присущую ей сложность (виновато неизвестное). В этой главе я покажу, как узнать, что происходит на самом деле, и эта информация может реабилитировать сеть для продолжения анализа.

Цели этой главы:

- познакомить с моделями и основными понятиями сетей;
- представить различные метрики, характеризующие задержки в сети;
- дать полное представление о работе сетевых протоколов;
- познакомить с внутренним устройством сетевого оборудования;
- познакомить с путями в коде ядра между сокетами и устройствами;
- представить различные методики анализа производительности сети;
- охарактеризовать общесистемный и индивидуальный сетевой ввод/вывод для каждого процесса;
- показать, как идентифицировать проблемы, вызванные повторной передачей пакетов TCP;

- познакомить с приемами использования инструментов трассировки для изучения внутреннего устройства сети;
- познакомить с параметрами настройки сети.

В главе шесть частей. В первых трех мы обсудим основы анализа производительности сети, а в следующих трех я покажу их практическое применение в системах на базе Linux:

- **Основы** знакомит с терминологией, моделями и ключевыми понятиями сетей.
- **Архитектура** содержит обобщенное описание физических компонентов сети и сетевого стека.
- **Методология** описывает методологии анализа эффективности, как наблюдаемые, так и экспериментальные.
- **Инструменты наблюдения** описывает инструменты наблюдения за работой сети в системах на базе Linux.
- **Экспериментирование** обобщает возможности инструментов для бенчмаркинга сети и проведения экспериментов.
- **Настройка** описывает параметры настройки сети.

Эта глава предполагает наличие у читателей базовых знаний о сети, таких как роль TCP и IP.

10.1. ТЕРМИНОЛОГИЯ

Ниже перечислены основные термины, связанные с сетью:

- **Интерфейс:** под *портом интерфейса* подразумевается физический разъем для подключения сетевого кабеля. Термин *интерфейс*, или *подключение* (link), обозначает логический порт сетевого интерфейса, который настраивается ОС. (Не все интерфейсы, доступные в ОС, имеют аппаратную поддержку: некоторые из них — виртуальные.)
- **Пакет:** термин *пакет* обозначает сообщение, распространяемое в сети с коммутацией пакетов, например IP-пакет.
- **Фрейм:** сообщение в сети физического уровня, например фрейм Ethernet.
- **Сокет:** программный интерфейс, представляющий конечные точки сети и первоначально появившийся в BSD.
- **Ширина полосы пропускания** (bandwidth): максимальная скорость передачи данных для данного типа сети, обычно измеряется в битах в секунду. «100 GbE» — это сеть Ethernet с шириной полосы 100 Гбит/с. Ширина полосы может ограничиваться отдельно для каждого направления, поэтому 100 GbE может одновременно передавать и принимать со скоростью 100 Гбит/с (в таком случае общая пропускная способность составит 200 Гбит/с).

- **Пропускная способность** (throughput): текущая скорость передачи данных между конечными точками сети, измеряется в битах в секунду или байтах в секунду.
- **Задержка**: под сетевой задержкой может подразумеваться время, необходимое для передачи сообщения между конечными точками, или время для установления соединения (например, трехэтапной процедуры «рукопожатия» в TCP), исключая время передачи данных, которая следует за этим.

На протяжении всей главы будут вводиться и другие термины. Глоссарий включает базовую терминологию, в том числе: *клиент*, *Ethernet*, *хост*, *IP*, *RFC*, *сервер*, *SYN*, *ACK*. См. также разделы «Терминология» в главах 2 и 3.

10.2. МОДЕЛИ

Ниже описаны простые модели, иллюстрирующие некоторые базовые принципы устройства сети и ее работы. Раздел 10.4 «Архитектура» содержит более подробную информацию и описывает тонкости, зависящие от реализации.

10.2.1. Сетевой интерфейс

Сетевой интерфейс — это конечная точка в ОС для сетевых подключений. Это абстракция, настраиваемая и управляемая системными администраторами.

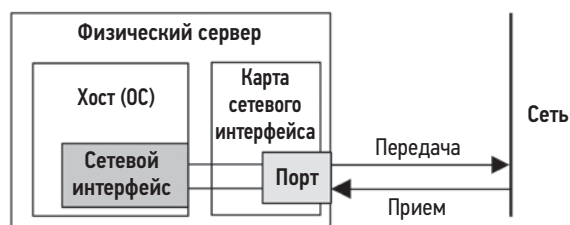


Рис. 10.1. Сетевой интерфейс

Сетевой интерфейс изображен на рис. 10.1. Сетевые интерфейсы отображаются в физические сетевые порты согласно их конфигурации. Порты подключаются к сети и обычно имеют отдельные каналы для передачи и приема.

10.2.2. Контроллер

Сетевая карта (network interface card, NIC) имеет один или несколько сетевых портов и содержит *сетевой контроллер*: микропроцессор для передачи пакетов между портами и транспортом (шиной) ввода/вывода системы. На рис. 10.2 показан пример контроллера с четырьмя портами и физическими компонентами.

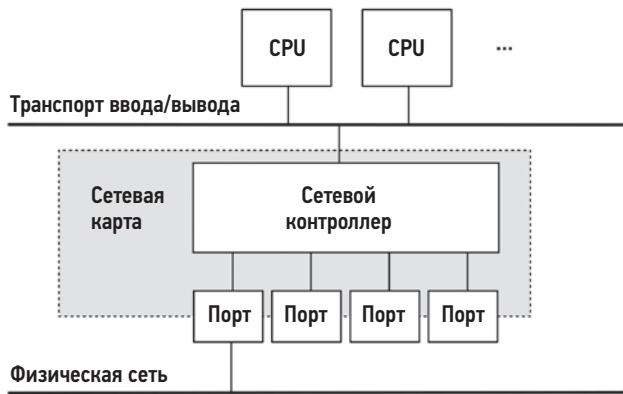


Рис. 10.2. Сетевой контроллер

Конструктивно контроллер обычно оформляется в виде отдельной платы расширения или набора микросхем на материнской плате. (Есть контроллеры, подключаемые к порту USB.)

10.2.3. Стек протоколов

Сетевые взаимодействия обеспечиваются стеком протоколов, каждый уровень которых служит определенной цели. На рис. 10.3 показаны две модели стека с примерами протоколов.

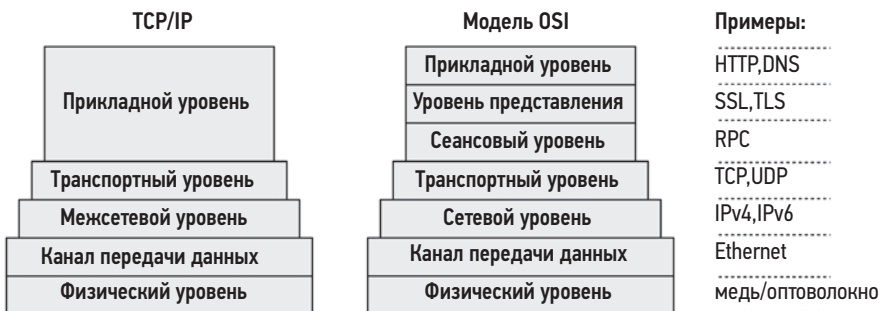


Рис. 10.3. Стек сетевых протоколов

Нижние уровни изображены шире, чтобы обозначить охват протокола. Отправляемые сообщения перемещаются вниз по стеку из приложения в физическую сеть. Принимаемые сообщения, наоборот, перемещаются вверх.

Обратите внимание: стандарт Ethernet тоже описывает физический уровень и использование меди или оптоволокна.

При использовании некоторых протоколов в модель могут добавляться дополнительные уровни, например Internet Protocol Security (IPsec) или Linux WireGuard, которые находятся над межсетевым уровнем и обеспечивают безопасность данных, передаваемых между конечными точками IP. Кроме того, при использовании туннелирования (например, для создания виртуальной расширяемой локальной сети (Virtual Extensible LAN, VXLAN) один стек протоколов может инкапсулироваться в другой.

Стек TCP/IP давно стал стандартом, но думаю, будет полезно кратко рассмотреть и модель OSI, поскольку она показывает уровни протокола внутри приложения¹. Термин «уровень» взят из OSI, где *Уровень 3* относится к сетевым протоколам.

На разных уровнях используются разные термины для обозначения сообщений. В модели OSI на транспортном уровне сообщение называется *сегментом* или *дейтаграммой*. На сетевом уровне — это *пакет*, а на канальном — *фрейм*.

10.3. ОСНОВНЫЕ ПОНЯТИЯ

Ниже описаны некоторые важные понятия, касающиеся сети и ее работы.

10.3.1. Сети и маршрутизация

Сеть — это группа хостов, которые могут обмениваться данными по сети с использованием адресов сетевых протоколов. Наличие нескольких сетей вместо одной гигантской всемирной сети предпочтительнее по ряду причин, главной из которых является масштабируемость. Некоторые сетевые сообщения — *широковещательные* и передаются всем соседним хостам. Создавая подсети меньшего размера, можно ограничить распространение таких широковещательных сообщений, чтобы они не создавали масштабных проблем с лавинной рассылкой. Кроме того, деление на подсети образует основу для ограничения передачи обычных сообщений только в границах сети между отправителем и получателем, что позволяет эффективнее использовать сетевую инфраструктуру.

Маршрутизация управляет доставкой сообщений, называемых пакетами, по этим сетям. Роль маршрутизации показана на рис. 10.4.

С точки зрения хоста А *локальный хост* (localhost) — это сам хост А. Все остальные хосты, изображенные на схеме, являются для него *удаленными хостами*.

Хост А может подключиться к хосту В через локальную сеть, обычно управляемую сетевым коммутатором (см. раздел 10.4 «Архитектура»). Также он может подключиться к хосту С через маршрутизатор 1 и к хосту D через маршрутизаторы 1, 2 и 3. Поскольку сетевые компоненты, такие как маршрутизаторы, общие, конкуренция

¹ Стоит *вкратце* рассмотреть эту модель, но включать ее в тест на знание сетевых технологий я не буду.

со стороны другого трафика (например, передаваемого от хоста С к хосту Е) может снизить производительность.

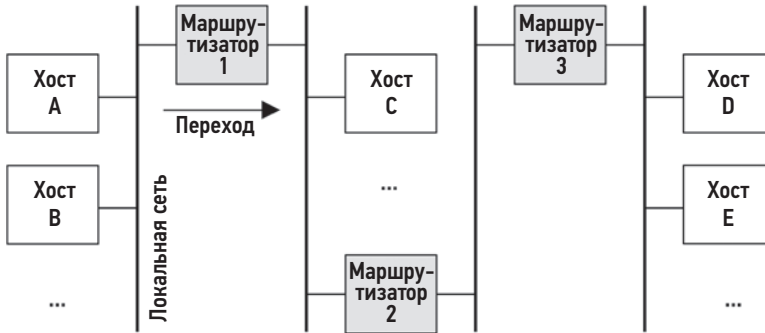


Рис. 10.4. Сети, соединенные через маршрутизаторы

Соединения между парами хостов предполагают *одноадресную* (unicast) передачу. *Многоадресная* (multicast) передача позволяет передавать данные сразу нескольким адресатам, которые могут находиться в разных подсетях, но такая возможность должна поддерживаться конфигурацией маршрутизатора. В общедоступных облачных средах она может быть отключена.

Кроме маршрутизаторов в типичных сетях также используются *брандмауэры* для повышения безопасности, блокирующие нежелательные соединения между хостами.

Информация об адресе, необходимая для маршрутизации пакетов, содержится в заголовке IP.

10.3.2. Протоколы

Стандарты сетевых протоколов — IP, TCP и UDP — необходимое требование для организации связи между системами и устройствами. Связь осуществляется путем передачи маршрутизируемых сообщений — *пакетов*, обычно инкапсуляцией данных полезной нагрузки в пакет.

Сетевые протоколы имеют разные характеристики производительности, обусловленные устройством протокола, поддержкой расширений или тем, как с ними оперируют программное или аппаратное обеспечение. Например, разные версии протокола IP — IPv4 и IPv6 — могут обрабатываться разными путями в коде ядра и иметь разные характеристики производительности. Другие протоколы могут иметь свои конструктивные особенности и выбираться как наиболее подходящие для данной рабочей нагрузки: примерами могут служить протокол передачи с управлением потоками (Stream Control Transmission Protocol, SCTP), многоканальный TCP (Multipath TCP, MPTCP) и QUIC.

Есть и системные настраиваемые параметры, которые могут влиять на производительность протокола, изменяя такие параметры, как размер буфера, алгоритмы и различные таймеры. Эти параметры для конкретных протоколов описаны далее.

Протоколы обычно передают данные с использованием инкапсуляции.

10.3.3. Инкапсуляция

Суть инкапсуляции заключается в добавлении метаданных к полезной нагрузке, в начало (*заголовок* — header), в конец (*подвал* — footer) или туда и туда. Инкапсуляция не изменяет данные полезной нагрузки, но немного увеличивает общий размер сообщения, что требует дополнительных затрат на передачу.

На рис. 10.5 показан пример инкапсуляции для стека TCP/IP с Ethernet.

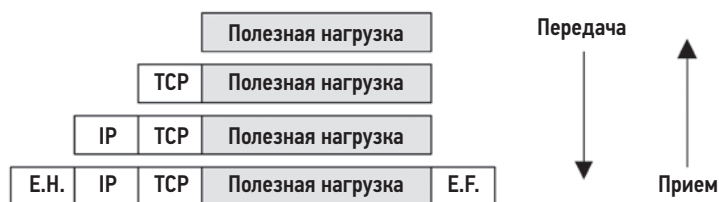


Рис. 10.5. Инкапсуляция сетевого протокола

Буквами «E.H.» здесь обозначен заголовок Ethernet (Ethernet header), а буквами «E.F.» — необязательный подвал Ethernet (Ethernet footer).

10.3.4. Размер пакета

Размер пакетов и их полезной нагрузки тоже влияет на производительность, при этом большие размеры улучшают пропускную способность и уменьшают оверхед. Размеры пакетов TCP/IP и Ethernet бывают от 54 до 9054 байтов, включая 54 байта (или больше, в зависимости от параметров или версии) заголовка протокола.

Размер пакета обычно ограничивается размером *максимального блока передачи* (maximum transmission unit, MTU) сетевого интерфейса, который для многих сетей Ethernet составляет 1500 байт. Размер 1500 возник в ранних версиях Ethernet и был обусловлен необходимостью сбалансировать стоимость буферной памяти в сетевой карте и задержку передачи [Nosachev 20]. Хосты конкурировали за совместно используемый носитель (коаксиальный кабель или концентратор Ethernet), и большие размеры увеличивали задержку для хостов, ожидающих своей очереди.

Современный Ethernet поддерживает большие пакеты (фреймы), размером до 9000 байт, которые называют *jumbo-фреймами*¹. Он может улучшить пропускную

¹ Дословно: сверхбольшие фреймы. — *Примеч. пер.*

способность сети, а также уменьшить задержку передачи данных за счет меньшего количества передаваемых пакетов.

Внедрению jumbo-фреймов помешало сочетание двух факторов: старое сетевое оборудование и неправильно настроенные брандмауэры. Старое оборудование, которое не поддерживает jumbo-фреймы, может либо фрагментировать пакет с использованием протокола IP (что приводит к снижению производительности из-за повторной сборки пакета), либо отвечать ICMP-ошибкой «can't fragment» (не удастся фрагментировать), давая понять отправителю, что нужно уменьшить размер пакета. В последнем случае в игру вступают неправильно настроенные брандмауэры: в прошлом происходили сетевые атаки на основе ICMP (включая «пинг смерти»), защищаясь от которых некоторые администраторы брандмауэров блокировали все ICMP-сообщения. Из-за этого полезные сообщения с ошибкой «can't fragment» не достигают отправителя, и сетевые пакеты автоматически отбрасываются, когда их размер превышает 1500. Если ICMP-сообщение получено и происходит фрагментация, возникает риск, что фрагментированные пакеты будут отброшены устройствами, которые их не поддерживают. Чтобы избежать этих проблем, многие системы используют для MTU значение по умолчанию 1500.

Производительность фреймов размером 1500 была улучшена за счет внедрения в сетевые карты новых функций, включая *разгрузку TCP* (TCP offload) и *разгрузку больших сегментов* (large segment offload). В системах с поддержкой этих функций сетевой карте передаются большие буферы, а та, в свою очередь, делит их на более мелкие фреймы, используя выделенное и оптимизированное оборудование. Это в некоторой степени сократило разрыв в производительности между сетями с MTU 1500 и 9000.

10.3.5. Задержка

Задержка — одна из важнейших метрик, характеризующих производительность сети. Она может измеряться разными способами, включая задержку разрешения имен, задержку проверки связи, задержку соединения, задержку первого байта, время приема-передачи и продолжительность жизни соединения. Они измеряются на стороне клиента, подключающегося к серверу.

Задержка разрешения имен

Прежде чем установить соединение с удаленным хостом, имя этого хоста обычно преобразуется в IP-адрес, например, с помощью разрешения DNS. Необходимое на это время можно измерить отдельно как задержку разрешения имен. В худшем случае такая задержка будет обусловлена тайм-аутом разрешения имен, который может составлять десятки секунд.

Операционные системы часто включают службу разрешения имен, которая обеспечивает кэширование, чтобы последующие DNS-запросы могли быстро удовлетворяться из кэша. Иногда приложения используют только IP-адреса, поэтому задержка DNS полностью устраняется.

Задержка проверки связи

Это время, измеряемое командой `ping(1)`, в течение которого на эхо-запрос ICMP приходит эхо-ответ. Это время используется для измерения задержки передачи данных между узлами в сети, включая переходы между ними. Оно измеряется как время, необходимое для выполнения сетевого запроса в оба конца. Благодаря простоте и доступности этот прием получил широкое распространение: многие ОС по умолчанию реагируют на эхо-запросы. Но эта метрика может неточно отражать время приема/передачи запросов приложений, потому что маршрутизаторы могут обрабатывать ICMP с другим приоритетом.

Пример задержки проверки связи показан в табл. 10.1.

Таблица 10.1. Примеры задержек проверки связи

Откуда	Куда	Через	Задержка	В масштабе
Локальный хост	Локальный хост	Ядро	0,05 мс	1 с
Хост	Хост (в той же подсети)	10 GbE	0,2 мс	4 с
Хост	Хост (в той же подсети)	1 GbE	0,6 мс	12 с
Хост	Хост (в той же подсети)	Wi-Fi	3 мс	1 мин
Сан-Франциско	Нью-Йорк	Интернет	40 мс	12 мин
Сан-Франциско	Великобритания	Интернет	81 мс	27 мин
Сан-Франциско	Австралия	Интернет	183 мс	1 час

Для лучшей иллюстрации соотношения порядков величин в столбце «В масштабе» показаны величины задержек относительно длительности воображаемой задержки проверки связи с локальным хостом в одну секунду.

Задержка подключения

Задержка подключения — это время, необходимое для установления сетевого подключения перед передачей каких-либо данных. *Задержка TCP-подключения* — это время выполнения трехэтапной процедуры «рукопожатия» в TCP. Измеряется на стороне клиента от отправки пакета SYN до получения соответствующего пакета SYN-ACK. Задержку подключения, возможно, лучше было бы назвать *задержкой установления соединения*, чтобы четко отделить ее от продолжительности жизни соединения.

Задержка подключения сопоставима с задержкой проверки связи, хотя при этом в ядре выполняется больше кода и может добавляться время для повторной передачи любых отброшенных пакетов. В частности, сервер может отбросить TCP-пакет SYN, если его очередь входящих запросов окажется заполненной до отказа, в результате чего клиент будет вынужден повторить отправку пакета SYN по истечении тайм-аута. Эта задержка происходит во время выполнения процедуры «рукопожатия»

TCP, поэтому может включать задержку повторной передачи, добавляя одну или несколько секунд.

За задержкой подключения следует задержка передачи первого байта.

Задержка первого байта

Задержка первого байта, или *время до первого байта* (time to first byte, TTFB), — это время от момента установления соединения до момента получения первого байта данных. Включает время, необходимое удаленному хосту, чтобы принять соединение, запустить поток, обслуживающий его, выполнить код потока и принять первый байт.

В отличие от задержек проверки связи и подключения, оценивающих задержку сети, задержка первого байта включает и время, необходимое целевому серверу на «обдумывание». Оно может увеличиваться, если сервер перегружен и ему требуется время для обработки запроса (задержка в очереди TCP) и для планирования сервера (задержка планировщика процессора).

Время приема-передачи

Время приема-передачи (round-trip time, RTT) описывает время, необходимое для доставки сетевого запроса серверу и получения ответа. Включает время передачи сигнала и время обработки на каждом сетевом переходе. Предполагаемое использование — определение задержки сети, поэтому в идеале RTT определяется временем, которое пакеты запроса и ответа проводят в сети (без учета времени, которое удаленный хост тратит на обслуживание запроса). Часто эта метрика измеряется как время приема-передачи эхо-запросов ICMP, потому что время обработки эхо-запросов на удаленном хосте минимально.

Продолжительность жизни соединения

Продолжительность жизни соединения — это время от момента установления сетевого подключения до его закрытия. Некоторые протоколы используют стратегию *постоянного соединения* (keep-alive), чтобы будущие операции могли использовать имеющиеся соединения и избегать оверхеда и задержек на подключение (и установление соединения TLS).

Дополнительные приемы измерения задержек в сети ищите в разделе 10.5.4 «Анализ задержек», где описывается их использование для диагностики производительности сети.

10.3.6. Буферизация

Несмотря на большое разнообразие потенциальных задержек, пропускную способность сети можно поддерживать на весьма высоком уровне за счет использования буферизации на стороне отправителя и получателя. Буферы большего размера

позволяют смягчить последствия увеличения времени приема/передачи, давая возможность продолжать отправлять данные до блокировки в ожидании подтверждения.

Для повышения пропускной способности протокол TCP использует буферизацию вместе со скользящим окном отправки. Сетевые сокетсы тоже имеют буферы, и приложения могут использовать свои собственные буферы для накопления данных перед отправкой.

В целях повышения собственной пропускной способности буферизация может обеспечиваться внешними сетевыми компонентами — коммутаторами и маршрутизаторами. К сожалению, использование больших буферов в этих компонентах может привести к проблеме *раздувания буфера* (bufferbloat), когда пакеты много времени проводят в очереди. Это предотвращает перегрузку TCP на хостах и снижает производительность. В ядра Linux 3.x добавили решение этой проблемы (включая ограничение на размер очередей, дисциплину очереди CoDel [Nichols 12] и поддержку коротких очередей TCP). Есть и сайт, где ведется обсуждение этой проблемы [Bufferbloat 20].

Буферизация (или большая буферизация) дает наибольший эффект при использовании на конечных точках — хостах, а не на промежуточных сетевых узлах, согласно принципу, который называется *сквозным аргументом* (end-to-end arguments) [Saltzer 84].

10.3.7. Очередь запросов на подключение

Еще одна разновидность буферизации — буферизация запросов на подключение. TCP реализует очередь (*backlog*) в ядре, куда помещаются запросы SYN перед тем, как они будут переданы пользовательскому процессу. Когда поступает слишком много запросов на TCP-подключение, которые процесс не может обслужить вовремя, очередь переполняется и SYN-пакеты отбрасываются, из-за чего клиенты вынуждены повторить их отправку позднее. Повторная передача этих пакетов вызывает задержку подключения на стороне клиента. Размер очереди настраивается: это параметр системного вызова `listen(2)`, а кроме того, ядро может поддерживать общесистемные ограничения.

Отбрасывание и повторная передача пакетов SYN являются индикаторами перегрузки хоста.

10.3.8. Согласование интерфейса

Сетевые интерфейсы могут работать в разных режимах, автоматически согласованных сторонами соединения. Вот несколько примеров:

- **ширина полосы пропускания:** например, 10, 100, 1000, 10 000, 40 000, 100 000 Мбит/с;
- **дуплекс:** полудуплексный или полнодуплексный.

Эти примеры взяты из стандарта Ethernet, в котором для ограничения полосы пропускания обычно используют округленные числа, кратные 10. Другие протоколы физического уровня, такие как SONET, имеют другой набор поддерживаемых полос пропускания.

Сетевые интерфейсы обычно описываются с точки зрения максимальной ширины полосы пропускания и протокола, например, Ethernet 1 Гбит/с (1 GbE). Но при необходимости интерфейсы могут автоматически переключаться на пониженные скорости. Это происходит, если другая конечная точка не может работать быстрее либо из-за физических проблем с носителем сигнала (некачественный сетевой кабель).

Полнодуплексный режим позволяет осуществлять передачу одновременно в двух направлениях по отдельным каналам для передачи и приема, каждый из которых может работать с максимальной пропускной способностью. Полудуплексный режим позволяет одновременно осуществлять передачу только в одном направлении.

10.3.9. Предотвращение перегрузки

Сети — это ресурсы общего пользования, они могут быть перегружены при передаче объемного трафика. Перегрузка может вызвать проблемы с производительностью: например, маршрутизаторы или коммутаторы могут отбрасывать пакеты, вызывая повторную передачу и, соответственно, задержку. Хосты тоже могут быть перегружены при поступлении пакетов с высокой скоростью и сами могут отбрасывать пакеты.

Есть множество механизмов, позволяющих избежать этих проблем. Эти механизмы следует изучать и при необходимости настраивать, чтобы улучшить масштабируемость под нагрузкой. Вот примеры для некоторых протоколов:

- **Ethernet:** перегруженный хост может посылать отправителю *фреймы паузы*, требуя приостановить передачу (IEEE 802.3x). Есть и классы приоритета, и *приоритетные фреймы паузы* для каждого класса.
- **IP:** включает поле явного уведомления о перегрузке (Explicit Congestion Notification, ECN).
- **TCP:** включает окно перегрузки и может использовать различные алгоритмы управления перегрузкой.

В разделах ниже более подробно описаны IP ECN и *алгоритмы управления перегрузкой* в TCP.

10.3.10. Потребление

Потребление сетевого интерфейса можно рассчитать как текущую пропускную способность при максимальной пропускной способности. Но учитывая изменчивый характер полосы пропускания и дуплексного режима из-за автоматического согласования, вычислить его не так просто, как кажется.

Для полнодуплексного режима потребление вычисляется для каждого направления и измеряется как текущая пропускная способность для этого направления с текущей согласованной полосой пропускания. Обычно наибольшее значение имеет только одно направление, поскольку хосты часто асимметричны: серверы передают большие объемы данных, а клиенты — принимают.

Когда потребление в сетевом интерфейсе в каком-либо направлении достигает 100 %, оно становится узким местом, ограничивающим производительность.

Некоторые инструменты оценки производительности измеряют активность только количеством пакетов, но не байтов. Поскольку размер пакета может сильно различаться (как упоминалось выше), по количеству пакетов невозможно определить количество байтов, чтобы рассчитать пропускную способность или потребление (на основе пропускной способности).

10.3.11. Локальные подключения

Сетевые подключения могут устанавливаться между двумя приложениями в одной системе. Это *локальные* подключения, использующие виртуальный сетевой интерфейс: *loopback*¹.

Распределенные приложения часто делятся на логические части, которые обмениваются данными по сети. Сюда могут входить веб-серверы, серверы баз данных, серверы кэширования, прокси-серверы и серверы приложений. Если они работают на одном хосте, то между ними устанавливаются локальные подключения.

Подключение к локальному хосту по IP — это метод межпроцессных взаимодействий (Inter-Process Communication, IPC) на основе *IP-сокетов*. Другой метод основан на использовании сокетов домена Unix (Unix Domain Sockets, UDS), которые для обмена данными создают файл в файловой системе. Производительность с использованием UDS может быть выше, потому что взаимодействия в этом методе осуществляются в обход стека TCP/IP в ядре и без оверхеда на инкапсуляцию данных в пакеты протокола.

При использовании сокетов TCP/IP после процедуры «рукопожатия» ядро может распознать, что подключение локальное, и выполнять передачу данных в обход стека TCP/IP, повышая производительность. Эта особенность в ядре Linux получила название *дружественного TCP* (TCP friends), но пока не попала в основную ветку ядра [Corbet 12]. Сейчас для этой цели в Linux можно использовать BPF, как, например, в программном обеспечении Cilium, где этот прием применяется для увеличения производительности и безопасности контейнерных сетей [Cilium 20a].

¹ Так называемый петлевой интерфейс. — *Примеч. пер.*

10.4. АРХИТЕКТУРА

В этом разделе описывается сетевая архитектура: протоколы, оборудование и программное обеспечение. Они были упомянуты как основа для анализа и настройки производительности с акцентом на характеристики производительности. Дополнительную информацию, включая общее описание сетей, ищите в специализированных книгах [Stevens 93], [Hassan 03], RFC и руководствах производителей сетевого оборудования. Некоторые из них перечислены в конце главы.

10.4.1. Протоколы

В этом подразделе представлены характеристики производительности IP, TCP, UDP и QUIC. Особенности реализации этих протоколов в аппаратном и программном обеспечении (в том числе уменьшение затрат на сегментацию, очереди запросов на подключение и буферизация) описаны в подразделах ниже, посвященных аппаратному и программному обеспечению.

IP

Межсетевой протокол (Internet Protocol, IP) версий 4 и 6 включает поле для выбора желаемой производительности соединения: поле Type of Service (тип обслуживания) в IPv4 и поле Traffic Class (класс трафика) в IPv6. Позднее эти поля были переопределены и теперь включают Differentiated Services Code Point (DSCP, поле кода дифференцирования обслуживания) (RFC 2474) [Nichols 98] и Explicit Congestion Notification (ECN; поле явного уведомления о перегрузке) (RFC 3168) [Ramakrishnan 01].

Поле DSCP предназначено для поддержки разных классов *обслуживания* с разными характеристиками, включая вероятность отбрасывания пакетов. Вот несколько примеров классов обслуживания: телефония, широкополосное видео, данные с малой задержкой, данные с высокой пропускной способностью и данные с низким приоритетом.

ECN — это механизм, позволяющий серверам, маршрутизаторам или коммутаторам явно сигнализировать о наличии перегрузки, устанавливая соответствующий бит в заголовке IP вместо отбрасывания пакета. Получатель вернет этот сигнал отправителю, который затем может ограничить скорость передачи. Это позволяет предотвратить перегрузку сети без потери пакетов (при условии правильного использования бита ECN).

TCP

Протокол управления передачей (Transmission Control Protocol, TCP) — это стандарт, широко используемый в интернете для создания надежных сетевых подключений. TCP определяется в RFC 793 [Postel 81] и более поздних дополнениях к нему.

В отношении производительности TCP может обеспечить высокую пропускную способность даже в сетях с высокой задержкой за счет использования буферизации и *скользящего окна*. TCP тоже имеет механизм управления перегрузкой (congestion control) — отправитель может установить *окно перегрузки*, чтобы обеспечить высокую и надежную скорость передачи в разных и изменяющихся сетях. Управление перегрузкой позволяет предотвратить отправку слишком большого количества пакетов, что может вызвать снижение производительности.

Ниже перечислены некоторые характеристики производительности TCP, включая дополнения к исходной спецификации:

- **Скользящее окно:** этот механизм позволяет отправить в сеть сразу несколько пакетов с суммарным размером вплоть до размера окна, не дожидаясь получения подтверждений, что обеспечивает высокую пропускную способность даже в сетях с высокой задержкой. Размер окна объявляется получателем, устанавливая который он сообщает, сколько пакетов хотел бы получить одновременно.
- **Предотвращение перегрузки:** используется для предотвращения отправки слишком большого количества данных и достижения насыщения, которое может привести к отбрасыванию пакетов и снижению производительности.
- **Медленный старт:** часть механизма управления перегрузкой в TCP. Передача данных начинается с небольшим окном перегрузки, которое постепенно увеличивается при получении подтверждений (пакетов ACK) в течение определенного времени. Если подтверждения отсутствуют или запаздывают, окно перегрузки уменьшается.
- **Выборочные подтверждения (selective acknowledgment, SACK):** позволяют протоколу TCP подтверждать пакеты не в порядке их поступления, а выборочно, что помогает уменьшить количество повторных передач.
- **Быстрая повторная передача:** протокол TCP может повторно передать отброшенные пакеты, не дожидаясь тайм-аута, если получено повторное подтверждение. Эта функция помогает сократить длительность задержки до уровня задержки приема/передачи (RTT) вместо обычно намного более длительной задержки по тайм-ауту.
- **Быстрое восстановление:** восстанавливает производительность TCP после обнаружения повторных подтверждений путем перенастройки соединения на медленный старт.
- **Быстрое открытие TCP (TCP fast open):** позволяет клиенту включать данные в пакет SYN, чтобы сервер мог начать обработку запроса раньше, не дожидаясь завершения процедуры «рукопожатия» (RFC7413). Может использовать криптографические ключи для аутентификации клиента.
- **Отметки времени TCP:** включает отметку времени получения отправленных пакетов, которая возвращается в ACK, чтобы можно было измерить время приема/передачи (RFC 1323) [Jacobson 92].
- **Поддержка ключей TCP SYN:** предоставляет клиентам криптографические ключи при высокой вероятности атак SYN-flood (переполнение очереди запросов

на подключение), чтобы законные клиенты могли продолжать подключаться и серверу не требовалось хранить дополнительные данные для этих попыток подключения.

В некоторых случаях эти возможности и механизмы реализуются за счет использования расширенных параметров TCP, добавленных в заголовок протокола.

С точки зрения производительности TCP особенно важно обсудить следующие темы: трехэтапное «рукопожатие», обнаружение повторных подтверждений, алгоритмы управления перегрузкой, алгоритм Нейгла, отложенное подтверждение, выборочное подтверждение (SACK) и прямое подтверждение (FACK).

Трехэтапное «рукопожатие»

Соединения устанавливаются с помощью трехэтапной процедуры «рукопожатия». Один хост пассивно принимает запросы на соединение, другой активно инициирует соединение. Чтобы было понятнее, термины *пассивный* и *активный* взяты из RFC 793 [Postel 81]. Их обычно называют *прослушивающей* и *подключающейся* стороной согласно терминологии API сокетов. В модели клиент/сервер сервер прослушивает (принимает запросы), а клиент инициирует подключение.

Трехэтапная процедура «рукопожатия» изображена на рис. 10.6.

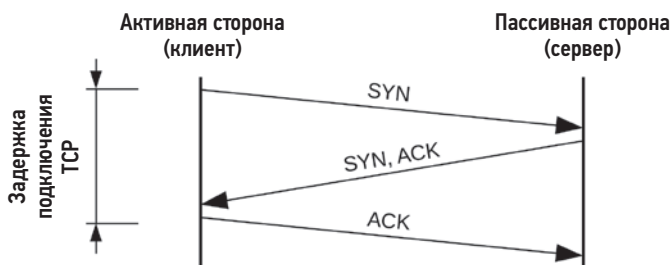


Рис. 10.6. Трехэтапная процедура «рукопожатия» в TCP

Задержка подключения измеряется на стороне клиента и завершается после отправки последнего пакета ACK. После этого может начаться передача данных.

На этом рисунке показана задержка подключения для лучшего случая. Любой пакет может быть отброшен, что увеличит задержку, потому что после истечения тайм-аута будет выполнена повторная отправка пакета.

После завершения процедуры «рукопожатия» сеанс TCP переводится в состояние ESTABLISHED (соединение установлено).

Состояния и таймеры

При получении определенных пакетов и возникновении событий в сокете происходит переключение состояний сеансов TCP. Вот эти состояния: LISTEN, SYN-SENT,

SYN-RECEIVED, ESTABLISHED, FIN-WAIT-1, FIN-WAIT-2, CLOSE-WAIT, CLOSING, LAST-ACK, TIME-WAIT и CLOSED [Postal 80]. При анализе производительности наибольший интерес вызывают сеансы в состоянии ESTABLISHED, которые представляют активные соединения. Такие сеансы могут активно передавать данные или находиться в режиме ожидания следующего события: передачи данных или события закрытия.

Полностью закрытый сеанс переходит в состояние TIME_WAIT¹, чтобы вновь поступившие пакеты не были по ошибке ассоциированы с новым подключением к тому же порту. Это может привести к проблеме с производительностью из-за нехватки портов, как описано в разделе 10.5.7 «Анализ TCP».

Продолжительность некоторых состояний ограничивается по времени. Состояние TIME_WAIT продолжается две минуты (ядро Windows, например, позволяет настраивать это время). Также в сеансе с состоянием ESTABLISHED может быть установлен таймер «поддержания в активном состоянии» с большим интервалом срабатывания (например, два часа), при срабатывании которого посылаются специальные пакеты для проверки соединения с удаленным хостом.

Обнаружение повторного подтверждения

Обнаружение повторного подтверждения (ACK) используется алгоритмами быстрой повторной передачи и быстрого восстановления, чтобы оперативно обнаружить потерю отправленного пакета (или подтвердить его получение). Вот как работает этот механизм:

1. Отправитель отправляет пакет с порядковым номером 10.
2. Получатель отвечает пакетом подтверждения ACK для порядкового номера 11.
3. Отправитель отправляет пакеты с порядковыми номерами 11, 12 и 13.
4. Пакет 11 теряется.
5. Получатель отвечает на пакеты 12 и 13 пакетом подтверждения ACK для порядкового номера 11, прибытия которого он все еще ожидает.
6. Отправитель получает повторные подтверждения для пакета 11.

Обнаружение повторного подтверждения также используется различными алгоритмами предотвращения перегрузки.

Повторная передача

Для обнаружения повторного подтверждения и повторной передачи потерянных пакетов обычно используются два механизма TCP:

- **Повторная передача по таймеру:** происходит, когда истекло время ожидания, а подтверждение получения пакета так и не было получено. Это время — тайм-аут

¹ Хотя он часто записывается (и программируется) как TIME_WAIT, в RFC 793 используется название TIME-WAIT.

повторной передачи TCP — рассчитывается динамически на основе времени приема/передачи (RTT) для соединения. В Linux оно составляет не менее 200 мс (TCP_RTO_MIN) для первой повторной передачи¹ и увеличивается в два раза для каждой следующей повторной передачи.

- **Быстрая повторная передача:** при получении повторного подтверждения TCP может предположить, что пакет был потерян, и немедленно передать его повторно.

Для дальнейшего повышения производительности были разработаны дополнительные механизмы, позволяющие избежать повторной передачи по таймеру. Одна из проблем связана с потерей последнего отправленного пакета, когда нет последующих пакетов, чтобы по ним определить повторное подтверждение. (Представьте, что в предыдущем примере потерялся пакет 13.) Она решается с помощью алгоритма проверки потери хвоста (Tail Loss Probe, TLP), который отправляет дополнительный (проверочный) пакет через короткий интервал времени после передачи последнего пакета с данными, чтобы помочь обнаружить потерю последнего пакета [Dukkipati 13].

Алгоритмы управления перегрузкой тоже могут ограничивать пропускную способность при повторных передачах.

Управление перегрузкой

Алгоритмы управления перегрузкой были разработаны для поддержания производительности в перегруженных сетях. Некоторые ОС (включая Linux) позволяют выбирать алгоритм управления перегрузкой как часть настройки системы. В число этих алгоритмов входят:

- **Reno:** триггер тройного повторения подтверждения: уменьшение вдвое окна перегрузки, уменьшение вдвое порога медленного старта, быстрая повторная передача и быстрое восстановление.
- **Tahoe:** триггер тройного повторения подтверждения: быстрая повторная передача, уменьшение вдвое порога медленного старта, выбор окна перегрузки с размером, равным одному максимальному размеру сегмента (maximum segment size, MSS), и переход в состояние медленного старта. (Первоначально был разработан для 4.3BSD вместе с Reno и Tahoe.)
- **CUBIC:** использует кубическую функцию (что обусловило такое название) для масштабирования окна и функцию «гибридного старта» для выхода из медленного старта. CUBIC действует более агрессивно, чем Reno, и используется в Linux по умолчанию.
- **BBR:** вместо использования окон BBR строит явную модель характеристик сетевого маршрута (RTT и ширины полосы пропускания), используя фазы

¹ Похоже, что это нарушает RFC 6298, который утверждает, что минимальное время ожидания не может быть меньше одной секунды [Paxson 11].

зондирования. BBR может обеспечить значительно более высокую производительность на одних сетевых маршрутах и снизить производительность на других. В настоящее время разрабатывается версия BBRv2, в которой обещано исправить некоторые недостатки первой версии.

- **DCTCP**: DataCenter TCP использует коммутаторы, настроенные на отправку флага в поле явного уведомления о перегрузке (explicit congestion notification, ECN) при очень малой загрузке очереди, чтобы быстро увеличить доступную полосу пропускания (RFC 8257) [Bensley 17]. Это делает DCTCP непригодным для развертывания в интернете, но в правильно настроенной и управляемой среде он может значительно увеличить производительность.

В числе других алгоритмов, не упоминавшихся выше, можно назвать: Vegas, New Reno и Hybla.

Алгоритм управления перегрузкой может оказывать большое влияние на производительность сети. Облачные сервисы в Netflix, например, используют BBR. Опытным путем было подтверждено, что этот алгоритм может в три раза увеличить пропускную способность в сети, отличающейся большой долей потерянных пакетов [Ather 17]. Понимать, как эти алгоритмы действуют в разных условиях, очень важно для анализа производительности TCP.

В версии ядра Linux 5.6, выпущенной в 2020 году, появилась поддержка разработки новых алгоритмов управления перегрузкой в BPF [Corbet 20]. Она позволяет конечному пользователю задавать свои алгоритмы и загружать их по требованию.

Алгоритм Нейгла

Алгоритм Нейгла (RFC 896) [Nagle 84] уменьшает количество небольших пакетов в сети, задерживая их передачу, чтобы накопить больше отправляемых данных и объединить их. Пакеты задерживаются только в том случае, если в конвейере имеются данные и задержки уже имели место.

Система может предоставлять свой настраиваемый параметр или параметр сокета для отключения алгоритма Нейгла, что может потребоваться при появлении конфликтов с отложенными подтверждениями (см. раздел 10.8.2 «Параметры сокетов»).

Отложенные подтверждения

Этот алгоритм (RFC 1122) [Braden 89] задерживает отправку ACK на срок до 500 мс, чтобы при возможности объединить несколько пакетов ACK. Другие управляющие сообщения TCP тоже можно объединять, чтобы уменьшить количество пакетов, передаваемых в сеть.

Так же как в случае с алгоритмом Нейгла, система может предоставлять свой настраиваемый параметр для отключения этого поведения.

SACK, FACK и RACK

Алгоритм выборочного подтверждения для протокола TCP (selective acknowledgment, SACK) позволяет получателю информировать отправителя о получении блока данных не по порядку. Без этого отбрасывание пакетов привело бы к повторной передаче всего окна отправки, как того требует схема последовательного подтверждения, что отрицательно сказывается на производительности TCP. Большинство современных систем решают эту проблему, поддерживая алгоритм SACK.

Впоследствии алгоритм SACK был дополнен алгоритмом прямых подтверждений (forward acknowledgment, FACK), который поддерживается в Linux по умолчанию. Алгоритм FACK определяет дополнительное состояние и лучше регулирует объем ожидающих обработки данных в сети, улучшая общую производительность [Mathis 96].

Оба алгоритма, SACK и FACK, используются для ускорения восстановления после потери пакетов. Более новый алгоритм недавних подтверждений (Recent ACKnowledgment, RACK, теперь называется RACK-TLP с включением TLP) использует не только порядковые номера, но также отметки времени из пакетов ACK, чтобы обеспечить надежное обнаружение потерь и быстрое восстановление [Cheng 20]. В Netflix был разработан новый реорганизованный стек TCP для FreeBSD под названием RACK, основанный на использовании алгоритмов RACK, TLP и др. [Stewart 18].

Начальное окно

Начальное окно (initial window, IW) — это количество пакетов, которое отправитель может передать после создания соединения до получения первого подтверждения от отправителя. Для коротких потоков, какими являются типичные HTTP-соединения, используется достаточно большое начальное окно, чтобы вместить передаваемые данные, и это может значительно сократить время выполнения передачи и повысить производительность. Но большое начальное окно для больших информационных потоков может привести к перегрузке и отбрасыванию пакетов. Проблема особенно усугубляется, когда одновременно запускается несколько потоков.

Значение по умолчанию в Linux (10 пакетов, или IW10) может оказаться слишком большим для медленных каналов или при создании большого количества соединений. Другие ОС по умолчанию используют размер начального окна, равный двум или четырем пакетам (IW2 или IW4).

UDP

Протокол пользовательских дейтаграмм (User Datagram Protocol, UDP) — это широко используемый интернет-стандарт для отправки сообщений, называемых *дейтаграммами* (RFC 768) [Postel 80]. В отношении производительности протокол UDP обеспечивает:

- **простоту:** простые и короткие заголовки протокола сокращают overhead на вычисления и хранение;
- **отсутствие состояния:** уменьшает overhead на подключение и передачу;
- **отсутствие поддержки повторной передачи:** поддержка этой возможности в TCP значительно увеличивает задержки.

Простой и высокопроизводительный протокол UDP не имеет средств обеспечения надежности, и данные, посылаемые по этому протоколу, могут теряться или приходить не по порядку. Это делает UDP непригодным для использования во многих типах взаимодействий. Кроме того, UDP не имеет средств предотвращения перегрузки и поэтому может вызывать перегрузку сети.

Некоторые сервисы, включая отдельные версии сетевой файловой системы (network file system, NFS), можно настроить для работы через TCP или UDP. Другие сервисы, выполняющие широковещательные или многоадресные рассылки, могут использовать только UDP.

Основное применение UDP — использование в сервисах DNS. Благодаря простоте, отсутствию средств управления перегрузкой и поддержки в интернете (обычно он не фильтруется брандмауэрами), появилось множество новых протоколов, основанных на UDP, которые реализуют собственное управление перегрузкой и другие функции. Например QUIC.

QUIC и HTTP/3

QUIC — это сетевой протокол, разработанный в Google Джимом Роскиндом (Jim Roskind), как более производительная альтернатива протоколу TCP. Он характеризуется меньшими задержками и оптимизирован для HTTP и TLS [Roskind 12]. QUIC построен на основе UDP и предоставляет несколько дополнительных функций, в том числе:

- возможность мультиплексирования нескольких потоков через одно и то же «соединение»;
- поддержка надежности передачи упорядоченных транспортных потоков по аналогии с TCP, которую можно отключать для отдельных подпотоков;
- возобновление подключения с использованием криптографической аутентификации идентификаторов подключения при изменении клиентом своего сетевого адреса;
- полное шифрование данных полезной нагрузки, включая заголовки QUIC;
- согласование соединения в режиме 0-RTT, включая криптографию (для узлов, которые ранее уже устанавливали соединение).

QUIC активно используется браузером Chrome.

Первоначально QUIC был разработан в Google, но Рабочая группа по проектированию интернета (Internet Engineering Task Force, IETF) стандартизировала

транспорт QUIC и конкретную конфигурацию использования HTTP поверх QUIC (последняя комбинация называется HTTP/3).

10.4.2. Оборудование

К сетевому оборудованию относятся интерфейсы, контроллеры, коммутаторы, маршрутизаторы и брандмауэры. Вам безусловно пригодится знание и понимание особенностей их работы, даже если ими управляете не вы, а другой персонал (администраторы сети).

Интерфейсы

Физические сетевые интерфейсы отправляют в сеть и принимают из сети сообщения, называемые *фреймами*. Они управляют передачей электрических, оптических или радиосигналов, а также обрабатывают ошибки передачи.

Типы интерфейсов основаны на стандартах уровня 2, каждый из которых обеспечивает максимальную ширину полосы пропускания. Интерфейсы с высокой пропускной способностью обеспечивают меньшую задержку передачи данных за счет более высоких затрат. При проектировании новых серверов ключевым решением зачастую становится баланс цены сервера и желаемой производительности сети.

Для Ethernet можно выбрать медный или оптический кабель с максимальной скоростью 1 Гбит/с (1 GbE), 10 GbE, 40 GbE, 100 GbE, 200 GbE и 400 GbE. Контроллеры интерфейса Ethernet производят многие вендоры, но ваша ОС может иметь драйверы лишь для некоторых из них.

Потребление интерфейса можно оценить как отношение текущей пропускной способности к текущей согласованной ширине полосы пропускания. Большинство интерфейсов имеют отдельные каналы для передачи и приема, и при работе в полнодуплексном режиме потребление каждого канала необходимо оценивать отдельно.

Проблемы с производительностью в беспроводных интерфейсах могут возникать из-за низкого уровня сигнала и помех¹.

Контроллеры

Физические сетевые интерфейсы доступны системе через контроллеры, конструктивно реализованные в виде набора микросхем на системной плате или отдельной карты расширения.

¹ Я разработал софт для BPF, который преобразует уровень сигнала Wi-Fi в слышимый звук, и презентовал его в докладе на AWS re:Invent 2019 [Gregg 19b]. Я бы включил его в эту главу, но к сожалению, мне пока не довелось опробовать его в промышленной или облачной среде — они еще остаются проводными.

Контроллеры управляются микропроцессорами и подключаются к системе через транспорт (шину) ввода/вывода (например, PCI). Любой из этих компонентов может ограничивать пропускную способность сети или IOPS.

Например, двояная сетевая карта 10 GbE подключается к четырехканальному слоту PCI Express (PCIe) Gen 2. Она имеет максимальную ширину полосы пропускания для канала передачи или приема $2 \times 10 \text{ GbE} = 20 \text{ Гбит/с}$ и общую ширину полосы 40 Гбит/с . Слот имеет максимальную ширину полосы $4 \times 4 \text{ Гбит/с} = 16 \text{ Гбит/с}$. Следовательно, пропускная способность сети на обоих портах будет ограничена пропускной способностью PCIe Gen 2, что не позволит достичь максимальной пропускной способности линии связи для них обоих (знаю это из практики).

Коммутаторы и маршрутизаторы

Коммутаторы обеспечивают выделенный канал связи между любыми двумя подключенными хостами, позволяя без помех осуществлять множественную передачу между парами хостов. Эта технология заменила концентраторы (а до этого — общие физические шины, в роли которых обычно выступал коаксиальный кабель Ethernet), рассылающие все пакеты всем хостам. Такое использование приводило к конфликтам, когда узлы передавали данные одновременно, что идентифицировались интерфейсами как *коллизия* с помощью алгоритма «множественного доступа с контролем несущей и обнаружением коллизий» (carrier sense multiple access with collision detection, CSMA/CD). При каждом обнаружении коллизии этот алгоритм увеличивал задержку по экспоненте и предпринимал повторную попытку передачи, пока не достигал успеха, что создавало проблемы с производительностью под нагрузкой. С появлением коммутаторов эта проблема осталась в прошлом, но некоторые инструменты наблюдения все еще подсчитывают коллизии, хотя обычно они возникают только из-за ошибок (недостаточно точное согласование или плохая проводка).

Маршрутизаторы отвечают за доставку пакетов между сетями и используют сетевые протоколы и таблицы маршрутизации для определения наиболее эффективных маршрутов. На пути между двумя городами пакет может пересечь десяток или даже больше маршрутизаторов, а также другое сетевое оборудование. Маршрутизаторы и маршруты обычно настраиваются на динамическое обновление, чтобы сеть могла автоматически реагировать на сбои, а маршрутизаторы — балансировать нагрузку. Это означает, что никогда нельзя точно предсказать, по какому маршруту движется пакет. При наличии нескольких альтернативных маршрутов есть вероятность доставки пакетов не по порядку, что может вызвать проблемы с производительностью TCP.

На эту неопределенность часто возлагают вину за низкую производительность, например, потому что интенсивный сетевой трафик от других узлов насыщает маршрутизатор, находящийся между отправителем и получателем. Поэтому группам администрирования сетей часто приходится реабилитировать свою инфраструктуру. Для этого они могут использовать передовые инструменты мониторинга в реальном времени и проверять все маршрутизаторы и другие сетевые компоненты.

И маршрутизаторы, и коммутаторы имеют буферы и микропроцессоры, которые сами могут стать узким местом под нагрузкой. Вот пример из моей практики: однажды я обнаружил, что довольно старый коммутатор 10 GbE мог суммарно передавать через все порты не более 11 Гбит/с из-за ограниченной производительности процессора.

Обратите внимание, что коммутаторы и маршрутизаторы также часто находятся в точках, где происходит *изменение скорости* (переход с одной полосы пропускания на другую, например переход с канала 10 Гбит/с на канал 1 Гбит/с). В таких случаях необходима некоторая буферизация, чтобы избежать чрезмерной потери пакетов, но многие коммутаторы и маршрутизаторы имеют избыточную буферизацию (см. описание проблемы разбухания буферов в разделе 10.3.6 «Буферизация»), которая приводит к большим задержкам. Устранить эту проблему можно было бы с использованием улучшенных алгоритмов управления очередями, но не все производители сетевых устройств их поддерживают. Ограничение по источнику тоже могло бы смягчить проблемы со сменой скорости, сделав трафик менее скачкообразным.

Брандмауэры

Брандмауэры часто используются для ограничения доступа на основе настроенного набора правил, что повышает безопасность сети. Брандмауэром может быть отдельное физическое сетевое устройство или программное обеспечение ядра.

Брандмауэры тоже могут оказаться узким местом для производительности, особенно если настроены на трассировку состояния. Правила с трассировкой состояния хранят метаданные для каждого обнаруженного соединения, из-за чего брандмауэр может оказывать чрезмерную нагрузку на память при обработке большого количества соединений. Перегрузка может быть обусловлена атаками типа «отказ в обслуживании» (denial of service, DoS), цель которых — затопить цель соединениями, или высокой частотой попыток установить исходящие соединения, потому что для них тоже может потребоваться осуществлять трассировку состояния.

Поскольку брандмауэры представляют собой специализированное оборудование или ПО, то круг инструментов для их анализа зависит от конкретной реализации. См. соответствующую документацию.

Использование фреймворка BPF для реализации брандмауэров на стандартном оборудовании ширится благодаря его производительности, программируемости, простоте использования и конечной стоимости. Среди компаний, использующих брандмауэры BPF и решения DDoS, можно назвать Facebook [Deepak 18], Cloudflare [Majkowski 18] и Cilium [Cilium 20a].

Брандмауэры могут мешать тестированию производительности: эксперименты с полосой пропускания при отладке проблем могут предусматривать изменение правил брандмауэра, чтобы разрешить соединение (а для этого приходится согласовывать эксперимент с группой поддержки безопасности).

Другое оборудование

Ваша среда может включать другие физические сетевые устройства, например концентраторы, мосты, повторители и модемы. Любое из них может стать узким местом и причиной потери пакетов.

10.4.3. Программное обеспечение

В состав ПО поддержки сетей входят: сетевой стек, реализация TCP и драйверы устройств. В этом разделе рассмотрим только темы, связанные с производительностью.

Сетевой стек

Используемые компоненты и уровни зависят от типа и версии ОС, поддерживаемых протоколов и интерфейсов. На рис. 10.7 изображена общая структура программных компонентов.

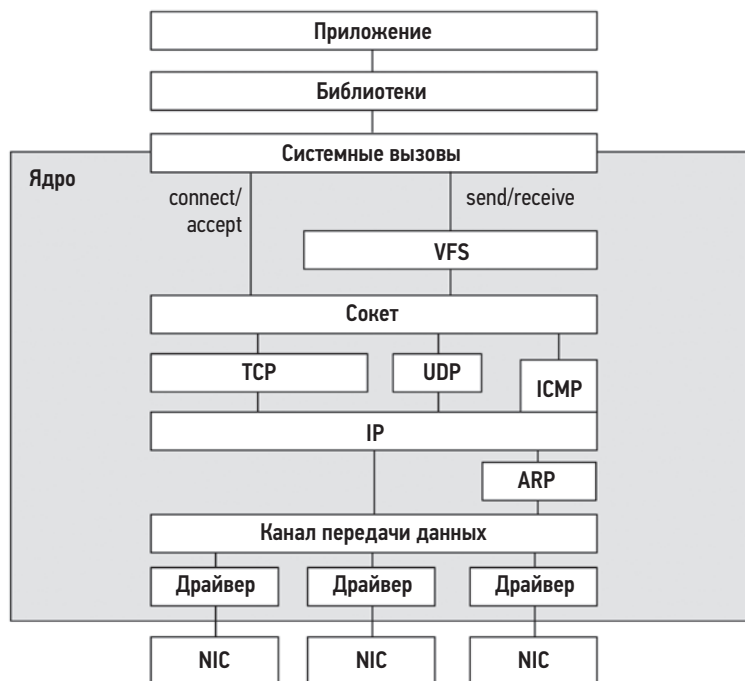


Рис. 10.7. Общая структура сетевого стека

В современных ядрах сетевой стек является многопоточным, поэтому входящие пакеты могут обрабатываться несколькими процессорами.

Linux

На рис. 10.8 изображен сетевой стек Linux, в том числе местоположение буферов передачи/приема в сокетах и очередей пакетов.

В системах Linux сетевой стек является основным компонентом ядра, а драйверы устройств — дополнительными модулями. Пакеты проходят через эти компоненты ядра в виде структуры `struct sk_buff` (буфер сокета). Обратите внимание, что на уровне IP (здесь не показан) тоже может быть очередь для повторной сборки пакетов.

В разделах ниже обсуждаются детали реализации сетевого стека в Linux, связанные с производительностью: очереди TCP-соединений, буферизация TCP, дисциплины очередей (`qdisc`), драйверы сетевых устройств, масштабирование процессора и выполнение операций в обход ядра. Протокол TCP описан в предыдущем разделе.

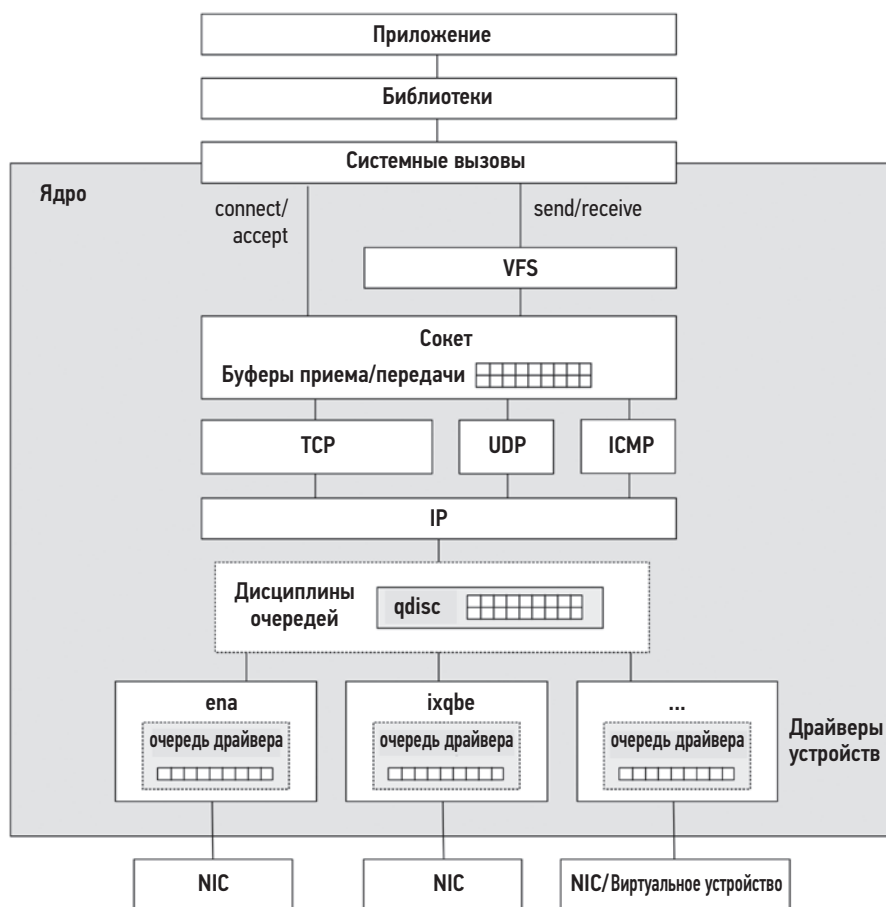


Рис. 10.8. Сетевой стек в Linux

Очереди соединений TCP

Входящие запросы на подключение обрабатываются с использованием очередей. Есть две такие очереди: одна для еще не установленных соединений (или *очередь пакетов SYN*) и одна для установленных соединений, ожидающих, пока они будут приняты приложением (или *очередь установленных соединений*). Они изображены на рис. 10.9.

В более ранних ядрах использовалась только одна очередь, и она была уязвима для атак типа SYN-flood. SYN-flood — это разновидность атаки DoS, суть которой заключается в отправке большого количества пакетов SYN в прослушивающий TCP-порт с поддельных IP-адресов, чтобы заполнить очередь до отказа, пока реализация TCP ждет завершения процедуры «рукопожатия», и воспрепятствовать подключению реальных клиентов.

При наличии двух очередей первая может выступать в качестве промежуточной области для хранения потенциально фиктивных соединений, которые перемещаются во вторую очередь только после подтверждения. Первую очередь можно сделать достаточно длинной, чтобы исключить возможность переполнения, и оптимизировать для хранения только минимального количества необходимых метаданных.

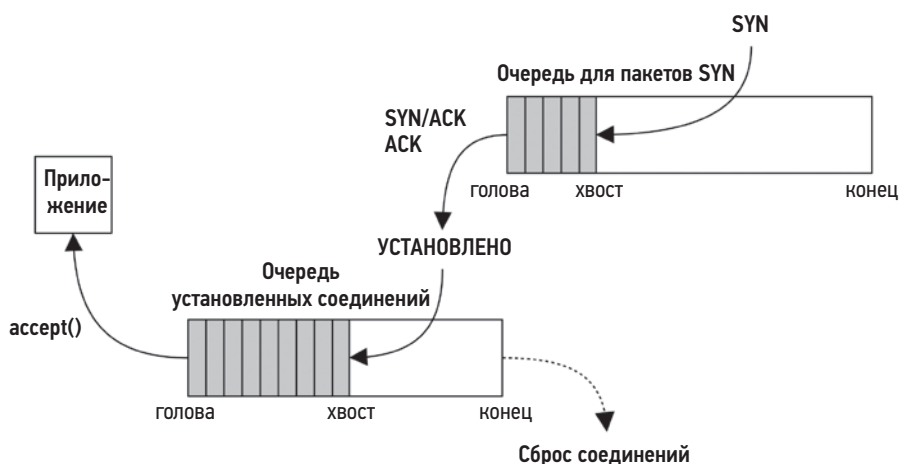


Рис. 10.9. Очереди соединений TCP

Соединения в ответ на пакеты SYN с криптографическими ключами устанавливаются в обход первой очереди, потому что ключи показывают, что клиент уже авторизован.

Длину этих очередей можно настраивать независимо (см. раздел 10.8 «Настройка»). Кроме того, размер второй очереди может устанавливаться приложением с помощью аргумента `backlog` системного вызова `listen(2)`.

Буферизация TCP

Пропускная способность TCP повышается за счет использования буферов передачи и приема в сокетах. Они показаны на рис. 10.10.

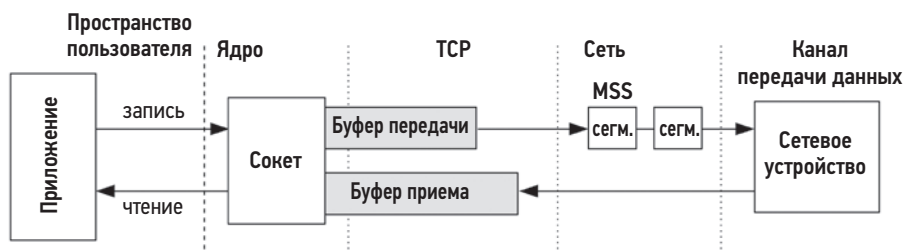


Рис. 10.10. Буферы передачи и приема TCP

Размеры буферов передачи и приема можно настраивать. Большие размеры увеличивают пропускную способность за счет увеличения потребления основной памяти каждым соединением. Буферы могут быть разного размера, если предполагается, что сервер будет больше отправлять или принимать. Кроме того, ядро Linux может динамически изменять размеры буферов в зависимости от активности соединения и позволяет настраивать их минимальный, типичный и максимальный размеры.

Уменьшение затрат на сегментацию: GSO и TSO

Сетевые устройства и сети принимают пакеты, размер которых не превышает максимального размера сегмента (MSS), который может составлять всего 1500 байт. Чтобы избежать оверхеда на отправку большого числа маленьких пакетов, Linux использует универсальный механизм уменьшения затрат на сегментацию (generic segmentation offload, GSO) для отправки пакетов размером до 64 Кбайт («суперпакетов»), которые разбиваются на сегменты размером MSS непосредственно перед передачей в сетевое устройство. Если сетевая карта и драйвер поддерживают уменьшение затрат на сегментацию TCP (TCP Segmentation Offload, TSO), то собственно сегментирование делегируется устройству, что дополнительно увеличивает пропускную способность сетевого стека¹. Есть и дополнение к GSO — универсальный механизм уменьшения затрат на прием (generic receive offload, GRO) [Linux 20i]². Оба механизма — GRO и GSO — реализованы в ядре, а TSO реализуется аппаратным обеспечением сетевых карт.

¹ Некоторые сетевые карты предоставляют механизм уменьшения затрат на TCP (TCP offload engine, TOE), позволяя делегировать ему часть или все функции протокола TCP/IP. Ядро Linux не поддерживает TOE по ряду причин, включая безопасность, сложность и даже производительность [Linux 16].

² Поддержка GSO и GRO для UDP была добавлена в Linux в 2018 году, в основном для использования с QUIC [Bruijn 18].

Дисциплины очередей

Этот необязательный уровень управляет классификацией трафика (traffic classification, tc), планированием, обработкой, фильтрацией и формированием сетевых пакетов. В Linux реализовано несколько алгоритмов, определяющих дисциплины очередей (qdisc), которые можно настраивать с помощью команды `tc(8)`. У каждого алгоритма есть своя страница в справочном руководстве `man(1)`:

```
# man -k tc-
actions (8)          - действия классификации трафика, определяемые независимо
tc-basic (8)         - простой фильтр управления трафиком
tc-bfifo (8)         - очередь пакетов "первым пришел, первым вышел"
tc-bpf (8)           - BPF-программируемые классификаторы и действия для добавления
в очередь и извлечения из нее
tc-cbq (8)           - организация очередей на основе классов
tc-cbq-details (8)   - организация очередей на основе классов
tc-cbs (8)           - дисциплина формирования очередей на основе разрешений
tc-cgroup (8)        - фильтр трафика на основе групп управления
tc-choke (8)         - планировщик выбора и сохранения
tc-codel (8)         - алгоритм активного управления очередью с контролируемой
задержкой
tc-connmark (8)      - действие для получения маркера соединения (CONNMARK),
установленного брандмауэром netfilter
tc-csum (8)          - действие по обновлению контрольной суммы
tc-drr (8)           - планировщик на основе циклического алгоритма с дефицитом
времени
tc-ematch (8)        - дополнительные средства сопоставления для использования
с фильтрами "basic" или "flow"
tc-flow (8)          - фильтр управления трафиком на основе потоков
tc-flower (8)        - фильтр управления трафиком на основе потоков
tc-fq (8)            - политика организации справедливых очередей
tc-fq_codel (8)      - комбинированная политика организации справедливых очередей
с контролируемой задержкой
[...]
```

Ядро Linux по умолчанию использует дисциплину очереди `pfifo_fast`, тогда как подсистема управления службами `systemd` менее консервативна и использует дисциплину `fq_codel`, чтобы уменьшить вероятность разбухания буферов за счет небольшой дополнительной сложности на уровне дисциплины.

BPF позволяет расширить возможности этого уровня с помощью программ типа `BPF_PROG_TYPE_SCHED_CLS` и `BPF_PROG_TYPE_SCHED_ACT`. Их можно привязать к точкам входа и выхода в ядре для фильтрации, изменения и пересылки пакетов, и эта возможность широко используется балансировщиками нагрузки и брандмауэрами.

Драйверы сетевых устройств

Драйвер сетевого устройства обычно имеет дополнительный кольцевой буфер между памятью ядра и сетевой картой для отправляемых и принимаемых пакетов. Он изображен на рис. 10.8 как очередь драйвера.

Одна из особенностей, повышающая производительность и получившая широкое распространение в высокоскоростных сетях, — использование *режима объединения прерываний*. При ее применении прерывания генерируются не для каждого полученного пакета, а при истечении определенного интервала времени (времени опроса) или получении определенного количества пакетов. Это уменьшает overhead на обмен данными между ядром и сетевой картой, позволяет буферизовать отправляемые данные и тем самым увеличить пропускную способность, хотя и за счет некоторой задержки.

Ядро Linux использует новую инфраструктуру API (New API, NAPI), в которой применяется прием согласования прерываний: когда пакеты передаются с невысокой частотой, используются прерывания (обработка происходит через программные прерывания). Когда пакеты передаются с высокой частотой, прерывания отключаются и используется метод опроса для поддержки объединения [Corbet 03], [Corbet 06b]. Этот прием обеспечивает низкую задержку или высокую пропускную способность, в зависимости от характера рабочей нагрузки. Вот некоторые другие особенности NAPI:

- Регулирование частоты следования пакетов: позволяет отбрасывать пакеты в сетевом адаптере, чтобы предотвратить перегрузку системы из-за наплыва большого количества пакетов.
- Планирование интерфейсов: ограничивает обработку буферов в цикле опроса на основе квот, чтобы обеспечить справедливое обслуживание высоконагруженных сетевых интерфейсов.
- Поддержка параметра `SO_BUSY_POLL` сокета: пользовательские приложения могут уменьшить задержку приема пакетов из сети, запрашивая *активное ожидание* (busy wait, когда поток продолжает выполняться в цикле ожидания на процессоре до тех пор, пока не произойдет событие) на сокет [Dumazet 17a].

Поддержка объединения прерываний особенно важна для повышения производительности сетевых взаимодействий между виртуальными машинами и реализуется сетевым драйвером `ena`, используемым в AWS EC2.

Отправка и получение по сетевой карте

Чтобы отправить пакеты, ядро уведомляет сетевую карту, и та извлекает пакеты (фреймы) из памяти ядра с использованием прямого доступа к памяти (direct memory access, DMA) для повышения эффективности. Сетевые карты предоставляют дескрипторы передачи для управления пакетами DMA. Если у сетевой карты нет свободных дескрипторов, то сетевой стек приостанавливает передачу, чтобы позволить сетевой карте отправить накопившиеся пакеты¹.

¹ Обычно для предотвращения исчерпания дескрипторов передачи используется алгоритм Byte Queue Limits (BQL), который описывается в подразделе «Другие оптимизации».

После получения пакетов сетевые карты могут использовать DMA, чтобы сохранить их в кольцевом буфере ядра, а затем уведомляют ядро с помощью прерывания (которое может игнорироваться при использовании приема объединения прерываний). Прерывание запускает обработчик, который передает пакеты в сетевой стек для дальнейшей обработки.

Масштабирование процессора

Если для обработки пакетов и выполнения стека TCP/IP использовать несколько процессоров, то можно достичь высокой скорости передачи пакетов. В Linux поддерживается несколько способов многопроцессорной обработки пакетов (см. `Documentation/network/scaling.txt`):

- **RSS (Receive Side Scaling — масштабирование на принимающей стороне):** используется с современными сетевыми картами, которые поддерживают несколько очередей и могут хешировать пакеты в разные очереди, которые, в свою очередь, обрабатываются разными процессорами, прерываемыми напрямую. Хеш может быть основан на IP-адресе и номерах портов TCP, поэтому пакеты из одного и того же соединения в итоге обрабатываются одним и тем же процессором¹.
- **RPS (Receive Packet Steering — управление принимаемыми пакетами):** программная реализация RSS для сетевых карт, которые поддерживают только одну очередь. Включает короткую процедуру обслуживания прерываний, которая выбирает процессор для обработки входящего пакета. Для выбора может использоваться хеш на основе полей из заголовков пакетов.
- **RFS (Receive Flow Steering — управление принимаемыми потоками):** реализуется подобно RPS, но опирается на привязку сокета к процессору, на котором он обрабатывался в последний раз, чтобы увеличить частоту попаданий в кэш ЦП и улучшить локальность памяти.
- **Accelerated Receive Flow Steering (ускоренное управление принимаемыми потоками):** аппаратная реализация RFS в сетевых картах, которые поддерживают эту функцию. Предусматривает передачу в сетевую карту информации о потоке, чтобы та могла определить, какой процессор прервать.
- **XPS (Transmit Packet Steering — управление передаваемыми пакетами):** для сетевых адаптеров с несколькими очередями передачи. Поддерживает передачу пакетов в разные очереди несколькими процессорами.

Если не используется никакая стратегия балансировки нагрузки на процессоры, сетевая карта может прерывать только один процессор, из-за чего уровень его потребления может достичь 100 % и процессор превратится в узкое место. Этот эффект

¹ Netflix FreeBSD CDN использует RSS в помощь механизму TCP уменьшения затрат на прием больших объемов данных (Large Receive Offload, LRO), позволяя объединять пакеты для одного и того же соединения, даже когда они разделены другими пакетами [Gallatin 17].

может выражаться в большом времени, затрачиваемом одним из процессоров на обработку программных прерываний (которое в Linux можно наблюдать, например, с помощью `mpstat(1)`: см. главу 6 «Процессоры», раздел 6.6.3 «`mpstat`»). Такая проблема особенно характерна для балансировщиков нагрузки или прокси-серверов (например, `nginx`), потому что их часто используют там, где есть высокая частота следования входящих пакетов.

Возможность выбора процессора для обработки прерывания на основе таких факторов, как когерентность кэша, как это сделано в RFS, может заметно улучшить производительность сети. То же самое можно реализовать с помощью процесса `irqbalance`, который связывает запросы на обработку прерываний (IRQ) с процессорами.

В обход ядра

На рис. 10.8 показан наиболее типичный путь следования пакетов через стек TCP/IP. Но приложения могут работать в обход сетевого стека ядра, применяя такие технологии, как Data Plane Development Kit (DPDK), чтобы достичь более высокой скорости передачи пакетов и производительности. К таким приложениям относятся приложения, реализующие свои сетевые протоколы в пользовательском пространстве и выполняющие запись в сетевой драйвер через библиотеку DPDK и драйвер ввода/вывода пользовательского пространства (user space I/O, UIO) или виртуальные функции ввода/вывода (virtual function I/O, VFIO). Расходов на копирование пакетов можно избежать за счет прямого доступа к памяти сетевой карты.

Технология eXpress Data Path (XDP) предлагает другой путь передачи сетевых пакетов: быстрый и программируемый, который использует расширенный BPF и интегрируется в существующий стек ядра [Høiland-Jørgensen 18]. (Сейчас DPDK поддерживает XDP для приема пакетов, делегируя выполнение некоторых функций ядру [DPDK 20].)

В случае выполнения операций в обход сетевого стека ядра анализ их производительности с применением традиционных инструментов и метрик невозможен, потому что счетчики и события трассировки, которые они используют, тоже не используются. Это затрудняет анализ производительности.

Помимо выполнения операций в обход всего стека, есть и возможности, позволяющие избежать затрат на копирование данных: флаг `MSG_ZEROCOPY` для системного вызова `send(2)` и получение данных через `mmap(2)` [Linux 20c], [Corbet 18b].

Другие оптимизации

В сетевом стеке Linux используются и другие алгоритмы для увеличения производительности. Они показаны на рис. 10.11, где изображен путь, преодолеваемый пакетами TCP при отправке (многие из них вызываются из функции ядра `tcp_write_xmit()`).

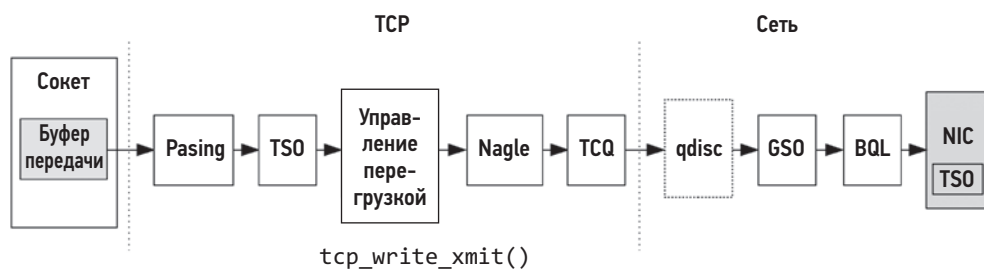


Рис. 10.11. Путь отправки пакетов TCP

Некоторые из этих механизмов и алгоритмов были описаны выше: буферы передачи в сокетах, TSO¹, управление перегрузкой, алгоритм Нейгла (Nagle) и дисциплины очередей (qdisc). Другие включают:

- **Pacing**: управляет отправкой пакетов, распределяя их так, чтобы избежать всплесков, способных снизить производительность (это помогает избежать отправки большого количества маленьких пакетов TCP и задержек при постановке в очередь или даже отбрасывания пакетов сетевыми коммутаторами, а также решить проблему *incast* (большого количества ответов), когда многие конечные точки одновременно отвечают серверу [Fritchie 12]).
- **TCP Small Queues (TSQ)**: управляет (уменьшает) количеством очередей в сетевом стеке, чтобы избежать проблем, в том числе разбухания буферов [Bufferbloat 20].
- **Byte Queue Limits (BQL)**: автоматически устанавливает достаточно большие размеры очередей драйверов, чтобы избежать их исчерпания, но также достаточно маленькие, чтобы уменьшить максимальное время пребывания пакетов в очереди и избежать исчерпания дескрипторов передачи сетевого интерфейса [Hrubý 12]. Приостанавливает добавление пакетов в очередь драйвера, когда это необходимо. Добавлено в Linux 3.3 [Siemon 13].
- **Early Departure Time (EDT)**: использует временное колесо вместо очереди для упорядочивания пакетов, отправляемых в сетевую карту. Каждому пакету, в зависимости от заданной политики и скорости, присваивается временная отметка. Был добавлен в Linux 4.20 и своими возможностями напоминает алгоритмы BQL и TSQ [Jacobson 18].

Эти алгоритмы часто используются в комбинации для повышения производительности. Отправляемый пакет TCP может обрабатываться каким-либо алгоритмом управления перегрузкой, TSO, TSQ, Pacing и дисциплинами очередей, прежде чем попасть в сетевую карту [Cheng 16].

¹ Обратите внимание, что TSO на этой схеме в двух местах: сразу после Pacing, для создания суперпакетов, а затем в сетевой карте для окончательной сегментации.

10.5. МЕТОДОЛОГИЯ

В этом разделе описаны различные методологии и упражнения анализа и настройки сети. Краткий перечень методологий приводится в табл. 10.2.

Таблица 10.2. Методологии анализа и настройки производительности сети

Раздел	Методология	Тип
10.5.1	Метод инструментов	Анализ наблюдения
10.5.2	Метод USE	Анализ наблюдения
10.5.3	Определение характеристик рабочей нагрузки	Анализ наблюдения, планирование мощности
10.5.4	Анализ задержек	Анализ наблюдения
10.5.5	Мониторинг производительности	Анализ наблюдения, планирование мощности
10.5.6	Перехват пакетов	Анализ наблюдения
10.5.7	Анализ TCP	Анализ наблюдения
10.5.8	Статическая настройка производительности	Анализ наблюдения, планирование мощности
10.5.9	Мониторинг производительности	Анализ наблюдения, планирование мощности
10.5.10	Микробенчмаркинг	Анализ наблюдения

Список дополнительных методологий и краткое введение в них ищите в главе 2 «Методологии».

Эти методологии можно применять по отдельности или в сочетании друг с другом. Предлагаю начинать со следующих стратегий в указанном порядке: метод USE, статическая настройка производительности и определение характеристик рабочей нагрузки.

В разделе 10.6 «Инструменты наблюдения» демонстрируются приемы применения этих методологий с использованием инструментов операционной системы.

10.5.1. Метод инструментов

Метод инструментов — это процесс перебора доступных инструментов для изучения ключевых метрик, которые они возвращают. Это очень простая методология, но при ее использовании могут оставаться незамеченными некоторые проблемы, которые плохо обнаруживаются или вообще не обнаруживаются инструментами. Также на ее применение может уйти много времени.

При анализе производительности сети методом инструментов можно выполнить следующие проверки:

- **nstat/netstat -s**: позволяют выявить высокую частоту повторных передач и нарушение порядка следования пакетов. Что означает «высокая» частота повторных передач, зависит от клиентов: система с выходом в интернет и ненадежными удаленными клиентами будет иметь более высокую частоту повторных передач, чем система, обслуживающая клиентов в том же центре обработки данных.
- **ip -s link/netstat -i**: позволяют проверить счетчики ошибок интерфейса, включая «errors» (ошибки), «dropped» (отброшено), «overrun» (переполнение).
- **ss -tiernm**: позволяет проверить наличие флага ограничителя для важных сокетов и узнать, где находится их узкое место, а также другие статистики, показывающие состояние сокета.
- **nicstat/ip -s link**: позволяют проверить скорость передачи и приема в байтах. Высокая пропускная способность может ограничиваться согласованной скоростью передачи данных или механизмами регулирования во внешней сети. Из-за этого также могут возникать задержки и конфликты между сетевыми пользователями при доступе к системе.
- **tcp life**: регистрирует информацию о сеансах TCP со сведениями о процессе, продолжительностью и статистиками пропускной способности.
- **tcp top**: позволяет наблюдать за наиболее активными сеансами TCP в режиме реального времени.
- **tcpdump**: у этого инструмента может быть большой оверхед с точки зрения потребления процессора и памяти. При этом, запуская tcpdump(8) на короткие промежутки времени, можно выявить необычный сетевой трафик или заголовки протокола.
- **perf(1)/BCC/bpftrace**: позволяет выборочно исследовать пакеты между приложением и сетью, в том числе проверять состояние ядра.

При обнаружении проблемы изучите все поля в выводе доступных инструментов. Дополнительную информацию о каждом инструменте ищите в разделе 9.6 «Инструменты наблюдения». Для выявления других типов проблем используйте другие методологии.

10.5.2. Метод USE

Метод USE используют для быстрого выявления узких мест и ошибок во всех компонентах. Для каждого сетевого интерфейса и направления — передачи (TX) и приема (RX) — проверьте:

- **Потребление**: время, в течение которого интерфейс был занят отправкой или получением фреймов.
- **Насыщенность**: размеры дополнительных очередей, буферов или наличие фактов блокировки из-за занятости сетевого интерфейса.

- **Ошибки:** для приема — неверная контрольная сумма, слишком короткий фрейм (меньше заголовка канала данных) или слишком длинный, наличие коллизий (маловероятно для коммутируемых сетей); для передачи — поздние коллизии (плохая проводка).

Сначала проверьте ошибки: для этого не требуется много времени и их легко интерпретировать.

Информация о потреблении обычно не предоставляется ОС или инструментами мониторинга напрямую (за исключением `nicstat(1)`). Но его можно рассчитать как отношение текущей пропускной способности к текущей согласованной скорости для каждого направления (RX, TX). Текущая пропускная способность должна измеряться в байтах в секунду, включая заголовки всех протоколов.

В средах, использующих механизмы ограничения ширины полосы пропускания сети (с помощью средств управления ресурсами), что особенно характерно для некоторых облачных сред, может потребоваться измерять потребление с учетом не только физических пределов, но и наложенных ограничений.

Насыщенность сетевого интерфейса измерить сложно из-за использования приемов буферизации, потому что приложения могут отправлять данные намного быстрее, чем интерфейс. Но можно измерить время блокировки потоков приложения в ожидании отправки пакетов в сеть, которое должно увеличиваться с увеличением насыщенности. Также проверьте, есть ли в ядре другие статистики, более тесно связанные с насыщением интерфейса, например «overrun» (переполнение). Обратите внимание, что Linux использует алгоритм BQL для регулирования размера очереди перед сетевым интерфейсом, что помогает избежать его насыщения.

Обычно в числе статистик доступно количество повторных передач на уровне TCP, которое тоже может служить индикатором насыщения сети. Но это количество характеризует весь маршрут между сервером и клиентом, и увеличение этого числа может быть вызвано проблемами на любом переходе.

Метод USE также применим к сетевым контроллерам и транспорту между ними и процессорами. Поскольку инструментов наблюдения для этих компонентов не так много, иногда проще вывести метрики на основе статистик и топологии сетевого интерфейса. Например, если сетевой контроллер A содержит порты A0 и A1, то пропускную способность сетевого контроллера можно рассчитать как сумму пропускных способностей портов A0 + A1. Затем, зная максимальную пропускную способность, можно рассчитать потребление сетевого контроллера.

10.5.3. Определение характеристик рабочей нагрузки

Определение характеристик приложенной нагрузки — важный шаг при планировании мощностей, сравнительном анализе и моделировании рабочих нагрузок. Это также помогает значительно увеличить производительность за счет выявления и устранения ненужной работы.

Вот основные измеряемые характеристики:

- пропускная способность сетевого интерфейса: количество принимаемых и передаваемых байтов в секунду;
- частота операций ввода/вывода, выполняемых сетевым интерфейсом: количество принимаемых и передаваемых фреймов в секунду;
- частота создания новых соединений ТСП: количество активных и пассивных соединений в секунду.

Термины *активный* и *пассивный* были описаны в подразделе «Трехэтапное “рукопожатие”», в разделе 10.4.1 «Протоколы».

Эти характеристики со временем могут меняться, потому что изменение характера потребления в течение дня — обычное дело. Мониторинг изменения характеристик с течением времени описан в разделе 10.5.5 «Мониторинг производительности».

Пример ниже показывает, как все эти атрибуты можно выразить в одном описании:

Пропускная способность сети зависит от активности пользователей, и выполняется больше операций записи (TX), чем чтения (RX). Пиковая скорость записи составляет 200 Мбайт/с и 210 000 пакетов в секунду, а пиковая скорость чтения — 10 Мбайт/с и 70 000 пакетов в секунду. Частота создания входящих (пассивных) ТСП-соединений достигает 3000 соединений в секунду.

Эти характеристики можно измерить не только в масштабе всей системы, но для каждого интерфейса. Это поможет выявить узкие места интерфейса, если есть возможность наблюдать достижение максимальной пропускной способности линии связи. Если в системе настроены механизмы ограничения ширины полосы пропускания сети (с помощью средств управления ресурсами), они могут ограничивать пропускную способность сети, не давая ей приблизиться к максимальной пропускной способности линии связи.

Чек-лист для определения дополнительных характеристик рабочей нагрузки

При необходимости выясните дополнительные сведения, характеризующие рабочую нагрузку. Они перечислены ниже в форме вопросов, которые также могут служить чек-листом при исследовании проблем с сетью:

- Каков средний размер пакетов? Принимаемых, посылаемых?
- Какие протоколы используются на каждом уровне? Для транспортного уровня: TCP, UDP (возможно QUIC).
- Какие порты TCP/UDP активны? Какова частота передачи данных через порт в байтах в секунду? Как часто устанавливаются соединения с этими портами?
- С какой частотой выполняются широковещательные и многоадресные рассылки?
- Какие процессы активно используют сеть?

Ответы на некоторые вопросы вы найдете в следующих разделах. См. главу 2 «Методологии», где более подробно описывается эта методология и характеристики, которые нужно измерить (кто, почему, зачем, как).

10.5.4. Анализ задержек

Чтобы понять и выразить особенности работы сети, можно измерить разные интервалы времени (задержки). Некоторые из них были представлены в разделе 10.3.5 «Задержка». Более полный список представлен в табл. 10.3. Измерьте как можно больше таких интервалов, чтобы сузить область поиска истинного источника задержек.

Таблица 10.3. Задержки в сети

Задержка	Описание
Задержка разрешения имен	Время, в течение которого имя хоста разрешается в IP-адрес, обычно с помощью DNS. Это довольно часто встречающийся источник проблем с производительностью
Задержка проверки связи	Время от отправки эхо-запроса ICMP до получения ответа. Измеряет время передачи пакетов в сети и их обработки сетевым стеком ядра на обоих хостах
Задержка инициализации подключения TCP	Время от момента отправки пакета SYN до получения пакета SYN,ACK. Поскольку никакие приложения не участвуют в подключении, это время измеряет задержку сети и стека ядра на обоих хостах, аналогично задержке проверки связи, плюс некоторые дополнительные затраты на создание сеанса TCP в ядре. Для уменьшения этой задержки может использоваться расширение TCP Fast Open (TFO)
Задержка первого байта	Также известна как время до первого байта (Time To First Byte, TTFB). Измеряет время от момента установления соединения до получения клиентом первого байта данных. Включает время, необходимое удаленному хосту, чтобы принять соединение, запустить поток, обслуживающий его, выполнить код потока и принять первый байт, вследствие чего эта метрика в большей степени характеризует производительность приложения и текущую нагрузку на него, чем задержку подключения TCP
Повторные передачи TCP	Если имеют место, могут добавлять тысячи миллисекунд задержки
Задержка TCP TIME_WAIT	Время, в течение которого сеансы TCP, закрытые на локальной стороне, продолжают ожидать поступления запоздавших пакетов
Продолжительность жизни соединения/сеанса	Время от инициализации до закрытия сетевого подключения. Некоторые протоколы, например HTTP, могут использовать стратегию поддержки соединения в открытом состоянии (keep-alive), чтобы избежать оверхеда и задержек при повторном подключении
Задержка системных вызовов, выполняющих прием/передачу	Время выполнения системных вызовов чтения/записи в сокет (любые системные вызовы, которые читают/записывают данные в сокеты, включая read(2), write(2), recv(2), send(2) и их варианты)

Таблица 10.3 (окончание)

Задержка	Описание
Задержка системного вызова, устанавливающего соединение	Время создания нового соединения. Обратите внимание, что некоторые приложения выполняют такой вызов в неблокирующем режиме
Время приема/передачи	Время, необходимое для доставки сетевого запроса серверу и получения ответа. Ядро может использовать эту метрику в алгоритме управления перегрузкой
Задержка обработки прерывания	Время от момента, когда контроллер сгенерировал прерывания, получив очередной пакет, до момента, когда пакет будет обработан ядром
Задержка стека	Время, в течение которого пакет перемещался по стеку TCP/IP в ядре

Задержки можно представить в виде:

- **средних значений за интервал:** лучше использовать для оценки производительности взаимодействия клиент/сервер, чтобы изолировать различия пропускных способностей сетей;
- **полных распределений:** в виде гистограмм или тепловых карт;
- **задержек отдельных операций:** с подробной информацией о каждом событии, включая IP-адреса отправителя и получателя.

Типичные источники проблем — это выбросы задержек, вызванные повторными передачами TCP. Их можно идентифицировать с помощью полных распределений или трассировкой задержек для каждой операции, в том числе путем фильтрации по минимальному порогу задержки.

Задержки можно измерить с помощью инструментов трассировки и, для некоторых задержек, параметров сокета. В Linux к таким параметрам относятся: `SO_TIMESTAMP`, который требует фиксировать время получения входящего пакета (и `SO_TIMESTAMPNS` с разрешением времени до наносекунд), и `SO_TIMESTAMPING`, требующий фиксировать время каждого события [Linux 20j]. Параметр `SO_TIMESTAMPING` помогает определять задержки передачи, время приема/передачи и задержки стека и может пригодиться при анализе сложных задержек пакетов, включающих задержки на туннелирование [Hassas Yeganeh 19].

Обратите внимание, что некоторые источники дополнительных задержек действуют непостоянно и происходят только во время загрузки системы. Для получения более реалистичной картины с задержками измерения следует производить не только в бездействующей системе, но и в находящейся под нагрузкой.

10.5.5. Мониторинг производительности

Мониторинг производительности помогает обнаружить проблемы и закономерности поведения, проявляющиеся с течением времени. Такое поведение включает,

в том числе, активность конечных пользователей в течение суток или выполнение операций по расписанию (например, резервное копирование по сети).

Ключевые сетевые метрики для мониторинга:

- **пропускная способность:** сетевого интерфейса в байтах в секунду для приема и передачи, в идеале для каждого интерфейса в отдельности;
- **соединения:** частота создания TCP-соединений в секунду, как еще один показатель сетевой нагрузки;
- **ошибки:** включая счетчики отброшенных пакетов;
- **повторные передачи TCP:** могут пригодиться для увязывания с проблемами в работе сети;
- **неупорядоченные пакеты TCP:** тоже могут вызывать проблемы с производительностью.

В средах, использующих механизмы ограничения ширины полосы пропускания сети (с помощью средств управления ресурсами), а также в некоторых облачных средах также могут собираться статистики, связанные с наложенными ограничениями.

10.5.6. Перехват пакетов

Перехват пакетов предполагает захват пакетов из сети для исследования их заголовков протоколов и данных. Для анализа наблюдения это крайнее средство, потому что перехват может сопровождаться значительным потреблением процессора и памяти. Код сетевого стека в ядре обычно хорошо оптимизирован, так как ему приходится обрабатывать до миллионов пакетов в секунду, и по этой же причине очень чувствителен к любому дополнительному оверхеду. Чтобы уменьшить оверхед, ядро может использовать кольцевые буферы для передачи данных из пакетов в инструмент трассировки на уровне пользователя через совместно используемую область памяти. Например, в BPF с использованием выходного кольцевого буфера в `perf(1)` и с применением `AF_XDP` [Linux 20k]¹. Еще одно решение проблемы оверхеда — использовать для перехвата пакетов отдельный сервер, подключенный к порту «tap» или «mirrortap» коммутатора. Облачные провайдеры Amazon и Google предлагают такую возможность как услугу [Amazon 19], [Google 20b].

Перехват пакетов обычно включает сохранение захваченных пакетов в файл и последующий анализ этого файла различными способами. Один из них — создание журнала, содержащего следующие данные для каждого пакета:

- отметка времени;
- пакет целиком, включая:
 - все заголовки всех протоколов (например, Ethernet, IP, TCP);
 - часть или все данные полезной нагрузки;

¹ Другой вариант — использовать для каждого пакета `PF_RING` вместо `PF_PACKET`, но поддержка `PF_RING` не реализована в ядре Linux [Deri 04].

- метаданные: общее количество пакетов, количество отброшенных пакетов;
- имя интерфейса.

Вот, например, вывод по умолчанию инструмента `tcpdump(8)` в Linux:

```
# tcpdump -ni eth4
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth4, link-type EN10MB (Ethernet), capture size 65535 bytes
01:20:46.769073 IP 10.2.203.2.22 > 10.2.0.2.33771: Flags [P.], seq
4235343542:4235343734, ack 4053030377, win 132, options [nop,nop,TS val 328647671 ecr
2313764364], length 192
01:20:46.769470 IP 10.2.0.2.33771 > 10.2.203.2.22: Flags [.], ack 192, win 501,
options [nop,nop,TS val 2313764392 ecr 328647671], length 0
01:20:46.787673 IP 10.2.203.2.22 > 10.2.0.2.33771: Flags [P.], seq 192:560, ack 1,
win 132, options [nop,nop,TS val 328647672 ecr 2313764392], length 368
01:20:46.788050 IP 10.2.0.2.33771 > 10.2.203.2.22: Flags [.], ack 560, win 501,
options [nop,nop,TS val 2313764394 ecr 328647672], length 0
01:20:46.808491 IP 10.2.203.2.22 > 10.2.0.2.33771: Flags [P.], seq 560:896, ack 1,
win 132, options [nop,nop,TS val 328647674 ecr 2313764394], length 336
[...]
```

Каждая строка в этом выводе содержит информацию об одном пакете, включая IP-адрес, порты TCP и другие сведения из заголовка TCP. Ее можно использовать для отладки многих проблем, включая задержку сообщений и потерю пакетов.

Поскольку перехват пакетов может требовать значительных вычислительных затрат, большинство реализаций позволяют в случае перегрузки отбрасывать события вместо их захвата. Счетчик отброшенных пакетов можно включить в журнал.

Помимо использования кольцевых буферов для уменьшения оверхеда, реализации перехвата пакетов обычно позволяют указать выражение для фильтрации в ядре. Это помогает уменьшить оверхед за счет отказа от передачи нежелательных пакетов в пространство пользователя. Выражение фильтра обычно оптимизируется с помощью Berkeley Packet Filter (BPF), который компилирует выражение в байт-код BPF и затем может динамически компилироваться ядром в машинный код. В последние годы BPF был серьезно доработан и превратился в универсальную среду выполнения, которая поддерживает многие инструменты наблюдения: см. главу 3 «Операционные системы», раздел 3.4.4 «Расширенный BPF» и главу 15 «BPF».

10.5.7. Анализ TCP

Кроме задержек, о которых рассказывалось в разделе 10.5.4 «Анализ задержек», можно исследовать другие специфические черты поведения TCP, в том числе:

- использование буферов приема/передачи TCP (сокета);
- использование очередей запросов на соединение TCP;
- отвергнутые соединения из-за переполнения очереди запросов;

- размер окна управления перегрузкой, включая анонсирование окна нулевого размера;
- получение пакетов SYN в течение интервала TIME_WAIT.

Последнее может вызвать проблемы масштабируемости, когда сервер часто подключается к другим серверам к тому же порту, используя те же IP-адреса отправителя и получателя. Единственной отличительной чертой каждого соединения является исходящий порт клиента — *эфемерный, или динамический, порт*, который в TCP имеет 16-битное значение и может дополнительно ограничиваться параметрами ОС (минимальным и максимальным значениями диапазона портов). В сочетании с интервалом TCP TIME_WAIT, который может составлять 60 с, при высокой частоте создания новых соединений (более 65 536 за 60 с) могут возникнуть конфликты. В этом сценарии пакет SYN будет отправлен, когда этот эфемерный порт все еще связан с предыдущим сеансом TCP, находящимся в состоянии TIME_WAIT, из-за чего новый пакет SYN может быть отклонен, если ошибочно будет идентифицирован как часть старого соединения (коллизия). Для решения этой проблемы ядро Linux пытается быстро утилизировать соединения (что обычно дает хорошие результаты). Еще одно возможное решение — использовать на сервере несколько IP-адресов, а также создавать сокет с параметром SO_LINGER и с малым временем задержки перед уничтожением.

10.5.8. Статическая настройка производительности

Статическая настройка производительности фокусируется на проблемах сконфигурированной среды. Анализируя производительность сети, изучите следующие аспекты статической конфигурации:

- Сколько сетевых интерфейсов доступно для использования? Сколько используется сейчас?
- Какова максимальная скорость сетевых интерфейсов?
- Какова согласованная скорость сетевых интерфейсов сейчас?
- В каком режиме действуют сетевые интерфейсы — дуплексном или полудуплексном?
- Какой размер MTU настроен для сетевых интерфейсов?
- Сетевые интерфейсы виртуальные (trunked)?
- Какие настраиваемые параметры имеются для драйвера устройства? Уровня IP? Уровня TCP?
- Были ли присвоены каким-либо параметрам значения, отличные от значений по умолчанию?
- Как настроена маршрутизация? Какой шлюз используется по умолчанию?
- Какова максимальная пропускная способность сетевых компонентов на пути данных (всех компонентов, включая системные платы коммутаторов и маршрутизаторов)?

- Какой максимальный размер MTU поддерживается для канала данных и происходит ли фрагментация?
- Есть ли на пути передачи данных какие-либо беспроводные соединения? Страдают ли они от помех?
- Включена ли сквозная пересылка? Система действует как маршрутизатор?
- Как настроена DNS? Как далеко находится сервер?
- Есть ли известные проблемы с производительностью (ошибки) в версии микропрограммы сетевого интерфейса или любого другого сетевого оборудования?
- Есть ли известные проблемы с производительностью (или ошибки) в драйвере сетевого устройства? В стеке TCP/IP ядра?
- Какие брандмауэры присутствуют?
- Есть ли программные ограничения пропускной способности сети (средства управления ресурсами)? Какие?

Эти ответы помогут выявить ранее упускавшиеся из виду варианты конфигурации.

Последний вопрос особенно актуален для облачных сред, где могут накладываться ограничения на пропускную способность сети.

10.5.9. Управление ресурсами

Операционная система может предоставлять средства управления для распределения ресурсов сети между процессами или группами процессов. Они могут регулировать:

- **Ширину полосы пропускания сети:** разрешенную ширину полосы (максимальную пропускную способность) для различных протоколов или приложений.
- **Качество обслуживания IP** (quality of service, QoS): обработка сетевого трафика компонентами сети (например, маршрутизаторами) с учетом приоритетов. Это может быть реализовано по-разному: заголовок IP включает биты типа обслуживания (type-of-service, ToS), в том числе и приоритет. Сейчас эти биты используются для определения новых схем качества обслуживания, включая дифференцированное обслуживание (см. раздел 10.4.1 «Протоколы», подраздел «IP»). Другие протоколы могут реализовать для той же цели другие приоритеты.
- **Задержку пакетов:** дополнительную задержку пакетов (например, при использовании tc-netem(8) в Linux), которую можно применять для моделирования других сетей при тестировании производительности.

Во многих сетях циркулирует смешанный трафик, который можно разделить на низко- и высокоприоритетный. К низкоприоритетному трафику можно отнести передачу резервных копий и данных мониторинга производительности, к высокоприоритетному — трафик между производственным сервером и клиентами. Для регулирования низкоприоритетного трафика можно использовать любую схему

управления ресурсами, обеспечивающую благоприятную производительность для высокоприоритетного трафика.

Как это работает, зависит от реализации и обсуждается в разделе 10.8 «Настройка».

10.5.10. Микробенчмаркинг

Для бенчмаркинга сети есть множество инструментов. Они особенно полезны при исследовании проблем с пропускной способностью в распределенных приложениях, чтобы подтвердить, что сеть может как минимум достичь ожидаемой пропускной способности. Если это невозможно, то производительность сети изучают с помощью инструмента микробенчмаркинга, который, как правило, намного проще в отладке, чем приложение. После настройки желаемой скорости сети можно вернуться к отладке приложения.

Вот основные факторы, которые можно проверить:

- **направление:** передача или прием;
- **протокол:** TCP или UDP и порт;
- **количество потоков выполнения;**
- **размер буфера;**
- **размер MTU интерфейса.**

Для высокоскоростных сетевых интерфейсов, например, с пропускной способностью 100 Гбит/с, может потребоваться увеличить количество клиентских потоков, чтобы достичь максимальной ширины полосы пропускания.

В разделе 10.7.4 «iperf» показан пример микробенчмаркинга сети с помощью `iperf(1)`, также в разделе 10.7 «Эксперименты» перечислены другие доступные инструменты.

10.6. ИНСТРУМЕНТЫ НАБЛЮДЕНИЯ

В этом разделе представлены инструменты наблюдения за работой сети, доступные в ОС на базе Linux. Стратегии их использования описываются в предыдущем разделе.

Инструменты перечислены в табл. 10.4.

Описание этого набора инструментов и возможностей дополняет раздел 10.5 «Методология». Он начинается с описания традиционных инструментов и статистик, затем переходит к инструментам трассировки и, наконец, к инструментам перехвата пакетов. Некоторые из традиционных инструментов доступны (а иногда первоначально созданы) в других Unix-подобных ОС, в том числе `ifconfig(8)`, `netstat(8)` и `sar(1)`. Инструменты трассировки основаны на BPF и используют интерфейсы BCC и `bpftrace` (глава 15), в том числе `socketio(8)`, `tcpife(8)`, `tcptop(8)` и `tcpretrans(8)`.

Таблица 10.4. Инструменты наблюдения за работой сети

Раздел	Инструмент	Описание
10.6.1	ss	Статистики, характеризующие работу сокетов
10.6.2	ip	Информация о сетевых интерфейсах и маршрутах
10.6.3	ifconfig	Информация о сетевых интерфейсах
10.6.4	nstat	Статистики, характеризующие работу сетевого стека
10.6.5	netstat	Различные статистики, характеризующие работу сетевого стека и интерфейсов
10.6.6	sar	Исторические статистики
10.6.7	nicstat	Информация о потреблении сетевых интерфейсов и их пропускной способности
10.6.8	ethtool	Статистики, характеризующие работу драйвера сетевого интерфейса
10.6.9	tcplife	Трассировка жизненного цикла сеансов TCP с информацией о соединениях
10.6.10	tcpdump	Показывает пропускную способность хоста и процессов
10.6.11	tcpdumps	Трассировка повторных передач TCP с адресами и информацией о состоянии TCP
10.6.12	bpftool	Трассировка стека TCP/IP: соединений, получаемых и отправляемых пакетов, сброшенных пакетов, задержек
10.6.13	tcpdump	Инструмент для перехвата сетевых пакетов
10.6.14	Wireshark	Инструмент с графическим интерфейсом для анализа сетевых пакетов

Первыми рассматриваются инструменты `ss(8)`, `ip(8)` и `nstat(8)`, так как входят в состав пакета `iproute2`, который поддерживается инженерами, разрабатывающими сетевую подсистему в ядре. Инструменты из этого пакета практически всегда поддерживают самые новые функции ядра Linux. Также здесь рассматриваются аналогичные инструменты из пакета `net-tools` — `ifconfig(8)` и `netstat(8)`, — потому что они получили широкое распространение, хотя инженеры сетевой подсистемы в ядре Linux считают их устаревшими.

10.6.1. ss

`ss(8)` сообщает статистики, характеризующие работу открытых сокетов. По умолчанию выводится обобщенная информация о сокетах, например:

```
# ss
Netid State  Recv-Q  Send-Q   Local Address:Port  Peer Address:Port
[...]
tcp    ESTAB   0        0      100.85.142.69:65264  100.82.166.11:6001
tcp    ESTAB   0        0      100.85.142.69:6028  100.82.16.200:6101
[...]
```

Этот вывод отражает текущее состояние. В первом столбце выводится протокол, используемый сокетами, в данном случае TCP. Поскольку в выводе перечислены все установленные соединения с информацией об IP-адресах, его можно использовать для определения характеристик текущей рабочей нагрузки и получения ответов на такие вопросы, как количество открытых клиентами соединений, количество одновременно действующих соединений и т. д.

Аналогичную информацию по сокетам можно получить через инструмент `netstat(8)`. При запуске с параметрами `ss(8)` может сообщить гораздо больше информации. Например, с параметром `-t` будут отображаться только сокеты TCP, с параметром `-i` будет выводиться внутренняя информация TCP, с параметром `-e` — расширенная информация о сокете, с параметром `-p` — информация о процессах и с параметром `-m` — информация о потреблении памяти:

```
# ss -tiepm
State      Recv-Q   Send-Q   Local Address:Port   Peer Address:Port
ESTAB      0         0        100.85.142.69:65264   100.82.166.11:6001
users:((("java",pid=4195,fd=10865)) uid:33 ino:2009918 sk:78 <->
      skmem:(r0,rb12582912,t0,tb12582912,f266240,w0,o0,bl0,d0) ts sack bbr ws
cale:9,9 rto:204 rtt:0.159/0.009 ato:40 mss:1448 pmtu:1500 rcvmss:1448 advmss:14
48 cwnd:152 bytes_acked:347681 bytes_received:1798733 segs_out:582 segs_in:1397
data_segs_out:294 data_segs_in:1318 bbr:(bw:328.6Mbps,mrtt:0.149,pacing_gain:2.8
8672,cwnd_gain:2.88672) send 11074.0Mbps lastsnd:1696 lastrcv:1660 lastack:1660
pacing_rate 2422.4Mbps delivery_rate 328.6Mbps app_limited busy:16ms rcv_rtt:39.
822 rcv_space:84867 rcv_ssthresh:3609062 minrtt:0.139
[...]
```

В этом примере жирным выделены адреса конечных точек и следующие метрики:

- **"java",pid = 4195**: имя процесса «java» и его идентификатор PID 4195;
- **fd = 10865**: дескриптор файла 10865 (для процесса с PID 4195);
- **rto:204**: тайм-аут повторной передачи TCP, в данном случае 204 мс;
- **rtt:0,159/0,009**: среднее время приема-передачи составляет 0,159 мс со средним отклонением 0,009 мс;
- **mss:1448**: максимальный размер сегмента 1448 байт;
- **cwnd:152**: размер окна перегрузки: $152 \times \text{MSS}$;
- **bytes_acked:347681**: успешно передано 340 Кбайт;
- **bytes_received:1798733**: получено 1,72 Мбайт;
- **bbr:...**: статистика алгоритма BBR управления перегрузкой;
- **pacing_rate 2422.4Mbps**: скорость синхронизации 2422,4 Мбит/с;
- **app_limited**: показывает, что окно перегрузки используется не полностью, из чего можно предположить, что пропускная способность ограничивается воздействием приложения;
- **minrtt:0.139**: минимальное время приема/передачи в миллисекундах. Сравните со средним значением и средним отклонением (см. выше), чтобы получить представление о переменчивости пропускной способности сети и ее перегрузке.

Это конкретное соединение определено как ограниченное приложением (`app_limited`), с малым временем приема-передачи (RTT) и небольшим общим объемом переданных байтов. Вот некоторые флаги, сообщающие об ограничениях, которые может выводить `ss(1)`:

- **app_limited**: ограничено приложением;
- **rwnd_limited:Xms**: ограничено размером окна приема; включает время в миллисекундах;
- **sndbuf_limited:Xms**: ограничено буфером отправки; включает время в миллисекундах.

В этом выводе нет одной детали, нужной для вычисления средней пропускной способности, — продолжительности существования соединения. Но я нашел обходное решение — использовать отметку времени последнего изменения файла в `/proc`, соответствующего дескриптору файла: чтобы получить ее, здесь можно было бы выполнить команду `stat(1)` в каталоге `/proc/4195/fd/10865`.

netlink

`ss(8)` получает эти расширенные сведения из интерфейса `netlink(7)`, который, в свою очередь, использует сокеты семейства `AF_NETLINK` для получения информации из ядра. Убедиться в этом можно с помощью `strace(1)` (см. предупреждения об переходе `strace(1)` в главе 5 «Приложения», в разделе 5.5.4 «`strace`»):

```
# strace -e sendmsg,recvmsg ss -t
sendmsg(3, {msg_name={sa_family=AF_NETLINK, nl_pid=0, nl_groups=00000000},
msg_namelen=12, msg_iov=[{iov_base={len=72, type=SOCK_DIAG_BY_FAMILY,
flags=NLM_F_REQUEST|NLM_F_DUMP, seq=123456, pid=0}, {sdiag_family=AF_INET,
sdiag_protocol=IPPROTO_TCP, iddiag_ext=1<<(INET_DIAG_MEMINFO-1)|...
recvmsg(3, {msg_name={sa_family=AF_NETLINK, nl_pid=0, nl_groups=00000000},...
```

`netstat(8)`, в отличие от `ss(8)`, получает эту же информацию из файлов в `/proc/net`:

```
# strace -e openat netstat -an
[...]
```

```
openat(AT_FDCWD, "/proc/net/tcp", O_RDONLY) = 3
openat(AT_FDCWD, "/proc/net/tcp6", O_RDONLY) = 3
[...]
```

Файлы в `/proc/net` содержат простой текст, поэтому я считаю их более удобными источниками информации, для обработки которой не требуется ничего, кроме `awk(1)`. Но серьезные инструменты мониторинга должны использовать интерфейс `netlink(7)`, который передает информацию в двоичном формате и позволяет избежать перехода на синтаксический анализ текста.

10.6.2. ip

`ip(8)` — это инструмент для управления маршрутизацией, сетевыми устройствами, интерфейсами и туннелями. Для целей наблюдения с его помощью можно выводить

информацию об интерфейсах, адресах, маршрутах и т. д. Ниже приведен вывод расширенных статистик (-s) об интерфейсах (link):

```
# ip -s link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT
group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    RX: bytes    packets  errors  dropped  overrun  mcast
    26550075    273178    0        0         0         0
    TX: bytes    packets  errors  dropped  carrier  collsns
    26550075    273178    0        0         0         0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP mode DEFAULT
group default qlen 1000
    link/ether 12:c0:0a:b0:21:b8 brd ff:ff:ff:ff:ff:ff
    RX: bytes    packets  errors  dropped  overrun  mcast
    512473039143 568704184 0        0         0         0
    TX: bytes    packets  errors  dropped  carrier  collsns
    573510263433 668110321 0        0         0         0
```

Информация о конфигурации интерфейсов пригодится во время статической настройки производительности для проверки правильности конфигурации. В вывод также включены метрики ошибок: для приема (RX): количество ошибок приема, отброшенных пакетов и переполнений; для передачи (TX): количество ошибок передачи, отброшенных пакетов, ошибок несущей и коллизий. Такие ошибки могут быть причиной проблем с производительностью и, в зависимости от ошибки, вызываться неисправным сетевым оборудованием. Это глобальные счетчики, суммирующие все ошибки с момента инициализации интерфейса (на сетевом жаргоне — с момента, когда он был «поднят»).

Передав параметр -s дважды (-s -s), можно получить более подробную информацию об ошибках.

Несмотря на то что инструмент ip(8) выводит количество принятых и переданных байтов (RX и TX), он не сообщает текущую пропускную способность за интервал. Для получения этой информации используйте sar(1) (раздел 10.6.6 «sar»).

Таблица маршрутизации

ip(1) позволяет наблюдать за другими сетевыми компонентами. Например, при вызове с параметром route этот инструмент выводит таблицу маршрутизации:

```
# ip route
default via 100.85.128.1 dev eth0
default via 100.85.128.1 dev eth0 proto dhcp src 100.85.142.69 metric 100
100.85.128.0/18 dev eth0 proto kernel scope link src 100.85.142.69
100.85.128.1 dev eth0 proto dhcp scope link src 100.85.142.69 metric 100
```

Неправильно настроенные маршруты тоже могут стать причиной проблем с производительностью (например, когда некоторые маршруты были добавлены администратором, но со временем потеряли актуальность и теперь работают хуже, чем маршрут по умолчанию).

Мониторинг

Для наблюдения за сообщениями интерфейса netlink используйте команду `monitoring`.

10.6.3. ifconfig

Команда `ifconfig(8)` — это традиционный инструмент для администрирования сетевых интерфейсов, который также отображает информацию о конфигурации. Версия для Linux выводит вместе с конфигурацией некоторые статистики¹:

```
$ ifconfig
eth0      Link encap:Ethernet HWaddr 00:21:9b:97:a9:bf
          inet addr:10.2.0.2 Bcast:10.2.0.255 Mask:255.255.255.0
          inet6 addr: fe80::221:9bff:fe97:a9bf/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
          RX packets:933874764 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1090431029 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:584622361619 (584.6 GB) TX bytes:537745836640 (537.7 GB)
          Interrupt:36 Memory:d6000000-d6012800
eth3      Link encap:Ethernet HWaddr 00:21:9b:97:a9:c5
[...]
```

Счетчики имеют те же значения, что и в команде `ip(8)`, которая была описана выше.

В Linux команда `ifconfig(8)` считается устаревшей, и на замену ей пришла команда `ip(8)`.

10.6.4. nstat

`nstat(8)` выводит разные метрики сети, поддерживаемые ядром, вместе с их SNMP-именами. Вот пример вывода команды, запущенной с параметром `-s`, чтобы предотвратить сброс счетчиков:

```
# nstat -s
#kernel
IpInReceives          462657733          0.0
IpInDelivers          462657733          0.0
IpOutRequests         497050986          0.0
IpOutDiscards         42                 0.0
IpFragOKs             2298               0.0
IpFragCreates         13788              0.0
IcmpInMsgs            91                 0.0
[...]
TcpActiveOpens        362997             0.0
```

¹ Он также показывает значение настраиваемого параметра `txqueuelen`, но не все драйверы используют его (он передается сетевому устройству вместе с уведомлением `NETDEV_CHANGE_TX_QUEUE_LEN`, поддержка которого не реализована в некоторых драйверах), а настройка очереди байтов ограничивается автоматической настройкой очередей в устройстве.

TcpPassiveOpens	9663983	0.0
TcpAttemptFails	12718	0.0
TcpEstabResets	14591	0.0
TcpInSegs	462181482	0.0
TcpOutSegs	938958577	0.0
TcpRetransSegs	129212	0.0
TcpOutRsts	52362	0.0
UdpInDatagrams	476072	0.0
UdpNoPorts	88	0.0
UdpOutDatagrams	476197	0.0
UdpIgnoredMulti	2	0.0
Ip6OutRequests	29	0.0
[...]		

К ключевым относятся следующие метрики:

- **IpInReceives**: количество полученных IP-пакетов;
- **IpOutRequests**: количество отправленных IP-пакетов;
- **TcpActiveOpens**: количество активных соединений TCP (созданных системным вызовом `connect(2)`);
- **TcpPassiveOpens**: количество пассивных соединений TCP (принятых системным вызовом `accept(2)`);
- **TcpInSegs**: количество полученных сегментов TCP;
- **TcpOutSegs**: количество отправленных сегментов TCP;
- **TcpRetransSegs**: количество повторно переданных сегментов TCP. Сравните с `TcpOutSegs`, чтобы оценить долю повторных передач.

При вызове без параметра `-s nstat(8)` сбрасывает счетчики в ядре. Это может пригодиться, когда планируется повторно запустить `nstat(8)`, чтобы оценить, как изменились счетчики за известный интервал времени. Если у вас проблема с сетью, которую можно воспроизвести, то команду `nstat(8)` можно запустить до и после воспроизведения проблемы, чтобы посмотреть, какие счетчики изменились.

Если вы по ошибке забыли добавить параметр `-s` и сбросили счетчики, то их можно восстановить в значения, накопленные с начала загрузки системы, передав команде параметр `-rs`.

`nstat(8)` также может выполняться в режиме демона (`-d`) и собирать статистику за интервал времени, которая отображается в последнем столбце.

10.6.5. netstat

Команда `netstat(8)` сообщает разные сетевые метрики, в зависимости от используемых параметров. Это универсальный инструмент, способный выполнять множество различных функций, в том числе:

- **без параметров (по умолчанию)**: список открытых сокетов;
- **-a**: информация обо всех сокетах;

- **-s**: статистики сетевого стека;
- **-i**: статистики сетевого интерфейса;
- **-r**: таблица маршрутизации.

Другие параметры могут изменять формат вывода, например, **-n** отключает разрешение IP-адресов в имена хостов, а **-v** добавляет дополнительные детали, если таковые имеются.

Вот пример получения статистик сетевого интерфейса с помощью `netstat(8)`:

```
$ netstat -i
Kernel Interface table
Iface  MTU      RX-OK RX-ERR RX-DRP RX-OVR      TX-OK TX-ERR TX-DRP TX-OVR Flg
eth0    1500  933760207      0      0 0      1090211545      0      0      0 BMRU
eth3    1500  718900017      0      0 0      587534567      0      0      0 BMRU
lo      16436  21126497      0      0 0      21126497      0      0      0 LRU
ppp5    1496      4225      0      0 0      3736      0      0      0 MOPRU
ppp6    1496      1183      0      0 0      1143      0      0      0 MOPRU
tun0    1500    695581      0      0 0      692378      0      0      0 MOPRU
tun1    1462      0      0      0 0      4      0      0      0 PRU
```

В колонках выводятся: имя сетевого интерфейса (**Iface**), размер MTU и последовательность статистик для приема (**RX-**) и передачи (**TX-**):

- **-OK**: успешно принятых/отправленных пакетов;
- **-ERR**: количество ошибок приема/отправки пакетов;
- **-DRP**: количество отброшенных пакетов;
- **-OVR**: переполнений.

Счетчики отброшенных пакетов и пакетов, вызвавших переполнение, — это показатели насыщения сетевого интерфейса, и их можно исследовать вместе с ошибками в рамках метода `USE`.

Вместе с параметром **-i** можно использовать параметр **-s**, включающий режим непрерывного вывода, в котором накопленные значения счетчиков выводятся каждую секунду. Это позволяет оценить частоту приема/передачи пакетов.

Вот пример (усеченный) вывода `netstat(8)` со статистиками сетевого стека:

```
$ netstat -s
Ip:
  Forwarding: 2
  454143446 total packets received
  0 forwarded
  0 incoming packets discarded
  454143446 incoming packets delivered
  487760885 requests sent out
  42 outgoing packets dropped
  2260 fragments received ok
  13560 fragments created
Icmp:
```

```

    91 ICMP messages received
[...]
```

Tcp:

```

    359286 active connection openings
    9463980 passive connection openings
    12527 failed connection attempts
    14323 connection resets received
    13545 connections established
    453673963 segments received
    922299281 segments sent out
    127247 segments retransmitted
    0 bad segments received
    51660 resets sent
```

Udp:

```

    469302 packets received
    88 packets to unknown port received
    0 packet receive errors
    469427 packets sent
    0 receive buffer errors
    0 send buffer errors
    IgnoredMulti: 2
```

TcpExt:

```

    21 resets received for embryonic SYN_RECV sockets
    12252 packets pruned from receive queue because of socket buffer overrun
    201219 TCP sockets finished time wait in fast timer
    11727438 delayed acks sent
    1445 delayed acks further delayed because of locked socket
    Quick ack mode was activated 17624 times
    169257582 packet headers predicted
    76058392 acknowledgments not containing data payload received
    111925821 predicted acknowledgments
    TCPSackRecovery: 1703
    Detected reordering 876 times using SACK
    Detected reordering 19 times using time stamp
    2 congestion windows fully recovered without slow start
    19 congestion windows partially recovered using Hoe heuristic
    TCPDSACKUndo: 164
    88 congestion windows recovered without slow start after partial ack
    TCPLostRetransmit: 901
    TCPSackFailures: 31
    28248 fast retransmits
    709 retransmits in slow start
    TCPTimeouts: 12684
    TCPLossProbes: 73383
    TCPLossProbeRecovery: 132
    TCPSackRecoveryFail: 24
    805315 packets collapsed in receive queue due to low socket buffer
[...]
```

TCPAutoCorking: 13520259

```

    TCPFromZeroWindowAdv: 257
    TCPToZeroWindowAdv: 257
    TCPWantZeroWindowAdv: 18941
    TCPSynRetrans: 24816
[...]
```

В выводе отображаются различные статистики, в основном для TCP, сгруппированные по протоколам. У большинства из них описательные имена, поэтому их назначение вполне очевидно. Некоторые из статистик в этом примере выделены жирным, чтобы показать, какая информация, связанная с производительностью, доступна. Для анализа многих из них требуется глубокое понимание поведения протокола TCP, включая новые возможности и алгоритмы, появившиеся в последние годы. Вот некоторые примеры, на которые стоит обращать внимание:

- большая доля пересылаемых пакетов по сравнению с общим количеством полученных пакетов: убедитесь, действительно ли сервер должен пересылать (передавать дальше) пакеты;
- количество создававшихся пассивных соединений: эта статистика может отражать нагрузку, оказываемую клиентами;
- большая доля повторно отправленных сегментов по сравнению с количеством отправленных сегментов: может свидетельствовать о ненадежности сети. Этого можно ожидать при обслуживании клиентов в интернете;
- **TCPSynRetrans**: показывает количество повторных попыток послать пакет SYN, которые могут быть вызваны удаленной конечной точкой, выбрасывающей SYN из очереди ожидания из-за высокой нагрузки;
- пакеты, удаленные из очереди приема из-за переполнения буфера сокета: это признак проблемы насыщения сети, которую можно ослабить увеличением буферов сокета, при условии, что в системе достаточно ресурсов, чтобы приложение могло своевременно обрабатывать входящие данные.

Названия некоторых статистик содержат опечатки (например, **packetes rejected**). Их нельзя просто исправить, потому что появилось множество инструментов мониторинга, учитывающих эти опечатки в выводе. В таких инструментах лучше было бы анализировать вывод команды `nstat(8)`, которая использует стандартные имена SNMP, а еще лучше читать статистики прямо из их источников в `/proc:/proc/net/snmp` и `/proc/net/netstat`. Например:

```
$ grep ^Tcp /proc/net/snmp
Tcp: RtoAlgorithm RtoMin RtoMax MaxConn ActiveOpens PassiveOpens AttemptFails
     EstabResets CurrEstab InSegs OutSegs RetransSegs InErrs OutRsts InCsumErrors
Tcp: 1 200 120000 -1 102378 126946 11940 19495 24 627115849 325815063 346455 5
24183
0
```

Статистики из `/proc/net/snmp` также включают базы управляющей информации (Management Information Bases, MIB) SNMP. Документация MIB описывает, какой должна быть каждая статистика (если ядро правильно ее реализовало). Дополнительные статистики находятся в `/proc/net/netstat`.

Команде `netstat(8)` можно также передать число — интервал (в секундах) вывода накопленных счетчиков. Такой вывод можно затем дополнительно обработать, чтобы вычислить скорость изменения каждого счетчика.

10.6.6. sar

Генератор отчетов о работе системы (system activity reporter) sar(1) можно использовать для наблюдения за текущей активностью или настроить для архивирования статистик и генерирования хронологических отчетов. Он был представлен в главе 4 «Инструменты наблюдения», в разделе 4.4 «sar» и упоминается в других главах, где это необходимо.

Версия для Linux предлагает следующие возможности для анализа производительности сети:

- **-n DEV:** статистики сетевого интерфейса DEV;
- **-n EDEV:** ошибки сетевого интерфейса DEV;
- **-n IP:** статистики по IP-дейтаграммам;
- **-n EIP:** ошибки IP;
- **-n TCP:** статистики TCP;
- **-n ETCP:** ошибки TCP;
- **-n SOCK:** информация о потреблении сокетов.

В табл. 10.5 перечислены некоторые сетевые статистики, сообщаемые этим инструментом.

Таблица 10.5. Сетевые статистики, предлагаемые утилитой sar в Linux

Параметр	Статистика	Описание	Единицы измерения
-n DEV	rxpkt/s	Принято пакетов	Пакетов в секунду
-n DEV	txpkt/s	Отправлено пакетов	Пакетов в секунду
-n DEV	rxkB/s	Принято килобайт	Кбайт/с
-n DEV	txkB/s	Отправлено килобайт	Кбайт/с
-n DEV	rxcmp/s	Принято сжатых пакетов	Пакетов в секунду
-n DEV	txcmp/s	Отправлено сжатых пакетов	Пакетов в секунду
-n DEV	rxmcst/s	Принято многоадресных пакетов	Пакетов в секунду
-n DEV	%ifutil	Потребление интерфейса: при работе в полнодуплексном режиме помогает оценить долю приема и передачи	Проценты
-n EDEV	rxerr/s	Ошибок при приеме пакетов	Пакетов в секунду
-n EDEV	txerr/s	Ошибок при передаче пакетов	Пакетов в секунду
-n EDEV	coll/s	Коллизий	Пакетов в секунду
-n EDEV	rxdrop/s	Сброшено полученных пакетов (из-за заполнения буфера)	Пакетов в секунду

Таблица 10.5 (продолжение)

Параметр	Статистика	Описание	Единицы измерения
-n EDEV	txdrop/s	Сброшено отправляемых пакетов (из-за заполнения буфера)	Пакетов в секунду
-n EDEV	txcarr/s	Ошибок несущей передачи	Ошибок в секунду
-n EDEV	rxfram/s	Ошибок выравнивания при приеме	Ошибок в секунду
-n EDEV	rxfifo/s	Ошибок постановки в очередь полученных пакетов	Пакетов в секунду
-n EDEV	txfifo/s	Ошибок постановки в очередь отправляемых пакетов	Пакетов в секунду
-n IP	irec/s	Входящих дейтаграмм (принято)	Дейтаграмм в секунду
-n IP	fwddgm/s	Переслано дейтаграмм	Дейтаграмм в секунду
-n IP	idel/s	Входящих IP-дейтаграмм (включая ICMP)	Дейтаграмм в секунду
-n IP	orq/s	Исходящих запросов на вывод дейтаграмм (отправку)	Дейтаграмм в секунду
-n IP	asmrq/s	Принято фрагментов IP	Фрагментов в секунду
-n IP	asmok/s	Повторно собрано IP-дейтаграмм	Дейтаграмм в секунду
-n IP	fragok/s	Фрагментировано IP-дейтаграмм	Дейтаграмм в секунду
-n IP	fragcrt/s	Создано фрагментов IP-дейтаграмм	Фрагментов в секунду
-n EIP	ihrerr/s	Ошибок в заголовках IP	Дейтаграмм в секунду
-n EIP	iadrerr/s	Ошибок в IP-адресе получателя	Дейтаграмм в секунду
-n EIP	iukwnpr/s	Неизвестных ошибок протокола	Дейтаграмм в секунду
-n EIP	idisc/s	Сброшено входящих дейтаграмм (например, из-за заполнения буфера)	Дейтаграмм в секунду
-n EIP	odisc/s	Сброшено исходящих дейтаграмм (например, из-за заполнения буфера)	Дейтаграмм в секунду
-n EIP	onort/s	Ошибок исходящих дейтаграмм из-за отсутствия маршрута	Дейтаграмм в секунду
-n EIP	asmf/s	Ошибок повторной сборки IP-дейтаграмм	Ошибок в секунду
-n EIP	fragf/s	Отброшено дейтаграмм, не подлежащих фрагментированию	Дейтаграмм в секунду
-n TCP	active/s	Новых активных соединений TCP (connect(2))	Соединений в секунду
-n TCP	passive/s	Новых пассивных соединений TCP (accept(2))	Соединений в секунду
-n TCP	iseg/s	Входящих сегментов	Сегментов в секунду
-n TCP	oseg/s	Исходящих сегментов	Сегментов в секунду
-n ETCP	atmptf/s	Потеряно активных соединений	Соединений в секунду

Параметр	Статистика	Описание	Единицы измерения
-n ETCP	estres/s	Сброшено установленных соединений	Сбросов в секунду
-n ETCP	retrans/s	Повторно переданных сегментов TCP	Сегментов в секунду
-n ETCP	isegerr/s	Ошибок сегментов	Сегментов в секунду
-n ETCP	orsts/s	Сброшено передач	Сегментов в секунду
-n SOCK	totsck	Количество используемых сокетов	Сокеты
-n SOCK	tcpsck/s	Количество используемых сокетов TCP	Сокеты
-n SOCK	udpsck/s	Количество используемых сокетов UDP	Сокеты
-n SOCK	rawsck/s	Количество используемых низкоуровневых сокетов	Сокеты
-n SOCK	ip-frag	Фрагментов IP в очереди в данный момент	Фрагменты
-n SOCK	tcp-tw	Сокетов TCP в состоянии TIME_WAIT	Сокеты

В таблице не указаны группы ICMP, NFS и SOFT (программная обработка сети), а также варианты IPv6: IP6, EIP6, SOCK6 и UDP6. Полный список статистик можно найти на странице справочного руководства man, где также отмечены некоторые эквивалентные имена SNMP (например, ipInReceives для irec/s). Многие из названий статистик в sar(1) легко запоминаются, потому что включают направление и измеряемые единицы: rx — «принято» (received), i — «входящий» (input), seg — «сегменты» и т. д.

Вот пример вывода статистик TCP раз в секунду:

```
$ sar -n TCP 1
Linux 5.3.0-1010-aws (ip-10-1-239-218) 02/27/20 _x86_64_ (2 CPU)
07:32:45 active/s passive/s iseg/s oseg/s
07:32:46 0.00 12.00 186.00 28837.00
07:32:47 0.00 13.00 203.00 33584.00
07:32:48 0.00 11.00 1999.00 24441.00
07:32:49 0.00 7.00 92.00 8908.00
07:32:50 0.00 10.00 114.00 13795.00
[...]
```

Этот вывод показывает частоту создания пассивных (входящих), близкую к 10 в секунду.

При исследовании сетевых устройств (DEV) в столбце с названием сетевого интерфейса (IFACE) перечисляются все интерфейсы. Но часто интерес представляет только один из них. В примере ниже показано применение небольшого сценария на awk(1) для фильтрации вывода:

```
$ sar -n DEV 1 | awk 'NR == 3 || $2 == "ens5"'
07:35:41 IFACE rxpck/s txpck/s rxkB/s txkB/s rxcmp/s txcmp/s rxcmt/s %ifutil
07:35:42 ens5 134.00 11483.00 10.22 6328.72 0.00 0.00 0.00 0.00
07:35:43 ens5 170.00 20354.00 13.62 6925.27 0.00 0.00 0.00 0.00
```

```

07:35:44 ens5 185.00 28228.00 14.33 8586.79 0.00 0.00 0.00 0.00
07:35:45 ens5 180.00 23093.00 14.59 7452.49 0.00 0.00 0.00 0.00
07:35:46 ens5 1525.00 19594.00 137.48 7044.81 0.00 0.00 0.00 0.00
07:35:47 ens5 146.00 10282.00 12.05 6876.80 0.00 0.00 0.00 0.00
[...]
```

Эта команда показывает пропускную способность сети на прием и передачу, а также другие статистики.

Используя инструмент `atop(1)`, можно архивировать статистики.

10.6.7. `nicstat`

`nicstat(1)`¹ выводит статистики сетевого интерфейса, включая пропускную способность и потребление. Он основан на модели традиционных инструментов `iostat(1)` и `mpstat(1)`.

Вот пример вывода, полученного с помощью версии 1.92 в Linux:

```
# nicstat -z 1
```

Time	Int	rKB/s	wKB/s	rPk/s	wPk/s	rAvs	wAvs	%Util	Sat
01:20:58	eth0	0.07	0.00	0.95	0.02	79.43	64.81	0.00	0.00
01:20:58	eth4	0.28	0.01	0.20	0.10	1451.3	80.11	0.00	0.00
01:20:58	vlan123	0.00	0.00	0.00	0.02	42.00	64.81	0.00	0.00
01:20:58	br0	0.00	0.00	0.00	0.00	42.00	42.07	0.00	0.00
Time	Int	rKB/s	wKB/s	rPk/s	wPk/s	rAvs	wAvs	%Util	Sat
01:20:59	eth4	42376.0	974.5	28589.4	14002.1	1517.8	71.27	35.5	0.00
Time	Int	rKB/s	wKB/s	rPk/s	wPk/s	rAvs	wAvs	%Util	Sat
01:21:00	eth0	0.05	0.00	1.00	0.00	56.00	0.00	0.00	0.00
01:21:00	eth4	41834.7	977.9	28221.5	14058.3	1517.9	71.23	35.1	0.00
Time	Int	rKB/s	wKB/s	rPk/s	wPk/s	rAvs	wAvs	%Util	Sat
01:21:01	eth4	42017.9	979.0	28345.0	14073.0	1517.9	71.24	35.2	0.00

Сначала выводится сводная информация, накопленная с момента загрузки, а затем идут статистики, полученные за истекший интервал. Как показывают интервальные данные, уровень потребления интерфейса `eth4` составил 35 % (это число соответствует наибольшему потреблению в одном из направлений, приема или передачи), а скорость чтения достигает 42 Мбайт/с.

В полях выводятся: имя интерфейса (**Int**), максимальное потребление (**%Util**), значение, отражающее насыщенность интерфейса (**Sat**), и ряд статистик с префиксом **r** для «чтения» (приема) и **w** для «записи» (передачи):

- **KB/s**: килобайт в секунду;
- **Pk/s**: пакетов в секунду;
- **Avs/s**: средний размер пакета в байтах.

¹ Первая версия была разработана мной для Solaris. Версию для Linux написал Тим Кук (Tim Cook) [Cook 09].

Из параметров, поддерживаемых в этой версии, можно отметить `-z`, запрещающий выводить строки с нулевыми значениями (неиспользуемые интерфейсы), и `-t` для вывода статистик ТСП.

`nicstat(1)` с успехом можно использовать в методе USE, поскольку он сообщает значения потребления и насыщения.

10.6.8. ethtool

`ethtool(8)` можно использовать для проверки параметров настройки сетевых интерфейсов, если добавить параметры `-i` и `-k`, а также статистик драйверов, если добавить параметр `-S`. Например:

```
# ethtool -S eth0
NIC statistics:
    tx_timeout: 0
    suspend: 0
    resume: 0
    wd_expired: 0
    interface_up: 1
    interface_down: 0
    admin_q_pause: 0
    queue_0_tx_cnt: 100219217
    queue_0_tx_bytes: 84830086234
    queue_0_tx_queue_stop: 0
    queue_0_tx_queue_wakeup: 0
    queue_0_tx_dma_mapping_err: 0
    queue_0_tx_linearize: 0
    queue_0_tx_linearize_failed: 0
    queue_0_tx_napi_comp: 112514572
    queue_0_tx_tx_poll: 112514649
    queue_0_tx_doorbells: 52759561
[...]
```

Эта команда извлекает статистики из инфраструктуры `ethtool` в ядре, которая поддерживает множество разных драйверов сетевых устройств. Драйверы устройств могут определять свои метрики `ethtool`.

При запуске с параметром `-i` команда выводит сведения о драйвере, а с параметром `-k` — параметры настройки интерфейса. Например:

```
# ethtool -i eth0
driver: ena
version: 2.0.3K
[...]
# ethtool -k eth0
Features for eth0:
rx-checksumming: on
[...]
tcp-segmentation-offload: off
    tx-tcp-segmentation: off [fixed]
    tx-tcp-ecn-segmentation: off [fixed]
```

```

tx-tcp-mangleid-segmentation: off [fixed]
tx-tcp6-segmentation: off [fixed]
udp-fragmentation-offload: off
generic-segmentation-offload: on
generic-receive-offload: on
large-receive-offload: off [fixed]
rx-vlan-offload: off [fixed]
tx-vlan-offload: off [fixed]
ntuple-filters: off [fixed]
receive-hashing: on
highdma: on
[...]
```

Этот пример был получен в облачном экземпляре с драйвером epa и с отключенной настройкой `tcp-segmentation-offload`. Для изменения этих настроек используйте параметр `-K`.

10.6.9. tcplife

`tcplife(8)`¹ — это инструмент ВСС и `bpfttrace` для трассировки сеансов TCP. Он отображает продолжительность каждого сеанса, адреса, пропускную способность и, если возможно, идентификатор и имя процесса.

Ниже показан пример вывода ВСС-версии `tcplife(8)`, полученный на производственном экземпляре с 48 процессорами:

```
# tcplife
PID  COMM      LADDR          LPORT  RADDR          RPORT  TX_KB  RX_KB  MS
4169  java      100.1.111.231  32648  100.2.0.48      6001    0      0    3.99
4169  java      100.1.111.231  32650  100.2.0.48      6001    0      0    4.10
4169  java      100.1.111.231  32644  100.2.0.48      6001    0      0    8.41
4169  java      100.1.111.231  40158  100.2.116.192   6001    7     33  3590.91
4169  java      100.1.111.231  56940  100.5.177.31    6101    0      0    2.48
4169  java      100.1.111.231  6001   100.2.176.45    49482   0      0   17.94
4169  java      100.1.111.231  18926  100.5.102.250   6101    0      0    0.90
4169  java      100.1.111.231  44530  100.2.31.140    6001    0      0    2.64
4169  java      100.1.111.231  44406  100.2.8.109     6001   11     28  3982.11
34781 sshd      100.1.111.231  22     100.2.17.121    41566   5      7  2317.30
4169  java      100.1.111.231  49726  100.2.9.217     6001   11     28  3938.47
4169  java      100.1.111.231  58858  100.2.173.248   6001    9     30  2820.51
[...]
```

Здесь показана последовательность соединений, часть из которых короткоживущие (менее 20 мс), а часть — долгоживущие (действовавшие более 3 с) согласно столбцу «MS», отображающему продолжительность в миллисекундах. Это пул серверов приложений, прослушивающий порт 6001. Большинство сеансов в этом примере соответствуют соединениям с портом 6001 на удаленных серверах приложений, и только одно соединение установлено с локальным портом 6001. Также в процессе

¹ Немного истории: первую версию `tcplife(8)` я написал 18 октября 2016 года, взяв за основу идею Джулии Эванс (Julia Evans), а версию для `bpfttrace` — 17 апреля 2019 года.

трассировки был сеанс ssh с локальным портом 22, который прослушивает демон sshd, — это входящий сеанс.

Версия tcplife(8) для BCC поддерживает следующие параметры:

- **-t**: включать в вывод отметки времени (в формате ЧЧ:ММ:СС);
- **-w**: широкие столбцы (лучше подходят для вывода адресов IPv6);
- **-p PID**: трассировать только этот процесс;
- **-L PORT[,PORT[,...]]**: трассировать сеансы только с этими локальными портами;
- **-D PORT[,PORT[,...]]**: трассировать сеансы только с этими удаленными портами.

Инструмент производит трассировку событий изменения состояния сокета TCP и выводит обобщенные данные, когда состояние изменяется на TCP_CLOSE. События изменения состояния случаются с намного меньшей частотой, чем частота следования пакетов, что делает этот подход гораздо менее затратным в плане оверхеда и позволяет использовать tcplife(8) для непрерывного анализа потока TCP на производственных серверах Netflix¹.

В главе 10 книги «BPF Performance Tools» [Gregg 19] приводится упражнение, где нужно создать udplife(8) для трассировки сеансов UDP. Свое начальное решение я опубликовал в [Gregg 19d].

10.6.10. tcptop

tcptop(8)² — это инструмент BCC, отображающий процессы, наиболее активно использующие TCP. Вот пример, полученный на производственном экземпляре Hadoop с 36 процессорами:

```
# tcptop
09:01:13 loadavg: 33.32 36.11 38.63 26/4021 123015
```

PID	COMM	LADDR	RADDR	RX_KB	TX_KB
118119	java	100.1.58.46:36246	100.2.52.79:50010	16840	0
122833	java	100.1.58.46:52426	100.2.6.98:50010	0	3112
122833	java	100.1.58.46:50010	100.2.50.176:55396	3112	0
120711	java	100.1.58.46:50010	100.2.7.75:23358	2922	0
121635	java	100.1.58.46:50010	100.2.5.101:56426	2922	0
121219	java	100.1.58.46:50010	100.2.62.83:40570	2858	0
121219	java	100.1.58.46:42324	100.2.4.58:50010	0	2858
122927	java	100.1.58.46:50010	100.2.2.191:29338	2351	0
[...]					

¹ С попутным выводом сведений обо всех открытых сеансах.

² Немного истории: версию для BCC я написал 2 сентября 2016 года, взяв за основу более раннюю версию, написанную мной в 2005 году, которая, в свою очередь, была написана по образу и подобию инструмента top(1) Уильяма Лефевра.

Как показывает этот вывод, в период трассировки через самое активное соединение (в начале списка) было получено более 16 Мбайт. По умолчанию информация обновляется каждую секунду.

Инструмент трассирует отправку и прием данных по протоколу TCP и суммирует результаты в высокопроизводительной карте BPF. Но события могут следовать очень часто, и в системах с высокой сетевой нагрузкой затраты на трассировку могут стать ощутимыми.

В числе параметров можно назвать:

- **-C**: не очищать экран;
- **-p PID**: выполнять измерения только для этого процесса.

Кроме того, `tcptop(8)` принимает необязательные числовые аргументы, определяющие интервал и счетчик.

10.6.11. tcpretrans

`tcpretrans(8)`¹ — это инструмент BCC и `bpftrace` для трассировки повторных передач TCP, который выводит IP-адреса и номера портов, а также состояние TCP. Вот пример вывода версии `tcpretrans(8)` для BCC на производственном экземпляре:

```
# tcpretrans
Tracing retransmits ... Hit Ctrl-C to end
TIME    PID    IP    LADDR:LPORT    T> RADDR:RPORT    STATE
00:20:11 72475  4    100.1.58.46:35908 R> 100.2.0.167:50010 ESTABLISHED
00:20:11 72475  4    100.1.58.46:35908 R> 100.2.0.167:50010 ESTABLISHED
00:20:11 72475  4    100.1.58.46:35908 R> 100.2.0.167:50010 ESTABLISHED
00:20:12 60695  4    100.1.58.46:52346 R> 100.2.6.189:50010 ESTABLISHED
00:20:12 60695  4    100.1.58.46:52346 R> 100.2.6.189:50010 ESTABLISHED
00:20:12 60695  4    100.1.58.46:52346 R> 100.2.6.189:50010 ESTABLISHED
00:20:12 60695  4    100.1.58.46:52346 R> 100.2.6.189:50010 ESTABLISHED
00:20:13 60695  6    ::ffff:100.1.58.46:13562 R> ::ffff:100.2.51.209:47356 FIN_WAIT1
00:20:13 60695  6    ::ffff:100.1.58.46:13562 R> ::ffff:100.2.51.209:47356 FIN_WAIT1
[...]
```

Как показывает этот вывод, в период трассировки повторные передачи выполнялись нечасто, всего несколько раз в секунду (об этом можно судить по столбцу TIME), и в основном в сеансах с состоянием ESTABLISHED. Высокая частота повторных передач в состоянии ESTABLISHED может указывать на проблемы с внешней сетью. Высокая частота в состоянии SYN_SENT может указывать на большую нагрузку на серверное приложение, которое не успевает обслуживать свою очередь запросов на соединение.

¹ Немного истории: в 2011 году я написал несколько похожих инструментов. Версию `tcpretrans(8)` на основе `Ftrace` я написал в 2014 году, а версию для BCC — 14 февраля 2016 года. Версию для `bpftrace` написал Дейл Хамел (Dale Hamel) 23 ноября 2018 года.

Этот инструмент трассирует события повторной передачи ТСП в ядре. Так как обычно они происходят нечасто, оверхед должен быть незначительным. Для сравнения, в традиционном подходе с использованием инструментов для перехвата всех пакетов и последующей их обработкой в поисках повторных передач оба шага могут приводить к значительным затратам вычислительных ресурсов. Кроме того, подход на основе перехвата пакетов может наблюдать только детали, имеющие отношение к сети, тогда как `tcpretrans(8)` выводит состояние ТСП-соединений непосредственно из ядра и его можно расширить для вывода большего количества информации, если потребуется.

Версия для ВСС поддерживает следующие параметры:

- **-1:** включать попытки повторной отправки «хвостового» сегмента (`Tail Loss Probe`, добавляет трассировку зонда ядра для `tcp_send_loss_probe()`);
- **-c:** подсчитывать повторные передачи по потокам данных.

Параметр `-c` меняет поведение `tcpretrans(8)`, заставляя выводить суммарные счетчики вместо подробностей о каждом событии.

10.6.12. bpftrace

`bpftrace` — это трассировщик, основанный на ВРФ, который поддерживает высокоуровневый язык программирования, позволяющий создавать мощные и короткие сценарии. Хорошо подходит для анализа работы сети на основе подсказок, полученных с помощью других инструментов.

Позволяет исследовать сетевые события в ядре и в приложениях, в том числе подключение к сокета, ввод/вывод в сокетах, события ТСП, передачу пакетов, отбрасывание запросов на соединение, повторные передачи ТСП и другие детали. Эти возможности помогают определять характер рабочей нагрузки и анализировать задержки.

Подробнее о `bpftrace` речь пойдет в главе 15. В этом разделе показано лишь несколько примеров однострочных сценариев для анализа производительности сети, трассировки сокетов и протокола ТСП.

Однострочные сценарии

Следующие однострочные сценарии показывают различные возможности `bpftrace`.

Подсчет обращений к системному вызову `accept(2)` с группировкой по PID и именам процессов:

```
bpftrace -e 't:syscalls:sys_enter_accept* { @[pid, comm] = count(); }'
```

Подсчет обращений к системному вызову `connect(2)` с группировкой по PID и именам процессов:

```
bpftrace -e 't:syscalls:sys_enter_connect { @[pid, comm] = count(); }'
```

Подсчет обращений к системному вызову `connect(2)` с группировкой по трассировкам стека в пространстве пользователя:

```
bpfttrace -e 't:syscalls:sys_enter_connect { @[ustack, comm] = count(); }'
```

Подсчет операций приема/передачи с группировкой по направлениям, а также PID и именам процессов, выполняющихся на процессоре¹:

```
bpfttrace -e 'k:sock_sendmsg,k:sock_recvmsg { @[func, pid, comm] = count(); }'
```

Подсчет принятых/отправленных байтов с группировкой по PID и именам процессов на процессоре:

```
bpfttrace -e 'kr:sock_sendmsg,kr:sock_recvmsg /(int32)retval > 0/ { @[pid, comm] = sum((int32)retval); }'
```

Подсчет иницированных соединений TCP с группировкой по PID и именам процессов на процессоре:

```
bpfttrace -e 'k:tcp_v*_connect { @[pid, comm] = count(); }'
```

Подсчет принятых соединений TCP с группировкой по PID и именам процессов на процессоре:

```
bpfttrace -e 'k:inet_csk_accept { @[pid, comm] = count(); }'
```

Подсчет операций приема/передачи с группировкой по PID и именам процессов на процессоре:

```
bpfttrace -e 'k:tcp_sendmsg,k:tcp_recvmsg { @[func, pid, comm] = count(); }'
```

Гистограмма с количествами отправленных байтов по протоколу TCP:

```
bpfttrace -e 'k:tcp_sendmsg { @send_bytes = hist(arg2); }'
```

Гистограмма с количествами полученных байтов по протоколу TCP:

```
bpfttrace -e 'kr:tcp_recvmsg /retval >= 0/ { @recv_bytes = hist(retval); }'
```

Подсчет повторных передач TCP с группировкой по типам и удаленным хостам (предполагается, что используется протокол IPv4):

```
bpfttrace -e 't:tcp:tcp_retransmit_* { @[probe, ntop(2, args->saddr)] = count(); }'
```

¹ Системные вызовы, которые трассируются предыдущими сценариями, выполняются в контексте процесса, где надежно определяются PID и имя процесса. Зонды `kprobes`, используемые в этом сценарии, находятся глубже в ядре и к моменту их срабатывания процесс, инициировавший соединение, может уже оставить процессор. Это означает, что значения `pid` и `comm`, доступные `bpfttrace`, могут быть связаны с другим процессом. Такое случается редко, но все же случается.

Подсчет вызовов всех функций TCP (добавляет значительный оверхед при большом количестве операций TCP):

```
bpfftrace -e 'k:tcp_* { @[func] = count(); }'
```

Подсчет операций приема/передачи по протоколу UDP с группировкой по PID и именам процессов на процессоре:

```
bpfftrace -e 'k:udp*_sendmsg,k:udp*_recvmsg { @[func, pid, comm] = count(); }'
```

Гистограмма с количествами отправленных байтов по протоколу UDP:

```
bpfftrace -e 'k:udp_sendmsg { @send_bytes = hist(arg2); }'
```

Гистограмма с количествами принятых байтов по протоколу UDP:

```
bpfftrace -e 'kr:udp_recvmsg /retval >= 0/ { @recv_bytes = hist(retval); }'
```

Подсчет операций передачи с группировкой по трассировкам стека в пространстве ядра:

```
bpfftrace -e 't:net:net_dev_xmit { @[kstack] = count(); }'
```

Выводит гистограмму потребления процессорного времени на прием для каждого устройства:

```
bpfftrace -e 't:net:netif_receive_skb { @[str(args->name)] = lhist(cpu, 0, 128, 1); }'
```

Подсчитывает вызовы функций `ieee80211` (добавляет значительный оверхед в операции с пакетами):

```
bpfftrace -e 'k:ieee80211_* { @[func] = count(); }'
```

Подсчитывает вызовы всех функций драйвера устройства `ixgbev` (добавляет значительный оверхед в работу `ixgbev`):

```
bpfftrace -e 'k:ixgbev_* { @[func] = count(); }'
```

Подсчитывает срабатывания всех точек трассировки драйвера устройства `iwl` (добавляет значительный оверхед в работу `iwl`):

```
bpfftrace -e 't:iwlfwifi:*,t:iwlfwifi_io:* { @[probe] = count(); }'
```

Трассировка сокетов

Трассировка сетевых событий на уровне сокетов имеет то преимущество, что соответствующий процесс по-прежнему выполняется на процессоре. Это упрощает определение приложения и путей в коде. Вот пример подсчета обращений к системному вызову `accept(2)` с группировкой по PID и именам процессов:

```
# bpfftrace -e 't:syscalls:sys_enter_accept { @[pid, comm] = count(); }'
Attaching 1 probe...
^C
```

```
@[573, sshd]: 2
@[1948, mysqld]: 41
```

Как показывает этот вывод, в период трассировки сервер `mysqld` вызвал `accept(2)` 41 раз, а `sshd` — 2 раза.

Есть возможность включить в вывод трассировки стека, чтобы показать путь в коде, который привел к вызову `accept(2)`. Вот пример подсчета обращений к `accept(2)` с группировкой по трассировкам стека в пространстве пользователя и именам процессов:

```
# bpftrace -e 't:syscalls:sys_enter_accept { @[ustack, comm] = count(); }'
Attaching 1 probe...
^C

@[
  accept+79
  Mysqld_socket_listener::listen_for_connection_event()+283
  mysqld_main(int, char**)+15577
  __libc_start_main+243
  0x49564100fe8c4b3d
, mysqld]: 22
```

Как показывает этот вывод, сервер `mysqld` принимал входящие соединения в функции `Mysqld_socket_listener::listen_for_connection_event()`. Заменяя «`accept`» на «`connect`», с помощью этого однострочного сценария можно определить пути в коде, ведущие к вызову `connect(2)`. Мне доводилось использовать такие однострочные сценарии, чтобы объяснить загадочные сетевые соединения, анализируя пути в коде, ведущие к ним.

Точки трассировки `sock`

Помимо системных вызовов есть и точки трассировки сокетов. Вот пример для ядра 5.3:

```
# bpftrace -l 't:sock:*'
tracepoint:sock:sock_rcvqueue_full
tracepoint:sock:sock_exceed_buf_limit
tracepoint:sock:inet_sock_set_state
```

Точка трассировки `sock:inet_sock_set_state` используется инструментом `tcplife(8)`, описанным выше. Вот пример однострочного сценария, использующего эту точку трассировки для подсчета IPv4-адресов отправителя и получателя в новых соединениях:

```
# bpftrace -e 't:sock:inet_sock_set_state
  /args->newstate == 1 && args->family == 2/ {
    @[ntop(args->saddr), ntop(args->daddr)] = count() }'
Attaching 1 probe...
^C
@[127.0.0.1, 127.0.0.1]: 2
@[10.1.239.218, 10.29.225.81]: 18
```

Этот однострочный сценарий довольно длинный, и его было бы проще сохранить в файл с программой на языке bpftrace (.bt), чтобы его легче было редактировать и запускать. Сохранив программу в файле, можно также подключить соответствующие заголовочные файлы из исходного кода ядра, чтобы строку фильтра можно было переписать с использованием имен констант, а не жестко закодированных чисел (которые ненадежны), например:

```
/args->newstate == TCP_ESTABLISHED && args->family == AF_INET/ {
```

Далее приводится пример такой программы в файле: socketio.bt.

socketio.bt

В качестве более сложного примера ниже приведен исходный код инструмента socketio(8), подсчитывающий операции ввода/вывода с сокетом и добавляющий информацию о процессе, направлении, протоколе и портах. Например:

```
# ./socketio.bt
Attaching 2 probes...
^C
[...]
@io[sshd, 21925, read, UNIX, 0]: 40
@io[sshd, 21925, read, TCP, 37408]: 41
@io[systemd, 1, write, UNIX, 0]: 51
@io[systemd, 1, read, UNIX, 0]: 57
@io[systemd-udevd, 241, write, NETLINK, 0]: 65
@io[systemd-udevd, 241, read, NETLINK, 0]: 75
@io[dbus-daemon, 525, write, UNIX, 0]: 98
@io[systemd-logind, 526, read, UNIX, 0]: 105
@io[systemd-udevd, 241, read, UNIX, 0]: 127
@io[snappd, 31927, read, NETLINK, 0]: 150
@io[dbus-daemon, 525, read, UNIX, 0]: 160
@io[mysqld, 1948, write, TCP, 55010]: 8147
@io[mysqld, 1948, read, TCP, 55010]: 24466
```

Как показывает этот вывод, большинство операций ввода/вывода с сокетами выполнял сервер mysqld, который читал и записывал данные в TCP-порт 55010 — временный порт, использовавшийся для подключения клиента.

Исходный код socketio(8):

```
#!/usr/local/bin/bpftrace

#include <net/sock.h>

kprobe:sock_recvmmsg
{
    $sock = (struct socket *)arg0;
    $dport = $sock->sk->__sk_common.skc_dport;
    $dport = ($dport >> 8) | (($dport << 8) & 0xff00);
    @io[comm, pid, "read", $sock->sk->__sk_common.skc_prot->name, $dport] =
        count();
}
```

```
kprobe:sock_sendmsg
{
    $sock = (struct socket *)arg0;
    $dport = $sock->sk->__sk_common.skc_dport;
    $dport = ($dport >> 8) | (($dport << 8) & 0xff00);
    @io[comm, pid, "write", $sock->sk->__sk_common.skc_prot->name, $dport] =
        count();
}
```

В этом сценарии пример извлечения информации из структур в ядра — из struct socket, которая содержит название протокола и порт назначения. Порт назначения представлен целым числом с прямым порядком следования байтов (big-endian) и преобразуется в число с обратным порядком байтов (little-endian, для процессора x86) перед добавлением в карту @io¹. Этот сценарий можно изменить и подсчитывать байты вместо операций ввода/вывода.

Трассировка ТСП

Трассировка на уровне ТСП позволяет получить представление о событиях и внутренних компонентах протокола ТСП, а также о событиях, не связанных с сокетами (например, сканирование портов ТСП).

Точки трассировки ТСП

Инструментация внутренних механизмов ТСП часто возможна только через зонды kprobes, но есть несколько точек трассировки ТСП. Вот пример для ядра 5.3:

```
# bpftrace -l 't:tcp:*'
tracepoint:tcp:tcp_retransmit_skb
tracepoint:tcp:tcp_send_reset
tracepoint:tcp:tcp_receive_reset
tracepoint:tcp:tcp_destroy_sock
tracepoint:tcp:tcp_rcv_space_adjust
tracepoint:tcp:tcp_retransmit_synack
tracepoint:tcp:tcp_probe
```

Точка трассировки tcp:tcp_retransmit_skb используется инструментом tcpretrans(8), описанным выше. Точки трассировки предпочтительнее из-за их стабильности, но если они не позволяют решить поставленную задачу, то для инструментации функций ТСП в ядре можно использовать зонды kprobes. Например:

```
# bpftrace -e 'k:tcp_* { @[func] = count(); }'
Attaching 336 probes...
^C
@[tcp_try_keep_open]: 1
@[tcp_ooo_try_coalesce]: 1
```

¹ Для правильной работы на процессорах с прямым порядком следования байтов инструмент должен проверять аппаратный порядок байтов и применять преобразование, только если это необходимо, например, используя #ifdef LITTLE_ENDIAN.

```
@[tcp_reset]: 1
[...]
@[tcp_push]: 3191
@[tcp_established_options]: 3584
@[tcp_wfree]: 4408
@[tcp_small_queue_check.isra.0]: 4617
@[tcp_rate_check_app_limited]: 7022
@[tcp_poll]: 8898
@[tcp_release_cb]: 18330
@[tcp_send_mss]: 28168
@[tcp_sendmsg]: 31450
@[tcp_sendmsg_locked]: 31949
@[tcp_write_xmit]: 33276
@[tcp_tx_timestamp]: 33485
```

Как показывает этот вывод, в период трассировки чаще других вызывалась функция `tcp_tx_timestamp()` — 33 485 раз. Подсчитывая вызовы функций, можно определить цели для более детальных исследований. Обратите внимание, что подсчет вызовов всех функций ТСП может добавить заметный оверхед из-за количества и частоты вызова трассируемых функций. Для этой конкретной задачи я бы использовал профилирование функций с помощью моего инструмента `funccount(8)`, основанного на `Ftrace`, — оверхед и время инициализации у него намного меньше. См. главу 14 «`Ftrace`».

tcpsynbl.bt

Инструмент `tcpsynbl(8)`¹ — это пример инструментации ТСП с использованием зондов `kprobes`. Он показывает размеры очередей с поступившими запросами на соединение, используемыми системным вызовом `listen(2)`, с разбивкой по размерам, чтобы можно было определить, насколько близки очереди к переполнению (что вызывает отбрасывание пакетов TCP SYN). Вот пример вывода:

```
# tcpsynbl.bt
Attaching 4 probes...
Tracing SYN backlog size. Ctrl-C to end.
04:44:31 dropping a SYN.
04:44:31 dropping a SYN.
04:44:31 dropping a SYN.
04:44:31 dropping a SYN.
04:44:31 dropping a SYN.
[...]
^C
@backlog[backlog limit]: histogram of backlog size

@backlog[128]:
[0]           473 |@
[1]           502 |@
[2, 4)        1001 |@@@
[4, 8)        1996 |@@@@@@
[8, 16)       3943 |@@@@@@@@@@@@
```

¹ Немного истории: я написал `tcpsynbl.bt` 19 апреля 2019 года для книги [Gregg 19].

[16, 32)	7718	@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@	
[32, 64)	14460	@@	
[64, 128)	17246	@@	
[128, 256)	1844	@@@@@@	

Во время работы tcpsynbl.bt выводит отметки времени, когда происходило отбрасывание пакетов SYN, если это происходило. После прерывания (нажатием Ctrl-C) выводится гистограмма с размерами очередей запросов на соединение для каждого используемого предела длины очереди. В этом выводе можно видеть, что в 4:44:31 было отброшено несколько пакетов SYN, а результирующая гистограмма показывает распределение размеров очередей для предела 128, согласно которой этот предел был достигнут 1844 раза (интервал от 128 до 256). Это распределение сообщает размеры очередей, бывших в момент получения пакета SYN.

Трассировка размеров очередей запросов на соединение позволяет проверить, растет ли очередь со временем, и предупредить заранее о неизбежности потери пакетов SYN. Такой анализ можно использовать при планировании мощностей.

Исходный код tcpsynbl(8):

```
#!/usr/local/bin/bpftrace

#include <net/sock.h>

BEGIN
{
    printf("Tracing SYN backlog size. Ctrl-C to end.\n");
}

kprobe:tcp_v4_syn_recv_sock,
kprobe:tcp_v6_syn_recv_sock
{
    $sock = (struct sock *)arg0;
    @backlog[$sock->sk_max_ack_backlog & 0xffffffff] =
        hist($sock->sk_ack_backlog);
    if ($sock->sk_ack_backlog > $sock->sk_max_ack_backlog) {
        time("%H:%M:%S dropping a SYN.\n");
    }
}

END
{
    printf("\n@backlog[backlog limit]: histogram of backlog size\n");
}
```

Форма распределения, представленного выше, во многом обусловлена масштабом log2, используемым функцией hist(), в соответствии с которым каждый следующий интервал охватывает более широкий диапазон. Вместо hist() можно использовать lhist():

```
lhist($sock->sk_ack_backlog, 0, 1000, 10);
```

В результате выводится линейная гистограмма с одинаковыми диапазонами в интервалах: в данном случае гистограмма охватывает диапазон от 0 до 1000 с шириной

интервала 10. Подробнее о программировании на bpftrace рассказывается в главе 15 «BPF».

Источники событий

bpftrace может инструментировать не только точки трассировки и зонды kprobes. В табл. 10.6 перечислены все источники сетевых событий, доступные для инструментации.

Таблица 10.6. Сетевые события и их источники

События	Источник
Прикладные протоколы	Зонды uprobes
Сокеты	Точки трассировки системных вызовов
TCP	Точки трассировки TCP, зонды kprobes
UDP	Зонды kprobes
IP и ICMP	Зонды kprobes
Пакеты	Точки трассировки skb, зонды kprobes
Дисциплины очередей и очереди драйверов	Точки трассировки qdisc и net, зонды kprobes
XDP	Точки трассировки xdp
Драйверы сетевых устройств	Зонды kprobes, некоторые драйверы имеют точки трассировки

По возможности используйте точки трассировки, так как они представляют более стабильный интерфейс.

10.6.13. tcpdump

Перехватывать и исследовать пакеты в Linux можно с помощью утилиты tcpdump(8). Она может выводить сводную информацию на стандартный вывод или записывать сведения о пакетах в файл для последующего анализа. Последний способ предпочтительнее с практической точки зрения: скорость передачи пакетов может быть слишком высокой, чтобы следить за ними в реальном времени.

Вот пример вывода пакетов из интерфейса eth4 в файл в каталоге /tmp:

```
# tcpdump -i eth4 -w /tmp/out.tcpdump
tcpdump: listening on eth4, link-type EN10MB (Ethernet), capture size 65535 bytes
^C273893 packets captured
275752 packets received by filter
1859 packets dropped by kernel
```

В выводе указывается, сколько пакетов отбросило ядро вместо передачи в tcpdump(8). Такое случается, когда частота следования пакетов слишком высокая.

Обратите внимание, что утилиту можно запустить с параметром `-i any`, чтобы заставить ее перехватывать пакеты со всех интерфейсов.

Вот пример содержимого файла с информацией о пакетах:

```
# tcpdump -nr /tmp/out.tcpdump
reading from file /tmp/out.tcpdump, link-type EN10MB (Ethernet)
02:24:46.160754 IP 10.2.124.2.32863 > 10.2.203.2.5001: Flags [.], seq
3612664461:3612667357, ack 180214943, win 64436, options [nop,nop,TS val 692339741
ecr 346311608], length 2896
02:24:46.160765 IP 10.2.203.2.5001 > 10.2.124.2.32863: Flags [.], ack 2896, win
18184, options [nop,nop,TS val 346311610 ecr 692339740], length 0
02:24:46.160778 IP 10.2.124.2.32863 > 10.2.203.2.5001: Flags [.], seq 2896:4344, ack
1, win 64436, options [nop,nop,TS val 692339741 ecr 346311608], length 1448
02:24:46.160807 IP 10.2.124.2.32863 > 10.2.203.2.5001: Flags [.], seq 4344:5792, ack
1, win 64436, options [nop,nop,TS val 692339741 ecr 346311608], length 1448
02:24:46.160817 IP 10.2.203.2.5001 > 10.2.124.2.32863: Flags [.], ack 5792, win
18184, options [nop,nop,TS val 346311610 ecr 692339741], length 0
[...]
```

Каждая строка в выводе сообщает время передачи пакета (с разрешением до микро-секунд), IP-адреса отправителя и получателя, а также значения полей в заголовке ТСР. Изучая их, можно понять, как работает ТСР, в том числе насколько хорошо расширенные возможности справляются с вашей рабочей нагрузкой.

Параметр `-n` отключает разрешение IP-адресов в имена хостов. В числе других параметров можно назвать вывод подробных сведений, если они доступны (`-v`), вывод заголовков канального уровня (`-e`) и вывод содержимого в шестнадцатеричном формате (`-x` или `-X`). Например:

```
# tcpdump -enr /tmp/out.tcpdump -vvv -X
reading from file /tmp/out.tcpdump, link-type EN10MB (Ethernet)
02:24:46.160754 80:71:1f:ad:50:48 > 84:2b:2b:61:b6:ed, ethertype IPv4 (0x0800),
length 2962: (tos 0x0, ttl 63, id 46508, offset 0, flags [DF], proto TCP (6), length
2948)
    10.2.124.2.32863 > 10.2.203.2.5001: Flags [.], cksum 0x667f (incorrect ->
0xc4da), seq 3612664461:3612667357, ack 180214943, win 64436, options [nop,nop,TS val
692339741 ecr 346311608], length 289
6
    0x0000: 4500 0b84 b5ac 4000 3f06 1fbf 0a02 7c02 E.....@.?.....|.
    0x0010: 0a02 cb02 805f 1389 d754 e28d 0abd dc9f ....._...T.....
    0x0020: 8010 fbb4 667f 0000 0101 080a 2944 441d ....f.....)DD.
    0x0030: 14a4 4bb8 3233 3435 3637 3839 3031 3233 ..K.234567890123
    0x0040: 3435 3637 3839 3031 3233 3435 3637 3839 4567890123456789
[...]
```

При анализе производительности иногда полезно изменить столбец с отметкой времени так, чтобы в нем отображалась разность времени между пакетами (`-ttt`) или время, прошедшее с момента первого пакета (`-ttttt`).

Также есть возможность определить выражение для фильтрации пакетов (см. описание `pcap-filter(7)`), чтобы сосредоточиться только на интересующих пакетах. Выражение вычисляется в ядре с использованием BPF для большей эффективности (кроме Linux 2.0 и старше).

Перехват пакетов требует больших затрат процессорного времени и памяти. Если возможно, используйте tcpdump(8) только в течение коротких периодов времени, чтобы ограничить отрицательное влияние на производительность. Ищите возможности использовать более эффективные инструменты на основе BPF, например bpftrace.

tshark(1) — аналогичный инструмент командной строки для перехвата пакетов, который предлагает улучшенные возможности фильтрации и вывода. Это версия Wireshark с интерфейсом командной строки.

10.6.14. Wireshark

tcpdump(8) отлично подходит для кратковременных и бессистемных исследований, но его использование для более глубокого анализа, тем более в командной строке, может занять много времени. На этом фоне намного предпочтительнее выглядит инструмент с графическим интерфейсом Wireshark (ранее Ethereal). Он позволяет перехватывать и исследовать пакеты, а также импортировать файлы с пакетами, созданными tcpdump(8) [Wireshark 20]. В числе его полезных возможностей — идентификация сетевых подключений и связанных с ними пакетов, чтобы их можно было изучать отдельно, а также перевод сотен заголовков протоколов.

На рис. 10.12 показано, как выглядит окно Wireshark. Оно разделено по горизонтали на три части. В верхней таблице отображаются строки с информацией о пакетах.

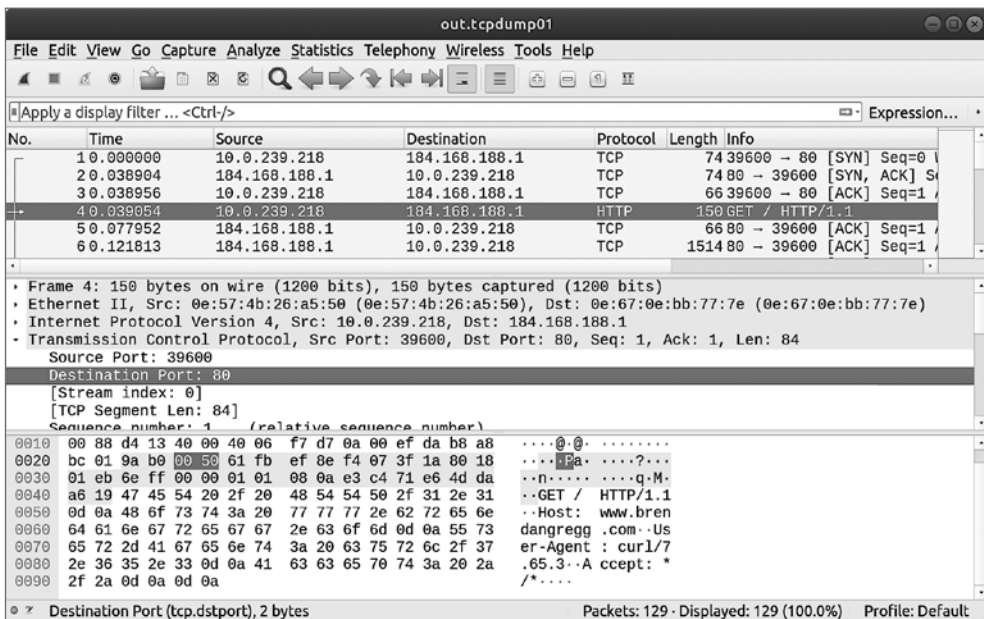


Рис. 10.12. Окно Wireshark

В средней части выводятся сведения о протоколе: в этом примере было раскрыто описание протокола TCP и выбран порт назначения. В нижней части слева показано содержимое пакета в шестнадцатеричном виде слева и справа — то же содержимое, но в текстовом виде: здесь выделены ячейки с номером порта назначения TCP.

10.6.15. Другие инструменты

В табл. 10.7 перечислены инструменты наблюдения за работой сети, представленные в других главах этой книги и в «BPF Performance Tools» [Gregg 19].

Таблица 10.7. Другие инструменты наблюдения за работой сети

Раздел	Инструмент	Описание
5.5.3	offcputime	Профилирование времени вне процессора, может профилировать операции сетевого ввода/вывода
[Gregg 19]	sockstat	Выводит обобщенную статистику по сокетам
[Gregg 19]	sofamily	Подсчет новых сокетов по семействам адресов и процессам
[Gregg 19]	soprotocol	Подсчет новых сокетов по транспортным протоколам и процессам
[Gregg 19]	soconnect	Трассирует запросы на соединение по протоколу IP
[Gregg 19]	soaccept	Трассирует прием соединений по протоколу IP
[Gregg 19]	socketio	Отображает сводную информацию о сокетах со счетчиками ввода/вывода
[Gregg 19]	socksize	Отображает объемы ввода/вывода через сокеты по процессам в виде гистограмм
[Gregg 19]	sormem	Отображает статистику использования и переполнения буферов чтения в сокетах
[Gregg 19]	soconnlat	Обобщает информацию о задержках соединения с IP-сокетами с трассировками стека
[Gregg 19]	so1stbyte	Обобщает информацию о задержках до получения первого байта
[Gregg 19]	tcpconnect	Трассирует создание активных TCP-соединений (connect())
[Gregg 19]	tcpaccept	Трассирует создание пассивных TCP-соединений (accept())
[Gregg 19]	tcpwin	Трассирует изменение параметра, определяющего размер окна насыщения
[Gregg 19]	tcpnagle	Трассирует работу алгоритма Нейгла и задержки передачи
[Gregg 19]	udpconnect	Трассирует новые локальные соединения UDP
[Gregg 19]	gethostlatency	Трассирует задержки поиска в DNS через библиотечные вызовы
[Gregg 19]	ipecn	Трассирует уведомления о перегрузке входящего IP-трафика
[Gregg 19]	superping	Измеряет время выполнения запросов ICMP ECHO
[Gregg 19]	qdisc-fq (...)	Отображает задержку в очереди с дисциплиной FQ
[Gregg 19]	netsize	Отображает объемы ввода/вывода через сетевые устройства

Раздел	Инструмент	Описание
[Gregg 19]	nettxlat	Отображает задержку передачи через сетевые устройства
[Gregg 19]	skbdrop	Трассирует события сброса буферов sk_buff и отображает соответствующие им трассировки стека ядра
[Gregg 19]	skblife	Измеряет продолжительность существования буферов sk_buff как задержки между вызовами функций сетевого стека
[Gregg 19]	ieee80211scan	Трассирует сканирование Wi-Fi согласно IEEE 802.11

Среди других инструментов для анализа работы сети в Linux можно назвать:

- **strace(1)**: для трассировки системных вызовов, обслуживающих сокеты, и исследования их параметров (имейте в виду, что **strace(1)** может создавать существенную нагрузку);
- **lsof(8)**: для получения списка файлов с некоторыми деталями, открытых процессами, в том числе и сокетами;
- **nfsstat(8)**: для получения статистик сервера и клиента NFS;
- **ifpps(8)**: для получения статистик сети и системы в стиле **top(1)**;
- **iftop(8)**: для получения обобщенной информации о пропускной способности сетевых интерфейсов (анализатор пакетов);
- **perf(1)**: для подсчета срабатываний и получения аргументов точек трассировки в сетевой подсистеме ядра;
- **/proc/net**: содержит множество файлов с информацией о сети;
- **итератор BPF**: позволяет программам для BPF экспортировать нестандартные статистики в `/sys/fs/bpf`.

Кроме того, есть множество решений для мониторинга сети, основанных либо на SNMP, либо на собственных нестандартных агентах.

10.7. ЭКСПЕРИМЕНТЫ

Производительность сети обычно исследуется с помощью инструментов, которые выполняют эксперименты, а не просто наблюдают за состоянием системы. К таким инструментам относятся **ping(8)**, **traceroute(8)** и средства микробенчмаркинга — **iperf(8)**. Их можно использовать для определения работоспособности сети между узлами и для выяснения того, обусловлена ли низкая производительность приложений недостаточной пропускной способностью сети.

10.7.1. ping

Команда **ping(8)** проверяет сетевое соединение, отправляя пакеты эхо-запроса ICMP. Например:

```
# ping www.netflix.com
PING www.netflix.com(2620:108:700f::3423:46a1 (2620:108:700f::3423:46a1)) 56 data
bytes
64 bytes from 2620:108:700f::3423:46a1 (2620:108:700f::3423:46a1): icmp_seq=1 ttl=43
time=32.3 ms
64 bytes from 2620:108:700f::3423:46a1 (2620:108:700f::3423:46a1): icmp_seq=2 ttl=43
time=34.3 ms
64 bytes from 2620:108:700f::3423:46a1 (2620:108:700f::3423:46a1): icmp_seq=3 ttl=43
time=34.0 ms
^C
--- www.netflix.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 32.341/33.579/34.389/0.889 ms
```

Команда выводит время приема/передачи (**time**) для каждого пакета и некоторую сводную информацию.

Старые версии `ping(8)` измеряли время приема/передачи в пространстве пользователя, немного увеличивая его из-за задержки планировщика. Новые ядра и версии `ping(8)` используют поддержку отметок времени ядра (`SIOCGSTAMP` или `SO_TIMESTAMP`), чтобы повысить точность отображаемых значений времени.

Пакеты ICMP могут обрабатываться маршрутизаторами с более низким приоритетом, чем прикладные протоколы, поэтому задержка может иметь более высокую дисперсию, чем обычно¹.

10.7.2. traceroute

Команда `traceroute(8)` отправляет серию тестовых пакетов для экспериментального определения текущего маршрута к хосту. Для этого в каждом следующем пакете на единицу увеличивается время жизни (**time to live, TTL**) в заголовке IP, в результате чего цепочка шлюзов на пути к хосту раскрывает себя, отправляя ICMP-сообщения о превышении времени (при условии, что брандмауэр не блокирует их).

Вот пример тестирования текущего маршрута от хоста в Калифорнии до моего сайта:

```
# traceroute www.brendangregg.com
traceroute to www.brendangregg.com (184.168.188.1), 30 hops max, 60 byte packets
 1 _gateway (10.0.0.1) 3.453 ms 3.379 ms 4.769 ms
 2 196.120.89.153 (196.120.89.153) 19.239 ms 19.217 ms 13.507 ms
 3 be-10006-rur01.sanjose.ca.sfba.comcast.net (162.151.1.145) 19.141 ms 19.102 ms
19.050 ms
 4 be-231-rar01.santaclara.ca.sfba.comcast.net (162.151.78.249) 19.018 ms 18.987
ms 18.941 ms
 5 be-299-ar01.santaclara.ca.sfba.comcast.net (68.86.143.93) 21.184 ms 18.849 ms
21.053 ms
 6 lag-14.ear3.SanJose1.Level3.net (4.68.72.105) 18.717 ms 11.950 ms 16.471 ms
 7 4.69.216.162 (4.69.216.162) 24.905 ms 4.69.216.158 (4.69.216.158) 21.705 ms
28.043 ms
```

¹ Хотя в некоторых сетях ICMP может иметь более высокий приоритет, чтобы лучше работать в бенчмарках на основе `ping`.

```

8 4.53.228.238 (4.53.228.238) 35.802 ms 37.202 ms 37.137 ms
9 ae0.ibrsa0107-01.lax1.bb.godaddy.com (148.72.34.5) 24.640 ms 24.610 ms 24.579
ms
10 148.72.32.16 (148.72.32.16) 33.747 ms 35.537 ms 33.598 ms
11 be38.trmc0215-01.ars.mgmt.phx3.gdg (184.168.0.69) 33.646 ms 33.590 ms 35.220
ms
12 * * *
13 * * *
[...]
```

Для каждого перехода выводится три значения времени приема/передачи (round-trip time, RTT), которые можно использовать для приблизительного выявления источников задержек в сети. Как и в случае с `ping(8)`, отправляемые пакеты имеют низкий приоритет и могут показывать завышенную задержку по сравнению с другими протоколами. В некоторых попытках выводятся звездочки («*»): они соответствуют случаям, когда сообщение ICMP о превышении времени жизни не было возвращено отправителю. Все три случая, в которых выводятся звездочки, могут быть результатом того, что шлюз, обслуживающий переход, вообще не возвращает сообщения ICMP или эти сообщения заблокированы брандмауэром. Более надежное решение — переключиться на TCP вместо ICMP, добавив в команду параметр `-T`. Команда с этим параметром часто доступна как `tcptraceroute(1)`. Есть более совершенная версия — `astraceroute(8)`, которая позволяет настраивать флаги.

Выявленный маршрут также можно исследовать в рамках статической настройки производительности. Сети проектируются так, чтобы могли быстро реагировать на перебои в работе и динамически менять маршруты при необходимости, поэтому производительность может снижаться из-за изменения маршрута. Обратите внимание, что маршрут может измениться даже во время выполнения `traceroute(8)`: ответ от перехода 7 в предыдущем выводе сначала возвращается с адреса 4.69.216.162, а затем с адреса 4.69.216.158. Адрес выводится, только когда меняется, в противном случае для последующих попыток выводится только время RTT.

Дополнительные сведения об интерпретации вывода `traceroute(8)` см. в [Steenbergen 09].

`traceroute(8)` был написан Ваном Якобсоном (Van Jacobson). Позже он создал удивительный инструмент под названием `pathchar`.

10.7.3. pathchar

`pathchar` — инструмент, похожий на `traceroute(8)`, но дополнительно он определяет ширину полосы пропускания между переходами. Для этого он многократно отправляет серии сетевых пакетов разного размера и выполняет статистический анализ. Вот пример вывода:

```

# pathchar 192.168.1.10
pathchar to 192.168.1.1 (192.168.1.1)
doing 32 probes at each of 64 to 1500 by 32
```

```

0 localhost
|   30 Mb/s, 79 us (562 us)
1 neptune.test.com (192.168.2.1)
|   44 Mb/s, 195 us (1.23 ms)
2 mars.test.com (192.168.1.1)
2 hops, rtt 547 us (1.23 ms), bottleneck 30 Mb/s, pipe 7555 bytes

```

К сожалению, pathchar не пользуется популярностью (возможно, из-за того, что не был открыт исходный код), и у него имеются сложности с запуском оригинальной версии (самый свежий двоичный файл для Linux 2.0.30 был выложен на сайте pathchar в 1997 году [Jacobson 97]). Новая версия, созданная Брюсом А. Ма (Bruce A. Mah) и названная rchar(8), более доступна. Кроме того, запуск pathchar занимал очень много времени, порой десятки минут, в зависимости от количества переходов, но позднее были предложены методы, позволяющие сократить это время [Downey 99].

10.7.4. iperf

iperf(1) — это инструмент с открытым исходным кодом для тестирования максимальной пропускной способности TCP и UDP. Он поддерживает множество возможностей, включая параллельный режим, в котором запускается несколько клиентских потоков, что может потребоваться для определения пределов возможностей сети. iperf(1) должен выполняться как на сервере, так и на клиенте.

Вот пример запуска iperf(1) на сервере:

```

$ iperf -s -l 128k
-----
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
-----

```

Эта команда увеличила размер буфера сокета до 128 Кбайт (-l 128k) со значения по умолчанию 8 Кбайт.

А вот результаты, полученные на стороне клиента:

```

# iperf -c 10.2.203.2 -l 128k -P 2 -i 1 -t 60
-----
Client connecting to 10.2.203.2, TCP port 5001
TCP window size: 48.0 KByte (default)
-----
[  4] local 10.2.124.2 port 41407 connected with 10.2.203.2 port 5001
[  3] local 10.2.124.2 port 35830 connected with 10.2.203.2 port 5001
[ ID] Interval      Transfer    Bandwidth
[  4] 0.0- 1.0 sec  6.00 MBytes  50.3 Mbits/sec
[  3] 0.0- 1.0 sec  22.5 MBytes  189 Mbits/sec
[SUM] 0.0- 1.0 sec  28.5 MBytes  239 Mbits/sec
[  3] 1.0- 2.0 sec  16.1 MBytes  135 Mbits/sec
[  4] 1.0- 2.0 sec  12.6 MBytes  106 Mbits/sec
[SUM] 1.0- 2.0 sec  28.8 MBytes  241 Mbits/sec
[...]
```

```
[ 4] 0.0-60.0 sec   748 MBytes   105 Mbits/sec
[ 3] 0.0-60.0 sec   996 MBytes   139 Mbits/sec
[SUM] 0.0-60.0 sec  1.70 GBytes   244 Mbits/sec
```

В этой команде использованы следующие параметры:

- **-c host**: имя хоста или IP-адрес для подключения;
- **-l 128k**: использовать для сокета буфер размером 128 Кбайт;
- **-P 2**: запустить в параллельном режиме два клиентских потока;
- **-i 1**: выводить статистики с интервалом в одну секунду;
- **-t 60**: общая продолжительность тестирования: 60 с.

Последняя строка сообщает среднюю пропускную способность за все время тестирования по всем параллельным потокам: 244 Мбит/с.

Чтобы увидеть, как меняются статистики с течением времени, можно выводить их через определенный интервал. Параметр **--reportstyle C** позволяет вывести данные в формате CSV, чтобы затем их можно было импортировать в другие инструменты, например средства для построения графиков.

10.7.5. netperf

netperf(1) — усовершенствованный инструмент для микробенчмаркинга, позволяющий оценить производительность запросов/ответов [HP 18]. Я использую **netperf(1)** для измерения задержки передачи пакетов TCP туда и обратно. Вот пример вывода:

```
server$ netserver -D -p 7001
Starting netserver with host 'IN(6)ADDR_ANY' port '7001' and family AF_UNSPEC
[...]
```

```
client$ netperf -v 100 -H 100.66.63.99 -t TCP_RR -p 7001
MIGRATED TCP REQUEST/RESPONSE TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to
100.66.63.99 () port 0 AF_INET : demo : first burst 0
Alignment      Offset      RoundTrip  Trans      Throughput
Local Remote  Local Remote  Latency   Rate      10^6bits/s
Send  Recv    Send  Recv    usec/Tran  per sec   Outbound  Inbound
   8     0      0     0    98699.102   10.132  0.000    0.000
```

Как показывает этот вывод, задержка приема/передачи пакетов TCP составляет 98,7 мс.

10.7.6. tc

Утилита управления трафиком **tc(8)** позволяет выбирать различные дисциплины очередей (qdiscs) для управления производительностью. Для экспериментов есть дисциплины, которые могут ограничивать или случайным образом менять производительность, что может пригодиться для тестирования и моделирования. В этом разделе представлена дисциплина **netem** (сетевой эмулятор).

Для начала представлю команду, которая отображает текущую дисциплину очередей для интерфейса `eth0`:

```
# tc qdisc show dev eth0
qdisc noqueue 0: root refcnt 2
```

Теперь добавим дисциплину `netem`. Каждая дисциплина имеет свои настраиваемые параметры. В этом примере я использую параметр для дисциплины `netem`, управляющий количеством теряемых пакетов, и установлю его равным 1 %:

```
# tc qdisc add dev eth0 root netem loss 1%
# tc qdisc show dev eth0
qdisc netem 8001: root refcnt 2 limit 1000 loss 1%
```

После этой операции сетевого ввода/вывода с `eth0` будут терять 1 % пакетов.

Параметр `-s` показывает статистики:

```
# tc -s qdisc show dev eth0
qdisc netem 8001: root refcnt 2 limit 1000 loss 1%
  Sent 75926119 bytes 89538 pkt (dropped 917, overlimits 0 requeues 0)
  backlog 0b 0p requeues 0
```

Здесь видно количество отброшенных пакетов.

Чтобы удалить дисциплину, достаточно выполнить команду:

```
# tc qdisc del dev eth0 root
# tc qdisc show dev eth0
qdisc noqueue 0: root refcnt 2
```

Полный перечень параметров можно найти на странице справочного руководства `man` для соответствующей дисциплины (для `netem` страница называется `tc-netem(8)`).

10.7.7. Другие инструменты

Другие инструменты для экспериментов включают:

- **pktgen**: генератор пакетов, включенный в ядро Linux [Linux 20];
- **Flent**: FLExible Network Tester запускает несколько микробенчмарков и выводит результаты в виде графика [Høiland-Jørgensen 20];
- **mtr(8)**: инструмент, похожий на `traceroute`, который включает статистики проверки связи (`ping`);
- **tcpreplay(1)**: воспроизводит ранее перехваченный (с помощью `tcpdump(8)`) сетевой трафик, вплоть до интервалов времени между пакетами. Этот инструмент предназначен скорее для общей отладки, чем для тестирования производительности, но иногда проблемы с производительностью возникают только с определенными последовательностями пакетов или битовыми шаблонами, и этот инструмент помогает воспроизвести их.

10.8. НАСТРОЙКА

Настраиваемые параметры сети обычно уже имеют значения, обеспечивающие высокую производительность. Кроме того, предполагается, что сетевой стек должен динамически реагировать на различные рабочие нагрузки, обеспечивая оптимальную производительность.

Прежде чем пробовать изменять настраиваемые параметры, полезно сначала понять, как используется сеть. Этот шаг помогает выявлять ненужную работу и исключать ее, что часто дает гораздо больший выигрыш в производительности. Попробуйте применить методологии определения характеристик рабочей нагрузки и статической настройки производительности с помощью инструментов, описанных в предыдущем разделе.

Круг доступных для настройки параметров зависит от версии ОС, поэтому дополнительную информацию ищите в соответствующей документации. Разделы ниже дают общее представление о доступных параметрах и особенностях их настройки. Рассматривайте их как отправную точку для исследований в зависимости от конкретной рабочей нагрузки и среды.

10.8.1. Общесистемные

Общесистемные настраиваемые параметры в Linux можно просмотреть и изменить с помощью команды `sysctl(8)` и сохранить в `/etc/sysctl.conf`. Их также можно читать и изменять с помощью файловой системы `/proc` — в каталоге `/proc/sys/net`.

Например, чтобы увидеть текущие настройки TCP, выполните поиск в выводе `sysctl(8)` по тексту «tcp»:

```
# sysctl -a | grep tcp
net.ipv4.tcp_abort_on_overflow = 0
net.ipv4.tcp_adv_win_scale = 1
net.ipv4.tcp_allowed_congestion_control = reno cubic
net.ipv4.tcp_app_win = 31
net.ipv4.tcp_autocorking = 1
net.ipv4.tcp_available_congestion_control = reno cubic
net.ipv4.tcp_available_ulp =
net.ipv4.tcp_base_mss = 1024
net.ipv4.tcp_challenge_ack_limit = 1000
net.ipv4.tcp_comp_sack_delay_ns = 1000000
net.ipv4.tcp_comp_sack_nr = 44
net.ipv4.tcp_congestion_control = cubic
net.ipv4.tcp_dsack = 1
[...]
```

В этом ядре (5.3) 70 параметров, в названиях которых есть символы «tcp», и еще множество параметров в разделе «net», в том числе параметры настройки для IP, Ethernet, маршрутизации и сетевых интерфейсов.

Некоторые из них можно настраивать отдельно для каждого сокета. Например, `net.ipv4.tcp_congestion_control` — это общесистемный алгоритм по умолчанию

управления перегрузкой, который можно назначить для каждого сокета с помощью параметра `TCP_CONGESTION` (см. раздел 10.8.2 «Параметры сокетов»).

Пример из промышленной среды

Ниже показаны настройки, используемые в Netflix для облачных экземпляров [Gregg 19c]. Они устанавливаются в сценарии запуска во время загрузки:

```
net.core.default_qdisc = fq
net.core.netdev_max_backlog = 5000
net.core.rmem_max = 16777216
net.core.somaxconn = 1024
net.core.wmem_max = 16777216
net.ipv4.ip_local_port_range = 10240 65535
net.ipv4.tcp_abort_on_overflow = 1
net.ipv4.tcp_congestion_control = bbr
net.ipv4.tcp_max_syn_backlog = 8192
net.ipv4.tcp_rmem = 4096 12582912 16777216
net.ipv4.tcp_slow_start_after_idle = 0
net.ipv4.tcp_syn_retries = 2
net.ipv4.tcp_tw_reuse = 1
net.ipv4.tcp_wmem = 4096 12582912 16777216
```

Здесь перечислены настройки только 14 параметров из доступных, и это лишь пример на определенный момент времени, а не рецепт. В течение 2020 года в Netflix предполагается изменить два из них (параметру `net.core.netdev_max_backlog` присвоить значение 1000 и параметру `net.core.somaxconn` — значение 4096)¹, но пока эти изменения ожидают нерегрессионного тестирования.

Далее мы более конкретно рассмотрим некоторые параметры.

Буферы сокетов и TCP

Максимальный размер буфера сокета для всех протоколов, как для чтения (`rmem_max`), так и для записи (`wmem_max`), можно установить с помощью параметров:

```
net.core.rmem_max = 16777216
net.core.wmem_max = 16777216
```

Значения задаются в байтах. Размер, равный 16 Мбайт или выше, может потребоваться установить для высокоскоростных подключений 10 GbE.

Включение автоматической настройки буфера приема для TCP:

```
net.ipv4.tcp_moderate_rcvbuf = 1
```

¹ Спасибо Даниэлю Боркманну (Daniel Borkmann) за предложения, сделанные во время рецензирования этой книги. Эти новые значения уже используются в Google [Dumazet 17b], [Dumazet 19].

Установка параметров автоматической настройки для буферов чтения и записи TCP:

```
net.ipv4.tcp_rmem = 4096 87380 16777216
net.ipv4.tcp_wmem = 4096 65536 16777216
```

Оба параметра имеют три значения: минимальный размер, размер по умолчанию и максимальный размер в байтах. Первоначально автоматически устанавливается размер по умолчанию. Чтобы увеличить пропускную способность TCP, можно попробовать увеличить максимальный размер. Увеличение минимального размера и размера по умолчанию приведет к выделению большего объема памяти для каждого соединения, чего может не потребоваться.

Очередь входящих запросов на соединение TCP

Всего есть две очереди. Первая предназначена для полуоткрытых соединений:

```
net.ipv4.tcp_max_syn_backlog = 4096
```

Вторая — для передачи запросов в accept(2):

```
net.core.somaxconn = 1024
```

Иногда нужно увеличить значения по умолчанию для обоих параметров, например, до 4096 и 1024 или выше, чтобы система могла лучше справляться со всплесками нагрузки.

Очередь устройства

Увеличение длины очереди входящих запросов на соединение для сетевого устройства на каждый процессор:

```
net.core.netdev_max_backlog = 10000
```

Для сетевых карт 10 GbE значение по умолчанию может потребоваться увеличить до 10 000.

Управление перегрузкой TCP

Linux поддерживает возможность выбора алгоритмов управления перегрузкой. Вот список алгоритмов, доступных сейчас:

```
# sysctl net.ipv4.tcp_available_congestion_control
net.ipv4.tcp_available_congestion_control = reno cubic
```

Некоторые из них могут быть доступны, но в текущий момент не загружены. Вот пример добавления алгоритма httpc:

```
# modprobe tcp_htcp
# sysctl net.ipv4.tcp_available_congestion_control
net.ipv4.tcp_available_congestion_control = reno cubic httpc
```

Назначить алгоритм текущим можно с помощью параметра:

```
net.ipv4.tcp_congestion_control = cubic
```

Параметры TCP

Вот еще несколько параметров TCP, которые можно изменить:

```
net.ipv4.tcp_sack = 1
net.ipv4.tcp_fack = 1
net.ipv4.tcp_tw_reuse = 1
net.ipv4.tcp_tw_recycle = 0
```

Расширения SACK и FACK могут улучшить пропускную способность в сетях с высокой задержкой за счет некоторой нагрузки на процессор.

Параметр `tcp_tw_reuse` разрешает повторно использовать сеансы в состоянии `TIME_WAIT`, когда это кажется безопасным. Это может ускорить создание соединений между двумя хостами, например между веб-сервером и базой данных, без риска исчерпать 16-битный диапазон эфемерных портов с сеансами в состоянии `TIME_WAIT`.

`tcp_tw_recycle` — еще один параметр, позволяющий повторно использовать сеансы в состоянии `TIME_WAIT`, хотя и не такой безопасный, как `tcp_tw_reuse`.

ECN

Управлять явными уведомлениями о перегрузке (explicit congestion notification, ECN) можно с помощью параметра:

```
net.ipv4.tcp_ecn = 1
```

Значение 0 отключает ECN, значение 1 разрешает входящие соединения и требует запрашивать ECN для исходящих соединений, значение 2 разрешает входящие соединения и не требует запрашивать ECN для исходящих. Значение по умолчанию: 2.

Есть и параметр `net.ipv4.tcp_ecn_fallback` со значением по умолчанию 1 (true), который отключает ECN для соединений, если ядро обнаружит, что механизм уведомлений работает некорректно.

Byte Queue Limits

Этот алгоритм поддерживает настройку через `/sys`. Вот как можно просмотреть содержимое управляющих файлов для этого алгоритма (имя каталога, усеченное в этом выводе, у вас будет отличаться):

```
# grep . /sys/devices/pci.../net/ens5/queues/tx-0/byte_queue_limits/limit*
/sys/devices/pci.../net/ens5/queues/tx-0/byte_queue_limits/limit:16654
/sys/devices/pci.../net/ens5/queues/tx-0/byte_queue_limits/limit_max:1879048192
/sys/devices/pci.../net/ens5/queues/tx-0/byte_queue_limits/limit_min:0
```

Ограничение для этого интерфейса составляет 16 654 байта и установлено автоматической настройкой. Чтобы ограничить допустимый диапазон, установите параметры `limit_min` и `limit_max`.

Управление ресурсами

В системе контрольных групп (`sgroups`) есть подсистема управления приоритетом сетевого трафика (`net_prio`), которая позволяет задавать приоритет исходящего сетевого трафика для процессов или групп процессов. С ее помощью можно дать преимущество высокоприоритетному сетевому трафику, например промышленной нагрузке, перед низкоприоритетным, таким как резервное копирование или мониторинг. Есть и подсистема сетевого классификатора (`net_cls`) для маркировки пакетов, принадлежащих контрольной группе, идентификатором класса: эти идентификаторы могут использоваться дисциплиной очереди для ограничения пакетов или полосы пропускания, а также программами BPF. Программы BPF могут также использовать другую информацию, такую как идентификатор `sgroup v2`, для получения сведений о контейнерах и могут улучшить масштабируемость за счет переноса классификации, оценки и перемаркировки в обработчик исходящего трафика `tc` и уменьшения давления на корневую блокировку дисциплины очереди [Fomichev 20].

Дополнительную информацию об управлении ресурсами ищите в подразделе «Сетевой ввод/вывод» в разделе 11.3.3 «Управление ресурсами», в главе 11 «Облачные вычисления».

Дисциплины очередей

Дисциплины очередей (`qdiscs`), описанные в разделе 10.4.3 «Программное обеспечение» и изображенные на рис. 10.8, — это алгоритмы планирования, управления, фильтрации и формирования сетевых пакетов. В разделе 10.7.6 «`tc`» было показано применение дисциплины `netem` для обеспечения потери пакетов. Но есть дисциплины, помогающие улучшить производительность разных рабочих нагрузок. Получить список дисциплин, доступных в системе, можно командой:

```
# man -k tc-
```

Для каждой дисциплины есть своя страница в справочном руководстве `man`. Дисциплины можно использовать для управления частотой передачи пакетов или шириной полосы пропускания, для установки флагов IP ECN и т. д.

Узнать, какая дисциплина используется по умолчанию, можно командой:

```
# sysctl net.core.default_qdisc
net.core.default_qdisc = fq_codel
```

Во многих дистрибутивах Linux уже выполнен переход на использование по умолчанию дисциплины `fq_codel`, которая в большинстве случаев обеспечивает хорошую производительность.

Проект Tuned

Из-за большого количества доступных настраиваемых параметров работать с ними сложно. Проект Tuned позволяет автоматизировать настройку некоторых из них на основе выбираемых профилей и поддерживает дистрибутивы Linux, включая RHEL, Fedora, Ubuntu и CentOS [Tuned Project 20]. После установки Tuned можно получить список доступных профилей, выполнив команду:

```
# tuned-adm list
Доступные профили:
[...]
- balanced                - Общий неспециализированный профиль tuned
[...]
- network-latency         - Оптимизированный для получения предсказуемой
производительности за счет увеличения энергопотребления с упором на низкую
задержку в сети
- network-throughput      - Оптимизированный для получения высокой пропускной
способности, обычно требуется только на старых процессорах или в сетях 40G+1
[...]
```

Это неполный список, всего есть 28 профилей. Активировать профиль network-latency, например, можно командой:

```
# tuned-adm profile network-latency
```

Чтобы узнать, какие настройки определены в этом профиле, можно заглянуть в файл конфигурации в исходных кодах tuned [Škarvada 20]:

```
$ more tuned/profiles/network-latency/tuned.conf
[...]
[main]
summary=Optimize for deterministic performance at the cost of increased power
consumption, focused on low latency network performance
include=latency-performance

[vm]
transparent_hugepages=never

[sysctl]
net.core.busy_read=50
net.core.busy_poll=50
net.ipv4.tcp_fastopen=3
kernel.numa_balancing=0

[bootloader]
cmdline_network_latency=skew_tick=1
```

Обратите внимание на директиву include, которая подключает настройки из профиля latency-performance.

¹ Это перевод вывода, который вы получите. — *Примеч. пер.*

10.8.2. Параметры сокетов

Сокеты можно настраивать по отдельности внутри приложений с помощью системного вызова `setsockopt(2)`. Но это реально, только если приложение ваше или вам доступен его исходный код и вы можете изменять его¹.

`setsockopt(2)` позволяет настраивать разные уровни (например, сокеты, протокол TCP). Некоторые из возможностей, доступных в Linux, перечислены в табл. 10.8.

Таблица 10.8. Настраиваемые параметры сокетов

Параметр	Описание
SO_SNDBUF, SO_RCVBUF	Размеры буфера передачи и приема (можно указать значения вплоть до системных пределов, описанных выше. Есть и параметр SO_SNDBUF - FORCE, настраивающий возможность превышения ограничения на размер буфера передачи)
SO_REUSEPORT	Позволяет нескольким процессам или потокам использовать один и тот же порт, поэтому ядро может распределять нагрузку между ними для масштабируемости (начиная с Linux 3.9)
SO_MAX_PACING_RATE	Позволяет установить максимальную скорость обработки пактов на транспортном уровне в байтах в секунду (см. описание <code>tc-fq(8)</code>)
SO_LINGER	Может использоваться для уменьшения времени пребывания сеансов в состоянии TIME_WAIT
SO_TXTIME	Позволяет запрашивать передачу пакетов в определенные интервалы времени (начиная с Linux 4.19) [Corbet 18c] (используется для регулирования UDP [Bruijn 18])
TCP_NODELAY	Отключает алгоритм Нейгла, отправляя сегменты при первой возможности. Это может уменьшить задержку за счет более высокого потребления сети (большого количества пакетов)
TCP_CORK	Приостанавливает передачу до тех пор, пока не будут отправлены полные пакеты, что повышает пропускную способность. (Есть и общесистемная настройка ядра для автоматической приостановки: <code>net.ipv4.tcp_autocorking</code>)
TCP_QUICKACK	Отправляет пакеты ACK немедленно (это может увеличить пропускную способность отправки)
TCP_CONGESTION	Алгоритм управления перегрузкой для сокетов

Описание доступных параметров сокетов ищите на страницах справочного руководства `man` для `socket(7)`, `tcp(7)`, `udp(7)` и т. д.

Системные вызовы ввода/вывода через сокеты поддерживают некоторые флаги, которые тоже могут влиять на производительность. Например, в Linux 4.14

¹ Есть несколько опасных способов изменить выполняемый двоичный файл, но с моей стороны было бы безответственно приводить их здесь.

в системный вызов `send(2)` добавили поддержку флага `MSG_ZEROCOPY`: он позволяет использовать во время передачи буфер в пространстве пользователя, чтобы избежать затрат на его копирование в пространство ядра¹ [Linux 20c].

10.8.3. Конфигурация

Для настройки производительности сети могут быть доступны следующие параметры конфигурации:

- **Jumbo-фреймы Ethernet**: увеличение размера MTU до ~9000 может повысить производительность сети, если сетевая инфраструктура поддерживает jumbo-фреймы.
- **Агрегирование каналов**: несколько сетевых интерфейсов можно сгруппировать, чтобы они работали как один интерфейс с объединенной полосой пропускания. Для этого требуется поддержка и настройка коммутатора.
- **Конфигурация брандмауэра**: правила iptables или программы BPF, обслуживающие исходящий трафик, можно использовать для установки уровня IP ToS (DSCP) в заголовках IP и, например, задавать приоритеты трафика на основе номера порта или использовать для других целей.

10.9. УПРАЖНЕНИЯ

1. Ответьте на следующие вопросы, касающиеся терминологии:
 - В чем разница между шириной полосы пропускания и пропускной способностью?
 - Что такое задержка подключения TCP?
 - Что такое задержка первого байта?
 - Что такое время приема/передачи?
2. Ответьте на следующие концептуальные вопросы:
 - Опишите такие характеристики сетевого интерфейса, как потребление и насыщение.
 - Что такое очередь установленных соединений TCP и как она используется?
 - Опишите плюсы и минусы объединения прерываний.
3. Ответьте на следующие, более сложные вопросы:
 - Объясните, как ошибка сетевого фрейма (или пакета) может повлиять на скорость создания TCP-соединения.

¹ При использовании `MSG_ZEROCOPY` недостаточно просто установить флаг: системный вызов `send(2)` может вернуть управление еще до того, как данные будут отправлены, поэтому отправляющее приложение должно дождаться уведомления ядра, прежде чем освободить или повторно использовать память, занятую буфером передачи.

- Опишите, что происходит, когда сетевой интерфейс оказывается перегружен, и как происходящее влияет на производительность приложения.
4. Разработайте следующие процедуры для своей ОС:
 - Чек-лист анализа потребления сетевых ресурсов (интерфейсов и контроллеров) для метода USE. Опишите, как получить каждую метрику (например, какую команду выполнить) и как интерпретировать результаты. Перед установкой или использованием дополнительных программных продуктов постарайтесь ограничиться инструментами наблюдения, которые есть в ОС.
 - Чек-лист для определения характеристик рабочей нагрузки, потребляющей сетевые ресурсы. Опишите, как получить каждую метрику, и для начала попробуйте ограничиться инструментами наблюдения, имеющимися в ОС.
 5. Решите следующие задачи (может потребоваться использовать средства и методы динамической трассировки):
 - Измерьте задержку первого байта для исходящих (активных) TCP-соединений.
 - Измерьте задержку подключения TCP. Сценарий должен учитывать неблокирующие вызовы `connect(2)`.
 6. (опционально) Измерьте задержку стека TCP/IP для приема и передачи. Для приема — время от прерывания до чтения из сокета; для передачи — от записи в сокет до передачи в устройство. Протестируйте под нагрузкой. Можно ли использовать какую-либо дополнительную информацию, чтобы объяснить причину выбросов по задержке?

10.10. ССЫЛКИ

[Postel 80] Postel, J., «RFC 768: User Datagram Protocol», Information Sciences Institute, <https://tools.ietf.org/html/rfc768>, 1980¹.

[Postel 81] Postel, J., «RFC 793: Transmission Control Protocol», Information Sciences Institute, <https://tools.ietf.org/html/rfc793>, 1981².

[Nagle 84] Nagle, J., «RFC 896: Congestion Control in IP/TCP Internetworks», <https://tools.ietf.org/html/rfc896>, 1984³.

[Saltzer 84] Saltzer, J., Reed, D., and Clark, D., «End-to-End Arguments in System Design», ACM TOCS, November 1984.

[Braden 89] Braden, R., «RFC 1122: Requirements for Internet Hosts — Communication Layers», <https://tools.ietf.org/html/rfc1122>, 1989⁴.

¹ Перевод на русский язык: <https://www.protocols.ru/WP/rfc768/>. — *Примеч. пер.*

² Перевод на русский язык: <https://www.protocols.ru/WP/rfc793/>. — *Примеч. пер.*

³ Перевод на русский язык: <https://www.protocols.ru/WP/rfc896/>. — *Примеч. пер.*

⁴ Перевод на русский язык: <https://www.protocols.ru/WP/rfc1122/>. — *Примеч. пер.*

- [Jacobson 92] Jacobson, V., et al., «TCP Extensions for High Performance», Network Working Group, <https://tools.ietf.org/html/rfc1323>, 1992¹.
- [Stevens 93] Stevens, W. R., TCP/IP Illustrated, Volume 1, Addison-Wesley, 1993².
- [Mathis 96] Mathis, M., and Mahdavi, J., «Forward Acknowledgement: Refining TCP Congestion Control», ACM SIGCOMM, 1996.
- [Jacobson 97] Jacobson, V., «pathchar-a1-linux-2.0.30.tar.gz», <ftp://ftp.ee.lbl.gov/pathchar>, 1997.
- [Nichols 98] Nichols, K., Blake, S., Baker, F., and Black, D., «Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers», Network Working Group, <https://tools.ietf.org/html/rfc2474>, 1998³.
- [Downey 99] Downey, A., «Using pathchar to Estimate Internet Link Characteristics», ACM SIGCOMM, October 1999.
- [Ramakrishnan 01] Ramakrishnan, K., Floyd, S., and Black, D., «The Addition of Explicit Congestion Notification (ECN) to IP», Network Working Group, <https://tools.ietf.org/html/rfc3168>, 2001⁴.
- [Corbet 03] Corbet, J., «Driver porting: Network drivers», LWN.net, <https://lwn.net/Articles/30107>, 2003.
- [Hassan 03] Hassan, M., and R. Jain., High Performance TCP/IP Networking, Prentice Hall, 2003.
- [Deri 04] Deri, L., «Improving Passive Packet Capture: Beyond Device Polling», Proceedings of SANE, 2004.
- [Corbet 06b] Corbet, J., «Reworking NAPI», LWN.net, <https://lwn.net/Articles/214457>, 2006.
- [Cook 09] Cook, T., «nicstat — the Solaris and Linux Network Monitoring Tool You Did Not Know You Needed», https://blogs.oracle.com/timc/entry/nicstat_the_solaris_and_linux, 2009.
- [Steenbergen 09] Steenbergen, R., «A Practical Guide to (Correctly) Troubleshooting with Traceroute», https://archive.nanog.org/meetings/nanog47/presentations/Sunday/RAS_Traceroute_N47_Sun.pdf, 2009.
- [Paxson 11] Paxson, V., Allman, M., Chu, J., and Sargent, M., «RFC 6298: Computing TCP's Retransmission Timer», Internet Engineering Task Force (IETF), <https://tools.ietf.org/html/rfc6298>, 2011⁵.
- [Corbet 12] «TCP friends», LWN.net, <https://lwn.net/Articles/511254>, 2012.
- [Fritchie 12] Fritchie, S. L., «TCP incast: What is it?», <http://www.snookles.com/slf-blog/slf-blog/2012/01/05/tcp-incast-what-is-it/>, 2012.
- [Hrubý 12] Hrubý, T., «Byte Queue Limits», Linux Plumber's Conference, https://blog.linuxplumbersconf.org/2012/wp-content/uploads/2012/08/bql_slide.pdf, 2012.

¹ Перевод на русский язык: <https://www.protocols.ru/WP/rfc1323/>. — Примеч. пер.

² У. Ричард Стивенс. «Протоколы TCP/IP. Практическое руководство».

³ Перевод на русский язык: <https://www.protocols.ru/WP/rfc2474/>. — Примеч. пер.

⁴ Перевод на русский язык: <https://www.protocols.ru/WP/rfc3168/>. — Примеч. пер.

⁵ Перевод на русский язык: <https://www.protocols.ru/WP/rfc6298/>. — Примеч. пер.

- [Nichols 12] Nichols, K., and Jacobson, V., «Controlling Queue Delay», Communications of the ACM, July 2012.
- [Roskind 12] Roskind, J., «QUIC: Quick UDP Internet Connections», https://docs.google.com/document/d/1RNNHkx_VvKWYWg6Lr8SZ-saqsQx7rFV-ev2jRFUoVD34/edit#, 2012.
- [Dukkipati 13] Dukkipati, N., Cardwell, N., Cheng, Y., and Mathis, M., «Tail Loss Probe (TLP): An Algorithm for Fast Recovery of Tail Losses», TCP Maintenance Working Group, <https://tools.ietf.org/html/draft-dukkipati-tcpm-tcp-loss-probe-01>, 2013.
- [Siemon 13] Siemon, D., «Queueing in the Linux Network Stack», <https://www.coverfire.com/articles/queueing-in-the-linux-network-stack>, 2013.
- [Cheng 16] Cheng, Y., and Cardwell, N., «Making Linux TCP Fast», netdev 1.2, <https://netdevconf.org/1.2/papers/bbr-netdev-1.2.new.new.pdf>, 2016.
- [Linux 16] «TCP Offload Engine (TOE)», <https://wiki.linuxfoundation.org/networking/toe>, 2016.
- [Ather 17] Ather, A., «BBR TCP congestion control offers higher network utilization and throughput during network congestion (packet loss, latencies)», <https://twitter.com/amernetflix/status/892787364598132736>, 2017.
- [Bensley 17] Bensley, S., et al., «Data Center TCP (DCTCP): TCP Congestion Control for Data Centers», Internet Engineering Task Force (IETF), <https://tools.ietf.org/html/rfc8257>, 2017.
- [Dumazet 17a] Dumazet, E., «Busy Polling: Past, Present, Future», netdev 2.1, <https://netdevconf.info/2.1/slides/apr6/dumazet-BUSY-POLLING-Netdev-2.1.pdf>, 2017.
- [Dumazet 17b] Dumazet, E., «Re: Something hitting my total number of connections to the server», netdev mailing list, <https://lore.kernel.org/netdev/1503423863.2499.39.camel@edumazet-glaptop3.roam.corp.google.com>, 2017.
- [Gallatin 17] Gallatin, D., «Serving 100 Gbps from an Open Connect Appliance», Netflix Technology Blog, <https://netflixtechblog.com/serving-100-gbps-from-an-open-connect-appliance-cdb51dda3b99>, 2017.
- [Bruijn 18] Bruijn, W., and Dumazet, E., «Optimizing UDP for Content Delivery: GSO, Pacing and Zero-copy», Linux Plumber's Conference, http://vger.kernel.org/lpc_net2018_talks/willemdebruijn-lpc2018-udpgso-paper-DRAFT-1.pdf, 2018.
- [Corbet 18b] Corbet, J., «Zero-copy TCP receive», LWN.net, <https://lwn.net/Articles/752188>, 2018.
- [Corbet 18c] Corbet, J., «Time-based packet transmission», LWN.net, <https://lwn.net/Articles/748879>, 2018.
- [Deepak 18] Deepak, A., «eBPF / XDP firewall and packet filtering», Linux Plumber's Conference, http://vger.kernel.org/lpc_net2018_talks/ebpf-firewall-LPC.pdf, 2018.
- [Jacobson 18] Jacobson, V., «Evolving from AFAP: Teaching NICs about Time», netdev 0x12, July 2018, <https://www.files.netdevconf.org/d/4ee0a09788fe49709855/files/?p=/Evolving%20from%20AFAP%20%E2%80%93%20Teaching%20NICs%20about%20time.pdf>, 2018.
- [Høiland-Jørgensen 18] Høiland-Jørgensen, T., et al., «The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel», Proceedings of the 14th International Conference on emerging Networking EXperiments and Technologies, 2018.
- [HP 18] «Netperf», <https://github.com/HewlettPackard/netperf>, 2018.

- [Majkowski 18] Majkowski, M., «How to Drop 10 Million Packets per Second», <https://blog.cloudflare.com/how-to-drop-10-million-packets>, 2018.
- [Stewart 18] Stewart, R., «This commit brings in a new refactored TCP stack called Rack», <https://reviews.freebsd.org/rS334804>, 2018.
- [Amazon 19] «Announcing Amazon VPC Traffic Mirroring for Amazon EC2 Instances», <https://aws.amazon.com/about-aws/whats-new/2019/06/announcing-amazon-vpc-traffic-mirroring-for-amazon-ec2-instances>, 2019.
- [Dumazet 19] Dumazet, E., «Re: [LKP] [net] 19f92a030c: apachebench.requests_per_second -37.9% regression», netdev mailing list, <https://lore.kernel.org/lkml/20191113172102.GA23306@1wt.eu>, 2019.
- [Gregg 19] Gregg, B., «BPF Performance Tools: Linux System and Application Observability»¹, Addison-Wesley, 2019.
- [Gregg 19b] Gregg, B., «BPF Theremin, Tetris, and Typewriters», <http://www.brendangregg.com/blog/2019-12-22/bpf-theremin.html>, 2019.
- [Gregg 19c] Gregg, B., «LISA2019 Linux Systems Performance», USENIX LISA, <http://www.brendangregg.com/blog/2020-03-08/lisa2019-linux-systems-performance.html>, 2019.
- [Gregg 19d] Gregg, B., «udplife.bt», https://github.com/brendangregg/bpf-perf-tools-book/blob/master/exercises/Ch10_Networking/udplife.bt, 2019.
- [Hassas Yeganeh 19] Hassas Yeganeh, S., and Cheng, Y., «TCP SO_TIMESTAMPING with OPT_STATS for Performance Analytics», netdev 0x13, <https://netdevconf.info/0x13/session.html?talk-tcp-timestamping>, 2019.
- [Bufferbloat 20] «Bufferbloat», <https://www.bufferbloat.net>, 2020.
- [Cheng 20] Cheng, Y., Cardwell, N., Dukkupati, N., and Jha, P., «RACK-TLP: A Time-Based Efficient Loss Detection for TCP», TCP Maintenance Working Group, <https://tools.ietf.org/html/draft-ietf-tcpm-rack-09>, 2020.
- [Cilium 20a] «API-aware Networking and Security», <https://cilium.io>, по состоянию на 2020.
- [Corbet 20] Corbet, J., «Kernel operations structures in BPF», LWN.net, <https://lwn.net/Articles/811631>, 2020.
- [DPDK 20] «AF_XDP Poll Mode Driver», DPDK documentation, <http://doc.dpdk.org/guides/index.html>, по состоянию на 2020.
- [Fomichev 20] Fomichev, S., et al., «Replacing HTB with EDT and BPF», netdev 0x14, <https://netdevconf.info/0x14/session.html?talk-replacing-HTB-with-EDT-and-BPF>, 2020.
- [Google 20b] «Packet Mirroring Overview», <https://cloud.google.com/vpc/docs/packetmirroring>, по состоянию на 2020.
- [Høiland-Jørgensen 20] Høiland-Jørgensen, T., «The FLExible Network Tester», <https://flent.org>, по состоянию на 2020.

¹ Грегг Б. «BPF: Профессиональная оценка производительности». Выходит в издательстве «Питер» в 2023 году.

- [Linux 20i] «Segmentation Offloads», Linux documentation, <https://www.kernel.org/doc/Documentation/networking/segmentation-offloads.rst>, по состоянию на 2020.
- [Linux 20c] «MSG_ZEROCOPY», Linux documentation, https://www.kernel.org/doc/html/latest/networking/msg_zerocopy.html, по состоянию на 2020.
- [Linux 20j] «timestamping.txt», Linux documentation, <https://www.kernel.org/doc/Documentation/networking/timestamping.txt>, по состоянию на 2020.
- [Linux 20k] «AF_XDP», Linux documentation, https://www.kernel.org/doc/html/latest/networking/af_xdp.html, по состоянию на 2020.
- [Linux 20l] «HOWTO for the Linux Packet Generator», Linux documentation, <https://www.kernel.org/doc/html/latest/networking/pktgen.html>, по состоянию на 2020.
- [Nosachev 20] Nosachev, D., «How 1500 Bytes Became the MTU of the Internet», <https://blog.benjojo.co.uk/post/why-is-ethernet-mtu-1500>, 2020.
- [Škarvada 20] Škarvada, J., «network-latency/tuned.conf», <https://github.com/redhatperformance/tuned/blob/master/profiles/network-latency/tuned.conf>, последнее обновление 2020.
- [Tuned Project 20] «The Tuned Project», <https://tuned-project.org>, по состоянию на 2020.
- [Wireshark 20] «Wireshark», <https://www.wireshark.org>, по состоянию на 2020.

Глава 11

ОБЛАЧНЫЕ ВЫЧИСЛЕНИЯ

Развитие облачных вычислений решает некоторые старые проблемы производительности, создавая при этом новые. Облачную среду можно создать практически мгновенно и масштабировать по мере необходимости без обычного оверхеда на создание и управление локальным центром обработки данных. Облака также обеспечивают лучшую детализацию для развертываний — разным клиентам могут выделяться разные доли сервера. Но это создает свои проблемы: оверхед на работу технологий виртуализации и конкуренцию за ресурсы между арендаторами.

Цели этой главы:

- познакомить с архитектурой облачных вычислений и ее влиянием на производительность;
- представить типы виртуализации: оборудования, операционной системы и легковесную виртуализацию оборудования;
- познакомить с внутренними функциями виртуализации, включая использование агентов (прокси-объектов) ввода/вывода, и с методами настройки;
- показать на примерах типичный оверхед при различных рабочих нагрузках для каждого типа виртуализации;
- научить диагностировать проблемы производительности гостевых и хост-систем и показать, как может меняться набор используемых инструментов в зависимости от типа виртуализации.

Все, о чем говорится в книге, применимо к анализу производительности облачных сред, но именно в этой главе основное внимание уделяется их уникальным особенностям: работе гипервизоров и виртуализации, влиянию средств управления ресурсами на гостевые системы и работе инструментов наблюдения на стороне гостевой и хост-системы. Облачные провайдеры обычно предлагают свои службы и API, которые здесь не рассматриваются: см. документацию с описанием набора сервисов, которые предоставляет каждый провайдер.

В главе четыре части:

- **Основы** представляет общую архитектуру облачных вычислений и ее влияние на производительность.

- **Виртуализация оборудования**, когда гипервизор управляет несколькими виртуальными машинами с экземплярами гостевых ОС, в каждой из которых выполняется свое ядро с виртуализированными устройствами. В качестве примеров используются гипервизоры Xen, KVM и Amazon Nitro.
- **Виртуализация операционной системы**, когда системой управляет единое ядро и создает виртуальные экземпляры ОС, изолированные друг от друга. В качестве примеров используются контейнеры Linux.
- **Легковесная виртуализация оборудования** вобрала в себя лучшее из обоих подходов: легковесные виртуализированные экземпляры оборудования запускаются с собственными ядрами и при этом сохраняются преимущества быстрой загрузки и плотности размещения, аналогичные контейнерам. В качестве примера гипервизора используется AWS Firecracker.

Разделы, посвященные виртуализации, упорядочены по времени широкого распространения этих технологий. Например, Amazon Elastic Compute Cloud (EC2) предлагал экземпляры виртуального оборудования еще в 2006 году, контейнеры с виртуальными ОС — в 2017 году (Amazon Fargate), а легковесные виртуальные машины — в 2019 году (Amazon Firecracker).

11.1. ОСНОВЫ

Облачные вычисления позволяют предоставлять вычислительные ресурсы как услугу и масштабировать их от небольшой доли одного сервера до систем со множеством серверов. Устройство облака зависит от выбора части программного стека для установки и настройки. В этой главе основное внимание уделяется облачным предложениям, предоставляющим *экземпляры серверов*, которыми могут быть:

- **Экземпляры оборудования** (иначе называются *инфраструктура как услуга* — infrastructure as a service, IaaS): предоставляются с использованием виртуализации оборудования. Каждый экземпляр сервера — это виртуальная машина.
- **Экземпляры операционной системы**: предоставляются легковесные экземпляры, обычно посредством виртуализации ОС.

Вместе они могут называться *экземплярами серверов*, *облачными экземплярами* или просто *экземплярами*. В числе облачных провайдеров, которые их поддерживают, можно назвать Amazon Web Services (AWS), Microsoft Azure и Google Cloud Platform (GCP). Есть и другие типы облачных примитивов, включая «функции как услуга» (functions as a service, FaaS; см. раздел 11.5 «Другие типы»).

В общем случае термин *облачные вычисления* описывает структуру динамического выделения ресурсов для экземпляров. В одной физической *хост-системе* действует один или несколько экземпляров *гостевых* систем. Иногда гостевые системы называют *арендаторами* (tenants), а если одна хост-система служит для нужд нескольких арендаторов, то используют термин *мультиарендность* (multitenancy). Хост может находиться под управлением внешнего провайдера, поддерживающего

публичное облако, или вашей компании, поддерживающей *частное облако* только для внутреннего пользования. Некоторые компании создают *гибридное облако*, охватывающее общедоступное и частное облака¹. Гостевые (арендуемые) системы в облаке управляются их конечными пользователями.

Для виртуализации оборудования используется технология под названием *гипервизор* (или *монитор виртуальных машин* — virtual machine monitor, VMM). Она позволяет создавать и управлять экземплярами виртуальных машин, которые внешне видны как отдельные компьютеры, и устанавливать полноценные ОС и ядра.

Обычно экземпляры могут создаваться (и уничтожаться) за минуты или даже секунды и немедленно запускаться в эксплуатацию. Нередко предлагаемый *облачный API* позволяет автоматизировать подготовку экземпляров с помощью другой программы.

Для обсуждения различных вопросов, связанных с производительностью, важно иметь некоторое представление о типах экземпляров, архитектуре, планировании мощности, организации хранилищ и мультиарендности. Кратко об этом в следующих разделах.

11.1.1. Типы экземпляров

Облачные провайдеры часто предлагают экземпляры разных типов и размеров.

Некоторые типы экземпляров универсальны и сбалансированы по ресурсам, другие могут быть оптимизированы по определенному ресурсу: памяти, процессорам, дискам и т. д. Например, AWS объединяет типы в семейства (обозначаемые буквами) и поколения (обозначаемые числами). Сейчас предлагаются следующие экземпляры:

- **m5**: общего назначения (сбалансированные);
- **c5**: оптимизированные для вычислительных задач;
- **i3**, **d2**: оптимизированные для хранения;
- **r4**, **x1**: оптимизированные по объему памяти;
- **p1**, **g3**, **f1**: с поддержкой ускорения вычислений (графические процессоры, FPGA и т. д.).

В каждом семействе есть экземпляры разных размеров. Семейство m5 в AWS, например, включает экземпляры от m5.large (2 виртуальных процессора и 8 Гбайт основной памяти) до m5.24xlarge (96 виртуальных процессоров и 384 Гбайт основной памяти).

Обычно соотношение цена/производительность остается неизменным для экземпляров всех размеров — это позволяет клиентам выбирать экземпляры, лучше подходящие для их рабочей нагрузки.

¹ Например, Google Anthos — это платформа управления приложениями, которая поддерживает локальный вариант движка Google Kubernetes Engine (GKE) с экземплярами GCP, а также другие облака.

Google Cloud Platform, например, дополнительно предлагает настраиваемые экземпляры, позволяя клиентам самим определять количество доступных ресурсов.

Благодаря такому разнообразию вариантов и простоте повторного развертывания, тип экземпляра стал больше похожим на настраиваемый параметр, который можно изменять по мере необходимости. Это значительное усовершенствование по сравнению с традиционной корпоративной моделью выбора и заказа физического оборудования, которое компания может не менять годами.

11.1.2. Масштабируемая архитектура

Чтобы справиться со все возрастающей нагрузкой, в корпоративных средах исторически использовалось *вертикальное масштабирование*, заключающееся в создании все более крупных и мощных систем (мейнфреймов). Этот подход имеет свои ограничения. Есть реальный предел физического размера компьютеров (он может ограничиваться размерами дверей лифта или транспортных контейнеров), а кроме того, по мере увеличения процессоров возникает все больше трудностей с синхронизацией их кэшей, а также с электропитанием и охлаждением. Обойти эти ограничения позволяет распределение нагрузки между несколькими (возможно, небольшими) системами. Это называется *горизонтальным масштабированием*. На предприятиях этот подход использовался для создания компьютерных ферм и кластеров, обеспечивавших, в частности, высокопроизводительные вычисления (предшествовавшие облачным вычислениям).

Облачные вычисления тоже основаны на идее горизонтального масштабирования. На рис. 11.1 показан пример такой среды, включающей балансировщики нагрузки, веб-серверы, серверы приложений и базы данных.

На каждом уровне среды есть один или несколько экземпляров серверов, работающих параллельно, и их число увеличивается с добавлением нагрузки. Архитектура может поддерживать возможность добавления экземпляров в каждый слой независимо либо делиться на вертикальные сегменты, каждый из которых состоит из серверов баз данных, серверов приложений и веб-серверов и добавляется как единое целое¹.

Наибольшую сложность в этой модели представляет развертывание традиционных серверов баз данных, из которых один экземпляр должен быть главным. Данные для этих БД, например MySQL, можно логически разделить на группы, которые называются *шардами* (shards), каждая из которых управляется своим собственным сервером БД (или парой серверов главный/вторичный). Архитектуры распределенных баз данных, например Riak, могут динамически распределять нагрузку между доступными экземплярами. Сейчас есть *полноценные облачные базы данных*, предназначенные для использования в облачных средах, включая Cassandra, CockroachDB, Amazon Aurora и Amazon DynamoDB.

¹ В Shopify, например, эти сегменты называются «подами» [Denis 18].

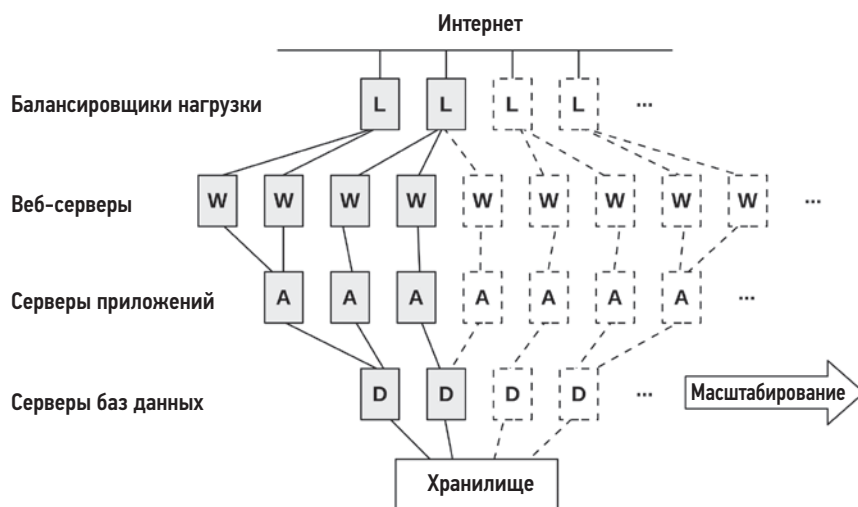


Рис. 11.1. Архитектура облака: горизонтальное масштабирование

Поскольку размер экземпляра обычно невелик, например 8 Гбайт (на физических хостах с 512 Гбайт DRAM и более), масштабирование можно делать небольшими приращениями для достижения оптимального соотношения цена/производительность, а не заранее вкладывать деньги в огромные системы, которые долгое время могут оставаться практически простаивающими.

11.1.3. Планирование мощности

Покупка отдельных серверов может потребовать значительных затрат на инфраструктуру и оплату многолетних контрактов на обслуживание. Кроме того, запуск нового сервера в промышленную эксплуатацию может занять месяцы, например, из-за необходимости согласования, ожидания появления деталей, доставки, установки, сборки и тестирования. Планирование мощностей — критически важный аспект в этом подходе, потому что помогает приобретать системы подходящего размера: слишком маленькие системы могут не справиться с возрастающей нагрузкой, а слишком большие стоят неоправданно дорого (а с учетом контрактов на обслуживание система может не окупиться за многие годы). Планирование мощности нужно и для прогнозирования роста потребностей, чтобы можно было вовремя завершить длительные процедуры закупок.

Отдельные серверы и вычислительные центры были нормой для корпоративных сред. Облачные вычисления — это совсем другое. Экземпляры серверов обходятся недорого и могут создаваться и уничтожаться практически мгновенно. Вместо того чтобы тратить время на планирование будущих потребностей, компании могут просто увеличивать количество используемых экземпляров *по мере необходимости*. Это можно делать автоматически через облачный API, опираясь на значения

метрик, которые собирает ПО мониторинга производительности. Малый бизнес или стартап может вырасти из одного небольшого экземпляра до нескольких тысяч, не затратив ни копейки на подробные исследования с целью планирования емкости¹.

Для растущих стартапов еще одним важным фактором является скорость изменения кода. Обычно сайты обновляют свой продакшен-код еженедельно, ежедневно или даже несколько раз в день. Исследования для планирования мощности могут занять недели, а поскольку результаты таких исследований основаны на моментальном снимке метрик производительности, они могут устареть к моменту их завершения. Этим они отличаются от корпоративных сред с коммерческим ПО, которое может меняться не чаще нескольких раз в год.

Вот что делается в облаке для планирования мощности:

- **динамическое изменение размера:** автоматическое добавление и удаление экземпляров серверов;
- **тестирование масштабируемости:** приобретение значительных облачных мощностей на короткий срок, чтобы протестировать масштабируемость с применением синтетической нагрузки (фактически это *бенчмаркинг*).

Принимая во внимание временные ограничения, можно также смоделировать масштабируемость (аналогично исследованиям на предприятиях), чтобы оценить, насколько фактическая масштабируемость отстает от теоретической.

Динамическое изменение размера (автомасштабирование)

Облачные провайдеры обычно поддерживают развертывание групп экземпляров серверов, которые могут автоматически масштабироваться с увеличением нагрузки (например, *группа автоматического масштабирования* AWS — *auto scaling group*, ASG). Они также поддерживают архитектуру *микросервисов*, когда приложение разделено на небольшие фрагменты, взаимодействующие между собой по сети, и их можно масштабировать индивидуально.

Автомасштабирование решает проблему быстрого реагирования на изменения нагрузки, но при этом может привести к избыточному выделению ресурсов, как показано на рис. 11.2. Например, увеличение нагрузки может быть вызвано DoS-атакой и привести к выделению избыточного количества дорогостоящих экземпляров сервера. Аналогичный риск несут изменения в приложениях, снижающие производительность и приводящие к необходимости выделения большего числа экземпляров для обслуживания той же нагрузки. Важно постоянно делать мониторинг, чтобы понимать, что такое масштабирование имеет смысл.

¹ Ситуация может усложниться, когда потребности компании вырастут до сотен тысяч экземпляров, потому что из-за высокого спроса у облачных провайдеров могут временно закончиться доступные экземпляры. Если вы дорастаете до такого масштаба, обсудите с представителем вашего провайдера способы смягчения ситуации (например, покупку резервных мощностей).



Рис. 11.2. Динамическое изменение размера

Облачные провайдеры выставляют счет по часам, минутам или даже секундам, что позволяет пользователям быстро увеличивать *и уменьшать* масштаб. Стоимость услуги снижается одновременно с уменьшением количества используемых экземпляров. Это снижение можно автоматизировать, выделяя достаточное количество экземпляров для каждой минуты в соответствии с шаблоном изменения рабочей нагрузки в течение дня¹. В Netflix реализовали такой подход в своем облаке, добавляя и удаляя десятки тысяч экземпляров ежедневно в соответствии со своим посекундным шаблоном изменения рабочей нагрузки, пример которого показан на рис. 11.3 [Gregg 14b].

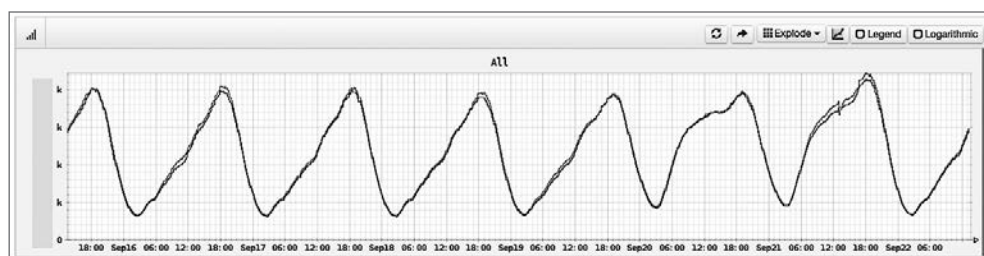


Рис. 11.3. Суточная закономерность изменения рабочей нагрузки в Netflix

Вот еще несколько примеров. В декабре 2012 года Pinterest сообщила о сокращении удельных расходов с \$54 в час до \$20 в час за счет автоматического отключения своих облачных систем в нерабочее время в ответ на уменьшение трафика [Hoff 12]. В 2018 году Shopify отметила большую экономию на инфраструктуре за счет перехода в облако: вместо серверов со средним временем простоя 61 % они начали использовать облачные экземпляры со средним временем простоя 19 % [Kwiatkowski 19]. Немедленная экономия также может быть результатом настрой-

¹ Обратите внимание, что автоматизация увеличения и уменьшения вычислительной мощности может стать сложной задачей. Масштабирование вниз может включать ожидание не только завершения обработки текущих запросов, но и завершения длительных пакетных заданий и передачи локальных данных между базами данных.

ки производительности, когда для обработки той же нагрузки требуется меньшее количество экземпляров.

Некоторые облачные архитектуры (см. раздел 11.3 «Виртуализация операционной системы») могут динамически выделять больше процессоров, если они доступны, используя стратегию *увеличения доступной вычислительной мощности* (bursting). Эта стратегия не требует дополнительных затрат и предназначена для предотвращения избыточного выделения ресурсов: резервная вычислительная мощность предоставляется на период, за который можно определить факт увеличения нагрузки и проверить, реальное ли это увеличение и продолжится ли нарастание нагрузки в будущем. В этом случае можно подготовить дополнительные экземпляры, чтобы гарантировать достаточность ресурсов в будущем.

Любой из этих методов обычно намного эффективнее, чем организация корпоративной среды, особенно с фиксированным размером, выбранным для обработки ожидаемой пиковой нагрузки в течение всего срока службы сервера: такие серверы могут большую часть времени работать в режиме ожидания.

11.1.4. Хранилище

Облачному экземпляру требуется хранилище для ОС, прикладного ПО и временных файлов. В системах Linux это корневой и другие тома. Хранилище может быть локальным физическим или сетевым. Хранилища экземпляров являются временными и уничтожаются вместе со своими экземплярами (поэтому они называются *эфемерными дисками*). В качестве постоянного хранилища обычно используется независимый сервис, который предоставляет экземплярам такие хранилища, как:

- **хранилища файлов:** например, через NFS;
- **блочные хранилища:** например, через iSCSI;
- **хранилища объектов:** через API, обычно на основе HTTP.

Они работают в сети, а сетевая инфраструктура и устройства хранения совместно используются несколькими арендаторами. По этим причинам производительность хранилища может быть менее предсказуемой, чем производительность локальных дисков, но с другой стороны, облачный провайдер может улучшить постоянство производительности за счет использования средств управления ресурсами.

Облачные провайдеры часто предлагают собственные услуги хранилищ. Например, Amazon предлагает для хранения файлов Amazon Elastic File System (EFS), блочное хранилище Amazon Elastic Block Store (EBS) и хранилище объектов Amazon Simple Storage Service (S3).

На рис. 11.4 изображены два вида хранилищ — локальное и сетевое.

Увеличенную задержку, характерную для сетевых хранилищ, обычно можно уменьшить за счет хранения часто используемых данных в кэше памяти.

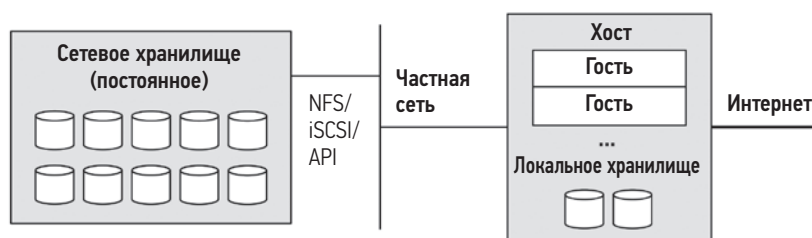


Рис. 11.4. Облачное хранилище

Некоторые сервисы хранения позволяют покупать гарантированное быстрое действие IOPS, когда требуется постоянная производительность (например, том Amazon EBS Provisioned IOPS).

11.1.5. Мультиарендность

Unix — многозадачная операционная система, поддерживающая одновременную работу с несколькими пользователями и процессами, которые обращаются к одним и тем же ресурсам. Позднее в Linux появилась возможность ограничивать потребление ресурсов и средства управления справедливым их распределением, а также инструменты наблюдения для выявления и количественной оценки проблем производительности, связанных с конкуренцией за ресурсы.

Облачные вычисления отличаются тем, что все экземпляры ОС могут сосуществовать в одной физической системе. Каждая гостевая система — это отдельная, изолированная ОС: гостевые системы (обычно¹) не имеют возможности наблюдать за пользователями и процессами других гостевых систем на том же хосте, потому что это будет считаться утечкой информации, даже если они используют одни и те же физические ресурсы.

Поскольку ресурсы распределяются между арендаторами, проблемы с производительностью могут быть вызваны так называемыми *шумными соседями*. Например, другая гостевая система на том же хосте может производить резервное копирование объемной базы данных в то же самое время, когда ваша система испытывает пиковую нагрузку, препятствуя вашим операциям с дисками и сетью. Хуже того, сосед может оценивать облачного провайдера, выполняя микробенчмаркинг и намеренно насыщая ресурсы, чтобы определить их предел.

¹ Контейнеры Linux могут собираться разными способами из пространств имен и контрольных групп. Также можно создавать контейнеры, совместно использующие пространство имен процессов, что может использоваться для создания контейнеров-прицепов («sidecar»), которые могут отлаживать процессы в другом контейнере. В Kubernetes основной абстракцией является под (Pod) — группа контейнеров, разделяющих общее сетевое пространство имен.

Есть несколько вариантов решения подобных проблем. Эффекты мультиарендности можно контролировать с помощью *средств управления ресурсами*: настроек, управляющих *распределением ресурсов* ОС, которые обеспечивают *изоляцию производительности* (также называемую изоляцией ресурсов). В этом случае для каждого арендатора устанавливаются ограничения или приоритеты на использование системных ресурсов: процессоров, памяти, дискового или файлового ввода/вывода, а также пропускной способности сети.

Кроме того, команда, обслуживающая облако, может наблюдать за конкуренцией арендаторов и настраивать ограничения, чтобы точнее сбалансировать распределение имеющихся ресурсов. Степень наблюдаемости зависит от типа виртуализации.

11.1.6. Оркестровка (Kubernetes)

Многие компании создают свои частные облака, используя *программное обеспечение оркестровки*, работающее на их собственном железе или в облачных системах. Самым популярным таким ПО является Kubernetes (сокращенно K8s), изначально созданное в Google. Kubernetes, по-гречески «рулевой», — это система с открытым исходным кодом, управляющая развертыванием приложений с использованием контейнеров (обычно контейнеров Docker, но вообще может использоваться любая среда выполнения, реализующая интерфейс Open Container, например containerd) [Kubernetes 20b]. Общедоступные облачные провайдеры тоже предлагают поддержку Kubernetes для упрощения развертывания контейнеров, в том числе Google Kubernetes Engine (GKE), Amazon Elastic Kubernetes Service (Amazon EKS) и Microsoft Azure Kubernetes Service (AKS).

Kubernetes развертывает контейнеры в виде объединенных групп — *подов* (Pods), в которых контейнеры могут совместно использовать ресурсы и взаимодействовать друг с другом локально. Каждому поду присваивается собственный IP-адрес, который можно использовать для взаимодействий (через сеть) с другими подами. *Сервис* в Kubernetes — это абстракция конечных точек, предоставляющая группу подов с метаданными, включая IP-адрес, которая служит постоянным и стабильным интерфейсом для этих конечных точек, в то время как сами поды можно добавлять и удалять, что позволяет считать их одноразовыми. Сервисы Kubernetes поддерживают архитектуру микросервисов. Kubernetes реализует стратегии автоматического масштабирования, такие как «Horizontal Pod Autoscaler», способные создавать дополнительные копии подов, исходя из уровня потребления ресурсов или других метрик. В Kubernetes физические машины называются *узлами*, а группы узлов объединяются в *кластеры* Kubernetes, если они подключены к одному и тому же серверу Kubernetes API.

Проблемы производительности в Kubernetes обычно обусловлены сложностями планирования (выбора узла в кластере с максимальной производительностью для запуска контейнера) и оверхедом на работу сети из-за использования дополнительных компонентов, реализующих сети контейнеров и осуществляющих балансировку нагрузки.

При планировании Kubernetes учитывает потребление и ограничения процессоров и памяти, а также метаданные, такие как *непригодность узлов* (node taints, когда узлы отмечаются как непригодные для планирования) и *селекторы меток* (настраиваемые метаданные). Сейчас Kubernetes не ограничивает блочный ввод/вывод (но такая поддержка, с помощью контрольной группы blkio, может быть добавлена в будущем [Xu 20]), что делает конкуренцию за диск еще одним возможным источником проблем с производительностью.

Для сетевых взаимодействий Kubernetes предлагает различные сетевые компоненты, поэтому определение наиболее подходящих для использования — важный шаг в обеспечении максимальной производительности. Сеть контейнеров может быть реализована с помощью подключаемого сетевого интерфейса контейнера (container network interface, CNI). Пример такого программного интерфейса — Calico, основанный на netfilter или iptables, и Cilium, основанный на BPF. Оба имеют открытый исходный код [Calico 20], [Cilium 20b]. Для балансировки нагрузки Cilium предлагает замену kube-proxy на основе BPF [Borkmann 19].

11.2. ВИРТУАЛИЗАЦИЯ ОБОРУДОВАНИЯ

При виртуализации оборудования создается виртуальная машина (virtual machine, VM), на которой может работать ОС, включая ее собственное ядро. Виртуальные машины создаются гипервизорами, которые также называют диспетчерами виртуальных машин (virtual machine manager, VMM). Распространенная классификация гипервизоров делит их на два типа: тип 1 и тип 2 [Goldberg 73]:

- **Тип 1** выполняется непосредственно на процессорах. Администрирование гипервизора может производиться привилегированной гостевой системой, которая создает и запускает новые гостевые системы. Тип 1 также называется *низкоуровневым* (native) *гипервизором* или *гипервизором на железе* (bare-metal). Этот гипервизор имеет собственный планировщик процессоров для гостевых виртуальных машин. Популярный пример — гипервизор Xen.
- **Тип 2** выполняется внутри хост-системы, обладающей привилегиями на администрирование гипервизора и запуск новых гостевых систем. В этом случае загружается обычная операционная система, которая затем запускает гипервизор. Этот гипервизор управляется планировщиком процессоров в ядре хост-системы, а гостевые виртуальные машины обслуживаются как процессы хост-системы.

Сейчас все еще можно встретить термины тип 1 и тип 2, но с развитием технологии гипервизоров эта классификация больше неприменима в строгом смысле [Liguori 07]: за счет использования модулей ядра, благодаря которым гипервизор получил прямой доступ к оборудованию, тип 2 практически превратился в тип 1. На рис. 11.5 показана классификация, более близкая к реальности, где представлены две типичные конфигурации, которые я назвал конфигурацией А и конфигурацией В [Gregg 19].

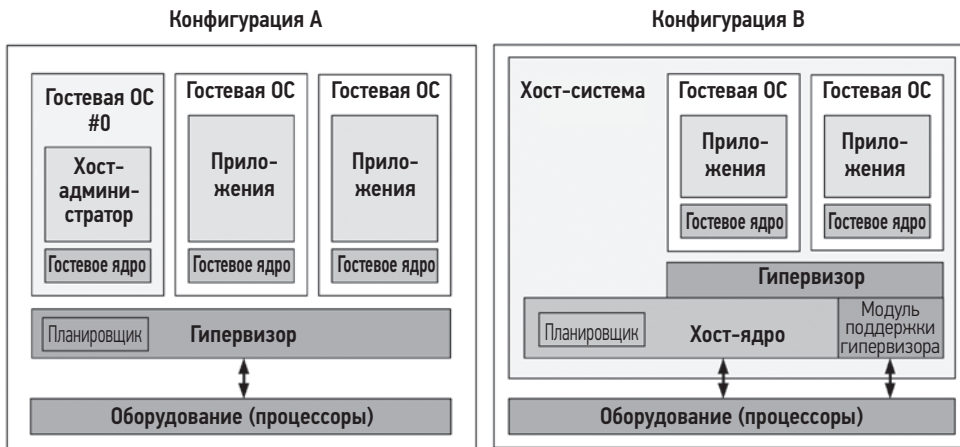


Рис. 11.5. Типичные конфигурации гипервизоров

Вот краткое описание этих конфигураций:

- **Конфигурация А:** также называется низкоуровневым гипервизором или гипервизором на железе. ПО гипервизора выполняется непосредственно на процессорах, создает домены для запуска гостевых виртуальных машин и распределяет виртуальные гостевые процессоры между физическими процессорами. Привилегированный домен (с номером 0 на рис. 11.5) может использоваться для администрирования остальных доменов. Популярный пример — гипервизор Xen.
- **Конфигурация В:** программное обеспечение гипервизора выполняется ядром хост-системы и может состоять из модулей ядра и процессов пользователя. Хост-система обладает привилегиями, необходимыми для администрирования гипервизора, а ее ядро распределяет процессорное время между виртуальными машинами и другими процессами в хост-системе. Благодаря использованию модулей ядра эта конфигурация обеспечивает прямой доступ к оборудованию. Популярный пример — гипервизор KVM.

Обе конфигурации могут предусматривать запуск агентов ввода/вывода (например, программного обеспечения QEMU) в домене 0 (Xen) или в хост-системе (KVM) для обслуживания гостевого ввода/вывода. Это решение увеличивает оверхед на ввод/вывод, но с годами оно было оптимизировано за счет добавления транспортов с разделяемой памятью и других технологий.

Первоначальный гипервизор оборудования, впервые предложенный компанией VMware в 1998 году, использовал *двоичную трансляцию* для полной виртуализации оборудования [VMware 07], включая переадресацию привилегированных инструкций, таких как системные вызовы и операции с таблицами страниц перед выполнением. Непривилегированные инструкции могут выполняться непосредственно на процессоре. Такой подход позволил создать полноценную виртуальную систему, состоящую из

виртуализированных аппаратных компонентов, на которую можно было установить немодифицированную ОС. Присущий ему оверхед часто оказывался вполне приемлемым с учетом экономии, которую давала консолидация серверов.

С тех пор были добавлены следующие улучшения:

- **Поддержка виртуализации процессоров:** в 2005–2006 годах были добавлены расширения AMD-V и Intel VT-x, обеспечившие ускорение процессорных операций на виртуальной машине. Эти расширения повысили скорость выполнения виртуализированных привилегированных инструкций и работу модуля управления памятью (MMU).
- **Паравиртуализация (paravirtualization, PV):** предлагает виртуальную систему, которая включает интерфейс для гостевых ОС с целью эффективного использования ресурсов хоста (через *гипервызовы*) без полной виртуализации всех компонентов. Например, включение таймера обычно предполагает выполнение нескольких привилегированных инструкций, которые должен эмулировать гипервизор. Эти инструкции могут объединяться паравиртуализированной гостевой системой в один гипервызов, что позволяет гипервизору более эффективно обрабатывать его. Для достижения еще большей эффективности гипервизор Xen группирует гипервызовы в *мультивызовы* (multicall). Паравиртуализация может допускать использование гостевой системой драйвера паравиртуального сетевого устройства для эффективной передачи пакетов физическим сетевым интерфейсам. В общем случае производительность увеличивается, но многое зависит от поддержки паравиртуализации гостевой системой (которой, например, традиционно нет в Windows).
- **Аппаратная поддержка устройств:** для еще большей оптимизации производительности VM в аппаратные устройства, кроме процессоров, была добавлена поддержка виртуальных машин. Сюда входит виртуализация ввода/вывода с одним корнем (SR-IOV) для сетевых устройств и устройств хранения, что позволяет гостевым виртуальным машинам напрямую обращаться к оборудованию. Для этого требуется поддержка драйверов (например, ixgbe, e1a, hv_netvsc и nvme).

С годами гипервизор Xen развился и улучшил производительность. Современные виртуальные машины Xen часто загружаются в режиме виртуализации оборудования (hardware VM, HVM), а затем используют драйверы PV с поддержкой HVM для повышения производительности: эта конфигурация получила название PVHVM. Ее можно усовершенствовать еще больше, если использовать полную аппаратную виртуализацию для некоторых драйверов, например SR-IOV, сетевых устройств и устройств хранения.

11.2.1. Реализация

Есть множество разных реализаций виртуализации оборудования, часть из которых уже упоминались выше (Xen и KVM). Например:

- **VMware ESX** (выпущен в 2001 году) — это корпоративный продукт для консолидации серверов и ключевой компонент VMware vSphere, продукта для

организации облачных вычислений. Его гипервизор представляет собой микро-ядро, работающее на «голом железе», а первая виртуальная машина называется *служебной консолью*, которая может управлять гипервизором и новыми виртуальными машинами.

- **Xen** (выпущен в 2003 году) начинался как исследовательский проект в Кембриджском университете, а затем был приобретен компанией Citrix. Xen — это гипервизор типа 1, который запускает паравиртуализированные гостевые системы для обеспечения высокой производительности. Позднее была добавлена поддержка немодифицированных гостевых систем (Windows). Виртуальные машины называются *доменами*, наиболее привилегированный из них — *dom0*, из которого производится администрирование гипервизора и запуск новых доменов. Xen распространяется с открытым исходным кодом и может запускаться из Linux. Ранее на Xen было основано вычислительное облако Amazon Elastic Compute Cloud (EC2).
- **Hyper-V** (появился вместе с Windows Server 2008) — это гипервизор типа 1, который создает *разделы* для запуска гостевых ОС. В общедоступном облаке Microsoft Azure есть возможность запускать свои версии Hyper-V (но детали этой возможности не раскрываются широкой публике).
- **KVM** (был разработан стартапом Qumranet, купленным Red Hat в 2008 году) — это гипервизор типа 2, работающий как модуль ядра. Поддерживает аппаратные расширения для обеспечения высокой производительности и использует паравиртуализацию для некоторых устройств, поддерживаемых гостевой системой. Чтобы создать полноценный экземпляр виртуальной машины с аппаратной поддержкой, он связывается с пользовательским процессом QEMU (Quick Emulator) — гипервизором, который может создавать виртуальные машины и управлять ими. QEMU изначально был высококачественным гипервизором типа 2 с открытым исходным кодом, который использовал двоичную трансляцию; он был написан Фабрисом Белларом (Fabrice Bellard). KVM распространяется с открытым исходным кодом и используется компанией Google в Google Compute Engine [Google 20c].
- **Nitro** (создан компанией AWS в 2017 году). Этот гипервизор использует компоненты на основе KVM с аппаратной поддержкой всех основных ресурсов: процессоров, сетевых устройств, устройств хранения, прерываний и таймеров [Gregg 17e]. Он не использует прокси-сервер QEMU. Nitro обеспечивает производительность гостевых виртуальных машин практически на уровне голого железа.

В разделах ниже рассмотрены вопросы производительности, связанные с виртуализацией оборудования: оверхед, управление ресурсами и наблюдаемость. Они различаются в зависимости от реализации и конфигурации.

11.2.2. Оверхед

При исследовании проблем с производительностью облачных сред важно понимать, какой оверхед можно ожидать от виртуализации.

Виртуализация оборудования производится по-разному. Для доступа к ресурсам может потребоваться проксирование и преобразование со стороны гипервизора с присущим ему оверхедом, но также могут использоваться аппаратные технологии, позволяющие избежать оверхеда. В разделах ниже в общих чертах описывается оверхед, обусловленный выполнением на процессоре, отображением памяти, размером памяти, операциями ввода/вывода и конкуренцией со стороны других арендаторов.

Процессор

Как правило, гостевые приложения выполняются непосредственно на процессорах, поэтому преимущественно вычислительные приложения могут иметь практически такую же производительность, что и в обычной системе. Оверхед может возникать при выполнении привилегированных машинных инструкций, обращении к оборудованию и отображении основной памяти в зависимости от того, как эти операции обрабатываются гипервизором. Ниже описано, как обрабатываются машинные инструкции различными типами механизмов виртуализации оборудования:

- **Двоичная трансляция:** гипервизор идентифицирует и транслирует инструкции, которые выполняются гостевым ядром и работают с физическими ресурсами. Двоичная трансляция применялась до появления виртуализации с аппаратной поддержкой. Схема виртуализации без аппаратной поддержки, используемая в VMware, включала запуск монитора виртуальных машин (virtual machine monitor, VMM) в кольце 0 процессора и перемещение гостевого ядра в кольцо 1, которое ранее не использовалось (приложения выполняются в кольце 3, а большинство процессоров поддерживают четыре кольца. О кольцах защиты шла речь в главе 3 «Операционные системы», в разделе 3.2.2 «Ядро и пользовательские режимы»). Поскольку некоторые инструкции гостевого ядра предполагают выполнение в кольце 0, то для выполнения из кольца 1 их необходимо транслировать в вызовы VMM, чтобы применить виртуализацию. Эта трансляция выполняется во время выполнения и требует значительного оверхеда.
- **Паравиртуализация:** инструкции в гостевой ОС, которые необходимо виртуализировать, заменяются гипервызовами к гипервизору. Производительность можно повысить, если модифицировать гостевую ОС для оптимизации гипервызовов, чтобы она знала, что работает на виртуализированном оборудовании.
- **Аппаратная поддержка:** немодифицированные инструкции гостевого ядра, обращающиеся к оборудованию, обрабатываются гипервизором, который использует монитор VMM на уровне ниже кольца 0. Вместо двоичной трансляции привилегированные инструкции гостевого ядра перехватываются более привилегированным монитором VMM, который затем может эмулировать привилегии для поддержки виртуализации [Adams 06].

Виртуализация с аппаратной поддержкой обычно предпочтительнее, в зависимости от реализации и рабочей нагрузки, но для увеличения производительности некоторых рабочих нагрузок (особенно ввода/вывода) можно использовать паравиртуализацию, если гостевая система ее поддерживает.

В качестве примера различий в реализации приведу модель двоичной трансляции в VMware, которая оптимизировалась на протяжении многих лет. Вот как о ней писали в 2007 году [VMware 07]:

Из-за высокого оверхеда на переходы между гипервизором и гостевой системой и жесткой модели программирования подход к двоичной трансляции в VMware сейчас превосходит большинство аппаратных реализаций первого поколения. Жесткая модель программирования в реализациях первого поколения оставляет мало места для гибкости в управлении частотой или оверхедом на переходы между гипервизором и гостевыми системами.

Частоту переходов между гостевой системой и гипервизором, а также время, проведенное в гипервизоре, можно интерпретировать как метрику, характеризующую оверхед. Эти события обычно называют *гостевыми выходами*, так как виртуальный процессор должен прекратить выполнение гостевого кода, когда происходит переход. На рис. 11.6 показан оверхед, связанный с гостевыми выходами внутри KVM.

Рисунок представляет поток гостевых выходов между пользовательским процессом, хост-ядром и гостевой системой. Время, потраченное на обработку гостевых выходов, — это оверхед при виртуализации оборудования. Чем больше времени тратится на обработку выходов, тем выше оверхед. Когда гость выходит, часть событий может обрабатываться непосредственно в ядре. События, которые не могут быть обработаны ядром, должны покинуть его и вернуться в пользовательский процесс. Это еще больше увеличивает оверхед по сравнению с выходами, которые могут обрабатываться ядром.

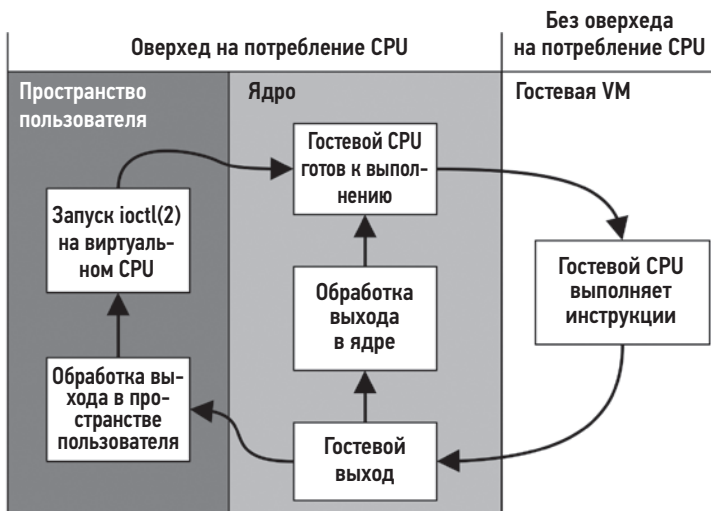


Рис. 11.6. Оверхед на потребление процессора при виртуализации оборудования

При использовании реализации KVM в Linux оверхед можно исследовать с помощью функций, выполняющих гостевой выход, которые отображаются в исходном коде вот так (фрагмент взят из файла `arch/x86/kvm/vmx/vmx.c` в исходном коде Linux 5.2):

```
/*
 * Обработчики выхода возвращают 1, если выход обработан полностью и гость
 * может продолжить выполнение. В противном случае они устанавливают
 * параметр kvm_run, чтобы показать, что нужно сделать в пространстве пользователя,
 * и возвращают 0.
 */
static int (*kvm_vmx_exit_handlers[])(struct kvm_vcpu *vcpu) = {
    [EXIT_REASON_EXCEPTION_NMI]      = handle_exception,
    [EXIT_REASON_EXTERNAL_INTERRUPT] = handle_external_interrupt,
    [EXIT_REASON_TRIPLE_FAULT]       = handle_triple_fault,
    [EXIT_REASON_NMI_WINDOW]         = handle_nmi_window,
    [EXIT_REASON_IO_INSTRUCTION]     = handle_io,
    [EXIT_REASON_CR_ACCESS]          = handle_cr,
    [EXIT_REASON_DR_ACCESS]          = handle_dr,
    [EXIT_REASON_CPUID]              = handle_cpuid,
    [EXIT_REASON_MSR_READ]           = handle_rdmshr,
    [EXIT_REASON_MSR_WRITE]          = handle_wrmsr,
    [EXIT_REASON_PENDING_INTERRUPT]  = handle_interrupt_window,
    [EXIT_REASON_HLT]                = handle_halt,
    [EXIT_REASON_INVD]               = handle_invd,
    [EXIT_REASON_INVLPG]             = handle_invlpg,
    [EXIT_REASON_RDPMSR]             = handle_rdpmsr,
    [EXIT_REASON_VMCALL]             = handle_vmcall,
    [...],
    [EXIT_REASON_XSAVES]             = handle_xsaves,
    [EXIT_REASON_XRSTORS]            = handle_xrstors,
    [EXIT_REASON_PML_FULL]           = handle_pml_full,
    [EXIT_REASON_INVPCID]            = handle_invpcid,
    [EXIT_REASON_VMFUNC]             = handle_vmx_instruction,
    [EXIT_REASON_PREEMPTION_TIMER]   = handle_preemption_timer,
    [EXIT_REASON_ENCLS]              = handle_encls,
};
```

Несмотря на краткость, имена могут дать некоторое представление о причинах, почему гость может вызвать гипервизор и тем самым способствовать увеличению оверхеда.

Одним из распространенных гостевых выходов является инструкция остановки `halt`, которая обычно вызывается потоком бездействия, когда задания для выполнения нет (что позволяет перевести процессор в режим с низким энергопотреблением до получения прерывания). Эта инструкция обрабатывается функцией `handle_halt()` (см. элемент `EXIT_REASON_HLT` в предыдущем листинге), которая в итоге вызывает `kvm_vcpu_halt()` (`arch/x86/kvm/x86.c`):

```
int kvm_vcpu_halt(struct kvm_vcpu *vcpu)
{
    ++vcpu->stat.halt_exits;
    if (lapic_in_kernel(vcpu)) {
```

```

        vcpu->arch.mp_state = KVM_MP_STATE_HALTED;
        return 1;
    } else {
        vcpu->run->exit_reason = KVM_EXIT_HLT;
        return 0;
    }
}

```

Как и во многих других обработчиках гостевых выходов, код этого обработчика очень короткий, чтобы минимизировать нагрузку на процессор. Этот обработчик начинается с приращения статистики виртуального процессора, подсчитывающей количество остановок. Остальной код выполняет аппаратную эмуляцию этой привилегированной инструкции. Эти функции можно инструментировать в Linux с помощью зондов kprobes в хост-ядре гипервизора для трассировки типов и продолжительности выходов. Выходы также можно трассировать глобально с помощью точки трассировки `kvm:kvm_exit`, пример использования которой показан в разделе 11.2.4 «Наблюдаемость».

Виртуализация оборудования, такого как контроллер прерываний и таймеры с высоким разрешением, тоже влечет за собой некоторый оверхед на процессор (и небольшой объем памяти).

Отображение памяти

Как рассказывалось в главе 7 «Память», операционная система использует блок управления памятью MMU для отображения страниц виртуальной памяти в физические адреса, кэшируя их в TLB для эффективности.

В случае виртуализации отображение новой страницы памяти (отказ страницы) в гостевой системе выполняется в два этапа:

1. Гостевое ядро выполняет преобразование виртуального адреса в гостевое физическое пространство.
2. Гипервизор VMM преобразует гостевой физический адрес в физический адрес хоста (фактический).

Затем отображение гостевого виртуального адреса в физический адрес хоста может быть кэшировано в TLB, чтобы последующие обращения обрабатывались с нормальной скоростью — без дополнительных затрат на трансляцию. Современные процессоры поддерживают виртуализацию MMU, благодаря чему отображения, остающиеся в TLB, могут обслуживаться быстрее (обход страниц) без вызова гипервизора. Эта особенность называется *расширенными таблицами страниц* (extended page tables, EPT) в Intel и *вложенными таблицами страниц* (nested page tables, NPT) в AMD [Milewski 11].

Другой подход, позволяющий увеличить производительность без поддержки EPT/NPT, заключается в организации *теневого таблиц страниц* (shadow page tables) для хранения соответствий между гостевыми виртуальными адресами и физическими адресами хоста, которые управляются гипервизором и доступны гостю

через гостевой регистр CR3. В этой стратегии гостевое ядро поддерживает свои собственные таблицы страниц, которые, как обычно, отображают гостевые виртуальные адреса в гостевые физические адреса. Гипервизор перехватывает изменения в этих таблицах страниц и создает в теневой таблице эквивалентные отображения в физические страницы хоста. Затем, передавая управление гостю, гипервизор перезаписывает регистр CR3 так, чтобы он указывал на теневые страницы.

Размер памяти

В отличие от виртуализации ОС, виртуализация оборудования добавляет дополнительных потребителей памяти. Для каждой гостевой системы запускается ее собственное ядро, которое потребляет некоторый объем памяти. Архитектура хранилища тоже может приводить к двойному кэшированию, когда и гость и хост кэшируют одни и те же данные. Гипервизоры в стиле KVM также запускают дополнительный процесс VMM для каждой виртуальной машины, например QEMU, который сам потребляет некоторый объем основной памяти.

Ввод/вывод

Ввод/вывод традиционно отличался самым большим оверхедом в виртуализации оборудования. Это было обусловлено необходимостью трансляции гипервизором всех операций с устройствами ввода/вывода. В высокоскоростных сетях, например в сети 10 Гбит/с, даже небольшой оверхед на каждую операцию ввода/вывода (пакет) может повлечь значительное снижение общей производительности. Но со временем были созданы технологии для уменьшения оверхеда, кульминацией которых стала аппаратная поддержка, полностью устраняющая такой оверхед. Такая аппаратная поддержка включает виртуализацию ввода/вывода MMU (AMD-Vi и Intel VT-d).

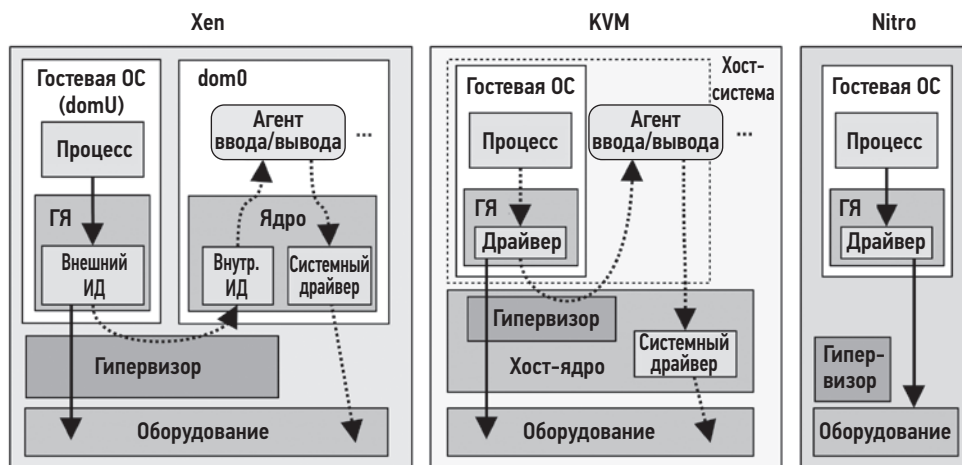
Одним из методов повышения производительности ввода/вывода является использование паравиртуализированных драйверов, которые могут объединять операции ввода/вывода и тем самым уменьшать количество прерываний, чтобы сократить оверхед на работу гипервизора.

Другой метод — *сквозной канал PCI*, когда устройство PCI отдается гостю под прямое управление и может использоваться гостевой системой, как если бы она работала непосредственно на железе. Сквозной канал PCI часто обеспечивает лучшую производительность по сравнению с другими доступными вариантами, но снижает гибкость при настройке системы с несколькими арендаторами, потому что некоторые устройства напрямую управляются гостевыми системами и не могут использоваться совместно. Это также может усложнять оперативную миграцию [Xen 19].

Есть несколько технологий повышения гибкости в использовании устройств PCI с виртуализацией, включая виртуализацию ввода/вывода с одним корнем (SR-IOV, упоминалась выше) и виртуализацию ввода/вывода с несколькими корнями (MR-IOV). Эти термины ссылаются на сложные корневые топологии PCI, поразному обеспечивающие виртуализацию оборудования. В облаке Amazon EC2

эти технологии применялись для ускорения сначала сетевых взаимодействий, а затем дискового ввода/вывода и по умолчанию используются с гипервизором Nitro [Gregg 17e].

На рис. 11.7 изображены типичные конфигурации гипервизоров Xen, KVM и Nitro.



ГЯ — Гостевое ядро

Внешний ИД — Внешний интерфейсный драйвер

Внутр. ИД — Внутренний интерфейсный драйвер

Рис. 11.7. Пути ввода/вывода в Xen, KVM и Nitro

Пунктирные стрелки показывают *поток управления* — порядок, в котором компоненты информируют друг друга, синхронно или асинхронно, о готовности к передаче дополнительных данных. *Поток данных* (сплошные стрелки) иногда может быть реализован с помощью разделяемой памяти и кольцевых буферов. Для Nitro поток управления не показан, потому что он совпадает с потоком данных благодаря прямому доступу к оборудованию.

Есть несколько вариантов настройки Xen и KVM, которые здесь не показаны. На этом рисунке представлен вариант с использованием агентов (прокси-процессов) ввода/вывода (обычно QEMU), которые создаются для каждой гостевой виртуальной машины. Но их также можно настроить на использование технологии SR-IOV, позволяющей гостевым виртуальным машинам напрямую обращаться к оборудованию (не показано для Xen или KVM на рис. 11.7). Nitro требует такой аппаратной поддержки, что избавляет от необходимости использовать агенты ввода/вывода.

Xen увеличивает производительность ввода/вывода за счет использования *канала устройства* — асинхронного транспорта на основе разделяемой памяти между dom0 и гостевыми доменами (domU). Это позволяет избежать оверхеда на создание дополнительных копий данных ввода/вывода при их передаче между доменами.

Кроме того, Xen может использовать отдельные домены для выполнения операций ввода/вывода, как описано в разделе 11.2.3 «Управление ресурсами».

Количество шагов на пути ввода/вывода, как в потоке управления, так и в потоке данных, очень важно для производительности: чем их меньше, тем лучше. В 2006 году разработчики KVM провели сравнение своего гипервизора с привилегированной гостевой системой — Xen и обнаружили, что в KVM ввод/вывод выполняется за вдвое меньшее количество шагов (пять против десяти; правда, тестирование проводилось без паравиртуализации, поэтому не отражает состояния дел в большинстве современных конфигураций) [Qumranet 06].

Поскольку гипервизор Nitro исключает дополнительные операции ввода/вывода, можно ожидать, что все крупные облачные провайдеры, стремящиеся к максимальной производительности, последуют их примеру и начнут использовать аппаратную поддержку, чтобы отказаться от использования агентов ввода/вывода.

Конкуренция между арендаторами

В зависимости от конфигурации гипервизора и количества совместно используемых процессоров арендаторы могут недополучать процессорное время и страдать от загрязнения кэшей процессоров другими арендаторами, что будет снижать производительность их приложений. Обычно эта проблема более характерна для конфигураций с контейнерами, чем с виртуальными машинами, потому что контейнеры особенно способствуют такому совместному использованию процессоров.

Другие арендаторы, производящие ввод/вывод, могут способствовать появлению прерываний, прерывающих работу приложений в зависимости от конфигурации гипервизора.

Конкуренцией за ресурсы можно управлять с помощью средств управления ресурсами.

11.2.3. Управление ресурсами

Процессоры и основная память, доступные гостевой системе, обычно определяются настройками ограничений. ПО гипервизора может также предоставлять средства управления ресурсами для сетевого и дискового ввода/вывода.

При использовании гипервизоров, подобных KVM, распределение физических ресурсов контролируется хост-системой, и кроме средств управления ресурсами в гипервизоре она может использовать свои средства управления для распределения ресурсов между гостевыми системами. В Linux к таким средствам относятся контрольные группы, наборы задач и т. д. Дополнительную информацию о средствах управления ресурсами, доступных хост-системе, ищите в разделе 11.3 «Виртуализация операционной системы». В следующих разделах в качестве примеров описаны средства управления ресурсами в гипервизорах Xen и KVM.

Процессоры

Процессоры обычно выделяются гостевым системам в виде виртуальных процессоров (vCPU), которые управляются гипервизором. Количество выделенных виртуальных процессоров ограничивает потребление вычислительных ресурсов.

В Xen поддерживается выделение точных квот на потребление процессоров гостевыми системами, которые контролируются планировщиком процессора в гипервизоре. В числе доступных планировщиков можно назвать [Cherkasova 07], [Matthews 08]:

- **Заемствованное виртуальное время** (borrowed virtual time, BVT): справедливый планировщик распределения виртуального времени, которое может быть заимствовано заранее для обеспечения выполнения с малой задержкой интерактивных приложений и приложений реального времени.
- **Самый ранний дедлайн первым** (simple earliest deadline first, SEDF): планировщик реального времени, который позволяет настраивать гарантии выполнения и при этом отдает приоритет задачам с самым ранним дедлайном.
- **На основе квот** (credit-based): поддерживает приоритеты (*веса*) и ограничения потребления процессора, а также балансировку нагрузки между несколькими процессорами.

При использовании KVM точные квоты на потребление процессора могут применяться хост-системой, например, при использовании *справедливого планировщика* в ядре хост-системы, описанного ранее. В Linux того же эффекта можно добиться применением контрольных групп, управляющих полосой пропускания процессоров.

Но обе технологии лишь ограниченно могут учитывать *приоритеты* гостевых систем. Распределение потребления процессоров в гостевой системе обычно непрозрачно для гипервизора, и приоритеты потоков гостевого ядра обычно невозможно определить или соблюсти. Например, низкоприоритетный демон ротации журналов в одной гостевой системе может иметь тот же приоритет гипервизора, что и критически важный сервер приложений в другой гостевой системе.

При использовании Xen управление распределением вычислительных ресурсов может дополнительно осложняться рабочими нагрузками, выполняющими большое количество операций ввода/вывода, которые потребляют дополнительные вычислительные ресурсы в dom0. Внутренний интерфейсный драйвер и агент ввода/вывода в гостевом домене могут потреблять больше процессорного времени, чем им выделено, но эту разницу невозможно учесть [Cherkasova 05]. Решение было найдено в создании изолированных доменов драйверов (Isolated Driver Domains, IDD), которые разделяют обслуживание ввода/вывода для обеспечения безопасности, изоляции производительности и учета, как показано на рис. 11.8.

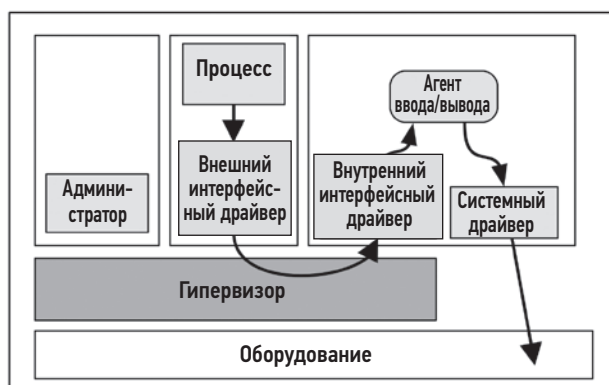


Рис. 11.8. Гипервизор Xen с изолированным доменом драйверов

Потребление процессоров изолированным доменом драйверов доступно для наблюдения и поэтому может учитываться при распределении вычислительных ресурсов между гостями. Вот что говорится в [Gupta 06]:

Наш модифицированный планировщик, SEDF-DC (SEDF-Debt Collector), периодически получает от XenMon информацию о потреблении процессоров изолированным доменом драйверов для обработки ввода/вывода от имени гостевых доменов. Используя эту информацию, SEDF-DC ограничивает выделение процессорного времени гостевым доменам в соответствии с заданным комбинированным пределом потребления.

Позднее для Xen был разработан еще один метод — *фиктивный домен* (stub domain), в котором выполняется мини-ОС.

Кэши процессоров

Помимо выделения виртуальных процессоров, можно управлять потреблением кэшей процессоров с помощью технологии распределения кэша (cache allocation technology, CAT), разработанной в Intel. Она позволяет разделить кэш последнего уровня (LLC) между гостями и предотвратить загрязнение кэша одного гостя другим гостем, но также может ухудшить производительность из-за ограничения размера доступного кэша.

Объем памяти

Ограничения на потребление памяти задаются в конфигурации гостевой системы, то есть гость видит только заданный объем памяти. А гостевое ядро предпринимает все необходимое (в том числе подкачку страниц), чтобы остаться в пределах своего лимита.

Для повышения гибкости статической конфигурации в VMware был разработан *драйвер воздушного шара* (balloon driver) [Waldspurger 02], который может уменьшить объем памяти, доступный запущенной гостевой системе, за счет «раздувания» модуля шара внутри нее, потребляющего гостевую память. Эта память затем может использоваться гипервизором для распределения между другими гостями. Шар также можно спустить и вернуть память гостевому ядру. Во время этого процесса гостевое ядро выполняет свои обычные процедуры управления памятью, чтобы освободить память (например, подкачку страниц). VMware, Xen и KVM поддерживают драйверы воздушного шара.

Когда используется драйвер шара (для проверки этого факта в гостевой системе можно выполнить поиск слова «balloon» в выводе `dmesg(1)`), я внимательно слежу за проблемами производительности, которые он может вызвать.

Объем файловой системы

Гости получают виртуальные дисковые тома от хоста. При использовании гипервизоров, подобных KVM, это могут быть программные тома, созданные хост-системой и имеющие соответствующий размер. Например, файловая система ZFS способна создавать виртуальные тома желаемого размера.

Дисковый ввод/вывод

Управление ресурсами с помощью ПО виртуализации оборудования традиционно было сосредоточено на потреблении процессоров, контролируя которое можно косвенно управлять потреблением ввода/вывода.

Пропускная способность сети может регулироваться внешними выделенными устройствами, а в случае гипервизоров вроде KVM функциями хост-ядра. Например, в Linux есть средства управления шириной полосы пропускания сети на основе контрольных групп, а также различных дисциплин очередей, которые могут применяться к интерфейсам гостевой сети.

После изучения изоляции производительности сети в Xen был сделан следующий вывод [Adamczyk 12]:

...в плане виртуализации сети слабое место Xen — это отсутствие должной изоляции производительности.

Авторы [Adamczyk 12] также предлагают решение для планирования сетевого ввода/вывода в Xen, которое добавляет параметры настройки приоритета и скорости сетевого ввода/вывода. Если вы используете Xen, проверьте: возможно, эта или подобная технология уже доступна.

Для гипервизоров с полной аппаратной поддержкой (например, Nitro) ограничения ввода/вывода могут поддерживаться оборудованием или внешними устройствами. В облаке Amazon EC2 сетевой и дисковый ввод/вывод для устройств, подключенных к сети, регулируются квотами с использованием внешних систем.

11.2.4. Наблюдаемость

Что именно доступно для наблюдения в виртуализированных системах, зависит от гипервизора и от того, где запускаются инструменты наблюдения:

- **В привилегированной гостевой системе (Xen) или хост-системе (KVM):** все физические ресурсы должны быть доступны для наблюдения с помощью стандартных инструментов ОС, описанных в предыдущих главах. Гостевой ввод/вывод можно изучать, наблюдая за агентами ввода/вывода, если они используются. Статистики потребления ресурсов каждой гостевой системой должны быть доступны в гипервизоре. Внутренние детали работы гостевой системы, включая их процессы, недоступны для наблюдения напрямую. Некоторые операции ввода/вывода могут быть недоступны для наблюдения, если устройство использует сквозной канал или SR-IOV.
- **В хост-системе с аппаратной поддержкой (Nitro):** использование SR-IOV может затруднить наблюдение из гипервизора за операциями ввода/вывода с устройствами, потому что гость обращается к оборудованию напрямую, а не через агента или хост-ядро. (Информация о том, как в Amazon на самом деле обеспечивают наблюдаемость на уровне гипервизора Nitro, не разглашается.)
- **В гостевой системе:** можно наблюдать за виртуализированными ресурсами и их потреблением в гостевой системе, а также делать выводы о проблемах с физическими ресурсами. Поскольку виртуальная машина имеет собственное выделенное ядро, можно изучить внутреннюю работу ядра и использовать для этого все инструменты трассировки, включая инструменты на основе BPF.

В привилегированной гостевой или хост-системе (гипервизоры Xen или KVM) можно получить общую информацию об использовании физических ресурсов: потребление, насыщение, ошибки, IOPS, пропускная способность, тип ввода/вывода. Обычно эта информация доступна для каждого гостя в отдельности, что позволяет быстро выявить активных пользователей. Подробная информация о том, какие гостевые процессы производят ввод/вывод, и их трассировки стеков на этом уровне недоступны. Но все это можно получить, войдя в гостевую систему (при условии, что настроены необходимые средства, например SSH) и используя инструменты наблюдения, которые она предлагает.

Когда используется сквозной канал или SR-IOV, гость может выполнять ввод/вывод, обращаясь непосредственно к оборудованию, в обход гипервизора и статистик, которые тот обычно собирает. В результате операции ввода/вывода могут стать невидимыми для гипервизора и не отображаться в `iostat(1)` или других инструментах. Одно из возможных решений — использовать счетчики производительности PMCS, связанные с вводом/выводом, и делать выводы на основе их значений.

Чтобы определить корень проблемы с производительностью гостевой системы, оператору облака может потребоваться войти в обе системы — гостевую и хост-систему — и запустить инструменты наблюдения в обеих. Трассировка путей ввода/вывода осложняется из-за задействованных шагов и может также

включать анализ внутренней работы гипервизора и агента ввода/вывода, если он используется.

В рядовой гостевой системе использование физических ресурсов может быть вообще недоступно для наблюдения. Из-за этого клиенты склонны обвинять в загадочных проблемах с производительностью и нехваткой ресурсов «шумных соседей». Для спокойствия клиентов (и сокращения количества обращений в службу поддержки) информация об использовании физических ресурсов (отредактированная) может предоставляться другими путями, включая SNMP или облачный API.

Чтобы упростить наблюдение за работой контейнеров, есть различные решения мониторинга, которые предоставляют графики, дашборды и ориентированные графы для представления информации о контейнерной среде. Среди них Google cAdvisor [Google 20d] и Cilium Hubble [Cilium 19] (оба распространяются с открытым исходным кодом).

В разделах ниже показаны простые инструменты наблюдения, которые можно использовать в разных местах, и описана стратегия анализа производительности. Для примера используются Xen и KVM (Nitro не рассматривается, потому что это собственность Amazon).

11.2.4.1. Привилегированный гость/хост

Все ресурсы системы (процессоры, память, файловая система, диск, сеть) должны быть доступными для наблюдения с использованием инструментов, описанных в предыдущих главах (за исключением ввода/вывода через сквозной канал или SR-IOV).

Xen

При использовании Xen-подобных гипервизоров гостевые виртуальные процессоры есть в гипервизоре и не видны для стандартных инструментов ОС из привилегированного гостя (dom0). В Xen вместо них используйте xentop(1):

```
# xentop
xentop - 02:01:05 Xen 3.3.2-rc1-xvm
2 domains: 1 running, 1 blocked, 0 paused, 0 crashed, 0 dying, 0 shutdown
Mem: 50321636k total, 12498976k used, 37822660k free CPUs: 16 @ 2394MHz
      NAME STATE CPU(sec) CPU(%) MEM(k) MEM(%) MAXMEM(k) MAXMEM(%) VCPUS NETS
NETTX(k) NETRX(k) VBDS VBD_OO VBD_RD VBD_WR SSID
Domain-0 -----r 6087972 2.6 9692160 19.3 no limit n/a 16 0
0 0 0 0 0 0
Doogle_Win --b--- 172137 2.0 2105212 4.2 2105344 4.2 1 2
0 0 2 0 0 0
[...]
```

Особый интерес представляют эти столбцы:

- **CPU(%)**: потребление процессора в процентах (в сумме для всех процессоров);
- **MEM(k)**: потребление основной памяти (в килобайтах);

- **MEM(%)**: потребление основной памяти в процентах от общего объема системной памяти;
- **MAXMEM(k)**: максимальный объем основной памяти (в килобайтах);
- **MAXMEM(%)**: максимальный объем основной памяти в процентах от общего объема системной памяти;
- **VCPUS**: количество виртуальных процессоров, выделенных гостю;
- **NETS**: количество виртуализированных сетевых интерфейсов;
- **NETTX(k)**: передано в сеть (в килобайтах);
- **NETRX(k)**: принято из сети (в килобайтах);
- **VBDS**: количество виртуальных блочных устройств;
- **VBD_OO**: количество обращений к виртуальному блочному устройству, заблокированных и поставленных в очередь (насыщение);
- **VBD_RD**: запросов на чтение из виртуального блочного устройства;
- **VBD_WR**: запросов на запись в виртуальное блочное устройство.

По умолчанию `xentop(1)` обновляет вывод каждые 3 с, но этот интервал можно изменить с помощью параметра `-d delay_secs`.

Для расширенного анализа особенностей работы Xen есть инструмент `xentrace(8)`, который может извлекать из гипервизора журнал с фиксированными типами событий. Этот журнал можно проанализировать с помощью `xenanalyze` на наличие проблем планирования в гипервизоре и узнать, какой планировщик процессора используется. В исходном коде Xen также есть инструмент `xenoprof` — профилировщик для Xen (MMU и гости).

KVM

При использовании KVM-подобных гипервизоров гостевые экземпляры доступны для наблюдения из хост-системы. Например:

```
host$ top
top - 15:27:55 up 26 days, 22:04,  1 user,  load average: 0.26, 0.24, 0.28
Tasks: 499 total,   1 running, 408 sleeping,   2 stopped,   0 zombie
%Cpu(s): 19.9 us,  4.8 sy,  0.0 ni, 74.2 id,  1.1 wa,  0.0 hi,  0.1 si,  0.0 st
KiB Mem : 24422712 total,  6018936 free, 12767036 used,  5636740 buff/cache
KiB Swap: 32460792 total, 31868716 free,   592076 used.  8715220 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM     TIME+ COMMAND
24881 libvirt+  20   0 6161864 1.051g 19448 S 171.9   4.5   0:25.88 qemu-system-x86

21897 root        0 -20       0       0       0 I   2.3   0.0   0:00.47 kworker/u17:8
23445 root        0 -20       0       0       0 I   2.3   0.0   0:00.24 kworker/u17:7
15476 root        0 -20       0       0       0 I   2.0   0.0   0:01.23 kworker/u17:2
23038 root        0 -20       0       0       0 I   2.0   0.0   0:00.28 kworker/u17:0

22784 root        0 -20       0       0       0 I   1.7   0.0   0:00.36 kworker/u17:1
[...]
```


Процесс `qemu-system-x86` — это гостевой гипервизор KVM, в рамках которого выполняются потоки для каждого vCPU и потоки агентов ввода/вывода. Общее потребление процессора гостем можно увидеть в верхней части вывода `top(1)`, а потребление каждого виртуального процессора определить с помощью других инструментов, например `pidstat(1)`:

```
host$ pidstat -tp 24881 1
03:40:44 PM  UID  TGID  TID  %usr %system %guest %wait  %CPU CPU Command
03:40:45 PM 64055 24881  - 17.00 17.00 147.00 0.00 181.00 0 qemu-system-x86
03:40:45 PM 64055  - 24881 9.00 5.00 0.00 0.00 14.00 0 |__qemu-system-x86
03:40:45 PM 64055  - 24889 0.00 0.00 0.00 0.00 0.00 6 |__qemu-system-x86
03:40:45 PM 64055  - 24897 1.00 3.00 69.00 1.00 73.00 4 |__CPU 0/KVM
03:40:45 PM 64055  - 24899 1.00 4.00 79.00 0.00 84.00 5 |__CPU 1/KVM
03:40:45 PM 64055  - 24901 0.00 0.00 0.00 0.00 0.00 2 |__vnc_worker
03:40:45 PM 64055  - 25811 0.00 0.00 0.00 0.00 0.00 7 |__worker
03:40:45 PM 64055  - 25812 0.00 0.00 0.00 0.00 0.00 6 |__worker
[...]
```

Как показывает этот вывод, потоки процессоров с именами `CPU 0/KVM` и `CPU 1/KVM` потребляют 73 % и 84 % процессорного времени соответственно.

Для отображения процессов QEMU в имена соответствующих гостевых экземпляров обычно достаточно изучить аргументы процесса (`ps -wwfp PID`) и прочитать параметр `-name`.

Еще одна важная область для анализа — выходы в гипервизор на гостевых виртуальных процессорах. Типы выходов могут подсказать, что делает гость: бездействует, выполняет ввод/вывод или производит вычисления. В Linux команда `perf(1)` имеет подкоманду `kvm`, которая сообщает высокоуровневые статистики по выходам в KVM, например:

```
host# perf kvm stat live
11:12:07.687968

Analyze events for all VMs, all VCPUs:
      VM-EXIT Samples Samples% Time% Min Time    Max Time      Avg time
      MSR_WRITE 1668   68.90% 0.28% 0.67us    31.74us    3.25us ( +- 2.20% )
      HLT       466   19.25% 99.63% 2.61us 100512.98us 4160.68us ( +- 14.77% )
PREEMPTION_TIMER 112    4.63% 0.03% 2.53us   10.42us    4.71us ( +- 2.68% )
PENDING_INTERRUPT 82    3.39% 0.01% 0.92us   18.95us    3.44us ( +- 6.23% )
EXTERNAL_INTERRUPT 53    2.19% 0.01% 0.82us    7.46us    3.22us ( +- 6.57% )
IO_INSTRUCTION  37    1.53% 0.04% 5.36us   84.88us   19.97us ( +- 11.87% )
      MSR_READ   2    0.08% 0.00% 3.33us    4.80us    4.07us ( +- 18.05% )
      EPT_MISCONFIG 1    0.04% 0.00% 19.94us   19.94us   19.94us ( +- 0.00% )
```

```
Total Samples:2421, Total events handled time:1946040.48us.
[...]
```

Этот вывод показывает причины выхода виртуальной машины в гипервизор и статистику по каждой причине. Наиболее продолжительные выходы в этом примере были обусловлены операцией `HLT` (остановка), выполняя которую виртуальные процессоры переходят в состояние ожидания. В столбцах выводятся:

- **VM-EXIT**: тип выхода;
- **Samples**: число выходов, зафиксированных за период трассировки;
- **Samples%**: доля от общего числа выходов в процентах;
- **Time%**: продолжительность от общего времени всех выходов в процентах;
- **Min Time**: минимальная продолжительность выхода;
- **Max Time**: максимальная продолжительность выхода;
- **Avg time**: средняя продолжительность выхода.

Даже при том, что оператору непросто наблюдать за внутренней работой гостевой виртуальной машины, изучение статистик выходов позволяет охарактеризовать то, как переход на виртуализацию оборудования влияет на арендатора. Если наблюдается небольшое количество выходов и большинство обусловлено выполнением инструкций HLT, то можно быть уверенными, что гостевой процессор бездействует долгое время. С другой стороны, если выполняется большое количество операций ввода/вывода с прерываниями, как сгенерированными, так и внедренными в гостевую систему, то весьма вероятно, что гость активно выполняет ввод/вывод через свои виртуальные сетевые адаптеры и диски.

Для расширенного анализа KVM есть множество точек трассировки:

```
host# perf list | grep kvm
kvm:kvm_ack_irq          [Tracepoint event]
kvm:kvm_age_page         [Tracepoint event]
kvm:kvm_apic              [Tracepoint event]
kvm:kvm_apic_accept_irq  [Tracepoint event]
kvm:kvm_apic_ipi          [Tracepoint event]
kvm:kvm_async_pf_completed [Tracepoint event]
kvm:kvm_async_pf_doublefault [Tracepoint event]
kvm:kvm_async_pf_not_present [Tracepoint event]
kvm:kvm_async_pf_ready    [Tracepoint event]
kvm:kvm_avic_incomplete_ipi [Tracepoint event]
kvm:kvm_avic_unaccelerated_access [Tracepoint event]
kvm:kvm_cpuid             [Tracepoint event]
kvm:kvm_cr                [Tracepoint event]
kvm:kvm_emulate_insn      [Tracepoint event]
kvm:kvm_enter_smm         [Tracepoint event]
kvm:kvm_entry             [Tracepoint event]
kvm:kvm_eoi               [Tracepoint event]
kvm:kvm_exit              [Tracepoint event]
[...]
```

Особый интерес представляют `kvm:kvm_exit` (упоминалась выше) и `kvm:kvm_entry`. Вот как можно получить список аргументов `kvm:kvm_exit` с помощью `bpfttrace`:

```
host# bpfttrace -lv t:kvm:kvm_exit
tracepoint:kvm:kvm_exit
    unsigned int exit_reason;
    unsigned long guest_rip;
    u32 isa;
    u64 info1;
    u64 info2;
```

Как видите, на основании аргументов можно узнать причину выхода (`exit_reason`), указатель гостевой инструкции для возврата (`guest_rip`) и другие детали. При трассировке `kvm:kvm_exit` в паре с `kvm:kvm_entry`, которая срабатывает при выполнении входа в гостевую систему из KVM (по завершении выхода в гипервизор), можно измерить продолжительность и узнать причину выхода. В книге «BPF Performance Tools» [Gregg 19] я опубликовал инструмент `kvmexits.bt` для `bpfttrace`, обобщающий причины выхода в виде гистограммы (он доступен в моем репозитории [Gregg 19e]). Вот пример вывода:

```
host# kvmexits.bt
Attaching 4 probes...
Tracing KVM exits. Ctrl-C to end
^C
[...]

@exit_ns[30, IO_INSTRUCTION]:
[1K, 2K)          1 |
[2K, 4K)          12 | @@@
[4K, 8K)          71 | @@@@@@@@@@@@@@@@@@
[8K, 16K)         198 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[16K, 32K)        129 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[32K, 64K)         94 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[64K, 128K)        37 | @@@@@@@@@@@@@@@@@@
[128K, 256K)       12 | @@@
[256K, 512K)       23 | @@@@@@@
[512K, 1M)         2 |
[1M, 2M)           0 |
[2M, 4M)           1 |
[4M, 8M)           2 |

@exit_ns[48, EPT_VIOLATION]:
[512, 1K)         6160 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[1K, 2K)          6885 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[2K, 4K)          7686 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[4K, 8K)          2220 | @@@@@@@@@@@@@@@@@@@
[8K, 16K)          582 | @@@
[16K, 32K)         244 | @
[32K, 64K)         47 |
[64K, 128K)         3 |
```

Вывод включает гистограммы для всех типов выходов, но здесь показаны только две. Судя по этим гистограммам, выходы `IO_INSTRUCTION` длятся обычно менее 512 мкс, но иногда случаются выбросы от 2 до 8 мс.

Другой пример расширенного анализа — профилирование содержимого регистра CR3. Каждый процесс в гостевой системе имеет собственное адресное пространство и набор таблиц страниц для преобразования виртуальных адресов в физические. Ссылка на эту таблицу страниц хранится в регистре CR3. Путем выборки регистра CR3 из хост-системы (например, с помощью `bpfttrace`) можно определить, активен ли какой-то один процесс в гостевой системе (одно и то же значение в CR3) или она переключается между несколькими процессами (разные значения CR3).

Для получения дополнительной информации нужно авторизоваться в гостевой системе.

11.2.4.2. Гость

При виртуализации оборудования из гостевой системы можно наблюдать только виртуальные устройства (если не используется сквозной канал/SR-IOV), включая количество виртуальных процессоров, выделенных гостю. Вот пример проверки процессоров гостя KVM с помощью `mpstat(1)`:

```
kvm-guest$ mpstat -P ALL 1
Linux 4.15.0-91-generic (ubuntu0) 03/22/2020 _x86_64_ (2 CPU)
10:51:34 PM CPU %usr %nice %sys %iowait %irq %soft %steal %guest %gnice %idle
10:51:35 PM all 14.95 0.00 35.57 0.00 0.00 0.00 0.00 0.00 0.00 49.48
10:51:35 PM 0 11.34 0.00 28.87 0.00 0.00 0.00 0.00 0.00 0.00 59.79
10:51:35 PM 1 17.71 0.00 42.71 0.00 0.00 0.00 0.00 0.00 0.00 39.58

10:51:35 PM CPU %usr %nice %sys %iowait %irq %soft %steal %guest %gnice %idle
10:51:36 PM all 11.56 0.00 37.19 0.00 0.00 0.00 0.50 0.00 0.00 50.75
10:51:36 PM 0 8.05 0.00 22.99 0.00 0.00 0.00 0.00 0.00 0.00 68.97
10:51:36 PM 1 15.04 0.00 48.67 0.00 0.00 0.00 0.00 0.00 0.00 36.28
[...]
```

Этот вывод показывает состояние всего двух гостевых процессоров.

Команда `vmstat(8)` в Linux выводит столбец с процентом недополученного (stolen) процессорного времени (`st`), и это редкий пример статистики виртуализации. Недополученное время — это процессорное время, недополученное гостем: оно могло быть использовано другими арендаторами или функциями гипервизора (например, для обслуживания собственного ввода/вывода или из-за ограничений, наложенных на экземпляр):

```
xen-guest$ vmstat 1
procs -----memory----- ---swap-- ----io---- -system-- -----cpu-----
r b swpd free buff cache si so bi bo in cs us sy id wa st
1 0 0 107500 141348 301680 0 0 0 0 1006 9 99 0 0 0 1
1 0 0 107500 141348 301680 0 0 0 0 1006 11 97 0 0 0 3
1 0 0 107500 141348 301680 0 0 0 0 978 9 95 0 0 0 5
3 0 0 107500 141348 301680 0 0 0 4 912 15 99 0 0 0 1
2 0 0 107500 141348 301680 0 0 0 0 33 7 3 0 0 0 97
3 0 0 107500 141348 301680 0 0 0 0 34 6 100 0 0 0 0
5 0 0 107500 141348 301680 0 0 0 0 35 7 1 0 0 0 99
2 0 0 107500 141348 301680 0 0 0 48 38 16 2 0 0 0 98
[...]
```

В этом примере тестировался гость Xen с агрессивной политикой ограничения потребления процессора. За первые 4 с более 90 % процессорного времени было потрачено в пользовательском режиме гостя, при этом недополучено было всего несколько процентов. Затем поведение начинает резко меняться, и гость недополучает большую часть процессорного времени.

Анализ потребления процессора на уровне тактов часто требует использования аппаратных счетчиков (см. главу 4 «Инструменты наблюдения», раздел 4.3.9 «Аппаратные счетчики (РМС)»). Они могут быть доступны или недоступны гостю в зависимости от конфигурации гипервизора. В Xen, например, есть модуль мониторинга

производительности виртуальной среды (virtual performance monitoring unit, vpmu), обеспечивающий доступность счетчиков РМС для гостевых систем и поддерживающий настройки, позволяющие определить, какие счетчики РМС будут доступны [Gregg 17f].

Поскольку дисковые и сетевые устройства виртуализированы, важной метрикой для анализа является *задержка*. Она показывает, как устройство реагирует на виртуализацию, ограничения и действия других арендаторов. Такие показатели, как процент занятости, трудно интерпретировать, не зная характеристик фактического устройства.

Задержку устройства можно подробно изучить с помощью инструментов трассировки ядра, включая perf(1), Ftrace и BPF (главы 13, 14 и 15). К счастью, все они должны работать в гостевых системах, потому что имеют собственные ядра, а пользователь root обладает неограниченным доступом к ядру. Вот пример запуска biosnoop(8) для BPF в гостевой системе KVM:

```
kvm-guest# biosnoop
TIME(s)      COMM          PID  DISK  T  SECTOR    BYTES    LAT(ms)
0.000000000  systemd-journ 389   vda   W 13103112   4096     3.41
0.001647000  jbd2/vda2-8   319   vda   W 8700872   360448    0.77
0.011814000  jbd2/vda2-8   319   vda   W 8701576   4096     0.20
1.711989000  jbd2/vda2-8   319   vda   W 8701584   20480    0.72
1.718005000  jbd2/vda2-8   319   vda   W 8701624   4096     0.67
[...]
```

Этот вывод показывает задержку устройства виртуального диска. Обратите внимание, что с контейнерами (раздел 11.3 «Виртуализация операционной системы») эти инструменты трассировки ядра могут не работать, из-за чего конечный пользователь не сможет детально анализировать работу устройств ввода/вывода и др.

11.2.4.3. Стратегия

В предыдущих главах мы рассмотрели методы анализа потребления физических ресурсов, которые могут использоваться администраторами физических систем для поиска узких мест и ошибок. Также изучают средства управления, выделяющие ресурсы для гостей. Так можно проверить, как часто гости достигают установленных пределов, и при необходимости проинформировать их и побудить к обновлению. Без входа в гостевую систему, что может потребоваться для любого серьезного исследования производительности, администраторы мало что могут определить¹.

В гостевых системах можно использовать инструменты и стратегии анализа потребления ресурсов, описанные в предыдущих главах. Но важно помнить, что

¹ Рецензент подсказал еще один возможный метод (обратите внимание, что это не рекомендация): можно проанализировать моментальный снимок гостевого хранилища (при условии, что он не шифруется). Например, с учетом журналирования адресов предыдущих операций дискового ввода/вывода моментальный снимок состояния файловой системы можно использовать, чтобы определить, к каким файлам происходили обращения.

ресурсы в этом случае обычно виртуальные. Потребление некоторых ресурсов невозможно довести до предела из-за невидимого контроля со стороны гипервизора или конкуренции с другими арендаторами. В идеальном случае облачное ПО или облачный провайдер предоставляют клиентам возможность проверять потребление физических ресурсов (с некоторыми ограничениями), чтобы они могли сами анализировать проблемы с производительностью. В других случаях выводы о конфликтах и ограничениях можно сделать на основе увеличения задержки ввода/вывода и планирования процессора. Такую задержку можно измерить на уровне системных вызовов или в гостевом ядре.

Стратегия, которую я использую для выявления конкуренции за дисковые и сетевые ресурсы из гостевой системы, заключается в тщательном анализе закономерностей ввода/вывода. Сюда может входить журналирование вывода `biosnoop(8)` (см. предыдущий пример), изучение последовательностей ввода/вывода на наличие каких-либо выбросов по задержке и их причин (ввод/вывод больших объемов данных выполняется медленнее), их закономерностей доступа (например, когда операция чтения ставится в очередь после выталкивания буферов, заполненных операциями записи). Если нет явных причин, то, возможно, мы имеем дело с физической конкуренцией или с проблемой в устройстве.

11.3. ВИРТУАЛИЗАЦИЯ ОПЕРАЦИОННОЙ СИСТЕМЫ

Виртуализация операционной системы делит ее на экземпляры, которые в Linux называются *контейнерами*. Контейнеры действуют как отдельные гостевые серверы и могут управляться и перезагружаться независимо от хост-системы. Эта технология позволяет получить небольшие, эффективные, быстро загружаемые экземпляры для облачных клиентов и серверы высокой плотности для облачных операторов. На рис. 11.9 показаны гостевые экземпляры в технологии виртуализации ОС.

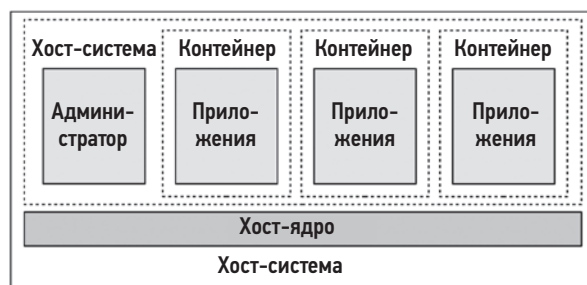


Рис. 11.9. Виртуализация операционной системы

Этот подход уходит корнями в команду Unix `chroot(8)`, которая ограничивает процесс поддеревом глобальной файловой системы Unix (изменяет каталог верхнего уровня, «/», видимый процессом). В 1998 году в FreeBSD было предложено дальнейшее развитие этой технологии в виде *клеток FreeBSD* (FreeBSD jails),

предоставлявшее защищенные ячейки, действующие как отдельные серверы. В 2005 году в Solaris 10 появилась версия *Solaris Zones* со средствами управления ресурсами. В то же время в Linux была добавлена возможность частичной изоляции процессов: в 2002 году в Linux 2.4.19 были добавлены *пространства имен*, и в 2008 году в Linux 2.6.24 — группы управления (cgroups) [Corbet 07a], [Corbet 07b], [Linux 20m]. Пространства имен и контрольные группы объединяются для создания контейнеров, которые обычно используются механизмом *seccomp-bpf*, а также средствами управления доступом к системным вызовам.

Ключевое отличие от технологий виртуализации оборудования в том, что работает только одно ядро. Ниже перечислены преимущества контейнеров по сравнению с виртуальными машинами при виртуализации оборудования (раздел 11.2 «Виртуализация оборудования»):

- короткое время инициализации: обычно не превышает нескольких миллисекунд;
- гости могут использовать всю память для приложений (без затрат на запуск собственного ядра);
- единый кэш файловой системы — это позволяет избежать двойного кэширования хостом и гостем;
- более точное управление совместным использованием ресурсов (контрольные группы);
- для операторов хоста: улучшенная наблюдаемость производительности благодаря возможности наблюдать за гостевыми процессами и их взаимодействиями;
- контейнеры могут совместно использовать страницы памяти для общих файлов, что обеспечивает более эффективное использование кэша страниц и увеличивает коэффициент попаданий в кэш процессора;
- процессоры — это настоящие физические процессоры; предположения, сделанные на основе адаптивных блокировок мьютексов, остаются в силе.

Есть и минусы:

- выше конкуренция за ресурсы ядра (блокировки, кэши, буферы, очереди);
- для гостей: хуже наблюдаемость производительности, так как обычно нет возможности проанализировать работу ядра;
- любой фатальный сбой в ядре оказывает фатальное влияние на всех гостей;
- гости не могут запускать собственные модули ядра;
- гости не могут запускать профильные ядра PGO (см. раздел 3.5.1 «Ядра PGO»);
- гости не могут запускать другие версии ядра¹.

¹ Есть технологии эмуляции других интерфейсов системных вызовов, чтобы другие ОС могли работать под управлением ядра, но на практике такая эмуляция отрицательно влияет на производительность. Кроме того, такие технологии обычно эмулируют только базовый набор возможностей системных вызовов и при попытке обратиться к расширенным возможностям возвращают код ошибки ENOTSUP (не поддерживается).

Рассмотрим два первых недостатка: после переноса из виртуальной машины в контейнер гость почти наверняка столкнется с проблемой конкуренции за ядро, а также потеряет возможность анализировать его работу. Они станут более зависимыми от оператора хоста для такого рода анализа.

Недостаток контейнеров, не связанный с производительностью, заключается в том, что контейнеры считаются менее безопасными, так как используют одно общее ядро.

Все эти недостатки решаются легковесной виртуализацией, описанной в разделе 11.4 «Легковесная виртуализация», хотя и за счет некоторых преимуществ.

В разделах ниже описаны особенности виртуализации ОС в Linux: реализация, оверхед, управление ресурсами и наблюдаемость.

11.3.1. Реализация

В ядре Linux нет понятия контейнера. Но есть пространства имен и контрольные группы (cgroups), которые используются ПО, действующим в пространстве пользователя (например, Docker), для создания так называемых *контейнеров*¹. На рис. 11.10 показана типичная конфигурация контейнера.

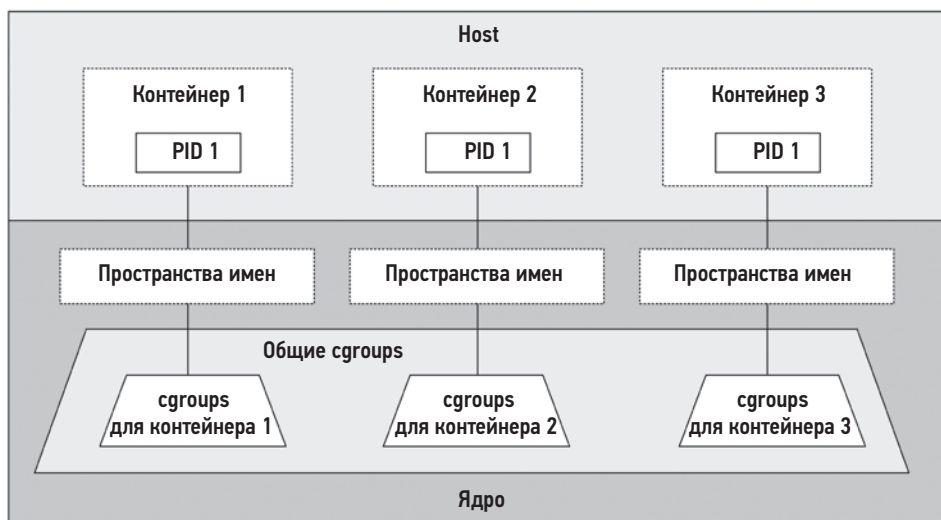


Рис. 11.10. Контейнеры в Linux

¹ Для связи процесса с контрольными группами в ядре используется структура `nsproху`. Поскольку эта структура определяет, как контейнеризуется процесс, ее можно считать лучшим представлением понятия контейнера в ядре.

Несмотря на то что в каждом контейнере есть процесс с идентификатором PID 1, это разные процессы, потому что принадлежат разным пространствам имен.

Поскольку чаще всего для развертывания контейнеров используется Kubernetes, его архитектура изображена на рис. 11.11. О Kubernetes шла речь в разделе 11.1.6 «Оркестровка (Kubernetes)».

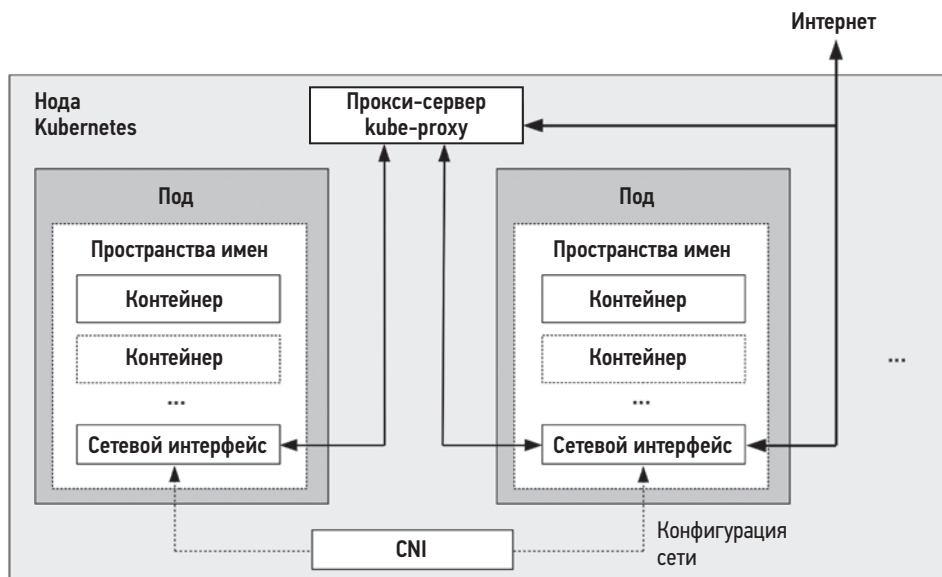


Рис. 11.11. Нода Kubernetes

На рис. 11.11 также показана связь по сети между подами (группами контейнеров) через прокси-сервер kube-proxu и между контейнерами через сетевой интерфейс контейнеров (container network interface, CNI).

Преимущество Kubernetes — это возможность с легкостью создать несколько контейнеров для совместного использования одних и тех же пространств имен, включив их в один под (группу). Это позволяет использовать более быстрые методы связи между контейнерами.

Пространства имен

Пространство имен фильтрует представление системы, поэтому контейнеры могут видеть и администрировать только собственные процессы, точки монтирования и другие ресурсы. Это основной механизм изоляции контейнеров друг от друга. В табл. 11.1 перечислены некоторые из пространств имен.

Таблица 11.1. Некоторые пространства имен в Linux

Пространство имен	Описание
cgroup	Для видимости контрольных групп
ipc	Для видимости межпроцессных взаимодействий
mnt	Для монтирования файловых систем
net	Для изоляции сетевого стека: управляет видимостью интерфейсов, сокетов, маршрутов и т. д.
pid	Для видимости процессов: фильтрует содержимое /proc
time	Для поддержки отдельных часов в каждом контейнере
user	Для идентификаторов пользователей
uts	Для информации о хосте; системный вызов uname(2)

Список текущих пространств имен в системе можно получить с помощью `lsns(8)`:

```
# lsns
      NS TYPE  NPROCS  PID USER      COMMAND
4026531835 cgroup   105    1 root      /sbin/init
4026531836 pid       105    1 root      /sbin/init
4026531837 user     105    1 root      /sbin/init
4026531838 uts       102    1 root      /sbin/init
4026531839 ipc       105    1 root      /sbin/init
4026531840 mnt        98    1 root      /sbin/init
4026531860 mnt         1   19 root      kdevtmpfs
4026531992 net       105    1 root      /sbin/init
4026532166 mnt         1  241 root      /lib/systemd/systemd-udev
4026532167 uts         1  241 root      /lib/systemd/systemd-udev
[...]
```

Как показывает вывод команды `lsns(8)`, процесс `init` имеет шесть разных пространств имен, используемых более чем 100 процессами.

Описание пространств имен можно найти в исходном коде Linux, а также на страницах справочного руководства `man`, начиная с `namespaces(7)`.

Контрольные группы

Контрольные группы (`cgroups`) ограничивают использование ресурсов. В ядре Linux есть две версии контрольных групп: 1 и 2¹. Многие проекты, такие как Kubernetes, все еще используют версию 1 (версия 2 находится в разработке). В табл. 11.2 перечислены некоторые контрольные группы версии 1.

Эти контрольные группы можно настроить для ограничения конкуренции за ресурсы между контейнерами, например, установив жесткие ограничения на потребление процессора и памяти или более мягкие (на основе относительных долей)

¹ Есть и смешанные конфигурации, в которых одновременно используются обе версии.

ограничения на потребление процессора и диска. Кроме того, контрольные группы можно организовать в иерархию, включающую системные контрольные группы, которые совместно используются контейнерами (см. рис. 11.10).

Таблица 11.2. Некоторые контрольные группы в Linux

Контрольная группа	Описание
bikio	Ограничивает блочный (дисковый) ввод/вывод: байты и частоту операций
cpu	Ограничивает потребление процессора на основе относительных долей
cpuacct	Учитывает потребление процессора процессами группы
cpuset	Назначает контейнерам процессоры и узлы памяти
devices	Управляет доступом к устройствам
hugetlb	Ограничивает использование огромных страниц
memory	Ограничивает объем памяти для процессов и ядра, а также использование подкачки
net_cls	Присваивает сетевым пакетам идентификатор класса (classid) для использования дисциплинами очередей и брандмауэрами
net_prio	Устанавливает приоритеты для сетевых интерфейсов
perf_event	Позволяет утилите perf наблюдать за процессами в контрольной группе
pids	Ограничивает количество процессов, которое можно запустить
rdma	Ограничивает потребление ресурсов RDMA (Remote Direct Memory Access — прямой доступ к удаленной памяти) и InfiniBand (высокоскоростная коммутируемая сеть)

Контрольные группы версии 2 изначально основаны на иерархии и свободны от различных недостатков версии 1. Ожидается, что в ближайшие годы контейнерные технологии будут переведены на версию 2, а версия 1 будет считаться устаревшей. Дистрибутив Fedora в версии 31, выпущенной в 2019 году, уже перешел на использование контрольных групп версии 2.

Документацию по контрольным группам можно найти в исходном коде Linux в Documentation/cgroup-v1 и Documentation/admin-guide/cgroup-v2.rst, а также на странице справочного руководства cgroups(7).

В разделах ниже описаны такие темы, связанные с виртуализацией контейнеров, как оверхед, управление ресурсами и наблюдаемость. Они различаются для разных реализаций контейнеров и конфигураций.

11.3.2. Оверхед

Контейнеры обычно дают незначительный оверхед. Потребление процессоров и потребление памяти приложениями в контейнере не должны отличаться от случая, когда те же приложения выполняются на «голом железе». Однако в ядре

могут производиться некоторые дополнительные вызовы при вводе/выводе из-за дополнительных слоев на пути к файловой системе и сети. Намного большую угрозу для производительности представляет конкуренция между несколькими арендаторами, так как контейнеры способствуют интенсивному совместному использованию ядра и физических ресурсов. В разделах ниже в общих чертах описывается оверхед, обусловленный выполнением на процессоре, отображением памяти, операциями ввода/вывода и конкуренцией со стороны других арендаторов.

Процессор

Когда поток контейнера выполняется в пользовательском режиме, то прямых оверхедов на потребление процессора нет: потоки выполняются непосредственно на процессоре, пока либо добровольно не уступят процессор, либо не будут вытеснены планировщиком. Также в Linux нет дополнительного оверхеда процессорного времени на запуск процессов в пространствах имен и контрольных группах: все процессы уже выполняются в пространстве имен и контрольных группах по умолчанию, независимо от того, используются контейнеры или нет.

Чаще всего снижение вычислительной производительности объясняется конкуренцией с другими арендаторами (см. следующий раздел «Конкуренция между арендаторами»).

При использовании ПО оркестровки, такой как Kubernetes, дополнительные сетевые компоненты могут использовать немного больше вычислительных ресурсов для обработки сетевых пакетов. Например, при наличии большого количества служб (измеряемого тысячами), Kube-proxu тратит довольно много процессорного времени на обслуживание первого пакета, когда требуется проверить его соответствие большим наборам правил iptables. Этот оверхед можно уменьшить, заменив kube-proxu на BPF [Borkmann 20].

Отображение памяти

Отображение памяти, загрузка и сохранение должны выполняться без дополнительного оверхеда.

Размер памяти

Приложения могут использовать весь объем памяти, выделенной контейнеру. Сравните это с виртуализацией оборудования, когда для каждого арендатора запускается отдельная виртуальная машина со своим ядром, причем каждому ядру требуется для работы некоторый объем оперативной памяти.

Обычно конфигурация контейнеров (с использованием OverlayFS) позволяет всем вместе использовать один кэш страниц при обращении к тем же файлам.

Это помогает эффективнее использовать память по сравнению с виртуальными машинами, которые загружают в память свои копии общих файлов (например, системных библиотек).

Ввод/вывод

Оверхед ввода/вывода зависит от конфигурации контейнера, потому что может включать дополнительные уровни для большей изоляции:

- **ввода/вывода через файловую систему:** например, overlayfs;
- **сетевого ввода/вывода:** например, доступ в сеть через мост.

Ниже показана трассировка стека ядра для операции записи в файловую систему из контейнера, которая была обработана overlayfs (и файловой системой XFS):

```
blk_mq_make_request+1
generic_make_request+420
submit_bio+108
__xfs_buf_ioapply+798
__xfs_buf_submit+226
xlog_bdstrat+48
xlog_sync+703
__xfs_log_force_lsn+469
xfs_log_force_lsn+143
xfs_file_fsync+244
xfs_file_buffered_aio_write+629
do_iter_readv_writev+316
do_iter_write+128
ovl_write_iter+376
__vfs_write+274
vfs_write+173
ksys_write+90
do_syscall_64+85
entry_SYSCALL_64_after_hwframe+68
```

Обращение к overlayfs можно увидеть в этой трассировке стека в виде вызова функции `ovl_write_iter()`.

Насколько это важно, зависит от рабочей нагрузки и частоты операций ввода/вывода. Для серверов с невысокой частотой операций ввода/вывода (скажем, < 1000 операций в секунду) оверхед не должен быть значительным.

Конкуренция между арендаторами

Присутствие других работающих арендаторов может привести к конкуренции за ресурсы и прерывания, которые снижают производительность:

- Кэши процессоров могут иметь более низкий коэффициент попаданий из-за того, что другие арендаторы будут добавлять свои и вытеснять чужие данные

из кэша. При переключении контекста между потоками контейнеров некоторые процессоры и конфигурации ядра могут даже очищать кэш первого уровня L1¹.

- Кэши TLB тоже могут иметь более низкий коэффициент попаданий из-за действий другого арендатора, а также из-за сбрасывания кэша при переключении контекста (чего можно избежать, если использовать идентификаторы контекста процесса (process-context ID, PCID)).
- Выполнение инструкций процессора может прерываться на короткие периоды для обслуживания устройств, с которыми работают другие арендаторы (например, сетевых интерфейсов).
- В ядре может возникать дополнительная конкуренция за буферы, кэши, очереди и блокировки, потому что нагрузка на мультиарендную контейнерную систему может увеличиться на порядок или даже больше. Такая конкуренция может снизить производительность приложения в зависимости от используемого ресурса ядра и характеристик его масштабируемости.
- Сетевой ввод/вывод может сопровождаться увеличенным потреблением процессора из-за использования iptables для реализации сети контейнеров.
- Возможна конкуренция за системные ресурсы (процессоры, диски, сетевые интерфейсы) со стороны других арендаторов, которые тоже их используют.

В своей статье Джанлука Борелло (Gianluca Borello) описывает, как было обнаружено медленное и неуклонное ухудшение производительности контейнера с течением времени, когда в системе были некоторые другие контейнеры [Borello 17]. Он выяснил, что ухудшение было обусловлено увеличенной задержкой `lstat(2)` из-за рабочей нагрузки другого контейнера и ее влиянием на кэш каталогов `dcache`.

Еще одна проблема, о которой сообщил Максим Леонович, сопровождала переходу от одноарендных виртуальных машин к мультиарендным контейнерам, что привело к увеличению количества вызовов `posix_fadvise()` в ядре и созданию узкого места [Leonovich 18].

Последний пункт в списке относится к средствам управления ресурсами. Некоторые из упомянутых факторов есть и в традиционной многопользовательской среде, но особенно сильно они проявляются именно в мультиарендной контейнерной системе.

¹ Например, в июне 2020 года Линус Торвалдс отклонил исправление для ядра, которое позволяло процессам производить очистку кэша данных L1 [Torvalds 20b]. Исправление, как предполагалось, должно было повысить безопасность в облачных средах, но было отклонено из-за опасений снижения производительности в тех случаях, когда в такой очистке нет никакой необходимости. Хотя это исправление не было включено в основную ветку ядра Linux, я не удивлюсь, если оно будет использоваться в некоторых дистрибутивах Linux в облаке.

11.3.3. Управление ресурсами

Механизмы управления ресурсами ограничивают доступ к ресурсам, обеспечивая их более справедливое распределение. В Linux они в основном предоставлены контрольными группами.

Отдельные средства управления ресурсами можно классифицировать как *приоритеты* или *пределы*. Приоритеты управляют потреблением ресурсов, стараясь сбалансировать потребление между соседями на основе их важности. Пределы — это максимальные значения потребления ресурсов. Оба используются по мере необходимости — для некоторых ресурсов используют и те и другие. В табл. 11.3 приводятся некоторые примеры.

Таблица 11.3. Управление ресурсами в контейнерах Linux

Ресурс	Приоритет	Предел
Процессор	Доли CFS	Наборы процессоров (в целых процессорах). Полоса пропускания CFS (в долях процессоров)
Объем памяти	Мягкое ограничение памяти	Пределы потребления памяти
Объем подкачки	—	Пределы подкачки
Объем файловой системы	—	Квоты/пределы на потребление пространства в файловой системе
Кэш файловой системы	—	Пределы потребления памяти ядра
Дисковый ввод/вывод	Весы blkio	Пределы количества операций ввода/вывода в blkio. Пределы пропускной способности blkio
Сетевой ввод/вывод	Приоритеты net_prio. Дисциплины очередей (fq и др.). Сценарии BPF	Дисциплины очередей (fq и др.). Сценарии BPF

В разделах ниже приведено общее описание этих средств на примере контрольных групп версии 1. Порядок их настройки зависит от контейнерной платформы (Docker, Kubernetes и т. д.). Подробности ищите в соответствующей документации.

Процессоры

Процессоры можно назначать контейнерам с помощью контрольной группы cpuset, а доли и ширину полосы пропускания — с помощью планировщика CFS.

cpuset

Контрольная группа cpuset позволяет назначать отдельные процессоры определенным контейнерам. Преимущество такого подхода в том, что контейнеры могут

выполняться на назначенных им процессорах, не приостанавливаясь для обработки прерываний, причиной которых являются другие контейнеры, и доступное им процессорное время остается неизменным. Обратная сторона — это невозможность передать недоиспользуемые ресурсы другими контейнерами.

Доли и пропускная способность

Распределение *долей* процессоров, поддерживаемое планировщиком CFS, — это другой подход к распределению вычислительных ресурсов, который позволяет контейнерам совместно использовать недоиспользованное процессорное время. Распределение долей основано на идее *увеличения доступной вычислительной мощности* (bursting), когда контейнер может работать быстрее за счет потребления вычислительных ресурсов, недоиспользованных другими контейнерами. Когда нет недоиспользованного процессорного времени, в том числе когда у хоста избыточное количество ресурсов, распределение долей обеспечивает оптимальное разделение вычислительных ресурсов между контейнерами, которым они нужны.

Процессор делит работу, назначая контейнерам единицы распределения ресурса. Они называются долями и используются для определения объема вычислительных ресурсов, который нагруженный контейнер получит в данный момент времени. Вычисления выполняются по формуле:

$$\text{Вычислительные ресурсы контейнера} = (\text{все вычислительные ресурсы} \times \text{количество долей контейнера}) / \text{количество занятых долей в системе}.$$

Рассмотрим систему, в которой распределяется 100 долей вычислительных ресурсов между несколькими контейнерами. В какой-то момент вычислительные ресурсы нужны только контейнерам А и В. Контейнеру А назначено 10 долей, а контейнеру В — 30 долей. Таким образом, контейнер А может использовать 25 % всех вычислительных ресурсов в системе: все вычислительные ресурсы $\times 10 / (10 + 30)$.

Теперь рассмотрим систему, где нет простаивающих контейнеров. В такой системе вычислительные ресурсы будут распределяться по формуле:

$$\text{Вычислительные ресурсы контейнера} = \text{все вычислительные ресурсы} \times \text{количество долей контейнера} / \text{общее количество долей в системе}.$$

В описанном сценарии контейнер А получит 10 % вычислительных ресурсов (все вычислительные ресурсы $\times 10/100$). Распределение долей гарантирует выделение минимального заданного объема вычислительных ресурсов. Но поддержка увеличения вычислительной мощности позволяет контейнеру получить больше. Контейнер А может использовать от 10 до 100 % вычислительных ресурсов, в зависимости от количества других работающих контейнеров.

Проблема с распределением ресурсов на основе долей заключается в том, что увеличение доступной вычислительной мощности может усложнить планирование емкости, в частности, потому, что многие системы мониторинга не показывают статистики такого увеличения (хотя должны бы). Конечный пользователь, тестирующий

контейнер, может посчитать его производительность удовлетворительной, не зная, что такая производительность возможна только за счет перераспределения вычислительных ресурсов. Позднее, когда появляются другие арендаторы, контейнер начинает получать только гарантированный минимум и его производительность снижается. Представьте, что контейнер А первоначально тестируется в неактивной системе и получает 100 % процессорного времени, а с появлением других контейнеров только 10 %. Я неоднократно наблюдал такой сценарий. Конечный пользователь думал, что у него возникла проблема с производительностью, и просил меня помочь устранить ее. Потом он разочаровывался, узнав, что система работает в точности как задумано и падение производительности в 10 раз — это норма, потому что в системе появились другие контейнеры. Такое положение вещей клиент может расценить как недобросовестную рекламу.

Проблему чрезмерного увеличения доступных вычислительных ресурсов можно ослабить, ограничив это увеличение так, чтобы последующее падение производительности не было таким серьезным (даже если при этом ограничивается производительность). В Linux для этого можно использовать управление *шириной полосы пропускания* в CFS, установив верхний предел потребления процессора. Например, для контейнера А можно установить максимальную ширину пропускания 20 % от общего объема вычислительных ресурсов в системе, чтобы с назначенными долями он получал от 10 до 20 % в зависимости от доступности недоиспользованных ресурсов. Такой диапазон вычислительных ресурсов — от минимального, гарантированного количеством долей, до максимального, ограниченного шириной полосы пропускания, — показан на рис. 11.12. Предполагается, что в каждом контейнере достаточно нагруженных потоков выполнения, чтобы использовать доступные вычислительные ресурсы (иначе контейнеры столкнутся с ограничениями из-за собственных рабочих нагрузок еще до того, как достигнут наложенных системой ограничений).

Ширина полосы пропускания обычно задается в долях целых процессоров: 2,5 означает два с половиной процессора. Доли отображаются в параметры ядра, которые

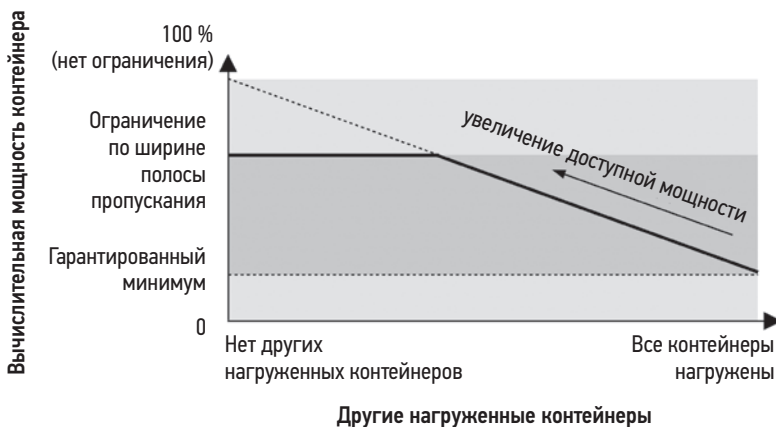


Рис. 11.12. Доли вычислительной мощности и ширина полосы пропускания

на самом деле представляют *периоды* и *квоты* в микросекундах: контейнер получает квоту в микросекундах процессорного времени за каждый период.

Другой способ управления увеличением доступной вычислительной мощности — когда операторы контейнеров уведомляют конечных пользователей, что они получили увеличенный объем вычислительных ресурсов в течение какого-то времени (например, дней), чтобы у них не возникало ложных ожиданий относительно производительности. Это может побудить конечных пользователей увеличить размер контейнера, чтобы получить большую долю и более высокую гарантированную вычислительную мощность.

Кэши процессоров

Использование кэша процессора можно контролировать с помощью технологии выделения кэша (cache allocation technology, CAT), разработанной в Intel. Он помогает избежать загрязнения кэшей другими контейнерами. Об этом рассказывалось в разделе 11.2.3 «Управление ресурсами», и здесь действует та же оговорка: ограничение доступа к кэшу также снижает производительность.

Объем памяти

Контрольная группа *memory* предлагает четыре механизма управления потреблением памяти. Они перечислены в табл. 11.4 вместе с соответствующими именами параметров настройки контрольной группы *memory*.

Таблица 11.4. Настройки контрольной группы *memory* в Linux

Имя	Описание
<code>memory.limit_in_bytes</code>	Ограничение размера памяти в байтах. Если контейнер попытается получить больше памяти, то столкнется с подкачкой страниц (если она настроена) или с ошибкой нехватки памяти, сгенерированной механизмом OOM Killer
<code>memory.soft_limit_in_bytes</code>	Ограничение размера памяти в байтах. Оптимальный подход, который предусматривает освобождение памяти, чтобы привести потребление памяти контейнером к мягкому пределу
<code>memory.kmem.limit_in_bytes</code>	Ограничение размера памяти для ядра в байтах
<code>memory.kmem.tcp.limit_in_bytes</code>	Ограничение размера памяти в байтах, выделенной для буферов TCP
<code>memory.pressure_level</code>	Позволяет организовать получение уведомлений о нехватке памяти с помощью системного вызова <code>eventfd(2)</code> . Для этого приложение должно поддерживать настройку уровня нехватки памяти, при котором будет отправляться уведомление, и использовать системный вызов

Есть и механизмы уведомления, позволяющие приложениям принимать меры при нехватке памяти: `memory.pressure_level` и `memory.oom_control`. Чтобы воспользоваться этими механизмами, приложение должно настроить отправку уведомлений с помощью системного вызова `eventfd(2)`.

Обратите внимание, что недоиспользованная контейнером память может использоваться другими контейнерами в кэше страниц ядра, повышая их производительность (форма увеличения доступных ресурсов для памяти).

Объем пространства в файле подкачки

Контрольная группа `memory` также позволяет ограничить подкачку. За это отвечает параметр `memory.memsw.limit_in_bytes`, определяющий объем памяти, а также объем пространства в файле подкачки.

Объем файловой системы

Объем файловой системы обычно ограничивается самой файловой системой. Например, файловая система XFS поддерживает как мягкие, так и жесткие квоты для пользователей, групп и проектов, где мягкие ограничения допускают некоторое временное избыточное потребление ниже жесткого ограничения. ZFS и btrfs тоже имеют квоты.

Кэш файловой системы

В Linux память, используемая кэшем страниц файловой системы для контейнера, относится к контейнеру в контрольной группе `memory` и не требует дополнительных настроек. Если контейнер настраивает поддержку подкачки, то степень предпочтения подкачки по сравнению с вытеснением кэша страниц можно контролировать с помощью параметра `memory.swappiness`, аналогичного общесистемному параметру `vm.swappiness` (глава 7 «Память», раздел 7.6.1 «Настраиваемые параметры»).

Дисковый ввод/вывод

Группа `blkio` предлагает механизмы для управления дисковым вводом/выводом. Они перечислены в табл. 11.5 вместе с соответствующими именами параметров настройки контрольной группы `blkio`.

Таблица 11.5. Настройки контрольной группы `blkio` в Linux

Имя	Описание
<code>blkio.weight</code>	Вес <code>cgroup</code> , который контролирует долю дисковых ресурсов во время нагрузки, аналогично долям вычислительных ресурсов. Используется с планировщиком ввода/вывода Bfq
<code>blkio.weight_device</code>	Настройка веса для конкретного устройства
<code>blkio.throttle.read_bps_device</code>	Предел для чтения в байтах в секунду
<code>blkio.throttle.write_bps_device</code>	Предел для записи в байтах в секунду
<code>blkio.throttle.read_iops_device</code>	Предел для чтения в операциях в секунду (IOPS)
<code>blkio.throttle.write_iops_device</code>	Предел для записи в операциях в секунду (IOPS)

Как и в случае с долями и шириной полосы пропускания вычислительных ресурсов, веса и пределы blkio позволяют совместно использовать дисковые ресурсы ввода/вывода с учетом приоритетов и пределов.

Сетевой ввод/вывод

Контрольная группа `net_prio` позволяет устанавливать приоритеты для исходящего сетевого трафика. Они совпадают с настройками параметра сокета `SO_PRIORITY` (см. `socket(7)`) и управляют приоритетом обработки пакетов в сетевом стеке. Контрольная группа `net_cls` может маркировать пакеты идентификатором класса для последующего управления дисциплинами очередей. (Этот прием применим также к подам Kubernetes, которые могут использовать `net_cls` для организации обслуживания контейнеров.)

Дисциплины очередей (см. главу 10 «Сеть», раздел 10.4.3 «Программное обеспечение») могут работать с идентификаторами классов или назначаться виртуальным сетевым интерфейсам в контейнерах для приоритизации и ограничения сетевого трафика. Есть более 50 различных типов дисциплин, каждая со своими правилами, особенностями и настройками. Например, настройки `kubernetes.io/ingress-bandwidth` и `kubernetes.io/egress-bandwidth` в Kubernetes реализуются путем создания фильтра маркерной корзины (Token Bucket Filter, TBF) [CNI 18]. Пример добавления и удаления дисциплин для сетевого интерфейса приводится в разделе 10.7.6 «tc».

К контрольным группам можно подключать программы BPF для создания нестандартных и программируемых средств управления ресурсами и брандмауэров. Примером может служить ПО Cilium, использующее комбинацию из программ BPF совместно с XDP, контрольными группами и дисциплинами очередей для поддержки безопасности, балансировки нагрузки и настройки брандмауэров между контейнерами [Cilium 20a].

11.3.4. Наблюдаемость

Что именно доступно для наблюдения, зависит от настроек безопасности хоста и от того, где запускаются инструменты наблюдения. Контейнеры могут быть настроены по-разному, поэтому я опишу наиболее типичный случай. В общем случае:

- **Из хост-системы** (наиболее привилегированного пространства имен): можно наблюдать все, включая аппаратные ресурсы, файловые системы, гостевые процессы, гостевые сеансы TCP и т. д. Гостевые процессы доступны для наблюдения и анализа без входа в гостевые системы. Гостевые файловые системы тоже доступны для наблюдения на уровне хоста (облачного провайдера).
- **Из гостевой среды:** как правило, на уровне контейнера можно наблюдать только за своими собственными процессами, файловыми системами, сетевыми интерфейсами и сеансами TCP. Основным исключением являются общесистемные статистики, например потребление процессоров и дисков: они часто отражают потребление на уровне хоста, а не только контейнера. Статус таких статистик

обычно не документируется (я написал свою документацию, которую вы найдете в следующем разделе «Традиционные инструменты»). Внутренние механизмы ядра обычно недоступны, поэтому инструменты анализа производительности, использующие инфраструктуру трассировки ядра (главы 13–15), часто оказываются бесполезными.

Последний момент был описан выше: контейнеры чаще сталкиваются с проблемами конкуренции за ядро и в то же время лишают конечного пользователя возможности диагностировать эти проблемы.

Распространенная проблема при анализе производительности контейнеров — «шумные соседи». Это другие арендаторы контейнеров, которые активно потребляют ресурсы и вступают в конкуренцию за доступ к ним с остальными арендаторами. Поскольку все контейнерные процессы обслуживаются одним ядром и могут анализироваться на уровне хоста, их анализ не отличается от традиционного анализа работы нескольких процессов, выполняемых в одной системе с разделением времени. Основное отличие состоит в том, что контрольные группы могут накладывать дополнительные программные ограничения, которые проверяются перед аппаратными ограничениями.

Многие инструменты мониторинга, написанные для обычных систем, не имеют поддержки виртуализации операционной системы (контейнеров) и не учитывают ограничения, накладываемые контрольными группами и другими программными средствами. При попытке использовать их в контейнерах клиенты могут обнаружить, что работают, но показывают только физические ресурсы системы. Без поддержки наблюдения за средствами управления облачными ресурсами эти инструменты могут ошибочно сообщать, что еще есть некоторый запас, хотя на самом деле был достигнут программный предел. Они также могут показывать высокий уровень потребления ресурсов, фактически обусловленный действиями других арендаторов.

В Linux наблюдение за контейнером как на уровне хоста, так и на уровне контейнеров еще более усложняется и требует много времени из-за того, что сейчас контейнеры не имеют идентификаторов в ядре¹. Также отсутствует специализированная поддержка контейнеров традиционными инструментами анализа производительности.

Эти проблемы описаны в следующих разделах. В них идет речь о состоянии традиционных инструментов анализа производительности, исследуется возможность наблюдения на уровне хоста и контейнеров, а также описывается стратегия анализа производительности.

¹ ПО для управления контейнерами может добавлять названия контрольных групп после идентификаторов контейнеров, и в этом случае названия контрольных групп в ядре действительно сообщают имя контейнера на уровне пользователя. Идентификатор по умолчанию в контрольных группах версии 2 — это еще один кандидат на использование в роли идентификатора контейнера внутри ядра и используется в таком качестве в BPF и bpftrace. В разделе 11.3.4 «Наблюдаемость» в подразделе «Трассировка с помощью BPF» показано еще одно возможное решение: имя узла из пространства имен uts, которое обычно используется в качестве имени контейнера.

11.3.4.1. Традиционные инструменты

В табл. 11.6 перечислены некоторые традиционные инструменты анализа производительности и описано, что они показывают при использовании на уровне хоста и типичного контейнера (который использует пространства имен процессов и точек монтирования) в ядре Linux 5.2. Ситуации, в которых возможны неожиданности, например, когда на уровне контейнера доступны статистики хоста, выделены полужирным шрифтом.

Таблица 11.6. Традиционные инструменты Linux

Инструмент	На уровне хоста	На уровне контейнера
top	В сводной части вывода отображается информация для хоста; таблица процессов содержит все процессы хоста и контейнера	В сводной части вывода отображаются смешанные статистики. Некоторые относятся к хосту, некоторые — к контейнеру. Таблица процессов содержит все процессы хоста и контейнера
ps	Показывает все процессы	Показывает процессы в контейнере
uptime	Показывает средние уровни нагрузки для хоста (системы в целом)	Показывает средние уровни нагрузки для хоста
mpstat	Показывает процессоры хоста и потребление процессоров на уровне хоста	Показывает процессоры хоста и потребление процессоров на уровне хоста
vmstat	Показывает потребление процессоров, памяти и другие статистики на уровне хоста	Показывает потребление процессоров, памяти и другие статистики на уровне хоста
pidstat	Показывает все процессы	Показывает процессы контейнера
free	Показывает потребление памяти хоста	Показывает потребление памяти хоста
iostat	Показывает потребление дисков на уровне хоста	Показывает потребление дисков на уровне хоста
pidstat -d	Показывает все процессы, выполняющие дисковый ввод/вывод	Показывает процессы контейнера, выполняющие дисковый ввод/вывод
sar -n DEV, TCP 1	Показывает сетевые интерфейсы и статистики TCP хоста	Показывает сетевые интерфейсы и статистики TCP контейнера
perf	Может профилировать все	Может не запускаться, но может быть разрешен и тогда позволяет профилировать процессы других арендаторов
tcpdump	Может перехватывать пакеты на всех интерфейсах	Может перехватывать пакеты только на интерфейсах контейнера
dmesg	Показывает журнал ядра	Не запускается

Со временем поддержка контейнеров в инструментах может улучшиться, и при использовании в контейнере они будут отображать только статистики, имеющие

отношение к контейнеру, или даже разбивку статистик для контейнера и хоста. На уровне хоста инструменты могут показывать все, но их тоже можно улучшить, добавив поддержку разбивки и фильтров по контейнерам или контрольным группам. Об этом я подробнее расскажу в разделах ниже, посвященных наблюдению за хостом и гостем.

11.3.4.2. Хост

После входа в систему хоста с помощью инструментов, описанных в предыдущих главах, можно изучить все системные ресурсы (процессоры, память, файловые системы, диски, сеть). При использовании контейнеров нужно изучить два дополнительных фактора:

- статистики по контейнерам;
- влияние средств управления ресурсами.

Как рассказывалось в разделе 11.3.1 «Реализация», в ядре нет понятия контейнера: контейнер — это просто набор пространств имен и контрольных групп. Идентификатор контейнера, который вы видите, создается и управляется программным обеспечением, действующим в пространстве пользователя. Вот примеры идентификаторов контейнеров в Kubernetes (в данном случае под с одним контейнером) и в Docker:

```
# kubectl get pod
NAME                                READY   STATUS             RESTARTS   AGE
kubernetes-b94cb9bff-kqvm1         0/1     ContainerCreating   0           3m
[...]
```

```
# docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
6280172ea7b9  ubuntu   "bash"    4 weeks ago  Up 4 weeks      eager_bhaskara
[...]
```

Это представляет проблему для традиционных инструментов анализа производительности — `ps(1)`, `top(1)` и т. д. Чтобы показать идентификатор контейнера, они должны поддерживать Kubernetes, Docker и любые другие контейнерные платформы. Если бы идентификация контейнеров выполнялась ядром, то такие идентификаторы стали бы стандартом и поддерживались всеми инструментами анализа производительности. Именно так обстоит дело с ядром Solaris, где контейнеры называются *зонами* и имеют *идентификатор зоны*, формируемый в ядре, который можно наблюдать с помощью `ps(1)` и других инструментов. (В следующем разделе «Трассировка с помощью BPF» показано решение для Linux, в котором в качестве идентификатора контейнера используется имя узла из пространства имен UTS.)

На практике статистики производительности по идентификаторам контейнера в Linux можно получить, используя:

- **Контейнерные инструменты**, предлагаемые контейнерной платформой. Например, в Docker есть инструмент, показывающий потребление ресурсов контейнером.

- **ПО для мониторинга производительности**, которое обычно имеет плагины для поддержки различных контейнерных платформ.
- **Статистики контрольных групп** и инструменты их поддержки. Для этого требуется выполнить дополнительный шаг, чтобы выяснить, какие контрольные группы соответствуют тому или иному контейнеру.
- **Отображение пространства имен** из хоста, например, с помощью `nsenter(1)`, чтобы иметь возможность запускать традиционные инструменты в контейнере. Если применить параметр `-p` (PID пространства имен), круг видимых процессов можно сузить до выполняющихся в контейнере. Однако важно помнить, что статистики, отображаемые инструментами, могут относиться не только к контейнеру: см. табл. 11.6. Также для запуска сетевых инструментов (`ping(8)`, `tcpdump(8)`) в том же сетевом пространстве имен пригодится параметр `-n` (сетевое пространство имен).
- **Трассировку с помощью BPF** для чтения информации о контрольных группах и пространствах имен из ядра.

В следующих разделах показаны примеры использования контейнерных инструментов, статистик контрольных групп, пространств имен и трассировки с помощью BPF, а также наблюдения за средствами управления ресурсами.

Контейнерные инструменты

Kubernetes позволяет изучать потребление основных ресурсов с помощью `kubectl top`.

Проверка хостов («узлов»):

```
# kubectl top nodes
NAME                                CPU(cores)   CPU%   MEMORY(bytes)   MEMORY%
bgregg-i-03cb3a7e46298b38e        1781m        10%    2880Mi          9%
```

В столбце `CPU(cores)` выводится накопленное процессорное время в миллисекундах, а в столбце `CPU%` — текущее потребление процессора узлом.

Проверка контейнеров («подов»):

```
# kubectl top pods
NAME                                CPU(cores)   MEMORY(bytes)
kubernetes-b94cb9bff-p7jzp         73m          9Mi
```

Здесь показаны накопленное процессорное время и текущий используемый объем памяти.

Эти команды требуют наличия действующего сервера метрик, который может запускаться по умолчанию в зависимости от настроек Kubernetes. Есть и инструменты мониторинга, отображающие эти же метрики в графическом интерфейсе, — `cAdvisor`, `Sysdig` и `Google Cloud Monitoring` [Kubernetes 20c].

Docker предлагает для анализа производительности несколько подкоманд в команде `docker(1)`, включая `stats`. Вот пример, полученный на рабочем хосте:

```
# docker stats
CONTAINER    CPU %       MEM USAGE / LIMIT     MEM %      NET I/O    BLOCK I/O      PIDS
353426a09db1 526.81%    4.061 GiB / 8.5 GiB   47.78%    0 B / 0 B   2.818 MB / 0 B  247
6bf166a66e08 303.82%    3.448 GiB / 8.5 GiB   40.57%    0 B / 0 B   2.032 MB / 0 B  267
58dcf8aed0a7 41.01%     1.322 GiB / 2.5 GiB   52.89%    0 B / 0 B   0 B / 0 B       229
61061566ffe5 85.92%     220.9 MiB / 3.023 GiB 7.14%     0 B / 0 B   43.4 MB / 0 B   61
bdc721460293 2.69%      1.204 GiB / 3.906 GiB 30.82%    0 B / 0 B   4.35 MB / 0 B   66
[...]
```

Здесь видно, что контейнер с UUID `353426a09db1` потребил в общей сложности 527 % процессорного времени в течение интервала обновления и использовал 4 Гбайт основной памяти при ограничении в 8,5 Гбайт. В течение этого интервала сетевой ввод/вывод не производился и имел место небольшой объем (несколько мегабайт) дискового ввода/вывода.

Статистики контрольных групп

В `/sys/fs/cgroups` доступны различные статистики для контрольных групп. Их можно получить и отобразить с помощью различных продуктов и инструментов мониторинга контейнеров, а также изучить непосредственно в командной строке:

```
# cd /sys/fs/cgroup/cpu,cpuacct/docker/02a7cf65f82e3f3e75283944caa4462e82f...
# cat cpuacct.usage
1615816262506

# cat cpu.stat
nr_periods 507
nr_throttled 74
throttled_time 3816445175
```

Файл `cpuacct.usage` содержит информацию о потреблении процессорного времени этой контрольной группой в наносекундах. Файл `cpu.stat` сообщает, сколько раз эта контрольная группа подвергалась ограничению на потребление процессора (`nr_throttled`), а также общее время ограничения в наносекундах. В этом примере видно, что эта контрольная группа подвергалась ограничению 74 раза в течение 507 периодов времени с общей продолжительностью 3,8 с.

Также есть файл `cpuacct.usage_percpu`, на этот раз с информацией о контрольной группе Kubernetes:

```
# cd /sys/fs/cgroup/cpu,cpuacct/kubepods/burstable/pod82e745...
# cat cpuacct.usage_percpu
37944772821 35729154566 35996200949 36443793055 36517861942 36156377488 36176348313
35874604278 37378190414 35464528409 35291309575 35829280628 36105557113 36538524246
36077297144 35976388595
```

В этой системе с 16 процессорами файл содержит 16 полей с потребленным процессорным временем в наносекундах. Эти метрики для контрольных групп версии 1 описаны в исходном коде ядра в `Documentation/cgroup-v1/cpuacct.txt`.

Также статистики контрольных групп можно получить с помощью инструментов командной строки `htop(1)` и `systemd-cgtop(1)`. Вот результат запуска `systemd-cgtop(1)` на промышленном хосте с контейнерами:

```
# systemd-cgtop
Control Group                                Tasks  %CPU   Memory  Input/s  Output/s
/                                              -    798.2  45.9G   -        -
/docker                                     1082   790.1  42.1G   -        -
/docker/dcf3a...9d28fc4a1c72bbaff4a24834    200   610.5  24.0G   -        -
/docker/370a3...e64ca01198f1e843ade7ce21    170   174.0  3.0G    -        -
/system.slice                               748    5.3    4.1G    -        -
/system.slice/daemontools.service           422    4.0    2.8G    -        -
/docker/dc277...42ab0603bbda2ac8af67996b    160    2.5    2.3G    -        -
/user.slice                                 5      2.0    34.5M   -        -
/user.slice/user-0.slice                     5      2.0    15.7M   -        -
/user.slice/u...slice/session-c26.scope      3      2.0    13.3M   -        -
/docker/ab452...c946f8447f2a4184f3ccff2a    174    1.0    6.3G    -        -
/docker/e18bd...26ffdd7368b870aa3d1deb7a    156    0.8    2.9G    -        -
[...]
```

Как показывает этот вывод, контрольная группа с именем `/docker/dcf3a...` потребила 610,5 % общего процессорного времени в течение этого интервала обновления (проценты суммируются по нескольким процессорам) и 24 Гбайт основной памяти при 200 выполняющихся задачах. В выводе также видны несколько контрольных групп, созданных `systemd` для системных служб (`/system.slice`) и пользовательских сеансов (`/user.slice`).

Отображение пространства имен

Обычно контейнеры используют отдельные пространства имен для идентификации процессов и точек монтирования.

Для пространств имен процессов это означает, что идентификатор процесса (PID) в гостевой системе может не совпадать с идентификатором процесса (PID) в хост-системе.

При диагностике проблем с производительностью я сначала захожу в контейнер, чтобы увидеть проявление проблемы с точки зрения конечного пользователя. Затем я могу войти в хост-систему, чтобы продолжить расследование с использованием общесистемных инструментов, но идентификатор процесса может быть другим. Соответствие между этими двумя идентификаторами можно узнать в файле `/proc/PID/status`. Например, на уровне хоста:

```
host# grep NSpid /proc/4915/status
NSpid: 4915 753
```

В данном случае идентификатору PID 4915 на хосте соответствует идентификатор PID 753 в гостевой системе. К сожалению, часто требуется найти обратное соответствие: определить идентификатор PID на хосте по идентификатору PID

в контейнере. Один из способов (не самый эффективный) сделать это — просканировать все файлы `status`:

```
host# awk '$1 == "NSpid:" && $3 == 753 { print $2 }' /proc/*/status
4915
```

В этом случае я определил, что гостевому идентификатору PID 753 соответствует PID 4915 в хост-системе. Обратите внимание, что в выводе может появиться несколько идентификаторов PID в хост-системе, потому что «753» может быть в нескольких пространствах имен процессов. В такой ситуации нужно выяснить, какому пространству имен принадлежит искомый идентификатор 753. Для этой цели можно использовать файлы `/proc/PID/ns` — символические ссылки на идентификаторы пространств имен — и проверить их на уровне гостя и хоста:

```
guest# ls -lh /proc/753/ns/pid
lrwxrwxrwx 1 root root 0 Mar 15 20:47 /proc/753/ns/pid -> 'pid:[4026532216]'

host# ls -lh /proc/4915/ns/pid
lrwxrwxrwx 1 root root 0 Mar 15 20:46 /proc/4915/ns/pid -> 'pid:[4026532216]'
```

Обратите внимание на совпадение идентификаторов пространств имен (4026532216): это подтверждает, что идентификатор PID 4915 в хост-системе соответствует гостевому идентификатору PID 753.

Пространства имен точек монтирования могут создавать аналогичные проблемы. Например, если запустить команду `perf(1)` на уровне хоста, она будет искать дополнительные файлы символов в `/tmp/perf-PID.map`, но приложения в контейнерах сохраняют их в `/tmp` внутри контейнера, а это совсем другой каталог, отличный от каталога `/tmp` в хост-системе. Кроме того, идентификатор PID с большой долей вероятности будет отличаться из-за использования разных пространств имен процессов. Элис Голдфасс (Alice Goldfuss) первой опубликовала обходное решение, предусматривающее перемещение и переименование файлов символов, чтобы они были доступны на хосте [Goldfuss 17]. Но спустя некоторое время в `perf(1)` была добавлена поддержка пространств имен, устраняющая эту проблему, а в ядро — отображение пространства имен `/proc/PID/root` для прямого доступа к корневому каталогу («/») контейнера. Например:

```
host# ls -lh /proc/4915/root/tmp
total 0
-rw-r--r-- 1 root root 0 Mar 15 20:54 I_am_in_the_container.txt
```

Это список файлов в каталоге `/tmp` контейнера.

Кроме чтения файлов в каталоге `/proc`, команда `nsenter(1)` может запускать другие команды в выбранных пространствах имен. Эта команда запускает `top(1)` на уровне хоста в пространствах имен точек монтирования (`-m`) и процессов (`-p`) внутри PID 4915 (`-t 4915`):

```
# nsenter -t 4915 -m -p top
top - 21:14:24 up 32 days, 23:23, 0 users, load average: 0.32, 0.09, 0.02
Tasks: 3 total, 2 running, 1 sleeping, 0 stopped, 0 zombie
```

```
%Cpu(s):  0.2 us,   0.1 sy,   0.0 ni, 99.4 id,   0.0 wa,   0.0 hi,   0.0 si,   0.2 st
KiB Mem : 1996844 total,   98400 free,   858060 used, 1040384 buff/cache
KiB Swap:    0 total,    0 free,    0 used.  961564 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
753	root	20	0	818504	93428	11996	R	100.0	0.2	0:27.88	java
1	root	20	0	18504	3212	2796	S	0.0	0.2	0:17.57	bash
766	root	20	0	38364	3420	2968	R	0.0	0.2	0:00.00	top

Здесь видно, что самым активным является процесс java с идентификатором PID 753.

Трассировка с помощью BPF

Некоторые инструменты трассировки на основе BPF уже поддерживают контейнеры, но многие еще не имеют этой поддержки. К счастью, добавить такую поддержку в инструменты bpftrace обычно несложно, как показано ниже. См. главу 15, где объясняются приемы программирования bpftrace.

Инструмент forks.bt подсчитывает количество новых процессов, запущенных в период, пока выполнялась трассировка, наблюдая за обращениями к системным вызовам clone(2), fork(2) и vfork(2). Вот его исходный код:

```
#!/usr/local/bin/bpftrace

tracepoint:syscalls:sys_enter_clone,
tracepoint:syscalls:sys_enter_fork,
tracepoint:syscalls:sys_enter_vfork
{
    @new_processes = count();
}
```

Пример вывода:

```
# ./forks.bt
Attaching 3 probes...
^C

@new_processes: 590
```

Как показывает вывод, за время трассировки было запущено 590 новых процессов.

Чтобы получить разбивку по контейнерам, можно вывести имя узла (хоста) из пространства имен uts. Такая возможность зависит от реализации ПО контейнера, настраивающего это пространство имен, но обычно она поддерживается. Вот исходный код того же инструмента с дополнениями, выделенными полужирным шрифтом:

```
#!/usr/local/bin/bpftrace

#include <linux/sched.h>
#include <linux/nsproxy.h>
#include <linux/utsname.h>

tracepoint:syscalls:sys_enter_clone,
tracepoint:syscalls:sys_enter_fork,
tracepoint:syscalls:sys_enter_vfork
```

```
{
    $task = (struct task_struct *)curtask;
    $nodename = $task->nsproxy->uts_ns->name.nodename;
    @new_processes[$nodename] = count();
}
```

Этот дополнительный код использует текущую структуру `task_struct` ядра, чтобы получить имя узла из пространства имен `uts`, и добавляет его как ключ в выходную карту `@new_processes`.

Пример вывода:

```
# ./forks.bt
Attaching 3 probes...
^C

@new_processes[ip-10-1-239-218]: 171
@new_processes[efe9f9be6185]: 743
```

Теперь результаты разбиты по контейнерам, и по ним видно, что узел `6280172ea7b9` (контейнер) в период трассировки запустил 252 процесса. Другой узел, `ip-10-1-239-218`, — это хост-система.

Такое возможно только потому, что системные вызовы выполняются в контексте задачи (процесса), поэтому `curtask` возвращает соответствующую структуру `task_struct`, через которую мы получаем имя узла. При трассировке асинхронных событий, например прерываний по завершении дискового ввода/вывода, процесс, запустивший операцию ввода/вывода, к моменту прерывания может покинуть процессор. Тогда `curtask` будет ссылаться на другое имя узла.

Поскольку извлечение имени узла из `uts` может превратиться в часто используемую операцию в `bpfftrace`, я собираюсь добавить встроенную переменную `nodename`, чтобы единственным был следующий код:

```
@new_processes[nodename] = count();
```

Проверьте обновления `bpfftrace`: возможно, эта переменная уже появится, когда вы будете читать эти строки.

Управление ресурсами

Наблюдение за средствами управления ресурсами из раздела 11.3.3 «Управление ресурсами» необходимо, чтобы определить, влияют ли на контейнер накладываемые ими ограничения. Традиционные инструменты анализа производительности и документация в основном уделяют внимание физическим ресурсам и не видят ограничений, накладываемых программно.

Проверка средств управления ресурсами описана в методе `USE` (глава 2 «Методологии», раздел 2.5.9 «Метод `USE`»), который предполагает наблюдение за ресурсами и проверку потребления, насыщения и ошибок. При наличии средств управления ресурсами их тоже нужно проверять для каждого ресурса.

В подразделе «Статистики контрольных групп» упоминался файл `/sys/fs/cgroup/.../cpu.stat` со статистиками, отражающими, сколько раз накладывалось ограничение на потребление процессора (`nr_throttled`) и общую продолжительность действия ограничений в наносекундах (`throttled_time`). Это регулирование заключается в ограничении пропускной способности процессоров. Определить, регулируется ли пропускная способность для контейнера, довольно легко: значение `throttled_time` будет увеличиваться. Если регулирование выполняется с помощью контрольной группы `cgroup`, то потребление процессорного времени можно проверить с помощью инструментов и метрик, включая `mpstat(1)`.

Распределением вычислительных ресурсов можно также управлять с помощью назначения долей, как описано в подразделе «Доли и пропускная способность». Ограниченный по производительности из-за долей контейнер обнаружить сложнее — у него нет соответствующих статистик. Я разработал блок-схему, показанную на рис. 11.13, которая поможет определить, выполняется ли регулирование вычислительных ресурсов доступных контейнеров и как это происходит [Gregg 17g]:

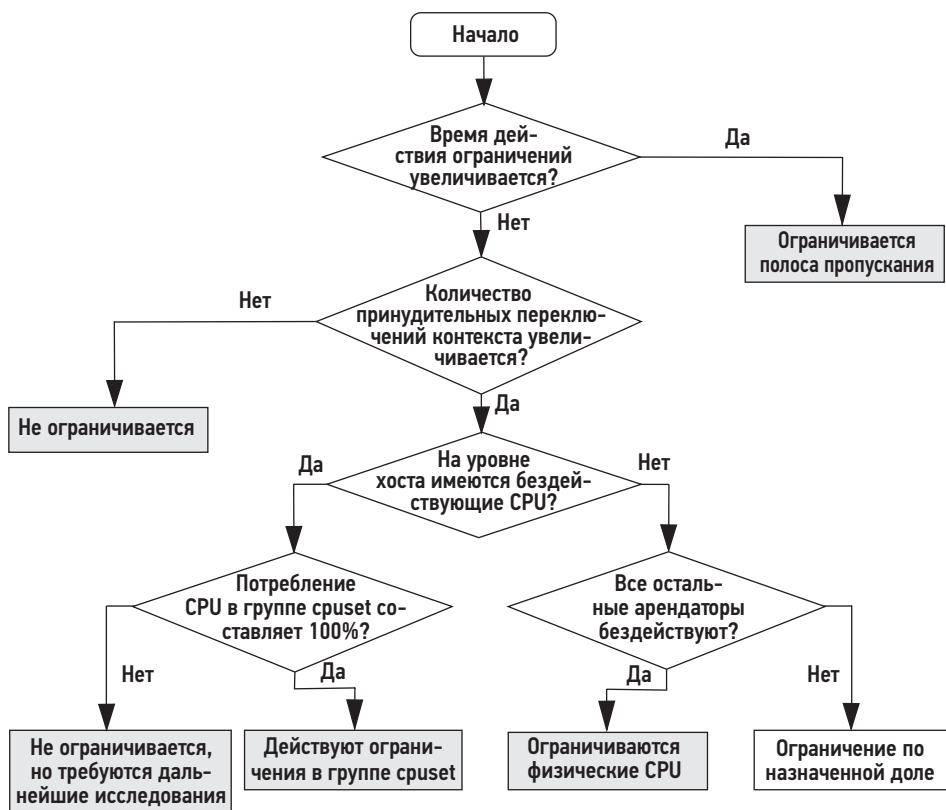


Рис. 11.13. Анализ регулирования вычислительных ресурсов, доступных контейнеру

Процедура анализа, представленная на рис. 11.13, помогает определить, ограничиваются ли вычислительные ресурсы контейнера и как. Для этого используются пять статистик:

- **Продолжительность действия ограничений:** значение `throttled_time` для контрольной группы.
- **Принудительные переключения контекста:** идентифицируются по увеличению значения `nonvolvention_ctxt_switches` в файле `/proc/PID/status`.
- **Бездействие процессоров на уровне хоста:** можно определить по значению столбца `%idle` в выводе `mpstat(1)`, в `/proc/stat` и с помощью других инструментов.
- **Потребление процессоров составляет 100 %:** в группе `cpuset` потребление процессоров можно узнать из вывода `mpstat(1)`, из файла `/proc/stat` и т. д.
- **Бездействие всех остальных арендаторов:** можно определить с помощью специализированного инструмента для конкретного контейнера (например, `docker stat`) или системных инструментов, которые показывают отсутствие конкуренции за вычислительные ресурсы (например, если `top(1)` показывает только один контейнер с ненулевым значением в столбце `%CPU`).

Аналогичную процедуру можно разработать для других ресурсов, а вспомогательная статистика, включая статистики контрольных групп, должна быть доступна в ПО и инструментах мониторинга. Идеальный продукт или инструмент мониторинга определит все автоматически и сообщит, как происходит регулирование ресурсов каждого контейнера, если оно есть.

11.3.4.3. Гость (контейнер)

Многие ожидают, что инструменты анализа производительности, которые запускаются в контейнере, будут отображать только статистики контейнера, но часто это не так. Вот пример запуска `iostat(1)` в неактивном контейнере:

```
container# iostat -sxz 1
[...]
```

avg-cpu:	%user	%nice	%system	%iowait	%steal	%idle		
	57.29	0.00	8.54	33.17	0.00	1.01		

Device	tps	kB/s	rqm/s	await	aqu-sz	areq-sz	%util
nvme0n1	2578.00	12436.00	331.00	0.33	0.00	4.82	100.00

avg-cpu:	%user	%nice	%system	%iowait	%steal	%idle		
	51.78	0.00	7.61	40.61	0.00	0.00		

Device	tps	kB/s	rqm/s	await	aqu-sz	areq-sz	%util
nvme0n1	2664.00	11020.00	88.00	0.32	0.00	4.14	98.80

```
[...]
```

Судя по этому выводу, есть существенная нагрузка на процессор и диск, но контейнер полностью бездействует. Это может сбивать с толку тех, кто плохо знаком с виртуализацией ОС, — может возникнуть ощущение, что контейнер работает под

нагрузкой. Причина в том, что инструменты показывают статистики хоста, которые суммируют активность других арендаторов.

Об этой особенности я кратко рассказал в разделе 11.3.4 «Наблюдаемость», в подразделе «Традиционные инструменты». Со временем в этих инструментах появляется поддержка контейнеров, и они начинают отображать статистики контрольных групп только по контейнерам, когда используются в контейнере.

Есть статистики контрольных групп для блочного ввода/вывода. Вот пример, полученный в том же контейнере:

```
container# cat /sys/fs/cgroup/blkio/blkio.throttle.io_serviced
259:0 Read 452
259:0 Write 352
259:0 Sync 692
259:0 Async 112
259:0 Discard 0
259:0 Total 804
Total 804
container# sleep 10
container# cat /sys/fs/cgroup/blkio/blkio.throttle.io_serviced
259:0 Read 452
259:0 Write 352
259:0 Sync 692
259:0 Async 112
259:0 Discard 0
259:0 Total 804
Total 804
```

Здесь показаны статистики, отражающие количество операций разных типов. Я вывел их дважды, выполнив команду `sleep 10` в промежутке, чтобы выждать некоторое время. Значения статистик не увеличились за это время, следовательно, контейнер не выполнял операции дискового ввода/вывода. Есть еще один файл со счетчиками байтов: `blkio.throttle.io_service_bytes`.

К сожалению, эти счетчики не обеспечивают всей полноты информации, необходимой инструменту `iostat(1)`. Для контрольных групп нужно организовать поддержку большего количества счетчиков, чтобы `iostat(1)` могла работать с контейнерами.

Поддержка контейнеров

Добавить в инструмент поддержку контейнеров не обязательно означает ограничивать видимую ему область рамками контейнера: возможность видеть состояние физических ресурсов из контейнера можно считать дополнительным преимуществом. Многое зависит от целей использования контейнеров:

А. Контейнер как изолированный сервер: если контейнер используется с этой целью, как обычно бывает с облачными провайдерами, то инструменты с поддержкой контейнеров должны показывать только текущую активность контейнера. Например, утилита `iostat(1)` должна показывать только дисковый ввод/вывод, инициированный контейнером и никем другим. Это можно реализовать,

изолировав все источники статистик, и отображать только метрики для текущего контейнера (`/proc`, `/sys`, `netlink` и т. д.). Но такая изоляция может как помогать, так и мешать анализу производительности на уровне гостя: диагностика проблем, связанных с конкуренцией за ресурсы со стороны других арендаторов, займет больше времени и будет основываться на наблюдении необъяснимого увеличения задержки устройства.

В. Контейнер как пакетное решение: компаниям, использующим собственные облака контейнеров, не обязательно отделять статистики, характеризующие работу контейнеров. Возможность видеть статистики хоста из контейнеров (как это часто бывает и как было показано выше на примере с `iostat(1)`) помогает конечным пользователям лучше понимать состояние аппаратных устройств и проблемы, вызванные шумными соседями. Под поддержкой контейнеров в `iostat(1)` в таком сценарии может пониматься разбивка, показывающая характеристики текущего контейнера в сопоставлении с характеристиками хоста или других контейнеров.

В обоих сценариях инструменты должны также поддерживать наблюдаемость средств управления ресурсами, где это необходимо, как часть поддержки контейнеров. Продолжая пример `iostat(1)`: помимо статистики `%util` для устройства (недоступна в сценарии А), также можно вывести ограничение `%cap`, исходя из пределов пропускной способности и IOPS в `blkio`, чтобы из контейнера можно было определить, ограничивается ли дисковый ввод/вывод средствами управления ресурсами¹.

Если наблюдение за физическими ресурсами разрешено, то гости смогут исключить из рассмотрения некоторые проблемы, в том числе проблему шумных соседей. А это может облегчить бремя поддержки для операторов контейнеров: люди склонны винить в проблемах то, что не могут наблюдать. Кроме того, в этом заключается важное отличие от виртуализации оборудования, при котором физические ресурсы скрываются от гостей и не имеют никакой возможности получить информацию о них (без использования внешних средств). В идеале в Linux должна появиться настройка для управления видимостью статистик хоста, чтобы любая контейнерная среда могла выбирать между сценариями А и В.

Инструменты трассировки

У продвинутых инструментов трассировки, основанных на использовании механизмов ядра — `perf(1)`, `Ftrace` и `BPF`, — такие же проблемы, и их создатели работают над реализацией поддержки контейнеров. Сейчас они не могут выполняться в контейнерах из-за отсутствия разрешений, необходимых для доступа к различным системным вызовам (`perf_event_open(2)`, `bpf(2)` и т. д.) и файлам в `/proc` и `/sys`. Кратко опишу их будущее для сценариев А и В, представленных выше:

¹ Вывод `iostat(1)` с параметром `-x` содержит так много полей, что не умещается по ширине ни в одном моем самом широком терминале, и я не решаюсь прямо предложить добавить дополнительные. Я бы предпочел добавить еще один параметр, например `-1`, для вывода столбцов со значениями программных пределов.

С. Требуется изоляция: разрешить контейнерам использовать инструменты трассировки можно через:

- **Фильтрацию в ядре:** события и их аргументы могут фильтроваться ядром, чтобы, например, в точке трассировки `block:block_rq_issue` контейнеру была доступна только информация о его текущем дисковом вводе/выводе.
- **API хоста:** хост может предоставлять доступ к определенным инструментам трассировки через безопасный API или графический интерфейс. Например, контейнер может запросить выполнение обычных инструментов ВСС, таких как `execsnoop(8)` и `biolatency(8)`, а хост может после проверки запроса выполнить ограниченную версию инструмента и вернуть его вывод.

Д. Изоляция не требуется: ресурсы (системные вызовы, `/proc` и `/sys`) делаются доступными контейнеру, чтобы инструменты трассировки работали без ограничений. Инструменты трассировки при этом могут иметь свою поддержку контейнеров, чтобы упростить фильтрацию посторонних событий, не имеющих отношения к текущему контейнеру.

Создание инструментов и статистик для контейнеров в Linux и других ядрах — медленный процесс, и, вероятно, потребуются много лет, прежде чем все они будут реализованы. Особенно это удручает опытных пользователей, которые входят в систему, чтобы воспользоваться инструментами командной строки для анализа производительности. Многие пользователи применяют продукты для мониторинга с агентами, которые уже поддерживают контейнеры (в той или иной степени).

11.3.4.4. Стратегия

Предыдущие главы охватывали методы анализа физических ресурсов системы и включали описание различных методологий. Они могут использоваться в хост-системе и, в некоторой степени, в гостевой, с учетом упоминавшихся выше ограничений. Гостям обычно доступны для наблюдения высокоуровневые статистики потребления ресурсов, но детальный анализ на уровне ядра, как правило, невозможен.

Помимо физических ресурсов, операторы хостов и арендаторы гостевых систем должны также проверять ограничения, накладываемые средствами управления ресурсами в облачных средах. Эти ограничения проявляются задолго до физических ограничений, поэтому доступность ресурсов гостю, скорее всего, будет ограничиваться ими, и они должны проверяться в первую очередь.

Многие традиционные инструменты наблюдения создавались до появления контейнеров и средств управления ресурсами (например, `top(1)`, `iostat(1)`), поэтому не включают по умолчанию информацию об управлении ресурсами, из-за чего пользователи могут упустить это из виду.

Вот несколько комментариев и стратегий для проверки средств управления каждым ресурсом:

- **Процессор:** см. схему на рис. 11.13. Проверьте настройки контрольной группы `cpuset`, ширину полосы пропускания и количество выделенных долей.

- **Память:** проверьте текущее потребление и его соответствие ограничениям контрольной группы памяти.
- **Объем файловой системы:** эта характеристика должна быть доступна для наблюдения, как и для любой другой файловой системы (включая использование `df(1)`).
- **Дисковый ввод/вывод:** проверьте настройки контрольной группы `blkio` (`/sys/fs/cgroup/blkio`) и статистики из файлов `blkio.throttle.io_service_bytes` и `blkio.throttle.io_service_bytes`: если они увеличиваются с течением времени, то это говорит о регулировании дискового ввода/вывода. Если доступна трассировка средствами BPF, то для подтверждения предположения о регулировании используйте инструмент `blkthrot(8)` [Gregg 19].
- **Сетевой ввод/вывод:** определите текущую пропускную способность сети и сравните с любым известным пределом полосы пропускания, который только можно наблюдать на уровне хоста. Достижение предела будет приводить к увеличению задержек сетевого ввода/вывода из-за регулирования сетевой активности арендаторов.

И заключительный комментарий: большая часть этого раздела описывает контрольные группы версии 1 и текущее состояние поддержки контейнеров в Linux. Возможности ядра быстро меняются, поэтому документация и поддержка контейнеров в инструментах оказываются в роли догоняющих. Чтобы быть в курсе последних достижений в области поддержки контейнеров в Linux, следите за изменениями в новых выпусках ядра и просматривайте каталог Documentation в исходном коде Linux. Рекомендую обращать внимание на любую документацию, написанную Теджуном Хео (Tejun Heo), ведущим разработчиком контрольных групп версии 2, включая Documentation/admin-guide/cgroup-v2.rst в исходном коде Linux [Heo 15].

11.4. ЛЕГКОВЕСНАЯ ВИРТУАЛИЗАЦИЯ

Легковесная виртуализация оборудования была разработана с целью объединить лучшее из двух подходов: безопасность виртуализации оборудования, с одной стороны, и эффективность с высокой скоростью загрузки контейнеров — с другой. На рис. 11.14 изображена технология легковесной виртуализации на основе Firecracker в сравнении с контейнерами.

В легковесной виртуализации оборудования используется легковесный гипервизор, основанный на виртуализации процессора и поддерживающий минимальное количество эмулируемых устройств. В этом его главное отличие от полноценных гипервизоров оборудования (раздел 11.2 «Виртуализация оборудования»), которые создавались на основе виртуальных машин для настольных компьютеров и включают поддержку видео, звука, BIOS, шины PCI и других устройств, а также различные уровни поддержки процессора. Гипервизор, предназначенный только для серверных вычислений, не нуждается в поддержке всех этих устройств. По поводу современных гипервизоров можно быть уверенными только в том, что в них доступны современные возможности виртуализации процессоров.

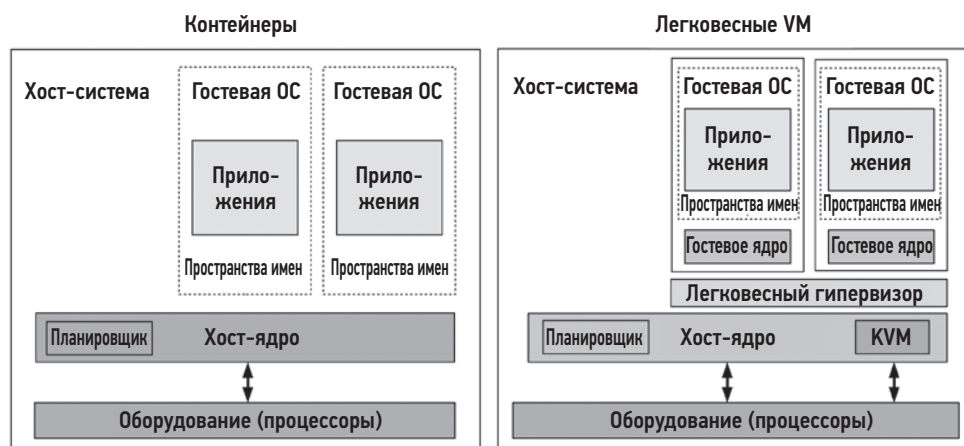


Рис. 11.14. Легковесная виртуализация

Для сравнения: QEMU (quick EMUlator) — это полноценный гипервизор оборудования для KVM, исходный код которого насчитывает более 1,4 миллиона строк (QEMU версии 4.2). Amazon Firecracker — это легковесный гипервизор, исходный код которого содержит всего 50 тысяч строк [Agache 20].

Легковесные виртуальные машины действуют как и виртуальные машины в конфигурации В, описанной в разделе 11.2. По сравнению с традиционными виртуальными машинами легковесные загружаются намного быстрее, имеют меньший оверхед на память и повышенную безопасность. Легковесные гипервизоры позволяют еще больше повысить безопасность за счет настройки пространства имен — еще одного уровня безопасности, — как показано на рис. 11.14.

В одних реализациях легковесные виртуальные машины называются *контейнерами*, в других используется термин *MicroVM*. Я предпочитаю использовать термин «MicroVM», так как термин «контейнеры» обычно ассоциируется с виртуализацией ОС.

11.4.1. Реализация

Есть несколько проектов легковесной виртуализации оборудования, в том числе:

- **Intel Clear Containers:** в этом проекте, запущенном в 2015 году, используются возможности Intel VT для создания легковесных виртуальных машин. Он доказал потенциал легковесных контейнеров, сократив время загрузки до 45 мс [Kadera 16]. В 2017 году проект Intel Clear Containers присоединился к проекту Kata Containers, в рамках которого продолжает развитие.
- **Kata Containers:** этот проект, запущенный в 2017 году, основан на Intel Clear Containers и Hyper.sh RunV и находится под управлением OpenStack Foundation.

Его девиз: «Скорость контейнеров, безопасность виртуальных машин» [Kata Containers 20].

- **Google gVisor:** этот проект с открытым исходным кодом, запущенный в 2018 году, использует специализированное ядро пользовательского пространства для гостевых систем, написанное на Go, которое улучшает безопасность контейнеров.
- **Amazon Firecracker:** этот проект с открытым исходным кодом был запущен в 2019 году [Firecracker 20]. Он использует KVM с новым легковесным диспетчером виртуальных машин (VMM) вместо QEMU и обеспечивает время загрузки системы около 100 мс [Agache 20].

В разделах ниже описана широко используемая реализация: легковесный гипервизор оборудования (Intel Clear Containers, Kata Containers и Firecracker). gVisor реализует другой подход, основанный на собственном легковесном ядре, и имеет характеристики, более схожие с характеристиками контейнеров (раздел 11.3 «Виртуализация операционной системы»).

11.4.2. Оверхед

Оверхед примерно такой же, как и на виртуализацию KVM (см. раздел 11.2.2 «Оверхед»), но с меньшим оверхедом памяти, потому что диспетчер виртуальных машин намного меньше. Так, оверхед памяти в Intel Clear Containers 2.0 составляет около 48–50 Мбайт на контейнер [Kadera 16]. Amazon Firecracker сообщает об оверхеде меньше 5 Мбайт [Agache 20].

11.4.3. Управление ресурсами

Поскольку процессы диспетчера виртуальных машин выполняются на хосте, ими можно управлять с помощью средств управления ресурсами ОС: контрольные группы, дисциплины очередей и т. д., по аналогии с виртуализацией KVM, как было описано в разделе 11.2.3 «Управление ресурсами». Эти средства управления ресурсами ОС подробно обсуждались с точки зрения контейнеров в разделе 11.3.3 «Управление ресурсами».

11.4.4. Наблюдаемость

Степень наблюдаемости аналогична наблюдаемости виртуализации KVM, описанной в разделе 11.2.4 «Наблюдаемость», то есть:

- **В хост-системе:** можно наблюдать все физические ресурсы, используя стандартные инструменты ОС, как было описано в предыдущих главах. Гостевые виртуальные машины отображаются как процессы. Внутренние детали работы гостевой системы, включая их процессы и файловые системы, недоступны для наблюдения напрямую. Чтобы оператор гипервизора мог изучить внутреннюю работу гостя, ему должен быть предоставлен доступ (например, через SSH).

- **В гостевой системе:** можно наблюдать виртуализированные ресурсы и их потребление гостем, а также делать выводы о физических проблемах. Работают все инструменты трассировки ядра, включая инструменты на основе BPF, потому что у виртуальной машины собственное выделенное ядро.

В качестве примера наблюдаемости показана виртуальная машина Firecracker, наблюдаемая с помощью `top(1)` на уровне хоста¹:

```
host# top
top - 15:26:22 up 25 days, 22:03,  2 users,  load average: 4.48, 2.10, 1.18
Tasks: 495 total,   1 running, 398 sleeping,   2 stopped,   0 zombie
%Cpu(s): 25.4 us,   0.1 sy,   0.0 ni, 74.4 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st
KiB Mem : 24422712 total, 8268972 free, 10321548 used, 5832192 buff/cache
KiB Swap: 32460792 total, 31906152 free,  554640 used. 11185060 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
 30785 root        20   0 1057360 297292 296772 S 200.0   1.2   0:22.03 firecracker
 31568 bgregg      20   0 110076   3336   2316 R  45.7   0.0   0:01.93 sshd
 31437 bgregg      20   0   57028   8052   5436 R  22.8   0.0   0:01.09 ssh
 30719 root        20   0 120320   16348 10756 S   0.3   0.1   0:00.83 ignite-spawn
    1 root        20   0  227044   7140   3540 S   0.0   0.0 15:32.13 systemd
[...]
```

Вся виртуальная машина выглядит как единый процесс с именем `firecracker`. Как показывает этот вывод, она потребляет 200 % процессорного времени (два процессора). На уровне хоста нельзя узнать, какие гостевые процессы потребляют столько процессорного времени.

Ниже показаны результаты запуска `top(1)` в гостевой системе:

```
guest# top
top - 22:26:30 up 16 min,  1 user,  load average: 1.89, 0.89, 0.38
Tasks: 67 total,   3 running, 35 sleeping,   0 stopped,   0 zombie
%Cpu(s): 81.0 us, 19.0 sy,   0.0 ni,  0.0 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st
KiB Mem : 1014468 total, 793660 free,   51424 used, 169384 buff/cache
KiB Swap:      0 total,      0 free,      0 used. 831400 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
 1104 root        20   0  18592   1232   700 R 100.0   0.1   0:05.77 bash
 1105 root        20   0  18592   1232   700 R 100.0   0.1   0:05.59 bash
 1106 root        20   0  38916   3468  2944 R   4.8   0.3   0:00.01 top
    1 root        20   0  77224   8352  6648 S   0.0   0.8   0:00.38 systemd
    3 root        0 -20      0      0      0 I   0.0   0.0   0:00.00 rcu_gp
[...]
```

Как показывает этот вывод, практически все процессорное время потребляется двумя программами `bash`.

Также сравните различие в сводной информации в начале вывода: на уровне хоста средняя нагрузка за минуту составляет 4,48, а на уровне гостя — 1,89. Другие детали

¹ Эта виртуальная машина была создана с помощью Weave Ignite, диспетчера контейнеров MicroVM [Weaveworks 20].

тоже отличаются, потому что в гостевой системе выполняется собственное ядро, которое накапливает статистики только для этого гостя. Как описано в разделе 11.3.4 «Наблюдаемость» в контейнерах, в отличие от виртуальных машин, статистики, полученные на уровне гостя, могут неожиданно оказаться общесистемными статистиками хоста.

В качестве другого примера показан вывод команды `mpstat(1)`, запущенной в гостевой системе:

```
guest# mpstat -P ALL 1
Linux 4.19.47 (cd41e0d846509816) 03/21/20 _x86_64_ (2 CPU)
22:11:07 CPU %usr %nice %sys %iowait %irq %soft %steal %guest %gnice %idle
22:11:08 all 81.50 0.00 18.50 0.00 0.00 0.00 0.00 0.00 0.00
22:11:08 0 82.83 0.00 17.17 0.00 0.00 0.00 0.00 0.00 0.00
22:11:08 1 80.20 0.00 19.80 0.00 0.00 0.00 0.00 0.00 0.00
[...]
```

В этом выводе отображается информация только о двух процессорах, потому что гостевой системе выделено только два процессора.

11.5. ДРУГИЕ ТИПЫ ВИРТУАЛИЗАЦИИ

В числе других примитивов и технологий облачных вычислений можно назвать:

- **Функции как услуга** (functions as a service, FaaS): разработчик отправляет функцию приложения в облако, которая вызывается по запросу. Эта технология упрощает процесс разработки ПО, но отсутствие управляющего сервера (потому она и называется «бессерверной») сказывается на производительности. Время запуска функции может быть значительным, и в отсутствие сервера конечный пользователь не может запускать традиционные инструменты наблюдения из командной строки. Анализ производительности обычно ограничивается отметками времени, которые фиксируются в приложении.
- **Программное обеспечение как услуга** (software as a service, SaaS): предоставляет ПО высокого уровня, при этом конечному пользователю не требуется настраивать серверы или приложения. Анализ производительности доступен только операторам: без доступа к серверам конечные пользователи мало что могут оценить, кроме времени на стороне клиента.
- **Одноцелевые ядра** (unikernels): эта технология предполагает компиляцию приложения вместе с минимальным ядром в единый двоичный файл, который может выполняться аппаратными гипервизорами напрямую без операционной системы. Несмотря на возможность увеличения производительности за счет минимизации выполняемого кода и, следовательно, загрязнения кэша процессора, а также повышения безопасности за счет удаления неиспользуемого кода, одноцелевые ядра создают проблемы наблюдаемости, обусловленные отсутствием ОС, где можно запускать инструменты наблюдения. Статистики ядра, подобные тем, что находятся в `/proc`, могут отсутствовать. К счастью, многие реализации

позволяют запускать одноцелевые ядра как обычные процессы, предоставляя один путь для анализа (хотя и в другой среде, отличной от гипервизора). Гипервизоры также могут поддерживать возможности исследования одноцелевых ядер, например профилирование стека: я разработал прототип, который генерирует флейм-графики для MirageOS Unikernel [Gregg 16a].

Во всех этих технологиях нет ОС, в которую конечный пользователь мог бы войти (или контейнера для входа) и провести традиционный анализ производительности. Анализ FaaS и SaaS доступен только операторам. Для анализа производительности одноцелевых ядер требуются специализированные инструменты и статистики, а в идеале — поддержки профилирования в гипервизоре.

11.6. СРАВНЕНИЕ

Сравнение технологий помогает лучше понять их, даже если вы не в состоянии изменить технологию, которая используется в вашей компании. В табл. 11.7 приведено сравнение характеристик производительности трех технологий из этой главы¹.

Таблица 11.7. Сравнение характеристик производительности технологий виртуализации

Характеристика	Виртуализация оборудования	Виртуализация операционной системы (контейнеры)	Легковесная виртуализация
Пример	KVM	Контейнеры	FireCracker
Производительность процессора	Высокая (поддержка процессоров)	Высокая	Высокая (поддержка процессоров)
Распределение процессоров	Фиксированное в виде виртуальных процессоров	Гибкое (доли и полоса пропускания)	Фиксированное в виде виртуальных процессоров
Пропускная способность ввода/вывода	Высокая (с SR-IOV)	Высокая (без внутреннего оверхеда)	Высокая (с SR-IOV)
Задержка ввода/вывода	Низкая (с SR-IOV и без QEMU)	Низкая (без внутреннего оверхеда)	Низкая (с SR-IOV)
Оверхед на доступ к памяти	Имеется (на EPT/NPT или теневые таблицы страниц)	Нет	Имеется (на EPT/NPT или теневые таблицы страниц)
Потери памяти	Имеются (на дополнительные ядра, таблицы страниц)	Нет	Имеются (на дополнительные ядра, таблицы страниц)

¹ Обратите внимание, что в этой таблице сравнивается только производительность. Есть и другие отличия, например, контейнеры были описаны как имеющие более слабую защиту [Agache 20].

Характеристика	Виртуализация оборудования	Виртуализация операционной системы (контейнеры)	Легковесная виртуализация
Распределение памяти	Фиксированное (возможно двойное кэширование)	Гибкое (память, неиспользуемая гостем, используется кэшем файловой системы)	Фиксированное (возможно двойное кэширование)
Управление ресурсами	Большинством (ядро плюс элементы управления гипервизором)	Многими (зависит от ядра)	Большинством (ядро плюс элементы управления гипервизором)
Наблюдаемость: на уровне хоста	Средняя (потребление ресурсов, статистики гипервизора, ОС для KVM-подобных гипервизоров, но нет возможности наблюдать внутреннюю работу гостя)	Высокая (доступно все)	Средняя (потребление ресурсов, статистики гипервизора, ОС для KVM-подобных гипервизоров, но нет возможности наблюдать внутреннюю работу гостя)
Наблюдаемость: на уровне гостя	Высокая (полная наблюдаемость ядра и виртуальных устройств)	Средняя (доступно только пространство пользователя, счетчики ядра, наблюдаемость ядра ограничена) с дополнительными метриками, характеризующими хост (например, iostat(1))	Высокая (полная наблюдаемость ядра и виртуальных устройств)
Наблюдаемость благоприятствует	Конечным пользователям	Операторам хостов	Конечным пользователям
Сложность гипервизора	Наивысшая (вдобавок к сложному гипервизору)	Средняя (ОС)	Высокая (вдобавок к легковесному гипервизору)
Разные гостевые системы	Да	Обычно нет (иногда возможно при использовании механизма трансляции системных вызовов, который может вносить свой overhead)	Да

Эта таблица будет устаревать по мере появления новых возможностей в технологиях виртуализации, но перечисленные в ней характеристики можно использовать для сравнения даже совершенно новых технологий виртуализации, которые не подпадают ни к одной из категорий.

Технологии виртуализации часто сравнивают по результатам микробенчмаркинга, помогающим определить, какая из них работает лучше всего. К сожалению, при

этом упускается из виду такая важная характеристика, как наблюдаемость системы, благодаря которой можно получить наибольший выигрыш в производительности. Хорошая наблюдаемость часто позволяет выявить и устранить ненужную работу, что увеличивает потенциальный выигрыш в производительности намного больше, чем незначительные различия в гипервизоре.

Наибольший уровень наблюдаемости для облачных операторов предлагают контейнеры, потому что, находясь на уровне хоста, они могут видеть все процессы и их взаимодействия. Для конечных пользователей предпочтительнее виртуальные машины, потому что они предоставляют пользователям доступ к ядру и позволяют запускать любые инструменты производительности на основе ядра, включая описанные в главах 13, 14 и 15. Другой вариант — контейнеры с доступом к ядру, дающие операторам и пользователям полную наблюдаемость всех ресурсов. Но этот вариант возможен, только если клиент сам управляет хостом для размещения контейнеров, — такой способ не обеспечивает должной изоляции безопасности между контейнерами.

Виртуализация все еще продолжает развиваться, например, легковесные гипервизоры оборудования появились совсем недавно. Учитывая их преимущества, особенно с точки зрения наблюдаемости для конечного пользователя, я ожидаю, что их популярность будет расти.

11.7. УПРАЖНЕНИЯ

1. Ответьте на следующие вопросы по терминологии виртуализации:
 - В чем разница между хост-системой и гостевой системой?
 - Что такое арендатор (tenant)?
 - Что такое гипервизор?
 - Что такое виртуализация оборудования?
 - Что такое виртуализация ОС?
2. Ответьте на следующие концептуальные вопросы:
 - Опишите роль изоляции производительности.
 - Опишите, какой оверхед с точки зрения производительности сопровождает современную виртуализацию оборудования (например, Nitro).
 - Опишите, какой оверхед с точки зрения производительности сопровождает виртуализацию ОС (например, контейнеры Linux).
 - Опишите наблюдаемость физической системы с точки зрения гостя в виртуализации оборудования (в XEN или KVM).
 - Опишите наблюдаемость физической системы с точки зрения гостя в виртуализации ОС.
 - Объясните разницу между виртуализацией оборудования (например, Xen или KVM) и легковесной виртуализацией оборудования (например, Firecracker).

3. Выберите технологию виртуализации и ответьте на следующие вопросы с позиции гостя:
 - Опишите, как применяется ограничение на объем доступной памяти и как это ограничение проявляется на стороне гостя. (Что наблюдает системный администратор, когда гостевая система исчерпывает доступную ей память?)
 - Опишите, как применяется ограничение на вычислительные ресурсы и как это ограничение проявляется на стороне гостя.
 - Опишите, как применяется ограничение на дисковый ввод/вывод и как это ограничение проявляется на стороне гостя.
 - Опишите, как применяется ограничение на сетевой ввод/вывод и как это ограничение проявляется на стороне гостя.
4. Разработайте чек-лист для метода USE, определяющий процедуру анализа средств управления ресурсами. Опишите, как получить каждую метрику (например, какую команду выполнить) и как интерпретировать результаты. Перед установкой или использованием дополнительных программных продуктов постарайтесь ограничиться инструментами наблюдения, имеющимися в ОС.

11.8. ССЫЛКИ

- [Goldberg 73] Goldberg, R. P., «Architectural Principles for Virtual Computer Systems», Harvard University (Thesis), 1972.
- [Waldspurger 02] Waldspurger, C., «Memory Resource Management in VMware ESX Server», Proceedings of the 5th Symposium on Operating Systems Design and Implementation, 2002.
- [Cherkasova 05] Cherkasova, L., and Gardner, R., «Measuring CPU Overhead for I/O Processing in the Xen Virtual Machine Monitor», USENIX ATEC, 2005.
- [Adams 06] Adams, K., and Agesen, O., «A Comparison of Software and Hardware Techniques for x86 Virtualization», ASPLOS, 2006.
- [Gupta 06] Gupta, D., Cherkasova, L., Gardner, R., and Vahdat, A., «Enforcing Performance Isolation across Virtual Machines in Xen», ACM/IFIP/USENIX Middleware, 2006.
- [Qumranet 06] «KVM: Kernel-based Virtualization Driver», Qumranet Whitepaper, 2006.
- [Cherkasova 07] Cherkasova, L., Gupta, D., and Vahdat, A., «Comparison of the Three CPU Schedulers in Xen», ACM SIGMETRICS, 2007.
- [Corbet 07a] Corbet, J., «Process containers», LWN.net, <https://lwn.net/Articles/236038>, 2007.
- [Corbet 07b] Corbet, J., «Notes from a container», LWN.net, <https://lwn.net/Articles/256389>, 2007.
- [Liguori, 07] Liguori, A., «The Myth of Type I and Type II Hypervisors», <http://blog.codemonkey.ws/2007/10/myth-of-type-i-and-type-ii-hypervisors.html>, 2007.
- [VMware 07] «Understanding Full Virtualization, Paravirtualization, and Hardware Assist», <https://www.vmware.com/techpapers/2007/understanding-full-virtualization-paravirtualizat-1008.html>, 2007.
- [Matthews 08] Matthews, J., et al. Running Xen: «A Hands-On Guide to the Art of Virtualization», Prentice Hall, 2008.

- [Milewski 11] Milewski, B., «Virtual Machines: Virtualizing Virtual Memory», <http://corensic.wordpress.com/2011/12/05/virtual-machines-virtualizing-virtual-memory>, 2011.
- [Adamczyk 12] Adamczyk, B., and Chydzinski, A., «Performance Isolation Issues in Network Virtualization in Xen», *International Journal on Advances in Networks and Services*, 2012.
- [Hoff 12] Hoff, T., «Pinterest Cut Costs from \$54 to \$20 Per Hour by Automatically Shutting Down Systems», <http://highscalability.com/blog/2012/12/12/pinterest-cut-costs-from-54-to-20-per-hour-by-automatically.html>, 2012.
- [Gregg 14b] Gregg, B., «From Clouds to Roots: Performance Analysis at Netflix», <http://www.brendangregg.com/blog/2014-09-27/from-clouds-to-roots.html>, 2014.
- [Heo 15] Heo, T., «Control Group v2», Linux documentation, <https://www.kernel.org/doc/Documentation/cgroup-v2.txt>, 2015.
- [Gregg 16a] Gregg, B., «Unikernel Profiling: Flame Graphs from dom0», <http://www.brendangregg.com/blog/2016-01-27/unikernel-profiling-from-dom0.html>, 2016.
- [Kadera 16] Kadera, M., «Accelerating the Next 10,000 Clouds», <https://www.slideshare.net/Docker/accelerating-the-next-10000-clouds-by-michael-kadera-intel>, 2016.
- [Borello 17] Borello, G., «Container Isolation Gone Wrong», Sysdig blog, <https://sysdig.com/blog/container-isolation-gone-wrong>, 2017.
- [Goldfuss 17] Goldfuss, A., «Making FlameGraphs with Containerized Java», <https://blog.alicegoldfuss.com/making-flamegraphs-with-containerized-java>, 2017.
- [Gregg 17e] Gregg, B., «AWS EC2 Virtualization 2017: Introducing Nitro», <http://www.brendangregg.com/blog/2017-11-29/aws-ec2-virtualization-2017.html>, 2017.
- [Gregg 17f] Gregg, B., «The PMCs of EC2: Measuring IPC», <http://www.brendangregg.com/blog/2017-05-04/the-pmcs-of-ec2.html>, 2017.
- [Gregg 17g] Gregg, B., «Container Performance Analysis at DockerCon 2017», <http://www.brendangregg.com/blog/2017-05-15/container-performance-analysis-dockercon-2017.html>, 2017.
- [CNI 18] «bandwidth plugin», <https://github.com/containernetworking/plugins/blob/master/plugins/meta/bandwidth/README.md>, 2018.
- [Denis 18] Denis, X., «A Pods Architecture to Allow Shopify to Scale», <https://engineering.shopify.com/blogs/engineering/a-pods-architecture-to-allow-shopify-to-scale>, 2018.
- [Leonovich 18] Leonovich, M., «Another reason why your Docker containers may be slow», <https://hackernoon.com/another-reason-why-your-docker-containers-may-be-slowd37207dec27f>, 2018.
- [Borkmann 19] Borkmann, D., and Pumputis, M., «Kube-proxy Removal», <https://cilium.io/blog/2019/08/20/cilium-16/#kubeproxy-removal>, 2019.
- [Cilium 19] «Announcing Hubble — Network, Service & Security Observability for Kubernetes», <https://cilium.io/blog/2019/11/19/announcing-hubble/>, 2019.
- [Gregg 19] Gregg, B., «BPF Performance Tools: Linux System and Application Observability»¹, Addison-Wesley, 2019.

¹ Грегг Б. «BPF: Профессиональная оценка производительности». Выходит в издательстве «Питер» в 2023 году.

- [Gregg 19e] Gregg, B., «kvmexits.bt», https://github.com/brendangregg/bpf-perf-tools-book/blob/master/originals/Ch16_Hypervisors/kvmexits.bt, 2019.
- [Kwiatkowski 19] Kwiatkowski, A., «Autoscaling in Reality: Lessons Learned from Adaptively Scaling Kubernetes», <https://conferences.oreilly.com/velocity/vl-eu/public/schedule/detail/78924>, 2019.
- [Xen 19] «Xen PCI Passthrough», http://wiki.xen.org/wiki/Xen_PCI_Passthrough, 2019.
- [Agache 20] Agache, A., et al., «Firecracker: Lightweight Virtualization for Serverless Applications», <https://www.amazon.science/publications/firecracker-lightweight-virtualization-for-serverless-applications>, 2020.
- [Calico 20] «Cloud Native Networking and Network Security», <https://github.com/projectcalico/calico>, последнее обновление 2020.
- [Cilium 20a] «API-aware Networking and Security», <https://cilium.io>, по состоянию на 2020.
- [Cilium 20b] «eBPF-based Networking, Security, and Observability», <https://github.com/cilium/cilium>, последнее обновление 2020.
- [Firecracker 20] «Secure and Fast microVMs for Serverless Computing», <https://github.com/firecracker-microvm/firecracker>, последнее обновление 2020.
- [Google 20c] «Google Compute Engine FAQ», <https://developers.google.com/compute/docs/faq#whatis>, по состоянию на 2020.
- [Google 20d] «Analyzes Resource Usage and Performance Characteristics of Running containers», <https://github.com/google/cadvisor>, последнее обновление 2020.
- [Kata Containers 20] «Kata Containers», <https://katacontainers.io>, по состоянию на 2020.
- [Linux 20m] «mount_namespaces(7)», http://man7.org/linux/man-pages/man7/mount_namespaces.7.html, по состоянию на 2020.
- [Weaveworks 20] «Ignite a Firecracker microVM», <https://github.com/weaveworks/ignite>, последнее обновление 2020.
- [Kubernetes 20b] «Production-Grade Container Orchestration», <https://kubernetes.io>, по состоянию на 2020.
- [Kubernetes 20c] «Tools for Monitoring Resources», <https://kubernetes.io/docs/tasks/debug-application-cluster/resource-usage-monitoring>, последнее обновление 2020.
- [Torvalds 20b] Torvalds, L., «Re: [GIT PULL] x86/mm changes for v5.8», <https://lkml.org/lkml/2020/6/1/1567>, 2020.
- [Xu 20] Xu, P., «iops limit for pod/pvc/pv #92287», <https://github.com/kubernetes/kubernetes/issues/92287>, 2020.

Глава 12

БЕНЧМАРКИНГ

Есть ложь, есть наглая ложь, и есть метрики производительности.

Anon et al., «A Measure of Transaction Processing Power» [Anon 85]

Контролируемый бенчмаркинг позволяет сравнивать выбранные варианты, выявлять регрессии и оценивать ограничения производительности еще до того, как они обнаружатся в промышленной среде. Ограничения могут быть обусловлены системными ресурсами, программными ограничениями в виртуализированной среде (облачные вычисления) или ограничениями в целевом приложении. В предыдущих главах были рассмотрены эти компоненты, описаны типы ограничений и инструменты, используемые для их анализа.

Кроме того, я показал инструменты для *микробенчмаркинга*, использующие простые искусственные рабочие нагрузки, — файловый ввод/вывод для тестирования компонентов. Есть также *макробенчмаркинг*, имитирующий рабочие нагрузки для проверки всей системы. Макробенчмаркинг может включать имитацию рабочей нагрузки или воспроизведение рабочей нагрузки по результатам трассировки. Независимо от выбранного типа бенчмаркинга важно проанализировать полученные результаты и убедиться, что вы понимаете, что именно было оценено. Бенчмарки сообщают лишь то, насколько быстро система может выполнять их, поэтому важно проанализировать результат и определить, насколько он применим к вашей среде.

Цели этой главы:

- познакомить с микро- и макробенчмаркингом;
- обозначить многочисленные подводные камни бенчмаркинга;
- познакомить с методологией активного бенчмаркинга;
- представить чек-лист для проверки результатов бенчмаркинга;
- показать, как наращивать точность при бенчмаркинге и интерпретации результатов.

В главе рассмотрен бенчмаркинг в целом. Даны рекомендации и методологии, которые помогут избежать распространенных ошибок и точно протестировать свои системы. Информация из этой главы пригодится вам при интерпретации других результатов, в том числе из промышленных бенчмарков или бенчмарков вендора.

12.1. ОСНОВЫ

В этом разделе описаны приемы эффективного бенчмаркинга и обобщены распространенные ошибки.

12.1.1. Причины

Бенчмаркинг можно проводить по следующим причинам:

- **Проектирование системы:** сравнение различных систем, компонентов или приложений. Бенчмаркинг коммерческих продуктов позволяет получить данные, помогающие принять решение о покупке, в частности соотношение *цена/производительность* для имеющихся вариантов¹. В некоторых случаях могут использоваться опубликованные бенчмарки, что позволит избежать самостоятельного проведения бенчмаркинга.
- **Проверка концепции:** для проверки производительности программного или аппаратного обеспечения под нагрузкой, перед покупкой или развертыванием в промышленной среде.
- **Настройка:** тестирование настраиваемых параметров и конфигураций с целью определения тех из них, которые заслуживают дальнейшего изучения с учетом рабочей нагрузки.
- **Разработка:** для *нерегрессионного тестирования* и *определения пределов* при разработке продукта. Нерегрессионное тестирование можно проводить с помощью пакета бенчмарков, и делать это регулярно, чтобы любое ухудшение производительности обнаруживалось на ранней стадии. Это позволит быстро сопоставить результаты с изменениями в продукте. В случае определения пределов бенчмаркинг можно использовать для доведения нагрузки на продукт до предела во время разработки и выяснить, куда направить усилия инженеров, чтобы повысить производительности продукта.
- **Планирование емкости:** определение ограничений системы и приложений для планирования емкости либо получение данных для моделирования производительности или прямого определения пределов емкости.
- **Устранение неисправностей** (troubleshooting): для проверки способности компонентов работать с максимальной производительностью, например, тестирование максимальной пропускной способности сети между хостами, чтобы проверить, не возникнет ли проблема с сетью.
- **Маркетинг:** определение максимальной производительности продукта для использования в рекламе (также называется *бенчмаркетингом* — *benchmarketing*).

¹ Несмотря на широкое использование соотношения цена/производительность, мне кажется, что соотношение производительность/цена выглядит понятнее. С точки зрения математики (а не просто привычного высказывания) такое соотношение имеет более привычную интерпретацию «чем больше, тем лучше» для людей, оценивающих показатели производительности.

Бенчмаркинг оборудования в корпоративных средах для проверки концепции (proof-of-concept, PoC) может стать важным этапом перед крупной закупкой оборудования и длиться неделями или даже месяцами. Сюда входит время на доставку, монтаж и прокладку кабельных линий, а также на установку ОС перед тестированием. Эта процедура может проводиться раз в год или в два, с выпуском нового оборудования.

В облачных средах ресурсы доступны по требованию без дорогостоящих инвестиций в оборудование, и при необходимости их количество можно быстро изменить (повторное развертывание на экземплярах различных типов). Эти среды тоже требуют долгосрочных инвестиций при выборе языка программирования и ОС, баз данных, веб-серверов и балансировщиков нагрузки. Некоторые из этих компонентов бывает сложно заменить на ходу. Бенчмаркинг помогает выяснить, насколько хорошо различные компоненты масштабируются при необходимости. Модель облачных вычислений упрощает бенчмаркинг: она позволяет за считанные минуты создать крупномасштабную среду для запуска бенчмарка, а затем уничтожить его, и все это обойдется очень недорого.

Обратите внимание, что отказоустойчивая и распределенная облачная среда также упрощает экспериментирование: с появлением новых типов экземпляров среда может быть немедленно подвергнута бенчмаркингу с помощью промышленных рабочих нагрузок, минуя традиционный бенчмаркинг. В этом сценарии бенчмаркинг можно использовать для объяснения различий путем сравнения производительности компонентов. Например, команда поддержки производительности в Netflix имеет автоматизированное ПО для анализа новых типов экземпляров с помощью различных микробенчмарков. Оно автоматически собирает системные статистики и профили процессоров, что позволяет проанализировать и объяснить любые обнаруженные различия.

12.1.2. Эффективный бенчмаркинг

Бенчмаркинг — удивительно сложная задача с массой возможностей для ошибок и упущений. Как отмечено в статье «A Nine Year Study of File System and Storage Benchmarking» [Traeger 08]:

Работая над этой статьей, мы изучили 415 бенчмарков файловых систем и хранилищ из 106 недавних статей. Мы обнаружили, что большинство популярных бенчмарков дают неверную картину, и многие исследовательские работы не позволяют получить четкое представление об истинной производительности.

В статье есть рекомендации относительно того, что можно сделать. В частности, оценки бенчмаркинга должны объяснять, *что* было протестировано и *почему*, и они должны включать некоторый анализ ожидаемого поведения системы.

Вот суть хорошего бенчмаркинга [Smaalders 06]:

- **повторяемость:** чтобы упростить сравнение;
- **наблюдаемость:** чтобы проанализировать и объяснить результаты;

- **переносимость:** чтобы провести сравнительный анализ конкурентов и различных версий продукта;
- **простота представления результатов:** чтобы каждый мог понять их;
- **реалистичность:** чтобы измерения отражали реальную картину, которую видят клиенты;
- **возможность запуска:** чтобы разработчики могли быстро протестировать свои изменения.

При сравнении различных систем с целью их приобретения стоит добавить еще одну характеристику: соотношение цена/производительность. Цену можно количественно определить как пятилетние капиталовложения в оборудование [Anon 85].

Эффективность также зависит от того, как используются бенчмарки: нужно проводить анализ и на его основе делать выводы.

Анализ бенчмаркинга

При использовании бенчмарков вы должны понимать:

- что тестируется;
- какие есть ограничивающие факторы;
- какие есть возмущающие факторы, которые могут повлиять на результаты;
- какие выводы можно сделать на основе результатов.

Для этого нужно глубокое понимание того, что делает ПО, осуществляющее бенчмарки, как реагирует система на его действия и как результаты соотносятся с целевой средой.

При наличии инструмента бенчмаркинга и доступа к системе эти потребности лучше всего удовлетворяются путем анализа производительности системы во время бенчмаркинга. Распространенная ошибка — когда бенчмарки выполняются младшим персоналом, а затем для интерпретации результатов приглашают экспертов по производительности. Намного лучше привлекать экспертов уже на этапе бенчмаркинга — так они смогут проанализировать работу системы в обычном режиме, в том числе сделать углубленный анализ для объяснения и количественной оценки ограничивающего фактора (факторов).

Ниже приведен интересный пример анализа:

Изучая производительность получившейся реализации TCP/IP, мы провели эксперимент и передали 4 Мбайт данных между двумя пользовательскими процессами, действующими на разных машинах. Передача выполнялась 1024-байтными записями, инкапсулированными в 1068-байтные пакеты Ethernet. Отправка данных с 11/750 на 11/780 по протоколу TCP/IP заняла 28 с. Сюда вошло все время, необходимое для установки и разрыва соединений и для обеспечения

пропускной способности 1,2 Мбод. Во время эксперимента процессор на 11/750 был нагружен на 100 %, а на 11/780 — на 70 %. Время, затраченное системой на обработку данных, распределилось между обработкой пакетов Ethernet (20 %), IP-пакетов (10 %), TCP-пакетов (30 %), вычислением контрольных сумм (25 %) и обработкой обращений пользователя к системным вызовам (15 %). При этом никакой этап обработки не оказался доминирующим по затраченному процессорному времени.

Здесь описаны ограничивающие факторы («процессор на 11/750 был нагружен на 100 %»¹), а затем объясняется, какие компоненты ядра внесли вклад в это ограничение. Кроме того, выполнить такой анализ и обобщить затраты процессорного времени разными компонентами ядра стало проще лишь недавно благодаря появлению флейм-графиков. Эти слова написаны задолго до появления флейм-графиков и принадлежат Биллу Джою, описавшему оригинальный стек TCP/IP BSD в 1981 году [Joy 81].

Нередко эффективнее разработать и использовать собственное ПО для бенчмаркинга вместо стандартных инструментов или, по крайней мере, генераторы нагрузки. Их можно сделать специализированными, сосредоточив внимание только на том, что важно для вашего теста; это ускорит их анализ и отладку.

Иногда инструменты бенчмаркинга или система могут быть недоступны, как, например, при изучении бенчмарков, проведенных другими пользователями. В таких случаях пройдите по пунктам списка из начала этого подраздела. Основываясь на доступных материалах, ответьте на вопросы: какая системная среда используется? как она настроена? Рассмотрите и другие вопросы, которые приведены в разделе 12.4 «Вопросы о бенчмаркинге».

12.1.3. Проблемы бенчмаркинга

В подразделах ниже представлен чек-лист различных ошибок при бенчмаркинге — оплошностей, заблуждений и неправильных действий. Также я рассказал о том, как их избежать. О том, как проводить бенчмаркинг, речь идет в разделе 12.3 «Методология».

1. Случайное тестирование

Бенчмаркинг — это не то занятие, о котором можно сразу забыть. Инструменты бенчмаркинга сообщают цифры, которые могут отражать совсем не то, что вы думаете, и ваши выводы на их основе могут быть ложными. Вот пример:

Случайный бенчмаркинг: вы тестируете А, но на самом деле получаете характеристики В и делаете вывод, что протестировали С.

¹ 11/750 — это сокращение от VAX-11/750, мини-компьютера, произведенного компанией DEC в 1980 году.

Хороший бенчмаркинг требует тщательности, чтобы проверить именно то, что предполагалось, и понимания того, что было протестировано, чтобы можно было сделать обоснованные выводы.

Например, многие инструменты заявляют или подразумевают, что они оценивают производительность диска, но на самом деле тестируют производительность файловой системы. Разница в оценках может достигать порядков, потому что файловые системы используют кэширование и буферизацию, подменяя дисковый ввод/вывод операциями с памятью. Даже если инструмент бенчмаркинга работает правильно и тестирует файловую систему, ваши выводы о дисках будут совершенно неверными.

Бенчмарки сложны для понимания, особенно новичками, у которых недостаточно опыта, чтобы понять, подозрительны ли цифры или нет. Если вдруг комнатный термометр покажет температуру в 100 градусов, вы сразу поймете, что что-то не так. Но так нельзя сказать о бенчмарках, сообщающих числа, которые ни о чем вам не говорят.

2. Слепая вера

Заманчиво довериться популярному инструменту бенчмаркинга, особенно если он давний и у него открытый исходный код. Это распространенное заблуждение, что популярность равна достоверности. Оно известно как *argumentsum ad populum* (лат. «аргумент к народу»¹).

Анализ используемых бенчмарков требует времени и специальных знаний. А если какой-то бенчмарк популярен, то анализ того, что *обязательно* должно быть достоверным, может показаться пустой тратой времени и сил.

Если бы популярный бенчмарк продвигался одной из ведущих компаний в индустрии IT, вы бы доверились ему? В прошлом уже были случаи, когда рекламируемый микробенчмарк, широко известный среди опытных перформанс-инженеров, оказывался ошибочным. К сожалению, простого способа избежать этого нет (я пробовал).

И проблема здесь даже не в ПО для бенчмаркинга (хотя ошибки случаются), а в интерпретации результатов.

3. Числа без анализа

Простые результаты бенчмаркинга, не содержащие аналитических деталей, могут служить признаком неопытности автора, наивно полагающего, что результаты заслуживают доверия и окончательные. Часто получение результатов — лишь начальный этап исследований, когда можно обнаружить, что результаты ошибочны или запутывают.

¹ Вид заведомо ошибочной логической аргументации, основанной на мнении, что большинство всегда право. — *Примеч. пер.*

Каждое число бенчмарка должно сопровождаться описанием обнаруженного предела и выполненного анализа:

Если вы потратили на изучение бенчмарка меньше недели, то, скорее всего, поступили неправильно.

Большая часть этой книги посвящена анализу производительности, который следует проводить во время бенчмаркинга. Когда у вас нет времени на тщательный анализ, рекомендую перечислить предположения, на проверку которых у вас не было времени, и включить этот перечень в результаты, например:

- предполагается, что инструмент бенчмаркинга не содержит ошибок;
- предполагается, что тест дискового ввода/вывода действительно измеряет характеристики дискового ввода/вывода;
- предполагается, что инструмент бенчмаркинга довел нагрузку дискового ввода/вывода до предела;
- предполагается, что этот вид дискового ввода/вывода подходит для этого приложения.

Такой перечень вполне может стать списком заданий для дальнейшей проверки, если позже результаты бенчмарка будут признаны достаточно важными, чтобы провести дополнительные исследования.

4. Сложные инструменты бенчмаркинга

Важно, чтобы инструмент бенчмаркинга не мешал анализу производительности своей сложностью. В идеале программа должна иметь открытый исходный код, который можно изучить, и быть достаточно короткой, чтобы ее можно было быстро прочитать и понять.

На роль микробенчмарков рекомендую выбирать программы, написанные на языке С. Для имитации поведения клиентского ПО рекомендуется использовать тот же язык программирования, на котором написан клиент, чтобы минимизировать различия.

Распространенная проблема — *бенчмаркинг бенчмарка*, когда сообщаемый результат ограничивается самим бенчмарком. Особенно часто это объясняется однопоточной природой ПО для бенчмаркинга. Сложные наборы бенчмарков могут затруднить анализ из-за большого объема кода, который нужно понять и проанализировать.

5. Тестирование не того, что надо

Несмотря на большое количество инструментов бенчмаркинга различных рабочих нагрузок, многие из них могут не подходить для целевого приложения.

Например, распространенная ошибка — бенчмаркинг диска с использованием доступных инструментов, даже при том, что потребности рабочей нагрузки в целевой

среде будут полностью удовлетворяться из кэша файловой системы и она не будет зависеть от дискового ввода/вывода.

Команда инженеров, разрабатывающая продукт, может взять за эталон конкретный бенчмарк и все силы направить на повышение производительности, измеряемой им. Но если бенчмарк имеет мало общего с фактической рабочей нагрузкой, то усилия инженеров будут направлены на оптимизацию не того поведения [Smaalders 06].

В реальной промышленной среде можно использовать методологию определения характеристик фактической рабочей нагрузки (описанную в предыдущих главах), чтобы оценить ее структуру и измерить показатели от устройства ввода/вывода до обработки запросов в приложении. Эти измерения помогут выбрать наиболее подходящие бенчмарки. Если промышленная среда недоступна для анализа, то можно симитировать ее или смоделировать предполагаемую рабочую нагрузку. Также познакомьте с данными бенчмарков целевую аудиторию и узнайте, согласна ли она с результатами тестов.

Возможно, бенчмарк когда-то соответствовал рабочей нагрузке, но из-за того, что не обновлялся годами, сейчас может тестировать совсем не то.

6. Игнорирование особенностей среды

Сопоставимы ли промышленная и тестовая среды? Представьте, что нужно оценить новую базу данных. Вы подготавливаете тестовый сервер и запускаете бенчмарк базы данных, а потом узнаете, что упустили из виду один важный аспект: тестовый сервер действует с настройками по умолчанию, с файловой системой по умолчанию и т. д. Обычно промышленные серверы баз данных настраиваются на выполнение максимального количества операций дискового ввода/вывода в секунду, поэтому тестирование в системе с настройками по умолчанию даст результаты, далекие от реальности: вы не сделали первый важный шаг — не изучили особенности промышленной среды.

7. Игнорирование ошибок

Тот факт, что инструмент бенчмаркинга дает какой-то результат, еще не означает, что результат отражает *успешное* выполнение теста. Некоторые или даже все запросы могли привести к ошибке. Эта проблема связана с предыдущими ошибками и упущениями, но она настолько распространена, что ее стоит упомянуть отдельно.

Об этом мне напомнили во время бенчмаркинга веб-сервера. Команда, проводившая тестирование, сообщила, что средняя задержка веб-сервера оказалась слишком высокой — более одной секунды *в среднем*. Одна секунда? В ходе короткого анализа удалось выяснить, что пошло не так: во время тестирования веб-сервер вообще ничего не делал, так как все запросы блокировались брандмауэром. *Все* запросы. Получившаяся задержка — это время, которое требовалось тестовому клиенту, чтобы прервать ожидание ответа по тайм-ауту!

8. Игнорирование изменчивости

Инструменты бенчмаркинга, особенно микробенчмаркинга, часто применяют стабильную и последовательную рабочую нагрузку, основанную на *среднем значении*, вычисленном по серии измерений характеристик, произведенных в разное время суток или за определенный интервал. Например, рабочая нагрузка может выполнять в среднем 500 операций чтения с диска в секунду и 50 операций записи. Инструмент бенчмаркинга, опираясь на эти значения, может смоделировать эту частоту или соотношение 10:1 операций чтения/записи, чтобы протестировать более высокую нагрузку.

Такой подход игнорирует дисперсию: скорость операций может меняться. Меняться могут и типы операций, а некоторые из них могут выполняться независимо друг от друга. Например, запись данных может производиться пакетами раз в 10 с (асинхронное выталкивание буферов отложенной записи), тогда как характер синхронных операций чтения почти не меняется. Операции пакетной записи могут вызвать проблемы в промышленной среде, например, вынуждая операции чтения простаивать долгое время в очереди, которые не воспроизводятся бенчмарком, соблюдающим постоянную среднюю нагрузку.

Одно из возможных решений для моделирования дисперсии — использовать в бенчмарках модели Маркова, отражающие вероятность того, что за одной операцией записи может последовать другая.

9. Игнорирование возмущающих факторов

Подумайте, какие внешние возмущающие факторы могут повлиять на результаты. Будут ли во время бенчмаркинга выполняться какие-либо запланированные по времени действия — резервное копирование или работа агентов мониторинга, собирающих статистику раз в минуту? В облачных средах таким возмущающим фактором также могут быть действия невидимых арендаторов в той же хост-системе.

На практике для сглаживания возмущений часто используется стратегия увеличения продолжительности выполнения бенчмарка — до нескольких минут вместо секунд. Обычно продолжительность бенчмаркинга не должна быть меньше одной секунды. Кратковременные тесты могут не столкнуться с прерываниями устройств (когда поток приостанавливается на время выполнения процедуры обработки прерывания), с решениями планировщика процессоров (ожидание в очереди перед переносом для сохранения привязки к процессору) и с эффектами разогрева кэша процессора. Попробуйте запустить бенчмарк несколько раз и проверьте стандартное отклонение — оно должно быть как можно меньше.

Также соберите данные, которые помогут изучить возмущения, если они есть. Пример таких данных — распределение задержек операций в дополнение к общему времени выполнения бенчмарка, чтобы можно было увидеть выбросы и выяснить их причины.

10. Изменение множества факторов

Сравнивая результаты двух бенчмарков, будьте внимательны и попробуйте опознать все факторы, отличающие их друг от друга.

Например, тестируя два хоста в сети, выясните: идентична ли сеть между ними? Не находится ли один хост на большем расстоянии и в более медленной или более нагруженной сети? Любые такие дополнительные факторы могут усложнить анализ результатов тестирования.

В облачных средах для бенчмарков иногда создают отдельные экземпляры, которые затем уничтожаются. При этом не учитываются многие факторы: экземпляры могут создаваться в более быстрых или медленных системах или в системах с более высокой нагрузкой и конкуренцией со стороны других арендаторов. Рекомендую протестировать несколько экземпляров и взять медианное значение (или, что еще лучше, записать распределение), чтобы исключить выбросы, вызванные тестированием в необычно быстрой или необычно медленной системе.

11. Парадокс бенчмарка

Потенциальные покупатели часто используют бенчмарки для оценки продукта перед покупкой, и они бывают настолько неточны, что с таким же успехом можно просто подбросить монетку. Один продавец как-то сказал мне, что был бы доволен таким шансом: выигрыш в половине случаев оценки его продукта вполне соответствовал бы его планам продаж. Но игнорирование бенчмарков — это подводный камень бенчмаркинга, и на практике шансы гораздо хуже:

Если шансы вашего продукта на победу в бенчмарке равны 50/50, то это означает, что в большинстве случаев вы будете проигрывать. [Gregg 14c]

Этот кажущийся парадокс можно объяснить простой вероятностью.

Покупая продукт, ориентируясь на производительность, клиенты хотят быть уверены в том, что он действительно обладает высокой производительностью. Они могут запустить несколько бенчмарков, желая во всех них увидеть продукт на первом месте. Если бенчмарк имеет вероятность выигрыша 50 %, то:

Вероятность выигрыша в трех тестах = $0,5 \times 0,5 \times 0,5 = 0,125 = 12,5 \%$.

Чем больше бенчмарков, тем меньше шансов выиграть их все.

12. Бенчмаркинг конкурентов

Ваш отдел маркетинга наверняка хотел бы иметь результаты бенчмарков, показывающие, насколько ваш продукт превосходит конкурентов. Это намного сложнее, чем кажется.

Когда покупатели выбирают товар, они пробуют его не пять минут, а анализируют и настраивают месяцами, чтобы добиться максимальной производительности. Они устраняют самые серьезные проблемы в первые несколько недель.

У вас нет нескольких недель, чтобы проанализировать и настроить продукт, *конкурирующий* с вашим. В отведенное время вы можете опробовать только ненастроенный продукт и, следовательно, получить результаты, далекие от реальности. Клиенты вашего конкурента, на которых направлена эта маркетинговая активность, вполне могут увидеть, что вы опубликовали результаты, полученные на ненастроенном продукте. Тогда ваша компания потеряет доверие у тех самых людей, на которых пыталась произвести впечатление.

Если понадобится провести сравнительный анализ с конкурирующим продуктом, то придется потратить довольно много времени на его настройку. Проанализируйте производительность, используя методы, описанные в предыдущих главах. Поищите рекомендации по настройке на форумах клиентов и в базах данных ошибок. Возможно, вы даже решите привлечь сторонних экспертов для настройки системы. Затем приложите те же усилия для настройки своего продукта и только потом проводите сравнительный бенчмаркинг.

13. Огонь по своим

При бенчмаркинге собственных продуктов приложите все силы, чтобы убедиться, что тестирование выполняется в самой эффективной системе и с оптимальной конфигурацией, а производительность системы доведена до предела. Поделитесь результатами с командой инженеров перед публикацией. Они могут заметить ошибки в конфигурации, которые вы пропустили. А если вы работаете в такой команде инженеров, внимательно следите за результатами бенчмаркинга — независимо от того, делает ли его ваша компания или подрядчик, — и помогайте коллегам.

Однажды я наблюдал, как команда инженеров упорно разрабатывала высокопроизводительный продукт, основанный на применении новой технологии, которая еще не была задокументирована. Для запуска продукта команде по бенчмаркингу предложили представить цифры. Они не понимали новой технологии (так как она не была задокументирована) и использовали неправильные настройки, а затем опубликовали цифры, не отражающие все преимущества продукта.

Иногда система может быть настроена правильно, но работает не на пределе возможностей. Задайте вопрос: что является узким местом для этого бенчмарка? Это может быть физический ресурс (например, процессоры, диски или внутренние соединения) с уровнем потребления, достигшим 100 %, который можно выявить при анализе. См. раздел 12.3.2 «Активный бенчмаркинг».

Еще одна проблема «огня по своим» возникает при бенчмаркинге старых версий ПО, имеющих проблемы с производительностью, которые были исправлены в более поздних версиях, или при тестировании на оборудовании с ограниченными возможностями, доступном на тот момент, но дающем не лучший результат. В большинстве случаев потенциальные клиенты склонны предполагать, что любые результаты,

опубликованные компанией, показывают наилучшую возможную производительность. Мало кто допускает, что эти результаты могли быть получены не в лучших условиях.

14. Бенчмарки, вводящие в заблуждение

Вводящие в заблуждение бенчмарки — обычное дело в индустрии. Это может быть обусловлено непреднамеренным ограничением информации о том, что на самом деле измеряет бенчмарк, или намеренным исключением некоторой информации из публикации. Часто результаты бенчмарка остаются технически правильными, но представляются клиенту в ложном свете.

Рассмотрим такую гипотетическую ситуацию: производитель достигает фантастического результата, создав нестандартный продукт, который получился настолько дорогим, что ни один реальный покупатель не купит его. Цена продукта не раскрывается вместе с результатами бенчмарков, в которых основное внимание уделяется показателям, не связанным с соотношением цена/производительность. Отдел маркетинга публикует неоднозначное обобщение результатов («Мы в два раза быстрее!»), связывая его в сознании клиентов либо с компанией в целом, либо с линейкой продуктов. Это случай исключения деталей с целью произвести благоприятное впечатление. Это нельзя назвать обманом — цифры не поддельные, но это *ложь из-за неполноты информации*.

Впрочем, даже такие результаты могут быть полезны как ориентир, показывающий верхнюю границу производительности. Это значения, превышения которых не следует ожидать (за исключением случаев огня по своим).

Рассмотрим другую гипотетическую ситуацию: у отдела маркетинга есть бюджет, который можно потратить на рекламную кампанию, и они хотят получить хорошие результаты бенчмарков. Они привлекают несколько подрядчиков для бенчмаркинга своего продукта и выбирают наилучший результат из полученных. Эти подрядчики выбираются не на основе опыта, а по способности дать результаты быстро и недорого. Фактически отсутствие опыта в такой ситуации оказывается выгоднее: чем больше результаты отклоняются от реальности, тем лучше — в идеале один из них будет сильно отклоняться в положительную сторону!

При использовании результатов, публикуемых производителем, внимательно проверьте, в какой системе производилось тестирование, какие типы дисков и какие сетевые интерфейсы использовались, какие настройки были выбраны, а также другие факторы. Чтобы узнать, чего следует опасаться, см. раздел 12.4 «Вопросы о бенчмаркинге».

15. Специализация под бенчмарки

Специализация под бенчмарки — это когда поставщик изучает популярный или отраслевой бенчмарк, а затем проектирует продукт так, чтобы тот показал хорошие результаты в этом бенчмарке, не принимая во внимание фактическую производительность клиента. Это также называется оптимизацией для бенчмарка.

Термин «специализация под бенчмарки» вошел в употребление в 1993 году вместе с бенчмарком TPC-A, как описано на странице с историей Transaction Processing Performance Council (TPC) [Shanley 98]:

Консалтинговая компания Standish Group из Массачусетса обвинила Oracle в добавлении специальной возможности (дискретных транзакций) в программное обеспечение баз данных с единственной целью — завysить результаты TPC-A. Standish Group заявила, что Oracle «нарушила дух TPC», потому что дискретные транзакции не используются типичными заказчиками, и следовательно, являются специализацией под бенчмарк. Oracle отвергла это обвинение, заявив, не без оснований, что они следовали букве закона при составлении спецификаций бенчмарков. Компания утверждала, что поскольку специализация под бенчмарки, не говоря уже о духе TPC, никак не оговаривается в спецификациях TPC, то несправедливо обвинять их в каких-либо нарушениях.

TPC добавила специальный пункт, запрещающий специализацию:

Запрещены все «специализированные» реализации, показывающие повышенную производительность при бенчмаркинге, но не реальную производительность или цены.

Поскольку TPC сосредоточена на соотношении цена/производительность, другой стратегией завышения цифр может быть использование *специальных цен* — больших скидок, которые на самом деле не получит ни один покупатель. Подобно специализации ПО, специализация цен не соответствует действительности, когда реальный клиент покупает систему. Компания TPC учла это в своих требованиях [TPC 19a]:

Согласно спецификациям TPC, общая цена не должна отличаться больше чем на 2 % от цены, которую покупатель заплатит за конфигурацию.

Эти примеры объясняют понятие специализации под бенчмарки, но TPC учла их в своих спецификациях много лет назад, и сейчас об этом уже можно не беспокоиться.

16. Мошенничество

Последняя проблема в бенчмаркинге — мошенничество: публикация фальшивых результатов. К счастью, это случается очень редко, по крайней мере мне не приходилось сталкиваться с выдуманными цифрами, даже в самой кровавой конкурентной борьбе.

12.2. ТИПЫ БЕНЧМАРКИНГА

На рис. 12.1 показан спектр типов бенчмарков, в зависимости от тестируемой рабочей нагрузки. Промышленная нагрузка также включена в этот спектр.

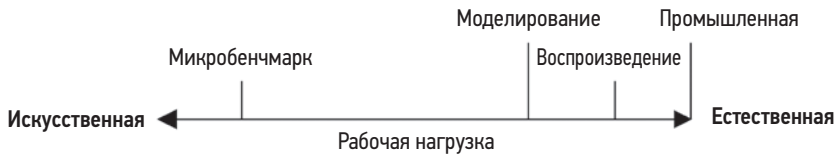


Рис. 12.1. Типы бенчмарков

В разделах ниже описаны три типа бенчмаркинга: микробенчмаркинг, моделирование и трассировка/воспроизведение. Рассмотрены и стандартные отраслевые бенчмарки.

12.2.1. Микробенчмаркинг

В микробенчмаркинге используются искусственные рабочие нагрузки, тестирующие операции определенного типа, например одного типа операций ввода/вывода с файловой системой, запросов к базе данных, машинных инструкций или одного системного вызова. Преимущество микробенчмарков в их простоте: сужение круга задействованных компонентов и путей в коде упрощает изучение цели и позволяет быстро устранить коренные причины уменьшения производительности. Микробенчмарки обычно легко повторить, потому что влияние других компонентов сведено к минимуму и их легко использовать в разных системах. А поскольку микробенчмарки намеренно сделаны искусственными, их трудно спутать с моделированием реальных рабочих нагрузок.

Чтобы использовать результаты микробенчмаркинга, их нужно сопоставить с целевой рабочей нагрузкой. Микробенчмарк может тестировать несколько параметров, но только один или два могут быть актуальными. Анализ производительности или моделирование целевой системы помогают определить, какие результаты микробенчмаркинга подходят и в какой степени.

Вот примеры инструментов для микробенчмаркинга по типам ресурсов, упомянутые в предыдущих главах:

- **процессоры:** SysBench;
- **ввод/вывод с памятью:** Imbencb (упоминается в главе 6 «Процессоры»);
- **файловые системы:** fio;
- **диски:** hdbarm, dd или fio с прямым вводом/выводом;
- **сети:** iperf.

Есть множество других инструментов для бенчмаркинга, но помните предупреждение [Traeger 08]: «Большинство популярных бенчмарков несовершенны».

При желании можно разработать свои инструменты. Старайтесь делать их как можно более простыми и определяйте атрибуты рабочей нагрузки, которые можно тестировать индивидуально. (Подробнее об этом рассказывается в разделе 12.3.6

«Собственные тесты».) Используйте сторонние инструменты, чтобы убедиться, что они выполняют те операции, которые должны выполнять.

Пример дизайна бенчмарка

Рассмотрим разработку микробенчмарка файловой системы для тестирования следующих характеристик: производительности последовательного и произвольного ввода/вывода; ввода/вывода с блоками разных размеров; и наконец, в разных направлениях (чтение и запись). В табл. 12.1 перечислены пять примеров тестов и их назначение.

Таблица 12.1. Примеры тестов для микробенчмаркинга файловой системы

№	Тест	Назначение
1	Последовательное чтение блоками по 512 байт ¹	Протестировать максимальное (реалистичное) количество операций ввода/вывода в секунду
2	Последовательное чтение блоками по 1 Мбайт ²	Протестировать максимальную пропускную способность чтения
3	Последовательная запись блоками по 1 Мбайт	Протестировать максимальную пропускную способность записи
4	Произвольное чтение блоками по 512 байт	Протестировать эффект произвольного ввода/вывода
5	Произвольная запись блоками по 512 байт	Протестировать эффект повторной записи

При желании можно добавить другие тесты. Эти тесты можно разнообразить применением двух дополнительных факторов:

- **Размер рабочего набора.** Объем данных, к которым осуществляется доступ (например, общий размер файла):

¹ Цель — максимизировать количество операций ввода/вывода в секунду за счет использования блоков небольшого размера. Для этой цели лучше было бы использовать блоки размером в 1 байт, но диски по крайней мере округляют размеры блоков до размеров секторов (512 байт или 4 Кбайт).

² Цель — максимизировать пропускную способность за счет уменьшения количества операций ввода/вывода и использования блоков большого размера (это позволяет уменьшить время, расходуемое на инициализацию ввода/вывода). Теоретически чем больше размер блока, тем лучше, но файловая система может иметь свою «золотую середину», обусловленную особенностями механизма распределения памяти в ядре, размерами страниц памяти и другими деталями. Например, ядро Solaris показывает наивысшую производительность при работе с блоками ввода/вывода 128 Кбайт, так как это самый большой размер блоков в кэше ядра (блоки большего размера размещаются в специальной области для блоков очень большого размера, которая обслуживается с меньшей производительностью).

- когда рабочий размер намного меньше объема основной памяти, данные могут полностью уместиться в кэше файловой системы, и в этом случае проводится бенчмаркинг производительности ПО файловой системы;
- когда рабочий размер намного больше объема основной памяти, влияние кэша файловой системы минимально, и в этом случае проводится бенчмаркинг дискового ввода/вывода.
- **Количество потоков выполнения.** Предполагается небольшой размер рабочего набора:
 - с одним потоком выполнения проверяется производительность файловой системы с учетом текущей тактовой частоты процессора;
 - с несколькими потоками, когда их количество достаточно велико для насыщения всех процессоров, проверяется максимальная производительность системы, файловой системы и процессоров.

Их можно быстро комбинировать и масштабировать, чтобы сформировать большую матрицу тестов. Также можно использовать методы статистического анализа, чтобы сократить требуемый набор для тестирования.

Создание бенчмарков, ориентированных на максимальную скорость, называется «солнечным» тестированием производительности. Чтобы не упустить из виду проблемы, также можно выполнить «пасмурное» и «дождливое» тестирование — они предусматривают тестирование в неидеальных ситуациях, включая конфликты, возмущения и переменчивость рабочей нагрузки.

12.2.2. Моделирование

Многие бенчмарки имитируют рабочие нагрузки, создаваемые клиентскими приложениями. Иногда их называют *микробенчмарками производительности*. Они могут создаваться на основе характеристик рабочей нагрузки в промышленной среде (см. главу 2 «Методологии»). Например, вы можете выяснить, что рабочая нагрузка на NFS в промышленной среде складывается из следующих типов операций: чтение — 40 %; запись — 7 %; чтение атрибутов файлов — 19 %; чтение содержимого каталогов — 1 %; и т. д. Аналогично можно измерить и симитировать другие характеристики.

Моделирование позволяет воспроизвести рабочую нагрузку, достаточно похожую на создаваемую клиентами в реальности, чтобы из этого можно было извлечь пользу. Моделирование может учитывать множество факторов, на изучение которых с помощью микробенчмаркинга потребуется много времени. Кроме того, моделирование может включать эффекты сложных взаимодействий в системе, которые могут быть полностью упущены из виду при микробенчмаркинге.

Примеры моделирования — бенчмарки Whetstone и Dhrystone для оценки производительности процессоров, представленные в главе 6 «Процессоры». Whetstone был разработан в 1972 году для моделирования научных вычислительных нагрузок того времени. Dhrystone, созданный в 1984 году, тоже моделирует рабочие нагрузки того времени, связанные с целочисленными вычислениями.

Многие компании моделируют HTTP-нагрузку, создаваемую клиентами, с помощью собственного или стороннего ПО (в числе примеров можно назвать wrk [Glozer 19], siege [Fulmer 12] и hey [Dogan 20]). Их можно использовать для оценки влияния изменений в программном или аппаратном обеспечении, а также для моделирования пиковой нагрузки (например, наплыва покупателей в периоды распродаж на маркетплейсе), чтобы выявить узкие места, проанализировать их и устранить.

Моделирование рабочей нагрузки может проводиться *без учета сохранения состояния*, когда каждый следующий запрос к серверу не связан с предыдущим. Например, описанную выше рабочую нагрузку на сервер NFS можно смоделировать путем запроса серии операций, когда тип каждой операции выбирается случайным образом на основе измеренной вероятности.

Моделирование также может учитывать *сохранение состояния*, когда каждый следующий запрос зависит от состояния клиента или, по меньшей мере, от предыдущего запроса. Вы можете обнаружить, что операции чтения и записи NFS, как правило, поступают группами и вероятность записи после записи намного выше, чем записи после чтения. Такую рабочую нагрузку лучше всего моделировать с помощью *модели Маркова*, представив запросы как состояния и измерив вероятность переходов между состояниями [Jain 91].

Проблема заключается в том, что моделирование может не учитывать дисперсию, как описано в разделе 12.1.3 «Проблемы бенчмаркинга». Закономерности работы клиентов тоже могут меняться со временем, что требует обновления и корректировки моделей, чтобы они оставались актуальными. Но такая корректировка может вызвать сопротивление, если уже есть опубликованные результаты, полученные с использованием более старой версии бенчмарка, которые нельзя использовать для сравнения с новой версией.

12.2.3. Воспроизведение

Третий тип бенчмаркинга основан на воспроизведении журнала трассировки в целевой системе для оценки ее производительности на реальных зафиксированных клиентских операциях. Звучит захватывающе — казалось бы, такой способ ничуть не хуже, чем тестирование в промышленной среде, не так ли? Но не все так гладко: когда характеристики сервера и величины задержек меняются, зафиксированная рабочая нагрузка вряд ли естественным образом отреагирует на эти различия, и в результате воспроизведение может оказаться не лучше моделирования. Если возлагать слишком большие надежды на этот метод, можно сильно разочароваться.

Рассмотрим гипотетическую ситуацию: заказчик рассматривает возможность модернизации инфраструктуры хранения. Он выполнил трассировку текущей промышленной нагрузки и воспроизвел ее на новом оборудовании. К сожалению, производительность оказалась хуже, что может привести к снижению продаж. Проблема: трассировка/воспроизведение выполнялись на уровне дискового ввода/вывода. В старой системе использовались диски со скоростью вращения 10 000 об/мин, а в новой — более медленные диски со скоростью вращения 7200 об/мин. Но в новой

системе объем кэша файловой системы в 16 раз больше и процессоры намного быстрее. Фактическая производительность должна вырасти, потому что данные извлекались бы в основном из кэша, чего не учитывает воспроизведение событий обращения к дискам.

Это пример «тестирования не того, что надо», но могут быть и другие тонкие временные эффекты, способные ухудшить ситуацию даже при правильно проведенных трассировке/воспроизведении. Как и во всех бенчмарках, очень важно анализировать и понимать происходящее.

12.2.4. Отраслевые стандарты

Стандартные отраслевые бенчмарки доступны в независимых организациях, которые стремятся добиться максимальной согласованности и актуальности бенчмарков. Обычно они имеют вид набора из различных микробенчмарков и симуляций рабочих нагрузок, которые четко определены, задокументированы и должны выполняться в соответствии с конкретными правилами, чтобы результаты были такими, как задумано. Вендоры могут участвовать в таких организациях (обычно за плату) и получать ПО для выполнения бенчмарков. Для публикации полученных результатов обычно требуется полное раскрытие настроек среды, которые можно проверить.

Эти бенчмарки помогают клиентам экономить массу времени благодаря возможному наличию результатов бенчмарка разных продуктов и разных вендоров. В таком случае вам остается лишь найти бенчмарк, наиболее точно соответствующий вашей будущей или текущей промышленной нагрузке. Соответствие текущей рабочей нагрузке можно определить, выяснив характеристики этой нагрузки.

Потребность в стандартных отраслевых бенчмарках была четко обозначена в статье «A Measure of Transaction Processing Power», написанной Джимом Греем (Jim Gray) и другими в 1985 году [Anon 85]. В ней обосновывается необходимость измерения соотношения цена/производительность и подробно описываются три бенчмарка, которые могут использовать производители: Sort, Scan и DebitCredit. В статье была предложена стандартная метрика — количество транзакций в секунду (transactions per second, TPS) на основе DebitCredit, — которую можно было бы использовать так же, как метрику расхода топлива в литрах на сто километров для автомобилей. Позже наработки Джима Грея способствовали созданию TPC [DeWitt 08].

Помимо TPS были предложены другие метрики для использования в качестве стандартных:

- **MIPS:** миллионов инструкций в секунду (millions of instructions per second). Это *определенно* показатель производительности, но его величина сильно зависит от типов инструкций, которые могут быть несопоставимыми для процессоров с разными архитектурами.
- **FLOPS:** количество операций с плавающей точкой в секунду (floating-point operations per second). Это метрика, подобная MIPS, но для рабочих нагрузок, в которых интенсивно используются вычисления с плавающей точкой.

Отраслевые бенчмарки обычно измеряют специальную метрику, которую можно сравнивать только с самой собой.

TPC

Совет по производительности обработки транзакций (Transaction Processing Performance Council, TPC) создает и администрирует отраслевые бенчмарки, уделяя особое внимание производительности баз данных, в том числе:

- **TPC-C:** моделирование полной вычислительной среды, в которой группа пользователей выполняет транзакции с базой данных.
- **TPC-DS:** моделирование системы поддержки принятия решений, включая запросы и обслуживание данных.
- **TPC-E:** рабочая нагрузка обработки транзакций в реальном времени (online transaction processing, OLTP), моделирующая базу данных брокерской компании с клиентами, которые генерируют транзакции, связанные с торговыми сделками, запросами по счетам и исследованиями рынка.
- **TPC-H:** бенчмарк для системы поддержки принятия решений, моделирующий специальные запросы и одновременное изменение данных.
- **TPC-VMS:** единая система виртуальных измерений TPC (TPC virtual measurement single system) позволяет собирать другие бенчмарки для виртуализированных баз данных.
- **TPCx-HS:** бенчмарк для оценки производительности обработки больших данных с использованием Hadoop.
- **TPCx-V:** тестирует рабочие нагрузки баз данных на виртуальных машинах.

Результаты TPC публикуются в интернете [TPC 19b] и включают соотношение цена/производительность.

SPEC

Корпорация по стандартной оценке производительности (Standard Performance Evaluation Corporation, SPEC) разрабатывает и публикует стандартизированный набор отраслевых бенчмарков, в том числе:

- **SPEC Cloud IaaS 2018:** для тестирования выделения вычислительных и сетевых ресурсов и ресурсов хранения путем создания нескольких рабочих нагрузок на нескольких экземплярах.
- **SPEC CPU 2017:** оценивает производительность рабочих нагрузок с интенсивными вычислениями; сообщает скорость вычислений с целыми числами и с числами с плавающей точкой, а также меру энергопотребления.
- **SPECjEnterprise 2018 Web Profile:** оценивает производительность всей системы для серверов приложений, баз данных и вспомогательной инфраструктуры Java Enterprise Edition (Java EE) Web Profile версии 7 или выше.

- **SPECsfs2014**: моделирует рабочие нагрузки доступа к файлам для серверов NFS, серверов с общей межсетевой файловой системой (common internet file system, CIFS) и с другими подобными файловыми системами.
- **SPECvirt_sc2013**: предназначен для виртуализированных сред. Оценивает сквозную производительность виртуализированного оборудования, платформы, гостевой ОС и прикладного ПО.

Результаты SPEC публикуются в интернете [SPEC 20] и включают подробную информацию о настройках системы и список компонентов, но их стоимость публикуется редко.

12.3. МЕТОДОЛОГИЯ

В этом разделе представлены методологии и примеры, описывающие приемы микробенчмаркинга, моделирования и воспроизведения. Краткий перечень методологий приводится в табл. 12.2.

Таблица 12.2. Методологии бенчмаркинга

Раздел	Методология	Тип
12.3.1	Пассивный бенчмаркинг	Анализ результатов экспериментов
12.3.2	Активный бенчмаркинг	Анализ наблюдения
12.3.3	Профилирование процессора	Анализ наблюдения
12.3.4	Метод USE	Анализ наблюдения
12.3.5	Определение характеристик рабочей нагрузки	Анализ наблюдения
12.3.6	Нестандартный бенчмаркинг	Разработка ПО
12.3.7	Пиковая нагрузка	Анализ результатов экспериментов
12.3.8	Проверка правильности	Анализ наблюдения
12.3.9	Статистический анализ	Статистический анализ

12.3.1. Пассивный бенчмаркинг

Эта стратегия бенчмаркинга относится к категории «запустил и забыл», когда бенчмарк запускается и игнорируется до его завершения. Основная задача таких бенчмарков — сбор данных, характеризующих производительность. Именно так обычно выполняются бенчмарки, но при сравнении с активным бенчмаркингом этот подход описывается как антиметодология.

Вот пример списка шагов, которые может включать пассивный бенчмаркинг:

1. Выбор инструмента бенчмаркинга.
2. Запуск со множеством параметров.

3. Создание сравнительных таблиц с результатами.
4. Передача таблиц руководству.

Проблемы этого подхода обсуждались выше. В общем случае результаты могут:

- быть недействительными из-за ошибок в ПО, реализующем бенчмарк;
- ограничиваться ПО бенчмарка (например, из-за однопоточной природы);
- ограничиваться компонентом, не связанным с целью бенчмарка (например, перегруженной сетью);
- ограничиваться конфигурацией (отключены параметры, обеспечивающие высокую производительность);
- иметь отклонения (и не воспроизводиться);
- отражать тестирование не того, что нужно.

Пассивный бенчмаркинг легко выполнить, но он подвержен ошибкам. Когда такой бенчмарк выполняется поставщиком, то может давать ошибочно заниженные результаты, которые напрасно расходуют инженерные ресурсы или приводят к потере продаж. В случае, когда это делает клиент, это может способствовать неправильному выбору продукта, который впоследствии будет сказываться на работе компании.

12.3.2. Активный бенчмаркинг

При активном бенчмаркинге производительность анализируется во время выполнения бенчмарка, а не после, для чего применяются инструменты наблюдения [Gregg 14d]. В этом случае вы можете убедиться, что бенчмарк проверяет именно то, что должен проверять, и что вы понимаете его суть. Активный бенчмаркинг также помогает определить истинные ограничения тестируемой системы или самого бенчмарка. Иногда очень полезно включить конкретную информацию о встреченных ограничениях при публикации результатов бенчмарка.

Кроме того, активный бенчмаркинг способствует развитию ваших навыков владения инструментами наблюдения за производительностью. Теоретически вы исследуете *известную нагрузку* и можете увидеть, как она проявляется в данных, возвращаемых этими инструментами. В идеале бенчмарк можно настроить и оставить работающим постоянно, чтобы получить возможность анализировать производительность в течение нескольких часов или даже дней.

Пример анализа

В качестве примера рассмотрим применение инструмента микробенчмаркинга `bonnie++`. На странице справочного руководства `man` он описан как (выделено мной):

```
NAME
    bonnie++ - это программа для тестирования производительности жесткого диска1.
```

¹ Это перевод текста из справочного руководства. — *Примеч. пер.*

А на домашней странице проекта приводится такое описание [Coker 01]:

Bonnie++ — это набор простых бенчмарков для оценки производительности жесткого диска и файловой системы.

Вот пример запуска bonnie++ в Ubuntu Linux:

```
# bonnie++
[...]
```

Version 1.97		-----Sequential Output-----				--Sequential Input--				--Random--			
Concurrency	1	-Per Chr-		--Block--		-Rewrite-		-Per Chr-		--Block--		--Seeks--	
Machine	Size	K/sec	%CP	K/sec	%CP	K/sec	%CP	K/sec	%CP	K/sec	%CP	/sec	%CP
ip-10-1-239-21	4G	739	99	549247	46	308024	37	1845	99	1156838	38	+++++	+++
Latency		18699us		983ms		280ms		11065us		4505us		7762us	

```
[...]
```

Судя по этому выводу, первый тест — «Sequential Output» (последовательный вывод) и «Per Chr» (посимвольно) — достиг скорости 739 Кбайт/с.

Проверка правильности: если это действительно посимвольный ввод/вывод, то полученный результат означает, что система выполняет до 739 000 операций ввода/вывода в секунду. Бенчмарк описывается как оценивающий производительность жесткого диска, но я сомневаюсь, что эта система способна обеспечить такую частоту операций ввода/вывода.

Во время выполнения первого теста я использовал `iostat(1)`, чтобы проверить частоту следования дисковых операций:

```
$ iostat -sxz 1
[...]
```

avg-cpu:	%user	%nice	%system	%iowait	%steal	%idle
	11.44	0.00	38.81	0.00	0.00	49.75

Device	tps	kB/s	rqm/s	await	aqu-sz	areq-sz	%util
[...]							

Никаких операций дискового ввода/вывода не наблюдалось.

Теперь используем `bpfftrace`, чтобы подсчитать события блочного ввода/вывода (см. главу 9 «Диски», раздел 9.6.11 «`bpfftrace`»):

```
# bpfftrace -e 'tracepoint:block:* { @[probe] = count(); }'
Attaching 18 probes...
^C
@[tracepoint:block:block_dirty_buffer]: 808225
@[tracepoint:block:block_touch_buffer]: 1025678
```

Этот однострочный сценарий тоже сообщает, что блочный ввод/вывод не запускался (событие `block:block_rq_issue`) и не завершался (событие `block:block_rq_complete`). Но буферы изменялись. Используем `cachestat(8)` (глава 8 «Файловые системы», раздел 8.6.12 «`cachestat`»), чтобы посмотреть состояние кэша файловой системы:

поэтому результаты бенчмарка при использовании `strace(1)` не следует принимать во внимание.

```
$ strace bonnie++ -b
[...]  
write(3, "6", 1)           = 1  
write(3, "7", 1)           = 1  
write(3, "8", 1)           = 1  
write(3, "9", 1)           = 1  
write(3, ":", 1)           = 1  
[...]
```

Как показывает вывод, `bonnie++` не вызывает `fsync(2)` после каждой операции записи. Этот инструмент поддерживает также параметр `-D` для прямого ввода/вывода, но не работает в моей системе. Фактически у меня не получилось провести тест посимвольной записи на диск.

Кто-то может возразить, что `bonnie++` действительно работает и выполняет тесты «Sequential Output» (последовательный вывод) и «Per Chr» (посимвольно), названия которых не говорят о том, что тестируется дисковый ввод/вывод. Но как мне кажется, для бенчмарка, в описании которого утверждается, что он тестирует производительность «жесткого диска», это по меньшей мере странно.

`Bonnie++` — неплохой инструмент для бенчмаркинга, во многих случаях он хорошо служит людям. Я выбрал его (а также самый подозрительный из его тестов) для этого примера, потому что он хорошо известен, я изучал его раньше, и подобные выводы не редкость. Но это всего лишь один пример.

Более старые версии `bonnie++` имели еще одну проблему с этим тестом: он позволял библиотеке `libc` буферизовать записываемые данные до передачи их в файловую систему, поэтому размер записи в VFS составлял 4 Кбайт или больше, в зависимости от версии `libc` и ОС¹. Это делало невозможным сравнение результатов `bonnie++`, полученных в разных ОС, в которых в `libc` использовались буферы разного размера. Эта проблема исправлена в последних версиях `bonnie++`, но возникла другая: результаты новой версии `bonnie++` нельзя сравнивать с результатами старой версии.

Подробнее об анализе производительности с помощью `bonnie++` идет речь в статье Роха Бурбонне (Roch Bourbonnais) «Decoding Bonnie++» [Bourbonnais 08].

12.3.3. Профилирование процессора

Профилирование процессора для бенчмаркинга его производительности стоит выделить особо, потому что такой подход может привести к некоторым быстрым открытиям. Профилирование часто выполняется в рамках активного бенчмаркинга.

¹ Если вам интересно, то размер буфера в `libc` можно настроить с помощью `setbuffer(3)`. Этот буфер использовался из-за того, что запись в `bonnie++` выполнялась вызовом функции `putc(3)` из библиотеки `libc`.

Цель состоит в том, чтобы быстро проверить, что делает ПО, и увидеть, не обнаружится ли что-нибудь интересное. Профилирование также помогает сузить поле поиска до наиболее важных программных компонентов из числа участвующих в бенчмаркинге.

Профилировать можно стеки как на уровне пользователя, так и на уровне ядра. Профилирование процессора на уровне пользователя было описано в главе 5 «Приложения». Затем обе разновидности профилирования были рассмотрены в главе 6 «Процессоры» с примерами в разделе 6.6 «Инструменты наблюдения», включая флейм-графики.

Пример

В новой системе был проведен микробенчмаркинг диска и получены некоторые неутешительные результаты: пропускная способность диска оказалась хуже, чем в старой системе. Меня попросили выяснить причину, высказав предположение, что проблема либо в дисках, либо в контроллере дисков, и их, возможно, следует заменить.

Я начал с метода USE (глава 2 «Методологии») и обнаружил, что диски работают с довольно низкой нагрузкой, но возникла несколько увеличенная нагрузка на процессор на уровне ядра (системное время).

Мало кто ожидает, что процессоры могут быть интересной целью для анализа при бенчмаркинге дисков. Увидев повышенную нагрузку на процессор на уровне ядра, я подумал, что стоит проверить, не обнаружится ли что-нибудь интересное, хотя, честно говоря, не ожидал увидеть ничего особенного. Я провел профилирование и построил флейм-график (рис. 12.2).

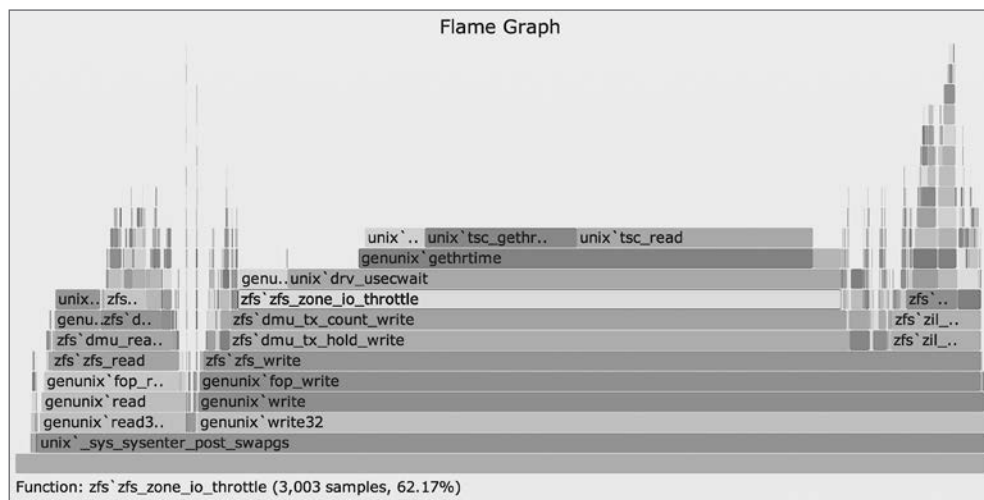


Рис. 12.2. Флейм-график профилирования процессора на уровне ядра

Обзор фреймов стека показал, что 62,17 % образцов включали функцию `zfs_zone_io_throttle()`. Мне не нужно было читать код этой функции, так как само ее имя было ключом к разгадке: средства управления ресурсами ввода/вывода в ZFS искусственно ограничивали бенчмарк! Это регулирование было включено по умолчанию в новой системе (и его не было в старой системе), и оно не учитывалось при бенчмаркинге.

12.3.4. Метод USE

Метод USE был представлен в главе 2 «Методологии» и описан в главах, посвященных конкретным ресурсам. Применение метода USE во время бенчмаркинга гарантирует обнаружение предела: либо вы обнаружите компонент (аппаратный или программный), потребление которого достигнет 100 %, либо вам не удастся довести систему до предела.

12.3.5. Определение характеристик рабочей нагрузки

Методология определения характеристик рабочей нагрузки была представлена в главе 2 «Методологии» и обсуждалась в следующих за ней главах. Эта методология позволяет определить, насколько хорошо бенчмарк соотносится с текущей промышленной средой, путем выяснения характеристик промышленной рабочей нагрузки для сравнения.

12.3.6. Собственный бенчмаркинг

Для простых бенчмарков иногда удобнее написать свою программу. Постарайтесь сделать такую программу как можно короче, чтобы избежать сложностей, затрудняющих анализ.

Лучший выбор для создания микробенчмарков — язык C, он позволяет достаточно точно выразить желаемое. При этом следует оценить, как оптимизации компилятора повлияют на ваш код: компилятор может исключить вызовы простых процедур бенчмарка, если посчитает, что их результаты не используются и они не нужны для расчетов. Выполняя бенчмарк, всегда проверяйте его работу с помощью других инструментов. Возможно, стоит дизассемблировать скомпилированный двоичный файл, чтобы увидеть, какие операции в действительности будут выполнены.

Программы на языках, включающих виртуальные машины, асинхронную сборку мусора и динамическую компиляцию, могут быть сложнее в отладке и менее точными. Возможно, вам все равно придется использовать эти языки, если при проведении макробенчмаркинга потребуется смоделировать работу клиентского ПО, написанного на таких языках.

Разработка собственных бенчмарков также поможет раскрыть тонкие особенности цели, которые могут оказаться полезными в дальнейшем. Например, при разработке

бенчмарка для базы данных можно обнаружить, что API поддерживает некоторые возможности увеличения производительности, которые сейчас не используются в промышленной среде, разработанной до появления этих возможностей.

Ваше ПО может просто генерировать нагрузку (*генератор нагрузки*) и перекладывать задачу измерения на другие инструменты. Этот подход часто используется для создания *пиковых нагрузок*.

12.3.7. Пиковая нагрузка

Это простой метод определения максимальной пропускной способности системы. Он предполагает последовательное увеличение нагрузки до предела и измерение получившейся пропускной способности. Результаты могут быть отображены в виде графика, показывающего профиль масштабируемости. Этот профиль можно исследовать визуально или с помощью моделей масштабируемости (см. главу 2 «Методологии»).

В качестве примера на рис. 12.3 показано масштабирование файловой системы и сервера с помощью потоков выполнения. Каждый поток производит произвольное чтение из кэшированного файла блоками размером 8 Кбайт, и они добавляются по одному.

Эта система достигла пика на уровне почти в полмиллиона операций чтения в секунду. Результаты были проверены с использованием статистик VFS, которые подтвердили, что размер блока ввода/вывода составлял 8 Кбайт, а на пике данные передавались со скоростью более 3,5 Гбайт/с.

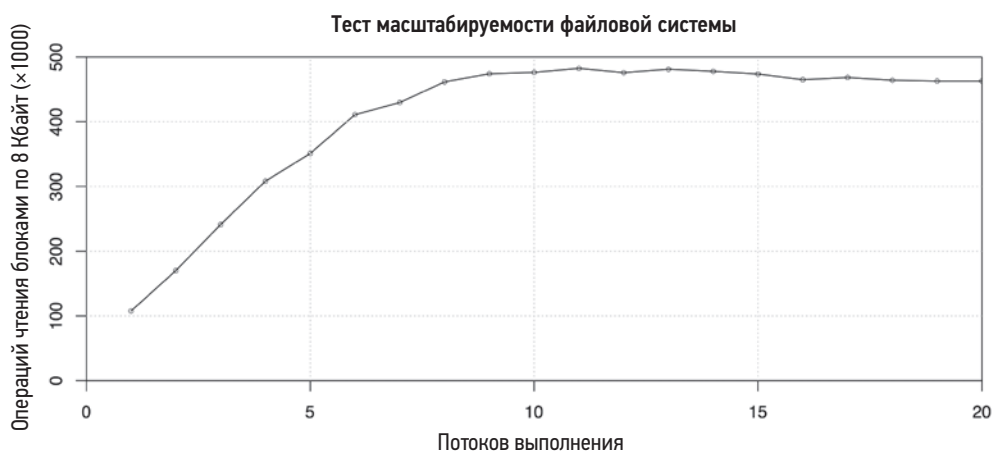


Рис. 12.3. Создание пиковой нагрузки на файловую систему

Генератор нагрузки для этого теста был написан на Perl, и он достаточно короткий, поэтому приведу его полностью:


```
#!/usr/bin/perl -w
#
# randread.pl - выполняет произвольное чтение из указанного файла.

use strict;

my $IOSIZE = 8192;                # размер блока ввода/вывода в байтах
my $QUANTA = $IOSIZE;            # шаг перемещения в файле

die "USAGE: randread.pl filename\n" if @ARGV != 1 or not -e $ARGV[0];

my $file = $ARGV[0];
my $span = -s $file;              # размер файла для выбора произвольного
                                  # блока, в байтах

my $junk;

open FILE, "$file" or die "ERROR: reading $file: $!\n";

while (1) {
    seek(FILE, int(rand($span / $QUANTA)) * $QUANTA, 0);
    sysread(FILE, $junk, $IOSIZE);
}

close FILE;
```

Для предотвращения буферизации используется функция `sysread()`, которая напрямую обращается к системному вызову `read(2)`.

Этот генератор был написан для микробенчмаркинга сервера NFS и запускался одновременно на нескольких клиентах, каждый из которых выполнял произвольное чтение файла из смонтированного тома NFS. Результаты микробенчмаркинга (операций чтения в секунду) измерялись на сервере NFS с использованием `nfsstat(8)` и других инструментов.

Количество используемых файлов и их общий размер контролировались (из них складывается *размер рабочего набора*) так, чтобы некоторые тесты могли получать данные из кэша на сервере, а другие — с диска. (См. подраздел «Пример дизайна бенчмарка» в разделе 12.2.1 «Микробенчмаркинг».)

Количество экземпляров, играющих роль клиентов, постепенно увеличивалось, чтобы довести нагрузку до предела. Как и в предыдущем примере, был построен график профиля масштабируемости с кривыми потребления ресурсов (метод USE), показавшими, какие ресурсы были исчерпаны. Здесь это оказался вычислительный ресурс (процессор) на сервере, что побудило провести еще одно исследование с целью дальнейшего повышения производительности.

Я использовал эту программу и этот подход, чтобы найти пределы для Sun ZFS Storage Appliance [Gregg 09b]. Впоследствии эти пределы были представлены как официальные результаты, которые, насколько нам известно, установили мировые рекорды. Кроме того, у меня был аналогичный набор ПО, написанного на C, который я обычно использую, но в этом случае он не пригодился — было множество клиентских процессоров. Хотя переход на C и уменьшил нагрузку на них, это

никак не повлияло на результат и было получено то же самое узкое место. Были опробованы и другие, более сложные бенчмарки и другие языки, но они также не смогли изменить результаты.

Следуя этому подходу, измерьте задержку, пропускную способность и обязательно распределение задержек. Когда система приближается к своему пределу, ожидание операций в очереди может стать значительным, что приведет к увеличению задержек. Если приложить слишком большую нагрузку, задержка будет настолько большой, что результат можно считать недействительным. Спросите себя: устроит ли заказчика получившаяся задержка?

Например: вы используете большой массив клиентов, чтобы довести нагрузку на целевую систему до уровня 990 000 операций ввода/вывода в секунду, соответствующего средней задержке ввода/вывода 5 мс. В действительности хотелось бы превысить уровень 1 миллион операций ввода/вывода в секунду, но система уже достигла насыщения. Добавляя все больше и больше клиентов, вам удастся преодолеть планку в 1 миллион операций ввода/вывода в секунду, но теперь все операции много времени проводят в очереди, и средняя задержка превышает 50 мс, что неприемлемо. Какой результат вы опубликуете для рекламы? (Ответ: 990 000 операций ввода/вывода в секунду.)

12.3.8. Проверка правильности

Цель этой методологии — проверка результатов бенчмарка путем выяснения, имеет ли смысл та или иная характеристика. Также проверяется, не потребовал ли результат превышения известных пределов какого-либо компонента, таких как пропускная способность сети, пропускная способность контроллера, пропускная способность внутренних соединений или максимального количества дисковых операций ввода/вывода в секунду. Если какой-либо предел оказался превышен, стоит изучить его более подробно. В большинстве случаев эта методология обнаруживает, что результат бенчмарка ложный.

Рассмотрим пример: по результатам бенчмаркинга сервера NFS операциями чтения блоками по 8 Кбайт было сообщено, что он способен выдержать нагрузку до 50 000 операций ввода/вывода в секунду. Сервер подключен к сети через единственный порт Ethernet со скоростью передачи 1 Гбит/с. Необходимая пропускная способность сети составляет $50\,000 \text{ операций в секунду} \times 8 \text{ Кбайт} = 400\,000 \text{ Кбайт/с}$ плюс заголовки протокола. Это более 3,2 Гбит/с, что значительно превышает известный предел в 1 Гбит/с. Здесь явно что-то не так.

Такие результаты обычно означают, что фактически бенчмарк проверил работу механизма *кэширования на стороне клиента* и не вся рабочая нагрузка достигла сервера NFS.

Я использовал подобные расчеты для выявления множества ошибочных результатов бенчмарков, которые включали следующие значения пропускной способности через один интерфейс 1 Гбит/с [Gregg 09c]:

- 120 Мбайт/с (0,96 Гбит/с);
- 200 Мбайт/с (1,6 Гбит/с);
- 350 Мбайт/с (2,8 Гбит/с);
- 800 Мбайт/с (6,4 Гбит/с);
- 1,15 Гбайт/с (9,2 Гбит/с).

Все это — пропускная способность в одном направлении. Результат 120 Мбайт/с кажется вполне правдоподобным, но на практике интерфейс 1 Гбит/с способен пропускать только где-то около 119 Мбайт/с. Результат 200 Мбайт/с возможен только при суммировании трафика в обоих направлениях, но в этом случае утверждалось, что измерялся однонаправленный трафик. Результаты 350 Мбайт/с и выше явно фальшивые.

Когда вам предложат результат бенчмарка для проверки, поищите любые простые суммы, которые сможете вычислить с указанными числами, чтобы определить такие пределы.

При наличии доступа к системе, вероятно, вы сможете получить дополнительные результаты, проведя новые наблюдения или эксперименты. Для этого используйте научный метод: ответьте на вопрос, верен ли результат бенчмарка. Исходя из этого можно делать гипотезы и прогнозы, а затем проверить их для ответа на вопрос.

12.3.9. Статистический анализ

Для изучения результатов бенчмарка можно использовать статистический анализ. Он состоит из трех этапов.

1. **Выбор** инструмента бенчмарка, настроек и метрик производительности системы для сбора.
2. **Выполнение** бенчмарка, сбор большого набора данных с результатами и метриками.
3. **Интерпретация** данных со статистическим анализом, составление отчета.

В отличие от активного бенчмаркинга, основное внимание в котором уделяется анализу системы во время выполнения бенчмарка, статистический анализ сосредоточен на анализе результатов. Он также отличается от пассивного бенчмаркинга, при котором анализ вообще не проводится.

Этот подход используется в средах, где доступ к крупномасштабной системе может ограничиваться по времени и стоить очень дорого. Например, может быть доступна только одна система в «максимальной конфигурации», и многим командам одновременно потребовалось запустить свои тесты, в том числе:

- **Отделу продаж:** для моделирования клиентской нагрузки в процессе проверки концепции, чтобы определить, что может дать система в максимальной конфигурации.

- **Отделу маркетинга:** для получения наилучших результатов маркетинговой кампании.
- **Отделу поддержки:** для исследования патологий, возникающих только в системе с максимальной конфигурацией при серьезной нагрузке.
- **Отделу разработки:** для проверки производительности новых функций и измененного кода.
- **Отделу QA:** для проведения нерегрессионного тестирования и сертификации.

Каждой команде выделяется ограниченное время для запуска бенчмарка в системе и намного больше времени для анализа полученных результатов.

Поскольку сбор метрик обходится довольно дорого, важно приложить все силы, чтобы обеспечить их надежность и достоверность и избежать повторного тестирования. Кроме проверки того, как они генерируются технически, также можно собрать дополнительные статистические характеристики, которые помогут ускорить обнаружение проблемы, в том числе статистики изменчивости, полные распределения, пределы ошибок и др. (см. главу 2 «Методологии», раздел 2.8 «Статистика»). При бенчмаркинге изменений в коде или проведении нерегрессионного тестирования важно понимать пределы вариаций и ошибок, чтобы осмыслить пары результатов.

Также желательно собрать как можно больше данных о производительности действующей системы (без ущерба для результатов из-за оверхеда на сбор), чтобы впоследствии можно было провести ретроспективный анализ этих данных. Сбор данных производят с помощью `sar(1)`, продуктов для мониторинга и собственных инструментов, которые собирают все доступные статистики.

Например, в Linux с помощью собственного сценария командной оболочки можно копировать содержимое файлов-счетчиков в `/proc` до и после запуска¹. При необходимости можно включить копирование всего, что доступно. Такой сценарий также может выполняться во время бенчмаркинга через определенные промежутки времени при условии, что он имеет приемлемый оверхед. Для сбора данных можно использовать и другие статистические инструменты.

Статистический анализ результатов и метрик может включать *анализ масштабируемости* и *теорию очередей* для моделирования системы как многоканальной системы обслуживания. Эти темы были представлены в главе 2 «Методологии». Также они рассмотрены в книгах [Jain 91], [Gunther 97], [Gunther 07].

12.3.10. Чек-лист бенчмаркинга

Вдохновившись чек-листом мантр производительности (глава 2 «Методологии», раздел 2.5.20 «Мантры производительности»), я создал чек-лист вопросов, ответы

¹ Не используйте утилиту `tar(1)` для этой цели — ее сбивают с толку файлы в `/proc`, которые по мнению `stat(2)` имеют нулевой размер, и она не читает их содержимое.

на которые можно найти в результатах бенчмаркинга, чтобы проверить их точность [Gregg 18d]:

- А почему не в два раза?
- Нарушены ли какие-то ограничения?
- Были ли ошибки?
- Можно ли воспроизвести результаты?
- Имеют ли результаты значение?
- Бенчмаркинг вообще выполнялся?

Точнее:

- **А почему не в два раза?** Почему скорость работы получилась именно такой, а не в два раза больше? На самом деле этот вопрос касается выявления ограничивающих компонентов. Ответ на него может решить многие проблемы бенчмаркинга, если обнаружится, что ограничивающий компонент — не предполагаемая цель тестирования.
- **Нарушены ли какие-то ограничения?** Это проверка правильности (раздел 12.3.8 «Проверка правильности»).
- **Были ли ошибки?** Столкнувшись с ошибкой, код работает иначе, и высокая частота ошибок может исказить результаты бенчмарка.
- **Можно ли воспроизвести результаты?** Насколько стабильны результаты бенчмарка?
- **Имеют ли результаты значение?** Рабочая нагрузка, которую оценивает конкретный бенчмарк, может не соответствовать вашим потребностям. Некоторые микробенчмарки тестируют отдельные системные вызовы и вызовы библиотечных функций, но ваше приложение может даже не использовать их.
- **Бенчмаркинг вообще выполнялся?** В разделе 12.1.3 «Проблемы бенчмаркинга», в подразделе «Игнорирование ошибок» описан случай, когда брандмауэр заблокировал запросы, посылаемые бенчмарком, поэтому тот фактически оценивал величину тайм-аутов.

В разделе ниже приведен гораздо более длинный список вопросов, который также охватывает сценарии, когда у вас может не быть доступа к целевой системе для самостоятельного анализа бенчмарка.

12.4. ВОПРОСЫ О БЕНЧМАРКИНГЕ

Получив от поставщика результаты бенчмарка, вы можете задать ему вопросы, чтобы лучше понять эти результаты и применить к своей среде, даже в отсутствие доступа к работающему бенчмарку для его анализа. Главная задача — определить, что в действительности измеряется и насколько результат воспроизводим и близок к реальности.

- **В целом:**

- Бенчмарк как-то связан с рабочей нагрузкой в моей промышленной среде?
- Как была настроена тестируемая система?
- Была ли протестирована одна система или результат описывает работу группы систем?
- Сколько стоит тестируемая система?
- Как были настроены клиенты бенчмарка?
- Как долго выполнялось тестирование? Сколько результатов было собрано?
- Результат описывает среднюю или пиковую нагрузку? Какова была средняя нагрузка?
- Известны ли другие параметры распределения (стандартное отклонение, процентиля или полные распределения)?
- Что было ограничивающим фактором бенчмарка?
- Каково соотношение успешных/неудачных операций?
- С какими атрибутами выполнялись операции?
- Использовались ли атрибуты, имитирующие рабочую нагрузку? Какие?
- Имитирует ли бенчмарк переменчивость рабочей нагрузки или оказывал усредненную нагрузку на систему?
- Подтверждаются ли результаты бенчмарка другими инструментами анализа? (Предоставьте скриншоты.)
- Можно ли оценить погрешность по результатам бенчмарка?
- Можно ли воспроизвести результат бенчмарка?

- Для бенчмарков, связанных с **процессором/памятью**:

- Какие процессоры использовались?
- Работали ли процессоры на повышенной частоте? Использовалось ли нестандартное охлаждение (например, водяное)?
- Сколько модулей памяти (например, DIMM) использовалось? Как они крепятся в разъемах?
- Были ли отключены какие-либо процессоры?
- Каким был уровень потребления процессоров в масштабе всей системы? (Ненагруженные системы могут работать быстрее из-за доступности более высоких уровней турбоускорения.)
- Что подразумевается под процессорами — ядра или гиперпоток?
- Какой объем основной памяти был установлен? Какого типа?
- Использовались ли какие-либо нестандартные настройки BIOS?

- Для бенчмарков, связанных с **устройствами хранения**:

- Как были настроены устройства хранения (сколько устройств использовалось, их типы, протокол хранения, конфигурация RAID, размер кэша, обратная или сквозная запись и т. д.)?

- Как были настроены файловые системы (типы, сколько файловых систем использовалось, их настройки, например поддержка журналирования)?
- Размер рабочего набора?
- Какая доля рабочего набора уместается в кэш? Где размещался кэш?
- Ко скольким файлам осуществлялся доступ?
- Для бенчмарков, связанных с **сетью**:
 - Конфигурация сети (сколько интерфейсов использовалось, их типы и настройки)?
 - Топология использовавшейся сети?
 - Какие протоколы использовались? Параметры сокетов?
 - Какие параметры сетевого стека настраивались? Параметры TCP/UDP?

Ответы на многие из этих вопросов при изучении отраслевых бенчмарков можно найти в сопроводительном описании.

12.5. УПРАЖНЕНИЯ

1. Ответьте на следующие вопросы, касающиеся ключевых понятий:
 - Что такое микробенчмаркинг?
 - Что такое размер рабочего набора и как он может повлиять на результаты бенчмарка хранилища?
 - Почему важно оценивать соотношение цена/производительность?
2. Выберите микробенчмарк и выполните следующие задания:
 - Масштабируйте какое-либо из измерений (количество потоков, размер блоков ввода/вывода) и проверьте, как меняется производительность.
 - Постройте график масштабируемости по результатам.
 - Используйте микробенчмарк, чтобы достичь максимальной производительности, и проанализируйте ограничивающий фактор.

12.6. ССЫЛКИ

- [Joy 81] Joy, W., «tcp-ip digest contribution», <http://www.rfc-editor.org/rfc/museum/tcp-ip-digest/tcp-ip-digest.v1n6.1>, 1981.
- [Anon 85] Anon et al., «A Measure of Transaction Processing Power», Datamation, April 1, 1985.
- [Jain 91] Jain, R., «The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling», Wiley, 1991.
- [Gunther 97] Gunther, N., «The Practical Performance Analyst», McGraw-Hill, 1997.

- [Shanley 98] Shanley, K., «History and Overview of the TPC», <http://www.tpc.org/information/about/history.asp>, 1998.
- [Coker 01] Coker, R., «bonnie++», <https://www.coker.com.au/bonnie++>, 2001.
- [Smaalders 06] Smaalders, B., «Performance Anti-Patterns», *ACM Queue* 4, no. 1, February 2006.
- [Gunther 07] Gunther, N., «Guerrilla Capacity Planning», Springer, 2007.
- [Bourbonnais 08] Bourbonnais, R., «Decoding Bonnie++», https://blogs.oracle.com/roch/entry/decoding_bonnie, 2008.
- [DeWitt 08] DeWitt, D., and Levine, C., «Not Just Correct, but Correct and Fast», *SIGMOD Record*, 2008.
- [Traeger 08] Traeger, A., Zadok, E., Joukov, N., and Wright, C., «A Nine Year Study of File System and Storage Benchmarking», *ACM Transactions on Storage*, 2008.
- [Gregg 09b] Gregg, B., «Performance Testing the 7000 series, Part 3 of 3», <http://www.brendangregg.com/blog/2009-05-26/performance-testing-the-7000-series3.html>, 2009.
- [Gregg 09c] Gregg, B., and Straughan, D., «Brendan Gregg at FROSUG, Oct 2009», <http://www.beginningwithi.com/2009/11/11/brendan-gregg-at-frosug-oct-2009>, 2009.
- [Fulmer 12] Fulmer, J., «Siege Home», <https://www.joedog.org/siege-home>, 2012.
- [Gregg 14c] Gregg, B., «The Benchmark Paradox», <http://www.brendangregg.com/blog/2014-05-03/the-benchmark-paradox.html>, 2014.
- [Gregg 14d] Gregg, B., «Active Benchmarking», <http://www.brendangregg.com/activebenchmarking.html>, 2014.
- [Gregg 18d] Gregg, B., «Evaluating the Evaluation: A Benchmarking Checklist», <http://www.brendangregg.com/blog/2018-06-30/benchmarking-checklist.html>, 2018.
- [Glozer 19] Glozer, W., «Modern HTTP Benchmarking Tool», <https://github.com/wg/wrk>, 2019.
- [TPC 19a] «Third Party Pricing Guideline», http://www.tpc.org/information/other/pricing_guidelines.asp, 2019.
- [TPC 19b] «TPC», <http://www.tpc.org>, 2019.
- [Dogan 20] Dogan, J., «HTTP load generator, ApacheBench (ab) replacement, formerly known as rakyll/boom», <https://github.com/rakyll/hey>, последнее обновление 2020.
- [SPEC 20] «Standard Performance Evaluation Corporation», <https://www.spec.org>, по состоянию на 2020.

Глава 13

PERF

`perf(1)` — это официальный профилировщик Linux, который находится в исходном коде ядра Linux в каталоге `tools/perf`¹. Это многофункциональный инструмент для профилирования, трассировки и создания сценариев, а также интерфейс для подсистемы наблюдения ядра `perf_events`, которая известна под названиями Performance Counters for Linux (PCL) и Linux Performance Events (LPE). Подсистема `perf_events` и интерфейс `perf(1)` начинались как поддержка счетчиков мониторинга производительности (performance monitoring counter, PMC), но постепенно развились и теперь поддерживают механизмы трассировки на основе событий: точки трассировки и зонды `kprobes`, `uprobes` и `USDT`.

Эта глава наряду с главами 14 «Ftrace» и 15 «BPF» необязательна для чтения и предназначена для желающих изучить один или несколько системных трассировщиков подробнее.

По сравнению с другими трассировщиками `perf(1)` особенно хорошо подходит для анализа потребления процессорного времени: профилирования методом выборки трассировок стека, трассировки поведения планировщика процессора и изучения счетчиков PMC для анализа производительности процессора на уровне микроархитектуры, включая анализ выполнения отдельных инструкций и тактов. Но имеющиеся в `perf(1)` возможности позволяют анализировать и другие цели, включая дисковый ввод/вывод и программные функции.

С помощью `perf(1)` можно ответить на такие вопросы:

- Какие пути в коде потребляют больше всего процессорного времени?
- Не приостанавливаются ли процессоры при выполнении операций с памятью?
- По каким причинам потоки оставляют процессор?
- Как выглядит типичный шаблон дискового ввода/вывода?

¹ Профилировщик `perf(1)` необычен тем, что это большая сложная программа, действующая в пространстве пользователя и находящаяся в дереве исходных текстов ядра Linux. Ее мейнтейнер Арнальдо Карвалью де Мело (Arnaldo Carvalho de Melo) описал мне эту ситуацию как «эксперимент». Такое положение вещей было выгодно для `perf(1)` и Linux, так как они разрабатывались синхронно, но некоторым не нравится включение `perf(1)` в исходный код ядра, и он остается единственным сложным программным продуктом пользовательского уровня, когда-либо включенным в исходный код Linux.

В разделах ниже дается введение в `perf(1)`, перечисляются источники событий, а затем обсуждаются подкоманды, которые их используют:

- 13.1: Обзор подкоманд
- 13.2: Примеры однострочных сценариев
- События:
 - 13.3: Обзор событий
 - 13.4: Аппаратные события
 - 13.5: Программные события
 - 13.6: Точки трассировки
 - 13.7: События зондов
- Команды:
 - 13.8: `perf stat`
 - 13.9: `perf record`
 - 13.10: `perf report`
 - 13.11: `perf script`
 - 13.12: `perf trace`
 - 13.13: Другие команды
- 13.14: Документация
- 13.15: Ссылки

В предыдущих главах не раз было показано, как использовать `perf(1)` для анализа конкретных целей. В этой главе основное внимание уделяется самому профилировщику `perf(1)`.

13.1. ОБЗОР ПОДКОМАНД

Разные возможности `perf(1)` доступны посредством подкоманд. Ниже демонстрируется типичный пример, использующий две подкоманды: `record` — для инструментации событий и записи полученных данных в файл и `report` — для обобщения содержимого файла. Эти подкоманды объясняются в разделах 13.9 «`perf record`» и 13.10 «`perf report`».

```
# perf record -F 99 -a -- sleep 30
[ perf record: Woken up 193 times to write data ]
[ perf record: Captured and wrote 48.916 MB perf.data (11880 samples) ]
# perf report --stdio
[...]
```

#	Overhead	Command	Shared	Object	Symbol
#					
#					
	21.10%	swapper		[kernel.vmlinux]	[k] native_safe_halt

```

6.39% mysqld      [kernel.vmlinux]      [k] _raw_spin_unlock_irqrest
4.66% mysqld      mysqld                 [.] _Z8ut_delay
2.64% mysqld      [kernel.vmlinux]      [k] finish_task_switch
[...]
```

В этом примере производится выборка трассировок стека любых программ, выполняющихся на любом процессоре, в течение 30 с с частотой 99 Гц, а затем выводятся функции, наиболее часто встречающиеся в выборке.

В табл. 13.1 перечислены подкоманды, которые поддерживаются последней версией `perf(1)` (начиная с Linux 5.6).

Таблица 13.1. Некоторые подкоманды `perf`

Раздел	Команда	Описание
–	<code>annotate</code>	Читает файл <code>perf.data</code> (созданный подкомандой <code>perf record</code>) и выводит аннотированный код
–	<code>archive</code>	Создает переносимую версию файла <code>perf.data</code> , содержащую отладочную информацию и символы
–	<code>bench</code>	Системные микробенчмарки
–	<code>buildid-cache</code>	Управляет кэшем идентификаторов сборки (используется зондами USDT)
–	<code>cdc</code>	Инструмент анализа кэша процессоров
–	<code>diff</code>	Читает два файла <code>perf.data</code> и отображает различия между профилями
–	<code>evlist</code>	Выводит список имен событий, зафиксированных в файле <code>perf.data</code>
14.12	<code>ftrace</code>	Интерфейс <code>perf(1)</code> к трассировщику <code>Ftrace</code>
–	<code>inject</code>	Фильтр для добавления дополнительной информации в поток событий
–	<code>kmem</code>	Трассировка/оценка свойств памяти (slab) ядра
11.3.3	<code>kvm</code>	Трассировка/оценка гостевых экземпляров <code>kvm</code>
13.3	<code>list</code>	Список типов событий
–	<code>lock</code>	Анализ событий блокировки
–	<code>mem</code>	Доступ к памяти профиля
13.7	<code>probe</code>	Определение новой динамической точки трассировки
13.9	<code>record</code>	Запускает заданную команду и записывает ее профиль в <code>perf.data</code>
13.10	<code>report</code>	Читает файл <code>perf.data</code> (созданный командой <code>perf record</code>) и выводит профиль
6.6.13	<code>sched</code>	Трассировка/оценка свойств планировщика (задержек)
5.5.1	<code>script</code>	Читает файл <code>perf.data</code> (созданный командой <code>perf record</code>) и выводит трассировку
13.8	<code>stat</code>	Запускает заданную команду и собирает статистики из счетчиков производительности

Таблица 13.1 (окончание)

Раздел	Команда	Описание
–	timechart	Визуализирует поведение системы под нагрузкой
–	top	Инструмент профилирования системы, динамически обновляющий вывод на экран
13.12	trace	Оперативный трассировщик (по умолчанию трассирует системные вызовы)

На рис. 13.1 показаны часто используемые подкоманды perf с их источниками данных и типами вывода.

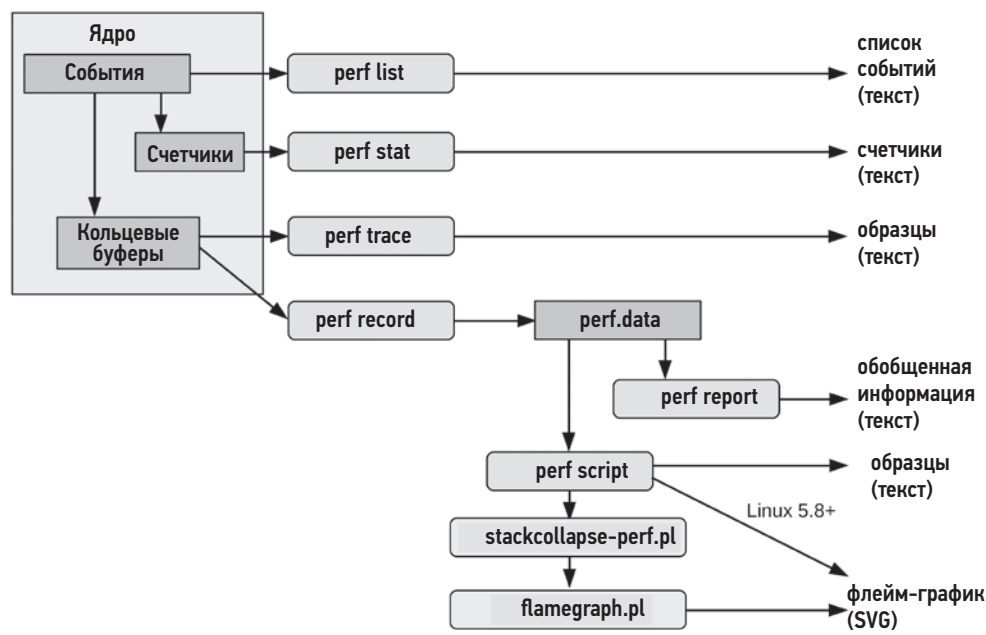


Рис. 13.1. Часто используемые подкоманды perf

Многие из этих и других подкоманд описаны в следующих разделах. Некоторые подкоманды рассматривались в предыдущих главах (см. табл. 13.1).

В будущих версиях perf(1) могут появиться дополнительные возможности: запуск perf без аргументов для получения полного списка подкоманд, доступных в системе.

13.2. ОДНОСТРОЧНЫЕ СЦЕНАРИИ

На примере следующих однострочных сценариев показаны различные возможности perf(1). Это лишь выдержка из большого списка, опубликованного мной

в интернете [Gregg 20h], который оказался эффективным способом объяснить возможности perf(1). Их синтаксис рассматривается в следующих разделах и на страницах справочного руководства man для perf (1).

Обратите внимание, что многие из этих сценариев используют параметр `-a` для указания всех процессоров. Но начиная с Linux 4.11, этот параметр подразумевается по умолчанию и его можно опустить в этой и более поздних версиях ядра.

Список событий

Список всех известных событий:

```
perf list
```

Список точек трассировки sched:

```
perf list 'sched:*
```

Список событий с именами, содержащими подстроку «block»:

```
perf list block
```

Список доступных в данный момент динамических зондов:

```
perf probe -l
```

Подсчет событий

Выводит статистики из счетчиков РМС для указанной команды:

```
perf stat команда
```

Выводит статистики из счетчиков РМС для процесса с указанным PID до нажатия Ctrl-C:

```
perf stat -p PID
```

Выводит статистики из счетчиков РМС для всей системы в целом в течение 5 с:

```
perf stat -a sleep 5
```

Выводит статистики, характеризующие работу кэша последнего уровня (LLC) процессора для указанной команды:

```
perf stat -e LLC-loads,LLC-load-misses,LLC-stores,LLC-prefetches команда
```

Посчитывает непрерывные такты ядра, опираясь на спецификацию РМС (Intel):

```
perf stat -e r003c -a sleep 5
```

Подсчитывает инструкции, простаивавшие в интерфейсе, опираясь на спецификацию РМС (Intel):

```
perf stat -e cpu/event=0x0e,umask=0x01,inv,cmask=0x01/ -a sleep 5
```

Подсчитывает количество обращений к системным вызовам в секунду для всей системы:

```
perf stat -e raw_syscalls:sys_enter -I 1000 -a
```

Подсчитывает количество обращений к системным вызовам по типам для процесса с указанным PID:

```
perf stat -e 'syscalls:sys_enter_*' -p PID
```

Подсчитывает события блочного устройства ввода/вывода для всей системы в течение 10 с:

```
perf stat -e 'block:*' -a sleep 10
```

Профилирование

Выборка трассировок стека указанной команды с частотой 99 Гц:

```
perf record -F 99 команда
```

Выборка трассировок стека (через указатели фреймов) в масштабе всей системы в течение 10 с:

```
perf record -F 99 -a -g sleep 10
```

Выборка трассировок стека для процесса с заданным идентификатором PID с использованием отладочной информации в формате dwarf для раскручивания стека:

```
perf record -F 99 -p PID --call-graph dwarf sleep 10
```

Выборка трассировок стека для контейнера по его контрольной группе из /sys/fs/cgroup/perf_event:

```
perf record -F 99 -e cpu-clock --cgroup=docker/1d567f439319...etc... -a sleep 10
```

Выборка трассировок стека в масштабе всей системы с использованием метода записи последней ветви (Last Branch Record, LBR; Intel):

```
perf record -F 99 -a --call-graph lbr sleep 10
```

Выборка трассировок стека, по одной на каждые 100 промахов в кэше последнего уровня в течение 5 с:

```
perf record -e LLC-load-misses -c 100 -ag sleep 5
```

Точная выборка пользовательских инструкций (например, с использованием Intel PEBS) в течение 5 с:

```
perf record -e cycles:up -a sleep 5
```

Выборка трассировок стека с частотой 49 Гц и с выводом на экран имен и сегментов наиболее часто встречающихся процессов:

```
perf top -F 49 -ns comm,dso
```

Статическая трассировка

Регистрация событий запуска новых процессов до нажатия Ctrl-C:

```
perf record -e sched:sched_process_exec -a
```

Регистрация подмножества событий переключения контекста с трассировками стека в течение 1 с:

```
perf record -e context-switches -a -g sleep 1
```

Регистрация всех переключений контекста с трассировками стека в течение 1 с:

```
perf record -e sched:sched_switch -a -g sleep 1
```

Регистрация всех переключений контекста с трассировками стека глубиной до пяти уровней в течение 1 с:

```
perf record -e sched:sched_switch/max-stack=5/ -a sleep 1
```

Регистрация вызовов connect(2) (исходящие соединения) с трассировками стека до нажатия Ctrl-C:

```
perf record -e syscalls:sys_enter_connect -a -g
```

Регистрация событий передачи запросов блочным устройствам не чаще 100 раз в секунду до нажатия Ctrl-C:

```
perf record -F 100 -e block:block_rq_issue -a
```

Регистрация всех событий передачи запросов блочным устройствам и завершения их обработки (с отметками времени) до нажатия комбинации Ctrl-C:

```
perf record -e block:block_rq_issue,block:block_rq_complete -a
```

Регистрация всех событий передачи запросов блочным устройствам, размер которых больше или равен 64 Кбайт, до нажатия Ctrl-C:

```
perf record -e block:block_rq_issue --filter 'bytes >= 65536'
```

Регистрация всех обращений к ext4 с записью в файловую систему, отличную от ext4, до нажатия Ctrl-C:

```
perf record -e 'ext4:*' -o /tmp/perf.data -a
```

Регистрация USDT-событий http__server__request (из Node.js; Linux 4.10+):

```
perf record -e sdt_node:http__server__request -a
```

Регистрация событий передачи запросов блочным устройствам с выводом на экран (не в perf.data) до нажатия Ctrl-C:

```
perf trace -e block:block_rq_issue
```

Регистрация всех событий передачи запросов блочным устройствам и завершения их обработки с выводом на экран:

```
perf trace -e block:block_rq_issue,block:block_rq_complete
```

Регистрация обращений к системным вызовам в масштабе всей системы с выводом (подробной информации) на экран:

```
perf trace
```

Динамическая трассировка

Вставить зонд на входе в функцию ядра `tcp_sendmsg()` (`--add` можно опустить):

```
perf probe --add tcp_sendmsg
```

Удалить зонд из функции `tcp_sendmsg()` (вместо `--del` можно использовать `-d`):

```
perf probe --del tcp_sendmsg
```

Список переменных, доступных функции `tcp_sendmsg()`, включая внешние (требуется отладочная информация для ядра):

```
perf probe -V tcp_sendmsg --externs
```

Список строк в `tcp_sendmsg()`, где можно установить зонд (требуется отладочная информация для ядра):

```
perf probe -L tcp_sendmsg
```

Список переменных, доступных в строке 81 функции `tcp_sendmsg()` (требуется отладочная информация для ядра):

```
perf probe -V tcp_sendmsg:81
```

Вставить зонд на входе в функцию ядра `tcp_sendmsg()`, фиксирующий аргументы в регистрах (зависит от модели процессора):

```
perf probe 'tcp_sendmsg %ax %dx %cx'
```

Вставить зонд на входе в функцию ядра `tcp_sendmsg()`, фиксирующий значение регистра `%cx` под псевдонимом «bytes»:

```
perf probe 'tcp_sendmsg bytes=%cx'
```

Регистрировать срабатывание предыдущего зонда, когда значение `bytes` (псевдоним) больше 100:

```
perf record -e probe:tcp_sendmsg --filter 'bytes > 100'
```

Вставить зонд на выходе из `tcp_sendmsg()` и фиксировать возвращаемое значение:

```
perf probe 'tcp_sendmsg%return $retval'
```


Вставить зонд на входе в `tcp_sendmsg()` и фиксировать объем отправляемых данных и состояние сокета (требуется отладочная информация):

```
perf probe 'tcp_sendmsg size sk->__sk_common.skc_state'
```

Вставить зонд на входе в `do_sys_open()` и фиксировать имя файла в виде строки (требуется отладочная информация):

```
perf probe 'do_sys_open filename:string'
```

Вставить зонд на входе в функцию пользовательского уровня `fopen(3)` из библиотеки `libc`:

```
perf probe -x /lib/x86_64-linux-gnu/libc.so.6 --add fopen
```

Отчеты

Вывод содержимого `perf.data` в браузере `ncurses` (TUI), если возможно:

```
perf report
```

Вывод содержимого `perf.data` в виде текстового отчета с объединенными данными, суммами и процентами:

```
perf report -n --stdio
```

Вывод всех событий, зафиксированных в `perf.data`, с заголовками (рекомендуется):

```
perf script --header
```

Вывод всех событий, зафиксированных в `perf.data`, с рекомендуемыми мной полями (файл должен создаваться подкомандой `record -a`. В Linux < 4.1 следует использовать параметр `-f` вместо `-F`):

```
perf script --header -F comm,pid,tid,cpu,time,event,ip,sym,dso
```

Создание флейм-графика (Linux 5.8+):

```
perf script report flamegraph
```

Дизассемблирование и аннотирование инструкций с процентами (требуется отладочная информация):

```
perf annotate --stdio
```

Это был перечень однострочных сценариев, которые я сам часто использую, но есть и другие возможности, которые здесь не рассматриваются. Дополнительные подкоманды `perf(1)` можно найти в предыдущем разделе, а также в последующих разделах этой главы и в других главах.

13.3. СОБЫТИЯ PERF

Список всех поддерживаемых событий можно получить командой `perf list`. Далее приводится неполный список событий, полученный в Linux 5.8, чтобы показать, какие типы событий существуют (выделены полужирным шрифтом>):

perf list

List of pre-defined events (to be used in -e):

```

branch-instructions OR branches          [Hardware event]
branch-misses                            [Hardware event]
bus-cycles                               [Hardware event]
cache-misses                             [Hardware event]
[...]
context-switches OR cs                   [Software event]
cpu-clock                                [Software event]
[...]
l1-dcache-load-misses                    [Hardware cache event]
l1-dcache-loads                          [Hardware cache event]
[...]
branch-instructions OR cpu/branch-instructions/ [Kernel PMU event]
branch-misses OR cpu/branch-misses/      [Kernel PMU event]
[...]
cache:
l1d.replacement
    [L1D data line replacements] [...]
floating point:
fp_arith_inst_retired.128b_packed_double
    [Number of SSE/AVX computational 128-bit packed double precision [...]]
frontend:
dsb2mte_switches.penalty_cycles
    [Decode Stream Buffer (DSB)-to-MITE switch true penalty cycles] [...]
memory:
cycle_activity.cycles_l3_miss
    [Cycles while L3 cache miss demand load is outstanding] [...]
offcore_response.demand_code_rd.l3_miss.any_snoop
    [DEMAND_CODE_RD & L3_MISS & ANY_SNOOP] [...]
other:
hw_interrupts.received
    [Number of hardware interrupts received by the processor]
pipeline:
arith.divider_active
    [Cycles when divide unit is busy executing divide or square root [...]]
uncore:
unc_arb_coh_trk_requests.all
    [Unit: uncore_arb Number of entries allocated. Account for Any type:
    e.g. Snoop, Core aperture, etc]
[...]
rNNN                                     [Raw hardware event descriptor]
cpu/t1=v1[,t2=v2,t3 ...]/modifier      [Raw hardware event descriptor]
    (see 'man perf-list' on how to encode it)

```

```

mem:<addr>[/len][:access]                [Hardware breakpoint]
alarmtimer:alarmtimer_cancel              [Tracepoint event]
alarmtimer:alarmtimer_fired              [Tracepoint event]
[...]
probe:do_nanosleep                        [Tracepoint event]
[...]
sdt_hotspot:class__initialization__clinit [SDT event]
sdt_hotspot:class__initialization__concurrent [SDT event]
[...]
List of pre-defined events (to be used in --pfm-events):

```

```

ix86arch:
  UNHALTED_CORE_CYCLES
    [count core clock cycles whenever the clock signal on the specific core is
running (not halted)]
  INSTRUCTION_RETIRED
[...]

```

Список сильно сокращен во многих местах, так как полный список в этой тестовой системе насчитывает 4402 строки. Вот основные типы событий:

- **Hardware event (аппаратное событие):** большинство событий этого типа генерируются процессором (с помощью счетчиков мониторинга PMC).
- **Software event (программное событие):** событие счетчика ядра.
- **Hardware cache event (событие аппаратного кэша):** события кэша процессора (PMC).
- **Kernel PMU event (событие модуля мониторинга производительности ядра):** события, генерируемые модулем мониторинга производительности (PMC).
- **cache, floating point... (кэш, плавающая точка...):** события, предусмотренные производителем процессора (PMC), с кратким описанием.
- **Raw hardware event descriptor (низкоуровневый дескриптор аппаратного события):** счетчики PMC, определяемые низкоуровневыми кодами.
- **Hardware breakpoint (аппаратная точка останова):** событие точки останова процессора.
- **Tracepoint event (событие точки трассировки):** события механизма статической инструментации ядра.
- **SDT event (событие SDT):** события механизма статической инструментации в пространстве пользователя (USDT).
- **pfm-events:** события библиотеки libpfm (добавлено в Linux 5.8).

В списке событий точек трассировки и SDT перечисляются точки, доступные для статической инструментации, но если вы создали несколько динамических зондов, они тоже будут перечислены в этом списке. Я включил в вывод один такой пример: `probe:do_nanosleep` — это установленный мной зонд `kprobe`, который описывается как «Tracepoint event».

Команда `perf list` может принимать аргумент со строкой поиска. Например, вот список событий, содержащих «`mem_load_l3`», с названиями, выделенными полужирным:

```
# perf list mem_load_l3
```

List of pre-defined events (to be used in `-e`):

cache:

```
  mem_load_l3_hit_retired.xsnp_hit
    [Retired load instructions which data sources were L3 and cross-core snoop
 hits in on-pkg core cache Supports address when precise (Precise event)]
  mem_load_l3_hit_retired.xsnp_hitm
    [Retired load instructions which data sources were HitM responses from shared
 L3 Supports address when precise (Precise event)]
  mem_load_l3_hit_retired.xsnp_miss
    [Retired load instructions which data sources were L3 hit and cross-core snoop
 missed in on-pkg core cache Supports address when precise (Precise event)]
  mem_load_l3_hit_retired.xsnp_none
    [Retired load instructions which data sources were hits in L3 without snoops
 required Supports address when precise (Precise event)]
  [...]
```

Это аппаратные события (на основе РМС), и вывод содержит их краткие описания. Аннотация (`Precise event` — точное событие) отмечает события, поддерживающие возможность точной выборки инструкций на основе событий (`precise event-based sampling`, PEBS).

13.4. АППАРАТНЫЕ СОБЫТИЯ

Об аппаратных событиях шла речь в главе 4 «Инструменты наблюдения», в разделе 4.3.9 «Аппаратные счетчики (РМС)». Обычно они реализуются посредством счетчиков РМС, которые настраиваются с помощью кодов, уникальных для разных процессоров. Например, инструкции ветвления в процессорах Intel обычно можно инструментировать в `perf(1)`, используя низкоуровневый дескриптор аппаратного события «`r00c4`», код которого формируется из значения `umask 0x0` и селектора события `0xc4`. Эти коды опубликованы в руководствах по процессорам [Intel 16], [AMD 18], [ARM 19]. Intel также публикует их списки в JSON [Intel 20c].

Не нужно запоминать эти коды, всегда сверяйте их с руководством по процессору. Для простоты в `perf(1)` поддерживаются удобочитаемые псевдонимы, которые можно использовать вместо кодов. Например, событие «`branch-instructions`» в вашей системе должно соответствовать счетчикам РМС для инструкций перехода¹.

¹ Раньше мне доводилось сталкиваться с проблемами несоответствия между удобочитаемыми псевдонимами и правильными кодами РМС. Их сложно диагностировать только по выводу `perf(1)`: для этого нужно иметь опыт работы с РМС и представления о том, какие значения нормальные. Имейте в виду возможность таких проблем. Учитывая скорость обновления процессора, я бы не исключал появления ошибок и в будущих определениях псевдонимов.

Некоторые из этих удобочитаемых псевдонимов можно заметить в предыдущем списке (аппаратные события и события PMU).

Сейчас есть огромное разнообразие типов процессоров, и регулярно выпускаются новые версии. Может быть так, что удобочитаемые псевдонимы еще не будут определены для вашего процессора в `perf(1)` или определены в более новой версии ядра. Кроме того, для некоторых РМС вообще нет таких псевдонимов. Мне постоянно приходится переключаться с удобочитаемых имен на низкоуровневые дескрипторы, когда требуется изучать более глубокие счетчики РМС, для которых псевдонимов нет. В псевдонимах также могут быть ошибки, и если вам доведется столкнуться с подозрительным результатом, то используйте низкоуровневый дескриптор, чтобы подтвердить или опровергнуть подозрения.

13.4.1. Дискретная выборка

При использовании команды `perf record` со счетчиками РМС выборка их значений происходит с определенной частотой по умолчанию, поэтому фиксируется далеко не каждое событие. Вот пример записи событий тактов:

```
# perf record -vve cycles -a sleep 1
Using CPUID GenuineIntel-6-8E
intel_pt default config: tsc,mtc,mtc_period=3,psb_period=3,pt,branch
-----
perf_event_attr:
  size                112
  { sample_period, sample_freq } 4000
  sample_type         IP|TID|TIME|CPU|PERIOD
  disabled            1
  inherit              1
  mmap                1
  comm                1
  freq                1
[...]
```

[perf record: Captured and wrote 3.360 MB perf.data (3538 samples)]

Как показывает этот вывод, здесь включена дискретная выборка (`freq 1`) с частотой 4000 Гц. Эти настройки требуют от ядра установить такую частоту выборки, чтобы фиксировалось примерно 4000 событий в секунду для каждого процессора. Дискретизация совершенно необходима, потому что некоторые события РМС могут происходить миллиарды раз в секунду (например, такты процессора), и оверхед на запись каждого события может оказаться недопустимым¹. Но проблема в том, что в выводе по умолчанию (без параметра очень подробного вывода: `-vv`) `perf(1)` не сообщает о том, что включена дискретизация, и вы могли бы рассчитывать на запись всех событий. Частота выборки событий влияет только на подкоманду `record`, подкоманда `stat` считает все события.

¹ Конечно, в таких случаях ядро будет защищать себя и уменьшать частоту выборки, отбрасывая события. Всегда проверяйте потерю событий, чтобы узнать, произошло ли это (например, проверьте сводные счетчики из: `perf report -D | tail -20`).

Частоту выборки событий можно изменить с помощью параметра `-F`, определяющего частоту в герцах, или с помощью параметра `-с`, определяющего *период* — количество событий, после накопления которых должна производиться одна выборка (такой способ выборки известен также как *выборка при переполнении*). Вот пример использования параметра `-F`:

```
perf record -F 99 -e cycles -a sleep 1
```

Эта команда будет производить выборку с частотой 99 Гц (событий в секунду). Она похожа на однострочные сценарии из раздела 13.2 «Однострочные сценарии»: они не определяют тип события (без `-e cycles`), что заставляет `perf(1)` по умолчанию использовать такты при доступности счетчиков РМС, или программное событие хода процессорных часов. Более подробно об этом идет речь в разделе 13.9.2 «Профилирование процессора».

Обратите внимание, что для `perf(1)` есть определенные ограничения на частоту и уровень потребления процессора. Эти ограничения можно просмотреть и установить с помощью `sysctl(8)`:

```
# sysctl kernel.perf_event_max_sample_rate
kernel.perf_event_max_sample_rate = 15500
# sysctl kernel.perf_cpu_time_max_percent
kernel.perf_cpu_time_max_percent = 25
```

На примере видно, что максимальная частота выборки в этой системе составляет 15 500 Гц, а максимальная нагрузка на процессор, которую может создавать `perf(1)` (в частности, прерыванием PMU), — 25 %.

13.5. ПРОГРАММНЫЕ СОБЫТИЯ

Эти события обычно соответствуют аппаратным событиями, но инструментируются программно. По аналогии с аппаратными событиями, для них может устанавливаться частота выборки по умолчанию, обычно 4000, из-за чего подкоманда `record` будет фиксировать только подмножество событий.

Обратите внимание на следующее различие между программным событием переключения контекста и эквивалентной точкой трассировки. Сначала программные события:

```
# perf record -vve context-switches -a -- sleep 1
[...]
-----
perf_event_attr:
  type                1
  size                112
  config              0x3
  { sample_period, sample_freq } 4000
  sample_type         IP|TID|TIME|CPU|PERIOD
[...]
```

```

    freq                                1
[...]
```

[perf record: Captured and wrote 3.227 MB perf.data (660 samples)]

Как показывает этот вывод, для регистрации программного события по умолчанию выбрана частота 4000 Гц. А теперь эквивалентная точка трассировки:

```

# perf record -vve sched:sched_switch -a sleep 1
[...]
```

```

perf_event_attr:
  type                2
  size                112
  config              0x131
  { sample_period, sample_freq } 1
  sample_type         IP|TID|TIME|CPU|PERIOD|RAW
[...]
```

[perf record: Captured and wrote 3.360 MB perf.data (3538 samples)]

В этом случае используется периодическая выборка (нет признака `freq 1`) с периодом 1 (эквивалент `-с 1`) и фиксируется каждое событие. То же самое можно сделать с программными событиями, добавив параметр `-с 1`, например:

```
perf record -vve context-switches -a -с 1 -- sleep 1
```

Но будьте внимательны: учитывайте объем данных, сохраняемых для каждого события, и связанный с этим оверхед, особенно в случае с переключениями контекста, которые могут следовать очень часто. Проверить их частоту можно с помощью `perf stat`: см. раздел 13.8 «perf stat».

13.6. СОБЫТИЯ ТОЧЕК ТРАССИРОВКИ

Точки трассировки были представлены в главе 4 «Инструменты наблюдения», в разделе 4.3.5 «Точки трассировки», с примерами их инструментации с помощью `perf(1)`. Напомню, что я использовал точку трассировки `block:block_rq_issue` и показал следующие примеры.

Трассировка всей системы в течение 10 с и вывод событий:

```
perf record -e block:block_rq_issue -a sleep 10; perf script
```

Вывод аргументов этой точки трассировки и ее строки формата (метаданных):

```
cat /sys/kernel/debug/tracing/events/block/block_rq_issue/format
```

Регистрация событий блочного ввода/вывода с размером больше 65 536 байт:

```
perf record -e block:block_rq_issue --filter 'bytes > 65536' -a sleep 10
```

Дополнительные примеры использования `perf(1)` и точек трассировки приведены в разделе 13.2 «Однострочные сценарии» и в других главах этой книги.

Обратите внимание, что `perf list` показывает все инициализированные зонды, включая `kprobes` (механизм динамической инструментации ядра), как «Tracpoint event» (события точек трассировки); см. раздел 13.7 «События зондов».

13.7. СОБЫТИЯ ЗОНДОВ

`perf(1)` использует термин *события зондов* (probe events) для обозначения зондов `kprobes`, `uprobes` и `USDT`. Они являются «динамическими» и поэтому должны инициализироваться перед трассировкой: они не присутствуют в выводе `perf list` по умолчанию (некоторые зонды `USDT` могут присутствовать, потому что были инициализированы автоматически). После инициализации они отображаются как «Tracpoint event» (события точек трассировки).

13.7.1. kprobes

Зонды `kprobes` были представлены в главе 4 «Инструменты наблюдения», в разделе 4.3.6 «`kprobes`». Вот типичный порядок инициализации и использования зонда `kprobe`; в этом примере инструментируется функция ядра `do_nanosleep()`:

```
perf probe --add do_nanosleep
perf record -e probe:do_nanosleep -a sleep 5
perf script
perf probe --del do_nanosleep
```

Инициализируется зонд `kprobe` с помощью подкоманды `probe` с параметром `--add` (`--add` можно опустить), а когда он становится ненужным, то удаляется с помощью подкоманды `probe` с параметром `--del`. Вот вывод этой последовательности команд, включая сами события:

```
# perf probe --add do_nanosleep
Added new event:
  probe:do_nanosleep (on do_nanosleep)
```

You can now use it in all perf tools, such as:

```
perf record -e probe:do_nanosleep -aR sleep 1
```

```
# perf list probe:do_nanosleep
```

List of pre-defined events (to be used in -e):

```
probe:do_nanosleep                                [Tracpoint event]
```

```
# perf record -e probe:do_nanosleep -aR sleep 1
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 3.368 MB perf.data (604 samples) ]
# perf script
sleep 11898 [002] 922215.458572: probe:do_nanosleep: (ffffffff83dbb6b0)
SendControllerT 15713 [002] 922215.459871: probe:do_nanosleep: (ffffffff83dbb6b0)
```



```
SendControllerT 5460 [001] 922215.459942: probe:do_nanosleep: (ffffffff83dbb6b0)
[...]
```

```
# perf probe --del probe:do_nanosleep
Removed event: probe:do_nanosleep
```

Вывод `perf script` показывает вызовы `do_nanosleep()`, произошедшие во время трассировки, сначала из команды `sleep(1)` (скорее всего, из команды `sleep(1)`, которую запустила команда `perf(1)`), за которыми следуют вызовы `SendControllerT` (имя усечено).

Инструментация точек выхода из функций выполняется добавлением `%return`:

```
perf probe --add do_nanosleep%return
```

В этом случае будет инициализирован зонд `kretprobe`.

Аргументы зондов *kprobe*

Есть минимум четыре разных способа получить значения аргументов функций ядра.

Во-первых, если доступна отладочная информация для ядра, то для `perf(1)` будет доступна информация о переменных функции, включая аргументы. Вот как можно получить список переменных в `do_nanosleep()` с помощью параметра `--vars`:

```
# perf probe --vars do_nanosleep
Available variables at do_nanosleep
    @<do_nanosleep+0>
        enum hrtimer_mode      mode
        struct hrtimer_sleeper* t
```

Согласно этому выводу, в функции `do_nanosleep()` доступны переменные с именами `mode` и `t`, которые одновременно являются входными аргументами. Их можно добавить в команду инициализации зонда, чтобы включить запись их значений. Вот как можно добавить переменную `mode`:

```
# perf probe 'do_nanosleep mode'
[...]
# perf record -e probe:do_nanosleep -a
[...]
# perf script
    svscan 1470 [012] 4731125.216396: probe:do_nanosleep: (ffffffffffa8e4e440)
mode=0x1
```

В этом примере видно, что `mode=0x1`.

Во-вторых, если отладочная информация для ядра недоступна (как это часто бывает в промышленных средах), то аргументы можно получить через соответствующие регистры. Один из приемов — использовать идентичную систему (на таком же оборудовании и с той же версией ядра) и установить на нее отладочную информацию для ядра. Затем в этой справочной системе определить соответствие регистров и аргументов, используя параметры `-n` (пробный запуск) и `-v` (подробный вывод):

```
# perf probe -nv 'do_nanosleep mode'
[...]
Writing event: p:probe/do_nanosleep _text+10806336 mode=%si:x32
[...]
```

Поскольку это пробный запуск, он не создает событий, но показывает, в каком регистре хранится значение переменной `mode` (выделено жирным): оно хранится в регистре `%si` как 32-битное шестнадцатеричное число (`x32`). (Этот синтаксис объясняется в следующем разделе, посвященном зондам `uprobes`.) Теперь эту информацию можно использовать в системе без отладочной информации, скопировав и вставив строку объявления переменной (`mode =% si: x32`):

```
# perf probe 'do_nanosleep mode=%si:x32'
[...]
# perf record -e probe:do_nanosleep -a
[...]
# perf script
    svscan 1470 [000] 4732120.231245: probe:do_nanosleep: (fffffffffa8e4e440)
mode=0x1
```

Этот прием работает, только если в системах используются процессоры с одинаковым ABI и одинаковые версии ядра, в противном случае аргументы могут оказаться в других регистрах.

В-третьих, если ABI процессора известен, вы можете сами определить размещение аргументов в регистрах. Пример такого подхода показан в следующем разделе о зондах `uprobes`.

В-четвертых, появился новый источник отладочной информации для ядра: формат представления типов BPF (BTF). Скорее всего, он будет использоваться по умолчанию и будущие версии `perf(1)` будут поддерживать его в качестве альтернативного источника отладочной информации.

Чтобы получить возвращаемое значение `do_nanosleep`, инструментируйте зонд `kretprobe` и прочтите специальную переменную `$retval`:

```
perf probe 'do_nanosleep%return $retval'
```

См. исходный код ядра, чтобы определить, что может содержать возвращаемое значение.

13.7.2. uprobes

Зонды `uprobes` были представлены в главе 4 «Инструменты наблюдения», в разделе 4.3.7 «`uprobes`». Зонды `uprobes` инициализируются аналогично зондам `kprobes`. Например, чтобы инициализировать зонд `uprobe` на входе в функцию `fopen(3)` из библиотеки `libc`:

```
# perf probe -x /lib/x86_64-linux-gnu/libc.so.6 --add fopen
Added new event:
    probe_libc:fopen      (on fopen in /lib/x86_64-linux-gnu/libc-2.27.so)
```

Теперь этот зонд с именем `probe_libc:fopen` можно использовать в команде `perf record` для регистрации событий:

```
perf record -e probe_libc:fopen -aR sleep 1
```

Когда зонд `uprobe` станет ненужным, удалите его с помощью параметра `--del`:

```
# perf probe --del probe_libc:fopen
Removed event: probe_libc:fopen
```

С помощью параметра `-x` можно указать путь к двоичному файлу для инструментации. Для получения возвращаемого значения добавьте переменную `%return`:

```
perf probe -x /lib/x86_64-linux-gnu/libc.so.6 --add fopen%return
```

В этом случае будет использоваться зонд `uretprobe`.

Аргументы зондов `uprobe`

При наличии в системе отладочной информации для целевого двоичного файла может быть доступна информация о переменных и аргументах функций. Получить их список можно с помощью `--vars`:

```
# perf probe -x /lib/x86_64-linux-gnu/libc.so.6 --vars fopen
Available variables at fopen
    @<_IO_vfscanf+15344>
        char*    filename
        char*    mode
```

Как показывает вывод, в функции `fopen(3)` доступны переменные `filename` и `mode`. Их можно добавить в команду инициализации зонда:

```
perf probe -x /lib/x86_64-linux-gnu/libc.so.6 --add 'fopen filename mode'
```

Отладочная информация может быть предоставлена через пакет `-dbg` или `-dbgsym`. Если он недоступен в целевой, но есть в другой системе, можно воспользоваться этой другой системой для справки, как было показано в предыдущем разделе о зондах `kprobes`.

Если отладочная информация вообще недоступна, варианты все равно есть. Один из них — скомпилировать ПО с включенной отладочной информацией (если это ПО с открытым исходным кодом). Другой вариант — самостоятельно определить использование регистров на основе ABI процессора. Вот пример для `x86_64`:

```
# perf probe -x /lib/x86_64-linux-gnu/libc.so.6 --add 'fopen filename=%0(%di):string
mode=%si:u8'
[...]
# perf record -e probe_libc:fopen -a
[...]
# perf script
run 28882 [013] 4503285.383830: probe_libc:fopen: (7fbe130e6e30)
filename="/etc/nsswitch.conf" mode=147
```

```

run 28882 [013] 4503285.383997: probe_libc:fopen: (7fbe130e6e30)
filename="/etc/passwd" mode=17
setuidgid 28882 [013] 4503285.384447: probe_libc:fopen: (7fed1ad56e30)
filename="/etc/nsswitch.conf" mode=147
setuidgid 28882 [013] 4503285.384589: probe_libc:fopen: (7fed1ad56e30)
filename="/etc/passwd" mode=17
run 28883 [014] 4503285.392096: probe_libc:fopen: (7f9be2f55e30)
filename="/etc/nsswitch.conf" mode=147
run 28883 [014] 4503285.392251: probe_libc:fopen: (7f9be2f55e30)
filename="/etc/passwd" mode=17
mkdir 28884 [015] 4503285.392913: probe_libc:fopen: (7fad6ea0be30)
filename="/proc/filesystems" mode=22
chown 28885 [015] 4503285.393536: probe_libc:fopen: (7efcd22d5e30)
filename="/etc/nsswitch.conf" mode=147
[...]
```

В этом выводе можно видеть несколько вызовов `fopen(3)` с именами файлов `/etc/nsswitch.conf`, `/etc/passwd` и т. д.

Разберем синтаксис, который я использовал:

- **filename=**: это псевдоним («filename»), который будет использоваться для аннотирования вывода.
- **%di, %si**: это регистры в архитектуре `x86_64`, которые, согласно `AMD64 ABI`, содержат первые два аргумента функции [Matz 13].
- **+0(...)**: разыменовать содержимое по нулевому смещению. Без этого вместо строки, находящейся по заданному адресу, были бы напечатаны сами адреса.
- **:string**: вывести как строку.
- **:u8**: вывести как 8-битное целое число без знака.

Синтаксис задокументирован на странице справочного руководства `man` для `perf-probe(1)`.

Прочитать возвращаемое значение при использовании зонда `uretprobe` можно с помощью `$retval`:

```
perf probe -x /lib/x86_64-linux-gnu/libc.so.6 --add 'fopen%return $retval'
```

См. исходный код приложения, чтобы узнать, что содержится в возвращаемом значении.

Зонды `uprobes` позволяют исследовать внутреннюю работу приложения, но у них нестабильный интерфейс, потому что они напрямую используют двоичный файл, который может меняться от версии к версии ПО. Намного предпочтительнее использовать зонды `USDT`, если они доступны.

13.7.3. USDT

О зондах `USDT` шла речь в главе 4 «Инструменты наблюдения», в разделе 4.3.8 «`USDT`». Они обеспечивают стабильный интерфейс для трассировки событий.

При наличии в двоичном файле зондов USDT¹ их список можно получить с помощью подкоманды `buildid-cache`. Вот пример для двоичного файла Node.js, скомпилированного с зондами USDT (сборка настраивалась командой: `./configure --with-dtrace`):

```
# perf buildid-cache --add $(which node)
```

После этого список доступных зондов USDT можно получить командой `perf list`:

```
# perf list | grep sdt_node
sdt_node:gc__done          [SDT event]
sdt_node:gc__start         [SDT event]
sdt_node:http__client__request [SDT event]
sdt_node:http__client__response [SDT event]
sdt_node:http__server__request [SDT event]
sdt_node:http__server__response [SDT event]
sdt_node:net__server__connection [SDT event]
sdt_node:net__stream__end    [SDT event]
```

В данном случае это всего лишь метаданные, описывающие местоположение статически определенных событий трассировки (statically defined tracing, SDT) в коде программы. Чтобы фактически их инструментировать, нужно инициализировать зонды примерно так же, как мы инициализировали зонды `uprobes` в предыдущем разделе (для инициализации зондов USDT используется все тот же механизм `uprobes`)². Вот пример инициализации зонда `sdt_node:http__server__request`:

```
# perf probe sdt_node:http__server__request
Added new event:
  sdt_node:http__server__request (on %http__server__request in
/home/bgregg/Build/node-v12.4.0/out/Release/node)
```

После инициализации зонд можно использовать в инструментах `perf`, например:

```
perf record -e sdt_node:http__server__request -aR sleep 1
```

```
# perf list | grep http__server__request
sdt_node:http__server__request [Tracepoint event]
sdt_node:http__server__request [SDT event]
```

Обратите внимание, что событие отображается как событие SDT («SDT event», из метаданных USDT) и как событие точки трассировки («Tracepoint event», которое можно инструментировать с помощью `perf(1)` и других инструментов). Немного странно видеть две записи для одного и того же события, но такое представление согласуется с поддержкой других событий. Также есть кортежи для точек трассировки,

¹ Также список зондов USDT, доступных в двоичном файле, можно получить командой `readelf -n`: они будут перечислены в разделе примечаний ELF.

² В будущем этот шаг может отпасть: при необходимости команда `perf record` может автоматически интерпретировать события SDT как точки трассировки.

за исключением того, что `perf(1)` выводит не точки трассировки, а только соответствующие им события (если они существуют¹).

Вот пример регистрации событий USDT:

```
# perf record -e sdt_node:http__server__request -a
^C[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 3.924 MB perf.data (2 samples) ]
# perf script
    node 16282 [006] 510375.595203: sdt_node:http__server__request:
(55c3d8b03530) arg1=140725176825920 arg2=140725176825888 arg3=140725176829208
arg4=39090 arg5=140725176827096 arg6=140725176826040 arg7=20
    node 16282 [006] 510375.844040: sdt_node:http__server__request:
(55c3d8b03530) arg1=140725176825920 arg2=140725176825888 arg3=140725176829208
arg4=39092 arg5=140725176827096 arg6=140725176826040 arg7=20
```

Как показывает вывод, в период трассировки зонд `sdt_node:http__server__request` сработал дважды. Здесь также видны аргументы для зонда USDT, но некоторые из них — это структуры и строки, поэтому `perf(1)` вывел их как адреса указателей. Чтобы получить фактические значения аргументов, *должна быть* какая-то возможность указывать типы аргументов при инициализации зонда. Например, чтобы преобразовать третий аргумент в строку с именем «address»:

```
perf probe --add 'sdt_node:http__server__request address=+0(arg3):string'
```

Но на момент написания этих строк такая возможность не поддерживалась.

Еще одна типичная проблема, которая была исправлена в Linux 4.20, заключается в том, что некоторые зонды USDT требуют наращивания семафора в адресном пространстве процесса для правильной активации. `sdt_node:http__server__request` — один из таких зондов, и без наращивания семафора он не будет регистрировать никакие события.

13.8. PERF STAT

Подкоманда `perf stat` подсчитывает количество событий. Ее можно использовать для измерения частоты событий или просто для проверки наличия каких-либо событий. `perf stat` имеет эффективную реализацию: она подсчитывает программные события в контексте ядра, а аппаратные события с использованием регистров РМС. Это позволяет прикинуть оверхед более дорогостоящей подкоманды `perf record`, предварительно измерив частоту событий с помощью `perf stat`.

Вот пример подсчета событий точки трассировки `sched:sched_switch` (с параметром `-e`, в котором указывается событие для подсчета) в системе в целом (`-a`) в течение одной секунды (`sleep 1`: фиктивная команда):

¹ В документации ядра действительно указано, что некоторые точки трассировки могут не иметь соответствующих им событий, хотя я еще не сталкивался с таким.

```
# perf stat -e sched:sched_switch -a -- sleep 1
Performance counter stats for 'system wide':
```

```
5,705 sched:sched_switch
1.001892925 seconds time elapsed
```

В данном случае за одну секунду точка трассировки sched:sched switch сработала 5705 раз.

Я часто использую разделитель «--» командной оболочки, чтобы отделить параметры команды perf(1) от фиктивной команды, которую она запускает, хотя здесь это не нужно.

В разделах ниже объясняются параметры подкоманды и демонстрируются примеры ее использования.

13.8.1. Параметры

Подкоманда **stat** поддерживает множество параметров, в том числе:

- **-a**: регистрировать события на всех процессорах (начиная с Linux 4.11 этот параметр подразумевается по умолчанию);
- **-e event**: регистрировать указанное событие *event*;
- **--filter filter**: установить логическое выражение фильтра *filter* для события;
- **-p PID**: регистрировать события только для процесса с этим *PID*;
- **-t TID**: регистрировать события только для потока с этим *TID*;
- **-G cgroup**: регистрировать события только для этой контрольной группы *cgroup* (используется для контейнеров);
- **-A**: вести подсчет для каждого процессора отдельно;
- **-I interval_ms**: выводить результаты через каждый интервал *interval_ms* (в миллисекундах);
- **-v**: выводить подробные сообщения; **-vv**: выводить дополнительные сообщения.

Событиями могут быть точки трассировки, программные события, аппаратные события, зонды kprobes, uprobes и USDT (см. разделы 13.3–13.7). Для перечисления нескольких событий можно использовать шаблонные символы («*» соответствует любому количеству любых символов, а «?» соответствует любому одному символу). Например, следующая команда подсчитает все события для всех точек трассировки типа sched:

```
# perf stat -e 'sched:*' -a
```

Также для перечисления нескольких описаний событий можно использовать несколько параметров **-e**. Например, события точек трассировки sched и block можно подсчитать любым из этих способов:

```
# perf stat -e 'sched:*' -e 'block:*' -a
# perf stat -e 'sched:*,block:*' -a
```

Если события не указаны, по умолчанию `perf stat` будет использовать счетчики РМС из архитектурного набора: соответствующий пример см. в главе 4 «Инструменты наблюдения», в разделе 4.3.9 «Аппаратные счетчики (РМС)».

13.8.2. Интервальные статистики

Интервальные статистики можно вывести с помощью параметра `-I`. Вот пример вывода количества событий `sched:sched_switch` каждые 1000 мс:

```
# perf stat -e sched:sched_switch -a -I 1000
#           time                counts unit events
1.000791768          5,308      sched:sched_switch
2.001650037          4,879      sched:sched_switch
3.002348559          5,112      sched:sched_switch
4.003017555          5,335      sched:sched_switch
5.003760359          5,300      sched:sched_switch
^C      5.217339333          1,256      sched:sched_switch
```

Столбец `counts` сообщает количество событий, происходивших в истекший интервал. При просмотре этого столбца можно заметить изменения во времени. Последняя строка показывает количество событий от начала текущего интервала до момента, когда я нажал `Ctrl-C`, чтобы завершить `perf(1)`. Это время составило 0,214 с, как нетрудно подсчитать по значениям в столбце `time`.

13.8.3. Баланс между процессорами

Баланс между процессорами можно оценить параметром `-A`:

```
# perf stat -e sched:sched_switch -a -A -I 1000
#           time CPU                counts unit events
1.000351429 CPU0          1,154      sched:sched_switch
1.000351429 CPU1           555      sched:sched_switch
1.000351429 CPU2           492      sched:sched_switch
1.000351429 CPU3           925      sched:sched_switch
[...]
```

В этом случае команда выводит количество событий за интервал отдельно для каждого логического процессора.

Есть также параметры `--per-socket` и `--per-core` для подсчета событий по физическим процессорам и ядрам.

13.8.4. Фильтрация событий

Для некоторых типов событий (событий точек трассировки) поддерживается возможность фильтрации по значениям аргументов. Событие будет зарегистрировано, только если выражение, определяющее фильтр, истинно. Вот пример подсчета события `sched:sched_switch`, когда предыдущим был процесс с PID 25467:


```
# perf stat -e sched:sched_switch --filter 'prev_pid == 25467' -a -I 1000
#           time              counts unit events
    1.000346518              131      sched:sched_switch
    2.000937838              145      sched:sched_switch
    3.001370500               11      sched:sched_switch
    4.001905444              217      sched:sched_switch
[...]
```

Описание всех этих аргументов вы найдете в главе 4 «Инструменты наблюдения», в разделе 4.3.5 «Точки трассировки», в подразделе «Аргументы и строка формата точки трассировки». Они уникальны для каждого события и перечисляются в соответствующих файлах format в /sys/kernel/debug/tracing/events.

13.8.5. Теневые статистики

perf(1) имеет множество теневых статистик, которые будут выводиться при инструментации определенных комбинаций событий. Например, при инструментации счетчиков РМС для циклов и инструкций выводится статистика *количество инструкций на такт* (instructions per cycle, IPC):

```
# perf stat -e cycles,instructions -a
^C
Performance counter stats for 'system wide':

    2,895,806,892 cycles
    6,452,798,206 instructions          # 2.23 insn per cycle

    1.040093176 seconds time elapsed
```

Как показывает этот вывод, на один такт приходится 2,23 инструкции. Теневые статистики выводятся справа от знака решетки. При вызове без событий perf stat выводит несколько таких теневых статистик (см. главу 4 «Инструменты наблюдения», раздел 4.3.9 «Аппаратные счетчики (РМС)»).

Для более подробного изучения событий их можно регистрировать с помощью perf record.

13.9. PERF RECORD

Подкоманда perf record записывает события в файл для последующего анализа. Требуемое событие указывается в параметре -e, поддерживается возможность регистрации нескольких событий одновременно (для этого команде можно передать несколько параметров -e или перечислить события через запятую).

По умолчанию события сохраняются в файл с именем perf.data. Например:

```
# perf record -e sched:sched_switch -a
^C[ perf record: Woken up 9 times to write data ]
[ perf record: Captured and wrote 6.060 MB perf.data (23526 samples) ]
```

Обратите внимание, что вывод включает размер файла `perf.data` (6,060 Мбайт), количество образцов (23 526) и число случаев, когда выполнение `perf (1)` возобновлялось для записи данных (9 раз). Данные передаются из ядра в пользовательское пространство через кольцевые буферы, по одному для каждого процессора; чтобы уменьшить оверхед на переключение контекста, `perf(1)` периодически возобновляется для их чтения.

Предыдущая команда продолжала запись до нажатия `Ctrl-C`. Чтобы установить определенную продолжительность трассировки, используйте фиктивную команду `sleep(1)` (или любую другую), как было показано на примере команды `perf stat` выше:

```
perf record -e tracepoint -a -- sleep 1
```

Эта команда регистрирует события `tracepoint` в масштабе всей системы (`-a`) в течение одной секунды.

13.9.1. Параметры

Подкоманда `record` поддерживает множество параметров, в том числе:

- **-a**: регистрировать события на всех процессорах (начиная с Linux 4.11, этот параметр подразумевается по умолчанию);
- **-e *event***: регистрировать указанное событие *event*;
- **--filter *filter***: установить логическое выражение фильтра *filter* для события;
- **-p *PID***: регистрировать события только для процесса с этим *PID*;
- **-t *TID***: регистрировать события только для потока с этим *TID*;
- **-G *cgroup***: регистрировать события только для этой контрольной группы *cgroup* (используется для контейнеров);
- **-g**: записывать трассировки стека;
- **--call-graph *mode***: записывать трассировки стека с использованием данного метода (`fp`, `dwarf` или `lbr`);
- **-o *file***: записывать события в файл с именем *file*;
- **-v**: выводить подробные сообщения; **-vv**: выводить дополнительные сообщения.

Те же события можно регистрировать с помощью подкоманды `perf stat` и выводить на экран в реальном времени (по мере их появления) с помощью `perf trace`.

13.9.2. Профилирование процессора

Команда `perf(1)` часто используется для профилирования процессора. Следующая команда будет записывать трассировки стека для всех процессоров с частотой 99 Гц в течение 30 с:

```
perf record -F 99 -a -g -- sleep 30
```

Когда событие не указано (нет параметра `-e`), `perf(1)` по умолчанию использует первое из доступных событий (многие используют *точные события*, представленные в главе 4 «Инструменты наблюдения», в разделе 4.3.9 «Аппаратные счетчики (PMC)»):

1. **cycles:ppp**: выборка частоты на основе тактов процессора с точной настройкой на нулевой занос (`skid`);
2. **cycles:pp**: выборка частоты на основе тактов процессора с точной настройкой на запрос нулевого заноса (который на практике может отличаться от нуля);
3. **cycles:p**: выборка частоты на основе тактов процессора с точной настройкой на запрос постоянного заноса;
4. **cycles**: выборка частоты на основе тактов ЦП (неточно);
5. **cpu-clock**: программная выборка частоты процессора.

При этом выбирается наиболее точный механизм профилирования процессора. Синтаксис `:ppp`, `:pp` и `:p` активирует режим выборки точных событий и может применяться к другим событиям (не только к тактам), которые его поддерживают. Кроме того, события могут поддерживать разные уровни точности. В процессорах Intel для точной выборки событий используется механизм PEBS, в процессорах AMD — механизм IBS. Они были кратко описаны в подразделе «Проблемы PMC» раздела 4.3.9.

13.9.3. Обход стека

Для записи трассировок стека вместо параметра `-g` можно использовать параметр конфигурации `max-stack`. У такого подхода два преимущества: он позволяет указать максимальную глубину стека и использовать разные настройки для разных событий. Например:

```
# perf record -e sched:sched_switch/max-stack=5/,sched:sched_wakeup/max-stack=1/ \
-a -- sleep 1
```

Это команда будет регистрировать события `sched_switch` с трассировками стека глубиной до пяти фреймов и события `sched_wakeup` с трассировками стека глубиной до одного фрейма.

Обратите внимание: если трассировки стека выглядят поврежденными, то это может быть связано с тем, что программное обеспечение не поддерживает регистр указателя фрейма. Эта проблема обсуждалась в главе 5 «Приложения», в разделе 5.6.2 «Отсутствующие стеки». Кроме перекомпиляции ПО с поддержкой указателей фреймов (например, `gcc(1) -fno-omit-frame-pointer`), можно также попробовать другой метод обхода стека, выбрав его с помощью параметра `--call-graph`, например:

- **--call-graph dwarf**: выбирает механизм обхода стека, основанный на использовании отладочной информации, который требует наличия отладочной

информации для выполняемого файла (для некоторых программ, например, можно установить пакет с именем, заканчивающимся на «-dbgsym» или «-dbg»);

- **--call-graph lbr**: выбирает механизм обхода стека, реализованный для процессоров Intel и основанный на записи последней ветви (Last Branch Record, LBR), который, впрочем, ограничивает глубину стека 16 фреймами¹ и поэтому имеет ограниченную полезность;
- **--call-graph fp**: выбирает механизм обхода стека, основанный на указателях фреймов (по умолчанию).

Обход стека на основе указателей фреймов описан в главе 3 «Операционные системы», в разделе 3.2.7 «Стеки». Другие механизмы (dwarf, LBR и ORC) описаны в главе 2 «Tech» («Основы технологии»), в разделе 2.4 «Stack Trace Walking» («Обход трассировки стека»), в книге «BPF Performance Tools» [Gregg 19].

После записи событий их можно исследовать с помощью `perf report` или `perf script`.

13.10. PERF REPORT

Подкоманда `perf report` обобщает содержимое файла `perf.data` и поддерживает следующие параметры:

- **--tui**: использовать интерфейс TUI (по умолчанию);
- **--stdio**: вывести отчет в текстовом виде;
- **-i file**: входной файл;
- **-n**: включить столбец со счетчиком образцов;
- **-g options**: установить параметры отображения трассировки стека.

Исследовать содержимое `perf.data` можно с помощью сторонних инструментов. Эти инструменты могут обрабатывать вывод команды `perf script`, описанной в разделе 13.11 «`perf script`». Команды `perf report` обычно достаточно во многих ситуациях, и сторонние инструменты нужны нечасто. Команда `perf report` выводит результаты либо с использованием интерактивного текстового пользовательского интерфейса (text user interface, TUI), либо в виде простого текстового отчета (STDIO).

13.10.1. TUI

Вот пример профилирования регистра указателя инструкций процессора с частотой 99 Гц в течение 10 с (без записи трассировок стека) и вывода результатов с использованием TUI:

¹ Для процессоров с микроархитектурой Haswell глубина стека составляет 16, для процессоров с микроархитектурой Skylake — 32.

```
# perf record -F 99 -a -- sleep 30
[ perf record: Woken up 193 times to write data ]
[ perf record: Captured and wrote 48.916 MB perf.data (11880 samples) ]
# perf report
Samples: 11K of event 'cpu-clock:pppH', Event count (approx.): 119999998800
Overhead Command Shared Object Symbol
 21.10% swapper [kernel.vmlinux] [k] native_safe_halt
  6.39% mysqld [kernel.vmlinux] [k] _raw_spin_unlock_irqrestor
  4.66% mysqld mysqld [.] _Z8ut_delay
  2.64% mysqld [kernel.vmlinux] [k] finish_task_switch
  2.59% oltpl_read_write [kernel.vmlinux] [k] finish_task_switch
  2.03% mysqld [kernel.vmlinux] [k] exit_to_usermode_loop
  1.68% mysqld mysqld [.] _Z15row_search_mvccPh15pag
  1.40% oltpl_read_write [kernel.vmlinux] [k] _raw_spin_unlock_irqrestor
[...]
```

perf report — это интерактивный интерфейс, позволяющий перемещаться по данным и выбирать функции и потоки для получения более подробной информации.

13.10.2. STDIO

Как выглядит тот же профиль процессора при выводе в виде текстового отчета (**--stdio**), было показано в разделе 13.1 «Обзор подкоманд». Этот отчет не имеет интерактивных функций, зато его удобно использовать для вывода отчета в файл в виде текста. Такие отчеты могут быть полезны для обмена с другими людьми через чат-системы, электронную почту и тикет-системы поддержки. Я обычно добавляю параметр **-n**, чтобы включить столбец со счетчиком образцов.

В качестве еще одного примера текстового отчета ниже показан профиль процессора, полученный с включенной трассировкой стека (**-g**):

```
# perf record -F 99 -a -g -- sleep 30
[ perf record: Woken up 8 times to write data ]
[ perf record: Captured and wrote 2.282 MB perf.data (11880 samples) ]
# perf report --stdio
[...]
```

#	Children	Self	Command	Shared Object	Symbol
#					
#					
50.45%		0.00%	mysqld	libpthread-2.27.so	[.] start_thread
	---	start_thread			
		--44.75%	pfs_spawn_thread		
		--44.70%	handle_connection		
		--44.55%	_Z10do_commandP3THD		
		--42.93%	_Z16dispatch_commandP3THD		
		--40.92%	_Z19mysqld_stm		

```
[...]
```

Трассировки стека объединяются в иерархию, начиная с корневой функции слева и двигаясь вниз и вправо. Самая правая функция — это функция, породившая событие (в данном случае функция, выполняющаяся на процессоре), а слева от нее — родительская. Путь в этом примере показывает, что процесс `mysqld` (демон) вызвал функцию `start_thread()`, которая вызывала `pfs_spawn_thread()`, которая вызывала `handle_connection()`, и т. д. Крайние правые функции в этом выводе не показаны.

Такой порядок — слева направо — в терминологии `perf(1)` называется *прямым, от вызывающей к вызываемой* (caller). Его можно изменить на *обратный, от вызываемой к вызывающей* (callee), когда функция, породившая событие, находится слева, а ее предки — внизу и справа. Для этого используйте параметр `-g callee` (раньше он подразумевался по умолчанию, но с версии Linux 4.4 в `perf(1)` стал использоваться прямой порядок).

13.11. PERF SCRIPT

Подкоманда `perf script` по умолчанию выводит каждый образец из `perf.data` и позволяет выявлять временные закономерности, которые могут затеряться в сводном отчете. Выходные данные можно использовать для создания флэйм-графиков. Также можно запускать *сценарии обработки трассировок*, автоматизирующие формирование и вывод отчетов о событиях разными способами. Об этом пойдет речь в данном разделе.

Для начала посмотрите, как выглядит вывод, сгенерированный на основе профиля процессора, собранного без трассировок стека:

```
# perf script
mysql 8631 [000] 4142044.582702: 10101010 cpu-clock:pppH:
c08fd9 _Z19close_thread_tablesP3THD+0x49 (/usr/sbin/mysqld)
mysql 8619 [001] 4142044.582711: 10101010 cpu-clock:pppH:
79f81d _ZN5Field10make_fieldEP10Send_field+0x1d (/usr/sbin/mysqld)
mysql 22432 [002] 4142044.582713: 10101010 cpu-clock:pppH:
fffffffff95530302 get_futex_key_refs.isra.12+0x32 (/lib/modules/5.4.0-rc8-virtua...
[...]
```

Ниже перечислены поля этого вывода вместе с содержимым из первой строки:

- **имя процесса:** `mysqld`;
- **идентификатор потока (TID):** `8631`;
- **идентификатор процессора:** `[000]`;
- **отметка времени:** `4142044.582702` (секунды);
- **период:** `10101010` (это значение — следствие использования параметра `-F 99`). Включается в некоторые режимы выборки;
- **имя события:** `cpu-clock:pppH`;

- **аргументы события:** это и последующие поля являются аргументами события. В аргументах события `cpu-clock` передаются указатель инструкции, имя и смещение функции, а также имя сегмента. Их описание приведено в главе 4, в разделе 4.3.5, в подразделе «Аргументы и строка формата точки трассировки».

Это перечень полей по умолчанию для этого события, но он может измениться в следующих версиях `perf(1)`. Другие события не включают поле периода.

Иногда важно обеспечить согласованность вывода, особенно когда требуется произвести дополнительную обработку. В таких случаях можно передать команде параметр `-F` с перечнем полей. Я часто использую его, чтобы включить идентификатор процесса, отсутствующий в наборе полей по умолчанию. Рекомендую добавлять параметр `--header`, чтобы включить метаданные из `perf.data`. Например, ниже показан профиль процессора с трассировкой стека:

```
# perf script --header -F comm,pid,tid,cpu,time,event,ip,sym,dso,trace
# =====
# captured on      : Sun Jan 5 23:43:56 2020
# header version   : 1
# data offset      : 264
# data size        : 2393000
# feat offset      : 2393264
# hostname         : bgregg-mysql
# os release       : 5.4.0
# perf version     : 5.4.0
# arch             : x86_64
# nr_cpus online   : 4
# nr_cpus avail    : 4
# cpudesc          : Intel(R) Xeon(R) Platinum 8175M CPU @ 2.50GHz
# cpuid            : GenuineIntel,6,85,4
# total memory     : 15923672 kB
# cmdline          : /usr/bin/perf record -F 99 -a -g -- sleep 30
# event            : name = cpu-clock:pppH, , id = { 5997, 5998, 5999, 6000 },
type = 1, size = 112, { sample_period, sample_freq } = 99, sample_ty
[...]
# =====
#
mysqlld 21616/8583 [000] 4142769.671581: cpu-clock:pppH:
c36299 [unknown] (/usr/sbin/mysqlld)
c3bad4 _ZN13QEP_tmp_table8end_sendEv (/usr/sbin/mysqlld)
c3c1a5 _Z13sub_select_opP4J0INP7QEP_TABb (/usr/sbin/mysqlld)
c346a8 _ZN4J0IN4execEv (/usr/sbin/mysqlld)
ca735a _Z12handle_queryP3THDP3LEXP12Query_resulttyy
[...]
```

Вывод включает заголовок с префиксом «#», описывающий систему и команду `perf(1)`, с помощью которой был создан файл `perf.data`. Сохранив этот вывод в файл, вы поблагодарите себя за предусмотрительность, потому что он содержит обширную информацию, которая понадобится позже. Эти файлы можно прочитать с помощью других инструментов визуализации, включая инструмент создания флейм-графиков.

13.11.1. Флейм-графики

Обычно флейм-графики используются для визуализации трассировок стека, полученных при профилировании процессора. Но с их помощью можно визуализировать коллекции трассировок стека для любых событий, собранные командой `perf(1)`, в том числе для событий переключения контекста, чтобы узнать, почему потоки оставляют процессор, либо для событий блочного ввода/вывода, чтобы увидеть, какие пути в коде ведут к дисковому вводу/выводу.

Две широко используемые реализации флейм-графиков (моя собственная и версия для d3) визуализируют вывод `perf script`. В `perf(1)` поддержка флейм-графиков была добавлена в Linux 5.8. Шаги, описывающие порядок создания флейм-графиков с использованием `perf(1)`, перечислены в главе 6 «Процессоры», в разделе 6.6.13 «perf», в подразделе «Флейм-графики процессора». Сама идея визуализации объясняется в разделе 6.7.3 «Флейм-графики».

FlameScope — еще один инструмент визуализации выходных данных `perf script`, объединяющий флейм-графики и тепловую карту с субсекундным смещением для изучения временных вариаций. Он также описан в главе 6 «Процессоры», в разделе 6.7.4 «FlameScope».

13.11.2. Сценарии обработки трассировок

Список доступных в `perf(1)` сценариев обработки трассировок можно получить с помощью параметра `-l`:

```
# perf script -l
List of available trace scripts:
[...]
event_analyzing_sample      analyze all perf samples
mem-phys-addr               resolve physical address samples
intel-pt-events             print Intel PT Power Events and PTWRITE
sched-migration              sched migration overview
net_dropmonitor              display a table of dropped frames
syscall-counts-by-pid [comm] system-wide syscall counts, by pid
failed-syscalls-by-pid [comm] system-wide failed syscalls, by pid
export-to-sqlite [database name] [columns] [calls] export perf data to a sqlite3
database
stackcollapse                produce callgraphs in short form for scripting
use
```

Их можно передавать в виде аргументов команде `perf script`. Также можно разрабатывать свои сценарии обработки трассировок на Perl или Python.

13.12. PERF TRACE

Подкоманда `perf trace` по умолчанию трассирует системные вызовы и выводит полученные данные на экран в реальном времени (без вывода в файл `perf.data`).

Этот способ был представлен в главе 5 «Приложения», в разделе 5.5.1 «perf» как альтернатива `strace(1)` с меньшим оверхедом, способная выполнять трассировку в масштабе всей системы. `perf trace` также может перехватывать любые события, используя синтаксис, аналогичный синтаксису `perf record`.

Вот пример трассировки проблемы с дисковым вводом/выводом:

```
# perf trace -e block:block_rq_issue,block:block_rq_complete
0.000 auditd/391 block:block_rq_issue:259,0 WS 8192 () 16046032 + 16 [auditd]
0.566 systemd-journald/28651 block:block_rq_complete:259,0 WS () 16046032 + 16 [0]
0.748 jbd2/nvme0n1p1/174 block:block_rq_issue:259,0 WS 61440 () 2100744 + 120
[jbd2/nvme0n1p1-]
1.436 systemd-journald/28651 block:block_rq_complete:259,0 WS () 2100744 + 120 [0]
1.515 kworker/0:1H-k/365 block:block_rq_issue:259,0 FF 0 () 0 + 0 [kworker/0:1H]
1.543 kworker/0:1H-k/365 block:block_rq_issue:259,0 WFS 4096 () 2100864 + 8
[kworker/0:1H]
2.074 sshd/6463 block:block_rq_complete:259,0 WFS () 2100864 + 8 [0]
2.077 sshd/6463 block:block_rq_complete:259,0 WFS () 2100864 + 0 [0]
1087.562 kworker/0:1H-k/365 block:block_rq_issue:259,0 W 4096 () 16046040 + 8
[kworker/0:1H]
[...]
```

Как и в случае с `perf record`, можно фильтровать события, используя некоторые строковые константы, сгенерированные из заголовков ядра. Вот пример трассировки обращений к системному вызову `mmap(2)` с флагом `MAP_SHARED` с использованием строки «`SHARED`» в качестве фильтра:

```
# perf trace -e syscalls:*enter_mmap --filter='flags==SHARED'
0.000 env/14780 syscalls:sys_enter_mmap(len: 27002, prot: READ, flags: SHARED,
fd: 3)
16.145 grep/14787 syscalls:sys_enter_mmap(len: 27002, prot: READ, flags: SHARED,
fd: 3)
18.704 cut/14791 syscalls:sys_enter_mmap(len: 27002, prot: READ, flags: SHARED,
fd: 3)
[...]
```

Обратите внимание, что для большей удобочитаемости строки формата `perf(1)` использует строки: вместо "`prot: 1`" команда вывела "`prot: READ`". В `perf(1)` эта возможность называется «украшательством».

13.12.1. Версии ядра

До Linux 4.19 команда `perf trace` по умолчанию инструментировала все системные вызовы (параметр `--syscalls`) в дополнение к указанным событиям (`-e`). Чтобы отключить трассировку других системных вызовов, нужно передать параметр `--no-syscalls` (теперь он подразумевается по умолчанию). Например:

```
# perf trace -e block:block_rq_issue,block:block_rq_complete --no-syscalls
```

Обратите внимание, что по умолчанию трассировка выполняется по всем процессам (`-a`), начиная с Linux 3.8. Фильтры (`--filter`) были добавлены в Linux 5.5.

13.13. ДРУГИЕ КОМАНДЫ

perf(1) поддерживает и другие подкоманды, часть которых используется в других главах. Напомню эти дополнительные подкоманды (полный список см. в табл. 13.1):

- **perf c2c** (Linux 4.10+): анализ ложного совместного использования строк кэша и операций копирования из кэша в кэш;
- **perf kmem**: анализирует распределение памяти ядра;
- **perf kvm**: анализ гостевой системы при использовании гипервизора KVM;
- **perf lock**: анализ блокировок;
- **perf mem**: анализ обращений к памяти;
- **perf sched**: статистики планировщика ядра;
- **perf script**: настраиваемый набор инструментов для perf.

В числе дополнительных возможностей можно назвать запуск программ BPF по событиям и использование механизмов аппаратной трассировки, таких как трассировка процессоров Intel (processor trace, PT) или ARM CoreSight, для анализа на уровне отдельных инструкций [Hunter 20].

Ниже приводится простой пример трассировки процессора Intel. Здесь **perf record** записывает такты в пользовательском режиме для команды **date(1)**:

```
# perf record -e intel_pt/cyc/u date
Sat Jul 11 05:52:40 PDT 2020
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.049 MB perf.data ]
```

Полученные результаты можно вывести как трассировку инструкций (инструкции выделены полужирным):

```
# perf script --insn-trace
      date 31979 [003] 653971.670163672: 7f3bfbf4d090 _start+0x0 (/lib/x86_64-
linux-gnu/ld-2.27.so) insn: 48 89 e7
      date 31979 [003] 653971.670163672: 7f3bfbf4d093 _start+0x3 (/lib/x86_64-
linux-gnu/ld-2.27.so) insn: e8 08 0e 00 00
[...]
```

Инструкции выводятся в виде машинного кода. После установки Intel X86 Encoder Decoder (XED) инструкции можно вывести в виде мнемоник на языке Ассемблера [Intelxed 19]:

```
# perf script --insn-trace --xed
date 31979 [003] 653971.670163672: ... (/lib/x86_64-linux-gnu/ld-2.27.so) mov %rsp,
%rdi
date 31979 [003] 653971.670163672: ... (/lib/x86_64-linux-gnu/ld-2.27.so) callq
0x7f3bfbf4dea0
date 31979 [003] 653971.670163672: ... (/lib/x86_64-linux-gnu/ld-2.27.so) pushq %rbp
[...]
```

```

date 31979 [003] 653971.670439432: ... (/bin/date) xor %ebp, %ebp
date 31979 [003] 653971.670439432: ... (/bin/date) mov %rdx, %r9
date 31979 [003] 653971.670439432: ... (/bin/date) popq %rsi
date 31979 [003] 653971.670439432: ... (/bin/date) mov %rsp, %rdx
date 31979 [003] 653971.670439432: ... (/bin/date) and $0xfffffffffffff0, %rsp
[...]
```

Вывод получается не только подробным, но и очень объемным. Полный вывод в этом примере составил 266 105 строк, и это только для команды `date(1)`. Другие примеры см. в Вики по `perf(1)` [Hunter 20].

13.14. ДОКУМЕНТАЦИЯ PERF

Для каждой подкоманды есть своя страница в справочном руководстве `man`, начинающаяся с «`perf-`», например, `perf-record(1)` — для подкоманды `record`. Они находятся в дереве исходных текстов Linux в каталоге `tools/perf/Documentation`.

На wiki.kernel.org есть руководство по `perf(1)` [Perf 15]. Винс Уивер (Vince Weaver) ведет свою неофициальную страницу `perf(1)` [Weaver 11]. Еще одна неофициальная страница с примерами использования `perf(1)` поддерживается мной [Gregg 20f].

Моя страница содержит исчерпывающий список однострочных сценариев для `perf(1)`, а также множество других примеров.

Поскольку `perf(1)` продолжает развиваться, обязательно проверьте наличие обновлений в более поздних версиях ядра. Хорошим источником послужит раздел `perf` в журнале изменений для каждой версии ядра, опубликованном на KernelNewbies [KernelNewbies 20].

13.15. ССЫЛКИ

[Weaver 11] Weaver, V., «The Unofficial Linux Perf Events Web-Page», http://web.eece.maine.edu/~vweaver/projects/perf_events, 2011.

[Matz 13] Matz, M., Hubička, J., Jaeger, A., and Mitchell, M., «System V Application Binary Interface, AMD64 Architecture Processor Supplement, Draft Version 0.99.6», <http://x86-64.org/documentation/abi.pdf>, 2013.

[Perf 15] «Tutorial: Linux kernel profiling with perf», `perf` wiki, <https://perf.wiki.kernel.org/index.php/Tutorial>, последнее обновление 2015.

[Intel 16] Intel 64 and IA-32 Architectures Software Developer's Manual Volume 3B: System Programming Guide, Part 2, September 2016, <https://www.intel.com/content/www/us/en/architecture-and-technology/64-ia-32-architectures-software-developer-vol-3b-part-2-manual.html>, 2016.

[AMD 18] Open-Source Register Reference for AMD Family 17h Processors Models 00h-2Fh, <https://developer.amd.com/resources/developer-guides-manuals>, 2018.

- [ARM 19] Arm® Architecture Reference Manual Armv8, for Armv8-A architecture profile, https://developer.arm.com/architectures/cpu-architecture/a-profile/docs?_ga=2.78191124.1893781712.1575908489-930650904.1559325573, 2019.
- [Intelxed 19] «Intel XED», <https://intelxed.github.io>, 2019.
- [Gregg 20h] Gregg, B., «One-Liners», <http://www.brendangregg.com/perf.html#OneLiners>, последнее обновление 2020.
- [Gregg 20f] Gregg, B., «perf Examples», <http://www.brendangregg.com/perf.html>, последнее обновление 2020.
- [Hunter 20] Hunter, A., «Perf tools support for Intel® Processor Trace», https://perf.wiki.kernel.org/index.php/Perf_tools_support_for_Intel%C2%AE_Processor_Trace, последнее обновление 2020.
- [Intel 20c] «/perfmon/», <https://download.01.org/perfmon>, по состоянию на 2020.
- [KernelNewbies 20] «KernelNewbies: LinuxVersions», <https://kernelnewbies.org/LinuxVersions>, по состоянию на 2020.

Глава 14

FTRACE

Ftrace — официальный трассировщик Linux, многофункциональный инструмент, включающий разнообразные утилиты трассировки. Ftrace был создан Стивеном Ростедтом (Steven Rostedt) и включен в Linux 2.6.27 (2008). Инструмент можно использовать без любых дополнительных интерфейсов уровня пользователя, что делает его особенно привлекательным для встраиваемых сред Linux, где пространство для хранения ограничено. Он пригодится и для анализа серверных сред.

Как и главы 13 «perf» и 15 «BPF», эта глава необязательна для чтения и предназначена для желающих изучить один или несколько системных трассировщиков более подробно.

С помощью Ftrace можно ответить на вопросы:

- Как часто вызываются те или иные функции ядра?
- Какие пути в коде приводят к вызову этой функции?
- Какие другие функции вызывает эта функция ядра?
- Какова самая высокая задержка, вызванная вытеснением выполняемого кода?

Следующие разделы представляют Ftrace более подробно, показывают некоторые из его профилировщиков и трассировщиков и перечисляют внешние интерфейсы, которые их используют:

- 14.1: Обзор возможностей
- 14.2: tracefs (/sys)
- Профилировщики:
 - 14.3: Профилировщик функций
 - 14.10: Триггеры hist
- Трассировщики:
 - 14.4: Трассировщик function
 - 14.5: Точки трассировки
 - 14.6: Зонды kprobes
 - 14.7: Зонды uprobes

- 14.8: Трассировщик `function_graph`
- 14.9: Трассировщик `hwlat`
- Внешние интерфейсы:
 - 14.11: `trace-cmd`
 - 14.12: `perf ftrace`
 - 14.13: `perf-tools`
- 14.14: Документация Ftrace
- 14.15: Ссылки

Триггеры `hist` — сложная тема, которая требует предварительного знакомства с профилировщиками и трассировщиками, поэтому в этой главе она рассматривается почти в самом конце. Разделы, посвященные зондам `kprobes` и `uprobes`, включают описание базовых возможностей профилирования.

На рис. 14.1 представлена структура Ftrace и внешних интерфейсов, где стрелками показаны пути от событий к выходным результатам в разных форматах.

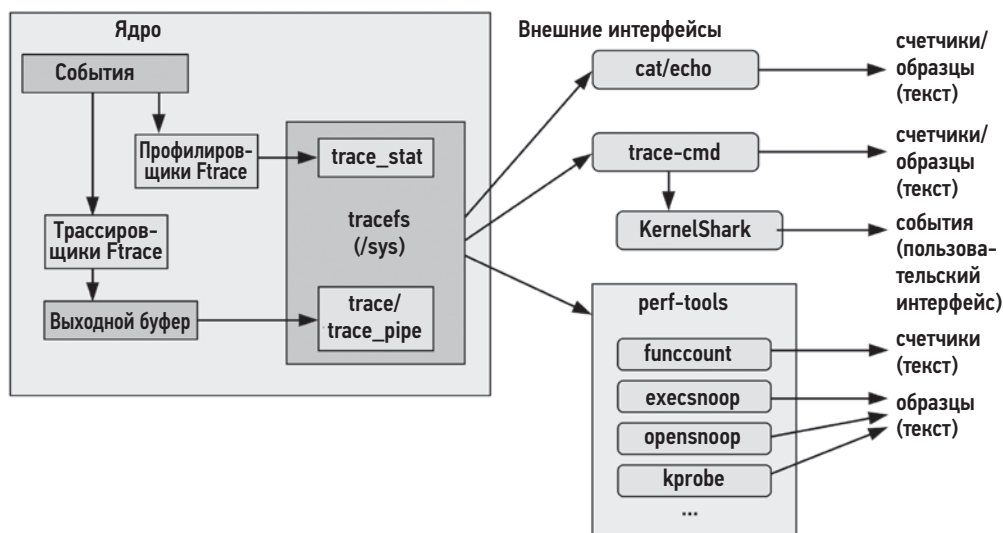


Рис. 14.1. Профилировщики, трассировщики и внешние интерфейсы в Ftrace

Обо всем этом подробно рассказано в следующих разделах.

14.1. ОБЗОР ВОЗМОЖНОСТЕЙ

В отличие от профилировщика `perf(1)`, в котором для решения разных задач используются подкоманды, в Ftrace есть отдельные *профилировщики* и *трассировщики*.

Профилировщики выводят сводные статистики, например счетчики и гистограммы, а трассировщики — подробные сведения об отдельных событиях.

В качестве примера использования Ftrace ниже показан инструмент funcgraph(8), использующий трассировщик Ftrace для функций, которые вызывает функция ядра `vfs_read()`:

```
# funcgraph vfs_read
Tracing "vfs_read"... Ctrl-C to end.
1)      | vfs_read() {
1)      |     rw_verify_area() {
1)      |         security_file_permission() {
1)      |             apparmor_file_permission() {
1)      |                 common_file_perm() {
1)      |                     aa_file_perm();
1) 0.763 us |                 }
1) 2.209 us |             }
1) 3.329 us |         }
1) 0.571 us |     __fsnotify_parent();
1) 0.612 us |     fsnotify();
1) 7.019 us | }
1) 8.416 us | }
1)      | __vfs_read() {
1)      |     new_sync_read() {
1)      |         ext4_file_read_iter() {
[...]
```

Как показывает вывод, `vfs_read()` вызывает `rw_verify_area()`, которая, в свою очередь, вызывает `security_file_permission()`, и т. д. Во втором столбце отображается продолжительность выполнения каждой функции («us» — это микросекунды), что позволяет проанализировать их производительности и выявить дочерние функции, из-за которых родительская функция работает медленно. Эта конкретная возможность в Ftrace называется трассировкой графа функций (и рассматривается в разделе 14.8 «Трассировщик `function_graph`»).

В табл. 14.1 и 14.2 перечислены профилировщики и трассировщики Ftrace из последней версии Linux (5.2) вместе с *трассировщиками событий* Linux: точки трассировки, а также зонды `kprobes` и `uprobes`. Трассировщики событий похожи на Ftrace, у них аналогичные настройки и интерфейсы вывода. Названия трассировщиков, выделенные моноширинным шрифтом в табл. 14.2, — это не только имена трассировщиков Ftrace, но и ключевые слова командной строки, используемые для их настройки.

Таблица 14.1. Профилировщики Ftrace

Профилировщик	Описание	Раздел
Профилировщик функций	Статистики функций ядра	14.3
Профилировщик зондов <code>kprobes</code>	Включает подсчет срабатываний зондов <code>kprobes</code>	14.6.5
Профилировщик зондов <code>uprobes</code>	Включает подсчет срабатываний зондов <code>uprobes</code>	14.7.4
Триггеры <code>hist</code>	Создает настраиваемые гистограммы распределения событий	14.10

Таблица 14.2. Ftrace и трассировщики событий

Трассировщик	Описание	Раздел
function	Трассировщик вызовов функций ядра	14.4
Точки трассировки	Статическая инструментация ядра (трассировщик событий)	14.5
Зонды kprobes	Динамическая инструментация ядра (трассировщик событий)	14.6
Зонды uprobes	Динамическая инструментация программного кода в пространстве пользователя (трассировщик событий)	14.7
function_graph	Трассировщик вызовов функций ядра с выводом графа вызовов дочерних функций	14.8
wakeup	Измеряет максимальную задержку планировщика процессорного времени	—
wakeup_rt	Измеряет максимальную задержку планировщика процессорного времени для задач реального времени (real-time, rt)	—
irqsoff	Трассирует события отключения прерываний, указывает их местоположение в коде и продолжительность (продолжительность выполнения с отключенными прерываниями) ¹	—
preemptoff	Трассирует события отключения вытеснения, указывает их местоположение в коде и продолжительность	—
preemptirqsoff	Трассировщик, объединяющий возможности irqsoff и preemptoff	—
blk	Трассировщик блочного ввода/вывода (используется командой blktrace(8))	—
hwlat	Трассировщик задержек оборудования: может обнаруживать внешние возмущения, вызывающие задержки	14.9
mmiotrace	Трассирует все обращения модуля к оборудованию	—
nop	Специальный трассировщик, отключающий все другие трассировщики	—

Получить список трассировщиков Ftrace, доступных в вашей версии ядра, можно командой:

```
# cat /sys/kernel/debug/tracing/available_tracers
hwlat blk mmiotrace function_graph wakeup_d1 wakeup_rt wakeup function nop
```

Она использует интерфейс файловой системы tracefs, смонтированной в /sys, о котором идет речь в следующем разделе. В последующих разделах говорится о профилировщиках, трассировщиках и инструментах, которые их используют.

Если хотите побыстрее познакомиться с инструментами на основе Ftrace, сразу переходите к разделу 14.13 «perf-tools», где также описывается инструмент funcgraph (8), показанный выше.

¹ Этот трассировщик (а также трассировщики preemptoff и preemptirqsoff) требует, чтобы ядро было скомпилировано с параметром CONFIG_PREEMPTIRQ_EVENTS.

В будущих версиях ядра в Ftrace могут добавиться дополнительные профилировщики и трассировщики: загляните в документацию с описанием Ftrace в исходном коде Linux в файле Documentation/trace/ftrace.rst [Rostedt 08].

14.2. TRACEFS (/SYS)

Интерфейсом для доступа к возможностям Ftrace служит файловая система tracefs. Она должна быть смонтирована в каталог /sys/kernel/tracing:

```
mount -t tracefs tracefs /sys/kernel/tracing
```

Ftrace изначально является частью файловой системы debugfs, но впоследствии был выделен в отдельные файлы в tracefs. Файловая система debugfs по-прежнему сохраняет исходную структуру каталогов, монтируя tracefs как подкаталог tracing. Узнать точки монтирования debugfs и tracefs можно командой:

```
# mount -t debugfs,tracefs
debugfs on /sys/kernel/debug type debugfs (rw,relatime)
tracefs on /sys/kernel/debug/tracing type tracefs (rw,relatime)
```

Этот вывод получен в Ubuntu 19.10 и показывает, что tracefs смонтирована в /sys/kernel/debug/tracing. Примеры в следующих разделах используют именно эту точку монтирования, поскольку она все еще распространена, но в будущем каталог должен измениться на /sys/kernel/tracing.

Обратите внимание: если у вас не получается смонтировать tracefs, то, возможно, ваше ядро скомпилировано с выключенными параметрами поддержки Ftrace (CONFIG_FTRACE и т. д.).

14.2.1. Содержимое tracefs

В смонтированной файловой системе tracefs — в каталоге tracing — должны быть доступны управляющие и выходные файлы:

```
# ls -F /sys/kernel/debug/tracing
available_events          max_graph_depth          stack_trace_filter
available_filter_functions options/                  synthetic_events
available_tracers        per_cpu/                 timestamp_mode
buffer_percent           printk_formats           trace
buffer_size_kb           README                   trace_clock
buffer_total_size_kb     saved_cmdlines           trace_marker
current_tracer           saved_cmdlines_size      trace_marker_raw
dynamic_events           saved_tgids              trace_options
dyn_ftrace_total_info    set_event                trace_pipe
enabled_functions        set_event_pid            trace_stat/
error_log                set_ftrace_filter        tracing_cpumask
events/                  set_ftrace_notrace       tracing_max_latency
free_buffer              set_ftrace_pid           tracing_on
function_profile_enabled set_graph_function       tracing_thresh
```

hwlat_detector/ instances/ kprobe_events kprobe_profile	set_graph_notrace snapshot stack_max_size stack_trace	uprobe_events uprobe_profile
--	--	---------------------------------

Назначение многих из них интуитивно понятно. Некоторые ключевые файлы и каталоги перечислены в табл. 14.3.

Таблица 14.3. Ключевые файлы tracefs

Файл	Доступен для	Описание
available_tracers	Чтения	Список доступных трассировщиков (см. табл. 14.2)
current_tracer	Чтения/записи	Показывает текущий активный трассировщик
function_profile_enabled	Чтения/записи	Активирует профилировщик function
available_filter_functions	Чтения	Список функций, доступных для трассировки
set_ftrace_filter	Чтения/записи	Выбирает функции для трассировки
tracing_on	Чтения/записи	Включает/выключает вывод содержимого кольцевого буфера
trace	Чтения/записи	Вывод трассировщиков (содержимое кольцевого буфера)
trace_pipe	Чтения	Вывод трассировщиков: эта версия является потребителем информации, передаваемой трассировщиками, и служит для потоковой трассировки
trace_options	Чтения/записи	Параметры настройки трассировщика и выходного буфера
trace_stat (каталог)	Чтения/записи	Вывод профилировщика function
kprobe_events	Чтения/записи	Активирует конфигурацию kprobe
uprobe_events	Чтения/записи	Активирует конфигурацию uprobe
events (каталог)	Чтения/записи	Файлы для управления трассировщиками: точками трассировки, зондами kprobes и uprobes
instances (каталог)	Чтения/записи	Экземпляры Ftrace для разных пользователей, работающих одновременно

Этот интерфейс /sys задокументирован в файле Documentation/trace/ftrace.rst [Rostedt 08]. Его можно использовать непосредственно из командной оболочки или с помощью внешних интерфейсов и библиотек. Например, чтобы увидеть, используются ли какие-либо трассировщики Ftrace в текущий момент, можно прочитать файл current_tracer командой cat(1):

```
# cat /sys/kernel/debug/tracing/current_tracer
nop
```

В этом примере команда вывела `por` (no operation — нет операции), то есть никакие трассировщики не используются. Чтобы включить трассировщик, нужно записать его имя в этот файл. Вот пример включения трассировщика `blk`:

```
# echo blk > /sys/kernel/debug/tracing/current_tracer
```

Другие управляющие и выходные файлы `Ftrace` тоже можно использовать в командах `echo(1)` и `cat(1)`. Это означает, что `Ftrace` практически не имеет зависимостей (достаточно обычной командной оболочки¹).

Стивен Ростедт создал `Ftrace` для собственного использования, когда разрабатывал свой набор исправлений поддержки режима реального времени в ядре; изначально не предполагалась одновременная работа с `Ftrace` нескольких пользователей. Например, в файл `current_tracer` можно было записать имя только одного трассировщика. Поддержка одновременной работы нескольких пользователей была добавлена позже в виде экземпляров, которые можно создавать в каталоге «instances». У каждого экземпляра есть свой файл `current_tracer` и выходные файлы, поэтому он выполняет трассировку независимо.

В следующих разделах (14.3–14.10) показаны дополнительные примеры использования интерфейса `/sys`, а в последующих разделах (14.11–14.13) будут представлены внешние интерфейсы, основанные на интерфейсе `/sys: trace-cmd`, подкоманда `ftrace` профилировщика `perf(1)` и `perf-tools`.

14.3. ПРОФИЛИРОВЩИК ФУНКЦИЙ

Профилировщик функций предоставляет статистики, характеризующие функции ядра, и помогает получить представление об их использовании и определить наиболее медленные из них. Я часто использую профилировщик функций в качестве отправной точки, чтобы понять, как работает код ядра с данной рабочей нагрузкой, — отчасти потому, что он очень эффективен и имеет относительно небольшой оверхед. С его помощью я могу выбрать функции для анализа с применением более дорогостоящей трассировки событий. Для работы этого профилировщика ядро должно быть скомпилировано с параметром `CONFIG_FUNCTION_PROFILER = y`.

Принцип действия профилировщика функций основан на использовании встроенных вызовов механизма профилирования в начале каждой функции ядра. Примерно так работают профилировщики компилятора, такие как параметр `-pg` в `gcc(1)`, вставляющий вызовы `mcount()` для использования с `gprof(1)`. Начиная с версии `gcc(1)` 4.6, функция `mcount()` называется `__fentry__()`. На первый взгляд кажется, что добавление вызовов в *каждую* функцию ядра должно повлечь значительный

¹ `echo(1)` — это встроенная команда оболочки, команду `cat(1)` можно симитировать в виде функции: `function shellcat { (while read line; do echo "$line"; done) < $1; }`. Также можно использовать `busybox`, чтобы включить командную оболочку, `cat(1)` и другие базовые команды.

оверхед, особенно если учесть, что подобная возможность используется нечасто, однако проблема оверхеда была решена: когда вызовы `__fentry__()` не используются, они заменяются быстрыми инструкциями `por` и включаются только при необходимости [Gregg 19f].

Ниже показано, как профилировщик функций использует интерфейс `tracefs` в `/sys`. Для справки ниже показано исходное неактивное состояние профилировщика функций:

```
# cd /sys/kernel/debug/tracing
# cat set_ftrace_filter
#### all functions enabled ####
# cat function_profile_enabled
0
```

Следующие команды (выполняемые в том же каталоге) используют профилировщик функций для подсчета вызовов всех функций ядра, имена которых начинаются с «tcp», в течение примерно 10 с:

```
# echo 'tcp*' > set_ftrace_filter
# echo 1 > function_profile_enabled
# sleep 10
# echo 0 > function_profile_enabled
# echo > set_ftrace_filter
```

Команда `sleep(1)` использовалась для установки (приблизительной) продолжительности профилирования. Команды, следующие за ней, остановили профилирование функций и сбросили фильтр. Совет: обязательно используйте «0 >», а не «0>» — это далеко не одно и то же, последнее — это перенаправление файлового дескриптора 0. Избегайте и комбинации «1>», поскольку она определяет перенаправление файлового дескриптора 1.

Статистики профиля теперь можно прочитать из каталога `trace_stat`, который хранит их в файлах «function» отдельно для каждого процессора. Этот пример был получен в системе с двумя процессорами. Следующий пример использует команду `head(1)`, только чтобы показать первые десять строк из каждого файла:

```
# head trace_stat/function*
==> trace_stat/function0 <==
Function                Hit      Time      Avg      s^2
-----
tcp_sendmsg             955912    2788479 us    2.917 us    3734541 us
tcp_sendmsg_locked      955912    2248025 us    2.351 us    2600545 us
tcp_push                 955912    852421.5 us    0.891 us    1057342 us
tcp_write_xmit          926777    674611.1 us    0.727 us    1386620 us
tcp_send_mss            955912    504021.1 us    0.527 us    95650.41 us
tcp_current_mss         964399    317931.5 us    0.329 us    136101.4 us
tcp_poll                966848    216701.2 us    0.224 us    201483.9 us
tcp_release_cb          956155    102312.4 us    0.107 us    188001.9 us

==> trace_stat/function1 <==
```

Function	Hit	Time	Avg	s^2
-----	---	----	---	---
tcp_sendmsg	317935	936055.4 us	2.944 us	13488147 us
tcp_sendmsg_locked	317935	770290.2 us	2.422 us	8886817 us
tcp_write_xmit	348064	423766.6 us	1.217 us	226639782 us
tcp_push	317935	310040.7 us	0.975 us	4150989 us
tcp_tasklet_func	38109	189797.2 us	4.980 us	2239985 us
tcp_tsq_handler	38109	180516.6 us	4.736 us	2239552 us
tcp_tsq_write.part.0	29977	173955.7 us	5.802 us	1037352 us
tcp_send_mss	317935	165881.9 us	0.521 us	352309.0 us

В столбцах выводятся: имя функции (Function), количество вызовов (Hit), суммарное время выполнения функции (Time), среднее время выполнения функции (Avg) и стандартное отклонение (s^2). Согласно полученным результатам, функция tcp_sendmsg() использовалась чаще других на обоих процессорах. Она была вызвана более 955 тысяч раз на CPU0 и более 317 тысяч раз на CPU1. Средняя продолжительность ее выполнения составила 2,9 мкс.

Профилирование функций добавляет небольшой оверхед к профилируемым функциям. Если set_fttrace_filter оставить пустым, то профилироваться будут все функции ядра (о чем предупреждает начальное содержимое файла, показанное выше: «all functions enabled» (все функции включены)). Помните об этом и старайтесь использовать фильтр функций, чтобы ограничить оверхед.

Внешние интерфейсы Ftrace, с которыми мы познакомимся позже, автоматизируют эти шаги и могут объединять результаты, полученные для отдельных процессоров в общесистемную сводку.

14.4. ТРАССИРОВЩИК FUNCTION

Трассировщик function выводит подробные сведения о вызовах функций для каждого события и использует механизм инструментации, описанный в предыдущем разделе. Он позволяет увидеть последовательность вызовов функций, заметить временные закономерности, а также узнать имя и идентификатор PID процесса, выполняющегося на процессоре, которому могут принадлежать эти функции. Оверхед на трассировку функций выше, чем на профилирование, поэтому ее лучше проводить для исследования функций, вызываемых относительно нечасто (менее 1000 раз в секунду). С этой целью перед трассировкой можно выполнить профилирование функций, как описано в предыдущем разделе, чтобы узнать, как часто они вызываются.

На рис. 14.2 показаны ключевые файлы tracefs, участвующие в трассировке функций.

Окончательный вывод трассировщика доступен в файлах trace или trace_pipe, описанных в следующих разделах. Оба этих интерфейса поддерживают также возможность очистить выходной буфер (это объясняет наличие стрелок на рис. 14.2, ведущих обратно в буфер).

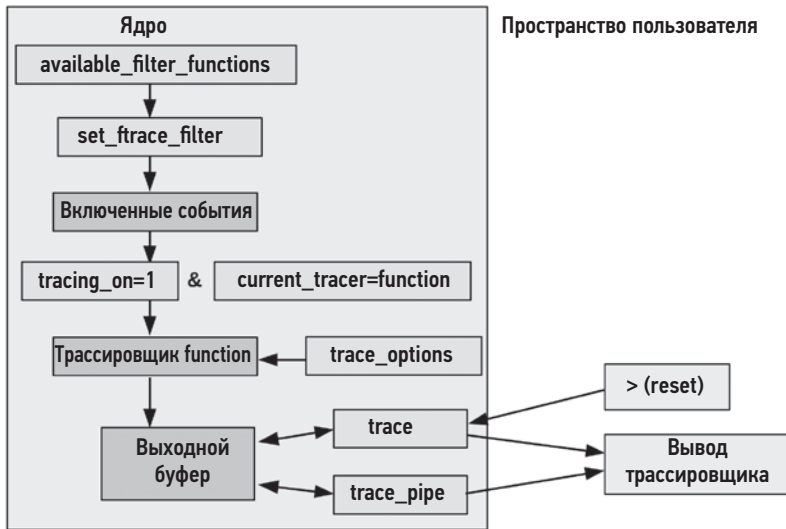


Рис. 14.2. Файлы в tracefs, участвующие в трассировке функций

14.4.1. Использование файла trace

В примере — получение результатов трассировки функций из выходного файла `trace`. Здесь для справки показано исходное неактивное состояние трассировщика `function`:

```
# cd /sys/kernel/debug/tracing
# cat set_ftrace_filter
#### all functions enabled ####
# cat current_tracer
nop
```

То есть в настоящий момент никакие другие трассировщики не используются.

В этом примере трассируются все функции ядра, имена которых оканчиваются на «sleep», и события в итоге сохраняются в файл `/tmp/out.trace01.txt`. Фиктивная команда `sleep(1)` используется для сбора данных трассировки не менее чем 10 с. Последовательность команд завершается выключением трассировщика `function` и возвратом системы в нормальное состояние:

```
# cd /sys/kernel/debug/tracing
# echo 1 > tracing_on
# echo '*sleep' > set_ftrace_filter
# echo function > current_tracer
# sleep 10
# cat trace > /tmp/out.trace01.txt
# echo nop > current_tracer
# echo > set_ftrace_filter
```

Запись числа 1 в `tracing_on` требуется не всегда (например, в моей системе Ubuntu этот файл содержит 1 по умолчанию). Я включил эту команду на случай, если в вашей системе этот файл содержит 0.

Фиктивная команда `sleep(1)` попадет в вывод трассировщика при трассировке вызовов функций «sleep»:

```
# more /tmp/out.trace01.txt
# tracer: function
#
# entries-in-buffer/entries-written: 57/57 #P:2
#
#
#          _-----> irqsoff
#          / _-----> need-resched
#          | / _-----> hardirq/softirq
#          || / _--> preempt-depth
#          ||| / _-----> delay
#
#          TASK-PID   CPU#  ||||   TIMESTAMP   FUNCTION
#          |   |     |   |   |   |   |   |
multipathd-348 [001] .... 332762.532877: __x64_sys_nanosleep <-do_syscall_64
multipathd-348 [001] .... 332762.532879: hrtimer_nanosleep <-
__x64_sys_nanosleep
multipathd-348 [001] .... 332762.532880: do_nanosleep <-hrtimer_nanosleep
sleep-4203 [001] .... 332762.722497: __x64_sys_nanosleep <-do_syscall_64
sleep-4203 [001] .... 332762.722498: hrtimer_nanosleep <-
__x64_sys_nanosleep
sleep-4203 [001] .... 332762.722498: do_nanosleep <-hrtimer_nanosleep
multipathd-348 [001] .... 332763.532966: __x64_sys_nanosleep <-do_syscall_64
[...]
```

Вывод включает заголовки полей и метаданные трассировки. В этом примере виден процесс с именем `multipathd` и идентификатором 348, который вызывает функции `sleep`, а также команду `sleep(1)`. Последние два поля показывают текущую функцию и вызвавшую ее родительскую функцию. Например, в первой строке текущей является функция `__x64_sys_nanosleep()`, и она была вызвана функцией `do_syscall_64()`.

Файл `trace` — это интерфейс к буферу событий трассировки. При чтении из файла вы получаете все содержимое буфера. Буфер можно очистить, записав в файл символ перевода строки:

```
# > trace
```

Буфер трассировки также очищается, когда в файл `current_tracer` снова записывается `por`, как я сделал это в примере выше, чтобы выключить трассировку, и при использовании файла `trace_pipe`.

14.4.2. Использование файла `trace_pipe`

Файл `trace_pipe` — это еще один интерфейс для чтения буфера трассировки. При чтении из этого файла возвращается бесконечный поток событий. Этот интерфейс также выступает в роли потребителя событий, удаляя их из буфера трассировки.

Вот пример использования `trace_pipe` для наблюдения за событиями `sleep` в реальном времени:

```
# echo '*sleep' > set_ftrace_filter
# echo function > current_tracer
# cat trace_pipe
multipathd-348    [001] .... 332624.519190: __x64_sys_nanosleep <-do_syscall_64
multipathd-348    [001] .... 332624.519192: hrtimer_nanosleep <-
__x64_sys_nanosleep
multipathd-348    [001] .... 332624.519192: do_nanosleep <-hrtimer_nanosleep
multipathd-348    [001] .... 332625.519272: __x64_sys_nanosleep <-do_syscall_64
multipathd-348    [001] .... 332625.519274: hrtimer_nanosleep <-
__x64_sys_nanosleep
multipathd-348    [001] .... 332625.519275: do_nanosleep <-hrtimer_nanosleep
cron-504          [001] .... 332625.560150: __x64_sys_nanosleep <-do_syscall_64
cron-504          [001] .... 332625.560152: hrtimer_nanosleep <-
__x64_sys_nanosleep
cron-504          [001] .... 332625.560152: do_nanosleep <-hrtimer_nanosleep
^C
# echo nop > current_tracer
# echo > set_ftrace_filter
```

Вывод показывает большое количество вызовов функций `sleep` в процессах `multipathd` и `cron`. Значения полей в `trace_pipe` такие же, как и в `trace` из примера выше, нет только заголовков.

Файл `trace_pipe` удобно использовать для наблюдения за относительно редкими событиями. Информацию о событиях, следующих с большой частотой, лучше записать в файл с помощью файла `trace`, из примера выше, чтобы затем без спешки ее проанализировать.

14.4.3. Параметры

Ftrace поддерживает возможность настройки представления результатов в файле `trace` с помощью файла `trace_options`, который находится в каталоге `options`. Вот пример отключения столбца флагов (в предыдущем выводе это было поле с многоточием «...»):

```
# echo 0 > options/irq-info
# cat trace
# tracer: function
#
# entries-in-buffer/entries-written: 3300/3300 #P:2
#
#
# TASK-PID      CPU#  TIMESTAMP  FUNCTION
#   |   |          |   |
multipathd-348  [001]  332762.532877: __x64_sys_nanosleep <-do_syscall_64
multipathd-348  [001]  332762.532879: hrtimer_nanosleep <-__x64_sys_nanosleep
multipathd-348  [001]  332762.532880: do_nanosleep <-hrtimer_nanosleep
[...]
```

Как видите, теперь флагов в выводе нет. Их можно вернуть командой:

```
# echo 1 > options/irq-info
```


Есть множество других параметров, список которых можно получить из каталога `options`. У них весьма говорящие имена.

```
# ls options/
annotate      funcgraph-abstime  hex              stacktrace
bin           funcgraph-cpu      irq-info         sym-addr
blk_cgname     funcgraph-duration latency-format    sym-offset
blk_cgroup     funcgraph-irqs     markers          sym-userobj
blk_classic    funcgraph-overhead overwrite         test_nop_accept
block         funcgraph-overflow print-parent     test_nop_refuse
context-info   funcgraph-proc     printk-msg-only  trace_printk
disable_on_free funcgraph-tail      raw             userstacktrace
display-graph  function-fork       record-cmd       verbose
event-fork     function-trace      record-tgid
func_stack_trace graph-time          sleep-time
```

В число поддерживаемых параметров входят `stacktrace` и `userstacktrace`, которые добавляют в выходные данные трассировки стека в пространствах ядра и пользователя: это поможет выяснить причины вызова интересующих функций. Все эти параметры описаны в документации `Ftrace` в исходном коде Linux [Rostedt 08].

14.5. ТОЧКИ ТРАССИРОВКИ

Точки трассировки представляют механизм статической инструментации ядра и были описаны в главе 4 «Инструменты наблюдения», в разделе 4.3.5 «Точки трассировки». Технически точки трассировки — это просто вызовы функций трассировки, помещенные в исходный код ядра. Они используются интерфейсом к событиям трассировки, который определяет и форматирует их аргументы. События трассировки доступны в `tracefs` посредством тех же управляющих и выходных файлов `Ftrace`.

Пример ниже активирует точку трассировки `block:block_rq_issue` и выводит информацию о вызовах в реальном времени. Он заканчивается выключением точки трассировки:

```
# cd /sys/kernel/debug/tracing
# echo 1 > events/block/block_rq_issue/enable
# cat trace_pipe
    sync-4844 [001] .... 343996.918805: block_rq_issue: 259,0 WS 4096 ()
2048 + 8 [sync]
    sync-4844 [001] .... 343996.918808: block_rq_issue: 259,0 WSM 4096 ()
10560 + 8 [sync]
    sync-4844 [001] .... 343996.918809: block_rq_issue: 259,0 WSM 4096 ()
38424 + 8 [sync]
    sync-4844 [001] .... 343996.918809: block_rq_issue: 259,0 WSM 4096 ()
4196384 + 8 [sync]
    sync-4844 [001] .... 343996.918810: block_rq_issue: 259,0 WSM 4096 ()
4462592 + 8 [sync]
^C
# echo 0 > events/block/block_rq_issue/enable
```

Первые пять столбцов совпадают со столбцами в примере из раздела 4.6.4 и включают: имя процесса, дефис «-», идентификатор процесса PID, идентификатор процессора, флаги, отметку времени (секунды) и имя события. Остальные поля — это строка формата для точки трассировки, описанная в разделе 4.3.5.

Как мы видим, точки трассировки имеют управляющие файлы в структуре каталогов events. Каждому семейству точек трассировки соответствует свой каталог (например, «block») с подкаталогами для каждого события (например, «block_rq_issue»). Вот содержимое этого каталога:

```
# ls events/block/block_rq_issue/  
enable filter format hist id trigger
```

Эти управляющие файлы описаны в исходном коде Linux в файле Documentation/trace/events.rst [Ts'o 20]. В этом примере для включения и выключения точки трассировки использовался файл enable. Также есть файлы для поддержки возможности фильтрации и запуска.

14.5.1. Фильтрация

Фильтр включает запись события, только если логическое выражение принимает истинное значение. Синтаксис определения таких выражений довольно ограничен:

поле оператор значение

Поле (field) — это определение из строки формата, описанной в разделе 4.3.5, в подразделе «Аргументы и строка формата точки трассировки» (эти поля также выводятся в строке формата, описанной ранее). Для чисел можно использовать операторы: ==, !=, <, <=, >, >=, &; а для строк: ==, !=, ~. Оператор «тильда» (~) выполняет сопоставление в стиле подстановки групповых символов: *, ?, []. Логические выражения можно группировать, заключая в круглые скобки, и объединять с помощью: &&, ||.

Вот пример установки фильтра для уже включенной точки трассировки block:block_rq_insert с целью оставить только событий, поле bytes в которых больше 64 Кбайт:

```
# echo 'bytes > 65536' > events/block/block_rq_insert/filter  
# cat trace_pipe  
kworker/u4:1-7173 [000] .... 378115.779394: block_rq_insert: 259,0 W 262144 ()  
5920256 + 512 [kworker/u4:1]  
kworker/u4:1-7173 [000] .... 378115.784654: block_rq_insert: 259,0 W 262144 ()  
5924336 + 512 [kworker/u4:1]  
kworker/u4:1-7173 [000] .... 378115.789136: block_rq_insert: 259,0 W 262144 ()  
5928432 + 512 [kworker/u4:1]  
^C
```

Теперь вывод содержит только события ввода/вывода больших блоков данных.

```
# echo 0 > events/block/block_rq_insert/filter
```

Эта команда echo 0 сбрасывает фильтр.

14.5.2. Триггеры

Триггер запускает дополнительную команду трассировки, когда возникает событие. Эта команда может включать или выключать другой трассировщик, выводить трассировку стека или делать снимок буфера трассировки. Получить список команд, доступных для триггеров, можно из файла `trigger`, если в данный момент триггер не установлен. Например:

```
# cat events/block/block_rq_issue/trigger
# Available triggers:
# traceon traceoff snapshot stacktrace enable_event disable_event enable_hist
disable_hist hist
```

Одно из применений триггеров — вывод событий, приведших к состоянию ошибки: триггер можно поместить в состояние ошибки, которое либо отключает трассировку (`traceoff`), чтобы не затереть предыдущие события в буфере трассировки, либо делает снимок (`snapshot`), чтобы сохранить его.

Триггеры можно комбинировать с фильтрами, описанными в предыдущем разделе, с помощью ключевого слова `if`. Это бывает нужно для проверки ошибочного состояния или обнаружения интересующего события. Вот как можно остановить запись событий, когда в очередь ввода/вывода будет поставлен блок размером более 64 Кбайт:

```
# echo 'traceoff if bytes > 65536' > events/block/block_rq_insert/trigger
```

Более сложные действия можно выполнить с помощью триггеров `hist` из раздела 14.10 «Триггеры `hist`».

14.6. ЗОНДЫ KPROBES

Kprobes — это механизм динамической инструментации ядра, который был представлен в главе 4 «Инструменты наблюдения», в разделе 4.3.6 «kprobes». Зонды kprobes генерируют события для трассировщиков, которые совместно с Ftrace используют общие выходные и управляющие файлы `tracefs`. Механизм kprobes похож на трассировщик `function` в Ftrace, описанный в разделе 14.4, — он тоже трассирует функции ядра. Но еще он позволяет инструментировать инструкции с некоторым смещением от начала функции и может сообщать ее аргументы и возвращаемое значение.

В этом разделе рассмотрена трассировка событий kprobe и профилировщик kprobe.

14.6.1. Трассировка событий

В примере ниже показана инструментация функции ядра `do_nanosleep()` с помощью механизма kprobes:

```
# echo 'p:brendan do_nanosleep' >> kprobe_events
# echo 1 > events/kprobes/brendan/enable
# cat trace_pipe
```

```

multipathd-348 [001] .... 345995.823380: brendan: (do_nanosleep+0x0/0x170)
multipathd-348 [001] .... 345996.823473: brendan: (do_nanosleep+0x0/0x170)
multipathd-348 [001] .... 345997.823558: brendan: (do_nanosleep+0x0/0x170)
^C
# echo 0 > events/kprobes/brendan/enable
# echo '-:brendan' >> kprobe_events

```

Зонд kprobe создается и удаляется путем записи специальных команд в файл kprobe_events. После создания файл зонда появляется в events вместе с точками трассировки и может использоваться аналогичным образом.

Синтаксис зондов kprobe полностью объясняется в исходном коде ядра в файле Documentation/trace/kprobetrace.rst [Hiramatsu 20]. Механизм kprobes выполняет трассировку точек входа и выхода, а также инструкции с некоторым смещением от начала функции. Вот краткое описание:

```

p[:[GRP/]EVENT] [MOD:]SYM[+offs]|MEMADDR [FETCHARGS] : Set a probe
r[MAXACTIVE][:[GRP/]EVENT] [MOD:]SYM[+0] [FETCHARGS] : Set a return probe
-:[GRP/]EVENT                                         : Clear a probe

```

В моем примере строка «p:brendan do_nanosleep» создает зонд (p:) с именем «brendan» для символа ядра do_nanosleep(). Строка «-:brendan» удаляет зонд «brendan».

Возможность выбора произвольных имен оказалась весьма полезной в случаях одновременного использования этого механизма несколькими пользователями. Трассировщик BCC (описанный в главе 15 «BPF», в разделе 15.1 «BCC») использует имена, включающие имя трассируемой функции, строку «bcc» и BCC PID. Например:

```

# cat /sys/kernel/debug/tracing/kprobe_events
p:kprobes/p_blk_account_io_start_bcc_19454 blk_account_io_start
p:kprobes/p_blk_mq_start_request_bcc_19454 blk_mq_start_request

```

Обратите внимание, что в новых ядрах трассировщик BCC перешел на интерфейс, основанный на perf_event_open(2), и использует kprobes вместо файла kprobe_events (и события, активированные с помощью perf_event_open(2), не отображаются в kprobe_events).

14.6.2. Аргументы

В отличие от трассировщика function (раздел 14.4 «Трассировщик function»), зонды kprobes позволяют исследовать аргументы и возвращаемые значения функций. Для примера взгляните на объявление функции do_nanosleep() из файла kernel/time/hrtimer с выделенными типами переменных-аргументов:

```

static int __sched do_nanosleep(struct hrtimer_sleeper *t, enum hrtimer_mode mode)
{
[...]
```

А вот как выглядит извлечение двух первых аргументов в системе Intel x86_64 и их вывод в шестнадцатеричном формате (по умолчанию):

```
# echo 'p:brendan do_nanosleep hrtimer_sleeper=$arg1 hrtimer_mode=$arg2' >>
kprobe_events
# echo 1 > events/kprobes/brendan/enable
# cat trace_pipe
    multipathd-348 [001] .... 349138.128610: brendan: (do_nanosleep+0x0/0x170)
hrtimer_sleeper=0xfffffaa6a4030be80 hrtimer_mode=0x1
    multipathd-348 [001] .... 349139.128695: brendan: (do_nanosleep+0x0/0x170)
hrtimer_sleeper=0xfffffaa6a4030be80 hrtimer_mode=0x1
    multipathd-348 [001] .... 349140.128785: brendan: (do_nanosleep+0x0/0x170)
hrtimer_sleeper=0xfffffaa6a4030be80 hrtimer_mode=0x1
^C
# echo 0 > events/kprobes/brendan/enable
# echo '-:brendan' >> kprobe_events
```

В описания событий в первой строке добавлены дополнительные элементы: например, строка «hrtimer_sleeper=\$arg1» извлекает первый аргумент функции и связывает его с выбранным именем «hrtimer_sleeper». Имена и значения аргументов выделены в выводе.

Возможность доступа к аргументам функций как \$arg1, \$arg2 и т. д. была добавлена в Linux 4.20. В предыдущих версиях требовалось использовать имена регистров¹. Вот эквивалентное определение зонда kprobe с использованием имен регистров:

```
# echo 'p:brendan do_nanosleep hrtimer_sleeper=%di hrtimer_mode=%si' >> kprobe_events
```

Чтобы использовать имена регистров, нужно знать тип процессора и используемое соглашение о вызове функций. В архитектуре используется AMD64 ABI [Matz 13], поэтому первые два аргумента доступны в регистрах rdi и rsi². Этот же синтаксис использует perf(1). Я уже приводил в главе 13 «perf» в разделе 13.7.2 «uprobes» довольно сложный пример разыменования указателя на строку.

14.6.3. Возвращаемые значения

Возвращаемое значение доступно под специальным псевдонимом \$retval при использовании kretprobes. Вот как можно получить значение, возвращаемое функцией do_nanosleep():

```
# echo 'r:brendan do_nanosleep ret=$retval' >> kprobe_events
# echo 1 > events/kprobes/brendan/enable
# cat trace_pipe
    multipathd-348 [001] d... 349782.180370: brendan:
```

¹ Это также может потребоваться в аппаратных архитектурах, где псевдонимы еще не были добавлены.

² Обобщенные соглашения о вызовах для различных процессоров можно найти на странице справочного руководства man для syscall(2). Выдержка из этой страницы приводится в разделе 14.13.4 «Однострочные сценарии для perf-tools».

```
(hrtimer_nanosleep+0xce/0x1e0 <- do_nanosleep) ret=0x0
multipathd-348 [001] d... 349783.180443: brendan:
(hrtimer_nanosleep+0xce/0x1e0 <- do_nanosleep) ret=0x0
multipathd-348 [001] d... 349784.180530: brendan:
(hrtimer_nanosleep+0xce/0x1e0 <- do_nanosleep) ret=0x0
^C
# echo 0 > events/kprobes/brendan/enable
# echo '-:brendan' >> kprobe_events
```

Как показывает этот вывод, в период, когда выполнялась трассировка, `do_nanosleep()` всегда возвращала 0 (успех).

14.6.4. Фильтры и триггеры

Фильтры и триггеры доступны в каталоге `events/kprobes/...`, по аналогии с точками трассировки (см. раздел 14.5 «Точки трассировки»). Вот файл формата для зонда `kprobe` в `do_nanosleep()` с аргументами (из раздела 14.6.2 «Аргументы»):

```
# cat events/kprobes/brendan/format
name: brendan
ID: 2024
format:
    field:unsigned short common_type; offset:0; size:2; signed:0;
    field:unsigned char common_flags; offset:2; size:1; signed:0;
    field:unsigned char common_preempt_count; offset:3; size:1; signed:0;
    field:int common_pid; offset:4; size:4; signed:1;

    field:unsigned long __probe_ip; offset:8; size:8; signed:0;
    field:u64 hrtimer_sleeper; offset:16; size:8; signed:0;
    field:u64 hrtimer_mode; offset:24; size:8; signed:0;

print fmt: "(%lx) hrtimer_sleeper=0x%lx hrtimer_mode=0x%lx", REC->__probe_ip, REC-
>hrtimer_sleeper, REC->hrtimer_mode
```

Обратите внимание, что выбранные мной имена переменных `hrtimer_sleeper` и `hrtimer_mode` отображаются как поля, которые можно использовать в фильтрах. Например:

```
# echo 'hrtimer_mode != 1' > events/kprobes/brendan/filter
```

Эта команда будет трассировать только вызовы `do_nanosleep()`, где в аргументе `hrtimer_mode` передается значение, отличное от 1.

14.6.5. Профилировщик kprobe

После установки зондов `kprobes` Ftrace начинает подсчитывать соответствующие им события. Эти счетчики доступны для чтения в файле `kprobe_profile`. Например:

```
# cat /sys/kernel/debug/tracing/kprobe_profile
```

<code>p_blk_account_io_start_bcc_19454</code>	1808	0
<code>p_blk_mq_start_request_bcc_19454</code>	677	0
<code>p_blk_account_io_completion_bcc_19454</code>	521	11
<code>p_kbd_event_1_bcc_1119</code>	632	0

В первом столбце выводится имя зонда (его определение можно увидеть в файле `kprobe_events`), во втором — счетчик попаданий и в третьем — счетчик промахов (когда зонд сработал, но затем возникла ошибка и событие не было записано, то есть было упущено).

Получить количество вызовов функций можно с помощью профилировщика `function` (раздел 14.3). Однако я обнаружил, что профилировщик `kprobe` намного удобнее для проверки постоянно включенных зондов `kprobes`, используемых ПО для мониторинга, в случаях, когда функции вызываются очень часто и их профилирование с помощью `function` нежелательно.

14.7. ЗОНДЫ UPROBES

Uprobes — это механизм динамической инструментации кода в пространстве пользователя. Он был представлен в главе 4 «Инструменты наблюдения», в разделе 4.3.7 «uprobes». Зонды uprobes генерируют события для трассировщиков, которые совместно с `Ftrace` используют общие выходные и управляющие файлы `tracefs`.

В этом разделе рассматриваются трассировка событий `uprobe` и профилировщик `uprobe`.

14.7.1. Трассировка событий

Управление зондами uprobes производится с помощью файла `uprobe_events`, синтаксис которого описан в исходном коде Linux в файле `Documentation/trace/uprobetracer.rst` [Dronamraju 20]. Вот краткое описание:

```
p[:[GRP/]EVENT] PATH:OFFSET [FETCHARGS] : Set a uprobe
r[:[GRP/]EVENT] PATH:OFFSET [FETCHARGS] : Set a return uprobe (uretprobe)
-:[GRP/]EVENT                             : Clear uprobe or uretprobe event
```

Современный синтаксис требует указывать пути и смещения для зондов. Ядро не имеет информации о символах в ПО пользовательского уровня, поэтому смещение должно быть определено и передано ядру с помощью инструментов, действующих в пространстве пользователя.

Вот пример использования uprobes для инструментации функции `readline()` из командной оболочки `bash`(1). Сначала определяется смещение символа:

```
# readelf -s /bin/bash | grep -w readline
882: 00000000000b61e0 153 FUNC GLOBAL DEFAULT 14 readline
# echo 'p:brendan /bin/bash:0xb61e0' >> uprobe_events
# echo 1 > events/uprobes/brendan/enable
# cat trace_pipe
bash-3970 [000] d... 347549.225818: brendan: (0x55d0857b71e0)
bash-4802 [000] d... 347552.666943: brendan: (0x560bcc1821e0)
bash-4802 [000] d... 347552.799480: brendan: (0x560bcc1821e0)
^C
# echo 0 > events/uprobes/brendan/enable
# echo '-:brendan' >> uprobe_events
```

ВНИМАНИЕ: если по ошибке указать смещение, попадающее в середину инструкции, вы повредите целевой процесс (а в случае разделяемой библиотеки — все процессы, которые ее используют). Показанный способ поиска смещения символа с помощью `readelf(1)` может не сработать, если целевой двоичный файл был скомпилирован как позиционно-независимый исполняемый файл (Position-Independent Executable, PIE) с рандомизацией размещения адресного пространства (Address Space Layout Randomization, ASLR). Рекомендую не использовать этот интерфейс: переключитесь на трассировщик более высокого уровня, который сам позаботится о поиске символов (например, BCC или `bpfttrace`).

14.7.2. Аргументы и возвращаемые значения

Доступ к аргументам и возвращаемым значениям выполняется так же, как при использовании механизма `kprobes` (см. раздел 14.6 «Зонды `kprobes`»). Аргументы и возвращаемые значения можно получить, указав их при создании зонда `uprobe`. Синтаксис описывается в файле `uprobetracer.rst` [Dronamraju 20].

14.7.3. Фильтры и триггеры

Фильтры и триггеры доступны в каталоге `events/uprobes/...`, по аналогии с `kprobes` (см. раздел 14.6 «Зонды `kprobes`»).

14.7.4. Профилировщик `uprobe`

После установки зондов `uprobes` Ftrace начинает подсчитывать соответствующие им события. Эти счетчики доступны для чтения в файле `uprobe_profile`. Например:

```
# cat /sys/kernel/debug/tracing/uprobe_profile
/bin/bash brendan                                     11
```

В первом столбце выводится путь, во втором — имя зонда (его определение можно увидеть в файле `uprobe_events`) и в третьем — счетчик попаданий.

14.8. ТРАССИРОВЩИК `FUNCTION_GRAPH`

Трассировщик `function_graph` выводит граф вызовов функций, показывая поток выполнения. В начале этой главы был приведен пример использования `funcgraph` (8) из `perf-tools`. А в этом разделе рассмотрим интерфейс Ftrace `tracefs`.

Вот как выглядит исходное неактивное состояние трассировщика `function_graph`:

```
# cd /sys/kernel/debug/tracing
# cat set_graph_function
#### all functions enabled ####
# cat current_tracer
nop
```

В данный момент не используются никакие трассировщики.

14.8.1. Трассировка графа

В примере ниже использован трассировщик function_graph для функции do_nanosleep(), чтобы показать вызовы ее дочерних функций:

```
# echo do_nanosleep > set_graph_function
# echo function_graph > current_tracer
# cat trace_pipe
1)  2.731 us | get_xsave_addr();
1)          | do_nanosleep() {
1)          |     hrtimer_start_range_ns() {
1)          |         lock_hrtimer_base.isra.0() {
1)  0.297 us |             _raw_spin_lock_irqsave();
1)  0.843 us |         }
1)  0.276 us |     ktime_get();
1)  0.340 us |     get_nohz_timer_target();
1)  0.474 us |     enqueue_hrtimer();
1)  0.339 us |     _raw_spin_unlock_irqrestore();
1)  4.438 us | }
1)          | schedule() {
1)          |     rcu_note_context_switch() {
[...]
```

5) \$ 1000383 us	}	/* do_nanosleep */
------------------	---	--------------------

```
^C
# echo nop > current_tracer
# echo > set_graph_function
```

Этот вывод показывает дочерние вызовы и поток выполнения кода: do_nanosleep() вызывает hrtimer_start_range_ns(), которая вызывает lock_hrtimer_base.isra.0(), и т. д. В первом столбце выводится номер процессора (в этом выводе в основном отображается процессор 1) и продолжительность выполнения функций, чтобы можно было определить причины задержек. Высокие задержки помечаются дополнительным символом для вашего внимания — в этом выводе символом «\$» отмечена задержка в 1 000 383 мкс (1 с). Вот значения этих символов [Rostedt 08]:

- \$: более 1 с;
- @: более 100 мс;
- *: более 10 мс;
- #: более 1 мс;
- !: более 100 мкс;
- +: более 10 мкс.

В этом примере намеренно не использовалась фильтрация функций (set_ftrace_filter), чтобы можно было видеть все дочерние вызовы. Но это влечет за собой некоторый оверхед, что приводит к завышению задержек. Но даже в этом случае такая трассировка помогает выявить источники высоких задержек, которые могут многократно превосходить добавленный оверхед. Если потребуется более точно определить время выполнения некоторой функции, настройте фильтр

функций — это уменьшит количество трассируемых функций. Вот как можно выполнить трассировку одной только `do_nanosleep()`:

```
# echo do_nanosleep > set_ftrace_filter
# cat trace_pipe
[...]
7) $ 1000130 us | } /* do_nanosleep */
^C
```

Здесь я повторил трассировку той же рабочей нагрузки (`sleep 1`). После применения фильтра сообщаемая длительность выполнения `do_nanosleep()` уменьшилась с 1 000 383 мкс до 1 000 130 мкс (в этом конкретном случае), потому что исчез оверхед на трассировку всех вызываемых функций.

В этих примерах также используется `trace_pipe`, чтобы просматривать вывод в реальном времени, но обычно вывод получается слишком объемным и намного практичнее перенаправить вывод трассировщика в файл, как было показано в разделе 14.4 «Трассировщик `function`».

14.8.2. Параметры

Для изменения формата вывода можно использовать дополнительные параметры, доступные в каталоге `options`:

```
# ls options/funcgraph-*
options/funcgraph-abstime   options/funcgraph-irqs      options/funcgraph-proc
options/funcgraph-cpu       options/funcgraph-overhead  options/funcgraph-tail
options/funcgraph-duration  options/funcgraph-overflow
```

Они управляют выводом и могут добавлять или исключать некоторые детали: идентификатор процессора (`funcgraph-cpu`), имя процесса (`funcgraph-proc`), продолжительность выполнения функции (`funcgraph-duration`) и маркеры задержки (`funcgraph-overhead`).

14.9. ТРАССИРОВЩИК HWLAT

Детектор задержек, обусловленных работой оборудования (`hwlat`), — пример специализированного трассировщика. Он может обнаруживать ситуации, когда внешние аппаратные события ухудшают производительность процессора: события, которые иначе невидимы для ядра и других инструментов. Например, прерывания управления системой (`system management interrupt`, SMI) и возмущения со стороны гипервизора (в том числе вызванные шумными соседями).

Для этого он выполняет цикл с отключенными прерываниями и измеряет время, затраченное на выполнение каждой итерации. Этот цикл поочередно выполняется на каждом процессоре, а затем выводится продолжительность выполнения самой медленной итерации для каждого процессора, если она превышает пороговое значение (по умолчанию 10 мкс, но порог можно настроить с помощью файла `tracing_thresh`).

Вот пример:

```
# cd /sys/kernel/debug/tracing
# echo hwlat > current_tracer
# cat trace_pipe
<...>-5820 [001] d... 354016.973699: #1    inner/outer(us): 2152/1933
ts:1578801212.559595228
<...>-5820 [000] d... 354017.985568: #2    inner/outer(us): 19/26
ts:1578801213.571460991
<...>-5820 [001] dn.. 354019.009489: #3    inner/outer(us): 1699/5894
ts:1578801214.595380588
<...>-5820 [000] d... 354020.033575: #4    inner/outer(us): 43/49
ts:1578801215.619463259
<...>-5820 [001] d... 354021.057566: #5    inner/outer(us): 18/45
ts:1578801216.643451721
<...>-5820 [000] d... 354022.081503: #6    inner/outer(us): 18/38
ts:1578801217.667385514
^C
# echo nop > current_tracer
```

Многие из этих полей были описаны в предыдущих разделах (см. раздел 14.4 «Трассировщик function»). Любопытно, что за отметкой времени следует порядковый номер (#1, ...), затем числа «inner/outer(us)» и окончательная отметка времени. Числа «inner/outer» показывают время в микросекундах выполнения одной итерации (внутреннее — inner) и логики перехода между итерациями (внешнее — outer). В первой строке видно, что самая медленная итерация выполнялась 2152 мкс (inner) и переход между итерациями длился 1933 мкс (outer). Эти задержки намного превышают порог в 10 мкс и явно обусловлены внешними возмущениями.

hwlat имеет параметры настройки: цикл выполняется в течение периода времени, который называется *шириной* (width), и запускает эксперименты, продолжительность которых называется *окном* (window). В течение каждого периода width регистрируется самая медленная итерация с продолжительностью, превышающей пороговое значение (10 мкс). Эти параметры можно изменить с помощью файлов width и window в /sys/kernel/debug/tracing/hwlat_detector; время в этих файлах измеряется в микросекундах.

ВНИМАНИЕ: я бы классифицировал hwlat как инструмент микробенчмаркинга, а не наблюдения, потому что он выполняет эксперименты, которые сами по себе влияют на производительность системы: поочередно нагружает каждый процессор на длительное время, отключая при этом прерывания.

14.10. ТРИГГЕРЫ HIST

Триггеры hist были добавлены в Linux 4.7 Томом Занусси (Tom Zanussi). Они позволяют создавать собственные гистограммы распределения событий. Это еще одна форма статистического анализа, позволяющая разбить счетчики на составляющие.

Вот обобщенный порядок получения гистограммы:

1. `echo 'hist:выражение' > events/.../trigger`: создать триггер hist.
2. `sleep продолжительность`: дать время для заполнения гистограммы.
3. `cat events/.../hist`: вывести гистограмму.
4. `echo '!hist:выражение' > events/.../trigger`: удалить триггер.

Выражения для триггеров hist определяются в следующем формате:

```
hist:keys=<field1[,field2,...]>[:values=<field1[,field2,...]>]
[:sort=<field1[,field2,...]>][:size=#entries][:pause][:continue]
[:clear][:name=histname1][:<handler>.<action>] [if <filter>]
```

Исчерпывающее описание синтаксиса можно найти в исходном коде Linux в файле Documentation/trace/histogram.rst, а далее приводятся несколько примеров [Zanussi 20].

14.10.1. Гистограмма с единственным ключом

В примере ниже показано применение триггеров hist для подсчета системных вызовов с использованием точки трассировки raw_syscalls:sys_enter. Выводится гистограмма с разбивкой по идентификатору процесса:

```
# cd /sys/kernel/debug/tracing
# echo 'hist:key=common_pid' > events/raw_syscalls/sys_enter/trigger
# sleep 10
# cat events/raw_syscalls/sys_enter/hist
# event histogram
#
# trigger info: hist:keys=common_pid.execname:vals=hitcount:sort=hitcount:size=2048
[active]
#

{ common_pid:      347 } hitcount:      1
{ common_pid:      345 } hitcount:      3
{ common_pid:      504 } hitcount:      8
{ common_pid:      494 } hitcount:     20
{ common_pid:      502 } hitcount:     30
{ common_pid:      344 } hitcount:     32
{ common_pid:      348 } hitcount:     36
{ common_pid:    32399 } hitcount:    136
{ common_pid:    32400 } hitcount:    138
{ common_pid:    32379 } hitcount:    177
{ common_pid:    32296 } hitcount:    187
{ common_pid:    32396 } hitcount:   882604

Totals:
  Hits: 883372
  Entries: 12
  Dropped: 0
# echo '!hist:key=common_pid' > events/raw_syscalls/sys_enter/trigger
```

Мы видим, что в период трассировки процесс с идентификатором PID 32396 обратился к системным вызовам 882 604 раза; также перечислены счетчики обращений для других процессов. В нескольких последних строках выводятся: количество операций записи в хеш (**Hits**), количество элементов в хеше (**Entries**) и количество прерванных операций записи из-за превышения размера хеша (**Dropped**). Если появляются прерванные операции записи, то можно увеличить размер хеша при его объявлении. По умолчанию хеш может содержать до 2048 элементов.

14.10.2. Поля

Поля хеша определяются файлом формата события. В этом примере использовалось поле `common_pid`:

```
# cat events/raw_syscalls/sys_enter/format
[...]
    field:int common_pid;      offset:4; size:4; signed:1;

    field:long id; offset:8; size:8; signed:1;
    field:unsigned long args[6]; offset:16; size:48; signed:0;
```

Также можно использовать другие поля. Поле `id` для этого события содержит идентификатор системного вызова. Вот пример использования этого поля в роли ключа хеша:

```
# echo 'hist:key=id' > events/raw_syscalls/sys_enter/trigger
# cat events/raw_syscalls/sys_enter/hist
[...]
{ id:      14 } hitcount:      48
{ id:       1 } hitcount:    80362
{ id:       0 } hitcount:    80396
[...]
```

Эта гистограмма показывает, что чаще других вызывались системные вызовы с идентификаторами 0 и 1. В моей системе идентификаторы системных вызовов определены в этом заголовочном файле:

```
# more /usr/include/x86_64-linux-gnu/asm/unistd_64.h
[...]
#define __NR_read 0
#define __NR_write 1
[...]
```

То есть идентификаторы 0 и 1 соответствуют системным вызовам `read(2)` и `write(2)`.

14.10.3. Модификаторы

Разбивка по идентификаторам процессов и системных вызовов используется настолько часто, что в триггеры `hist` была добавлена поддержка модификаторов, аннотирующих вывод: `.hexesname` — для идентификаторов PID и `.syscall` — для

идентификаторов системных вызовов. Вот пример добавления модификатора `.execname` к предыдущему примеру:

```
# echo 'hist:key=common_pid.execname' > events/raw_syscalls/sys_enter/trigger
[...]
{ common_pid: bash           [ 32379 ] } hitcount:      166
{ common_pid: sshd          [ 32296 ] } hitcount:       259
{ common_pid: dd            [ 32396 ] } hitcount:    869024
[...]
```

Этот модификатор добавляет в вывод имена процессов, за которыми следуют их идентификаторы PID в квадратных скобках.

14.10.4. Фильтры PID

Основываясь на предыдущих результатах с разбивкой по идентификаторам процессов и системных вызовов, можно предположить, что они связаны, а к системным вызовам `read(2)` и `write(2)` действительно обращалась команда `dd(1)`. Чтобы убедиться в этом, можно запросить создание гистограммы с разбивкой по идентификаторам системных вызовов и добавить фильтр, чтобы выполнить подсчет обращений только для одного конкретного процесса:

```
# echo 'hist:key=id.syscall if common_pid==32396' > \
    events/raw_syscalls/sys_enter/trigger
# cat events/raw_syscalls/sys_enter/hist
# event histogram
#
# trigger info: hist:keys=id.syscall:vals=hitcount:sort=hitcount:size=2048 if
common_
pid==32396 [active]
#

{ id: sys_write           [ 1 ] } hitcount:    106425
{ id: sys_read            [ 0 ] } hitcount:    106425

Totals:
  Hits: 212850
  Entries: 2
  Dropped: 0
```

Теперь гистограмма показывает количество обращений к системным вызовам только для процесса с указанным PID, а модификатор `.syscall` добавил имена системных вызовов. Это подтверждает, что `dd(1)` вызывает `read(2)` и `write(2)`. Другое решение — использовать несколько ключей, как показано в следующем разделе.

14.10.5. Гистограмма с несколькими ключами

В следующем примере используется *второй ключ* — идентификатор системного вызова:

```
# echo 'hist:key=common_pid.execname,id' > events/raw_syscalls/sys_enter/trigger
# sleep 10
# cat events/raw_syscalls/sys_enter/hist
# event histogram
#
# trigger info: hist:keys=common_pid.execname,id:vals=hitcount:sort=hitcount:si
ze=2048
[active]
#
[...]
```

{ common_pid: sshd	[14250], id:	23 }	hitcount:	36
{ common_pid: bash	[14261], id:	13 }	hitcount:	42
{ common_pid: sshd	[14250], id:	14 }	hitcount:	72
{ common_pid: dd	[14325], id:	0 }	hitcount:	9195176
{ common_pid: dd	[14325], id:	1 }	hitcount:	9195176

```
Totals:
  Hits: 18391064
  Entries: 75
  Dropped: 0
  Dropped: 0
```

Теперь вывод включает имя и идентификатор PID процесса, выполняется дополнительная разбивка по идентификатору системного вызова. Как показывает этот вывод, процесс dd с идентификатором PID 142325 обращался к двум системным вызовам с идентификаторами 0 и 1. Ко второму ключу можно добавить модификатор .syscall, чтобы включить в вывод имена системных вызовов.

14.10.6. Трассировки стека в роли ключей

Часто бывает нужно узнать путь кода, который привел к событию, и я предложил Тому Занусси добавить в Ftrace возможность использовать в роли ключа трассировку стека ядра.

Вот подсчет путей в коде, которые привели к точке трассировки block:block_rq_issue:

```
# echo 'hist:key=stacktrace' > events/block/block_rq_issue/trigger
# sleep 10
# cat events/block/block_rq_issue/hist
[...]
```

```
{ stacktrace:
  nvme_queue_rq+0x16c/0x1d0
  __blk_mq_try_issue_directly+0x116/0x1c0
  blk_mq_request_issue_directly+0x4b/0xe0
  blk_mq_try_issue_list_directly+0x46/0xb0
  blk_mq_sched_insert_requests+0xae/0x100
  blk_mq_flush_plug_list+0x1e8/0x290
  blk_flush_plug_list+0xe3/0x110
  blk_finish_plug+0x26/0x34
  read_pages+0x86/0x1a0
  __do_page_cache_readahead+0x180/0x1a0
  ondemand_readahead+0x192/0x2d0
  page_cache_sync_readahead+0x78/0xc0
  generic_file_buffered_read+0x571/0xc00
```

```

        generic_file_read_iter+0xdc/0x140
        ext4_file_read_iter+0x4f/0x100
        new_sync_read+0x122/0x1b0
} hitcount:      266

Totals:
  Hits: 522
  Entries: 10
  Dropped: 0

```

Я сократил вывод и оставил только последнюю, наиболее частую трассировку стека. Она показывает, что дисковый ввод/вывод производился через вызов функции `new_sync_read()`, которая вызывала `ext4_file_read_iter()` и т. д.

14.10.7. Синтетические события

Именно здесь все начинает становиться по-настоящему странным (если еще не стало). При желании можно создать *синтетическое событие*, которое запускается другими событиями и может комбинировать их аргументы настраиваемыми способами. Чтобы получить доступ к аргументам предыдущих событий, сохраните их в гистограмму и извлеките из более позднего синтетического события.

Синтетические события наиболее полезны для основного варианта использования: создания нестандартных гистограмм задержки. С помощью синтетического события можно сохранить отметку времени для одного события, а затем извлечь ее по другому событию и рассчитать разность времени.

Вот пример использования синтетического события `syscall_latency` для вычисления задержки всех системных вызовов и представления ее распределения в виде гистограммы с разбивкой по идентификатору и имени системного вызова:

```

# cd /sys/kernel/debug/tracing
# echo 'syscall_latency u64 lat_us; long id' >> synthetic_events
# echo 'hist:keys=common_pid:ts0=common_timestamp.usecs' >> \
    events/raw_syscalls/sys_enter/trigger
# echo 'hist:keys=common_pid:lat_us=common_timestamp.usecs-$ts0:\
    'onmatch(raw_syscalls.sys_enter).trace(syscall_latency,$lat_us,id)' >>\
    events/raw_syscalls/sys_exit/trigger
# echo 'hist:keys=lat_us,id:syscall:sort=lat_us' >> \
    events/synthetic/syscall_latency/trigger
# sleep 10
# cat events/synthetic/syscall_latency/hist
[...]
```

{ lat_us: 5779085, id: sys_epoll_wait	[232] }	hitcount: 1
{ lat_us: 6232897, id: sys_poll	[7] }	hitcount: 1
{ lat_us: 6233840, id: sys_poll	[7] }	hitcount: 1
{ lat_us: 6233884, id: sys_futex	[202] }	hitcount: 1
{ lat_us: 7028672, id: sys_epoll_wait	[232] }	hitcount: 1
{ lat_us: 9999049, id: sys_poll	[7] }	hitcount: 1
{ lat_us: 10000097, id: sys_nanosleep	[35] }	hitcount: 1
{ lat_us: 10001535, id: sys_wait4	[61] }	hitcount: 1
{ lat_us: 10002176, id: sys_select	[23] }	hitcount: 1

```

[...]
```


Я сократил вывод, чтобы показать только самые большие задержки. Гистограмма подсчитывает задержки (в микросекундах) по идентификаторам системных вызовов и, как следует из этого вывода, `sys_nanosleep` один раз показала задержку в 10 000 097 мкс. Скорее всего, она соответствует команде `sleep 10`, которую я запустил, чтобы установить продолжительности сбора данных.

Вывод получился очень объемным, потому что записывает ключ для каждой комбинации микросекунд и идентификатора системного вызова, и в действительности я превысил размер гистограммы, по умолчанию равный 2048. При необходимости размер можно увеличить, добавив оператор: `size = ...` в объявление гистограммы. Также можно использовать модификатор `.log2` для записи задержек с шагом, кратным `log2`. Это значительно сократит количество элементов в гистограмме и при этом обеспечит достаточное разрешение для анализа задержек.

Чтобы отключить и очистить синтетическое событие, нужно выполнить все команды в обратном порядке, добавив префикс «!».

В табл. 14.4 на примерах фрагментов кода я объясняю, как работает это синтетическое событие.

Таблица 14.4. Объяснение примера с синтетическим событием

Описание	Синтаксис
Создается синтетическое событие с именем <code>syscall_latency</code> и двумя аргументами: <code>lat_us</code> и <code>id</code>	<pre>echo 'syscall_latency u64 lat_us; long id' >> synthetic_events</pre>
При появлении события <code>sys_enter</code> выполнить запись в гистограмму, используя в качестве ключа <code>common_pid</code> (идентификатор PID текущего процесса),	<pre>echo 'hist:keys=common_pid: ... >> events/raw_syscalls/sys_enter/ trigger</pre>
и сохранить текущее время в микросекундах в гистограмме — в переменной с именем <code>ts0</code> , связанной с ключом гистограммы (<code>common_pid</code>)	<pre>ts0=common_timestamp.usecs</pre>
При появлении события <code>sys_exit</code> использовать <code>common_pid</code> в качестве ключа гистограммы и	<pre>echo 'hist:keys=common_pid: ... >> events/raw_syscalls/sys_exit/ trigger</pre>
вычислить задержку как разность между текущим и начальным временем, сохраненным в <code>ts0</code> предыдущим событием, и записать ее в гистограмму — в переменную с именем <code>lat_us</code> ,	<pre>lat_us=common_timestamp.usecs- \$ts0</pre>
сравнить ключи в гистограмме для этого события и события <code>sys_enter</code> . Если они совпадают (одно и то же значение <code>common_pid</code>), тогда задержка в <code>lat_us</code> вычислена правильно (от <code>sys_enter</code> до <code>sys_exit</code> для одного и того же PID), поэтому,	<pre>onmatch(raw_syscalls.sys_enter)</pre>

Таблица 14.4 (окончание)

Описание	Синтаксис
наконец, запускаем наше синтетическое событие <code>syscall_latency</code> с аргументами <code>lat_us</code> и <code>id</code>	<code>.trace(syscall_latency, \$lat_us,id)</code>
Вывести это синтетическое событие в виде гистограммы с полями <code>lat_us</code> и <code>id</code>	<code>echo 'hist:keys=lat_us, id.syscall:sort=lat_us' >> events/synthetic/ syscall_latency/trigger</code>

Гистограммы Ftrace реализованы на основе хеш-объектов (хранилищ пар ключ/значение), и в более ранних примерах эти хеши использовались только для вывода: для подсчета количества системных вызовов по их идентификаторам и идентификаторам PID процессов. При использовании синтетических событий мы выполняем две дополнительные операции с этими хешами: а) сохраняем значения, которые не являются частью вывода (отметки времени) и б) в одном событии извлекаем пары ключ/значение, которые были установлены другим событием. Также мы выполняем арифметические операции, в частности вычитание. В каком-то смысле начинаем писать мини-программы.

Дополнительную информацию о синтетических событиях ищите в документации [Zanussi 20]. Я не раз делился своим мнением и взглядами, прямо или косвенно, с инженерами Ftrace и BPF. Считаю, что в эволюции Ftrace есть смысл, потому что проблемы, которые я поднимал ранее, постепенно решаются. Я бы резюмировал развитие так:

- Ftrace великолепен, но мне приходится использовать BPF для подсчета событий по PID и трассировкам стека.
- Нет проблем, вот тебе триггеры `hist`.
- Здорово! Но я все равно вынужден использовать BPF для вычисления нестандартных задержек.
- Нет проблем, вот тебе синтетические события.
- Отлично, я проверю эту возможность, когда закончу писать «BPF Performance Tools».
- *Ты серьезно?*

Да, теперь я хочу изучить возможность применения синтетических событий в определенных случаях. Это невероятно мощная возможность, встроенная в ядро, которую можно использовать с помощью обычных сценариев на языке командной оболочки. (И да, я закончил писать книгу о BPF, но потом занялся этой книгой.)

14.11. TRACE-CMD

trace-cmd — это интерфейс к Ftrace с открытым исходным кодом, разработанный Стивеном Ростедтом (Steven Rostedt) и другими [trace-cmd 20]. Он поддерживает подкоманды и параметры для настройки системы трассировки, двоичный формат вывода и другие функции. Для доступа к источникам событий он может использовать трассировщики function и function_graph в Ftrace, а также точки трассировки и уже настроенные зонды kprobes и uprobes.

Вот пример использования trace-cmd для регистрации вызовов функции ядра do_nanosleep() с помощью трассировщика function в течение 10 с (выполнением фиктивной команды sleep(1)):

```
# trace-cmd record -p function -l do_nanosleep sleep 10
  plugin 'function'
CPU0 data recorded at offset=0x4fe000
    0 bytes in size
CPU1 data recorded at offset=0x4fe000
    4096 bytes in size
# trace-cmd report
CPU 0 is empty
cpus=2
      sleep-21145 [001] 573259.213076: function:      do_nanosleep
multipathd-348  [001] 573259.523759: function:      do_nanosleep
multipathd-348  [001] 573260.523923: function:      do_nanosleep
multipathd-348  [001] 573261.524022: function:      do_nanosleep
multipathd-348  [001] 573262.524119: function:      do_nanosleep
[...]
```

Вывод начинается с команды sleep(1), которая была вызвана командой trace-cmd (сначала она настраивает трассировку, а затем запускает указанную команду), за которой следуют различные вызовы из процесса multipathd с PID 348. Этот пример также показывает, что trace-cmd более лаконичен, чем эквивалентные команды tracefs в /sys. Кроме того, этот интерфейс безопаснее: многие подкоманды сбрасывают состояние трассировки, когда это нужно.

Интерфейс trace-cmd обычно распространяется в составе пакета «trace-cmd», но при желании его исходный код можно получить на сайте проекта [trace-cmd 20].

Далее в этом разделе я покажу некоторые подкоманды и возможности трассировки trace-cmd. Полную информацию обо всех возможностях и синтаксисе из примеров ниже ищите в документации по trace-cmd.

14.11.1. Обзор подкоманд

Основные возможности trace-cmd доступны в виде подкоманд — trace-cmd record. Подкоманды, поддерживаемые последней версией trace-cmd (2.8.3), перечислены в табл. 14.5.

Таблица 14.5. Некоторые подкоманды trace-cmd

Команда	Описание
record	Выполняет трассировку и записывает результаты в файл trace.dat
report	Читает данные из файла trace.dat
stream	Выполняет трассировку и выводит результаты в стандартный вывод
list	Перечисляет доступные события трассировки
stat	Сообщает состояние подсистемы трассировки в ядре
profile	Трассирует и генерирует отчет со значениями времени и задержек в ядре
listen	Принимает сетевые запросы на выполнение трассировки

Также в число поддерживаемых входят подкоманды `start`, `stop`, `restart` и `clear`, управляющие трассировкой аналогично подкоманде `record`. В будущих версиях в `trace-cmd` могут появиться дополнительные подкоманды. Чтобы не пропустить их, выполните команду `trace-cmd` без аргументов, которая выведет полный список подкоманд.

Каждая подкоманда имеет множество параметров. Получить их список можно с помощью параметра `-h`. Вот список параметров для подкоманды `record`:

```
# trace-cmd record -h
```

```
trace-cmd version 2.8.3
```

```
usage:
```

```
trace-cmd record [-v][-e event [-f filter]][-p plugin][-F][-d][-D][-o file] \
  [-q][-s usecs][-O option ][-l func][-g func][-n func] \
  [-P pid][-N host:port][-t][-r prio][-b size][-B buf][command ...]
  [-m max][-C clock]
  -e запустить команду, активировав трассировку указанного события
  -f фильтр по содержимому для события, указанного в параметре -e
  -R триггер для события, указанного в параметре
  -p запустить команду, активировав указанный плагин
  -F фильтровать события только для указанного процесса
  -P трассировать только процесс с указанным PID, действует так же,
    как параметр -F для команды
  -c трассировать также дочерние процессы при наличии параметра -F
    (или -P, если ядро поддерживает его)
  -C установить значение часов трассировки
  -T включить сохранение трассировки стека для каждого события
  -l трассировать только указанную функцию
  -g создать граф для указанной функции
  -n не трассировать указанную функцию1
```

```
[...]
```

¹ Это перевод вывода команды. — *Примеч. пер.*

Я обрезал список в этом выводе, оставив только первые 12 из 35 параметров, которые используются особенно часто. Обратите внимание, что под термином *плагин* (*plugin*; -p) здесь подразумеваются трассировщики Ftrace, в том числе function, function_graph и hwlat.

14.11.2. Однострочные сценарии для trace-cmd

В этих примерах однострочных сценариев показаны различные возможности trace-cmd. Их синтаксис описан на соответствующих страницах справочного руководства man.

Список событий

Вывести список всех источников событий и параметров:

```
trace-cmd list
```

Вывести список трассировщиков, доступных в Ftrace:

```
trace-cmd list -t
```

Вывести список источников событий (точек трассировки, зондов kprobe и uprobe):

```
trace-cmd list -e
```

Вывести список точек трассировки системных вызовов:

```
trace-cmd list -e syscalls:
```

Показать файл формата для указанной точки трассировки:

```
trace-cmd list -e syscalls:sys_enter_nanosleep -F
```

Трассировка функций

Трассировать функцию ядра в масштабе всей системы:

```
trace-cmd record -p function -l имя_функции
```

Трассировать все функции ядра с именами, начинающимися с «tcp_», в масштабе всей системы до нажатия Ctrl-C:

```
trace-cmd record -p function -l 'tcp_*
```

Трассировать все функции ядра с именами, начинающимися с «tcp_», в масштабе всей системы в течение 10 с:

```
trace-cmd record -p function -l 'tcp_*' sleep 10
```

Трассировать все функции ядра с именами, начинающимися с «vfs_», которые вызываются командой ls(1):

```
trace-cmd record -p function -l 'vfs_*' -F ls
```

Трассировать все функции ядра с именами, начинающимися с «vfs_», которые вызываются командной оболочкой bash(1) и всеми ее дочерними процессами:

```
trace-cmd record -p function -l 'vfs_*' -F -c bash
```

Трассировать все функции ядра с именами, начинающимися с «vfs_», которые вызываются процессом с PID 21124:

```
trace-cmd record -p function -l 'vfs_*' -P 21124
```

Трассировка графа функций

Трассировать функцию ядра и вызываемые ею функции в масштабе всей системы:

```
trace-cmd record -p function_graph -g имя_функции
```

Трассировать функцию ядра do_nanosleep() и вызываемые ею функции в масштабе всей системы в течение 10 с:

```
trace-cmd record -p function_graph -g do_nanosleep sleep 10
```

Трассировка событий

Трассировать событие запуска нового процесса с использованием точки трассировки sched:sched_process_exec до нажатия Ctrl-C:

```
trace-cmd record -e sched:sched_process_exec
```

Трассировать событие запуска нового процесса с использованием точки трассировки sched:sched_process_exec (короткая версия):

```
trace-cmd record -e sched_process_exec
```

Трассировать запросы блочного ввода/вывода с записью трассировок стека ядра:

```
trace-cmd record -e block_rq_issue -T
```

Трассировать все точки трассировки из семейства block до нажатия Ctrl-C:

```
trace-cmd record -e block
```

Трассировать предварительно созданный зонд kprobe с именем «brendan» в течение 10 с:

```
trace-cmd record -e probe:brendan sleep 10
```

Трассировать все системные вызовы, к которым обращается команда ls(1):

```
trace-cmd record -e syscalls -F ls
```

Отчеты

Вывести содержимое файла trace.dat:

```
trace-cmd report
```

Вывести содержимое файла trace.dat, только относящееся к процессору CPU 0:

```
trace-cmd report --cpu 0
```

Другие возможности

Трассировать события из плагина sched_switch:

```
trace-cmd record -p sched_switch
```

Принимать запросы на трассировку на порту TCP с номером 8081:

```
trace-cmd listen -p 8081
```

Подключиться к удаленному хосту для выполнения подкоманды record:

```
trace-cmd record ... -N addr:port
```

14.11.3. Сравнение trace-cmd и perf(1)

Своим стилем использования подкоманд trace-cmd напоминает профилировщик perf(1), описанный в главе 13. Они действительно имеют схожие возможности. В табл. 14.6 проводится сравнение trace-cmd и perf(1).

Таблица 14.6. Сравнение perf(1) и trace-cmd

Атрибут	perf(1)	trace-cmd
Двоичный выходной файл	perf.data	trace.dat
Точки трассировки	Да	Да
Зонды kprobes	Да	Частично(1)
Зонды uprobes	Да	Частично(1)
Зонды USDT	Да	Частично(1)
Счетчики РМС	Да	Нет
Выборка по времени	Да	Нет
Трассировка функций	Частично(2)	Да
Трассировка графов функций	Частично(2)	Да
Трассировка по сети (клиент/сервер)	Нет	Да
Оверхед на вывод в файл	Низкий	Очень низкий
Внешние интерфейсы	Разнообразные	KernelShark
Исходный код	В исходном коде ядра Linux в tools/perf	git.kernel.org

- Частично(1): trace-cmd поддерживает эти события, только если они уже были созданы другими способами и есть в /sys/kernel/debug/tracing/events.

- Частично(2): perf(1) поддерживает их с помощью подкоманды **ftrace**, но интеграция с perf(1) неполная (например, не поддерживается файл perf.data).

Чтобы показать эту схожесть, ниже приводятся команды, запускающие трассировку системных вызовов через точку трассировки `sys_enter_read` в масштабе всей системы в течение 10 с, а затем выводящие результаты трассировки. С помощью perf(1):

```
# perf record -e syscalls:sys_enter_nanosleep -a sleep 10
# perf script
```

...и trace-cmd:

```
# trace-cmd record -e syscalls:sys_enter_nanosleep sleep 10
# trace-cmd report
```

Одно из преимуществ trace-cmd — лучшая поддержка трассировщиков `function` и `function_graph`.

14.11.4. trace-cmd function_graph

В начале этого раздела я показал применение трассировщика `function` с использованием trace-cmd. Следующая команда демонстрирует применение `function_graph` для трассировки той же функции ядра `do_nanosleep()`:

```
# trace-cmd record -p function_graph -g do_nanosleep sleep 10
  plugin 'function_graph'
CPU0 data recorded at offset=0x4fe000
  12288 bytes in size
CPU1 data recorded at offset=0x501000
  45056 bytes in size
# trace-cmd report | cut -c 66-

      | do_nanosleep() {
      |     hrtimer_start_range_ns() {
      |         lock_hrtimer_base.isra.0() {
0.250 us |             _raw_spin_lock_irqsave();
0.688 us |         }
0.190 us |     }
0.153 us |     ktime_get();
      |     get_nohz_timer_target();
[...]
```

В этом примере я использовал команду `cut(1)`, чтобы убрать из графа функции столбцы с временами. Из-за этого в выводе нет полей трассировки, показанных в предыдущем примере.

14.11.5. KernelShark

KernelShark — это визуальный пользовательский интерфейс для изучения выходных файлов trace-cmd, созданный автором Ftrace Стивеном Ростедтом. Первоначально

использовавший фреймворк GTK, KernelShark был переписан на Qt Йорданом Караджовым (Yordan Karadzov) — нынешним мейнтейнером проекта. KernelShark можно установить в составе пакета kernelshark, если он доступен, или из исходного кода с сайта проекта [KernelShark 20]. Версия 1.0 — это версия на Qt, а версии 0.99 и ниже — это версии на GTK.

Вот пример регистрации событий всех точек трассировки планировщика и последующая их визуализация с помощью KernelShark:

```
# trace-cmd record -e 'sched:*'
# kernelshark
```

По умолчанию KernelShark читает выходной файл trace.dat, созданный trace-cmd (при необходимости с помощью параметра `-i` можно указать другой файл). На рис. 14.3 показано окно KernelShark с содержимым файла.

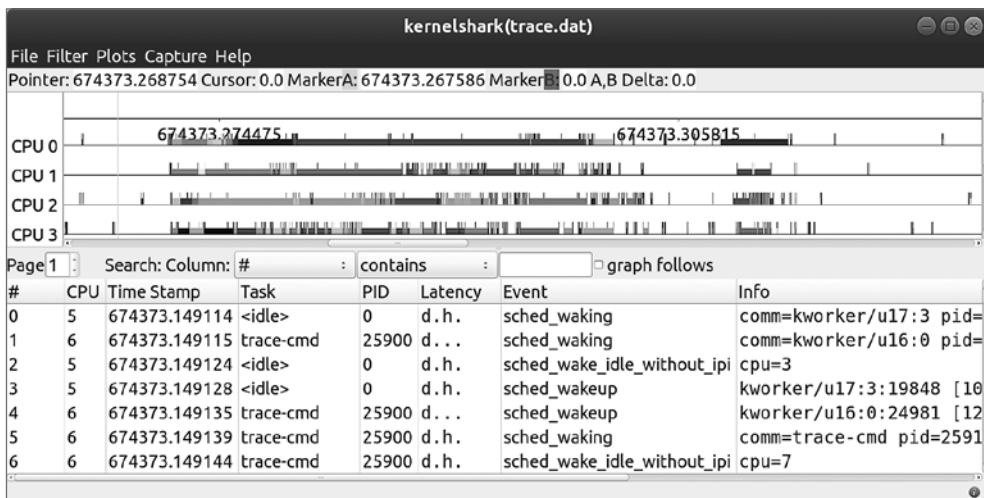


Рис. 14.3. KernelShark

В верхней части окна отображается шкала времени для каждого процессора и задачи, окрашенные в разные цвета. В нижней части таблица событий. KernelShark поддерживает интерактивные возможности: щелчок и перетаскивание вправо увеличивают масштаб до выбранного диапазона времени, а щелчок и перетаскивание влево уменьшают масштаб. В ответ на щелчок правой кнопкой мыши на события предлагаются дополнительные действия, например настройка фильтров.

KernelShark можно использовать для выявления проблем с производительностью, вызванных взаимодействиями между различными потоками.

14.11.6. Документация для trace-cmd

После установки пакета документация для trace-cmd должна быть доступна в виде страниц в справочном руководстве man (trace-cmd-record(1)), которые также можно найти в каталоге Documentation в дереве исходных текстов trace-cmd. Рекомендую посмотреть доклад «Understanding the Linux Kernel (via ftrace)» Стивена Ростедта, ведущего разработчика Ftrace и trace-cmd:

- презентация: <https://www.slideshare.net/ennael/kernel-recipes-2017-understanding-the-linux-kernel-via-ftrace-steven-rostedt>;
- видео: <https://www.youtube.com/watch?v=2ff-7UTg5rE>.

14.12. PERF FTRACE

Утилита perf(1), о которой шла речь в главе 13, имеет подкоманду ftrace, благодаря которой можно использовать трассировщики function и function_graph.

Вот пример использования трассировщика function для трассировки функции ядра do_nanosleep():

```
# perf ftrace -T do_nanosleep -a sleep 10
0) sleep-22821 | do_nanosleep() {
1) multipa-348 | do_nanosleep() {
1) multipa-348 | $ 1000068 us | }
1) multipa-348 | do_nanosleep() {
1) multipa-348 | $ 1000068 us | }
[...]
```

А это пример использования трассировщика function_graph:

```
# perf ftrace -G do_nanosleep -a sleep 10
1) sleep-22828 | do_nanosleep() {
1) sleep-22828 | =====> |
1) sleep-22828 | smp_irq_work_interrupt() {
1) sleep-22828 | irq_enter() {
1) sleep-22828 | rcu_irq_enter();
1) sleep-22828 | 0.258 us | }
1) sleep-22828 | 0.800 us | }
1) sleep-22828 | __wake_up() {
1) sleep-22828 | __wake_up_common_lock() {
1) sleep-22828 | 0.491 us | _raw_spin_lock_irqsave();
[...]
```

Подкоманда ftrace поддерживает несколько параметров, включая -p для фильтрации по значению PID. Это простая обертка, не интегрированная с другими возможностями perf(1): например, она выводит результаты трассировки на стандартный вывод и не использует файл perf.data.

14.13. PERF-TOOLS

perf-tools — это набор инструментов с открытым исходным кодом, основанных на Ftrace и perf(1), для анализа производительности, который был разработан мной и по умолчанию устанавливается на серверы Netflix [Gregg 20i]. Я спроектировал эти инструменты так, чтобы их было легко установить (минимум зависимостей) и просто использовать: каждый инструмент должен делать что-то одно, и делать это хорошо. Сами инструменты perf-tools в большинстве своем реализованы в виде сценариев командной оболочки, автоматизирующих установку файлов tracefs /sys.

Вот пример использования execsnoop(8) для трассировки событий запуска новых процессов:

```
# execsnoop
Tracing exec()s. Ctrl-C to end.
  PID  PPID  ARGS
6684   6682  cat -v trace_pipe
6683   6679  gawk -v o=1 -v opt_name=0 -v name= -v opt_duration=0 [...]
6685  20997  man ls
6695   6685  pager
6691   6685  preconv -e UTF-8
6692   6685  tbl
6693   6685  nroff -mandoc -rLL=148n -rLT=148n -Tutf8
6698   6693  locale charmap
6699   6693  groff -mtty-char -Tutf8 -mandoc -rLL=148n -rLT=148n
6700   6699  troff -mtty-char -mandoc -rLL=148n -rLT=148n -Tutf8
6701   6699  grotty
[...]
```

Вывод начинается с отображения команд cat(1) и gawk(1), которые используются самой утилитой execsnoop(8). Далее следуют команды, которые запускает man ls. Этот подход можно использовать для отладки скоротечных процессов, которые могут оставаться незамеченными другими инструментами.

execsnoop(8) поддерживает параметры, включая -t — для вывода отметок времени и -h — для вывода справки по использованию. У execsnoop(8) и других инструментов есть страница в справочном руководстве man и файлы примеров.

14.13.1. Покрываем инструментами

На рис. 14.4 показаны различные инструменты perf и области системы, которые они покрывают.

Многие из них являются узкоспециализированными инструментами и обозначены однонаправленной стрелкой, некоторые — многоцелевыми, они перечислены слева рядом с двунаправленными стрелками, показывающими их покрытие.

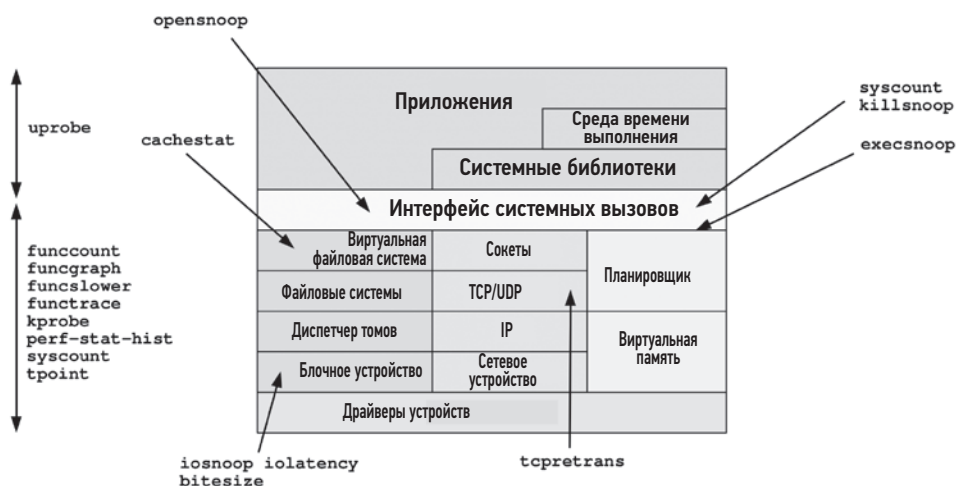


Рис. 14.4. perf-tools

14.13.2. Специализированные инструменты

Специализированные инструменты отмечены на рис. 14.4 однонаправленной стрелкой. Некоторые из них были представлены в предыдущих главах.

Специализированные инструменты, например `execsnoop(8)`, делают что-то одно, и делают это хорошо (философия Unix). Согласно этой философии, выходные данные должны быть по умолчанию краткими и очевидными, что оказывает немалую помощь в их изучении. Вы можете «просто запустить `execsnoop`», не вдаваясь в изучение параметров командной строки, и получить достаточный объем информации для решения проблемы. Параметры обычно предназначены для тонкой настройки.

Специализированные инструменты перечислены в табл. 14.7.

Таблица 14.7. Специализированные инструменты perf-tools

Инструмент	Используется	Описание
<code>bitesize(8)</code>	perf	Отображает размеры дискового ввода/вывода в виде гистограммы
<code>cachestat(8)</code>	Ftrace	Выводит статистику попаданий и промахов кэша страниц
<code>execsnoop(8)</code>	Ftrace	Трассирует запуск новых процессов (через <code>execve(2)</code>) и их аргументы
<code>iolateness(8)</code>	Ftrace	Отображает задержки дискового ввода/вывода в виде гистограммы
<code>iosnoop(8)</code>	Ftrace	Трассирует дисковый ввод/вывод и отображает подробную информацию, включая задержки
<code>killsnoop(8)</code>	Ftrace	Трассирует отправку сигналов процессам с помощью <code>kill(2)</code> и отображает информацию о сигналах

Инструмент	Используй- зует	Описание
opensnoop(8)	Ftrace	Трассирует семейство системных вызовов open(2) и отображает имена открывавшихся файлов
tcpretrans(8)	Ftrace	Трассирует повторные передачи TCP, выводит IP-адреса и состояние ядра

Инструмент execsnoop(8) был показан выше. Посмотрим также на инструмент `iolatency(8)`, отображающий задержки дискового ввода/вывода в виде гистограммы:

iolatency

Tracing block I/O. Output every 1 seconds. Ctrl-C to end.

```

>=(ms) .. <(ms)   : I/O      |Distribution|
0 -> 1           : 731      |#####|
1 -> 2           : 318      |#####|
2 -> 4           : 160      |#####|

>=(ms) .. <(ms)   : I/O      |Distribution|
0 -> 1           : 2973     |#####|
1 -> 2           : 497      |#####|
2 -> 4           : 26       |#|
4 -> 8           : 3        |#|

>=(ms) .. <(ms)   : I/O      |Distribution|
0 -> 1           : 3130     |#####|
1 -> 2           : 177      |###|
2 -> 4           : 1        |#|
^C

```

Как показывает этот вывод, задержка ввода/вывода обычно оставалась довольно низкой, от 0 до 1 мс.

Порядок реализации этого инструмента наглядно объясняет необходимость расширенного BPF. `iolatency(8)` трассирует события запуска и завершения блочного ввода/вывода, передает все события в пространство пользователя, анализирует их и преобразует в гистограммы с помощью `awk(1)`. Поскольку на большинстве серверов дисковый ввод/вывод выполняется относительно редко, этот подход почти не влечет за собой оверхед. Но для более частых событий, таких как сетевой ввод/вывод или планирование процессов, оверхед может оказаться недопустимо высоким. В расширенном BPF эта проблема решается за счет формирования гистограммы в пространстве ядра; в пространство пользователя передается только окончательный результат, что значительно сокращает оверхед. Ftrace поддерживает некоторые похожие возможности в триггерах `hist` и синтетических событиях, описанных в разделе 14.10 «Триггеры `hist`» (я собираюсь обновить утилиту `iolatency(8)` и использовать в ней эти новые возможности).

Я разработал решение для создания гистограмм еще до появления BPF и представил его как многоцелевой инструмент `perf-stat-hist(8)`.

14.13.3. Многоцелевые инструменты

Многоцелевые инструменты перечислены и описаны в табл. 14.8. Каждый из них поддерживает несколько источников событий и может играть множество ролей, как `perf(1)` и `trace-cmd`, хотя это также усложняет их использование.

Таблица 14.8. Многоцелевые инструменты `perf-tools`

Инструмент	Использует	Описание
<code>funccount(8)</code>	Ftrace	Подсчитывает вызовы функций ядра
<code>funcgraph(8)</code>	Ftrace	Трассирует функции ядра и показывает путь выполнения через дочерние функции
<code>functrace(8)</code>	Ftrace	Трассирует функции ядра
<code>funclower(8)</code>	Ftrace	Трассирует функции ядра, выполняющиеся дольше заданного порога
<code>kprobe(8)</code>	Ftrace	Динамическая трассировка функций ядра
<code>perf-stat-hist(8)</code>	<code>perf(1)</code>	Мощный инструмент для агрегирования аргументов точек трассировки
<code>syscount(8)</code>	<code>perf(1)</code>	Обобщает статистики о системных вызовах
<code>tpoint(8)</code>	Ftrace	Трассирует точки трассировки
<code>uprobe(8)</code>	Ftrace	Динамическая трассировка функций в пространстве пользователя

Чтобы облегчить использование этих инструментов, вы можете собрать свою коллекцию однострочных сценариев. Некоторые из таких сценариев, похожих на однострочные сценарии для `perf(1)` и `trace-cmd`, приводятся в разделе ниже.

14.13.4. Однострочные сценарии для `perf-tools`

Следующие однострочные сценарии выполняют трассировку в масштабе всей системы, пока не будет нажато `Ctrl-C`, если явно не указано иное. Они объединены в группы по признаку использования профилировщиков Ftrace, трассировщиков Ftrace и трассировки событий (точки трассировки, `kprobes`, `uprobes`).

Профилировщики Ftrace

Подсчитать вызовы функций ядра с именами, начинающимися на «`tcp_`»:

```
funccount 'tcp_*
```

Подсчитать вызовы функций VFS. Выводит 10 наиболее часто вызывавшихся функций каждую секунду:

```
funccount -t 10 -i 1 'vfs*
```

Трассировщики *Ftrace*

Трассировать функцию ядра `do_nanosleep()` и показать вызываемые ею функции:

```
funcgraph do_nanosleep
```

Трассировать функцию ядра `do_nanosleep()` и показать вызываемые ею функции вплоть до третьего уровня вложенности:

```
funcgraph -m 3 do_nanosleep
```

Подсчитать вызовы функций ядра с именами, оканчивающимися на «sleep», которые вызываются процессом с PID 198:

```
functrace -p 198 '*sleep'
```

Трассировать вызовы `vfs_read()`, продолжающиеся дольше 10 мс:

```
funcslower vfs_read 10000
```

Трассировка событий

Трассировать функцию ядра `do_sys_open()` с помощью `kprobe`:

```
kprobe p:do_sys_open
```

Трассировать возврат из `do_sys_open()` с помощью `kretprobe` и вывести возвращаемое значение:

```
kprobe 'r:do_sys_open $retval'
```

Трассировать аргумент с режимом открытия файла в вызове `do_sys_open()`:

```
kprobe 'p:do_sys_open mode=$arg3:u16'
```

Трассировать аргумент с режимом открытия файла в вызове `do_sys_open()` (в архитектуре `x86_64`):

```
kprobe 'p:do_sys_open mode=%dx:u16'
```

Трассировать аргумент с именем файла в вызове `do_sys_open()` и вывести его как строку:

```
kprobe 'p:do_sys_open filename=+0($arg2):string'
```

Трассировать аргумент с именем файла в вызове `do_sys_open()` и вывести его как строку (в архитектуре `x86_64`):

```
kprobe 'p:do_sys_open filename=+0(%si):string'
```

Трассировать `do_sys_open()`, когда имя файла совпадает с шаблоном «*stat»:

```
kprobe 'p:do_sys_open file=+0($arg2):string' 'file ~ "*stat"'
```

Трассировать `tcp_retransmit_skb()` и вывести трассировки стека ядра:

```
kprobe -s p:tcp_retransmit_skb
```

Вывести список точек трассировки:

```
tpoint -l
```

Трассировать дисковый ввод/вывод и вывести трассировки стека ядра:

```
tpoint -s block:block_rq_issue
```

Трассировать вызовы функции `readline()` в пространстве пользователя во всех процессах «bash»:

```
uprobe p:bash:readline
```

Трассировать возврат из `readline()` во всех процессах «bash» и вывести возвращаемое значение как строку:

```
uprobe 'r:bash:readline +0($retval):string'
```

Трассировать точку входа в `readline()` в `/bin/bash` и вывести аргумент как строку (x86_64):

```
uprobe 'p:/bin/bash:readline prompt=+0(%di):string'
```

Трассировать вызов `gettimeofday()` из библиотеки `libc` только для процесса с PID 1234:

```
uprobe -p 1234 p:libc:gettimeofday
```

Трассировать возврат из `fopen()`, только когда возвращается `NULL` (и использовать псевдоним «file»):

```
uprobe 'r:libc:fopen file=$retval' 'file == 0'
```

Регистры процессора

Псевдонимы аргументов функций (`$arg1`, ..., `$argN`) — это новая возможность Ftrace, появившаяся в Linux 4.20. В системах с более старыми ядрами (или с аппаратными архитектурами без псевдонимов) вместо псевдонимов используют имена регистров процессоров, как описано в разделе 14.6.2 «Аргументы». Представленные там однострочные сценарии включали несколько примеров использования регистров x86_64 (`%di`, `%si`, `%dx`). Соглашения о вызовах задокументированы на странице справочного руководства `man` для `syscall(2)`:

```
$ man 2 syscall
```

```
[...]
```

Arch/ABI	arg1	arg2	arg3	arg4	arg5	arg6	arg7	Notes
----------	------	------	------	------	------	------	------	-------

```
[...]
```

sparc/32	o0	o1	o2	o3	o4	o5	-	
----------	----	----	----	----	----	----	---	--


```

sparc/64      o0    o1    o2    o3    o4    o5    -
tile          R00   R01   R02   R03   R04   R05   -
x86-64        rdi   rsi   rdx   r10   r8    r9    -
x32           rdi   rsi   rdx   r10   r8    r9    -
[...]
```

14.13.5. Пример

Ниже показано применение `funccount(8)` для подсчета вызовов VFS (функция с именами, соответствующими шаблону «`vfs_*`»):

```

# funccount 'vfs_*'
Tracing "vfs_*"... Ctrl-C to end.
^C
FUNC                                COUNT
vfs_fsync_range                     10
vfs_statfs                          10
vfs_readlink                        35
vfs_statx                           673
vfs_write                           782
vfs_statx_fd                         922
vfs_open                           1003
vfs_getattr                         1390
vfs_getattr_nosec                   1390
vfs_read                           2604
```

Как показывает этот вывод, в течение трассировки функция `vfs_read()` была вызвана 2604 раза. Я регулярно использую `funccount(8)`, чтобы определить, какие функции ядра вызываются и какие из них вызываются особенно часто. Этот инструмент имеет относительно низкий оверхед, поэтому его можно использовать для проверки частоты вызова функций перед более дорогостоящей трассировкой.

14.13.6. Сравнение perf-tools и BCC/BPF

Изначально набор инструментов `perf-tools` разрабатывался для использования в облаке Netflix, когда оно работало под управлением Linux 3.2, где не было расширенного BPF. Сейчас в Netflix используются новые ядра, и я переписал многие из этих инструментов так, чтобы они использовали BPF. Например `perf-tools` и BCC имеют свои собственные версии `funccount(8)`, `hexsnoop(8)`, `opensnoop(8)` и других инструментов.

BPF обеспечивает возможность программирования и обладает более мощными возможностями (его интерфейсы BCC и `bpftool` рассматриваются в главе 15). Однако и у `perf-tools` есть некоторые преимущества¹:

- **funccount(8)**: версия в `perf-tools` использует более эффективные и менее ограниченные средства профилирования функций из `Ftrace`, чем имеющиеся в текущей версии BPF и основанные на `kprobe`.

¹ Первоначально я полагал, что откажусь от `perf-tools`, когда закончу реализацию трассировки в BPF, но затем решил продолжить использовать их по этим причинам.

- **funcgraph(8)**: в BCC вообще нет такого инструмента, потому что он использует трассировщик `function_graph` из Ftrace.
- **Триггеры hist**: в будущем этот механизм сможет обеспечить более эффективную работу инструментов `perf-tools` по сравнению с их версиями в BPF, основанными на `kprobe`.
- **Зависимости**: инструменты `perf-tools` сохраняют свою актуальность для сред с ограниченными ресурсами (например, для встраиваемых систем Linux), потому что им обычно достаточно лишь командной оболочки и `awk(1)`.

Сам я иногда использую инструменты `perf-tools` для перекрестной проверки и отладки проблем с инструментами BPF¹.

14.13.7. Документация

Обычно инструменты выводят сообщение о порядке использования, описывающее их синтаксис. Например:

```
# funccount -h
ИСПОЛЬЗОВАНИЕ: funccount [-hT] [-i secs] [-d secs] [-t top] funcstring
                -d seconds      # общая продолжительность трассировки
                -h               # это сообщение о порядке использования
                -i seconds      # сводная информация за интервал
                -t top           # показать только top самых часто вызываемых функций
                -T               # включить отметки времени (для -i)
```

Примеры:

```
funccount 'vfs*'          # трассировать все функции, соответствующие "vfs*"
funccount -d 5 'tcp*'      # трассировать все функции "tcp*" в течение 5 с
funccount -t 10 'ext3*'    # показать 10 самых часто вызываемых функций "ext3*"
funccount -i 1 'ext3*'     # выводить сводку каждую секунду
funccount -i 1 -d 5 'ext3*' # выводить сводку каждую секунду в течение 5 с
```

У каждого инструмента есть своя страница в справочном руководстве `man` и файл с примерами в репозитории `perf-tools` (`funccount_example.txt`).

14.14. ДОКУМЕНТАЦИЯ ДЛЯ FTRACE

Ftrace (и события трассировки) подробно описываются в исходном коде Linux в каталоге `Documentation/trace`. Эта документация также доступна в интернете:

- <https://www.kernel.org/doc/html/latest/trace/ftrace.html>;
- <https://www.kernel.org/doc/html/latest/trace/kprobetrace.html>;
- <https://www.kernel.org/doc/html/latest/trace/uprobetracer.html>;

¹ Я бы сказал так: «Человек с одним трассировщиком знает, какие события произошли. Человек с двумя трассировщиками знает, что один из них сломан, и обращается к `lkm1` в надежде найти патч».

- <https://www.kernel.org/doc/html/latest/trace/events.html>;
- <https://www.kernel.org/doc/html/latest/trace/histogram.html>.

Ресурсы, где доступны внешние интерфейсы:

- **trace-cmd**: <https://trace-cmd.org>;
- **perf ftrace**: исходный код Linux: [tools/perf/Documentation/perf-ftrace.txt](https://www.kernel.org/doc/html/latest/trace/perf-ftrace.txt);
- **perf-tools**: <https://github.com/brendangregg/perf-tools>.

14.15. ССЫЛКИ

- [Rostedt 08] Rostedt, S., «ftrace — Function Tracer», Linux documentation, <https://www.kernel.org/doc/html/latest/trace/ftrace.html>, 2008+.
- [Matz 13] Matz, M., Hubička, J., Jaeger, A., and Mitchell, M., «System V Application Binary Interface, AMD64 Architecture Processor Supplement, Draft Version 0.99.6», <http://x86-64.org/documentation/abi.pdf>, 2013.
- [Gregg 19f] Gregg, B., «Two Kernel Mysteries and the Most Technical Talk I've Ever Seen», <http://www.brendangregg.com/blog/2019-10-15/kernelrecipes-kernel-ftrace-internals.html>, 2019.
- [Dronamraju 20] Dronamraju, S., «Uprobe-tracer: Uprobe-based Event Tracing», Linux documentation, <https://www.kernel.org/doc/html/latest/trace/uprobetracer.html>, по состоянию на 2020.
- [Gregg 20i] Gregg, B., «Performance analysis tools based on Linux perf_events (aka perf) and ftrace», <https://github.com/brendangregg/perf-tools>, последнее обновление 2020.
- [Hiramatsu 20] Hiramatsu, M., «Kprobe-based Event Tracing», Linux documentation, <https://www.kernel.org/doc/html/latest/trace/kprobetrace.html>, по состоянию на 2020.
- [KernelShark 20] «KernelShark», <https://www.kernelshark.org>, по состоянию на 2020.
- [trace-cmd 20] «TRACE-CMD», <https://trace-cmd.org>, по состоянию на 2020.
- [Ts'o 20] Ts'o, T., Zefan, L., and Zanussi, T., «Event Tracing», Linux documentation, <https://www.kernel.org/doc/html/latest/trace/events.html>, по состоянию на 2020.
- [Zanussi 20] Zanussi, T., «Event Histograms», Linux documentation, <https://www.kernel.org/doc/html/latest/trace/histogram.html>, по состоянию на 2020.

Глава 15

BPF

В этой главе описываются BCC и bpftrace — внешние интерфейсы трассировки для расширенного BPF. Они включают набор инструментов для анализа производительности, которые мы уже рассматривали в предыдущих главах. Технология BPF была представлена в главе 3 «Операционная система», в разделе 3.4.4 «Расширенный BPF». Вкратце, расширенный BPF — это среда выполнения в пространстве ядра, предоставляющая трассировщикам возможности программирования.

Как и главы 13 «perf» и 14 «Ftrace», эта глава необязательна для чтения и предназначена для желающих изучить один или несколько системных трассировщиков более подробно.

С помощью инструментов расширенного BPF можно ответить на вопросы:

- Как выглядит гистограмма распределения задержек дискового ввода/вывода?
- Не обусловлены ли проблемы слишком большой задержкой планировщика процессора?
- Не страдают ли приложения из-за задержек в файловой системе?
- Какие сеансы TCP были и как долго существовали?
- Какие пути в коде блокируются и на какой срок?

Главное отличие BPF от других трассировщиков — возможность программирования. Он позволяет запускать по событиям написанные пользователем программы, которые могут фильтровать, сохранять и извлекать информацию, вычислять задержки, производить агрегирование в ядре и формировать обобщенные сводки, а также многое другое. В отличие от других трассировщиков, которым может потребоваться копировать все события в пространство пользователя для их обработки, BPF позволяет выполнять такую обработку более эффективно в контексте ядра. Тем самым можно создавать инструменты анализа производительности, у которых иначе был бы слишком высокий оверхед в промышленной среде.

В этой главе каждому интерфейсу отводится отдельный большой раздел. Вот основные разделы с подразделами:

- 15.1: BCC
 - 15.1.1: Установка
 - 15.1.2: Покрывтие инструментами
 - 15.1.3: Специализированные инструменты
 - 15.1.4: Многоцелевые инструменты
 - 15.1.5: Однострочные сценарии
- 15.2: bpftrace
 - 15.2.1: Установка
 - 15.2.2: Инструменты
 - 15.2.3: Однострочные сценарии
 - 15.2.4: Программирование
 - 15.2.5: Справочник

Различия между BCC и bpftrace очевидно вытекают из примеров их использования в предыдущих главах: BCC лучше подходит для создания сложных инструментов, а bpftrace — для специализированных программ. Некоторые инструменты могут быть реализованы с использованием обоих интерфейсов, как показано на рис. 15.1.

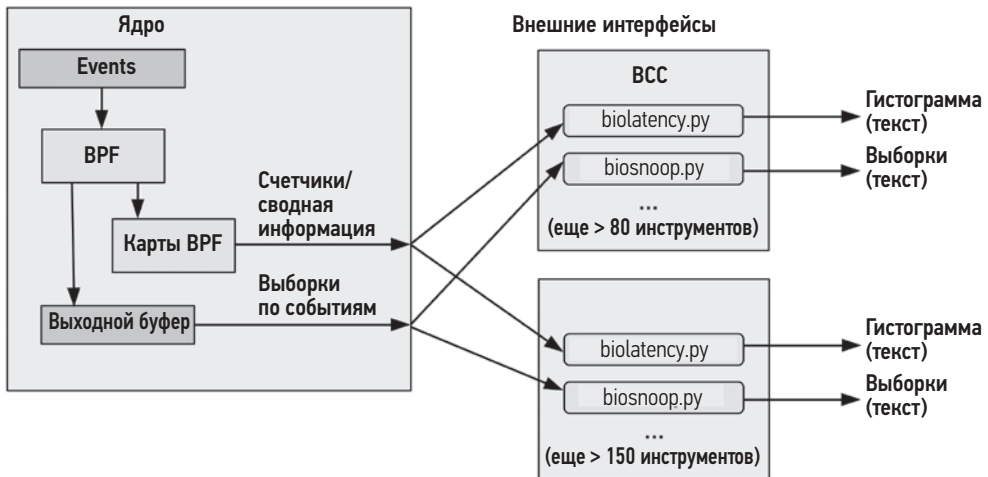


Рис. 15.1. Внешние интерфейсы трассировки для BPF

Конкретные различия между BCC и bpftrace перечислены в табл. 15.1.

Таблица 15.1. Сравнение BCC и bpftrace

Характеристика	BCC	bpftrace
Количество инструментов в репозитории	> 80 (bcc)	> 30 (bpftrace) > 120 (в книге «BPF Performance Tools»)
Использование	Обычно сложное: с параметрами (-h, -P PID, и т. д.) и аргументами	Обычно простое: без параметров и иногда без аргументов
Документация	Страницы справочного руководства man, файлы с примерами	Страницы справочного руководства man, файлы с примерами
Язык программирования	В пространстве пользователя: Python, Lua, C или C++. В пространстве ядра: C	bpftrace
Сложность программирования	Сложно	Просто
Типы вывода событий	Любой	Текст, JSON
Типы сводок	Любой	Счетчики, минимальное, максимальное, сумма, среднее, гистограммы с масштабом log2, линейные гистограммы; по нулевому и большему количеству ключей
Поддержка библиотек	Да (например, библиотеки для Python)	Нет
Средний размер программы ¹ (без комментариев)	228 строк	28 строк

Оба интерфейса, BCC и bpftrace, используются во многих компаниях, включая Facebook и Netflix. В Netflix они устанавливаются по умолчанию во все облачные экземпляры и используются для глубокого анализа, если в ходе мониторинга облака выявляются проблемы [Gregg 18e]:

- **BCC:** стандартные инструменты используются в командной строке для анализа дискового и сетевого ввода/вывода и выполнения процессов, когда это необходимо. Некоторые инструменты BCC автоматически используются системой мониторинга производительности, чтобы сформировать тепловые карты задержек планировщика и дискового ввода/вывода, флейм-графики распределения процессорного времени и т. д. Кроме того, пользовательский инструмент BCC всегда работает как демон (на основе tcplife(8)), регистрируя сетевые события в облачном хранилище для анализа потока данных.

¹ На основе инструментов в официальной репозитории и в моей репозитории для книги о BPF.

- **bpfftrace**: пользовательские инструменты bpfftrace разрабатываются по мере необходимости для изучения патологий в ядре и приложениях.

В следующих разделах подробно описываются инструменты BCC и bpfftrace, а также приемы программирования на bpfftrace.

15.1. BCC

Коллекция компиляторов BPF (BPF Compiler Collection, BCC) — это проект с открытым исходным кодом, содержащий большой набор инструментов расширенного анализа производительности, а также платформу для их создания. BCC был создан Бренденом Бланко (Brenden Blanco). Я помогал ему в разработке и сам создал множество инструментов для трассировки.

Примером инструмента на основе BCC может служить **biolatency(8)**, показывающий распределение задержки дискового ввода/вывода в виде гистограммы с масштабом, кратным степени двойки, и с возможностью разбивки по флагам ввода/вывода:

```
# biolatency.py -mF
Tracing block device I/O... Hit Ctrl-C to end.
^C

flags = Priority-Metadata-Read
  msec      : count      distribution
  0 -> 1    : 90         | ***** |
flags = Write
  msec      : count      distribution
  0 -> 1    : 24         | ***** |
  2 -> 3    : 0          |         |
  4 -> 7    : 8          | ***** |
flags = ReadAhead-Read
  msec      : count      distribution
  0 -> 1    : 3031       | ***** |
  2 -> 3    : 10         |         |
  4 -> 7    : 5          |         |
  8 -> 15   : 3          |         |
```

Здесь видно бимодальное распределение задержек в операциях записи и множество операций ввода/вывода с флагами «ReadAhead-Read». Этот инструмент использует BPF для создания гистограмм в пространстве ядра, чтобы достичь большей эффективности. Поэтому компоненту, действующему в пространстве пользователя, остается только прочитать уже готовые гистограммы (счетчики) и вывести их.

Инструменты на основе BCC обычно предусматривают вывод сообщения о порядке использования (-h), у них есть свои страницы в справочном руководстве man и файлы с примерами в репозитории BCC:

<https://github.com/iovisor/bcc>

В этом разделе кратко описаны BCC и его специализированные и многоцелевые инструменты анализа производительности.

15.1.1. Установка

Пакеты BCC доступны для многих дистрибутивов Linux, включая Ubuntu, Debian, RHEL, Fedora и Amazon Linux, что весьма упрощает установку. Найдите пакет с названием «bcc-tools», «bpfcc-tools» или «bcc» (в разных дистрибутивах пакет называется по-разному).

Можно скомпилировать BCC из исходного кода. Актуальные инструкции по установке и сборке ищите в файле `INSTALL.md` в репозитории BCC [Iovisor 20b]. В `INSTALL.md` перечислены требования к конфигурации ядра (включая `CONFIG_BPF=y`, `CONFIG_BPF_SYSCALL=y`, `CONFIG_BPF_EVENTS=y`). Для некоторых инструментов BCC требуется версия Linux не ниже 4.4, но для большинства инструментов нужна версия 4.9 или выше.

15.1.2. Покрытие инструментами

Инструменты трассировки BCC изображены на рис. 15.2 (некоторые сгруппированы с использованием подстановочных знаков: например, под `java*` подразумеваются все инструменты, названия которых начинаются с «`java`»).

Многие из них — специализированные, они обозначены однонаправленной стрелкой, другие — многоцелевые, они перечислены слева рядом с двунаправленной стрелкой, показывающей покрытие.

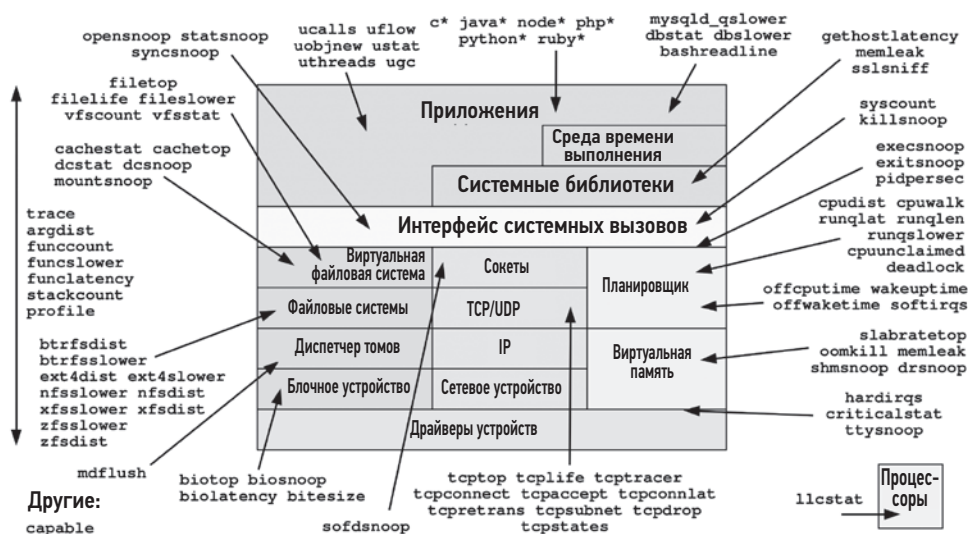


Рис. 15.2. Инструменты BCC

15.1.3. Специализированные инструменты

Многие из этих инструментов я разрабатывал, следуя все той же философии «делать что-то одно, и делать это хорошо». Согласно этой философии, выходные данные должны быть по умолчанию краткими и очевидными. Вы можете «просто запустить biolatency», не вдаваясь в изучение параметров командной строки, и получить достаточно информации для решения проблемы. Параметры обычно предназначены для тонкой настройки. Например, параметр `-F` команды `biolatency(8)` служит для разбивки результатов по флагам ввода/вывода, как было показано выше.

В табл. 15.2 перечислены некоторые специализированные инструменты, а также номера разделов этой книги, где они упоминаются. Полный список см. в репозитории BCC [Iovisor 20a].

Таблица 15.2. Некоторые специализированные инструменты BCC

Инструмент	Описание	Раздел
<code>biolatency(8)</code>	Обобщает информацию о задержках блочного (дискового) ввода/вывода и представляет ее в виде гистограммы	9.6.6
<code>biotop(8)</code>	Обобщает информацию о блочном вводе/выводе по процессам	9.6.8
<code>biosnoop(8)</code>	Трассирует блочный ввод/вывод, сообщает информацию о задержках и другие детали	9.6.7
<code>bitesize(8)</code>	Обобщает информацию о размерах блочного ввода/вывода по процессам и представляет ее в виде гистограммы	—
<code>btfsdist(8)</code>	Обобщает информацию о работе файловой системы <code>btfs</code> и представляет ее в виде гистограммы	8.6.13
<code>btfs slower(8)</code>	Трассирует медленные операции <code>btfs</code>	8.6.14
<code>cpudist(8)</code>	Обобщает время, проведенное потоками выполнения на процессоре и вне процессора, и представляет ее в виде гистограммы	6.6.15, 16.1.7
<code>cpuunclaimed(8)</code>	Показывает процессоры, которые простаивают, несмотря на спрос	—
<code>criticalstat(8)</code>	Трассирует долго выполняющиеся атомарные критические секции в коде ядра	—
<code>db slower(8)</code>	Трассирует долго обрабатываемые запросы к базе данных	—
<code>dbstat(8)</code>	Обобщает информацию о задержках в обработке запросов к базе данных и представляет ее в виде гистограммы	—
<code>drsnop(8)</code>	Трассирует события прямого освобождения памяти и показывает затронутые процессы и задержки	7.5.11
<code>hexesnop(8)</code>	Трассирует запуск новых процессов через обращение к системному вызову <code>hexesve(2)</code>	1.7.3, 5.5.5
<code>ext4dist(8)</code>	Обобщает информацию о задержках в файловой системе <code>ext4</code> и представляет ее в виде гистограммы	8.6.13
<code>ext4 slower(8)</code>	Трассирует медленные операции <code>ext4</code>	8.6.14

Таблица 15.2 (продолжение)

Инструмент	Описание	Раздел
filelife(8)	Трассирует продолжительность жизни короткоживущих файлов	—
gethostlatency(8)	Трассирует задержки обращений к службе разрешения имен (DNS) через вызовы библиотечных функций	—
hardirqs(8)	Обобщает время, затраченное на обработку аппаратных прерываний	6.6.19
killsnoop(8)	Трассирует сигналы, посылаемые через системный вызов kill(2)	—
klockstat(8)	Обобщает статистики использования мьютексов ядра	—
llcstat(8)	Обобщает статистики обращений и промахов кэша процессора по процессам	—
memleak(8)	Показывает операции выделения памяти, не имеющие парных им операций освобождения памяти	—
mysqld_qslower(8)	Трассирует долго обрабатывающиеся запросы к базе данных MySQL	—
nfsdist(8)	Обобщает информацию о задержках в сетевой файловой системе NFS и представляет ее в виде гистограммы	8.6.13
nfsslower(8)	Трассирует медленные операции NFS	8.6.14
offcputime(8)	Обобщает время, проведенное вне процессора, по трассировкам стека	5.5.3
offwaketime(8)	Обобщает время ожидания на блокировках по трассировкам стека	—
oomkill(8)	Трассирует события механизма OOM Killer, срабатывающего в случае нехватки памяти	—
opensnoop(8)	Трассирует системные вызовы из семейства open(2)	8.6.10
profile(8)	Профилирует потребление процессора, отбирая образцы трассировок стека через регулярные интервалы времени	5.5.2
runqlat(8)	Обобщает информацию о задержках в очереди на выполнение (планировщика) и представляет ее в виде гистограммы	6.6.16
runqlen(8)	Обобщает информацию о размерах очередей на выполнение, выбирая их через регулярные интервалы времени, и представляет ее в виде гистограммы	6.6.17
runqslower(8)	Трассирует длительные задержки в очереди на выполнение	—
syncsnoop(8)	Трассирует системные вызовы из семейства sync(2)	—
syscount(8)	Обобщает информацию о количестве обращений к системным вызовам и задержках в них	5.5.6
tcpife(8)	Трассирует сеансы TCP и обобщает продолжительность их жизни	10.6.9
tcpretrans(8)	Трассирует повторные передачи TCP и выводит некоторые подробности, включая состояние ядра	10.6.11
tcptop(8)	Обобщает информацию об объеме приема/передачи по протоколу TCP с разбивкой по процессам и хостам	10.6.10
wakeuptime(8)	Обобщает время ожидания до возобновления по трассировкам стека	—

Инструмент	Описание	Раздел
xfsdist(8)	Обобщает информацию о задержках в файловой системе xfs и представляет ее в виде гистограммы	8.6.13
xfsslower(8)	Трассирует медленные операции xfs	8.6.14
zfsdist(8)	Обобщает информацию о задержках в файловой системе zfs и представляет ее в виде гистограммы	8.6.13
zfsslower(8)	Трассирует медленные операции zfs	8.6.14

Примеры использования этих инструментов ищите в предыдущих главах, а также в файлах *_example.txt в репозитории BCC (многие из которых написал я). Информацию об инструментах, не описанных в этой книге, ищите в [Gregg 19].

15.1.4. Многоцелевые инструменты

Многоцелевые инструменты перечислены слева на рис. 15.2. Каждый из них поддерживает несколько источников событий и может играть множество ролей, как и perf(1), что помимо прочего усложняет их использование. Они описываются в табл. 15.3.

Таблица 15.3. Многоцелевые инструменты BCC

Инструмент	Описание	Раздел
argdist(8)	Выводит значения параметров функций в виде гистограммы или счетчиков	15.1.15
funccount(8)	Подсчитывает вызовы функций в пространстве ядра или пользователя	15.1.15
funclslower(8)	Трассирует медленные функции в пространстве ядра или пользователя	—
funclatency(8)	Обобщает информацию о задержках в функциях и представляет ее в виде гистограммы	—
stackcount(8)	Подсчитывает трассировки стека, приводящие к заданному событию	15.1.15
trace(8)	Трассирует произвольные функции с фильтрами	15.1.15

Чтобы облегчить использование этих инструментов, вы можете собрать свою коллекцию однострочных сценариев. Некоторые из таких сценариев, похожие на однострочные сценарии для perf(1) и trace-cmd, приводятся в следующем разделе.

15.1.5. Однострочные сценарии

Однострочные сценарии ниже выполняют трассировку в масштабе всей системы, пока не будет нажато Ctrl-C, если явно не указано иное. Они объединены в группы по используемому инструменту.

funccount(8)

Подсчитать вызовы функций VFS в ядре:

```
funcgraph 'vfs_*
```

Подсчитать вызовы функций TCP в ядре:

```
funccount 'tcp_*
```

Подсчитать количество операций отправки TCP в секунду:

```
funccount -i 1 'tcp_send*'
```

Показать частоту следования событий блочного ввода/вывода в секунду:

```
funccount -i 1 't:block:*
```

Показать частоту вызовов getaddrinfo() (разрешение имен) из libc в секунду:

```
funccount -i 1 c:getaddrinfo
```

stackcount(8)

Подсчитать трассировки стека, ведущие к блочному вводу/выводу:

```
stackcount t:block:block_rq_insert
```

Подсчитать трассировки стека, ведущие к отправке IP-пакетов, с разбивкой по процессам:

```
stackcount -P ip_output
```

Подсчитать трассировки стека, ведущие к блокировке потока выполнения и оставлению им процессора:

```
stackcount t:sched:sched_switch
```

trace(8)

Трассировать функцию ядра do_sys_open() и вывести имя файла:

```
trace 'do_sys_open "%s", arg2'
```

Трассировать значение, возвращаемое функцией ядра do_sys_open(), и вывести его:

```
trace 'r::do_sys_open "ret: %d", retval'
```

Трассировать функцию ядра do_nanosleep() и вывести значение параметра режима и трассировки стека в пространстве пользователя:

```
trace -U 'do_nanosleep "mode: %d", arg2'
```

Трассировать запросы на аутентификацию через библиотеку pam:

```
trace 'pam:pam_start "%s: %s", arg1, arg2'
```

argdist(8)

Обобщить операции чтения VFS по возвращаемому значению (размер или код ошибки):

```
argdist -H 'r::vfs_read()'
```

Обобщить вызовы read() из libc по возвращаемому значению (размер или код ошибки) для процесса с PID 1005:

```
argdist -p 1005 -H 'r:c:read()'
```

Подсчитать обращения к системным вызовам с разбивкой по их идентификаторам:

```
argdist.py -C 't:raw_syscalls:sys_enter():int:args->id'
```

Обобщить вызовы функции ядра tcp_sendmsg() по аргументу size и представить результаты в виде счетчиков:

```
argdist -C 'p::tcp_sendmsg(struct sock *sk, struct msghdr *msg,
size_t size):u32:size'
```

Обобщить вызовы функции ядра tcp_sendmsg() по аргументу size и представить результаты в виде гистограммы с масштабом, кратным степени двойки:

```
argdist -H 'p::tcp_sendmsg(struct sock *sk, struct msghdr *msg,
size_t size):u32:size'
```

Подсчитать вызовы write() из libc, выполненные процессом с PID 181, с разбивкой по файловым дескрипторам:

```
argdist -p 181 -C 'p:c:write(int fd):int:fd'
```

Обобщить операции чтения, длившиеся дольше 100 мкс, с разбивкой по процессам:

```
argdist -C 'r::__vfs_read():u32:$PID:$latency > 100000'
```

15.1.6. Пример многоцелевого инструмента

В качестве примера использования многоцелевого инструмента показано применение trace(8), где с его помощью трассируется функция ядра do_sys_open() и выводится второй аргумент в виде строки:

```
# trace 'do_sys_open "%s", arg2'
PID    TID    COMM      FUNC          -
28887  28887  1s        do_sys_open   /etc/ld.so.cache
28887  28887  1s        do_sys_open   /lib/x86_64-linux-gnu/libselinux.so.1
28887  28887  1s        do_sys_open   /lib/x86_64-linux-gnu/libc.so.6
28887  28887  1s        do_sys_open   /lib/x86_64-linux-gnu/libpcre2-8.so.0
28887  28887  1s        do_sys_open   /lib/x86_64-linux-gnu/libdl.so.2
28887  28887  1s        do_sys_open   /lib/x86_64-linux-gnu/libpthread.so.0
28887  28887  1s        do_sys_open   /proc/filesystems
28887  28887  1s        do_sys_open   /usr/lib/locale/locale-archive
[...]
```

Синтаксис `trace(8)` основан на `printf(3)` и поддерживает строку формата и аргументы. Здесь `arg2`, второй аргумент, выводится в виде строки, потому что содержит имя файла.

Оба инструмента, `trace(8)` и `argdist(8)`, поддерживают синтаксис, позволяющий создавать множество гибких однострочных сценариев. `bpfttrace`, описанный в следующих разделах, идет еще дальше, предлагая полноценный язык программирования для создания однострочных и многострочных программ.

15.1.7. Сравнение BCC и bpfttrace

Различия между этими двумя интерфейсами кратко изложены в начале главы. BCC лучше подходит для создания гибких и сложных инструментов, поддерживающих множество аргументов или использующих множество библиотек. `bpfttrace` лучше подходит для однострочных или коротких сценариев без параметров или с одним целочисленным параметром. BCC позволяет разрабатывать программы для BPF на языке C, обеспечивая полный контроль. Но за большие возможности приходится платить сложностью: на разработку инструмента BCC может потребоваться в десять раз больше времени, чем на разработку инструмента на `bpfttrace`, а его исходный код может включать в десять раз больше строк. Обычно разработка инструмента выполняется в несколько итераций, и я обнаружил, что гораздо эффективнее разработать сначала инструмент на `bpfttrace`, а затем, если понадобится, перенести в BCC.

Разница между BCC и `bpfttrace` — это как разница между программированием на C и языке командной оболочки, где BCC можно сравнить с программированием на C (некоторые инструменты *фактически* пишутся на C), а `bpfttrace` — с языком командной оболочки. В обычной работе я использую множество готовых программ на C (`top(1)`, `vmstat(1)` и т. д.) и пишу одноразовые сценарии командной оболочки. Точно так же я использую множество встроенных инструментов BCC и разрабатываю одноразовые инструменты на `bpfttrace`.

В этой книге я показал достаточно примеров такого подхода: во многих главах есть примеры использования инструментов BCC, а в следующих разделах этой главы вы увидите, как можно создавать собственные инструменты на `bpfttrace`.

15.1.8. Документация

Обычно инструменты выводят сообщение о порядке использования, описывающее их синтаксис. Например:

```
# funccount -h
использование: funccount [-h] [-p PID] [-i INTERVAL] [-d DURATION] [-T] [-r] [-D]
                    pattern
```

Подсчитывает вызовы функций, срабатывания точек трассировки и зондов USDT

позиционные аргументы:

pattern шаблон для поиска событий

необязательные аргументы:

```
-h, --help          вывести это справочное сообщение и выйти
-p PID, --pid PID   трассировать только процесс с этим PID
-i INTERVAL, --interval INTERVAL
                    выводить сводку через указанный интервал в секундах
-d DURATION, --duration DURATION
                    продолжительность трассировки в секундах
-T, --timestamp     включить в вывод отметки времени
-r, --regex          использовать регулярные выражения. По умолчанию
                    допускается использовать только подстановочный
                    символ "*".
-D, --debug         вывести программу BPF перед запуском (для отладки)
```

примеры:

```
./funccount 'vfs_*'          # подсчет вызовов функций ядра с именами,
                             # начинающимися с "vfs"
./funccount -r '^vfs.*'      # то же, что и выше, но с использованием
                             # регулярного выражения
./funccount -Ti 5 'vfs_*'    # выводить сводку с отметками времени
                             # каждые 5 с
./funccount -d 10 'vfs_*'    # трассировать 10 с
./funccount -p 185 'vfs_*'   # подсчет вызовов vfs только для PID 181
./funccount t:sched:sched_fork # подсчет срабатываний точки трассировки
                             # sched_fork
./funccount -p 185 u:node:gc* # подсчет срабатываний всех USDT-зондов GC
                             # на узле для PID 185
./funccount c:malloc         # подсчет всех вызовов malloc() из libc
./funccount go:os.*          # подсчет всех вызовов "os.*" из libgo
./funccount -p 185 go:os.*   # подсчет всех вызовов "os.*" из libgo
                             # для PID 185
./funccount ./test:read*     # подсчет вызовов "read*" в двоичном файле
                             # ./test
```

Для каждого инструмента есть своя страница в справочном руководстве (man/man8/funccount.8) и файл с примерами (examples/funccount_example.txt) в репозитории bcc. Файлы с примерами содержат примеры вывода с комментариями.

Я также добавил следующую документацию в репозиторий BCC [Iovisor 20b]:

- tutorial для конечных пользователей: docs/tutorial.md;
- tutorial для разработчиков BCC: docs/tutorial_bcc_python_developer.md;
- справочник: docs/reference_guide.md.

Глава 4 в моей книге «BPF Performance Tools» полностью посвящена BCC [Gregg 19].

15.2. BPFTRACE

bpftrace — это трассировщик с открытым исходным кодом, основанный на BPF и BCC. Он включает не только набор инструментов для анализа производительности,

но и язык высокого уровня для разработки новых инструментов. Язык проектировался с прицелом на простоту изучения. Это аналог `awk(1)` в мире трассировки, и он основан на `awk(1)`. Используя `awk(1)`, вы пишете короткую программу для обработки входной строки, а используя `bpfftrace` — пишете короткую программу для обработки события. `bpfftrace` был создан Аластером Робертсоном (Alastair Robertson), и я стал одним из основных участников этого проекта.

Ниже приводится однострочный сценарий, отображающий распределение размеров принимаемых сообщений TCP с разбивкой по именам процессов:

```
# bpfftrace -e 'kr:tcp_recvmmsg /retval >= 0/ { @recv_bytes[comm] = hist(retval); }'
Attaching 1 probe...
^C

@recv_bytes[sshd]:
[32, 64)          7 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[64, 128)         2 | @@@@@@@@@@@@@@@@@@ |

@recv_bytes[nodejs]:
[0]              82 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[1]             135 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[2, 4)          153 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[4, 8)           12 | @@@ |
[8, 16)           6 | @ |
[16, 32)          32 | @@@@@@@@@@@@@@ |
[32, 64)          158 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[64, 128)         155 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |
[128, 256)        14 | @@@@ |
```

Как показывает этот вывод, распределение размеров сообщений, принимаемых процессом `nodejs`, имеет бимодальный характер, одна мода находится в диапазоне от 0 до 4 байт, а другая в диапазоне от 32 до 128 байт.

Этот компактный однострочный сценарий на `bpfftrace` инициализирует `kretprobe` в функции `tcp_recvmmsg()`, фильтрует отрицательные возвращаемые значения (чтобы избавиться от отрицательных кодов ошибок) и заполняет объект карты BPF с именем `@recv_bytes` гистограммами возвращаемых значений, сохраненными с использованием имени процесса (`comm`) в качестве ключа. После нажатия `Ctrl-C` `bpfftrace` получает сигнал (`SIGINT`) и завершается, автоматически распечатывая карту BPF. Этот синтаксис более подробно объясняется в следующих разделах.

Но `bpfftrace` позволяет не только писать свои однострочные сценарии, но и включает множество готовых инструментов, которые можно найти в репозитории проекта:

<https://github.com/iovisor/bpfftrace>

В этом разделе кратко описаны инструменты и язык программирования `bpfftrace`. За основу взята информация из моей книги [Gregg 19], где `bpfftrace` рассматривается более подробно.

15.2.1. Установка

Пакеты bpftrace доступны для многих дистрибутивов Linux, включая Ubuntu, что весьма упрощает установку. Найдите пакет с именем «bpftrace». Такие пакеты есть для Ubuntu, Fedora, Gentoo, Debian, OpenSUSE и CentOS. В RHEL 8.2 bpftrace включен как технология для предварительного ознакомления.

Помимо пакетов, есть образы Docker с bpftrace, двоичные файлы bpftrace, не требующие никаких зависимостей, кроме glibc, и инструкции по сборке bpftrace из исходного кода. Описание этих вариантов установки можно найти в файле INSTALL.md в репозитории bpftrace [Iovisor 20a], где перечисляются требования к конфигурации ядра (включая CONFIG_BPF=y, CONFIG_BPF_SYSCALL=y, CONFIG_BPF_EVENTS=y). Для bpftrace требуется версия Linux 4.9 или выше.

15.2.2. Инструменты

Инструменты трассировки bpftrace изображены на рис. 15.3.

Инструменты, которые можно найти в репозитории bpftrace, показаны черным цветом. Для моей предыдущей книги я разработал еще множество инструментов на bpftrace и опубликовал их исходный код в репозитории bpf-perf-tools-book: они показаны серым цветом [Gregg 19g].



Рис. 15.3. Инструменты bpftrace

15.2.3. Однострочные сценарии

Однострочные сценарии ниже выполняют трассировку в масштабе всей системы, пока не будет нажато Ctrl-C, если явно не указано иное. Они не только несут практическую пользу, но также служат мини-примерами программирования на языке bpftrace. Сценарии сгруппированы по целям. Более длинные списки однострочных сценариев на bpftrace ищите в главах, посвященных конкретным ресурсам.

Процессоры

Трассировать запуск новых процессов с выводом их аргументов:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_execve { join(args->argv); }'
```

Подсчитать обращения к системным вызовам с разбивкой по процессам:

```
bpftrace -e 'tracepoint:raw_syscalls:sys_enter { @[pid, comm] = count(); }'
```

Выбирать трассировки стека в пространстве пользователя с частотой 49 Гц для PID 189:

```
bpftrace -e 'profile:hz:49 /pid == 189/ { @[ustack] = count(); }'
```

Память

Подсчитать события расширения кучи по процессам (brk()) по трассировкам стека:

```
bpftrace -e tracepoint:syscalls:sys_enter_brk { @[ustack, comm] = count(); }
```

Подсчитать сбои страниц по трассировкам стека в пространстве пользователя:

```
bpftrace -e 'tracepoint:exceptions:page_fault_user { @[ustack, comm] =  
count(); }'
```

Подсчитать операции в vmscan с использованием точек трассировки:

```
bpftrace -e 'tracepoint:vmscan:* { @[probe]++; }'
```

Файловые системы

Трассировать операции открытия файлов обращением к openat(2) с разбивкой по именам процессов:

```
bpftrace -e 't:syscalls:sys_enter_openat { printf("%s %s\n", comm,  
str(args->filename)); }'
```

Показать распределение вызовов read() по размерам прочитанных блоков (и кодам ошибок):

```
bpftrace -e 'tracepoint:syscalls:sys_exit_read { @ = hist(args->ret); }'
```

Подсчитать вызовы VFS:

```
bpftrace -e 'kprobe:vfs_* { @[probe] = count(); }'
```

Подсчитать количество прохождений через точки трассировки в ext4:

```
bpftrace -e 'tracepoint:ext4:* { @[probe] = count(); }'
```

Диск

Показать гистограмму с распределением объемов данных, вовлеченных в операции блочного ввода/вывода:

```
bpftrace -e 't:block:block_rq_issue { @bytes = hist(args->bytes); }'
```

Подсчитать количество запросов на выполнение блочного ввода/вывода по трассировкам стека в пространстве пользователя:

```
bpftrace -e 't:block:block_rq_issue { @[ustack] = count(); }'
```

Подсчитать операции блочного ввода/вывода по флагам типа:

```
bpftrace -e 't:block:block_rq_issue { @[args->rwbs] = count(); }'
```

Сеть

Подсчитать вызовы accept(2) с разбивкой по идентификаторам (PID) и именам процессов:

```
bpftrace -e 't:syscalls:sys_enter_accept* { @[pid, comm] = count(); }'
```

Подсчитать байты, вовлеченные в операции приема/передачи с сокетами с разбивкой по идентификаторам (PID) и именам процессов, выполняющихся на процессоре:

```
bpftrace -e 'kr:sock_sendmsg,kr:sock_recvmsg /retval > 0/ {
    @[pid, comm] = sum(retval); }'
```

Показать гистограмму распределения размеров отправленных сообщений TCP в байтах:

```
bpftrace -e 'k:tcp_sendmsg { @send_bytes = hist(arg2); }'
```

Показать гистограмму распределения размеров полученных сообщений TCP в байтах:

```
bpftrace -e 'kr:tcp_recvmsg /retval >= 0/ { @recv_bytes = hist(retval); }'
```

Показать гистограмму с размерами отправленных пакетов UDP:

```
bpftrace -e 'k:udp_sendmsg { @send_bytes = hist(arg2); }'
```

Приложения

Суммировать объем памяти в байтах, выделяемой вызовом `malloc()`, с разбивкой по трассировкам стека в пространстве пользователя (имеет высокий оверхед):

```
bpfttrace -e 'u:/lib/x86_64-linux-gnu/libc-2.27.so:malloc { @[ustack(5)] =
    sum(arg0); }'
```

Трассировать вызовы `kill()` для отправки сигналов и показать имя процесса-отправителя, идентификатор (PID) получателя и номер сигнала:

```
bpfttrace -e 't:syscalls:sys_enter_kill { printf("%s -> PID %d SIG %d\n",
    comm, args->pid, args->sig); }'
```

Ядро

Подсчитать обращения к системным вызовам по их именам:

```
bpfttrace -e 'tracepoint:raw_syscalls:sys_enter {
    @[ksym(*(kaddr("sys_call_table") + args->id * 8))] = count(); }'
```

Подсчитать вызовы функций ядра с именами, начинающимися с «attach»:

```
bpfttrace -e 'kprobe:attach* { @[probe] = count(); }'
```

Показать гистограмму с распределением значений в третьем аргументе `vfs_write()` (размер):

```
bpfttrace -e 'kprobe:vfs_write { @[arg2] = count(); }'
```

Измерить время выполнения функции ядра `vfs_read()` и вывести результаты в виде гистограммы:

```
bpfttrace -e 'k:vfs_read { @ts[tid] = nsecs; } kr:vfs_read /@ts[tid]/ {
    @ = hist(nsecs - @ts[tid]); delete(@ts[tid]); }'
```

Подсчитать количество переключений контекста по трассировкам стека:

```
bpfttrace -e 't:sched:sched_switch { @[kstack, ustack, comm] = count(); }'
```

Выбирать трассировки стека в пространстве ядра с частотой 99 Гц, исключая поток, обслуживающий бездействие системы:

```
bpfttrace -e 'profile:hz:99 /pid/ { @[kstack] = count(); }'
```

15.2.4. Программирование

В этом разделе — краткое руководство по использованию `bpfttrace` и программированию на языке `bpfttrace`. Оформление раздела заимствовано из оригинальной статьи с описанием `awk` [Aho 78], [Aho 88], уместившейся на шести страницах. Сам язык `bpfttrace` создавался по образу и подобию `awk` и `C`, а также трассировщиков — `DTrace` и `SystemTap`.

Ниже пример сценария на bpftrace, который измеряет время выполнения функции ядра `vfs_read()` и выводит результаты в виде гистограммы.

```
#!/usr/local/bin/bpftrace

// эта программа измеряет время выполнения vfs_read()

kprobe:vfs_read
{
    @start[tid] = nsecs;
}

kretprobe:vfs_read
/@start[tid]/
{
    $duration_us = (nsecs - @start[tid]) / 1000;
    @us = hist($duration_us);
    delete(@start[tid]);
}
```

В подразделах ниже подробно описаны компоненты инструмента bpftrace, рассматривайте это как tutorial. В разделе 15.2.5 «Справочник» описаны типы зондов, проверки, операторы, переменные, функции и типы карт.

1. Использование

Команда

```
bpftrace -e program
```

выполнит программу `program` и инструментирует любые события, которые в ней определены. Программа будет выполняться до нажатия Ctrl-C или пока не будет вызвана функция `exit()`. Программа на bpftrace, которая передается как аргумент в параметре `-e`, называется *однострочным сценарием*. Однако программу можно сохранить в файл и запустить командой:

```
bpftrace file.bt
```

Расширение `.bt` необязательное, но помогает идентифицировать программы. Добавив в начало файла строку, определяющую интерпретатор¹,

```
#!/usr/local/bin/bpftrace
```

файл с программой можно сделать исполняемым (`chmod a+x file.bt`) и запускать как любую другую программу:

```
./file.bt
```

¹ Некоторые предпочитают использовать `#!/usr/bin/env bpftrace`, чтобы система могла найти bpftrace в любом каталоге, включенном в `$PATH`. Но у утилиты `env(1)` есть проблемы, и в некоторых проектах она больше не используется.

Команда `bpfttrace` должна запускаться пользователем `root` (суперпользователем)¹. В некоторых средах для запуска программы можно использовать командную оболочку `root` непосредственно, тогда как бывает предпочтительнее запускать привилегированные команды через `sudo(1)`:

```
sudo ./file.bt
```

2. Структура программы

Программа на `bpfttrace` определяет собой серию зондов с соответствующими действиями:

```
зонды { действия }
зонды { действия }
...
```

При срабатывании зондов выполняются соответствующие действия. Перед действиями можно указать необязательное выражение фильтра:

```
зонды /фильтр/ { действия }
```

В этом случае действия выполняются, только если выражение фильтра, стоящее перед действиями, вернет истинное значение. Своей структурой это напоминает программу на `awk(1)`:

```
/шаблон/ { действия }
```

Программирование на `awk(1)` напоминает программирование на `bpfttrace`: можно определить несколько блоков действий, и они могут выполняться в любом порядке, когда обнаруживается соответствующий шаблон или срабатывает зонд + выражение фильтра оказывается истинным.

3. Комментарии

В файлах программ на `bpfttrace` можно вставлять однострочные комментарии, начинающиеся с «`//`»:

```
// это комментарий
```

Комментарии не выполняются. Также можно использовать многострочные комментарии в стиле C:

```
/*
 * Это многострочный
 * комментарий.
 */
```

Этот же синтаксис можно использовать, чтобы закомментировать часть строки (например, `/* комментарий */`).

¹ `bpfttrace` проверяет `UID 0`, но возможно, появятся проверки конкретных привилегий.

4. Формат определения зондов

Определение зонда начинается с имени типа зонда, за которым следует иерархия идентификаторов, разделенных двоеточиями:

```
тип:идентификатор1[:идентификатор2[...]]
```

Иерархия зависит от типа зонда. Взгляните на два следующих примера:

```
kprobe:vfs_read
uprobe:/bin/bash:readline
```

Зонды типа `kprobe` инструментируют вызовы функций ядра и требуют указать только один идентификатор: имя функции. Зонды типа `uprobe` инструментируют вызовы функций в пространстве пользователя и требуют указать путь к двоичному файлу и имя функции.

Для выполнения одних и тех же действий перечислите несколько зондов через запятую. Например:

```
зонд1, зонд2, ... { действия }
```

Есть два специальных типа зондов, для которых не нужно указывать дополнительные идентификаторы: `BEGIN` и `END`. Они срабатывают в начале и в конце программы `bpftrace` (как и в `awk(1)`). Вот пример вывода сообщения перед началом трассировки:

```
BEGIN { printf("Tracing. Hit Ctrl-C to end.\n"); }
```

Дополнительную информацию о типах зондов и их использовании ищите в разделе 15.2.5 «Справочник», в подразделе 1 «Типы зондов».

5. Подстановочные знаки в определениях зондов

Некоторые типы зондов допускают использование подстановочных знаков. Определение

```
kprobe:vfs_*
```

инструментирует все зонды `kprobes` (функции ядра) с именами, начинающимися с «`vfs_`». Инструментация слишком большого количества зондов может привести к избыточному оверхеду. Чтобы избежать этого, в `bpftrace` есть параметр настройки, ограничивающий максимальное количество зондов, которые можно активировать. Этот параметр устанавливается с помощью переменной среды `BPFTRACE_MAX_PROBES` (сейчас у него значение по умолчанию 512¹).

¹ Большее количество зондов замедлит запуск и завершение программы на `bpftrace`, так как они обрабатываются последовательно. В будущем предполагается добавить в ядро пакетную обработку зондов, после чего этот предел можно значительно увеличить или вообще убрать.

Вы можете протестировать определения с подстановочными знаками перед использованием, запустив программу командой `bpfttrace -l`, чтобы вывести список зондов, совпавших с определениями:

```
# bpfttrace -l 'kprobe:vfs_*'
kprobe:vfs_fallocate
kprobe:vfs_truncate
kprobe:vfs_open
kprobe:vfs_setpos
kprobe:vfs_llseek
[...]
bpfttrace -l 'kprobe:vfs_*' | wc -l
56
```

Этому определению соответствует 56 зондов. Имя указывается в кавычках, чтобы избежать нежелательной интерпретации подстановочных знаков командной оболочкой.

6. Фильтры

Фильтры — это логические выражения, определяющие возможность выполнения действий. Фильтр

```
/pid == 123/
```

разрешит выполнение действия, только если встроенная переменная `pid` (идентификатор процесса) содержит значение 123.

Если проверка не указана

```
/pid/
```

то фильтр проверяет значение на неравенство нулю (выражение `/pid/` эквивалентно выражению `/pid != 0/`). Фильтры можно комбинировать с помощью логических операторов, таких как логическое И (`&&`). Например, фильтр:

```
/pid > 100 && pid < 1000/
```

требует, чтобы оба выражения оценивались как «истинные».

7. Действия

Действие может быть одной или несколькими инструкциями, разделенными точкой с запятой:

```
{ действие первое; действие второе; действие третье }
```

К последней инструкции можно добавить точку с запятой. Инструкции записываются на языке `bpfttrace`, который похож на C, и могут управлять переменными и вызывать функции `bpfttrace`. Следующее действие:

```
{ $x = 42; printf("$x is %d", $x); }
```


присваивает переменной `$x` значение 42, а затем выводит ее вызовом функции `printf()`. Список других доступных функций ищите в разделе 15.2.5 «Справочник», в подразделах 4 «Функции» и 5 «Функции карты».

8. Hello, World!

После этого краткого введения вы легко поймете, как написать простую программу, которая выводит текст «Hello, World!»:

```
# bpftrace -e 'BEGIN { printf("Hello, World!\n"); }'
Attaching 1 probe...
Hello, World!
^C
```

В файле эту программу можно структурировать так:

```
#!/usr/local/bin/bpftrace

BEGIN
{
    printf("Hello, World!\n");
}
```

Оформление отступов и пустые строки — условие необязательное, но это повышает удобочитаемость кода.

9. Функции

Помимо функции форматированного вывода `printf()`, в число встроенных функций входят также:

- **`exit()`**: выход из программы на `bpftrace`;
- **`str(char *)`**: возвращает строку, находящуюся по адресу в указателе;
- **`system(формат [, аргументы ...])`**: запускает команду в оболочке.

Действие

```
printf("got: %llx %s\n", $x, str($x)); exit();
```

выведет значение переменной `$x` в виде шестнадцатеричного целого числа, а затем интерпретирует ее как указатель на массив символов с завершающим символом `NULL (char *)`, выведет его как строку и завершит выполнение программы.

10. Переменные

Есть три типа переменных: встроенные, временные и карты.

Встроенные переменные заранее определены, доступны в программах на `bpftrace` по умолчанию, как правило, только для чтения и обычно являются источниками информации. К ним относятся: `pid` — идентификатор процесса, `comm` — имя процесса,

`nsecs` — отметка времени в наносекундах и `curtask` — адрес структуры `task_struct` текущего потока.

Временные переменные (scratch variables) могут использоваться для хранения промежуточных результатов вычислений; их имена начинаются префиксом «\$». Имена и тип определяются по первой инструкции присваивания. Инструкции:

```
$x = 1;
$y = "hello";
$z = (struct task_struct *)curtask;
```

объявляют целочисленную переменную `$x`, строковую переменную `$y` и указатель `$z` на структуру `task_struct`. Эти переменные можно использовать только внутри блока с действиями, в котором они были созданы. При попытке сослаться на переменную, которой не было присвоено значение, `bpftool` сообщит об ошибке (это поможет отловить опечатки).

Переменные-карты (map variables) хранятся в хранилище карт BPF; их имена должны начинаться с префикса «@». В этих переменных можно сохранять данные для передачи между действиями. Программа:

```
probe1 { @a = 1; }
probe2 { $x = @a; }
```

присвоит число 1 переменной `@a`, когда возникнет событие `probe1`, а затем, когда возникнет событие `probe2`, присвоит значение `@a` переменной `$x`. Если сначала возникнет событие `probe1`, а затем `probe2`, переменная `$x` получит значение 1, в противном случае — значение 0 (значение по умолчанию для неинициализированной переменной).

После имени переменной-карты можно указать ключ и использовать такие переменные на манер хеш-таблицы (ассоциативного массива). Инструкция:

```
@start[tid] = nsecs;
```

часто используется в реальности: здесь она сохранит значение встроенной переменной `nsecs` в карте с именем `@start` и с ключом `tid` (идентификатором текущего потока). Такой прием позволяет потокам хранить отметки времени, которые не будут затерты другими потоками.

```
@path[pid, $fd] = str(arg0);
```

Это пример карты с несколькими ключами, здесь роль ключей играют значение встроенной переменной `pid` и значение временной переменной `$fd`.

11. Функции карт

Картам можно присваивать специальные функции. Эти функции хранят и выводят данные. Например, инструкция

```
@x = count();
```

подсчитывает события и при выводе вернет значение счетчика. Она использует карту, хранящую данные для каждого процессора отдельно, соответственно, `@x` превращается в специальный объект с типом счетчика. Следующая инструкция тоже подсчитывает события:

```
@x++;
```

Но здесь используется карта с одним общим целочисленным счетчиком, а не со счетчиками для каждого процессора в отдельности. Такие глобальные целочисленные переменные пригодятся в программах, где нужно целое число, а не счетчик. Но имейте в виду, что из-за одновременных изменений такой переменной в нескольких потоках выполнения может возникнуть некоторая погрешность подсчета.

Инструкция:

```
@y = sum($x);
```

суммирует значения переменной `$x` и при выводе вернет общий результат. Инструкция:

```
@z = hist($x);
```

сохранит значение `$x` в гистограмму и при выводе вернет интервальные счетчики и гистограмму в формате ASCII.

Некоторые функции оперируют картами как единым целым. Например:

```
print(@x);
```

выведет карту `@x`. Этот прием можно использовать, например, для вывода содержимого карты через некоторые интервалы времени. Однако он используется нечасто, потому что после завершения `bpftrace` все карты выводятся автоматически¹.

Некоторые функции карт оперируют ключами в картах. Например, инструкция:

```
delete(@start[tid]);
```

удалит из карты `@start` пару ключ/значение с ключом `tid`.

12. Определение продолжительности выполнения `vfs_read()`

Теперь, после знакомства с новыми синтаксическими конструкциями, более сложный пример будет понять проще. Вот программа `vfsread.bt`, которая измеряет продолжительность выполнения функции ядра `vfs_read` и выводит гистограмму распределения продолжительностей в микросекундах:

¹ Кроме того, вывод карты по завершении программы имеет более низкий оверхед, потому что во время выполнения карты обновляются и их вывод может замедлить процедуру обхода карты.

```
#!/usr/local/bin/bpftrace

// эта программа измеряет продолжительность выполнения vfs_read()

kprobe:vfs_read
{
    @start[tid] = nsecs;
}

kretprobe:vfs_read
/@@start[tid]/
{
    $duration_us = (nsecs - @start[tid]) / 1000;
    @us = hist($duration_us);
    delete(@start[tid]);
}
```

Чтобы измерить продолжительность выполнения функции ядра `vfs_read()`, по событию входа в функцию программа сохраняет отметку времени в карте `@start` с ключом, соответствующим идентификатору потока, а затем, по событию выхода, вычисляет разность между текущим временем и временем входа. Фильтр при событии выхода проверяет наличие отметки времени вызова функции, иначе вычисление разности возвращало бы ошибочный результат в случаях, когда трассировка была запущена в момент выполнения `vfs_read()`.

Пример вывода:

```
# bpftrace vfsread.bt
Attaching 2 probes...
^C
```

```
@us:
[0]          23 | @
[1]         138 | @@@@@@@@@@
[2, 4)      538 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[4, 8)     744 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[8, 16)    641 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
[16, 32)   122 | @@@@@@@@@@
[32, 64)    13 | 
[64, 128)   17 | @
[128, 256)   2  | 
[256, 512)   0  | 
[512, 1K)    1  |
```

Программа продолжает выполняться до нажатия Ctrl-C, затем выводит эти строки и завершается. Этой карте с гистограммой было дано имя «us», чтобы в выводе напомнить единицы измерения, потому что имя карты выводится вместе с данными. Давая картам осмысленные имена, такие как «bytes» и «latency_ns», можно аннотировать вывод и сделать его более понятным.

Эту программу можно немного модифицировать. Взгляните на изменения в строке, где присваивается функция `hist()`:

```
@us[pid, comm] = hist($duration us);
```

Теперь для каждой пары имя/идентификатор процесса будет создана своя гистограмма. Традиционные системные инструменты — `iostat(1)` и `vmstat(1)` — имеют фиксированный вывод, который очень непросто настроить. Но с помощью `bpftrace` наблюдаемые метрики можно разбивать на части и дополнять метриками из других событий, пока не будут найдены ответы на интересующие вас вопросы.

См. также главу 8 «Файловые системы», раздел 8.6.15 «`bpftrace`», подраздел «Трассировка VFS», где приводится расширенный пример, разбивающий задержку в `vfs_read()` по типу: файловая система, сокет и т. д.

15.2.5. Справочник

Ниже кратко описаны основные аспекты, связанные с программированием на `bpftrace`: типы зондов, управление потоком выполнения, переменные, функции и функции карт.

1. Типы зондов

В табл. 15.4 перечислены доступные типы зондов. У многих из них короткие псевдонимы, упрощающие создание коротких однострочных сценариев.

Таблица 15.4. Типы зондов в `bpftrace`

Тип	Псевдоним	Описание
<code>tracepoint</code>	<code>t</code>	Точки статической инструментации в коде ядра
<code>usdt</code>	<code>U</code>	Статически определяемые точки трассировки на уровне пользователя
<code>kprobe</code>	<code>k</code>	Динамически определяемые точки инструментации функций ядра
<code>kretprobe</code>	<code>kr</code>	Динамически определяемые точки инструментации выхода из функций ядра
<code>kfunc</code>	<code>f</code>	Динамически определяемые точки инструментации функций ядра (на основе BPF)
<code>kretfunc</code>	<code>fr</code>	Динамически определяемые точки инструментации выхода из функций ядра (на основе BPF)
<code>uprobe</code>	<code>u</code>	Динамически определяемые точки инструментации функций на уровне пользователя
<code>uretprobe</code>	<code>ur</code>	Динамически определяемые точки инструментации выхода из функций на уровне пользователя
<code>software</code>	<code>s</code>	Программные события ядра
<code>hardware</code>	<code>h</code>	Аппаратные события ядра на основе счетчиков
<code>watchpoint</code>	<code>w</code>	Точки наблюдения для инструментации событий обращения к памяти
<code>profile</code>	<code>p</code>	Выборка трассировок стека по всем процессорам с некоторой частотой
<code>interval</code>	<code>i</code>	Вывод отчета по времени (с одного процессора)
<code>BEGIN</code>		Пуск программы на <code>bpftrace</code>
<code>END</code>		Окончание программы на <code>bpftrace</code>

Большинство из этих типов зондов — интерфейсы к существующим механизмам ядра. В главе 4 я объяснил, как работают эти технологии: kprobes, uprobes, точки трассировки, USDT и PMC (счетчики производительности, используются зондами типа hardware). Тип зондов kfunc/kretfunc — это новый интерфейс с низким оверхедом, использующий трамплины eBPF и BTF.

Некоторые зонды, например, инструментирующие события планировщика, распределения памяти и приема/передачи сетевых пакетов, могут срабатывать очень часто. Чтобы уменьшить оверхед, старайтесь решать проблемы, по возможности используя менее частые события. Если вы не знаете, с какой частотой срабатывает зонд, измерьте ее с помощью bpftrace, например, подсчитав через kprobe вызовы `vfs_read()` за одну секунду:

```
# bpftrace -e 'k:vfs_read { @ = count(); } interval:s:1 { exit(); }'
```

Экспериментируйте в течение короткого времени, чтобы минимизировать оверхед, если он окажется значительным. То, что мы считаем высокой или низкой частотой событий, зависит от быстродействия процессоров, их количества и запаса мощности, а также стоимости инструментации зондов. Чтобы задать приблизительный ориентир: для современных компьютеров я бы посчитал низкой частоту менее 100 000 событий в секунду.

Аргументы зондов

Зонды разных типов поддерживают разные типы аргументов для передачи контекста событий. Например, в точках трассировки доступны поля из файла формата, при этом имена полей используются в структуре данных `args`. Следующий сценарий инструментирует точку трассировки `syscalls:sys_enter_read` и использует аргумент `args->count` для записи гистограммы из аргумента `count` (запрошенный размер чтения):

```
bpftrace -e 'tracepoint:syscalls:sys_enter_read { @req_bytes = hist(args->count); }'
```

Узнать, какие поля поддерживаются, можно в файле формата в каталоге `/sys` или выполнив `bpftrace` с параметрами `-lv`:

```
# bpftrace -lv 'tracepoint:syscalls:sys_enter_read'
tracepoint:syscalls:sys_enter_read
    int __syscall_nr;
    unsigned int fd;
    char * buf;
    size_t count;
```

В справочнике по `bpftrace` «bpftrace Reference Guide», доступном в интернете, вы найдете описание каждого типа зондов и их аргументов [Iovisor 20c].

2. Управление потоком выполнения

В `bpftrace` есть три типа проверок условий: фильтры, тернарные операторы и операторы `if`. Все они изменяют ход выполнения программы в соответствии с логическими выражениями, поддерживающими операторы, которые перечислены в табл. 15.5.

Таблица 15.5. Логические операторы в bpftrace

Оператор	Описание
==	Равно
!=	Не равно
>	Больше, чем
<	Меньше, чем
>=	Больше или равно
<=	Меньше или равно
&&	И
	Включающее ИЛИ

Выражения можно группировать с помощью круглых скобок.

Фильтры

Фильтры, представленные выше, определяют, будет ли выполнено действие. Формат:

```
зонд /фильтр/ { действие }
```

В фильтрах можно использовать логические операторы. Фильтр `/pid == 123/` разрешает выполнить действие, только если встроенная переменная `pid` имеет значение 123.

Тернарные операторы

Тернарный оператор — это оператор с тремя элементами: условным выражением и двумя результатами. Формат:

```
проверка ? инструкция_для_пройденной_проверки : инструкция_для_непройденной_проверки
```

Тернарный оператор можно использовать для определения абсолютного значения `$x`:

```
$abs = $x >= 0 ? $x : - $x;
```

Операторы if

Операторы `if` имеют следующий синтаксис:

```
if (проверка) { инструкции_для_пройденной_проверки }
if (test) { инструкции_для_пройденной_проверки } else { инструкции_для_
непройденной_проверки }
```

Один из вариантов использования — выполнение разных действий для IPv4 и IPv6. Например (для простоты игнорируются семейства, отличные от IPv4 и IPv6):

```

if ($inet_family == $AF_INET) {
    // IPv4
    ...
} else {
    // предполагается IPv6
    ...
}

```

Начиная с версии bpftrace v0.10.0, поддерживается также оператор `else if`¹.

Циклы

bpftrace поддерживает циклы, доступные для развертывания с помощью `unroll()`. В ядре Linux версии 5.3 и выше поддерживаются также циклы `while()`²:

```

while (проверка) {
    инструкции
}

```

Они основаны на поддержке циклов в BPF, добавленной в Linux 5.3.

Операторы

В предыдущем разделе были перечислены логические операторы, которые можно использовать в условных выражениях (проверках). Однако bpftrace поддерживает еще ряд операторов, перечисленных в табл. 15.6.

Таблица 15.6. Операторы в bpftrace

Оператор	Описание
=	Присваивание
+, -, *, /	Сложение, вычитание, умножение, деление (только целочисленное)
++, --	Автоинкремент, автодекремент
&, , ^	Битовые И, ИЛИ и ИСКЛЮЧАЮЩЕЕ ИЛИ
!	Логическое НЕ
<<, >>	Логический сдвиг влево, логический сдвиг вправо
+=, -=, *=, /=, %=, &=, ^=, <<=, >>=	Составные операторы присваивания

Эти операторы действуют так же, как и аналогичные операторы в C.

¹ Спасибо Даниэлю Сю (Daniel Xu; PR#1211).

² Спасибо Базу Смиту (Bas Smit) за добавление логики в bpftrace (PR#1066).

3. Переменные

Встроенные переменные, поддерживаемые в bpftrace, обычно доступны только для чтения. В табл. 15.7 перечислены некоторые особенно важные встроенные переменные.

Таблица 15.7. Некоторые встроенные переменные в bpftrace

Встроенная переменная	Тип	Описание
pid	Целое число	Идентификатор процесса (группы потоков ядра)
tid	Целое число	Идентификатор потока выполнения (процесса ядра)
uid	Целое число	Идентификатор пользователя
username	Строка	Имя пользователя
nsecs	Целое число	Отметка времени в наносекундах
elapsed	Целое число	Время в наносекундах, прошедшее с момента инициализации bpftrace
cpu	Целое число	Идентификатор процессора
comm	Строка	Имя процесса
kstack	Строка	Трассировка стека ядра
ustack	Строка	Трассировка стека в пространстве пользователя
arg0, ..., argN	Целое число	Аргументы в зондах некоторых типов
args	Целое число	Аргументы в зондах некоторых типов
sarg0, ..., sargN	Целое число	Аргументы на основе стека в зондах некоторых типов
retval	Целое число	Возвращаемое значение в зондах некоторых типов
func	Строка	Имя трассируемой функции
probe	Строка	Полное имя текущего зонда
curstack	Структура/целое число	Структура task_struct в ядре (либо как task_struct, либо как 64-битное целое без знака, в зависимости от доступности информации о типе)
cgroup	Целое число	Идентификатор cgroup v2 по умолчанию текущего процесса (для сравнения с cgroupid())
\$1, ..., \$N	int, char *	Позиционные параметры программы на bpftrace

Сейчас все целочисленные переменные имеют тип uint64. Все перечисленные переменные относятся к текущему выполняющемуся потоку, зонду, функции и процессору.

Выше в этой главе были показаны встроенные переменные: `retval`, `comm`, `tid` и `nsecs`. Полный и актуальный список встроенных переменных ищите в справочнике по `bpftool` «`bpftool Reference Guide`», доступном в интернете [Iovisor 20c].

4. Функции

В табл. 15.8 перечислены некоторые встроенные функции для решения разных задач. Некоторые из них уже использовались в примерах, например, `printf()`.

Таблица 15.8. Некоторые встроенные функции в `bpftool`

Функция	Описание
<code>printf(char *fmt [, ...])</code>	Форматированный вывод
<code>time(char *fmt)</code>	Форматированный вывод времени
<code>join(char *arr[])</code>	Вывод массива строк через пробел
<code>str(char *s [, int len])</code>	Возвращает строку, находящуюся по указателю <code>s</code> , позволяет ограничить длину строки
<code>buf(void *d [, int length])</code>	Возвращает блок данных, доступный по указателю <code>d</code> , в виде строки шестнадцатеричных чисел
<code>strncmp(char *s1, char *s2, int length)</code>	Сравнивает до <code>length</code> первых символов в строках
<code>sizeof(expression)</code>	Возвращает размер выражения <code>expression</code> или типа данных
<code>kstack([int limit])</code>	Возвращает стек ядра с глубиной до <code>limit</code> фреймов
<code>ustack([int limit])</code>	Возвращает стек в пространстве пользователя с глубиной до <code>limit</code> фреймов
<code>ksym(void *p)</code>	По заданному адресу определяет и возвращает символ ядра в виде строки
<code>usym(void *p)</code>	По заданному адресу определяет и возвращает символ в пространстве пользователя в виде строки
<code>kaddr(char *name)</code>	По заданному символу ядра определяет его адрес
<code>uaddr(char *name)</code>	По заданному символу в пространстве пользователя определяет его адрес
<code>reg(char *name)</code>	Возвращает значение, хранящееся в указанном регистре
<code>ntop([int af,] int addr)</code>	Возвращает строковое представление адреса IPv4/IPv6
<code>cgroupid(char *path)</code>	Возвращает идентификатор контрольной группы (<code>cgroup</code>) по заданному пути (<code>/sys/fs/cgroup/...</code>)
<code>system(char *fmt [, ...])</code>	Выполняет команду оболочки
<code>cat(char *filename)</code>	Выводит содержимое файла
<code>signal(char[] sig u32 sig)</code>	Посылает сигнал текущей задаче (например, <code>SIGTERM</code>)

Функция	Описание
<code>override(u64 rc)</code>	Переопределяет возвращаемое значение в <code>kprobe</code> ¹
<code>exit()</code>	Завершает программу на <code>bpftrace</code>

Некоторые из этих функций действуют асинхронно: ядро ставит событие в очередь и через короткое время обрабатывает его в пространстве пользователя. К числу асинхронных относятся функции: `printf()`, `time()`, `cat()`, `join()` и `system()`. Функции `kstack()`, `ustack()`, `ksym()` и `usym()` действуют синхронно, но трансляцию символов выполняют асинхронно.

Ниже показан пример применения функций `printf()` и `str()` для вывода имени файла, переданного системному вызову `openat(2)`:

```
# bpftrace -e 't:syscalls:sys_enter_open { printf("%s %s\n", comm,
    str(args->filename)); }'
Attaching 1 probe...
top /etc/ld.so.cache
top /lib/x86_64-linux-gnu/libprocps.so.7
top /lib/x86_64-linux-gnu/libtinfo.so.6
top /lib/x86_64-linux-gnu/libc.so.6
[...]
```

Полный и актуальный список функций ищите в справочнике по `bpftrace` «`bpftrace Reference Guide`», доступном в интернете [Iovisor 20c].

5. Функции карт

Карты — это специальные объекты в ВРФ, предназначенные для хранения хеш-таблиц, которые можно использовать для разных целей, например для хранения пар ключ/значение или статистических сводок. Для работы с картами `bpftrace` предоставляет ряд встроенных функций. Наиболее важные из них перечислены в табл. 15.9.

Таблица 15.9. Некоторые функции карт в `bpftrace`

Функция	Описание
<code>count()</code>	Подсчитывает вхождения
<code>sum(int n)</code>	Суммирует указанное значение
<code>avg(int n)</code>	Усредняет указанное значение
<code>min(int n)</code>	Сохраняет минимальное значение
<code>max(int n)</code>	Сохраняет максимальное значение

¹ ВНИМАНИЕ: используйте эту функцию, только если точно знаете, что делаете: небольшая ошибка может вызвать крах ядра.

15.2.6. Документация

В предыдущих главах этой книги можно найти дополнительную информацию о `bpfttrace`:

- глава 5 «Приложения», раздел 5.5.7;
- глава 6 «Процессоры», раздел 6.6.20;
- глава 7 «Память», раздел 7.5.13;
- глава 8 «Файловые системы», раздел 8.6.15;
- глава 9 «Диски», раздел 9.6.11;
- глава 10 «Сеть», раздел 10.6.12.

Примеры использования `bpfttrace` можно найти в главе 4 «Инструменты наблюдения» и в главе 11 «Облачные вычисления».

В репозитории `bpfttrace` доступна следующая документация:

- справочник: `docs/reference_guide.md` [Iovisor 20c];
- туториал: `docs/tutorial_one_liners.md` [Iovisor 20d].

Подробную информацию о `bpfttrace` ищите в моей книге «BPF Performance Tools» [Gregg 19], где в главе 5 «`bpfttrace`» на множестве примеров показаны возможности языка программирования `bpfttrace`, а в последующих главах представлено еще больше программ на `bpfttrace` для разных целей.

Обратите внимание, что некоторые возможности `bpfttrace`, описанные в [Gregg 19] как «запланированные», уже были добавлены в `bpfttrace` и включены в эту главу. Это циклы `while()`, операторы `else-if`, `signal()`, `override()` и события точек наблюдения. В числе других дополнений, появившихся в `bpfttrace`, тип зондов `kfunc` и функции `buf()` и `sizeof()`. Ознакомьтесь с примечаниями к релизу в репозитории `bpfttrace`, где перечислены будущие дополнения, хотя их не так много: `bpfttrace` уже обладает достаточно широкими возможностями и на нем написано более 120 инструментов.

15.3. ССЫЛКИ

[Aho 78] Aho, A. V., Kernighan, B. W., and Weinberger, P. J., «Awk: A Pattern Scanning and Processing Language (Second Edition)», Unix 7th Edition man pages, 1978. Доступно в Интернете по адресу: <http://plan9.bell-labs.com/7thEdMan/index.html>.

[Aho 88] Aho, A. V., Kernighan, B. W., and Weinberger, P. J., «The AWK Programming Language», Addison Wesley, 1988.

[Gregg 18e] Gregg, B., «YOW! 2018 Cloud Performance Root Cause Analysis at Netflix», <http://www.brendangregg.com/blog/2019-04-26/yow2018-cloud-performance-netflix.html>, 2018.

- [Gregg 19] Gregg, B., «BPF Performance Tools: Linux System and Application Observability»¹, Addison-Wesley, 2019.
- [Gregg 19g] Gregg, B., «BPF Performance Tools (book): Tools», <http://www.brendangregg.com/bpf-performance-tools-book.html#tools>, 2019.
- [Iovisor 20a] «bpfttrace: High-level Tracing Language for Linux eBPF», <https://github.com/iovisor/bpfttrace>, последнее обновление 2020.
- [Iovisor 20b] «BCC - Tools for BPF-based Linux IO Analysis, Networking, Monitoring, and More», <https://github.com/iovisor/bcc>, последнее обновление 2020.
- [Iovisor 20c] «bpfttrace Reference Guide», https://github.com/iovisor/bpfttrace/blob/master/docs/reference_guide.md, последнее обновление 2020.
- [Iovisor 20d] Gregg, B., et al., «The bpfttrace One-Liner Tutorial», https://github.com/iovisor/bpfttrace/blob/master/docs/tutorial_one_liners.md, последнее обновление 2020.

¹ Грегг Б. «BPF: Профессиональная оценка производительности». Выходит в издательстве «Питер» в 2023 году.

Глава 16

ПРИМЕР ИЗ ПРАКТИКИ

В этой главе описан пример исследования производительности системы. Я расскажу о реальной проблеме — от первоначального отчета до окончательного решения. Эта проблема возникла в производственной облачной среде, и я выбрал ее как наглядный пример анализа производительности системы.

Я не буду вводить здесь никаких новых технических понятий. Рассказанная история покажет вам, как можно применять инструменты и методологии на практике, в реальной рабочей среде. Это будет особенно полезно новичкам, которым по жизни еще предстоит столкнуться с проблемами производительности. Они смогут узнать, как такие проблемы решают эксперты, о чем эти эксперты думают во время анализа и почему. Моя история не документирование наилучшего подхода, а скорее объяснение, почему тот или иной подход был выбран.

16.1. НЕОБЪЯСНИМЫЙ ВЫИГРЫШ

В Netflix протестировали микросервис на новой платформе на основе контейнеров и обнаружили, что задержка обработки запроса снизилась в 3–4 раза. У контейнерной платформы множество преимуществ, но такой большой выигрыш стал полной неожиданностью! Звучало слишком хорошо, чтобы быть правдой, и меня попросили изучить ситуацию и объяснить, как это произошло.

Для анализа я использовал различные инструменты, в том числе основанные на счетчиках, статической конфигурации, РМС, программных событиях и трассировке. Все эти типы инструментов сыграли свою роль и дали подсказки, сложившиеся в общую непротиворечивую картину. Поскольку это исследование делалось для широкой аудитории, то стало вступительным материалом к моему докладу на USENIX LISA 2019 о производительности систем [Gregg 19h]. Я включил его в эту книгу для примера.

16.1.1. Постановка задачи

Поговорив с командой обслуживания, я выяснил подробности о микросервисе: это было приложение на Java, которое работает на экземплярах виртуальных машин

в облаке AWS EC2 и определяет рекомендации для клиентов. Микросервис состоит из двух компонентов, и один из них тестировался на новой контейнерной платформе Netflix под названием Titus, также работающей на AWS EC2. Задержка запроса этого компонента составляла 3–4 с на экземплярах виртуальных машин, а на контейнерах — 1 с: в 3–4 раза быстрее!

Мне нужно было объяснить эту разницу в производительности. Если такой прирост производительности был вызван простым перемещением микросервиса в контейнер, то можно было бы рассчитывать на выигрыш в 3–4 раза при перемещении других микросервисов. Если это связано с каким-то другим фактором, то стоило бы понять, что это за фактор и будет ли он проявляться постоянно. Возможно, его влияние может проявиться где-то еще и в большей степени.

Первое, что пришло мне в голову: проявились преимущества изолированного запуска одного компонента рабочей нагрузки, и теперь микросервис может использовать все кэши процессора, не конкурируя с другими компонентами. Из-за этого увеличивается коэффициент попаданий в кэш и, следовательно, производительность. Другое предположение — *bursting*, то есть увеличение доступной вычислительной мощности. Это когда контейнер может использовать простаивающие вычислительные ресурсы из других контейнеров.

16.1.2. Стратегия анализа

Трафик обрабатывается балансировщиком нагрузки (AWS ELB), поэтому можно разделить трафик между виртуальной машиной и контейнерами, чтобы проанализировать одновременно обе системы. Это идеальная ситуация для сравнительного анализа: можно одновременно запустить одну и ту же команду анализа в обеих системах (что гарантирует одинаковый трафик и нагрузку) и сразу сравнить результат.

В этом случае у меня был доступ не только к контейнеру, но и к хосту контейнера. Это позволило использовать любые инструменты анализа и дать этим инструментам разрешение на доступ к любому системному вызову. Если бы мне был доступен только контейнер, то анализ стал бы более трудоемким из-за ограниченности источников наблюдаемости и привилегий и потребовал бы проанализировать гораздо больший объем косвенных метрик из-за недоступности прямых показателей. Некоторые проблемы производительности нецелесообразно анализировать только из контейнера (см. главу 11 «Облачные вычисления»).

Что касается методологий, я планировал начать с 60-секундного чек-листа (глава 1 «Введение», раздел 1.10.1 «Анализ производительности Linux за 60 секунд») и метода USE (глава 2 «Методологии», раздел 2.5.9 «Метод USE») и уже по их результатам сделать детальный анализ (раздел 2.5.12 «Анализ с увеличением детализации») и использовать другие методологии.

Я включил команды, которые выполнял, и их вывод в следующие разделы. В них я использовал запрос «serverA#» для обозначения экземпляра виртуальной машины и «serverB#» — для хоста контейнера.

16.1.3. Статистики

Для начала я запустил `uptime(1)`, чтобы проверить среднюю величину нагрузки. В обеих системах

```
serverA# uptime
22:07:23 up 15 days, 5:01, 1 user, load average: 85.09, 89.25, 91.26

serverB# uptime
22:06:24 up 91 days, 23:52, 1 user, load average: 17.94, 16.92, 16.62
```

нагрузка была примерно стабильной: она немного уменьшилась для экземпляра виртуальной машины (с 91,26 до 85,09) и немного увеличилась для контейнера (с 16,62 до 17,94). Я проверил тенденции, чтобы увидеть, увеличивается ли проблема, уменьшается или остается постоянной: это особенно важно в облачных средах, которые могут автоматически переносить нагрузку с неработоспособного экземпляра. Я неоднократно заходил в проблемный экземпляр и обнаруживал, что активность невысока и средняя нагрузка за минуту приближается к нулю.

Средние значения также показали, что виртуальная машина нагружена гораздо больше, чем хост контейнера (85,09 против 17,94), но чтобы понять, что это означает, нужно воспользоваться другими инструментами. Средняя высокая нагрузка обычно указывает на нагрузку на процессор, но иногда может быть обусловлена интенсивным вводом/выводом (см. главу 6 «Процессоры», раздел 6.6.1 «uptime»).

Чтобы изучить нагрузку на процессор, я обратился к `mpstat(1)`, начав со средних значений для всей системы. На виртуальной машине:

```
serverA# mpstat 10
Linux 4.4.0-130-generic (...) 07/18/2019      _x86_64_ (48 CPU)

10:07:55 PM  CPU    %usr   %nice    %sys  %iowait  %irq   %soft  %steal  %guest  %gnice   %idle
10:08:05 PM  all    89.72    0.00   7.84    0.00    0.00   0.04   0.00   0.00   0.00   2.40
10:08:15 PM  all    88.60    0.00   9.18    0.00    0.00   0.05   0.00   0.00   0.00   2.17
10:08:25 PM  all    89.71    0.00   9.01    0.00    0.00   0.05   0.00   0.00   0.00   1.23
10:08:35 PM  all    89.55    0.00   8.11    0.00    0.00   0.06   0.00   0.00   0.00   2.28
10:08:45 PM  all    89.87    0.00   8.21    0.00    0.00   0.05   0.00   0.00   0.00   1.86
^C
Average:      all    89.49    0.00   8.47    0.00    0.00   0.05   0.00   0.00   0.00   1.99
```

И в контейнере:

```
serverB# mpstat 10
Linux 4.19.26 (...) 07/18/2019 _x86_64_ (64 CPU)

09:56:11 PM  CPU    %usr   %nice    %sys  %iowait  %irq   %soft  %steal  %guest  %gnice   %idle
09:56:21 PM  all    23.21    0.01   0.32    0.00    0.00   0.10   0.00   0.00   0.00   76.37
09:56:31 PM  all    20.21    0.00   0.38    0.00    0.00   0.08   0.00   0.00   0.00   79.33
09:56:41 PM  all    21.58    0.00   0.39    0.00    0.00   0.10   0.00   0.00   0.00   77.92
09:56:51 PM  all    21.57    0.01   0.39    0.02    0.00   0.09   0.00   0.00   0.00   77.93
09:57:01 PM  all    20.93    0.00   0.35    0.00    0.00   0.09   0.00   0.00   0.00   78.63
^C
Average:      all    21.50    0.00   0.36    0.00    0.00   0.09   0.00   0.00   0.00   78.04
```

`mpstat(1)` выводит количество процессоров в первой строке. Вывод показывает, что виртуальная машина имеет 48 процессоров, а хост контейнера — 64. Это помогло мне интерпретировать средние значения нагрузки: если бы они были обусловлены нехваткой вычислительных ресурсов, то это означало бы, что экземпляр VM работает в условиях насыщения процессоров, потому что средняя нагрузка примерно вдвое превышает количество процессоров, в то время как хост контейнера используется недостаточно интенсивно. Метрики, полученные с помощью `mpstat(1)`, подтвердили эту гипотезу: время бездействия на виртуальной машине составило около 2 %, тогда как на хосте контейнера — около 78 %.

Изучив другие статистики `mpstat(1)`, я обнаружил новые зацепки:

- Потребление процессора (`%usr + %sys + ...`) виртуальной машиной составило 98 % по сравнению с 22 % у контейнера. Процессоры имеют по два гиперпотока на каждое ядро, поэтому превышение уровня потребления 50 % обычно означает конкуренцию между гиперпотоками и снижение производительности. Виртуальная машина уже перешагнула этот рубеж, тогда как узел контейнера мог продолжать получать выгоду от нагрузки, с которой прекрасно справляется один гиперпоток в каждом ядре.
- Время выполнения в пространстве системы (`%sys`) на виртуальной машине намного выше: около 8 % против 0,38 %. Если виртуальная машина выполняется на нагруженном процессоре, это дополнительное время `%sys` может включать время, расходуемое на переключение контекста ядра. Подтвердить эту догадку можно трассировкой или профилированием ядра.

Я прошелся по другим пунктам 60-секундного чек-листа. `vmstat(8)` показала длину очереди выполнения, аналогичную средней нагрузке, подтверждая, что средняя нагрузка зависит от CPU. `iostat(1)` показала небольшое количество операций дискового ввода/вывода, а `sar(1)` — небольшое количество операций сетевого ввода/вывода. (Результаты этих измерений я здесь не привожу.) Это подтвердило, что виртуальная машина работает с насыщением процессора, в результате чего потоки выполнения вынуждены подолгу ждать своей очереди, в то время как хост контейнера — нет. `top(1)` на хосте контейнера также показала, что работает только один контейнер.

Эти команды дали мне статистику для метода USE, который также выявил проблему высокой нагрузки на процессор.

Решил ли я задачу? Я выяснил, что средняя нагрузка на виртуальную машину в системе с 48 CPU составляет около 85 и что эта средняя нагрузка обусловлена большой нагрузкой на процессор. Это означает, что примерно 77 % времени ($85/48 - 1$) потоки ожидают своей очереди на выполнение и избавление от этих простоев привело бы к ускорению примерно в 4 раза ($1/(1 - 0,77)$). Хотя эта величина соответствовала проблеме, я не мог объяснить, почему средняя нагрузка оказалась выше: требовался дополнительный анализ.

16.1.4. Конфигурация

Зная, что проблема в процессоре, я проверил конфигурацию и настроенные пределы (статическая настройка производительности: разделы 2.5.17 и 6.5.7). Для виртуальных машин и контейнеров использовались разные процессоры. Вот содержимое `/proc/cpuinfo` на хосте виртуальной машины:

```
serverA# cat /proc/cpuinfo
processor       : 47
vendor_id     : GenuineIntel
cpu family    : 6
model         : 85
model name    : Intel(R) Xeon(R) Platinum 8175M CPU @ 2.50GHz
stepping      : 4
microcode    : 0x200005e
cpu MHz       : 2499.998
cache size   : 33792 KB
physical id   : 0
siblings     : 48
core id      : 23
cpu cores    : 24
apicid       : 47
initial apicid : 47
fpu          : yes
fpu_exception : yes
cpuid level  : 13
wp           : yes
flags        : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat
pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc
arch_perfmon rep_good nopl xtopology nonstop_tsc aperfmperf eagerfpu pni pclmulqdq
monitor ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes
xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch invpcid_single kaiser
fsgsbase tsc_adjust bmi1 hle avx2 smep bmi2 erms invpcid rtm mpx avx512f rdseed adx
smap clflushopt clwb avx512cd xsaveopt xsavec xgetbv1 ida arat
bugs         : cpu_meltdown spectre_v1 spectre_v2 spec_store_bypass
bogomips     : 4999.99
clflush size : 64
cache_alignment : 64
address sizes : 46 bits physical, 48 bits virtual
power management:
```

и на хосте контейнера:

```
serverB# cat /proc/cpuinfo
processor       : 63
vendor_id     : GenuineIntel
cpu family    : 6
model         : 79
model name    : Intel(R) Xeon(R) CPU E5-2686 v4 @ 2.30GHz
stepping      : 1
microcode    : 0xb000033
```

```

cpu MHz          : 1200.601
cache size      : 46080 KB
physical id     : 1
siblings       : 32
core id        : 15
cpu cores      : 16
apicid         : 95
initial apicid : 95
fpu            : yes
fpu_exception  : yes
cpuid level    : 13
wp             : yes
flags          : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat
pse36 clflush mmx fxsr sse sse2 ht syscall nx pdpe1gb rdtscp lm constant_tsc arch_
perfmon rep_good nopl xtopology nonstop_tsc cpuid aperfmperf pni pclmulqdq monitor
est ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes
xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch cpuid_fault
invpcid_single pti fsgsbase bmi1 hle avx2 smep bmi2 erms invpcid rtm rdseed adx
xsaveopt ida
bugs           : cpu_meltdown spectre_v1 spectre_v2 spec_store_bypass l1tf
bogomips      : 4662.22
clflush size   : 64
cache_alignm   : 64
address sizes  : 46 bits physical, 48 bits virtual
power manage   :

```

Процессоры на хосте контейнера имели более низкую базовую тактовую частоту (2,30 против 2,50 ГГц), зато объем кэша последнего уровня у них был гораздо больше (45 против 33 Мбайт). В зависимости от рабочей нагрузки размер кэша может существенно сказываться на производительности. Дальше я решил исследовать счетчики РМС.

16.1.5. Счетчики РМС

Счетчики мониторинга производительности (РМС) могут объяснить, куда расходуются такты процессора, и доступны в некоторых экземплярах AWS EC2. Я опубликовал набор инструментов для анализа РМС в облаке [Gregg 20e], куда входит `rmcarch(8)` (раздел 6.6.11 «`rmcarch`»). `rmcarch(8)` показывает «архитектурный набор» счетчиков РМС от Intel, который является базовым общедоступным набором.

В виртуальной машине:

```

serverA# ./rmcarch -p 4093 10
K_CYCLES  K_INSTR   IPC BR_RETIRED  BR_MISPRED  BMR% LLCREF      LLCMISS      LLC%
982412660 575706336 0.59 126424862460 2416880487 1.91 15724006692 10872315070 30.86
999621309 555043627 0.56 120449284756 2317302514 1.92 15378257714 11121882510 27.68
991146940 558145849 0.56 126350181501 2530383860 2.00 15965082710 11464682655 28.19
996314688 562276830 0.56 122215605985 2348638980 1.92 15558286345 10835594199 30.35
979890037 560268707 0.57 125609807909 2386085660 1.90 15828820588 11038597030 30.26
[...]
```

В экземпляре с контейнером:

```
serverB# ./pmcarch -p 1928219 10
K_CYCLES K_INSTR IPC BR_RETIRED BR_MISPRED BMR% LLCREF LLCMISS LLC%
147523816 222396364 1.51 46053921119 641813770 1.39 8880477235 968809014 89.09
156634810 229801807 1.47 48236123575 653064504 1.35 9186609260 1183858023 87.11
152783226 237001219 1.55 49344315621 692819230 1.40 9314992450 879494418 90.56
140787179 213570329 1.52 44518363978 631588112 1.42 8675999448 712318917 91.79
136822760 219706637 1.61 45129020910 651436401 1.44 8689831639 617678747 92.89
[...]
```

Судя по результатам, количество инструкций на такт (IPC) составило около 0,57 для виртуальной машины и 1,52 для контейнера: разница в 2,6 раза.

Одна из причин более низкого значения IPC — возможная конкуренция между гиперпотоками, так как хост виртуальной машины работал с нагрузкой на процессор выше 50 %. В последнем столбце видна еще одна причина: коэффициент попаданий в кэш последнего уровня (LLC) составлял всего 30 % для виртуальной машины и примерно 90 % для контейнера. Из-за этого процессор на виртуальной машине может часто останавливаться, ожидая доступа к основной памяти, что снижает IPC и пропускную способность инструкций (производительность).

Более низкий коэффициент попаданий в LLC на виртуальной машине может быть вызван как минимум тремя факторами:

- меньший размер LLC (33 против 45 Мбайт);
- выполнение полной рабочей нагрузки вместо одного компонента (как указано в формулировке проблемы); компонент, выполняющийся в одиночку, вероятно, будет иметь лучший коэффициент попаданий в кэш, так как в нем меньше инструкций и данных;
- высокая нагрузка на процессор вызывает большее количество переключений контекста и переключений между путями в коде (включая пользователя и ядро), увеличивая нагрузку на кэш.

Последний фактор можно изучить с помощью инструментов трассировки.

16.1.6. Программные события

Исследование переключений контекста я начал с команды `perf(1)`, чтобы подсчитать частоту переключения контекста в системе в целом. Для этого я использовал программное событие, похожее на аппаратное (PMC), но реализованное программно (см. главу 4 «Инструменты наблюдения», рис. 4.5 и главу 13 «perf», раздел 13.5 «Программные события»).

В виртуальной машине:

```
serverA# perf stat -e cs -a -I 1000
#           time          counts unit events
    1.000411740          2,063,105    cs
    2.000977435          2,065,354    cs
```

3.001537756	1,527,297	cs
4.002028407	515,509	cs
5.002538455	2,447,126	cs
6.003114251	2,021,182	cs
7.003665091	2,329,157	cs
8.004093520	1,740,898	cs
9.004533912	1,235,641	cs
10.005106500	2,340,443	cs
^C 10.513632795	1,496,555	cs

Как показывает этот вывод, частота переключений контекста составила около двух миллионов в секунду. Затем я запустил ту же команду на хосте контейнера, добавив фильтрацию по PID приложения контейнера, чтобы исключить другие возможные контейнеры (такую же фильтрацию я пробовал включить на виртуальной машине, и это не сильно изменило предыдущие результаты¹):

```
serverB# perf stat -e cs -p 1928219 -I 1000
#           time              counts unit events
 1.001931945          1,172      cs
 2.002664012          1,370      cs
 3.003441563          1,034      cs
 4.004140394          1,207      cs
 5.004947675          1,053      cs
 6.005605844           955      cs
 7.006311221           619      cs
 8.007082057          1,050      cs
 9.007716475          1,215      cs
10.008415042          1,373      cs
^C 10.584617028           894      cs
```

Этот вывод показал скорость всего около тысячи переключений контекста в секунду.

Высокая частота переключений контекста может оказывать большое давление на кэши процессора, которые переключаются между различными путями в коде, включая код ядра, управляющий переключением контекста и, возможно, различными процессами². Чтобы дальше исследовать переключения контекста, я использовал инструменты трассировки.

16.1.7. Трассировка

Есть несколько инструментов трассировки на основе BPF, с помощью которых можно проанализировать потребление процессора и переключения контекста, в том числе из BCC: `cpudist(8)`, `cpuwalk(8)`, `runqlen(8)`, `runqlat(8)`, `runqslower(8)`, `cpuunclaimed(8)`) и т. д. (см. рис. 15.1).

¹ Тогда почему я не привел вывод с результатами для виртуальной машины, полученными при включенной фильтрации по PID? У меня его нет.

² Некоторые конфигурации процессора и ядра могут также предусматривать очистку кэша L1 при переключении контекста.

cpudist(8) показывает продолжительность выполнения потоков на процессоре. В виртуальной машине:

```
serverA# cpudist -p 4093 10 1
```

Tracing on-CPU time... Hit Ctrl-C to end.

usecs	: count	distribution
0 -> 1	: 3618650	*****
2 -> 3	: 2704935	*****
4 -> 7	: 421179	****
8 -> 15	: 99416	*
16 -> 31	: 16951	
32 -> 63	: 6355	
64 -> 127	: 3586	
128 -> 255	: 3400	
256 -> 511	: 4004	
512 -> 1023	: 4445	
1024 -> 2047	: 8173	
2048 -> 4095	: 9165	
4096 -> 8191	: 7194	
8192 -> 16383	: 11954	
16384 -> 32767	: 1426	
32768 -> 65535	: 967	
65536 -> 131071	: 338	
131072 -> 262143	: 93	
262144 -> 524287	: 28	
524288 -> 1048575	: 4	

Как показывает этот вывод, обычно приложение выполнялось на процессоре очень короткое время, часто менее 7 мкс. Другие инструменты (stackcount(8) для t:sched:sched_switch и /proc/PID/status) показали, что приложение обычно покидает CPU из-за принудительного¹ переключения контекста.

На хосте контейнера:

```
serverB# cpudist -p 1928219 10 1
```

Tracing on-CPU time... Hit Ctrl-C to end.

usecs	: count	distribution
0 -> 1	: 0	
2 -> 3	: 16	
4 -> 7	: 6	
8 -> 15	: 7	
16 -> 31	: 8	
32 -> 63	: 10	
64 -> 127	: 18	
128 -> 255	: 40	
256 -> 511	: 44	
512 -> 1023	: 156	*
1024 -> 2047	: 238	**
2048 -> 4095	: 4511	*****
4096 -> 8191	: 277	**

¹ В /proc/PID/status они называются nonvolption_ctxt_switches.

8192 -> 16383	:	286		**	
16384 -> 32767	:	77			
32768 -> 65535	:	63			
65536 -> 131071	:	44			
131072 -> 262143	:	9			
262144 -> 524287	:	14			
524288 -> 1048575	:	5			

В этом случае приложение выполнялось на процессоре в среднем от 2 до 4 мс. Другие инструменты показали, что приложение не слишком часто прерывается принудительными переключениями контекста.

Принудительные переключения контекста и высокая частота переключений на виртуальной машине вызвали проблемы с производительностью. Приложение вынуждено было оставлять процессор, проработав менее 10 мкс, что не давало времени кэшам процессора «прогреться» под особенности текущего исполняемого кода.

16.1.8. Заключение

По результатам анализа я пришел к выводу, что увеличение производительности обусловлено следующими причинами:

- **Отсутствие соседних контейнеров:** хост контейнера почти бездействовал, обслуживая единственный контейнер. Это позволило контейнеру использовать все кэши процессора и не конкурировать с другими контейнерами за процессор. Это обусловило высокие показатели производительности во время тестирования, но едва ли эта тенденция сохранится в промышленной среде в долгосрочной перспективе, когда появятся соседние контейнеры. Падение производительности микросервиса в 3–4 раза может обнаружиться, когда появятся другие арендаторы.
- **Разница в размере LLC и рабочей нагрузке:** величина IPC в виртуальной машине оказалась в 2,6 раза ниже, что может объяснить замедление в 2,6 раза. Одной из причин, вероятно, была конкуренция между гиперпотоками, потому что хост виртуальной машины работал с нагрузкой выше 50 % (и имел по два гиперпотока на ядро). Но главная причина, скорее всего, заключается в низком коэффициенте попаданий в LLC: 30 % на виртуальной машине по сравнению с 90 % на контейнере. Такой низкий коэффициент попаданий объясняется тремя факторами.
 - Меньший размер LLC на виртуальной машине: 33 Мбайт против 45 Мбайт.
 - Более сложная рабочая нагрузка на виртуальной машине: полное приложение с большим объемом кода и данных по сравнению с компонентом, выполняемым в контейнере.
 - Высокая частота переключения контекста на виртуальной машине: около 2 миллионов в секунду. Это не позволяет потокам занимать процессор на долгое время и не дает прогреть кэш. Длительность выполнения потоков на процессоре в виртуальной машине обычно составляла менее 10 мкс, тогда как на хосте контейнера потоки выполнялись на процессоре в среднем 2–4 мс.

- **Разница в нагрузке на процессор:** виртуальная машина испытывала более высокую нагрузку, что привело к насыщению процессора: средний уровень нагрузки оказался равным 85 в системе с 48 CPU. Это привело к увеличению частоты переключений контекста до 2 миллионов в секунду и длительному ожиданию потоков в очереди на выполнение. Задержка в очереди на выполнение, подразумеваемая средними показателями нагрузки, показала, что виртуальная машина работает примерно в 4 раза медленнее.

Эти проблемы объясняют наблюдаемую разницу в производительности.

16.2. ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ

Чтобы познакомиться с другими примерами анализа производительности, загляните в базу данных ошибок (или в систему отслеживания тикетов) вашей компании и поищите отчеты о решении предыдущих проблем с производительностью, а также публичные базы данных ошибок для приложений и ОС. Эти отчеты часто начинаются с описания проблемы и заканчиваются рецептом ее исправления. Многие базы данных ошибок также включают историю комментариев, которые можно изучить и увидеть ход анализа, включая исследованные гипотезы и принятые неверные решения. Поворот не туда и выявление нескольких факторов — это нормально.

Время от времени в интернете публикуются тематические исследования производительности систем, например, в моем блоге [Gregg 20j]. Технические журналы с упором на практику, например «USENIX; login:» [USENIX 20] и «ACM Queue» [ACM 20], также часто используют тематические исследования в качестве контекста при описании новых решений проблем.

16.3. ССЫЛКИ

[Gregg 19h] Gregg, B., «LISA2019 Linux Systems Performance», USENIX LISA, <http://www.brendangregg.com/blog/2020-03-08/lisa2019-linux-systems-performance.html>, 2019.

[ACM 20] «acmqueue», <http://queue.acm.org>, по состоянию на 2020.

[Gregg 20e] Gregg, B., «PMC (Performance Monitoring Counter) Tools for the Cloud», <https://github.com/brendangregg/pmc-cloud-tools>, последнее обновление 2020.

[Gregg 20j] «Brendan Gregg's Blog», <http://www.brendangregg.com/blog>, последнее обновление 2020.

[USENIX 20] «login: The USENIX Magazine», <https://www.usenix.org/publications/login>, по состоянию на 2020.

Приложение А

МЕТОД USE: LINUX

В этом приложении — чек-лист для Linux, созданный на основе метода USE [Gregg 13d]. Этот метод проверки работоспособности системы и выявления типичных узких мест и ошибок был описан в главе 2 «Методологии», в разделе 2.5.9 «Метод USE». В последующих главах (5, 6, 7, 9, 10) он описан в конкретном контексте вместе с инструментами для его поддержки.

Инструменты производительности совершенствуются, постоянно разрабатываются новые, поэтому рассматривайте этот чек-лист как отправную точку, которую нужно корректировать и обновлять. Также могут появиться новые фреймворки и инструменты наблюдения, упрощающие выполнение метода USE.

А1. ФИЗИЧЕСКИЕ РЕСУРСЫ

Компонент	Тип	Метрика
CPU	Потребление	С разбивкой по процессорам: mpstat -P ALL 1 столбцы, отражающие потребление процессоров (%usr, %nice, %sys, %irq, %soft, %guest, %gnice), и обратные им столбцы, характеризующие бездействие (%iowait, %steal, %idle); sar -P ALL столбцы, отражающие потребление процессоров (%user, %nice, %system), и обратные им столбцы, характеризующие бездействие (%iowait, %steal, %idle) Для системы в целом: vmstat 1, столбцы us + sy; sar -u, столбцы %user + %nice + %system С разбивкой по процессам: top, столбец %CPU; htop, столбец CPU%; ps -o pcpu; pidstat 1, столбец %CPU С разбивкой по потокам ядра: top/htop (K для переключения), в строках с VIRT == 0 (эвристика)

Компонент	Тип	Метрика
CPU	Насыщен- ние	Для системы в целом: vmstat 1, r > количества процессоров¹; sar -q, runq-sz > количества процессоров; runqlat; runqlen С разбивкой по процессам: /proc/PID/schedstat второе поле (sched_info.run_delay); getdelays.c, CPU²; perf sched latency (средняя и максимальная задержки планирования)³
CPU	Ошибки	Исключения проверки компьютера (Machine Check Exceptions, MCE) можно увидеть с помощью dmesg или rasdaemon и ras-mc-ctl -summary; события ошибок (PMC), характерные для процессоров, доступны в perf(1); например, для AMD64: «04Ah Single-bit ECC Errors Recorded by Scrubber»⁴ (которую также можно классифицировать как ошибку памяти); ipmtool sel list; ipmitool sdr list
Память	Потребле- ние	Для системы в целом: free -m, поле Mem: (основная память), поле Swap: (виртуальная память); vmstat 1, поле free (основная память), поле swap (виртуальная память); sar -r, столбец %memused; slabtop -s с для определения потребления памяти ядра С разбивкой по процессам: top/htop, столбец RES (резидентная основная память), столбец VIRT (виртуальная память), поле Mem — для системы в целом
Память	Насыщен- ние	Для системы в целом: vmstat 1, столбцы si/so (подкачка); sar -B, столбцы pgscan + pgscand (сканирование); sar -W

¹ В столбце r указано количество потоков, ожидающих в очереди на выполнение плюс выполняющихся на процессоре. См. описание vmstat(1) в главе 6 «Процессоры».

² Использует систему учета задержек, см. главу 4 «Инструменты наблюдения».

³ Также для perf(1) есть точка трассировки sched:sched_process_wait. Будьте осторожны с оверхедом при трассировке, так как события планировщика следуют друг за другом очень часто.

⁴ В последних руководствах по процессорам Intel и AMD перечислено не так много событий, связанных с ошибками.

Компонент	Тип	Метрика
		С разбивкой по процессам: getdelays.c, столбец SWAP2; 10-е поле (min_flt) в /proc/PID/stat для наблюдения за частотой отказов или для динамической инструментации ⁵ ; dmesg grep killed (события механизма OOM Killer)
Память	Ошибки	dmesg для поиска ошибок физической памяти или rasdaemon и ras-mc-ctl --summary или edac-util; dmidecode тоже можно использовать для выявления ошибок физической памяти; ipmtool sel list; ipmitool sdr list; динамическая инструментация, например, зондов uretprobe для выявления отказов в malloc() (bpftrace)
Сетевые интерфейсы	Потребление	ip -s link, прием/передача, пропускная способность и ширина полосы пропускания; sar -n DEV, скорость приема/передачи, ширина полосы пропускания; /proc/net/dev, пропускная способность приема/передачи
Сетевые интерфейсы	Насыщение	nstat, поле TcpRetransSegs; sar -n EDEV, столбцы *drop/s, *fifo/s6; /proc/net/dev, принято/получено/сброшено байтов; динамическая инструментация других механизмов очередей в стеке TCP/IP (bpftrace)
Сетевые интерфейсы	Ошибки	ip -s link, столбец errors; sar -n EDEV all; /proc/net/dev, поля errs, drop ⁶ ; дополнительные счетчики можно найти в /sys/class/net/*/statistics/*error*; динамическая инструментация возврата из функций драйвера
Устройства хранения, ввод/вывод	Потребление	Для системы в целом: iostat -xz 1, столбец %util; sar -d, столбец %util; С разбивкой по процессам: iotop, biotop; /proc/PID/sched поля se.statistics. iowait_sum

⁵ Используйте, чтобы выяснить, какие процессы потребляют память и вызывают насыщение, путем наблюдения за незначительными отказами. Должно быть доступно также в htop(1) в столбце MINFLT.

⁶ Информация о количестве сброшенных пакетов может служить признаком насыщения и ошибок, так как пакеты могут сбрасываться в обоих случаях.

Компонент	Тип	Метрика
Устройства хранения, ввод/вывод	Насыщение	<code>iostat -xnz 1</code> , где <code>avgqu-sz > 1</code> или высокое значение <code>await</code> ; <code>sar -d</code> то же самое; <code>perf(1)</code> точки трассировки из семейства <code>block</code> для определения длины очереди и задержки в ней; <code>biolatency</code>
Устройства хранения, ввод/вывод	Ошибки	<code>/sys/devices/.../ioerr_cnt</code> ; <code>smartctl</code> ; <code>bioerr</code> ; динамическая/статическая инструментация подсистемы ввода/вывода для получения кодов ответа ⁷
Устройства хранения, емкость	Потребление	Подкачка: <code>swapon -s</code> ; <code>free</code> ; <code>/proc/meminfo</code> поля <code>SwapFree/SwapTotal</code> ; Файловые системы: <code>df -h</code>
Устройства хранения, емкость	Насыщение	Не уверен, что в этом есть смысл, — когда устройство хранения заполнено, возвращается признак ошибки <code>ENOSPC</code> (хотя когда устройство близко к заполнению, производительность может снижаться в зависимости от алгоритма поиска свободных блоков в файловой системе)
Устройства хранения, емкость	Ошибки файловых систем	<code>strace</code> для выявления ошибок <code>ENOSPC</code> ; динамическая инструментация для выявления ошибок <code>ENOSPC</code> ; <code>/var/log/messages</code> поле <code>errs</code> , в зависимости от файловой системы; журнал ошибок приложения
Контроллер устройств хранения	Потребление	<code>iostat -sxz 1</code> , обобщенные статистики по устройствам и их сравнение с известными пределами пропускной способности контроллеров
Контроллер устройств хранения	Насыщение	См. «Устройства хранения, ввод/вывод» — «Насыщение»
Контроллер устройств хранения	Ошибки	См. «Устройства хранения, ввод/вывод» — «Ошибки»
Сетевой контроллер	Потребление	Может косвенно определяться по выводу <code>ip -s link</code> (или <code>sar</code> , или <code>/proc/net/dev</code>) с последующим сравнением с известными пределами пропускной способности контроллера

⁷ В том числе трассировка функций из разных уровней подсистемы ввода/вывода: блочное устройство, SCSI, SATA, IDE... Также доступны некоторые статические зонды (точки трассировки `scsi` и `block` для `perf(1)`). Для прочих случаев используйте динамическую трассировку.

Компонент	Тип	Метрика
Сетевой контроллер	Насыщение	См. «Сетевые интерфейсы» — «Насыщение»
Сетевой контроллер	Ошибки	См. «Сетевые интерфейсы» — «Ошибки»
Внутренняя шина между процессорами	Потребление	perf stat со счетчиками РМС для портов внутренней шины между процессорами, сравнение пропускной способности с шириной полосы пропускания
Внутренняя шина между процессорами	Насыщение	perf stat со счетчиками РМС циклов простоя
Внутренняя шина между процессорами	Ошибки	perf stat со всеми доступными счетчиками РМС
Внутренняя шина памяти	Потребление	perf stat со счетчиками РМС для памяти шины памяти и сравнение пропускной способности с шириной полосы пропускания; например, Intel uncore_imc/data_reads/, uncore_imc/data_writes/; или IPC меньше, чем, например, 0.2. Счетчики также могут быть локальными и удаленными
Внутренняя шина памяти	Насыщение	perf stat со счетчиками РМС циклов простоя
Внутренняя шина памяти	Ошибки	perf stat со всеми доступными счетчиками РМС; dmi decode тоже может дать полезную информацию
Внутренняя шина ввода/вывода	Потребление	perf stat со счетчиками РМС для определения пропускной способности и полосы пропускания, если таковые доступны; косвенное определение пропускной способности по выводу iostat/ip/...
Внутренняя шина ввода/вывода	Ошибки	perf stat со всеми доступными счетчиками РМС

Общие замечания: метрика «load average», которую выводит команда `uptime` (или значение в `/proc/loadavg`) не учитывается в метриках процессора, потому что среднее значение нагрузки в Linux включает задачи, действующие в режиме прерываемого ввода/вывода.

`perf(1)`: это мощный набор инструментов наблюдения, которые читают счетчики РМС, а также могут выполнять динамическую и статическую инструментацию. Интерфейсом к ним служит команда `perf(1)`. См. главу 13 «`perf`».

Счетчики РМС: счетчики мониторинга производительности. См. главу 6 «Процессоры» и примеры их использования с `perf(1)`.

Внутренние шины ввода/вывода: к ним относятся шины контроллера ввода/вывода, контроллер(-ы) ввода/вывода и шины устройств (например, PCIe).

Динамическая инструментация: позволяет конструировать собственные метрики. См. главу 4 «Инструменты наблюдения» и примеры в последующих главах. К числу инструментов динамической трассировки для Linux входят: perf(1) (глава 13), Ftrace (глава 14), BCC и bpftrace (глава 15).

В любой среде, где действуют средства управления ресурсами (например, в облачных средах), примените метод USE для каждого такого средства. Установленные ими ограничения могут быть существенно ниже фактических ограничений оборудования.

A2. ПРОГРАММНЫЕ РЕСУРСЫ

Компонент	Тип	Метрика
Мьютексы ядра	Потребление	При условии CONFIG_LOCK_STATS = y, /proc/lock_stat поля holdtime-total/acquisitions (см. также holdtime-min, holdtime-max) ¹ ; динамическая инструментация функций или инструкций блокировки (если возможно)
Мьютексы ядра	Насыщение	При условии CONFIG_LOCK_STATS = y, /proc/lock_stat поля waittimetotal/contentions (см. также waittime-min, waittime-max); динамическая инструментация функций блокировки, например, с помощью mlock.bt [Gregg 19]; спин-блокировки можно увидеть при профилировании командой perf record -a -g -F 99 ...
Мьютексы ядра	Ошибки	Динамическая инструментация (например, рекурсивные попытки приобретения мьютекса); другие ошибки, способные вызвать зависание или крах ядра, отслеживаются с помощью kdump/crash
Мьютексы в пространстве пользователя	Потребление	valgrind --tool=drd --exclusive-threshold=... (время удержания); динамическая инструментация функций приобретения/освобождения блокировок ²
Мьютексы в пространстве пользователя	Насыщение	valgrind --tool=drd для выявления признаков конкуренции по времени удержания; динамическая инструментация функций синхронизации для определения времени ожидания, например, с помощью pmlock.bt; профилирование по трассировкам стека в пространстве пользователя для выявления спин-блокировок (perf(1))

¹ Раньше анализ блокировок ядра проводился с помощью инструмента lockmeter, доступного через интерфейс lockstat.

² Эти функции могут вызываться очень часто, остерегайтесь оверхеда: трассировка каждого вызова может замедлить приложение в два и более раза.

Компонент	Тип	Метрика
Мьютексы в пространстве пользователя	Ошибки	valgrind --tool=drd различные ошибки; динамическая инструментация pthread_mutex_lock() для определения ошибок EAGAIN, EINVAL, EPERM, EDEADLK, ENOMEM, EOWNERDEAD, ...
Емкость пространства задач	Потребление	top/htop, поле Tasks (текущее значение); sysctl kernel.threads-max, /proc/sys/kernel/threads-max (поле max)
Емкость пространства задач	Насыщение	Блокировка потоков выполнения при выделении памяти; на этом этапе должен быть запущен сканер страниц (sar -B, pgscan*), в противном случае попробуйте выполнить динамическую трассировку
Емкость пространства задач	Ошибки	Ошибки "can't fork()"; для потоков выполнения в пространстве пользователя: pthread_create() завершается с кодом ошибки EAGAIN, EINVAL, ...; для потоков ядра: выполните динамическую трассировку kernel_thread() на предмет ошибки ENOMEM
Файловые дескрипторы	Потребление	Для системы в целом: sar -v, сравните file-nr с /proc/sys/fs/file-max; или просто проверьте /proc/sys/fs/file-nr С разбивкой по процессам: сравните echo /proc/PID/fd/* wc -w с ulimit -n
Файловые дескрипторы	Насыщение	Этот анализ может не иметь смысла
Файловые дескрипторы	Ошибки	strace возврат errno == EMFILE системными вызовами, возвращающими файловые дескрипторы (например, open(2), accept(2), ...); opensnoop -x

А3. ССЫЛКИ

[Gregg 13d] Gregg, B., «USE Method: Linux Performance Checklist», <http://www.brendangregg.com/USEmethod/use-linux.html>, впервые опубликована в 2013 г.

Приложение В

КРАТКИЙ СПРАВОЧНИК ПО SAR

В этом приложении — краткое описание параметров и метрик генератора отчетов по работе системы (system activity reporter) sar(1). Используйте это приложение как памятку о доступных метриках и параметрах. Полный список ищите на странице справочного руководства man.

О sar(1) подробно рассказывалось в главе 4 «Инструменты наблюдения», в разделе 4.4, а некоторые параметры описывались в последующих главах (6, 7, 8, 9, 10).

Параметр	Метрики	Описание
-u -P ALL	%user %nice %system %iowait %steal %idle	Потребление процессоров по отдельности (-u — необязательный параметр)
-u	%user %nice %system %iowait %steal %idle	Потребление процессоров
-u ALL	... %irq %soft %guest %gnice	Расширенная информация о потреблении процессоров
-m CPU -P ALL	МГц	Тактовая частота с разбивкой по процессорам
-q	runq-sz plist-sz ldavg-1 ldavg-5 ldavg-15 blocked	Размер очереди на выполнение
-w	proc/s cswch/s	События планировщика процессора
-B	pgpgin/s pgpgout/s fault/s majflt/s pgfree/s pgscank/s pgscand/s pgsteal/s %vmeff	Статистики подкачки страниц
-H	kbhugfree kbhugused %hugused	Огромные страницы
-r	kbmemfree kbavail kbmempused %memused kbbuffers kbcached kbcommit %commit kbactive kbinact kbdirty	Потребление памяти
-S	kbswpfree kbswpused %swpused kbswpcad %swpcad	Потребление виртуальной памяти
-W	pswpin/s pswpout/s	Статистики подкачки

Параметр	Метрики	Описание
-v	dentunusd file-nr inode-nr pty-nr	Таблицы ядра
-d	tps rkB/s wkB/s areq-sz aqu-sz await svctm %util	Статистики дисков
-n DEV	rxpck/s txpck/s rxkB/s txkB/s rxcmp/s txcmp/s rxmcst/s %ifutil	Статистики сетевого интерфейса
-n EDEV	rxerr/s txerr/s coll/s rxdrop/s txdrop/s txcarr/s rxfram/s rxfifo/s txfifo/s	Ошибки сетевого интерфейса
-n IP	irec/s fwddgm/s idel/s orq/s asmrq/s asmok/s fragok/s fragcrt/s	Статистики протокола IP
-n EIP	ihrerr/s iadrerr/s iukwnpr/s idisc/s odisc/s onort/s asmf/s fragf/s	Ошибки протокола IP
-n TCP	active/s passive/s iseg/s oseg/s	Статистики протокола ECP
-n ETCP	atmptf/s estres/s retrans/s isegerr/s orsts/s	Ошибки протокола ECP
-n SOCK	totsck tcpsck udpsck rawsck ip-frag tcp- tw	Статистики сокетов

Полужирным шрифтом я выделил ключевые метрики, на которые обращаю внимание.

Некоторые параметры `sar(1)` могут требовать того, чтобы ядро было скомпилировано с определенными параметрами (например, поддержка огромных страниц). Некоторые метрики были добавлены в более поздних версиях `sar(1)` (здесь перечислены метрики и параметры для версии `sar 12.0.6`).

Приложение С

ОДНОСТРОЧНЫЕ СЦЕНАРИИ ДЛЯ BPFTRACE

В этом приложении приведены удобные однострочные сценарии для bpftrace. Они полезны не только сами по себе: они помогут освоить программирование на языке bpftrace. Большинство из них уже приводилось в предыдущих главах. Многие могут не запускаться в некоторых системах, так как зависят от наличия определенных точек трассировки или функций либо от конкретной версии или конфигурации ядра.

Для знакомства с bpftrace см. главу 15, раздел 15.2.

ПРОЦЕССОРЫ

Трассировка создания новых процессов с регистрацией аргументов:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_execve { join(args->argv); }'
```

Подсчет количества обращений к системным вызовам с разбивкой по процессам:

```
bpftrace -e 'tracepoint:raw_syscalls:sys_enter { @[pid, comm] = count(); }'
```

Подсчет количества обращений к системным вызовам по их именам:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_* { @[probe] = count(); }'
```

Выборка имен работающих процессов с частотой 99 Гц:

```
bpftrace -e 'profile:hz:99 { @[comm] = count(); }'
```

Выборка всех трассировок стека в системе в пространствах пользователя и ядра с частотой 49 Гц с именами процессов:

```
bpftrace -e 'profile:hz:49 { @[kstack, ustack, comm] = count(); }'
```

Выборка трассировок стека в пространстве пользователя с частотой 49 Гц для PID 189:

```
bpftrace -e 'profile:hz:49 /pid == 189/ { @[ustack] = count(); }'
```

Выборка трассировок стека глубиной до пяти фреймов в пространстве пользователя с частотой 49 Гц для PID 189:

```
bpftrace -e 'profile:hz:49 /pid == 189/ { @[ustack(5)] = count(); }'
```

Выборка трассировок стека в пространстве пользователя с частотой 49 Гц для процесса с именем «mysqld»:

```
bpftrace -e 'profile:hz:49 /comm == "mysqld"/ { @[ustack] = count(); }'
```

Подсчет пересечений точек трассировки планировщика в ядре:

```
bpftrace -e 'tracepoint:sched:* { @[probe] = count(); }'
```

Подсчет трассировок стека ядра вне процессора по событиям переключения контекста:

```
bpftrace -e 'tracepoint:sched:sched_switch { @[kstack] = count(); }'
```

Подсчет количества вызовов функций ядра, имена которых начинаются с «vfs_»:

```
bpftrace -e 'kprobe:vfs_* { @[func] = count(); }'
```

Трассировка создания новых потоков выполнения вызовом функции pthread_create():

```
bpftrace -e 'u:/lib/x86_64-linux-gnu/libpthread-2.27.so:pthread_create {  
    printf("%s by %s (%d)\n", probe, comm, pid); }'
```

ПАМЯТЬ

Суммировать объем памяти в байтах, выделяемой вызовом функции malloc() из библиотеки libc с разбивкой по трассировкам стека в пространстве пользователя (имеет высокий оверхед):

```
bpftrace -e 'u:/lib/x86_64-linux-gnu/libc.so.6:malloc {  
    @[ustack, comm] = sum(arg0); }'
```

Суммировать объем памяти в байтах, выделяемой вызовом функции malloc() из библиотеки libc с разбивкой по трассировкам стека в пространстве пользователя для процесса с PID 181 (имеет высокий оверхед):

```
bpftrace -e 'u:/lib/x86_64-linux-gnu/libc.so.6:malloc /pid == 181/ {  
    @[ustack] = sum(arg0); }'
```

Показать гистограмму с шагом, равным степени 2, для распределения объемов выделяемой памяти через вызов функции malloc() из библиотеки libc с разбивкой по трассировкам стека в пространстве пользователя для процесса с PID 181 (имеет высокий оверхед):

```
bpftrace -e 'u:/lib/x86_64-linux-gnu/libc.so.6:malloc /pid == 181/ {  
    @[ustack] = hist(arg0); }'
```

Подсчет количества байтов, выделенных в кэше ядра kmem, по трассировкам стека в пространстве ядра:

```
bpftrace -e 't:kmem:kmem_cache_alloc { @bytes[kstack] = sum(args->bytes_alloc); }'
```

Подсчет событий увеличения кучи процесса (brk(2)) с разбивкой по трассировкам стека:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_brk { @[ustack, comm] = count(); }'
```

Подсчет сбоев страниц с разбивкой по процессам:

```
bpftrace -e 'software:page-fault:1 { @[comm, pid] = count(); }'
```

Подсчет сбоев страниц в пространстве пользователя с разбивкой по трассировкам стека в пространстве пользователя:

```
bpftrace -e 't:exceptions:page_fault_user { @[ustack, comm] = count(); }'
```

Подсчет операций в vmscan с использованием точек трассировки:

```
bpftrace -e 'tracepoint:vmscan:* { @[probe]++; }'
```

Подсчет событий подкачки страниц с разбивкой по процессам:

```
bpftrace -e 'kprobe:swap_readpage { @[comm, pid] = count(); }'
```

Подсчет миграций страниц:

```
bpftrace -e 'tracepoint:migrate:mm_migrate_pages { @ = count(); }'
```

Трассировка событий уплотнения памяти:

```
bpftrace -e 't:compaction:mm_compaction_begin { time(); }'
```

Список зондов USDT в libc:

```
bpftrace -l 'usdt:/lib/x86_64-linux-gnu/libc.so.6:*
```

Список точек трассировки kmem в ядре:

```
bpftrace -l 't:kmem:*
```

Список всех точек трассировки в подсистеме управления памятью (mm):

```
bpftrace -l 't:*:mm_*
```

ФАЙЛОВЫЕ СИСТЕМЫ

Трассировка операций открытия файлов через вызов openat(2) с именами процессов:

```
bpftrace -e 't:syscalls:sys_enter_openat { printf("%s %s\n", comm, str(args->filename)); }'
```

Подсчет обращений к системным вызовам из семейства read:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_*read* { @[probe] = count(); }'
```

Подсчет обращений к системным вызовам из семейства write:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_*write* { @[probe] = count(); }'
```

Показать гистограмму с распределением вызовов read() по запрошенным размерам блоков:

```
bpftrace -e 'tracepoint:syscalls:sys_enter_read { @ = hist(args->count); }'
```

Показать гистограмму с распределением вызовов read() по размерам прочитанных блоков (и кодам ошибок):

```
bpftrace -e 'tracepoint:syscalls:sys_exit_read { @ = hist(args->ret); }'
```

Подсчет количества вызовов read(), завершившихся ошибкой, с разбивкой по кодам ошибок:

```
bpftrace -e 't:syscalls:sys_exit_read /args->ret < 0/ { @[- args->ret] = count(); }'
```

Подсчет количества вызовов VFS:

```
bpftrace -e 'kprobe:vfs_* { @[probe] = count(); }'
```

Подсчет количества вызовов VFS для процесса с PID 181:

```
bpftrace -e 'kprobe:vfs_* /pid == 181/ { @[probe] = count(); }'
```

Подсчет количества пересечений точек трассировки в ext4:

```
bpftrace -e 'tracepoint:ext4:* { @[probe] = count(); }'
```

Подсчет количества пересечений точек трассировки в xfs:

```
bpftrace -e 'tracepoint:xfs:* { @[probe] = count(); }'
```

Подсчет количества операций чтения файлов в файловой системе ext4 с разбивкой по именам процессов и трассировкам стека в пространстве пользователя:

```
bpftrace -e 'kprobe:ext4_file_read_iter { @[ustack, comm] = count(); }'
```

Определяет время выполнения spa_sync() в ZFS:

```
bpftrace -e 'kprobe:spa_sync { time("%H:%M:%S ZFS spa_sync()\n"); }'
```

Подсчет количества обращений к кэшу каталогов с разбивкой по именам и идентификаторам (PID) процессов:

```
bpftrace -e 'kprobe:lookup_fast { @[comm, pid] = count(); }'
```

ДИСКИ

Подсчет событий блочного ввода/вывода по точкам трассировки:

```
bpfftrace -e 'tracepoint:block:* { @[probe] = count(); }'
```

Вывод размеров блочного ввода/вывода в виде гистограммы:

```
bpfftrace -e 't:block:block_rq_issue { @bytes = hist(args->bytes); }'
```

Подсчет операций ввода/вывода с разбивкой по трассировкам стека в пространстве пользователя:

```
bpfftrace -e 't:block:block_rq_issue { @[ustack] = count(); }'
```

Подсчет операций ввода/вывода с разбивкой по типам:

```
bpfftrace -e 't:block:block_rq_issue { @[args->rwbs] = count(); }'
```

Подсчет ошибок блочного ввода/вывода с разбивкой по типам операций и устройствам:

```
bpfftrace -e 't:block:block_rq_complete /args->error/ {  
    printf("dev %d type %s error %d\n", args->dev, args->rwbs, args->error); }'
```

Подсчет операций SCSI с разбивкой по их кодам:

```
bpfftrace -e 't:scsi:scsi_dispatch_cmd_start { @opcode[args->opcode] =  
    count(); }'
```

Подсчет операций SCSI с разбивкой по кодам завершения:

```
bpfftrace -e 't:scsi:scsi_dispatch_cmd_done { @result[args->result] = count(); }'
```

Подсчет количества вызовов функций в драйвере SCSI:

```
bpfftrace -e 'kprobe:scsi* { @[func] = count(); }'
```

СЕТИ

Подсчет обращений к системному вызову accept(2) с разбивкой по PID и именам процессов:

```
bpfftrace -e 't:syscalls:sys_enter_accept* { @[pid, comm] = count(); }'
```

Подсчет обращений к системному вызову connect(2) с разбивкой по PID и именам процессов:

```
bpfftrace -e 't:syscalls:sys_enter_connect { @[pid, comm] = count(); }'
```

Подсчет обращений к системному вызову connect(2) с разбивкой по трассировкам стека в пространстве пользователя:

```
bpftrace -e 't:syscalls:sys_enter_connect { @[ustack, comm] = count(); }'
```

Подсчет операций приема/передачи с разбивкой по направлениям, а также PID и именам процессов на процессоре:

```
bpftrace -e 'k:sock_sendmsg,k:sock_recvmsg { @[func, pid, comm] = count(); }'
```

Подсчет принятых/отправленных байтов с разбивкой по PID и именам процессов на процессоре:

```
bpftrace -e 'kr:sock_sendmsg,kr:sock_recvmsg /(int32)retval > 0/ { @[pid, comm] = sum((int32)retval); }'
```

Подсчет соединений TCP с разбивкой по PID и именам процессов на процессоре:

```
bpftrace -e 'k:tcp_v*_connect { @[pid, comm] = count(); }'
```

Подсчет принятых соединений TCP с разбивкой по PID и именам процессов на процессоре:

```
bpftrace -e 'k:inet_csk_accept { @[pid, comm] = count(); }'
```

Подсчет операций приема/передачи с разбивкой по PID и именам процессов на процессоре:

```
bpftrace -e 'k:tcp_sendmsg,k:tcp_recvmsg { @[func, pid, comm] = count(); }'
```

Гистограмма с распределением количества отправленных байтов по протоколу TCP:

```
bpftrace -e 'k:tcp_sendmsg { @send_bytes = hist(arg2); }'
```

Гистограмма с распределением количества полученных байтов по протоколу TCP:

```
bpftrace -e 'kr:tcp_recvmsg /retval >= 0/ { @recv_bytes = hist(retval); }'
```

Подсчет повторных передач TCP с разбивкой по типам и удаленным хостам (предполагается, что используется протокол IPv4):

```
bpftrace -e 't:tcp:tcp_retransmit_* { @[probe, ntop(2, args->saddr)] = count(); }'
```

Подсчет вызовов всех функций TCP (добавляет значительный оверхед при большом количестве операций TCP):

```
bpftrace -e 'k:tcp_* { @[func] = count(); }'
```

Подсчет операций приема/передачи по протоколу UDP с разбивкой по PID и именам процессов на процессоре:

```
bpftrace -e 'k:udp*_sendmsg,k:udp*_recvmsg { @[func, pid, comm] = count(); }'
```


Гистограмма с распределением количества отправленных байтов по протоколу UDP:

```
bpfttrace -e 'k:udp_sendmsg { @send_bytes = hist(arg2); }'
```

Гистограмма с распределением количества принятых байтов по протоколу UDP:

```
bpfttrace -e 'k:udp_recvmsg /retval >= 0/ { @recv_bytes = hist(retval); }'
```

Подсчет операций передачи с разбивкой по трассировкам стека в пространстве ядра:

```
bpfttrace -e 't:net:net_dev_xmit { @[kstack] = count(); }'
```

Показать гистограмму потребления процессорного времени на прием для каждого устройства:

```
bpfttrace -e 't:net:netif_receive_skb { @[str(args->name)] = lhist(cpu, 0, 128, 1); }'
```

Подсчет вызовов функций ieee80211 (добавляет значительный оверхед в операции с пакетами):

```
bpfttrace -e 'k:ieee80211_* { @[func] = count(); }'
```

Подсчет вызовов всех функций драйвера устройства ixgbev (добавляет значительный оверхед в работу ixgbev):

```
bpfttrace -e 'k:ixgbev_* { @[func] = count(); }'
```

Подсчет пересечения всех точек трассировки драйвера устройства iwl (добавляет значительный оверхед в работу iwl):

```
bpfttrace -e 't:iwlfwifi:*,t:iwlfwifi_io:* { @[probe] = count(); }'
```

Приложение D

РЕШЕНИЯ НЕКОТОРЫХ УПРАЖНЕНИЙ

Ниже приводятся ответы на вопросы и варианты решения некоторых упражнений.

ГЛАВА 2. МЕТОДОЛОГИИ

В. Что такое задержка?

О. Мера времени, обычно время ожидания завершения чего-либо. В IT-индустрии термин используется по-разному в зависимости от контекста.

ГЛАВА 3. ОПЕРАЦИОННЫЕ СИСТЕМЫ

В. Перечислите причины, почему поток выполнения может оставить процессор.

О. В результате блокировки на операции ввода/вывода, приостановки на блокировке вызова `yield`, по истечении выделенного кванта времени, в результате вытеснения другим потоком, при получении прерывания от устройства, по завершении работы.

ГЛАВА 6. ПРОЦЕССОРЫ

В. Вычислите среднюю нагрузку...

О. 34.

ГЛАВА 7. ПАМЯТЬ

В. В чем разница между подкачкой страниц (`paging`) и анонимной подкачкой (`swapping`) с точки зрения терминологии Linux?

- О. Подкачка страниц (paging) — это передача страниц памяти между основной памятью и запоминающими устройствами. Подкачка — это анонимная передача страниц на устройство/файл подкачки и обратно.
- В. Опишите характеристики потребления и насыщения памяти.
- О. Потребление — это отношение объема памяти, используемой и недоступной для других нужд, к общему количеству доступной памяти. Потребление можно выразить в процентах, как емкость файловой системы. Насыщение — это мера потребности в доступной памяти, превышающей размер имеющейся памяти. Обычно когда наступает насыщение, запускается процедура ядра для освобождения памяти и удовлетворения этой потребности.

ГЛАВА 8. ФАЙЛОВЫЕ СИСТЕМЫ

- В. В чем разница между логическим и физическим вводом/выводом?
- О. Логический ввод/вывод — это ввод/вывод между приложением и файловой системой. Физический ввод/вывод — это ввод/вывод между файловой системой и устройствами хранения (дисками).
- В. Объясните, как использование алгоритма копирования при записи может повысить производительность файловой системы.
- О. Поскольку произвольная запись может быть записана в новое место, такие операции группируются (за счет увеличения объема ввода/вывода), и запись производится последовательно. Оба этих фактора обычно улучшают производительность в зависимости от типа устройства хранения.

ГЛАВА 9. ДИСКИ

- В. Опишите, что происходит, когда диски перегружены работой, включая влияние на производительность приложений.
- О. Такие диски работают с постоянно высоким уровнем потребления (до 100 %) и насыщения (запросы ставятся в очередь). Задержка ввода/вывода таких дисков увеличивается из-за вероятности постановки запроса в очередь (что можно смоделировать). Если приложение выполняет ввод/вывод, увеличенная задержка *может* снизить его производительность при условии, что ввод/вывод выполняется синхронно: чтение или синхронная запись. Это также всегда происходит при выполнении критических путей в коде приложения, например во время обслуживания запроса, но не во время выполнения асинхронной фоновой задачи (которая может лишь *косвенно* вызвать снижение производительности приложения). Обычно правильная реакция в ответ на увеличение задержки ввода/вывода позволяет держать частоту запросов ввода/вывода под контролем и не вызвать неограниченного увеличения задержки.

ГЛАВА 11. ОБЛАЧНЫЕ ВЫЧИСЛЕНИЯ

- В. Опишите наблюдаемость физической системы с точки зрения гостя в виртуализации операционной системы.
- О. В зависимости от реализации ядра хоста гость может видеть высокоуровневые метрики всех физических ресурсов, включая процессоры и диски, и замечать, когда они используются другими арендаторами. Метрики, которые могут привести к утечке пользовательских данных, должны блокироваться ядром. Например, ядро позволяет наблюдать уровень потребления процессора (скажем, 50 %), но не идентификаторы и имена процессов других арендаторов, влияющих на потребление.

Приложение Е

ПРОИЗВОДИТЕЛЬНОСТЬ СИСТЕМ, КТО ЕСТЬ КТО

Бывает полезно знать по именам создателей технологий, которыми мы пользуемся. Ниже — список «кто есть кто» в области производительности систем, основанный на технологиях, приведенных в этой книге. Я решил добавить его, вдохновившись списком «Кто есть кто в Unix», опубликованным в [Libes 89]. Заранее прошу прощения у всех, кого упустил из виду или включил в список по ошибке. Если вы хотите больше узнать о людях и истории, обратите внимание на разделы со ссылками в главах, имена, перечисленные в исходном коде Linux, а также на историю репозитория Linux и файл MAINTAINERS в исходном коде Linux. В разделе «Благодарности» моей книги о BPF [Gregg 19] также перечислены различные технологии, в частности расширенный BPF, BCC, bpftrace, kprobes и uprobes, и люди, стоящие за ними.

Джон Олспоу (John Allspaw): планирование мощности [Allspaw 08].

Джин М. Амдал (Gene M. Amdahl): ранние работы над масштабируемостью компьютерных систем [Amdahl 67].

Йенс Аксбо (Jens Axboe): планировщик ввода/вывода CFQ, fio, blktrace, io_uring.

Бренден Бланко (Brenden Blanco): BCC.

Джефф Бонвик (Jeff Bonwick): автор механизма распределения памяти для объектов (slab allocation), соавтор механизма распределения памяти для объектов в пространстве пользователя, соавтор файловой системы ZFS, kstat, первый разработчик mpstat.

Даниэль Боркманн (Daniel Borkmann): соавтор и мейнтейнер расширенного BPF.

Рох Бурбонне (Roch Bourbonnais): эксперт в области анализа производительности систем Sun Microsystems.

Тим Брей (Tim Bray): автор микробенчмарка Bonnie дискового ввода/вывода, известен как один из авторов спецификации XML.

Брайан Кэнтрилл (Bryan Cantrill): соавтор DTrace; Oracle ZFS Storage Appliance Analytics.

Реми Кард (Rémy Card): основной разработчик файловых систем ext2 и ext3.

Надя Иветт Чамберс (Nadia Yvette Chambers): файловая система hugetlbfs для Linux.

Гийом Шазарен (Guillaume Chazarain): iotop(1) для Linux.

Адриан Кокрофт (Adrian Cockcroft): книги об анализе производительности [Cockcroft 95], [Cockcroft 98], Virtual Adrian (SE Toolkit).

Тим Кук (Tim Cook): nicstat(1) для Linux и расширения.

Алан Кокс (Alan Cox): анализ производительности сетевого стека Linux.

Матье Дезнойерс (Mathieu Desnoyers): Linux Trace Toolkit (LTTng), точки трассировки ядра, ведущий автор RCU для пространства пользователя.

Фрэнк Ч. Эйглер (Frank Ch. Eigler): ведущий разработчик SystemTap.

Ричард Эллинг (Richard Elling): методология статической настройки производительности.

Джулия Эванс (Julia Evans): документация и инструменты анализа производительности и отладки.

Кевин Роберт Эльз (Kevin Robert Elz): DNLC.

Роджер Фолкнер (Roger Faulkner): разработчик файловой системы /proc для UNIX System V, реализация потоков выполнения для Solaris и трассировщик системных вызовов truss(1).

Томас Глейкснер (Thomas Gleixner): различные разработки для повышения производительности ядра Linux, включая таймеры высокого разрешения.

Себастьян Годар (Sebastian Godard): пакет sysstat для Linux, содержащий множество инструментов анализа производительности, включая iostat(1), mpstat(1), pidstat(1), nfsiostat(1), cifsioat(1), расширенную версию sar(1), sadc(8), sadf(1) (см. метрики в приложении В).

Саша Гольдштейн (Sasha Goldshtein): инструменты BPF (argdist(8), trace(8) и др.), один из разработчиков BCC.

Брендан Грегг (Brendan Gregg): nicstat(1), DTraceToolkit, ZFS L2ARC, инструменты BPF (execsnoop, biosnoop, ext4slower, tcptop и др.), один из разработчиков BCC/bpfttrace, метод USE, тепловые карты (задержек, потребления, субсекундное смещение), флейм-графики, эта книга и предыдущая [Gregg 11a], [Gregg 19], другие работы, связанные с анализом производительности.

Доктор Нил Гюнтер (Dr. Neil Gunther): универсальный закон масштабируемости, тройные графики потребления процессора, книги по производительности [Gunther 97].

Джефффри Холлингсворт (Jeffrey Hollingsworth): динамическая инструментация [Hollingsworth 94].

Ван Якобсон (Van Jacobson): traceroute(8), pathchar, анализ производительности TCP/IP.

Радж Джайн (Raj Jain): теория производительности систем [Jain 91].

Джерри Елинек (Jerry Jelinek): зоны в Solaris.

Билл Джой (Bill Joy): vmstat(1), участие в разработке виртуальной памяти BSD, анализ производительности TCP/IP, FFS.

Энди Клин (Andi Kleen): анализ производительности Intel, участие в разработке ядра Linux.

Кристоф Ламетер (Christoph Lameter): распределитель SLUB.

Уильям Лефебвр (William LeFebvre): автор первой версии top(1), вдохновившей на создание многих других инструментов.

Дэвид Левинталь (David Levinthal): эксперт по производительности процессоров Intel.

Джон Левон (John Levon): OProfile.

Майк Лукидес (Mike Loukides): первая книга об анализе производительности систем Unix [Loukides 90], заложившая и поддерживавшая традицию анализа на основе ресурсов: процессор, память, диск, сеть.

Роберт Лав (Robert Love): разработки, связанные с увеличением производительности ядра Linux, включая вытеснение.

Мэри Марчини (Mary Marchini): libstapsdt: поддержка зондов USDT для разных языков.

Джим Мауро (Jim Mauro): соавтор книг «Solaris Performance and Tools» [McDougall 06a] и «DTrace: Dynamic Tracing in Oracle Solaris, Mac OS X, and FreeBSD» [Gregg 11].

Ричард Макдугалл (Richard McDougall): система учета микросостояний в Solaris, соавтор «Solaris Performance and Tools» [McDougall 06a].

Маршалл Кирк Маккьюсик (Marshall Kirk McKusick): FFS, один из разработчиков BSD.

Арнальдо Карвальо де Мело (Arnaldo Carvalho de Melo): мейнтейнер perf(1) в Linux.

Бартон Миллер (Barton Miller): динамическая инструментация [Hollingsworth 94].

Дэвид С. Миллер (David S. Miller): мейнтейнер сетевого стека и архитектуры SPARC в Linux. Многочисленные улучшения производительности и поддержка расширенного BPF.

Кэри Миллсап (Cary Millsap): метод R.

Инго Молнар (Ingo Molnar): планировщик O(1), абсолютно справедливый планировщик, добровольное вытеснение ядра, ftrace, perf и работа над механизмом

вытеснения в реальном времени, мьютексы, фьютексы, профилирование планировщика, очереди заданий.

Ричард Дж. Мур (Richard J. Moore): DProbes, kprobes.

Эндрю Мортон (Andrew Morton): fadvise, read-ahead.

Джан-Паоло Д. Мусумечи (Gian-Paolo D. Musumeci): «System Performance Tuning, 2nd Ed.» [Musumeci 02].

Майк Муусс (Mike Muuss): ping(8).

Шайлаб Нарар (Shailabh Nagar): учет задержек, taskstats.

Рич Петтит (Rich Pettit): SE Toolkit.

Ник Пиггин (Nick Piggin): планировщики Linux.

Билл Пиевски (Bill Pijewski): Solaris vfsstat(1M), регулирование ввода/вывода в ZFS.

Деннис Ричи (Dennis Ritchie): Unix и ее механизмы обеспечения производительности: приоритеты процессов, подкачка, кэш буферов и т. д.

Аластер Робертсон (Alastair Robertson): создатель bpftrace.

Стивен Ростедт (Steven Rostedt): Ftrace, KernelShark, поддержка реального времени в Linux, адаптивные мьютексы, поддержка трассировки в Linux.

Расти Рассел (Rusty Russell): оригинальные фьютексы, участие в разработке ядра Linux.

Майкл Шапиро (Michael Shapiro): соавтор DTrace.

Алексей Шипилёв (Aleksey Shipilev): эксперт по производительности Java.

Бальбир Сингх (Balbir Singh): контроллер памяти в Linux, учет задержек, taskstats, cgroupstats, учет производительности процессоров.

Юнхонг Сон (Yonghong Song): BTF и участие в разработке расширенного BPF и BCC.

Алексей Старовойтов (Alexei Starovoitov): соавтор и мейнтейнер расширенного BPF.

Кен Томпсон (Ken Thompson): Unix и ее механизмы обеспечения производительности: приоритеты процессов, подкачка, кэш буферов и т. д.

Мартин Томпсон (Martin Thompson): Mechanical Sympathy.

Линус Торвалдс (Linus Torvalds): ядро Linux и многочисленные базовые компоненты, необходимые для производительности системы, планировщик ввода/вывода для Linux, Git.

Арьян ван де Вен (Arjan van de Ven): latencytop, PowerTOP, irqbalance, профилирование планировщика Linux.

Ницан Вакарт (Nitsan Wakart): эксперт по производительности Java.

Тобиас Вальдекранц (Tobias Waldekranz): ply (первый высокоуровневый трассировщик в BPF).

Дэг Вирс (Dag Wieers): dstat.

Карим Ягмур (Karim Yaghmour): LTT, развитие трассировки в Linux.

Джови Чжанвэй (Jovi Zhangwei): ktap.

Том Занусси (Tom Zanussi): триггеры hist в Ftrace.

Питер Зийлстра (Peter Zijlstra): реализация адаптивных мьютексов, фреймворк обратных вызовов для аппаратных прерываний, другие работы в сфере анализа производительности Linux.

Е.1. ССЫЛКИ

[Amdahl 67] Amdahl, G., «Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities», AFIPS, 1967.

[Libes 89] Libes, D., and Ressler, S., «Life with UNIX: A Guide for Everyone», Prentice Hall, 1989.

[Loukides 90] Loukides, M., «System Performance Tuning», O'Reilly, 1990.

[Hollingsworth 94] Hollingsworth, J., Miller, B., and Cargille, J., «Dynamic Program Instrumentation for Scalable Performance Tools», Scalable High-Performance Computing Conference (SHPCC), May 1994.

[Cockcroft 95] Cockcroft, A., «Sun Performance and Tuning», Prentice Hall, 1995.

[Cockcroft 98] Cockcroft, A., and Pettit, R., «Sun Performance and Tuning: Java and the Internet», Prentice Hall, 1998.

[Musumeci 02] Musumeci, G. D., and Loukidas, M., «System Performance Tuning, 2nd Edition», O'Reilly, 2002¹.

[McDougall 06a] McDougall, R., Mauro, J., and Gregg, B., «Solaris Performance and Tools: DTrace and MDB Techniques for Solaris 10 and OpenSolaris», Prentice Hall, 2006.

[Gunther 07] Gunther, N., «Guerrilla Capacity Planning», Springer, 2007.

[Allspaw 08] Allspaw, J., «The Art of Capacity Planning», O'Reilly, 2008².

[Gregg 11a] Gregg, B., and Mauro, J., «DTrace: Dynamic Tracing in Oracle Solaris, Mac OS X and FreeBSD», Prentice Hall, 2011.

[Gregg 19] Gregg, B., «BPF Performance Tools: Linux System and Application Observability»³, Addison-Wesley, 2019.

¹ Джан-Паоло Д. Мусумеси, Майк Лукидес. «Настройка производительности UNIX-систем, 2-е издание».

² Джон Оллспоу. «Искусство планирования мощностей». СПб., издательство «Питер».

³ Грегг Б. «BPF: Профессиональная оценка производительности». Выходит в издательстве «Питер» в 2023 году.

ГЛОССАРИЙ

ABI	Application Binary Interface — прикладной двоичный интерфейс.
ACK	Пакет подтверждения в TCP.
AMD	Производитель процессоров.
API	Application Programming Interface — прикладной программный интерфейс.
ARM	Производитель процессоров.
ARP	Address Resolution Protocol — протокол разрешения адресов.
ASIC	Application-Specific Integrated Circuit — специализированная интегральная схема.
AT&T	Компания American Telephone and Telegraph Company, в состав которой входила Bell Laboratories, где была разработана ОС Unix.
BCC	BPF Compiler Collection — коллекция компиляторов BPF. BCC — это проект, включающий компилятор для фреймворка BPF, а также множество инструментов анализа производительности BPF. См. главу 15.
BIOS	Basic Input/Output System — базовая система ввода/вывода. Прошивка, выполняющая инициализацию компьютерного оборудования и управляющая процессом загрузки.
BPF	Berkeley Packet Filter — фильтр пакетов Беркли. Легковесная технология 1992 года, встроенная в ядро и созданная для увеличения производительности фильтрации пакетов. С 2014 года была расширена и превратилась в среду выполнения общего назначения (см. <i>eBPF</i>).
BSD	Berkeley Software Distribution, производная Unix.
C	Язык программирования C.
CDN	Content Delivery Network — сеть доставки содержимого.
CPI	Cycles Per Instruction — тактов на инструкцию. См. главу 6 «Процессы».
CSV	Comma Separated Values — значения, разделенные запятыми. Формат файлов.

CTSS	Compatible Time-Sharing System — совместимая система с разделением времени. Одна из первых систем с разделением времени.
DEC	Digital Equipment Corporation.
DNS	Domain Name Service — служба доменных имен.
DRAM	Dynamic Random-Access Memory — динамическая память с произвольным доступом. Тип энергозависимой памяти, обычно используемой в качестве основной.
eBPF	Extended BPF — расширенный BPF (см. <i>BPF</i>). Аббревиатура eBPF первоначально описывала расширенную версию BPF 2014 года с обновленными размером регистра и набором инструкций, с дополнительной поддержкой хранилища карт и ограниченной возможностью вызова функций ядра. В 2015 году eBPF решили называть просто BPF.
ECC	Error-Correcting Code — код исправления ошибки. Алгоритм обнаружения ошибок и исправления некоторых из них (обычно однобитовых).
ELF	Executable and Linkable Format — формат исполнимых и компоуемых модулей. Распространенный формат файлов для исполняемых программ.
errno	Переменная, содержащая последнюю ошибку в виде числа согласно стандарту (POSIX.1-2001).
Ethernet	Набор стандартов для сетей физического уровня и уровня передачи данных.
FPGA	Field-Programmable Gate Array — программируемая логическая матрица. Перепрограммируемая интегральная схема, используемая в вычислениях для ускорения определенной операции.
FreeBSD	Unix-подобная операционная система с открытым исходным кодом.
fsck	Команда проверки файловой системы. Используется для восстановления файловых систем после сбоя, например, из-за отключения питания или краха ядра. Этот процесс может занять несколько часов.
Gbps	Gigabits per second — гигабит в секунду.
GUI	Graphical User Interface — графический интерфейс пользователя.
HDD	Hard Disk Drive — привод жесткого диска. Устройство хранения с вращающимися магнитными дисками. См. главу 9 «Диски».
HTTP	Hyper Text Transfer Protocol — протокол передачи гипертекста.
ICMP	Internet Control Message Protocol — протокол управляющих сообщений интернета. Используется командой ping(1) (ICMP echo request/reply).

I/O	Input/Output — ввод/вывод.
IO Visor	Проект Linux Foundation. Его репозитории bcc и bpftrace размещены в GitHub, что упрощает сотрудничество между разработчиками BPF в разных компаниях.
IOPS	I/O Operations Per Second — операций ввода/вывода в секунду. Мера скорости ввода/вывода.
Intel	Производитель процессоров.
IP	Internet Protocol — протокол интернета. Основные версии — IPv4 и IPv6. См. главу 10 «Сети».
IPC	Может означать: либо количество инструкций на такт (Instructions Per Cycle) — низкоуровневая метрика производительности процессора, либо межпроцессные взаимодействия (Inter-Process Communication) — средства обмена данными между процессами. Один из примеров механизмов межпроцессных взаимодействий — сокет.
IRIX	Производная Unix. Операционная система, разработанная в Silicon Graphics, Inc. (SGI).
IRQ	Interrupt Request — запрос на прерывание. Аппаратный сигнал процессору, запрашивающий выполнение некоторого задания. См. главу 3 «Операционные системы».
kprobes	Технология Linux для динамической инструментации ядра.
LRU	Least Recently Used — наиболее давно использовавшийся. См. главу 2 «Методологии», раздел 2.3.14 «Кэширование».
malloc	Выделение памяти — memory allocate. Под этим термином обычно подразумевают функцию, реализующую выделение памяти.
Mbps	Megabits per second — мегабит в секунду.
MMU	Memory Management Unit — блок управления памятью. Отвечает за представление памяти процессору и преобразование виртуальных адресов в физические.
mysqld	Демон базы данных MySQL.
NVMe	Non-Volatile Memory express — энергонезависимая память. Спецификация шины PCIe для устройств хранения.
PC	Program Counter — счетчик инструкций. Регистр процессора с адресом выполняемой в данный момент инструкции.

PCID	Process-Context ID — идентификатор контекста процесса. Особенность процессора/MMU для маркировки виртуальных адресов идентификатором процесса. Это помогает избежать сброса кэша при переключении контекста.
PCIe	Peripheral Component Interconnect Express: стандарт шины, широко используемый для контроллеров устройств хранения и сетевых контроллеров.
PDP	Programmed Data Processor, серия мини-компьютеров, производившихся компанией Digital Equipment Corporation (DEC).
PEBS	Precise Event-Based Sampling, иногда Processor Event-Based Sampling — точная выборка инструкций на основе событий. Технология, реализованная в процессорах Intel для использования со счетчиками РМС. Она обеспечивает более точное определение состояния процессора во время событий.
PID	Process Identifier — идентификатор процесса. Уникальный числовой идентификатор процесса в операционной системе.
PMC	Performance Monitoring Counters — счетчики мониторинга производительности. Специальные аппаратные регистры процессора, которые можно запрограммировать для подсчета низкоуровневых событий: тактов, тактов простоя, инструкций, чтения/записи в память и т. д.
POSIX	Portable Operating System Interface — переносимый интерфейс операционных систем. Семейство стандартов, которыми управляет IEEE для определения Unix API. Сюда входит интерфейс файловой системы, используемый приложениями и предоставляемый через системные вызовы или системные библиотеки, использующие системные вызовы.
PSI	Pressure Stall Information. Набор метрик, используемых для выявления проблем с производительностью, обусловленных ресурсами.
RCU	Read-Copy-Update: механизм синхронизации в Linux.
RFC	Request For Comments — запрос на комментарии. Общедоступный документ, который публикует Рабочая группа по проектированию интернета (Internet Engineering Task Force, IETF) для обмена сетевыми стандартами и передовым опытом. Документы RFC используются для определения сетевых протоколов: RFC 793, например, определяет протокол TCP.
RSS	Resident Set Size — размер резидентного набора. Мера основной памяти.
ROI	Return On Investment — показатель возврата инвестиций, бизнес-метрика.
RX	Прием (используется в сетевых технологиях).

SCSI	Small Computer System Interface — интерфейс малых вычислительных систем. Стандарт интерфейса для устройств хранения.
SLA	Service Level Agreement — соглашение об уровне обслуживания.
SMP	Symmetric Multiprocessing — симметричная многопроцессорность. Многопроцессорная архитектура, в которой несколько одинаковых процессоров совместно используют одну и ту же основную память.
SMT	Simultaneous Multithreading — одновременная многопоточность. Особенность процессора, позволяющая запускать несколько потоков на ядрах. См. <i>гиперпоточность</i> .
SNMP	Simple Network Management Protocol — простой протокол управления сетью.
Solaris	Операционная система Unix, первоначально разработанная в Sun Microsystems. Славилась своей масштабируемостью и надежностью и была популярна в корпоративных средах. После покупки Sun корпорацией Oracle была переименована в Oracle Solaris.
SONET	Synchronous Optical Networking, физический протокол для оптоволоконных линий.
SPARC	Архитектура процессоров (от Scalable Processor Architecture) — масштабируемая архитектура процессоров.
SRE	Site Reliability Engineer: технический сотрудник, специализирующийся на инфраструктуре и надежности. SRE-инженеры анализируют производительность в ходе расследования инцидентов в короткие сроки.
SSD	Solid-State Drive — твердотельный накопитель. Устройство хранения, обычно на основе флеш-памяти. См. главу 9 «Диски».
SSH	Secure Shell. Безопасный протокол обмена с удаленной командной оболочкой.
SunOS	Операционная система Sun Microsystems Operating System. Позднее была переименована в Solaris.
SUT	System Under Test — тестируемая система.
SVG	Scalable Vector Graphs — масштабируемая векторная графика. Формат файлов.
SYN	TCP-пакет синхронизации.
TCP	Transmission Control Protocol — протокол управления передачей. Первоначально определен в RFC 793. См. главу 10 «Сети».

TENEX	Операционная система TEN-Extended, основанная на TOPS-10 для PDP-10.
TLB	Translation Lookaside Buffer — буфер ассоциативной трансляции. Кэш для трансляции адресов в системах виртуальной памяти, который используется блоком управления памяти MMU (см. <i>MMU</i>).
TLS	Transport Layer Security — протокол безопасности транспортного уровня. Используется для шифрования данных, передаваемых по сети.
TPU	Tensor Processing Unit — тензорный процессор. Специализированная интегральная схема, ускоритель вычислений для задач искусственного интеллекта и машинного обучения, разработанный в Google и названный в честь TensorFlow (программная платформа для машинного обучения).
TX	Передача (используется в сетевых технологиях).
UDP	User Datagram Protocol — протокол пользовательских дейтаграмм. Первоначально определен в RFC 768. См. главу 10 «Сети».
uprobes	Технология ядра Linux для динамической инструментации программного обеспечения пользовательского уровня.
us	Микросекунды. Обычно используется аббревиатура μs , но в выводе инструментов оценки производительности вы часто будете видеть «us». (Обратите внимание, что вывод <code>vmstat(8)</code> , примеры которого много раз приводились в этой книге, включает столбец <code>us</code> , сокращенно от «user time» — время в пространстве пользователя.)
μs	Микросекунды. См. <i>us</i> .
USDT	User-land Statically Defined Tracing — статически определяемые точки трассировки на стороне пользователя. Включает размещение программистом статических зондов трассировки в коде приложения в местах, где это может пригодиться.
vCPU	Виртуальный процессор. Современные процессоры могут предоставлять несколько виртуальных процессоров на каждое ядро (например, благодаря технологии гиперпоточности Intel).
VFS	Virtual File System — виртуальная файловая система. Абстракция, используемая ядром для единообразной поддержки файловых систем разных типов.
VMS	Virtual Memory System — система виртуальной памяти. Операционная система, выпущенная компанией DEC.
x86	Архитектура процессоров, основанная на Intel 8086.
ZFS	Комбинированная файловая система и диспетчер томов, созданные в Sun Microsystems.

Адаптивный мьютекс (adaptive mutex)	Мьютекс (mutual exclusion — взаимоисключающая блокировка) — тип блокировки, использующийся для синхронизации. См. главу 5 «Приложения», раздел 5.2.5 «Конкурентность и параллелизм».
Адрес (address)	Местоположение в памяти.
Адресное пространство (address space)	Контекст виртуальной памяти. См. главу 7 «Память».
Ассоциативный массив (associative array)	Тип данных в языках программирования, в котором доступ к элементам производится с помощью произвольного ключа или нескольких ключей.
Байт (byte)	Единица представления цифровых данных. Эта книга следует промышленному стандарту, согласно которому один байт равен восьми битам, а бит — это двоичная цифра, ноль или один.
Бенчмарк (benchmark)	В информационных технологиях бенчмарком называют инструмент, помогающий создать экспериментальную рабочую нагрузку и измерить производительность: результат бенчмаркинга. Обычно бенчмарки используются для оценки влияния различных параметров настройки на производительность.
Буфер (buffer)	Область памяти для временного хранения данных.
Вне процессора (off-CPU)	Этот термин применяется к потокам, которые в данный момент не выполняются на процессоре, то есть находятся «вне процессора», например, ожидая завершения ввода/вывода, простаивая на блокировке, добровольно перейдя в режим ожидания или вследствие некоторого другого события.
Внешний интерфейс, внешняя часть (front end)	Обозначает интерфейс конечного пользователя и программное обеспечение, представляющее информацию. Веб-приложение — это внешний интерфейс. См. <i>внутренний интерфейс</i> .
Внутренний интерфейс, внутренняя часть (back end)	Этот термин относится к хранилищам данных и компонентам инфраструктуры. Веб-сервер — это внутренняя часть программного комплекса. См. <i>внешний интерфейс</i> .
Виртуальная память (virtual memory)	Абстракция основной памяти, поддерживающая многозадачность и возможность выделения большего объема памяти, чем есть.

Выборка (sampling)	Метод наблюдения за целью путем проведения подмножества измерений: выборки.
Герц (Гц)	Такты в секунду.
Гиперпоточность (hyperthread)	Реализация SMT (Simultaneous Multithreading — одновременная многопоточность), выполненная компанией Intel. Технология масштабирования процессоров, позволяющая ОС создавать несколько виртуальных процессоров для одного ядра и планировать задания для них, которые процессор будет пытаться выполнять параллельно.
Графический процессор (Graphics Processing Unit, GPU)	Графические процессоры можно использовать для обработки некоторых рабочих нагрузок, например для машинного обучения.
Дейтаграмма (datagram)	См. <i>сегмент</i> .
Демон (daemon)	Системная программа, выполняющаяся постоянно для предоставления услуги.
Дескриптор файла (file descriptor)	Идентификатор, используемый программой для обращения к открытому файлу.
Динамическая трассировка (dynamic tracing)	Может означать ПО, реализующее динамическую instrumentation.
Динамическая инструментация (dynamic instrumentation)	Динамическая instrumentation, или динамическая трассировка, — это технология, позволяющая инструментировать любое программное событие, включая вызовы функций и возврат из них, путем изменения команд в реальном времени и вставки временных команд трассировки.
Диск (disk)	Физическое устройство для хранения данных. См. также <i>HDD</i> и <i>SSD</i> .
Дуплекс (duplex)	Режим, когда обмен данными происходит одновременно в обоих направлениях.
Загрузка/выгрузка страниц (pagein/pageout)	Действия, выполняемые ОС (ядром) для перемещения фрагментов памяти (страниц) с внешних устройств хранения и обратно.
Задача (task)	Запущенный выполняемый объект, который может быть процессом, потоком в многопоточном процессе или потоком ядра. См. главу 3 «Операционные системы».

Задержка (latency)	Время, потраченное на ожидание. В анализе производительности этот термин часто используется для описания времени ввода/вывода. Задержка играет важную роль, потому что часто она — наиболее эффективное средство оценки проблемы с производительностью. Где именно измеряется задержка, не всегда ясно без дополнительных уточнений. Например, «задержка диска» может означать время, потраченное на ожидание только в очереди драйвера диска, или все время ожидания завершения дискового ввода/вывода, включая и время ожидания в очереди, и время обслуживания. Задержка имеет нижнюю границу, полоса пропускания — верхнюю.
Значительный сбой (major fault)	Сбой доступа к памяти, требующий подкачки с запоминающих устройств (дисков). См. главу 3 «Операционные системы».
Карта расширения (expander card)	Физическое устройство (карта), подключенное к системе, обычно для подключения дополнительных контроллеров ввода/вывода.
Килобайт (Кбайт)	Международная система единиц (СИ) определяет килобайт как 1000 байт, но в информатике килобайт обычно составляет 1024 байта (в системе СИ обозначается как кибибайт). Инструменты, описываемые в книге, которые сообщают размеры данных в килобайтах, обычно используют определение $1024 (2^{10})$ байт.
Клиент (client)	Потребитель сетевой службы. Обычно клиентом называют клиентский хост или клиентское приложение.
Кольцо процессора (processor ring)	Режим защиты процессора.
Команда (command)	Программа, выполняемая в командной оболочке.
Командная оболочка (shell)	Интерпретатор командной строки и язык сценариев.
Конкурентность (concurrency)	См. главу 5 «Приложения», раздел 5.2.5 «Конкурентность и параллелизм».
Конкуренция (contention)	Состязание за обладание ресурсом.

Контроллер диска (disk controller)	Компонент, управляющий подключенными дисками и обеспечивающий их доступность для системы напрямую или через виртуальные диски. Контроллеры дисков могут встраиваться в системную материнскую плату, подключаться как карты расширения или встраиваться в массивы хранения. Они поддерживают один или несколько типов интерфейсов хранения (например, SCSI, SATA, SAS) и обычно называются адаптерами главной шины (Host Bus Adapter, HBA) с указанием типа интерфейса, например SAS HBA.
Коэффициент попаданий (hit ratio)	Часто используется для описания производительности кэша: отношение попаданий в кэш к общему числу обращений к кэшу (число попаданий плюс число промахов), выражается в процентах. Чем выше, тем лучше.
Логический процессор (logical processor)	Другое название <i>виртуального процессора</i> . См. главу 6 «Процессоры».
Локальные диски (local disks)	Диски, подключенные непосредственно к серверу. К локальным относятся диски внутри корпуса сервера и диски, подключенные напрямую через транспортную шину.
Массив (array)	Набор значений. Тип данных в языках программирования. В низкоуровневых языках массивы обычно хранятся в непрерывных областях памяти, а индекс определяет смещение элемента массива от начала этой области.
Массив хранения (storage array)	Набор дисков в одном корпусе, который можно подключить к системе. Массивы хранения обычно предлагают различные дополнительные возможности для повышения надежности и производительности.
Материнская плата (main board)	Печатная плата, на которой размещены процессоры и системные соединения. Также называется <i>системной платой</i> .
Мегабайт (Мбайт)	Международная система единиц (СИ) определяет мегабайт как 1 000 000 байт, но в информатике мегабайт обычно составляет 1 048 576 байт (в системе СИ обозначается как мебибайт). Инструменты, описываемые в книге, которые сообщают размеры данных в мегабайтах, обычно используют определение 1 048 576 (2 ²⁰) байт.

Микробенчмарк (microbenchmark)	Бенчмарк для оценки одной или простой операции.
Мьютекс (mutex)	Взаимоисключающая блокировка. Мьютексы могут быть узким местом и часто исследуются на предмет проблем с производительностью. См. главу 5 «Приложения».
На процессоре (on-CPU)	Этот термин применяется к потокам, которые в текущий момент времени выполняются на процессоре.
Наблюдаемость (observability)	Возможность наблюдения за вычислительной системой. Включает методики и инструменты, используемые для анализа состояния вычислительных систем.
Настраиваемый параметр (tunable parameter)	Параметр для настройки различных аспектов работы программного обеспечения.
Незначительный сбой (minor fault)	Сбой доступа к памяти, не требующий подкачки с запоминающих устройств (дисков). См. главу 3 «Операционные системы».
ОС (OS)	Операционная система. Набор ПО, включая ядро, осуществляющий управление ресурсами и процессами пользователя уровня.
Основная память (main memory)	Оперативная память системы, обычно реализованная на микросхемах DRAM.
Отказ страницы (page fault)	Системное прерывание; срабатывает, когда программа ссылается на область виртуальной памяти, которая еще не отображена в физическую память. Это нормальное следствие модели распределения памяти по требованию.
Очередь на выполнение (run queue)	Очередь планировщика процессора, в которой находятся задачи, ожидающие своей очереди выполнения на процессоре. В реальности очередь может быть реализована в виде древовидной структуры, но до сих пор широко используется термин «очередь на выполнение».
Пакет (packet)	Сетевое сообщение на сетевом уровне сетевой модели OSI (см. раздел 10.2.3).
Память (memory)	См. <i>основная память</i> .
Параллельное выполнение (concurrency)	См. главу 5 «Приложения», раздел 5.2.5 «Конкуренция и параллелизм».

Перекрестный вызов (cross call)	См. <i>перекрестный вызов процессоров</i> .
Перекрестный вызов процессоров (CPU cross call)	Вызовы между процессорами в многопроцессорной системе для передачи заданий друг другу. Перекрестные вызовы могут выполняться для обработки общесистемных событий, например для согласования записей в кэше процессора. См. главу 6 «Процессоры». В Linux эти вызовы называются вызовами SMP.
Переменная (variable)	Именованный объект, использующийся в языках программирования для хранения данных.
Перформанс-инженер (performance engineer)	Технический сотрудник, занимающийся в основном вопросами производительности: планированием, оценкой, анализом и улучшениями. См. главу 1 «Введение», раздел 1.2 «Роли».
Полоса пропускания (bandwidth)	Частотный диапазон канала связи, измеряемый в герцах. В информатике под полосой пропускания подразумевается максимальная скорость передачи для канала связи. Этот термин также часто <i>ошибочно</i> используют для описания текущей пропускной способности (throughput).
Попадание в кэш (cache hit)	Запрос на получение данных, которые могут быть возвращены из кэша.
Поток/поток выполнения (thread)	Программная абстракция выполняемого экземпляра программы, которую можно запланировать для выполнения на процессоре. Ядро имеет несколько потоков выполнения, а процесс содержит один или несколько таких потоков. См. главу 3 «Операционные системы».
Приложение (application)	Программа, обычно выполняющаяся в пространстве пользователя.
Проммах кэша (cache miss)	Запрос на получение данных, которые не могут быть возвращены из кэша.
Промышленная среда/продакшен (production)	Термин, использующийся для описания рабочей нагрузки, создаваемой реальными клиентами, и среды, которая их обрабатывает. У многих компаний есть «тестовая» среда, обслуживающая синтетические рабочие нагрузки и предназначенная для тестирования ПО перед развертыванием в промышленной среде.

Пропускная способность (throughput)	Для сетевых устройств под пропускной способностью обычно понимается скорость передачи данных в битах в секунду или байтах в секунду. В статическом анализе под пропускной способностью также может пониматься частота выполнения операций ввода/вывода в секунду (IOPS).
Пространство пользователя (user-space)	Адресное пространство процессов пользовательского уровня.
Пространство ядра (kernel-space)	Адресное пространство ядра.
Профилирование (profiling)	Метод сбора данных, характеризующих эффективность цели. Распространенный метод профилирования — выборка по времени (см. <i>выборка</i>).
Процесс (process)	Выполняющийся экземпляр программы. Многозадачные ОС могут выполнять несколько процессов одновременно и запускать несколько экземпляров одной и той же программы в разных процессах. Процесс содержит один или несколько потоков, выполняющихся на процессорах.
Процессор (Central Processing Unit, CPU)	Этим термином обозначается набор функциональных блоков, которые выполняют инструкции, включая регистры и арифметико-логический блок (Arithmetic Logic Unit, ALU). Используется для обозначения физических и виртуальных процессоров.
Прошивка (firmware)	Программное обеспечение, встроенное в устройство.
Рабочая нагрузка (workload)	Описывает запросы к системе или ресурсам.
Рабочая нагрузка реального времени (real-time workload)	Рабочая нагрузка, имеющая фиксированные требования к задержкам, обычно довольно низкие.
Регистры (registers)	Маленькие запоминающие устройства в процессоре, используемые машинными инструкциями при обработке данных.
Сбалансированная система (balanced system)	Система без узких мест.
Сегмент (segment)	Сообщение на транспортном уровне сетевой модели OSI (см. раздел 10.2.3 «Стек протоколов»).

Сектор (sector)	Минимальная единица данных для устройств хранения, обычно 512 байт или 4 Кбайт. См. главу 9 «Диски».
Сервер (server)	В сети — сетевой хост, который предоставляет услуги сетевым клиентам, например HTTP-сервер или сервер базы данных. Термин «сервер» также может относиться к физической системе.
Системный вызов (system call/syscall)	Интерфейс для процессов, запрашивающих выполнение привилегированных действий у ядра. См. главу 3 «Операционные системы».
Сокет (socket)	Программная абстракция, представляющая конечную точку сетевого соединения.
Статическая инструментация/трассировка (static instrumentation/tracing)	Инструментация ПО с использованием предварительно скомпилированных точек зондирования. См. главу 4 «Инструменты наблюдения».
Стек (stack)	В контексте инструментов наблюдения под стеком обычно понимаются трассировки стека.
Сторона пользователя (user-land)	Относится к ПО и файлам пользовательского уровня, включая исполняемые программы в /usr/bin, /lib и т. д.
Сторона ядра (kernel-land)	Программное обеспечение ядра.
Страница (page)	Фрагмент памяти, управляемой ядром и процессором. Вся память, используемая системой, разбита на страницы. Обычно используются страницы размерами 4 Кбайт и 2 Мбайт (в зависимости от процессора).
Структура (struct)	Экземпляр структуры, обычно в языке С.
Сценарий/скрипт (script)	В информатике — выполняемая программа, обычно короткая и написанная на языке высокого уровня.
Такт процессора (CPU cycle)	Единица времени, основанная на тактовой частоте процессора: при тактовой частоте 2 ГГц каждый такт длится 0,5 нс. Сам такт представляет собой электрический сигнал — перепад напряжения, запускающий цифровую логику.
Теплота кэша (cash warmth)	См. главу 2 «Методологии», раздел 2.3.14 «Кэширование», подраздел «Горячие, холодные и теплые кэши».

Точки трассировки (tracepoints)	Технология ядра Linux для поддержки статической инструментации.
Трассировка (tracing)	Наблюдение за событиями.
Трассировка стека (stack trace)	Стек вызовов, состоящий из нескольких фреймов стека, охватывающих путь выполнения кода. Трассировки стека часто исследуют в ходе анализа производительности, особенно при профилировании процессора.
Трассировщик (tracer)	Инструмент для трассировки. См. <i>трассировка</i> .
Удаленные диски (remote disks)	Диски (включая виртуальные), которые используются сервером, но подключены к удаленной системе.
Узкое место (bottleneck)	Что-то, ограничивающее производительность.
Уровень пользователя (user-level)	Режим привилегий процессора, используемый при выполнении на уровне пользователя. Это более низкий уровень привилегий, чем у ядра. Он запрещает прямой доступ к ресурсам, вынуждая ПО пользовательского уровня запрашивать доступ к ним через ядро.
Уровень ядра (kernel-level)	Уровень привилегий процессора, устанавливаемый при выполнении кода ядра.
Файл с отладочной информацией (debuginfo file)	Файл с символами и отладочной информацией, которые используют отладчики и профилировщики.
Флейм-график/график пламени (flame graph)	Метод визуализации набора трассировок стека. См. главу 2 «Методологии».
Фрейм (frame)	Сообщение на канальном уровне сетевой модели OSI (см. раздел 10.2.3 «Стек протоколов»).
Фрейм стека (stack frame)	Структура данных, содержащая информацию о состоянии функции, включая ее аргументы и адрес возврата.
Хост (host)	Система, подключенная к сети. Также называют сетевым <i>узлом</i> .
Частота операций (operation rate)	Количество операций, выполняемых в единицу времени (например, операций в секунду), могут подразумеваться любые операции, в том числе не связанные с вводом/выводом.

Экземпляр (instance)	Виртуальный сервер. Облачные среды предоставляют экземпляры серверов.
Ядро (core)	Конвейер выполнения в процессоре. ОС может видеть каждое ядро как отдельный процессор или, если поддерживается технология гиперпоточности, как несколько процессоров.
Ядро (kernel)	Базовая программа в системе, которая работает в привилегированном режиме и управляет ресурсами и процессами уровня пользователя.

Брендан Грегг

Производительность систем

Перевел с английского А. Киселев

Руководитель дивизиона	<i>Ю. Сергиенко</i>
Руководитель проекта	<i>А. Питиримов</i>
Ведущий редактор	<i>Е. Строганова</i>
Литературный редактор	<i>К. Тульцева</i>
Художественный редактор	<i>В. Мостипан</i>
Корректоры	<i>С. Беляева, Н. Викторова</i>
Верстка	<i>Л. Егорова</i>

Изготовлено в России. Изготовитель: ООО «Прогресс книга».
Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург,
Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 05.2023. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 06.03.23. Формат 70×100/16. Бумага офсетная. Усл. п. л. 79,980. Тираж 700. Заказ 0000.

Карл Олбинг, Джей Пи Фоссен

ИДИОМЫ BASH



Сценарии на языке командной оболочки получили самое широкое распространение, особенно написанные на языках, совместимых с bash. Но эти сценарии часто сложны и непонятны. Сложность — враг безопасности и причина неудобочитаемости кода. Эта книга на практических примерах покажет, как расшифровывать старые сценарии и писать новый код, максимально понятный и легко читаемый.

Авторы Карл Олбинг (Carl Albing) и Джей Пи Фоссен (JP Vossen) покажут, как использовать мощь и гибкость командной оболочки. Даже если вы умеете писать сценарии на bash, эта книга поможет расширить ваши знания и навыки. Независимо от используемой ОС — Linux, Unix, Windows или Mac — к концу книги вы научитесь понимать и писать сценарии на экспертном уровне. Это вам обязательно пригодится.

Вы познакомитесь с идиомами, которые следует использовать, и такими, которых следует избегать.

КУПИТЬ

Кристофер Негус

БИБЛИЯ LINUX

10-е издание



Полностью обновленное 10-е издание «Библии Linux» поможет как начинающим, так и опытным пользователям приобрести знания и навыки, которые выведут на новый уровень владения Linux. Известный эксперт и автор бестселлеров Кристофер Негус делает акцент на инструментах командной строки и новейших версиях Red Hat Enterprise Linux, Fedora и Ubuntu. Шаг за шагом на подробных примерах и упражнениях вы досконально поймете операционную систему Linux и пустите знания в дело. Кроме того, в 10-м издании содержатся материалы для подготовки к экзаменам на различные сертификаты по Linux.

КУПИТЬ